SOURCE CODE DOCUMENTATION

Capacity Connect

Frontend Application Architecture, Pages, Features & Components

TOTAL FILES INCLUDED
146 Files

TOTAL CODE LINES
48,573 Lines

EXPORT DATE & TIME
October 22, 2026 at 1:02 PM

TECHNOLOGY STACK
React 19, TypeScript, Supabase

ARCHITECTURE & CORE DEPENDENCIES

React 19 | TypeScript | Vite | Tailwind CSS | Supabase Backend | PostgreSQL & RLS Policies | Supabase Edge Functions (Deno) | Radix UI Primitives | TanStack React Query | React Hook Form + Zod | Three.js / React Three Fiber | TensorFlow.js (BlazeFace/COCO-SSD)

Capacity Connect Application Repository

Directory: E:\Projects\capacity connect

Table of Contents

01. Configuration & Build Setup (11 files, 568 lines)

components.json	21 lines | 0.4 KB
index.html	43 lines | 1.7 KB
package.json	71 lines | 2.1 KB
postcss.config.js	7 lines | 0.1 KB
README.md	34 lines | 1.3 KB
tailwind.config.js	278 lines | 7.3 KB
tsconfig.app.json	26 lines | 0.6 KB
tsconfig.json	13 lines | 0.2 KB
tsconfig.node.json	24 lines | 0.5 KB
vercel.json	9 lines | 0.1 KB
vite.config.ts	42 lines | 1.2 KB

02. Application Entry & Core Setup (4 files, 685 lines)

src/App.css	185 lines | 2.8 KB
src/App.tsx	264 lines | 15.3 KB
src/index.css	225 lines | 5.4 KB
src/main.tsx	11 lines | 0.2 KB

03. Integrations, Supabase Client & Utilities (10 files, 2,557 lines)

src/integrations/supabase/types.ts	1,116 lines | 32.9 KB
src/lib/ai.ts	90 lines | 2.3 KB
src/lib/certificateGenerator.ts	900 lines | 32.7 KB
src/lib/courseGrading.ts	210 lines | 7.1 KB
src/lib/query-client.ts	19 lines | 0.5 KB
src/lib/storage.ts	55 lines | 1.3 KB
src/lib/supabase.ts	22 lines | 0.6 KB
src/lib/useStorageUrl.ts	38 lines | 0.9 KB
src/lib/utils.ts	7 lines | 0.2 KB
src/utils/cropImage.ts	100 lines | 3.2 KB

04. Custom Hooks & State Management (4 files, 1,157 lines)

src/hooks/useAuth.tsx	181 lines | 6.1 KB
src/hooks/useConfirm.tsx	35 lines | 0.9 KB
src/hooks/useHomePageSettings.ts	563 lines | 19.2 KB
src/hooks/useNotifications.ts	378 lines | 11.7 KB

05. Application Pages & Views (17 files, 8,389 lines)

src/pages/AboutPage.tsx	515 lines | 29.2 KB
src/pages/AccessDeniedPage.tsx	48 lines | 2.4 KB
src/pages/AccountSuspendedPage.tsx	50 lines | 2.4 KB
src/pages/ContactPage.tsx	431 lines | 21.2 KB
src/pages/Dashboards.tsx	2,474 lines | 138.4 KB
src/pages/ForgotPassword.tsx	117 lines | 5.6 KB
src/pages/LandingPage.tsx	1,144 lines | 53.7 KB
src/pages/Login.tsx	172 lines | 7.4 KB
src/pages/NotFoundPage.tsx	38 lines | 1.8 KB
src/pages/PendingApprovalPage.tsx	54 lines | 2.8 KB
src/pages/ProfilePage.tsx	1,613 lines | 77.4 KB
src/pages/PublicCourseCatalog.tsx	311 lines | 17.5 KB
src/pages/PublicCourseDetails.tsx	551 lines | 31.3 KB
src/pages/Register.tsx	475 lines | 23.5 KB
src/pages/SettingsPage.tsx	7 lines | 0.1 KB
src/pages/SetupPassword.tsx	160 lines | 7.3 KB
src/pages/SupabaseTest.tsx	229 lines | 12.0 KB

06. Features — Trainer Management (14 files, 8,761 lines)

src/features/trainer/AssessmentsPage.tsx	2,072 lines | 111.2 KB
src/features/trainer/CourseCreatePage.tsx	2,461 lines | 124.3 KB
src/features/trainer/CourseDetailPage.tsx	1,443 lines | 78.7 KB
src/features/trainer/CourseEditPage.tsx	714 lines | 35.9 KB
src/features/trainer/CourseListPage.tsx	409 lines | 19.8 KB
src/features/trainer/CourseSessionsPage.tsx	508 lines | 24.6 KB
src/features/trainer/NotificationsPage.tsx	7 lines | 0.2 KB
src/features/trainer/OverallAttendanceDialog.tsx	194 lines | 8.1 KB
src/features/trainer/PerformancePage.tsx	342 lines | 17.5 KB
src/features/trainer/SettingsPage.tsx	7 lines | 0.1 KB
src/features/trainer/TrainerDashboard.tsx	205 lines | 10.4 KB
src/features/trainer/TrainerLayout.tsx	196 lines | 9.8 KB
src/features/trainer/TrainerProfile.tsx	7 lines | 0.1 KB
src/features/trainer/TrainerSkills.tsx	196 lines | 8.4 KB

07. Features — Courses, Sessions & Learning (17 files, 11,743 lines)

src/features/courses/AdminCourses.tsx	1,506 lines | 77.9 KB
src/features/courses/CourseAnnouncements.tsx	217 lines | 9.8 KB
src/features/courses/CourseCertificateStep.tsx	284 lines | 14.0 KB
src/features/courses/CourseChat.tsx	288 lines | 12.0 KB
src/features/courses/CourseFeedback.tsx	405 lines | 17.3 KB
src/features/courses/CourseFormDialog.tsx	184 lines | 6.4 KB
src/features/courses/CourseMaterials.tsx	738 lines | 29.8 KB
src/features/courses/LiveAttendanceTraineePanel.tsx	266 lines | 9.4 KB
src/features/courses/LiveAttendanceTrainerPanel.tsx	417 lines | 16.2 KB
src/features/courses/TraineeAssessmentResult.tsx	462 lines | 34.4 KB
src/features/courses/TraineeAssessmentsHub.tsx	532 lines | 30.9 KB
src/features/courses/TraineeAssessmentTest.tsx	1,202 lines | 55.4 KB
src/features/courses/TraineeCourseCatalog.tsx	310 lines | 17.1 KB
src/features/courses/TraineeCourseDetails.tsx	2,515 lines | 139.5 KB
src/features/courses/TraineeCourseLearnPage.tsx	1,900 lines | 93.5 KB
src/features/courses/TraineeMyLearning.tsx	311 lines | 16.7 KB
src/features/courses/TrainerCourses.tsx	206 lines | 7.9 KB

08. Features — Super Admin & Governance (8 files, 7,540 lines)

src/features/admin/AdminAnnouncements.tsx	216 lines | 9.8 KB
src/features/admin/AdminAssessmentsHub.tsx	1,556 lines | 73.0 KB
src/features/admin/AdminContactMessages.tsx	423 lines | 19.0 KB
src/features/admin/AdminCourseCreatePage.tsx	2,428 lines | 118.0 KB
src/features/admin/AdminCourseEditPage.tsx	516 lines | 32.6 KB
src/features/admin/AdminHomePageSettings.tsx	1,620 lines | 82.8 KB
src/features/admin/AdminUserDetailsModal.tsx	632 lines | 31.5 KB
src/features/admin/TrainerAssignmentBanner.tsx	149 lines | 6.8 KB

09. Features — Notifications & Certificates (1 files, 338 lines)

src/features/notifications/UniversalNotificationsPage.tsx	338 lines | 14.0 KB

10. UI Primitives & Design System (Radix/Tailwind) (22 files, 2,102 lines)

src/components/ui/alert-dialog.tsx	140 lines | 4.3 KB
src/components/ui/AnnouncementModal.tsx	177 lines | 6.4 KB
src/components/ui/AnnouncementsFeed.tsx	113 lines | 4.5 KB
src/components/ui/badge.tsx	47 lines | 1.5 KB
src/components/ui/button.tsx	68 lines | 2.7 KB
src/components/ui/card.tsx	77 lines | 1.8 KB
src/components/ui/ConfirmModal.tsx	78 lines | 2.7 KB
src/components/ui/dialog.tsx	121 lines | 3.7 KB
src/components/ui/dropdown-menu.tsx	202 lines | 8.6 KB
src/components/ui/form.tsx	177 lines | 4.1 KB
src/components/ui/ImageCropperModal.tsx	208 lines | 7.2 KB
src/components/ui/input.tsx	23 lines | 0.8 KB
src/components/ui/label.tsx	25 lines | 0.7 KB
src/components/ui/MaterialPreviewDialog.tsx	133 lines | 7.0 KB
src/components/ui/progress.tsx	27 lines | 0.8 KB
src/components/ui/select.tsx	160 lines | 5.9 KB
src/components/ui/separator.tsx	30 lines | 0.7 KB
src/components/ui/sonner.tsx	32 lines | 0.9 KB
src/components/ui/table.tsx	121 lines | 2.8 KB
src/components/ui/tabs.tsx	54 lines | 1.7 KB
src/components/ui/textarea.tsx	23 lines | 0.7 KB
src/components/ui/Thumbnail.tsx	66 lines | 2.7 KB

11. Shared Layout & Dialog Components (13 files, 3,143 lines)

src/components/ChatBot.tsx	385 lines | 16.4 KB
src/components/ChatBotLogo.tsx	95 lines | 2.8 KB
src/components/ErrorBoundary.tsx	29 lines | 0.8 KB
src/components/landing/AchievementsSection.tsx	277 lines | 14.1 KB
src/components/landing/DynamicUpdatesSection.tsx	304 lines | 15.1 KB
src/components/landing/HorizontalAnnouncementBar.tsx	193 lines | 7.4 KB
src/components/landing/MissionExplodedView.tsx	299 lines | 10.7 KB
src/components/landing/StickyScrollFeatures.tsx	346 lines | 15.6 KB
src/components/layout/DashboardShell.tsx	217 lines | 9.0 KB
src/components/notifications/NotificationPanel.tsx	337 lines | 15.3 KB
src/components/profile/FirstTimeProfileSetupModal.tsx	484 lines | 18.9 KB
src/components/ProtectedRoute.tsx	65 lines | 1.6 KB
src/components/space/StarfieldCanvas.tsx	112 lines | 3.1 KB

14. System, Audit & Database Scripts (25 files, 1,590 lines)

.oxlintrc.json	9 lines | 0.2 KB
check_db.ts	19 lines | 0.5 KB
check_fetch.js	38 lines | 1.0 KB
check_fetch2.js	38 lines | 1.0 KB
check_fetch3.js	38 lines | 1.0 KB
check_rls.ts	27 lines | 0.9 KB
check-db.js	16 lines | 0.5 KB
check.js	24 lines | 0.7 KB
clean_update_types.js	86 lines | 2.4 KB
dedup_types.js	32 lines | 0.7 KB
fix_assessment_attempts_rls.ts	31 lines | 0.9 KB
fix_assessments_rls.ts	95 lines | 3.0 KB
fix.js	38 lines | 1.2 KB
fix.ts	24 lines | 0.8 KB
get_logs.cjs	19 lines | 0.6 KB
repair_types.js	54 lines | 2.0 KB
scripts/db-audit.mjs	102 lines | 3.7 KB
scripts/debug-score.mjs	65 lines | 2.4 KB
scripts/fix-attempt-answers.mjs	93 lines | 3.1 KB
scripts/generate_code_pdf.cjs	613 lines | 18.9 KB
test_admin.mjs	8 lines | 0.4 KB
test_admin2.mjs	9 lines | 0.4 KB
test_login.mjs	9 lines | 0.4 KB
test-db.mjs	22 lines | 0.8 KB
update_types.js	81 lines | 2.3 KB

components.json JSON 0.4 KB 21 Lines

```json
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "tailwind.config.js",
    "css": "src/index.css",
    "baseColor": "slate",
    "cssVariables": true,
    "prefix": ""
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  }
}

```

index.html HTML 1.7 KB 43 Lines

```
<!doctype html>
<html lang="en" class="dark">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <link rel="icon" type="image/png" sizes="32x32" href="/favicon.png" />
    <link rel="shortcut icon" href="/favicon.ico" />
    <link rel="apple-touch-icon" href="/apple-touch-icon.png" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- Preconnect to external font origins -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />


    <!-- Non-render-blocking Google Fonts loading -->
    <link
      rel="preload"
      as="style"
      href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=Space+Grotesk:wght@400;500;600;700;800&display=swap"
    />
    <link
      rel="stylesheet"
      href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=Space+Grotesk:wght@400;500;600;700;800&display=swap"
      media="print"
      onload="this.media='all'"
    />
    <noscript>
      <link
        rel="stylesheet"
        href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=Space+Grotesk:wght@400;500;600;700;800&display=swap"
      />
    </noscript>

    <title>Capacity Connect — Digital Capacity Building & Learning Portal | MoES & IMD</title>
  </head>
  <body class="bg-[#05060F] text-slate-100 antialiased selection:bg-pink-500 selection:text-white">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>


```

package.json JSON 2.1 KB 71 Lines

```
{
  "name": "capacity-connect",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "@google/genai": "^2.23.0",
    "@hookform/resolvers": "^5.9.1",
    "@radix-ui/react-alert-dialog": "^1.1.23",
    "@radix-ui/react-dialog": "^1.1.23",
    "@radix-ui/react-dropdown-menu": "^2.1.24",
    "@radix-ui/react-label": "^2.1.15",
    "@radix-ui/react-progress": "^1.1.16",
    "@radix-ui/react-select": "^2.3.7",
    "@radix-ui/react-separator": "^1.1.15",
    "@radix-ui/react-slot": "^1.3.3",
    "@radix-ui/react-tabs": "^1.1.21",
    "@react-three/drei": "^10.7.8",
    "@react-three/fiber": "^9.7.0",
    "@supabase/supabase-js": "^2.116.0",
    "@tanstack/react-query": "^5.102.8",
    "@tensorflow-models/blazeface": "^0.1.0",
    "@tensorflow-models/coco-ssd": "^2.2.3",
    "@tensorflow/tfjs": "^4.22.0",
    "@types/three": "^0.186.0",
    "bcrypt": "^6.0.0",
    "bcryptjs": "^3.0.3",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "date-fns": "^4.4.0",
    "framer-motion": "^13.2.0",
    "gsap": "^3.15.0",
    "jspdf": "^4.2.1",
    "jszip": "^3.10.2",
    "lucide-react": "^1.45.0",
    "next-themes": "^0.4.6",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-easy-crop": "^6.2.3",
    "react-hook-form": "^7.87.0",
    "react-router-dom": "^7.18.3",
    "recharts": "^3.10.1",
    "sonner": "^2.0.8",
    "tailwind-merge": "^3.6.0",
    "tailwindcss-animate": "^1.0.7",
    "three": "^0.186.0",
    "zod": "^3.25.76"
  },
  "devDependencies": {
    "@types/jszip": "^3.4.0",
    "@types/node": "^24.13.4",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "autoprefixer": "^10.5.6",
    "oxlint": "^1.81.0",
    "postcss": "^8.5.28",
    "prismjs": "^1.30.0",
    "sharp": "^0.35.4",
    "tailwindcss": "^3.4.19",
    "typescript": "~6.0.2",
    "vite": "^8.3.0"
  }
}

```

postcss.config.js JS 0.1 KB 7 Lines

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}

```

README.md MD 1.3 KB 34 Lines

```
# React + TypeScript + Vite

This template provides a minimal setup to get React working in Vite with HMR and some Oxlint rules.

Currently, two official plugins are available:

- [@vitejs/plugin-react](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react) uses [Oxc](https://oxc.rs)
- [@vitejs/plugin-react-swc](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react-swc) uses [SWC](https://swc.rs/)

## React Compiler

The React Compiler is not enabled on this template because of its impact on dev & build performances. To add it, see [this documentation](https://react.dev/learn/react-compiler/installation).

## Expanding the Oxlint configuration

If you are developing a production application, we recommend enabling type-aware lint rules by installing `oxlint-tsgolint` and editing `.oxlintrc.json`:

```json
{
  "$schema": "./node_modules/oxlint/configuration_schema.json",
  "plugins": ["react", "typescript", "oxc"],
  "options": {
    "typeAware": true
  },
  "rules": {
    "react/rules-of-hooks": "error",
    "react/only-export-components": ["warn", { "allowConstantExport": true }]
  }
}
```

See the [Oxlint rules documentation](https://oxc.rs/docs/guide/usage/linter/rules) for the full list of rules and categories.
Capacity_Connect

```

**tailwind.config.js** JS 7.3 KB 278 Lines

```
/** @type {import('tailwindcss').Config} */

export default {
  darkMode: ["class"],
  content: [
    "./index.html",
    "./src/**/*.{ts,tsx,js,jsx}",
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ['"SF Pro Display"', '-apple-system', 'BlinkMacSystemFont', '"SF Pro Text"', 'Inter', 'system-ui', 'sans-seri
f'],
        display: ['"SF Pro Display"', '-apple-system', 'BlinkMacSystemFont', '"SF Pro Text"', 'Inter', 'system-ui', 'sans-seri
f'],
        mono: ['ui-monospace', 'SFMono-Regular', 'Menlo', 'Monaco', 'Consolas', 'monospace'],
      },
      colors: {
        space: {
          950: '#05060F',
          900: '#0B0F2A',
          800: '#111740',
          700: '#1A235C',
          card: '#0D1333',
        },
        // --- Sunset Vibrant Palette (#F3841D, #FF8E7A, #EA519D, #6B4BA3, #FFFFFF) ---

        orange: {
          DEFAULT: '#F3841D',
          50: '#FEF7EE',
          100: '#FDEDD7',
          200: '#FBD8AE',
          300: '#F7BD7B',
          400: '#F59F47',
          500: '#F3841D',
          600: '#D86B0E',
          700: '#B4520D',
          800: '#8F4112',
          900: '#743712',
        },
        coral: {
          DEFAULT: '#FF8E7A',
          50: '#FFF4F2',
          100: '#FFE7E3',
          200: '#FFD0C8',
          300: '#FFB1A2',
          400: '#FF8E7A',
          500: '#F96A50',
          600: '#E74C2F',
          700: '#C2381E',
          800: '#A0301B',
          900: '#832B1B',
        },
        pink: {
          DEFAULT: '#EA519D',
          50: '#FDF2F8',
          100: '#FCE7F3',
          200: '#FBCFE8',
          300: '#F8A2D2',
          400: '#F274B5',
          500: '#EA519D',
          600: '#D42C7E',
          700: '#B61B64',
          800: '#961952',
          900: '#7C1A46',
        },
        purple: {
          DEFAULT: '#6B4BA3',
          50: '#F5F2FB',
          100: '#ECE5F6',
          200: '#DCCFED',
          300: '#C3ADE0',
          400: '#9875C4',
          500: '#6B4BA3',
          600: '#5A3C8E',
          700: '#4A3075',
```

```
          800: '#3D275F',
          900: '#1E1333',
        },
        midnight: {
          DEFAULT: '#1E1333',
          50: '#F8F7FB',
          100: '#EFEBF5',
          200: '#DDD6E9',
          300: '#B6A6D2',
          400: '#7B66A5',
          500: '#4F3C74',
          600: '#3D2C5E',
          700: '#2D1E4A',
          800: '#1E1333',
          900: '#110A1F',
        },

        // --- Cross-compatibility Aliases ---

        magenta: {
          DEFAULT: '#EA519D',
          500: '#EA519D',
          600: '#D42C7E',
        },
        violet: {
          DEFAULT: '#6B4BA3',
          500: '#6B4BA3',
          600: '#5A3C8E',
        },
        amber: {
          DEFAULT: '#F3841D',
          500: '#F3841D',
          600: '#D86B0E',
        },
        cream: {
          DEFAULT: '#FFFFFF',
          50: '#FFFFFF',
          100: '#FAFAFC',
          200: '#F5F4F8',
          300: '#ECE5F6',
          400: '#DCCFED',
          500: '#9875C4',
        },
        ink: {
          DEFAULT: '#1E1333',
          50: '#F8F7FB',
          100: '#EFEBF5',
          200: '#DDD6E9',
          300: '#B6A6D2',
          400: '#7B66A5',
          500: '#4F3C74',
          600: '#3D2C5E',
          700: '#2D1E4A',
          800: '#1E1333',
          900: '#110A1F',
        },
        obsidian: {
          DEFAULT: '#1E1333',
          50: '#F8F7FB',
          100: '#EFEBF5',
          200: '#DDD6E9',
          300: '#B6A6D2',
          400: '#7B66A5',
          500: '#4F3C74',
          600: '#3D2C5E',
          700: '#2D1E4A',
          800: '#1E1333',
          900: '#110A1F',
        },
        navy: {
          DEFAULT: '#1E1333',
          50: '#F8F7FB',
          100: '#EFEBF5',
          200: '#DDD6E9',
          300: '#B6A6D2',
          400: '#7B66A5',
          500: '#4F3C74',
          600: '#3D2C5E',
          700: '#2D1E4A',
```

```
          800: '#1E1333',
          900: '#110A1F',
        },
        burgundy: {
          DEFAULT: '#EA519D',
          50: '#FDF2F8',
          100: '#FCE7F3',
          200: '#FBCFE8',
          300: '#F8A2D2',
          400: '#F274B5',
          500: '#EA519D',
          600: '#D42C7E',
          700: '#B61B64',
          800: '#961952',
          900: '#7C1A46',
        },
        teal: {
          DEFAULT: '#6B4BA3',
          50: '#F5F2FB',
          100: '#ECE5F6',
          200: '#DCCFED',
          300: '#C3ADE0',
          400: '#9875C4',
          500: '#6B4BA3',
          600: '#5A3C8E',
          700: '#4A3075',
          800: '#3D275F',
          900: '#1E1333',
        },
        olive: {
          DEFAULT: '#F3841D',
          50: '#FEF7EE',
          100: '#FDEDD7',
          200: '#FBD8AE',
          300: '#F7BD7B',
          400: '#F59F47',
          500: '#F3841D',
          600: '#D86B0E',
          700: '#B4520D',
          800: '#8F4112',
          900: '#743712',
        },
        ice: {
          DEFAULT: '#ECE5F6',
          50: '#F8F7FB',
          100: '#F5F2FB',
          200: '#ECE5F6',
          300: '#DCCFED',
          400: '#C3ADE0',
          500: '#9875C4',
          600: '#6B4BA3',
        },
        gold: {
          DEFAULT: '#F3841D',
          50: '#FEF7EE',
          100: '#FDEDD7',
          200: '#FBD8AE',
          300: '#F7BD7B',
          400: '#F59F47',
          500: '#F3841D',
          600: '#D86B0E',
          700: '#B4520D',
          800: '#8F4112',
          900: '#743712',
        },
        wheat: {
          DEFAULT: '#FF8E7A',
          50: '#FFF4F2',
          100: '#FFE7E3',
          200: '#FFD0C8',
          300: '#FFB1A2',
          400: '#FF8E7A',
          500: '#F96A50',
          600: '#E74C2F',
        },

        // --- Semantic Tokens ---

        background: 'hsl(var(--background))',
```

```
        foreground: 'hsl(var(--foreground))',
        card: {
          DEFAULT: 'hsl(var(--card))',
          foreground: 'hsl(var(--card-foreground))'
        },
        popover: {
          DEFAULT: 'hsl(var(--popover))',
          foreground: 'hsl(var(--popover-foreground))'
        },
        primary: {
          DEFAULT: 'hsl(var(--primary))',
          foreground: 'hsl(var(--primary-foreground))'
        },
        secondary: {
          DEFAULT: 'hsl(var(--secondary))',
          foreground: 'hsl(var(--secondary-foreground))'
        },
        muted: {
          DEFAULT: 'hsl(var(--muted))',
          foreground: 'hsl(var(--muted-foreground))'
        },
        accent: {
          DEFAULT: 'hsl(var(--accent))',
          foreground: 'hsl(var(--accent-foreground))'
        },
        destructive: {
          DEFAULT: 'hsl(var(--destructive))',
          foreground: 'hsl(var(--destructive-foreground))'
        },
        border: 'hsl(var(--border))',
        input: 'hsl(var(--input))',
        ring: 'hsl(var(--ring))',
        chart: {
          '1': '#6B4BA3',
          '2': '#EA519D',
          '3': '#F3841D',
          '4': '#FF8E7A',
          '5': '#9875C4'
        }
      },
      borderRadius: {
        lg: 'var(--radius)',
        md: 'calc(var(--radius) - 2px)',
        sm: 'calc(var(--radius) - 4px)'
      }
    }
  },
  plugins: [require("tailwindcss-animate")],
}
```

tsconfig.app.json JSON 0.6 KB 26 Lines

```
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./src/*"]
    },
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "module": "esnext",
    "types": ["vite/client"],
    "allowArbitraryExtensions": true,
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}

```

tsconfig.json JSON 0.2 KB 13 Lines

```json
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

tsconfig.node.json JSON 0.5 KB 24 Lines

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023"],
    "types": ["node"],
    "skipLibCheck": true,

    /* Bundler mode */
    "module": "nodenext",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["vite.config.ts"]
}

```

vercel.json JSON 0.1 KB 9 Lines

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}

```

vite.config.ts TS 1.2 KB 42 Lines

```ts
import { defineConfig } from "vite"
import react from "@vitejs/plugin-react"

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      "@": `${import.meta.dirname}/src`,
    },
  },
  build: {
    chunkSizeWarningLimit: 800,
    cssCodeSplit: true,
    rollupOptions: {
      output: {
        manualChunks(id) {
          if (id.includes('node_modules/react/') || id.includes('node_modules/react-dom/') || id.includes('node_modules/react-router-dom/')) {
            return 'vendor-react'
          }
          if (id.includes('node_modules/framer-motion/') || id.includes('node_modules/gsap/')) {
            return 'vendor-motion'
          }
          if (id.includes('node_modules/lucide-react/')) {
            return 'vendor-icons'
          }
          if (id.includes('node_modules/@supabase/')) {
            return 'vendor-supabase'
          }
          if (id.includes('node_modules/recharts/')) {
            return 'vendor-charts'
          }
          if (id.includes('node_modules/@tanstack/') || id.includes('node_modules/sonner/') || id.includes('node_modules/date-fns/')) {
            return 'vendor-misc'
          }
        },
      },
    },
  },
})

```

src/App.css CSS 2.8 KB 185 Lines

```css
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;
```

```
77 |
78 |   & > div {
79 |     flex: 1 1 0;
80 |     padding: 32px;
81 |     @media (max-width: 1024px) {
82 |       padding: 24px 20px;
83 |     }
84 |   }
85 |
86 |   .icon {
87 |     margin-bottom: 16px;
88 |     width: 22px;
89 |     height: 22px;
90 |   }
91 |
92 |   @media (max-width: 1024px) {
93 |     flex-direction: column;
94 |     text-align: center;
95 |   }
96 | }
97 |
98 | #docs {
99 |   border-right: 1px solid var(--border);
100 |
101 |   @media (max-width: 1024px) {
102 |     border-right: none;
103 |     border-bottom: 1px solid var(--border);
104 |   }
105 | }
106 |
107 | #next-steps ul {
108 |   list-style: none;
109 |   padding: 0;
110 |   display: flex;
111 |   gap: 8px;
112 |   margin: 32px 0 0;
113 |
114 |   .logo {
115 |     height: 18px;
116 |   }
117 |
118 |   a {
119 |     color: var(--text-h);
120 |     font-size: 16px;
121 |     border-radius: 6px;
122 |     background: var(--social-bg);
123 |     display: flex;
124 |     padding: 6px 12px;
125 |     align-items: center;
126 |     gap: 8px;
127 |     text-decoration: none;
128 |     transition: box-shadow 0.3s;
129 |
130 |     &:hover {
131 |       box-shadow: var(--shadow);
132 |     }
133 |     .button-icon {
134 |       height: 18px;
135 |       width: 18px;
136 |     }
137 |   }
138 |
139 |   @media (max-width: 1024px) {
140 |     margin-top: 20px;
141 |     flex-wrap: wrap;
142 |     justify-content: center;
143 |
144 |     li {
145 |       flex: 1 1 calc(50% - 8px);
146 |     }
147 |
148 |     a {
149 |       width: 100%;
150 |       justify-content: center;
151 |       box-sizing: border-box;
152 |     }
153 |   }
154 | }
155 |
```

```
#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}

```

src/App.tsx TSX 15.3 KB 264 Lines

```tsx
import React, { Suspense, lazy } from 'react'
import { BrowserRouter, Routes, Route, Navigate, useLocation } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { Toaster } from '@/components/ui/sonner'
import { QueryClientProvider } from '@tanstack/react-query'
import { queryClient } from './lib/query-client'
import { AuthProvider, useAuth } from './hooks/useAuth'
import { ProtectedRoute, ApprovedRoute, RoleRoute } from './components/ProtectedRoute'
import { LandingPage } from './pages/LandingPage'
import { FirstTimeProfileSetupModal } from './components/profile/FirstTimeProfileSetupModal'

// Lazy loaded page components to drastically reduce initial bundle size and TBT
const Login = lazy(() => import('./pages/Login').then((m) => ({ default: m.Login })))
const Register = lazy(() => import('./pages/Register').then((m) => ({ default: m.Register })))
const SetupPassword = lazy(() => import('./pages/SetupPassword').then((m) => ({ default: m.SetupPassword })))
const ForgotPassword = lazy(() => import('./pages/ForgotPassword').then((m) => ({ default: m.ForgotPassword })))
const PendingApprovalPage = lazy(() => import('./pages/PendingApprovalPage').then((m) => ({ default: m.PendingApprovalPage
})))
const AccountSuspendedPage = lazy(() => import('./pages/AccountSuspendedPage').then((m) => ({ default: m.AccountSuspendedPage
})))
const AccessDeniedPage = lazy(() => import('./pages/AccessDeniedPage').then((m) => ({ default: m.AccessDeniedPage })))
const NotFoundPage = lazy(() => import('./pages/NotFoundPage').then((m) => ({ default: m.NotFoundPage })))
const SupabaseTest = lazy(() => import('./pages/SupabaseTest').then((m) => ({ default: m.SupabaseTest })))

// Dashboards (heavy Recharts & Supabase admin logic)
const TraineeDashboard = lazy(() => import('./pages/Dashboards').then((m) => ({ default: m.TraineeDashboard })))
const AdminDashboard = lazy(() => import('./pages/Dashboards').then((m) => ({ default: m.AdminDashboard })))
const SuperAdminDashboard = lazy(() => import('./pages/Dashboards').then((m) => ({ default: m.SuperAdminDashboard })))

// Course & Trainee features
const TraineeCourseCatalog = lazy(() => import('./features/courses/TraineeCourseCatalog').then((m) => ({ default: m.TraineeCou
rseCatalog })))
const TraineeCourseDetails = lazy(() => import('./features/courses/TraineeCourseDetails').then((m) => ({ default: m.TraineeCou
rseDetails })))
const TraineeCourseLearnPage = lazy(() => import('./features/courses/TraineeCourseLearnPage').then((m) => ({ default: m.Traine
eCourseLearnPage })))
const TraineeAssessmentTest = lazy(() => import('./features/courses/TraineeAssessmentTest').then((m) => ({ default: m.TraineeA
ssessmentTest })))
const TraineeAssessmentsHub = lazy(() => import('./features/courses/TraineeAssessmentsHub').then((m) => ({ default: m.TraineeA
ssessmentsHub })))
const TraineeMyLearning = lazy(() => import('./features/courses/TraineeMyLearning').then((m) => ({ default: m.TraineeMyLearnin
g })))
const CourseMaterials = lazy(() => import('./features/courses/CourseMaterials').then((m) => ({ default: m.CourseMaterials })))

// Trainer features
const TrainerDashboard = lazy(() => import('./features/trainer/TrainerDashboard').then((m) => ({ default: m.TrainerDashboard
})))
const CourseListPage = lazy(() => import('./features/trainer/CourseListPage').then((m) => ({ default: m.CourseListPage })))
const CourseCreatePage = lazy(() => import('./features/trainer/CourseCreatePage').then((m) => ({ default: m.CourseCreatePage
})))
const CourseEditPage = lazy(() => import('./features/trainer/CourseEditPage').then((m) => ({ default: m.CourseEditPage })))
const AssessmentsPage = lazy(() => import('./features/trainer/AssessmentsPage').then((m) => ({ default: m.AssessmentsPage })))
const PerformancePage = lazy(() => import('./features/trainer/PerformancePage').then((m) => ({ default: m.PerformancePage })))
const CourseSessionsPage = lazy(() => import('./features/trainer/CourseSessionsPage').then((m) => ({ default: m.CourseSessions
Page })))
const TrainerProfile = lazy(() => import('./features/trainer/TrainerProfile').then((m) => ({ default: m.TrainerProfile })))
const TrainerSkills = lazy(() => import('./features/trainer/TrainerSkills').then((m) => ({ default: m.TrainerSkills })))
const NotificationsPage = lazy(() => import('./features/notifications/UniversalNotificationsPage').then((m) => ({ default: m.U
niversalNotificationsPage })))
const CourseDetailPage = lazy(() => import('./features/trainer/CourseDetailPage').then((m) => ({ default: m.CourseDetailPage
})))

// Public & Profile/Settings pages
const ProfilePage = lazy(() => import('./pages/ProfilePage').then((m) => ({ default: m.ProfilePage })))
const SettingsPage = lazy(() => import('./pages/SettingsPage').then((m) => ({ default: m.SettingsPage })))
const PublicCourseCatalog = lazy(() => import('./pages/PublicCourseCatalog').then((m) => ({ default: m.PublicCourseCatalog
})))
const PublicCourseDetails = lazy(() => import('./pages/PublicCourseDetails').then((m) => ({ default: m.PublicCourseDetails
})))
const AboutPage = lazy(() => import('./pages/AboutPage').then((m) => ({ default: m.AboutPage })))
const ContactPage = lazy(() => import('./pages/ContactPage').then((m) => ({ default: m.ContactPage })))
```

```
const AdminCourseCreatePage = lazy(() => import('./features/admin/AdminCourseCreatePage').then((m) => ({ default: m.AdminCourseCreatePage })))
const AdminCourseEditPage = lazy(() => import('./features/admin/AdminCourseEditPage').then((m) => ({ default: m.AdminCourseEditPage })))
const ChatBot = lazy(() => import('./components/ChatBot').then((m) => ({ default: m.ChatBot })))

function RouteThemeManager() {
  const location = useLocation()

  React.useEffect(() => {
    // Smooth scroll to top on navigation
    window.scrollTo(0, 0)

    document.documentElement.classList.add('theme-light')
    document.documentElement.classList.remove('theme-dark')
    document.body.classList.add('theme-light')
    document.body.classList.remove('theme-dark')
    document.body.style.backgroundColor = '#FAF9F6'
  }, [location.pathname])

  return null
}

function AtmosphericThemeVeil() {
  return (
    <motion.div
      initial={false}
      animate={{
        opacity: 1,
      }}
      transition={{
        duration: 0.85,
        ease: [0.16, 1, 0.3, 1],
      }}
      className="fixed inset-0 pointer-events-none -z-10 overflow-hidden"
    >
      {/* Radiant dawn ambient glow layer */}
      <div className="absolute inset-0 bg-[#FAF9F6] transition-opacity duration-700" />
      <div className="absolute -top-40 left-1/2 -translate-x-1/2 w-[800px] h-[500px] rounded-full bg-gradient-to-r from-purple-400/10 via-pink-400/10 to-orange-400/10 blur-[120px]" />
    </motion.div>
  )
}

function DashboardRedirect() {
  const { session, profile, loading } = useAuth()
  if (loading) return (
    <div className="min-h-screen bg-[#040814] flex items-center justify-center">
      <div className="w-8 h-8 border-2 border-cyan-500/30 border-t-transparent rounded-full animate-spin" />
    </div>
  )
  if (!session) return <Navigate to="/login" replace />
  if (profile?.role === 'super_admin') return <Navigate to="/super-admin" replace />
  if (profile?.role === 'admin') return <Navigate to="/admin" replace />
  if (profile?.role === 'trainer') return <Navigate to="/trainer" replace />
  return <Navigate to="/trainee" replace />
}

function AnimatedAppRoutes() {
  const location = useLocation()
  const isLandingPage = location.pathname === '/'

  // Transition smoothly between different page tiers
  const routeKey = isLandingPage
    ? 'landing'
    : location.pathname === '/login' || location.pathname === '/register'
    ? 'auth'
    : location.pathname.startsWith('/trainer')
    ? 'trainer'
    : location.pathname.startsWith('/trainee')
    ? 'trainee'
    : location.pathname.startsWith('/admin') || location.pathname.startsWith('/super-admin')
    ? 'admin'
    : location.pathname

  return (
    <>
```

```
      <RouteThemeManager />
      <AtmosphericThemeVeil />
      <AnimatePresence mode="wait" initial={false}>
        <motion.div
          key={routeKey}
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          transition={{ duration: 0.35, ease: 'easeInOut' }}
          className="min-h-screen w-full"
        >
          <Suspense
            fallback={
              <div className="min-h-screen bg-[#030712] flex items-center justify-center">
                <div className="w-8 h-8 border-2 border-cyan-500 border-t-transparent rounded-full animate-spin" />
              </div>
            }
          >
            <Routes location={location}>
              <Route path="/" element={<LandingPage />} />
              <Route path="/dashboard" element={<DashboardRedirect />} />
              <Route path="/login" element={<Login />} />
              <Route path="/register" element={<Register />} />
              <Route path="/setup-password" element={<SetupPassword />} />
              <Route path="/forgot-password" element={<ForgotPassword />} />
              <Route path="/supabase-test" element={<SupabaseTest />} />
              <Route path="/db" element={<SupabaseTest />} />

              {/* Public Course Pages & About */}
              <Route path="/courses" element={<PublicCourseCatalog />} />
              <Route path="/courses/:courseId" element={<PublicCourseDetails />} />
              <Route path="/about" element={<AboutPage />} />
              <Route path="/contact" element={<ContactPage />} />

              {/* Auth Fallbacks */}
              <Route path="/pending-approval" element={<PendingApprovalPage />} />
              <Route path="/account-suspended" element={<AccountSuspendedPage />} />
              <Route path="/access-denied" element={<AccessDeniedPage />} />

              {/* Base Protected Routes */}
              <Route element={<ProtectedRoute />}>
                <Route element={<ApprovedRoute />}>

                  {/* Shared / Permitted Preview Assessment Route for Trainees and Admins */}
                  <Route element={<RoleRoute allowedRoles={['trainee', 'admin', 'super_admin']} />}>
                    <Route path="/trainee/courses/:courseId/assessments/:assessmentId" element={<TraineeAssessmentTest />} />
                  </Route>

                  {/* Trainee routes */}
                  <Route element={<RoleRoute allowedRoles={['trainee']} />}>
                    <Route path="/trainee" element={<TraineeDashboard />} />
                    <Route path="/trainee/courses" element={<TraineeCourseCatalog />} />
                    <Route path="/trainee/courses/:courseId" element={<TraineeCourseDetails />} />
                    <Route path="/trainee/courses/:courseId/learn" element={<TraineeCourseLearnPage />} />
                    <Route path="/trainee/assessments" element={<TraineeAssessmentsHub />} />
                    <Route path="/trainee/my-learning" element={<TraineeMyLearning />} />
                    <Route path="/trainee/notifications" element={<NotificationsPage />} />
                    <Route path="/trainee/profile" element={<ProfilePage />} />
                    <Route path="/trainee/settings" element={<Navigate to="/trainee/profile" replace />} />
                  </Route>

                  {/* Trainer routes */}
                  <Route element={<RoleRoute allowedRoles={['trainer']} />}>
                    <Route path="/trainer" element={<TrainerDashboard />} />
                    <Route path="/trainer/courses" element={<CourseListPage />} />
                    <Route path="/trainer/courses/new" element={<CourseCreatePage />} />
                    <Route path="/trainer/courses/:courseId" element={<CourseDetailPage />} />
                    <Route path="/trainer/courses/:courseId/edit" element={<CourseEditPage />} />
                    <Route path="/trainer/courses/:courseId/sessions" element={<CourseSessionsPage />} />
                    <Route path="/trainer/courses/:courseId/materials" element={<CourseMaterials />} />
                    <Route path="/trainer/courses/:courseId/assessments" element={<AssessmentsPage />} />
                    <Route path="/trainer/courses/:courseId/performance" element={<PerformancePage />} />
                    <Route path="/trainer/notifications" element={<NotificationsPage />} />
                    <Route path="/trainer/profile" element={<ProfilePage />} />
                    <Route path="/trainer/settings" element={<Navigate to="/trainer/profile" replace />} />
                    <Route path="/trainer/skills" element={<TrainerSkills />} />
                  </Route>

                  {/* Admin routes */}
```

```
                  <Route element={<RoleRoute allowedRoles={['admin', 'super_admin']} />}>
                    <Route path="/admin" element={<AdminDashboard />} />
                    <Route path="/admin/courses" element={<AdminDashboard />} />
                    <Route path="/admin/courses/:courseId" element={<CourseDetailPage />} />
                    <Route path="/admin/courses/:courseId/edit" element={<AdminCourseEditPage />} />
                    <Route path="/admin/courses/:courseId/sessions" element={<CourseSessionsPage />} />
                    <Route path="/admin/courses/:courseId/materials" element={<CourseMaterials />} />
                    <Route path="/admin/courses/:courseId/assessments" element={<AssessmentsPage />} />
                    <Route path="/admin/courses/:courseId/performance" element={<PerformancePage />} />
                    <Route path="/admin/assessments" element={<AdminDashboard />} />
                    <Route path="/admin/notifications" element={<AdminDashboard />} />
                    <Route path="/admin/announcements" element={<AdminDashboard />} />
                    <Route path="/admin/messages" element={<AdminDashboard />} />
                    <Route path="/admin/courses/new" element={<AdminCourseCreatePage />} />
                    <Route path="/admin/profile" element={<ProfilePage />} />
                    <Route path="/admin/settings" element={<Navigate to="/admin/profile" replace />} />
                  </Route>

                  {/* Super Admin routes */}
                  <Route element={<RoleRoute allowedRoles={['super_admin']} />}>
                    <Route path="/super-admin" element={<SuperAdminDashboard />} />
                    <Route path="/super-admin/profile" element={<ProfilePage />} />
                    <Route path="/super-admin/settings" element={<Navigate to="/super-admin/profile" replace />} />
                  </Route>

                </Route>
              </Route>

              <Route path="*" element={<NotFoundPage />} />
            </Routes>
          </Suspense>
        </motion.div>
      </AnimatePresence>
    </>
  )
}

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <BrowserRouter>
          <AnimatedAppRoutes />
          <FirstTimeProfileSetupModal />
          <Suspense fallback={null}>
            <ChatBot />
          </Suspense>
          <Toaster position="top-right" richColors />
        </BrowserRouter>
      </AuthProvider>
    </QueryClientProvider>
  )
}
```

src/index.css CSS 5.4 KB 225 Lines

```css
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  * {
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
  }
  html {
    scroll-behavior: smooth;
  }
  body {
    font-family: "SF Pro Display", -apple-system, BlinkMacSystemFont, "SF Pro Text", "Inter", sans-serif;
    letter-spacing: -0.011em;
  }
}

@layer base {
  :root {
    /* Warm White / Cream Background #FAF9F6 */
    --background: 40 20% 98%;
    /* Deep Midnight Purple #1E1333 */
    --foreground: 260 45% 14%;

    /* Cards: Pure White with subtle crisp contrast */
    --card: 0 0% 100%;
    --card-foreground: 260 45% 14%;

    /* Popover */
    --popover: 0 0% 100%;
    --popover-foreground: 260 45% 14%;

    /* Primary: Royal Purple #6B4BA3 */
    --primary: 262 37% 47%;
    --primary-foreground: 0 0% 100%;

    /* Secondary: Radiant Magenta Pink #EA519D */
    --secondary: 330 80% 62%;
    --secondary-foreground: 0 0% 100%;

    /* Accent: Vivid Amber Orange #F3841D */
    --accent: 29 90% 53%;
    --accent-foreground: 0 0% 100%;

    /* Muted: Soft Violet Mist #ECE5F6 */
    --muted: 260 40% 96%;
    --muted-foreground: 262 25% 45%;

    /* Destructive */
    --destructive: 0 84% 60%;
    --destructive-foreground: 0 0% 100%;

    /* Borders & Inputs */
    --border: 260 20% 91%;
    --input: 260 20% 91%;
    --ring: 262 37% 47%;

    --radius: 0.75rem;
  }

  .dark {
    /* Balanced Deep Space Navy #0B0F19 */
```

```
    --background: 222 47% 7%;
    /* Soft Crisp Text #F8FAFC */
    --foreground: 210 40% 98%;

    /* Cards: Elevated Dark Slate #111827 */
    --card: 222 47% 11%;
    --card-foreground: 210 40% 98%;

    /* Popover */
    --popover: 222 47% 11%;
    --popover-foreground: 210 40% 98%;

    /* Primary: Royal Indigo / Purple */
    --primary: 262 60% 60%;
    --primary-foreground: 0 0% 100%;

    /* Secondary: Radiant Magenta Pink */
    --secondary: 330 80% 62%;
    --secondary-foreground: 0 0% 100%;

    /* Accent: Vivid Amber */
    --accent: 29 90% 53%;
    --accent-foreground: 0 0% 100%;

    /* Muted: Soft Slate */
    --muted: 217 33% 17%;
    --muted-foreground: 215 20% 65%;

    /* Destructive */
    --destructive: 0 62% 30%;
    --destructive-foreground: 210 40% 98%;

    /* Borders & Inputs */
    --border: 217 33% 20%;
    --input: 217 33% 20%;
    --ring: 262 60% 60%;
  }
}

@layer base {
  * {
    border-color: hsl(var(--border));
  }
  body {
    background-color: hsl(var(--background));
    color: hsl(var(--foreground));
    transition: background-color 0.4s cubic-bezier(0.16, 1, 0.3, 1), color 0.4s cubic-bezier(0.16, 1, 0.3, 1);
  }
  body.theme-light {
    background-color: #FAF9F6;
    color: #1E1333;
  }
  body.theme-dark {
    background-color: #0B0F19;
    color: #F8FAFC;
  }
}

/* Custom Space / Glassmorphism / Earth Sciences Utilities */
@layer utilities {
  .bg-cream {
    background-color: #FAF9F6;
  }
  .text-midnight {
    color: #1E1333;
  }
  .bg-space-mesh {
```

```
    background-image:
      radial-gradient(at 15% 20%, rgba(107, 75, 163, 0.18) 0px, transparent 50%),
      radial-gradient(at 85% 30%, rgba(234, 81, 157, 0.14) 0px, transparent 50%),
      radial-gradient(at 50% 80%, rgba(243, 132, 29, 0.12) 0px, transparent 50%),
      radial-gradient(at 80% 90%, rgba(14, 165, 233, 0.12) 0px, transparent 50%);
  }

  .cosmic-grid {
    background-size: 40px 40px;
    background-image:
      linear-gradient(to right, rgba(255, 255, 255, 0.03) 1px, transparent 1px),
      linear-gradient(to bottom, rgba(255, 255, 255, 0.03) 1px, transparent 1px);
  }

  .cosmic-grid-fine {
    background-size: 20px 20px;
    background-image:
      linear-gradient(to right, rgba(255, 255, 255, 0.02) 1px, transparent 1px),
      linear-gradient(to bottom, rgba(255, 255, 255, 0.02) 1px, transparent 1px);
  }

  .text-glow-purple {
    text-shadow: 0 0 24px rgba(107, 75, 163, 0.65), 0 0 48px rgba(234, 81, 157, 0.35);
  }

  .text-glow-orange {
    text-shadow: 0 0 24px rgba(243, 132, 29, 0.65), 0 0 48px rgba(255, 142, 122, 0.35);
  }

  .box-glow-purple {
    box-shadow: 0 0 30px -5px rgba(107, 75, 163, 0.35);
  }

  .box-glow-pink {
    box-shadow: 0 0 30px -5px rgba(234, 81, 157, 0.35);
  }

  .box-glow-cyan {
    box-shadow: 0 0 30px -5px rgba(14, 165, 233, 0.35);
  }
}

@keyframes orbitSlow {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

@keyframes pulseGlow {
  0%, 100% { opacity: 0.4; transform: scale(1); }
  50% { opacity: 0.8; transform: scale(1.05); }
}

@keyframes marqueeScroll {
  0% {
    transform: translate3d(0, 0, 0);
  }
  100% {
    transform: translate3d(-50%, 0, 0);
  }
}

.animate-orbit-slow {
  animation: orbitSlow 60s linear infinite;
}

.animate-pulse-glow {
  animation: pulseGlow 6s ease-in-out infinite;
}

.animate-marquee-infinite {
  display: flex;
  width: max-content;
  animation: marqueeScroll 45s linear infinite;
  will-change: transform;
}

.group:hover .animate-marquee-infinite,
.animate-marquee-infinite:hover {
  animation-play-state: paused !important;
```

```
}

/* Utility to hide scrollbars while preserving scroll functionality */

.hide-scrollbar {
  -ms-overflow-style: none;
  scrollbar-width: none;
}
.hide-scrollbar::-webkit-scrollbar {
  display: none;
}
```

src/main.tsx TSX 0.2 KB 11 Lines

```tsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.tsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

src/integrations/supabase/types.ts TS 32.9 KB 1,116 Lines

```ts
export type Json =
  | string
  | number
  | boolean
  | null
  | { [key: string]: Json | undefined }
  | Json[]

export interface Database {
  public: {
    Tables: {
      home_page_settings: {
        Row: {
          id: string
          featured_programs_enabled: boolean
          featured_programs_tag: string
          featured_programs_title: string
          featured_programs_subtitle: string
          featured_programs_btn_text: string
          featured_programs_btn_link: string
          featured_programs_items: Json
          announcements_bar_enabled: boolean
          announcements_bar_label: string
          announcements_bar_speed: number
          announcements_items: Json
          upcoming_tracks_enabled: boolean
          upcoming_tracks_tag: string
          upcoming_tracks_title: string
          upcoming_tracks_subtitle: string
          upcoming_tracks_btn_text: string
          upcoming_tracks_items: Json
          updated_at: string
          updated_by: string | null
        }
        Insert: {
          id?: string
          featured_programs_enabled?: boolean
          featured_programs_tag?: string
          featured_programs_title?: string
          featured_programs_subtitle?: string
          featured_programs_btn_text?: string
          featured_programs_btn_link?: string
          featured_programs_items?: Json
          announcements_bar_enabled?: boolean
          announcements_bar_label?: string
          announcements_bar_speed?: number
          announcements_items?: Json
          upcoming_tracks_enabled?: boolean
          upcoming_tracks_tag?: string
          upcoming_tracks_title?: string
          upcoming_tracks_subtitle?: string
          upcoming_tracks_btn_text?: string
          upcoming_tracks_items?: Json
          updated_at?: string
          updated_by?: string | null
        }
        Update: {
          id?: string
          featured_programs_enabled?: boolean
          featured_programs_tag?: string
          featured_programs_title?: string
          featured_programs_subtitle?: string
          featured_programs_btn_text?: string
          featured_programs_btn_link?: string
          featured_programs_items?: Json
          announcements_bar_enabled?: boolean
          announcements_bar_label?: string
          announcements_bar_speed?: number
          announcements_items?: Json
          upcoming_tracks_enabled?: boolean
          upcoming_tracks_tag?: string
          upcoming_tracks_title?: string
          upcoming_tracks_subtitle?: string
          upcoming_tracks_btn_text?: string
          upcoming_tracks_items?: Json
          updated_at?: string
```

```
          updated_by?: string | null
        }
        Relationships: []
      }
      admins: {
        Row: {
          id: string
          full_name: string
          email: string | null
          role: 'admin' | 'super_admin'
          approval_status: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path: string | null
          password: string | null
          created_at: string
          updated_at: string
        }
        Insert: {
          id: string
          full_name?: string
          email?: string | null
          role?: 'admin' | 'super_admin'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          password?: string | null
          created_at?: string
          updated_at?: string
        }
        Update: {
          id?: string
          full_name?: string
          email?: string | null
          role?: 'admin' | 'super_admin'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          password?: string | null
          created_at?: string
          updated_at?: string
        }
        Relationships: []
      }
      trainers: {
        Row: {
          id: string
          full_name: string
          email: string | null
          role: 'trainer'
          approval_status: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path: string | null
          proof_path: string | null
          biodata_path: string | null
          study_details: string | null
          bio: string | null
          years_of_experience: number | null
          qualifications: string | null
          availability: string | null
          mobile_number: string | null
          is_mobile_verified: boolean
          password: string | null
          created_at: string
          updated_at: string
          work_experience: string | null
          interests: string | null
        }
        Insert: {
          id: string
          full_name?: string
          email?: string | null
          role?: 'trainer'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          proof_path?: string | null
          biodata_path?: string | null
          study_details?: string | null
          bio?: string | null
          years_of_experience?: number | null
          qualifications?: string | null
          availability?: string | null
          mobile_number?: string | null
          is_mobile_verified?: boolean
```

```
          is_email_verified?: boolean
          work_experience?: string | null
          interests?: string | null
          created_at?: string
          updated_at?: string
        }
        Update: {
          id?: string
          full_name?: string
          email?: string | null
          role?: 'trainer'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          proof_path?: string | null
          biodata_path?: string | null
          study_details?: string | null
          bio?: string | null
          years_of_experience?: number | null
          qualifications?: string | null
          availability?: string | null
          mobile_number?: string | null
          is_mobile_verified?: boolean
          is_email_verified?: boolean
          password?: string | null
          work_experience?: string | null
          interests?: string | null
          created_at?: string
          updated_at?: string
        }
        Relationships: []
      }
      trainees: {
        Row: {
          id: string
          full_name: string
          email: string | null
          role: 'trainee'
          approval_status: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path: string | null
          proof_path: string | null
          department: string | null
          designation: string | null
          mobile_number: string | null
          is_mobile_verified: boolean
          is_email_verified: boolean
          password: string | null
          created_at: string
          updated_at: string
          qualifications: string | null
          work_experience: string | null
          interests: string | null
        }
        Insert: {
          id: string
          full_name?: string
          email?: string | null
          role?: 'trainee'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          proof_path?: string | null
          department?: string | null
          designation?: string | null
          mobile_number?: string | null
          is_mobile_verified?: boolean
          is_email_verified?: boolean
          password?: string | null
          created_at?: string
          updated_at?: string
        }
        Update: {
          id?: string
          full_name?: string
          email?: string | null
          role?: 'trainee'
          approval_status?: 'pending' | 'approved' | 'rejected' | 'suspended'
          avatar_path?: string | null
          proof_path?: string | null
          department?: string | null
          designation?: string | null
```

```
          mobile_number?: string | null
          is_mobile_verified?: boolean
          is_email_verified?: boolean
          password?: string | null
          created_at?: string
          updated_at?: string
        }
        Relationships: []
      }
      courses: {
        Row: {
          id: string
          title: string
          description: string | null
          course_type: string
          trainer_id: string | null
          department: string | null
          duration_minutes: number | null
          passing_score: number | null
          status: 'draft' | 'pending_review' | 'published' | 'archived'
          thumbnail_path: string | null
          learning_objectives: Json | null
          competencies: Json | null
          created_at: string
          updated_at: string
          published_at: string | null
          meet_link: string | null
          start_date: string | null
          end_date: string | null
          delivery_mode: string | null
          session_flow_text: string | null
          session_flow_document_path: string | null
          live_class_timing: string | null
          mock_test_timing: string | null
          final_exam_timing: string | null
          final_test_date: string | null
          final_test_start_time: string | null
          final_test_end_time: string | null
          planned_assessments_count: number | null
          planned_mock_tests_count: number | null
          max_trainees: number | null
          trainer_suggestion: string | null
        }
        Insert: {
          id?: string
          title: string
          description?: string | null
          course_type?: string
          trainer_id?: string | null
          department?: string | null
          duration_minutes?: number | null
          passing_score?: number
          status?: 'draft' | 'pending_review' | 'published' | 'archived'
          thumbnail_path?: string | null
          learning_objectives?: Json | null
          competencies?: Json | null
          created_at?: string
          updated_at?: string
          published_at?: string | null
          meet_link?: string | null
          start_date?: string | null
          end_date?: string | null
          delivery_mode?: string | null
          session_flow_text?: string | null
          session_flow_document_path?: string | null
          live_class_timing?: string | null
          mock_test_timing?: string | null
          final_exam_timing?: string | null
          final_test_date?: string | null
          final_test_start_time?: string | null
          final_test_end_time?: string | null
          planned_assessments_count?: number | null
          planned_mock_tests_count?: number | null
          max_trainees?: number | null
          trainer_suggestion?: string | null
        }
        Update: {
          id?: string
          title?: string
```

```
          description?: string | null
          course_type?: string
          trainer_id?: string | null
          department?: string | null
          duration_minutes?: number | null
          passing_score?: number
          status?: 'draft' | 'pending_review' | 'published' | 'archived'
          thumbnail_path?: string | null
          learning_objectives?: Json | null
          competencies?: Json | null
          created_at?: string
          updated_at?: string
          published_at?: string | null
          meet_link?: string | null
          start_date?: string | null
          end_date?: string | null
          delivery_mode?: string | null
          session_flow_text?: string | null
          session_flow_document_path?: string | null
          live_class_timing?: string | null
          mock_test_timing?: string | null
          final_exam_timing?: string | null
          final_test_date?: string | null
          final_test_start_time?: string | null
          final_test_end_time?: string | null
          planned_assessments_count?: number | null
          planned_mock_tests_count?: number | null
          max_trainees?: number | null
          trainer_suggestion?: string | null
        }
        Relationships: []
      }
      course_sessions: {
        Row: {
          id: string
          course_id: string
          title: string
          description: string | null
          start_time: string | null
          end_time: string | null
          meet_link: string | null
          order_index: number
          created_at: string
          updated_at: string
          session_type: string | null
          location: string | null
        }
        Insert: {
          id?: string
          course_id: string
          title: string
          description?: string | null
          start_time?: string | null
          end_time?: string | null
          meet_link?: string | null
          order_index?: number
          created_at?: string
          updated_at?: string
          session_type?: string | null
          location?: string | null
        }
        Update: {
          id?: string
          course_id?: string
          title?: string
          description?: string | null
          start_time?: string | null
          end_time?: string | null
          meet_link?: string | null
          order_index?: number
          created_at?: string
          updated_at?: string
          session_type?: string | null
          location?: string | null
        }
        Relationships: [
          {
            foreignKeyName: "course_sessions_course_id_fkey"
            columns: ["course_id"]
```

```
            isOneToOne: false
            referencedRelation: "courses"
            referencedColumns: ["id"]
          }
        ]
      }
      materials: {
        Row: {
          id: string
          course_id: string
          uploaded_by: string
          file_name: string
          storage_path: string
          mime_type: string | null
          file_size: number | null
          extracted_text: string | null
          extraction_status: 'pending' | 'completed' | 'failed'
          material_type: 'file' | 'link' | 'video'
          url: string | null
          created_at: string
          session_id: string | null
        }
        Insert: {
          id?: string
          course_id: string
          uploaded_by: string
          file_name: string
          storage_path: string
          mime_type?: string | null
          file_size?: number | null
          extracted_text?: string | null
          extraction_status?: 'pending' | 'completed' | 'failed'
          material_type?: 'file' | 'link' | 'video'
          url?: string | null
          created_at?: string
          session_id?: string | null
        }
        Update: {
          id?: string
          course_id?: string
          uploaded_by?: string
          file_name?: string
          storage_path?: string
          mime_type?: string | null
          file_size?: number | null
          extracted_text?: string | null
          extraction_status?: 'pending' | 'completed' | 'failed'
          material_type?: 'file' | 'link' | 'video'
          url?: string | null
          created_at?: string
          session_id?: string | null
        }
        Relationships: []
      }
      enrollments: {
        Row: {
          id: string
          user_id: string
          course_id: string
          status: 'pending_approval' | 'enrolled' | 'in_progress' | 'completed' | 'withdrawn' | 'rejected'
          progress_percent: number
          enrolled_at: string
          completed_at: string | null
        }
        Insert: {
          id?: string
          user_id: string
          course_id: string
          status?: 'pending_approval' | 'enrolled' | 'in_progress' | 'completed' | 'withdrawn' | 'rejected'
          progress_percent?: number
          enrolled_at?: string
          completed_at?: string | null
        }
        Update: {
          id?: string
          user_id?: string
          course_id?: string
          status?: 'pending_approval' | 'enrolled' | 'in_progress' | 'completed' | 'withdrawn' | 'rejected'
          progress_percent?: number
```

```
          enrolled_at?: string
          completed_at?: string | null
        }
        Relationships: []
      }
      assessments: {
        Row: {
          id: string
          course_id: string
          title: string
          instructions: string | null
          passing_score: number
          status: 'draft' | 'pending_review' | 'published' | 'archived'
          created_by: string
          created_at: string
          updated_at: string
          assessment_type: string
          requires_sea: boolean
          sea_link: string | null
          scheduled_date: string | null
          start_time: string | null
          end_time: string | null
          duration_minutes: number | null
            results_publish_date: string | null
            is_adaptive: boolean | null
            is_simulation: boolean | null
            simulation_dataset_url: string | null
        }
        Insert: {
          id?: string
          course_id: string
          title: string
          instructions?: string | null
          passing_score?: number
          status?: 'draft' | 'pending_review' | 'published' | 'archived'
          created_by: string
          created_at?: string
          updated_at?: string
          assessment_type?: string
          requires_sea?: boolean
          sea_link?: string | null
          scheduled_date?: string | null
          start_time?: string | null
          end_time?: string | null
          duration_minutes?: number | null
            results_publish_date?: string | null
            is_adaptive?: boolean | null
            is_simulation?: boolean | null
            simulation_dataset_url?: string | null
        }
        Update: {
          id?: string
          course_id?: string
          title?: string
          instructions?: string | null
          passing_score?: number
          status?: 'draft' | 'pending_review' | 'published' | 'archived'
          created_by?: string
          created_at?: string
          updated_at?: string
          assessment_type?: string
          requires_sea?: boolean
          sea_link?: string | null
          scheduled_date?: string | null
          start_time?: string | null
          end_time?: string | null
          duration_minutes?: number | null
            results_publish_date?: string | null
            is_adaptive?: boolean | null
            is_simulation?: boolean | null
            simulation_dataset_url?: string | null
        }
        Relationships: []
      }
      questions: {
        Row: {
          id: string
          assessment_id: string
          question_text: string
```

```
          options: Json
          correct_answer: string
          explanation: string | null
          position: number
          approved: boolean
          created_at: string
            difficulty: 'easy' | 'medium' | 'hard' | null
          question_type: string | null
        }
        Insert: {
          id?: string
          assessment_id: string
          question_text: string
          options: Json
          correct_answer: string
          explanation?: string | null
          position?: number
          approved?: boolean
          created_at?: string
            difficulty?: 'easy' | 'medium' | 'hard' | null
          question_type?: string | null
        }
        Update: {
          id?: string
          assessment_id?: string
          question_text?: string
          options?: Json
          correct_answer?: string
          explanation?: string | null
          position?: number
          approved?: boolean
          created_at?: string
            difficulty?: 'easy' | 'medium' | 'hard' | null
          question_type?: string | null
        }
        Relationships: []
      }
      assessment_attempts: {
        Row: {
          id: string
          assessment_id: string
          user_id: string
          score: number | null
          passed: boolean | null
          started_at: string
          submitted_at: string | null
        }
        Insert: {
          id?: string
          assessment_id: string
          user_id: string
          score?: number | null
          passed?: boolean | null
          started_at?: string
          submitted_at?: string | null
        }
        Update: {
          id?: string
          assessment_id?: string
          user_id?: string
          score?: number | null
          passed?: boolean | null
          started_at?: string
          submitted_at?: string | null
        }
        Relationships: []
      }
      attempt_answers: {
        Row: {
          id: string
          attempt_id: string
          question_id: string
          selected_answer: string
          is_correct: boolean
          created_at: string
        }
        Insert: {
          id?: string
          attempt_id: string
```

```
          question_id: string
          selected_answer: string
          is_correct: boolean
          created_at?: string
        }
        Update: {
          id?: string
          attempt_id?: string
          question_id?: string
          selected_answer?: string
          is_correct?: boolean
          created_at?: string
        }
        Relationships: []
      }
      certificates: {
        Row: {
          id: string
          certificate_number: string
          user_id: string
          course_id: string
          final_score: number
          storage_path: string
          verification_hash: string
          issued_at: string
          expires_at: string | null
          status: 'valid' | 'revoked'
          revoked_at: string | null
          revoked_by: string | null
        }
        Insert: {
          id?: string
          certificate_number: string
          user_id: string
          course_id: string
          final_score: number
          storage_path: string
          verification_hash: string
          issued_at?: string
          expires_at?: string | null
          status?: 'valid' | 'revoked'
          revoked_at?: string | null
          revoked_by?: string | null
        }
        Update: {
          id?: string
          certificate_number?: string
          user_id?: string
          course_id?: string
          final_score?: number
          storage_path?: string
          verification_hash?: string
          issued_at?: string
          expires_at?: string | null
          status?: 'valid' | 'revoked'
          revoked_at?: string | null
          revoked_by?: string | null
        }
        Relationships: []
      }
      skills: {
        Row: {
          id: string
          name: string
          category: string | null
          created_at: string
        }
        Insert: {
          id?: string
          name: string
          category?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          name?: string
          category?: string | null
          created_at?: string
        }
```

```
        Relationships: []
      }
      user_skills: {
        Row: {
          user_id: string
          skill_id: string
          level: number
        }
        Insert: {
          user_id: string
          skill_id: string
          level?: number
        }
        Update: {
          user_id?: string
          skill_id?: string
          level?: number
        }
        Relationships: []
      }
      course_skills: {
        Row: {
          course_id: string
          skill_id: string
          required_level: number
        }
        Insert: {
          course_id: string
          skill_id: string
          required_level?: number
        }
        Update: {
          course_id?: string
          skill_id?: string
          required_level?: number
        }
        Relationships: []
      }
      notifications: {
        Row: {
          id: string
          user_id: string
          type: string
          title: string
          message: string
          related_certificate_id: string | null
          read_at: string | null
          created_at: string
        }
        Insert: {
          id?: string
          user_id: string
          type: string
          title: string
          message: string
          related_certificate_id?: string | null
          read_at?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          user_id?: string
          type?: string
          title?: string
          message?: string
          related_certificate_id?: string | null
          read_at?: string | null
          created_at?: string
        }
        Relationships: []
      }
      audit_logs: {
        Row: {
          id: string
          actor_id: string | null
          action: string
          entity_type: string
          entity_id: string | null
          metadata: Json | null
```

```
          created_at: string
        }
        Insert: {
          id?: string
          actor_id?: string | null
          action: string
          entity_type: string
          entity_id?: string | null
          metadata?: Json | null
          created_at?: string
        }
        Update: {
          id?: string
          actor_id?: string | null
          action?: string
          entity_type?: string
          entity_id?: string | null
          metadata?: Json | null
          created_at?: string
        }
        Relationships: []
      }
      announcements: {
        Row: {
          id: string
          title: string
          content: string
          author_id: string
          target_audience: string
          is_active: boolean
          created_at: string
        }
        Insert: {
          id?: string
          title: string
          content: string
          author_id: string
          target_audience?: string
          is_active?: boolean
          created_at?: string
        }
        Update: {
          id?: string
          title?: string
          content?: string
          author_id?: string
          target_audience?: string
          is_active?: boolean
          created_at?: string
        }
        Relationships: []
      }

      course_announcements: {
        Row: {
          id: string
          course_id: string
          trainer_id: string
          title: string
          content: string
          created_at: string
          updated_at: string
        }
        Insert: {
          id?: string
          course_id: string
          trainer_id: string
          title: string
          content: string
          created_at?: string
          updated_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          trainer_id?: string
          title?: string
          content?: string
          created_at?: string
```

```
          updated_at?: string
        }
        Relationships: [
          {
            foreignKeyName: "course_announcements_course_id_fkey"
            columns: ["course_id"]
            isOneToOne: false
            referencedRelation: "courses"
            referencedColumns: ["id"]
          },
          {
            foreignKeyName: "course_announcements_trainer_id_fkey"
            columns: ["trainer_id"]
            isOneToOne: false
            referencedRelation: "users"
            referencedColumns: ["id"]
          }
        ]
      }
      course_messages: {
        Row: {
          id: string
          course_id: string
          sender_id: string
          content: string
          is_private: boolean
          recipient_id: string | null
          created_at: string
        }
        Insert: {
          id?: string
          course_id: string
          sender_id: string
          content: string
          is_private?: boolean
          recipient_id?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          sender_id?: string
          content?: string
          is_private?: boolean
          recipient_id?: string | null
          created_at?: string
        }
        Relationships: [
          {
            foreignKeyName: "course_messages_course_id_fkey"
            columns: ["course_id"]
            isOneToOne: false
            referencedRelation: "courses"
            referencedColumns: ["id"]
          },
          {
            foreignKeyName: "course_messages_sender_id_fkey"
            columns: ["sender_id"]
            isOneToOne: false
            referencedRelation: "users"
            referencedColumns: ["id"]
          }
        ]
      }
      course_feedback: {
        Row: {
          id: string
          course_id: string
          user_id: string
          rating: number
          comments: string | null
          created_at: string
        }
        Insert: {
          id?: string
          course_id: string
          user_id: string
          rating: number
          comments?: string | null
```

```
          created_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          user_id?: string
          rating?: number
          comments?: string | null
          created_at?: string
        }
        Relationships: []
      }
      scenario_modules: {
        Row: {
          id: string
          course_id: string
          title: string
          description: string | null
          created_at: string
        }
        Insert: {
          id?: string
          course_id: string
          title: string
          description?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          title?: string
          description?: string | null
          created_at?: string
        }
        Relationships: []
      }
      scenario_nodes: {
        Row: {
          id: string
          scenario_id: string
          question: string
          options: Json
          correct_option: string
          explanation: string | null
          position: number
          next_node_id: string | null
          score_value: number
        }
        Insert: {
          id?: string
          scenario_id: string
          question: string
          options: Json
          correct_option: string
          explanation?: string | null
          position?: number
          next_node_id?: string | null
          score_value?: number
        }
        Update: {
          id?: string
          scenario_id?: string
          question?: string
          options?: Json
          correct_option?: string
          explanation?: string | null
          position?: number
          next_node_id?: string | null
          score_value?: number
        }
        Relationships: []
      }
    }
    Views: {
      questions_safe: {
        Row: {
          id: string | null
          assessment_id: string | null
          question_text: string | null
```

```
          options: Json | null
          position: number | null
        }
        Relationships: []
      }
      scenario_nodes_safe: {
        Row: {
          id: string | null
          scenario_id: string | null
          question: string | null
          options: Json | null
          position: number | null
        }
        Relationships: []
      }
    }
    Functions: {
      current_user_role: {
        Args: Record<PropertyKey, never>
        Returns: string
      }
      admin_update_user: {
        Args: {
          target_user_id: string
          new_role: string
          new_status: string
        }
        Returns: void
      }
      admin_delete_user: {
        Args: {
          target_user_id: string
        }
        Returns: void
      }
      super_admin_promote_to_admin: {
        Args: {
          target_user_id: string
        }
        Returns: void
      }
      admin_update_course: {
        Args: {
          target_course_id: string
          new_status: string
        }
        Returns: void
      }
      generate_mobile_otp: {
        Args: {
          p_mobile: string
        }
        Returns: string
      }
      verify_mobile_otp: {
        Args: {
          p_mobile: string
          p_otp: string
          p_user_id: string
        }
        Returns: boolean
      }
      generate_otp: {
        Args: {
          p_identifier: string
          p_type: string
        }
        Returns: string
      }
      verify_otp: {
        Args: {
          p_identifier: string
          p_otp: string
          p_user_id: string
          p_type: string
        }
        Returns: boolean
      }
      check_otp_match: {
```

```
        Args: {
          p_identifier: string
          p_otp: string
          p_type: string
        }
        Returns: boolean
      }
    }
    Enums: {}
    CompositeTypes: {}
  }
}

```

src/lib/ai.ts TS 2.3 KB 90 Lines

```ts
import { supabase } from './supabase'

interface GeneratedQuestion {
  question_text: string
  options: { A: string; B: string; C: string; D: string }
  correct_answer: string
  explanation: string | null
}

interface GenerateQuestionsResponse {
  success: boolean
  questions_generated: number
  assessment_id: string
  questions: Array<{
    id: string
    question_text: string
    options: { A: string; B: string; C: string; D: string }
    correct_answer: string
    explanation: string | null
    position: number
  }>
}

export async function generateQuestionsFromMaterial(
  materialId: string,
  courseId: string,
  content: string,
  topic?: string
): Promise<GenerateQuestionsResponse> {
  const { data: { session } } = await supabase.auth.getSession()
  if (!session) throw new Error('Not authenticated')

  const { data, error } = await supabase.functions.invoke('generate-questions', {
    body: {
      material_id: materialId,
      course_id: courseId,
      content,
      topic,
    },
  })

  if (error) throw error
  return data as GenerateQuestionsResponse
}

export async function extractTextFromFile(
  file: File,
  mimeType: string
): Promise<string> {
  // For text-based files, read directly
  if (
    mimeType.startsWith('text/') ||
    mimeType === 'application/pdf' ||
    mimeType.includes('word') ||
    mimeType.includes('document') ||
    mimeType.includes('presentation')
  ) {
    return new Promise((resolve, reject) => {
      const reader = new FileReader()
      reader.onload = () => resolve(reader.result as string)
      reader.onerror = () => reject(new Error('Failed to read file'))
      reader.readAsText(file)
    })
  }

  // For other files, return empty (edge function will handle)
  return ''
}

export async function generateQuestionsFromUrl(
  url: string,
  courseId: string,
  title?: string
): Promise<GenerateQuestionsResponse> {
  // For URLs, we fetch the content server-side
  const { data: { session } } = await supabase.auth.getSession()
```

```
  if (!session) throw new Error('Not authenticated')

  const { data, error } = await supabase.functions.invoke('generate-questions', {
    body: {
      course_id: courseId,
      content: `External resource: ${url}\nTitle: ${title || url}`,
      topic: title || 'External Resource',
    },
  })

  if (error) throw error
  return data as GenerateQuestionsResponse
}

```

src/lib/certificateGenerator.ts TS 32.7 KB 900 Lines

```ts
import { jsPDF } from 'jspdf'
import JSZip from 'jszip'
import { supabase } from './supabase'

export interface CertificateData {
  traineeName: string
  traineeEmail?: string
  traineeId: string
  courseId: string
  courseTitle: string
  trainerName?: string
  percentage?: number | string
  completedAt?: string
  certificateId?: string
}

/**
 * Triggers a file download in the browser
 */
export function triggerFileDownload(blob: Blob, fileName: string) {
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a')
  a.href = url
  a.download = fileName
  document.body.appendChild(a)
  a.click()
  document.body.removeChild(a)
  setTimeout(() => URL.revokeObjectURL(url), 1000)
}

/**
 * Helper to draw text with letter spacing
 */
function drawCenteredSpacedText(
  ctx: CanvasRenderingContext2D,
  text: string,
  centerX: number,
  y: number,
  spacingPx: number
) {
  if ('letterSpacing' in ctx) {
    try {
      ;(ctx as any).letterSpacing = `${spacingPx}px`
      ctx.textAlign = 'center'
      ctx.fillText(text, centerX, y)
      ;(ctx as any).letterSpacing = '0px'
      return
    } catch {
      // fallback if letterSpacing assignment fails
    }
  }

  const chars = text.split('')
  ctx.textAlign = 'center'
  const charWidths = chars.map(c => ctx.measureText(c).width)
  const totalWidth = charWidths.reduce((a, b) => a + b, 0) + (chars.length - 1) * spacingPx
  let currentX = centerX - totalWidth / 2

  for (let i = 0; i < chars.length; i++) {
    ctx.fillText(chars[i], currentX + charWidths[i] / 2, y)
    currentX += charWidths[i] + spacingPx
  }
}

/**
 * Generates an exact 1-to-1 matching PDF certificate conforming to DBMS_Certificate_Template.pptx
 * Only dynamic texts (trainee name, course title, percentage score, date, trainer name) are replaced.
```

```
 */

export async function generatePdfCertificate(data: CertificateData): Promise<Blob> {
  // 16:9 Standard widescreen presentation canvas (high-res 2400 x 1350)

  const width = 2400
  const height = 1350

  const canvas = document.createElement('canvas')
  canvas.width = width
  canvas.height = height
  const ctx = canvas.getContext('2d')

  if (!ctx) {
    throw new Error('Canvas 2D context unavailable')
  }

  // 1. Pure Crisp White Background

  ctx.fillStyle = '#FFFFFF'
  ctx.fillRect(0, 0, width, height)

  // Scaling factor: PPTX uses 12192000 x 6858000 EMU

  const scale = width / 12192000

  // 2. Outer Deep Navy Border (Shape 0)

  // PPTX: x=365760, y=365760, w=11457432, h=6126480, line width 38100 (3pt), color #1F3864

  ctx.strokeStyle = '#1F3864'
  ctx.lineWidth = 7.5
  ctx.strokeRect(365760 * scale, 365760 * scale, 11457432 * scale, 6126480 * scale)

  // 3. Inner Muted Gold Border (Shape 1)

  // PPTX: x=548640, y=548640, w=11091672, h=5760720, line width 15875 (1.25pt), color #B8892B

  ctx.strokeStyle = '#B8892B'
  ctx.lineWidth = 3.2
  ctx.strokeRect(548640 * scale, 548640 * scale, 11091672 * scale, 5760720 * scale)

  // 4. Header: "CERTIFICATE" (Text 2)

  // PPTX: Georgia bold 46pt, color #1F3864, letter-spacing 6pt

  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 84px "Georgia", "Times New Roman", serif'
  drawCenteredSpacedText(ctx, 'CERTIFICATE', width / 2, 275, 12)

  // 5. Subtitle: "OF COMPLETION" (Text 3)

  // PPTX: Georgia 20pt, color #B8892B, letter-spacing 8pt

  ctx.fillStyle = '#B8892B'
  ctx.font = '36px "Georgia", "Times New Roman", serif'
  drawCenteredSpacedText(ctx, 'OF COMPLETION', width / 2, 395, 14)

  // 6. Lead-in: "This is to certify that" (Text 4)

  // PPTX: Georgia italic 16pt, color #595959

  ctx.fillStyle = '#595959'
  ctx.font = 'italic 30px "Georgia", "Times New Roman", serif'
  ctx.textAlign = 'center'
  ctx.fillText('This is to certify that', width / 2, 535)

  // 7. Dynamic Trainee Name: <<NAME>> (Text 5)

  // PPTX: Georgia bold 40pt, color #1F3864

  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 74px "Georgia", "Times New Roman", serif'
  ctx.textAlign = 'center'
  ctx.fillText(data.traineeName || 'Trainee Name', width / 2, 665)

  // 8. Description: "has successfully completed the training course" (Text 6)
```

```
  // PPTX: Georgia italic 16pt, color #595959

  ctx.fillStyle = '#595959'
  ctx.font = 'italic 30px "Georgia", "Times New Roman", serif'
  ctx.textAlign = 'center'
  ctx.fillText('has successfully completed the training course', width / 2, 795)

  // 9. Dynamic Course Title: <<COURSE_NAME>> (Text 7)

  // PPTX: Georgia bold 24pt, color #1F3864

  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 44px "Georgia", "Times New Roman", serif'
  ctx.textAlign = 'center'
  ctx.fillText(data.courseTitle || 'Course Title', width / 2, 895)

  // 10. Dynamic Score / Percentage: "with a final score of <<PERCENTAGE>>" (Text 8)

  const percentageStr = typeof data.percentage === 'number'
    ? `${data.percentage}%`
    : data.percentage
    ? String(data.percentage).includes('%') ? String(data.percentage) : `${data.percentage}%`
    : '100%'

  const textPart1 = 'with a final score of '
  ctx.font = 'italic 30px "Georgia", "Times New Roman", serif'
  const widthPart1 = ctx.measureText(textPart1).width
  ctx.font = 'bold 30px "Georgia", "Times New Roman", serif'
  const widthPart2 = ctx.measureText(percentageStr).width
  const totalScoreWidth = widthPart1 + widthPart2
  const scoreStartX = width / 2 - totalScoreWidth / 2

  ctx.textAlign = 'left'
  ctx.fillStyle = '#595959'
  ctx.font = 'italic 30px "Georgia", "Times New Roman", serif'
  ctx.fillText(textPart1, scoreStartX, 995)

  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 30px "Georgia", "Times New Roman", serif'
  ctx.fillText(percentageStr, scoreStartX + widthPart1, 995)

  // 11. Left Footer: Date Block (Shape 9, Text 10, Text 11)

  // PPTX Line: x=1280160, y=5760720, w=2926080, line width 12700 (1pt), color #595959

  const dateLineX1 = 1280160 * scale
  const dateLineX2 = (1280160 + 2926080) * scale
  const dateLineY = 5760720 * scale
  const dateCenterX = (dateLineX1 + dateLineX2) / 2

  // Date Underline

  ctx.strokeStyle = '#595959'
  ctx.lineWidth = 2.5
  ctx.beginPath()
  ctx.moveTo(dateLineX1, dateLineY)
  ctx.lineTo(dateLineX2, dateLineY)
  ctx.stroke()

  // Date Value: <<DATE>> (Text 10)

  const formattedDate = data.completedAt
    ? new Date(data.completedAt).toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' })
    : new Date().toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' })

  ctx.textAlign = 'center'
  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 28px "Georgia", "Times New Roman", serif'
  ctx.fillText(formattedDate, dateCenterX, dateLineY - 18)

  // Date Label: "Date of Issue" (Text 11)

  ctx.fillStyle = '#595959'
  ctx.font = '22px "Georgia", "Times New Roman", serif'
  ctx.fillText('Date of Issue', dateCenterX, dateLineY + 38)

  // 12. Right Footer: Instructor Block (Shape 12, Text 13, Text 14)

  // PPTX Line: x=7982712, y=5760720, w=2926080, line width 12700 (1pt), color #595959
```

```
  const instLineX1 = 7982712 * scale
  const instLineX2 = (7982712 + 2926080) * scale
  const instLineY = 5760720 * scale
  const instCenterX = (instLineX1 + instLineX2) / 2

  // Instructor Underline

  ctx.strokeStyle = '#595959'
  ctx.lineWidth = 2.5
  ctx.beginPath()
  ctx.moveTo(instLineX1, instLineY)
  ctx.lineTo(instLineX2, instLineY)
  ctx.stroke()

  // Trainer Name: <<TRAINER_NAME>> (Text 13)

  ctx.textAlign = 'center'
  ctx.fillStyle = '#1F3864'
  ctx.font = 'bold 28px "Georgia", "Times New Roman", serif'
  ctx.fillText(data.trainerName || 'Lead Instructor', instCenterX, instLineY - 18)

  // Instructor Label: "Lead Instructor" (Text 14)

  ctx.fillStyle = '#595959'
  ctx.font = '22px "Georgia", "Times New Roman", serif'
  ctx.fillText('Lead Instructor', instCenterX, instLineY + 38)

  // Convert canvas to High Quality PNG image

  const imgDataUrl = canvas.toDataURL('image/png', 1.0)

  // Create jsPDF in exact 16:9 standard slide dimensions (338.67 mm x 190.5 mm)

  const pdf = new jsPDF({
    orientation: 'landscape',
    unit: 'mm',
    format: [338.67, 190.5],
    compress: true,
  })

  // Render image to cover full PDF page precisely

  pdf.addImage(imgDataUrl, 'PNG', 0, 0, 338.67, 190.5, undefined, 'FAST')

  return pdf.output('blob')
}

/**

 * Generates a customized PDF certificate and saves it to Supabase storage

 */

export async function generateTraineeCertificate(
  _templateUrl: string | null | undefined,
  data: CertificateData
): Promise<{ blob: Blob; fileName: string; storageUrl?: string }> {
  const outputBlob = await generatePdfCertificate(data)

  const safeTraineeName = (data.traineeName || 'Trainee').replace(/[^a-zA-Z0-9_-]/g, '_')
  const safeCourseName = (data.courseTitle || 'Course').replace(/[^a-zA-Z0-9_-]/g, '_').substring(0, 30)
  const fileName = `${safeTraineeName}_${safeCourseName}_Certificate.pdf`

  let storageUrl: string | undefined
  try {
    const storagePath = `${data.courseId}/${data.traineeId}_${Date.now()}.pdf`
    const { data: uploadData, error: uploadErr } = await supabase.storage
      .from('certificates')
      .upload(storagePath, outputBlob, {
        contentType: 'application/pdf',
        upsert: true,
      })

    if (!uploadErr && uploadData) {
      const { data: publicUrlData } = supabase.storage
        .from('certificates')
        .getPublicUrl(storagePath)
      storageUrl = publicUrlData.publicUrl
```

```
      try {
        const percentageNum = typeof data.percentage === 'number'
          ? data.percentage
          : parseFloat(String(data.percentage).replace('%', '')) || 100

        await (supabase as any).from('certificates').upsert({
          course_id: data.courseId,
          trainee_id: data.traineeId,
          trainee_name: data.traineeName,
          course_title: data.courseTitle,
          percentage: percentageNum,
          file_url: storageUrl,
          file_name: fileName,
          issued_at: new Date().toISOString(),
        }, { onConflict: 'course_id,trainee_id' })
      } catch (dbErr) {
        console.warn('Could not record certificate in DB table:', dbErr)
      }
    }
  } catch (storageErr) {
    console.warn('Storage upload error (proceeding with direct PDF download):', storageErr)
  }

  return { blob: outputBlob, fileName, storageUrl }
}

function escapeXml(unsafe: string): string {
  return unsafe
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&apos;')
}

/**
 * Creates a clean, standalone default MoES Capacity Connect PPTX certificate template matching DBMS_Certificate_Template.pptx
 */

export async function createDefaultPptxTemplate(courseTitle = 'Database Management Systems (DBMS)'): Promise<ArrayBuffer> {
  const zip = new JSZip()

  // [Content_Types].xml

  zip.file('[Content_Types].xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
  <Default Extension="xml" ContentType="application/xml"/>
  <Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>
  <Override PartName="/ppt/presentation.xml" ContentType="application/vnd.openxmlformats-officedocument.presentationml.presentation.main+xml"/>
  <Override PartName="/ppt/slides/slide1.xml" ContentType="application/vnd.openxmlformats-officedocument.presentationml.slide+xml"/>
  <Override PartName="/ppt/slideLayouts/slideLayout1.xml" ContentType="application/vnd.openxmlformats-officedocument.presentationml.slideLayout+xml"/>
  <Override PartName="/ppt/slideMasters/slideMaster1.xml" ContentType="application/vnd.openxmlformats-officedocument.presentationml.slideMaster+xml"/>
  <Override PartName="/ppt/theme/theme1.xml" ContentType="application/vnd.openxmlformats-officedocument.theme+xml"/>
</Types>`)

  // _rels/.rels

  zip.file('_rels/.rels', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/officeDocument" Target="ppt/presentation.xml"/>
</Relationships>`)

  // ppt/_rels/presentation.xml.rels

  zip.file('ppt/_rels/presentation.xml.rels', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slideMaster" Target="slideMasters/slideMaster1.xml"/>
  <Relationship Id="rId2" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slide" Target="slides/slide1.xml"/>
</Relationships>`)
```

```
|
|   // ppt/presentation.xml

|   zip.file('ppt/presentation.xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
| <p:presentation xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main" xmlns:r="http://schemas.openxmlformats.org/off
iceDocument/2006/relationships" xmlns:p="http://schemas.openxmlformats.org/presentationml/2006/main" saveSubsetFonts="1" autoCompres
sPictures="0">
|   <p:sldMasterIdLst>
|     <p:sldMasterId id="2147483648" r:id="rId1"/>
|   </p:sldMasterIdLst>
|   <p:sldIdLst>
|     <p:sldId id="256" r:id="rId2"/>
|   </p:sldIdLst>
|   <p:sldSz cx="12192000" cy="6858000"/>
|   <p:notesSz cx="6858000" cy="12192000"/>
| </p:presentation>`)
|
|   // ppt/slides/_rels/slide1.xml.rels

|   zip.file('ppt/slides/_rels/slide1.xml.rels', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
| <Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
|   <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slideLayout" Target="../sl
ideLayouts/slideLayout1.xml"/>
| </Relationships>`)
|
|   // ppt/slides/slide1.xml matching DBMS_Certificate_Template.pptx

|   zip.file('ppt/slides/slide1.xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
| <p:sld xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main" xmlns:r="http://schemas.openxmlformats.org/officeDocume
nt/2006/relationships" xmlns:p="http://schemas.openxmlformats.org/presentationml/2006/main">
|   <p:cSld name="Slide 1">
|     <p:bg>
|       <p:bgPr>
|         <a:solidFill>
|           <a:srgbClr val="FFFFFF"/>
|         </a:solidFill>
|       </p:bgPr>
|     </p:bg>
|     <p:spTree>
|       <p:nvGrpSpPr>
|         <p:cNvPr id="1" name=""/>
|         <p:cNvGrpSpPr/>
|         <p:nvPr/>
|       </p:nvGrpSpPr>
|       <p:grpSpPr>
|         <a:xfrm>
|           <a:off x="0" y="0"/>
|           <a:ext cx="0" cy="0"/>
|           <a:chOff x="0" y="0"/>
|           <a:chExt cx="0" cy="0"/>
|         </a:xfrm>
|       </p:grpSpPr>
|
|       <!-- Shape 0: Outer Navy Border -->
|       <p:sp>
|         <p:nvSpPr>
|           <p:cNvPr id="2" name="Shape 0"/>
|           <p:cNvSpPr/>
|           <p:nvPr/>
|         </p:nvSpPr>
|         <p:spPr>
|           <a:xfrm>
|             <a:off x="365760" y="365760"/>
|             <a:ext cx="11457432" cy="6126480"/>
|           </a:xfrm>
|           <a:prstGeom prst="rect">
|             <a:avLst/>
|           </a:prstGeom>
|           <a:ln w="38100">
|             <a:solidFill>
|               <a:srgbClr val="1F3864"/>
|             </a:solidFill>
|             <a:prstDash val="solid"/>
|           </a:ln>
|         </p:spPr>
|       </p:sp>
|
|       <!-- Shape 1: Inner Gold Border -->
|       <p:sp>
```

```
        <p:nvSpPr>
          <p:cNvPr id="3" name="Shape 1"/>
          <p:cNvSpPr/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="548640" y="548640"/>
            <a:ext cx="11091672" cy="5760720"/>
          </a:xfrm>
          <a:prstGeom prst="rect">
            <a:avLst/>
          </a:prstGeom>
          <a:ln w="15875">
            <a:solidFill>
              <a:srgbClr val="B8892B"/>
            </a:solidFill>
            <a:prstDash val="solid"/>
          </a:ln>
        </p:spPr>
      </p:sp>

      <!-- Text 2: CERTIFICATE -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="4" name="Text 2"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="914400"/>
            <a:ext cx="10360152" cy="822960"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="4600" b="1" spc="600" kern="0">
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>CERTIFICATE</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 3: OF COMPLETION -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="5" name="Text 3"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="1691640"/>
            <a:ext cx="10360152" cy="457200"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="2000" spc="800" kern="0">
                <a:solidFill><a:srgbClr val="B8892B"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>OF COMPLETION</a:t>
```

```
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 4: This is to certify that -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="6" name="Text 4"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="2514600"/>
            <a:ext cx="10360152" cy="365760"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1600" i="1">
                <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>This is to certify that</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 5: <<NAME>> Placeholder -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="7" name="Text 5"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="2926080"/>
            <a:ext cx="10360152" cy="822960"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="4000" b="1">
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>&lt;&lt;NAME&gt;&gt;</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 6: has successfully completed the training course -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="8" name="Text 6"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="3840480"/>
            <a:ext cx="10360152" cy="365760"/>
          </a:xfrm>
```

```
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1600" i="1">
                <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>has successfully completed the training course</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 7: <<COURSE_NAME>> Placeholder -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="9" name="Text 7"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="1371600" y="4251960"/>
            <a:ext cx="9445752" cy="548640"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="2400" b="1">
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>${escapeXml(courseTitle || '<<COURSE_NAME>>')}</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 8: with a final score of <<PERCENTAGE>> -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="10" name="Text 8"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="914400" y="4846320"/>
            <a:ext cx="10360152" cy="365760"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1600" i="1">
                <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>with a final score of </a:t>
            </a:r>
            <a:r>
              <a:rPr lang="en-US" sz="1600" b="1">
```

```
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>&lt;&lt;PERCENTAGE&gt;&gt;</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Shape 9: Date Underline -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="11" name="Shape 9"/>
          <p:cNvSpPr/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="1280160" y="5760720"/>
            <a:ext cx="2926080" cy="0"/>
          </a:xfrm>
          <a:prstGeom prst="line"><a:avLst/></a:prstGeom>
          <a:noFill/>
          <a:ln w="12700">
            <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
            <a:prstDash val="solid"/>
          </a:ln>
        </p:spPr>
      </p:sp>

      <!-- Text 10: <<DATE>> -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="12" name="Text 10"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="1280160" y="5349240"/>
            <a:ext cx="2926080" cy="365760"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1600" b="1">
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>&lt;&lt;DATE&gt;&gt;</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 11: Date of Issue -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="13" name="Text 11"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="1280160" y="5806440"/>
            <a:ext cx="2926080" cy="274320"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
```

```
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1200">
                <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>Date of Issue</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Shape 12: Trainer Underline -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="14" name="Shape 12"/>
          <p:cNvSpPr/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="7982712" y="5760720"/>
            <a:ext cx="2926080" cy="0"/>
          </a:xfrm>
          <a:prstGeom prst="line"><a:avLst/></a:prstGeom>
          <a:noFill/>
          <a:ln w="12700">
            <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
            <a:prstDash val="solid"/>
          </a:ln>
        </p:spPr>
      </p:sp>

      <!-- Text 13: <<TRAINER_NAME>> -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="15" name="Text 13"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="7982712" y="5349240"/>
            <a:ext cx="2926080" cy="365760"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1600" b="1">
                <a:solidFill><a:srgbClr val="1F3864"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>&lt;&lt;TRAINER_NAME&gt;&gt;</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

      <!-- Text 14: Lead Instructor -->
      <p:sp>
        <p:nvSpPr>
          <p:cNvPr id="16" name="Text 14"/>
          <p:cNvSpPr txBox="1"/>
          <p:nvPr/>
        </p:nvSpPr>
        <p:spPr>
          <a:xfrm>
            <a:off x="7982712" y="5806440"/>
            <a:ext cx="2926080" cy="274320"/>
          </a:xfrm>
          <a:prstGeom prst="rect"><a:avLst/></a:prstGeom>
          <a:noFill/><a:ln/>
```

```
        </p:spPr>
        <p:txBody>
          <a:bodyPr wrap="square" lIns="0" tIns="0" rIns="0" bIns="0" rtlCol="0" anchor="ctr"/>
          <a:lstStyle/>
          <a:p>
            <a:pPr algn="ctr" indent="0" marL="0"><a:buNone/></a:pPr>
            <a:r>
              <a:rPr lang="en-US" sz="1200">
                <a:solidFill><a:srgbClr val="595959"/></a:solidFill>
                <a:latin typeface="Georgia" pitchFamily="34" charset="0"/>
              </a:rPr>
              <a:t>Lead Instructor</a:t>
            </a:r>
          </a:p>
        </p:txBody>
      </p:sp>

    </p:spTree>
  </p:cSld>
</p:sld>`)

  // ppt/slideLayouts/slideLayout1.xml

  zip.file('ppt/slideLayouts/slideLayout1.xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<p:sldLayout xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main" xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships" xmlns:p="http://schemas.openxmlformats.org/presentationml/2006/main" type="blank">
  <p:cSld>
    <p:spTree>
      <p:nvGrpSpPr>
        <p:cNvPr id="1" name=""/>
        <p:cNvGrpSpPr/>
        <p:nvPr/>
      </p:nvGrpSpPr>
      <p:grpSpPr/>
    </p:spTree>
  </p:cSld>
</p:sldLayout>`)

  // ppt/slideLayouts/_rels/slideLayout1.xml.rels

  zip.file('ppt/slideLayouts/_rels/slideLayout1.xml.rels', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slideMaster" Target="../slideMasters/slideMaster1.xml"/>
</Relationships>`)

  // ppt/slideMasters/slideMaster1.xml

  zip.file('ppt/slideMasters/slideMaster1.xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<p:sldMaster xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main" xmlns:r="http://schemas.openxmlformats.org/officeDocument/2006/relationships" xmlns:p="http://schemas.openxmlformats.org/presentationml/2006/main">
  <p:cSld>
    <p:spTree>
      <p:nvGrpSpPr>
        <p:cNvPr id="1" name=""/>
        <p:cNvGrpSpPr/>
        <p:nvPr/>
      </p:nvGrpSpPr>
      <p:grpSpPr/>
    </p:spTree>
  </p:cSld>
  <p:sldLayoutIdLst>
    <p:sldLayoutId id="2147483649" r:id="rId1"/>
  </p:sldLayoutIdLst>
</p:sldMaster>`)

  // ppt/slideMasters/_rels/slideMaster1.xml.rels

  zip.file('ppt/slideMasters/_rels/slideMaster1.xml.rels', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/slideLayout" Target="../slideLayouts/slideLayout1.xml"/>
  <Relationship Id="rId2" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/theme" Target="../theme/theme1.xml"/>
</Relationships>`)

  // ppt/theme/theme1.xml

  zip.file('ppt/theme/theme1.xml', `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

```
<a:theme xmlns:a="http://schemas.openxmlformats.org/drawingml/2006/main" name="Office Theme">
  <a:themeElements>
    <a:clrScheme name="Office">
      <a:dk1><a:sysClr val="windowText" lastClr="000000"/></a:dk1>
      <a:lt1><a:sysClr val="window" lastClr="FFFFFF"/></a:lt1>
      <a:dk2><a:srgbClr val="1F3864"/></a:dk2>
      <a:lt2><a:srgbClr val="EEECE1"/></a:lt2>
      <a:accent1><a:srgbClr val="B8892B"/></a:accent1>
      <a:accent2><a:srgbClr val="1F3864"/></a:accent2>
      <a:accent3><a:srgbClr val="595959"/></a:accent3>
      <a:accent4><a:srgbClr val="8B5CF6"/></a:accent4>
      <a:accent5><a:srgbClr val="F59E0B"/></a:accent5>
      <a:accent6><a:srgbClr val="EC4899"/></a:accent6>
      <a:hlink><a:srgbClr val="1F3864"/></a:hlink>
      <a:folHlink><a:srgbClr val="595959"/></a:folHlink>
    </a:clrScheme>
    <a:fontScheme name="Office">
      <a:majorFont><a:latin typeface="Georgia"/></a:majorFont>
      <a:minorFont><a:latin typeface="Georgia"/></a:minorFont>
    </a:fontScheme>
    <a:fmtScheme name="Office">
      <a:fillStyleLst><a:solidFill><a:schemeClr val="phClr"/></a:solidFill></a:fillStyleLst>
      <a:lnStyleLst><a:ln><a:solidFill><a:schemeClr val="phClr"/></a:solidFill></a:ln></a:lnStyleLst>
      <a:effectStyleLst><a:effectStyle><a:effectLst/></a:effectStyle></a:effectStyleLst>
      <a:bgFillStyleLst><a:solidFill><a:schemeClr val="phClr"/></a:solidFill></a:bgFillStyleLst>
    </a:fmtScheme>
  </a:themeElements>
</a:theme>`)

  return await zip.generateAsync({ type: 'arraybuffer' })
}
```

src/lib/courseGrading.ts TS 7.1 KB 210 Lines

```ts
export interface AssessmentItem {
  id: string
  course_id: string
  title: string
  assessment_type: 'daily' | 'mock' | 'final' | 'assessment' | string | null
  passing_score?: number | null
  status?: string | null
  scheduled_date?: string | null
  start_time?: string | null
  end_time?: string | null
  duration_minutes?: number | null
}

export interface AssessmentAttemptItem {
  id?: string
  assessment_id: string
  user_id: string
  score?: number | null
  grade_status?: string | null
  submitted_at?: string | null
  created_at?: string | null
}

export interface CourseGradeBreakdown {
  // Course Module Progress (25% max)

  moduleProgressPercent: number
  moduleScore: number // 0 to 25

  // Regular Assessments (25% max, averaged)

  regularAssessmentsTotal: number
  regularAssessmentsCompleted: number
  regularAssessmentAveragePercent: number // 0 to 100
  regularAssessmentScore: number // 0 to 25

  // Final Assessment (50% max)

  hasFinalAssessment: boolean
  finalAssessmentId: string | null
  finalAssessmentTitle: string | null
  finalAssessmentCompleted: boolean
  finalAssessmentScorePercent: number | null // 0 to 100
  finalAssessmentWeightedScore: number // 0 to 50
  isFinalUnlocked: boolean

  // Overall Totals

  totalScore: number // 0 to 100
  passingScore: number // e.g. 50 or 60
  isPassed: boolean
  isCompleted: boolean // 100% modules + final passed (if final exists)
}

export function isPracticeAssessment(type?: string | null): boolean {
  if (!type) return false
  const t = type.toLowerCase()
  return t === 'mock' || t === 'daily' || t === 'practice'
}

export function isRegularAssessment(type?: string | null): boolean {
  if (!type) return false
  const t = type.toLowerCase()
  return t === 'assessment' || t === 'quiz' || t === 'module_test'
}

export function isFinalAssessment(type?: string | null): boolean {
  if (!type) return false
  const t = type.toLowerCase()
  return t === 'final' || t === 'final_exam'
}

export function getAssessmentTypeLabel(type?: string | null): {
  label: string
  weightLabel: string
  isPractice: boolean
```

```
  badgeColor: string
} {
  if (isPracticeAssessment(type)) {
    const isMock = type?.toLowerCase() === 'mock'
    return {
      label: isMock ? 'Mock Test' : 'Daily Test',
      weightLabel: 'Practice Only (0% Grade Weight)',
      isPractice: true,
      badgeColor: 'bg-slate-100 text-slate-700 border-slate-200',
    }
  }

  if (isFinalAssessment(type)) {
    return {
      label: 'Final Assessment',
      weightLabel: '50% Final Grade Weight',
      isPractice: false,
      badgeColor: 'bg-purple-50 text-purple-700 border-purple-200',
    }
  }

  return {
    label: 'Assessment Test',
    weightLabel: '25% Grade Weight (Averaged)',
    isPractice: false,
    badgeColor: 'bg-amber-50 text-amber-700 border-amber-200',
  }
}

/**
 * Calculates the complete course grade breakdown according to the rules:
 * - Course completion / modules: 25% mark
 * - Regular assessments average: 25% mark (taking average of all completed regular assessments)
 * - Final assessment: 50% mark
 * - Final assessment unlocks ONLY when modules are 100% complete
 * - Mock & Daily tests are practice only (0% weight)
 */
export function calculateCourseGradeBreakdown({
  moduleProgressPercent = 0,
  courseAssessments = [],
  traineeAttempts = [],
  passingScore = 50,
}: {
  moduleProgressPercent?: number
  courseAssessments?: AssessmentItem[]
  traineeAttempts?: AssessmentAttemptItem[]
  passingScore?: number
}): CourseGradeBreakdown {
  const cleanModuleProgress = Math.min(100, Math.max(0, moduleProgressPercent))
  const isFinalUnlocked = cleanModuleProgress >= 100

  // 1. Course Modules Progress: 25% weight
  const moduleScore = Number(((cleanModuleProgress / 100) * 25).toFixed(1))

  // 2. Regular Assessments: 25% weight (average of assessments)
  const regularAssessments = courseAssessments.filter(a => isRegularAssessment(a.assessment_type))
  const regularAssessmentsTotal = regularAssessments.length

  let regularScoresSum = 0
  let regularAssessmentsCompleted = 0

  regularAssessments.forEach(assessment => {
    // Find best attempt for this assessment
    const attempts = traineeAttempts.filter(
      at => at.assessment_id === assessment.id && at.score !== null && at.score !== undefined
    )
    if (attempts.length > 0) {
      const bestScore = Math.max(...attempts.map(at => Number(at.score) || 0))
```

```
      regularScoresSum += Math.min(100, Math.max(0, bestScore))
      regularAssessmentsCompleted++
    }
  })

  let regularAssessmentAveragePercent = 0
  let regularAssessmentScore = 0

  if (regularAssessmentsTotal > 0) {
    if (regularAssessmentsCompleted > 0) {
      regularAssessmentAveragePercent = Math.round(regularScoresSum / regularAssessmentsCompleted)
      regularAssessmentScore = Number(((regularAssessmentAveragePercent / 100) * 25).toFixed(1))
    }
  } else {
    // If the course doesn't have separate regular assessments, award the 25% proportionally based on module completion
    regularAssessmentAveragePercent = cleanModuleProgress
    regularAssessmentScore = Number(((cleanModuleProgress / 100) * 25).toFixed(1))
  }

  // 3. Final Assessment: 50% weight
  const finalAssessments = courseAssessments.filter(a => isFinalAssessment(a.assessment_type))
  const hasFinalAssessment = finalAssessments.length > 0
  const finalAssessment = finalAssessments[0] || null

  let finalAssessmentCompleted = false
  let finalAssessmentScorePercent: number | null = null
  let finalAssessmentWeightedScore = 0

  if (finalAssessment) {
    const finalAttempts = traineeAttempts.filter(
      at => at.assessment_id === finalAssessment.id && at.score !== null && at.score !== undefined
    )
    if (finalAttempts.length > 0) {
      const bestFinalScore = Math.max(...finalAttempts.map(at => Number(at.score) || 0))
      finalAssessmentScorePercent = Math.min(100, Math.max(0, bestFinalScore))
      finalAssessmentCompleted = true
      finalAssessmentWeightedScore = Number(((finalAssessmentScorePercent / 100) * 50).toFixed(1))
    }
  }

  // Calculate Overall Total (out of 100)
  const totalScore = Math.round(moduleScore + regularAssessmentScore + finalAssessmentWeightedScore)
  const isPassed = totalScore >= passingScore

  // Completion requirement: 100% modules + (if final assessment exists, must be completed and passed)
  const isCompleted =
    cleanModuleProgress >= 100 &&
    (hasFinalAssessment ? finalAssessmentCompleted && isPassed : isPassed)

  return {
    moduleProgressPercent: cleanModuleProgress,
    moduleScore,
    regularAssessmentsTotal,
    regularAssessmentsCompleted,
    regularAssessmentAveragePercent,
    regularAssessmentScore,
    hasFinalAssessment,
    finalAssessmentId: finalAssessment?.id || null,
    finalAssessmentTitle: finalAssessment?.title || null,
    finalAssessmentCompleted,
    finalAssessmentScorePercent,
    finalAssessmentWeightedScore,
    isFinalUnlocked,
    totalScore,
    passingScore,
    isPassed,
    isCompleted,
  }
}
```

src/lib/query-client.ts TS 0.5 KB 19 Lines

```ts
import { QueryClient } from '@tanstack/react-query'

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5, // 5 minutes
      retry: 1, // Only retry once on failure
      refetchOnWindowFocus: false, // Don't refetch automatically when switching tabs
    },
    mutations: {
      onError: (error) => {
        // We handle toast notifications in the components usually,
        // but this catches any unhandled mutation errors.
        console.error('Mutation error:', error)
      },
    },
  },
})

```

src/lib/storage.ts TS 1.3 KB 55 Lines

```ts
import { supabase } from './supabase'

/**
 * Get a signed URL for a file in Supabase storage.
 * @param bucket - The storage bucket name (e.g., 'materials')
 * @param path - The file path in storage
 * @param expiresIn - URL expiry time in seconds (default: 1 hour)
 * @returns Signed URL string or null
 */
export async function getSignedUrl(
  bucket: string,
  path: string,
  expiresIn: number = 3600
): Promise<string | null> {
  if (!path) return null

  try {
    const { data, error } = await supabase.storage
      .from(bucket)
      .createSignedUrl(path, expiresIn)

    if (error) {
      console.error('Error getting signed URL:', error)
      return null
    }

    return data.signedUrl
  } catch (err) {
    console.error('Error getting signed URL:', err)
    return null
  }
}

/**
 * Get a public URL for a file in Supabase storage.
 * Note: Only works if the bucket has public access.
 * @param bucket - The storage bucket name
 * @param path - The file path in storage
 * @returns Public URL string or null
 */
export function getPublicUrl(bucket: string, path: string): string | null {
  if (!path) return null

  try {
    const { data } = supabase.storage
      .from(bucket)
      .getPublicUrl(path)

    return data.publicUrl
  } catch (err) {
    console.error('Error getting public URL:', err)
    return null
  }
}

```

src/lib/supabase.ts TS 0.6 KB 22 Lines

```ts
import { createClient } from '@supabase/supabase-js'
import { Database } from '@/integrations/supabase/types'

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabasePublishableKey = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;

if (!supabaseUrl || !supabasePublishableKey) {
  throw new Error("Missing VITE_SUPABASE_URL or VITE_SUPABASE_PUBLISHABLE_KEY");
}

export const supabase = createClient<Database>(
  supabaseUrl,
  supabasePublishableKey,
  {
    auth: {
      persistSession: true,
      autoRefreshToken: true,
      detectSessionInUrl: true,
    },
  }
);

```

src/lib/useStorageUrl.ts TS 0.9 KB 38 Lines

```ts
import { useState, useEffect } from 'react'
import { getSignedUrl } from './storage'

/**
 * Hook to get a signed URL for a storage file.
 * @param bucket - The storage bucket name
 * @param path - The file path in storage
 * @returns Object with url, loading, and error states
 */
export function useStorageUrl(bucket: string, path: string | null) {
  const [url, setUrl] = useState<string | null>(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    if (!path) {
      setUrl(null)
      setLoading(false)
      return
    }

    setLoading(true)
    setError(null)

    getSignedUrl(bucket, path)
      .then(signedUrl => {
        setUrl(signedUrl)
        setLoading(false)
      })
      .catch(err => {
        setError(err.message)
        setLoading(false)
      })
  }, [bucket, path])

  return { url, loading, error }
}

```

src/lib/utils.ts TS 0.2 KB 7 Lines

```ts
import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

```

src/utils/cropImage.ts TS 3.2 KB 100 Lines

```ts
export const createImage = (url: string): Promise<HTMLImageElement> =>
  new Promise((resolve, reject) => {
    const image = new Image()
    image.crossOrigin = 'anonymous'
    image.onload = () => resolve(image)
    image.onerror = () => {
      // Fallback: try loading without crossOrigin if CORS header is missing

      const fallbackImg = new Image()
      fallbackImg.onload = () => resolve(fallbackImg)
      fallbackImg.onerror = (err) => reject(err)
      fallbackImg.src = url
    }
    image.src = url
  })

export async function getCroppedImg(
  imageSrc: string,
  pixelCrop: { x: number; y: number; width: number; height: number },
  fileName: string = 'cropped.jpeg'
): Promise<File> {
  let sourceToLoad = imageSrc

  // If remote URL, attempt to convert to local blob first to prevent canvas tainting

  if (imageSrc.startsWith('http://') || imageSrc.startsWith('https://')) {
    try {
      const response = await fetch(imageSrc, { mode: 'cors' })
      if (response.ok) {
        const blob = await response.blob()
        sourceToLoad = URL.createObjectURL(blob)
      }
    } catch (fetchErr) {
      console.warn('Direct blob fetch failed, falling back to direct image loading:', fetchErr)
    }
  }

  const image = await createImage(sourceToLoad)
  const canvas = document.createElement('canvas')
  const ctx = canvas.getContext('2d')

  if (!ctx) {
    throw new Error('No 2d context')
  }

  const targetWidth = Math.max(1, Math.round(pixelCrop.width))
  const targetHeight = Math.max(1, Math.round(pixelCrop.height))

  canvas.width = targetWidth
  canvas.height = targetHeight

  ctx.drawImage(
    image,
    pixelCrop.x,
    pixelCrop.y,
    pixelCrop.width,
    pixelCrop.height,
    0,
    0,
    targetWidth,
    targetHeight
  )

  return new Promise((resolve, reject) => {
    try {
      canvas.toBlob((file) => {
        if (file) {
          resolve(new File([file], fileName, { type: 'image/jpeg' }))
        } else {
          // Fallback: toDataURL conversion

          try {
            const dataUrl = canvas.toDataURL('image/jpeg', 0.9)
            const byteString = atob(dataUrl.split(',')[1])
            const ab = new ArrayBuffer(byteString.length)
            const ia = new Uint8Array(ab)
```

```
            for (let i = 0; i < byteString.length; i++) {
              ia[i] = byteString.charCodeAt(i)
            }
            resolve(new File([ab], fileName, { type: 'image/jpeg' }))
          } catch (dataUrlErr) {
            reject(new Error('Canvas export failed: ' + dataUrlErr))
          }
        }
      }, 'image/jpeg', 0.9)
    } catch (toBlobErr) {
      // If toBlob threw SecurityError (tainted canvas)

      try {
        const dataUrl = canvas.toDataURL('image/jpeg', 0.9)
        const byteString = atob(dataUrl.split(',')[1])
        const ab = new ArrayBuffer(byteString.length)
        const ia = new Uint8Array(ab)
        for (let i = 0; i < byteString.length; i++) {
          ia[i] = byteString.charCodeAt(i)
        }
        resolve(new File([ab], fileName, { type: 'image/jpeg' }))
      } catch (finalErr) {
        reject(new Error('Failed to process image crop: ' + finalErr))
      }
    }
  })
}

```

src/hooks/useAuth.tsx TSX 6.1 KB 181 Lines

```tsx
import React, { createContext, useContext, useEffect, useState, useCallback } from 'react'
import { Session, User } from '@supabase/supabase-js'
import { supabase } from '@/lib/supabase'
import { Database } from '@/integrations/supabase/types'

export type Profile = {
  id: string;
  full_name: string;
  email: string | null;
  role: 'trainee' | 'trainer' | 'admin' | 'super_admin';
  approval_status: 'pending' | 'approved' | 'rejected' | 'suspended';
  avatar_path: string | null;
  created_at: string;
  updated_at: string;

  // Role-specific fields

  department?: string | null;
  designation?: string | null;
  proof_path?: string | null;
  bio?: string | null;
  years_of_experience?: number | null;
  expertise_areas?: string[] | null;
  learning_goals?: string | null;
  admin_level?: string | null;
  permissions_granted_at?: string | null;
};

type AuthContextType = {
  session: Session | null
  user: User | null
  profile: Profile | null
  loading: boolean
  signIn: (email: string, password: string) => Promise<void>
  signUp: (email: string, password: string, metadata: { full_name: string; department?: string; designation?: string; proof_pa
th?: string; signup_role: string; is_email_verified: boolean; mobile_number: string; study_details?: string }) => Promise<void>
  signOut: () => Promise<void>
  resetPassword: (email: string) => Promise<void>
  refreshProfile: () => Promise<void>
}

const AuthContext = createContext<AuthContextType | undefined>(undefined)

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [session, setSession] = useState<Session | null>(null)
  const [user, setUser] = useState<User | null>(null)
  const [profile, setProfile] = useState<Profile | null>(null)
  const [loading, setLoading] = useState(true)

  const fetchProfile = useCallback(async (userId: string) => {
    try {
      // Try to find the user in any of the three role tables concurrently

      const [adminRes, trainerRes, traineeRes] = await Promise.all([
        supabase.from('admins').select('*').eq('id', userId).maybeSingle(),
        supabase.from('trainers').select('*').eq('id', userId).maybeSingle(),
        supabase.from('trainees').select('*').eq('id', userId).maybeSingle()
      ]);

      const foundProfile = adminRes.data || trainerRes.data || traineeRes.data;

      if (!foundProfile) {
        console.error('Failed to load profile from any role table for user:', userId);
        setProfile(null);
        return;
      }

      setProfile(foundProfile as Profile);
    } catch (e) {
      console.error('Unexpected error loading profile:', e);
      setProfile(null);
    }
  }, [])

  const refreshProfile = useCallback(async () => {
    if (user) {
```

```
74 |       await fetchProfile(user.id)
75 |     }
76 |   }, [user, fetchProfile])
77 |
78 |   useEffect(() => {
79 |     const initializeAuth = async () => {
80 |       try {
81 |         const { data: { session }, error } = await supabase.auth.getSession()
82 |         if (error) throw error
83 |
84 |         setSession(session)
85 |         setUser(session?.user ?? null)
86 |
87 |         if (session?.user) {
88 |           await fetchProfile(session.user.id)
89 |         }
90 |       } catch (err) {
91 |         console.error('Auth initialization error:', err)
92 |       } finally {
93 |         setLoading(false)
94 |       }
95 |     }
96 |
97 |     initializeAuth()
98 |
99 |     const { data: { subscription } } = supabase.auth.onAuthStateChange(
100|       async (event, newSession) => {
101|         // If the session changes and we have a new user, we must set loading to true
102|         // so that the router doesn't try to redirect before the profile is fetched.
103|         if (newSession?.user && newSession.user.id !== user?.id) {
104|           setLoading(true)
105|         }
106|
107|         setSession(newSession)
108|         setUser(newSession?.user ?? null)
109|
110|         if (newSession?.user) {
111|           await fetchProfile(newSession.user.id)
112|         } else {
113|           setProfile(null)
114|         }
115|         setLoading(false)
116|
117|         // Automatically redirect to password setup when clicking an email link, but only if not already there
118|         if (event === 'PASSWORD_RECOVERY' && !window.location.pathname.includes('/setup-password')) {
119|           window.location.href = '/setup-password'
120|         }
121|       }
122|     )
123|
124|     return () => subscription.unsubscribe()
125|   }, [fetchProfile, user?.id])
126|
127|   const signIn = async (email: string, password: string) => {
128|     const { error } = await supabase.auth.signInWithPassword({ email, password })
129|     if (error) throw error
130|   }
131|
132|   const signUp = async (email: string, password: string, metadata: { full_name: string; department?: string; designation?: str
ing; proof_path?: string; signup_role: string; is_email_verified: boolean; mobile_number: string; study_details?: string }) => {
133|     const { error } = await supabase.auth.signUp({
134|       email,
135|       password,
136|       options: {
137|         data: {
138|           full_name: metadata.full_name,
139|           department: metadata.department,
140|           designation: metadata.designation,
141|           proof_path: metadata.proof_path,
142|           signup_role: metadata.signup_role,
143|           is_email_verified: metadata.is_email_verified,
144|           mobile_number: metadata.mobile_number,
145|           study_details: metadata.study_details,
146|         }
147|       }
148|     })
```

```
    if (error) throw error
  }

  const signOut = async () => {
    const { error } = await supabase.auth.signOut()
    if (error) throw error
    setSession(null)
    setUser(null)
    setProfile(null)
  }

  const resetPassword = async (email: string) => {
    const { error } = await supabase.auth.resetPasswordForEmail(email, {
      redirectTo: `${window.location.origin}/setup-password`,
    })
    if (error) throw error
  }

  return (
    <AuthContext.Provider value={{ session, user, profile, loading, signIn, signUp, signOut, resetPassword, refreshProfile }}>
      {children}
    </AuthContext.Provider>
  )
}

export const useAuth = () => {
  const context = useContext(AuthContext)
  if (context === undefined) {
    throw new Error('useAuth must be used within an AuthProvider')
  }
  return context
}

```

src/hooks/useConfirm.tsx TSX 0.9 KB 35 Lines

```tsx
import React, { useState } from 'react'
import { ConfirmModal } from '@/components/ui/ConfirmModal'

export function useConfirm() {
  const [promise, setPromise] = useState<{ resolve: (value: boolean) => void } | null>(null)
  const [config, setConfig] = useState({ title: '', message: '' })

  const confirm = (message: string, title: string = 'Confirm Action') => new Promise<boolean>((resolve) => {
    setConfig({ title, message })
    setPromise({ resolve })
  })

  const handleConfirm = () => {
    promise?.resolve(true)
    setPromise(null)
  }

  const handleCancel = () => {
    promise?.resolve(false)
    setPromise(null)
  }

  const ConfirmationDialog = () => (
    <ConfirmModal
      isOpen={promise !== null}
      title={config.title}
      message={config.message}
      onConfirm={handleConfirm}
      onCancel={handleCancel}
    />
  )

  return [ConfirmationDialog, confirm] as const
}

```

src/hooks/useHomePageSettings.ts TS 19.2 KB 563 Lines

```ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { supabase } from '@/lib/supabase';
import { toast } from 'sonner';

export interface FeaturedProgramItem {
  id: string;
  title: string;
  description: string;
  department: string;
  badge: string;
  duration_hours: number;
  trainer_name: string;
  link: string;
  thumbnail_url?: string | null;
}

export interface AnnouncementItem {
  id: string;
  tag: string;
  tag_color: string;
  title: string;
  link: string;
}

export interface UpcomingTrackItem {
  id: string;
  title: string;
  status: string;
  status_color: string;
  level: string;
  date: string;
  duration: string;
  format: string;
  desc: string;
  key_skills: string;
  btn_text: string;
  link: string;
}

export interface AchieverItem {
  id?: string;
  name: string;
  initials: string;
  role: string;
  institute: string;
  achievementBadge: string;
  score: string;
  avatar_url?: string | null;
}

export interface AchievementStoryItem {
  id: string;
  bannerTitle: string;
  bannerSubtitle: string;
  highlightStatTitle: string;
  highlightStatNumber: string;
  highlightStatDescription: string;
  image_url?: string | null;
  achievers: AchieverItem[];
}

export interface HomePageSettings {
  id: string;
  // Section 1: Featured Programs
  featured_programs_enabled: boolean;
  featured_programs_tag: string;
  featured_programs_title: string;
  featured_programs_subtitle: string;
  featured_programs_btn_text: string;
  featured_programs_btn_link: string;
  featured_programs_items: FeaturedProgramItem[];

  // Section 2: Announcements Marquee
  announcements_bar_enabled: boolean;
```

```
  announcements_bar_label: string;
  announcements_bar_speed: number;
  announcements_items: AnnouncementItem[];

  // Section 3: Specialized Earth Sciences Tracks

  upcoming_tracks_enabled: boolean;
  upcoming_tracks_tag: string;
  upcoming_tracks_title: string;
  upcoming_tracks_subtitle: string;
  upcoming_tracks_btn_text: string;
  upcoming_tracks_items: UpcomingTrackItem[];

  // Section 4: Achievements / Proven Results

  achievements_enabled: boolean;
  achievements_tag: string;
  achievements_title: string;
  achievements_subtitle: string;
  achievements_moes_badge: string;
  achievements_moes_desc: string;
  achievements_stories: AchievementStoryItem[];

  updated_at?: string;
}

export const defaultFeaturedPrograms: FeaturedProgramItem[] = [
  {
    id: 'prog-1',
    title: 'Ocean Observation and Marine Data Analysis',
    description: 'This course introduces learners to ocean observation systems, marine data collection, oceanographic parameters, satellite observations, and basic marine data analysis.',
    department: 'INCOIS, MoES',
    badge: 'Top Rated',
    duration_hours: 80,
    trainer_name: 'Dr. V. Ramakrishnan',
    link: '/courses',
    thumbnail_url: null,
  },
  {
    id: 'prog-2',
    title: 'Cyclone Response and Early Warning Systems',
    description: 'Learn the fundamentals of weather forecasting, atmospheric processes, climate variability, climate change, and the use of meteorological data for disaster preparedness.',
    department: 'IMD, MoES',
    badge: 'Featured',
    duration_hours: 100,
    trainer_name: 'Dr. A. Sengupta',
    link: '/courses',
    thumbnail_url: null,
  },
  {
    id: 'prog-3',
    title: 'Doppler Weather Radar & Advanced Nowcasting Systems',
    description: 'Master operational radar telemetry, severe storm identification, mesocyclone tracking, and rapid nowcasting protocols for extreme weather events.',
    department: 'IMD, MoES',
    badge: 'Popular',
    duration_hours: 72,
    trainer_name: 'Dr. N. Mukherjee',
    link: '/courses',
    thumbnail_url: null,
  },
];

export const defaultAnnouncementItems: AnnouncementItem[] = [
  {
    id: 'ann-1',
    tag: 'MoES Live',
    tag_color: 'cyan',
    title: 'IMD Doppler Radar Network Calibration Workshop Open for Trainee Registration',
    link: '/courses',
  },
  {
    id: 'ann-2',
    tag: 'INCOIS Notice',
    tag_color: 'amber',
    title: 'Ocean State Forecast & In-Situ Buoy Data Analytics Winter Cohort Starts Oct 15',
    link: '/courses',
```

```
  },
  {
    id: 'ann-3',
    tag: 'Certification',
    tag_color: 'emerald',
    title: 'New Government-Accredited Micro-Credential Standards Published for Field Scientists',
    link: '/courses',
  },
  {
    id: 'ann-4',
    tag: 'Operational Alert',
    tag_color: 'sky',
    title: 'Numerical Weather Prediction (NWP) High-Performance Computing Portal Now Active',
    link: '/courses',
  },
  {
    id: 'ann-5',
    tag: 'Admissions',
    tag_color: 'purple',
    title: 'Dual-Pol Radar Nowcasting Batch Enrollment Closing Soon — Reserve Your Seat',
    link: '/register',
  },
];

export const defaultUpcomingTracks: UpcomingTrackItem[] = [
  {
    id: 'track-1',
    title: 'Dual-Pol Doppler Radar & Severe Storm Nowcasting',
    status: 'Pre-Registration Open',
    status_color: 'emerald',
    level: 'Advanced Specialist',
    date: 'Starts Oct 15, 2026',
    duration: '4 Weeks',
    format: 'Live Radar Labs',
    desc: "Go deep on polarimetric radar and real-time storm tracking, with live radar feeds you'll actually use in the fiel
d.",
    key_skills: 'Polarimetric Refl., Hydrometeor Class., Mesocyclone Detection',
    btn_text: 'Pre-Register Cohort',
    link: '/register',
  },
  {
    id: 'track-2',
    title: 'Coastal Early Warning & Ocean Telemetry Protocol',
    status: 'Fast Filling',
    status_color: 'amber',
    level: 'Executive Protocol',
    date: 'Starts Dec 01, 2026',
    duration: '3 Weeks',
    format: 'Field Buoy & In-Situ',
    desc: 'Master storm surge alerting and ocean buoy data workflows alongside joint INCOIS and IMD teams.',
    key_skills: 'Tsunami Sensor Array, Storm Surge Modeling, In-Situ Buoy QC',
    btn_text: 'Pre-Register Cohort',
    link: '/register',
  },
  {
    id: 'track-3',
    title: 'AI & Deep Learning in Numerical Weather Prediction',
    status: 'Limited Cohort',
    status_color: 'sky',
    level: 'Specialized Track',
    date: 'Starts Nov 02, 2026',
    duration: '6 Weeks',
    format: 'HPC Supercomputing',
    desc: 'Hands-on machine learning for weather models and ensemble forecasts, built for real computational pipelines.',
    key_skills: 'Graph Neural Nets, Ensemble Prediction, CUDA Acceleration',
    btn_text: 'Pre-Register Cohort',
    link: '/register',
  },
];

export const defaultAchievementStories: AchievementStoryItem[] = [
  {
    id: 'imd-2025',
    bannerTitle: 'Capacity Connect Trainees Shine Bright in',
    bannerSubtitle: 'NATIONAL WEATHER FORECASTER BENCHMARK 2025',
    highlightStatTitle: 'OVER',
    highlightStatNumber: '94%',
    highlightStatDescription: 'Trainees Secured Top Tier-1 Operational Deployment in IMD & RSMC Desks',
    image_url: null,
```

```
    achievers: [
      {
        id: 'ach-1',
        name: 'Dr. Aarav Sharma',
        initials: 'AS',
        role: 'Radar Meteorologist',
        institute: 'IMD New Delhi',
        achievementBadge: 'AIR 1 (99.8%)',
        score: 'Top Forecaster',
        avatar_url: null,
      },
      {
        id: 'ach-2',
        name: 'Priya Venkatesh',
        initials: 'PV',
        role: 'Severe Weather Analyst',
        institute: 'RMC Chennai',
        achievementBadge: 'AIR 3 (98.9%)',
        score: 'Nowcast Specialist',
        avatar_url: null,
      },
      {
        id: 'ach-3',
        name: 'Rahul Sengupta',
        initials: 'RS',
        role: 'Cyclone Warning Desk',
        institute: 'IMD Kolkata',
        achievementBadge: 'AIR 7 (97.6%)',
        score: 'Early Warning Star',
        avatar_url: null,
      },
    ],
  },
  {
    id: 'ocean-2025',
    bannerTitle: 'Ocean Science Cohorts Excel in',
    bannerSubtitle: 'COASTAL & TSUNAMI EARLY WARNING CERTIFICATION',
    highlightStatTitle: 'EVERY',
    highlightStatNumber: '9 IN 10',
    highlightStatDescription: 'Fellows Successfully Deployed Real-Time Coastal Telemetry Systems',
    image_url: null,
    achievers: [
      {
        id: 'ach-4',
        name: 'Ananya Deshmukh',
        initials: 'AD',
        role: 'Ocean Dynamics Fellow',
        institute: 'INCOIS Hyderabad',
        achievementBadge: 'Top Rank (99.4%)',
        score: 'Tsunami Warning Lead',
        avatar_url: null,
      },
      {
        id: 'ach-5',
        name: 'Karthik Pillai',
        initials: 'KP',
        role: 'Marine Data Specialist',
        institute: 'NIOT Chennai',
        achievementBadge: 'Excellence (98.2%)',
        score: 'Deep Ocean Mission',
        avatar_url: null,
      },
      {
        id: 'ach-6',
        name: 'Sneha Roy',
        initials: 'SR',
        role: 'Wave Modeling Officer',
        institute: 'INCOIS Coastal Hub',
        achievementBadge: 'Distinction (97.8%)',
        score: 'Wave Climate Expert',
        avatar_url: null,
      },
    ],
  },
  {
    id: 'modelling-2025',
    bannerTitle: 'Atmospheric Modellers Lead in',
    bannerSubtitle: 'HIGH-PERFORMANCE NUMERICAL WEATHER PREDICTION',
    highlightStatTitle: 'OVER',
```

```
    highlightStatNumber: '88%',
    highlightStatDescription: 'Achieved Advanced Parallel Computing & HPC Model Optimization Benchmarks',
    image_url: null,
    achievers: [
      {
        id: 'ach-7',
        name: 'Vikramaditya Iyer',
        initials: 'VI',
        role: 'NWP HPC Engineer',
        institute: 'NCMRWF Noida',
        achievementBadge: 'AIR 2 (99.5%)',
        score: 'GPU HPC Specialist',
        avatar_url: null,
      },
      {
        id: 'ach-8',
        name: 'Dr. Meera Nambiar',
        initials: 'MN',
        role: 'Data Assimilation Specialist',
        institute: 'IITM Pune',
        achievementBadge: 'AIR 5 (98.6%)',
        score: 'Monsoon Mission',
        avatar_url: null,
      },
      {
        id: 'ach-9',
        name: 'Sanjay Rawat',
        initials: 'SR',
        role: 'Climate AI Researcher',
        institute: 'MoES Earth System Lab',
        achievementBadge: 'AIR 8 (97.9%)',
        score: 'Neural Weather AI',
        avatar_url: null,
      },
    ],
  },
  {
    id: 'climate-2025',
    bannerTitle: 'Climate Scientists Achieve Milestones in',
    bannerSubtitle: 'POLAR, CRYOSPHERE & MOUNTAIN METEOROLOGY',
    highlightStatTitle: 'OVER',
    highlightStatNumber: '96%',
    highlightStatDescription: 'Expedition Readiness & Polar Field Data Collection Certification Passed',
    image_url: null,
    achievers: [
      {
        id: 'ach-10',
        name: 'Tenzin Norbu',
        initials: 'TN',
        role: 'Glaciology Observer',
        institute: 'NCPOR Goa / Arctic Track',
        achievementBadge: 'Top Star (99.1%)',
        score: 'Himansh Station Lead',
        avatar_url: null,
      },
      {
        id: 'ach-11',
        name: 'Pooja Bhatt',
        initials: 'PB',
        role: 'Antarctic Weather Specialist',
        institute: 'Maitri Research Base',
        achievementBadge: 'Excellence (98.4%)',
        score: 'Polar Met Lead',
        avatar_url: null,
      },
      {
        id: 'ach-12',
        name: 'Deepak Joshi',
        initials: 'DJ',
        role: 'Western Ghats Field Scientist',
        institute: 'NCESS Thiruvananthapuram',
        achievementBadge: 'Distinction (97.5%)',
        score: 'Hydro-Met Expert',
        avatar_url: null,
      },
    ],
  },
];

```

```
export const defaultHomePageSettings: HomePageSettings = {
  id: 'default_settings',
  // Section 1

  featured_programs_enabled: true,
  featured_programs_tag: 'Top Certified Tracks',
  featured_programs_title: 'Featured Learning Programs',
  featured_programs_subtitle: 'Explore government-certified, high-impact earth science and meteorological training tracks led by premier MoES scientific institutions.',
  featured_programs_btn_text: 'Explore All Courses',
  featured_programs_btn_link: '/courses',
  featured_programs_items: defaultFeaturedPrograms,

  // Section 2

  announcements_bar_enabled: true,
  announcements_bar_label: 'Updates',
  announcements_bar_speed: 65,
  announcements_items: defaultAnnouncementItems,

  // Section 3

  upcoming_tracks_enabled: true,
  upcoming_tracks_tag: 'Upcoming Announcements',
  upcoming_tracks_title: 'Specialized Earth Sciences Tracks.',
  upcoming_tracks_subtitle: 'Pre-register for next-generation cohorts and masterclasses.',
  upcoming_tracks_btn_text: 'View Complete Catalog',
  upcoming_tracks_items: defaultUpcomingTracks,

  // Section 4

  achievements_enabled: true,
  achievements_tag: 'Proven National Impact',
  achievements_title: 'Inspired trainees. Inspired results',
  achievements_subtitle: 'National Benchmark Results & Field Recognitions',
  achievements_moes_badge: '100% Verified Operational Skill Credentials',
  achievements_moes_desc: 'Direct alignment with WMO, IMD & INCOIS forecast protocols.',
  achievements_stories: defaultAchievementStories,
};

const LOCAL_STORAGE_KEY = 'capacity_connect_home_page_settings';

export function getLocalSettings(): HomePageSettings {
  try {
    const saved = localStorage.getItem(LOCAL_STORAGE_KEY);
    if (saved) {
      const parsed = JSON.parse(saved);
      return {
        ...defaultHomePageSettings,
        ...parsed,
        featured_programs_items: parsed.featured_programs_items?.length ? parsed.featured_programs_items : defaultFeaturedPrograms,
        announcements_items: parsed.announcements_items?.length ? parsed.announcements_items : defaultAnnouncementItems,
        upcoming_tracks_items: parsed.upcoming_tracks_items?.length ? parsed.upcoming_tracks_items : defaultUpcomingTracks,
        achievements_stories: parsed.achievements_stories?.length ? parsed.achievements_stories : defaultAchievementStories,
      };
    }
  } catch (e) {
    // Ignore storage errors

  }
  return defaultHomePageSettings;
}

export function useHomePageSettings() {
  const queryClient = useQueryClient();

  const { data: settings = defaultHomePageSettings, isLoading } = useQuery<HomePageSettings>({
    queryKey: ['home_page_settings'],
    queryFn: async () => {
      try {
        const { data, error } = await (supabase as any)
          .from('home_page_settings')
          .select('*')
          .limit(1)
          .maybeSingle();

        if (error) {
          return getLocalSettings();
```

```
        }

        if (data && typeof data === 'object') {
          const raw = data as Record<string, any>;
          const merged: HomePageSettings = {
            ...defaultHomePageSettings,
            ...raw,
            featured_programs_items: raw.featured_programs_items?.length ? raw.featured_programs_items : defaultFeaturedPrograms,
            announcements_items: raw.announcements_items?.length ? raw.announcements_items : defaultAnnouncementItems,
            upcoming_tracks_items: raw.upcoming_tracks_items?.length ? raw.upcoming_tracks_items : defaultUpcomingTracks,
            achievements_stories: raw.achievements_stories?.length ? raw.achievements_stories : defaultAchievementStories,
          };
          localStorage.setItem(LOCAL_STORAGE_KEY, JSON.stringify(merged));
          return merged;
        }
      } catch (err) {
        // Fallback gracefully
      }
      return getLocalSettings();
    },
    staleTime: 1000 * 60 * 5,
  });

  const updateMutation = useMutation({
    mutationFn: async (newSettings: Partial<HomePageSettings>) => {
      const updated: HomePageSettings = {
        ...settings,
        ...newSettings,
        id: 'default_settings',
        updated_at: new Date().toISOString(),
      };

      // Always save to local cache for instant zero-latency UI update
      localStorage.setItem(LOCAL_STORAGE_KEY, JSON.stringify(updated));

      const payload = {
        id: 'default_settings',
        featured_programs_enabled: updated.featured_programs_enabled ?? true,
        featured_programs_tag: updated.featured_programs_tag || 'Top Certified Tracks',
        featured_programs_title: updated.featured_programs_title || 'Featured Learning Programs',
        featured_programs_subtitle: updated.featured_programs_subtitle || '',
        featured_programs_btn_text: updated.featured_programs_btn_text || 'Explore All Courses',
        featured_programs_btn_link: updated.featured_programs_btn_link || '/courses',
        featured_programs_items: updated.featured_programs_items || [],

        announcements_bar_enabled: updated.announcements_bar_enabled ?? true,
        announcements_bar_label: updated.announcements_bar_label || 'Updates',
        announcements_bar_speed: updated.announcements_bar_speed || 65,
        announcements_items: updated.announcements_items || [],

        upcoming_tracks_enabled: updated.upcoming_tracks_enabled ?? true,
        upcoming_tracks_tag: updated.upcoming_tracks_tag || 'Upcoming Announcements',
        upcoming_tracks_title: updated.upcoming_tracks_title || 'Specialized Earth Sciences Tracks.',
        upcoming_tracks_subtitle: updated.upcoming_tracks_subtitle || '',
        upcoming_tracks_btn_text: updated.upcoming_tracks_btn_text || 'View Complete Catalog',
        upcoming_tracks_items: updated.upcoming_tracks_items || [],

        achievements_enabled: updated.achievements_enabled ?? true,
        achievements_tag: updated.achievements_tag || 'Proven National Impact',
        achievements_title: updated.achievements_title || 'Inspired trainees. Inspired results',
        achievements_subtitle: updated.achievements_subtitle || 'National Benchmark Results & Field Recognitions',
        achievements_moes_badge: updated.achievements_moes_badge || '100% Verified Operational Skill Credentials',
        achievements_moes_desc: updated.achievements_moes_desc || 'Direct alignment with WMO, IMD & INCOIS forecast protocols.',
        achievements_stories: updated.achievements_stories || [],

        updated_at: new Date().toISOString(),
      };

      // Attempt DB upsert
      try {
        const { error } = await (supabase as any)
          .from('home_page_settings')
          .upsert(payload, { onConflict: 'id' });

```

```
531 |         if (error) {
532 |           console.error('Database upsert error on home_page_settings:', error);
533 |           return { ...updated, _dbError: error.message };
534 |         }
535 |       } catch (err: any) {
536 |         console.error('Database network error:', err);
537 |         return { ...updated, _dbError: err?.message || 'Database connection error' };
538 |       }
539 |
540 |       return updated;
541 |     },
542 |     onSuccess: (result: any) => {
543 |       queryClient.setQueryData(['home_page_settings'], result);
544 |       queryClient.invalidateQueries({ queryKey: ['home_page_settings'] });
545 |       if (result._dbError) {
546 |         toast.warning(`Saved locally. Supabase DB returned: ${result._dbError}. Please run the SQL migration in Supabase to sy
nc DB.`);
547 |       } else {
548 |         toast.success('Home page sections successfully published to Database and Live Site!');
549 |       }
550 |     },
551 |     onError: (err: any) => {
552 |       toast.error('Failed to update settings: ' + (err.message || 'Unknown error'));
553 |     },
554 |   });
555 |
556 |   return {
557 |     settings,
558 |     isLoading,
559 |     updateSettings: updateMutation.mutate,
560 |     isUpdating: updateMutation.isPending,
561 |   };
562 | }
563 |
```

src/hooks/useNotifications.ts TS 11.7 KB 378 Lines

```ts
import { useState, useEffect, useCallback } from 'react'
import { toast } from 'sonner'
import { supabase } from '@/lib/supabase'
import { useAuth } from './useAuth'
import { Database } from '@/integrations/supabase/types'

type Notification = Database['public']['Tables']['notifications']['Row']

const getDismissedIds = (userId: string): string[] => {
  try {
    return JSON.parse(localStorage.getItem(`cc_dismissed_notifs_${userId}`) || '[]')
  } catch {
    return []
  }
}

const saveDismissedIds = (userId: string, ids: string[]) => {
  try {
    localStorage.setItem(`cc_dismissed_notifs_${userId}`, JSON.stringify(ids))
  } catch (e) {
    console.error('Failed to save dismissed notification IDs:', e)
  }
}

const getReadIds = (userId: string): string[] => {
  try {
    return JSON.parse(localStorage.getItem(`cc_read_notifs_${userId}`) || '[]')
  } catch {
    return []
  }
}

const saveReadIds = (userId: string, ids: string[]) => {
  try {
    localStorage.setItem(`cc_read_notifs_${userId}`, JSON.stringify(ids))
  } catch (e) {
    console.error('Failed to save read notification IDs:', e)
  }
}

const getClearedTimestamp = (userId: string): number => {
  try {
    return Number(localStorage.getItem(`cc_cleared_notifs_ts_${userId}`) || 0)
  } catch {
    return 0
  }
}

const saveClearedTimestamp = (userId: string, ts: number) => {
  try {
    localStorage.setItem(`cc_cleared_notifs_ts_${userId}`, String(ts))
  } catch (e) {
    console.error('Failed to save cleared notification timestamp:', e)
  }
}

export function useNotifications() {
  const { profile } = useAuth()
  const [notifications, setNotifications] = useState<Notification[]>([])
  const [loading, setLoading] = useState(true)

  const processAndFilterNotifications = useCallback((rawList: Notification[], userId: string): Notification[] => {
    const dismissedIds = getDismissedIds(userId)
    const readIds = getReadIds(userId)
    const clearedTs = getClearedTimestamp(userId)

    return rawList
      .filter(n => {
        // Exclude dismissed items
        if (dismissedIds.includes(n.id)) return false
        // Exclude items created before global clear timestamp
        if (n.created_at && new Date(n.created_at).getTime() <= clearedTs) return false
        return true
      })
```

```
      .map(n => {
        // Enforce local read status if marked read previously
        if (readIds.includes(n.id) && !n.read_at) {
          return { ...n, read_at: new Date().toISOString() }
        }
        return n
      })
  }, [])

  useEffect(() => {
    if (!profile?.id) {
      setNotifications([])
      setLoading(false)
      return
    }

    const userId = profile.id
    let isMounted = true

    const fetchNotifications = async () => {
      try {
        const { data, error } = await supabase
          .from('notifications')
          .select('*')
          .eq('user_id', userId)
          .order('created_at', { ascending: false })
          .limit(30)

        if (!error && data && isMounted) {
          const filtered = processAndFilterNotifications(data, userId)
          setNotifications(filtered)
        }
      } catch (err) {
        console.error('Error fetching notifications:', err)
      } finally {
        if (isMounted) setLoading(false)
      }
    }

    fetchNotifications()

    // Real-time subscription with unique channel ID per hook instance
    const channelName = `public:notifications:${userId}:${Math.random().toString(36).slice(2, 9)}`
    const channel = supabase
      .channel(channelName)
      .on(
        'postgres_changes',
        {
          event: 'INSERT',
          schema: 'public',
          table: 'notifications',
          filter: `user_id=eq.${userId}`
        },
        (payload) => {
          const newNotif = payload.new as Notification
          const dismissedIds = getDismissedIds(userId)
          const clearedTs = getClearedTimestamp(userId)

          if (
            !dismissedIds.includes(newNotif.id) &&
            (!newNotif.created_at || new Date(newNotif.created_at).getTime() > clearedTs)
          ) {
            setNotifications((prev) => [newNotif, ...prev.filter(n => n.id !== newNotif.id)].slice(0, 30))
            toast.info(newNotif.title, { description: newNotif.message })
          }
        }
      )
      .on(
        'postgres_changes',
        {
          event: 'UPDATE',
          schema: 'public',
          table: 'notifications',
          filter: `user_id=eq.${userId}`
        },
        (payload) => {
          const updatedNotif = payload.new as Notification
```

```
          setNotifications((prev) => prev.map(n => n.id === updatedNotif.id ? updatedNotif : n))
        }
      )
      .subscribe()

    return () => {
      isMounted = false
      supabase.removeChannel(channel)
    }
  }, [profile?.id, processAndFilterNotifications])

  const markAsRead = async (id: string) => {
    if (!profile?.id) return
    const userId = profile.id
    const nowIso = new Date().toISOString()

    // Persist to local read set immediately
    const readIds = getReadIds(userId)
    if (!readIds.includes(id)) {
      saveReadIds(userId, [...readIds, id])
    }

    // Update state immediately
    setNotifications(prev => prev.map(n => n.id === id ? { ...n, read_at: nowIso } : n))

    // Update database asynchronously
    try {
      await supabase
        .from('notifications')
        .update({ read_at: nowIso })
        .eq('id', id)
        .eq('user_id', userId)
    } catch (e) {
      console.warn('DB update failed, local state retained:', e)
    }
  }

  const markAllAsRead = async () => {
    if (!profile?.id) return
    const userId = profile.id
    const nowIso = new Date().toISOString()

    // Persist all current notification IDs to local read set
    const currentIds = notifications.map(n => n.id)
    const readIds = getReadIds(userId)
    const mergedReadIds = Array.from(new Set([...readIds, ...currentIds]))
    saveReadIds(userId, mergedReadIds)

    // Update state immediately
    setNotifications(prev => prev.map(n => ({ ...n, read_at: n.read_at || nowIso })))
    toast.success('All notifications marked as read')

    // Update database asynchronously
    try {
      await supabase
        .from('notifications')
        .update({ read_at: nowIso })
        .eq('user_id', userId)
        .is('read_at', null)
    } catch (e) {
      console.warn('DB update failed, local state retained:', e)
    }
  }

  const deleteNotification = async (id: string) => {
    if (!profile?.id) return
    const userId = profile.id

    // Persist to local dismissed set immediately
    const dismissedIds = getDismissedIds(userId)
    if (!dismissedIds.includes(id)) {
      saveDismissedIds(userId, [...dismissedIds, id])
```

```
    }

    // Update state immediately

    setNotifications(prev => prev.filter(n => n.id !== id))
    toast.success('Notification dismissed')

    // Delete from database asynchronously

    try {
      await supabase
        .from('notifications')
        .delete()
        .eq('id', id)
        .eq('user_id', userId)
    } catch (e) {
      console.warn('DB delete failed, local state retained:', e)
    }
  }

  const clearAllNotifications = async () => {
    if (!profile?.id) return
    const userId = profile.id
    const nowTs = Date.now()

    // Record clear timestamp and all current IDs to local storage

    saveClearedTimestamp(userId, nowTs)
    const currentIds = notifications.map(n => n.id)
    const dismissedIds = getDismissedIds(userId)
    saveDismissedIds(userId, Array.from(new Set([...dismissedIds, ...currentIds])))

    // Update state immediately

    setNotifications([])
    toast.success('All notifications cleared')

    // Delete from database asynchronously

    try {
      await supabase
        .from('notifications')
        .delete()
        .eq('user_id', userId)
    } catch (e) {
      console.warn('DB delete failed, local state retained:', e)
    }
  }

  const unreadCount = notifications.filter(n => !n.read_at).length

  return {
    notifications,
    loading,
    markAsRead,
    markAllAsRead,
    deleteNotification,
    clearAllNotifications,
    unreadCount
  }
}

export function getNotificationRedirectUrl(type: string, role: string | undefined): string {
  if (!role) return '/'

  if (type.startsWith('enrollment_request:')) {
    const courseId = type.split(':')[1]
    return `/trainer/courses/${courseId}`
  }
  if (type.startsWith('enrollment_status:')) {
    const courseId = type.split(':')[1]
    return `/trainee/courses/${courseId}`
  }
  if (type.startsWith('course_message:')) {
    const courseId = type.split(':')[1]
    return role === 'trainer' ? `/trainer/courses/${courseId}` : `/trainee/courses/${courseId}`
  }
  if (type.startsWith('course_announcement:') || type.startsWith('announcement:')) {
    const courseId = type.split(':')[1]
```

```
    return role === 'trainer' ? `/trainer/courses/${courseId}` : `/trainee/courses/${courseId}`
  }
  if (type.startsWith('course_session:') || type.startsWith('session:') || type.startsWith('live_session:')) {
    const courseId = type.split(':')[1]
    return role === 'trainer' ? `/trainer/courses/${courseId}/sessions` : `/trainee/courses/${courseId}`
  }

  switch (type) {
    case 'course_announcement':
    case 'platform_announcement':
    case 'announcement':
      return role === 'trainer' ? '/trainer' : role === 'trainee' ? '/trainee' : '/admin'
    case 'course_session':
    case 'session':
    case 'live_session':
      return role === 'trainer' ? '/trainer/courses' : '/trainee/courses'
    case 'user_registration':
    case 'course_submission':
      return role === 'super_admin' ? '/super-admin' : '/admin'
    case 'course_status':
    case 'course_assigned':
    case 'enrollment_request':
      return '/trainer/courses'
    case 'enrollment_status':
    case 'certificate_issued':
    case 'assessment_result':
      return '/trainee/my-learning'
    default:
      return `/${role}`
  }
}

export interface NotificationMeta {
  category: 'announcement' | 'session' | 'course' | 'chat' | 'user' | 'system'
  label: string
  color: string
}

export function getNotificationMeta(type: string): NotificationMeta {
  if (type.startsWith('course_announcement:') || type.startsWith('announcement:') || type === 'platform_announcement' || type
=== 'announcement' || type === 'course_announcement') {
    return {
      category: 'announcement',
      label: 'Announcement',
      color: 'bg-amber-50 text-amber-700 border-amber-200'
    }
  }
  if (type.startsWith('course_session:') || type.startsWith('session:') || type.startsWith('live_session:') || type === 'cours
e_session' || type === 'session' || type === 'live_session') {
    return {
      category: 'session',
      label: 'Live Session',
      color: 'bg-indigo-50 text-indigo-700 border-indigo-200'
    }
  }
  if (type.startsWith('course_message:')) {
    return {
      category: 'chat',
      label: 'Message',
      color: 'bg-cyan-50 text-cyan-700 border-cyan-200'
    }
  }
  if (type === 'user_registration') {
    return {
      category: 'user',
      label: 'User Account',
      color: 'bg-blue-50 text-blue-700 border-blue-200'
    }
  }
  if (type === 'certificate_issued' || type === 'assessment_result' || type === 'enrollment' || type === 'enrollment_status')
{
    return {
      category: 'course',
      label: 'Enrollment / Result',
      color: 'bg-emerald-50 text-emerald-700 border-emerald-200'
    }
  }
  return {
    category: 'system',
```

```
    label: 'Platform',
    color: 'bg-slate-100 text-slate-700 border-slate-200'
  }
}

```

src/pages/AboutPage.tsx TSX 29.2 KB 515 Lines

```tsx
import React, { useState } from 'react'
import { Link } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import {
  Compass, Shield, Award, Users, BookOpen, CheckCircle2,
  ArrowRight, Globe, Sparkles, Building, Landmark, Target,
  Eye, GraduationCap, Server, Layers, BarChart3, WifiOff,
  Cpu, Check, ChevronRight, LogIn, Menu, X, Mail, Phone
} from 'lucide-react'
import { StarfieldCanvas } from '@/components/space/StarfieldCanvas'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

export function AboutPage() {
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false)

  return (
    <div className="min-h-screen bg-[#030712] font-sans selection:bg-cyan-500 selection:text-white overflow-x-hidden">
      {/* Background Starfield Particle Canvas */}
      <StarfieldCanvas />

      {/* Decorative ambient atmospheric glow orbs */}
      <div className="pointer-events-none fixed -top-40 -left-40 w-[650px] h-[650px] rounded-full bg-cyan-500/10 blur-[150px] z-0" />
      <div className="pointer-events-none fixed top-1/3 -right-40 w-[600px] h-[600px] rounded-full bg-blue-500/10 blur-[150px] z-0" />

      {/* 1. Header Navigation Bar (Dark Theme with Center Links) */}
      <header className="fixed top-0 left-0 right-0 z-50 bg-[#040814]/95 backdrop-blur-xl border-b border-cyan-500/20 shadow-2xl shadow-cyan-950/60">
        <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
          {/* Brand Logo & Name */}
          <Link to="/" className="flex items-center gap-3 group cursor-pointer shrink-0">
            <img src="/logo.png" alt="Capacity Connect Logo" className="w-8 h-8 object-contain shrink-0 group-hover:scale-105 transition-transform" />
            <span className="text-lg font-bold font-display tracking-tight leading-tight">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
                {' '}Connect
              </span>
            </span>
          </Link>

          {/* Center Navigation Links (Desktop: Courses, About, Contact, Get Started) */}
          <nav className="hidden md:flex items-center gap-8">
            <Link to="/courses" className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors">
              Courses
            </Link>
            <span className="text-sm font-bold text-cyan-400 border-b-2 border-cyan-400 pb-0.5 cursor-default">
              About
            </span>
            <Link to="/contact" className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors">
              Contact
            </Link>
            <Link to="/register" className="text-sm font-semibold text-cyan-300 hover:text-white transition-colors">
              Get Started
            </Link>
          </nav>

          {/* Desktop Right Placeholder (Empty for clean balanced alignment) */}
          <div className="hidden md:block w-32" />

          {/* Mobile Hamburger Button */}
          <div className="flex md:hidden items-center gap-2">
            <button
              type="button"
              onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
              className="p-2 rounded-xl text-white hover:text-cyan-300 transition-colors cursor-pointer"
              aria-label="Toggle menu"
            >
              {mobileMenuOpen ? <X className="w-5 h-5" /> : <Menu className="w-5 h-5" />}
            </button>
```

```
          </div>
        </div>

        {/* Mobile Dropdown Menu */}
        <AnimatePresence>
          {mobileMenuOpen && (
            <motion.div
              initial={{ opacity: 0, height: 0 }}
              animate={{ opacity: 1, height: 'auto' }}
              exit={{ opacity: 0, height: 0 }}
              className="md:hidden border-t border-cyan-500/20 bg-[#040814]/98 backdrop-blur-xl px-6 py-4 space-y-3 shadow-2x
l"
            >
              <Link to="/courses" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-slate-3
00 hover:text-cyan-400 py-1.5">
                Courses
              </Link>
              <span className="block text-sm font-bold text-cyan-400 py-1.5">
                About
              </span>
              <Link to="/contact" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-slate-3
00 hover:text-cyan-400 py-1.5">
                Contact
              </Link>
              <Link to="/register" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-cyan-3
00 hover:text-white py-1.5">
                Get Started
              </Link>
            </motion.div>
          )}
        </AnimatePresence>
      </header>

      {/* 2. Hero Section (Clean Crisp Light Theme) */}
      <section className="relative pt-32 pb-20 md:pt-40 md:pb-28 text-center px-6 overflow-hidden bg-gradient-to-b from-[#F8FA
FC] via-[#F1F5F9] to-[#E2E8F0] text-slate-900 border-b border-slate-200/80">
        {/* Subtle decorative radial gradients */}
        <div className="pointer-events-none absolute top-0 left-1/2 -translate-x-1/2 w-full max-w-7xl h-full bg-[radial-gradie
nt(ellipse_at_top,_var(--tw-gradient-stops))] from-cyan-100/60 via-transparent to-transparent opacity-70" />

        <div className="max-w-4xl mx-auto space-y-6 relative z-10">
          <motion.div variants={fadeUp} initial="hidden" animate="visible" className="space-y-4">
            <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-cyan-50 border border-cyan-200 text-cya
n-800 text-xs font-bold uppercase tracking-wider shadow-sm backdrop-blur-md">
              <Sparkles className="w-3.5 h-3.5 text-cyan-600" />
              Smart India Hackathon • Problem Statement SIH26075
            </div>

            <h1 className="text-3xl sm:text-5xl md:text-6xl font-black text-slate-900 tracking-tight leading-tight">
              About the <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-70
0">Learning Management System</span>
            </h1>

            <p className="text-slate-600 text-sm sm:text-base md:text-lg leading-relaxed max-w-3xl mx-auto font-normal">
              A dedicated digital training and institutional capacity-building platform developed for the{' '}
              <span className="text-cyan-800 font-bold">Ministry of Earth Sciences (MoES)</span>, Government of India, and its
constituent premier scientific units.
            </p>
          </motion.div>

          {/* Institutional Badges Banner (Light Themed Cards) */}
          <motion.div
            variants={stagger}
            initial="hidden"
            animate="visible"
            className="pt-6 grid grid-cols-2 sm:grid-cols-4 gap-3.5 max-w-4xl mx-auto"
          >
            {[
              { code: 'MoES', label: 'Ministry of Earth Sciences', sub: 'Govt. of India' },
              { code: 'IMD', label: 'India Meteorological Dept.', sub: 'Est. 1875' },
              { code: 'IITM', label: 'Tropical Meteorology', sub: 'Pune Research Hub' },
              { code: 'NCMRWF', label: 'Medium Range Forecasting', sub: 'Noida Center' },
            ].map((inst) => (
              <motion.div
                key={inst.code}
                variants={fadeUp}
                className="p-4 rounded-2xl bg-white border border-slate-200/90 shadow-md hover:border-cyan-400 hover:shadow-xl
hover:shadow-slate-300/50 transition-all duration-300 text-center"
              >
```

```
                <span className="text-cyan-600 font-black text-lg block tracking-wider">{inst.code}</span>
                <span className="text-slate-800 text-xs font-bold block truncate mt-0.5">{inst.label}</span>
                <span className="text-[10px] text-slate-500 font-medium">{inst.sub}</span>
              </motion.div>
            ))}
          </motion.div>
        </div>
      </section>

      {/* 3. Featured Institutional Landmarks (Visual Heritage Gallery - Dark BG with Light Themed Cards) */}
      <section className="py-20 sm:py-28 bg-gradient-to-b from-[#030712] via-[#07132a] to-[#040814] text-white relative z-10 b
order-t border-cyan-500/20">
        <div className="max-w-7xl mx-auto px-6">
          <div className="text-center max-w-2xl mx-auto mb-14">
            <h2 className="text-2xl sm:text-3xl font-black text-white tracking-tight mb-2 flex items-center justify-center gap
-2">
              <Landmark className="w-6 h-6 text-cyan-400" />
              Institutions of Scientific Excellence
            </h2>
            <p className="text-xs sm:text-sm text-slate-300">
              Anchored at the forefront of atmospheric, oceanographic, seismological, and climate sciences in India.
            </p>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            {/* Image Card 1: MoES Headquarters (Light Themed Card) */}
            <motion.div
              variants={fadeUp}
              initial="hidden"
              whileInView="visible"
              viewport={{ once: true }}
              className="group relative rounded-3xl overflow-hidden bg-white border border-slate-200/90 shadow-2xl shadow-cyan
-950/80 hover:border-cyan-400 hover:shadow-cyan-500/20 transition-all duration-300 flex flex-col"
            >
              <div className="h-72 sm:h-80 w-full overflow-hidden relative bg-slate-100">
                <img
                  src="/MOES.jpg"
                  alt="Ministry of Earth Sciences (MoES) Headquarters"
                  className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-700"
                />
                <div className="absolute top-4 left-4">
                  <span className="px-3.5 py-1 rounded-full bg-white/95 backdrop-blur-md border border-slate-200 text-cyan-800
text-xs font-extrabold uppercase tracking-wide shadow-md">
                    Prithvi Bhavan • New Delhi
                  </span>
                </div>
              </div>
              <div className="p-6 sm:p-7 bg-white text-slate-900 border-t border-slate-100 flex-1 flex flex-col justify-betwee
n">
                <div>
                  <h3 className="text-xl font-black text-slate-900 mb-2 group-hover:text-cyan-700 transition-colors">
                    Ministry of Earth Sciences (MoES)
                  </h3>
                  <p className="text-xs sm:text-sm text-slate-600 leading-relaxed font-normal">
                    Reorganized in 2006 to bring together premier scientific institutions under an integrated mandate spanning
atmospheric science, ocean technology, polar exploration, and seismological research for socio-economic development.
                  </p>
                </div>
              </div>
            </motion.div>

            {/* Image Card 2: IMD Iconic Campus (Light Themed Card) */}
            <motion.div
              variants={fadeUp}
              initial="hidden"
              whileInView="visible"
              viewport={{ once: true }}
              className="group relative rounded-3xl overflow-hidden bg-white border border-slate-200/90 shadow-2xl shadow-cyan
-950/80 hover:border-amber-400 hover:shadow-amber-500/20 transition-all duration-300 flex flex-col"
            >
              <div className="h-72 sm:h-80 w-full overflow-hidden relative bg-slate-100">
                <img
                  src="/IMD.jpg"
                  alt="India Meteorological Department (IMD) Headquarters"
                  className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-700"
                />
                <div className="absolute top-4 left-4">
                  <span className="px-3.5 py-1 rounded-full bg-white/95 backdrop-blur-md border border-slate-200 text-amber-80
0 text-xs font-extrabold uppercase tracking-wide shadow-md">
```

```
                    Mausam Bhavan • Est. 1875
                  </span>
                </div>
              </div>
              <div className="p-6 sm:p-7 bg-white text-slate-900 border-t border-slate-100 flex-1 flex flex-col justify-between">
                <div>
                  <h3 className="text-xl font-black text-slate-900 mb-2 group-hover:text-amber-700 transition-colors">
                    India Meteorological Department (IMD)
                  </h3>
                  <p className="text-xs sm:text-sm text-slate-600 leading-relaxed font-normal">
                    Tracing its lineage back to 1875, IMD manages one of the world's most extensive meteorological and Doppler radar networks, delivering 24/7 weather forecasting, cyclone warnings, and climate monitoring.
                  </p>
                </div>
              </div>
            </motion.div>
          </div>
        </div>
      </section>

      {/* 4. Balanced Light Content Section: Genesis, Background & Vision/Mission */}
      <section className="py-20 sm:py-28 bg-gradient-to-b from-[#f8fafc] via-white to-[#f1f5f9] text-slate-900 relative z-10 border-t border-slate-200">
        <div className="max-w-7xl mx-auto px-6">
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-12 items-start">

            {/* Left Column: Background & The Challenge */}
            <motion.div
              variants={fadeUp}
              initial="hidden"
              whileInView="visible"
              viewport={{ once: true }}
              className="lg:col-span-7 space-y-6"
            >
              <div className="space-y-3">
                <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-cyan-100 text-cyan-800 text-xs font-extrabold uppercase tracking-wider">
                  <Globe className="w-3.5 h-3.5 text-cyan-700" /> Genesis & Background
                </div>
                <h2 className="text-2xl sm:text-4xl font-black text-slate-900 tracking-tight leading-tight">
                  Bridging the Gap from Scattered Resources to Unified Learning
                </h2>
              </div>

              <div className="text-slate-700 space-y-4 text-sm sm:text-base leading-relaxed">
                <p>
                  The Ministry of Earth Sciences depends on a highly specialized, continuously evolving workforce of scientists, forecasters, radar engineers, and observation officers. With its mandate spanning atmospheric sciences, ocean science and technology, seismology, and polar research, standardizing operational competencies across diverse field stations is a genuine operational imperative.
                </p>
                <p>
                  Historically, training for MoES personnel — from field-level observers at remote meteorological stations to high-performance computing research scientists — relied on fragmented channels: in-person workshops, disconnected PDFs, circulars, and institutional knowledge passed down informally.
                </p>
                <div className="p-4 rounded-2xl bg-cyan-50 border border-cyan-200 text-cyan-900 text-sm font-semibold leading-relaxed">
                  This LMS was conceived under Smart India Hackathon problem statement SIH26075 to close that gap — providing a unified, centralized, and measurable digital learning environment accessible anytime, anywhere.
                </div>
              </div>
            </motion.div>

            {/* Right Column: Vision & Mission Cards */}
            <motion.div
              variants={stagger}
              initial="hidden"
              whileInView="visible"
              viewport={{ once: true }}
              className="lg:col-span-5 space-y-6"
            >
              {/* Vision Card */}
              <motion.div variants={fadeUp} className="p-7 rounded-3xl bg-gradient-to-br from-[#040814] to-[#07132a] text-white border border-cyan-500/30 shadow-xl shadow-slate-200/80">
                <div className="w-10 h-10 rounded-2xl bg-cyan-500/20 border border-cyan-400/40 flex items-center justify-center text-cyan-300 mb-4 shadow-sm">
                  <Eye className="w-5 h-5" />
```

```
                </div>
                <h3 className="text-lg font-bold text-white mb-2">Our Vision</h3>
                <p className="text-xs sm:text-sm text-slate-300 leading-relaxed font-normal">
                  To build a unified knowledge and training ecosystem that mirrors MoES's own vision — <em>"to excel as a know
ledge and technology enterprise in the earth system science realm towards socio-economic benefit of the society"</em> — ensuring eve
ry scientist, officer, and technical staff member has structured access to the training they need.
                </p>
              </motion.div>

              {/* Mission Pillars Card */}
              <motion.div variants={fadeUp} className="p-7 rounded-3xl bg-white border border-slate-200/90 shadow-xl shadow-sl
ate-200/60">
                <div className="w-10 h-10 rounded-2xl bg-blue-50 border border-blue-200 flex items-center justify-center text-
blue-600 mb-4 shadow-sm">
                  <Target className="w-5 h-5" />
                </div>
                <h3 className="text-lg font-bold text-slate-900 mb-3">Our Mission</h3>
                <ul className="space-y-2.5 text-xs sm:text-sm text-slate-700">
                  <li className="flex items-start gap-2.5">
                    <CheckCircle2 className="w-4 h-4 text-cyan-600 shrink-0 mt-0.5" />
                    <span>Delivers structured courses on meteorology, climatology, oceanography, and seismology.</span>
                  </li>
                  <li className="flex items-start gap-2.5">
                    <CheckCircle2 className="w-4 h-4 text-cyan-600 shrink-0 mt-0.5" />
                    <span>Tracks learner progress, certifications, and skill development across MoES units.</span>
                  </li>
                  <li className="flex items-start gap-2.5">
                    <CheckCircle2 className="w-4 h-4 text-cyan-600 shrink-0 mt-0.5" />
                    <span>Reduces dependence on ad hoc training by enabling self-paced & blended learning.</span>
                  </li>
                  <li className="flex items-start gap-2.5">
                    <CheckCircle2 className="w-4 h-4 text-cyan-600 shrink-0 mt-0.5" />
                    <span>Gives ministry heads instant visibility into compliance and institutional skill gaps.</span>
                  </li>
                </ul>
              </motion.div>
            </motion.div>
          </div>
        </div>
      </section>

      {/* 5. "Who It's For" Stakeholder Grid */}
      <section className="py-20 sm:py-28 bg-[#040814] text-white border-t border-cyan-500/20 relative z-10">
        <div className="max-w-7xl mx-auto px-6">
          <div className="text-center max-w-2xl mx-auto mb-16">
            <span className="text-xs font-bold uppercase tracking-wider text-cyan-400">Target Cadres</span>
            <h2 className="text-2xl sm:text-4xl font-black text-white tracking-tight mt-1 mb-3">
              Who Is Capacity Connect For?
            </h2>
            <p className="text-xs sm:text-sm text-slate-400">
              Configured specifically to support roles and operational workflows across every tier of the Ministry.
            </p>
          </div>

          <motion.div
            variants={stagger}
            initial="hidden"
            whileInView="visible"
            viewport={{ once: true }}
            className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6"
          >
            {[
              {
                title: 'New Recruits & Trainees',
                badge: 'Onboarding',
                icon: GraduationCap,
                color: 'from-cyan-500 to-blue-600',
                border: 'border-cyan-500/30',
                desc: 'Structured onboarding pathways for new personnel joining IMD, IITM, NCMRWF, and other MoES units to bui
ld domain fundamentals quickly.',
              },
              {
                title: 'Field Officers & Observers',
                badge: 'Field Operations',
                icon: Shield,
                color: 'from-blue-500 to-indigo-600',
                border: 'border-blue-500/30',
                desc: 'Refreshers on weather instrumentation, Doppler radar operations, surface observation protocols, and aut
omated data reporting standards.',
```

```
352 |               },
353 |               {
354 |                 title: 'Scientists & Forecasters',
355 |                 badge: 'Research & NWP',
356 |                 icon: Cpu,
357 |                 color: 'from-teal-500 to-emerald-600',
358 |                 border: 'border-teal-500/30',
359 |                 desc: 'Upskilling on numerical weather prediction models, satellite meteorology, cyclone analysis, and seasona
l forecasting techniques.',
360 |               },
361 |               {
362 |                 title: 'Department Administrators',
363 |                 badge: 'Compliance',
364 |                 icon: BarChart3,
365 |                 color: 'from-amber-500 to-orange-600',
366 |                 border: 'border-amber-500/30',
367 |                 desc: 'Visibility to assign mandatory modules, monitor completion metrics, verify certifications, and address
organizational skill gaps.',
368 |               },
369 |             ].map((cadre) => (
370 |               <motion.div
371 |                 key={cadre.title}
372 |                 variants={fadeUp}
373 |                 className={`p-6 sm:p-7 rounded-3xl bg-slate-900/90 border ${cadre.border} shadow-2xl hover:-translate-y-1.5 tr
ansition-all duration-300 flex flex-col justify-between`}
374 |               >
375 |                 <div>
376 |                   <div className="flex items-center justify-between mb-4">
377 |                     <div className={`w-12 h-12 rounded-2xl bg-gradient-to-br ${cadre.color} flex items-center justify-center t
ext-white shadow-md`}>
378 |                       <cadre.icon className="w-6 h-6" />
379 |                     </div>
380 |                     <span className="text-[10px] font-mono font-bold px-2.5 py-1 rounded-full bg-white/10 text-cyan-300 border
border-white/15">
381 |                       {cadre.badge}
382 |                     </span>
383 |                   </div>
384 |                   <h3 className="text-base font-bold text-white mb-2">{cadre.title}</h3>
385 |                   <p className="text-xs text-slate-400 leading-relaxed">{cadre.desc}</p>
386 |                 </div>
387 |               </motion.div>
388 |             ))}
389 |           </motion.div>
390 |         </div>
391 |       </section>
392 |
393 |       {/* 6. Key Platform Features Grid (Balanced Light Backdrop) */}
394 |       <section className="py-20 sm:py-28 bg-[#f8fafc] text-slate-900 border-t border-slate-200 relative z-10">
395 |         <div className="max-w-7xl mx-auto px-6">
396 |           <div className="text-center max-w-2xl mx-auto mb-16">
397 |             <span className="text-xs font-bold uppercase tracking-wider text-cyan-700">Platform Features</span>
398 |             <h2 className="text-2xl sm:text-4xl font-black text-slate-900 tracking-tight mt-1 mb-3">
399 |               Key Platform Capabilities
400 |             </h2>
401 |             <p className="text-xs sm:text-sm text-slate-600">
402 |               Engineered to meet the mission-critical training requirements of scientific organizations.
403 |             </p>
404 |           </div>
405 |
406 |           <motion.div
407 |             variants={stagger}
408 |             initial="hidden"
409 |             whileInView="visible"
410 |             viewport={{ once: true }}
411 |             className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6"
412 |           >
413 |             {[
414 |               {
415 |                 title: 'Role-Based Course Catalogs',
416 |                 icon: Layers,
417 |                 desc: 'Tailored course tracks aligned specifically to observers, radar technicians, scientific researchers, an
d administrative personnel.',
418 |               },
419 |               {
420 |                 title: 'Digital Certification & Badges',
421 |                 icon: Award,
422 |                 desc: 'Automated certificate issuance upon 100% curriculum completion and verified 80%+ quiz score achievemen
t.',
423 |               },
```

```
              {
                title: 'Multilingual Accessibility',
                icon: Globe,
                desc: 'English and Hindi interface support mirroring MoES national accessibility standards for government work force portals.',
              },
              {
                title: 'Offline-Resilient Architecture',
                icon: WifiOff,
                desc: 'Optimized performance for remote coastal observatories, island radars, and high-altitude weather stations with limited connectivity.',
              },
              {
                title: 'Department-Wide Dashboards',
                icon: BarChart3,
                desc: 'Real-time analytics for department heads to inspect completion percentages, test scores, and compliance across ministries.',
              },
              {
                title: 'Integration-Ready Design',
                icon: Server,
                desc: 'Modular schema ready for future connection with HRMS service records and MoES's existing digital infrastructure.',
              },
            ].map((feat) => (
              <motion.div
                key={feat.title}
                variants={fadeUp}
                className="p-6 rounded-3xl bg-white border border-slate-200/90 hover:border-cyan-400 hover:shadow-xl hover:shadow-slate-200/60 transition-all duration-300 flex items-start gap-4"
              >
                <div className="w-11 h-11 rounded-2xl bg-cyan-50 border border-cyan-200 flex items-center justify-center text-cyan-600 shrink-0">
                  <feat.icon className="w-5 h-5" />
                </div>
                <div>
                  <h4 className="text-sm font-bold text-slate-900 mb-1.5">{feat.title}</h4>
                  <p className="text-xs text-slate-600 leading-relaxed">{feat.desc}</p>
                </div>
              </motion.div>
            ))}
          </motion.div>
        </div>
      </section>

      {/* 7. Alignment with MoES Broader Goals & Call to Action Banner */}
      <section className="py-20 sm:py-28 bg-[#02050E] text-white border-t border-cyan-500/20 relative z-10">
        <div className="max-w-5xl mx-auto px-6 text-center space-y-8">
          <div className="p-8 sm:p-14 rounded-3xl bg-gradient-to-b from-[#040814] to-[#07132a] border border-cyan-500/30 shadow-2xl relative overflow-hidden">
            <div className="absolute top-0 right-0 w-80 h-80 bg-cyan-500/15 rounded-full blur-3xl pointer-events-none" />

            <div className="relative z-10 space-y-4">
              <span className="text-xs font-bold uppercase tracking-wider text-amber-300">National Mandate</span>
              <h2 className="text-2xl sm:text-4xl font-black text-white tracking-tight">
                Empowering India's Scientific Workforce for Tomorrow
              </h2>
              <p className="text-xs sm:text-base text-slate-300 max-w-2xl mx-auto leading-relaxed font-normal">
                Directly supporting MoES's core function to <em>"develop skilled and trained manpower in Earth Sciences with the support of academic institutions in the country and abroad"</em> by making training scalable, measurable, and consistent across every national unit.
              </p>

              <div className="pt-6 flex flex-wrap items-center justify-center gap-4">
                <Link
                  to="/courses"
                  className="px-6 py-3 rounded-2xl bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-white font-black text-sm shadow-lg shadow-cyan-500/25 transition-all hover:scale-105 flex items-center gap-2 cursor-pointer"
                >
                  <BookOpen className="w-4 h-4" />
                  <span>Explore Course Catalog</span>
                </Link>
                <Link
                  to="/register"
                  className="px-6 py-3 rounded-2xl bg-white/10 hover:bg-white/20 border border-white/20 text-white font-bold text-sm transition-all flex items-center gap-2 cursor-pointer"
                >
                  <span>Register for Training</span>
```

```
                  <ArrowRight className="w-4 h-4" />
                </Link>
              </div>
            </div>
          </div>
        </div>
      </section>

      {/* 8. Deep Space Clean Footer */}
      <footer className="border-t border-slate-800 bg-[#02050E] py-8 text-xs text-slate-400 relative z-10">
        <div className="max-w-7xl mx-auto px-6 flex flex-col sm:flex-row items-center justify-between gap-4">
          <p>© 2026 Capacity Connect • Ministry of Earth Sciences (MoES), Govt. of India.</p>
          <div className="flex items-center gap-6">
            <Link to="/courses" className="hover:text-cyan-400 transition-colors">Courses</Link>
            <Link to="/about" className="hover:text-cyan-400 font-bold text-cyan-400 transition-colors">About</Link>
            <Link to="/contact" className="hover:text-cyan-400 transition-colors">Contact</Link>
            <Link to="/login" className="hover:text-cyan-400 transition-colors">Login</Link>
            <Link to="/register" className="hover:text-cyan-400 transition-colors">Register</Link>
          </div>
        </div>
      </footer>
    </div>
  )
}
```

src/pages/AccessDeniedPage.tsx TSX 2.4 KB 48 Lines

```tsx
import React from 'react'
import { motion } from 'framer-motion'
import { ShieldAlert, ArrowLeft } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { useNavigate, Link } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'

export function AccessDeniedPage() {
  const navigate = useNavigate()
  const { session } = useAuth()

  return (
    <div className="min-h-screen bg-[#040814] text-white flex items-center justify-center p-4 relative overflow-hidden">
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-rose-500/20 via-orange-500/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10"
      >
        <div className="bg-white text-slate-900 border border-slate-200/90 rounded-3xl p-6 md:p-10 shadow-2xl text-center">
          <div className="w-16 h-16 md:w-20 md:h-20 bg-rose-50 border border-rose-200 rounded-2xl flex items-center justify-center mx-auto mb-4 md:mb-6">
            <ShieldAlert className="w-8 h-8 md:w-10 md:h-10 text-rose-500" />
          </div>
          <h1 className="text-xl md:text-2xl font-bold text-slate-900 mb-3">Access Denied</h1>
          <p className="text-slate-600 leading-relaxed mb-8 text-sm">
            You do not have permission to access this page. Your account role does not grant the required privileges.
          </p>
          <div className="space-y-3">
            {session ? (
              <Button onClick={() => navigate('/dashboard')} className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold rounded-xl shadow-md transition-all">
                <ArrowLeft className="w-4 h-4 mr-2" /> Go to Dashboard
              </Button>
            ) : (
              <Link to="/login" className="block w-full">
                <Button className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold rounded-xl shadow-md transition-all">
                  <ArrowLeft className="w-4 h-4 mr-2" /> Sign In
                </Button>
              </Link>
            )}
          </div>
        </div>
      </motion.div>
    </div>
  )
}

```

src/pages/AccountSuspendedPage.tsx TSX 2.4 KB 50 Lines

```tsx
import React from 'react'
import { motion } from 'framer-motion'
import { Ban, LogOut, Home } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { useAuth } from '@/hooks/useAuth'
import { useNavigate } from 'react-router-dom'

export function AccountSuspendedPage() {
  const { signOut } = useAuth()
  const navigate = useNavigate()

  const handleSignOut = async () => {
    await signOut()
    navigate('/login', { replace: true })
  }

  return (
    <div className="min-h-screen bg-[#040814] flex items-center justify-center p-4 relative overflow-hidden">
      {/* Background radiant glow */}
      <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[500px] h-[500px] bg-red-600/10 rounded-full blur-3xl pointer-events-none" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10"
      >
        <div className="bg-white border border-slate-200/90 rounded-3xl p-6 md:p-10 shadow-2xl text-center">
          <div className="w-16 h-16 md:w-20 md:h-20 bg-red-50 border border-red-200 rounded-2xl flex items-center justify-center mx-auto mb-4 md:mb-6 shadow-sm">
            <Ban className="w-8 h-8 md:w-10 md:h-10 text-red-600" />
          </div>
          <h1 className="text-xl md:text-2xl font-black text-slate-900 mb-3 tracking-tight">Account Suspended</h1>
          <p className="text-slate-600 leading-relaxed mb-8 text-sm font-medium">
            Your account has been suspended by an administrator. You currently do not have access to the platform.
            Please contact support if you believe this is an error.
          </p>
          <div className="space-y-3">
            <Button onClick={handleSignOut} className="w-full h-11 bg-red-600 hover:bg-red-700 text-white font-semibold rounded-xl shadow-md shadow-red-500/20 transition-all">
              <LogOut className="w-4 h-4 mr-2" /> Sign Out
            </Button>
            <Button variant="outline" onClick={() => navigate('/')} className="w-full h-11 border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold rounded-xl transition-all">
              <Home className="w-4 h-4 mr-2" /> Back to Home
            </Button>
          </div>
        </div>
      </motion.div>
    </div>
  )
}

```

src/pages/ContactPage.tsx TSX 21.2 KB 431 Lines

```tsx
import React, { useState } from 'react'
import { Link } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import {
  MapPin, Phone, Mail, Clock, Globe, Send, CheckCircle2,
  Building, User, Shield, FileText, Sparkles, ExternalLink,
  LogIn, Menu, X, HelpCircle, MessageSquare, AlertCircle, RefreshCw
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { supabase } from '@/lib/supabase'
import { toast } from 'sonner'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

export function ContactPage() {
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false)
  const [isSubmitting, setIsSubmitting] = useState(false)
  const [submitted, setSubmitted] = useState(false)
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    department: 'IMD',
    subject: '',
    message: '',
  })

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!formData.name.trim() || !formData.email.trim() || !formData.message.trim()) {
      toast.error('Please complete all required fields (Name, Email, Message).')
      return
    }

    setIsSubmitting(true)
    try {
      // Attempt insert into contact_messages

      const { error } = await (supabase as any).from('contact_messages').insert({
        name: formData.name.trim(),
        email: formData.email.trim().toLowerCase(),
        department: formData.department || 'MoES',
        subject: formData.subject.trim() || 'General Inquiry',
        message: formData.message.trim(),
        status: 'unread',
        created_at: new Date().toISOString(),
      })

      if (error) {
        console.warn('Supabase contact_messages table insert notice:', error.message)
        // Store in localStorage as fallback so admin dashboard can still read it

        const existing = JSON.parse(localStorage.getItem('local_contact_messages') || '[]')
        existing.unshift({
          id: 'local_' + Date.now(),
          name: formData.name.trim(),
          email: formData.email.trim(),
          department: formData.department,
          subject: formData.subject.trim() || 'General Inquiry',
          message: formData.message.trim(),
          status: 'unread',
          created_at: new Date().toISOString(),
        })
        localStorage.setItem('local_contact_messages', JSON.stringify(existing))
      }

      setSubmitted(true)
      toast.success('Your message has been received! Our support team will respond shortly.')
      setFormData({ name: '', email: '', department: 'IMD', subject: '', message: '' })
    } catch (err: any) {
```

```
      console.error('Contact submission error:', err)
      toast.error('Could not submit message. Please try again or use direct email.')
    } finally {
      setIsSubmitting(false)
    }
  }

  return (
    <div className="min-h-screen bg-gradient-to-b from-[#F8FAFC] via-[#F1F5F9] to-[#FFFFFF] text-slate-900 font-sans selectio
n:bg-cyan-500 selection:text-white overflow-x-hidden">

      {/* 1. Header Navigation Bar (Dark Theme with Center Links) */}
      <header className="fixed top-0 left-0 right-0 z-50 bg-[#040814]/95 backdrop-blur-xl border-b border-cyan-500/20 shadow-2
xl shadow-cyan-950/60">
        <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
          {/* Brand Logo & Name */}
          <Link to="/" className="flex items-center gap-3 group cursor-pointer shrink-0">
            <img src="/logo.png" alt="Capacity Connect Logo" className="w-8 h-8 object-contain shrink-0 group-hover:scale-105
transition-transform" />
            <span className="text-lg font-bold font-display tracking-tight leading-tight">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
                {' '}Connect
              </span>
            </span>
          </Link>

          {/* Center Navigation Links (Desktop: Courses, About, Contact, Get Started) */}
          <nav className="hidden md:flex items-center gap-8">
            <Link to="/courses" className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors">
              Courses
            </Link>
            <Link to="/about" className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors">
              About
            </Link>
            <span className="text-sm font-bold text-cyan-400 border-b-2 border-cyan-400 pb-0.5 cursor-default">
              Contact
            </span>
            <Link to="/register" className="text-sm font-semibold text-cyan-300 hover:text-white transition-colors">
              Get Started
            </Link>
          </nav>

          {/* Desktop Right Placeholder (Empty for clean balanced alignment) */}
          <div className="hidden md:block w-32" />

          {/* Mobile Hamburger Button */}
          <div className="flex md:hidden items-center gap-2">
            <button
              type="button"
              onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
              className="p-2 rounded-xl text-slate-300 hover:text-cyan-400 transition-colors cursor-pointer"
              aria-label="Toggle menu"
            >
              {mobileMenuOpen ? <X className="w-5 h-5" /> : <Menu className="w-5 h-5" />}
            </button>
          </div>
        </div>

        {/* Mobile Dropdown Menu */}
        <AnimatePresence>
          {mobileMenuOpen && (
            <motion.div
              initial={{ opacity: 0, height: 0 }}
              animate={{ opacity: 1, height: 'auto' }}
              exit={{ opacity: 0, height: 0 }}
              className="md:hidden border-t border-cyan-500/20 bg-[#040814]/98 backdrop-blur-xl px-6 py-4 space-y-3 shadow-2x
l"
            >
              <Link to="/courses" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-slate-3
00 hover:text-cyan-400 py-1.5">
                Courses
              </Link>
              <Link to="/about" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-slate-300
hover:text-cyan-400 py-1.5">
                About
              </Link>
              <span className="block text-sm font-bold text-cyan-400 py-1.5">
                Contact
```

```
148 |               </span>
149 |               <Link to="/register" onClick={() => setMobileMenuOpen(false)} className="block text-sm font-semibold text-cyan-3
00 hover:text-white py-1.5">
150 |                 Get Started
151 |               </Link>
152 |             </motion.div>
153 |           )}
154 |         </AnimatePresence>
155 |       </header>
156 |
157 |       {/* 2. Hero Section (Clean Light Theme) */}
158 |       <section className="relative pt-32 pb-14 md:pt-40 md:pb-20 px-6 text-center overflow-hidden border-b border-slate-200/8
0">
159 |         <div className="max-w-4xl mx-auto space-y-4 relative z-10">
160 |           <motion.div variants={fadeUp} initial="hidden" animate="visible" className="space-y-3">
161 |             <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-cyan-50 border border-cyan-200 text-cya
n-800 text-xs font-bold uppercase tracking-wider shadow-sm">
162 |               <Building className="w-3.5 h-3.5 text-cyan-600" />
163 |               Ministry of Earth Sciences • Contact & Communication Directory
164 |             </div>
165 |
166 |             <h1 className="text-3xl sm:text-5xl font-black text-slate-900 tracking-tight leading-tight">
167 |               Get in Touch with <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-600 via-sky-600 to-b
lue-700">Capacity Connect</span>
168 |             </h1>
169 |
170 |             <p className="text-slate-600 text-sm sm:text-base md:text-lg leading-relaxed max-w-2xl mx-auto font-normal">
171 |               Official institutional directory, grievance redressal, RTI authorities, and direct inquiry portal for the Minist
ry of Earth Sciences, Government of India.
172 |             </p>
173 |           </motion.div>
174 |         </div>
175 |       </section>
176 |
177 |       {/* 3. Official Officers & RTI Authority Directory Grid (Appears First) */}
178 |       <section className="py-14 sm:py-20 bg-white border-b border-slate-200">
179 |         <div className="max-w-7xl mx-auto px-6">
180 |           <div className="text-center max-w-2xl mx-auto mb-12">
181 |             <span className="text-xs font-extrabold uppercase tracking-wider text-cyan-700">Official Cadre</span>
182 |             <h2 className="text-2xl sm:text-3xl font-black text-slate-900 tracking-tight mt-1 mb-2">
183 |               Key Officers &amp;amp; Institutional Contacts
184 |             </h2>
185 |             <p className="text-xs sm:text-sm text-slate-600">
186 |               Direct administrative touchpoints for Web Information, CIO, CPIO, and RTI Appellate inquiries.
187 |             </p>
188 |           </div>
189 |
190 |           <motion.div
191 |             variants={stagger}
192 |             initial="hidden"
193 |             whileInView="visible"
194 |             viewport={{ once: true }}
195 |             className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6"
196 |           >
197 |             {[
198 |               {
199 |                 role: 'Content Related & Web Information Manager (WIM)',
200 |                 name: 'Dr. R S Maheskumar',
201 |                 designation: 'Scientist G',
202 |                 address: 'Ministry of Earth Sciences, Prithvi Bhawan, opposite India Habitat Centre, Lodhi Road, New Delhi 110
003',
203 |                 phone: '+91-11-24669725',
204 |                 email: 'mahesh.rs@gov.in',
205 |                 badgeBg: 'bg-cyan-50 text-cyan-800 border-cyan-200',
206 |               },
207 |               {
208 |                 role: 'Chief Information Officer (CIO)',
209 |                 name: 'Dr. R S Maheskumar',
210 |                 designation: 'Scientist G',
211 |                 address: 'Ministry of Earth Sciences, Prithvi Bhawan, opposite India Habitat Centre, Lodhi Road, New Delhi 110
003',
212 |                 phone: '+91-11-24669725',
213 |                 email: 'mahesh.rs@gov.in',
214 |                 badgeBg: 'bg-blue-50 text-blue-800 border-blue-200',
215 |               },
216 |               {
217 |                 role: 'Central Public Information Officer (CPIO)',
218 |                 name: 'Dr. Bhavya Khanna',
219 |                 designation: 'Scientist E',
```

```
                  address: 'Ministry of Earth Sciences, Prithvi Bhawan, opposite India Habitat Centre, Lodhi Road, New Delhi 110003',
                  phone: '+91-11-24669647',
                  email: 'cpio-moes@gov.in',
                  badgeBg: 'bg-emerald-50 text-emerald-800 border-emerald-200',
                },
                {
                  role: 'First Appellate Authority (RTI)',
                  name: 'Director(ICC)',
                  designation: 'Director',
                  address: 'Ministry of Earth Sciences, Prithvi Bhawan, opposite India Habitat Centre, Lodhi Road, New Delhi 110003',
                  phone: '+91-011-24669718',
                  email: 'rajendrakumhar@ord.gov.in',
                  badgeBg: 'bg-purple-50 text-purple-800 border-purple-200',
                },
              ].map(officer => (
                <motion.div
                  key={officer.role + officer.name}
                  variants={fadeUp}
                  className="p-6 rounded-3xl bg-slate-50 border border-slate-200/90 hover:border-cyan-400 hover:shadow-xl hover:shadow-slate-200/60 transition-all duration-300 flex flex-col justify-between space-y-4"
                >
                  <div className="space-y-2.5">
                    <span className={`inline-block text-[10px] font-bold px-2.5 py-1 rounded-full border ${officer.badgeBg} leading-tight`}>
                      {officer.role}
                    </span>

                    <div>
                      <h3 className="text-base font-black text-slate-900">{officer.name}</h3>
                      <p className="text-xs text-cyan-700 font-bold">{officer.designation}</p>
                    </div>

                    <p className="text-[11px] text-slate-600 leading-relaxed">
                      {officer.address}
                    </p>
                  </div>

                  <div className="pt-3 border-t border-slate-200 space-y-1.5 text-xs">
                    <div className="flex items-center gap-2 text-slate-700">
                      <Phone className="w-3.5 h-3.5 text-emerald-600 shrink-0" />
                      <a href={`tel:${officer.phone.replace(/[^0-9+]/g, '')}`} className="font-semibold hover:text-emerald-700 hover:underline">
                        {officer.phone}
                      </a>
                    </div>
                    <div className="flex items-center gap-2 text-slate-700">
                      <Mail className="w-3.5 h-3.5 text-cyan-600 shrink-0" />
                      <a href={`mailto:${officer.email}`} className="font-semibold text-cyan-700 hover:underline truncate">
                        {officer.email}
                      </a>
                    </div>
                  </div>
                </motion.div>
              ))}
            </motion.div>
          </div>
        </section>

        {/* 4. Send Us a Message Form */}
        <section className="py-14 sm:py-20 max-w-4xl mx-auto px-6">
          <div className="bg-white border border-slate-200/90 rounded-3xl p-6 sm:p-10 shadow-lg shadow-slate-200/50 space-y-6">
            <div className="border-b border-slate-100 pb-4 text-center max-w-xl mx-auto">
              <div className="inline-flex items-center gap-2 text-cyan-700 text-xs font-extrabold uppercase tracking-wider mb-1">
                <MessageSquare className="w-4 h-4 text-cyan-600" /> Inquiries &amp;amp; Support
              </div>
              <h2 className="text-2xl sm:text-3xl font-black text-slate-900">Send Us a Message</h2>
              <p className="text-xs sm:text-sm text-slate-500 mt-1">
                Have questions about courses, certifications, or technical onboarding? Fill out this form and our team will review it.
              </p>
            </div>

            {submitted ? (
              <motion.div
                initial={{ opacity: 0, scale: 0.95 }}
                animate={{ opacity: 1, scale: 1 }}
```

```
              className="p-8 rounded-2xl bg-emerald-50 border border-emerald-200 text-center space-y-3 my-6"
            >
              <div className="w-14 h-14 rounded-full bg-emerald-100 text-emerald-600 flex items-center justify-center mx-auto
shadow-sm">
                <CheckCircle2 className="w-7 h-7" />
              </div>
              <h3 className="text-lg font-bold text-emerald-900">Message Dispatched Successfully!</h3>
              <p className="text-xs sm:text-sm text-emerald-700 max-w-md mx-auto">
                Your inquiry has been submitted and recorded in the administrator portal. A representative will contact you vi
a email shortly.
              </p>
              <Button
                onClick={() => setSubmitted(false)}
                variant="outline"
                size="sm"
                className="mt-3 rounded-xl border-emerald-300 text-emerald-800 hover:bg-emerald-100/60 font-semibold text-xs c
ursor-pointer"
              >
                Send Another Message
              </Button>
            </motion.div>
          ) : (
            <form onSubmit={handleSubmit} className="space-y-4 max-w-2xl mx-auto">
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div className="space-y-1.5">
                  <Label htmlFor="contact-name" className="text-xs font-bold text-slate-700">
                    Full Name <span className="text-rose-500">*</span>
                  </Label>
                  <Input
                    id="contact-name"
                    placeholder="e.g. Dr. Rajesh Kumar"
                    value={formData.name}
                    onChange={e => setFormData({ ...formData, name: e.target.value })}
                    required
                    className="rounded-xl border-slate-300 focus:border-cyan-500 text-xs sm:text-sm h-11"
                  />
                </div>

                <div className="space-y-1.5">
                  <Label htmlFor="contact-email" className="text-xs font-bold text-slate-700">
                    Official / Personal Email <span className="text-rose-500">*</span>
                  </Label>
                  <Input
                    id="contact-email"
                    type="email"
                    placeholder="e.g. rajesh.kumar@imd.gov.in"
                    value={formData.email}
                    onChange={e => setFormData({ ...formData, email: e.target.value })}
                    required
                    className="rounded-xl border-slate-300 focus:border-cyan-500 text-xs sm:text-sm h-11"
                  />
                </div>
              </div>

              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div className="space-y-1.5">
                  <Label htmlFor="contact-dept" className="text-xs font-bold text-slate-700">
                    Unit / Institution
                  </Label>
                  <select
                    id="contact-dept"
                    value={formData.department}
                    onChange={e => setFormData({ ...formData, department: e.target.value })}
                    className="w-full h-11 rounded-xl border border-slate-300 bg-white px-3 text-xs sm:text-sm text-slate-800
focus:outline-none focus:ring-2 focus:ring-cyan-500/20 focus:border-cyan-500"
                  >
                    <option value="MoES">Ministry of Earth Sciences (MoES HQ)</option>
                    <option value="IMD">India Meteorological Department (IMD)</option>
                    <option value="IITM">Indian Institute of Tropical Meteorology (IITM)</option>
                    <option value="NCMRWF">NCMRWF Noida</option>
                    <option value="INCOIS">INCOIS Hyderabad</option>
                    <option value="NIOT">NIOT Chennai</option>
                    <option value="NCPOR">NCPOR Goa</option>
                    <option value="Other">Other Organization / Trainee</option>
                  </select>
                </div>

                <div className="space-y-1.5">
                  <Label htmlFor="contact-sub" className="text-xs font-bold text-slate-700">
```

```
                    Subject
                  </Label>
                  <Input
                    id="contact-sub"
                    placeholder="e.g. Assessment Verification / Training Inquiry"
                    value={formData.subject}
                    onChange={e => setFormData({ ...formData, subject: e.target.value })}
                    className="rounded-xl border-slate-300 focus:border-cyan-500 text-xs sm:text-sm h-11"
                  />
                </div>
              </div>

              <div className="space-y-1.5">
                <Label htmlFor="contact-msg" className="text-xs font-bold text-slate-700">
                  Your Message <span className="text-rose-500">*</span>
                </Label>
                <Textarea
                  id="contact-msg"
                  rows={4}
                  placeholder="Provide details about your query or technical assistance request..."
                  value={formData.message}
                  onChange={e => setFormData({ ...formData, message: e.target.value })}
                  required
                  className="rounded-xl border-slate-300 focus:border-cyan-500 text-xs sm:text-sm resize-none"
                />
              </div>

              <Button
                type="submit"
                disabled={isSubmitting}
                className="w-full h-12 rounded-xl bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue
-700 text-white font-bold text-sm shadow-md shadow-cyan-600/20 gap-2 cursor-pointer mt-2"
              >
                {isSubmitting ? (
                  <>
                    <RefreshCw className="w-4 h-4 animate-spin" /> Submitting Message...
                  </>
                ) : (
                  <>
                    <Send className="w-4 h-4" /> Send Message
                  </>
                )}
              </Button>
            </form>
          )}
        </div>
      </section>

      {/* 5. Clean Footer */}
      <footer className="border-t border-slate-200 bg-slate-100 py-8 text-xs text-slate-600">
        <div className="max-w-7xl mx-auto px-6 flex flex-col sm:flex-row items-center justify-between gap-4">
          <p>© 2026 Capacity Connect • Ministry of Earth Sciences (MoES), Govt. of India.</p>
          <div className="flex items-center gap-6">
            <Link to="/courses" className="hover:text-cyan-700 transition-colors">Courses</Link>
            <Link to="/about" className="hover:text-cyan-700 transition-colors">About</Link>
            <Link to="/contact" className="hover:text-cyan-700 font-bold text-cyan-700 transition-colors">Contact</Link>
            <Link to="/login" className="hover:text-cyan-700 transition-colors">Login</Link>
            <Link to="/register" className="hover:text-cyan-700 transition-colors">Register</Link>
          </div>
        </div>
      </footer>

    </div>
  )
}

```

src/pages/Dashboards.tsx TSX 138.4 KB 2,474 Lines

```tsx
import React, { useState, useEffect, useMemo } from 'react'
import { useQuery } from '@tanstack/react-query'
import { Link, useNavigate, useLocation } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { Database } from '@/integrations/supabase/types'
import { AdminCourses } from '@/features/courses/AdminCourses'
import { TrainerAssignmentBanner } from '@/features/admin/TrainerAssignmentBanner'
import { AdminAnnouncements } from '@/features/admin/AdminAnnouncements'
import { AdminHomePageSettings } from '@/features/admin/AdminHomePageSettings'
import { AdminContactMessages } from '@/features/admin/AdminContactMessages'
import { AdminAssessmentsHub } from '@/features/admin/AdminAssessmentsHub'
import { UniversalNotificationsPage } from '@/features/notifications/UniversalNotificationsPage'
import { NotificationPanel } from '@/components/notifications/NotificationPanel'
import { Button } from '@/components/ui/button'
import { ResponsiveContainer, Tooltip, Radar, RadarChart, PolarGrid, PolarAngleAxis, PolarRadiusAxis } from 'recharts'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { AnnouncementModal } from '@/components/ui/AnnouncementModal'
import { AnnouncementsFeed } from '@/components/ui/AnnouncementsFeed'
import { AdminUserDetailsModal } from '@/features/admin/AdminUserDetailsModal'
import {
  Globe, LogOut, Users, BookOpen, BarChart3, Shield,
  GraduationCap, ChevronRight, CheckCircle, Search,
  XCircle, Clock, Ban, ArrowUpRight, Compass, Bell,
  Award, Target, FileText, Settings, User, Mail,
  RefreshCw, Star, MessageSquare, Eye,
  Menu, X, Trash2, Loader2, LayoutDashboard, Megaphone, FileCheck, UserCheck
} from 'lucide-react'
import { toast } from 'sonner'
import { useNotifications, getNotificationRedirectUrl, getNotificationMeta } from '@/hooks/useNotifications'
import { formatDistanceToNow } from 'date-fns'
import { useConfirm } from '@/hooks/useConfirm'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'

type AuditLog = Database['public']['Tables']['audit_logs']['Row']
type Profile = (
  | Database['public']['Tables']['admins']['Row']
  | Database['public']['Tables']['trainers']['Row']
  | Database['public']['Tables']['trainees']['Row']
) & { department?: string | null; proof_path?: string | null }


const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }
const scaleIn = {
  hidden: { opacity: 0, scale: 0.95 },
  visible: { opacity: 1, scale: 1, transition: { duration: 0.3 } },
}

import { DashboardShell } from '@/components/layout/DashboardShell'
export { DashboardShell }


/* ── Status Badge ──────────────────────────────────────── */

function StatusBadge({ status }: { status: string }) {
  const config: Record<string, { color: string; icon: React.ElementType }> = {
    approved: { color: 'bg-emerald-50 text-emerald-700 border-emerald-200', icon: CheckCircle },
    pending: { color: 'bg-amber-50 text-amber-700 border-amber-200', icon: Clock },
    rejected: { color: 'bg-rose-50 text-rose-700 border-rose-200', icon: XCircle },
    suspended: { color: 'bg-rose-50 text-rose-700 border-rose-200', icon: Ban },
  }
  const c = config[status] ?? { color: 'bg-slate-100 text-slate-700 border-slate-200', icon: Clock }
  return (
    <span className={`inline-flex items-center gap-1.5 text-xs px-2.5 py-1 rounded-full border capitalize font-semibold shado
```

```
w-xs ${c.color}`}>
        <c.icon className="w-3.5 h-3.5" />
        {status}
      </span>
    )
}

/* ── Stat Card ──────────────────────────────────────────── */

function StatCard({
  label,
  value,
  icon: Icon,
  gradient,
  subtext,
  badgeText
}: {
  label: string
  value: string | number
  icon: React.ElementType
  gradient: string
  subtext?: string
  badgeText?: string
}) {
  return (
    <motion.div
      variants={fadeUp}
      className="group relative p-5 md:p-6 rounded-3xl bg-white border border-slate-200/90 shadow-sm hover:shadow-xl hover:sh
adow-slate-200/60 hover:-translate-y-1 transition-all duration-300 cursor-default overflow-hidden"
    >
      <div className="relative flex items-start justify-between mb-4">
        <div className={`w-12 h-12 rounded-2xl bg-gradient-to-br ${gradient} text-white flex items-center justify-center shad
ow-md group-hover:scale-105 transition-transform duration-300`}>
          <Icon className="w-5 h-5" />
        </div>
        {badgeText && (
          <span className="text-[11px] font-bold px-2.5 py-0.5 rounded-full bg-slate-100 text-slate-700 border border-slate-2
00">
            {badgeText}
          </span>
        )}
      </div>

      <div className="relative">
        <div className="text-3xl font-extrabold text-slate-900 tracking-tight mb-1">{value}</div>
        <div className="text-xs font-semibold text-slate-500">{label}</div>
        {subtext && <div className="text-[11px] text-cyan-700 font-medium mt-1">{subtext}</div>}
      </div>
    </motion.div>
  )
}

/* ── Activity Item ──────────────────────────────────────── */

function ActivityItem({
  icon: Icon,
  title,
  desc,
  time,
  iconBg
}: {
  icon: React.ElementType
  title: string
  desc: string
  time: string
  iconBg: string
}) {
  return (
    <div className="flex items-start gap-3.5 p-3.5 rounded-2xl hover:bg-slate-50 transition-colors group">
      <div className={`w-9 h-9 rounded-xl ${iconBg} flex items-center justify-center shrink-0 shadow-xs group-hover:scale-105
transition-transform`}>
        <Icon className="w-4 h-4" />
      </div>
      <div className="flex-1 min-w-0">
        <p className="text-xs md:text-sm text-slate-900 font-bold truncate group-hover:text-cyan-600 transition-colors">{titl
e}</p>
        <p className="text-xs text-slate-500 mt-0.5 truncate">{desc}</p>
      </div>
      <span className="text-[11px] text-slate-600 font-semibold whitespace-nowrap bg-slate-100 border border-slate-200 px-2 p
```

```
y-0.5 rounded-full">{time}</span>
    </div>
  )
}

/* ── Progress Bar ──────────────────────────────────────────── */

function ProgressBar({ value, max = 100, color = 'from-cyan-600 to-blue-600' }: { value: number; max?: number; color?: string
}) {
  const pct = Math.min(Math.round((value / max) * 100), 100)
  return (
    <div className="w-full h-2.5 bg-slate-100 rounded-full overflow-hidden p-0.5 border border-slate-200">
      <motion.div
        initial={{ width: 0 }}
        animate={{ width: `${pct}%` }}
        transition={{ duration: 0.8, ease: 'easeOut', delay: 0.2 }}
        className={`h-full bg-gradient-to-r ${color} rounded-full shadow-xs`}
      />
    </div>
  )
}

/* ── Trainee Dashboard ─────────────────────────────────────── */

export function TraineeDashboard() {
  const { profile } = useAuth()

  // Fetch real skills from Supabase (User's acquired skills)

  const { data: dbSkills = [], isLoading: skillsLoading } = useQuery({
    queryKey: ['trainee-skills', profile?.id],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('user_skills')
        .select(`
          level,
          skill:skills (name, category)
        `)
        .eq('user_id', profile!.id);

      if (error) {
        console.error("Error fetching user skills:", error);
        return [];
      }
      return data || [];
    },
    enabled: !!profile?.id,
  });

  // Fetch ALL available skills to dynamically build the radar axes

  const { data: allSkills = [], isLoading: allSkillsLoading } = useQuery({
    queryKey: ['all-available-skills'],
    queryFn: async () => {
      const { data, error } = await supabase.from('skills').select('*').order('name');
      if (error) {
        console.error("Error fetching all skills:", error);
        return [];
      }
      return data || [];
    }
  });

  // Calculate competency data merging live DB data with dynamically fetched baseline

  const competencyData = React.useMemo(() => {
    let baseline: any[] = [];

    if (allSkills && allSkills.length > 0) {
      baseline = allSkills.map((s: any) => ({
        subject: s.name,
        A: 0,
        B: 85, // Default target
        fullMark: 100
      }));
    } else {
      // Fallback if the skills table is completely empty

      baseline = [
```

```
        { subject: 'All Courses', A: 0, B: 85, fullMark: 100 },
        { subject: 'Standard Courses', A: 0, B: 85, fullMark: 100 },
        { subject: 'Scenario Courses', A: 0, B: 85, fullMark: 100 }
      ];
    }

    if (!dbSkills || dbSkills.length === 0) {
      return baseline.slice(0, 5);
    }

    // Merge DB skills into baseline

    const merged = [...baseline];

    dbSkills.forEach((ds: any) => {
      const skillName = ds.skill?.name;
      if (!skillName) return;

      const existingIdx = merged.findIndex(m => m.subject.toLowerCase() === skillName.toLowerCase());
      if (existingIdx >= 0) {
        merged[existingIdx].A = ds.level || 0;
      } else {
        // Add completely new skills from DB that aren't in baseline

        merged.push({
          subject: skillName,
          A: ds.level || 0,
          B: 85, // Default target
          fullMark: 100
        });
      }
    });

    // Sort to prioritize skills where the user has a level > 0, then by name

    merged.sort((a, b) => {
      if (b.A !== a.A) return b.A - a.A;
      return a.subject.localeCompare(b.subject);
    });

    // Limit to exactly 5 items for a basic, clean pentagon shape (prevents any overlap)

    return merged.slice(0, 5);
  }, [dbSkills, allSkills]);

  // Find top gaps for recommendations

  const topGaps = React.useMemo(() => {
    return [...competencyData]
      .map(c => ({ ...c, gap: Math.max(0, c.B - c.A) }))
      .filter(c => c.gap > 0)
      .sort((a, b) => b.gap - a.gap)
      .slice(0, 2);
  }, [competencyData]);

  const handleResetPassword = async () => {
    if (!profile?.email) return;
    try {
      const { error } = await supabase.auth.resetPasswordForEmail(profile.email, {
        redirectTo: `${window.location.origin}/setup-password`,
      });
      if (error) throw error;
      toast.success('Secure password reset email sent! Please check your inbox.');
    } catch (e: any) {
      toast.error(e.message || 'Failed to send reset email.');
    }
  };

  // ── Real data from Supabase ──────────────────────────────

  const { data: enrollments = [], isLoading: enrollmentsLoading } = useQuery({
    queryKey: ['trainee-dashboard-enrollments', profile?.id],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('enrollments')
        .select(`
          *,

          course:courses!enrollments_course_id_fkey(
```

```
290 |             id, title, status, duration_minutes,
291 |             trainer:trainers!courses_trainer_id_fkey(full_name)
292 |           )
293 |         `)
294 |         .eq('user_id', profile!.id)
295 |         .order('enrolled_at', { ascending: false })
296 |       if (error) throw error
297 |       return (data || []).filter((e: any) => e.course && e.course.status !== 'archived')
298 |     },
299 |     enabled: !!profile?.id,
300 |   })
301 |
302 |   const { data: certificates = [], isLoading: certLoading } = useQuery({
303 |     queryKey: ['trainee-dashboard-certs', profile?.id],
304 |     queryFn: async () => {
305 |       const { data, error } = await supabase
306 |         .from('certificates')
307 |         .select('id, issued_at, status')
308 |         .eq('user_id', profile!.id)
309 |         .eq('status', 'valid')
310 |       if (error) throw error
311 |       return data || []
312 |     },
313 |     enabled: !!profile?.id,
314 |   })
315 |
316 |   const { data: notifications = [], isLoading: notifLoading } = useQuery({
317 |     queryKey: ['trainee-dashboard-notifications', profile?.id],
318 |     queryFn: async () => {
319 |       const { data, error } = await supabase
320 |         .from('notifications')
321 |         .select('*')
322 |         .eq('user_id', profile!.id)
323 |         .order('created_at', { ascending: false })
324 |         .limit(5)
325 |       if (error) throw error
326 |       return data || []
327 |     },
328 |     enabled: !!profile?.id,
329 |   })
330 |
331 |   const enrolledCount = enrollments.length
332 |   const completedEnrollments = enrollments.filter(
333 |     (e: any) => e.status === 'completed' || (e.progress_percent ?? 0) === 100
334 |   )
335 |   const completionRate = enrolledCount > 0
336 |     ? Math.round((completedEnrollments.length / enrolledCount) * 100)
337 |     : 0
338 |   const totalMinutes = enrollments.reduce((sum: number, e: any) => {
339 |     const mins = e.course?.duration_minutes ?? 0
340 |     const pct = (e.progress_percent ?? 0) / 100
341 |     return sum + Math.round(mins * pct)
342 |   }, 0)
343 |   const hoursLearned = Math.round(totalMinutes / 60)
344 |   const earnedCertificatesCount = Math.max(
345 |     certificates.length,
346 |     completedEnrollments.length
347 |   )
348 |
349 |   const stats = [
350 |     {
351 |       label: 'Enrolled Courses',
352 |       value: enrollmentsLoading ? '…' : String(enrolledCount),
353 |       icon: BookOpen,
354 |       gradient: 'from-cyan-600 to-blue-700',
355 |       badgeText: 'Active',
356 |       subtext: enrolledCount === 0 ? 'No courses yet' : `${enrollments.filter((e: any) => e.status !== 'completed' && e.statu
s !== 'withdrawn' && (e.progress_percent ?? 0) < 100).length} active`,
357 |     },
358 |     {
359 |       label: 'Hours Learned',
360 |       value: enrollmentsLoading ? '…' : String(hoursLearned),
361 |       icon: Clock,
362 |       gradient: 'from-blue-600 to-indigo-700',
363 |       badgeText: 'Total',
364 |       subtext: hoursLearned === 0 ? 'Start learning!' : `Based on progress`,
365 |     },
366 |     {
367 |       label: 'Certificates',
```

```
      value: certLoading && enrollmentsLoading ? '…' : String(earnedCertificatesCount),
      icon: Award,
      gradient: 'from-amber-500 to-orange-600',
      badgeText: earnedCertificatesCount > 0 ? 'Verified' : 'Pending',
      subtext: earnedCertificatesCount === 0 ? 'None yet' : `${earnedCertificatesCount} earned • Ready to view`,
    },
    {
      label: 'Completion Rate',
      value: enrollmentsLoading ? '…' : `${completionRate}%`,
      icon: Target,
      gradient: 'from-emerald-500 to-teal-700',
      badgeText: completionRate >= 75 ? 'Top 25%' : completionRate >= 50 ? 'On Track' : 'Keep Going',
      subtext: enrolledCount === 0 ? 'No data yet' : `${completedEnrollments.length} of ${enrolledCount} done`,
    },
  ]

  const recentCourses = enrollments
    .filter((e: any) => e.status !== 'withdrawn')
    .slice(0, 3)
    .map((e: any) => ({
      id: e.course?.id,
      title: e.course?.title ?? 'Untitled Course',
      progress: e.progress_percent ?? 0,
      status: e.status === 'completed' || (e.progress_percent ?? 0) === 100 ? 'Completed' : 'In Progress',
      instructor: e.course?.trainer?.full_name ?? 'Instructor',
      duration: e.status === 'completed' || (e.progress_percent ?? 0) === 100
        ? 'Completed'
        : e.course?.duration_minutes
          ? `${Math.round(e.course.duration_minutes * (1 - (e.progress_percent ?? 0) / 100))} min left`
          : 'Self-paced',
    }))

  const activities = notifications.length > 0
    ? notifications.map((n: any) => ({
        icon: n.type === 'certificate_issued' ? Award
          : n.type === 'enrollment' ? BookOpen
          : n.type === 'assessment_result' ? CheckCircle
          : Star,
        title: n.title,
        desc: n.message,
        time: formatDistanceToNow(new Date(n.created_at), { addSuffix: true }),
        iconBg: n.type === 'certificate_issued' ? 'bg-amber-100 text-amber-700'
          : n.type === 'enrollment' ? 'bg-cyan-100 text-cyan-700'
          : n.type === 'assessment_result' ? 'bg-emerald-100 text-emerald-700'
          : 'bg-blue-100 text-blue-700',
      }))
    : enrollments.slice(0, 4).map((e: any) => ({
        icon: e.status === 'completed' || (e.progress_percent ?? 0) === 100 ? CheckCircle : BookOpen,
        title: e.status === 'completed' || (e.progress_percent ?? 0) === 100 ? 'Completed Course' : 'Enrolled in Course',
        desc: e.course?.title ?? 'Course',
        time: formatDistanceToNow(new Date(e.enrolled_at), { addSuffix: true }),
        iconBg: e.status === 'completed' || (e.progress_percent ?? 0) === 100 ? 'bg-emerald-100 text-emerald-700' : 'bg-cyan-
100 text-cyan-700',
      }))

  return (
    <DashboardShell
      title="Trainee Dashboard"
      icon={GraduationCap}
      navLinks={[
        { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
        { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
        { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
        { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
        { to: '/trainee/profile', label: 'Profile', icon: User },
      ]}
    >
      <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-6 max-w-full">
        {/* Welcome Hero Banner (Dark Midnight Aesthetic) */}
        <motion.div
          variants={fadeUp}
          className="relative p-6 sm:p-8 rounded-3xl bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white ov
erflow-hidden shadow-xl border border-cyan-500/30"
        >
          <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blu
r-3xl pointer-events-none" />
          <div className="absolute bottom-0 left-1/3 w-60 h-60 bg-sky-500/15 rounded-full blur-2xl pointer-events-none" />
```

```
          <div className="relative z-10 flex flex-col sm:flex-row sm:items-center sm:justify-between gap-5">
            <div className="max-w-xl">
              <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md text-amber-300 text-xs font-semibold mb-3 border border-white/15">
                <Star className="w-3.5 h-3.5 fill-amber-300" />
                <span>Keep up the momentum!</span>
              </div>
              <h2 className="text-2xl sm:text-3xl font-extrabold tracking-tight text-white mb-2">
                Welcome back, {profile?.full_name?.split(' ')[0] ?? 'Trainee'}! 👋
              </h2>
              <p className="text-slate-300 text-sm leading-relaxed">
                Continue your learning journey.
                {completionRate > 0
                  ? ` You've completed ${completionRate}% of your enrolled courses!`
                  : enrolledCount > 0
                    ? ` You have ${enrolledCount} active course${enrolledCount > 1 ? 's' : ''} in progress.`
                    : ' Browse the catalog to start your learning journey!'}
              </p>
            </div>
            <Link to="/trainee/courses" className="shrink-0">
              <Button className="w-full sm:w-auto bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600 hover:to-blue-700 text-white font-bold px-6 py-3 rounded-2xl shadow-lg shadow-cyan-500/30 hover:scale-105 active:scale-95 transition-all">
                <Compass className="w-4 h-4 mr-2" />
                Browse Catalog
              </Button>
            </Link>
          </div>
        </motion.div>

        {/* Stats Grid */}
        <motion.div variants={fadeUp} className="grid grid-cols-2 md:grid-cols-4 gap-4">
          {stats.map(s => <StatCard key={s.label} {...s} />)}
        </motion.div>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
          {/* Course Progress */}
          <motion.div variants={fadeUp} className="lg:col-span-2 bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden flex flex-col justify-between">
            <div>
              <div className="pb-4 border-b border-slate-100 flex items-center justify-between">
                <div>
                  <h3 className="text-base font-bold text-slate-900">In-Progress Courses</h3>
                  <p className="text-xs text-slate-500">Your active learning roadmap</p>
                </div>
                <Link to="/trainee/my-learning" className="text-xs font-bold text-cyan-600 hover:text-cyan-700 transition-colors flex items-center gap-1 bg-cyan-50 hover:bg-cyan-100 border border-cyan-200 px-3 py-1.5 rounded-full">
                  View All <ArrowUpRight className="w-3.5 h-3.5" />
                </Link>
              </div>

              <div className="py-4 space-y-3.5">
                {enrollmentsLoading ? (
                  <div className="flex items-center justify-center py-8">
                    <Loader2 className="w-6 h-6 text-cyan-600 animate-spin" />
                  </div>
                ) : recentCourses.length === 0 ? (
                  <div className="text-center py-8">
                    <BookOpen className="w-8 h-8 text-slate-300 mx-auto mb-2" />
                    <p className="text-sm text-slate-500">No courses yet. Browse the catalog to get started!</p>
                  </div>
                ) : (
                  recentCourses.map((course: any) => (
                    <div key={course.id ?? course.title} className="p-4 rounded-2xl bg-slate-50/70 hover:bg-slate-50 border border-slate-200 hover:border-cyan-300 transition-all duration-200 group">
                      <div className="flex items-center justify-between mb-2.5">
                        <div className="min-w-0 pr-3">
                          <p className="text-sm font-bold text-slate-900 truncate group-hover:text-cyan-600 transition-colors">{course.title}</p>
                          <p className="text-xs text-slate-500">{course.instructor} • {course.duration}</p>
                        </div>
                        <span className={`text-xs px-2.5 py-1 rounded-full font-bold shrink-0 ${
                          course.status === 'Completed'
                            ? 'bg-emerald-50 text-emerald-700 border border-emerald-200'
                            : 'bg-cyan-50 text-cyan-800 border border-cyan-200'
                        }`}>
                          {course.status}
                        </span>
                      </div>
```

```
                      <div className="flex items-center gap-3">
                        <ProgressBar
                          value={course.progress}
                          color={course.progress === 100 ? 'from-emerald-500 to-teal-600' : 'from-cyan-500 to-blue-600'}
                        />
                        <span className="text-xs font-bold text-slate-700 w-10 text-right">{course.progress}%</span>
                      </div>
                    </div>
                  ))
                )}
              </div>
            </div>

            <div className="pt-3 border-t border-slate-100 flex items-center justify-between text-xs text-slate-600">
              <span>{enrolledCount > 0 ? `${enrolledCount} course${enrolledCount > 1 ? 's' : ''} enrolled` : 'No enrollments yet'}</span>
              <span className="font-semibold text-cyan-700">
                {completionRate >= 75 ? 'Excellent 🚀' : completionRate >= 50 ? 'On Track 📈' : completionRate > 0 ? 'Keep Going 💪' : 'Get Started!'}
              </span>
            </div>
          </motion.div>

          {/* Recent Activity */}
          <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm flex flex-col">
            <div className="pb-4 border-b border-slate-100 flex items-center justify-between">
              <div>
                <h3 className="text-base font-bold text-slate-900">Recent Activity</h3>
                <p className="text-xs text-slate-500">Your recent updates & badges</p>
              </div>
              <button className="p-1.5 rounded-xl hover:bg-slate-100 text-slate-500 hover:text-slate-900 transition-colors">
                <RefreshCw className="w-4 h-4" />
              </button>
            </div>
            <div className="py-2 space-y-1 flex-1">
              {notifLoading || enrollmentsLoading ? (
                <div className="flex items-center justify-center py-8">
                  <Loader2 className="w-5 h-5 text-cyan-600 animate-spin" />
                </div>
              ) : activities.length === 0 ? (
                <div className="text-center py-8">
                  <Star className="w-7 h-7 text-slate-300 mx-auto mb-2" />
                  <p className="text-xs text-slate-400">No activity yet. Enroll in a course to get started!</p>
                </div>
              ) : (
                activities.map((act: any, i: number) => (
                  <ActivityItem key={i} {...act} />
                ))
              )}
            </div>
          </motion.div>
        </div>

        {/* Trainee Digital Twin */}
        <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden relative">
          <div className="relative z-10 flex flex-col lg:flex-row gap-8 items-center">
            <div className="lg:w-1/2 space-y-5">
              <div>
                <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-50 text-cyan-700 border border-cyan-200 text-xs font-bold mb-2">
                  <Target className="w-3.5 h-3.5" />
                  <span>AI Skill Analysis</span>
                </div>
                <h3 className="text-xl font-bold text-slate-900 leading-tight">Your Skill Profile</h3>
                <p className="text-sm text-slate-600 leading-relaxed mt-2">
                  See how your current skills compare to your target goals.
                  {topGaps.length > 0 && (
                    <span> We recommend focusing on <strong className="text-cyan-700">{topGaps.map(g => g.subject).join(' and ')}</strong> to improve.</span>
                  )}
                </p>
              </div>

              <div className="flex flex-wrap gap-4 text-xs font-medium text-slate-700">
                <div className="flex items-center gap-2">
                  <div className="w-3 h-3 rounded-full bg-cyan-600 shadow-sm" /> Your Current Level
                </div>
```

```
                <div className="flex items-center gap-2">
                  <div className="w-3 h-3 rounded-full bg-amber-500 shadow-sm" /> Target Level
                </div>
              </div>

              {/* Actionable Insights */}
              {topGaps.length > 0 && (
                <div className="mt-4 space-y-3">
                  <h4 className="text-xs font-bold text-slate-900 uppercase tracking-wider">Top Areas for Improvement</h4>
                  {topGaps.map((gap, i) => (
                    <div key={i} className="flex items-center justify-between p-3 rounded-2xl bg-slate-50 border border-slate
-100">

                      <div>
                        <p className="text-sm font-bold text-slate-900">{gap.subject}</p>
                        <p className="text-xs text-slate-500">Current: {gap.A}% <span className="mx-1 text-slate-300">•</span
> Target: {gap.B}%</p>
                      </div>
                      <Link to={`/trainee/courses?search=${encodeURIComponent(gap.subject)}`}>
                        <Button variant="outline" size="sm" className="h-8 text-xs bg-white border-slate-200 text-cyan-700 ho
ver:bg-cyan-50 hover:border-cyan-200">
                          Find Course <ChevronRight className="w-3 h-3 ml-1" />
                        </Button>
                      </Link>
                    </div>
                  ))}
                </div>
              )}
            </div>
            <div className="lg:w-1/2 h-80 w-full relative">
              {(skillsLoading || allSkillsLoading) && (
                <div className="absolute inset-0 flex items-center justify-center bg-white/50 z-20 backdrop-blur-sm rounded-3
xl">

                  <Loader2 className="w-8 h-8 text-cyan-600 animate-spin" />
                </div>
              )}
              {competencyData.length >= 3 ? (
                <ResponsiveContainer width="100%" height="100%">
                  <RadarChart cx="50%" cy="50%" outerRadius="70%" data={competencyData}>
                    <PolarGrid stroke="#f1f5f9" />
                    <PolarAngleAxis dataKey="subject" tick={{ fill: '#64748b', fontSize: 12 }} />
                    <Radar name="Current Level" dataKey="A" stroke="#0ea5e9" strokeWidth={2} fill="#0ea5e9" fillOpacity={0.2}
/>

                    <Radar name="Target Level" dataKey="B" stroke="#cbd5e1" strokeWidth={2} fill="none" />
                    <Tooltip
                      wrapperStyle={{ outline: 'none' }}
                      contentStyle={{ borderRadius: '8px', border: 'none', boxShadow: '0 4px 6px -1px rgba(0,0,0,0.1)' }}
                    />
                  </RadarChart>
                </ResponsiveContainer>
              ) : (
                <div className="w-full h-full flex flex-col items-center justify-center text-slate-400">
                  <Target className="w-12 h-12 mb-3 text-slate-200" />
                  <p className="text-sm font-medium">Add more skills to view chart</p>
                  <p className="text-xs text-slate-400 max-w-[200px] text-center mt-1">Add at least 2 skill categories to dis
play the chart.</p>
                </div>
              )}
            </div>
          </div>
        </motion.div>

        {/* Quick Actions */}
        <motion.div variants={fadeUp} className="grid grid-cols-1 md:grid-cols-3 gap-4">
          {[
            { icon: Compass, title: 'Explore Catalog', desc: 'Discover new specialized programs', to: '/trainee/courses', gra
dient: 'from-cyan-600 to-blue-700' },
            { icon: BookOpen, title: 'My Learning Space', desc: 'Resume current lessons & quizzes', to: '/trainee/my-learnin
g', gradient: 'from-blue-600 to-indigo-700' },
            { icon: Award, title: 'Earned Certificates', desc: 'View, download, and share badges', to: '/trainee/my-learnin
g', gradient: 'from-amber-500 to-orange-600' },
          ].map(action => (
            <Link key={action.title} to={action.to}>
              <div className="p-5 rounded-3xl bg-white border border-slate-200/90 hover:border-cyan-400 hover:shadow-xl hove
r:shadow-slate-200/60 hover:-translate-y-1 transition-all duration-300 group cursor-pointer flex items-center justify-between">
                <div className="flex items-center gap-4">
                  <div className={`w-12 h-12 rounded-2xl bg-gradient-to-br ${action.gradient} text-white flex items-center ju
stify-center shadow-md group-hover:scale-105 transition-transform`}>
                    <action.icon className="w-5 h-5" />
                  </div>
```

```
                  <div>
                    <h4 className="text-sm font-bold text-slate-900 group-hover:text-cyan-600 transition-colors">{action.title}</h4>
                    <p className="text-xs text-slate-500">{action.desc}</p>
                  </div>
                </div>
                <ChevronRight className="w-5 h-5 text-slate-300 group-hover:text-cyan-600 group-hover:translate-x-1 transition-all" />
              </div>
            </Link>
          ))}
        </motion.div>
      </motion.div>
    </DashboardShell>
  )
}

/* ─── Trainer Dashboard ──────────────────────────────────── */

export function TrainerDashboard() {
  const { profile } = useAuth()

  const stats = [
    { label: 'Active Courses', value: '8', icon: BookOpen, gradient: 'from-cyan-600 to-blue-700', badgeText: 'Published' },
    { label: 'Total Trainees', value: '156', icon: Users, gradient: 'from-blue-600 to-indigo-700', badgeText: 'Enrolled', subtext: '+14 this week' },
    { label: 'Avg. Completion', value: '82%', icon: BarChart3, gradient: 'from-amber-500 to-orange-600', badgeText: 'High Rate' },
    { label: 'Avg. Rating', value: '4.7', icon: Star, gradient: 'from-emerald-500 to-teal-700', badgeText: '⭐ 4.7 / 5.0' },
  ]

  const recentSubmissions = [
    { name: 'Amit Kumar', course: 'Data Science Fundamentals', module: 'Module 6 — ML Basics', time: '1h ago' },
    { name: 'Priya Singh', course: 'Python for Analytics', module: 'Module 3 — DataFrames', time: '3h ago' },
    { name: 'Rahul Verma', course: 'Cloud Computing', module: 'Final Assessment', time: '5h ago' },
    { name: 'Sneha Patel', course: 'Data Science Fundamentals', module: 'Module 5 — Statistics', time: '1d ago' },
  ]

  const topCourses = [
    { title: 'Data Science Fundamentals', trainees: 89, completion: 76, rating: 4.8 },
    { title: 'Python for Analytics', trainees: 67, completion: 64, rating: 4.6 },
    { title: 'Cloud Computing Basics', trainees: 45, completion: 82, rating: 4.7 },
  ]

  const activities = [
    { icon: FileText, title: 'New Submission', desc: 'Amit Kumar — Module 6 Quiz (94%)', time: '1h ago', iconBg: 'bg-emerald-100 text-emerald-700' },
    { icon: Users, title: '5 New Enrollments', desc: 'Data Science Fundamentals', time: '3h ago', iconBg: 'bg-cyan-100 text-cyan-700' },
    { icon: MessageSquare, title: 'New Question', desc: 'Python for Analytics — Forum Post', time: '6h ago', iconBg: 'bg-amber-100 text-amber-700' },
    { icon: Star, title: '5-Star Review', desc: 'Cloud Computing Basics — Rating', time: '1d ago', iconBg: 'bg-blue-100 text-blue-700' },
  ]

  return (
    <DashboardShell
      title="Trainer Dashboard"
      icon={BookOpen}
      navLinks={[
        { to: '/trainer', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainer/courses', label: 'My Courses', icon: BookOpen },
      ]}
    >
      <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-6 max-w-full">
        {/* Welcome Banner (Midnight Dark Gradient) */}
        <motion.div
          variants={fadeUp}
          className="relative p-6 sm:p-8 rounded-3xl bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white overflow-hidden shadow-xl border border-cyan-500/30"
        >
          <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />
          <div className="relative z-10 flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
            <div>
              <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md text-amber-300 text-xs font-semibold mb-3 border border-white/15">
                <span>Trainer Command Center</span>
              </div>
```

```
              <h2 className="text-2xl sm:text-3xl font-extrabold tracking-tight text-white mb-2">
                Welcome, {profile?.full_name?.split(' ')[0] ?? 'Trainer'}!
              </h2>
              <p className="text-slate-300 text-sm">Manage your course curriculum and track real-time trainee engagement.</p>
            </div>
            <Link to="/trainer/courses" className="shrink-0">
              <Button className="w-full sm:w-auto bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600
hover:to-blue-700 text-white font-bold px-6 py-3 rounded-2xl shadow-lg shadow-cyan-500/30 hover:scale-105 transition-all">
                <BookOpen className="w-4 h-4 mr-2" />
                Manage Courses
              </Button>
            </Link>
          </div>
        </motion.div>

        {/* Assignment Announcement Banner */}
        <TrainerAssignmentBanner />

        {/* Stats Grid */}
        <motion.div variants={fadeUp} className="grid grid-cols-2 md:grid-cols-4 gap-4">
          {stats.map(s => <StatCard key={s.label} {...s} />)}
        </motion.div>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
          {/* Top Courses */}
          <motion.div variants={fadeUp} className="lg:col-span-2 bg-white border border-slate-200/90 rounded-3xl p-6 shadow-s
m overflow-hidden">
            <div className="pb-4 border-b border-slate-100 flex items-center justify-between">
              <div>
                <h3 className="text-base font-bold text-slate-900">Top Performing Courses</h3>
                <p className="text-xs text-slate-500">Engagement and completion breakdown</p>
              </div>
              <Link to="/trainer/courses" className="text-xs font-bold text-cyan-600 hover:text-cyan-700 transition-colors fl
ex items-center gap-1 bg-cyan-50 hover:bg-cyan-100 border border-cyan-200 px-3 py-1.5 rounded-full">
                View All <ArrowUpRight className="w-3.5 h-3.5" />
              </Link>
            </div>
            <div className="py-4 space-y-3.5">
              {topCourses.map(course => (
                <div key={course.title} className="p-4 rounded-2xl bg-slate-50/70 hover:bg-slate-50 border border-slate-200 t
ransition-all">
                  <div className="flex items-center justify-between mb-3">
                    <p className="text-sm font-bold text-slate-900">{course.title}</p>
                    <div className="flex items-center gap-1 text-amber-700 font-bold text-xs bg-amber-50 px-2 py-0.5 rounded-
full border border-amber-200">
                      <Star className="w-3.5 h-3.5 fill-amber-500 text-amber-500" />
                      <span>{course.rating}</span>
                    </div>
                  </div>
                  <div className="grid grid-cols-3 gap-4 items-center">
                    <div>
                      <p className="text-base font-extrabold text-slate-900">{course.trainees}</p>
                      <p className="text-[11px] font-semibold text-slate-500">Trainees</p>
                    </div>
                    <div>
                      <p className="text-base font-extrabold text-slate-900">{course.completion}%</p>
                      <p className="text-[11px] font-semibold text-slate-500">Completion</p>
                    </div>
                    <div>
                      <ProgressBar value={course.completion} />
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </motion.div>

          {/* Recent Activity */}
          <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm flex flex-co
l">
            <div className="pb-4 border-b border-slate-100">
              <h3 className="text-base font-bold text-slate-900">Recent Activity</h3>
              <p className="text-xs text-slate-500">Live trainee actions</p>
            </div>
            <div className="py-2 space-y-1 flex-1">
              {activities.map((act, i) => (
                <ActivityItem key={i} {...act} />
              ))}
            </div>
```

```
797 |           </motion.div>
798 |         </div>
799 |
800 |         {/* Recent Submissions */}
801 |         <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidde
n">
802 |           <div className="pb-4 border-b border-slate-100 flex items-center justify-between">
803 |             <div>
804 |               <h3 className="text-base font-bold text-slate-900">Recent Trainee Submissions</h3>
805 |               <p className="text-xs text-slate-500">Assessments and assignments pending review</p>
806 |             </div>
807 |             <button className="text-xs font-bold text-cyan-600 hover:text-cyan-700 transition-colors flex items-center gap-1
bg-cyan-50 hover:bg-cyan-100 border border-cyan-200 px-3 py-1.5 rounded-full">
808 |               View All <ArrowUpRight className="w-3.5 h-3.5" />
809 |             </button>
810 |           </div>
811 |           <div className="overflow-x-auto mt-2">
812 |             <table className="w-full min-w-[500px]">
813 |               <thead>
814 |                 <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slate-50">
815 |                   <th className="px-4 py-3">Trainee</th>
816 |                   <th className="px-4 py-3">Course</th>
817 |                   <th className="px-4 py-3">Module</th>
818 |                   <th className="px-4 py-3 text-right">Time</th>
819 |                 </tr>
820 |               </thead>
821 |               <tbody className="divide-y divide-slate-100">
822 |                 {recentSubmissions.map((s, i) => (
823 |                   <tr key={i} className="hover:bg-slate-50/80 transition-colors">
824 |                     <td className="px-4 py-3">
825 |                       <div className="flex items-center gap-2.5">
826 |                         <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center just
ify-center text-white text-xs font-bold shrink-0">
827 |                           {s.name.charAt(0)}
828 |                         </div>
829 |                         <span className="text-xs sm:text-sm font-bold text-slate-900 truncate">{s.name}</span>
830 |                       </div>
831 |                     </td>
832 |                     <td className="px-4 py-3 text-xs sm:text-sm text-slate-600 truncate">{s.course}</td>
833 |                     <td className="px-4 py-3 text-xs sm:text-sm text-slate-600">{s.module}</td>
834 |                     <td className="px-4 py-3 text-xs text-slate-400 text-right">{s.time}</td>
835 |                   </tr>
836 |                 ))}
837 |               </tbody>
838 |             </table>
839 |           </div>
840 |         </motion.div>
841 |
842 |         {/* Announcements Feed Section */}
843 |         <motion.div variants={fadeUp}>
844 |           <AnnouncementsFeed />
845 |         </motion.div>
846 |       </motion.div>
847 |     </DashboardShell>
848 |   )
849 | }
850 |
851 | /* ─── Admin Dashboard ──────────────────────────────────────── */

852 | export function AdminDashboard() {
853 |   const navigate = useNavigate()
854 |   const location = useLocation()
855 |   const { profile } = useAuth()
856 |   const [users, setUsers] = useState<Profile[]>([])
857 |   const [logs, setLogs] = useState<AuditLog[]>([])
858 |   const [loading, setLoading] = useState(true)
859 |   const getInitialTab = () => {
860 |     const queryTab = new URLSearchParams(location.search).get('tab')
861 |     if (queryTab) return queryTab as any
862 |     if ((location.state as any)?.tab) return (location.state as any).tab
863 |     if (location.pathname === '/admin/courses') return 'courses'
864 |     if (location.pathname === '/admin/assessments') return 'assessments'
865 |     if (location.pathname === '/admin/announcements') return 'announcements'
866 |     if (location.pathname === '/admin/notifications') return 'notifications'
867 |     if (location.pathname === '/admin/messages') return 'messages'
868 |     return 'overview'
869 |   }
870 |
871 |   const [activeTab, setActiveTab] = useState<'overview' | 'trainees' | 'trainers' | 'admins' | 'courses' | 'assessments' | 'l
```

```
ogs' | 'announcements' | 'notifications' | 'messages' | 'home_page'>(
    getInitialTab()
  )

  useEffect(() => {
    const queryTab = new URLSearchParams(location.search).get('tab')
    if (queryTab) {
      setActiveTab(queryTab as any)
    } else if ((location.state as any)?.tab) {
      setActiveTab((location.state as any).tab)
    } else if (location.pathname === '/admin/courses') {
      setActiveTab('courses')
    } else if (location.pathname === '/admin/assessments') {
      setActiveTab('assessments')
    } else if (location.pathname === '/admin/announcements') {
      setActiveTab('announcements')
    } else if (location.pathname === '/admin/notifications') {
      setActiveTab('notifications')
    } else if (location.pathname === '/admin/messages') {
      setActiveTab('messages')
    } else if (location.pathname === '/admin') {
      setActiveTab('overview')
    }
  }, [location.pathname, location.search, location.state])
  const [searchQuery, setSearchQuery] = useState('')
  const [statusFilter, setStatusFilter] = useState<'all' | 'pending' | 'approved' | 'suspended' | 'rejected'>('all')
  const [ConfirmDialog, confirm] = useConfirm()

  const [traineeSubTab, setTraineeSubTab] = useState<'enrollment_requests' | 'accounts'>('enrollment_requests')
  const [detailedEnrollments, setDetailedEnrollments] = useState<any[]>([])
  const [isProcessingEnrollmentId, setIsProcessingEnrollmentId] = useState<string | null>(null)
  const [enrollmentSearchQuery, setEnrollmentSearchQuery] = useState('')
  const [enrollmentStatusFilter, setEnrollmentStatusFilter] = useState<'all' | 'pending_approval' | 'enrolled' | 'rejected' |
'waitlisted'>('pending_approval')

  const [previewMaterial, setPreviewMaterial] = useState<{file_name: string, storage_path: string} | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [pendingCoursesCount, setPendingCoursesCount] = useState(0)
  const [coursesList, setCoursesList] = useState<any[]>([])
  const [enrollmentsList, setEnrollmentsList] = useState<any[]>([])
  const [selectedUserForModal, setSelectedUserForModal] = useState<any | null>(null)
  const [isUserDetailsModalOpen, setIsUserDetailsModalOpen] = useState(false)

  const fetchData = async () => {
    setLoading(true)
    try {
      const [trRes, trnRes, admRes, logRes, pendingCoursesRes, allCoursesRes, enrollmentsRes, detailedEnrollmentsRes] = await
Promise.all([
        supabase.from('trainees').select('*').order('created_at', { ascending: false }),
        supabase.from('trainers').select('*').order('created_at', { ascending: false }),
        supabase.from('admins').select('*').order('created_at', { ascending: false }),
        supabase.from('audit_logs').select('*').order('created_at', { ascending: false }).limit(50),
        supabase.from('courses').select('id', { count: 'exact' }).eq('status', 'pending_review'),
        supabase.from('courses').select('id, title, status, delivery_mode, department, course_type, created_at'),
        supabase.from('enrollments').select('id, course_id, user_id, status, progress_percent, completed_at, enrolled_at'),
        supabase.from('enrollments').select(`
          id, course_id, user_id, status, progress_percent, completed_at, enrolled_at,
          course:courses(id, title, course_type, department, trainer:trainers(full_name)),
          trainee:trainees(id, full_name, email, department, avatar_path, phone)
        `).order('enrolled_at', { ascending: false }),
      ])

      const allUsers = [
        ...(trRes.data || []),
        ...(trnRes.data || []),
        ...(admRes.data || [])
      ]
      setUsers(allUsers as any)
      if (logRes.data) setLogs(logRes.data)
      setPendingCoursesCount(pendingCoursesRes.count || 0)
      if (allCoursesRes.data) setCoursesList(allCoursesRes.data)
      if (enrollmentsRes.data) setEnrollmentsList(enrollmentsRes.data)
      if (detailedEnrollmentsRes.data) setDetailedEnrollments(detailedEnrollmentsRes.data)
    } catch (e) {
      console.error('Exception in fetchData:', e)
    } finally {
      setLoading(false)
    }
  }
```

```
948 |
949 |   useEffect(() => { fetchData() }, [])
950 |
951 |   const platformAnalytics = useMemo(() => {
952 |     const totalEnrollments = enrollmentsList.length
953 |     const completedEnrollments = enrollmentsList.filter(e => e.status === 'completed' || (e.progress_percent ?? 0) >= 100).le
ngth
954 |     const activeEnrollments = enrollmentsList.filter(e => e.status === 'in_progress' || e.status === 'enrolled' || ((e.progre
ss_percent ?? 0) > 0 && (e.progress_percent ?? 0) < 100)).length
955 |     const completionRate = totalEnrollments > 0 ? Math.round((completedEnrollments / totalEnrollments) * 100) : 0
956 |
957 |     const totalProgress = enrollmentsList.reduce((acc, curr) => acc + (Number(curr.progress_percent) || 0), 0)
958 |     const avgProgress = totalEnrollments > 0 ? Math.round(totalProgress / totalEnrollments) : 0
959 |
960 |     // Department grouping - normalized to aggregate clean institutional units

961 |     const deptMap: Record<string, number> = {}
962 |     users.forEach(u => {
963 |       let d = (u.department || 'MoES HQ').trim()
964 |       if (d) {
965 |         const upper = d.toUpperCase()
966 |         if (upper === 'CSE' || upper === 'IMD' || upper === 'NCMRWF' || upper === 'INCOIS' || upper === 'IITM' || upper ===
'NIOT' || upper === 'NCPOR') {
967 |           d = upper
968 |         }
969 |       }
970 |       deptMap[d] = (deptMap[d] || 0) + 1
971 |     })
972 |     const deptArray = Object.entries(deptMap)
973 |       .map(([name, count]) => ({
974 |         name,
975 |         count,
976 |         percent: users.length > 0 ? Math.round((count / users.length) * 100) : 0,
977 |       }))
978 |       .sort((a, b) => b.count - a.count)
979 |
980 |     const publishedCourses = coursesList.filter(c => c.status === 'published').length
981 |     const draftCourses = coursesList.filter(c => c.status === 'draft').length
982 |     const pendingCourses = coursesList.filter(c => c.status === 'pending_review').length
983 |
984 |     const recordedCount = coursesList.filter(c => c.delivery_mode === 'recorded' || !c.delivery_mode).length
985 |     const liveCount = coursesList.filter(c => c.delivery_mode === 'live').length
986 |     const hybridCount = coursesList.filter(c => c.delivery_mode === 'hybrid').length
987 |
988 |     return {
989 |       totalEnrollments,
990 |       completedEnrollments,
991 |       activeEnrollments,
992 |       completionRate,
993 |       avgProgress,
994 |       deptArray: deptArray.slice(0, 6),
995 |       totalCourses: coursesList.length,
996 |       publishedCourses,
997 |       draftCourses,
998 |       pendingCourses,
999 |       deliveryModes: {
1000 |         recorded: recordedCount,
1001 |         live: liveCount,
1002 |         hybrid: hybridCount,
1003 |       }
1004 |     }
1005 |   }, [enrollmentsList, coursesList, users])
1006 |
1007 |   const filteredUsers = useMemo(() => {
1008 |     let filtered = users
1009 |     if (activeTab === 'trainees') filtered = users.filter(u => u.role === 'trainee')
1010 |     else if (activeTab === 'trainers') filtered = users.filter(u => u.role === 'trainer')
1011 |     else if (activeTab === 'admins') filtered = users.filter(u => u.role === 'admin' || u.role === 'super_admin')
1012 |
1013 |     if (statusFilter !== 'all') {
1014 |       filtered = filtered.filter(u => u.approval_status === statusFilter)
1015 |     }
1016 |
1017 |     if (!searchQuery) return filtered
1018 |     const q = searchQuery.toLowerCase()
1019 |     return filtered.filter(u =>
1020 |       u.full_name?.toLowerCase().includes(q) ||
1021 |       u.email?.toLowerCase().includes(q) ||
1022 |       u.department?.toLowerCase().includes(q) ||
```

```
      (u as any).qualifications?.toLowerCase().includes(q)
    )
  }, [users, searchQuery, activeTab, statusFilter])

  const filteredEnrollments = useMemo(() => {
    let list = detailedEnrollments
    if (enrollmentStatusFilter !== 'all') {
      list = list.filter(e => e.status === enrollmentStatusFilter)
    }
    if (!enrollmentSearchQuery.trim()) return list
    const q = enrollmentSearchQuery.toLowerCase().trim()
    return list.filter(e =>
      e.trainee?.full_name?.toLowerCase().includes(q) ||
      e.trainee?.email?.toLowerCase().includes(q) ||
      e.trainee?.department?.toLowerCase().includes(q) ||
      e.course?.title?.toLowerCase().includes(q) ||
      e.course?.trainer?.full_name?.toLowerCase().includes(q)
    )
  }, [detailedEnrollments, enrollmentStatusFilter, enrollmentSearchQuery])

  const handleUpdateUser = async (userId: string, email: string | null, role: Profile['role'], status: Profile['approval_stat
us']) => {
    try {
      const { error } = await supabase.rpc('admin_update_user', {
        target_user_id: userId,
        new_role: role,
        new_status: status,
      })
      if (error) throw error

      if (status === 'approved' && email) {
        const { error: resetError } = await supabase.auth.resetPasswordForEmail(email, {
          redirectTo: `${window.location.origin}/setup-password`,
        })
        if (resetError) {
          console.error("Failed to send setup email:", resetError)
          toast.error("User approved, but failed to send setup email.")
        } else {
          toast.success('User approved and setup email sent!')
        }
      } else {
        toast.success('User updated successfully')
      }
      fetchData()
    } catch (e: any) {
      toast.error(`Failed to update user: ${e?.message || JSON.stringify(e)}`)
      console.error('Update user error:', e)
    }
  }

  const handleDeleteUser = async (userId: string) => {
    const isConfirmed = await confirm("Are you sure you want to permanently delete this user? This action cannot be undone.",
"Delete User")
    if (!isConfirmed) return;
    try {
      const { error } = await supabase.rpc('admin_delete_user', {
        target_user_id: userId,
      })
      if (error) throw error

      toast.success('User account completely deleted.')
      fetchData()
    } catch (e) {
      toast.error('Failed to delete user. Check Supabase connection.')
      console.error(e)
    }
  }

  const handleViewProof = async (proofPath: string) => {
    try {
      setPreviewMaterial({ file_name: 'ID Proof Document', storage_path: proofPath })
      setPreviewUrl(null)
      const { data, error } = await supabase.storage.from('proofs').createSignedUrl(proofPath, 60)
      if (error) throw error
      if (data?.signedUrl) {
        setPreviewUrl(data.signedUrl)
      }
    } catch (e) {
      console.error('Error viewing proof:', e)
```

```
      toast.error('Failed to open proof document')
      setPreviewMaterial(null)
    }
  }

  const handlePromoteToAdmin = async (userId: string) => {
    try {
      const { error } = await supabase.rpc('super_admin_promote_to_admin', {
        target_user_id: userId,
      })
      if (error) throw error
      toast.success('User promoted to Admin successfully')
      fetchData()
    } catch (e: any) {
      const msg = e?.message?.includes('super_admin')
        ? 'Only Super Admins can promote users to Admin.'
        : 'Failed to promote user.'
      toast.error(msg)
      console.error(e)
    }
  }

  const handleEnrollmentDecision = async (enrollmentId: string, action: 'approve' | 'reject', enrollment: any) => {
    setIsProcessingEnrollmentId(enrollmentId)
    try {
      const newStatus = action === 'approve' ? 'enrolled' : 'rejected'
      const { error } = await supabase
        .from('enrollments')
        .update({ status: newStatus } as any)
        .eq('id', enrollmentId)
      if (error) throw error

      if (enrollment.trainee?.id && enrollment.course?.title) {
        await supabase.from('notifications').insert({
          user_id: enrollment.trainee.id,
          title: action === 'approve' ? 'Course Enrollment Approved! 🎉' : 'Course Enrollment Update',
          message: action === 'approve'
            ? `Your enrollment for "${enrollment.course.title}" has been approved. You can now access all learning modules an
d assessments.`
            : `Your enrollment request for "${enrollment.course.title}" was not approved by administration.`,
          type: 'enrollment'
        } as any)
      }

      toast.success(`Course enrollment ${action === 'approve' ? 'approved' : 'rejected'} successfully!`)
      fetchData()
    } catch (err: any) {
      toast.error(err.message || 'Failed to update enrollment')
    } finally {
      setIsProcessingEnrollmentId(null)
    }
  }

  const [isCleaningFiles, setIsCleaningFiles] = useState(false)
  const cleanupOrphanedFiles = async () => {
    setIsCleaningFiles(true)
    try {
      const { data: courses } = await supabase.from('courses').select('id')
      const { data: trainees } = await supabase.from('trainees').select('id')
      const { data: trainers } = await supabase.from('trainers').select('id')
      const { data: admins } = await supabase.from('admins').select('id')

      const courseIds = new Set(courses?.map(c => c.id) || [])
      const userIds = new Set([
        ...(trainees?.map(u => u.id) || []),
        ...(trainers?.map(u => u.id) || []),
        ...(admins?.map(u => u.id) || [])
      ])

      // Clean materials bucket (folders match course id)

      const { data: materialFolders } = await supabase.storage.from('materials').list()
      if (materialFolders) {
        for (const folder of materialFolders) {
          if (folder.name && folder.name !== '.emptyFolderPlaceholder' && !courseIds.has(folder.name)) {
            const { data: files } = await supabase.storage.from('materials').list(folder.name)
            if (files && files.length > 0) {
              await supabase.storage.from('materials').remove(files.map(f => `${folder.name}/${f.name}`))
            }
```

```
          }
        }
      }

      // Clean proofs bucket (folders match user id)

      const { data: proofFolders } = await supabase.storage.from('proofs').list()
      if (proofFolders) {
        for (const folder of proofFolders) {
          if (folder.name && folder.name !== '.emptyFolderPlaceholder' && !userIds.has(folder.name)) {
            const { data: files } = await supabase.storage.from('proofs').list(folder.name)
            if (files && files.length > 0) {
              await supabase.storage.from('proofs').remove(files.map(f => `${folder.name}/${f.name}`))
            }
          }
        }
      }

      toast.success('Successfully cleaned up orphaned files!')
    } catch (e: any) {
      console.error(e)
      toast.error('Failed to clean up files.')
    } finally {
      setIsCleaningFiles(false)
    }
  }

  const isSuperAdmin = profile?.role === 'super_admin'

  const tabs = [
    { key: 'overview', label: 'Dashboard', icon: LayoutDashboard },
    { key: 'trainees', label: 'Trainees', icon: Users },
    { key: 'trainers', label: 'Trainers', icon: Users },
    ...(isSuperAdmin ? [{ key: 'admins', label: 'Admins', icon: Shield }] : []),
    { key: 'courses', label: 'Courses', icon: BookOpen },
    { key: 'assessments', label: 'Assessments', icon: FileCheck },
    { key: 'announcements', label: 'Announcements', icon: Megaphone },
    { key: 'notifications', label: 'Notifications', icon: Bell },
    { key: 'messages', label: 'Messages', icon: Mail },
    { key: 'home_page', label: 'Home Page', icon: Globe },
    ...(isSuperAdmin ? [{ key: 'logs', label: 'Audit Logs', icon: BarChart3 }] : []),
    { key: 'profile', label: 'Profile', icon: User, to: isSuperAdmin ? '/super-admin/profile' : '/admin/profile' },
  ] as const

  const pendingEnrollmentsCount = detailedEnrollments.filter(e => e.status === 'pending_approval').length
  const pendingTraineeAccountsCount = users.filter(u => u.approval_status === 'pending' && u.role === 'trainee').length
  const totalPendingTrainees = pendingEnrollmentsCount + pendingTraineeAccountsCount

  const pendingCount = users.filter(u => u.approval_status === 'pending').length + pendingEnrollmentsCount
  const pendingTrainees = totalPendingTrainees
  const pendingTrainers = users.filter(u => u.approval_status === 'pending' && u.role === 'trainer').length
  const pendingAdmins = isSuperAdmin ? users.filter(u => u.approval_status === 'pending' && u.role === 'admin').length : 0

  const stats = [
    { label: 'Total Users', value: users.length, icon: Users, gradient: 'from-cyan-600 to-blue-700', badgeText: 'Registered'
},
    { label: 'Pending Approvals', value: pendingCount, icon: Clock, gradient: 'from-amber-500 to-orange-600', badgeText: pend
ingCount > 0 ? `${pendingCount} Action Needed` : 'All Clear' },
    { label: 'Pending Course Requests', value: pendingEnrollmentsCount, icon: GraduationCap, gradient: 'from-purple-600 to-in
digo-700', badgeText: pendingEnrollmentsCount > 0 ? 'Course Requests' : 'None' },
  ]

  const activities = useMemo(() => {
    const list: Array<{
      id: string
      icon: React.ElementType
      title: string
      desc: string
      time: string
      iconBg: string
      timestamp: number
    }> = []

    // 1. From real audit_logs

    if (logs && logs.length > 0) {
      logs.forEach((log) => {
        let icon = Shield
        let iconBg = 'bg-slate-100 text-slate-700'
```

```
1251 |         let title = log.action || 'System Event'
1252 |         let desc = log.entity_type ? `${log.entity_type} event` : 'Audit log recorded'
1253 |
1254 |         const actionLower = (log.action || '').toLowerCase()
1255 |         if (actionLower.includes('create') || actionLower.includes('register') || actionLower.includes('insert')) {
1256 |           icon = Users
1257 |           iconBg = 'bg-cyan-100 text-cyan-700'
1258 |           title = 'Registration / Creation'
1259 |         } else if (actionLower.includes('approve') || actionLower.includes('verify') || actionLower.includes('complete')) {
1260 |           icon = CheckCircle
1261 |           iconBg = 'bg-emerald-100 text-emerald-700'
1262 |           title = 'Account Approved'
1263 |         } else if (actionLower.includes('suspend') || actionLower.includes('reject') || actionLower.includes('delete') || act
ionLower.includes('ban')) {
1264 |           icon = Ban
1265 |           iconBg = 'bg-rose-100 text-rose-700'
1266 |           title = 'Security & Status'
1267 |         } else if (actionLower.includes('course') || actionLower.includes('publish')) {
1268 |           icon = BookOpen
1269 |           iconBg = 'bg-blue-100 text-blue-700'
1270 |           title = 'Course Updated'
1271 |         }
1272 |
1273 |         if (log.metadata && typeof log.metadata === 'object') {
1274 |           const meta = log.metadata as any
1275 |           if (meta.email) desc = `${meta.email} — ${log.action}`
1276 |           else if (meta.title) desc = `"${meta.title}" — ${log.action}`
1277 |           else if (meta.target_user_id) desc = `User ${meta.target_user_id.slice(0, 8)}... — ${log.action}`
1278 |         }
1279 |
1280 |         const createdAt = log.created_at ? new Date(log.created_at) : new Date()
1281 |         list.push({
1282 |           id: `log-${log.id}`,
1283 |           icon,
1284 |           title,
1285 |           desc,
1286 |           time: formatDistanceToNow(createdAt, { addSuffix: true }),
1287 |           iconBg,
1288 |           timestamp: createdAt.getTime(),
1289 |         })
1290 |       })
1291 |     }
1292 |
1293 |     // 2. From real users (new registrations / status changes)

1294 |     users.forEach((u) => {
1295 |       const createdAt = u.created_at ? new Date(u.created_at) : null
1296 |       if (createdAt) {
1297 |         let icon = Users
1298 |         let iconBg = 'bg-cyan-100 text-cyan-700'
1299 |         let title = `${u.role === 'trainer' ? 'Trainer' : u.role === 'admin' || u.role === 'super_admin' ? 'Admin' : 'Traine
e'} Registered`
1300 |         let desc = `${u.full_name || u.email} ${u.department ? `(${u.department})` : ''}`
1301 |
1302 |         if (u.approval_status === 'approved') {
1303 |           icon = CheckCircle
1304 |           iconBg = 'bg-emerald-100 text-emerald-700'
1305 |           title = `${u.role === 'trainer' ? 'Trainer' : 'User'} Verified`
1306 |           desc = `${u.full_name || u.email} is active`
1307 |         } else if (u.approval_status === 'suspended') {
1308 |           icon = Ban
1309 |           iconBg = 'bg-rose-100 text-rose-700'
1310 |           title = 'User Suspended'
1311 |           desc = `${u.full_name || u.email} suspended`
1312 |         } else if (u.approval_status === 'pending') {
1313 |           icon = Clock
1314 |           iconBg = 'bg-amber-100 text-amber-700'
1315 |           title = 'Application Pending'
1316 |           desc = `${u.full_name || u.email} awaiting review`
1317 |         }
1318 |
1319 |         list.push({
1320 |           id: `user-${u.id}-${u.approval_status}`,
1321 |           icon,
1322 |           title,
1323 |           desc,
1324 |           time: formatDistanceToNow(createdAt, { addSuffix: true }),
1325 |           iconBg,
1326 |           timestamp: createdAt.getTime(),
```

```
        })
      }
    })

    // 3. From real courses created

    coursesList.forEach((c) => {
      const createdAt = c.created_at ? new Date(c.created_at) : null
      if (createdAt) {
        list.push({
          id: `course-${c.id}`,
          icon: BookOpen,
          title: c.status === 'published' ? 'Course Published' : 'Course Created',
          desc: `"${c.title}" (${c.delivery_mode || 'recorded'})`,
          time: formatDistanceToNow(createdAt, { addSuffix: true }),
          iconBg: c.status === 'published' ? 'bg-emerald-100 text-emerald-700' : 'bg-blue-100 text-blue-700',
          timestamp: createdAt.getTime(),
        })
      }
    })

    // 4. From real enrollments

    enrollmentsList.forEach((e) => {
      const enrolledAt = e.enrolled_at ? new Date(e.enrolled_at) : null
      if (enrolledAt) {
        const isDone = e.status === 'completed' || (e.progress_percent ?? 0) >= 100
        list.push({
          id: `enroll-${e.id}`,
          icon: isDone ? Award : GraduationCap,
          title: isDone ? 'Certification Completed' : 'Program Enrollment',
          desc: isDone ? `Trainee completed certification (100%)` : `Trainee enrolled in course (${e.progress_percent ?? 0}%
progress)`,
          time: formatDistanceToNow(enrolledAt, { addSuffix: true }),
          iconBg: isDone ? 'bg-emerald-100 text-emerald-700' : 'bg-amber-100 text-amber-700',
          timestamp: enrolledAt.getTime(),
        })
      }
    })

    // Deduplicate and sort by newest

    const seen = new Set()
    const unique = list.filter(item => {
      if (seen.has(item.id)) return false
      seen.add(item.id)
      return true
    })

    unique.sort((a, b) => b.timestamp - a.timestamp)
    return unique.slice(0, 6)
  }, [logs, users, coursesList, enrollmentsList])

  return (
    <>
      <DashboardShell
        title={isSuperAdmin ? "Super Admin Dashboard" : "Admin Dashboard"}
        icon={Shield}
        navLinks={tabs.map(tab => ({
          id: tab.key,
          label: tab.label,
          icon: tab.icon,
          to: 'to' in tab ? (tab as any).to : undefined,
          badge: tab.key === 'trainees' ? pendingTrainees || undefined :
                 tab.key === 'trainers' ? pendingTrainers || undefined :
                 tab.key === 'admins' ? pendingAdmins || undefined :
                 tab.key === 'courses' ? pendingCoursesCount || undefined : undefined,
          isActive: 'to' in tab ? location.pathname === (tab as any).to : activeTab === tab.key,
          onClick: 'to' in tab ? undefined : () => setActiveTab(tab.key as any)
        }))}
      >
        <ConfirmDialog />
        <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-6 max-w-full">
          {/* Welcome Banner (Midnight Dark Aesthetic) */}
          <motion.div
            variants={fadeUp}
            className="relative p-6 sm:p-8 rounded-3xl bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white
overflow-hidden shadow-xl border border-cyan-500/30"
          >
```

```
            <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />
            <div className="relative z-10 flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
              <div>
                <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md text-amber-300 text-xs font-semibold mb-3 border border-white/15">
                  <Shield className="w-3.5 h-3.5" />
                  <span>Administration & Governance</span>
                </div>
                <h2 className="text-2xl sm:text-3xl font-extrabold tracking-tight text-white mb-2">
                  {isSuperAdmin ? "Super Administrator Control Center" : "Administrator Control Center"}
                </h2>
                <p className="text-slate-300 text-sm">
                  {pendingCount > 0
                    ? `${pendingCount} user application(s) require verification.`
                    : 'All applications are processed. Platform security and integrity verified.'}
                </p>
              </div>
              <div className="flex flex-wrap items-center gap-3">
                {pendingCount > 0 && (
                  <Button
                    onClick={() => setActiveTab('trainees')}
                    className="w-full sm:w-auto bg-gradient-to-r from-amber-500 to-orange-600 hover:from-amber-600 hover:to-orange-700 text-white font-bold px-6 py-3 rounded-2xl shadow-lg shadow-amber-500/30 hover:scale-105 transition-all"
                  >
                    <Clock className="w-4 h-4 mr-2" />
                    Review {pendingCount} Pending
                  </Button>
                )}
                {isSuperAdmin && (
                  <Button
                    onClick={cleanupOrphanedFiles}
                    disabled={isCleaningFiles}
                    className="w-full sm:w-auto bg-white/10 hover:bg-white/20 text-white border border-white/20 font-bold px-6 py-3 rounded-2xl transition-all"
                  >
                    {isCleaningFiles ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Trash2 className="w-4 h-4 mr-2" />}
                    Cleanup Files
                  </Button>
                )}
              </div>
            </div>
          </motion.div>

          <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
            {/* Main Content Area */}
            <motion.div variants={fadeUp} className={`space-y-4 min-w-0 ${activeTab === 'overview' ? 'lg:col-span-2' : 'lg:col-span-3'}`}>

              {/* Tab: Overview */}
              <AnimatePresence mode="wait">
                {activeTab === 'overview' && (
                  <motion.div
                    key="overview"
                    variants={scaleIn}
                    initial="hidden"
                    animate="visible"
                    exit="hidden"
                    className="space-y-6"
                  >
                    <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                      {stats.map(s => <StatCard key={s.label} {...s} />)}
                    </div>

                    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
                      <div
                        onClick={() => navigate('/admin/courses/new')}
                        className="group cursor-pointer p-5 rounded-3xl bg-gradient-to-br from-cyan-600 to-blue-700 text-white shadow-md shadow-cyan-600/20 hover:scale-105 transition-all flex flex-col justify-between h-32"
                      >
                        <BookOpen className="w-6 h-6 text-white/80 group-hover:text-white group-hover:scale-110 transition-transform" />
                        <div>
                          <h4 className="font-bold text-sm">Create Course</h4>
                          <p className="text-xs text-white/80">Assign a trainer & publish</p>
                        </div>
                      </div>
                      <div
```

```
                        onClick={() => setActiveTab('trainees')}
                        className="group cursor-pointer p-5 rounded-3xl bg-gradient-to-br from-amber-500 to-orange-600 text-white shadow-md shadow-amber-500/20 hover:scale-105 transition-all flex flex-col justify-between h-32 relative overflow-hidden"
                      >
                        <Users className="w-6 h-6 text-white/80 group-hover:text-white group-hover:scale-110 transition-transform" />
                        <div>
                          <h4 className="font-bold text-sm">Review Trainees</h4>
                          <p className="text-xs text-white/80">Manage user accounts</p>
                        </div>
                        {pendingTrainees > 0 && (
                          <div className="absolute top-4 right-4 bg-black/30 px-2 py-1 rounded-lg text-xs font-bold backdrop-blur-sm">
                            {pendingTrainees} Pending
                          </div>
                        )}
                      </div>
                      <div
                        onClick={() => setActiveTab('trainers')}
                        className="group cursor-pointer p-5 rounded-3xl bg-gradient-to-br from-slate-700 to-slate-900 text-white shadow-md shadow-slate-500/20 hover:scale-105 transition-all flex flex-col justify-between h-32 relative overflow-hidden"
                      >
                        <GraduationCap className="w-6 h-6 text-white/80 group-hover:text-white group-hover:scale-110 transition-transform" />
                        <div>
                          <h4 className="font-bold text-sm">Review Trainers</h4>
                          <p className="text-xs text-white/80">Manage trainer accounts</p>
                        </div>
                        {pendingTrainers > 0 && (
                          <div className="absolute top-4 right-4 bg-black/30 px-2 py-1 rounded-lg text-xs font-bold backdrop-blur-sm">
                            {pendingTrainers} Pending
                          </div>
                        )}
                      </div>
                    </div>

                    <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden">
                      <div className="pb-4 border-b border-slate-100 mb-4">
                        <h3 className="text-base font-bold text-slate-900">Recent Activity</h3>
                        <p className="text-xs text-slate-500 mt-0.5">Live platform activity & system logs.</p>
                      </div>
                      <div className="space-y-4">
                        {activities.length === 0 ? (
                          <p className="text-xs text-slate-400 py-4 text-center">No platform activity recorded yet.</p>
                        ) : (
                          activities.map((act) => (
                            <div key={act.id} className="flex items-center gap-4">
                              <div className={`w-10 h-10 rounded-full flex items-center justify-center shrink-0 ${act.iconBg}`}>
                                <act.icon className="w-5 h-5" />
                              </div>
                              <div className="flex-1 min-w-0">
                                <p className="text-sm font-bold text-slate-900 truncate">{act.title}</p>
                                <p className="text-xs text-slate-500 truncate">{act.desc}</p>
                              </div>
                              <span className="text-xs text-slate-400 font-medium whitespace-nowrap">{act.time}</span>
                            </div>
                          ))
                        )}
                      </div>
                    </div>
                  </motion.div>
                )}
              </AnimatePresence>

              {/* Tab: Trainees */}
              <AnimatePresence mode="wait">
                {activeTab === 'trainees' && (
                  <motion.div
                    key="trainees"
                    variants={scaleIn}
                    initial="hidden"
                    animate="visible"
                    exit="hidden"
                    className="space-y-6"
                  >
                    {/* Header & Sub-Tab Switcher */}
                    <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm">
```

```
                        <div className="flex flex-col lg:flex-row lg:items-center lg:justify-between gap-4 pb-5 border-b border
-slate-100">
                          <div>
                            <h3 className="text-lg font-extrabold text-slate-900 tracking-tight flex items-center gap-2.5">
                              <Users className="w-5 h-5 text-cyan-600" />
                              Trainee Operations & Approvals
                            </h3>
                            <p className="text-xs text-slate-500 mt-1">
                              Review course enrollment requests, approve trainee access, and manage user accounts.
                            </p>
                          </div>

                          {/* Top Sub-Tab Navigation */}
                          <div className="flex items-center gap-2 bg-slate-100/90 p-1 rounded-2xl border border-slate-200/80 sh
rink-0">
                            <button
                              onClick={() => setTraineeSubTab('enrollment_requests')}
                              className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-bold transition-all ${
                                traineeSubTab === 'enrollment_requests'
                                  ? 'bg-white text-cyan-800 shadow-sm border border-slate-200/80'
                                  : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/50'
                              }`}
                            >
                              <GraduationCap className="w-4 h-4 text-cyan-600" />
                              <span>Course Enrollment Requests</span>
                              {pendingEnrollmentsCount > 0 && (
                                <span className="px-2 py-0.5 rounded-full text-[10px] font-extrabold bg-amber-500 text-white an
imate-pulse">
                                  {pendingEnrollmentsCount}
                                </span>
                              )}
                            </button>
                            <button
                              onClick={() => setTraineeSubTab('accounts')}
                              className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-bold transition-all ${
                                traineeSubTab === 'accounts'
                                  ? 'bg-white text-cyan-800 shadow-sm border border-slate-200/80'
                                  : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/50'
                              }`}
                            >
                              <UserCheck className="w-4 h-4 text-cyan-600" />
                              <span>Trainee Accounts Directory</span>
                              {pendingTraineeAccountsCount > 0 && (
                                <span className="px-2 py-0.5 rounded-full text-[10px] font-extrabold bg-rose-500 text-white">
                                  {pendingTraineeAccountsCount}
                                </span>
                              )}
                            </button>
                          </div>
                        </div>

                        {/* SUB-TAB 1: COURSE ENROLLMENT REQUESTS */}
                        {traineeSubTab === 'enrollment_requests' && (
                          <div className="pt-5 space-y-4">
                            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
                              {/* Filter Buttons */}
                              <div className="flex bg-slate-100 p-1 rounded-xl border border-slate-200 w-full sm:w-fit overflow
-x-auto hide-scrollbar">
                                {[
                                  { key: 'pending_approval', label: 'Pending Approval', count: detailedEnrollments.filter(e =>
e.status === 'pending_approval').length },
                                  { key: 'enrolled', label: 'Approved / Active', count: detailedEnrollments.filter(e => e.statu
s === 'enrolled').length },
                                  { key: 'waitlisted', label: 'Waitlisted', count: detailedEnrollments.filter(e => e.status ===
'waitlisted').length },
                                  { key: 'rejected', label: 'Rejected', count: detailedEnrollments.filter(e => e.status === 're
jected').length },
                                  { key: 'all', label: 'All Requests', count: detailedEnrollments.length },
                                ].map(f => (
                                  <button
                                    key={f.key}
                                    onClick={() => setEnrollmentStatusFilter(f.key as any)}
                                    className={`flex items-center gap-1.5 px-3.5 py-1.5 rounded-lg text-xs font-bold transition
-all whitespace-nowrap ${
                                      enrollmentStatusFilter === f.key
                                        ? 'bg-white text-cyan-700 shadow-xs border border-slate-200'
                                        : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/60'
                                    }`}
                                  >
```

```
                                    <span>{f.label}</span>
                                    <span className={`text-[10px] px-1.5 py-0.2 rounded-full ${
                                      enrollmentStatusFilter === f.key ? 'bg-cyan-100 text-cyan-800' : 'bg-slate-200 text-slate
-600'
                                    }`}>
                                      {f.count}
                                    </span>
                                  </button>
                                ))}
                              </div>

                              {/* Search Box */}
                              <div className="relative w-full sm:w-72 shrink-0">
                                <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
                                <input
                                  type="text"
                                  placeholder="Search by trainee, course, trainer..."
                                  value={enrollmentSearchQuery}
                                  onChange={(e) => setEnrollmentSearchQuery(e.target.value)}
                                  className="w-full pl-9 pr-4 py-2 text-xs bg-slate-50 border border-slate-200 rounded-xl text-
slate-900 placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-white transition-all shadow-xs"
                                />
                              </div>
                            </div>

                            {/* Enrollments Table */}
                            {loading ? (
                              <div className="p-12 text-center text-slate-400 text-sm flex items-center justify-center gap-2">
                                <Loader2 className="w-4 h-4 animate-spin text-cyan-600" />
                                <span>Loading enrollment requests...</span>
                              </div>
                            ) : (
                              <div className="overflow-x-auto rounded-2xl border border-slate-100">
                                <table className="w-full min-w-[700px]">
                                  <thead>
                                    <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slat
e-50/80">
                                      <th className="px-4 py-3.5">Trainee</th>
                                      <th className="px-4 py-3.5">Target Course</th>
                                      <th className="px-4 py-3.5">Trainer</th>
                                      <th className="px-4 py-3.5">Applied Date</th>
                                      <th className="px-4 py-3.5">Status</th>
                                      <th className="px-4 py-3.5 text-right">Approval Decision</th>
                                    </tr>
                                  </thead>
                                  <tbody className="divide-y divide-slate-100">
                                    {filteredEnrollments.length === 0 ? (
                                      <tr>
                                        <td colSpan={6} className="text-center py-12 text-slate-400 text-xs">
                                          <GraduationCap className="w-8 h-8 text-slate-300 mx-auto mb-2 opacity-60" />
                                          <p className="font-semibold text-slate-600">No enrollment requests match this criteri
a</p>
                                          <p className="text-[11px] text-slate-400 mt-0.5">Trainee course registration submissi
ons will appear here for review.</p>
                                        </td>
                                      </tr>
                                    ) : (
                                      filteredEnrollments.map((item) => {
                                        const isProcessing = isProcessingEnrollmentId === item.id
                                        const trainee = item.trainee || {}
                                        const course = item.course || {}
                                        const trainerName = course.trainer?.full_name || 'Assigned Trainer'
                                        const enrolledDate = item.enrolled_at ? new Date(item.enrolled_at) : null

                                        return (
                                          <tr key={item.id} className="hover:bg-slate-50/80 transition-colors">
                                            {/* Trainee Info */}
                                            <td className="px-4 py-3.5">
                                              <div className="flex items-center gap-3">
                                                <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600
flex items-center justify-center text-white text-xs font-bold shrink-0 shadow-xs overflow-hidden">
                                                  {trainee.avatar_path ? (
                                                    <img src={trainee.avatar_path} alt={trainee.full_name} className="w-full h-
full object-cover" />
                                                  ) : (
                                                    trainee.full_name?.charAt(0)?.toUpperCase() ?? 'T'
                                                  )}
                                                </div>
                                                <div>
```

```
                                        <p className="text-xs font-bold text-slate-900">{trainee.full_name || 'Traine
e'}</p>
                                        <p className="text-[11px] text-slate-500 truncate max-w-[150px]">{trainee.ema
il || '—'}</p>
                                        {trainee.department && (
                                          <span className="text-[10px] text-slate-400 font-medium">{trainee.departmen
t}</span>
                                        )}
                                      </div>
                                    </div>
                                  </td>

                                  {/* Course Info */}
                                  <td className="px-4 py-3.5">
                                    <div className="max-w-[220px]">
                                      <p className="text-xs font-bold text-slate-900 line-clamp-1">{course.title ||
'Course Details'}</p>
                                      <div className="flex items-center gap-1.5 mt-0.5">
                                        {course.course_type && (
                                          <span className="text-[9px] font-bold uppercase tracking-wider px-1.5 py-0.
2 rounded bg-cyan-50 text-cyan-700 border border-cyan-200">
                                            {course.course_type}
                                          </span>
                                        )}
                                        {course.department && (
                                          <span className="text-[10px] text-slate-500 font-medium truncate">
                                            {course.department}
                                          </span>
                                        )}
                                      </div>
                                    </div>
                                  </td>

                                  {/* Trainer */}
                                  <td className="px-4 py-3.5">
                                    <span className="text-xs text-slate-700 font-medium">{trainerName}</span>
                                  </td>

                                  {/* Applied Date */}
                                  <td className="px-4 py-3.5">
                                    <span className="text-xs text-slate-500 font-medium">
                                      {enrolledDate ? formatDistanceToNow(enrolledDate, { addSuffix: true }) : '—'}
                                    </span>
                                  </td>

                                  {/* Status */}
                                  <td className="px-4 py-3.5">
                                    {item.status === 'pending_approval' ? (
                                      <span className="inline-flex items-center gap-1 text-[11px] font-bold px-2.5 py
-1 rounded-full bg-amber-50 text-amber-700 border border-amber-200/80">
                                        <Clock className="w-3 h-3 text-amber-600" />
                                        Pending Approval
                                      </span>
                                    ) : item.status === 'enrolled' ? (
                                      <span className="inline-flex items-center gap-1 text-[11px] font-bold px-2.5 py
-1 rounded-full bg-emerald-50 text-emerald-700 border border-emerald-200/80">
                                        <CheckCircle className="w-3 h-3 text-emerald-600" />
                                        Enrolled & Approved
                                      </span>
                                    ) : item.status === 'rejected' ? (
                                      <span className="inline-flex items-center gap-1 text-[11px] font-bold px-2.5 py
-1 rounded-full bg-rose-50 text-rose-700 border border-rose-200/80">
                                        <X className="w-3 h-3 text-rose-600" />
                                        Rejected
                                      </span>
                                    ) : item.status === 'waitlisted' ? (
                                      <span className="inline-flex items-center gap-1 text-[11px] font-bold px-2.5 py
-1 rounded-full bg-purple-50 text-purple-700 border border-purple-200/80">
                                        <Clock className="w-3 h-3 text-purple-600" />
                                        Waitlisted
                                      </span>
                                    ) : (
                                      <span className="text-xs text-slate-600 font-medium capitalize">{item.status}</
span>
                                    )}
                                  </td>

                                  {/* Actions */}
                                  <td className="px-4 py-3.5 text-right">
```

```
                                          <div className="flex items-center justify-end gap-1.5">
                                            {isProcessing ? (
                                              <Loader2 className="w-4 h-4 animate-spin text-cyan-600" />
                                            ) : item.status === 'pending_approval' ? (
                                              <>
                                                <button
                                                  onClick={() => handleEnrollmentDecision(item.id, 'approve', item)}
                                                  className="text-xs px-3 py-1.5 rounded-xl bg-emerald-600 hover:bg-emerald
-700 text-white font-bold transition-all shadow-xs flex items-center gap-1 cursor-pointer hover:scale-105"
                                                  title="Approve Trainee Enrollment"
                                                >
                                                  <CheckCircle className="w-3.5 h-3.5" /> Approve
                                                </button>
                                                <button
                                                  onClick={() => handleEnrollmentDecision(item.id, 'reject', item)}
                                                  className="text-xs px-2.5 py-1.5 rounded-xl bg-rose-50 hover:bg-rose-100
text-rose-700 border border-rose-200 font-bold transition-all shadow-xs flex items-center gap-1 cursor-pointer"
                                                  title="Reject Enrollment"
                                                >
                                                  <X className="w-3.5 h-3.5" /> Reject
                                                </button>
                                              </>
                                            ) : item.status === 'enrolled' ? (
                                              <button
                                                onClick={() => handleEnrollmentDecision(item.id, 'reject', item)}
                                                className="text-xs px-2.5 py-1 rounded-lg bg-slate-100 text-slate-600 hove
r:bg-rose-50 hover:text-rose-700 hover:border-rose-200 border border-slate-200 font-bold transition-all"
                                              >
                                                Revoke Access
                                              </button>
                                            ) : (
                                              <button
                                                onClick={() => handleEnrollmentDecision(item.id, 'approve', item)}
                                                className="text-xs px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 bo
rder border-emerald-200 hover:bg-emerald-100 font-bold transition-all"
                                              >
                                                Re-Approve
                                              </button>
                                            )}
                                          </div>
                                        </td>
                                      </tr>
                                    )
                                  })
                                )}
                              </tbody>
                            </table>
                          </div>
                        )}
                      </div>
                    )}

                    {/* SUB-TAB 2: TRAINEE USER ACCOUNTS DIRECTORY */}
                    {traineeSubTab === 'accounts' && (
                      <div className="pt-5 space-y-4">
                        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
                          <div className="flex bg-slate-100 p-1 rounded-xl border border-slate-200 w-full sm:w-fit overflow
-x-auto hide-scrollbar">
                            {(['all', 'pending', 'approved', 'suspended', 'rejected'] as const).map(f => (
                              <button
                                key={f}
                                onClick={() => setStatusFilter(f)}
                                className={`flex-1 sm:flex-none px-4 py-1.5 rounded-lg text-xs font-bold transition-all cap
italize whitespace-nowrap ${
                                  statusFilter === f ? 'bg-white text-cyan-700 shadow-sm border border-slate-200' : 'text-s
late-600 hover:text-slate-900 hover:bg-slate-200/60'
                                }`}
                              >
                                {f}
                              </button>
                            ))}
                          </div>

                          <div className="relative w-full sm:w-64 shrink-0">
                            <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
                            <input
                              type="text"
                              placeholder="Search trainee accounts..."
                              value={searchQuery}
```

```
                                onChange={(e) => setSearchQuery(e.target.value)}
                                className="w-full pl-9 pr-4 py-2 text-xs bg-slate-50 border border-slate-200 rounded-xl text-slate-900 placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-white transition-all shadow-xs"
                              />
                            </div>
                          </div>

                          {loading ? (
                            <div className="p-8 text-center text-slate-400 text-sm">Loading trainees...</div>
                          ) : (
                            <div className="overflow-x-auto mt-2">
                              <table className="w-full min-w-[620px]">
                                <thead>
                                  <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slate-50">
                                    <th className="px-3 py-3">Name</th>
                                    <th className="px-3 py-3">Email</th>
                                    <th className="px-3 py-3">Department</th>
                                    <th className="px-3 py-3">Proof</th>
                                    <th className="px-3 py-3">Status</th>
                                    <th className="px-3 py-3 text-right">Actions</th>
                                  </tr>
                                </thead>
                                <tbody className="divide-y divide-slate-100">
                                  {filteredUsers.length === 0 ? (
                                    <tr>
                                      <td colSpan={6} className="text-center py-8 text-slate-400 text-sm">
                                        {searchQuery ? 'No trainees match your search.' : 'No trainees found.'}
                                      </td>
                                    </tr>
                                  ) : filteredUsers.map(u => (
                                    <tr key={u.id} className="hover:bg-slate-50/80 transition-colors">
                                      <td className="px-3 py-3">
                                        <div
                                          onClick={() => { setSelectedUserForModal(u); setIsUserDetailsModalOpen(true); }}
                                          className="flex items-center gap-2.5 cursor-pointer group"
                                          title="Click to view full profile details"
                                        >
                                          <div className="w-7 h-7 rounded-lg bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white text-xs font-bold shrink-0 group-hover:scale-105 transition-transform shadow-xs overflow-hidden">
                                            {u.avatar_path ? (
                                              <img src={u.avatar_path} alt={u.full_name} className="w-full h-full object-cover" />
                                            ) : (
                                              u.full_name?.charAt(0)?.toUpperCase() ?? '?'
                                            )}
                                          </div>
                                          <span className="text-xs sm:text-sm font-bold text-slate-900 group-hover:text-cyan-700 transition-colors truncate max-w-[130px]">{u.full_name}</span>
                                        </div>
                                      </td>
                                      <td className="px-3 py-3 text-xs text-slate-600 truncate max-w-[150px]">{u.email}</td>
                                      <td className="px-3 py-3 text-xs text-slate-600 truncate max-w-[90px]">{u.department ?? '—'}</td>
                                      <td className="px-3 py-3">
                                        {u.proof_path ? (
                                          <div className="flex flex-col gap-1.5">
                                            <div className="flex items-center gap-1 text-[10px] text-slate-600 bg-slate-100 px-1.5 py-0.5 rounded border border-slate-200 w-fit" title="Securely stored in MoES Cloud">
                                              <Shield className="w-3 h-3 text-emerald-600" />
                                              <span className="truncate max-w-[100px]">{u.proof_path.split('/').pop()}</span>
                                            </div>
                                            <button onClick={() => handleViewProof(u.proof_path!)} className="text-[10px] px-2 py-0.5 rounded-lg bg-cyan-50 text-cyan-700 border border-cyan-200 hover:bg-cyan-100 font-bold transition-all w-fit cursor-pointer">
                                              View Document
                                            </button>
                                          </div>
                                        ) : (
                                          <span className="text-xs text-slate-400">—</span>
                                        )}
                                      </td>
                                      <td className="px-3 py-3"><StatusBadge status={u.approval_status} /></td>
                                      <td className="px-3 py-3 text-right">
                                        <div className="flex items-center justify-end gap-1.5">
                                          <button
                                            type="button"
                                            onClick={(e) => {
```

```
                                            e.preventDefault()
                                            e.stopPropagation()
                                            setSelectedUserForModal(u)
                                            setIsUserDetailsModalOpen(true)
                                          }}
                                          className="text-xs px-2.5 py-1 rounded-lg bg-sky-50 text-sky-800 border border-sky-300 hover:bg-sky-100 hover:border-sky-400 font-bold transition-all shadow-xs flex items-center gap-1 cursor-pointer"
                                          title="View Complete Trainee Info"
                                        >
                                          <Eye className="w-3.5 h-3.5 text-sky-700" /> Info
                                        </button>
                                        {u.approval_status === 'pending' && (
                                          <>
                                            <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'approved')} className="text-xs px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 border border-emerald-200 hover:bg-emerald-100 font-bold transition-all shadow-xs cursor-pointer">Approve</button>
                                            <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'rejected')} className="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-xs cursor-pointer">Reject</button>
                                          </>
                                        )}
                                        {u.approval_status === 'approved' && (
                                          <>
                                            <button onClick={() => handleUpdateUser(u.id, u.email, 'trainer', 'approved')} className="text-xs px-2.5 py-1 rounded-lg bg-cyan-50 text-cyan-700 border border-cyan-200 hover:bg-cyan-100 font-bold transition-all shadow-xs cursor-pointer">↔ Trainer</button>
                                            <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'suspended')} className="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-xs cursor-pointer">Suspend</button>
                                          </>
                                        )}
                                        {u.approval_status === 'suspended' && (
                                          <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'approved')} className="text-xs px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 border border-emerald-200 hover:bg-emerald-100 font-bold transition-all shadow-xs cursor-pointer">Unsuspend</button>
                                        )}
                                        {u.approval_status !== 'pending' && (
                                          <button onClick={() => handleDeleteUser(u.id)} className="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-xs cursor-pointer" title="Permanently Delete Account">Delete</button>
                                        )}
                                      </div>
                                    </td>
                                  </tr>
                                ))}
                              </tbody>
                            </table>
                          </div>
                        )}
                      </div>
                    )}
                  </div>
                </motion.div>
              )}

              {/* Tab: Trainers */}
              {activeTab === 'trainers' && (
                <motion.div
                  key="trainers"
                  variants={scaleIn}
                  initial="hidden"
                  animate="visible"
                  exit="hidden"
                  className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden"
                >
                  <div className="pb-4 border-b border-slate-100 space-y-4">
                    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
                      <div>
                        <h3 className="text-base font-bold text-slate-900 capitalize">Trainers Management</h3>
                        <p className="text-xs text-slate-500 mt-0.5">Approve, reject, suspend or promote trainers.</p>
                      </div>
                      <div className="relative w-full sm:w-64 shrink-0">
                        <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
                        <input
                          type="text"
                          placeholder="Search trainers..."
                          value={searchQuery}
                          onChange={(e) => setSearchQuery(e.target.value)}
                          className="w-full pl-9 pr-4 py-2 text-sm bg-slate-50 border border-slate-200 rounded-xl text-slat
```

```
e-900 placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-white transition-all shadow-xs"
1962 |                           />
1963 |                         </div>
1964 |                       </div>
1965 |                       <div className="flex bg-slate-100 p-1 rounded-xl border border-slate-200 w-full sm:w-fit overflow-x-aut
o hide-scrollbar">
1966 |                         {(['all', 'pending', 'approved', 'suspended', 'rejected'] as const).map(f => (
1967 |                           <button
1968 |                             key={f}
1969 |                             onClick={() => setStatusFilter(f)}
1970 |                             className={`flex-1 sm:flex-none px-4 py-1.5 rounded-lg text-xs font-bold transition-all capitaliz
e whitespace-nowrap ${
1971 |                               statusFilter === f ? 'bg-white text-cyan-700 shadow-sm border border-slate-200' : 'text-slate-6
00 hover:text-slate-900 hover:bg-slate-200/60'
1972 |                             }`}
1973 |                           >
1974 |                             {f}
1975 |                           </button>
1976 |                         ))}
1977 |                       </div>
1978 |                     </div>
1979 |
1980 |                     {loading ? (
1981 |                       <div className="p-8 text-center text-slate-400 text-sm">Loading trainers...</div>
1982 |                     ) : (
1983 |                       <div className="overflow-x-auto mt-2">
1984 |                         <table className="w-full min-w-[620px]">
1985 |                           <thead>
1986 |                             <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slate-50">
1987 |                               <th className="px-3 py-3">Name</th>
1988 |                               <th className="px-3 py-3">Email</th>
1989 |                               <th className="px-3 py-3">Qualifications</th>
1990 |                               <th className="px-3 py-3">Experience</th>
1991 |                               <th className="px-3 py-3">Proof</th>
1992 |                               <th className="px-3 py-3">Status</th>
1993 |                               <th className="px-3 py-3 text-right">Actions</th>
1994 |                             </tr>
1995 |                           </thead>
1996 |                           <tbody className="divide-y divide-slate-100">
1997 |                             {filteredUsers.length === 0 ? (
1998 |                               <tr>
1999 |                                 <td colSpan={7} className="text-center py-8 text-slate-400 text-sm">
2000 |                                   {searchQuery ? 'No trainers match your search.' : 'No trainers found.'}
2001 |                                 </td>
2002 |                               </tr>
2003 |                             ) : filteredUsers.map(u => (
2004 |                               <tr key={u.id} className="hover:bg-slate-50/80 transition-colors">
2005 |                                 <td className="px-3 py-3">
2006 |                                   <div
2007 |                                     onClick={() => { setSelectedUserForModal(u); setIsUserDetailsModalOpen(true); }}
2008 |                                     className="flex items-center gap-2.5 cursor-pointer group"
2009 |                                     title="Click to view full profile details"
2010 |                                   >
2011 |                                     <div className="w-7 h-7 rounded-lg bg-gradient-to-br from-cyan-600 to-blue-600 flex items
-center justify-center text-white text-xs font-bold shrink-0 group-hover:scale-105 transition-transform shadow-xs overflow-hidden">
2012 |                                       {u.avatar_path ? (
2013 |                                         <img src={u.avatar_path} alt={u.full_name} className="w-full h-full object-cover" />
2014 |                                       ) : (
2015 |                                         u.full_name?.charAt(0)?.toUpperCase() ?? '?'
2016 |                                       )}
2017 |                                     </div>
2018 |                                     <span className="text-xs sm:text-sm font-bold text-slate-900 group-hover:text-cyan-700 tr
ansition-colors truncate max-w-[130px]">{u.full_name}</span>
2019 |                                   </div>
2020 |                                 </td>
2021 |                                 <td className="px-3 py-3 text-xs text-slate-600 truncate max-w-[150px]">{u.email}</td>
2022 |                                 <td className="px-3 py-3 text-xs text-slate-600 truncate max-w-[120px]">{(u as any).qualifica
tions ?? '—'}</td>
2023 |                                 <td className="px-3 py-3 text-xs text-slate-600">{(u as any).years_of_experience ? `${(u as a
ny).years_of_experience} yrs` : '—'}</td>
2024 |                                 <td className="px-3 py-3">
2025 |                                   {u.proof_path ? (
2026 |                                     <div className="flex flex-col gap-1.5">
2027 |                                       <div className="flex items-center gap-1 text-[10px] text-slate-600 bg-slate-100 px-1.5
py-0.5 rounded border border-slate-200 w-fit" title="Securely stored in MoES Cloud">
2028 |                                         <Shield className="w-3 h-3 text-emerald-600" />
2029 |                                         <span className="truncate max-w-[100px]">{u.proof_path.split('/').pop()}</span>
2030 |                                       </div>
2031 |                                       <button onClick={() => handleViewProof(u.proof_path!)} className="text-[10px] px-2 py-
```

```
0.5 rounded-lg bg-cyan-50 text-cyan-700 border border-cyan-200 hover:bg-cyan-100 font-bold transition-all w-fit cursor-pointer">
                                        View Document
                                      </button>
                                    </div>
                                  ) : (
                                    <span className="text-xs text-slate-400">—</span>
                                  )}
                                </td>
                                <td className="px-3 py-3"><StatusBadge status={u.approval_status} /></td>
                                <td className="px-3 py-3 text-right">
                                  <div className="flex items-center justify-end gap-1.5">
                                    <button
                                      type="button"
                                      onClick={(e) => {
                                        e.preventDefault()
                                        e.stopPropagation()
                                        setSelectedUserForModal(u)
                                        setIsUserDetailsModalOpen(true)
                                      }}
                                      className="text-xs px-2.5 py-1 rounded-lg bg-sky-50 text-sky-800 border border-sky-300
hover:bg-sky-100 hover:border-sky-400 font-bold transition-all shadow-xs flex items-center gap-1 cursor-pointer"
                                      title="View Complete Trainer Info"
                                    >
                                      <Eye className="w-3.5 h-3.5 text-sky-700" /> Info
                                    </button>
                                    {u.approval_status === 'pending' && (
                                      <>
                                        <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'approved')} className
="text-xs px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 border border-emerald-200 hover:bg-emerald-100 font-bold transition-
all shadow-xs cursor-pointer">Approve</button>
                                        <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'rejected')} className
="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-x
s cursor-pointer">Reject</button>
                                      </>
                                    )}
                                    {u.approval_status === 'approved' && (
                                      <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'suspended')} className
="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-x
s cursor-pointer">Suspend</button>
                                    )}
                                    {u.approval_status === 'suspended' && (
                                      <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'approved')} className
="text-xs px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 border border-emerald-200 hover:bg-emerald-100 font-bold transition-
all shadow-xs cursor-pointer">Unsuspend</button>
                                    )}
                                    {u.approval_status !== 'pending' && (
                                      <button onClick={() => handleDeleteUser(u.id)} className="text-xs px-2.5 py-1 rounded-l
g bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-xs cursor-pointer" title="Perman
ently Delete Account">Delete</button>
                                    )}
                                    {isSuperAdmin && u.approval_status === 'approved' && (
                                      <button onClick={() => handlePromoteToAdmin(u.id)} className="text-xs px-2.5 py-1 round
ed-lg bg-gradient-to-r from-cyan-600 to-blue-600 text-white font-bold hover:opacity-95 transition-all shadow-xs cursor-pointer">Prom
ote to Admin</button>
                                    )}
                                  </div>
                                </td>
                              </tr>
                            ))}
                          </tbody>
                        </table>
                      </div>
                    )}
                  </motion.div>
                )}

                {/* Tab: Admins */}
                {activeTab === 'admins' && (
                  <motion.div
                    key="admins"
                    variants={scaleIn}
                    initial="hidden"
                    animate="visible"
                    exit="hidden"
                    className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden"
                  >
                    <div className="pb-4 border-b border-slate-100 flex flex-col sm:flex-row sm:items-center justify-between
gap-4">
                      <div>
```

```
                        <h3 className="text-base font-bold text-slate-900 capitalize">Admins Management</h3>
                        <p className="text-xs text-slate-500 mt-0.5">Manage administrative access to the platform.</p>
                      </div>
                      <div className="relative w-full sm:w-64 shrink-0">
                        <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
                        <input
                          type="text"
                          placeholder="Search admins..."
                          value={searchQuery}
                          onChange={(e) => setSearchQuery(e.target.value)}
                          className="w-full pl-9 pr-4 py-2 text-sm bg-slate-50 border border-slate-200 rounded-xl text-slate-
900 placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-white transition-all shadow-xs"
                        />
                      </div>
                    </div>

                    {loading ? (
                      <div className="p-8 text-center text-slate-400 text-sm">Loading admins...</div>
                    ) : (
                      <div className="overflow-x-auto mt-2">
                        <table className="w-full min-w-[620px]">
                          <thead>
                            <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slate-50">
                              <th className="px-3 py-3">Name</th>
                              <th className="px-3 py-3">Email</th>
                              <th className="px-3 py-3">Role</th>
                              <th className="px-3 py-3">Status</th>
                              <th className="px-3 py-3 text-right">Actions</th>
                            </tr>
                          </thead>
                          <tbody className="divide-y divide-slate-100">
                            {filteredUsers.length === 0 ? (
                              <tr>
                                <td colSpan={5} className="text-center py-8 text-slate-400 text-sm">
                                  {searchQuery ? 'No admins match your search.' : 'No admins found.'}
                                </td>
                              </tr>
                            ) : filteredUsers.map(u => (
                              <tr key={u.id} className="hover:bg-slate-50/80 transition-colors">
                                <td className="px-3 py-3">
                                  <div
                                    onClick={() => { setSelectedUserForModal(u); setIsUserDetailsModalOpen(true); }}
                                    className="flex items-center gap-2.5 cursor-pointer group"
                                    title="Click to view full profile details"
                                  >
                                    <div className="w-7 h-7 rounded-lg bg-gradient-to-br from-cyan-600 to-blue-600 flex items
-center justify-center text-white text-xs font-bold shrink-0 group-hover:scale-105 transition-transform shadow-xs overflow-hidden">
                                      {u.avatar_path ? (
                                        <img src={u.avatar_path} alt={u.full_name} className="w-full h-full object-cover" />
                                      ) : (
                                        u.full_name?.charAt(0)?.toUpperCase() ?? '?'
                                      )}
                                    </div>
                                    <span className="text-xs sm:text-sm font-bold text-slate-900 group-hover:text-cyan-700 tr
ansition-colors truncate max-w-[130px]">{u.full_name}</span>
                                  </div>
                                </td>
                                <td className="px-3 py-3 text-xs text-slate-600 truncate max-w-[150px]">{u.email}</td>
                                <td className="px-3 py-3">
                                  <span className="text-[11px] font-semibold capitalize text-cyan-800 bg-cyan-50 border borde
r-cyan-200 px-2 py-0.5 rounded-md">
                                    {u.role.replace('_', ' ')}
                                  </span>
                                </td>
                                <td className="px-3 py-3"><StatusBadge status={u.approval_status} /></td>
                                <td className="px-3 py-3 text-right">
                                  <div className="flex items-center justify-end gap-1.5">
                                    <button
                                      type="button"
                                      onClick={(e) => {
                                        e.preventDefault()
                                        e.stopPropagation()
                                        setSelectedUserForModal(u)
                                        setIsUserDetailsModalOpen(true)
                                      }}
                                      className="text-xs px-2.5 py-1 rounded-lg bg-sky-50 text-sky-800 border border-sky-300
hover:bg-sky-100 hover:border-sky-400 font-bold transition-all shadow-xs flex items-center gap-1 cursor-pointer"
                                      title="View Complete Admin Info"
                                    >
```

```
                                    <Eye className="w-3.5 h-3.5 text-sky-700" /> Info
                                  </button>
                                  {u.approval_status === 'approved' && u.role !== 'super_admin' && (
                                    <button onClick={() => handleUpdateUser(u.id, u.email, u.role, 'suspended')} className
="text-xs px-2.5 py-1 rounded-lg bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-x
s cursor-pointer">Suspend</button>
                                  )}
                                  {u.approval_status !== 'pending' && u.role !== 'super_admin' && (
                                    <button onClick={() => handleDeleteUser(u.id)} className="text-xs px-2.5 py-1 rounded-l
g bg-rose-50 text-rose-700 border border-rose-200 hover:bg-rose-100 font-bold transition-all shadow-xs cursor-pointer" title="Perman
ently Delete Account">Delete</button>
                                  )}
                                </div>
                              </td>
                            </tr>
                          ))}
                        </tbody>
                      </table>
                    </div>
                  )}
                </motion.div>
              )}
            </AnimatePresence>

            {/* Tab: Courses */}
            {activeTab === 'courses' && (
              <motion.div key="courses" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <AdminCourses />
              </motion.div>
            )}

            {/* Tab: Assessments */}
            {activeTab === 'assessments' && (
              <motion.div key="assessments" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <AdminAssessmentsHub />
              </motion.div>
            )}

            {/* Tab: Announcements */}
            {activeTab === 'announcements' && (
              <motion.div key="announcements" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <AdminAnnouncements />
              </motion.div>
            )}

            {/* Tab: Notifications */}
            {activeTab === 'notifications' && (
              <motion.div key="notifications" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <UniversalNotificationsPage hideShell />
              </motion.div>
            )}

            {/* Tab: Home Page Management */}
            {activeTab === 'home_page' && (
              <motion.div key="home_page" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <AdminHomePageSettings />
              </motion.div>
            )}

            {/* Tab: Messages */}
            {activeTab === 'messages' && (
              <motion.div key="messages" variants={scaleIn} initial="hidden" animate="visible" exit="hidden">
                <AdminContactMessages />
              </motion.div>
            )}

            {/* Tab: Audit Logs */}
            {activeTab === 'logs' && (
              <motion.div key="logs" variants={scaleIn} initial="hidden" animate="visible" exit="hidden" className="bg-whit
e border border-slate-200/90 rounded-3xl p-6 shadow-sm overflow-hidden">
                <div className="pb-4 border-b border-slate-100">
                  <h3 className="text-base font-bold text-slate-900">Audit Logs</h3>
                  <p className="text-xs text-slate-500 mt-0.5">Complete platform activity and compliance record.</p>
                </div>
                {loading ? (
                  <div className="p-8 text-center text-slate-400 text-sm">Loading logs...</div>
                ) : (
                  <div className="overflow-x-auto mt-2">
                    <table className="w-full min-w-[500px]">
```

```
                        <thead>
                          <tr className="border-b border-slate-200 text-left text-xs text-slate-600 font-bold bg-slate-50">
                            <th className="px-4 py-3">Time</th>
                            <th className="px-4 py-3">Actor</th>
                            <th className="px-4 py-3">Action</th>
                            <th className="px-4 py-3">Entity</th>
                            <th className="px-4 py-3">Details</th>
                          </tr>
                        </thead>
                        <tbody className="divide-y divide-slate-100">
                          {logs.length === 0 ? (
                            <tr><td colSpan={5} className="text-center py-8 text-slate-400 text-sm">No audit logs found.</td></tr>
                          ) : logs.map(log => (
                            <tr key={log.id} className="hover:bg-slate-50/80 transition-colors">
                              <td className="px-4 py-3 text-xs text-slate-600 whitespace-nowrap">{new Date(log.created_at || '').toLocaleString()}</td>
                              <td className="px-4 py-3 text-xs font-mono text-cyan-700">{log.actor_id?.slice(0, 8)}…</td>
                              <td className="px-4 py-3 text-xs text-slate-900 font-bold">{log.action}</td>
                              <td className="px-4 py-3 text-xs text-slate-600">{log.entity_type}</td>
                              <td className="px-4 py-3 text-xs font-mono text-slate-500 max-w-xs truncate">{JSON.stringify(log.metadata)}</td>
                            </tr>
                          ))}
                        </tbody>
                      </table>
                    </div>
                  )}
                </motion.div>
              )}
            </motion.div>

            {/* Recent Activity Sidebar */}
            {activeTab === 'overview' && (
              <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm flex flex-col">
                <div className="pb-4 border-b border-slate-100">
                  <h3 className="text-base font-bold text-slate-900">System Activity</h3>
                  <p className="text-xs text-slate-500">Live audit events</p>
                </div>
                <div className="py-2 space-y-1 flex-1">
                  {activities.length === 0 ? (
                    <p className="text-xs text-slate-400 py-6 text-center">No live events logged.</p>
                  ) : (
                    activities.map((act) => (
                      <ActivityItem key={act.id} {...act} />
                    ))
                  )}
                </div>
              </motion.div>
            )}
          </div>

          {/* Announcements Feed Section */}
          <motion.div variants={fadeUp}>
            <AnnouncementsFeed />
          </motion.div>

          {/* Platform Analytics & Governance Statistics Section */}
          {activeTab === 'overview' && (
            <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-3xl p-6 sm:p-7 shadow-sm overflow-hidden space-y-6">
              {/* Header */}
              <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
                <div className="flex items-center gap-3.5">
                  <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-amber-500 to-orange-600 text-white flex items-center justify-center shadow-md shadow-orange-500/20 shrink-0">
                    <BarChart3 className="w-6 h-6" />
                  </div>
                  <div>
                    <div className="flex items-center gap-2">
                      <h3 className="text-lg font-bold text-slate-900">Platform Analytics & Insights</h3>
                      <span className="inline-flex items-center px-2 py-0.5 rounded-full text-[10px] font-extrabold tracking-wider uppercase bg-emerald-100 text-emerald-800 border border-emerald-200">
                        Live Metrics
                      </span>
                    </div>
                    <p className="text-xs text-slate-500 mt-0.5">Real-time training metrics, completion trends, and instituti
```

```
onal capacity distribution.</p>
                  </div>
                </div>

                <div className="flex items-center gap-2">
                  <Button
                    variant="outline"
                    size="sm"
                    onClick={fetchData}
                    className="rounded-xl border-slate-200 text-slate-700 hover:bg-slate-50 text-xs font-semibold cursor-poin
ter"
                  >
                    <RefreshCw className="w-3.5 h-3.5 mr-1.5" /> Refresh Stats
                  </Button>
                  <Button
                    size="sm"
                    onClick={() => setActiveTab('courses')}
                    className="rounded-xl bg-gradient-to-r from-cyan-600 to-blue-600 text-white hover:opacity-95 text-xs font
-semibold shadow-xs cursor-pointer"
                  >
                    <BookOpen className="w-3.5 h-3.5 mr-1.5" /> View Courses
                  </Button>
                </div>
              </div>

              {/* 4 Stat Metric Cards */}
              <div className="grid grid-cols-2 md:grid-cols-4 gap-3.5">
                <div className="p-4 rounded-2xl bg-slate-50/80 border border-slate-200/80">
                  <p className="text-xs font-semibold text-slate-500">Total Enrollments</p>
                  <p className="text-2xl font-extrabold text-slate-900 mt-1">{platformAnalytics.totalEnrollments}</p>
                  <div className="flex items-center gap-1.5 mt-2 text-[11px] font-medium text-cyan-700">
                    <span className="w-1.5 h-1.5 rounded-full bg-cyan-600"></span>
                    <span>{platformAnalytics.activeEnrollments} Active in training</span>
                  </div>
                </div>

                <div className="p-4 rounded-2xl bg-slate-50/80 border border-slate-200/80">
                  <p className="text-xs font-semibold text-slate-500">Course Completion Rate</p>
                  <p className="text-2xl font-extrabold text-emerald-600 mt-1">{platformAnalytics.completionRate}%</p>
                  <div className="flex items-center gap-1.5 mt-2 text-[11px] font-medium text-emerald-700">
                    <CheckCircle className="w-3.5 h-3.5 text-emerald-600" />
                    <span>{platformAnalytics.completedEnrollments} Certified Trainees</span>
                  </div>
                </div>

                <div className="p-4 rounded-2xl bg-slate-50/80 border border-slate-200/80">
                  <p className="text-xs font-semibold text-slate-500">Average Learner Progress</p>
                  <p className="text-2xl font-extrabold text-blue-600 mt-1">{platformAnalytics.avgProgress}%</p>
                  <div className="w-full bg-slate-200 h-1.5 rounded-full mt-2.5 overflow-hidden">
                    <div className="bg-blue-600 h-full rounded-full transition-all duration-500" style={{ width: `${platformA
nalytics.avgProgress}%` }} />
                  </div>
                </div>

                <div className="p-4 rounded-2xl bg-slate-50/80 border border-slate-200/80">
                  <p className="text-xs font-semibold text-slate-500">Published Programs</p>
                  <p className="text-2xl font-extrabold text-slate-900 mt-1">{platformAnalytics.publishedCourses}</p>
                  <div className="flex items-center gap-1.5 mt-2 text-[11px] font-medium text-slate-500">
                    <span>{platformAnalytics.totalCourses} total created</span>
                    {platformAnalytics.pendingCourses > 0 && (
                      <span className="text-amber-600 font-bold">({platformAnalytics.pendingCourses} pending)</span>
                    )}
                  </div>
                </div>
              </div>

              {/* 2 Detailed Breakdown Panels */}
              <div className="grid grid-cols-1 md:grid-cols-2 gap-5 pt-1">
                {/* Department Distribution */}
                <div className="p-5 rounded-2xl border border-slate-200/80 bg-white shadow-xs space-y-3.5">
                  <div className="flex items-center justify-between">
                    <h4 className="text-xs font-bold uppercase tracking-wider text-slate-700 flex items-center gap-2">
                      <Users className="w-4 h-4 text-cyan-600" />
                      MoES Departmental Representation
                    </h4>
                    <span className="text-[11px] text-slate-400 font-medium">Top Units</span>
                  </div>

                  <div className="space-y-3">
```

```
                    {platformAnalytics.deptArray.length > 0 ? (
                      platformAnalytics.deptArray.map(dept => (
                        <div key={dept.name} className="space-y-1.5">
                          <div className="flex justify-between text-xs font-medium">
                            <span className="text-slate-800 font-bold">{dept.name}</span>
                            <span className="text-slate-500">{dept.count} users ({dept.percent}%)</span>
                          </div>
                          <div className="h-2 w-full bg-slate-100 rounded-full overflow-hidden">
                            <div
                              className="h-full bg-gradient-to-r from-cyan-500 to-blue-600 rounded-full transition-all durati
on-500"
                              style={{ width: `${Math.max(dept.percent, 8)}%` }}
                            />
                          </div>
                        </div>
                      ))
                    ) : (
                      <p className="text-xs text-slate-400 py-3 text-center">No departmental data available yet.</p>
                    )}
                  </div>
                </div>

                {/* Course Delivery & Operational Status */}
                <div className="p-5 rounded-2xl border border-slate-200/80 bg-white shadow-xs space-y-3.5">
                  <div className="flex items-center justify-between">
                    <h4 className="text-xs font-bold uppercase tracking-wider text-slate-700 flex items-center gap-2">
                      <Target className="w-4 h-4 text-orange-600" />
                      Program Delivery Modes
                    </h4>
                    <span className="text-[11px] text-slate-400 font-medium">Distribution</span>
                  </div>

                  <div className="grid grid-cols-3 gap-2.5 pt-1">
                    <div className="p-3 rounded-xl bg-cyan-50/60 border border-cyan-100 text-center">
                      <p className="text-[11px] font-semibold text-cyan-800">Recorded</p>
                      <p className="text-lg font-extrabold text-cyan-950 mt-0.5">{platformAnalytics.deliveryModes.recorded}</
p>
                      <p className="text-[10px] text-cyan-600 mt-0.5">Self-paced</p>
                    </div>
                    <div className="p-3 rounded-xl bg-amber-50/60 border border-amber-100 text-center">
                      <p className="text-[11px] font-semibold text-amber-800">Live</p>
                      <p className="text-lg font-extrabold text-amber-950 mt-0.5">{platformAnalytics.deliveryModes.live}</p>
                      <p className="text-[10px] text-amber-600 mt-0.5">Real-time cohorts</p>
                    </div>
                    <div className="p-3 rounded-xl bg-purple-50/60 border border-purple-100 text-center">
                      <p className="text-[11px] font-semibold text-purple-800">Hybrid</p>
                      <p className="text-lg font-extrabold text-purple-950 mt-0.5">{platformAnalytics.deliveryModes.hybrid}</
p>
                      <p className="text-[10px] text-purple-600 mt-0.5">Blended sessions</p>
                    </div>
                  </div>

                  <div className="pt-2 border-t border-slate-100 flex items-center justify-between text-xs text-slate-600">
                    <span className="flex items-center gap-1.5 font-medium">
                      <Award className="w-4 h-4 text-emerald-600" />
                      Digital Certifications:
                    </span>
                    <span className="font-bold text-slate-900">{platformAnalytics.completedEnrollments} issued</span>
                  </div>
                </div>
              </div>
            </motion.div>
          )}
        </motion.div>
      </DashboardShell>
      <MaterialPreviewDialog
        material={previewMaterial}
        previewUrl={previewUrl}
        onClose={() => setPreviewMaterial(null)}
      />
      <AdminUserDetailsModal
        user={selectedUserForModal}
        isOpen={isUserDetailsModalOpen}
        onClose={() => {
          setIsUserDetailsModalOpen(false)
          setSelectedUserForModal(null)
        }}
        onUpdateStatus={handleUpdateUser}
        onViewProof={handleViewProof}
```

```
      />
    </>
  )
}

export function SuperAdminDashboard() {
  return <AdminDashboard />
}

```

src/pages/ForgotPassword.tsx TSX 5.6 KB 117 Lines

```tsx
import React, { useState } from 'react'
import { Link } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { motion } from 'framer-motion'
import * as z from 'zod'
import { Loader2, ArrowLeft, Mail, CheckCircle } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { useAuth } from '@/hooks/useAuth'
import { toast } from 'sonner'

const schema = z.object({
  email: z.string().email('Please enter a valid email address'),
})
type FormValues = z.infer<typeof schema>

export function ForgotPassword() {
  const { resetPassword } = useAuth()
  const [isLoading, setIsLoading] = useState(false)
  const [isSuccess, setIsSuccess] = useState(false)

  const { register, handleSubmit, formState: { errors } } = useForm<FormValues>({ resolver: zodResolver(schema) })

  const onSubmit = async (data: FormValues) => {
    setIsLoading(true)
    try {
      await resetPassword(data.email)
      setIsSuccess(true)
    } catch (err: unknown) {
      toast.error(err instanceof Error ? err.message : 'Failed to send reset link.')
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div className="min-h-screen bg-[#040814] text-white flex items-center justify-center p-4 relative overflow-hidden">
      {/* Decorative ambient gradients */}
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-500/20 via-blue-500/15 to-transparent blur-[120px]" />
      <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-sky-500/20 via-indigo-500/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10"
      >
        <div className="text-center mb-6">
          <Link to="/" className="inline-flex items-center gap-2.5 mb-4">
            <img src="/logo.png" alt="Logo" className="w-10 h-10 object-contain" />
            <span className="text-lg font-bold">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
            </span>
          </Link>
          <h1 className="text-2xl md:text-3xl font-extrabold text-white tracking-tight">Reset Password</h1>
          <p className="text-slate-400 mt-1.5 text-sm">We'll send a link to your email address</p>
        </div>

        <div className="bg-white text-slate-900 border border-slate-200/90 rounded-3xl p-6 md:p-8 shadow-2xl shadow-black/40">
          {isSuccess ? (
            <motion.div initial={{ opacity: 0, scale: 0.9 }} animate={{ opacity: 1, scale: 1 }} className="text-center py-4 space-y-5">
              <div className="w-16 h-16 bg-emerald-50 rounded-full flex items-center justify-center mx-auto border border-emerald-200">
                <CheckCircle className="w-8 h-8 text-emerald-600" />
              </div>
              <div>
                <h2 className="text-lg font-bold text-slate-900 mb-2">Check your inbox</h2>
                <p className="text-sm text-slate-600 leading-relaxed">
                  A password reset link has been sent to your email. Please also check your spam folder.
                </p>
```

```
              </div>
              <Link to="/login">
                <Button className="w-full bg-slate-100 hover:bg-slate-200 text-slate-800 border border-slate-200 transition-al
l font-semibold rounded-xl h-11">
                  <ArrowLeft className="mr-2 w-4 h-4" /> Back to Sign In
                </Button>
              </Link>
            </motion.div>
          ) : (
            <form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
              <div className="space-y-1.5">
                <Label htmlFor="email" className="text-slate-700 text-sm font-semibold flex items-center gap-1.5">
                  <Mail className="w-3.5 h-3.5" /> Email Address
                </Label>
                <Input
                  id="email"
                  type="email"
                  placeholder="name@example.com"
                  {...register('email')}
                  className={`bg-slate-50 border-slate-200 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:bord
er-cyan-500 h-11 rounded-xl ${errors.email ? 'border-rose-500' : ''}`}
                  disabled={isLoading}
                />
                {errors.email && <p className="text-xs text-rose-600 font-medium">{errors.email.message}</p>}
              </div>

              <Button
                type="submit"
                className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blu
e-700 text-white shadow-md shadow-cyan-600/20 border-0 transition-all font-bold rounded-xl"
                disabled={isLoading}
              >
                {isLoading ? <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Sending...</> : 'Send Reset Link'}
              </Button>

              <div className="text-center pt-2">
                <Link to="/login" className="text-sm text-slate-600 hover:text-cyan-700 inline-flex items-center gap-1 font-se
mibold transition-colors">
                  <ArrowLeft className="w-3.5 h-3.5" /> Back to sign in
                </Link>
              </div>
            </form>
          )}
        </div>
      </motion.div>
    </div>
  )
}
```

src/pages/LandingPage.tsx TSX 53.7 KB 1,144 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react';
import { Link, Navigate } from 'react-router-dom';
import { motion, AnimatePresence, useInView } from 'framer-motion';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogDescription,
} from '@/components/ui/dialog';
import { toast } from 'sonner';
import { useAuth } from '@/hooks/useAuth';
import { StarfieldCanvas } from '@/components/space/StarfieldCanvas';
import { StickyScrollFeatures } from '@/components/landing/StickyScrollFeatures';
import { DynamicUpdatesSection } from '@/components/landing/DynamicUpdatesSection';
import { HorizontalAnnouncementBar } from '@/components/landing/HorizontalAnnouncementBar';
import { MissionExplodedView } from '@/components/landing/MissionExplodedView';
import { AchievementsSection } from '@/components/landing/AchievementsSection';
import { useHomePageSettings } from '@/hooks/useHomePageSettings';
import { supabase } from '@/lib/supabase';
import {
  BookOpen,
  GraduationCap,
  ArrowRight,
  Zap,
  Globe,
  Users,
  BarChart3,
  Lock,
  Star,
  Sparkles,
  Compass,
  ChevronDown,
  Satellite,
  Radio,
  CloudRain,
  ShieldCheck,
  Award,
  Layers,
  Flame,
  ArrowUpRight,
  CheckCircle2,
  Quote,
  Building2,
  WifiOff,
  Cpu,
  HelpCircle,
  ExternalLink,
  Calendar,
  Mail,
  Phone,
  MapPin,
  Send,
  Menu,
  X,
  Loader2,
} from 'lucide-react';

function AnimatedCounter({
  target,
  suffix = '',
  duration = 1.8,
}: {
  target: number;
  suffix?: string;
  duration?: number;
}) {
  const [count, setCount] = useState(0);
  const ref = useRef<HTMLSpanElement | null>(null);
  const isInView = useInView(ref, { once: false, margin: '-20px' });

  useEffect(() => {
    let animationFrameId: number;
```

```
    if (isInView) {
      let startTimestamp: number | null = null;
      const step = (timestamp: number) => {
        if (!startTimestamp) startTimestamp = timestamp;
        const progress = Math.min((timestamp - startTimestamp) / (duration * 1000), 1);
        // Ease out cubic for smooth accelerating-to-settling counter
        const easeOut = 1 - Math.pow(1 - progress, 3);
        setCount(Math.floor(easeOut * target));

        if (progress < 1) {
          animationFrameId = requestAnimationFrame(step);
        } else {
          setCount(target);
        }
      };

      animationFrameId = requestAnimationFrame(step);
    } else {
      setCount(0);
    }

    return () => {
      if (animationFrameId) cancelAnimationFrame(animationFrameId);
    };
  }, [isInView, target, duration]);

  return (
    <span ref={ref}>
      {count.toLocaleString()}
      {suffix}
    </span>
  );
}

function IndianAshokaChakraIcon({ className = 'w-4 h-4' }: { className?: string }) {
  return (
    <svg
      viewBox="0 0 24 24"
      fill="none"
      xmlns="http://www.w3.org/2000/svg"
      className={className}
      aria-label="Ashoka Chakra - Emblem of India"
    >
      <circle cx="12" cy="12" r="10" stroke="#FF9933" strokeWidth="1.2" />
      <circle cx="12" cy="12" r="9" stroke="#FFFFFF" strokeWidth="0.8" />
      <circle cx="12" cy="12" r="8" stroke="#138808" strokeWidth="0.8" />
      <circle cx="12" cy="12" r="2.2" fill="#000080" />
      {Array.from({ length: 24 }).map((_, i) => (
        <line
          key={i}
          x1="12"
          y1="12"
          x2="12"
          y2="4"
          stroke="#000080"
          strokeWidth="0.6"
          strokeLinecap="round"
          transform={`rotate(${i * 15} 12 12)`}
        />
      ))}
    </svg>
  );
}

const fadeUp = {
  hidden: { opacity: 0, y: 30 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.7, ease: 'easeOut' as const } },
};

const stagger = {
  hidden: {},
  visible: { transition: { staggerChildren: 0.12 } },
};

const scaleIn = {
  hidden: { opacity: 0, scale: 0.92 },
  visible: { opacity: 1, scale: 1, transition: { duration: 0.5, ease: 'easeOut' as const } },
```

```
};

const stats = [
  {
    label: 'Courses Available',
    target: 200,
    suffix: '+',
    icon: BookOpen,
    desc: 'All MoES-certified',
    glow: 'from-cyan-500/10 to-transparent',
    border: 'border-slate-200/80 hover:border-cyan-500/50',
    iconColor: 'text-cyan-600',
    iconBg: 'bg-cyan-50 border border-cyan-200 shadow-sm',
  },
  {
    label: 'Active Users',
    target: 5000,
    suffix: '+',
    icon: Users,
    desc: 'Real scientists, real officers',
    glow: 'from-sky-500/10 to-transparent',
    border: 'border-slate-200/80 hover:border-sky-500/50',
    iconColor: 'text-sky-600',
    iconBg: 'bg-sky-50 border border-sky-200 shadow-sm',
  },
  {
    label: 'Completion Rate',
    target: 94,
    suffix: '%',
    icon: BarChart3,
    desc: 'Learners who actually finish',
    glow: 'from-amber-500/10 to-transparent',
    border: 'border-slate-200/80 hover:border-amber-500/50',
    iconColor: 'text-amber-600',
    iconBg: 'bg-amber-50 border border-amber-200 shadow-sm',
  },
  {
    label: 'Certificates Issued',
    target: 12000,
    suffix: '+',
    icon: Star,
    desc: 'Credentials that hold up',
    glow: 'from-emerald-500/10 to-transparent',
    border: 'border-slate-200/80 hover:border-emerald-500/50',
    iconColor: 'text-emerald-600',
    iconBg: 'bg-emerald-50 border border-emerald-200 shadow-sm',
  },
];

const roles = [
  {
    icon: GraduationCap,
    title: 'Trainees & Field Scientists',
    badge: 'Operational Learning',
    desc: 'Learn the way field work actually happens — hands-on radar labs, real telemetry assessments, and verifiable govern
ment micro-credentials.',
    highlights: [
      'Interactive satellite & telemetry lab modules',
      'Progress tracking with skill competency maps',
      'Verifiable government micro-credentials',
    ],
    border: 'border-cyan-500/30 hover:border-cyan-400',
    bg: 'bg-gradient-to-b from-[#081226] via-[#060D1E] to-[#040814]',
    iconBg: 'bg-gradient-to-br from-cyan-600 to-blue-700 shadow-lg shadow-cyan-600/30',
    badgeColor: 'text-cyan-300 bg-cyan-950/80 border-cyan-500/40',
  },
  {
    icon: BookOpen,
    title: 'Trainers & Domain Experts',
    badge: 'Content Authoring',
    desc: 'Spend less time grading, more time teaching. Build your course once, let the platform handle telemetry simulation
grading and cohort analytics.',
    highlights: [
      'Multi-format lecture & simulation builder',
      'Real-time student cohort performance analytics',
      'Automated assessment generation & scoring',
    ],
    border: 'border-amber-500/30 hover:border-amber-400',
    bg: 'bg-gradient-to-b from-[#081226] via-[#060D1E] to-[#040814]',
```

```
    iconBg: 'bg-gradient-to-br from-amber-500 to-orange-600 shadow-lg shadow-amber-500/30',
    badgeColor: 'text-amber-300 bg-amber-950/80 border-amber-500/40',
  },
];

const upcomingCourses = [
  {
    title: 'Dual-Pol Doppler Radar & Severe Storm Nowcasting',
    status: 'Pre-Registration Open',
    statusColor: 'text-emerald-300 bg-emerald-950/80 border-emerald-500/40',
    dotColor: 'bg-emerald-400',
    date: 'Starts Oct 15, 2026',
    duration: '4 Weeks',
    format: 'Live Radar Labs',
    seatsLeft: '24 Seats Left',
    level: 'Advanced Specialist',
    desc: "Go deep on polarimetric radar and real-time storm tracking, with live radar feeds you'll actually use in the fiel
d.",
    keySkills: ['Polarimetric Refl.', 'Hydrometeor Class.', 'Mesocyclone Detection'],
    icon: Radio,
    color: 'from-[#081226] via-[#060D1E] to-[#040814]',
    border: 'border-cyan-500/30 hover:border-cyan-400',
    glow: 'from-cyan-500/20 via-sky-500/10 to-transparent',
    iconBg: 'bg-gradient-to-br from-cyan-950 to-blue-950 border border-cyan-500/40 text-cyan-400',
    accentColor: 'text-cyan-400',
    link: '/register',
    btnText: 'Pre-Register Cohort',
  },
  {
    title: 'Coastal Early Warning & Ocean Telemetry Protocol',
    status: 'Fast Filling',
    statusColor: 'text-amber-300 bg-amber-950/80 border-amber-500/40',
    dotColor: 'bg-amber-400',
    date: 'Starts Dec 01, 2026',
    duration: '3 Weeks',
    format: 'Field Buoy & In-Situ',
    seatsLeft: '12 Seats Left',
    level: 'Executive Protocol',
    desc: 'Master storm surge alerting and ocean buoy data workflows alongside joint INCOIS and IMD teams.',
    keySkills: ['Tsunami Sensor Array', 'Storm Surge Modeling', 'In-Situ Buoy QC'],
    icon: Satellite,
    color: 'from-[#081226] via-[#060D1E] to-[#040814]',
    border: 'border-amber-500/30 hover:border-amber-400',
    glow: 'from-amber-500/20 via-orange-500/10 to-transparent',
    iconBg: 'bg-gradient-to-br from-amber-950 to-orange-950 border border-amber-500/40 text-amber-400',
    accentColor: 'text-amber-400',
    link: '/register',
    btnText: 'Pre-Register Cohort',
  },
  {
    title: 'AI & Deep Learning in Numerical Weather Prediction',
    status: 'Limited Cohort',
    statusColor: 'text-sky-300 bg-sky-950/80 border-sky-500/40',
    dotColor: 'bg-sky-400',
    date: 'Starts Nov 02, 2026',
    duration: '6 Weeks',
    format: 'HPC Supercomputing',
    seatsLeft: '18 Seats Left',
    level: 'Specialized Track',
    desc: 'Hands-on machine learning for weather models and ensemble forecasts, built for real computational pipelines.',
    keySkills: ['Graph Neural Nets', 'Ensemble Prediction', 'CUDA Acceleration'],
    icon: Cpu,
    color: 'from-[#081226] via-[#060D1E] to-[#040814]',
    border: 'border-sky-500/30 hover:border-sky-400',
    glow: 'from-sky-500/20 via-blue-500/10 to-transparent',
    iconBg: 'bg-gradient-to-br from-sky-950 to-indigo-950 border border-sky-500/40 text-sky-400',
    accentColor: 'text-sky-400',
    link: '/register',
    btnText: 'Pre-Register Cohort',
  },
];

const testimonials = [
  {
    quote:
      'Capacity Connect revolutionized how our regional meteorologists train on Doppler radar feeds. The interactive simulati
ons give real-time confidence before high-impact monsoon events.',
    author: 'Dr. V. Ramakrishnan',
    role: 'Scientist-F, Regional Meteorological Centre (Chennai)',
```

```
    division: 'India Meteorological Department',
  },
  {
    quote:
      'The ability to track field officers' competency in AWS calibration across 500+ remote stations ensures high data fidel
ity for our Numerical Weather Prediction models.',
    author: 'Shri A. Sengupta',
    role: 'Director of Observational Networks',
    division: 'Ministry of Earth Sciences (MoES)',
  },
  {
    quote:
      'The role-based architecture and government-compliant learning paths allow us to seamlessly induct new trainees into cy
clone tracking protocols with measurable certification.',
    author: 'Dr. N. Mukherjee',
    role: 'Lead Trainer & Synoptic Analyst',
    division: 'Mausam Bhawan, New Delhi',
  },
];

export function LandingPage() {
  const { session, profile, loading } = useAuth();
  const { settings } = useHomePageSettings();
  const [scrolled, setScrolled] = useState(false);
  const [isContactOpen, setIsContactOpen] = useState(false);
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
  const [contactForm, setContactForm] = useState({
    name: '',
    email: '',
    department: '',
    message: '',
  });
  const [isSubmitting, setIsSubmitting] = useState(false);

  useEffect(() => {
    const handleScroll = () => {
      setScrolled(window.scrollY > 40);
    };
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  const displayedUpcomingTracks = React.useMemo(() => {
    if (settings?.upcoming_tracks_items && settings.upcoming_tracks_items.length > 0) {
      return settings.upcoming_tracks_items.map((t, index) => {
        const fallback = upcomingCourses[index % upcomingCourses.length];
        const rawSkills = t.key_skills;
        const skills = Array.isArray(rawSkills)
          ? rawSkills
          : typeof rawSkills === 'string'
          ? rawSkills.split(',').map((s: string) => s.trim()).filter(Boolean)
          : fallback.keySkills;

        return {
          title: t.title || fallback.title,
          status: t.status || fallback.status,
          level: t.level || fallback.level,
          date: t.date || fallback.date,
          duration: t.duration || fallback.duration,
          format: t.format || fallback.format,
          desc: t.desc || fallback.desc,
          keySkills: skills.length > 0 ? skills : fallback.keySkills,
          link: t.link || '/register',
          btnText: t.btn_text || 'Pre-Register Cohort',
          icon: fallback.icon,
          color: fallback.color,
          border: fallback.border,
          glow: fallback.glow,
          iconBg: fallback.iconBg,
          accentColor: fallback.accentColor,
        };
      });
    }
    return upcomingCourses;
  }, [settings?.upcoming_tracks_items]);

  if (!loading && session) {
    if (profile?.approval_status === 'pending') return <Navigate to="/pending-approval" replace />;
    if (profile?.approval_status === 'suspended') return <Navigate to="/account-suspended" replace />;
```

```
    if (profile?.approval_status === 'rejected') return <Navigate to="/access-denied" replace />;
    if (profile?.role === 'admin' || profile?.role === 'super_admin') return <Navigate to="/admin" replace />;
    if (profile?.role === 'trainer') return <Navigate to="/trainer" replace />;
    return <Navigate to="/trainee" replace />;
  }

  const scrollToFeatures = () => {
    const el = document.getElementById('sticky-features');
    el?.scrollIntoView({ behavior: 'smooth' });
  };

  const scrollToAbout = () => {
    setMobileMenuOpen(false);
    const el = document.getElementById('mission-section') || document.getElementById('sticky-features');
    el?.scrollIntoView({ behavior: 'smooth' });
  };

  const scrollToTop = (e?: React.MouseEvent) => {
    if (e) e.preventDefault();
    window.scrollTo({ top: 0, behavior: 'smooth' });
  };

  const handleContactSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!contactForm.name || !contactForm.email || !contactForm.message) {
      toast.error('Please fill in all required fields.');
      return;
    }
    setIsSubmitting(true);
    try {
      await (supabase as any).from('contact_messages').insert({
        name: contactForm.name.trim(),
        email: contactForm.email.trim().toLowerCase(),
        department: contactForm.department || 'MoES',
        subject: 'Inquiry via Landing Page',
        message: contactForm.message.trim(),
        status: 'unread',
        created_at: new Date().toISOString(),
      });
      const local = JSON.parse(localStorage.getItem('local_contact_messages') || '[]');
      local.unshift({
        id: 'local_' + Date.now(),
        name: contactForm.name.trim(),
        email: contactForm.email.trim(),
        department: contactForm.department || 'MoES',
        subject: 'Inquiry via Landing Page',
        message: contactForm.message.trim(),
        status: 'unread',
        created_at: new Date().toISOString(),
      });
      localStorage.setItem('local_contact_messages', JSON.stringify(local));

      toast.success('Thank you for reaching out! A ministry support representative will respond shortly.');
      setIsContactOpen(false);
      setContactForm({ name: '', email: '', department: '', message: '' });
    } catch (err) {
      toast.error('Failed to submit message.');
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen bg-[#030712] text-white font-sans relative selection:bg-cyan-500 selection:text-white overfl
ow-x-hidden">
      {/* Background Starfield Particle Canvas */}
      <StarfieldCanvas />

      {/* Decorative subtle ambient atmospheric glow orbs */}
      <div className="pointer-events-none fixed -top-40 -left-40 w-[650px] h-[650px] rounded-full bg-cyan-500/10 blur-[150px]
z-0" />
      <div className="pointer-events-none fixed top-1/3 -right-40 w-[600px] h-[600px] rounded-full bg-blue-500/10 blur-[150p
x] z-0" />
      <div className="pointer-events-none fixed bottom-10 left-1/4 w-[500px] h-[500px] rounded-full bg-amber-500/10 blur-[150
px] z-0" />

      {/* ========================================================================== */}
      {/* 1. TOP NAVBAR (Clean Transparent, Get Started in Middle, No Sign In)      */}
      {/* ========================================================================== */}
```

```
      <header
        className={`fixed top-0 left-0 right-0 z-50 transition-all duration-300 ${
          scrolled
            ? 'bg-[#040814]/90 backdrop-blur-xl border-b border-cyan-500/20 py-3 shadow-2xl shadow-cyan-950/60'
            : 'bg-transparent py-5'
        }`}
      >
        <div className="max-w-7xl mx-auto px-6 flex items-center justify-between">
          {/* Brand Logo & Name */}
          <Link to="/" onClick={scrollToTop} className="flex items-center gap-3 group cursor-pointer shrink-0">
            <img src="/logo.webp" alt="Capacity Connect Logo" width={32} height={32} className="w-8 h-8 object-contain shrink
-0 group-hover:scale-105 transition-transform" />
            <span className="text-lg font-bold font-display tracking-tight leading-tight">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
                {' '}
                Connect
              </span>
            </span>
          </Link>

          {/* Center Navigation Links (Desktop) */}
          <nav className="hidden md:flex items-center gap-6 lg:gap-8">
            <Link
              to="/courses"
              className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors drop-shadow-sm"
            >
              Courses
            </Link>
            <Link
              to="/about"
              className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors drop-shadow-sm"
            >
              About
            </Link>
            <Link
              to="/contact"
              className="text-sm font-semibold text-slate-300 hover:text-cyan-400 transition-colors drop-shadow-sm"
            >
              Contact
            </Link>
            <Link
              to="/register"
              className="text-sm font-semibold text-cyan-300 hover:text-white transition-colors drop-shadow-sm"
            >
              Get Started
            </Link>
          </nav>

          {/* Right side spacer for balanced symmetrical centering */}
          <div className="hidden md:block w-36 shrink-0" />

          {/* Mobile Hamburger Button */}
          <div className="flex md:hidden items-center gap-2">
            <button
              type="button"
              onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
              className="p-2 rounded-xl transition-colors cursor-pointer text-white hover:text-cyan-300"
              aria-label="Toggle menu"
            >
              {mobileMenuOpen ? <X className="w-5 h-5" /> : <Menu className="w-5 h-5" />}
            </button>
          </div>
        </div>

        {/* Mobile Dropdown Menu */}
        <AnimatePresence>
          {mobileMenuOpen && (
            <motion.div
              initial={{ opacity: 0, height: 0 }}
              animate={{ opacity: 1, height: 'auto' }}
              exit={{ opacity: 0, height: 0 }}
              className="md:hidden border-t border-cyan-500/20 bg-[#040814]/98 backdrop-blur-xl px-6 py-4 mt-3 space-y-3 shad
ow-2xl"
            >
              <Link
                to="/courses"
                onClick={() => setMobileMenuOpen(false)}
                className="block text-sm font-semibold text-slate-300 hover:text-cyan-400 py-1.5"
```

```
              >
                Courses
              </Link>
              <Link
                to="/about"
                onClick={() => setMobileMenuOpen(false)}
                className="block text-sm font-semibold text-slate-300 hover:text-cyan-400 py-1.5"
              >
                About
              </Link>
              <button
                type="button"
                onClick={() => {
                  setMobileMenuOpen(false);
                  setIsContactOpen(true);
                }}
                className="block w-full text-left text-sm font-semibold text-slate-300 hover:text-cyan-400 py-1.5 cursor-poin
ter"
              >
                Contact
              </button>
              <Link
                to="/register"
                onClick={() => setMobileMenuOpen(false)}
                className="block text-sm font-semibold text-cyan-300 hover:text-white py-1.5"
              >
                Get Started
              </Link>
            </motion.div>
          )}
        </AnimatePresence>
      </header>

      {/* ========================================================================== */}
      {/* 2. HERO SECTION (Apple SF Pro Display Typographic Hierarchy)               */}
      {/* ========================================================================== */}
      <section className="relative min-h-screen w-full flex items-center justify-center overflow-hidden pt-28 pb-20 sm:pt-36
sm:pb-28 md:pt-40 md:pb-32 bg-black">
        {/* Background Image: Spacecraft & Satellite in Earth Orbit with Cinematic Dark Contrast */}
        <div className="absolute inset-0 z-0 pointer-events-none overflow-hidden bg-black">
          <img
            src="/home%20page/hero-spacecraft.webp"
            alt="Spacecraft and satellite in Earth orbit"
            width={1920}
            height={1080}
            loading="eager"
            fetchPriority="high"
            decoding="async"
            className="w-full h-full object-cover object-center brightness-[0.52] contrast-[1.15] saturate-[1.1] scale-[1.0
1]"
          />
        </div>

        {/* Hero Container: Center Aligned Content */}
        <div className="relative z-10 max-w-5xl mx-auto px-6 w-full flex flex-col items-center justify-center text-center my-
auto">
          <motion.div
            variants={stagger}
            initial="hidden"
            animate="visible"
            className="w-full flex flex-col items-center text-center justify-center max-w-4xl mx-auto"
          >
            {/* Tier 1: Eyebrow Tag Pill (Uppercase, letter-spaced, Ashoka Chakra Indian Emblem) */}
            <motion.div variants={fadeUp} className="mb-4 sm:mb-5 max-w-full flex justify-center">
              <div className="inline-flex items-center gap-1.5 sm:gap-2 px-3 py-1 sm:px-4 sm:py-1 rounded-full bg-[#070E20]/9
0/10 border border-white/20 text-zinc-200 backdrop-blur-md text-[8.5px] min-[380px]:text-[9.5px] sm:text-xs font-semibold tracking-w
ide sm:tracking-wider uppercase shadow-sm whitespace-nowrap max-w-full">
                <IndianAshokaChakraIcon className="w-3.5 h-3.5 sm:w-4 sm:h-4 shrink-0 text-white" />
                <span className="truncate">Ministry of Earth Sciences, Government of India</span>
              </div>
            </motion.div>

            {/* Tier 2: Headline: SF Pro Display Bold (700) */}
            <motion.h1
              variants={fadeUp}
              className="text-4xl sm:text-6xl md:text-7xl lg:text-[76px] xl:text-[80px] font-bold tracking-[-0.035em] leading
-[1.04] text-white text-center select-none mb-3 sm:mb-4"
            >
              <span>Building India's Next Generation of</span>
```

```
610 |               <br />
611 |               <span className="bg-gradient-to-b from-white via-white/95 to-zinc-300 bg-clip-text text-transparent">
612 |                 Earth Scientists.
613 |               </span>
614 |             </motion.h1>
615 |
616 |             {/* Tier 3: Subhead: SF Pro Display Medium (500) */}
617 |             <motion.p
618 |               variants={fadeUp}
619 |               className="text-base sm:text-xl md:text-2xl lg:text-[25px] font-medium tracking-[-0.015em] leading-snug text-zi
nc-200 text-center max-w-3xl mx-auto mb-2.5 sm:mb-3 select-none"
620 |             >
621 |               Live satellite telemetry. Real forecasting simulations. Government-certified expertise.
622 |             </motion.p>
623 |
624 |             {/* Tier 4: Smaller / Lighter LMS Supporting Line */}
625 |             <motion.p
626 |               variants={fadeUp}
627 |               className="text-xs sm:text-sm md:text-base text-zinc-400 font-normal leading-relaxed text-center max-w-2xl mx-a
uto tracking-[-0.01em]"
628 |             >
629 |               A digital learning platform for MoES and IMD field officers — courses, certifications, and simulations in one p
lace.
630 |             </motion.p>
631 |           </motion.div>
632 |         </div>
633 |
634 |         {/* Scroll-down Indicator (Bottom Center) */}
635 |         <button
636 |           onClick={scrollToFeatures}
637 |           className="absolute bottom-4 left-1/2 -translate-x-1/2 z-20 flex flex-col items-center gap-1.5 text-white/70 hover:
text-white transition-colors group cursor-pointer"
638 |           aria-label="Scroll to explore"
639 |         >
640 |           <span className="text-[10px] font-mono uppercase tracking-widest text-white/70 group-hover:text-cyan-400 transition
-colors">
641 |             Scroll to explore
642 |           </span>
643 |           <div className="w-7 h-7 rounded-full border border-white/30 bg-black/40 backdrop-blur-md flex items-center justify-
center group-hover:border-white shadow-sm transition-all">
644 |             <ChevronDown className="w-3.5 h-3.5 text-white animate-bounce" />
645 |           </div>
646 |         </button>
647 |       </section>
648 |
649 |       {/* ========================================================================== */}
650 |       {/* 1.5 DYNAMIC UPDATES (Announcements & New Courses)                          */}
651 |       {/* ========================================================================== */}
652 |       <DynamicUpdatesSection />
653 |
654 |       {/* ========================================================================== */}
655 |       {/* 2. CORE PLATFORM ARCHITECTURE (Sticky Scroll Scrollytelling)              */}
656 |       {/* ========================================================================== */}
657 |       <StickyScrollFeatures />
658 |
659 |       {/* ========================================================================== */}
660 |       {/* 3. STATS OVERVIEW SECTION (200+ Courses, 5000+ Users, 94% Completion, etc) */}
661 |       {/* ========================================================================== */}
662 |       <section id="stats-section" className="relative py-20 bg-gradient-to-b from-[#f8fafc] via-white to-[#f1f5f9] text-slate
-900 border-t border-slate-200">
663 |         <div className="max-w-7xl mx-auto px-6">
664 |           <motion.div
665 |             variants={stagger}
666 |             initial="hidden"
667 |             whileInView="visible"
668 |             viewport={{ once: false, amount: 0.2 }}
669 |             className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6"
670 |           >
671 |             {stats.map((s) => (
672 |               <motion.div
673 |                 key={s.label}
674 |                 variants={fadeUp}
675 |                 className={`group relative bg-white border ${s.border} rounded-2xl p-6 text-center hover:scale-[1.02] shadow-
xl shadow-slate-200/60 hover:shadow-2xl transition-all duration-300 overflow-hidden`}
676 |               >
677 |                 {/* Background gradient glow on hover */}
678 |                 <div
679 |                   className={`absolute inset-0 bg-gradient-to-br ${s.glow} opacity-0 group-hover:opacity-100 transition-opaci
ty duration-300 pointer-events-none`}
```

```
                />

                <div className="relative z-10">
                  <div
                    className={`w-12 h-12 rounded-xl ${s.iconBg} flex items-center justify-center mx-auto mb-4 ${s.iconColor}
group-hover:scale-110 transition-transform duration-300`}
                  >
                    <s.icon className="w-6 h-6" />
                  </div>
                  <div className="text-3xl sm:text-4xl font-extrabold font-display text-slate-900 mb-1 tracking-tight">
                    <AnimatedCounter target={s.target} suffix={s.suffix} />
                  </div>
                  <div className="text-sm font-semibold text-slate-800 mb-1 tracking-[-0.01em]">{s.label}</div>
                  <div className="text-xs text-slate-500 font-medium">{s.desc}</div>
                </div>
              </motion.div>
            ))}
          </motion.div>
        </div>
      </section>

      {/* ========================================================================== */}
      {/* 3.5 HORIZONTAL LIVE ANNOUNCEMENTS TICKER BAR                              */}
      {/* ========================================================================== */}
      <HorizontalAnnouncementBar />

      {/* ========================================================================== */}
      {/* 4. "BUILT FOR EVERY ROLE" (Trainees, Trainers, Coordinators)              */}
      {/* ========================================================================== */}
      <section id="roles-section" className="relative py-24 bg-gradient-to-b from-[#f1f5f9] via-white to-[#f8fafc] text-slate
-900 border-t border-slate-200">
        <div className="max-w-7xl mx-auto px-6">
          <div className="text-center max-w-3xl mx-auto mb-16">
            <div className="inline-flex items-center gap-2 px-3.5 py-1 rounded-full bg-cyan-50 border border-cyan-200 text-cy
an-800 text-xs font-semibold uppercase tracking-wider mb-4 shadow-sm">
              <Layers className="w-3.5 h-3.5 text-cyan-600" />
              Role-Specific Workspaces
            </div>
            <h2 className="text-3xl sm:text-4xl md:text-5xl font-bold font-display text-slate-900 tracking-[-0.02em] mb-4">
              Designed for Every Stakeholder
            </h2>
            <p className="text-slate-600 text-base sm:text-lg tracking-[-0.01em]">
              Tailored dashboards and tools configured specifically for field trainees, scientific
              instructors, and ministry administrators.
            </p>
          </div>

          <div className="grid grid-cols-1 lg:grid-cols-2 gap-8 max-w-5xl mx-auto">
            {roles.map((r) => (
              <motion.div
                key={r.title}
                variants={scaleIn}
                initial="hidden"
                whileInView="visible"
                viewport={{ once: true }}
                className={`group relative p-8 sm:p-10 rounded-3xl border ${r.border} ${r.bg} backdrop-blur-xl shadow-2xl sha
dow-black/70 hover:shadow-cyan-500/10 transition-all duration-500 hover:-translate-y-1 overflow-hidden flex flex-col justify-between
`}
              >
                <div className="relative z-10">
                  <div className="flex items-center justify-between gap-4 mb-6">
                    <div className={`w-14 h-14 rounded-2xl ${r.iconBg} flex items-center justify-center text-white`}>
                      <r.icon className="w-7 h-7" />
                    </div>
                    <span
                      className={`text-xs font-mono font-semibold px-3 py-1 rounded-full border ${r.badgeColor}`}
                    >
                      {r.badge}
                    </span>
                  </div>

                  <h3 className="text-2xl font-bold font-display text-white mb-3 tracking-[-0.015em]">
                    {r.title}
                  </h3>
                  <p className="text-slate-300 text-sm sm:text-base leading-relaxed mb-6 tracking-[-0.01em]">
                    {r.desc}
                  </p>

                  <div className="space-y-3 mb-8">
```

```
                    {r.highlights.map((h) => (
                      <div key={h} className="flex items-start gap-3 text-xs sm:text-sm text-slate-200 tracking-[-0.005em]">
                        <CheckCircle2 className="w-4 h-4 text-emerald-400 shrink-0 mt-0.5" />
                        <span>{h}</span>
                      </div>
                    ))}
                  </div>
                </div>

                <div className="pt-6 border-t border-cyan-500/20 flex items-center justify-between relative z-10">
                  <Link
                    to="/register"
                    className="inline-flex items-center gap-2 text-sm font-semibold text-cyan-400 hover:text-cyan-300 transition-colors tracking-[-0.01em]"
                  >
                    <span>Get Started in this Role</span>
                    <ArrowRight className="w-4 h-4" />
                  </Link>
                  <span className="text-xs font-mono text-slate-400">Fast-track Onboarding</span>
                </div>
              </motion.div>
            ))}
          </div>
        </div>
      </section>

      {/* ========================================================================== */}
      {/* 5. UPCOMING COURSES & ANNOUNCEMENTS PREVIEW                                */}
      {/* ========================================================================== */}
      {(!settings || settings.upcoming_tracks_enabled) && (
        <section
          id="courses-preview"
          className="relative py-24 sm:py-32 bg-gradient-to-b from-[#f8fafc] via-white to-[#f1f5f9] text-slate-900 border-t border-slate-200 overflow-hidden font-['SF_Pro_Display',-apple-system,BlinkMacSystemFont,'Inter',sans-serif]"
        >
          {/* Ambient atmospheric backdrop glow */}
          <div className="absolute top-1/4 left-1/2 -translate-x-1/2 w-[800px] h-[350px] bg-cyan-500/5 rounded-full blur-[140px] pointer-events-none" />

          <div className="max-w-6xl mx-auto px-6 relative z-10">
            <div className="flex flex-col md:flex-row md:items-end justify-between mb-12 sm:mb-16 gap-6">
              <div>
                <div className="inline-flex items-center gap-2 px-3.5 py-1 rounded-full bg-cyan-50 border border-cyan-200 text-cyan-800 text-[11px] font-semibold tracking-wide uppercase mb-3 sm:mb-4 shadow-sm">
                  <Sparkles className="w-3.5 h-3.5 text-cyan-600" />
                  <span>{settings?.upcoming_tracks_tag || 'Upcoming Announcements'}</span>
                </div>
                <h2 className="text-3xl sm:text-5xl md:text-6xl font-bold tracking-[-0.025em] leading-[1.08] text-slate-900">
                  {settings?.upcoming_tracks_title || 'Specialized Earth Sciences Tracks.'}
                </h2>
                <p className="text-base sm:text-xl md:text-2xl font-normal text-slate-600 tracking-[-0.015em] leading-snug mt-2 sm:mt-3 max-w-2xl">
                  {settings?.upcoming_tracks_subtitle || 'Pre-register for next-generation cohorts and masterclasses.'}
                </p>
              </div>
              <Link to="/courses">
                <Button
                  variant="outline"
                  className="border-slate-300 bg-white hover:bg-slate-50 text-slate-800 rounded-full shadow-sm transition-all duration-300 tracking-[-0.01em] group h-11 px-6 font-semibold text-sm"
                >
                  <span>{settings?.upcoming_tracks_btn_text || 'View Complete Catalog'}</span>
                  <ArrowRight className="w-4 h-4 ml-2 transition-transform duration-300 group-hover:translate-x-1" />
                </Button>
              </Link>
            </div>

            <div className="max-w-6xl mx-auto grid grid-cols-1 md:grid-cols-3 gap-5 lg:gap-6">
              {displayedUpcomingTracks.map((c) => (
                <motion.div
                  key={c.title}
                  variants={fadeUp}
                  initial="hidden"
                  whileInView="visible"
                  viewport={{ once: true }}
                  className={`relative p-5 sm:p-6 rounded-2xl bg-gradient-to-b ${c.color} border ${c.border} backdrop-blur-xl flex flex-col justify-between group transition-all duration-300 shadow-2xl shadow-black/70 hover:shadow-[0_0_30px_rgba(0,210,255,0.16)] hover:-translate-y-1 overflow-hidden`}
                >
```

```
                {/* Glow effect on hover */}
                <div
                  className={`absolute inset-0 bg-gradient-to-br ${c.glow} opacity-0 group-hover:opacity-100 transition-opacity duration-500 pointer-events-none`}
                />
                <div className="absolute top-0 right-0 w-32 h-32 bg-cyan-500/10 rounded-full blur-3xl pointer-events-none" />

                <div className="relative z-10">
                  {/* Top Header: Icon + Single Clean Tag */}
                  <div className="flex items-center justify-between gap-3 mb-4">
                    <div className={`w-10 h-10 rounded-xl flex items-center justify-center shrink-0 ${c.iconBg} shadow-sm group-hover:scale-105 transition-transform duration-300`}>
                      <c.icon className="w-5 h-5" />
                    </div>
                    <span className="text-xs font-mono font-medium px-2.5 py-1 rounded-full bg-white/10 border border-white/15 text-slate-200 tracking-wide">
                      {c.level}
                    </span>
                  </div>

                  {/* Title & Desc */}
                  <h3 className="text-lg sm:text-xl font-bold text-white mb-2 group-hover:text-cyan-300 transition-colors leading-snug tracking-[-0.02em] min-h-[52px] flex items-center">
                    {c.title}
                  </h3>
                  <p className="text-xs sm:text-sm text-slate-300 font-normal leading-relaxed mb-4 tracking-[-0.01em] line-clamp-3">
                    {c.desc}
                  </p>

                  {/* Key Skills Tags */}
                  <div className="mb-4 flex flex-wrap gap-1.5">
                    {c.keySkills.map((skill) => (
                      <span
                        key={skill}
                        className="text-[10px] font-mono px-2 py-0.5 rounded-md bg-white/5 border border-white/10 text-slate-300 group-hover:border-cyan-500/30 transition-colors"
                      >
                        {skill}
                      </span>
                    ))}
                  </div>
                </div>

                {/* Card Footer with Schedule Grid and Pre-Register CTA */}
                <div className="relative z-10 pt-3.5 border-t border-cyan-500/20 flex flex-col gap-3">
                  <div className="flex items-center justify-between text-[11px] font-mono text-slate-300 px-1">
                    <div className="flex items-center gap-1.5 text-slate-200 font-medium">
                      <Calendar className="w-3.5 h-3.5 text-cyan-400 shrink-0" />
                      <span>{c.date}</span>
                    </div>
                    <div className="text-slate-400">
                      <span>{c.duration}</span>
                    </div>
                  </div>

                  <Link
                    to={c.link || '/register'}
                    className="w-full flex items-center justify-center gap-1.5 py-2.5 px-3 rounded-xl bg-cyan-950/80 hover:bg-gradient-to-r hover:from-cyan-500 hover:via-blue-600 hover:to-amber-500 border border-cyan-500/30 hover:border-transparent text-cyan-200 hover:text-white font-semibold text-xs tracking-wide transition-all duration-300 shadow-none hover:shadow-lg hover:shadow-cyan-500/20 group/btn"
                  >
                    <span>{c.btnText || 'Pre-Register Cohort'}</span>
                    <ArrowRight className="w-3.5 h-3.5 transition-transform duration-300 group-hover/btn:translate-x-1" />
                  </Link>
                </div>
              </motion.div>
            ))}
          </div>
        </div>
      </section>
      )}

      {/* ========================================================================== */}
      {/* 6. "3D LAYER SPLIT / EXPLODED VIEW" MISSION SECTION                        */}
      {/* ========================================================================== */}
      <MissionExplodedView id="mission-section" />
```

```
      {/* ========================================================================== */}
      {/* 7. FINAL CALL TO ACTION (CTA BAND)                                         */}
      {/* ========================================================================== */}
      <section className="relative py-28 bg-gradient-to-b from-[#040814] via-[#081226] to-[#030712] border-t border-cyan-500/20 overflow-hidden">
        {/* Orbital rings background visual */}
        <div className="pointer-events-none absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[700px] h-[700px] rounded-full border border-cyan-500/20 animate-orbit-slow" />
        <div className="pointer-events-none absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[900px] h-[900px] rounded-full border border-amber-500/15 animate-orbit-slow" />
        <div className="pointer-events-none absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[400px] h-[400px] rounded-full bg-gradient-to-r from-cyan-500/15 via-blue-500/15 to-amber-500/15 blur-3xl" />

        <div className="relative z-10 max-w-4xl mx-auto px-6 text-center">
          <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-amber-950/60 border border-amber-500/30 text-amber-300 text-xs font-semibold uppercase tracking-wider mb-6 shadow-lg shadow-amber-950/50">
            <Sparkles className="w-3.5 h-3.5 text-amber-400 animate-pulse" />
            Join the Next Generation of Earth Scientists
          </div>

          <h2 className="text-3xl sm:text-4xl md:text-5xl lg:text-6xl font-bold font-display text-white tracking-[-0.02em] leading-tight mb-6">
            Ready to Build Capacity for a{' '}
            <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
              Changing Climate?
            </span>
          </h2>

          <p className="text-slate-300 text-base sm:text-lg max-w-2xl mx-auto leading-relaxed mb-10 tracking-[-0.01em]">
            Join hundreds of field officers, meteorologists, and data analysts across India. Get started
            today with official government certification tracks.
          </p>

          <div className="flex flex-col sm:flex-row items-center justify-center gap-4">
            <Link to="/register" className="w-full sm:w-auto">
              <Button
                size="lg"
                className="w-full sm:w-auto h-14 px-10 text-base font-semibold bg-gradient-to-r from-cyan-500 via-blue-600 to-amber-500 hover:opacity-95 text-white shadow-2xl shadow-cyan-500/25 rounded-xl tracking-[-0.01em]"
              >
                Join the Platform
                <ArrowRight className="ml-2 w-4 h-4" />
              </Button>
            </Link>
            <Link to="/courses" className="w-full sm:w-auto">
              <Button
                size="lg"
                variant="outline"
                className="w-full sm:w-auto h-14 px-8 text-base font-semibold border-cyan-500/40 bg-cyan-950/30 hover:bg-cyan-900/50 text-white rounded-xl shadow-none hover:border-cyan-400 transition-all duration-300 tracking-[-0.01em]"
              >
                Browse All Courses
                <Compass className="ml-2 w-4 h-4 text-cyan-400" />
              </Button>
            </Link>
          </div>
        </div>
      </section>

      {/* ========================================================================== */}
      {/* 7.5. PROVEN RESULTS & ACHIEVEMENTS SHOWCASE SECTION                        */}
      {/* ========================================================================== */}
      <AchievementsSection />

      {/* ========================================================================== */}
      {/* 8. TESTIMONIALS & TRUST BAND                                               */}
      {/* ========================================================================== */}
      <section className="relative py-24 bg-gradient-to-b from-[#f8fafc] via-white to-[#f1f5f9] text-slate-900 border-t border-slate-200">
        <div className="max-w-7xl mx-auto px-6">
          <div className="text-center max-w-3xl mx-auto mb-16">
            <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-cyan-50 border border-cyan-200 text-cyan-800 text-xs font-semibold uppercase tracking-wider mb-4 shadow-sm">
              <Quote className="w-3.5 h-3.5 text-cyan-600" />
              Field Impact
            </div>
            <h2 className="text-3xl sm:text-4xl font-bold font-display text-slate-900 tracking-[-0.02em] mb-4">
              Trusted by Meteorological Centres Across India
            </h2>
```

```
          </div>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            {testimonials.map((t) => (
              <motion.div
                key={t.author}
                variants={fadeUp}
                initial="hidden"
                whileInView="visible"
                viewport={{ once: true }}
                className="p-8 rounded-3xl bg-white border border-slate-200/90 shadow-xl shadow-slate-200/60 hover:shadow-2xl hover:border-cyan-500/40 flex flex-col justify-between relative overflow-hidden transition-all duration-300"
              >
                <div className="text-cyan-600/30 mb-4">
                  <Quote className="w-8 h-8" />
                </div>
                <p className="text-slate-700 text-sm leading-relaxed mb-6 italic tracking-[-0.01em]">&ldquo;{t.quote}&rdquo;</p>
                <div className="pt-4 border-t border-slate-100">
                  <div className="text-sm font-bold text-slate-900 tracking-[-0.01em]">{t.author}</div>
                  <div className="text-xs text-cyan-700 font-semibold">{t.role}</div>
                  <div className="text-[11px] font-medium text-slate-500 mt-1">{t.division}</div>
                </div>
              </motion.div>
            ))}
          </div>
        </div>
      </section>

      {/* ========================================================================== */}
      {/* 9. DEEP SPACE FOOTER                                                       */}
      {/* ========================================================================== */}
      <footer className="relative z-10 bg-[#02050E] border-t border-cyan-500/20 py-12 text-slate-400">
        <div className="max-w-7xl mx-auto px-6 flex flex-col md:flex-row items-center justify-between gap-6">
          <div className="flex flex-col items-center md:items-start text-center md:text-left">
            <Link to="/" onClick={scrollToTop} className="flex items-center gap-3 mb-2 group cursor-pointer">
              <img src="/logo.png" alt="Capacity Connect" className="w-8 h-8 object-contain shrink-0 group-hover:scale-105 transition-transform" />
              <span className="text-xl font-bold font-display">
                <span className="text-white">Capacity</span>
                <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
                  {' '}
                  Connect
                </span>
              </span>
            </Link>
            <p className="text-sm text-slate-400 max-w-xl">
              Digital Capacity Building & Learning Portal developed for the Ministry of Earth Sciences (MoES) & India Meteorological Department (IMD).
            </p>
          </div>

          <div className="flex flex-col md:flex-row items-center gap-6 text-xs text-slate-400">
            <div className="flex items-center gap-4">
              <Link to="/courses" className="hover:text-cyan-400 transition-colors">Courses</Link>
              <Link to="/about" className="hover:text-cyan-400 transition-colors">About</Link>
              <Link to="/contact" className="hover:text-cyan-400 transition-colors">Contact</Link>
              <Link to="/login" className="hover:text-cyan-400 transition-colors">Login</Link>
              <Link to="/register" className="hover:text-cyan-400 transition-colors">Register</Link>
            </div>
            <div className="font-mono text-slate-500">
              © 2026 Capacity Connect • Ministry of Earth Sciences (MoES), Govt. of India.
            </div>
          </div>
        </div>
      </footer>

      {/* ========================================================================== */}
      {/* 10. GET IN TOUCH MODAL DIALOG                                              */}
      {/* ========================================================================== */}
      <Dialog open={isContactOpen} onOpenChange={setIsContactOpen}>
        <DialogContent className="max-w-lg rounded-3xl bg-[#081022] border border-cyan-500/30 shadow-2xl text-white p-6 sm:p-8">
          <DialogHeader className="text-left space-y-1.5">
            <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-950/70 text-cyan-300 text-xs font-semibold uppercase tracking-wider w-fit mb-1 border border-cyan-500/30">
              <Mail className="w-3.5 h-3.5 text-cyan-400" />
              Contact & Inquiries
            </div>
```

```
              <DialogTitle className="text-2xl font-bold font-display text-white tracking-tight">
                Get in Touch
              </DialogTitle>
              <DialogDescription className="text-sm text-slate-300">
                Have questions about MoES certifications, Doppler radar training tracks, or institutional onboarding? Reach out to our team.
              </DialogDescription>
            </DialogHeader>

            {/* Quick contact info cards */}
            <div className="grid grid-cols-2 gap-3 my-1 text-xs">
              <div className="p-3 rounded-2xl bg-[#040816] border border-cyan-500/20 flex items-start gap-2.5">
                <Mail className="w-4 h-4 text-cyan-400 shrink-0 mt-0.5" />
                <div>
                  <span className="font-bold text-white block">Email Support</span>
                  <span className="text-slate-400 text-[11px] block break-all">support@moes.gov.in</span>
                </div>
              </div>
              <div className="p-3 rounded-2xl bg-[#040816] border border-amber-500/20 flex items-start gap-2.5">
                <Phone className="w-4 h-4 text-amber-400 shrink-0 mt-0.5" />
                <div>
                  <span className="font-bold text-white block">Helpline</span>
                  <span className="text-slate-400 text-[11px] block">+91 11 24669500</span>
                </div>
              </div>
            </div>

            <form onSubmit={handleContactSubmit} className="space-y-3.5 mt-2">
              <div className="space-y-1.5">
                <Label htmlFor="contact-name" className="text-xs font-semibold text-slate-200">
                  Full Name <span className="text-red-400">*</span>
                </Label>
                <Input
                  id="contact-name"
                  required
                  placeholder="e.g. Dr. Rajesh Sharma"
                  value={contactForm.name}
                  onChange={(e) => setContactForm({ ...contactForm, name: e.target.value })}
                  className="bg-[#040816] border-slate-700 text-white placeholder:text-slate-500 text-sm rounded-xl focus:border-cyan-400"
                />
              </div>

              <div className="space-y-1.5">
                <Label htmlFor="contact-email" className="text-xs font-semibold text-slate-200">
                  Work / Official Email <span className="text-red-400">*</span>
                </Label>
                <Input
                  id="contact-email"
                  type="email"
                  required
                  placeholder="name@imd.gov.in"
                  value={contactForm.email}
                  onChange={(e) => setContactForm({ ...contactForm, email: e.target.value })}
                  className="bg-[#040816] border-slate-700 text-white placeholder:text-slate-500 text-sm rounded-xl focus:border-cyan-400"
                />
              </div>

              <div className="space-y-1.5">
                <Label htmlFor="contact-dept" className="text-xs font-semibold text-slate-200">
                  Department / Organization (Optional)
                </Label>
                <Input
                  id="contact-dept"
                  placeholder="e.g. IMD Radar Operations, RMC Chennai"
                  value={contactForm.department}
                  onChange={(e) => setContactForm({ ...contactForm, department: e.target.value })}
                  className="bg-[#040816] border-slate-700 text-white placeholder:text-slate-500 text-sm rounded-xl focus:border-cyan-400"
                />
              </div>

              <div className="space-y-1.5">
                <Label htmlFor="contact-msg" className="text-xs font-semibold text-slate-200">
                  Message / Inquiry <span className="text-red-400">*</span>
                </Label>
                <textarea
                  id="contact-msg"
```

```
                required
                rows={3}
                placeholder="How can we help your division?"
                value={contactForm.message}
                onChange={(e) => setContactForm({ ...contactForm, message: e.target.value })}
                className="w-full bg-[#040816] border border-slate-700 rounded-xl p-3 text-sm text-white placeholder:text-sla
te-500 focus:outline-none focus:border-cyan-400 focus:ring-1 focus:ring-cyan-400/30 resize-none"
              />
            </div>

            <Button
              type="submit"
              disabled={isSubmitting}
              className="w-full h-11 bg-gradient-to-r from-cyan-500 via-blue-600 to-amber-500 text-white font-semibold rounde
d-xl shadow-lg shadow-cyan-500/25 hover:opacity-95"
            >
              {isSubmitting ? (
                <>
                  <Loader2 className="w-4 h-4 mr-2 animate-spin" />
                  Sending Message...
                </>
              ) : (
                <>
                  Send Message
                  <Send className="w-4 h-4 ml-2" />
                </>
              )}
            </Button>
          </form>
        </DialogContent>
      </Dialog>
    </div>
  );
}
```

src/pages/Login.tsx TSX 7.4 KB 172 Lines

```tsx
import React, { useState } from 'react'
import { useNavigate, Link } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { motion } from 'framer-motion'
import * as z from 'zod'
import { Eye, EyeOff, Loader2, ArrowRight, ArrowLeft } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { useAuth } from '@/hooks/useAuth'
import { toast } from 'sonner'

const loginSchema = z.object({
  email: z.string().email('Please enter a valid email address'),
  password: z.string().min(1, 'Password is required'),
})

type LoginFormValues = z.infer<typeof loginSchema>

export function Login() {
  const { user, signIn } = useAuth()
  const navigate = useNavigate()
  const [showPassword, setShowPassword] = useState(false)
  const [isLoading, setIsLoading] = useState(false)

  React.useEffect(() => {
    if (user) {
      navigate('/dashboard', { replace: true })
    }
  }, [user, navigate])

  const { register, handleSubmit, formState: { errors } } = useForm<LoginFormValues>({
    resolver: zodResolver(loginSchema),
  })

  const onSubmit = async (data: LoginFormValues) => {
    setIsLoading(true)
    try {
      await signIn(data.email, data.password)
      navigate('/dashboard', { replace: true })
    } catch (err: unknown) {
      const msg = err instanceof Error ? err.message : 'Your email or password is incorrect.'
      toast.error(msg)
    } finally {
      setIsLoading(false)
    }
  }

  const [isExiting, setIsExiting] = useState(false)

  const handleGoToRegister = (e: React.MouseEvent) => {
    e.preventDefault()
    setIsExiting(true)
    setTimeout(() => navigate('/register'), 300)
  }

  return (
    <motion.div
      initial={false}
      animate={{ backgroundColor: isExiting ? "#f8fafc" : "#040814" }}
      transition={{ duration: 0.4, ease: "easeInOut" }}
      className="min-h-screen text-white flex items-center justify-center p-4 md:p-6 relative overflow-hidden"
      style={{ perspective: 1200 }}
    >
      {/* Decorative ambient gradients */}
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-500/20 via-blue-500/15 to-transparent blur-[120px]" />
      <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-sky-500/20 via-indigo-500/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, rotateY: -90 }}
        animate={isExiting ? { opacity: 0, rotateY: -90 } : { opacity: 1, rotateY: 0 }}
        transition={{ duration: 0.4, ease: "easeInOut" }}
        className="w-full max-w-md relative z-10"
```

```
        style={{ transformStyle: "preserve-3d" }}
      >
        <Link
          to="/"
          className="inline-flex items-center gap-1.5 text-sm font-semibold text-slate-300 hover:text-white transition-colors mb-6"
        >
          <ArrowLeft className="w-4 h-4" />
          Back to Home
        </Link>

        {/* Logo */}
        <div className="text-center mb-6">
          <Link to="/" className="inline-flex items-center gap-2.5 mb-4">
            <img src="/logo.png" alt="Logo" className="w-10 h-10 object-contain" />
            <span className="text-lg font-bold">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
            </span>
          </Link>
          <h1 className="text-2xl md:text-3xl font-extrabold text-white tracking-tight">Welcome Back</h1>
          <p className="text-slate-400 mt-1.5 text-sm">Sign in to continue to your dashboard</p>
        </div>

        {/* Card (Elevated Pure White) */}
        <div className="bg-white text-slate-900 border border-slate-200/90 rounded-3xl p-6 md:p-8 shadow-2xl shadow-black/40">
          <form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
            <div className="space-y-1.5">
              <Label htmlFor="email" className="text-slate-700 text-sm font-semibold">Email Address</Label>
              <Input
                id="email"
                type="email"
                placeholder="name@example.com"
                {...register('email')}
                className={`bg-slate-50 border-slate-200 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:border-cyan-500 h-11 rounded-xl ${errors.email ? 'border-rose-500' : ''}`}
                disabled={isLoading}
              />
              {errors.email && <p className="text-xs text-rose-600 font-medium">{errors.email.message}</p>}
            </div>

            <div className="space-y-1.5">
              <div className="flex items-center justify-between">
                <Label htmlFor="password" className="text-slate-700 text-sm font-semibold">Password</Label>
                <Link to="/forgot-password" className="text-xs text-cyan-700 hover:text-cyan-800 transition-colors font-semibold">
                  Forgot password?
                </Link>
              </div>
              <div className="relative">
                <Input
                  id="password"
                  type={showPassword ? 'text' : 'password'}
                  placeholder="••••••••"
                  {...register('password')}
                  className={`bg-slate-50 border-slate-200 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:border-cyan-500 h-11 pr-10 rounded-xl ${errors.password ? 'border-rose-500' : ''}`}
                  disabled={isLoading}
                />
                <button
                  type="button"
                  onClick={() => setShowPassword(!showPassword)}
                  className="absolute right-1 top-1/2 -translate-y-1/2 p-2 text-slate-400 hover:text-slate-700 transition-colors"
                  tabIndex={-1}
                >
                  {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
                </button>
              </div>
              {errors.password && <p className="text-xs text-rose-600 font-medium">{errors.password.message}</p>}
            </div>

            <Button
              type="submit"
              className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white shadow-md shadow-cyan-600/20 border-0 transition-all font-bold rounded-xl"
              disabled={isLoading}
            >
              {isLoading ? (
```

```
148 |                 <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Signing In...</>
149 |               ) : (
150 |                 <>Sign In <ArrowRight className="ml-2 w-4 h-4" /></>
151 |               )}
152 |             </Button>
153 |           </form>
154 |
155 |           <div className="mt-6 text-center pt-4 border-t border-slate-100">
156 |             <p className="text-sm text-slate-600 font-medium">
157 |               Don't have an account?{' '}
158 |               <a href="/register" onClick={handleGoToRegister} className="text-cyan-700 hover:text-cyan-800 font-bold transiti
on-colors">
159 |                 Create one
160 |               </a>
161 |             </p>
162 |           </div>
163 |         </div>
164 |
165 |         <p className="text-center text-xs text-slate-500 mt-6">
166 |           Ministry of Earth Sciences  •  Capacity Connect
167 |         </p>
168 |       </motion.div>
169 |     </motion.div>
170 |   )
171 | }
172 |
```

src/pages/NotFoundPage.tsx TSX 1.8 KB 38 Lines

```tsx
import React from 'react'
import { motion } from 'framer-motion'
import { Compass } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Link } from 'react-router-dom'

export function NotFoundPage() {
  return (
    <div className="min-h-screen bg-[#040814] flex items-center justify-center p-4 relative overflow-hidden">
      {/* Background ambient lighting */}
      <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[600px] h-[600px] bg-cyan-600/10 rounded-full blur-3xl pointer-events-none" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10 text-center"
      >
        <div className="bg-white border border-slate-200/90 rounded-3xl p-8 md:p-10 shadow-2xl">
          <div className="text-6xl md:text-8xl font-black text-slate-100 mb-2 select-none tracking-tighter">404</div>
          <div className="w-16 h-16 bg-gradient-to-br from-cyan-500 to-blue-600 rounded-2xl flex items-center justify-center mx-auto mb-6 -mt-8 shadow-lg shadow-cyan-500/20 text-white">
            <Compass className="w-8 h-8" />
          </div>
          <h1 className="text-xl md:text-2xl font-black text-slate-900 mb-3 tracking-tight">Page Not Found</h1>
          <p className="text-slate-600 leading-relaxed mb-8 text-sm font-medium">
            The page you're looking for doesn't exist or has been moved.
          </p>
          <Link to="/">
            <Button className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-semibold rounded-xl shadow-lg shadow-cyan-600/20 border-0 transition-all">
              Return to Home
            </Button>
          </Link>
        </div>
      </motion.div>
    </div>
  )
}

```

src/pages/PendingApprovalPage.tsx TSX 2.8 KB 54 Lines

```tsx
import React from 'react'
import { motion } from 'framer-motion'
import { Clock, LogOut, Home } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { useAuth } from '@/hooks/useAuth'
import { useNavigate } from 'react-router-dom'

export function PendingApprovalPage() {
  const { signOut, profile } = useAuth()
  const navigate = useNavigate()

  const handleSignOut = async () => {
    await signOut()
    navigate('/login', { replace: true })
  }

  return (
    <div className="min-h-screen bg-[#040814] text-white flex items-center justify-center p-4 relative overflow-hidden">
      {/* Decorative ambient gradients */}
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-500/20 via-blue-500/15 to-transparent blur-[120px]" />
      <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-amber-500/15 via-orange-500/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10"
      >
        <div className="bg-white text-slate-900 border border-slate-200/90 rounded-3xl p-6 md:p-10 shadow-2xl text-center">
          <div className="w-16 h-16 md:w-20 md:h-20 bg-amber-50 border border-amber-200 rounded-2xl flex items-center justify-center mx-auto mb-4 md:mb-6 shadow-xs">
            <Clock className="w-8 h-8 md:w-10 md:h-10 text-amber-500" />
          </div>
          <h1 className="text-xl md:text-2xl font-bold text-slate-900 mb-2">Approval Pending</h1>
          {profile?.full_name && (
            <p className="text-cyan-700 text-sm font-semibold mb-3">Hello, {profile.full_name}!</p>
          )}
          <p className="text-slate-600 leading-relaxed mb-8 text-sm">
            Your account has been registered and is awaiting administrator approval.
            You'll receive full access once an administrator reviews your application.
          </p>
          <div className="space-y-3">
            <Button onClick={handleSignOut} className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white shadow-md shadow-cyan-600/20 border-0 transition-all font-bold rounded-xl">
              <LogOut className="w-4 h-4 mr-2" /> Sign Out
            </Button>
            <Button variant="outline" onClick={() => navigate('/')} className="w-full h-11 border-slate-200 text-slate-700 hover:bg-slate-50 text-sm font-semibold rounded-xl">
              <Home className="w-4 h-4 mr-2" /> Back to Home
            </Button>
          </div>
        </div>
      </motion.div>
    </div>
  )
}

```

src/pages/ProfilePage.tsx TSX 77.4 KB 1,613 Lines

```tsx
import React, { useState, useEffect, useCallback } from 'react'
import { Link } from 'react-router-dom'
import { useQuery } from '@tanstack/react-query'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { TrainerLayout } from '@/features/trainer/TrainerLayout'
import { DashboardShell } from './Dashboards'
import { motion, AnimatePresence } from 'framer-motion'
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@/components/ui/card'
import { Input } from '@/components/ui/input'
import { Button } from '@/components/ui/button'
import { Textarea } from '@/components/ui/textarea'
import { Badge } from '@/components/ui/badge'
import { Label } from '@/components/ui/label'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select'
import { toast } from 'sonner'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'
import {
  User, Mail, Phone, MapPin, GraduationCap, Briefcase, Sparkles,
  Shield, Key, Globe, Save, Loader2, Plus, X,
  CheckCircle2, Compass, BookOpen, BarChart3, Award, Calendar,
  Building, Layers, Lock, FileText, Check, Code, ExternalLink,
  Trophy, Medal, Star, ChevronRight, ArrowRight, Eye, EyeOff,
  Camera, Upload, Trash2, Image as ImageIcon, Bell, Download, Crop, Sliders
} from 'lucide-react'
import { ImageCropperModal } from '@/components/ui/ImageCropperModal'

function LinkedinIcon({ className = 'w-4 h-4' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path d="M19 3a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h14m-.5 15.5v-5.3a3.26 3.26 0 0 0-3.26-3.26c-.85 0-1.84.52-2.28 1.3v-1.11h-2.79v8.37h2.79v-4.93c0-.77.62-1.4 1.39-1.4a1.4 1.4 0 0 1 1.4 1.4v4.93h2.75M6.88 8.56a1.68 1.68 0 0 0 1.68-1.68c0-.93-.75-1.69-1.68-1.69a1.69 1.69 0 0 0-1.69 1.69c0 .93.76 1.68 1.69 1.68m1.39 9.94v-8.37H5.5v8.37h2.77z" />
    </svg>
  )
}

function GithubIcon({ className = 'w-4 h-4' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path fillRule="evenodd" clipRule="evenodd" d="M12 2C6.477 2 2 6.484 2 12.017c0 4.425 2.865 8.18 6.839 9.504.5.092.682-.217.682-.483 0-.237-.008-.868-.013-1.703-2.782.605-3.369-1.343-3.369-1.343-.454-1.158-1.11-1.466-1.11-1.466-.908-.62.069-.608.069-.608 1.003.07 1.53 1.032 1.53 1.032.892 1.53 2.341 1.088 2.91.832.092-.647.35-1.088.636-1.338-2.22-.253-4.555-1.113-4.555-4.951 0-1.093.39-1.988 1.029-2.688-.103-.253-.446-1.272.098-2.65 0 0 .84-.27 2.75 1.026A9.564 9.564 0 0112 6.844c.85.004 1.705.115 2.504.337 1.909-1.296 2.747-1.027 2.747-1.027.546 1.379.202 2.398.1 2.651.64.7 1.028 1.595 1.028 2.688 0 3.848-2.339 4.695-4.566 4.943.359.309.678.92.678 1.855 0 1.338-.012 2.419-.012 2.747 0 .268.18.58.688.482A10.019 10.019 0 0022 12.017C22 6.484 17.522 2 12 2z" />
    </svg>
  )
}

const fadeUp = {
  hidden: { opacity: 0, y: 15 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.3 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.06 } } }

const SUGGESTED_SKILLS = [
  'Python', 'Atmospheric Modeling', 'WRF Model', 'Numerical Weather Prediction (NWP)',
  'Doppler Weather Radar', 'Satellite Remote Sensing', 'GIS & ArcGIS', 'Ocean State Forecasting',
  'Buoy Data Analysis', 'Climate Data Operators (CDO)', 'MATLAB', 'NetCDF / GRIB',
  'Machine Learning & AI', 'High-Performance Computing (HPC)', 'R Programming', 'Fortran'
]

const SUGGESTED_INTERESTS = [
  'Monsoon Dynamics', 'Tropical Cyclones', 'Ocean Buoy Networks', 'Satellite Oceanography',
  'Flood & Drought Modeling', 'Urban Meteorology', 'Deep Ocean Mission', 'Data Assimilation',
  'Cloud Physics & Radar', 'Renewable Energy Forecasting', 'Coastal Disaster Warning'
]

export function ProfilePage() {
  const { user, profile, refreshProfile } = useAuth()
  const [activeTab, setActiveTab] = useState<'general' | 'qualifications' | 'experience' | 'skills' | 'links' | 'security'>('general')
  const [loading, setLoading] = useState(true)
  const [saving, setSaving] = useState(false)
  const [passwordLoading, setPasswordLoading] = useState(false)
```

```
  const [uploadingAvatar, setUploadingAvatar] = useState(false)
  const fileInputRef = React.useRef<HTMLInputElement>(null)

  // Fetch completed course enrollments for Badges section

  const { data: completedEnrollments = [], isLoading: badgesLoading } = useQuery({
    queryKey: ['trainee-profile-completed-badges', profile?.id, user?.id],
    queryFn: async () => {
      const uid = profile?.id || user?.id
      if (!uid) return []

      let enrollmentsList: any[] = []
      try {
        const { data: enrollmentsData, error } = await supabase
          .from('enrollments')
          .select(`
            id, status, progress_percent, enrolled_at, course_id, user_id,
            course:courses!enrollments_course_id_fkey(
              id, title, course_type, thumbnail_path, duration_minutes, modules,
              trainer:trainers!courses_trainer_id_fkey(full_name)
            )
          `)
          .eq('user_id', uid)
          .order('enrolled_at', { ascending: false })

        if (!error && enrollmentsData && enrollmentsData.length > 0) {
          enrollmentsList = enrollmentsData
        } else {
          // Resilient fallback query

          const { data: fallbackEnrs } = await supabase
            .from('enrollments')
            .select('*')
            .eq('user_id', uid)

          if (fallbackEnrs && fallbackEnrs.length > 0) {
            const courseIds = fallbackEnrs.map((e: any) => e.course_id).filter(Boolean)
            const { data: coursesData } = await supabase
              .from('courses')
              .select('id, title, course_type, thumbnail_path, duration_minutes, modules, trainer:trainers!courses_trainer_id
_fkey(full_name)')
              .in('id', courseIds)

            const courseMap = new Map((coursesData || []).map((c: any) => [c.id, c]))
            enrollmentsList = fallbackEnrs.map((e: any) => ({
              ...e,
              course: courseMap.get(e.course_id)
            }))
          }
        }
      } catch (err) {
        console.warn('Error fetching enrollments for badges:', err)
      }

      // Also check certificates if any

      const { data: certsData } = await supabase
        .from('certificates')
        .select('course_id')
        .eq('user_id', uid)

      const certCourseIds = new Set((certsData || []).map((c: any) => c.course_id))

      const list = (enrollmentsList || []).filter((e: any) => Boolean(e.course || e.course_id))
      return list
    },
    enabled: !!(profile?.id || user?.id) && (profile?.role === 'trainee' || !profile?.role),
  })

  const [downloadingBadgeId, setDownloadingBadgeId] = useState<string | null>(null)

  const handleDownloadBadgeCertificate = async (enr: any) => {
    const course = enr.course
    if (!course) return
    const traineeName = profile?.full_name || user?.email?.split('@')[0] || 'Trainee'
    const percentage = `${enr.progress_percent ?? 100}%`

    setDownloadingBadgeId(enr.id)
    try {
```

```
      const { blob, fileName } = await generateTraineeCertificate(
        course.certificate_template_url,
        {
          traineeName,
          traineeEmail: profile?.email || user?.email,
          traineeId: user!.id,
          courseId: course.id,
          courseTitle: course.title,
          trainerName: course.trainer?.full_name || 'Lead Trainer',
          percentage,
          completedAt: new Date().toISOString(),
        }
      )
      triggerFileDownload(blob, fileName)
      toast.success('Certificate downloaded successfully! 🎉')
    } catch (err: any) {
      console.error('Certificate download error:', err)
      toast.error(err?.message || 'Failed to download certificate')
    } finally {
      setDownloadingBadgeId(null)
    }
  }

  // Profile Form State

  const [form, setForm] = useState({
    full_name: '',
    email: '',
    avatar_path: '',
    phone: '',
    alternate_phone: '',
    department: '',
    designation: '',
    address: '',
    city: '',
    state: '',
    country: 'India',
    pincode: '',
    qualifications: '',
    work_experience: '',
    years_of_experience: '',
    bio: '',
    learning_goals: '',
    availability: 'available',
    linkedin_url: '',
    website_url: '',
    github_url: '',
    skills: [] as string[],
    interests: [] as string[],
  })

  // Skill & Interest input fields

  const [newSkillInput, setNewSkillInput] = useState('')
  const [newInterestInput, setNewInterestInput] = useState('')

  // Password update states

  const [currentPassword, setCurrentPassword] = useState('')
  const [newPassword, setNewPassword] = useState('')
  const [confirmPassword, setConfirmPassword] = useState('')
  const [showCurrentPassword, setShowCurrentPassword] = useState(false)
  const [showNewPassword, setShowNewPassword] = useState(false)
  const [showConfirmPassword, setShowConfirmPassword] = useState(false)

  // Profile Photo Cropper & View Adjustment State

  const [isCropperOpen, setIsCropperOpen] = useState(false)
  const [rawAvatarFile, setRawAvatarFile] = useState<File | null>(null)
  const [rawAvatarUrl, setRawAvatarUrl] = useState<string | null>(null)

  // Load profile data

  const loadProfile = useCallback(async () => {
    if (!user) return
    setLoading(true)
    try {
      // Fetch user data from role tables
```

```
      const [adminRes, trainerRes, traineeRes] = await Promise.all([
        supabase.from('admins').select('*').eq('id', user.id).maybeSingle(),
        supabase.from('trainers').select('*').eq('id', user.id).maybeSingle(),
        supabase.from('trainees').select('*').eq('id', user.id).maybeSingle()
      ])

      const data: any = adminRes.data || trainerRes.data || traineeRes.data || {}

      setForm({
        full_name: data.full_name || profile?.full_name || '',
        email: data.email || user.email || '',
        avatar_path: data.avatar_path || profile?.avatar_path || '',
        phone: data.phone || '',
        alternate_phone: data.alternate_phone || '',
        department: data.department || '',
        designation: data.designation || '',
        address: data.address || '',
        city: data.city || '',
        state: data.state || '',
        country: data.country || 'India',
        pincode: data.pincode || '',
        qualifications: data.qualifications || '',
        work_experience: data.work_experience || '',
        years_of_experience: data.years_of_experience ? String(data.years_of_experience) : '',
        bio: data.bio || '',
        learning_goals: data.learning_goals || '',
        availability: data.availability || 'available',
        linkedin_url: data.linkedin_url || '',
        website_url: data.website_url || '',
        github_url: data.github_url || '',
        skills: Array.isArray(data.skills) ? data.skills : (Array.isArray(data.expertise_areas) ? data.expertise_areas : []),
        interests: Array.isArray(data.interests) ? data.interests : [],
      })
    } catch (err: any) {
      console.error('Error loading profile:', err)
      toast.error('Failed to load profile details')
    } finally {
      setLoading(false)
    }
  }, [user, profile?.full_name, profile?.avatar_path])

  useEffect(() => {
    loadProfile()
  }, [loadProfile])

  const updateField = (field: string, value: any) => {
    setForm(prev => ({ ...prev, [field]: value }))
  }

  // Handle File Selection for Avatar - Opens Cropper / View Adjuster

  const handleAvatarSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file || !user) return

    if (!file.type.startsWith('image/')) {
      toast.error('Please select a valid image file (PNG, JPG, JPEG, WEBP)')
      return
    }

    if (file.size > 10 * 1024 * 1024) {
      toast.error('Image size cannot exceed 10MB limit')
      return
    }

    setRawAvatarFile(file)
    setRawAvatarUrl(null)
    setIsCropperOpen(true)
    if (fileInputRef.current) fileInputRef.current.value = ''
  }

  // Open Adjust View for existing photo - Loads the Original Uncropped Photo

  const handleOpenAdjustView = () => {
    if (!form.avatar_path || !user) return
    const storedRaw = localStorage.getItem(`cc_raw_avatar_${user.id}`)
    setRawAvatarUrl(storedRaw || form.avatar_path)
    setRawAvatarFile(null)
    setIsCropperOpen(true)
```

```
  }

  // Handle Crop Complete

  const handleCropComplete = async (croppedFile: File) => {
    const origFile = rawAvatarFile
    setIsCropperOpen(false)
    setRawAvatarFile(null)
    setRawAvatarUrl(null)
    await uploadCroppedAvatar(croppedFile, origFile)
  }

  // Upload Cropped Photo to Storage and Update Profile

  const uploadCroppedAvatar = async (file: File, originalRawFile?: File | null) => {
    if (!user) return
    setUploadingAvatar(true)
    const fileExt = 'jpg'
    const fileName = `avatar_${user.id}_${Date.now()}.${fileExt}`
    const filePath = `avatars/${fileName}`
    const table = profile?.role === 'trainer' ? 'trainers' : profile?.role === 'trainee' ? 'trainees' : 'admins'

    try {
      // 1. If we have the original raw file, upload & persist it so user can re-adjust losslessly

      if (originalRawFile) {
        const rawExt = originalRawFile.name.split('.').pop() || 'jpg'
        const rawFileName = `raw_avatar_${user.id}_${Date.now()}.${rawExt}`
        const rawFilePath = `avatars/${rawFileName}`

        try {
          const { error: rawUploadErr } = await supabase.storage
            .from('Homepage')
            .upload(rawFilePath, originalRawFile, { upsert: true })

          if (!rawUploadErr) {
            const { data: rawUrlData } = supabase.storage.from('Homepage').getPublicUrl(rawFilePath)
            localStorage.setItem(`cc_raw_avatar_${user.id}`, rawUrlData.publicUrl)
          } else {
            // Fallback DataURL for original

            const reader = new FileReader()
            reader.onload = (e) => {
              if (e.target?.result) {
                localStorage.setItem(`cc_raw_avatar_${user.id}`, e.target.result as string)
              }
            }
            reader.readAsDataURL(originalRawFile)
          }
        } catch {
          // Non-blocking

        }
      }

      let finalAvatarUrl: string | null = null

      // 2. Upload cropped avatar to storage

      const { error: uploadError } = await supabase.storage
        .from('Homepage')
        .upload(filePath, file, { upsert: true, contentType: 'image/jpeg' })

      if (uploadError) {
        const { error: matError } = await supabase.storage
          .from('materials')
          .upload(filePath, file, { upsert: true, contentType: 'image/jpeg' })

        if (!matError) {
          const { data: urlData } = supabase.storage.from('materials').getPublicUrl(filePath)
          finalAvatarUrl = urlData.publicUrl
        } else {
          // DataURL fallback

          await new Promise<void>((resolve) => {
            const reader = new FileReader()
            reader.onload = (event) => {
              finalAvatarUrl = event.target?.result as string
              resolve()
```

```
            }
            reader.readAsDataURL(file)
          })
        }
      } else {
        const { data: urlData } = supabase.storage.from('Homepage').getPublicUrl(filePath)
        finalAvatarUrl = urlData.publicUrl
      }

      if (finalAvatarUrl) {
        setForm(prev => ({ ...prev, avatar_path: finalAvatarUrl! }))

        // Update database table for user's role

        await supabase
          .from(table as any)
          .update({ avatar_path: finalAvatarUrl })
          .eq('id', user.id)

        await refreshProfile()
        toast.success('Profile photo adjusted and updated successfully!')
      }
    } catch (err: any) {
      console.error('Error uploading avatar:', err)
      toast.error('Failed to update profile photo')
    } finally {
      setUploadingAvatar(false)
    }
  }

  // Handle Avatar Photo Removal

  const handleRemoveAvatar = async () => {
    if (!user) return
    setUploadingAvatar(true)
    const table = profile?.role === 'trainer' ? 'trainers' : profile?.role === 'trainee' ? 'trainees' : 'admins'
    try {
      setForm(prev => ({ ...prev, avatar_path: '' }))
      await supabase
        .from(table as any)
        .update({ avatar_path: null })
        .eq('id', user.id)

      await refreshProfile()
      toast.success('Profile photo removed')
    } catch (err: any) {
      toast.error('Failed to remove profile photo')
    } finally {
      setUploadingAvatar(false)
    }
  }

  // Handle Tag Addition

  const handleAddSkill = (skill: string) => {
    const trimmed = skill.trim()
    if (!trimmed) return
    if (!form.skills.includes(trimmed)) {
      setForm(prev => ({ ...prev, skills: [...prev.skills, trimmed] }))
    }
    setNewSkillInput('')
  }

  const handleRemoveSkill = (skillToRemove: string) => {
    setForm(prev => ({ ...prev, skills: prev.skills.filter(s => s !== skillToRemove) }))
  }

  const handleAddInterest = (interest: string) => {
    const trimmed = interest.trim()
    if (!trimmed) return
    if (!form.interests.includes(trimmed)) {
      setForm(prev => ({ ...prev, interests: [...prev.interests, trimmed] }))
    }
    setNewInterestInput('')
  }

  const handleRemoveInterest = (interestToRemove: string) => {
    setForm(prev => ({ ...prev, interests: prev.interests.filter(i => i !== interestToRemove) }))
  }
```

```
442 |
443 |   // Save Profile Handler

444 |   const handleSaveProfile = async (e?: React.FormEvent) => {
445 |     if (e) e.preventDefault()
446 |     if (!user) return
447 |     setSaving(true)
448 |     try {
449 |       const table = profile?.role === 'trainer' ? 'trainers' : profile?.role === 'trainee' ? 'trainees' : 'admins'
450 |       const payload: Record<string, any> = {
451 |         full_name: form.full_name,
452 |         avatar_path: form.avatar_path || null,
453 |         phone: form.phone || null,
454 |         alternate_phone: form.alternate_phone || null,
455 |         department: form.department || null,
456 |         designation: form.designation || null,
457 |         address: form.address || null,
458 |         city: form.city || null,
459 |         state: form.state || null,
460 |         country: form.country || 'India',
461 |         pincode: form.pincode || null,
462 |         qualifications: form.qualifications || null,
463 |         work_experience: form.work_experience || null,
464 |         years_of_experience: form.years_of_experience ? parseInt(form.years_of_experience) : null,
465 |         bio: form.bio || null,
466 |         linkedin_url: form.linkedin_url || null,
467 |         website_url: form.website_url || null,
468 |         github_url: form.github_url || null,
469 |         skills: form.skills,
470 |         interests: form.interests,
471 |       }
472 |
473 |       if (profile?.role === 'trainer') {
474 |         payload.availability = form.availability || 'available'
475 |         payload.expertise_areas = form.skills
476 |       }
477 |
478 |       if (profile?.role === 'trainee') {
479 |         payload.learning_goals = form.learning_goals || null
480 |       }
481 |
482 |       const { error } = await supabase
483 |         .from(table as any)
484 |         .update(payload)
485 |         .eq('id', user.id)
486 |
487 |       if (error) throw error
488 |
489 |       await refreshProfile()
490 |       toast.success('Profile details saved successfully!')
491 |     } catch (err: any) {
492 |       console.error('Error saving profile:', err)
493 |       toast.error(err.message || 'Failed to save profile')
494 |     } finally {
495 |       setSaving(false)
496 |     }
497 |   }
498 |
499 |   // Update Password Handler

500 |   const handleUpdatePassword = async (e: React.FormEvent) => {
501 |     e.preventDefault()
502 |     if (!currentPassword) {
503 |       toast.error('Please enter your current password')
504 |       return
505 |     }
506 |     if (!newPassword) {
507 |       toast.error('Please enter a new password')
508 |       return
509 |     }
510 |     if (newPassword.length < 6) {
511 |       toast.error('New password must be at least 6 characters')
512 |       return
513 |     }
514 |     if (newPassword === currentPassword) {
515 |       toast.error('New password must be different from your current password')
516 |       return
517 |     }
518 |     if (newPassword !== confirmPassword) {
```

```
519 |       toast.error('New passwords do not match')
520 |       return
521 |     }
522 |
523 |     const userEmail = user?.email || form.email
524 |     if (!userEmail) {
525 |       toast.error('Account email not found')
526 |       return
527 |     }
528 |
529 |     setPasswordLoading(true)
530 |     try {
531 |       // 1. Verify current password by signing in

532 |       const { error: verifyErr } = await supabase.auth.signInWithPassword({
533 |         email: userEmail,
534 |         password: currentPassword,
535 |       })
536 |       if (verifyErr) {
537 |         toast.error('Current password is incorrect. Please check and try again.')
538 |         setPasswordLoading(false)
539 |         return
540 |       }
541 |
542 |       // 2. Update to new password

543 |       const { error: updateErr } = await supabase.auth.updateUser({ password: newPassword })
544 |       if (updateErr) throw updateErr
545 |
546 |       // 3. Keep role tables in sync if needed

547 |       try {
548 |         const table = profile?.role === 'trainer' ? 'trainers' : profile?.role === 'trainee' ? 'trainees' : 'admins'
549 |         await (supabase.from(table as any) as any).update({ password: newPassword }).eq('id', user!.id)
550 |       } catch {
551 |         // non-fatal

552 |       }
553 |
554 |       toast.success('Password updated successfully! 🔒')
555 |       setCurrentPassword('')
556 |       setNewPassword('')
557 |       setConfirmPassword('')
558 |     } catch (err: any) {
559 |       toast.error(err.message || 'Failed to update password')
560 |     } finally {
561 |       setPasswordLoading(false)
562 |     }
563 |   }
564 |
565 |   if (loading) {
566 |     const loadingView = (
567 |       <div className="flex flex-col items-center justify-center py-28 space-y-4">
568 |         <div className="w-10 h-10 border-3 border-cyan-600 border-t-transparent rounded-full animate-spin" />
569 |         <p className="text-sm font-semibold text-slate-500">Loading profile data...</p>
570 |       </div>
571 |     )
572 |     if (profile?.role === 'trainer') return <TrainerLayout>{loadingView}</TrainerLayout>
573 |     return (
574 |       <DashboardShell title="Profile" icon={User}>
575 |         {loadingView}
576 |       </DashboardShell>
577 |     )
578 |   }
579 |
580 |   const mainContent = (
581 |     <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-5xl mx-auto space-y-6 pb-12">
582 |       {/* Header Profile Hero Card */}
583 |       <motion.div variants={fadeUp}>
584 |         <div className="relative overflow-hidden rounded-3xl bg-gradient-to-r from-[#040814] via-[#081329] to-[#040d21] text-
white p-6 sm:p-8 border border-cyan-500/30 shadow-xl">
585 |           <div className="absolute top-0 right-0 w-96 h-96 bg-gradient-to-br from-cyan-500/15 via-blue-500/10 to-transparent
rounded-full blur-3xl pointer-events-none" />
586 |
587 |           <div className="relative z-10 flex flex-col sm:flex-row sm:items-center justify-between gap-6">
588 |             <div className="flex items-center gap-5">
589 |               <div className="relative group/avatar w-20 h-20 rounded-2xl bg-gradient-to-br from-cyan-500 via-sky-500 to-blue
-600 flex items-center justify-center text-white text-3xl font-black shadow-lg shadow-cyan-500/30 ring-4 ring-white/10 shrink-0 over
flow-hidden">
```

```
590 |                 {form.avatar_path ? (
591 |                   <img src={form.avatar_path} alt={form.full_name} className="w-full h-full object-cover" />
592 |                 ) : (
593 |                   form.full_name?.charAt(0)?.toUpperCase() || <User className="w-10 h-10" />
594 |                 )}
595 |                 <button
596 |                   type="button"
597 |                   onClick={() => fileInputRef.current?.click()}
598 |                   disabled={uploadingAvatar}
599 |                   className="absolute inset-0 bg-black/60 opacity-0 group-hover/avatar:opacity-100 flex flex-col items-center
justify-center text-white text-[10px] font-bold transition-opacity cursor-pointer backdrop-blur-2xs"
600 |                   title="Change profile photo"
601 |                 >
602 |                   {uploadingAvatar ? <Loader2 className="w-5 h-5 animate-spin" /> : <Camera className="w-5 h-5 mb-0.5" />}
603 |                   <span>Edit</span>
604 |                 </button>
605 |               </div>
606 |               <div className="min-w-0">
607 |                 <div className="flex flex-wrap items-center gap-2 mb-1">
608 |                   <h1 className="text-2xl sm:text-3xl font-black text-white tracking-tight truncate">
609 |                     {form.full_name || 'My Profile'}
610 |                   </h1>
611 |                   <span className="text-xs px-2.5 py-0.5 rounded-full border bg-cyan-500/20 text-cyan-300 border-cyan-500/40
capitalize font-bold">
612 |                     {profile?.role?.replace('_', ' ')}
613 |                   </span>
614 |                 </div>
615 |                 <p className="text-slate-300 text-xs sm:text-sm font-medium flex items-center gap-2 truncate">
616 |                   <Mail className="w-3.5 h-3.5 text-cyan-400 shrink-0" /> {form.email || user?.email}
617 |                 </p>
618 |                 {(form.designation || form.department) && (
619 |                   <p className="text-slate-400 text-xs mt-1 flex items-center gap-2">
620 |                     <Building className="w-3.5 h-3.5 text-cyan-400 shrink-0" />
621 |                     {[form.designation, form.department].filter(Boolean).join(' • ')}
622 |                   </p>
623 |                 )}
624 |               </div>
625 |             </div>
626 |
627 |             {/* Verified Account Status Badge */}
628 |             <div className="flex items-center gap-3.5 bg-slate-900/80 backdrop-blur-md px-5 py-3.5 rounded-2xl border border-
emerald-500/30 shadow-lg shadow-emerald-950/40 shrink-0">
629 |               <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-emerald-400 via-teal-500 to-emerald-600 flex items-
center justify-center text-white shadow-md shadow-emerald-500/30 shrink-0">
630 |                 <CheckCircle2 className="w-5 h-5 text-white" />
631 |               </div>
632 |               <div>
633 |                 <div className="flex items-center gap-2">
634 |                   <span className="text-sm font-bold text-white tracking-wide">
635 |                     {profile?.approval_status === 'rejected' ? 'Action Required' : profile?.approval_status === 'pending' ?
'Verification Pending' : 'Verified Member'}
636 |                   </span>
637 |                 </div>
638 |                 <p className="text-[11px] text-slate-400 mt-0.5 font-medium">
639 |                   Official Capacity Connect Member
640 |                 </p>
641 |               </div>
642 |             </div>
643 |           </div>
644 |         </div>
645 |       </motion.div>
646 |
647 |       {/* Trainee Course Badges Section */}
648 |       {profile?.role === 'trainee' && (
649 |         <motion.div variants={fadeUp}>
650 |           <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
651 |             <CardHeader className="p-5 sm:p-6 border-b border-slate-100 bg-gradient-to-r from-amber-500/10 via-cyan-500/5 to-
transparent flex flex-col sm:flex-row sm:items-center justify-between gap-4">
652 |               <div className="flex items-center gap-3">
653 |                 <div className="w-10 h-10 rounded-2xl bg-gradient-to-br from-amber-400 via-amber-500 to-orange-500 flex items
-center justify-center text-white shadow-md shadow-amber-500/25 shrink-0">
654 |                   <Trophy className="w-5 h-5" />
655 |                 </div>
656 |                 <div>
657 |                   <div className="flex items-center gap-2">
658 |                     <CardTitle className="text-base sm:text-lg font-black text-slate-900">
659 |                       Earned Course Badges
660 |                     </CardTitle>
661 |                     <Badge className="bg-amber-100 text-amber-900 border-amber-300 text-[10px] font-extrabold px-2">
```

```
                      {completedEnrollments.length} Earned
                    </Badge>
                  </div>
                  <CardDescription className="text-xs text-slate-500 mt-0.5">
                    Official milestone achievement credentials awarded for 100% course curriculum completion.
                  </CardDescription>
                </div>
              </div>

              <Link
                to="/trainee/my-learning"
                className="text-xs font-bold text-cyan-700 hover:text-cyan-800 flex items-center gap-1 shrink-0 group transition-colors"
              >
                <span>View All in My Learning</span>
                <ChevronRight className="w-4 h-4 group-hover:translate-x-0.5 transition-transform" />
              </Link>
            </CardHeader>

            <CardContent className="p-5 sm:p-6">
              {badgesLoading ? (
                <div className="flex items-center justify-center py-8 text-slate-400 gap-2">
                  <Loader2 className="w-5 h-5 animate-spin text-cyan-600" />
                  <span className="text-xs font-semibold">Loading badges...</span>
                </div>
              ) : completedEnrollments.length > 0 ? (
                <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
                  {completedEnrollments.map((enr: any, idx: number) => {
                    const course = enr.course
                    if (!course) return null

                    const colorThemes = [
                      {
                        bg: 'from-amber-500/15 via-orange-500/5 to-amber-500/10 border-amber-200/90 hover:border-amber-400',
                        medal: 'from-amber-400 via-amber-500 to-orange-500 text-white shadow-amber-500/30',
                        tag: 'bg-amber-100 text-amber-900 border-amber-300',
                        icon: Trophy,
                        tier: 'Mastery Badge',
                      },
                      {
                        bg: 'from-emerald-500/15 via-teal-500/5 to-emerald-500/10 border-emerald-200/90 hover:border-emerald-400',
                        medal: 'from-emerald-400 via-teal-500 to-emerald-600 text-white shadow-emerald-500/30',
                        tag: 'bg-emerald-100 text-emerald-900 border-emerald-300',
                        icon: Award,
                        tier: 'Certified Specialist',
                      },
                      {
                        bg: 'from-cyan-500/15 via-blue-500/5 to-cyan-500/10 border-cyan-200/90 hover:border-cyan-400',
                        medal: 'from-cyan-400 via-sky-500 to-blue-600 text-white shadow-cyan-500/30',
                        tag: 'bg-cyan-100 text-cyan-900 border-cyan-300',
                        icon: Medal,
                        tier: 'Graduate Badge',
                      },
                      {
                        bg: 'from-purple-500/15 via-indigo-500/5 to-purple-500/10 border-purple-200/90 hover:border-purple-400',
                        medal: 'from-purple-400 via-indigo-500 to-violet-600 text-white shadow-purple-500/30',
                        tag: 'bg-purple-100 text-purple-900 border-purple-300',
                        icon: Star,
                        tier: 'Excellence Award',
                      },
                    ]
                    const theme = colorThemes[idx % colorThemes.length]
                    const IconComp = theme.icon

                    return (
                      <div
                        key={enr.id}
                        className={`p-4 rounded-2xl border bg-gradient-to-br transition-all duration-200 shadow-xs hover:shadow-md flex items-start gap-3.5 group relative ${theme.bg}`}
                      >
                        <div className={`w-12 h-12 rounded-2xl bg-gradient-to-br flex items-center justify-center shadow-md shrink-0 group-hover:scale-105 transition-transform ${theme.medal}`}>
                          <IconComp className="w-6 h-6" />
                        </div>

                        <div className="min-w-0 flex-1 space-y-1">
                          <div className="flex items-center gap-1.5 flex-wrap">
```

```
                            <span className={`text-[10px] font-extrabold px-2 py-0.5 rounded-full border ${theme.tag}`}>
                              {theme.tier}
                            </span>
                            {course.department && (
                              <span className="text-[10px] text-slate-500 font-semibold truncate">
                                • {course.department}
                              </span>
                            )}
                          </div>

                          <h4 className="text-xs font-bold text-slate-900 line-clamp-2 leading-snug group-hover:text-cyan-700 transition-colors">
                            {course.title}
                          </h4>

                          <div className="flex items-center justify-between pt-1 gap-2">
                            <span className={`text-[11px] font-extrabold flex items-center gap-1 ${enr.status === 'completed' || (enr.progress_percent ?? 0) >= 100
                                ? 'text-emerald-600'
                                : 'text-cyan-700'
                              }`}>
                              <CheckCircle2 className="w-3.5 h-3.5" />
                              {enr.status === 'completed' || (enr.progress_percent ?? 0) >= 100
                                ? 'Completed'
                                : `${enr.progress_percent ?? 0}% In Progress`}
                            </span>

                            <div className="flex items-center gap-1.5">
                              <button
                                type="button"
                                onClick={() => handleDownloadBadgeCertificate(enr)}
                                disabled={downloadingBadgeId === enr.id}
                                className="px-2.5 py-1 rounded-lg bg-emerald-600 hover:bg-emerald-700 text-white text-[10px] font-bold flex items-center gap-1 transition-all cursor-pointer shadow-xs disabled:opacity-50"
                                title="Download Certificate"
                              >
                                {downloadingBadgeId === enr.id ? (
                                  <Loader2 className="w-3 h-3 animate-spin" />
                                ) : (
                                  <Award className="w-3 h-3 text-amber-300" />
                                )}
                                <span>Certificate</span>
                              </button>

                              <Link
                                to={`/trainee/courses/${course.id}`}
                                className="text-[11px] font-bold text-slate-600 hover:text-cyan-600 flex items-center gap-0.5"
                              >
                                View <ChevronRight className="w-3 h-3" />
                              </Link>
                            </div>
                          </div>
                        </div>
                      </div>
                    )
                  })}
                </div>
              ) : (
                <div className="p-6 rounded-2xl border border-dashed border-slate-200 bg-slate-50/60 text-center space-y-2">
                  <div className="flex justify-center items-center gap-2 text-slate-300">
                    <Trophy className="w-8 h-8 opacity-40" />
                    <Award className="w-9 h-9 opacity-60 text-amber-400" />
                    <Star className="w-8 h-8 opacity-40" />
                  </div>
                  <div>
                    <p className="text-sm font-bold text-slate-800">No Course Badges Earned Yet</p>
                    <p className="text-xs text-slate-500 mt-1 max-w-md mx-auto">
                      Complete all modules in your enrolled courses and score at least 80% on module quizzes to unlock your official skill achievement badges here.
                    </p>
                  </div>
                </div>
              )}
            </CardContent>
          </Card>
        </motion.div>
      )}

```

```
      {/* Tabs Navigation Bar */}
      <motion.div variants={fadeUp}>
        <div className="flex items-center gap-1.5 p-1.5 bg-slate-200/70 backdrop-blur-sm rounded-2xl border border-slate-200 overflow-x-auto scrollbar-none shadow-xs">
          {[
            { id: 'general', label: 'Personal & Contact', icon: User },
            { id: 'qualifications', label: 'Qualifications', icon: GraduationCap },
            { id: 'experience', label: 'Experience & Bio', icon: Briefcase },
            { id: 'skills', label: 'Skills & Interests', icon: Sparkles },
            { id: 'links', label: 'Social & Links', icon: Globe },
            { id: 'security', label: 'Security & Settings', icon: Shield },
          ].map(tab => {
            const isActive = activeTab === tab.id
            return (
              <button
                key={tab.id}
                onClick={() => setActiveTab(tab.id as any)}
                className={`flex items-center gap-2 px-4 py-2.5 rounded-xl text-xs font-bold whitespace-nowrap transition-all ${isActive
                    ? 'bg-white text-slate-900 shadow-sm border border-slate-200/80'
                    : 'text-slate-600 hover:text-slate-900 hover:bg-white/50'
                  }`}
              >
                <tab.icon className={`w-4 h-4 ${isActive ? 'text-cyan-600' : 'text-slate-400'}`} />
                <span>{tab.label}</span>
              </button>
            )
          })}
        </div>
      </motion.div>

      {/* Tab Panels */}
      <AnimatePresence mode="wait">
        {/* 1. Personal & Contact Tab */}
        {activeTab === 'general' && (
          <motion.div key="general" variants={fadeUp} initial="hidden" animate="visible" exit="hidden">
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <User className="w-5 h-5 text-cyan-600" /> Personal & Contact Details
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Manage your personal contact info, official affiliations, and address.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-6">
                {/* Profile Photo Uploader Section */}
                <div className="p-4 sm:p-5 rounded-2xl bg-slate-50 border border-slate-200/90 flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4">
                  <div className="flex items-center gap-4">
                    <div className="relative w-16 h-16 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white text-2xl font-black shadow-md shrink-0 ring-2 ring-cyan-500/30 overflow-hidden">
                      {form.avatar_path ? (
                        <img src={form.avatar_path} alt={form.full_name} className="w-full h-full object-cover" />
                      ) : (
                        form.full_name?.charAt(0)?.toUpperCase() || <User className="w-8 h-8" />
                      )}
                      {uploadingAvatar && (
                        <div className="absolute inset-0 bg-black/60 flex items-center justify-center">
                          <Loader2 className="w-5 h-5 text-white animate-spin" />
                        </div>
                      )}
                    </div>
                    <div>
                      <h4 className="text-sm font-bold text-slate-900">Profile Photo</h4>
                      <p className="text-xs text-slate-500 mt-0.5">
                        PNG, JPG, or WEBP (Max 10MB). High-resolution square image recommended.
                      </p>
                    </div>
                  </div>

                  <div className="flex flex-wrap items-center gap-2 w-full sm:w-auto">
                    <input
                      ref={fileInputRef}
                      type="file"
                      accept="image/*"
                      className="hidden"
                      onChange={handleAvatarSelect}
                    />
```

```
                    <Button
                      type="button"
                      variant="outline"
                      size="sm"
                      onClick={() => fileInputRef.current?.click()}
                      disabled={uploadingAvatar}
                      className="border-cyan-200 bg-cyan-50 hover:bg-cyan-100 text-cyan-800 text-xs font-bold rounded-xl h-9
flex-1 sm:flex-initial cursor-pointer"
                    >
                      {uploadingAvatar ? (
                        <Loader2 className="w-3.5 h-3.5 mr-1.5 animate-spin" />
                      ) : (
                        <Upload className="w-3.5 h-3.5 mr-1.5 text-cyan-600" />
                      )}
                      {form.avatar_path ? 'Change Photo' : 'Upload Photo'}
                    </Button>

                    {form.avatar_path && (
                      <Button
                        type="button"
                        variant="outline"
                        size="sm"
                        onClick={handleOpenAdjustView}
                        disabled={uploadingAvatar}
                        className="border-slate-300 bg-white hover:bg-slate-100 text-slate-700 text-xs font-bold rounded-xl h
-9 px-3 cursor-pointer shadow-xs"
                      >
                        <Crop className="w-3.5 h-3.5 mr-1 text-cyan-600" /> Adjust View
                      </Button>
                    )}

                    {form.avatar_path && (
                      <Button
                        type="button"
                        variant="ghost"
                        size="sm"
                        onClick={handleRemoveAvatar}
                        disabled={uploadingAvatar}
                        className="text-rose-600 hover:text-rose-700 hover:bg-rose-50 text-xs font-semibold rounded-xl h-9 px
-3 cursor-pointer"
                      >
                        <Trash2 className="w-3.5 h-3.5 mr-1" /> Remove
                      </Button>
                    )}
                  </div>
                </div>

                <div className="grid grid-cols-1 sm:grid-cols-2 gap-5">
                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">Full Name *</Label>
                    <Input
                      value={form.full_name}
                      onChange={e => updateField('full_name', e.target.value)}
                      placeholder="e.g. Dr. Rajesh Kumar"
                      className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                    />
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">Email Address (Registered)</Label>
                    <Input
                      value={form.email}
                      disabled
                      className="bg-slate-100 border-slate-200 rounded-xl h-10 text-slate-500 cursor-not-allowed"
                    />
                  </div>
                </div>

                <div className="grid grid-cols-1 sm:grid-cols-2 gap-5">
                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">Primary Phone / Mobile Number *</Label>
                    <div className="relative">
                      <Phone className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
                      <Input
                        value={form.phone}
                        onChange={e => updateField('phone', e.target.value)}
                        placeholder="+91 98765 43210"
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 pl-9 text-slate-900"
                      />
                    </div>
```

```
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-xs font-bold text-slate-700">Alternate Contact / WhatsApp</Label>
                      <div className="relative">
                        <Phone className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
                        <Input
                          value={form.alternate_phone}
                          onChange={e => updateField('alternate_phone', e.target.value)}
                          placeholder="+91 98765 00000"
                          className="bg-slate-50 border-slate-200 rounded-xl h-10 pl-9 text-slate-900"
                        />
                      </div>
                    </div>
                  </div>

                  <div className="grid grid-cols-1 sm:grid-cols-2 gap-5 pt-2 border-t border-slate-100">
                    <div className="space-y-1.5">
                      <Label className="text-xs font-bold text-slate-700">Department / Institute / Ministry</Label>
                      <Input
                        value={form.department}
                        onChange={e => updateField('department', e.target.value)}
                        placeholder="e.g. INCOIS / IMD / NCMRWF / IITM"
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                      />
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-xs font-bold text-slate-700">Designation / Role Title</Label>
                      <Input
                        value={form.designation}
                        onChange={e => updateField('designation', e.target.value)}
                        placeholder="e.g. Scientist-D / Meteorologist / Field Trainee"
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                      />
                    </div>
                  </div>

                  <div className="space-y-3 pt-2 border-t border-slate-100">
                    <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                      <MapPin className="w-4 h-4 text-cyan-600" /> Complete Address & Location
                    </Label>
                    <Textarea
                      value={form.address}
                      onChange={e => updateField('address', e.target.value)}
                      rows={2}
                      placeholder="Street address, building, campus, postal location..."
                      className="bg-slate-50 border-slate-200 rounded-xl text-slate-900 resize-none"
                    />
                    <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
                      <div className="space-y-1">
                        <Label className="text-[11px] font-semibold text-slate-600">City</Label>
                        <Input
                          value={form.city}
                          onChange={e => updateField('city', e.target.value)}
                          placeholder="e.g. Hyderabad"
                          className="bg-slate-50 border-slate-200 rounded-xl h-9 text-xs text-slate-900"
                        />
                      </div>
                      <div className="space-y-1">
                        <Label className="text-[11px] font-semibold text-slate-600">State</Label>
                        <Input
                          value={form.state}
                          onChange={e => updateField('state', e.target.value)}
                          placeholder="e.g. Telangana"
                          className="bg-slate-50 border-slate-200 rounded-xl h-9 text-xs text-slate-900"
                        />
                      </div>
                      <div className="space-y-1">
                        <Label className="text-[11px] font-semibold text-slate-600">Postal PIN Code</Label>
                        <Input
                          value={form.pincode}
                          onChange={e => updateField('pincode', e.target.value)}
                          placeholder="e.g. 500090"
                          className="bg-slate-50 border-slate-200 rounded-xl h-9 text-xs text-slate-900"
                        />
                      </div>
                      <div className="space-y-1">
                        <Label className="text-[11px] font-semibold text-slate-600">Country</Label>
                        <Input
                          value={form.country}
```

```
                        onChange={e => updateField('country', e.target.value)}
                        placeholder="India"
                        className="bg-slate-50 border-slate-200 rounded-xl h-9 text-xs text-slate-900"
                      />
                    </div>
                  </div>
                </div>

                <div className="flex justify-end pt-4">
                  <Button
                    onClick={() => handleSaveProfile()}
                    disabled={saving}
                    className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold r
ounded-xl px-6 h-11 shadow-sm"
                  >
                    {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                    Save Changes
                  </Button>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        {/* 2. Qualifications Tab */}
        {activeTab === 'qualifications' && (
          <motion.div key="qualifications" variants={fadeUp} initial="hidden" animate="visible" exit="hidden">
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <GraduationCap className="w-5 h-5 text-cyan-600" /> Academic & Professional Qualifications
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Add your degrees, certifications, specializations, and institutional credentials.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-6">
                <div className="space-y-2">
                  <Label className="text-xs font-bold text-slate-700">
                    Degrees, Diplomas & Institutional Certifications
                  </Label>
                  <Textarea
                    value={form.qualifications}
                    onChange={e => updateField('qualifications', e.target.value)}
                    rows={5}
                    placeholder="e.g.
• Ph.D. in Atmospheric Sciences - IIT Delhi (2020)
• M.Tech in Meteorology & Oceanography - Andhra University (2015)
• WMO Level 2 Meteorological Forecaster Certification (2018)
• B.Sc. in Physics & Mathematics - University of Hyderabad (2013)"
                    className="bg-slate-50 border-slate-200 rounded-xl text-slate-900 leading-relaxed font-sans"
                  />
                  <p className="text-[11px] text-slate-500 font-medium">
                    Tip: Mention degree name, university / institute, year of graduation, and special honors.
                  </p>
                </div>

                <div className="flex justify-end pt-4">
                  <Button
                    onClick={() => handleSaveProfile()}
                    disabled={saving}
                    className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold r
ounded-xl px-6 h-11 shadow-sm"
                  >
                    {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                    Save Qualifications
                  </Button>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        {/* 3. Experience & Bio Tab */}
        {activeTab === 'experience' && (
          <motion.div key="experience" variants={fadeUp} initial="hidden" animate="visible" exit="hidden">
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
```

```
                    <Briefcase className="w-5 h-5 text-cyan-600" /> Work Experience & Biography
                  </CardTitle>
                  <CardDescription className="text-xs text-slate-500">
                    Highlight your scientific track record, years of service, and professional summary.
                  </CardDescription>
                </CardHeader>
                <CardContent className="p-6 space-y-6">
                  <div className="grid grid-cols-1 sm:grid-cols-2 gap-5">
                    <div className="space-y-1.5">
                      <Label className="text-xs font-bold text-slate-700">Total Years of Experience</Label>
                      <Input
                        type="number"
                        value={form.years_of_experience}
                        onChange={e => updateField('years_of_experience', e.target.value)}
                        placeholder="e.g. 8"
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                      />
                    </div>

                    {profile?.role === 'trainer' && (
                      <div className="space-y-1.5">
                        <Label className="text-xs font-bold text-slate-700">Trainer Availability</Label>
                        <Select value={form.availability} onValueChange={v => updateField('availability', v)}>
                          <SelectTrigger className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900 font-semibold">
                            <SelectValue />
                          </SelectTrigger>
                          <SelectContent>
                            <SelectItem value="available">🟢 Available for Sessions</SelectItem>
                            <SelectItem value="busy">🟡 Busy with Ongoing Cohorts</SelectItem>
                            <SelectItem value="unavailable">🔴 Currently Unavailable</SelectItem>
                          </SelectContent>
                        </Select>
                      </div>
                    )}
                  </div>

                  <div className="space-y-2">
                    <Label className="text-xs font-bold text-slate-700">Professional Summary / Biography</Label>
                    <Textarea
                      value={form.bio}
                      onChange={e => updateField('bio', e.target.value)}
                      rows={4}
                      placeholder="Write a brief professional overview outlining your research interests, teaching methodology,
or scientific contributions..."
                      className="bg-slate-50 border-slate-200 rounded-xl text-slate-900 leading-relaxed font-sans"
                    />
                  </div>

                  <div className="space-y-2 pt-2 border-t border-slate-100">
                    <Label className="text-xs font-bold text-slate-700">Detailed Work Experience & Key Projects</Label>
                    <Textarea
                      value={form.work_experience}
                      onChange={e => updateField('work_experience', e.target.value)}
                      rows={4}
                      placeholder="e.g.
• Senior Research Scientist - INCOIS (2021 - Present): Leading numerical ocean modeling and storm surge advisory pipelines.
• Meteorological Officer - IMD (2016 - 2021): Handled Doppler Radar observations and daily aviation forecasting."
                      className="bg-slate-50 border-slate-200 rounded-xl text-slate-900 leading-relaxed font-sans"
                    />
                  </div>

                  {profile?.role === 'trainee' && (
                    <div className="space-y-2 pt-2 border-t border-slate-100">
                      <Label className="text-xs font-bold text-slate-700">Learning Goals & Aspirations</Label>
                      <Textarea
                        value={form.learning_goals}
                        onChange={e => updateField('learning_goals', e.target.value)}
                        rows={3}
                        placeholder="What specific skills or masterclasses are you aiming to master on Capacity Connect?"
                        className="bg-slate-50 border-slate-200 rounded-xl text-slate-900"
                      />
                    </div>
                  )}

                  <div className="flex justify-end pt-4">
                    <Button
                      onClick={() => handleSaveProfile()}
                      disabled={saving}
                      className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold r
```

```
ounded-xl px-6 h-11 shadow-sm"
                  >
                    {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                    Save Experience
                  </Button>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        {/* 4. Skills & Interests Tab */}
        {activeTab === 'skills' && (
          <motion.div key="skills" variants={fadeUp} initial="hidden" animate="visible" exit="hidden" className="space-y-6">
            {/* Skills Card */}
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <Sparkles className="w-5 h-5 text-cyan-600" /> Technical & Domain Skills
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Tag your technical software, meteorological modeling frameworks, and observational tools.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-4">
                {/* Active Skills Pills */}
                <div className="flex flex-wrap gap-2 min-h-[44px] p-3 rounded-2xl bg-slate-50 border border-slate-200">
                  {form.skills.length > 0 ? (
                    form.skills.map(skill => (
                      <span
                        key={skill}
                        className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-100 text-cyan-800 border b
order-cyan-300 text-xs font-bold shadow-xs animate-in fade-in"
                      >
                        {skill}
                        <button
                          type="button"
                          onClick={() => handleRemoveSkill(skill)}
                          className="w-4 h-4 rounded-full hover:bg-cyan-200 flex items-center justify-center text-cyan-900 tr
ansition-colors"
                        >
                          <X className="w-3 h-3" />
                        </button>
                      </span>
                    ))
                  ) : (
                    <span className="text-xs text-slate-400 font-medium py-1">
                      No skills added yet. Type below or pick suggestions.
                    </span>
                  )}
                </div>

                {/* Add Custom Skill Input */}
                <div className="flex gap-2">
                  <Input
                    value={newSkillInput}
                    onChange={e => setNewSkillInput(e.target.value)}
                    onKeyDown={e => {
                      if (e.key === 'Enter') {
                        e.preventDefault()
                        handleAddSkill(newSkillInput)
                      }
                    }}
                    placeholder="Type a skill and press Enter..."
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                  />
                  <Button
                    type="button"
                    onClick={() => handleAddSkill(newSkillInput)}
                    className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold rounded-xl px-4 shrink-0"
                  >
                    <Plus className="w-4 h-4 mr-1" /> Add Skill
                  </Button>
                </div>

                {/* Quick Add Suggestions */}
                <div className="space-y-2 pt-2">
                  <Label className="text-[11px] font-bold text-slate-500 uppercase tracking-wider">
                    Recommended Suggestions:
```

```
                  </Label>
                  <div className="flex flex-wrap gap-1.5">
                    {SUGGESTED_SKILLS.filter(s => !form.skills.includes(s)).slice(0, 10).map(s => (
                      <button
                        key={s}
                        type="button"
                        onClick={() => handleAddSkill(s)}
                        className="px-2.5 py-1 rounded-lg bg-slate-100 hover:bg-cyan-50 hover:text-cyan-700 hover:border-cyan
-200 border border-slate-200 text-xs text-slate-600 font-medium transition-all"
                      >
                        + {s}
                      </button>
                    ))}
                  </div>
                </div>
              </CardContent>
            </Card>

            {/* Interests Card */}
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <Compass className="w-5 h-5 text-indigo-600" /> Research & Learning Interests
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Select domain themes, meteorological specializations, or earth science topics.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-4">
                {/* Active Interests Pills */}
                <div className="flex flex-wrap gap-2 min-h-[44px] p-3 rounded-2xl bg-slate-50 border border-slate-200">
                  {form.interests.length > 0 ? (
                    form.interests.map(interest => (
                      <span
                        key={interest}
                        className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-indigo-100 text-indigo-800 bord
er border-indigo-300 text-xs font-bold shadow-xs animate-in fade-in"
                      >
                        {interest}
                        <button
                          type="button"
                          onClick={() => handleRemoveInterest(interest)}
                          className="w-4 h-4 rounded-full hover:bg-indigo-200 flex items-center justify-center text-indigo-90
0 transition-colors"
                        >
                          <X className="w-3 h-3" />
                        </button>
                      </span>
                    ))
                  ) : (
                    <span className="text-xs text-slate-400 font-medium py-1">
                      No interests tagged yet. Type below or pick suggestions.
                    </span>
                  )}
                </div>

                {/* Add Custom Interest Input */}
                <div className="flex gap-2">
                  <Input
                    value={newInterestInput}
                    onChange={e => setNewInterestInput(e.target.value)}
                    onKeyDown={e => {
                      if (e.key === 'Enter') {
                        e.preventDefault()
                        handleAddInterest(newInterestInput)
                      }
                    }}
                    placeholder="Type an interest area and press Enter..."
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                  />
                  <Button
                    type="button"
                    onClick={() => handleAddInterest(newInterestInput)}
                    className="bg-indigo-600 hover:bg-indigo-700 text-white font-bold rounded-xl px-4 shrink-0"
                  >
                    <Plus className="w-4 h-4 mr-1" /> Add Interest
                  </Button>
                </div>

```

```
                {/* Quick Add Suggestions */}
                <div className="space-y-2 pt-2">
                  <Label className="text-[11px] font-bold text-slate-500 uppercase tracking-wider">
                    Suggested Earth Science Topics:
                  </Label>
                  <div className="flex flex-wrap gap-1.5">
                    {SUGGESTED_INTERESTS.filter(i => !form.interests.includes(i)).slice(0, 8).map(i => (
                      <button
                        key={i}
                        type="button"
                        onClick={() => handleAddInterest(i)}
                        className="px-2.5 py-1 rounded-lg bg-slate-100 hover:bg-indigo-50 hover:text-indigo-700 hover:border-indigo-200 border border-slate-200 text-xs text-slate-600 font-medium transition-all"
                      >
                        + {i}
                      </button>
                    ))}
                  </div>
                </div>

                <div className="flex justify-end pt-4 border-t border-slate-100">
                  <Button
                    onClick={() => handleSaveProfile()}
                    disabled={saving}
                    className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold rounded-xl px-6 h-11 shadow-sm"
                  >
                    {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                    Save Skills & Interests
                  </Button>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        {/* 5. Social & Links Tab */}
        {activeTab === 'links' && (
          <motion.div key="links" variants={fadeUp} initial="hidden" animate="visible" exit="hidden">
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <Globe className="w-5 h-5 text-cyan-600" /> Professional Profiles & External Links
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Connect your LinkedIn, GitHub, ResearchGate, or personal website.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-5">
                <div className="space-y-1.5">
                  <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                    <LinkedinIcon className="w-4 h-4 text-blue-600" /> LinkedIn Profile
                  </Label>
                  <Input
                    value={form.linkedin_url}
                    onChange={e => updateField('linkedin_url', e.target.value)}
                    placeholder="https://linkedin.com/in/username"
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                  />
                </div>

                <div className="space-y-1.5">
                  <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                    <Globe className="w-4 h-4 text-emerald-600" /> ResearchGate / Google Scholar / Portfolio URL
                  </Label>
                  <Input
                    value={form.website_url}
                    onChange={e => updateField('website_url', e.target.value)}
                    placeholder="https://researchgate.net/profile/username"
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                  />
                </div>

                <div className="space-y-1.5">
                  <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                    <GithubIcon className="w-4 h-4 text-slate-800" /> GitHub / Code Repository
                  </Label>
                  <Input
                    value={form.github_url}
```

```
                    onChange={e => updateField('github_url', e.target.value)}
                    placeholder="https://github.com/username"
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900"
                  />
                </div>

                <div className="flex justify-end pt-4 border-t border-slate-100">
                  <Button
                    onClick={() => handleSaveProfile()}
                    disabled={saving}
                    className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold r
ounded-xl px-6 h-11 shadow-sm"
                  >
                    {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                    Save Links
                  </Button>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        {/* 6. Security & Settings Tab */}
        {activeTab === 'security' && (
          <motion.div key="security" variants={fadeUp} initial="hidden" animate="visible" exit="hidden" className="space-y-
6">
            {/* Password Update Card */}
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <Key className="w-5 h-5 text-amber-600" /> Change Account Password
                </CardTitle>
                <CardDescription className="text-xs text-slate-500">
                  Update your authentication credentials to keep your workspace secure.
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6">
                <form onSubmit={handleUpdatePassword} className="space-y-4 max-w-md">
                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">Current Password</Label>
                    <div className="relative">
                      <Input
                        type={showCurrentPassword ? "text" : "password"}
                        value={currentPassword}
                        onChange={e => setCurrentPassword(e.target.value)}
                        placeholder="Enter your current password"
                        required
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 pr-10 text-slate-900"
                      />
                      <button
                        type="button"
                        onClick={() => setShowCurrentPassword(prev => !prev)}
                        className="absolute right-3 top-1/2 -translate-y-1/2 text-slate-400 hover:text-slate-600 transition-c
olors cursor-pointer"
                        title={showCurrentPassword ? "Hide password" : "Show password"}
                      >
                        {showCurrentPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
                      </button>
                    </div>
                  </div>

                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">New Password</Label>
                    <div className="relative">
                      <Input
                        type={showNewPassword ? "text" : "password"}
                        value={newPassword}
                        onChange={e => setNewPassword(e.target.value)}
                        placeholder="At least 6 characters"
                        required
                        minLength={6}
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 pr-10 text-slate-900"
                      />
                      <button
                        type="button"
                        onClick={() => setShowNewPassword(prev => !prev)}
                        className="absolute right-3 top-1/2 -translate-y-1/2 text-slate-400 hover:text-slate-600 transition-c
olors cursor-pointer"
                        title={showNewPassword ? "Hide password" : "Show password"}
```

```
                      >
                        {showNewPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
                      </button>
                    </div>
                  </div>

                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700">Confirm New Password</Label>
                    <div className="relative">
                      <Input
                        type={showConfirmPassword ? "text" : "password"}
                        value={confirmPassword}
                        onChange={e => setConfirmPassword(e.target.value)}
                        placeholder="Re-enter new password"
                        required
                        minLength={6}
                        className="bg-slate-50 border-slate-200 rounded-xl h-10 pr-10 text-slate-900"
                      />
                      <button
                        type="button"
                        onClick={() => setShowConfirmPassword(prev => !prev)}
                        className="absolute right-3 top-1/2 -translate-y-1/2 text-slate-400 hover:text-slate-600 transition-c
olors cursor-pointer"
                        title={showConfirmPassword ? "Hide password" : "Show password"}
                      >
                        {showConfirmPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
                      </button>
                    </div>
                  </div>

                  <Button
                    type="submit"
                    disabled={passwordLoading}
                    className="bg-amber-600 hover:bg-amber-700 text-white font-bold rounded-xl px-5 h-10 shadow-sm cursor-poi
nter"
                  >
                    {passwordLoading ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Lock className="w-4 h-4 mr-2" />}
                    Update Password
                  </Button>
                </form>
              </CardContent>
            </Card>

            {/* Account Metadata Card */}
            <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <Shield className="w-5 h-5 text-cyan-600" /> Account Information & Access
                </CardTitle>
              </CardHeader>
              <CardContent className="p-6 space-y-4 text-xs">
                <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200/80">
                    <span className="text-slate-400 block font-semibold">User ID</span>
                    <span className="text-slate-800 font-mono font-bold break-all">{user?.id}</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200/80">
                    <span className="text-slate-400 block font-semibold">Account Role</span>
                    <span className="text-cyan-700 font-bold capitalize">{profile?.role?.replace('_', ' ')}</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200/80">
                    <span className="text-slate-400 block font-semibold">Verification Status</span>
                    <span className="text-emerald-700 font-bold capitalize">{profile?.approval_status || 'Approved'}</span>
                  </div>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}
      </AnimatePresence>

      <ImageCropperModal
        isOpen={isCropperOpen}
        imageFile={rawAvatarFile}
        imageUrl={rawAvatarUrl}
        aspectRatio={1}
        cropShape="round"
        title="Adjust Profile Photo"
        onClose={() => {
```

```
          setIsCropperOpen(false)
          setRawAvatarFile(null)
          setRawAvatarUrl(null)
        }}
        onCropComplete={handleCropComplete}
      />
    </motion.div>
  )

  if (profile?.role === 'trainer') {
    return <TrainerLayout>{mainContent}</TrainerLayout>
  }

  const adminNavLinks = [
    { to: profile?.role === 'super_admin' ? '/super-admin' : '/admin', label: 'Dashboard', icon: BarChart3 },
    { to: `/${profile?.role}/profile`, label: 'Profile', icon: User, isActive: true },
  ]

  const traineeNavLinks = [
    { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
    { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
    { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
    { to: '/trainee/assessments', label: 'Assessments', icon: Award },
    { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
    { to: '/trainee/profile', label: 'Profile', icon: User, isActive: true },
  ]

  return (
    <DashboardShell
      title="Profile & Settings"
      icon={User}
      navLinks={profile?.role?.includes('admin') ? adminNavLinks : traineeNavLinks}
    >
      {mainContent}
    </DashboardShell>
  )
}

```

src/pages/PublicCourseCatalog.tsx TSX 17.5 KB 311 Lines

```tsx
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import { Link } from 'react-router-dom'
import { motion } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Compass, Clock, Search, ArrowRight, Sparkles, LogIn, Play, CheckCircle2, User, BookOpen } from 'lucide-react'
import { Thumbnail } from '@/components/ui/Thumbnail'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

const categories = ['All', 'Standard', 'Scenario']

function formatDurationDisplay(minutes?: number | null) {
  if (!minutes || minutes <= 0) return 'Self-paced'
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  if (h > 0 && m > 0) return `${h}h ${m}m`
  if (h > 0) return `${h}h`
  return `${m}m`
}

export function PublicCourseCatalog() {
  const { user } = useAuth()
  const [searchQuery, setSearchQuery] = React.useState('')
  const [selectedCategory, setSelectedCategory] = React.useState('All')

  const { data: courses, isLoading } = useQuery({
    queryKey: ['public_published_courses'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('courses')
        .select(`
          *,

          trainer:trainers!courses_trainer_id_fkey(full_name)
        `)
        .eq('status', 'published')
        .order('created_at', { ascending: false })

      if (error) throw error
      return data
    }
  })

  // If user is logged in, query their enrollments to show accurate status & CTAs

  const { data: userEnrollments = [] } = useQuery({
    queryKey: ['public-catalog-user-enrollments', user?.id],
    queryFn: async () => {
      if (!user?.id) return []
      const { data } = await supabase
        .from('enrollments')
        .select('id, course_id, status, progress_percent')
        .eq('user_id', user.id)
      return data || []
    },
    enabled: !!user?.id,
  })

  const enrollmentMap = React.useMemo(() => {
    const map = new Map<string, any>()
    userEnrollments.forEach((e: any) => {
      map.set(e.course_id, e)
    })
    return map
  }, [userEnrollments])

  const filteredCourses = React.useMemo(() => {
    if (!courses) return []
    let list = courses
    if (selectedCategory !== 'All') {
```

```
      list = list.filter((c: any) => c.course_type?.toLowerCase() === selectedCategory.toLowerCase())
    }
    if (searchQuery.trim()) {
      const q = searchQuery.toLowerCase()
      list = list.filter((course: any) =>
        course.title?.toLowerCase().includes(q) ||
        course.description?.toLowerCase().includes(q) ||
        course.course_type?.toLowerCase().includes(q) ||
        course.trainer?.full_name?.toLowerCase().includes(q)
      )
    }
    return list
  }, [courses, searchQuery, selectedCategory])

  return (
    <div className="min-h-screen bg-[#f8fafc] text-slate-900 font-sans flex flex-col">
      {/* Dark Shell Topbar */}
      <header className="sticky top-0 z-50 border-b border-slate-800 bg-[#040814] text-white">
        <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
          <Link to="/" className="flex items-center gap-3">
            <div className="w-8 h-8 flex items-center justify-center shrink-0">
              <img src="/logo.png" alt="Capacity Connect" className="w-full h-full object-contain" />
            </div>
            <span className="text-lg font-bold tracking-tight">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
            </span>
          </Link>
          {user ? (
            <Link to="/trainee" className="text-sm font-semibold text-cyan-400 hover:text-white transition-colors flex items-center gap-1.5 bg-white/10 hover:bg-white/20 border border-white/20 px-3.5 py-1.5 rounded-xl">
              <BookOpen className="w-4 h-4" /> Go to Dashboard
            </Link>
          ) : (
            <Link to="/login" className="text-sm font-semibold text-cyan-400 hover:text-white transition-colors flex items-center gap-1.5 bg-white/10 hover:bg-white/20 border border-white/20 px-3.5 py-1.5 rounded-xl">
              <LogIn className="w-4 h-4" /> Login
            </Link>
          )}
        </div>
      </header>

      <main className="max-w-7xl mx-auto px-6 py-10 space-y-8 flex-1 w-full">
        {/* Header & Search Bar in Midnight Dark Banner */}
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex flex-col md:flex-row md:items-center justify-between gap-6 bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white p-8 rounded-3xl border border-cyan-500/30 shadow-xl relative overflow-hidden">
          <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />

          <div className="relative z-10">
            <div className="flex items-center gap-2 mb-2">
              <span className="px-2.5 py-0.5 rounded-full bg-white/10 text-amber-300 text-xs font-bold uppercase tracking-wider flex items-center gap-1 border border-white/15">
                <Sparkles className="w-3 h-3 text-amber-300" /> Catalog
              </span>
            </div>
            <h1 className="text-3xl font-extrabold text-white tracking-tight mb-2">Explore Courses</h1>
            <p className="text-slate-300 text-sm max-w-md">Discover high-impact MoES competency tracks available on the Capacity Connect platform.</p>
          </div>

          <div className="relative w-full md:w-80 shrink-0 z-10">
            <Search className="w-5 h-5 text-slate-400 absolute left-4 top-1/2 -translate-y-1/2" />
            <input
              type="text"
              placeholder="Search courses, trainers..."
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              className="w-full bg-slate-900/80 border border-slate-700/80 rounded-2xl pl-12 pr-4 py-3 text-sm text-white placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-slate-900 focus:ring-2 focus:ring-cyan-500/20 transition-all shadow-xs"
            />
          </div>
        </motion.div>

        {/* Category Pills */}
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex items-center gap-2.5 overflow-x-auto py-2 px-1 scrollbar-none">
```

```
144 |           {categories.map(cat => {
145 |             const isSelected = selectedCategory === cat
146 |             return (
147 |               <button
148 |                 key={cat}
149 |                 onClick={() => setSelectedCategory(cat)}
150 |                 className={`px-5 py-2 rounded-full text-sm font-bold transition-all duration-200 whitespace-nowrap cursor-poin
ter ${
151 |                   isSelected
152 |                     ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white shadow-md shadow-cyan-600/30 ring-2 r
ing-cyan-500/20'
153 |                     : 'bg-white border border-slate-200 text-slate-700 hover:text-slate-900 hover:bg-slate-50 shadow-xs hover:
border-slate-300'
154 |                 }`}
155 |               >
156 |                 {cat === 'All' ? 'All Courses' : `${cat} Courses`}
157 |               </button>
158 |             )
159 |           })}
160 |         </motion.div>
161 |
162 |         {/* Course Grid */}
163 |         {isLoading ? (
164 |           <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
165 |             {[1, 2, 3, 4, 5, 6].map(i => (
166 |               <div key={i} className="h-80 bg-white animate-pulse rounded-3xl border border-slate-200" />
167 |             ))}
168 |           </div>
169 |         ) : filteredCourses.length === 0 ? (
170 |           <div className="p-16 text-center bg-white border border-slate-200 rounded-3xl shadow-sm">
171 |             <div className="w-16 h-16 bg-cyan-50 rounded-2xl flex items-center justify-center mx-auto mb-4 border border-cyan-
200">
172 |               <Compass className="w-8 h-8 text-cyan-600" />
173 |             </div>
174 |             <h3 className="text-lg font-bold text-slate-900 mb-1">{searchQuery ? 'No Matching Courses Found' : 'No Courses Ava
ilable'}</h3>
175 |             <p className="text-slate-500 text-sm max-w-sm mx-auto">{searchQuery ? 'Try clearing your filters or searching for
different keywords.' : 'Check back shortly for newly published training programs.'}</p>
176 |           </div>
177 |         ) : (
178 |           <motion.div variants={stagger} initial="hidden" animate="visible" className="grid grid-cols-1 md:grid-cols-2 lg:grid
-cols-3 gap-6">
179 |             {filteredCourses.map((course: any) => {
180 |               const enrollment = enrollmentMap.get(course.id)
181 |               const isCompleted = enrollment?.status === 'completed' || (enrollment?.progress_percent ?? 0) === 100
182 |               const isEnrolled = enrollment && (enrollment.status === 'enrolled' || enrollment.status === 'in_progress') && !i
sCompleted
183 |               const isPending = enrollment?.status === 'pending'
184 |               const isRejected = enrollment?.status === 'rejected'
185 |
186 |               let targetUrl = `/courses/${course.id}`
187 |               if (user && isEnrolled) {
188 |                 targetUrl = `/trainee/learn/${course.id}`
189 |               } else if (user) {
190 |                 targetUrl = `/trainee/courses/${course.id}`
191 |               }
192 |
193 |               return (
194 |                 <motion.div
195 |                   key={course.id}
196 |                   variants={fadeUp}
197 |                   className={`group flex flex-col bg-white border rounded-3xl overflow-hidden shadow-sm hover:shadow-xl hover:
shadow-slate-200/60 transition-all duration-300 hover:-translate-y-1.5 ${
198 |                     isCompleted
199 |                       ? 'border-emerald-200/90 hover:border-emerald-400'
200 |                       : isEnrolled
201 |                       ? 'border-cyan-300 hover:border-cyan-500'
202 |                       : isPending
203 |                       ? 'border-amber-200/90 hover:border-amber-400'
204 |                       : 'border-slate-200/90 hover:border-cyan-300'
205 |                   }`}
206 |                 >
207 |                   {/* Thumbnail Header */}
208 |                   <div className="h-44 relative overflow-hidden bg-slate-100">
209 |                     <Thumbnail path={course.thumbnail_path} alt={course.title} type={course.course_type} />
210 |
211 |                     {/* Top Badges Overlay */}
212 |                     <div className="absolute top-3 left-3 right-3 flex items-center justify-between pointer-events-none gap-
2">
```

```
                      <span className="bg-black/60 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-bold text-white uppercase tracking-wider border border-white/20 shadow-sm shrink-0">
                        {course.course_type || 'Standard'}
                      </span>

                      <div className="flex items-center gap-1.5 shrink-0">
                        {isCompleted ? (
                          <span className="bg-emerald-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabold text-white flex items-center gap-1 shadow-sm">
                            <CheckCircle2 className="w-3 h-3 text-white" /> Completed
                          </span>
                        ) : isEnrolled ? (
                          <span className="bg-gradient-to-r from-cyan-600 to-blue-600 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabold text-white flex items-center gap-1 shadow-sm">
                            <Play className="w-2.5 h-2.5 fill-current text-white" /> {enrollment.progress_percent ? `${enrollment.progress_percent}%` : 'Enrolled'}
                          </span>
                        ) : isPending ? (
                          <span className="bg-amber-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabold text-white flex items-center gap-1 shadow-sm">
                            <Clock className="w-3 h-3 text-white" /> Pending
                          </span>
                        ) : isRejected ? (
                          <span className="bg-rose-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabold text-white flex items-center gap-1 shadow-sm">
                            Declined
                          </span>
                        ) : (
                          <span className="bg-black/60 backdrop-blur-md px-2.5 py-1 rounded-full text-[11px] font-medium text-white flex items-center gap-1 shadow-sm">
                            <Clock className="w-3 h-3 text-amber-400" />
                            <span>{formatDurationDisplay(course.duration_minutes)}</span>
                          </span>
                        )}
                      </div>
                    </div>
                  </div>

                  {/* Card Content */}
                  <div className="p-6 flex-1 flex flex-col">
                    <div className="flex items-center gap-2 text-xs text-cyan-700 font-bold mb-2">
                      <span className="inline-block w-2 h-2 rounded-full bg-cyan-500 shrink-0" />
                      <span className="truncate">{course.department || 'MoES'} • {course.course_type || 'Standard'} Track</span>
                    </div>

                    <h3 className="text-base font-bold text-slate-900 mb-2 group-hover:text-cyan-600 transition-colors line-clamp-2 min-h-[3rem] leading-snug">
                      {course.title}
                    </h3>

                    <p className="text-xs text-slate-500 mb-5 line-clamp-2 leading-relaxed flex-1">
                      {course.description || 'Comprehensive training module designed to enhance core competencies.'}
                    </p>

                    {/* Instructor row */}
                    <div className="flex items-center justify-between pt-3 border-t border-slate-100 text-xs text-slate-600 mb-5">
                      <div className="flex items-center gap-2 min-w-0">
                        <div className="w-6 h-6 rounded-full bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white text-[10px] font-bold shrink-0">
                          {course.trainer?.full_name?.[0]?.toUpperCase() ?? 'T'}
                        </div>
                        <span className="truncate max-w-[140px] font-medium text-slate-700">
                          {course.trainer?.full_name || 'Assigned Trainer'}
                        </span>
                      </div>
                      <span className="text-[11px] text-cyan-700 font-bold bg-cyan-50 px-2.5 py-0.5 rounded-full border border-cyan-200 shrink-0">
                        {isCompleted ? 'Completed' : isEnrolled ? 'In Progress' : isPending ? 'Pending' : 'Open Access'}
                      </span>
                    </div>

                    {/* Action Button */}
                    <Link to={targetUrl} className="w-full mt-auto">
                      {isCompleted ? (
                        <button className="w-full py-2.5 rounded-2xl bg-emerald-50 hover:bg-emerald-100 text-emerald-800 border border-emerald-300 text-sm font-bold transition-all shadow-xs flex items-center justify-center gap-1.5 cursor-pointer">
                          <CheckCircle2 className="w-4 h-4 text-emerald-600" />
```

```
                          <span>Review Course (Completed)</span>
                          <ArrowRight className="w-4 h-4" />
                        </button>
                      ) : isEnrolled ? (
                        <button className="w-full py-2.5 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white text-sm font-extrabold transition-all shadow-md shadow-cyan-600/25 flex items-center justify-center gap-1.5 hover:scale-[1.01] cursor-pointer">
                          <Play className="w-3.5 h-3.5 fill-current" />
                          <span>{enrollment.progress_percent && enrollment.progress_percent > 0 ? `Resume Learning (${enrollment.progress_percent}%)` : 'Start Learning'}</span>
                          <ArrowRight className="w-4 h-4" />
                        </button>
                      ) : isPending ? (
                        <button className="w-full py-2.5 rounded-2xl bg-amber-50 hover:bg-amber-100 text-amber-900 border border-amber-300 text-sm font-bold transition-all shadow-xs flex items-center justify-center gap-1.5 cursor-pointer">
                          <Clock className="w-4 h-4 text-amber-600" />
                          <span>Approval Pending ⏳</span>
                          <ArrowRight className="w-4 h-4" />
                        </button>
                      ) : (
                        <button className="w-full py-2.5 rounded-2xl bg-gradient-to-r from-cyan-50 to-blue-50 group-hover:from-cyan-600 group-hover:via-sky-600 group-hover:to-blue-600 text-cyan-800 group-hover:text-white border border-cyan-200 group-hover:border-transparent text-sm font-bold transition-all duration-300 shadow-xs group-hover:shadow-md group-hover:shadow-cyan-600/20 flex items-center justify-center gap-1.5 cursor-pointer">
                          <span>Enroll Now</span>
                          <ArrowRight className="w-4 h-4 group-hover:translate-x-1 transition-transform" />
                        </button>
                      )}
                    </Link>
                  </div>
                </motion.div>
              )
            })}
          </motion.div>
        )}
      </main>
    </div>
  )
}
```

src/pages/PublicCourseDetails.tsx TSX 31.3 KB 551 Lines

```tsx
import React, { useState } from 'react'
import { useQuery } from '@tanstack/react-query'
import { useParams, Link } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import {
  Clock, User, ArrowLeft, Loader2, Layers, LogIn, GraduationCap,
  Globe, Mail, BookOpen, CheckCircle2, Award, Video, FileText,
  Calendar, Target, ChevronDown, ChevronUp, Play, Sparkles, HelpCircle,
  Shield, Check
} from 'lucide-react'
import { Button } from '@/components/ui/button'

function LinkedinIcon({ className = 'w-3.5 h-3.5' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path d="M19 3a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h14m-.5 15.5v-5.3a3.26 3.26 0 0 0-3.26-3.26c-.85 0-1.84.52-2.28 1.3v-1.11h-2.79v8.37h2.79v-4.93c0-.77.62-1.4 1.39-1.4a1.4 1.4 0 0 1 1.4 1.4v4.93h2.75M6.88 8.56a1.68 1.68 0 0 0 1.68-1.68c0-.93-.75-1.69-1.68-1.69a1.69 1.69 0 0 0-1.69 1.69c0 .93.76 1.68 1.69 1.68m1.39 9.94v-8.37H5.5v8.37h2.77z" />
    </svg>
  )
}

function GithubIcon({ className = 'w-3.5 h-3.5' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path fillRule="evenodd" clipRule="evenodd" d="M12 2C6.477 2 2 6.484 2 12.017c0 4.425 2.865 8.18 6.839 9.504.5.092.682-.217.682-.483 0-.237-.008-.868-.013-1.703-2.782.605-3.369-1.343-3.369-1.343-.454-1.158-1.11-1.466-1.11-1.466-.908-.62.069-.608.069-.608 1.003.07 1.53 1.032 1.53 1.032.892 1.53 2.341 1.088 2.91.832.092-.647.35-1.088.636-1.338-2.22-.253-4.555-1.113-4.555-4.951 0-1.093.39-1.988 1.029-2.688-.103-.253-.446-1.272.098-2.65 0 0 .84-.27 2.75 1.026A9.564 9.564 0 0112 6.844c.85.004 1.705.115 2.504.337 1.909-1.296 2.747-1.027 2.747-1.027.546 1.379.202 2.398.1 2.651.64.7 1.028 1.595 1.028 2.688 0 3.848-2.339 4.695-4.566 4.943.359.309.678.92.678 1.855 0 1.338-.012 2.419-.012 2.747 0 .268.18.58.688.482A10.019 10.019 0 0022 12.017C22 6.484 17.522 2 12 2z" />
    </svg>
  )
}

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}

function formatDurationDisplay(minutes?: number | null) {
  if (!minutes || minutes <= 0) return 'Self-paced'
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  if (h > 0 && m > 0) return `${h}h ${m}m (${minutes} mins)`
  if (h > 0) return `${h} hours (${minutes} mins)`
  return `${m} minutes`
}

export function PublicCourseDetails() {
  const { courseId } = useParams<{ courseId: string }>()
  const [openModules, setOpenModules] = useState<Set<string>>(new Set())

  const toggleModule = (id: string) => {
    setOpenModules(prev => {
      const next = new Set(prev)
      next.has(id) ? next.delete(id) : next.add(id)
      return next
    })
  }

  const { data: course, isLoading } = useQuery({
    queryKey: ['public_course_full', courseId],
    queryFn: async () => {
      const { data: courseData, error } = await supabase
        .from('courses')
        .select(`
          *,
          trainer:trainers!courses_trainer_id_fkey(full_name, department, bio, qualifications, study_details, work_experience, years_of_experience, expertise_areas, linkedin_url, website_url, github_url, email)
        `)
        .eq('id', courseId!)
        .eq('status', 'published')
        .single() as any
```

```
68 |
69 |       if (error) throw error
70 |
71 |       let modulesList = (courseData.modules && Array.isArray(courseData.modules) && courseData.modules.length > 0)
72 |         ? courseData.modules
73 |         : []
74 |
75 |       if (modulesList.length === 0) {
76 |         try {
77 |           const { data: cmData } = await (supabase as any)
78 |             .from('course_modules')
79 |             .select('*')
80 |             .eq('course_id', courseId!)
81 |             .order('order_index')
82 |           if (cmData && cmData.length > 0) {
83 |             modulesList = cmData.map((m: any) => ({
84 |               id: m.id,
85 |               title: m.title,
86 |               description: m.description,
87 |               items: m.content_items || [],
88 |               quiz_questions: m.quiz_questions || [],
89 |               passing_score: m.passing_score ?? 80,
90 |             }))
91 |           }
92 |         } catch {
93 |           // ignore

94 |         }
95 |       }
96 |
97 |       const { data: sessionsData } = await supabase
98 |         .from('course_sessions')
99 |         .select('*')
100 |        .eq('course_id', courseId!)
101 |        .order('order_index')
102 |
103 |      return {
104 |        ...courseData,
105 |        modules: modulesList,
106 |        sessions: sessionsData || [],
107 |      }
108 |    },
109 |    enabled: !!courseId
110 |  })
111 |
112 |  return (
113 |    <div className="min-h-screen bg-[#f8fafc] text-slate-900 font-sans flex flex-col">
114 |      {/* Dark Shell Topbar */}
115 |      <header className="sticky top-0 z-50 border-b border-slate-800 bg-[#040814] text-white">
116 |        <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
117 |          <Link to="/" className="flex items-center gap-3">
118 |            <div className="w-8 h-8 flex items-center justify-center shrink-0">
119 |              <img src="/logo.png" alt="Capacity Connect" className="w-full h-full object-contain" />
120 |            </div>
121 |            <span className="text-lg font-bold tracking-tight">
122 |              <span className="text-white">Capacity</span>
123 |              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
124 |            </span>
125 |          </Link>
126 |          <div className="flex items-center gap-4">
127 |            <Link to="/courses" className="text-sm text-slate-300 hover:text-white transition-colors hidden sm:block">
128 |              All Courses
129 |            </Link>
130 |            <Link to="/about" className="text-sm text-slate-300 hover:text-white transition-colors hidden sm:block">
131 |              About
132 |            </Link>
133 |            <Link to="/login" className="text-sm font-semibold text-cyan-400 hover:text-white transition-colors flex items-cen
ter gap-1.5 bg-white/10 hover:bg-white/20 border border-white/20 px-3.5 py-1.5 rounded-xl">
134 |              <LogIn className="w-4 h-4" /> Login
135 |            </Link>
136 |          </div>
137 |        </div>
138 |      </header>
139 |
140 |      <main className="max-w-7xl mx-auto px-6 py-10 flex-1 w-full space-y-8">
141 |        <Link to="/courses" className="inline-flex items-center gap-2 text-sm text-slate-600 hover:text-cyan-700 transition-co
lors font-semibold">
142 |          <ArrowLeft className="w-4 h-4" /> Back to Course Catalog
143 |        </Link>
```

```
144 |
145 |         {isLoading ? (
146 |           <div className="flex items-center justify-center h-64">
147 |             <Loader2 className="w-8 h-8 text-cyan-600 animate-spin" />
148 |           </div>
149 |         ) : !course ? (
150 |           <div className="text-center p-12 bg-white border border-slate-200 rounded-3xl shadow-sm">
151 |             <h3 className="text-xl font-bold text-slate-900">Course Not Found</h3>
152 |             <p className="text-slate-500 mt-2 text-sm">The course you are looking for does not exist or is not publicly availa
ble.</p>
153 |           </div>
154 |         ) : (
155 |           <motion.div variants={fadeUp} initial="hidden" animate="visible" className="space-y-8">
156 |
157 |             {/* 1. Hero Banner Card */}
158 |             <div className="bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white border border-cyan-500/30 ro
unded-3xl overflow-hidden shadow-xl relative p-8 md:p-12">
159 |               <div className="absolute top-0 right-0 w-96 h-96 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full
blur-3xl pointer-events-none" />
160 |
161 |               <div className="relative z-10 max-w-4xl">
162 |                 <div className="flex flex-wrap items-center gap-2 mb-5">
163 |                   <span className="bg-cyan-500/20 text-cyan-300 border border-cyan-500/40 px-3 py-1 rounded-full text-xs font-
bold uppercase tracking-wider">
164 |                     {course.department || 'MoES'}
165 |                   </span>
166 |                   <span className="bg-white/10 backdrop-blur-md px-3 py-1 rounded-full text-xs font-bold text-amber-300 upperc
ase tracking-wider border border-white/15">
167 |                     {course.course_type || 'Standard'} Track
168 |                   </span>
169 |                   {course.has_certificate !== false && (
170 |                     <span className="bg-emerald-500/20 text-emerald-300 border border-emerald-500/40 px-3 py-1 rounded-full te
xt-xs font-bold flex items-center gap-1">
171 |                       <Award className="w-3.5 h-3.5" /> Official Certification
172 |                     </span>
173 |                   )}
174 |                 </div>
175 |
176 |                 <h1 className="text-3xl md:text-5xl font-black text-white mb-4 leading-tight">
177 |                   {course.title}
178 |                 </h1>
179 |
180 |                 <div className="flex flex-wrap items-center gap-x-6 gap-y-3 text-sm text-slate-300 mb-8 font-medium">
181 |                   <div className="flex items-center gap-2">
182 |                     <User className="w-4 h-4 text-cyan-400 shrink-0" />
183 |                     <span>{course.trainer?.full_name || 'Assigned Instructor'}</span>
184 |                   </div>
185 |                   <div className="flex items-center gap-2">
186 |                     <Clock className="w-4 h-4 text-amber-400 shrink-0" />
187 |                     <span>{formatDurationDisplay(course.duration_minutes)}</span>
188 |                   </div>
189 |                   <div className="flex items-center gap-2">
190 |                     <Layers className="w-4 h-4 text-sky-400 shrink-0" />
191 |                     <span>{course.modules?.length || 0} Modules</span>
192 |                   </div>
193 |                   {course.sessions?.length > 0 && (
194 |                     <div className="flex items-center gap-2">
195 |                       <Calendar className="w-4 h-4 text-emerald-400 shrink-0" />
196 |                       <span>{course.sessions.length} Live Sessions</span>
197 |                     </div>
198 |                   )}
199 |                 </div>
200 |
201 |                 <div className="flex flex-wrap items-center gap-4">
202 |                   <Link to="/login">
203 |                     <Button
204 |                       className="bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500
text-white font-black text-sm sm:text-base rounded-2xl px-8 py-6 shadow-xl shadow-cyan-500/30 hover:scale-105 active:scale-95 transi
tion-all cursor-pointer"
205 |                     >
206 |                       <LogIn className="w-4 h-4 mr-2" />
207 |                       Login to Enroll & Start Learning
208 |                     </Button>
209 |                   </Link>
210 |                   <Link to="/register">
211 |                     <Button
212 |                       variant="outline"
213 |                       className="bg-white/10 hover:bg-white/20 border-white/20 text-white font-bold text-sm sm:text-base round
ed-2xl px-6 py-6 transition-all cursor-pointer"
```

```
                    >
                      New Trainee Registration
                    </Button>
                  </Link>
                </div>
              </div>
            </div>

            {/* 2. Main 2-Column Content Grid */}
            <div className="grid grid-cols-1 lg:grid-cols-12 gap-8 items-start">

              {/* Left Column (8 cols): Description, Modules, Sessions, Learning Outcomes */}
              <div className="lg:col-span-8 space-y-6">

                {/* About this Course Card */}
                <div className="bg-white border border-slate-200/90 rounded-3xl p-6 sm:p-8 shadow-sm">
                  <h2 className="text-lg font-bold text-slate-900 mb-3 flex items-center gap-2">
                    <BookOpen className="w-5 h-5 text-cyan-600" /> About this Course
                  </h2>
                  <div className="text-slate-700 leading-relaxed text-sm whitespace-pre-line">
                    {course.description || 'This course offers comprehensive, competency-aligned training designed to enhance operational and scientific skills across MoES divisions.'}
                  </div>
                </div>

                {/* Course Outline & Roadmap Section */}
                <div className="bg-white border border-slate-200/90 rounded-3xl p-6 sm:p-8 shadow-sm">
                  <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 mb-6 pb-4 border-b border-slate-100">
                    <div>
                      <h2 className="text-lg font-bold text-slate-900 flex items-center gap-2">
                        <Layers className="w-5 h-5 text-cyan-600" /> Course Outline & Roadmap
                      </h2>
                      <p className="text-xs text-slate-500 mt-1">
                        Structured curriculum with video lessons, study materials, and interactive quizzes.
                      </p>
                    </div>
                    <span className="text-xs font-bold text-cyan-800 bg-cyan-50 px-3 py-1 rounded-full border border-cyan-200 self-start sm:self-auto">
                      {course.modules?.length || 0} Learning Modules
                    </span>
                  </div>

                  {course.modules && course.modules.length > 0 ? (
                    <div className="space-y-4">
                      {course.modules.map((mod: any, index: number) => {
                        const isOpen = openModules.has(mod.id || String(index))
                        const items = mod.items || mod.content_items || []
                        const quizQuestions = mod.quiz_questions || []
                        const videoCount = items.filter((i: any) => i.type === 'video').length
                        const notesCount = items.filter((i: any) => i.type === 'notes' || i.type === 'pdf' || i.type === 'doc').length
                        const quizCount = quizQuestions.length + items.filter((i: any) => i.type === 'quiz').length

                        return (
                          <div
                            key={mod.id || index}
                            className="border border-slate-200/90 rounded-2xl overflow-hidden hover:border-cyan-300 transition-colors bg-slate-50/40"
                          >
                            <button
                              onClick={() => toggleModule(mod.id || String(index))}
                              className="w-full p-4 sm:p-5 flex items-start sm:items-center justify-between gap-4 text-left hover:bg-slate-50 transition-colors cursor-pointer"
                            >
                              <div className="flex items-start gap-3.5 min-w-0">
                                <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white font-extrabold text-xs shadow-xs shrink-0 mt-0.5 sm:mt-0">
                                  {index + 1}
                                </div>
                                <div className="min-w-0">
                                  <h3 className="text-sm sm:text-base font-bold text-slate-900 leading-snug">
                                    {mod.title || `Module ${index + 1}`}
                                  </h3>
                                  {mod.description && (
                                    <p className="text-xs text-slate-500 line-clamp-1 mt-0.5">
                                      {mod.description}
                                    </p>
                                  )}
```

```
                                    <div className="flex items-center gap-3 text-[11px] text-slate-500 mt-2 font-medium flex-wra
p">
                                      {videoCount > 0 && (
                                        <span className="flex items-center gap-1 text-blue-700 bg-blue-50 px-2 py-0.5 rounded-m
d">
                                          <Video className="w-3 h-3" /> {videoCount} Video{videoCount > 1 ? 's' : ''}
                                        </span>
                                      )}
                                      {notesCount > 0 && (
                                        <span className="flex items-center gap-1 text-emerald-700 bg-emerald-50 px-2 py-0.5 roun
ded-md">
                                          <FileText className="w-3 h-3" /> {notesCount} Note{notesCount > 1 ? 's' : ''}
                                        </span>
                                      )}
                                      {quizCount > 0 && (
                                        <span className="flex items-center gap-1 text-purple-700 bg-purple-50 px-2 py-0.5 rounde
d-md">
                                          <HelpCircle className="w-3 h-3" /> {quizCount} Quiz Question{quizCount > 1 ? 's' : ''}
                                        </span>
                                      )}
                                    </div>
                                  </div>
                                </div>

                                <div className="shrink-0 text-slate-400 p-1">
                                  {isOpen ? <ChevronUp className="w-5 h-5 text-cyan-600" /> : <ChevronDown className="w-5 h-5" /
>}
                                </div>
                              </button>

                              <AnimatePresence>
                                {isOpen && (
                                  <motion.div
                                    initial={{ opacity: 0, height: 0 }}
                                    animate={{ opacity: 1, height: 'auto' }}
                                    exit={{ opacity: 0, height: 0 }}
                                    className="border-t border-slate-200/80 bg-white p-4 sm:p-5 space-y-3"
                                  >
                                    {mod.description && (
                                      <p className="text-xs text-slate-600 leading-relaxed pb-2 border-b border-slate-100">
                                        {mod.description}
                                      </p>
                                    )}

                                    <div className="space-y-2">
                                      <p className="text-[11px] font-bold text-slate-400 uppercase tracking-wide">
                                        Module Content & Activities
                                      </p>
                                      {items.length > 0 ? (
                                        items.map((it: any, itIdx: number) => (
                                          <div
                                            key={itIdx}
                                            className="p-2.5 rounded-xl bg-slate-50 border border-slate-100 flex items-center ju
stify-between text-xs text-slate-700"
                                          >
                                            <div className="flex items-center gap-2 min-w-0">
                                              {it.type === 'video' ? (
                                                <Play className="w-3.5 h-3.5 text-blue-600 shrink-0" />
                                              ) : it.type === 'quiz' ? (
                                                <HelpCircle className="w-3.5 h-3.5 text-purple-600 shrink-0" />
                                              ) : (
                                                <FileText className="w-3.5 h-3.5 text-emerald-600 shrink-0" />
                                              )}
                                              <span className="font-semibold truncate">{it.title || `Lesson ${itIdx + 1}`}</span
>
                                            </div>
                                            <span className="text-[10px] text-slate-400 capitalize shrink-0 ml-2">
                                              {it.type || 'Lesson'}
                                            </span>
                                          </div>
                                        ))
                                      ) : (
                                        <p className="text-xs text-slate-400 italic">Self-guided interactive lessons and quizzes
included.</p>
                                      )}
                                    </div>
                                  </motion.div>
                                )}
                              </AnimatePresence>
```

```
                          </div>
                        )
                      })}
                    </div>
                  ) : (
                    <div className="p-8 rounded-2xl bg-slate-50 border border-dashed border-slate-200 text-center">
                      <Layers className="w-8 h-8 text-slate-300 mx-auto mb-2" />
                      <p className="text-sm font-bold text-slate-700">Course Curriculum In Progress</p>
                      <p className="text-xs text-slate-400 mt-0.5">The instructor is finalizing the interactive module items for this course.</p>
                    </div>
                  )}
                </div>

                {/* Live Sessions (if any) */}
                {course.sessions && course.sessions.length > 0 && (
                  <div className="bg-white border border-slate-200/90 rounded-3xl p-6 sm:p-8 shadow-sm">
                    <h2 className="text-lg font-bold text-slate-900 mb-2 flex items-center gap-2">
                      <Calendar className="w-5 h-5 text-cyan-600" /> Scheduled Live Sessions
                    </h2>
                    <p className="text-xs text-slate-500 mb-5">
                      Real-time interactive webinars, radar demonstrations, and Q&A classes with the trainer.
                    </p>

                    <div className="grid grid-cols-1 sm:grid-cols-2 gap-3.5">
                      {course.sessions.map((sess: any, sIdx: number) => (
                        <div key={sess.id || sIdx} className="p-4 rounded-2xl bg-slate-50 border border-slate-200/80 space-y-2">
                          <div className="flex items-center justify-between">
                            <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-cyan-100 text-cyan-800 uppercase tracking-wide">
                              Session {sIdx + 1}
                            </span>
                            <span className="text-xs text-slate-500 font-medium">
                              {sess.session_date ? new Date(sess.session_date).toLocaleDateString() : 'TBA'}
                            </span>
                          </div>
                          <h4 className="text-sm font-bold text-slate-900 line-clamp-1">{sess.title}</h4>
                          {sess.description && (
                            <p className="text-xs text-slate-500 line-clamp-2">{sess.description}</p>
                          )}
                        </div>
                      ))}
                    </div>
                  </div>
                )}

              </div>

              {/* Right Column (4 cols): Course Summary & Trainer Details */}
              <div className="lg:col-span-4 space-y-6">

                {/* Course Fast Facts Card */}
                <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm space-y-4">
                  <h3 className="text-xs font-bold text-slate-400 uppercase tracking-wider">Course Highlights</h3>

                  <div className="space-y-3 text-xs">
                    <div className="flex items-center justify-between pb-2.5 border-b border-slate-100">
                      <span className="text-slate-500">Duration</span>
                      <span className="font-bold text-slate-900">{formatDurationDisplay(course.duration_minutes)}</span>
                    </div>
                    <div className="flex items-center justify-between pb-2.5 border-b border-slate-100">
                      <span className="text-slate-500">Curriculum Modules</span>
                      <span className="font-bold text-slate-900">{course.modules?.length || 0} Modules</span>
                    </div>
                    <div className="flex items-center justify-between">
                      <span className="text-slate-500">Digital Certificate</span>
                      <span className="font-bold text-cyan-700">Included</span>
                    </div>
                  </div>

                  <Link to="/login" className="block pt-2">
                    <Button className="w-full bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold rounded-2xl h-11 shadow-md shadow-cyan-600/20 cursor-pointer">
                      Enroll into Program
                    </Button>
                  </Link>
                </div>

```

```
                {/* Trainer Details Card */}
                <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm">
                  <h3 className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-4">Trainer Details</h3>
                  <div className="flex items-start gap-3.5 mb-4">
                    <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white font-bold text-sm shadow-md shadow-cyan-600/20 shrink-0">
                      {course.trainer?.full_name?.charAt(0) || 'T'}
                    </div>
                    <div className="min-w-0 flex-1">
                      <p className="text-sm font-bold text-slate-900 leading-tight">{course.trainer?.full_name || 'Assigned Instructor'}</p>
                      <p className="text-xs text-slate-500 font-medium mt-0.5">
                        {course.trainer?.department || 'MoES Trainer'}
                        {course.trainer?.years_of_experience ? ` • ${course.trainer.years_of_experience} yrs exp` : ''}
                      </p>

                      {/* Trainer Social Links below name */}
                      <div className="flex items-center gap-1.5 mt-2.5 flex-wrap">
                        <a
                          href={
                            course.trainer?.linkedin_url
                              ? (course.trainer.linkedin_url.startsWith('http') ? course.trainer.linkedin_url : `https://${course.trainer.linkedin_url}`)
                              : `https://www.linkedin.com/search/results/all/?keywords=${encodeURIComponent(course.trainer?.full_name || 'MoES Trainer')}`
                          }
                          target="_blank"
                          rel="noreferrer"
                          className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-blue-50 hover:bg-blue-100 text-blue-700 border border-blue-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                          title={course.trainer?.linkedin_url ? "LinkedIn Profile" : "Search LinkedIn Profile"}
                        >
                          <LinkedinIcon className="w-3.5 h-3.5" />
                          <span>LinkedIn</span>
                        </a>

                        <a
                          href={
                            course.trainer?.github_url
                              ? (course.trainer.github_url.startsWith('http') ? course.trainer.github_url : `https://${course.trainer.github_url}`)
                              : `https://github.com/search?q=${encodeURIComponent(course.trainer?.full_name || 'MoES')}`
                          }
                          target="_blank"
                          rel="noreferrer"
                          className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-slate-100 hover:bg-slate-200 text-slate-800 border border-slate-300 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                          title={course.trainer?.github_url ? "GitHub / Research Profile" : "Search GitHub Profile"}
                        >
                          <GithubIcon className="w-3.5 h-3.5" />
                          <span>GitHub</span>
                        </a>

                        <a
                          href={
                            course.trainer?.website_url
                              ? (course.trainer.website_url.startsWith('http') ? course.trainer.website_url : `https://${course.trainer.website_url}`)
                              : 'https://moes.gov.in'
                          }
                          target="_blank"
                          rel="noreferrer"
                          className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-emerald-50 hover:bg-emerald-100 text-emerald-700 border border-emerald-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                          title="Official Website / Portal"
                        >
                          <Globe className="w-3.5 h-3.5" />
                          <span>Website</span>
                        </a>

                        <a
                          href={`mailto:${course.trainer?.email || 'trainer@capacityconnect.in'}?subject=Regarding: ${encodeURIComponent(course.title || 'Course')}`}
                          className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-cyan-50 hover:bg-cyan-100 text-cyan-700 border border-cyan-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                          title="Email Instructor"
                        >
                          <Mail className="w-3.5 h-3.5" />
                          <span>Email</span>
```

```
                        </a>
                      </div>
                    </div>
                  </div>

                  {course.trainer?.bio && (
                    <div className="p-3.5 bg-slate-50 rounded-2xl border border-slate-200/80 mb-3">
                      <p className="text-[10px] font-bold text-cyan-700 uppercase tracking-wider mb-1">
                        Professional Summary
                      </p>
                      <p className="text-xs text-slate-700 leading-relaxed whitespace-pre-line font-normal">
                        {course.trainer.bio}
                      </p>
                    </div>
                  )}

                  {course.trainer?.qualifications && (
                    <div className="p-3.5 bg-blue-50/70 rounded-2xl border border-blue-100">
                      <p className="text-[10px] font-bold text-blue-700 uppercase tracking-wider mb-1 flex items-center gap-1.
5">

                        <GraduationCap className="w-3.5 h-3.5 text-blue-600" />
                        Academic & Professional Qualifications
                      </p>
                      <p className="text-xs text-slate-800 leading-relaxed whitespace-pre-line font-medium">
                        {course.trainer.qualifications}
                      </p>
                    </div>
                  )}

                  {course.trainer?.expertise_areas && course.trainer.expertise_areas.length > 0 && (
                    <div className="mt-3.5">
                      <p className="text-[10px] font-bold text-slate-400 uppercase tracking-wide mb-1.5">Expertise</p>
                      <div className="flex flex-wrap gap-1.5">
                        {course.trainer.expertise_areas.map((skill: string, idx: number) => (
                          <span key={idx} className="px-2 py-0.5 rounded-md bg-emerald-50 border border-emerald-100 text-[10p
x] font-semibold text-emerald-700">
                            {skill}
                          </span>
                        ))}
                      </div>
                    </div>
                  )}
                </div>

              </div>
            </div>

          </motion.div>
        )}
      </main>
    </div>
  )
}

```

src/pages/Register.tsx TSX 23.5 KB 475 Lines

```tsx
import React, { useState } from 'react'
import { useNavigate, Link } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { motion } from 'framer-motion'
import * as z from 'zod'
import { Loader2, ArrowRight, CheckCircle, ShieldCheck, ArrowLeft } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Tabs, TabsList, TabsTrigger, TabsContent } from '@/components/ui/tabs'
import { Textarea } from '@/components/ui/textarea'
import { supabase } from '@/lib/supabase'
import { toast } from 'sonner'
import { useAuth } from '@/hooks/useAuth'

const countryCodes = [
  { code: '+91', flag: 'IN', name: 'India' },
  { code: '+1', flag: 'US', name: 'USA/Canada' },
  { code: '+44', flag: 'GB', name: 'UK' },
  { code: '+61', flag: 'AU', name: 'Australia' },
  { code: '+81', flag: 'JP', name: 'Japan' },
  { code: '+49', flag: 'DE', name: 'Germany' },
  { code: '+33', flag: 'FR', name: 'France' },
  { code: '+971', flag: 'AE', name: 'UAE' },
  { code: '+65', flag: 'SG', name: 'Singapore' },
  { code: '+60', flag: 'MY', name: 'Malaysia' },
  { code: '+94', flag: 'LK', name: 'Sri Lanka' },
  { code: '+880', flag: 'BD', name: 'Bangladesh' },
  { code: '+977', flag: 'NP', name: 'Nepal' },
  { code: '+27', flag: 'ZA', name: 'South Africa' },
  { code: '+55', flag: 'BR', name: 'Brazil' },
]

const baseSchema = z.object({
  fullName: z.string().min(2, 'Full name must be at least 2 characters'),
  email: z.string().email('Please enter a valid email address'),
  mobileNumber: z.string().min(8, 'Please enter a valid mobile number'),
})

const traineeSchema = baseSchema.extend({
  department: z.string().optional(),
  designation: z.string().optional(),
  proofFile: z.any().refine((files) => files?.length === 1, 'Proof document is required'),
})

const trainerSchema = baseSchema.extend({
  studyDetails: z.string().min(10, 'Please provide details about your studies/qualifications'),
  biodataFile: z.any().refine((files) => files?.length === 1, 'Biodata/Resume document is required'),
})

type TraineeFormValues = z.infer<typeof traineeSchema>
type TrainerFormValues = z.infer<typeof trainerSchema>

export function Register() {
  const { user, signUp } = useAuth()
  const navigate = useNavigate()
  const [isLoading, setIsLoading] = useState(false)
  const [role, setRole] = useState<'trainee' | 'trainer'>('trainee')
  const [registrationSuccess, setRegistrationSuccess] = useState(false)
  const [isExiting, setIsExiting] = useState(false)

  const handleGoToLogin = (e: React.MouseEvent) => {
    e.preventDefault()
    setIsExiting(true)
    setTimeout(() => navigate('/login'), 300)
  }

  React.useEffect(() => {
    if (user) {
      navigate('/dashboard', { replace: true })
    }
  }, [user, navigate])

  // Verification states
```

```tsx
  const [countryCode, setCountryCode] = useState('+91')

  const [emailOtpSent, setEmailOtpSent] = useState(false)
  const [isEmailVerified, setIsEmailVerified] = useState(false)
  const [emailOtpInput, setEmailOtpInput] = useState('')
  const [isSendingEmailOtp, setIsSendingEmailOtp] = useState(false)

  const traineeForm = useForm<TraineeFormValues>({ resolver: zodResolver(traineeSchema) })
  const trainerForm = useForm<TrainerFormValues>({ resolver: zodResolver(trainerSchema) })

  const currentForm = role === 'trainee' ? traineeForm : trainerForm

  const handleSendEmailOtp = async () => {
    const email = role === 'trainee' ? traineeForm.getValues('email') : trainerForm.getValues('email')
    if (!email || currentForm.formState.errors.email) {
      toast.error('Please enter a valid email first.')
      return
    }

    setIsSendingEmailOtp(true)

    // First check if the email is already registered

    try {
      const { data: emailExists, error: checkError } = await (supabase.rpc as any)('check_email_exists', {
        p_email: email
      })

      if (checkError) {
        console.error("Error checking email:", checkError)
      } else if (emailExists) {
        toast.error('This email is already registered. Please log in instead.')
        setIsSendingEmailOtp(false)
        return
      }
    } catch (err) {
      console.error("Email check failed:", err)
    }

    try {
      const { data, error } = await supabase.functions.invoke('send-otp', {
        body: { identifier: email, type: 'email' }
      })
      if (error) throw error
      if (data?.error) throw new Error(data.error)

      if (data?._simulated) {
        console.log(`[SIMULATED] Email OTP for ${email}: ${data.otp}`)
      }

      setEmailOtpSent(true)
      toast.success(`Verification code sent to ${email}`)
    } catch (err: any) {
      toast.error(err.message || 'Failed to send email verification code.')
    } finally {
      setIsSendingEmailOtp(false)
    }
  }

  const handleVerifyEmailOtp = async () => {
    const email = role === 'trainee' ? traineeForm.getValues('email') : trainerForm.getValues('email')
    if (!emailOtpInput || emailOtpInput.length < 6) {
      toast.error('Please enter a 6-digit OTP code')
      return
    }

    setIsSendingEmailOtp(true)
    try {
      const { data, error } = await supabase.rpc('verify_otp', {
        p_identifier: email,
        p_otp: emailOtpInput,
        p_user_id: '00000000-0000-0000-0000-000000000000',
        p_type: 'email'
      } as any)
      if (error) throw error
      if (!data) throw new Error("Invalid or expired OTP.")

      setIsEmailVerified(true)
      toast.success('Email successfully verified!')
```

```
    } catch (err: any) {
      toast.error(err.message || 'Invalid or expired OTP.')
    } finally {
      setIsSendingEmailOtp(false)
    }
  }

  const onSubmit = async (data: TraineeFormValues | TrainerFormValues) => {
    if (!isEmailVerified) {
      toast.error('Please verify your email address first.')
      return
    }

    setIsLoading(true)
    try {
      let uploadedFilePath: string | null = null
      const rawFile = (role === 'trainee' ? (data as TraineeFormValues).proofFile : (data as TrainerFormValues).biodataFile)?.
[0]


      if (rawFile) {
        const fileExt = rawFile.name.split('.').pop()
        const uniqueFileName = `${Date.now()}_${Math.random().toString(36).substring(2, 8)}.${fileExt}`
        const filePath = `registrations/${uniqueFileName}`

        const { error: uploadError } = await supabase.storage
          .from('proofs')
          .upload(filePath, rawFile, { upsert: false })

        if (uploadError) {
          console.warn('Storage upload error (fallback path used):', uploadError)
        } else {
          uploadedFilePath = filePath
        }
      }

      const fullMobile = `${countryCode} ${data.mobileNumber}`

      // Generate a secure random password since they will set it later via email link

      const randomPassword = Array.from(crypto.getRandomValues(new Uint8Array(16)))
        .map(b => b.toString(16).padStart(2, '0')).join('') + 'Aa1!'

      await signUp(data.email, randomPassword, {
        full_name: data.fullName,
        department: role === 'trainee' ? (data as TraineeFormValues).department : undefined,
        designation: role === 'trainee' ? (data as TraineeFormValues).designation : undefined,
        study_details: role === 'trainer' ? (data as TrainerFormValues).studyDetails : undefined,
        proof_path: uploadedFilePath || undefined,
        signup_role: role,
        is_email_verified: true,
        mobile_number: fullMobile
      })

      setRegistrationSuccess(true)
      toast.success('Account registered successfully!')
    } catch (err: any) {
      console.error('Registration Error:', err)
      const msg = err.message || 'Registration failed. Please try again.'
      toast.error(msg)
    } finally {
      setIsLoading(false)
    }
  }

  const inputClass = (hasError: boolean) =>
    `bg-slate-950/60 border-slate-800 text-slate-100 placeholder:text-slate-500 focus:bg-slate-950 focus:border-cyan-500 h-11
rounded-xl ${hasError ? 'border-rose-500/80' : ''}`

  const renderEmailField = (formObj: any) => (
    <>
    <div className="space-y-1.5">
      <Label htmlFor="email" className="text-slate-200 text-sm font-semibold">Email Address</Label>
      <div className="flex gap-2">
        <Input
          id="email"
          type="email"
          placeholder="name@moes.gov.in"
          {...formObj.register('email')}
          className={inputClass(!!formObj.formState.errors.email)}
```

```
230 |           disabled={isLoading || isEmailVerified || emailOtpSent}
231 |         />
232 |         {!isEmailVerified && !emailOtpSent && (
233 |           <Button type="button" onClick={handleSendEmailOtp} disabled={isSendingEmailOtp || isLoading} className="h-11 bg-cyan
-500/15 text-cyan-400 hover:bg-cyan-500/25 border border-cyan-500/30 font-bold rounded-xl px-4">
234 |             {isSendingEmailOtp ? <Loader2 className="w-4 h-4 animate-spin" /> : 'Verify'}
235 |           </Button>
236 |         )}
237 |         {isEmailVerified && (
238 |           <div className="h-11 w-11 flex items-center justify-center bg-emerald-950/60 rounded-xl border border-emerald-800/60
shrink-0">
239 |             <CheckCircle className="w-5 h-5 text-emerald-400" />
240 |           </div>
241 |         )}
242 |       </div>
243 |       {formObj.formState.errors.email && <p className="text-xs text-rose-400 font-medium">{formObj.formState.errors.email.mess
age}</p>}
244 |
245 |       {emailOtpSent && !isEmailVerified && (
246 |         <motion.div initial={{ opacity: 0, height: 0 }} animate={{ opacity: 1, height: 'auto' }} className="flex gap-2 mt-2">
247 |           <Input
248 |             placeholder="Enter 6-digit OTP"
249 |             maxLength={6}
250 |             value={emailOtpInput}
251 |             onChange={e => setEmailOtpInput(e.target.value.replace(/\D/g, ''))}
252 |             className="h-11 bg-slate-950/60 border-slate-800 text-white font-bold tracking-widest text-center"
253 |           />
254 |           <Button type="button" onClick={handleVerifyEmailOtp} disabled={isSendingEmailOtp || emailOtpInput.length < 6} classN
ame="h-11 bg-cyan-600 hover:bg-cyan-500 text-white font-bold rounded-xl px-4">
255 |             Confirm
256 |           </Button>
257 |         </motion.div>
258 |       )}
259 |     </div>
260 |     </>
261 |   )
262 |
263 |   const renderMobileField = (formObj: any) => (
264 |     <div className="space-y-1.5">
265 |       <Label htmlFor="mobileNumber" className="text-slate-200 text-sm font-semibold">Mobile Number</Label>
266 |       <div className="flex gap-2">
267 |         <div className="relative shrink-0">
268 |           <select
269 |             value={countryCode}
270 |             onChange={(e) => setCountryCode(e.target.value)}
271 |             disabled={isLoading}
272 |             className="h-11 appearance-none bg-slate-950/60 border border-slate-800 text-slate-100 rounded-xl pl-3 pr-8 text-s
m focus:outline-none focus:border-cyan-500 cursor-pointer font-medium"
273 |           >
274 |             {countryCodes.map(c => (
275 |               <option key={c.code} value={c.code} className="bg-slate-900 text-white">{c.flag} {c.code}</option>
276 |             ))}
277 |           </select>
278 |           <div className="pointer-events-none absolute inset-y-0 right-0 flex items-center px-2 text-slate-400">
279 |             <svg className="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 20 20"><path d="M9.293 12.95
l.707.707L15.657 8l-1.414-1.414L10 10.828 5.757 6.586 4.343 8z"/></svg>
280 |           </div>
281 |         </div>
282 |         <Input
283 |           id="mobileNumber"
284 |           type="tel"
285 |           placeholder="9876543210"
286 |           {...formObj.register('mobileNumber')}
287 |           className={inputClass(!!formObj.formState.errors.mobileNumber)}
288 |           disabled={isLoading}
289 |         />
290 |       </div>
291 |       {formObj.formState.errors.mobileNumber && <p className="text-xs text-rose-400 font-medium">{formObj.formState.errors.mob
ileNumber.message}</p>}
292 |     </div>
293 |   )
294 |
295 |   // SUCCESS VIEW

296 |   if (registrationSuccess) {
297 |     return (
298 |       <motion.div
299 |         initial={false}
300 |         animate={{ backgroundColor: isExiting ? "#040814" : "#f8fafc" }}
```

```
        transition={{ duration: 0.4, ease: "easeInOut" }}
        className="min-h-screen text-slate-900 flex items-center justify-center p-4 md:p-6 py-8 md:py-12 relative overflow-hidden"
        style={{ perspective: 1200 }}
      >
        <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-400/20 via-blue-400/15 to-transparent blur-[120px]" />
        <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-sky-400/20 via-indigo-400/15 to-transparent blur-[120px]" />
        <motion.div
          initial={{ opacity: 0, scale: 0.95, rotateY: 90 }}
          animate={isExiting ? { opacity: 0, rotateY: 90 } : { opacity: 1, scale: 1, rotateY: 0 }}
          transition={{ duration: 0.4, ease: "easeInOut" }}
          className="w-full max-w-md relative z-10 text-center bg-slate-900 text-white border border-slate-800 rounded-3xl p-8 shadow-2xl shadow-slate-900/20"
          style={{ transformStyle: "preserve-3d" }}
        >
          <div className="w-16 h-16 bg-emerald-950/60 text-emerald-400 border border-emerald-800/60 rounded-full flex items-center justify-center mx-auto mb-6 shadow-md">
            <ShieldCheck className="w-8 h-8" />
          </div>
          <h2 className="text-2xl font-bold mb-3 text-white">Verification Complete!</h2>
          <p className="text-slate-300 mb-6 leading-relaxed text-sm">
            Your account is now pending admin approval. You will receive an email to set your password once your account is approved.
          </p>
          <Button onClick={handleGoToLogin} className="w-full bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-white font-bold h-11 rounded-xl shadow-md shadow-cyan-600/20">
            Return to Login
          </Button>
        </motion.div>
      </motion.div>
    )
  }

  // REGISTRATION FORM VIEW

  return (
    <motion.div
      initial={false}
      animate={{ backgroundColor: isExiting ? "#040814" : "#f8fafc" }}
      transition={{ duration: 0.4, ease: "easeInOut" }}
      className="min-h-screen text-slate-900 flex items-center justify-center p-4 md:p-6 py-8 md:py-12 relative overflow-hidden"
      style={{ perspective: 1200 }}
    >
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-400/20 via-blue-400/15 to-transparent blur-[120px]" />
      <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-sky-400/20 via-indigo-400/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, rotateY: 90 }}
        animate={isExiting ? { opacity: 0, rotateY: 90 } : { opacity: 1, rotateY: 0 }}
        transition={{ duration: 0.4, ease: "easeInOut" }}
        className="w-full max-w-md relative z-10"
        style={{ transformStyle: "preserve-3d" }}
      >
        <Link
          to="/"
          className="inline-flex items-center gap-1.5 text-sm font-semibold text-slate-600 hover:text-slate-900 transition-colors mb-6"
        >
          <ArrowLeft className="w-4 h-4" />
          Back to Home
        </Link>

        <div className="text-center mb-6">
          <Link to="/" className="inline-flex items-center gap-2.5 mb-4">
            <img src="/logo.png" alt="Logo" className="w-10 h-10 object-contain" />
            <span className="text-lg font-bold">
              <span className="text-slate-900">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-600 to-blue-600"> Connect</span>
            </span>
          </Link>
          <h1 className="text-2xl md:text-3xl font-extrabold text-slate-900 tracking-tight">Create Account</h1>
          <p className="text-slate-500 mt-1.5 text-sm">Join the MoES Capacity Connect platform</p>
        </div>

```

```
368 |         <div className="bg-slate-900 text-white border border-slate-800 rounded-3xl p-6 md:p-8 shadow-2xl shadow-slate-900/2
0">
369 |           <Tabs value={role} onValueChange={(v) => {
370 |             setRole(v as 'trainee' | 'trainer')
371 |             setIsEmailVerified(false); setEmailOtpSent(false); setEmailOtpInput('')
372 |           }} className="w-full">
373 |             <TabsList className="grid w-full grid-cols-2 mb-6 p-1 bg-slate-950/80 rounded-xl border border-slate-800">
374 |               <TabsTrigger value="trainee" className="rounded-lg font-bold text-slate-400 data-[state=active]:bg-gradient-to-r
data-[state=active]:from-cyan-600 data-[state=active]:to-blue-600 data-[state=active]:text-white data-[state=active]:shadow-sm trans
ition-all">Trainee</TabsTrigger>
375 |               <TabsTrigger value="trainer" className="rounded-lg font-bold text-slate-400 data-[state=active]:bg-gradient-to-r
data-[state=active]:from-cyan-600 data-[state=active]:to-blue-600 data-[state=active]:text-white data-[state=active]:shadow-sm trans
ition-all">Trainer</TabsTrigger>
376 |             </TabsList>
377 |
378 |             <TabsContent value="trainee" className="mt-0">
379 |               <form onSubmit={traineeForm.handleSubmit(onSubmit)} className="space-y-4">
380 |                 <div className="space-y-1.5">
381 |                   <Label htmlFor="fullName" className="text-slate-200 text-sm font-semibold">Full Name</Label>
382 |                   <Input id="fullName" placeholder="John Doe" {...traineeForm.register('fullName')} className={inputClass(!!tr
aineeForm.formState.errors.fullName)} disabled={isLoading} />
383 |                   {traineeForm.formState.errors.fullName && <p className="text-xs text-rose-400 font-medium">{traineeForm.form
State.errors.fullName.message}</p>}
384 |                 </div>
385 |
386 |                 {renderEmailField(traineeForm)}
387 |                 {renderMobileField(traineeForm)}
388 |
389 |                 <div className="space-y-1.5">
390 |                   <Label htmlFor="proofFile" className="text-slate-200 text-sm font-semibold">Proof Document</Label>
391 |                   <Input id="proofFile" type="file" accept=".pdf,.jpg,.jpeg,.png" {...traineeForm.register('proofFile')} class
Name={`${inputClass(!!traineeForm.formState.errors.proofFile)} pt-2.5 file:text-slate-300 file:border-0 file:bg-transparent file:tex
t-xs file:font-medium`} disabled={isLoading} />
392 |                   <p className="text-xs text-slate-400">Please upload your ID or employment proof (PDF, JPG, PNG)</p>
393 |                   {traineeForm.formState.errors.proofFile && <p className="text-xs text-rose-400 font-medium">{traineeForm.for
mState.errors.proofFile.message as string}</p>}
394 |                 </div>
395 |
396 |                 <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
397 |                   <div className="space-y-1.5">
398 |                     <Label htmlFor="department" className="text-slate-200 text-sm font-semibold">Department <span className="t
ext-slate-400">(optional)</span></Label>
399 |                     <Input id="department" placeholder="e.g. IMD" {...traineeForm.register('department')} className={inputClas
s(false)} disabled={isLoading} />
400 |                   </div>
401 |                   <div className="space-y-1.5">
402 |                     <Label htmlFor="designation" className="text-slate-200 text-sm font-semibold">Designation <span className
="text-slate-400">(optional)</span></Label>
403 |                     <Input id="designation" placeholder="e.g. Scientist" {...traineeForm.register('designation')} className={i
nputClass(false)} disabled={isLoading} />
404 |                   </div>
405 |                 </div>
406 |
407 |                 <SubmitButton isLoading={isLoading} disabled={!isEmailVerified} />
408 |               </form>
409 |             </TabsContent>
410 |
411 |             <TabsContent value="trainer" className="mt-0">
412 |               <form onSubmit={trainerForm.handleSubmit(onSubmit)} className="space-y-4">
413 |                 <div className="space-y-1.5">
414 |                   <Label htmlFor="t-fullName" className="text-slate-200 text-sm font-semibold">Full Name</Label>
415 |                   <Input id="t-fullName" placeholder="Jane Smith" {...trainerForm.register('fullName')} className={inputClass
(!!trainerForm.formState.errors.fullName)} disabled={isLoading} />
416 |                   {trainerForm.formState.errors.fullName && <p className="text-xs text-rose-400 font-medium">{trainerForm.form
State.errors.fullName.message}</p>}
417 |                 </div>
418 |
419 |                 {renderEmailField(trainerForm)}
420 |                 {renderMobileField(trainerForm)}
421 |
422 |                 <div className="space-y-1.5">
423 |                   <Label htmlFor="studyDetails" className="text-slate-200 text-sm font-semibold">Study Details & Qualification
s</Label>
424 |                   <Textarea id="studyDetails" placeholder="PhD in Meteorology, 10 years teaching experience..." {...trainerFor
m.register('studyDetails')} className={`min-h-[100px] resize-none ${inputClass(!!trainerForm.formState.errors.studyDetails)}`} disab
led={isLoading} />
425 |                   {trainerForm.formState.errors.studyDetails && <p className="text-xs text-rose-400 font-medium">{trainerForm.
formState.errors.studyDetails.message}</p>}
426 |                 </div>
```

```
                <div className="space-y-1.5">
                  <Label htmlFor="biodataFile" className="text-slate-200 text-sm font-semibold">Biodata / Resume</Label>
                  <Input id="biodataFile" type="file" accept=".pdf,.doc,.docx" {...trainerForm.register('biodataFile')} className={`${inputClass(!!trainerForm.formState.errors.biodataFile)} pt-2.5 file:text-slate-300 file:border-0 file:bg-transparent file:text-xs file:font-medium`} disabled={isLoading} />
                  <p className="text-xs text-slate-400">Please upload your CV or Biodata (PDF, DOC)</p>
                  {trainerForm.formState.errors.biodataFile && <p className="text-xs text-rose-400 font-medium">{trainerForm.formState.errors.biodataFile.message as string}</p>}
                </div>

                <SubmitButton isLoading={isLoading} disabled={!isEmailVerified} />
              </form>
            </TabsContent>
          </Tabs>

          <div className="mt-5 text-center pt-4 border-t border-slate-800">
            <p className="text-sm text-slate-400 font-medium">
              Already have an account?{' '}
              <a href="/login" onClick={handleGoToLogin} className="text-cyan-400 hover:text-cyan-300 font-bold transition-colors">Sign in</a>
            </p>
          </div>
        </div>
      </motion.div>
    </motion.div>
  )
}

function SubmitButton({ isLoading, disabled }: { isLoading: boolean, disabled: boolean }) {
  return (
    <>
      <div className="flex items-start gap-2 p-3 mt-4 bg-cyan-950/40 border border-cyan-800/60 rounded-xl">
        <CheckCircle className="w-4 h-4 text-cyan-400 shrink-0 mt-0.5" />
        <p className="text-xs text-cyan-200 leading-relaxed font-medium">
          New accounts require <strong className="text-cyan-100 font-bold">admin approval</strong> before full platform access.
        </p>
      </div>
      <Button
        type="submit"
        className="w-full h-11 mt-4 bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-white shadow-lg shadow-cyan-500/25 border-0 transition-all font-bold rounded-xl disabled:opacity-50 disabled:grayscale"
        disabled={isLoading || disabled}
      >
        {isLoading ? (
          <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Creating Account...</>
        ) : (
          <>Create Account <ArrowRight className="ml-2 w-4 h-4" /></>
        )}
      </Button>
    </>
  )
}

```

src/pages/SettingsPage.tsx TSX 0.1 KB 7 Lines

```tsx
import React from 'react'
import { ProfilePage } from './ProfilePage'

export function SettingsPage() {
  return <ProfilePage />
}

```

src/pages/SetupPassword.tsx TSX 7.3 KB 160 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { useNavigate } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { motion } from 'framer-motion'
import * as z from 'zod'
import { Eye, EyeOff, Loader2, CheckCircle } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { toast } from 'sonner'

const setupPasswordSchema = z.object({
  password: z.string().min(6, 'Password must be at least 6 characters'),
  confirmPassword: z.string(),
}).refine(d => d.password === d.confirmPassword, {
  message: "Passwords don't match",
  path: ['confirmPassword'],
})

type SetupPasswordFormValues = z.infer<typeof setupPasswordSchema>

export function SetupPassword() {
  const navigate = useNavigate()
  const { refreshProfile } = useAuth()
  const [showPassword, setShowPassword] = useState(false)
  const [isLoading, setIsLoading] = useState(false)
  const [hasSession, setHasSession] = useState(false)
  const [userEmail, setUserEmail] = useState<string | null>(null)

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      if (session) {
        setHasSession(true)
        setUserEmail(session.user.email ?? null)
      } else {
        toast.error('Invalid or expired reset link. Please try again.')
        navigate('/login')
      }
    })
  }, [navigate])

  const { register, handleSubmit, formState: { errors } } = useForm<SetupPasswordFormValues>({
    resolver: zodResolver(setupPasswordSchema),
  })

  const onSubmit = async (data: SetupPasswordFormValues) => {
    setIsLoading(true)
    try {
      const { error } = await supabase.auth.updateUser({ password: data.password })
      if (error) throw error

      const { data: sessionData } = await supabase.auth.getSession()
      const userId = sessionData.session?.user.id

      if (userId) {
        await Promise.all([
          supabase.from('trainees').update({ password: data.password }).eq('id', userId),
          supabase.from('trainers').update({ password: data.password }).eq('id', userId),
          supabase.from('admins').update({ password: data.password }).eq('id', userId)
        ])
      }

      await refreshProfile()
      toast.success('Password set successfully! Welcome to your dashboard.')
      window.location.href = '/dashboard'
    } catch (err: any) {
      toast.error(err.message || 'Failed to set password. Please try again.')
    } finally {
      setIsLoading(false)
    }
  }

  const inputClass = (hasError: boolean) =>
```

```
    `bg-slate-50 border-slate-200 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:border-cyan-500 h-11 rounded-xl ${hasError ? 'border-rose-500' : ''}`

  if (!hasSession) {
    return (
      <div className="min-h-screen bg-[#040814] flex items-center justify-center p-4">
        <Loader2 className="w-8 h-8 animate-spin text-cyan-500" />
      </div>
    )
  }

  return (
    <div className="min-h-screen bg-[#040814] text-white flex items-center justify-center p-4 py-12 relative overflow-hidden">
      {/* Decorative ambient gradients */}
      <div className="pointer-events-none absolute -top-32 -left-32 w-96 h-96 rounded-full bg-gradient-to-br from-cyan-500/20 via-blue-500/15 to-transparent blur-[120px]" />
      <div className="pointer-events-none absolute -bottom-32 -right-32 w-96 h-96 rounded-full bg-gradient-to-bl from-sky-500/20 via-indigo-500/15 to-transparent blur-[120px]" />

      <motion.div
        initial={{ opacity: 0, y: 30 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.5 }}
        className="w-full max-w-md relative z-10"
      >
        <div className="text-center mb-6">
          <div className="inline-flex items-center gap-2.5 mb-4">
            <img src="/logo.png" alt="Logo" className="w-10 h-10 object-contain" />
            <span className="text-lg font-bold">
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
            </span>
          </div>
          <h1 className="text-2xl md:text-3xl font-extrabold text-white tracking-tight">Set New Password</h1>
          <p className="text-slate-400 mt-1.5 text-sm">Please enter a new password for your account.</p>
          {userEmail && (
            <div className="mt-4 inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-white/10 border border-white/20 text-xs font-semibold text-cyan-300">
              <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"></span>
              {userEmail}
            </div>
          )}
        </div>

        <div className="bg-white text-slate-900 border border-slate-200/90 rounded-3xl p-6 md:p-8 shadow-2xl shadow-black/40">
          <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
            <div className="space-y-1.5">
              <Label htmlFor="password" className="text-slate-700 text-sm font-semibold">New Password</Label>
              <div className="relative">
                <Input
                  id="password"
                  type={showPassword ? 'text' : 'password'}
                  placeholder="Min. 6 characters"
                  {...register('password')}
                  className={`${inputClass(!!errors.password)} pr-10`}
                  disabled={isLoading}
                />
                <button type="button" onClick={() => setShowPassword(!showPassword)} className="absolute right-1 top-1/2 -translate-y-1/2 p-2 text-slate-400 hover:text-slate-700 transition-colors" tabIndex={-1}>
                  {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
                </button>
              </div>
              {errors.password && <p className="text-xs text-rose-600 font-medium">{errors.password.message}</p>}
            </div>

            <div className="space-y-1.5">
              <Label htmlFor="confirmPassword" className="text-slate-700 text-sm font-semibold">Confirm Password</Label>
              <Input id="confirmPassword" type={showPassword ? 'text' : 'password'} placeholder="••••••••" {...register('confirmPassword')} className={inputClass(!!errors.confirmPassword)} disabled={isLoading} />
              {errors.confirmPassword && <p className="text-xs text-rose-600 font-medium">{errors.confirmPassword.message}</p>}
            </div>

            <Button
              type="submit"
              className="w-full h-11 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white shadow-md shadow-cyan-600/20 border-0 transition-all font-bold rounded-xl mt-4"
              disabled={isLoading}
            >
```

```
148 |               {isLoading ? (
149 |                 <><Loader2 className="mr-2 h-4 w-4 animate-spin" />Saving...</>
150 |               ) : (
151 |                 <>Save and Continue <CheckCircle className="ml-2 w-4 h-4" /></>
152 |               )}
153 |             </Button>
154 |           </form>
155 |         </div>
156 |       </motion.div>
157 |     </div>
158 |   )
159 | }
160 |
```

src/pages/SupabaseTest.tsx TSX 12.0 KB 229 Lines

```tsx
import React, { useEffect, useState } from 'react'
import { supabase } from '@/lib/supabase'
import { Loader2, CheckCircle, XCircle, ShieldCheck, ExternalLink, Database, Globe, Lock, Users, BookOpen, Shield, GraduationCap, RefreshCw } from 'lucide-react'
import { Link } from 'react-router-dom'
import { motion } from 'framer-motion'

type TestResult = {
  label: string
  status: 'ok' | 'warn' | 'error' | 'loading'
  detail: string
}

const appPages = [
  { label: 'Landing Page', path: '/', icon: Globe, desc: 'Home / Marketing page', public: true },
  { label: 'Login', path: '/login', icon: Lock, desc: 'Sign in page', public: true },
  { label: 'Register', path: '/register', icon: Users, desc: 'New user registration', public: true },
  { label: 'Forgot Password', path: '/forgot-password', icon: Lock, desc: 'Password reset request', public: true },
  { label: 'Setup Password', path: '/setup-password', icon: Lock, desc: 'Set password from email link', public: true },
  { label: 'Pending Approval', path: '/pending-approval', icon: Shield, desc: 'Awaiting admin approval', public: true },
  { label: 'Account Suspended', path: '/account-suspended', icon: XCircle, desc: 'Suspended account page', public: true },
  { label: 'Access Denied', path: '/access-denied', icon: Shield, desc: 'Unauthorized access page', public: true },
  { label: 'Trainee Dashboard', path: '/trainee', icon: GraduationCap, desc: 'Role: Trainee', public: false },
  { label: 'Course Catalog', path: '/trainee/courses', icon: BookOpen, desc: 'Role: Trainee', public: false },
  { label: 'My Learning', path: '/trainee/my-learning', icon: BookOpen, desc: 'Role: Trainee', public: false },
  { label: 'Trainer Dashboard', path: '/trainer', icon: Users, desc: 'Role: Trainer', public: false },
  { label: 'Trainer Courses', path: '/trainer/courses', icon: BookOpen, desc: 'Role: Trainer', public: false },
  { label: 'Admin Dashboard', path: '/admin', icon: Shield, desc: 'Role: Admin / Super Admin', public: false },
  { label: 'DB Test (this page)', path: '/supabase-test', icon: Database, desc: 'Connection health check', public: true },
]

export function SupabaseTest() {
  const [results, setResults] = useState<TestResult[]>([
    { label: 'Environment Variables', status: 'loading', detail: '' },
    { label: 'Network Reachability', status: 'loading', detail: '' },
    { label: 'RLS Protection Active', status: 'loading', detail: '' },
  ])
  const [refreshKey, setRefreshKey] = useState(0)

  const resetAndRefresh = () => {
    setResults([
      { label: 'Environment Variables', status: 'loading', detail: '' },
      { label: 'Network Reachability', status: 'loading', detail: '' },
      { label: 'RLS Protection Active', status: 'loading', detail: '' },
    ])
    setRefreshKey(k => k + 1)
  }

  useEffect(() => {
    const run = async () => {
      const next = [...results]

      const url = import.meta.env.VITE_SUPABASE_URL
      const key = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY
      if (url && key && url.startsWith('https://') && key.startsWith('sb_publishable')) {
        next[0] = { label: 'Environment Variables', status: 'ok', detail: `URL: ${url}` }
      } else {
        next[0] = { label: 'Environment Variables', status: 'error', detail: 'Missing or malformed VITE_SUPABASE_URL / VITE_SUPABASE_PUBLISHABLE_KEY in .env.local' }
        setResults([...next])
        return
      }

      try {
        const res = await fetch(`${url}/rest/v1/`, {
          headers: { apikey: key },
        })
        if (res.ok || res.status === 404 || res.status === 401) {
          next[1] = { label: 'Network Reachability', status: 'ok', detail: `HTTP ${res.status} — Supabase REST API is reachable` }
        } else {
          next[1] = { label: 'Network Reachability', status: 'warn', detail: `Unexpected HTTP ${res.status}` }
        }
      } catch (e) {
        next[1] = { label: 'Network Reachability', status: 'error', detail: `Network error: ${e instanceof Error ? e.message : String(e)}` }
```

```
        setResults([...next])
        return
      }

      const { error } = await supabase.from('trainees').select('id').limit(1)
      if (!error) {
        next[2] = { label: 'RLS Protection Active', status: 'warn', detail: 'Trainees table readable without auth — check RLS policies!' }
      } else if (error.code === '42501' || error.message?.includes('permission denied')) {
        next[2] = { label: 'RLS Protection Active', status: 'ok', detail: 'Permission denied for anon — RLS is enforced correctly ✓' }
      } else if (error.code === 'PGRST301') {
        next[2] = { label: 'RLS Protection Active', status: 'ok', detail: 'JWT required — RLS is enforced correctly ✓' }
      } else {
        next[2] = { label: 'RLS Protection Active', status: 'error', detail: `${error.message} (code: ${error.code})` }
      }

      setResults([...next])
    }

    run()
  // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [refreshKey])

  const allOk = results.every(r => r.status === 'ok')
  const hasError = results.some(r => r.status === 'error')
  const stillLoading = results.some(r => r.status === 'loading')

  return (
    <div className="min-h-screen bg-[#040814] p-4 md:p-8">
      <div className="max-w-4xl mx-auto space-y-6">

        {/* Header */}
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ duration: 0.4 }}>
          <div className="flex items-center gap-3 mb-1">
            <div className="w-9 h-9 flex items-center justify-center shrink-0">
              <img src="/logo.png" alt="Capacity Connect" className="w-full h-full object-contain" />
            </div>
            <div>
              <h1 className="text-xl font-bold text-zinc-200">Capacity Connect — Health Dashboard</h1>
              <p className="text-xs text-zinc-200/50">DB connectivity check + full app page map</p>
            </div>
          </div>
        </motion.div>

        {/* DB Tests */}
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ duration: 0.4, delay: 0.1 }}
          className="bg-[#070E20]/90 border border-cyan-500/30 rounded-2xl overflow-hidden shadow-sm"
        >
          <div className="px-6 py-4 border-b border-cyan-500/30 flex items-center gap-2">
            <Database className="w-4 h-4 text-zinc-200/60" />
            <h2 className="text-sm font-semibold text-zinc-200">Supabase Connection Tests</h2>
            <button
              onClick={resetAndRefresh}
              disabled={stillLoading}
              className="ml-auto flex items-center gap-1.5 text-xs px-3 py-1.5 rounded-lg border border-cyan-500/30 text-zinc-200/60 hover:text-zinc-200 hover:bg-ink/5 transition-all disabled:opacity-40 disabled:cursor-not-allowed"
            >
              <RefreshCw className={`w-3 h-3 ${stillLoading ? 'animate-spin' : ''}`} />
              {stillLoading ? 'Checking...' : 'Refresh'}
            </button>
          </div>
          <div className="p-4 space-y-3">
            {results.map((r) => (
              <div key={r.label} className={`flex items-start gap-3 p-4 rounded-xl border transition-all ${
                r.status === 'ok' ? 'bg-green-50 border-green-200' :
                r.status === 'error' ? 'bg-red-50 border-red-200' :
                r.status === 'warn' ? 'bg-yellow-50 border-yellow-200' :
                'bg-ink/5 border-cyan-500/30'
              }`}>
                <div className="mt-0.5 shrink-0">
                  {r.status === 'loading' && <Loader2 className="w-4 h-4 text-zinc-200/50 animate-spin" />}
                  {r.status === 'ok' && <CheckCircle className="w-4 h-4 text-green-600" />}
                  {r.status === 'warn' && <ShieldCheck className="w-4 h-4 text-yellow-600" />}
                  {r.status === 'error' && <XCircle className="w-4 h-4 text-red-600" />}
                </div>
                <div className="min-w-0">
                  <p className={`text-sm font-medium ${
                    r.status === 'ok' ? 'text-green-700' :
```

```
                    r.status === 'error' ? 'text-red-700' :
                    r.status === 'warn' ? 'text-yellow-700' :
                    'text-zinc-200/60'
                  }`}>{r.label}</p>
                  {r.detail && <p className="text-xs text-zinc-200/50 mt-0.5 break-all">{r.detail}</p>}
                </div>
              </div>
            ))}
          </div>

          {!stillLoading && (
            <div className="px-4 pb-4">
              {allOk && (
                <div className="p-4 rounded-xl bg-green-50 border border-green-200 text-center">
                  <p className="text-green-700 font-semibold text-sm">✅ All systems operational — Supabase is fully connected!</p>
                </div>
              )}
              {hasError && (
                <div className="p-4 rounded-xl bg-red-50 border border-red-200 text-center">
                  <p className="text-red-700 font-semibold text-sm">❌ Connection issue detected</p>
                  <p className="text-xs text-zinc-200/50 mt-1">Check your <code className="text-zinc-200/60">.env.local</code> file and Vercel environment variables.</p>
                </div>
              )}
            </div>
          )}
        </motion.div>

        {/* App Pages Map */}
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ duration: 0.4, delay: 0.2 }}
          className="bg-[#070E20]/90 border border-cyan-500/30 rounded-2xl overflow-hidden shadow-sm"
        >
          <div className="px-6 py-4 border-b border-cyan-500/30 flex items-center gap-2">
            <Globe className="w-4 h-4 text-zinc-200/60" />
            <h2 className="text-sm font-semibold text-zinc-200">App Pages — Full Sitemap</h2>
            <span className="ml-auto text-xs text-zinc-200/40">{appPages.length} pages total</span>
          </div>
          <div className="p-4 grid grid-cols-1 sm:grid-cols-2 gap-2">
            {appPages.map((page) => (
              <Link
                key={page.path}
                to={page.path}
                className="flex items-center gap-3 p-3 rounded-xl border border-cyan-500/30 hover:border-cyan-500/30 hover:bg-ink/5 transition-all group"
              >
                <div className={`w-8 h-8 rounded-lg flex items-center justify-center shrink-0 ${page.public ? 'bg-ink/10' : 'bg-ink/5'}`}>
                  <page.icon className="w-4 h-4 text-zinc-200/60" />
                </div>
                <div className="min-w-0 flex-1">
                  <p className="text-sm font-medium text-zinc-200 group-hover:text-zinc-200 truncate">{page.label}</p>
                  <p className="text-xs text-zinc-200/50 truncate">{page.desc}</p>
                </div>
                <div className="flex items-center gap-1.5 shrink-0">
                  <span className={`text-[10px] px-1.5 py-0.5 rounded-full border ${page.public ? 'bg-green-50 text-green-700 border-green-200' : 'bg-ink/5 text-zinc-200/50 border-cyan-500/30'}`}>
                    {page.public ? 'Public' : 'Auth'}
                  </span>
                  <ExternalLink className="w-3 h-3 text-zinc-200/30 group-hover:text-zinc-200/60 transition-colors" />
                </div>
              </Link>
            ))}
          </div>
        </motion.div>

        {/* Quick Actions */}
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ duration: 0.4, delay: 0.3 }}
          className="grid grid-cols-1 sm:grid-cols-3 gap-3"
        >
          <Link to="/login" className="p-4 bg-ink text-cream rounded-xl text-center text-sm font-semibold hover:bg-ink/90 transition-all">
            Go to Login →
          </Link>
          <Link to="/register" className="p-4 bg-[#070E20]/90 border border-cyan-500/30 text-zinc-200 rounded-xl text-center text-sm font-semibold hover:bg-ink/5 transition-all">
            Register New User →
          </Link>
          <Link to="/" className="p-4 bg-[#070E20]/90 border border-cyan-500/30 text-zinc-200 rounded-xl text-center text-sm f
```

```
ont-semibold hover:bg-ink/5 transition-all">
221 |             Back to Landing Page →
222 |           </Link>
223 |         </motion.div>
224 |
225 |       </div>
226 |     </div>
227 |   )
228 | }
229 |
```

src/features/trainer/AssessmentsPage.tsx TSX 111.2 KB 2,072 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useParams, Link } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from '@/components/ui/select'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
} from '@/components/ui/dialog'
import {
  ArrowLeft, Plus, Trash2, Loader2, FileText,
  Save, Send, Target, AlertCircle, Calendar, Clock, Brain, Sparkles, CheckCircle2, XCircle, Eye, RefreshCw, Download
} from 'lucide-react'
import { toast } from 'sonner'
import { format } from 'date-fns'
import { TraineeAssessmentResult } from '../courses/TraineeAssessmentResult'
import { useConfirm } from '@/hooks/useConfirm'

type Course = Database['public']['Tables']['courses']['Row']
type Assessment = Database['public']['Tables']['assessments']['Row']
type Question = Database['public']['Tables']['questions']['Row']
type Material = Database['public']['Tables']['materials']['Row']

interface QuestionForm {
  question_type: 'mcq' | 'open_ended'
  question_text: string
  option_a: string
  option_b: string
  option_c: string
  option_d: string
  correct_answer: string
  explanation: string
  difficulty: 'easy' | 'medium' | 'hard'
}

const emptyQuestion: QuestionForm = {
  question_type: 'mcq', question_text: '', option_a: '', option_b: '', option_c: '', option_d: '', correct_answer: 'A', expla
nation: '', difficulty: 'medium'
}

export function AssessmentsPage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user, profile } = useAuth()
  const isTrainer = profile?.role === 'trainer'
  const baseCoursePath = isTrainer ? '/trainer/courses' : '/admin/courses'
  const [course, setCourse] = useState<Course | null>(null)
  const [assessments, setAssessments] = useState<Assessment[]>([])
  const [selectedAssessmentId, setSelectedAssessmentId] = useState<string | null>(null)
  const [questions, setQuestions] = useState<Question[]>([])
  const [materials, setMaterials] = useState<Material[]>([])
  const [attemptsStats, setAttemptsStats] = useState<{ total: number, passRatio: number, avgScore: number } | null>(null)
  const [attempts, setAttempts] = useState<any[]>([])
  const [enrollmentsList, setEnrollmentsList] = useState<any[]>([])
  const [activeTab, setActiveTab] = useState<'questions' | 'grading'>('questions')
  const [gradingAttemptId, setGradingAttemptId] = useState<string | null>(null)
  const [gradingScore, setGradingScore] = useState<number>(0)

  const [loading, setLoading] = useState(true)
  const [saving, setSaving] = useState(false)

  const [questionDialogOpen, setQuestionDialogOpen] = useState(false)
  const [editingQuestion, setEditingQuestion] = useState<QuestionForm>(emptyQuestion)
  const [editingQuestionId, setEditingQuestionId] = useState<string | null>(null)

  const [assessmentDialogOpen, setAssessmentDialogOpen] = useState(false)
  const [editingAssessmentId, setEditingAssessmentId] = useState<string | null>(null)
```

```
  const [ConfirmDialog, confirm] = useConfirm()

  const [assessmentForm, setAssessmentForm] = useState({
    title: '',
    assessment_type: 'final' as 'daily' | 'mock' | 'final' | 'assessment',
    requires_sea: true,
    is_adaptive: false,
    is_simulation: false,
    simulation_dataset_url: '',
    scheduled_date: '',
    start_time: '',
    end_time: '',
    duration_minutes: 30,
    results_publish_date: '',
    passing_score: 50,
  })

  const [aiGenDialogOpen, setAiGenDialogOpen] = useState(false)
  const [aiGenForm, setAiGenForm] = useState({
    title: '',
    type: 'daily_test',
    topic: '',
    material_ids: [] as string[],
    count: 5,
    marks_per_question: 10,
    difficulty: 'mixed',
    scheduled_date: '',
    start_time: '',
    end_time: '',
    duration_minutes: 30,
    results_publish_date: '',
    question_format: 'mcq' as 'mcq' | 'open_ended' | 'both',
    requires_sea: false,
    passing_score: 50,
    instant_result_publication: true,
  })

  const [viewingResultForAttempt, setViewingResultForAttempt] = useState<any>(null)

  const selectedAssessment = assessments.find(a => a.id === selectedAssessmentId)

  const cleanTitle = (raw: string | undefined) => {
    return raw || '';
  }

  const formatSafeTime = (timeStr: string | null) => {
    if (!timeStr) return '';
    let d = new Date(timeStr);
    if (isNaN(d.getTime())) {
      d = new Date(`2000-01-01T${timeStr}`);
      if (isNaN(d.getTime())) return timeStr.substring(0, 5);
    }
    return format(d, 'h:mm a');
  }

  const extractTime = (timeStr: string | null) => {
    if (!timeStr) return '';
    if (timeStr.includes('T')) {
      const d = new Date(timeStr);
      if (isNaN(d.getTime())) return '';
      return format(d, 'HH:mm');
    }
    return timeStr;
  }

  const calculateWindowDuration = (start: string, end: string) => {
    if (!start || !end) return null;
    const [startH, startM] = start.split(':').map(Number);
    const [endH, endM] = end.split(':').map(Number);
    if (isNaN(startH) || isNaN(startM) || isNaN(endH) || isNaN(endM)) return null;

    let diffMin = (endH * 60 + endM) - (startH * 60 + startM);
    if (diffMin < 0) diffMin += 24 * 60; // overnight

    const h = Math.floor(diffMin / 60);
    const m = diffMin % 60;
    return `${h.toString().padStart(2, '0')}:${m.toString().padStart(2, '0')}:00`;
  }

```

```
155 |   const minutesToTimeStr = (mins: number) => {
156 |     const h = Math.floor(mins / 60);
157 |     const m = mins % 60;
158 |     return `${h.toString().padStart(2, '0')}:${m.toString().padStart(2, '0')}:00`;
159 |   }
160 |
161 |   const timeStrToMinutes = (timeStr: string) => {
162 |     if (!timeStr) return 30;
163 |     const [h, m, s] = timeStr.split(':').map(Number);
164 |     return (h * 60) + (m || 0) + (s && s >= 30 ? 1 : 0);
165 |   }
166 |
167 |   const fetchData = useCallback(async () => {
168 |     if (!user || !courseId) return
169 |     setLoading(true)
170 |     try {
171 |       let courseQuery = supabase.from('courses').select('*').eq('id', courseId)
172 |       if (profile?.role === 'trainer') {
173 |         courseQuery = courseQuery.eq('trainer_id', user.id)
174 |       }
175 |       const { data: c } = await courseQuery.single()
176 |       if (c) setCourse(c)
177 |
178 |       let assessQuery = supabase.from('assessments').select('*').eq('course_id', courseId).order('created_at', { ascending: f
alse })
179 |       if (profile?.role === 'trainer') {
180 |         assessQuery = assessQuery.eq('created_by', user.id)
181 |       }
182 |       const { data: aList } = await assessQuery
183 |       if (aList) setAssessments(aList)
184 |
185 |       if (selectedAssessmentId) {
186 |         const { data: q } = await supabase.from('questions').select('*').eq('assessment_id', selectedAssessmentId).order('pos
ition')
187 |         if (q) {
188 |           setQuestions(q.map((question: any) => ({
189 |              ...question,
190 |              question_type: (question.options as any)?._question_type || 'mcq',
191 |              difficulty: (question.options as any)?._difficulty || 'medium',
192 |           })))
193 |         }
194 |
195 |         const { data: attRaw } = await supabase.from('assessment_attempts' as any).select('*, attempt_answers(question_id, se
lected_answer)').eq('assessment_id', selectedAssessmentId).order('submitted_at', { ascending: false })
196 |
197 |         let fetchedAttempts: any[] = []
198 |         if (attRaw && attRaw.length > 0) {
199 |            const allAttempts = await Promise.all(attRaw.map(async (a: any) => {
200 |              let parsedAnswers = a.answers;
201 |              if (typeof parsedAnswers === 'string') {
202 |                  try { parsedAnswers = JSON.parse(parsedAnswers); } catch(e) {}
203 |              }
204 |              const ansObj: Record<string, string> = typeof parsedAnswers === 'object' && parsedAnswers ? { ...parsedAnswers }
: {}
205 |              if (a.attempt_answers && Array.isArray(a.attempt_answers)) {
206 |                a.attempt_answers.forEach((ans: any) => {
207 |                  ansObj[ans.question_id] = ans.selected_answer
208 |                })
209 |              }
210 |
211 |              // Manually fetch profile to avoid foreign key issues
212 |              let profile = null;
213 |              const { data: tData } = await supabase.from('trainees').select('full_name').eq('id', a.user_id).maybeSingle();
214 |              if (tData) {
215 |                  profile = { first_name: (tData as any).full_name, last_name: '' };
216 |              }
217 |
218 |              return { ...a, answers: ansObj, profiles: profile }
219 |            }))
220 |
221 |            // Deduplicate attempts: Only keep the most recent attempt per user
222 |            const uniqueAttemptsMap = new Map();
223 |            allAttempts.forEach(attempt => {
224 |              if (!uniqueAttemptsMap.has(attempt.user_id)) {
225 |                uniqueAttemptsMap.set(attempt.user_id, attempt);
226 |              }
227 |            });
```

```
            fetchedAttempts = Array.from(uniqueAttemptsMap.values());

            setAttempts(fetchedAttempts)
            const total = fetchedAttempts.length;
            const passed = fetchedAttempts.filter((d: any) => d.passed).length;
            const passRatio = total > 0 ? Math.round((passed / total) * 100) : 0;
            const avgScore = total > 0 ? Math.round(fetchedAttempts.reduce((acc: any, curr: any) => acc + (curr.score || 0), 0) / total) : 0;
            setAttemptsStats({ total, passRatio, avgScore });
         } else {
            setAttempts([])
            setAttemptsStats(null);
         }

         const { data: enrollData } = await supabase.from('enrollments').select('user_id').eq('course_id', courseId).eq('status', 'enrolled')
         if (enrollData) {
            const enrichedEnrollments = await Promise.all(enrollData.map(async (enr: any) => {
                const { data: tData } = await supabase.from('trainees').select('full_name').eq('id', enr.user_id).maybeSingle();

                return {
                    ...enr,
                    profiles: tData ? { first_name: (tData as any).full_name, last_name: '' } : null
                }
            }));
            setEnrollmentsList(enrichedEnrollments)
         } else {
            setEnrollmentsList([])
         }
       } else {
          setAttempts([])
          setAttemptsStats(null);
          setEnrollmentsList([]);
       }

       const { data: mList } = await supabase.from('materials').select('*').eq('course_id', courseId).order('created_at', { ascending: false })
       if (mList) setMaterials(mList)
     } catch (err) {
       console.error(err)
     } finally {
       setLoading(false)
     }
   }, [user, courseId, selectedAssessmentId])

   useEffect(() => { fetchData() }, [fetchData])

   const handleDeleteAttempt = async (attempt: any) => {
     const isConfirmed = await confirm('Are you sure you want to delete this attempt? The trainee will be able to take the assessment again.', 'Confirm Action')
     if (!isConfirmed) return
     setSaving(true)
     try {
       // Delete ALL attempts for this user and assessment to fix duplicates

       const { error } = await supabase.from('assessment_attempts' as any)
         .delete()
         .eq('user_id', attempt.user_id)
         .eq('assessment_id', attempt.assessment_id)

       if (error) throw error

       setAttempts(prev => {
         const deletedCount = prev.filter(a => a.user_id === attempt.user_id && a.assessment_id === attempt.assessment_id).length;
         setAttemptsStats(stats => {
           if (!stats) return stats;
           return { ...stats, total: Math.max(0, stats.total - deletedCount) }
         });
         return prev.filter(a => !(a.user_id === attempt.user_id && a.assessment_id === attempt.assessment_id));
       });

       toast.success('Attempt(s) deleted successfully.')
       fetchData() // Refresh to ensure sync
     } catch (e: any) {
       toast.error(e.message)
     } finally {
       setSaving(false)
     }
```

```
  }

  const handleUnblockAttempt = async (attemptId: string) => {
    const attempt = attempts.find(a => a.id === attemptId);
    if (!attempt) return;

    // We bypass the confirm dialog since unblocking implicitly means we want to delete it to allow retest

    setSaving(true)
    try {
      const { error } = await supabase.from('assessment_attempts' as any)
        .delete()
        .eq('user_id', attempt.user_id)
        .eq('assessment_id', attempt.assessment_id)

      if (error) throw error

      setAttempts(prev => {
        const deletedCount = prev.filter(a => a.user_id === attempt.user_id && a.assessment_id === attempt.assessment_id).length;

        setAttemptsStats(stats => {
          if (!stats) return stats;
          return { ...stats, total: Math.max(0, stats.total - deletedCount) }
        });
        return prev.filter(a => !(a.user_id === attempt.user_id && a.assessment_id === attempt.assessment_id));
      });

      toast.success('Trainee unblocked! They can now retake the assessment.')
      fetchData()
    } catch (e: any) {
      toast.error(e.message)
    } finally {
      setSaving(false)
    }
  }

  const handleGradeAttempt = async (attemptId: string) => {
    if (gradingScore < 0 || gradingScore > 100) {
      toast.error('Score must be between 0 and 100')
      return
    }
    setSaving(true)
    try {
      const { error } = await supabase.from('assessment_attempts' as any)
        .update({ score: gradingScore, passed: gradingScore >= (selectedAssessment?.passing_score ?? 50), grade_status: 'graded' })
        .eq('id', attemptId)

      if (error) throw error
      toast.success('Attempt graded successfully')
      setGradingAttemptId(null)
      fetchData()
    } catch(err: any) {
      toast.error(err.message || 'Failed to grade attempt')
    } finally {
      setSaving(false)
    }
  }

  const handleManualGradeSubmission = async (userId: string) => {
    if (gradingScore < 0 || gradingScore > 100) {
      toast.error('Score must be between 0 and 100')
      return
    }
    setSaving(true)
    try {
      const { error } = await supabase.from('assessment_attempts').insert({
        assessment_id: selectedAssessmentId,
        user_id: userId,
        score: gradingScore,
        passed: gradingScore >= (selectedAssessment?.passing_score ?? 50),
        grade_status: 'graded',
        submitted_at: new Date().toISOString()
      } as any)

      if (error) throw error
      toast.success('Manual grade added successfully')
      setGradingAttemptId(null)
      fetchData()
```

```
    } catch(err: any) {
      toast.error(err.message || 'Failed to add manual grade')
    } finally {
      setSaving(false)
    }
  }

  const handleAutoGradeAttempt = async (attemptId: string) => {
    const attempt = attempts.find(a => a.id === attemptId)
    if (!attempt || !(attempt as any).answers) return

    setSaving(true)
    const toastId = toast.loading('AI is evaluating the answers...')
    try {
      const openEndedQs = questions.filter(q => (q as any).question_type === 'open_ended')
      if (openEndedQs.length === 0) throw new Error("No open-ended questions to grade.")

      let prompt = "You are an expert evaluator grading a student's open-ended test answers based on a reference rubric. Assi
gn a score between 0 and 100 for each answer. 0 means completely wrong, 100 means perfect match with rubric. Return ONLY a valid JSO
N object mapping the question ID to the numerical score. DO NOT return markdown formatting.\n\n"

      openEndedQs.forEach(q => {
        prompt += `Question ID: ${q.id}\nReference Rubric: ${q.correct_answer}\nStudent Answer: ${(attempt as any).answers[q.
id] || 'No answer'}\n\n`
      })

      const allKeys = (import.meta.env.VITE_GEMINI_API_KEYS || import.meta.env.VITE_GEMINI_API_KEY || '').split(',').map((k:
string) => k.trim()).filter(Boolean)
      if (!allKeys.length) throw new Error("Missing Gemini API Key")

      const key = allKeys[Math.floor(Math.random() * allKeys.length)]

      const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent?
key=${key}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contents: [{ parts: [{ text: prompt }] }],
          generationConfig: { temperature: 0.1 }
        })
      })

      if (!response.ok) throw new Error("AI Evaluation request failed")

      const data = await response.json()
      const responseText = data.candidates?.[0]?.content?.parts?.[0]?.text
      if (!responseText) throw new Error("Empty AI response")

      const jsonMatch = responseText.match(/\{[\s\S]*\}/);
      if (!jsonMatch) throw new Error("AI did not return valid JSON format");
      const scores = JSON.parse(jsonMatch[0]);

      let totalCorrectPoints = 0
      questions.forEach(q => {
        if ((q as any).question_type === 'open_ended') {
          const aiScore = scores[q.id] || 0
          totalCorrectPoints += (aiScore / 100)
        } else {
          if ((attempt as any).answers[q.id] === q.correct_answer) {
             totalCorrectPoints += 1
          }
        }
      })

      const finalScore = Math.round((totalCorrectPoints / questions.length) * 100)

      const { error } = await supabase.from('assessment_attempts' as any)
        .update({ score: finalScore, passed: finalScore >= (selectedAssessment?.passing_score ?? 50), grade_status: 'graded'
})
        .eq('id', attemptId)

      if (error) throw error
      toast.success('AI Evaluation complete!', { id: toastId })
      setGradingAttemptId(null)
      fetchData()
    } catch (err: any) {
      console.error(err)
      toast.error(err.message || "AI Auto-grading failed.", { id: toastId })
    } finally {
```

```
449 |       setSaving(false)
450 |     }
451 |   }
452 |
453 |   const handleSaveAssessment = async () => {
454 |     if (!user || !courseId) return
455 |
456 |     if (assessmentForm.scheduled_date && course) {
457 |       const testDate = new Date(assessmentForm.scheduled_date)
458 |       const courseStart = course.start_date ? new Date(course.start_date) : null
459 |       const courseEnd = course.end_date ? new Date(course.end_date) : null
460 |
461 |       // Removed strict validation against course start/end dates to allow pre-assessments and make-up tests

462 |     }
463 |
464 |     setSaving(true)
465 |     try {
466 |       const editingAssessment = assessments.find(a => a.id === editingAssessmentId);
467 |
468 |       const payload = {
469 |         title: assessmentForm.title || `${assessmentForm.assessment_type} Test`,
470 |         assessment_type: assessmentForm.assessment_type,
471 |         status: editingAssessment ? editingAssessment.status : ('draft' as const),
472 |         requires_sea: assessmentForm.requires_sea,
473 |         scheduled_date: assessmentForm.scheduled_date || null,
474 |         start_time: assessmentForm.start_time || null,
475 |         end_time: assessmentForm.end_time || null,
476 |         duration_minutes: assessmentForm.duration_minutes,
477 |         results_publish_date: assessmentForm.results_publish_date || null,
478 |       }
479 |
480 |       if (editingAssessmentId) {
481 |         const { error } = await supabase.from('assessments').update(payload as any).eq('id', editingAssessmentId)
482 |         if (error) throw error
483 |         toast.success('Assessment updated')
484 |       } else {
485 |         const { error } = await supabase.from('assessments').insert({
486 |           ...payload,
487 |           course_id: courseId,
488 |           created_by: user.id,
489 |           passing_score: assessmentForm.passing_score
490 |         } as any)
491 |         if (error) throw error
492 |         toast.success('Assessment created')
493 |       }
494 |       setAssessmentDialogOpen(false)
495 |       fetchData()
496 |     } catch (err) {
497 |       toast.error((err as any)?.message || 'Failed to save assessment')
498 |     } finally {
499 |       setSaving(false)
500 |     }
501 |   }
502 |
503 |   const handleDeleteAssessment = async (id: string) => {
504 |     const isConfirmed = await confirm('Are you sure you want to delete this assessment and all its questions?', 'Delete Asses
sment')
505 |     if (!isConfirmed) return
506 |
507 |     try {
508 |       // Clean up dependencies first to satisfy foreign key constraints

509 |       await supabase.from('assessment_attempts').delete().eq('assessment_id', id)
510 |       await supabase.from('questions').delete().eq('assessment_id', id)
511 |
512 |       const { error } = await supabase.from('assessments').delete().eq('id', id)
513 |       if (error) throw error
514 |
515 |       toast.success('Assessment deleted')
516 |       setAssessments(prev => prev.filter(a => a.id !== id))
517 |       fetchData()
518 |     } catch (err: any) {
519 |       console.error('Delete error:', err)
520 |       toast.error(err.message || 'Failed to delete assessment')
521 |     }
522 |   }
523 |
524 |   const handleUpdateStatus = async (id: string, newStatus: 'draft' | 'pending_review' | 'published' | 'archived') => {
```

```
    try {
      const { error } = await supabase.from('assessments').update({ status: newStatus }).eq('id', id)
      if (error) throw error
      setAssessments(prev => prev.map(a => a.id === id ? { ...a, status: newStatus as any } : a))
      toast.success('Status updated')
    } catch (err: any) {
      console.error('Update status error:', err)
      toast.error(err.message || 'Failed to update status')
    }
  }



  const handleSaveQuestion = async () => {
    if (!selectedAssessment) return
    setSaving(true)
    try {
      const qData = {
        assessment_id: selectedAssessment.id,
        question_text: editingQuestion.question_text,
        options: {
          ...(editingQuestion.question_type === 'mcq' ? { A: editingQuestion.option_a, B: editingQuestion.option_b, C: editin
gQuestion.option_c, D: editingQuestion.option_d } : {}),
          _question_type: editingQuestion.question_type,
          _difficulty: editingQuestion.difficulty
        },
        correct_answer: editingQuestion.correct_answer,
        explanation: editingQuestion.explanation || null,
        position: editingQuestionId ? undefined : questions.length + 1,
        approved: false,
      }

      if (editingQuestionId) {
        const { error } = await supabase.from('questions').update(qData as any).eq('id', editingQuestionId)
        if (error) throw error
      } else {
        const { error } = await supabase.from('questions').insert({ ...qData, position: questions.length + 1 } as any)
        if (error) throw error
      }
      toast.success(editingQuestionId ? 'Question updated' : 'Question added')
      setQuestionDialogOpen(false)
      setEditingQuestion(emptyQuestion)
      setEditingQuestionId(null)
      fetchData()
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed')
    } finally {
      setSaving(false)
    }
  }

  const handleDeleteQuestion = async (id: string) => {
    const isConfirmed = await confirm('Are you sure you want to delete this question?', 'Delete Question')
    if (!isConfirmed) return

    try {
      await supabase.from('questions').delete().eq('id', id)
      toast.success('Question deleted')
      fetchData()
    } catch (err) {
      toast.error('Failed to delete')
    }
  }

  const handlePublishResultsNow = async (id?: string) => {
    const targetId = id || selectedAssessment?.id;
    if (!targetId) return;
    try {
      setSaving(true)
      const { error } = await supabase.from('assessments').update({ results_publish_date: null } as any).eq('id', targetId)
      if (error) throw error
      toast.success('Results published instantly!')
      fetchData()
    } catch(err) {
      toast.error('Failed to publish results')
    } finally {
      setSaving(false)
    }
  }
```

```
603 |
604 |   const handleHideResults = async (id?: string) => {
605 |     const targetId = id || selectedAssessment?.id;
606 |     if (!targetId) return;
607 |     try {
608 |       setSaving(true)
609 |       const futureDate = new Date()
610 |       futureDate.setFullYear(futureDate.getFullYear() + 10)
611 |       const { error } = await supabase.from('assessments').update({ results_publish_date: futureDate.toISOString() } as any).
eq('id', targetId)
612 |       if (error) throw error
613 |       toast.success('Results are now hidden from trainees')
614 |       fetchData()
615 |     } catch(err) {
616 |       toast.error('Failed to hide results')
617 |     } finally {
618 |       setSaving(false)
619 |     }
620 |   }
621 |
622 |   const handlePublishAssessment = async (assessmentToPublish: Assessment) => {
623 |     setSaving(true)
624 |     try {
625 |       if (assessmentToPublish.assessment_type === 'final') {
626 |         await supabase.from('assessments').update({ status: 'pending_review' }).eq('id', assessmentToPublish.id)
627 |         toast.success('Final Exam submitted to Admin for review')
628 |       } else {
629 |         await supabase.from('assessments').update({ status: 'published' }).eq('id', assessmentToPublish.id)
630 |         // Also approve all questions for this assessment

631 |         await supabase.from('questions').update({ approved: true } as any).eq('assessment_id', assessmentToPublish.id)
632 |
633 |         // Notify enrolled trainees

634 |         const { data: enrollments } = await supabase.from('enrollments').select('user_id').eq('course_id', assessmentToPublis
h.course_id).eq('status', 'enrolled')
635 |         if (enrollments && enrollments.length > 0) {
636 |             const notifications = enrollments.map(e => ({
637 |                 user_id: e.user_id,
638 |                 type: 'assessment',
639 |                 title: assessmentToPublish.status === 'published' ? 'Assessment Updated' : 'New Assessment Published',
640 |                 message: assessmentToPublish.status === 'published'
641 |                    ? `The assessment "${cleanTitle(assessmentToPublish.title)}" has been updated in ${course?.title || 'your
course'}.`
642 |                    : `A new assessment "${cleanTitle(assessmentToPublish.title)}" is available for ${course?.title || 'your c
ourse'}.`,
643 |             }))
644 |             await supabase.from('notifications').insert(notifications as any)
645 |         }
646 |
647 |         toast.success(assessmentToPublish.status === 'published' ? 'Assessment updated and trainees notified!' : 'Assessment
approved and published! Trainees notified.')
648 |       }
649 |       fetchData()
650 |     } catch (err) {
651 |       toast.error('Failed to publish assessment')
652 |     } finally {
653 |       setSaving(false)
654 |     }
655 |   }
656 |
657 |   const handleSubmitForReview = () => {
658 |     if (selectedAssessment) handlePublishAssessment(selectedAssessment)
659 |   }
660 |
661 |   const handleAIGenerate = async () => {
662 |     if (!aiGenForm.title.trim()) {
663 |       toast.error('Please enter a test title');
664 |       return;
665 |     }
666 |     if (aiGenForm.material_ids.length === 0 && !aiGenForm.topic) {
667 |       toast.error('Please enter a topic or select at least one material');
668 |       return;
669 |     }
670 |     if (!aiGenForm.scheduled_date) {
671 |       toast.error('Please select a test date');
672 |       return;
673 |     }
674 |     if (aiGenForm.type !== 'daily_test' && !aiGenForm.start_time) {
```

```
675 |       toast.error('Please set a start time');
676 |       return;
677 |     }
678 |     setSaving(true)
679 |     try {
680 |       const allKeys = (import.meta.env.VITE_GEMINI_API_KEYS || import.meta.env.VITE_GEMINI_API_KEY || '')
681 |         .split(',').map((k: string) => k.trim()).filter(Boolean);
682 |       if (!allKeys.length) throw new Error("Missing Gemini API Key in .env.local");
683 |
684 |       let contextStr = "";
685 |       let testTitle = aiGenForm.title.trim();
686 |
687 |       let inlineData: { mimeType: string, data: string } | null = null;
688 |
689 |       if (aiGenForm.material_ids.length > 0) {
690 |         const selectedMaterials = materials.filter(m => aiGenForm.material_ids.includes(m.id));
691 |         if (!testTitle && !aiGenForm.topic && selectedMaterials.length === 1) {
692 |           testTitle = `${selectedMaterials[0].file_name} Test`;
693 |         } else if (!testTitle && !aiGenForm.topic && selectedMaterials.length > 1) {
694 |           testTitle = `Combined Materials Test`;
695 |         }
696 |
697 |         const toBase64 = (blob: Blob) => new Promise<string>((resolve, reject) => {
698 |           const reader = new FileReader();
699 |           reader.readAsDataURL(blob);
700 |           reader.onload = () => resolve((reader.result as string).split(',')[1]);
701 |           reader.onerror = error => reject(error);
702 |         });
703 |
704 |         const combinedTexts: string[] = [];
705 |         for (const mat of selectedMaterials) {
706 |           if (mat.extracted_text) {
707 |             combinedTexts.push(`--- ${mat.file_name} ---\n${mat.extracted_text.substring(0, Math.floor(15000 / selectedMateri
als.length))}`);
708 |           } else if (mat.storage_path) {
709 |             if (selectedMaterials.length === 1) {
710 |               // For a single file with no extracted text, send inline to Gemini

711 |               const { data: fileBlob, error: downloadError } = await supabase.storage.from('materials').download(mat.storage_
path);
712 |               if (!downloadError && fileBlob) {
713 |                 try {
714 |                   const base64data = await toBase64(fileBlob);
715 |                   inlineData = { mimeType: fileBlob.type || 'application/pdf', data: base64data };
716 |                   combinedTexts.push(`--- ${mat.file_name} ---\n(File attached for AI to read directly)`);
717 |                 } catch {
718 |                   combinedTexts.push(`--- ${mat.file_name} ---`);
719 |                 }
720 |               } else {
721 |                 combinedTexts.push(`--- ${mat.file_name} ---`);
722 |               }
723 |             } else {
724 |               combinedTexts.push(`--- ${mat.file_name} ---`);
725 |             }
726 |           } else {
727 |             combinedTexts.push(`--- ${mat.file_name} ---`);
728 |           }
729 |         }
730 |         contextStr = `\n\nCOURSE MATERIAL CONTENT FOR CONTEXT:\n=======================\n${combinedTexts.join('\n\n')}\n====
==================\n`;
731 |       }
732 |
733 |       const baseTopic = aiGenForm.topic ? `about the following topic: "${aiGenForm.topic}"` : "based primarily on the provide
d course material";
734 |
735 |       // Build format-specific prompt

736 |       const formatInstructions = aiGenForm.question_format === 'mcq'
737 |         ? `All questions must be multiple-choice (MCQ). Each object must have:
738 |       - "question_type": "mcq"
739 |       - "question_text" (string)
740 |       - "options" (object with keys "A", "B", "C", "D")
741 |       - "correct_answer" (string: "A", "B", "C", or "D")
742 |       - "explanation" (string)
743 |       - "difficulty" (string: "easy", "medium", or "hard")`
744 |         : aiGenForm.question_format === 'open_ended'
745 |         ? `All questions must be open-ended / Q&A (short answer). Each object must have:
746 |       - "question_type": "open_ended"
747 |       - "question_text" (string)
```

```
        - "options": {}
        - "correct_answer" (string: a model answer or key points expected)
        - "explanation" (string: grading guidance)
        - "difficulty" (string: "easy", "medium", or "hard")`
          : `Mix MCQ and open-ended questions roughly equally. Each object must have:
        - "question_type" (string: "mcq" or "open_ended")
        - "question_text" (string)
        - "options" (object: keys A/B/C/D for mcq, empty object {} for open_ended)
        - "correct_answer" (string: "A"/"B"/"C"/"D" for mcq, model answer for open_ended)
        - "explanation" (string)
        - "difficulty" (string: "easy", "medium", or "hard")`;

        // System prompt for generating assessment

        const prompt = `You are an expert educator. Generate a strictly formatted JSON array of ${aiGenForm.count} questions ${baseTopic}. ${contextStr}
        The difficulty of these questions should be: ${aiGenForm.difficulty}.
        ${formatInstructions}
        Only return the raw JSON array. Do not include markdown code blocks.`;

        const parts: any[] = [{ text: prompt }];
        if (inlineData) {
            parts.push({ inlineData });
        }

        let text = '';
        let jsonRes = null;
        let lastError = null;
        const shuffledKeys = [...allKeys].sort(() => Math.random() - 0.5);

        // Try each key, with a 2-second delay if we hit a rate limit (429) to let the API recover

        for (const currentKey of shuffledKeys) {
          try {
            const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent?key=${currentKey}`, {
              method: 'POST',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({
                contents: [{ parts }],
                generationConfig: { temperature: 0.7 }
              })
            });

            if (!response.ok) {
              const errorText = await response.text();
              console.warn(`GEMINI API ERROR (${response.status}) for a key, trying next if available...`, errorText);
              lastError = new Error(`Failed to reach Gemini API: ${response.status} - ${errorText.substring(0,100)}`);
              if (response.status === 429 || response.status === 503) {
                await new Promise(r => setTimeout(r, 2000)); // wait 2s before trying next key
                continue; // Try next key
              }
              throw lastError; // Non-retryable error
            }

            jsonRes = await response.json();
            text = jsonRes.candidates[0].content.parts[0].text;
            break; // Success
          } catch (e) {
            lastError = e;
          }
        }

        // If Gemini completely fails (e.g. quota exhausted), fallback to Groq!

        if (!text) {
          console.warn("All Gemini keys failed. Falling back to Groq...", lastError);
          const groqKey = import.meta.env.VITE_GROQ_API_KEY;
          if (!groqKey) {
              throw lastError || new Error("All Gemini API keys failed and no Groq fallback key found.");
          }

          const groqResponse = await fetch('https://api.groq.com/openai/v1/chat/completions', {
            method: 'POST',
            headers: {
              'Content-Type': 'application/json',
              Authorization: `Bearer ${groqKey.trim()}`,
            },
            body: JSON.stringify({
```

```
822 |             model: 'openai/gpt-oss-20b',
823 |             messages: [
824 |               { role: 'system', content: 'You are an expert educator that generates multiple choice quiz questions in strict
JSON format.' },
825 |               { role: 'user', content: prompt }
826 |             ],
827 |             temperature: 0.7,
828 |             max_tokens: 4096,
829 |           }),
830 |         });
831 |
832 |         if (!groqResponse.ok) {
833 |             const groqErr = await groqResponse.text();
834 |             console.error('Groq API error:', groqErr);
835 |             throw new Error(`AI Generation failed on both Gemini and Groq. Groq error: ${groqResponse.status} - ${groqErr.sub
string(0, 150)}`);
836 |         }
837 |
838 |         const groqData = await groqResponse.json();
839 |         text = groqData.choices?.[0]?.message?.content || "";
840 |         if (!text) throw new Error("Groq returned empty response");
841 |       }
842 |
843 |       // Clean up markdown formatting if AI includes it

844 |       text = text.replace(/```json/g, '').replace(/```/g, '').trim();
845 |       let generatedQuestions = [];
846 |       try {
847 |         generatedQuestions = JSON.parse(text);
848 |         if (generatedQuestions && !Array.isArray(generatedQuestions)) {
849 |             const arrayProp = Object.values(generatedQuestions).find(Array.isArray);
850 |             if (arrayProp) generatedQuestions = arrayProp;
851 |             else throw new Error("Expected an array of questions");
852 |         }
853 |       } catch (e) {
854 |          console.error("Failed to parse AI JSON:", text);
855 |          throw new Error("AI returned invalid JSON format.");
856 |       }
857 |
858 |       // Map form type value to DB column value

859 |       const typeMap: Record<string, string> = {
860 |         'daily_test': 'daily',
861 |         'mock_test': 'mock',
862 |         'assessment_test': 'assessment',
863 |         'final': 'final',
864 |       };
865 |       const dbType = typeMap[aiGenForm.type] || 'daily';
866 |
867 |       // Daily test: result on the spot, no SEA; others need SEA

868 |       const isDailyTest = aiGenForm.type === 'daily_test';
869 |
870 |       // Create Assessment record

871 |       const { data: assessment, error: assessmentError } = await supabase.from('assessments').insert({
872 |         course_id: courseId || '',
873 |         title: testTitle,
874 |         assessment_type: dbType,
875 |         requires_sea: aiGenForm.requires_sea,
876 |         created_by: user?.id || '',
877 |         passing_score: aiGenForm.passing_score,
878 |         status: 'draft',
879 |         scheduled_date: aiGenForm.scheduled_date || null,
880 |         start_time: (!isDailyTest && aiGenForm.start_time) ? aiGenForm.start_time : null,
881 |         end_time: (!isDailyTest && aiGenForm.end_time) ? aiGenForm.end_time : null,
882 |         duration_minutes: aiGenForm.duration_minutes,
883 |         results_publish_date: aiGenForm.instant_result_publication
884 |             ? null
885 |             : (aiGenForm.results_publish_date ? new Date(aiGenForm.results_publish_date).toISOString() : new Date(new Date().
setFullYear(new Date().getFullYear() + 10)).toISOString())
886 |       }).select().single();
887 |
888 |       if (assessmentError) throw assessmentError;
889 |
890 |       // Insert questions

891 |       const questionsToInsert = generatedQuestions.map((q: any, i: number) => ({
892 |         assessment_id: assessment.id,
```

```
893 |         question_text: q.question_text,
894 |         options: {
895 |           ...(q.question_type === 'open_ended' ? {} : (q.options || {})),
896 |           _question_type: q.question_type === 'open_ended' ? 'open_ended' : 'mcq',
897 |           _difficulty: q.difficulty && ['easy', 'medium', 'hard'].includes(q.difficulty.toLowerCase()) ? q.difficulty.toLower
Case() : 'medium'
898 |         },
899 |         correct_answer: q.correct_answer,
900 |         explanation: q.explanation,
901 |         position: i + 1,
902 |         approved: false
903 |       }));
904 |
905 |       const { error: questionsError } = await supabase.from('questions').insert(questionsToInsert);
906 |       if (questionsError) throw questionsError;
907 |
908 |       toast.success('Assessment generated successfully!')
909 |       setAiGenDialogOpen(false)
910 |       setAiGenForm({ title: '', type: 'daily_test', topic: '', material_ids: [], count: 5, marks_per_question: 10, difficult
y: 'mixed', scheduled_date: '', start_time: '', end_time: '', duration_minutes: 30, results_publish_date: '', question_format: 'mc
q', requires_sea: false, passing_score: 50, instant_result_publication: true })
911 |       fetchData()
912 |     } catch (err) {
913 |       console.error(err);
914 |       toast.error((err as any)?.message || (err as any)?.details || String(err) || 'Failed to generate assessment.')
915 |     } finally {
916 |       setSaving(false)
917 |     }
918 |   }
919 |
920 |   const openEditQuestion = (q: Question) => {
921 |     const opts = (q.options as Record<string, string>) || {}
922 |     setEditingQuestion({
923 |       question_type: ((opts as any)._question_type as 'mcq'|'open_ended') || ((q as any).question_type as 'mcq'|'open_ended')
|| 'mcq',
924 |       question_text: q.question_text,
925 |       option_a: opts.A || '', option_b: opts.B || '', option_c: opts.C || '', option_d: opts.D || '',
926 |       correct_answer: q.correct_answer, explanation: q.explanation ?? '',
927 |       difficulty: ((opts as any)._difficulty as 'easy'|'medium'|'hard') || ((q as any).difficulty as 'easy'|'medium'|'hard')
|| 'medium',
928 |     })
929 |     setEditingQuestionId(q.id)
930 |     setQuestionDialogOpen(true)
931 |   }
932 |
933 |   const openNewAssessmentDialog = () => {
934 |     setEditingAssessmentId(null)
935 |     setAssessmentForm({
936 |       title: '', assessment_type: 'final', requires_sea: true, scheduled_date: '', start_time: '', end_time: '', duration_min
utes: 30, results_publish_date: '',
937 |       is_adaptive: false, is_simulation: false, simulation_dataset_url: '', passing_score: course?.passing_score ?? 60
938 |     })
939 |     setAssessmentDialogOpen(true)
940 |   }
941 |
942 |   const openEditAssessmentDialog = (a: Assessment) => {
943 |     setEditingAssessmentId(a.id)
944 |     setAssessmentForm({
945 |       title: a.title,
946 |       assessment_type: (a.assessment_type || 'final') as 'daily' | 'mock' | 'final',
947 |       requires_sea: a.requires_sea,
948 |       scheduled_date: a.scheduled_date ? new Date(a.scheduled_date).toISOString().slice(0, 10) : '',
949 |       start_time: extractTime(a.start_time),
950 |       end_time: extractTime(a.end_time),
951 |       duration_minutes: a.duration_minutes || 30,
952 |       results_publish_date: (a as any).results_publish_date ? new Date((a as any).results_publish_date).toISOString().slice
(0, 10) : '',
953 |       is_adaptive: (a as any).is_adaptive || false,
954 |       is_simulation: (a as any).is_simulation || false,
955 |       simulation_dataset_url: (a as any).simulation_dataset_url || '',
956 |       passing_score: a.passing_score ?? course?.passing_score ?? 60
957 |     })
958 |     setAssessmentDialogOpen(true)
959 |   }
960 |
961 |   const handleDownloadCSV = () => {
962 |     if (!attempts || attempts.length === 0) {
963 |       toast.error('No results available to download.');
964 |       return;
```

```
    }

    const headers = ['Trainee Name', 'Score', 'Passed', 'Status', 'Submitted At'];

    const rows = attempts.map(att => {
      const name = `${att.profiles?.first_name || ''} ${att.profiles?.last_name || ''}`.trim();
      const score = att.score === -1 ? 'Blocked (SEA Violation)' : `${att.score}%`;
      const passed = att.score === -1 ? 'N/A' : (att.passed ? 'Yes' : 'No');
      const status = att.grade_status === 'pending_manual' ? 'Needs Grading' : 'Graded';
      const submittedAt = new Date(att.submitted_at).toLocaleString();

      return [
        `"${name.replace(/"/g, '""')}"`,
        `"${score}"`,
        `"${passed}"`,
        `"${status}"`,
        `"${submittedAt}"`
      ].join(',');
    });

    const csvContent = [headers.join(','), ...rows].join('\n');
    const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.href = url;
    link.setAttribute('download', `${cleanTitle(selectedAssessment?.title || 'Assessment')}_Results.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };

  if (loading && !assessments.length) {
    return (
      <TrainerLayout>
        <div className="flex items-center justify-center py-20"><Loader2 className="w-6 h-6 animate-spin text-cyan-600" /></div>
      </TrainerLayout>
    )
  }

  return (
    <TrainerLayout>
      <ConfirmDialog />
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-4xl mx-auto space-y-6">

        {/* LIST VIEW */}
        {!selectedAssessmentId ? (
          <>
            <motion.div variants={fadeUp} className="flex items-center justify-between">
              <div>
                <Link to={`${baseCoursePath}/${courseId}`} className="flex items-center gap-2 text-sm text-slate-500 hover:text-cyan-600 transition-colors mb-4 font-medium">
                  <ArrowLeft className="w-4 h-4" /> Back to Course
                </Link>
                <h2 className="text-2xl font-black tracking-tight text-slate-900">Tests & Assessments</h2>
                <p className="text-slate-500 text-sm mt-1 font-medium">{course?.title}</p>
              </div>
              <div className="flex gap-2">
                <Button onClick={() => setAiGenDialogOpen(true)} variant="outline" className="border-cyan-200 text-cyan-700 bg-cyan-50 hover:bg-cyan-100 font-bold rounded-xl">
                  <Brain className="w-4 h-4 mr-2 text-cyan-600" /> Auto-Generate with AI
                </Button>
                <Button onClick={openNewAssessmentDialog} className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-bold rounded-xl shadow-md shadow-cyan-600/10">
                  <Plus className="w-4 h-4 mr-2" /> Create Test
                </Button>
              </div>
            </motion.div>

            {assessments.length === 0 ? (
              <motion.div variants={fadeUp} className="grid grid-cols-1 md:grid-cols-2 gap-6 mt-8">
                {/* AI Generation Card */}
                <motion.div whileHover={{ scale: 1.02, y: -4 }} transition={{ type: "spring", stiffness: 300 }}>
                  <Card className="relative overflow-hidden group border border-cyan-200 hover:border-cyan-400 shadow-sm hover:shadow-lg h-full cursor-pointer bg-gradient-to-br from-white to-cyan-50/50 rounded-3xl" onClick={() => setAiGenDialogOpen(true)}>
                    <CardContent className="p-8 relative z-10 h-full flex flex-col justify-between">
                      <div>
                        <div className="w-14 h-14 bg-cyan-100 rounded-2xl flex items-center justify-center mb-6 border border
```

```
-cyan-200 shadow-sm text-cyan-600">
                            <Brain className="w-7 h-7" />
                          </div>
                          <h3 className="text-2xl font-black text-slate-900 mb-3 tracking-tight">Auto-Generate with AI</h3>
                          <p className="text-sm text-slate-600 mb-8 leading-relaxed font-medium">
                            Instantly generate a complete, high-quality assessment tailored perfectly to your course content, o
bjectives, and desired difficulty level.
                          </p>
                        </div>
                        <Button className="w-full bg-gradient-to-r from-cyan-600 to-blue-600 text-white hover:opacity-95 shadow
-md font-bold text-sm h-12 rounded-xl">
                          <Brain className="w-4 h-4 mr-2" /> Start AI Generation
                        </Button>
                      </CardContent>
                    </Card>
                  </motion.div>

                  {/* Manual Creation Card */}
                  <motion.div whileHover={{ scale: 1.02, y: -4 }} transition={{ type: "spring", stiffness: 300 }}>
                    <Card className="relative overflow-hidden group border border-slate-200 hover:border-slate-300 shadow-sm ho
ver:shadow-lg h-full cursor-pointer bg-white rounded-3xl" onClick={openNewAssessmentDialog}>
                      <CardContent className="p-8 relative z-10 h-full flex flex-col justify-between">
                        <div>
                          <div className="w-14 h-14 bg-slate-100 text-slate-800 rounded-2xl flex items-center justify-center mb
-6 border border-slate-200">
                            <Plus className="w-7 h-7" />
                          </div>
                          <h3 className="text-2xl font-black text-slate-900 mb-3 tracking-tight">Create Manually</h3>
                          <p className="text-sm text-slate-600 mb-8 leading-relaxed font-medium">
                            Build your assessment from scratch. Define your own questions, options, and passing criteria with a
bsolute precision and control.
                          </p>
                        </div>
                        <Button variant="outline" className="w-full border-slate-200 text-slate-700 hover:bg-slate-50 font-bold
text-sm h-12 rounded-xl">
                          Create Blank Test
                        </Button>
                      </CardContent>
                    </Card>
                  </motion.div>
                </motion.div>
              ) : (
                <motion.div variants={fadeUp} className="space-y-4">
                  {assessments.map((a) => (
                    <Card key={a.id} className="bg-white border border-slate-200/90 hover:border-slate-300 shadow-sm hover:shad
ow-md transition-all overflow-hidden rounded-3xl group">
                      <CardHeader className="bg-slate-50/80 border-b border-slate-100 pb-4">
                        <div className="flex items-start justify-between">
                          <div>
                            <div className="flex items-center gap-2 mb-1">
                              <Badge variant="outline" className="capitalize text-[10px] bg-white border-slate-200 text-slate-7
00">{a.assessment_type} Test</Badge>
                              {a.requires_sea && <Badge className="bg-cyan-50 text-cyan-700 hover:bg-cyan-100 border border-cya
n-200 text-[10px]">SEA Enabled</Badge>}
                              <CardTitle className="text-lg text-slate-900 font-bold">{cleanTitle(a.title)}</CardTitle>
                            </div>
                            <div className="flex flex-wrap gap-4 mt-3 text-xs text-slate-500 font-medium">
                              {a.scheduled_date && (
                                <span className="flex items-center gap-1.5"><Calendar className="w-3.5 h-3.5 text-slate-400" />
{new Date(a.scheduled_date).toLocaleDateString()}</span>
                              )}
                              {a.start_time && (
                                <span className="flex items-center gap-1.5"><Clock className="w-3.5 h-3.5 text-slate-400" /> {f
ormatSafeTime(a.start_time)} {a.end_time ? `- ${formatSafeTime(a.end_time)}` : '- TBD'}</span>
                              )}
                            </div>
                          </div>
                          <div className="flex items-center gap-2">
                            <Button variant="ghost" size="sm" onClick={() => openEditAssessmentDialog(a)} className="text-slate
-600 hover:text-slate-900">Settings</Button>
                            <Button variant="ghost" size="sm" className="text-red-500 hover:text-red-600 hover:bg-red-50" onCli
ck={() => handleDeleteAssessment(a.id)}>
                              <Trash2 className="w-4 h-4" />
                            </Button>
                          </div>
                        </div>
                      </CardHeader>
                      <CardContent className="p-5 bg-white flex justify-between items-center">
                        <div>
```

```
                        <p className="text-sm font-semibold text-slate-700">Status: <span className="capitalize text-slate-900">{a.status.replace('_', ' ')}</span></p>
                      </div>
                      <div className="flex items-center gap-2">
                        {a.status === 'published' ? (
                          <>
                            <Button
                              variant="outline"
                              onClick={() => handleUpdateStatus(a.id, 'draft')}
                              className="border-rose-200 text-rose-700 font-semibold rounded-xl hover:bg-rose-50"
                            >
                              Unpublish
                            </Button>
                            <Button
                              variant="outline"
                              onClick={() => handlePublishAssessment(a)}
                              disabled={saving}
                              className="border-cyan-200 text-cyan-700 font-semibold rounded-xl hover:bg-cyan-50"
                            >
                              Update Published
                            </Button>
                            {a.results_publish_date && new Date(a.results_publish_date) > new Date() ? (
                              <Button
                                variant="outline"
                                onClick={() => handlePublishResultsNow(a.id)}
                                disabled={saving}
                                className="border-indigo-200 text-indigo-700 font-semibold rounded-xl hover:bg-indigo-50"
                              >
                                Publish Results
                              </Button>
                            ) : (
                              <Button
                                variant="outline"
                                onClick={() => handleHideResults(a.id)}
                                disabled={saving}
                                className="border-amber-200 text-amber-700 font-semibold rounded-xl hover:bg-amber-50"
                              >
                                Hide Results
                              </Button>
                            )}
                            <Button
                              variant="outline"
                              onClick={() => { setSelectedAssessmentId(a.id); setActiveTab('grading'); }}
                              className="border-emerald-200 text-emerald-700 font-semibold rounded-xl hover:bg-emerald-50"
                            >
                              View Results
                            </Button>
                          </>
                        ) : a.status === 'pending_review' ? (
                            <Button
                              variant="outline"
                              disabled
                              className="border-amber-200 text-amber-700 font-semibold rounded-xl bg-amber-50"
                            >
                              Under Review
                            </Button>
                        ) : (
                          <Button
                            onClick={() => handlePublishAssessment(a)}
                            disabled={saving}
                            className="bg-emerald-100 text-emerald-700 hover:bg-emerald-200 font-bold rounded-xl shadow-sm border border-emerald-200"
                          >
                            {a.assessment_type === 'final' ? 'Submit for Review' : 'Publish'}
                          </Button>
                        )}
                        <Button onClick={() => { setSelectedAssessmentId(a.id); setActiveTab('questions'); }} className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-semibold rounded-xl shadow-sm">
                          Manage Questions
                        </Button>
                      </div>
                    </CardContent>
                  </Card>
                ))}
              </motion.div>
            )}
          </>
        ) : (
          /* DETAIL VIEW (Question Builder) */
```

```
1180 |           <>
1181 |             <motion.div variants={fadeUp}>
1182 |               <button onClick={() => setSelectedAssessmentId(null)} className="flex items-center gap-2 text-sm text-slate-500
hover:text-cyan-600 transition-colors mb-4 font-medium">
1183 |                 <ArrowLeft className="w-4 h-4" /> Back to Assessments
1184 |               </button>
1185 |
1186 |               <div className="flex items-center justify-between mb-6">
1187 |                 <div>
1188 |                   <div className="flex items-center gap-2">
1189 |                     <h2 className="text-2xl font-black tracking-tight text-slate-900">{cleanTitle(selectedAssessment?.title)}
</h2>
1190 |                     <Badge variant="outline" className="capitalize text-[10px] py-0 border-slate-200">{selectedAssessment?.as
sessment_type}</Badge>
1191 |                   </div>
1192 |                   <p className="text-xs text-slate-500 mt-1 font-medium">{questions.length} questions | Passing: {selectedAss
essment?.passing_score}%</p>
1193 |                 </div>
1194 |                 <div className="flex gap-2">
1195 |                   {selectedAssessment?.results_publish_date && new Date(selectedAssessment.results_publish_date) > new Date()
? (
1196 |                     <Button onClick={() => handlePublishResultsNow(selectedAssessment.id)} disabled={saving} variant="seconda
ry" className="bg-indigo-50 text-indigo-700 hover:bg-indigo-100 font-bold rounded-xl border border-indigo-200">
1197 |                       Publish Results Now
1198 |                     </Button>
1199 |                   ) : (
1200 |                     <Button onClick={() => handleHideResults(selectedAssessment?.id)} disabled={saving} variant="secondary" c
lassName="bg-amber-50 text-amber-700 hover:bg-amber-100 font-bold rounded-xl border border-amber-200">
1201 |                       Hide Results
1202 |                     </Button>
1203 |                   )}
1204 |                   <Button onClick={handleDownloadCSV} variant="outline" className="border-slate-200 text-slate-700 hover:bg-s
late-50 font-semibold rounded-xl">
1205 |                     <Download className="w-4 h-4 mr-2" /> Download CSV
1206 |                   </Button>
1207 |                   <Button onClick={() => { setEditingQuestion(emptyQuestion); setEditingQuestionId(null); setQuestionDialogOp
en(true) }}
1208 |                     className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-semibold rounded-x
l shadow-sm">
1209 |                     <Plus className="w-4 h-4 mr-2" /> Add Question
1210 |                   </Button>
1211 |                   <Button onClick={handleSubmitForReview} disabled={saving || questions.length === 0} variant="outline" class
Name="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold rounded-xl">
1212 |                     <Send className="w-4 h-4 mr-2" /> {selectedAssessment?.assessment_type === 'final' ? 'Submit for Admin Re
view' : (selectedAssessment?.status === 'published' ? 'Update Published' : 'Approve & Publish')}
1213 |                   </Button>
1214 |                 </div>
1215 |               </div>
1216 |
1217 |               {/* Analytics Dashboard */}
1218 |               {attemptsStats && (
1219 |                 <div className="grid grid-cols-3 md:grid-cols-6 gap-4 mb-8">
1220 |                   <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
1221 |                     <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Total Trainees
</p>
1222 |                     <p className="text-2xl font-black text-slate-900">{enrollmentsList.length}</p>
1223 |                   </div>
1224 |                   <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
1225 |                     <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Trainees Attemp
ted</p>
1226 |                     <p className="text-2xl font-black text-slate-900">{attemptsStats.total}</p>
1227 |                   </div>
1228 |                   <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
1229 |                     <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Passed</p>
1230 |                     <p className="text-2xl font-black text-emerald-600">{attempts.filter(a => a.passed).length}</p>
1231 |                   </div>
1232 |                   <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
1233 |                     <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Failed</p>
1234 |                     <p className="text-2xl font-black text-rose-600">{attempts.filter(a => !a.passed && a.score !== -1).lengt
h}</p>
1235 |                   </div>
1236 |                   <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
1237 |                     <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Average Score</
```

```
p>
                    <p className="text-2xl font-black text-cyan-600">{attemptsStats.avgScore}%</p>
                  </div>
                  <div className="bg-white p-5 rounded-2xl border border-slate-200/90 shadow-sm flex flex-col justify-center
items-center">
                    <p className="text-xs text-slate-500 uppercase font-bold tracking-wider mb-1 text-center">Pass Ratio</p>
                    <p className={`text-2xl font-black ${attemptsStats.passRatio >= 50 ? 'text-emerald-600' : 'text-rose-60
0'}`}>{attemptsStats.passRatio}%</p>
                  </div>
                </div>
              )}
            </motion.div>

            {/* Tabs */}
            <div className="flex border-b border-slate-200 mb-6">
              <button
                onClick={() => setActiveTab('questions')}
                className={`px-6 py-3 text-sm font-bold border-b-2 transition-colors ${activeTab === 'questions' ? 'border-
cyan-600 text-cyan-600' : 'border-transparent text-slate-500 hover:text-slate-800'}`}
              >
                Questions
              </button>
              <button
                onClick={() => setActiveTab('grading')}
                className={`px-6 py-3 text-sm font-bold border-b-2 transition-colors ${activeTab === 'grading' ? 'border-cy
an-600 text-cyan-600' : 'border-transparent text-slate-500 hover:text-slate-800'}`}
              >
                Results & Grading Queue
                {attempts.filter(a => a.grade_status === 'pending_manual').length > 0 && (
                  <span className="ml-2 px-2 py-0.5 bg-rose-500 text-white text-[10px] rounded-full font-bold">
                    {attempts.filter(a => a.grade_status === 'pending_manual').length}
                  </span>
                )}
              </button>
            </div>

            {activeTab === 'questions' ? (
              questions.length === 0 ? (
              <motion.div variants={fadeUp}>
                <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl">
                  <CardContent className="py-12 text-center">
                    <p className="text-slate-500 font-medium">No questions yet. Add your first question.</p>
                  </CardContent>
                </Card>
              </motion.div>
            ) : (
              <motion.div variants={fadeUp} className="space-y-3">
                {questions.map((q) => {
                  const opts = q.options as Record<string, string>
                  return (
                    <div key={q.id} className="p-5 rounded-2xl bg-white border border-slate-200/90 hover:border-slate-300 sha
dow-sm transition-all">
                      <div className="flex items-start justify-between gap-3">
                        <div className="flex-1">
                          <h4 className="font-bold text-slate-900 text-base mb-1">Q{q.position}. {q.question_text}</h4>
                          <span className="text-xs px-2.5 py-0.5 rounded-full bg-slate-100 text-slate-600 font-semibold capit
alize mb-3 inline-block">
                            {(q as any).question_type?.replace('_', ' ') || 'Question'}
                          </span>
                          <div className="grid grid-cols-2 gap-2 text-xs">
                            {(q as any).question_type === 'open_ended' ? (
                              <div className="col-span-2 px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-s
late-700 italic font-medium">
                                Open-Ended Question. Reference Answer: {q.correct_answer}
                              </div>
                            ) : (
                              ['A', 'B', 'C', 'D'].map(opt => (
                                <div key={opt} className={`px-3.5 py-2.5 rounded-xl border ${
                                  q.correct_answer === opt
                                    ? 'bg-emerald-50 border-emerald-300 text-emerald-900 font-semibold'
                                    : 'bg-slate-50 border-slate-200 text-slate-700 font-medium'
                                }`}>
                                  <span className="font-bold mr-1">{opt}.</span> {opts[opt]}
                                </div>
                              ))
                            )}
                          </div>
                          {q.explanation && (
                            <p className="text-xs text-slate-500 mt-2.5 italic">Explanation: {q.explanation}</p>
```

```
                          )}
                        </div>
                        <div className="flex gap-1 shrink-0">
                          <button onClick={() => openEditQuestion(q)} className="p-2 rounded-lg hover:bg-slate-100 text-slate
-500 hover:text-slate-800 transition-all">
                            <FileText className="w-4 h-4" />
                          </button>
                          <button onClick={() => handleDeleteQuestion(q.id)} className="p-2 rounded-lg hover:bg-red-50 text-s
late-500 hover:text-red-600 transition-all">
                            <Trash2 className="w-4 h-4" />
                          </button>
                        </div>
                      </div>
                    </div>
                  )
                })}
              </motion.div>
            )) : (
              <motion.div variants={fadeUp} className="space-y-4">
                {attempts.length === 0 && enrollmentsList.length === 0 ? (
                  <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl">
                    <CardContent className="py-12 text-center">
                      <p className="text-slate-500 font-medium">No trainees enrolled or attempts made in this course yet.</p>
                    </CardContent>
                  </Card>
                ) : (
                  <>
                  {[...attempts].sort((a, b) => (a.profiles?.first_name || '').localeCompare(b.profiles?.first_name || '')).m
ap((att) => {
                      return (
                    <Card key={att.id} className="bg-white border border-slate-200/90 shadow-sm rounded-3xl overflow-hidden">
                      <CardHeader className={`border-b py-3.5 px-5 ${att.score === -1 ? 'bg-red-50 border-red-100' : 'bg-slat
e-50 border-slate-100'}`}>
                        <div className="flex items-center justify-between">
                          <div className="flex items-center gap-2">
                            <span className="font-bold text-slate-900">{att.profiles?.first_name} {att.profiles?.last_name}</
span>
                            <span className="text-xs text-slate-400 font-medium">{new Date(att.submitted_at).toLocaleString
()}</span>
                          </div>
                          {att.score === -1 ? (
                             <div className="flex items-center gap-2">
                               <Badge className="bg-red-100 text-red-800 hover:bg-red-100">
                                 Blocked (Suspicious Activity)
                               </Badge>
                               <Button
                                 onClick={() => handleUnblockAttempt(att.id)}
                                 size="sm"
                                 className="bg-red-600 hover:bg-red-700 text-white rounded-lg h-7 px-3 text-xs"
                               >
                                 Unblock
                               </Button>
                           </div>
                          ) : (
                          <div className="flex items-center gap-3">
                             {att.passed ? (
                                <Badge className="bg-emerald-100 text-emerald-800 hover:bg-emerald-100 border-0">Passed</Badg
e>
                             ) : (
                                <Badge className="bg-rose-100 text-rose-800 hover:bg-rose-100 border-0">Failed</Badge>
                             )}
                             <Badge className={att.grade_status === 'pending_manual' ? 'bg-amber-100 text-amber-800 hover:bg-
amber-100' : 'bg-cyan-100 text-cyan-800 hover:bg-cyan-100'}>
                               {att.grade_status === 'pending_manual' ? 'Needs Grading' : `Score: ${att.score}%`}
                             </Badge>
                             <Button
                               onClick={() => handleDeleteAttempt(att)}
                               variant="outline"
                               size="sm"
                               className="border-rose-200 text-rose-700 hover:bg-rose-50 hover:text-rose-800 h-7 text-xs px-2
rounded-lg"
                               title="Delete this attempt and allow the trainee to re-take the test"
                             >
                               <RefreshCw className="w-3.5 h-3.5 mr-1.5" /> Allow Retest
                             </Button>
                          </div>
                          )}
                        </div>
                      </CardHeader>
```

```
                      <CardContent className="p-0">
                        {att.score === -1 ? (
                          <div className="text-center py-6 px-5">
                            <p className="text-red-600 font-medium text-sm">This attempt was automatically blocked by the sys
tem due to repeated Secure Exam Area (SEA) violations.</p>
                            <p className="text-red-500/80 text-xs mt-1">Unblocking will delete this attempt record, allowing
the trainee to retake the assessment.</p>
                          </div>
                        ) : (
                          <>
                            <div className="p-5 bg-slate-50 border-b border-slate-100">
                               <Button variant="outline" className="w-full text-cyan-700 border-cyan-200 hover:bg-cyan-50" on
Click={() => setViewingResultForAttempt(att)}>
                                 <Eye className="w-4 h-4 mr-2" /> View Detailed Result
                               </Button>
                            </div>

                            <div className="flex flex-wrap items-center gap-3 p-5 bg-white">
                              <Label className="font-bold text-slate-900 whitespace-nowrap">Manual Correction Score (0-100):
</Label>
                              <Input
                                type="number"
                                min="0" max="100"
                                className="w-24 bg-slate-50 border-slate-200 text-slate-900 rounded-xl"
                                placeholder={att.score?.toString()}
                                value={gradingAttemptId === att.id ? gradingScore : (att.score ?? '')}
                                onChange={(e) => {
                                  setGradingAttemptId(att.id)
                                  setGradingScore(parseInt(e.target.value) || 0)
                                }}
                              />
                              <Button
                                size="sm"
                                className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-semibo
ld rounded-xl"
                                onClick={() => handleGradeAttempt(att.id)}
                                disabled={saving || gradingAttemptId !== att.id}
                              >
                                Save Grade
                              </Button>

                              {att.grade_status === 'pending_manual' && (
                                <Button
                                  size="sm"
                                  className="bg-purple-600 hover:bg-purple-700 text-white font-semibold rounded-xl ml-auto fl
ex items-center gap-2"
                                  onClick={() => handleAutoGradeAttempt(att.id)}
                                  disabled={saving}
                                >
                                  <Sparkles className="w-4 h-4" /> AI Auto-Grade
                                </Button>
                              )}
                            </div>
                          </>
                        )}
                      </CardContent>
                    </Card>
                          )
                        })}
                    {enrollmentsList.filter(enr => !attempts.find(a => a.user_id === enr.user_id)).map((enr) => {
                        return (
                          <Card key={enr.user_id} className="bg-white border border-slate-200/90 shadow-sm rounded-3xl overflow
-hidden">
                            <CardHeader className="border-b py-3.5 px-5 bg-slate-50 border-slate-100">
                              <div className="flex items-center justify-between">
                                <div className="flex items-center gap-2">
                                  <span className="font-bold text-slate-900">{enr.profiles?.first_name} {enr.profiles?.last_nam
e}</span>
                                  <span className="text-xs text-slate-400 font-medium">Not Attempted</span>
                                </div>
                                <Badge className="bg-slate-200 text-slate-600 hover:bg-slate-200">
                                  Missing Submission
                                </Badge>
                              </div>
                            </CardHeader>
                            <CardContent className="p-5 space-y-6">
                              <div className="text-center py-4 text-slate-500 text-sm">
                                Trainee has not submitted this assessment online. You can manually enter a score if they took i
t offline.
```

```
                            </div>
                            <div className="flex items-center gap-3 pt-4 border-t border-slate-100">
                                <Label className="font-bold text-slate-900 whitespace-nowrap">Manual Score (0-100):</Label>
                                <Input
                                  type="number"
                                  min="0" max="100"
                                  className="w-24 bg-slate-50 border-slate-200 text-slate-900 rounded-xl"
                                  placeholder="0"
                                  value={gradingAttemptId === enr.user_id ? gradingScore : ''}
                                  onChange={(e) => {
                                    setGradingAttemptId(enr.user_id)
                                    setGradingScore(parseInt(e.target.value) || 0)
                                  }}
                                />
                                <Button
                                  size="sm"
                                  className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-semi
bold rounded-xl"
                                  onClick={() => handleManualGradeSubmission(enr.user_id)}
                                  disabled={saving || gradingAttemptId !== enr.user_id}
                                >
                                  Submit Score
                                </Button>
                            </div>
                          </CardContent>
                        </Card>
                      )
                    })}
                    </>
                  )}
                </motion.div>
              )}
            </>
          )}

          {/* Question Dialog */}
          <Dialog open={questionDialogOpen} onOpenChange={(o) => { if (!o) { setQuestionDialogOpen(false); setEditingQuestionId
(null) } }}>
            <DialogContent className="max-w-lg bg-white border border-slate-200/90 shadow-2xl rounded-3xl text-slate-900">
              <DialogHeader>
                <DialogTitle className="text-slate-900 font-black text-xl">{editingQuestionId ? 'Edit Question' : 'Add Questio
n'}</DialogTitle>
              </DialogHeader>
              <div className="space-y-4">
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs">Question Type</Label>
                  <Select value={editingQuestion.question_type} onValueChange={(v: 'mcq' | 'open_ended') => setEditingQuestion
(p => ({ ...p, question_type: v }))}>
                    <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"><SelectValue /></SelectTr
igger>
                    <SelectContent>
                      <SelectItem value="mcq">Multiple Choice</SelectItem>
                      <SelectItem value="open_ended">Open-Ended (Text)</SelectItem>
                    </SelectContent>
                  </Select>
                </div>
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs">Difficulty</Label>
                  <Select value={editingQuestion.difficulty} onValueChange={(v: 'easy' | 'medium' | 'hard') => setEditingQuesti
on(p => ({ ...p, difficulty: v }))}>
                    <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"><SelectValue /></SelectTr
igger>
                    <SelectContent>
                      <SelectItem value="easy">Easy</SelectItem>
                      <SelectItem value="medium">Medium</SelectItem>
                      <SelectItem value="hard">Hard</SelectItem>
                    </SelectContent>
                  </Select>
                </div>
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs">Question *</Label>
                  <Textarea value={editingQuestion.question_text} onChange={e => setEditingQuestion(p => ({ ...p, question_tex
t: e.target.value }))} rows={3} className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" />
                </div>

                {editingQuestion.question_type === 'mcq' ? (
                  <>
                    {['A', 'B', 'C', 'D'].map(opt => (
                      <div key={opt} className="space-y-1.5">
```

```
                      <Label className="text-slate-700 font-semibold text-xs">Option {opt} *</Label>
                      <Input value={editingQuestion[`option_${opt.toLowerCase()}` as keyof QuestionForm] as string}
                        onChange={e => setEditingQuestion(p => ({ ...p, [`option_${opt.toLowerCase()}`]: e.target.value }))}
                        className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" />
                    </div>
                  ))}
                  <div className="grid grid-cols-2 gap-4">
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs">Correct Answer</Label>
                      <select value={editingQuestion.correct_answer}
                        onChange={e => setEditingQuestion(p => ({ ...p, correct_answer: e.target.value }))}
                        className="w-full h-10 px-3 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-sm app
earance-none cursor-pointer focus:outline-none focus:ring-2 focus:ring-cyan-500">
                        <option value="A">A</option>
                        <option value="B">B</option>
                        <option value="C">C</option>
                        <option value="D">D</option>
                      </select>
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs">Explanation</Label>
                      <Input value={editingQuestion.explanation} onChange={e => setEditingQuestion(p => ({ ...p, explanation:
e.target.value }))} className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" placeholder="Optional" />
                    </div>
                  </div>
                </>
              ) : (
                <div className="space-y-4">
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 font-semibold text-xs">Reference Answer / Grading Rubric</Label>
                    <Textarea value={editingQuestion.correct_answer}
                      onChange={e => setEditingQuestion(p => ({ ...p, correct_answer: e.target.value }))}
                      className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" rows={3} placeholder="What should a
good answer contain?" />
                  </div>
                </div>
              )}
            </div>
            <DialogFooter>
              <Button variant="outline" onClick={() => { setQuestionDialogOpen(false); setEditingQuestionId(null) }} classNam
e="border-slate-200 text-slate-700 hover:bg-slate-50 rounded-xl">Cancel</Button>
              <Button onClick={handleSaveQuestion} disabled={saving} className="bg-gradient-to-r from-cyan-600 to-blue-600 ho
ver:opacity-95 text-white font-semibold rounded-xl shadow-sm">
                {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
                {editingQuestionId ? 'Update' : 'Add'} Question
              </Button>
            </DialogFooter>
          </DialogContent>
        </Dialog>

        {/* Assessment Settings Dialog */}
        <Dialog open={assessmentDialogOpen} onOpenChange={setAssessmentDialogOpen}>
          <DialogContent className="w-[95vw] max-w-lg bg-white border border-slate-200/90 shadow-2xl rounded-3xl text-slate-9
00 flex flex-col max-h-[90vh] p-0 overflow-hidden">
            {/* Scrollable body */}
            <div className="overflow-y-auto flex-1 px-4 sm:px-6 pt-6 pb-2">
              <DialogHeader className="mb-4">
                <DialogTitle className="text-slate-900 font-black text-xl">{editingAssessmentId ? 'Edit Test Details' : 'New
Test / Assessment'}</DialogTitle>
              </DialogHeader>
              <div className="space-y-4">
                {(() => {
                  const editingAssessment = assessments.find(a => a.id === editingAssessmentId);

                  return (
                    <>
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs">Title</Label>
                  <Input value={assessmentForm.title} onChange={e => setAssessmentForm({...assessmentForm, title: e.target.va
lue})} placeholder="e.g. Midterm Mock Test" className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl h-10" />
                </div>
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs">Assessment Type</Label>
                  <Select disabled={!!editingAssessmentId} value={assessmentForm.assessment_type} onValueChange={(v) => setAs
sessmentForm({ ...assessmentForm, assessment_type: v as any })}>
                    <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl disabled:opacity-60"><Se
lectValue /></SelectTrigger>
                    <SelectContent>
                      <SelectItem value="daily">Daily Assessment (Trainer Can Publish)</SelectItem>
```

```
                        <SelectItem value="assessment">Assessment Test (Trainer Can Publish)</SelectItem>
                        <SelectItem value="mock">Mock Test (Trainer Can Publish)</SelectItem>
                        <SelectItem value="final">Final Exam (Requires Admin Approval)</SelectItem>
                      </SelectContent>
                    </Select>
                  </div>

                  <div className="space-y-1.5">
                    <Label className="text-slate-700 font-semibold text-xs">Scheduled Date</Label>
                    <Input
                      type="date"
                      value={assessmentForm.scheduled_date}
                      onChange={e => setAssessmentForm({...assessmentForm, scheduled_date: e.target.value})}
                      className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl h-10"
                    />
                  </div>

                  <div className="space-y-1.5 bg-slate-50 p-3 rounded-xl border border-slate-200">
                    <Label className="text-slate-700 font-semibold text-xs">Passing Score</Label>
                    <p className="text-sm font-bold text-slate-900">
                      {editingAssessmentId && questions.length > 0
                        ? `${Math.round(questions.length * 10 * (assessmentForm.passing_score / 100))} marks (${assessmentForm.
passing_score}%)`
                        : `${assessmentForm.passing_score}% of total marks (Auto-calculated)`}
                    </p>
                    <p className="text-[10px] text-slate-500">Passing score is automatically set based on the number of questio
ns.</p>
                  </div>

                  <div className="grid grid-cols-2 gap-3">
                    <div className="flex items-center space-x-2 border border-cyan-200 p-3 rounded-xl bg-cyan-50/50 transition-
all">
                      <input type="checkbox" id="is_adaptive" className="w-4 h-4 rounded text-cyan-600 border-cyan-300" checked
={assessmentForm.is_adaptive} onChange={(e) => setAssessmentForm({ ...assessmentForm, is_adaptive: e.target.checked })} />
                      <Label htmlFor="is_adaptive" className="text-cyan-800 font-bold flex-1 cursor-pointer text-xs">Adaptive M
CQ Mode</Label>
                    </div>
                    <div className="flex items-center space-x-2 border border-emerald-200 p-3 rounded-xl bg-emerald-50 transiti
on-all">
                      <input type="checkbox" id="is_simulation" className="w-4 h-4 rounded text-emerald-600 border-emerald-300"
checked={assessmentForm.is_simulation} onChange={(e) => setAssessmentForm({ ...assessmentForm, is_simulation: e.target.checked })} /
>
                      <Label htmlFor="is_simulation" className="text-emerald-800 font-bold flex-1 cursor-pointer text-xs">IMD S
imulation Mode</Label>
                    </div>
                  </div>

                  {assessmentForm.is_simulation && (
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs">Dataset URL / Image Link (Optional)</Label>
                      <Input value={assessmentForm.simulation_dataset_url} onChange={e => setAssessmentForm({...assessmentForm,
simulation_dataset_url: e.target.value})} placeholder="https://example.com/weather-chart.jpg" className="bg-slate-50 border-slate-20
0 text-slate-900 rounded-xl h-10" />
                      <p className="text-[10px] text-slate-500">Link to weather chart or dataset to display alongside question
s.</p>
                    </div>
                  )}

                  <div className="grid grid-cols-3 gap-3">
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs">Start Time</Label>
                      <Input type="time" value={assessmentForm.start_time} onChange={e => setAssessmentForm({...assessmentForm,
start_time: e.target.value})} className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl h-10 disabled:opacity-60" />
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs">End Time</Label>
                      <Input type="time" value={assessmentForm.end_time} onChange={e => setAssessmentForm({...assessmentForm, e
nd_time: e.target.value})} className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl h-10 disabled:opacity-60" />
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs whitespace-nowrap overflow-hidden text-ellipsis">D
uration (hh:mm:ss)</Label>
                      <Input type="time" step="1" value={minutesToTimeStr(assessmentForm.duration_minutes)} onChange={e => setA
ssessmentForm({...assessmentForm, duration_minutes: timeStrToMinutes(e.target.value)})} className="bg-slate-50 border-slate-200 text
-slate-900 rounded-xl h-10" />
                    </div>
                  </div>
                  {assessmentForm.start_time && assessmentForm.end_time && (
                    <p className="text-[11px] text-slate-500 font-medium -mt-1">
```

```
                    Time window reference: <span className="text-cyan-600 font-semibold">{calculateWindowDuration(assessmentForm.start_time, assessmentForm.end_time)}</span> (hh:mm:ss)
                  </p>
                )}

                  </>
                );
              })()}

              <div
                className={`flex items-center space-x-2 border p-3 rounded-xl transition-all cursor-pointer ${
                  assessmentForm.requires_sea
                    ? 'border-cyan-400 bg-cyan-50 shadow-sm'
                    : 'border-slate-200 bg-slate-50'
                }`}
                onClick={() => setAssessmentForm({ ...assessmentForm, requires_sea: !assessmentForm.requires_sea })}
              >
                <input
                  type="checkbox"
                  id="requires_sea"
                  className="w-4 h-4 rounded accent-cyan-600 border-cyan-300 cursor-pointer"
                  checked={assessmentForm.requires_sea}
                  onChange={e => setAssessmentForm({ ...assessmentForm, requires_sea: e.target.checked })}
                  onClick={e => e.stopPropagation()}
                />
                <div className="flex-1">
                  <Label htmlFor="requires_sea" className="text-cyan-800 font-bold text-xs cursor-pointer block">Require SEA (Secure Exam Mode)</Label>
                  <p className="text-[10px] text-slate-500 mt-0.5">Enable secure browser lock during the exam.</p>
                </div>
              </div>
            </div>

            <div className="space-y-1.5 border-t border-slate-100 pt-4 mt-4">
              <Label className="text-slate-700 font-semibold text-xs">Result Publication</Label>
              <div className="flex flex-col gap-3 mb-2 bg-slate-50 border border-slate-200 rounded-xl px-4 py-3">
                <div className="flex items-center gap-2">
                  <input type="checkbox" id="instant_publish" className="w-4 h-4 rounded text-indigo-600 border-indigo-300" checked={!assessmentForm.results_publish_date} onChange={(e) => setAssessmentForm({ ...assessmentForm, results_publish_date: e.target.checked ? '' : new Date().toISOString().slice(0, 10) })} />
                  <Label htmlFor="instant_publish" className="text-indigo-800 font-bold flex-1 cursor-pointer text-xs">Instant Result Publication</Label>
                </div>
                {assessmentForm.results_publish_date !== '' && (
                  <div className="space-y-1.5 border-t border-slate-200 pt-3 mt-1">
                    <Label className="text-slate-500 font-semibold text-[10px] uppercase">Scheduled Publish Date</Label>
                    <Input type="date" value={assessmentForm.results_publish_date} onChange={e => setAssessmentForm({...assessmentForm, results_publish_date: e.target.value})} className="bg-white border-slate-200 text-slate-900 rounded-lg h-9 text-sm" />
                  </div>
                )}
              </div>
            </div>

          </div>

          {/* Sticky footer */}
          <div className="flex justify-end gap-3 px-4 sm:px-6 py-4 border-t border-slate-100 bg-white shrink-0">
            <Button variant="outline" onClick={() => setAssessmentDialogOpen(false)} className="border-slate-200 text-slate-700 hover:bg-slate-50 rounded-xl">Cancel</Button>
            <Button onClick={handleSaveAssessment} disabled={saving} className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-semibold rounded-xl shadow-sm">
              {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
              Save
            </Button>
          </div>
        </DialogContent>
      </Dialog>

      <Dialog open={aiGenDialogOpen} onOpenChange={setAiGenDialogOpen}>
        <DialogContent className="w-[95vw] max-w-lg bg-white border border-slate-200/90 shadow-2xl rounded-3xl overflow-hidden p-0 text-slate-900 flex flex-col max-h-[90vh]">
          {/* Scrollable body */}
          <div className="overflow-y-auto flex-1 p-4 sm:p-6">
            <DialogHeader className="mb-5">
              <DialogTitle className="text-lg sm:text-xl font-black text-slate-900 flex items-center gap-3">
                <div className="p-2 bg-cyan-100 rounded-xl border border-cyan-200 text-cyan-600 shrink-0">
                  <Brain className="w-5 h-5"/>
                </div>
                Auto-Generate Test
```

```
                </DialogTitle>
                <p className="text-slate-500 text-sm mt-1 font-medium">Harness AI to instantly create a highly effective asse
ssment.</p>
              </DialogHeader>

              <div className="space-y-5">
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Test Title <span className
="text-red-500">*</span></Label>
                  <Input
                    placeholder="e.g. Advanced Marine Biology Assessment..."
                    value={aiGenForm.title}
                    onChange={e => setAiGenForm({...aiGenForm, title: e.target.value})}
                    className="bg-slate-50 border-slate-200 text-slate-900 h-11 rounded-xl"
                  />
                </div>

                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Assessment Type</Label>
                  <Select value={aiGenForm.type} onValueChange={v => setAiGenForm({...aiGenForm, type: v})}>
                    <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 h-11 rounded-xl">
                      <SelectValue />
                    </SelectTrigger>
                    <SelectContent>
                      <SelectItem value="daily_test">Daily Test (Trainer Can Publish)</SelectItem>
                      <SelectItem value="assessment_test">Assessment Test (Trainer Can Publish)</SelectItem>
                      <SelectItem value="mock_test">Mock Test (Trainer Can Publish)</SelectItem>
                      <SelectItem value="final">Final Exam (Requires Admin Approval)</SelectItem>
                    </SelectContent>
                  </Select>
                </div>

                {/* ── Scheduling fields (vary by type) ── */}
                {aiGenForm.type === 'daily_test' ? (
                  <div className="grid grid-cols-2 gap-3">
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap-
1.5">

                        Test Date <span className="text-red-500">*</span>
                      </Label>
                      <input
                        type="date"
                        value={aiGenForm.scheduled_date}
                        onChange={e => setAiGenForm({ ...aiGenForm, scheduled_date: e.target.value })}
                        className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm foc
us:outline-none focus:ring-2 focus:ring-cyan-500"
                      />
                      <p className="text-[11px] text-slate-400">Trainees can attend on this date only. Result published insta
ntly. Expires after this day.</p>
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap-
1.5 whitespace-nowrap">
                        Duration (hh:mm:ss) <span className="text-red-500">*</span>
                      </Label>
                      <input
                        type="time"
                        step="1"
                        value={minutesToTimeStr(aiGenForm.duration_minutes)}
                        onChange={e => setAiGenForm({ ...aiGenForm, duration_minutes: timeStrToMinutes(e.target.value) })}
                        className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm foc
us:outline-none focus:ring-2 focus:ring-cyan-500"
                      />
                    </div>
                  </div>
                ) : (
                  <div className="space-y-3">
                    <div className="grid grid-cols-2 gap-3">
                      <div className="space-y-1.5">
                        <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap
-1">

                          Test Date <span className="text-red-500">*</span>
                        </Label>
                        <input
                          type="date"
                          value={aiGenForm.scheduled_date}
                          onChange={e => setAiGenForm({ ...aiGenForm, scheduled_date: e.target.value })}
                          className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm f
ocus:outline-none focus:ring-2 focus:ring-cyan-500"
```

```
                        />
                      </div>
                      <div className="space-y-1.5">
                        <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap
-1 whitespace-nowrap">
                          Duration (hh:mm:ss) <span className="text-red-500">*</span>
                        </Label>
                        <input
                          type="time"
                          step="1"
                          value={minutesToTimeStr(aiGenForm.duration_minutes)}
                          onChange={e => setAiGenForm({ ...aiGenForm, duration_minutes: timeStrToMinutes(e.target.value) })}
                          className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm f
ocus:outline-none focus:ring-2 focus:ring-cyan-500"
                        />
                      </div>
                      <div className="space-y-1.5">
                        <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap
-1">
                          Start Time <span className="text-red-500">*</span>
                        </Label>
                        <input
                          type="time"
                          value={aiGenForm.start_time}
                          onChange={e => setAiGenForm({ ...aiGenForm, start_time: e.target.value })}
                          className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm f
ocus:outline-none focus:ring-2 focus:ring-cyan-500"
                        />
                      </div>
                      <div className="space-y-1.5">
                        <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">
                          End Time
                        </Label>
                        <input
                          type="time"
                          value={aiGenForm.end_time}
                          onChange={e => setAiGenForm({ ...aiGenForm, end_time: e.target.value })}
                          className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm f
ocus:outline-none focus:ring-2 focus:ring-cyan-500"
                        />
                      </div>
                    </div>
                    {aiGenForm.start_time && aiGenForm.end_time && (
                      <p className="text-[11px] text-slate-500 font-medium">
                        Time window reference: <span className="text-cyan-600 font-semibold">{calculateWindowDuration(aiGenFo
rm.start_time, aiGenForm.end_time)}</span> (hh:mm:ss)
                      </p>
                    )}

                    <div className="space-y-1.5 bg-slate-50 p-4 rounded-xl border border-slate-200">
                      <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider block mb-1">
                        Passing Score
                      </Label>
                      <p className="text-lg font-black text-cyan-700">
                        {Math.round(aiGenForm.count * aiGenForm.marks_per_question * (aiGenForm.passing_score / 100))} <span
className="text-sm font-bold text-slate-500">marks</span>
                      </p>
                      <p className="text-[11px] font-medium text-slate-500">Calculated as {aiGenForm.passing_score}% of {aiGe
nForm.count * aiGenForm.marks_per_question} total marks.</p>
                    </div>
                  </div>
                )}

                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap-1">
                    Result Publication
                  </Label>
                  <div
                    className="flex items-center space-x-3 border border-slate-200 p-3 rounded-xl transition-all cursor-point
er bg-slate-50 hover:bg-slate-100"
                    onClick={() => setAiGenForm({ ...aiGenForm, instant_result_publication: !aiGenForm.instant_result_publica
tion })}
                  >
                    <input
                      type="checkbox"
                      className="w-4 h-4 rounded accent-blue-600 cursor-pointer shrink-0"
                      checked={aiGenForm.instant_result_publication}
                      onChange={e => setAiGenForm({ ...aiGenForm, instant_result_publication: e.target.checked })}
                      onClick={e => e.stopPropagation()}
```

```
                    />
                    <div className="flex-1 font-bold text-sm text-[#3b34b1]">
                      Instant Result Publication
                    </div>
                  </div>
                  {!aiGenForm.instant_result_publication && (
                    <div className="mt-3">
                      <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider flex items-center gap-
1">

                        Result Publication Date (Optional)
                      </Label>
                      <input
                        type="date"
                        value={aiGenForm.results_publish_date}
                        onChange={e => setAiGenForm({ ...aiGenForm, results_publish_date: e.target.value })}
                        className="w-full h-11 rounded-xl border border-slate-200 bg-slate-50 text-slate-900 px-3 text-sm foc
us:outline-none focus:ring-2 focus:ring-cyan-500 mt-1"
                        />
                        <p className="text-[11px] text-slate-400 mt-1">If set, results and answers will be hidden from trainees
until this date. Leave blank to hide indefinitely.</p>
                    </div>
                  )}
                </div>

                <div className="space-y-1.5">
                  <div className="flex items-center justify-between">
                    <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Source Material (Optiona
l)</Label>
                    {materials.length > 0 && (
                      <button
                        type="button"
                        className="text-xs text-cyan-600 hover:text-cyan-700 font-medium"
                        onClick={() => {
                          if (aiGenForm.material_ids.length === materials.length) {
                            setAiGenForm({ ...aiGenForm, material_ids: [] })
                          } else {
                            setAiGenForm({ ...aiGenForm, material_ids: materials.map(m => m.id) })
                          }
                        }}
                      >
                        {aiGenForm.material_ids.length === materials.length ? 'Deselect All' : 'Select All'}
                      </button>
                    )}
                  </div>
                  <div className="border border-slate-200 rounded-xl bg-slate-50 divide-y divide-slate-100 max-h-40 overflow-
y-auto">
                    {materials.length === 0 ? (
                      <p className="text-xs text-slate-400 px-3 py-3">No materials uploaded yet.</p>
                    ) : (
                      materials.map(m => {
                        const checked = aiGenForm.material_ids.includes(m.id)
                        return (
                          <label
                            key={m.id}
                            className={`flex items-center gap-3 px-3 py-2.5 cursor-pointer hover:bg-slate-100 transition-colo
rs ${
                              checked ? 'bg-cyan-50' : ''
                            }`}
                          >
                            <input
                              type="checkbox"
                              checked={checked}
                              onChange={() => {
                                const ids = checked
                                  ? aiGenForm.material_ids.filter(id => id !== m.id)
                                  : [...aiGenForm.material_ids, m.id]
                                setAiGenForm({ ...aiGenForm, material_ids: ids })
                              }}
                              className="w-4 h-4 accent-cyan-600 rounded shrink-0"
                            />
                            <span className="text-sm text-slate-800 truncate flex-1">{m.file_name}</span>
                          </label>
                        )
                      })
                    )}
                  </div>
                  {aiGenForm.material_ids.length > 0 && (
                    <p className="text-xs text-cyan-600 font-medium">{aiGenForm.material_ids.length} material{aiGenForm.mater
ial_ids.length > 1 ? 's' : ''} selected</p>
```

```
                  )}
                </div>

                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Topic / Instructions for A
I {aiGenForm.material_ids.length > 0 && '(Optional)'}</Label>
                  <div className="relative">
                    <Textarea
                      placeholder="e.g. Generate a test about advanced marine biology and coral reefs..."
                      value={aiGenForm.topic}
                      onChange={e => setAiGenForm({...aiGenForm, topic: e.target.value})}
                      className="bg-slate-50 border-slate-200 text-slate-900 h-24 rounded-xl resize-none p-3 placeholder:text
-slate-400"
                    />
                  </div>
                </div>

                <div className="grid grid-cols-3 gap-3">
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider whitespace-nowrap overflo
w-hidden text-ellipsis">No. of Questions</Label>
                    <Input
                      type="number"
                      min="1"
                      max="50"
                      value={aiGenForm.count}
                      onChange={e => setAiGenForm({...aiGenForm, count: parseInt(e.target.value) || 5})}
                      className="bg-slate-50 border-slate-200 text-slate-900 h-11 rounded-xl"
                    />
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider whitespace-nowrap overflo
w-hidden text-ellipsis">Marks per Q.</Label>
                    <Input
                      type="number"
                      min="1"
                      value={aiGenForm.marks_per_question}
                      onChange={e => setAiGenForm({...aiGenForm, marks_per_question: parseInt(e.target.value) || 10})}
                      className="bg-slate-50 border-slate-200 text-slate-900 h-11 rounded-xl"
                    />
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Difficulty Level</Label>
                    <Select value={aiGenForm.difficulty} onValueChange={v => setAiGenForm({...aiGenForm, difficulty: v})}>
                      <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 h-11 rounded-xl">
                        <SelectValue />
                      </SelectTrigger>
                      <SelectContent>
                        <SelectItem value="mixed">Mixed Combinations</SelectItem>
                        <SelectItem value="easy">Easy</SelectItem>
                        <SelectItem value="medium">Medium</SelectItem>
                        <SelectItem value="hard">Hard</SelectItem>
                      </SelectContent>
                    </Select>
                  </div>
                </div>

                {/* Question Format */}
                <div className="space-y-1.5">
                  <Label className="text-slate-700 font-semibold text-xs uppercase tracking-wider">Question Format</Label>
                  <div className="grid grid-cols-3 gap-2">
                    {([['mcq', 'MCQ Only'], ['open_ended', 'Q&A Only'], ['both', 'MCQ + Q&A']] as const).map(([val, label]) =
> (
                      <button
                        key={val}
                        type="button"
                        onClick={() => setAiGenForm({ ...aiGenForm, question_format: val })}
                        className={`py-2 px-2 rounded-xl text-xs font-semibold border transition-all ${
                          aiGenForm.question_format === val
                            ? 'bg-cyan-600 text-white border-cyan-600 shadow-sm'
                            : 'bg-slate-50 text-slate-600 border-slate-200 hover:border-cyan-300'
                        }`}
                      >
                        {label}
                      </button>
                    ))}
                  </div>
                </div>

```

```
                {/* SEA toggle */}
                <div
                  className={`flex items-center space-x-3 border p-3 rounded-xl transition-all cursor-pointer ${
                    aiGenForm.requires_sea
                      ? 'border-cyan-400 bg-cyan-50 shadow-sm'
                      : 'border-slate-200 bg-slate-50'
                  }`}
                  onClick={() => setAiGenForm({ ...aiGenForm, requires_sea: !aiGenForm.requires_sea })}
                >
                  <input
                    type="checkbox"
                    id="ai_requires_sea"
                    className="w-4 h-4 rounded accent-cyan-600 border-cyan-300 cursor-pointer shrink-0"
                    checked={aiGenForm.requires_sea}
                    onChange={e => setAiGenForm({ ...aiGenForm, requires_sea: e.target.checked })}
                    onClick={e => e.stopPropagation()}
                  />
                  <div className="flex-1">
                    <Label htmlFor="ai_requires_sea" className="text-cyan-800 font-bold text-xs cursor-pointer block">Require SEA (Secure Exam Mode)</Label>
                    <p className="text-[10px] text-slate-500 mt-0.5">Enable secure browser lock during the exam.</p>
                  </div>
                </div>
              </div>
            </div>

            {/* Sticky footer */}
            <div className="flex justify-end gap-3 px-4 sm:px-6 py-4 border-t border-slate-100 bg-white shrink-0">
              <Button variant="outline" onClick={() => setAiGenDialogOpen(false)} className="border-slate-200 text-slate-700 hover:bg-slate-50 h-11 rounded-xl px-5">
                Cancel
              </Button>
              <Button onClick={handleAIGenerate} disabled={saving} className="bg-gradient-to-r from-cyan-600 to-blue-600 text-white hover:opacity-95 font-semibold h-11 rounded-xl px-6 shadow-md shadow-cyan-600/10">
                {saving ? (
                  <><Loader2 className="w-4 h-4 mr-2 animate-spin text-white/80" /> Synthesizing...</>
                ) : (
                  <><Brain className="w-4 h-4 mr-2" /> Generate Now</>
                )}
              </Button>
            </div>
          </DialogContent>
        </Dialog>

        <Dialog open={!!viewingResultForAttempt} onOpenChange={(o) => { if (!o) setViewingResultForAttempt(null) }}>
          <DialogContent className="max-w-6xl w-[95vw] max-h-[90vh] overflow-y-auto bg-slate-50 border border-slate-200/90 shadow-2xl rounded-3xl p-0 text-slate-900">
            <div className="p-6">
             {viewingResultForAttempt && selectedAssessment && (
                 <TraineeAssessmentResult
                    assessment={selectedAssessment}
                    questions={questions}
                    attemptData={viewingResultForAttempt}
                    attemptAnswers={viewingResultForAttempt.attempt_answers || []}
                    areResultsHidden={false}
                    displayTitle={`${viewingResultForAttempt.profiles?.first_name}'s Result: ${selectedAssessment.title}`}
                    traineeNavLinks={[]}
                    profile={viewingResultForAttempt.profiles}
                    course={{ title: 'Trainer Dashboard' }}
                    isStandalone={true}
                 />
             )}
            </div>
          </DialogContent>
        </Dialog>

      </motion.div>
    </TrainerLayout>
  )
}
```

src/features/trainer/CourseCreatePage.tsx TSX 124.3 KB 2,461 Lines

```tsx
import React, { useState, useEffect, useRef, useCallback, useMemo } from 'react'
import { useNavigate } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from '@/components/ui/select'
import {
  ArrowLeft, ArrowRight, CheckCircle, Check, Loader2, Plus, Upload, X,
  BookOpen, Settings, Target, Eye, AlertCircle, FileText, File, Image,
  Video, Link2, ExternalLink, Globe, Trash2, Layers, Trophy, Film, Camera,
  Sparkles, AlignLeft, HelpCircle, CheckSquare, Award, ArrowUp, ArrowDown, Edit3, GripVertical, Clock
} from 'lucide-react'
import { toast } from 'sonner'
import { ImageCropperModal } from '@/components/ui/ImageCropperModal'
import { CourseCertificateStep } from '@/features/courses/CourseCertificateStep'

type Skill = Database['public']['Tables']['skills']['Row']

export interface QuizQuestion {
  id: string
  question: string
  options: string[]
  correct_option: number
  explanation?: string
}

export interface ModuleItem {
  id: string
  type: 'photo' | 'video' | 'link' | 'text' | 'quiz'
  title: string
  content?: string
  url?: string
  duration_minutes?: number
  duration_seconds?: number
  file?: File
  previewUrl?: string
  storagePath?: string
  fileName?: string
  fileSize?: number
  quiz_data?: QuizQuestion
}

export interface CourseModule {
  id: string
  title: string
  description: string
  items: ModuleItem[]
  quiz_questions?: QuizQuestion[]
  passing_score?: number
}

const detailsSchema = z.object({
  title: z.string().min(5, 'Title must be at least 5 characters'),
  description: z.string().min(10, 'Description must be at least 10 characters'),
  course_type: z.string().min(2, 'Course type is required'),
  department: z.string().optional(),
})

const settingsSchema = z.object({
  duration_hours: z.coerce.number().positive('Duration must be positive').optional(),
  passing_score: z.coerce.number().min(1).max(100).optional(),
  meet_link: z.string().url('Must be a valid URL').optional().or(z.literal('')),
  start_date: z.string().optional(),
```

```
  end_date: z.string().optional(),
  planned_assessments_count: z.coerce.number().min(0).optional(),
  planned_mock_tests_count: z.coerce.number().min(0).optional(),
  final_test_date: z.string().optional(),
  final_test_start_time: z.string().optional(),
  final_test_end_time: z.string().optional(),
  delivery_mode: z.enum(['recorded', 'live', 'hybrid']).optional(),
  max_trainees: z.coerce.number().min(50, 'Minimum capacity is 50').max(250, 'Maximum capacity is 250').optional(),
  trainer_suggestion: z.string().optional(),
}).superRefine((data, ctx) => {
  if (data.start_date) {
    const start = new Date(data.start_date)
    const today = new Date()
    today.setHours(0, 0, 0, 0)
    const diffDays = Math.ceil((start.getTime() - today.getTime()) / (1000 * 60 * 60 * 24))
    if (diffDays < 30) {
      ctx.addIssue({
        code: z.ZodIssueCode.custom,
        message: 'Course must start at least 1 month from today',
        path: ['start_date']
      })
    }
  }
})

const objectivesSchema = z.object({
  learning_objectives: z.string().min(5, 'Describe what learners will achieve from this course'),
})

const sessionFlowSchema = z.object({
  session_flow_text: z.string().optional(),
})

type DetailsData = z.infer<typeof detailsSchema>
type SettingsData = z.infer<typeof settingsSchema>
type ObjectivesData = z.infer<typeof objectivesSchema>
type SessionFlowData = z.infer<typeof sessionFlowSchema>

interface PendingMaterial {
  id: string
  file?: File
  fileName: string
  fileSize: number
  mimeType: string
  materialType: 'file' | 'link' | 'video'
  url?: string
  storagePath?: string
  preview?: string
}

const MAX_FILE_SIZE = 100 * 1024 * 1024
const ACCEPTED_EXTENSIONS = ['.pdf', '.doc', '.docx', '.pptx', '.ppt', '.txt', '.mp4', '.webm', '.mov', '.png', '.jpg', '.jpeg', '.gif', '.webp']

function getFileIcon(mime: string) {
  if (mime.startsWith('video/')) return Video
  if (mime.startsWith('image/')) return Image
  if (mime.includes('pdf')) return FileText
  return File
}

function formatFileSize(bytes: number) {
  if (bytes < 1024) return `${bytes} B`
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`
}

const STEPS = [
  { id: 1, label: 'Course Details', icon: BookOpen },
  { id: 2, label: 'Course Configuration', icon: Settings },
  { id: 3, label: 'Course Outline', icon: FileText },
  { id: 4, label: 'Learning & Skills', icon: Target },
  { id: 5, label: 'Modules', icon: Layers },
  { id: 6, label: 'Materials', icon: FileText },
  { id: 7, label: 'Certificate', icon: Award },
  { id: 8, label: 'Review & Submit', icon: Eye },
]

export function CourseCreatePage() {
```

```tsx
  const { user } = useAuth()
  const navigate = useNavigate()

  const [step, setStep] = useState(1)
  const [saving, setSaving] = useState(false)
  const [hasCertificate, setHasCertificate] = useState(true)
  const [certificateTemplateUrl, setCertificateTemplateUrl] = useState<string | null>(null)
  const [certificateTemplateName, setCertificateTemplateName] = useState<string | null>(null)
  const [skills, setSkills] = useState<Skill[]>([])
  const [selectedSkills, setSelectedSkills] = useState<string[]>([])
  const [thumbnail, setThumbnail] = useState<File | null>(null)
  const [thumbnailPreview, setThumbnailPreview] = useState<string | null>(null)
  const [customSkillName, setCustomSkillName] = useState('')
  const [addingSkill, setAddingSkill] = useState(false)
  const [isCustomType, setIsCustomType] = useState(false)
  const [isCropperOpen, setIsCropperOpen] = useState(false)
  const [rawImageFile, setRawImageFile] = useState<File | null>(null)
  const thumbRef = useRef<HTMLInputElement>(null)

  // Modules state

  const [modules, setModules] = useState<CourseModule[]>([
    {
      id: crypto.randomUUID(),
      title: 'Module 1: Introduction & Fundamentals',
      description: 'Foundational concepts, overview content, photos, and video resources.',
      items: [],
    },
  ])

  // Active module item adding state

  const [activeModuleId, setActiveModuleId] = useState<string | null>(null)
  const [editingItemId, setEditingItemId] = useState<string | null>(null)
  const [addingItemType, setAddingItemType] = useState<'photo' | 'video' | 'link' | 'text' | null>(null)
  const [itemTitle, setItemTitle] = useState('')
  const [itemContent, setItemContent] = useState('')
  const [itemUrl, setItemUrl] = useState('')
  const [itemDurationMinutes, setItemDurationMinutes] = useState<string>('5')
  const [itemFile, setItemFile] = useState<File | null>(null)
  const [itemPreview, setItemPreview] = useState<string | null>(null)
  const moduleFileInputRef = useRef<HTMLInputElement>(null)

  // Active module quiz question state

  const [activeQuizModuleId, setActiveQuizModuleId] = useState<string | null>(null)
  const [editingQuestionId, setEditingQuestionId] = useState<string | null>(null)
  const [quizQuestionText, setQuizQuestionText] = useState('')
  const [quizOptions, setQuizOptions] = useState<string[]>(['', '', '', ''])
  const [quizCorrectOption, setQuizCorrectOption] = useState<number>(0)
  const [quizExplanation, setQuizExplanation] = useState('')
  const [isGeneratingAiQuiz, setIsGeneratingAiQuiz] = useState(false)

  // Materials state

  const [pendingMaterials, setPendingMaterials] = useState<PendingMaterial[]>([])
  const [dragOver, setDragOver] = useState(false)
  const [activeTab, setActiveTab] = useState<'files' | 'links'>('files')
  const [linkUrl, setLinkUrl] = useState('')
  const [linkTitle, setLinkTitle] = useState('')
  const fileInputRef = useRef<HTMLInputElement>(null)

  const detailsForm = useForm<DetailsData>({
    resolver: zodResolver(detailsSchema),
    defaultValues: { title: '', description: '', course_type: 'standard', department: '' },
  })

  const settingsForm = useForm<SettingsData>({
    resolver: zodResolver(settingsSchema),
    defaultValues: { passing_score: 60, delivery_mode: 'recorded' },
  })

  const objectivesForm = useForm<ObjectivesData>({
    resolver: zodResolver(objectivesSchema),
    defaultValues: { learning_objectives: '' },
  })

  const sessionFlowForm = useForm<SessionFlowData>({
    resolver: zodResolver(sessionFlowSchema),
```

```
    defaultValues: { session_flow_text: '' },
  })

  const [sessionFlowDoc, setSessionFlowDoc] = useState<File | null>(null)

  // Calculate course days dynamically

  const courseDays = useMemo(() => {
    const start = settingsForm.watch('start_date')
    const end = settingsForm.watch('end_date')
    if (start && end) {
      const diff = new Date(end).getTime() - new Date(start).getTime()
      return Math.max(1, Math.ceil(diff / (1000 * 60 * 60 * 24)))
    }
    return null
  }, [settingsForm.watch('start_date'), settingsForm.watch('end_date')])

  const isUrgent = useMemo(() => {
    const start = settingsForm.watch('start_date')
    if (start) {
      const diff = new Date(start).getTime() - new Date().getTime()
      const diffDays = diff / (1000 * 60 * 60 * 24)
      return diffDays < 30
    }
    return false
  }, [settingsForm.watch('start_date')])

  // Calculate final exam duration dynamically

  const startTime = settingsForm.watch('final_test_start_time')
  const endTime = settingsForm.watch('final_test_end_time')
  let examDuration = ''
  if (startTime && endTime) {
    const [startH, startM] = startTime.split(':').map(Number)
    const [endH, endM] = endTime.split(':').map(Number)
    let diffMins = (endH * 60 + endM) - (startH * 60 + startM)
    if (diffMins < 0) diffMins += 24 * 60 // handle overnight
    const h = Math.floor(diffMins / 60)
    const m = diffMins % 60
    examDuration = `${h > 0 ? `${h}h ` : ''}${m > 0 ? `${m}m` : ''}` || '0m'
  }

  useEffect(() => {
    supabase.from('skills').select('*').order('name').then(({ data, error }) => {
      if (data && data.length > 0) {
        setSkills(data)
      } else {
        const defaultSkills: Skill[] = [
          { id: 'sk-1', name: 'Cyclone Tracking & Analysis', category: 'Meteorology', created_at: '' },
          { id: 'sk-2', name: 'Emergency Warning Communication', category: 'Operations', created_at: '' },
          { id: 'sk-3', name: 'Satellite & Radar Telemetry', category: 'Technical', created_at: '' },
          { id: 'sk-4', name: 'Disaster Preparedness', category: 'Disaster Management', created_at: '' },
          { id: 'sk-5', name: 'Flood Hazard Mapping', category: 'Hydrology', created_at: '' },
          { id: 'sk-6', name: 'GIS & Spatial Modeling', category: 'GIS', created_at: '' },
          { id: 'sk-7', name: 'Ocean Sensor Calibration', category: 'Oceanography', created_at: '' },
          { id: 'sk-8', name: 'Search & Rescue Protocols', category: 'Operations', created_at: '' },
          { id: 'sk-9', name: 'Weather Data Interpretation', category: 'Meteorology', created_at: '' },
          { id: 'sk-10', name: 'Risk Assessment & Mitigation', category: 'Safety', created_at: '' },
        ] as any
        setSkills(defaultSkills)
      }
    })
  }, [])

  const handleThumbnailChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return
    if (file.size > 5 * 1024 * 1024) {
      toast.error('Thumbnail must be under 5MB')
      return
    }
    setRawImageFile(file)
    setIsCropperOpen(true)
    if (thumbRef.current) thumbRef.current.value = ''
  }

  const handleCropComplete = (croppedFile: File) => {
    setThumbnail(croppedFile)
    setThumbnailPreview(URL.createObjectURL(croppedFile))
```

```
    setIsCropperOpen(false)
    setRawImageFile(null)
  }

  // Modules helpers

  const addModule = () => {
    const newMod: CourseModule = {
      id: crypto.randomUUID(),
      title: `Module ${modules.length + 1}: New Topic`,
      description: '',
      items: [],
      quiz_questions: [],
    }
    setModules(prev => [...prev, newMod])
    toast.success('New module added')
  }

  const removeModule = (id: string) => {
    if (modules.length <= 1) {
      toast.error('You must keep at least one module')
      return
    }
    setModules(prev => prev.filter(m => m.id !== id))
    toast.info('Module removed')
  }

  const updateModule = (id: string, updates: Partial<CourseModule>) => {
    setModules(prev => prev.map(m => m.id === id ? { ...m, ...updates } : m))
  }

  const moveModuleItem = (moduleId: string, fromIndex: number, toIndex: number) => {
    setModules(prev => prev.map(m => {
      if (m.id !== moduleId) return m
      if (toIndex < 0 || toIndex >= m.items.length) return m
      const newItems = [...m.items]
      const [moved] = newItems.splice(fromIndex, 1)
      newItems.splice(toIndex, 0, moved)
      return { ...m, items: newItems }
    }))
  }

  const openAddItem = (moduleId: string, type: 'photo' | 'video' | 'link' | 'text') => {
    setActiveModuleId(moduleId)
    setEditingItemId(null)
    setAddingItemType(type)
    setItemTitle('')
    setItemContent('')
    setItemUrl('')
    setItemDurationMinutes('5')
    setItemFile(null)
    setItemPreview(null)
  }

  const openEditItem = (moduleId: string, item: ModuleItem) => {
    if (item.type === 'quiz') {
      const qData = item.quiz_data || {
        id: item.id,
        question: item.title.replace(/^Quiz:\s*/, '') || '',
        options: ['', '', '', ''],
        correct_option: 0,
        explanation: item.content || '',
      }
      openEditQuizQuestion(moduleId, qData)
      return
    }

    setActiveModuleId(moduleId)
    setEditingItemId(item.id)
    setAddingItemType(item.type)
    setItemTitle(item.title)
    setItemContent(item.content || '')
    setItemUrl(item.url || '')
    setItemDurationMinutes(
      item.duration_minutes
        ? String(item.duration_minutes)
        : item.duration_seconds
        ? String(Math.round(item.duration_seconds / 60))
        : '5'
```

```
    )
    setItemFile(item.file || null)
    setItemPreview(item.previewUrl || null)
  }

  const handleModuleFileSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return
    if (file.size > 100 * 1024 * 1024) {
      toast.error('File exceeds 100MB limit')
      return
    }
    setItemFile(file)
    if (!itemTitle) {
      setItemTitle(file.name.replace(/\.[^/.]+$/, ""))
    }
    if (file.type.startsWith('image/')) {
      setItemPreview(URL.createObjectURL(file))
    }
  }

  const saveModuleItem = () => {
    if (!activeModuleId || !addingItemType) return
    if (!itemTitle.trim() && !itemFile?.name && !itemUrl.trim()) {
      toast.error('Please enter a title or URL')
      return
    }

    const durationMin = addingItemType === 'video' ? Math.max(0.5, parseFloat(itemDurationMinutes) || 5) : undefined
    const durationSec = durationMin ? Math.round(durationMin * 60) : undefined

    setModules(prev => prev.map(m => {
      if (m.id !== activeModuleId) return m

      if (editingItemId) {
        return {
          ...m,
          items: m.items.map(item => item.id === editingItemId ? {
            ...item,
            type: addingItemType,
            title: itemTitle.trim() || itemFile?.name || (addingItemType === 'link' ? itemUrl : 'Untitled Item'),
            content: itemContent.trim() || undefined,
            url: itemUrl.trim() || undefined,
            duration_minutes: durationMin,
            duration_seconds: durationSec,
            file: itemFile || item.file || undefined,
            previewUrl: itemPreview || item.previewUrl || (itemFile && itemFile.type.startsWith('image/') ? URL.createObjectU
RL(itemFile) : undefined),
            fileName: itemFile?.name || item.fileName,
            fileSize: itemFile?.size || item.fileSize,
          } : item)
        }
      }

      const newItem: ModuleItem = {
        id: crypto.randomUUID(),
        type: addingItemType,
        title: itemTitle.trim() || itemFile?.name || (addingItemType === 'link' ? itemUrl : 'Untitled Item'),
        content: itemContent.trim() || undefined,
        url: itemUrl.trim() || undefined,
        duration_minutes: durationMin,
        duration_seconds: durationSec,
        file: itemFile || undefined,
        previewUrl: itemPreview || (itemFile && itemFile.type.startsWith('image/') ? URL.createObjectURL(itemFile) : undefine
d),
        fileName: itemFile?.name,
        fileSize: itemFile?.size,
      }

      return {
        ...m,
        items: [...m.items, newItem]
      }
    }))

    toast.success(editingItemId ? `Updated ${addingItemType} item` : `Added ${addingItemType} to module`)
    setAddingItemType(null)
    setActiveModuleId(null)
    setEditingItemId(null)
```

```
    setItemFile(null)
    setItemPreview(null)
    setItemTitle('')
    setItemContent('')
    setItemUrl('')
    setItemDurationMinutes('5')
  }

  const removeModuleItem = (moduleId: string, itemId: string) => {
    setModules(prev => prev.map(m => {
      if (m.id !== moduleId) return m
      return {
        ...m,
        items: m.items.filter(i => i.id !== itemId),
        quiz_questions: (m.quiz_questions || []).filter(q => q.id !== itemId)
      }
    }))
    toast.info('Item removed')
  }

  // Quiz questions helpers

  const openAddQuizQuestion = (moduleId: string) => {
    setActiveQuizModuleId(moduleId)
    setEditingQuestionId(null)
    setQuizQuestionText('')
    setQuizOptions(['', '', '', ''])
    setQuizCorrectOption(0)
    setQuizExplanation('')
  }

  const openEditQuizQuestion = (moduleId: string, q: QuizQuestion) => {
    setActiveQuizModuleId(moduleId)
    setEditingQuestionId(q.id)
    setQuizQuestionText(q.question)
    setQuizOptions(q.options && q.options.length >= 2 ? [...q.options] : ['', '', '', ''])
    setQuizCorrectOption(q.correct_option ?? 0)
    setQuizExplanation(q.explanation || '')
  }

  const saveQuizQuestion = () => {
    if (!activeQuizModuleId) return
    if (!quizQuestionText.trim()) {
      toast.error('Please enter the question text')
      return
    }
    const cleanOptions = quizOptions.map(o => o.trim())
    if (cleanOptions.filter(Boolean).length < 2) {
      toast.error('Please provide at least 2 answer choices')
      return
    }

    const questionData: QuizQuestion = {
      id: editingQuestionId || crypto.randomUUID(),
      question: quizQuestionText.trim(),
      options: cleanOptions,
      correct_option: quizCorrectOption,
      explanation: quizExplanation.trim() || undefined,
    }

    setModules(prev => prev.map(m => {
      if (m.id !== activeQuizModuleId) return m

      let updatedItems = [...m.items]
      const existingItemIdx = updatedItems.findIndex(i => i.id === editingQuestionId || i.quiz_data?.id === editingQuestionI
d)

      if (existingItemIdx !== -1) {
        updatedItems[existingItemIdx] = {
          ...updatedItems[existingItemIdx],
          type: 'quiz',
          title: `Quiz: ${quizQuestionText.trim().slice(0, 50)}${quizQuestionText.trim().length > 50 ? '...' : ''}`,
          content: quizExplanation.trim() || undefined,
          quiz_data: questionData
        }
      } else {
        const newQuizItem: ModuleItem = {
          id: questionData.id,
          type: 'quiz',
```

```
          title: `Quiz: ${quizQuestionText.trim().slice(0, 50)}${quizQuestionText.trim().length > 50 ? '...' : ''}`,
          content: quizExplanation.trim() || undefined,
          quiz_data: questionData
        }
        updatedItems.push(newQuizItem)
      }

      // Keep quiz_questions in sync

      const currentQuestions = m.quiz_questions || []
      const updatedQuestions = editingQuestionId
        ? currentQuestions.map(q => q.id === editingQuestionId ? questionData : q)
        : [...currentQuestions, questionData]

      return {
        ...m,
        items: updatedItems,
        quiz_questions: updatedQuestions,
        passing_score: m.passing_score ?? 80,
      }
    }))

    toast.success(editingQuestionId ? 'Quiz question updated' : 'Quiz question added to module sequence')
    setActiveQuizModuleId(null)
    setEditingQuestionId(null)
    setQuizQuestionText('')
    setQuizOptions(['', '', '', ''])
    setQuizCorrectOption(0)
    setQuizExplanation('')
  }

  const removeQuizQuestion = (moduleId: string, questionId: string) => {
    setModules(prev => prev.map(m => {
      if (m.id !== moduleId) return m
      return {
        ...m,
        items: m.items.filter(i => i.id !== questionId && i.quiz_data?.id !== questionId),
        quiz_questions: (m.quiz_questions || []).filter(q => q.id !== questionId),
      }
    }))
    toast.info('Quiz question removed')
  }

  const handleGenerateAiQuiz = (moduleId: string) => {
    const mod = modules.find(m => m.id === moduleId)
    if (!mod) return
    setIsGeneratingAiQuiz(true)
    setTimeout(() => {
      const topic = mod.title.replace(/^Module \d+:\s*/, '') || 'Module Topic'
      const aiQuestions: QuizQuestion[] = [
        {
          id: crypto.randomUUID(),
          question: `What is the core prerequisite protocol for ${topic}?`,
          options: [
            'Adhere to verified operational checklists and standard ministry guidelines',
            'Bypass preliminary verification to speed up execution',
            'Conduct unrecorded subjective assessments',
            'Defer telemetry log synchronization until post-incident review'
          ],
          correct_option: 0,
          explanation: 'Standard operational guidelines ensure consistent execution and data integrity.'
        },
        {
          id: crypto.randomUUID(),
          question: `What minimum passing score is required to unlock subsequent modules in Capacity Connect?`,
          options: [
            '80% passing threshold in module checkpoint verification',
            '50% general participation threshold',
            '60% optional assessment threshold',
            'No benchmark requirement'
          ],
          correct_option: 0,
          explanation: 'Capacity Connect requires an 80% passing score on module quizzes before proceeding.'
        },
        {
          id: crypto.randomUUID(),
          question: `How should abnormal telemetry flags or critical discrepancies in ${topic} be handled?`,
          options: [
            'Log anomaly immediately and escalate to emergency response command',
```

```
            'Suppress the alert to prevent alarm fatigue',
            'Recalibrate sensor data without logging',
            'Wait for end-of-day reconciliation before taking action'
          ],
          correct_option: 0,
          explanation: 'Immediate escalation ensures rapid risk mitigation and disaster preparedness.'
        }
      ]

      const aiQuizItems: ModuleItem[] = aiQuestions.map(q => ({
        id: q.id,
        type: 'quiz',
        title: `Quiz: ${q.question.slice(0, 50)}...`,
        content: q.explanation,
        quiz_data: q
      }))

      setModules(prev => prev.map(m => m.id === moduleId ? {
        ...m,
        items: [...m.items, ...aiQuizItems],
        quiz_questions: [...(m.quiz_questions || []), ...aiQuestions],
        passing_score: 80,
      } : m))
      setIsGeneratingAiQuiz(false)
      toast.success('Generated 3 AI quiz questions and added to module sequence!')
    }, 500)
  }

  const validateStep = async () => {
    switch (step) {
      case 1: return await detailsForm.trigger()
      case 2: return await settingsForm.trigger()
      case 3: return await sessionFlowForm.trigger()
      case 4: return await objectivesForm.trigger()
      case 5: {
        if (modules.length === 0) {
          toast.error('Please add at least one module')
          return false
        }
        for (let i = 0; i < modules.length; i++) {
          if (!modules[i].title.trim()) {
            toast.error(`Please provide a title for Module ${i + 1}`)
            return false
          }
        }
        return true
      }
      default: return true
    }
  }

  const handleNext = async () => {
    const valid = await validateStep()
    if (!valid) return
    setStep(s => Math.min(s + 1, 8))
  }

  const handleBack = () => {
    setStep(s => Math.max(s - 1, 1))
  }

  // Materials handling

  const validateFile = (file: File): string | null => {
    if (file.size > MAX_FILE_SIZE) return `File too large (${formatFileSize(file.size)}). Max 100MB.`
    const ext = '.' + file.name.split('.').pop()?.toLowerCase()
    if (!ACCEPTED_EXTENSIONS.includes(ext)) return `File type not supported: ${ext}`
    return null
  }

  const addFileMaterial = useCallback((file: File) => {
    const error = validateFile(file)
    if (error) {
      toast.error(error)
      return
    }
    const isVideo = file.type.startsWith('video/')
    const material: PendingMaterial = {
      id: crypto.randomUUID(),
```

```
      file,
      fileName: file.name,
      fileSize: file.size,
      mimeType: file.type || 'application/octet-stream',
      materialType: isVideo ? 'video' : 'file',
      preview: file.type.startsWith('image/') ? URL.createObjectURL(file) : undefined,
    }
    setPendingMaterials(prev => [...prev, material])
    toast.success(`Added: ${file.name}`)
  }, [])

  const handleFileInput = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = e.target.files
    if (!files) return
    Array.from(files).forEach(addFileMaterial)
    if (fileInputRef.current) fileInputRef.current.value = ''
  }

  const handleDrop = (e: React.DragEvent) => {
    e.preventDefault()
    setDragOver(false)
    const files = e.dataTransfer.files
    Array.from(files).forEach(addFileMaterial)
  }

  const addLinkMaterial = () => {
    const url = linkUrl.trim()
    if (!url) {
      toast.error('Enter a URL')
      return
    }
    try {
      new URL(url)
    } catch {
      toast.error('Please enter a valid URL')
      return
    }
    const isVideo = /youtube\.com|youtu\.be|vimeo\.com|\.mp4|\.webm/i.test(url)
    const title = linkTitle.trim() || new URL(url).hostname
    const material: PendingMaterial = {
      id: crypto.randomUUID(),
      fileName: title,
      fileSize: 0,
      mimeType: 'text/uri-list',
      materialType: isVideo ? 'video' : 'link',
      url,
    }
    setPendingMaterials(prev => [...prev, material])
    setLinkUrl('')
    setLinkTitle('')
    toast.success(`Added: ${title}`)
  }

  const removeMaterial = (id: string) => {
    setPendingMaterials(prev => prev.filter(m => m.id !== id))
  }

  // Submit

  const handleSubmit = async (status: 'draft' | 'pending_review') => {
    if (status === 'pending_review' && pendingMaterials.length === 0) {
      toast.error('Add at least one material before submitting for review')
      return
    }
    if (selectedSkills.length === 0) {
      toast.error('Select at least one skill for the course')
      return
    }
    setSaving(true)
    try {
      const d = detailsForm.getValues()
      const s = settingsForm.getValues()
      const f = sessionFlowForm.getValues()
      const o = objectivesForm.getValues()

      // Create course first (without thumbnail or session doc path)

      const { data: course, error } = await supabase
        .from('courses')
```

```
        .insert({
          title: d.title,
          description: d.description,
          course_type: d.course_type,
          department: d.department || null,
          duration_minutes: s.duration_hours ? s.duration_hours * 60 : null,
          passing_score: s.passing_score ?? 60,
          trainer_id: user!.id,
          status,
          learning_objectives: {
            description: o.learning_objectives,
            understand: o.learning_objectives,
          },
          meet_link: s.meet_link || null,
          start_date: s.start_date ? new Date(s.start_date).toISOString() : null,
          end_date: s.end_date ? new Date(s.end_date).toISOString() : null,
          planned_assessments_count: s.planned_assessments_count || 0,
          planned_mock_tests_count: s.planned_mock_tests_count || 0,
          final_test_date: s.final_test_date ? new Date(s.final_test_date).toISOString() : null,
          final_test_start_time: s.final_test_start_time || null,
          final_test_end_time: s.final_test_end_time || null,
          delivery_mode: s.delivery_mode,
          max_trainees: s.max_trainees || null,
          trainer_suggestion: s.trainer_suggestion || null,
          session_flow_text: f.session_flow_text || null,
          has_certificate: hasCertificate,
          certificate_template_url: certificateTemplateUrl,
          certificate_template_name: certificateTemplateName,
        } as any)
        .select()
        .single()
      if (error) throw error

      // Upload thumbnail & session doc after course is created (needs course_id for storage RLS)

      let sessionDocPath = null
      if (sessionFlowDoc && user && course) {
        const ext = sessionFlowDoc.name.split('.').pop()
        const storagePath = `${course.id}/session_flow_${crypto.randomUUID()}.${ext}`
        const { error: uploadErr } = await supabase.storage
          .from('materials')
          .upload(storagePath, sessionFlowDoc)
        if (!uploadErr) {
          sessionDocPath = storagePath
        }
      }

      if (thumbnail && user && course) {
        const ext = thumbnail.name.split('.').pop()
        const thumbnailStoragePath = `${course.id}/thumbnail.${ext}`
        const { error: uploadErr } = await supabase.storage
          .from('materials')
          .upload(thumbnailStoragePath, thumbnail)
        if (uploadErr) {
          console.error('Thumbnail upload error:', uploadErr)
        } else {
          // Update course with paths

          await supabase
            .from('courses')
            .update({
              thumbnail_path: thumbnailStoragePath,
              session_flow_document_path: sessionDocPath
            })
            .eq('id', course.id)
        }
      } else if (sessionDocPath && course) {
         await supabase
          .from('courses')
          .update({
            session_flow_document_path: sessionDocPath
          })
          .eq('id', course.id)
      }

      // Process and upload Modules to 'Course modules' bucket

      const uploadedModules = []
      for (let mIdx = 0; mIdx < modules.length; mIdx++) {
```

```
        const mod = modules[mIdx]
        const uploadedItems: any[] = []
        for (let iIdx = 0; iIdx < mod.items.length; iIdx++) {
          const item = mod.items[iIdx]
          if (item.file) {
            const ext = item.file.name.split('.').pop() || 'dat'
            const storagePath = `${course.id}/module_${mIdx + 1}/${crypto.randomUUID()}.${ext}`

            // Try uploading to 'Course modules' bucket

            let uploadRes = await supabase.storage
              .from('Course modules')
              .upload(storagePath, item.file)

            if (uploadRes.error) {
              console.warn('Upload to Course modules had error, trying fallback:', uploadRes.error)
              uploadRes = await supabase.storage
                .from('materials')
                .upload(storagePath, item.file)
            }

            const { data: publicUrlData } = supabase.storage
              .from('Course modules')
              .getPublicUrl(storagePath)

            uploadedItems.push({
              id: item.id,
              type: item.type,
              title: item.title,
              content: item.content || '',
              url: publicUrlData?.publicUrl || item.url || '',
              duration_minutes: item.duration_minutes,
              duration_seconds: item.duration_seconds,
              storagePath: storagePath,
              fileName: item.fileName,
              fileSize: item.fileSize,
              quiz_data: item.quiz_data,
            })
          } else {
            uploadedItems.push({
              id: item.id,
              type: item.type,
              title: item.title,
              content: item.content || '',
              url: item.url || '',
              duration_minutes: item.duration_minutes,
              duration_seconds: item.duration_seconds,
              fileName: item.fileName,
              fileSize: item.fileSize,
              quiz_data: item.quiz_data,
            })
          }
        }

        uploadedModules.push({
          id: mod.id,
          title: mod.title,
          description: mod.description,
          order_index: mIdx,
          is_final_assessment: false,
          passing_score: mod.passing_score ?? 80,
          quiz_questions: mod.quiz_questions || [],
          items: uploadedItems,
        })
      }

      // Update course with modules JSON

      try {
        await supabase
          .from('courses')
          .update({ modules: uploadedModules } as any)
          .eq('id', course.id)
      } catch (err) {
        console.warn('Could not update courses.modules directly:', err)
      }

      // Try inserting into course_modules table if present
```

```
      try {
        const moduleRows = uploadedModules.map(m => ({
          course_id: course.id,
          title: m.title,
          description: m.description,
          order_index: m.order_index,
          is_final_assessment: false,
          content_items: m.items,
        }))
        await (supabase as any).from('course_modules').insert(moduleRows)
      } catch (err) {
        console.warn('course_modules table insert optional/skipped:', err)
      }

      // Insert skills

      if (selectedSkills.length > 0 && course) {
        try {
          const skillInserts: any[] = []
          for (const skillId of selectedSkills) {
            if (skillId.startsWith('custom-')) {
              const skillObj = skills.find(s => s.id === skillId)
              if (skillObj) {
                const { data: createdSkill } = await supabase
                  .from('skills')
                  .insert({ name: skillObj.name } as any)
                  .select()
                  .single()
                if (createdSkill) {
                  skillInserts.push({
                    course_id: course.id,
                    skill_id: createdSkill.id,
                    required_level: 3,
                  })
                }
              }
            } else {
              skillInserts.push({
                course_id: course.id,
                skill_id: skillId,
                required_level: 3,
              })
            }
          }
          if (skillInserts.length > 0) {
            await supabase.from('course_skills').insert(skillInserts)
          }
        } catch (sErr) {
          console.warn('Could not insert course_skills:', sErr)
        }
      }

      // Upload materials

      if (course && pendingMaterials.length > 0) {
        for (const mat of pendingMaterials) {
          if (mat.materialType === 'link' || mat.materialType === 'video') {
            // Link/video - just insert record

            await supabase.from('materials').insert({
              course_id: course.id,
              uploaded_by: user!.id,
              file_name: mat.fileName,
              storage_path: `link/${crypto.randomUUID()}`,
              material_type: mat.materialType,
              url: mat.url,
              mime_type: 'text/uri-list',
              extraction_status: 'completed',
            })
          } else if (mat.file) {
            // File - upload to storage then insert record

            const fileExt = mat.file.name.split('.').pop()
            const materialId = mat.id
            const storagePath = `${course.id}/${materialId}.${fileExt}`

            const { error: uploadError } = await supabase.storage
              .from('materials')
              .upload(storagePath, mat.file)
```

```
            if (uploadError) throw uploadError

            await supabase.from('materials').insert({
              id: materialId,
              course_id: course.id,
              uploaded_by: user!.id,
              file_name: mat.fileName,
              storage_path: storagePath,
              mime_type: mat.mimeType,
              file_size: mat.fileSize,
              extraction_status: 'pending',
            })
          }
        }
      }

      toast.success(status === 'draft' ? 'Course saved as draft' : 'Course submitted for review')
      if (course) {
        navigate(`/trainer/courses/${course.id}/materials`)
      } else {
        navigate('/trainer/courses')
      }
    } catch (err) {
      console.error('Course creation error:', err)
      toast.error(err && typeof err === 'object' && 'message' in err ? String(err.message) : 'Failed to create course')
    } finally {
      setSaving(false)
    }
  }

  const toggleSkill = (id: string) => {
    setSelectedSkills(prev => prev.includes(id) ? prev.filter(s => s !== id) : [...prev, id])
  }

  const addCustomSkill = async () => {
    const name = customSkillName.trim()
    if (!name) return
    const existing = skills.find(s => s.name.toLowerCase() === name.toLowerCase())
    if (existing) {
      if (!selectedSkills.includes(existing.id)) {
        setSelectedSkills(prev => [...prev, existing.id])
        setCustomSkillName('')
        toast.success(`Selected skill: "${existing.name}"`)
        return
      }
      toast.info('Skill already selected')
      setCustomSkillName('')
      return
    }

    setAddingSkill(true)
    try {
      const { data, error } = await supabase.from('skills').insert({ name } as any).select().single()
      if (error || !data) {
        // Local custom skill fallback so trainer is never blocked

        const localSkill: Skill = {
          id: `custom-${crypto.randomUUID()}`,
          name: name,
          category: 'Custom Outcome',
          created_at: new Date().toISOString()
        } as any
        setSkills(prev => [...prev, localSkill].sort((a, b) => a.name.localeCompare(b.name)))
        setSelectedSkills(prev => [...prev, localSkill.id])
        setCustomSkillName('')
        toast.success(`Added outcome: "${name}"`)
      } else {
        setSkills(prev => [...prev, data].sort((a, b) => a.name.localeCompare(b.name)))
        setSelectedSkills(prev => [...prev, data.id])
        setCustomSkillName('')
        toast.success(`Skill "${name}" added`)
      }
    } catch (err) {
      const localSkill: Skill = {
        id: `custom-${crypto.randomUUID()}`,
        name: name,
        category: 'Custom Outcome',
        created_at: new Date().toISOString()
      } as any
```

```
      setSkills(prev => [...prev, localSkill].sort((a, b) => a.name.localeCompare(b.name)))
      setSelectedSkills(prev => [...prev, localSkill.id])
      setCustomSkillName('')
      toast.success(`Added outcome: "${name}"`)
    } finally {
      setAddingSkill(false)
    }
  }

  const watchedValues = {
    title: detailsForm.watch('title'),
    description: detailsForm.watch('description'),
    course_type: detailsForm.watch('course_type'),
    department: detailsForm.watch('department'),
    session_flow_text: sessionFlowForm.watch('session_flow_text'),
    duration_hours: settingsForm.watch('duration_hours'),
    passing_score: settingsForm.watch('passing_score'),
    learning_objectives: objectivesForm.watch('learning_objectives'),
  }

  const totalMaterialSize = pendingMaterials.reduce((sum, m) => sum + m.fileSize, 0)

  return (
    <TrainerLayout>
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-3xl mx-auto space-y-6">
        {/* Header */}
        <motion.div variants={fadeUp}>
          <button onClick={() => navigate('/trainer/courses')} className="flex items-center gap-2 text-sm font-semibold text-
slate-600 hover:text-slate-900 transition-colors mb-4">
            <ArrowLeft className="w-4 h-4" /> Back to Courses
          </button>
          <h2 className="text-2xl font-bold tracking-tight text-slate-900">Create Course</h2>
          <p className="text-sm text-slate-500 mt-1">Fill in the details, add materials, and submit for review.</p>
        </motion.div>

        {/* Progress Steps */}
        <motion.div variants={fadeUp}>
          <div className="flex items-center gap-1">
            {STEPS.map((s, i) => (
              <React.Fragment key={s.id}>
                <button
                  onClick={() => { if (s.id < step) setStep(s.id) }}
                  className={`flex items-center gap-1.5 transition-all ${s.id <= step ? 'text-slate-900 font-bold' : 'text-sl
ate-400'} ${s.id < step ? 'cursor-pointer hover:text-cyan-700' : 'cursor-default'}`}
                >
                  <div className={`w-7 h-7 rounded-full flex items-center justify-center text-[10px] font-bold border transit
ion-all ${
                    step > s.id ? 'bg-emerald-50 border-emerald-300 text-emerald-700' :
                    step === s.id ? 'bg-gradient-to-r from-cyan-600 to-blue-600 text-white border-transparent shadow-xs' :
                    'border-slate-300 text-slate-400 bg-white'
                  }`}>
                    {step > s.id ? <CheckCircle className="w-3.5 h-3.5" /> : <s.icon className="w-3.5 h-3.5" />}
                  </div>
                  <span className="text-[10px] hidden md:inline">{s.label}</span>
                </button>
                {i < STEPS.length - 1 && <div className={`flex-1 h-px ${step > s.id ? 'bg-cyan-500' : 'bg-slate-200'}`} />}
              </React.Fragment>
            ))}
          </div>
        </motion.div>

        {/* Step Content */}
        <motion.div variants={fadeUp}>
          {/* Step 1: Course Details */}
          {step === 1 && (
            <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
              <CardContent className="p-6 space-y-4">
                <div className="flex items-center gap-2 mb-2">
                  <BookOpen className="w-4 h-4 text-cyan-600" />
                  <h3 className="text-sm font-bold text-slate-900">Course Details</h3>
                </div>
                <div className="space-y-1.5">
                  <Label className="text-slate-700 text-xs font-semibold">Course Title *</Label>
                  <Input {...detailsForm.register('title')} placeholder="e.g. Cyclone Response and Warning Communication" cla
ssName="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                  {detailsForm.formState.errors.title && <p className="text-xs text-rose-600 font-medium">{detailsForm.formSt
ate.errors.title.message}</p>}
                </div>
                <div className="space-y-1.5">
```

```
                  <Label className="text-slate-700 text-xs font-semibold">Description *</Label>
                  <Textarea {...detailsForm.register('description')} rows={4} placeholder="Describe what this course covers, its target audience, and key topics..." className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white rounded-xl resize-none" />
                  {detailsForm.formState.errors.description && <p className="text-xs text-rose-600 font-medium">{detailsForm.formState.errors.description.message}</p>}
                </div>
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 text-xs font-semibold">Course Type</Label>
                    <Select
                      value={isCustomType ? 'custom' : detailsForm.watch('course_type')}
                      onValueChange={v => {
                        if (v === 'custom') {
                          setIsCustomType(true)
                          detailsForm.setValue('course_type', '')
                        } else {
                          setIsCustomType(false)
                          detailsForm.setValue('course_type', v)
                        }
                      }}
                    >
                      <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 h-10 rounded-xl"><SelectValue /></SelectTrigger>
                      <SelectContent>
                        <SelectItem value="standard">Standard Training</SelectItem>
                        <SelectItem value="scenario">Scenario-Based Training</SelectItem>
                        <SelectItem value="technical">Technical Training</SelectItem>
                        <SelectItem value="custom">+ Add Custom Course Type...</SelectItem>
                      </SelectContent>
                    </Select>
                    {isCustomType && (
                      <Input
                        {...detailsForm.register('course_type')}
                        placeholder="Enter custom course type"
                        className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 mt-2 rounded-xl"
                      />
                    )}
                    {detailsForm.formState.errors.course_type && <p className="text-xs text-rose-600 font-medium">{detailsForm.formState.errors.course_type.message}</p>}
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 text-xs font-semibold">Department</Label>
                    <Input {...detailsForm.register('department')} placeholder="e.g. IMD, CWC, NIOT" className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                  </div>
                </div>
              </CardContent>
            </Card>
          )}

          {/* Step 2: Course Configuration */}
          {step === 2 && (
            <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
              <CardContent className="p-6 space-y-4">
                <div className="flex items-center gap-2 mb-2">
                  <Settings className="w-4 h-4 text-cyan-600" />
                  <h3 className="text-sm font-bold text-slate-900">Course Configuration</h3>
                </div>
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div className="space-y-1.5 col-span-1 sm:col-span-2">
                    <Label className="text-slate-700 text-xs font-semibold">Duration (hours)</Label>
                    <Input type="number" {...settingsForm.register('duration_hours')} placeholder="e.g. 20" className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl max-w-sm" />
                    <p className="text-[10px] text-slate-400 font-medium">Leave empty for self-paced</p>
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 text-xs font-semibold">Start Date</Label>
                    <Input type="date" {...settingsForm.register('start_date')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                    {settingsForm.formState.errors.start_date && (
                      <p className="text-[10px] text-rose-600 font-medium">{settingsForm.formState.errors.start_date.message}</p>
                    )}
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 text-xs font-semibold">End Date</Label>
                    <Input type="date" {...settingsForm.register('end_date')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
```

```
                    {settingsForm.formState.errors.end_date && (
                      <p className="text-[10px] text-rose-600 font-medium">{settingsForm.formState.errors.end_date.message}</p>
                    )}
                  </div>
                  {isUrgent && settingsForm.watch('start_date') && !settingsForm.formState.errors.start_date && (
                    <div className="col-span-1 sm:col-span-2 bg-amber-50 border border-amber-200 text-amber-800 p-3 rounded-xl text-xs font-medium flex items-center gap-2">
                      <span className="text-lg">⚠️</span> Course starts in less than 30 days! This will be flagged as <strong className="font-bold">URGENT</strong> for fast-track Admin approval.
                    </div>
                  )}

                  {/* Capacity */}
                  <div className="space-y-1.5 col-span-1 sm:col-span-2">
                    <Label className="text-slate-700 text-xs font-semibold">Trainee Capacity Limit</Label>
                    <Input type="number" {...settingsForm.register('max_trainees')} placeholder="e.g. 50" className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                    <p className="text-[10px] text-slate-400 font-medium">Must be between 50 and 250</p>
                  </div>

                  {/* Trainer Suggestion / Notice */}
                  <div className="space-y-1.5 col-span-1 sm:col-span-2">
                    <Label className="text-slate-700 text-xs font-semibold flex items-center gap-2">Notice to Admin <Badge variant="secondary" className="text-[9px] h-4 bg-slate-100 text-slate-600">Optional</Badge></Label>
                    <Textarea {...settingsForm.register('trainer_suggestion')} placeholder="Add any notes or suggestions for the admin approving this course..." className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white min-h-[60px] rounded-xl resize-none" />
                  </div>

                  {/* Test Planning */}
                  <div className="space-y-1.5 col-span-1 sm:col-span-2 mt-2 pt-4 border-t border-slate-100">
                    <Label className="text-slate-900 font-bold text-sm">Test & Assessment Plan</Label>
                  </div>
                  <div className="space-y-1.5 col-span-1 sm:col-span-2">
                    <Label className="text-slate-700 text-xs font-bold mt-1">Final Exam Details</Label>
                  </div>
                  <div className="grid grid-cols-1 sm:grid-cols-3 gap-3 col-span-1 sm:col-span-2">
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 text-xs font-semibold">Date</Label>
                      <Input type="date" {...settingsForm.register('final_test_date')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 text-xs font-semibold">Start Time</Label>
                      <Input type="time" {...settingsForm.register('final_test_start_time')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                    </div>
                    <div className="space-y-1.5">
                      <Label className="text-slate-700 text-xs font-semibold">End Time</Label>
                      <Input type="time" {...settingsForm.register('final_test_end_time')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
                    </div>
                  </div>
                  {examDuration && (
                    <div className="col-span-1 sm:col-span-2 text-xs font-semibold text-slate-700">
                      Duration: <span className="text-cyan-600 font-bold">{examDuration}</span>
                    </div>
                  )}
                </div>
                <div className="space-y-1.5 pt-4 border-t border-slate-100">
                  <Label className="text-slate-700 text-xs font-semibold">Course Thumbnail</Label>
                  <input ref={thumbRef} type="file" accept="image/*" className="hidden" onChange={handleThumbnailChange} />
                  {thumbnailPreview ? (
                    <div className="relative w-full h-40 rounded-xl overflow-hidden border border-slate-200 shadow-xs">
                      <img src={thumbnailPreview} alt="" className="w-full h-full object-cover" />
                      <button onClick={() => { setThumbnail(null); setThumbnailPreview(null) }} className="absolute top-2 right-2 p-1.5 rounded-full bg-slate-900/80 text-white hover:bg-rose-600 transition-colors">
                        <X className="w-3.5 h-3.5" />
                      </button>
                    </div>
                  ) : (
                    <button onClick={() => thumbRef.current?.click()} className="w-full h-32 border-2 border-dashed border-slate-200 hover:border-cyan-500 rounded-xl flex flex-col items-center justify-center gap-2 text-slate-400 hover:text-cyan-600 bg-slate-50/50 hover:bg-cyan-50/20 transition-all">
                      <Upload className="w-6 h-6" />
                      <span className="text-xs font-semibold">Click to upload thumbnail</span>
                      <span className="text-[10px] text-slate-400">PNG, JPG up to 5MB (16:5 ratio, e.g. 1600x500px)</span>
                    </button>
```

```
                  )}
                </div>
              </CardContent>
            </Card>
          )}

          {/* Step 3: Course Outline */}
          {step === 3 && (
            <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
              <CardContent className="p-6 space-y-4">
                <div className="flex items-center gap-2 mb-2">
                  <FileText className="w-4 h-4 text-cyan-600" />
                  <h3 className="text-sm font-bold text-slate-900">Course Outline</h3>
                </div>
                <p className="text-xs text-slate-500">Outline the structure, modules, and topics covered in this course.</p>

                <div className="space-y-1.5">
                  <Label className="text-slate-700 text-xs font-semibold">Course Outline Details</Label>
                  <Textarea {...sessionFlowForm.register('session_flow_text')} rows={6} placeholder="Describe the course outl
ine, topics covered, and learning plan..." className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white rounded-xl resize-n
one" />
                </div>

                <div className="space-y-1.5 mt-4">
                  <Label className="text-slate-700 text-xs font-semibold">Course Outline Document (Optional)</Label>
                  {sessionFlowDoc ? (
                    <div className="flex items-center justify-between p-3 bg-slate-50 border border-slate-200 rounded-xl">
                      <div className="flex items-center gap-2 text-sm text-slate-900 font-medium truncate">
                        <FileText className="w-4 h-4 shrink-0 text-cyan-600" />
                        <span className="truncate">{sessionFlowDoc.name}</span>
                      </div>
                      <button onClick={() => setSessionFlowDoc(null)} className="p-1.5 rounded-full hover:bg-rose-50 text-sla
te-400 hover:text-rose-600 transition-colors">
                        <X className="w-4 h-4" />
                      </button>
                    </div>
                  ) : (
                    <div className="flex items-center gap-2">
                      <Input
                        type="file"
                        accept=".pdf,.doc,.docx,.txt"
                        className="bg-slate-50 border-slate-200 text-slate-900 cursor-pointer rounded-xl h-10"
                        onChange={(e) => {
                          const file = e.target.files?.[0]
                          if (file) {
                            if (file.size > 20 * 1024 * 1024) {
                              toast.error('File size must be less than 20MB')
                              return
                            }
                            setSessionFlowDoc(file)
                          }
                        }}
                      />
                    </div>
                  )}
                  <p className="text-[10px] text-slate-400">Upload a PDF or Word document outlining the flow (max 20MB).</p>
                </div>
              </CardContent>
            </Card>
          )}

          {/* Step 4: Learning Objectives & Skills */}
          {step === 4 && (
            <div className="space-y-4">
              <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
                <CardContent className="p-6 space-y-4">
                  <div className="flex items-center gap-2 mb-2">
                    <Target className="w-4 h-4 text-cyan-600" />
                    <h3 className="text-sm font-bold text-slate-900">Learning Objectives</h3>
                  </div>
                  <div className="space-y-1.5">
                    <Label className="text-slate-700 text-xs font-semibold">Course Learning Objectives *</Label>
                    <Textarea
                      {...objectivesForm.register('learning_objectives')}
                      rows={5}
                      placeholder="e.g. Describe the core knowledge, skills, and practical competencies learners will acquire
upon completing this course..."
                      className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white rounded-xl resize-none"
                    />
```

```
                    {objectivesForm.formState.errors.learning_objectives && (
                      <p className="text-xs text-rose-600 font-medium">{objectivesForm.formState.errors.learning_objectives.message}</p>
                    )}
                  </div>
                </CardContent>
              </Card>

              <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
                <CardContent className="p-6 space-y-4">
                  <div className="flex items-center gap-2 mb-2">
                    <Target className="w-4 h-4 text-cyan-600" />
                    <h3 className="text-sm font-bold text-slate-900">Outcomes of Learning (Skills Developed)</h3>
                  </div>
                  <p className="text-xs text-slate-500">
                    Add custom learning outcomes or select standard competencies trainees will master upon completing this course.
                  </p>

                  {/* Add Custom Outcome / Skill */}
                  <div className="flex gap-2">
                    <Input
                      value={customSkillName}
                      onChange={e => setCustomSkillName(e.target.value)}
                      onKeyDown={e => { if (e.key === 'Enter') { e.preventDefault(); addCustomSkill() } }}
                      placeholder="Type custom skill or outcome (e.g. Cyclone Track Prediction, Database Normalization)..."
                      className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:text-slate-900 h-10 text-xs rounded-xl font-medium"
                      disabled={addingSkill}
                    />
                    <Button
                      type="button"
                      onClick={addCustomSkill}
                      disabled={!customSkillName.trim() || addingSkill}
                      className="bg-cyan-600 hover:bg-cyan-700 text-white h-10 px-4 shrink-0 rounded-xl font-bold text-xs shadow-xs"
                    >
                      {addingSkill ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1.5" /> : <Plus className="w-3.5 h-3.5 mr-1.5" />}
                      Add Outcome
                    </Button>
                  </div>

                  {/* Selected Skills Summary */}
                  {selectedSkills.length > 0 && (
                    <div className="p-3 bg-cyan-50/60 border border-cyan-200 rounded-xl space-y-2">
                      <div className="flex items-center justify-between text-xs">
                        <span className="font-bold text-cyan-950 flex items-center gap-1.5">
                          <CheckCircle className="w-3.5 h-3.5 text-cyan-700" />
                          Selected Course Outcomes ({selectedSkills.length})
                        </span>
                        <button
                          type="button"
                          onClick={() => setSelectedSkills([])}
                          className="text-[11px] text-slate-500 hover:text-rose-600 font-medium"
                        >
                          Clear all
                        </button>
                      </div>
                      <div className="flex flex-wrap gap-1.5">
                        {selectedSkills.map(id => {
                          const skill = skills.find(s => s.id === id)
                          if (!skill) return null
                          return (
                            <span
                              key={id}
                              className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full text-xs font-bold bg-cyan-600 text-white shadow-xs"
                            >
                              <span>{skill.name}</span>
                              <button
                                type="button"
                                onClick={() => toggleSkill(id)}
                                className="hover:bg-cyan-700 rounded-full p-0.5"
                              >
                                <X className="w-3 h-3" />
                              </button>
                            </span>
                          )
```

```
                        })}
                      </div>
                    </div>
                  )}

                  {/* Available Suggested Competencies */}
                  <div className="space-y-2 pt-2">
                    <Label className="text-xs font-bold text-slate-700">Click to select suggested competencies:</Label>
                    {skills.length === 0 ? (
                      <p className="text-sm text-slate-400 py-4 text-center">No skills available yet. Type an outcome above to add.</p>
                    ) : (
                      <div className="flex flex-wrap gap-2 max-h-48 overflow-y-auto p-1">
                        {skills.map(s => {
                          const isSelected = selectedSkills.includes(s.id)
                          return (
                            <button
                              key={s.id}
                              type="button"
                              onClick={() => toggleSkill(s.id)}
                              className={`px-3 py-1.5 rounded-full text-xs font-semibold border transition-all cursor-pointer flex items-center gap-1.5 ${
                                isSelected
                                  ? 'bg-cyan-600 text-white border-cyan-600 shadow-xs'
                                  : 'bg-white border-slate-200 text-slate-700 hover:bg-slate-100 hover:border-slate-300'
                              }`}
                            >
                              {isSelected && <Check className="w-3 h-3" />}
                              <span>{s.name}</span>
                            </button>
                          )
                        })}
                      </div>
                    )}
                  </div>
                  <p className="text-xs text-slate-500 font-medium">{selectedSkills.length} skill{selectedSkills.length !== 1 ? 's' : ''} selected</p>
                </CardContent>
              </Card>
            </div>
          )}

          {/* Step 5: Modules */}
          {step === 5 && (
            <div className="space-y-6">
              <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 bg-gradient-to-r from-cyan-500/10 via-sky-500/10 to-blue-500/10 p-5 rounded-2xl border border-cyan-200/80">
                <div>
                  <div className="flex items-center gap-2">
                    <Layers className="w-5 h-5 text-cyan-700" />
                    <h3 className="text-base font-bold text-slate-900">Course Modules</h3>
                  </div>
                  <p className="text-xs text-slate-600 mt-1">
                    Structure the learning content into sequential modules. Add photos, video lessons, links, and content descriptions.
                  </p>
                </div>
                <Button
                  type="button"
                  onClick={addModule}
                  className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white text-xs font-bold rounded-xl shadow-xs shrink-0 self-start sm:self-auto"
                >
                  <Plus className="w-3.5 h-3.5 mr-1.5" /> Add Module
                </Button>
              </div>

              {/* Module Cards List */}
              <div className="space-y-6">
                {modules.map((mod, mIndex) => {
                  return (
                    <Card
                      key={mod.id}
                      className="transition-all rounded-2xl overflow-hidden border border-slate-200/90 bg-white shadow-xs"
                    >
                      <CardContent className="p-6 space-y-5">
                        {/* Module Header */}
                        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pb-4 border-b border-slate-100">
```

```
                          <div className="flex items-center gap-2.5">
                            <Badge
                              className="text-xs font-bold px-3 py-1 rounded-xl bg-cyan-50 text-cyan-800 border border-cyan-200"
                            >
                              Module {mIndex + 1}
                            </Badge>
                          </div>
                          {modules.length > 1 && (
                            <Button
                              type="button"
                              variant="ghost"
                              size="sm"
                              onClick={() => removeModule(mod.id)}
                              className="text-slate-400 hover:text-rose-600 hover:bg-rose-50 rounded-xl text-xs font-semibold h-8"
                            >
                              <Trash2 className="w-3.5 h-3.5 mr-1.5" /> Remove Module
                            </Button>
                          )}
                        </div>

                        {/* Title & Description */}
                        <div className="space-y-3">
                          <div className="space-y-1.5">
                            <Label className="text-slate-700 text-xs font-semibold">Module Title *</Label>
                            <Input
                              value={mod.title}
                              onChange={e => updateModule(mod.id, { title: e.target.value })}
                              placeholder={`e.g. Module ${mIndex + 1}: Core Disaster Response`}
                              className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rounded-xl text-sm font-semibold"
                            />
                          </div>
                          <div className="space-y-1.5">
                            <Label className="text-slate-700 text-xs font-semibold">Module Description / Overview</Label>
                            <Textarea
                              value={mod.description}
                              onChange={e => updateModule(mod.id, { description: e.target.value })}
                              placeholder="Describe what trainees will learn and review in this module..."
                              rows={2}
                              className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white rounded-xl text-xs resize-none"
                            />
                          </div>
                        </div>

                        {/* Unified Module Content & Assessment Sequence */}
                        <div className="space-y-4 pt-2">
                          <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pb-2 border-b border-slate-100">
                            <div>
                              <div className="flex items-center gap-2">
                                <Label className="text-slate-900 text-xs font-bold flex items-center gap-1.5">
                                  <Layers className="w-3.5 h-3.5 text-cyan-600" />
                                  Module Content & Assessment Sequence ({mod.items.length})
                                </Label>
                                {(mod.items.some(i => i.type === 'quiz') || (mod.quiz_questions || []).length > 0) && (
                                  <Badge className="bg-amber-50 text-amber-800 border-amber-200 text-[10px] font-bold px-2 py-0.5">
                                    80% Quiz Pass Gate Active
                                  </Badge>
                                )}
                              </div>
                              <p className="text-[11px] text-slate-500 mt-0.5">
                                Add and sequence videos, diagrams, notes, and quiz questions in the exact order trainees will experience them.
                              </p>
                            </div>

                            {/* Toolbar Buttons */}
                            <div className="flex flex-wrap items-center gap-1.5 shrink-0">
                              <Button
                                type="button"
                                variant="outline"
                                size="sm"
                                onClick={() => openAddItem(mod.id, 'text')}
                                className="h-7 px-2 text-[11px] border-purple-200 text-purple-800 bg-purple-50/50 hover:bg-purple-100 rounded-lg font-semibold"
```

```
                                >
                                  <AlignLeft className="w-3 h-3 mr-1 text-purple-600" /> + Text
                                </Button>
                                <Button
                                  type="button"
                                  variant="outline"
                                  size="sm"
                                  onClick={() => openAddItem(mod.id, 'video')}
                                  className="h-7 px-2 text-[11px] border-blue-200 text-blue-800 bg-blue-50/50 hover:bg-blue-100
rounded-lg font-semibold"
                                >
                                  <Film className="w-3 h-3 mr-1 text-blue-600" /> + Video
                                </Button>
                                <Button
                                  type="button"
                                  variant="outline"
                                  size="sm"
                                  onClick={() => openAddItem(mod.id, 'photo')}
                                  className="h-7 px-2 text-[11px] border-cyan-200 text-cyan-800 bg-cyan-50/50 hover:bg-cyan-100
rounded-lg font-semibold"
                                >
                                  <Camera className="w-3 h-3 mr-1 text-cyan-600" /> + Photo
                                </Button>
                                <Button
                                  type="button"
                                  variant="outline"
                                  size="sm"
                                  onClick={() => openAddItem(mod.id, 'link')}
                                  className="h-7 px-2 text-[11px] border-emerald-200 text-emerald-800 bg-emerald-50/50 hover:bg
-emerald-100 rounded-lg font-semibold"
                                >
                                  <Link2 className="w-3 h-3 mr-1 text-emerald-600" /> + Link
                                </Button>
                                <Button
                                  type="button"
                                  size="sm"
                                  onClick={() => openAddQuizQuestion(mod.id)}
                                  className="h-7 px-2.5 text-[11px] bg-gradient-to-r from-amber-500 to-orange-600 hover:from-am
ber-600 hover:to-orange-700 text-white rounded-lg font-bold shadow-xs"
                                >
                                  <HelpCircle className="w-3 h-3 mr-1" /> + Quiz Qn
                                </Button>
                                <Button
                                  type="button"
                                  variant="outline"
                                  size="sm"
                                  disabled={isGeneratingAiQuiz}
                                  onClick={() => handleGenerateAiQuiz(mod.id)}
                                  className="h-7 px-2 text-[11px] border-amber-300 bg-amber-50/70 text-amber-900 hover:bg-amber
-100 rounded-lg font-bold"
                                >
                                  <Sparkles className="w-3 h-3 mr-1 text-amber-600" />
                                  {isGeneratingAiQuiz ? '...' : 'AI Quiz'}
                                </Button>
                              </div>
                            </div>

                            {/* Unified Sequential List of Items */}
                            {mod.items.length > 0 ? (
                              <div className="space-y-2.5 pt-1">
                                {mod.items.map((item, itemIdx) => {
                                  const isFirst = itemIdx === 0
                                  const isLast = itemIdx === mod.items.length - 1
                                  const isQuiz = item.type === 'quiz'

                                  return (
                                    <div
                                      key={item.id || itemIdx}
                                      className={`p-3.5 rounded-xl border transition-all flex items-start gap-3 relative group
${
                                        isQuiz
                                          ? 'bg-amber-50/40 border-amber-200/90 hover:border-amber-300'
                                          : 'bg-slate-50 border-slate-200/90 hover:border-cyan-300'
                                      }`}
                                    >
                                      {/* Sequential Step Badge */}
                                      <div className="flex flex-col items-center gap-1 shrink-0 pt-0.5">
                                        <span className={`w-6 h-6 rounded-full flex items-center justify-center text-xs font-bl
ack shadow-xs ${
```

```
                                        isQuiz
                                          ? 'bg-amber-600 text-white'
                                          : 'bg-slate-900 text-white'
                                      }`}>
                                        {itemIdx + 1}
                                      </span>
                                      <span className="text-[9px] font-bold text-slate-400 uppercase">Step</span>
                                    </div>

                                    {/* Icon & Type Badge */}
                                    <div className="flex-1 min-w-0 pr-24">
                                      <div className="flex flex-wrap items-center gap-2 mb-1">
                                        <Badge className={`text-[10px] font-bold px-2 py-0.5 rounded-md flex items-center gap
-1 ${
                                          item.type === 'photo' ? 'bg-cyan-50 text-cyan-800 border-cyan-200' :
                                          item.type === 'video' ? 'bg-blue-50 text-blue-800 border-blue-200' :
                                          item.type === 'link' ? 'bg-emerald-50 text-emerald-800 border-emerald-200' :
                                          item.type === 'text' ? 'bg-purple-50 text-purple-800 border-purple-200' :
                                          'bg-amber-100 text-amber-900 border-amber-300'
                                        }`}>
                                          {item.type === 'photo' && <Camera className="w-3 h-3" />}
                                          {item.type === 'video' && <Film className="w-3 h-3" />}
                                          {item.type === 'link' && <Link2 className="w-3 h-3" />}
                                          {item.type === 'text' && <AlignLeft className="w-3 h-3" />}
                                          {item.type === 'quiz' && <HelpCircle className="w-3 h-3 text-amber-700" />}
                                          <span className="capitalize">{item.type === 'quiz' ? 'Quiz Question' : item.type ==
= 'text' ? 'Text / Notes' : item.type}</span>
                                        </Badge>

                                        {item.type === 'video' && (
                                          <Badge className="bg-blue-100/80 text-blue-800 border-blue-300 text-[10px] font-bol
d flex items-center gap-1">
                                            <Clock className="w-2.5 h-2.5" />
                                            {item.duration_minutes || (item.duration_seconds ? Math.round(item.duration_secon
ds / 60) : 5)} min (90% required)
                                          </Badge>
                                        )}

                                        <p className="text-xs font-bold text-slate-900 truncate">
                                          {item.title}
                                        </p>
                                      </div>

                                      {/* Quiz Details */}
                                      {isQuiz && item.quiz_data && (
                                        <div className="space-y-1.5 mt-1.5 pl-1">
                                          <p className="text-xs text-slate-800 font-semibold">{item.quiz_data.question}</p>
                                          <div className="grid grid-cols-1 sm:grid-cols-2 gap-1.5">
                                            {item.quiz_data.options.map((opt, oIdx) => {
                                              const isCorrect = oIdx === item.quiz_data?.correct_option
                                              return (
                                                <div
                                                  key={oIdx}
                                                  className={`p-1.5 px-2 rounded-lg text-[11px] flex items-center gap-1.5 bor
der ${
                                                    isCorrect
                                                      ? 'bg-emerald-50 border-emerald-300 text-emerald-900 font-semibold'
                                                      : 'bg-white border-slate-200 text-slate-600'
                                                  }`}
                                                >
                                                  <span className={`w-3.5 h-3.5 rounded-full text-[9px] font-black flex items
-center justify-center shrink-0 ${
                                                    isCorrect ? 'bg-emerald-600 text-white' : 'bg-slate-100 text-slate-500'
                                                  }`}>
                                                    {String.fromCharCode(65 + oIdx)}
                                                  </span>
                                                  <span className="truncate">{opt}</span>
                                                  {isCorrect && (
                                                    <span className="text-[9px] font-bold text-emerald-700 ml-auto bg-emerald
-100 px-1 rounded">
                                                      Correct
                                                    </span>
                                                  )}
                                                </div>
                                              )
                                            })}
                                          </div>
                                          {item.quiz_data.explanation && (
                                            <p className="text-[10px] text-slate-500 italic mt-1">💡 {item.quiz_data.explanat
```

```
ion}</p>
                                        )}
                                      </div>
                                    )}

                                    {/* Media & Content Details */}
                                    {!isQuiz && (
                                      <div className="space-y-1 mt-0.5">
                                        {item.url && (
                                          <p className="text-[11px] text-cyan-700 font-medium truncate flex items-center gap-1">
                                            <ExternalLink className="w-3 h-3" /> {item.url}
                                          </p>
                                        )}
                                        {item.content && (
                                          <p className="text-xs text-slate-600 line-clamp-2 leading-relaxed bg-white/70 p-2 rounded-lg border border-slate-100">
                                            {item.content}
                                          </p>
                                        )}
                                        {item.previewUrl && (
                                          <div className="mt-1.5 w-40 h-24 rounded-lg overflow-hidden border border-slate-200 bg-white">
                                            <img src={item.previewUrl} alt="" className="w-full h-full object-cover" />
                                          </div>
                                        )}
                                        {item.fileSize && (
                                          <span className="text-[10px] text-slate-400 font-medium">
                                            File: {item.fileName || 'Uploaded media'} ({formatFileSize(item.fileSize)})
                                          </span>
                                        )}
                                      </div>
                                    )}
                                  </div>

                                  {/* Action Controls: Move Up, Move Down, Edit, Delete */}
                                  <div className="absolute top-3 right-3 flex items-center gap-1 bg-white/90 p-1 rounded-lg border border-slate-200/90 shadow-xs">
                                    <button
                                      type="button"
                                      disabled={isFirst}
                                      onClick={() => moveModuleItem(mod.id, itemIdx, itemIdx - 1)}
                                      className="p-1 rounded text-slate-500 hover:text-slate-900 hover:bg-slate-100 disabled:opacity-30 disabled:hover:bg-transparent"
                                      title="Move Up in sequence"
                                    >
                                      <ArrowUp className="w-3.5 h-3.5" />
                                    </button>
                                    <button
                                      type="button"
                                      disabled={isLast}
                                      onClick={() => moveModuleItem(mod.id, itemIdx, itemIdx + 1)}
                                      className="p-1 rounded text-slate-500 hover:text-slate-900 hover:bg-slate-100 disabled:opacity-30 disabled:hover:bg-transparent"
                                      title="Move Down in sequence"
                                    >
                                      <ArrowDown className="w-3.5 h-3.5" />
                                    </button>
                                    <button
                                      type="button"
                                      onClick={() => openEditItem(mod.id, item)}
                                      className="p-1 rounded text-slate-500 hover:text-cyan-600 hover:bg-cyan-50"
                                      title="Edit item"
                                    >
                                      <Edit3 className="w-3.5 h-3.5" />
                                    </button>
                                    <button
                                      type="button"
                                      onClick={() => removeModuleItem(mod.id, item.id)}
                                      className="p-1 rounded text-slate-500 hover:text-rose-600 hover:bg-rose-50"
                                      title="Delete item"
                                    >
                                      <Trash2 className="w-3.5 h-3.5" />
                                    </button>
                                  </div>
                                </div>
                              )
                            })}
                          </div>
```

```
                          ) : (
                            <div className="p-6 rounded-2xl border border-dashed border-slate-200 bg-slate-50/60 text-center space-y-2">
                              <Layers className="w-8 h-8 text-slate-300 mx-auto" />
                              <div>
                                <p className="text-xs font-bold text-slate-700">No items added to this module yet.</p>
                                <p className="text-[11px] text-slate-500 mt-0.5">
                                  Use the toolbar buttons (+ Text, + Video, + Photo, + Link, + Quiz Qn) to build this module's curriculum.
                                </p>
                              </div>
                            </div>
                          )}
                        </div>
                      </CardContent>
                    </Card>
                  )
                })}
              </div>
            </div>
          )}

          {/* Module Quiz Question Addition / Edit Modal */}
          {activeQuizModuleId && (
            <div className="fixed inset-0 z-50 bg-slate-900/40 backdrop-blur-xs flex items-center justify-center p-4">
              <div className="bg-white border border-slate-200 rounded-2xl shadow-xl max-w-lg w-full p-5 space-y-4 animate-in fade-in zoom-in-95 duration-150">
                <div className="flex items-center justify-between pb-2 border-b border-slate-100">
                  <div className="flex items-center gap-2">
                    <HelpCircle className="w-4 h-4 text-amber-500" />
                    <h3 className="text-sm font-bold text-slate-900">
                      {editingQuestionId ? 'Edit Quiz Question' : 'Add Quiz Question to Module'}
                    </h3>
                  </div>
                  <button
                    type="button"
                    onClick={() => {
                      setActiveQuizModuleId(null)
                      setEditingQuestionId(null)
                    }}
                    className="p-1 rounded-lg text-slate-400 hover:text-slate-600 hover:bg-slate-100"
                  >
                    <X className="w-4 h-4" />
                  </button>
                </div>

                <div className="p-2.5 rounded-xl bg-amber-50 border border-amber-200 flex items-center gap-2">
                  <Badge className="bg-amber-600 text-white text-[10px] font-bold">Rule</Badge>
                  <p className="text-xs text-amber-900 font-medium">
                    Trainees must score <strong>≥80%</strong> on this module quiz to unlock the next module.
                  </p>
                </div>

                <div className="space-y-3">
                  <div className="space-y-1">
                    <Label className="text-xs font-semibold text-slate-700">Question Prompt *</Label>
                    <Textarea
                      value={quizQuestionText}
                      onChange={e => setQuizQuestionText(e.target.value)}
                      placeholder="e.g. What is the standard protocol for classifying cyclone landfall zones?"
                      rows={2}
                      className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:text-slate-900 rounded-xl text-xs resize-none font-medium"
                    />
                  </div>

                  <div className="space-y-2">
                    <Label className="text-xs font-semibold text-slate-700 flex items-center justify-between">
                      <span>Answer Choices (Select the correct answer) *</span>
                      <span className="text-[10px] text-slate-400 font-normal">Click letter circle to set correct</span>
                    </Label>
                    <div className="space-y-2">
                      {quizOptions.map((opt, idx) => (
                        <div key={idx} className="flex items-center gap-2">
                          <button
                            type="button"
                            onClick={() => setQuizCorrectOption(idx)}
                            className={`w-6 h-6 rounded-full flex items-center justify-center font-black text-xs shrink-0 transition-all ${
```

```
                              quizCorrectOption === idx
                                ? 'bg-emerald-600 text-white shadow-xs scale-105'
                                : 'bg-slate-100 text-slate-500 hover:bg-slate-200'
                            }`}
                            title="Set as correct answer"
                          >
                            {String.fromCharCode(65 + idx)}
                          </button>
                          <Input
                            value={opt}
                            onChange={e => {
                              const newOpts = [...quizOptions]
                              newOpts[idx] = e.target.value
                              setQuizOptions(newOpts)
                            }}
                            placeholder={`Option ${String.fromCharCode(65 + idx)}...`}
                            className={`h-9 rounded-xl text-xs font-medium placeholder:text-slate-400 ${
                              quizCorrectOption === idx
                                ? 'bg-emerald-50/70 border-emerald-300 text-slate-900 focus:bg-white focus:text-slate-900'
                                : 'bg-white border-slate-300 text-slate-900 focus:bg-white focus:text-slate-900'
                            }`}
                          />
                          {quizCorrectOption === idx && (
                            <Badge className="bg-emerald-100 text-emerald-800 text-[10px] font-bold shrink-0">
                              Correct
                            </Badge>
                          )}
                        </div>
                      ))}
                    </div>
                  </div>

                  <div className="space-y-1">
                    <Label className="text-xs font-semibold text-slate-700">Explanation / Feedback (Optional)</Label>
                    <Input
                      value={quizExplanation}
                      onChange={e => setQuizExplanation(e.target.value)}
                      placeholder="Why is this the correct answer?"
                      className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:tex
t-slate-900 h-9 rounded-xl text-xs font-medium"
                    />
                  </div>
                </div>

                <div className="flex items-center justify-end gap-2 pt-3 border-t border-slate-100">
                  <Button
                    type="button"
                    variant="ghost"
                    size="sm"
                    onClick={() => {
                      setActiveQuizModuleId(null)
                      setEditingQuestionId(null)
                    }}
                    className="text-xs text-slate-600"
                  >
                    Cancel
                  </Button>
                  <Button
                    type="button"
                    size="sm"
                    onClick={saveQuizQuestion}
                    className="bg-gradient-to-r from-amber-500 to-orange-600 hover:from-amber-600 hover:to-orange-700 text-wh
ite text-xs font-bold rounded-xl"
                  >
                    {editingQuestionId ? 'Save Changes' : 'Add Question'}
                  </Button>
                </div>
              </div>
            </div>
          )}

          {/* Module Item Addition Modal */}
          {addingItemType && activeModuleId && (
            <div className="fixed inset-0 z-50 bg-slate-900/40 backdrop-blur-xs flex items-center justify-center p-4">
              <div className="bg-white border border-slate-200 rounded-2xl shadow-xl max-w-md w-full p-5 space-y-4 animate-in
fade-in zoom-in-95 duration-150">
                <div className="flex items-center justify-between pb-2 border-b border-slate-100">
                  <div className="flex items-center gap-2">
                    {addingItemType === 'photo' && <Camera className="w-4 h-4 text-cyan-600" />}
```

```
                    {addingItemType === 'video' && <Film className="w-4 h-4 text-blue-600" />}
                    {addingItemType === 'link' && <Link2 className="w-4 h-4 text-emerald-600" />}
                    {addingItemType === 'text' && <AlignLeft className="w-4 h-4 text-purple-600" />}
                    <h3 className="text-sm font-bold text-slate-900 capitalize">Add {addingItemType} to Module</h3>
                  </div>
                  <button
                    type="button"
                    onClick={() => {
                      setAddingItemType(null)
                      setActiveModuleId(null)
                    }}
                    className="p-1 rounded-lg text-slate-400 hover:text-slate-600 hover:bg-slate-100"
                  >
                    <X className="w-4 h-4" />
                  </button>
                </div>

                <div className="space-y-3">
                  <div className="space-y-1">
                    <Label className="text-xs font-semibold text-slate-700">Item Title *</Label>
                    <Input
                      value={itemTitle}
                      onChange={e => setItemTitle(e.target.value)}
                      placeholder={`e.g. ${addingItemType === 'video' ? 'Introductory Lecture Video' : addingItemType === 'photo' ? 'Chart Diagram' : 'Topic Overview'}`}
                      className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:text-slate-900 h-10 rounded-xl text-xs font-medium"
                    />
                  </div>

                  {addingItemType === 'photo' && (
                    <div className="space-y-2">
                      <Label className="text-xs font-semibold text-slate-700">Upload Photo or Image</Label>
                      <input
                        ref={moduleFileInputRef}
                        type="file"
                        accept="image/*"
                        className="hidden"
                        onChange={handleModuleFileSelect}
                      />
                      {itemPreview ? (
                        <div className="relative rounded-xl overflow-hidden border border-slate-200 h-32 bg-slate-50">
                          <img src={itemPreview} alt="" className="w-full h-full object-cover" />
                          <button
                            type="button"
                            onClick={() => { setItemFile(null); setItemPreview(null) }}
                            className="absolute top-2 right-2 p-1 rounded-full bg-slate-900/80 text-white hover:bg-rose-600"
                          >
                            <X className="w-3.5 h-3.5" />
                          </button>
                        </div>
                      ) : (
                        <button
                          type="button"
                          onClick={() => moduleFileInputRef.current?.click()}
                          className="w-full h-24 border-2 border-dashed border-slate-200 hover:border-cyan-500 rounded-xl flex flex-col items-center justify-center gap-1.5 text-slate-400 hover:text-cyan-600 bg-slate-50/50"
                        >
                          <Upload className="w-5 h-5" />
                          <span className="text-xs font-semibold">Select image file (PNG, JPG, WebP)</span>
                        </button>
                      )}
                      <div className="pt-1">
                        <Label className="text-[11px] font-medium text-slate-500">Or Image URL</Label>
                        <Input
                          value={itemUrl}
                          onChange={e => setItemUrl(e.target.value)}
                          placeholder="https://..."
                          className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:text-slate-900 h-9 rounded-xl text-xs font-medium mt-1"
                        />
                      </div>
                    </div>
                  )}

                  {addingItemType === 'video' && (
                    <div className="space-y-3">
                      <div className="space-y-1.5 p-3 rounded-xl bg-blue-50/70 border border-blue-200/80">
                        <div className="flex items-center justify-between">
```

```
                          <Label className="text-xs font-bold text-slate-800 flex items-center gap-1.5">
                            <Clock className="w-3.5 h-3.5 text-blue-600" /> Video Length / Duration (Minutes) *
                          </Label>
                          <Badge className="bg-blue-600 text-white text-[10px] font-extrabold shadow-xs">90% Gate</Badge>
                        </div>
                        <div className="relative">
                          <Input
                            type="number"
                            min="0.5"
                            step="0.5"
                            value={itemDurationMinutes}
                            onChange={e => setItemDurationMinutes(e.target.value)}
                            placeholder="e.g. 5"
                            className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white foc
us:text-slate-900 h-9 rounded-xl text-xs font-semibold pr-16"
                          />
                          <span className="absolute right-3 top-1/2 -translate-y-1/2 text-xs font-bold text-slate-400">minute
s</span>
                        </div>
                        <p className="text-[11px] text-slate-600 leading-snug">
                          Trainees must watch at least <strong className="text-blue-700 font-bold">90%</strong> ({itemDuratio
nMinutes ? (parseFloat(itemDurationMinutes) * 0.9).toFixed(1) : '4.5'} min) of this video before they can finish this module and sub
mit the quiz.
                        </p>
                      </div>

                      <div className="space-y-2">
                        <Label className="text-xs font-semibold text-slate-700">Video Link (YouTube, Vimeo, MP4)</Label>
                        <Input
                          value={itemUrl}
                          onChange={e => setItemUrl(e.target.value)}
                          placeholder="e.g. https://www.youtube.com/watch?v=... or https://example.com/video.mp4"
                          className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focu
s:text-slate-900 h-9 rounded-xl text-xs font-medium"
                        />
                        <div className="text-center text-[10px] text-slate-400 font-semibold">— OR UPLOAD VIDEO FILE —</div>
                        <input
                          ref={moduleFileInputRef}
                          type="file"
                          accept="video/mp4,video/webm,video/quicktime"
                          className="hidden"
                          onChange={handleModuleFileSelect}
                        />
                        {itemFile ? (
                          <div className="flex items-center justify-between p-2.5 bg-slate-50 border border-slate-200 rounded
-xl text-xs font-semibold text-slate-700">
                            <span className="truncate">{itemFile.name} ({formatFileSize(itemFile.size)})</span>
                            <button
                              type="button"
                              onClick={() => setItemFile(null)}
                              className="p-1 rounded-full hover:bg-rose-50 text-slate-400 hover:text-rose-600"
                            >
                              <X className="w-3.5 h-3.5" />
                            </button>
                          </div>
                        ) : (
                          <button
                            type="button"
                            onClick={() => moduleFileInputRef.current?.click()}
                            className="w-full h-16 border-2 border-dashed border-slate-200 hover:border-blue-500 rounded-xl f
lex items-center justify-center gap-2 text-slate-400 hover:text-blue-600 bg-slate-50/50"
                          >
                            <Upload className="w-4 h-4" />
                            <span className="text-xs font-semibold">Upload MP4/WebM file (max 100MB)</span>
                          </button>
                        )}
                      </div>
                    </div>
                  )}

                  {addingItemType === 'link' && (
                    <div className="space-y-1">
                      <Label className="text-xs font-semibold text-slate-700">External URL *</Label>
                      <Input
                        value={itemUrl}
                        onChange={e => setItemUrl(e.target.value)}
                        placeholder="https://..."
                        className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:t
ext-slate-900 h-9 rounded-xl text-xs font-medium"
```

```
                      />
                    </div>
                  )}

                  <div className="space-y-1">
                    <Label className="text-xs font-semibold text-slate-700">
                      {addingItemType === 'text' ? 'Content & Explanation *' : 'Description / Caption (Optional)'}
                    </Label>
                    <Textarea
                      value={itemContent}
                      onChange={e => setItemContent(e.target.value)}
                      rows={addingItemType === 'text' ? 5 : 2}
                      placeholder="Enter description, study notes, or reading material..."
                      className="bg-white border-slate-300 text-slate-900 placeholder:text-slate-400 focus:bg-white focus:tex
t-slate-900 rounded-xl text-xs resize-none font-medium leading-relaxed"
                    />
                  </div>
                </div>

                <div className="flex items-center justify-end gap-2 pt-2 border-t border-slate-100">
                  <Button
                    type="button"
                    variant="outline"
                    size="sm"
                    onClick={() => {
                      setAddingItemType(null)
                      setActiveModuleId(null)
                    }}
                    className="border-slate-200 text-slate-700 rounded-xl text-xs"
                  >
                    Cancel
                  </Button>
                  <Button
                    type="button"
                    size="sm"
                    onClick={saveModuleItem}
                    className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white ro
unded-xl text-xs font-bold shadow-xs"
                  >
                    Save to Module
                  </Button>
                </div>
              </div>
            </div>
          )}

          {/* Step 6: Materials */}
          {step === 6 && (
            <div className="space-y-4">
              <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
                <CardContent className="p-6 space-y-4">
                  <div className="flex items-center gap-2 mb-2">
                    <FileText className="w-4 h-4 text-cyan-600" />
                    <h3 className="text-sm font-bold text-slate-900">Additional Course Materials</h3>
                  </div>
                  <p className="text-xs text-slate-500">Add supplementary documents, reference guides, and links.</p>

                  {/* Tab Toggle */}
                  <div className="flex gap-2 p-1 bg-slate-100 rounded-xl border border-slate-200">
                    <button onClick={() => setActiveTab('files')} className={`flex-1 py-2 text-xs font-bold rounded-lg transi
tion-all ${activeTab === 'files' ? 'bg-white text-slate-900 shadow-xs' : 'text-slate-500 hover:text-slate-900'}`}>
                      <FileText className="w-3.5 h-3.5 inline mr-1.5 text-cyan-600" /> Files & Videos
                    </button>
                    <button onClick={() => setActiveTab('links')} className={`flex-1 py-2 text-xs font-bold rounded-lg transi
tion-all ${activeTab === 'links' ? 'bg-white text-slate-900 shadow-xs' : 'text-slate-500 hover:text-slate-900'}`}>
                      <Globe className="w-3.5 h-3.5 inline mr-1.5 text-cyan-600" /> Links
                    </button>
                  </div>

                  {/* Files Tab */}
                  {activeTab === 'files' && (
                    <div
                      onDragOver={e => { e.preventDefault(); setDragOver(true) }}
                      onDragLeave={() => setDragOver(false)}
                      onDrop={handleDrop}
                      className={`border-2 border-dashed rounded-2xl p-6 text-center transition-all ${dragOver ? 'border-cyan
-500 bg-cyan-50/20' : 'border-slate-200 hover:border-cyan-400 bg-slate-50/50'}`}
                    >
                      <input ref={fileInputRef} type="file" multiple accept=".pdf,.doc,.docx,.pptx,.ppt,.txt,.mp4,.webm,.mo
```

```
v,.png,.jpg,.jpeg,.gif,.webp" className="hidden" onChange={handleFileInput} />
                        <Upload className="w-8 h-8 text-slate-400 mx-auto mb-2" />
                        <p className="text-sm text-slate-700 font-semibold mb-1">Drag & drop files here, or click to browse</p>
                        <p className="text-[10px] text-slate-400 mb-3">PDF, DOC, PPTX, MP4, images — up to 100MB each</p>
                        <Button variant="outline" size="sm" onClick={() => fileInputRef.current?.click()} className="border-sla
te-200 text-slate-700 hover:bg-white font-semibold rounded-xl">
                          <Upload className="w-3.5 h-3.5 mr-1.5 text-cyan-600" /> Choose Files
                        </Button>
                      </div>
                    )}

                    {/* Links Tab */}
                    {activeTab === 'links' && (
                      <div className="space-y-3">
                        <div className="space-y-1.5">
                          <Label className="text-slate-700 text-xs font-semibold">URL *</Label>
                          <Input value={linkUrl} onChange={e => setLinkUrl(e.target.value)} placeholder="https://example.com/vi
deo-or-document" className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 text-xs rounded-xl" />
                        </div>
                        <div className="space-y-1.5">
                          <Label className="text-slate-700 text-xs font-semibold">Title (optional)</Label>
                          <Input value={linkTitle} onChange={e => setLinkTitle(e.target.value)} placeholder="Descriptive title"
 className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 text-xs rounded-xl" />
                        </div>
                        <Button variant="outline" size="sm" onClick={addLinkMaterial} disabled={!linkUrl.trim()} className="bor
der-slate-200 text-slate-700 hover:bg-slate-50 font-semibold rounded-xl">
                          <Plus className="w-3.5 h-3.5 mr-1.5 text-cyan-600" /> Add Link
                        </Button>
                      </div>
                    )}

                    {/* Material List */}
                    {pendingMaterials.length > 0 && (
                      <div className="space-y-2 pt-2">
                        <div className="flex items-center justify-between">
                          <h4 className="text-xs font-bold text-slate-900">Added Materials ({pendingMaterials.length})</h4>
                          <span className="text-[10px] text-slate-400 font-semibold">{formatFileSize(totalMaterialSize)}</span>
                        </div>
                        <div className="space-y-1.5 max-h-60 overflow-y-auto">
                          {pendingMaterials.map(mat => {
                            const Icon = mat.materialType === 'link' || mat.materialType === 'video'
                              ? (mat.url?.includes('youtube') || mat.url?.includes('vimeo') ? Video : Globe)
                              : getFileIcon(mat.mimeType)
                            return (
                              <div key={mat.id} className="flex items-center gap-3 p-2.5 rounded-xl bg-slate-50 border border-s
late-200 group">
                                <div className="w-8 h-8 rounded-lg bg-white border border-slate-200 flex items-center justify-c
enter shrink-0">
                                  <Icon className="w-4 h-4 text-cyan-600" />
                                </div>
                                <div className="flex-1 min-w-0">
                                  <p className="text-xs font-semibold text-slate-900 truncate">{mat.fileName}</p>
                                  <p className="text-[10px] text-slate-500">
                                    {mat.materialType === 'link' || mat.materialType === 'video' ? mat.url : formatFileSize(ma
t.fileSize)}
                                  </p>
                                </div>
                                <Badge className="text-[9px] h-5 bg-cyan-50 text-cyan-700 border-cyan-200 font-semibold capital
ize">
                                  {mat.materialType}
                                </Badge>
                                <button onClick={() => removeMaterial(mat.id)} className="p-1 rounded-lg hover:bg-rose-50 text-
slate-400 hover:text-rose-600 transition-colors shrink-0">
                                  <Trash2 className="w-3.5 h-3.5" />
                                </button>
                              </div>
                            )
                          })}
                        </div>
                      </div>
                    )}
                  </CardContent>
                </Card>
              </div>
            )}

            {/* Step 7: Certificate */}
            {step === 7 && (
              <CourseCertificateStep
```

```
2232 |               hasCertificate={hasCertificate}
2233 |               setHasCertificate={setHasCertificate}
2234 |               templateUrl={certificateTemplateUrl}
2235 |               setTemplateUrl={setCertificateTemplateUrl}
2236 |               templateName={certificateTemplateName}
2237 |               setTemplateName={setCertificateTemplateName}
2238 |               courseTitle={watchedValues.title || 'Specialized Training Course'}
2239 |             />
2240 |           )}
2241 |
2242 |           {/* Step 8: Review & Submit */}
2243 |           {step === 8 && (
2244 |             <div className="space-y-4">
2245 |               <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
2246 |                 <CardContent className="p-6 space-y-4">
2247 |                   <div className="flex items-center gap-2 mb-2">
2248 |                     <Eye className="w-4 h-4 text-cyan-600" />
2249 |                     <h3 className="text-sm font-bold text-slate-900">Review Course</h3>
2250 |                   </div>
2251 |                   <p className="text-xs text-slate-500">Review everything before creating the course.</p>
2252 |
2253 |                   {/* Course Details */}
2254 |                   <div className="space-y-2">
2255 |                     <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
2256 |                       <BookOpen className="w-3 h-3 text-cyan-600" /> Course Details
2257 |                     </div>
2258 |                     <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5 space-y-2">
2259 |                       <div><span className="text-[10px] font-semibold text-slate-400">Title</span><p className="text-sm font-
bold text-slate-900">{watchedValues.title || '—'}</p></div>
2260 |                       <div><span className="text-[10px] font-semibold text-slate-400">Description</span><p className="text-xs
text-slate-600 line-clamp-2 leading-relaxed">{watchedValues.description || '—'}</p></div>
2261 |                       <div className="grid grid-cols-2 gap-3 pt-1">
2262 |                         <div><span className="text-[10px] font-semibold text-slate-400">Type</span><p className="text-xs font
-semibold text-slate-800 capitalize">{watchedValues.course_type}</p></div>
2263 |                         <div><span className="text-[10px] font-semibold text-slate-400">Department</span><p className="text-x
s font-semibold text-slate-800">{watchedValues.department || '—'}</p></div>
2264 |                       </div>
2265 |                     </div>
2266 |                   </div>
2267 |
2268 |                   {/* Certificate Settings Review */}
2269 |                   <div className="space-y-2">
2270 |                     <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
2271 |                       <Award className="w-3 h-3 text-cyan-600" /> Certificate Configuration
2272 |                     </div>
2273 |                     <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5 flex items-center justify-between">
2274 |                       <div>
2275 |                         <p className="text-xs font-bold text-slate-800">
2276 |                           {hasCertificate ? 'Certificate Enabled' : 'No Certificate'}
2277 |                         </p>
2278 |                         <p className="text-[11px] text-slate-500 mt-0.5">
2279 |                           {hasCertificate
2280 |                             ? (certificateTemplateName ? `Custom Template: ${certificateTemplateName}` : 'Using Standard MoES
Capacity Connect PPTX Template')
2281 |                             : 'Trainees will not receive a certificate upon completion'}
2282 |                         </p>
2283 |                       </div>
2284 |                       {hasCertificate && (
2285 |                         <span className="text-[10px] font-bold px-2.5 py-1 rounded-full bg-emerald-50 text-emerald-700 border
border-emerald-200">
2286 |                           Auto-Issue on 100%
2287 |                         </span>
2288 |                       )}
2289 |                     </div>
2290 |                   </div>
2291 |
2292 |                   {/* Settings */}
2293 |                   <div className="space-y-2">
2294 |                     <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
2295 |                       <Settings className="w-3 h-3 text-cyan-600" /> Settings
2296 |                     </div>
2297 |                     <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5">
2298 |                       <div><span className="text-[10px] font-semibold text-slate-400">Duration</span><p className="text-xs fo
nt-semibold text-slate-800">{watchedValues.duration_hours ? `${watchedValues.duration_hours} hours` : 'Self-paced'}</p></div>
2299 |                     </div>
2300 |                   </div>
2301 |
2302 |                   {/* Course Outline */}
2303 |                   {(watchedValues.session_flow_text || sessionFlowDoc) && (
```

```
                    <div className="space-y-2">
                      <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                        <FileText className="w-3 h-3 text-cyan-600" /> Course Outline
                      </div>
                      <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5 space-y-2">
                        {watchedValues.session_flow_text && (
                          <p className="text-xs text-slate-600 line-clamp-3 leading-relaxed">{watchedValues.session_flow_tex
t}</p>
                        )}
                        {sessionFlowDoc && (
                          <div className="flex items-center gap-2 text-xs font-semibold text-slate-800">
                            <File className="w-3.5 h-3.5 text-cyan-600" /> {sessionFlowDoc.name}
                          </div>
                        )}
                      </div>
                    </div>
                  )}

                  {/* Learning Objectives */}
                  <div className="space-y-2">
                    <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                      <Target className="w-3 h-3 text-cyan-600" /> Learning Objectives
                    </div>
                    <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5">
                      <p className="text-xs text-slate-600 leading-relaxed whitespace-pre-wrap">{watchedValues.learning_objec
tives || '—'}</p>
                    </div>
                  </div>

                  {/* Outcomes of Learning (Skills) */}
                  <div className="space-y-2">
                    <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                      <Target className="w-3 h-3 text-cyan-600" /> Outcomes of Learning (Skills Developed)
                    </div>
                    <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5">
                      {selectedSkills.length > 0 ? (
                        <div className="flex flex-wrap gap-1.5">
                          {selectedSkills.map(id => {
                            const skill = skills.find(s => s.id === id)
                            return skill ? <span key={id} className="px-2.5 py-0.5 rounded-full text-xs font-semibold bg-cyan
-50 border border-cyan-200 text-cyan-800">{skill.name}</span> : null
                          })}
                        </div>
                      ) : <p className="text-xs text-slate-400">No skills selected</p>}
                    </div>
                  </div>

                  {/* Course Modules Review */}
                  <div className="space-y-2">
                    <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                      <Layers className="w-3 h-3 text-cyan-600" /> Course Modules ({modules.length})
                    </div>
                    <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5 space-y-2.5">
                      {modules.map((m) => {
                        return (
                          <div key={m.id} className="p-2.5 rounded-lg bg-white border border-slate-200/90 text-xs space-y-1.
5">
                            <div className="flex items-center justify-between font-bold text-slate-900">
                              <span>{m.title}</span>
                              <div className="flex items-center gap-1.5">
                                <Badge variant="secondary" className="text-[9px] font-semibold">
                                  {m.items.length} Item{m.items.length !== 1 ? 's' : ''}
                                </Badge>
                                {(m.quiz_questions || []).length > 0 && (
                                  <Badge className="text-[9px] font-bold bg-amber-50 text-amber-800 border-amber-200">
                                    {(m.quiz_questions || []).length} Quiz Qs (≥80% Pass)
                                  </Badge>
                                )}
                              </div>
                            </div>
                            {m.description && (
                              <p className="text-[11px] text-slate-500 line-clamp-1">{m.description}</p>
                            )}
                          </div>
                        )
                      })}
                    </div>
                  </div>
```

```
                  {/* Materials */}
                  <div className="space-y-2">
                    <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                      <FileText className="w-3 h-3 text-cyan-600" /> Supplementary Materials ({pendingMaterials.length})
                    </div>
                    <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5">
                      {pendingMaterials.length > 0 ? (
                        <div className="space-y-1.5">
                          {pendingMaterials.slice(0, 5).map(mat => (
                            <div key={mat.id} className="flex items-center gap-2 text-xs text-slate-700 font-medium">
                              <File className="w-3.5 h-3.5 shrink-0 text-cyan-600" />
                              <span className="truncate">{mat.fileName}</span>
                            </div>
                          ))}
                          {pendingMaterials.length > 5 && <p className="text-[10px] text-slate-400 font-semibold">+{pendingMaterials.length - 5} more</p>}
                        </div>
                      ) : <p className="text-xs text-slate-400">No supplementary materials added</p>}
                    </div>
                  </div>

                  {/* Thumbnail */}
                  {thumbnailPreview && (
                    <div className="space-y-2">
                      <div className="flex items-center gap-2 text-[10px] font-bold text-slate-500 uppercase tracking-wider">
                        <Upload className="w-3 h-3 text-cyan-600" /> Thumbnail
                      </div>
                      <div className="bg-slate-50 border border-slate-200 rounded-xl p-3">
                        <img src={thumbnailPreview} alt="Thumbnail" className="w-full h-24 object-cover rounded-lg" />
                      </div>
                    </div>
                  )}
                </CardContent>
              </Card>

              {/* Warning */}
              <div className="flex items-start gap-2 p-3.5 bg-amber-50 border border-amber-200 rounded-2xl">
                <AlertCircle className="w-4 h-4 text-amber-600 shrink-0 mt-0.5" />
                <div className="text-xs text-amber-900 leading-relaxed">
                  <p className="font-bold mb-0.5">After admin approval:</p>
                  <p>You can add more materials, generate AI assessment questions, and manage course content from the course materials and modules page.</p>
                </div>
              </div>
            </div>
          )}
        </motion.div>

        {/* Navigation */}
        <motion.div variants={fadeUp} className="flex items-center justify-between pt-2">
          <Button variant="outline" onClick={handleBack} disabled={step === 1} className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold rounded-xl">
            <ArrowLeft className="w-4 h-4 mr-2" /> Back
          </Button>
          <div className="flex items-center gap-3">
            {step < 8 ? (
              <Button onClick={handleNext} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold shadow-md shadow-cyan-600/20 rounded-xl">
                Next <ArrowRight className="w-4 h-4 ml-2" />
              </Button>
            ) : (
              <div className="flex gap-2">
                <Button variant="outline" onClick={() => handleSubmit('draft')} disabled={saving} className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold rounded-xl">
                  {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null} Save as Draft
                </Button>
                <Button onClick={() => handleSubmit('pending_review')} disabled={saving} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold shadow-md shadow-cyan-600/20 rounded-xl">
                  {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null} Submit for Review
                </Button>
              </div>
            )}
          </div>
        </motion.div>
      </motion.div>

      <ImageCropperModal
        isOpen={isCropperOpen}
        imageFile={rawImageFile}
```

```
2452 |         onClose={() => {
2453 |           setIsCropperOpen(false)
2454 |           setRawImageFile(null)
2455 |         }}
2456 |         onCropComplete={handleCropComplete}
2457 |       />
2458 |     </TrainerLayout>
2459 |   )
2460 | }
2461 |
```

src/features/trainer/CourseDetailPage.tsx TSX 78.7 KB 1,443 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useParams, Link as RouterLink, useNavigate } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion, AnimatePresence } from 'framer-motion'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import {
  ArrowLeft, Edit3, Target, BarChart3, FileText, Clock, Users, Loader2, Calendar,
  Video, BookOpen, CheckCircle2, XCircle, UserCheck, Layers, Trophy, Globe,
  Image as ImageIcon, ExternalLink, HelpCircle, ChevronDown, ChevronUp, Sparkles,
  Eye, AlertCircle, Send, Check, Film, Camera, AlignLeft, Link2, Download, Trash2,
  Star, MessageSquare, Info, BookMarked
} from 'lucide-react'
import { Thumbnail } from '@/components/ui/Thumbnail'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription
} from '@/components/ui/dialog'
import { toast } from 'sonner'
import { CourseAnnouncements } from '../courses/CourseAnnouncements'
import { CourseChat } from '../courses/CourseChat'
import { CourseFeedback } from '../courses/CourseFeedback'
import { CourseMaterials } from '../courses/CourseMaterials'
import { LiveAttendanceTrainerPanel } from '../courses/LiveAttendanceTrainerPanel'

type Course = Database['public']['Tables']['courses']['Row'] & {
  modules?: any[] | null
  edit_request_status?: string | null
  edit_request_reason?: string | null
  edit_request_at?: string | null
  edit_window_expires_at?: string | null
  edit_window_duration_hours?: number | null
  admin_edit_notes?: string | null
}
type CourseSkill = Database['public']['Tables']['course_skills']['Row'] & { skills: { name: string } | null }
type Session = Database['public']['Tables']['course_sessions']['Row']
type Material = Database['public']['Tables']['materials']['Row']

const statusColors: Record<string, string> = {
  draft: 'bg-slate-100 text-slate-700 border border-slate-200',
  pending_review: 'bg-amber-50 text-amber-700 border border-amber-200',
  published: 'bg-emerald-50 text-emerald-700 border border-emerald-200',
  archived: 'bg-rose-50 text-rose-700 border border-rose-200',
}

function getYouTubeEmbedUrl(url?: string): string | null {
  if (!url) return null
  const regExp = /^.*(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|&v=)([^#&?]*).*/
  const match = url.match(regExp)
  return (match && match[2].length === 11) ? `https://www.youtube.com/embed/${match[2]}` : null
}

function getFormattedObjectives(raw: any): string {
  if (!raw) return ''
  if (typeof raw === 'string') return raw
  if (Array.isArray(raw)) return raw.join('\n\n')
  if (typeof raw === 'object') {
    const parts: string[] = []
    if (raw.description) parts.push(raw.description)
    if (raw.understand) parts.push(`What you'll understand: ${raw.understand}`)
    if (raw.able_to_do) parts.push(`What you'll be able to do: ${raw.able_to_do}`)
    return parts.length > 0 ? parts.join('\n\n') : JSON.stringify(raw)
  }
  return String(raw)
}

function formatTime12(timeStr?: string | null): string {
  if (!timeStr) return ''
  if (timeStr.includes('AM') || timeStr.includes('PM')) return timeStr
  const parts = timeStr.split(':')
```

```
  if (parts.length < 2) return timeStr
  let hour = parseInt(parts[0], 10)
  const minute = parts[1]
  const ampm = hour >= 12 ? 'PM' : 'AM'
  hour = hour % 12 || 12
  return `${hour.toString().padStart(2, '0')}:${minute} ${ampm}`
}

function parseSessionFlowText(text: string): Array<{ number: string; title: string; description: string }> {
  if (!text) return []
  const lines = text.split(/\r?\n/).filter(l => l.trim())
  const results: Array<{ number: string; title: string; description: string }> = []

  for (const line of lines) {
    const sessionMatch = line.match(/^[Ss]ession\s+(\d+)\s*[-\-:]\s*([^:]+?)(?::\s*(.+))?$/)
    if (sessionMatch) {
      results.push({
        number: sessionMatch[1],
        title: sessionMatch[2].trim(),
        description: sessionMatch[3]?.trim() || '',
      })
      continue
    }
    const numMatch = line.match(/^(\d+)\.\s+([^:]+?)(?::\s*(.+))?$/)
    if (numMatch) {
      results.push({
        number: numMatch[1],
        title: numMatch[2].trim(),
        description: numMatch[3]?.trim() || '',
      })
      continue
    }
    if (results.length > 0 && !results[results.length - 1].description) {
      results[results.length - 1].description = line.trim()
    } else {
      results.push({
        number: String(results.length + 1),
        title: line.trim(),
        description: '',
      })
    }
  }

  return results
}

export function CourseDetailPage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user, profile } = useAuth()
  const navigate = useNavigate()
  const [course, setCourse] = useState<Course | null>(null)
  const [courseSkills, setCourseSkills] = useState<CourseSkill[]>([])
  const [sessions, setSessions] = useState<Session[]>([])
  const [materials, setMaterials] = useState<Material[]>([])
  const [assessmentCount, setAssessmentCount] = useState(0)
  const [enrollmentCount, setEnrollmentCount] = useState(0)
  const [loading, setLoading] = useState(true)
  const [docLoading, setDocLoading] = useState(false)
  const [aboutModalOpen, setAboutModalOpen] = useState(false)
  const [aboutActiveTab, setAboutActiveTab] = useState<'about' | 'outline'>('about')
  const [previewMaterial, setPreviewMaterial] = useState<Material | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [allEnrollments, setAllEnrollments] = useState<any[]>([])
  const [isProcessingId, setIsProcessingId] = useState<string | null>(null)

  const isAdmin = profile?.role === 'admin' || profile?.role === 'super_admin'
  const isTrainer = profile?.role === 'trainer'
  const baseCoursePath = isTrainer ? '/trainer/courses' : '/admin/courses'

  // Module Preview Expansion State

  const [openModules, setOpenModules] = useState<Set<string>>(new Set())
  const [zoomImage, setZoomImage] = useState<{ url: string; title: string } | null>(null)

  // Edit Request Modal State

  const [editRequestOpen, setEditRequestOpen] = useState(false)
  const [editReason, setEditReason] = useState('')
  const [submittingEditRequest, setSubmittingEditRequest] = useState(false)
```

```
154 |
155 |   // Attendance Modal state

156 |   const [attendanceDialogOpen, setAttendanceDialogOpen] = useState(false)
157 |   const [attendanceSession, setAttendanceSession] = useState<Session | null>(null)
158 |   const [attendanceRecords, setAttendanceRecords] = useState<Record<string, 'present' | 'absent' | 'late'>>({})
159 |
160 |   const pendingEnrollments = allEnrollments.filter(e => e.status === 'pending_approval')
161 |   const waitlistedEnrollments = allEnrollments.filter(e => e.status === 'waitlisted')
162 |   const activeEnrollments = allEnrollments.filter(e => ['enrolled', 'in_progress', 'completed'].includes(e.status))
163 |   const historyEnrollments = allEnrollments.filter(e => e.status !== 'pending_approval')
164 |
165 |   const isSessionFinished = (session: Session) => {
166 |     if (!session.end_time && !session.start_time) return false
167 |     const sessionEndTime = session.end_time ? new Date(session.end_time) : new Date(session.start_time!)
168 |     return sessionEndTime < new Date()
169 |   }
170 |
171 |   const activeSessions = sessions.filter(s => !isSessionFinished(s))
172 |   const pastSessions = sessions.filter(s => isSessionFinished(s))
173 |   const activeUpcomingLiveSession = sessions.find(s => s.meet_link && !isSessionFinished(s))
174 |
175 |   const handleOpenAttendance = (session: Session) => {
176 |     setAttendanceSession(session)
177 |     const initial: Record<string, 'present' | 'absent' | 'late'> = {}
178 |     activeEnrollments.forEach(e => {
179 |       initial[e.user_id] = 'present'
180 |     })
181 |     setAttendanceRecords(initial)
182 |     setAttendanceDialogOpen(true)
183 |   }
184 |
185 |   const toggleModule = (modId: string) => {
186 |     setOpenModules(prev => {
187 |       const next = new Set(prev)
188 |       if (next.has(modId)) next.delete(modId)
189 |       else next.add(modId)
190 |       return next
191 |     })
192 |   }
193 |
194 |   const toggleAllModules = (expand: boolean) => {
195 |     if (!course?.modules || !Array.isArray(course.modules)) return
196 |     if (expand) {
197 |       setOpenModules(new Set(course.modules.map((m: any) => m.id)))
198 |     } else {
199 |       setOpenModules(new Set())
200 |     }
201 |   }
202 |
203 |   const fetchData = useCallback(async (isInitial = false) => {
204 |     if (!user || !courseId) return
205 |     if (isInitial) setLoading(true)
206 |     try {
207 |       let courseQuery = supabase.from('courses').select('*').eq('id', courseId)
208 |       if (profile?.role === 'trainer') {
209 |         courseQuery = courseQuery.eq('trainer_id', user.id)
210 |       }
211 |
212 |       let assessQuery = supabase.from('assessments').select('id', { count: 'exact', head: true }).eq('course_id', courseId)
213 |       if (profile?.role === 'trainer') {
214 |         assessQuery = assessQuery.eq('created_by', user.id)
215 |       }
216 |
217 |       const [cRes, csRes, mRes, aRes, eRes, sRes, enrollmentsRes] = await Promise.all([
218 |         courseQuery.single(),
219 |         supabase.from('course_skills').select('*, skills(name)').eq('course_id', courseId),
220 |         supabase.from('materials').select('*').eq('course_id', courseId),
221 |         assessQuery,
222 |         supabase.from('enrollments').select('*', { count: 'exact', head: true }).eq('course_id', courseId).in('status', ['enr
olled', 'in_progress', 'completed']),
223 |         supabase.from('course_sessions').select('*').eq('course_id', courseId).order('order_index'),
224 |         supabase.from('enrollments').select('*').eq('course_id', courseId).order('enrolled_at', { ascending: false }),
225 |       ])
226 |
227 |       let mergedEnrollments: any[] = []
228 |       if (enrollmentsRes.error) {
229 |         console.error('Error fetching enrollments:', enrollmentsRes.error)
230 |       } else if (enrollmentsRes.data && enrollmentsRes.data.length > 0) {
```

```
231 |         const userIds = enrollmentsRes.data.map(e => e.user_id)
232 |         const { data: traineesData, error: traineesError } = await supabase.from('trainees').select('id, full_name, email').i
n('id', userIds)
233 |         if (traineesError) console.error('Error fetching trainees:', traineesError)
234 |
235 |         mergedEnrollments = enrollmentsRes.data.map(e => ({
236 |           ...e,
237 |           trainee: traineesData?.find(t => t.id === e.user_id) || { full_name: 'Unknown Trainee', email: '' }
238 |         }))
239 |       }
240 |
241 |       if (cRes.data) {
242 |         let loadedCourse = cRes.data as Course
243 |         if (!loadedCourse.modules || !Array.isArray(loadedCourse.modules) || loadedCourse.modules.length === 0) {
244 |           const { data: dbModules } = await (supabase as any)
245 |             .from('course_modules')
246 |             .select('*, quiz_questions:quiz_questions(*)')
247 |             .eq('course_id', courseId)
248 |             .order('order_index', { ascending: true })
249 |           if (dbModules && dbModules.length > 0) {
250 |             loadedCourse = {
251 |               ...loadedCourse,
252 |               modules: dbModules
253 |             }
254 |           }
255 |         }
256 |         setCourse(loadedCourse)
257 |         // Default open first 2 modules on initial load

258 |         if (isInitial && loadedCourse.modules && Array.isArray(loadedCourse.modules)) {
259 |           setOpenModules(new Set(loadedCourse.modules.slice(0, 2).map((m: any) => m.id)))
260 |         }
261 |       }
262 |       if (csRes.data) setCourseSkills(csRes.data as any)
263 |       if (mRes.data) setMaterials(mRes.data)
264 |       if (aRes.count !== null) setAssessmentCount(aRes.count)
265 |       setEnrollmentCount(eRes.count ?? 0)
266 |       if (sRes.data) setSessions(sRes.data)
267 |       setAllEnrollments(mergedEnrollments)
268 |     } catch (err) {
269 |       console.error(err)
270 |     } finally {
271 |       if (isInitial) setLoading(false)
272 |     }
273 |   }, [user, profile?.role, courseId])
274 |
275 |   useEffect(() => { fetchData(true) }, [fetchData])
276 |
277 |   const handleOpenSessionDoc = async () => {
278 |     if (!course?.session_flow_document_path) return
279 |     setDocLoading(true)
280 |     try {
281 |       const { data, error } = await supabase.storage
282 |         .from('materials')
283 |         .createSignedUrl(course.session_flow_document_path, 300)
284 |       if (error) throw error
285 |       if (data?.signedUrl) {
286 |         setPreviewMaterial({
287 |           id: 'syllabus-doc',
288 |           file_name: 'Course Syllabus Document',
289 |           storage_path: course.session_flow_document_path,
290 |           material_type: 'document',
291 |           course_id: course.id,
292 |           created_at: new Date().toISOString(),
293 |           description: 'Uploaded Course Syllabus Document',
294 |           duration_minutes: null,
295 |           external_url: null,
296 |           order_index: 0,
297 |           thumbnail_url: null,
298 |           title: 'Course Outline & Syllabus',
299 |           topic_name: 'Outline',
300 |           uploader_id: course.trainer_id
301 |         } as any)
302 |         setPreviewUrl(data.signedUrl)
303 |       }
304 |     } catch {
305 |       toast.error('Could not open syllabus document.')
306 |     } finally {
307 |       setDocLoading(false)
```

```
    }
  }

  const handleDownload = async (mat: Material) => {
    if (!mat.storage_path) return
    try {
      const { data, error } = await supabase.storage
        .from('materials')
        .download(mat.storage_path)
      if (error) throw error
      const url = URL.createObjectURL(data)
      const a = document.createElement('a')
      a.href = url
      a.download = mat.file_name || 'download'
      document.body.appendChild(a)
      a.click()
      document.body.removeChild(a)
      URL.revokeObjectURL(url)
      toast.success('Downloaded successfully')
    } catch {
      toast.error('Failed to download file')
    }
  }

  const handleApproval = async (enrollmentId: string, action: 'approve' | 'reject', trainee: any) => {
    setIsProcessingId(enrollmentId)
    try {
      const newStatus = action === 'approve' ? 'enrolled' : 'rejected'

      const { error: updateError } = await supabase
        .from('enrollments')
        .update({ status: newStatus })
        .eq('id', enrollmentId)

      if (updateError) throw updateError

      const { error: notificationError } = await supabase.from('notifications').insert({
        user_id: trainee.id,
        type: `enrollment_status:${course?.id}`,
        title: `Enrollment ${action === 'approve' ? 'Approved' : 'Rejected'}`,
        message: `Your request to enroll in ${course?.title} was ${action === 'approve' ? 'approved' : 'rejected'}.`,
      })
      if (notificationError) console.error(notificationError)

      toast.success(`Enrollment ${action === 'approve' ? 'approved' : 'rejected'} successfully`)
      fetchData()
    } catch (e: any) {
      toast.error(e.message || 'Failed to process enrollment')
      console.error(e)
    } finally {
      setIsProcessingId(null)
    }
  }

  // Handle Trainer Requesting Edit Permission from Admin

  const handleRequestEditPermission = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!editReason.trim() || editReason.trim().length < 10) {
      toast.error('Please enter a valid description (at least 10 characters) explaining why edits are needed.')
      return
    }
    if (!course || !user) return

    setSubmittingEditRequest(true)
    try {
      // 1. Insert into course_edit_requests

      const { error: reqErr } = await (supabase as any).from('course_edit_requests').insert({
        course_id: course.id,
        trainer_id: user.id,
        reason: editReason.trim(),
        status: 'pending',
        created_at: new Date().toISOString()
      })
      if (reqErr) console.warn('Edit request table insert warning:', reqErr)

      // 2. Update courses table
```

```
      const { error: courseErr } = await supabase
        .from('courses')
        .update({
          edit_request_status: 'pending',
          edit_request_reason: editReason.trim(),
          edit_request_at: new Date().toISOString(),
          admin_edit_notes: null
        } as any)
        .eq('id', course.id)

      if (courseErr) throw courseErr

      // 3. Notify Admins

      try {
        const { data: admins } = await supabase.from('admins').select('id')
        if (admins && admins.length > 0) {
          await supabase.from('notifications').insert(
            admins.map(adm => ({
              user_id: adm.id,
              type: `course_edit_request:${course.id}`,
              title: `Course Edit Permission Requested: ${course.title}`,
              message: `Trainer requested permission to edit "${course.title}". Reason: ${editReason.trim().slice(0, 100)}...
`,
            }))
          )
        }
      } catch (err) {
        console.warn('Could not notify admins', err)
      }

      toast.success('Edit permission request submitted to Admin! You will be notified once a time limit is granted.')
      setEditRequestOpen(false)
      setEditReason('')
      fetchData()
    } catch (err: any) {
      toast.error(err.message || 'Failed to submit edit request')
    } finally {
      setSubmittingEditRequest(false)
    }
  }

  // Determine active editing window

  const isEditWindowActive = Boolean(
    course?.edit_request_status === 'approved' &&
    course?.edit_window_expires_at &&
    new Date(course.edit_window_expires_at).getTime() > Date.now()
  )

  const canDirectEdit = isAdmin || course?.status === 'draft' || course?.status === 'pending_review' || isEditWindowActive

  // Format countdown string for active editing window

  const getRemainingTimeStr = () => {
    if (!course?.edit_window_expires_at) return ''
    const diff = new Date(course.edit_window_expires_at).getTime() - Date.now()
    if (diff <= 0) return 'Expired'
    const hours = Math.floor(diff / (1000 * 60 * 60))
    const mins = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60))
    return `${hours}h ${mins}m remaining`
  }

  const objectives = (course?.learning_objectives as Record<string, string> | null) ?? null

  if (loading) {
    return (
      <TrainerLayout>
        <div className="flex justify-center py-20">
          <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
        </div>
      </TrainerLayout>
    )
  }

  if (!course) {
    return (
      <TrainerLayout>
        <div className="max-w-3xl mx-auto py-20 text-center">
```

```
459 |           <p className="text-slate-500 mb-4">Course not found or you don't have access.</p>
460 |           <RouterLink to={isTrainer ? '/trainer/courses' : '/admin'}><Button variant="outline" className="border-slate-200 te
xt-slate-700">Back to Courses</Button></RouterLink>
461 |         </div>
462 |       </TrainerLayout>
463 |     )
464 |   }
465 |
466 |   return (
467 |     <TrainerLayout>
468 |       <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-4xl mx-auto space-y-6">
469 |         <motion.div variants={fadeUp}>
470 |           <RouterLink to={isTrainer ? '/trainer/courses' : '/admin'} className="flex items-center gap-2 text-sm font-semibold
text-slate-600 hover:text-slate-900 transition-colors mb-4">
471 |             <ArrowLeft className="w-4 h-4" /> Back to Courses
472 |           </RouterLink>
473 |         </motion.div>
474 |
475 |         {/* 1. Active Editing Window Banner (if authorized by Admin) */}
476 |         {isEditWindowActive && !isAdmin && (
477 |           <motion.div variants={fadeUp} className="p-5 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 tex
t-white shadow-lg flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4">
478 |             <div className="space-y-1">
479 |               <div className="flex items-center gap-2">
480 |                 <span className="w-2.5 h-2.5 rounded-full bg-emerald-300 animate-ping" />
481 |                 <h3 className="font-bold text-sm text-white">⚡ Course Editing Permission Active</h3>
482 |                 <span className="text-[11px] font-extrabold bg-white/20 text-white px-2.5 py-0.5 rounded-full backdrop-blur-x
s">
483 |                   {getRemainingTimeStr()}
484 |                 </span>
485 |               </div>
486 |               <p className="text-xs text-cyan-100 leading-relaxed">
487 |                 Admin has approved your edit request. Changes must be submitted for re-approval before{' '}
488 |                 <strong>{new Date(course.edit_window_expires_at!).toLocaleString()}</strong>.
489 |               </p>
490 |             </div>
491 |             <RouterLink to={isTrainer ? `/trainer/courses/${courseId}/edit` : `/admin/courses/${courseId}/edit`}>
492 |               <Button size="sm" className="bg-white text-cyan-900 hover:bg-cyan-50 font-bold text-xs rounded-xl shadow-md shr
ink-0">
493 |                 <Edit3 className="w-3.5 h-3.5 mr-1.5 text-cyan-700" /> Open Course Editor
494 |               </Button>
495 |             </RouterLink>
496 |           </motion.div>
497 |         )}
498 |
499 |         {/* 2. Pending Edit Request Status Banner */}
500 |         {!isEditWindowActive && !isAdmin && course.edit_request_status === 'pending' && (
501 |           <motion.div variants={fadeUp} className="p-4 rounded-2xl bg-amber-50 border border-amber-200 text-amber-900 flex it
ems-start gap-3">
502 |             <Clock className="w-5 h-5 text-amber-600 shrink-0 mt-0.5" />
503 |             <div className="flex-1 min-w-0 text-xs">
504 |               <div className="flex items-center justify-between">
505 |                 <p className="font-bold text-sm text-amber-950">⏳ Edit Permission Request Pending Admin Review</p>
506 |                 <span className="text-[10px] font-semibold text-amber-700">
507 |                   {course.edit_request_at ? new Date(course.edit_request_at).toLocaleDateString() : ''}
508 |                 </span>
509 |               </div>
510 |               <p className="mt-1 text-slate-700 leading-relaxed">
511 |                 <strong>Your Reason:</strong> <em>"{course.edit_request_reason}"</em>
512 |               </p>
513 |               <p className="text-[11px] text-amber-800 mt-1.5">
514 |                 The administrator has been notified and will set a time window for your edits.
515 |               </p>
516 |             </div>
517 |           </motion.div>
518 |         )}
519 |
520 |         {/* 3. Hero Header Card */}
521 |         <motion.div variants={fadeUp} className="p-6 rounded-2xl bg-white border border-slate-200/90 shadow-xs relative overf
low-hidden">
522 |           <div className="relative z-10">
523 |             <div className="flex flex-col sm:flex-row items-start gap-4 mb-4">
524 |               {course.thumbnail_path && (
525 |                 <div className="w-full sm:w-32 h-20 rounded-xl bg-slate-100 border border-slate-200 overflow-hidden shrink-
0">
526 |                   <Thumbnail path={course.thumbnail_path} alt={course.title} />
527 |                 </div>
528 |               )}
529 |               <div className="flex-1 min-w-0">
```

```
                <div className="flex flex-col sm:flex-row sm:items-start justify-between gap-3 mb-2">
                  <div>
                    <Badge className={`${statusColors[course.status]} text-[10px] font-semibold mb-2`}>{course.status.replace('_', ' ')}</Badge>
                    <div className="flex flex-wrap items-center gap-2.5">
                      <h1 className="text-2xl font-black text-slate-900 leading-tight">{course.title}</h1>
                      <button
                        onClick={() => {
                          setAboutActiveTab('about')
                          setAboutModalOpen(true)
                        }}
                        title="About Course"
                        className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-50 hover:bg-cyan-100 text-cyan-800 border border-cyan-300 text-xs font-bold transition-all shadow-xs cursor-pointer group shrink-0"
                      >
                        <Info className="w-3.5 h-3.5 text-cyan-600 group-hover:scale-110 transition-transform" />
                        <span>About</span>
                      </button>
                    </div>
                  </div>

                  <div className="flex items-center gap-2 flex-wrap">
                    {canDirectEdit ? (
                      <RouterLink to={isTrainer ? `/trainer/courses/${courseId}/edit` : `/admin/courses/${courseId}/edit`}>
                        <Button size="sm" className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold rounded-xl shadow-xs">
                          <Edit3 className="w-3.5 h-3.5 mr-1.5" /> Edit Course
                        </Button>
                      </RouterLink>
                    ) : (
                      <Button
                        size="sm"
                        onClick={() => setEditRequestOpen(true)}
                        disabled={course.edit_request_status === 'pending'}
                        className="bg-gradient-to-r from-amber-600 to-orange-600 hover:from-amber-700 hover:to-orange-700 text-white font-bold rounded-xl shadow-xs text-xs"
                      >
                        <Edit3 className="w-3.5 h-3.5 mr-1.5" />
                        {course.edit_request_status === 'pending' ? 'Edit Request Pending' : 'Request Edit Permission'}
                      </Button>
                    )}
                  </div>
                </div>
                <p className="text-sm text-slate-600 mb-4 leading-relaxed">{course.description}</p>
                <div className="flex flex-wrap items-center gap-4 text-xs font-semibold text-slate-500">
                  <span className="flex items-center gap-1"><Clock className="w-3.5 h-3.5 text-cyan-600" /> {course.duration_minutes ? Math.floor(course.duration_minutes / 60) : 0} hours</span>
                  <span>&bull;</span>
                  <span>{course.course_type === 'standard' ? 'Standard' : 'Scenario'} Training</span>
                  <span>&bull;</span>
                  <span>{course.department || 'General'}</span>
                  <span>&bull;</span>
                  <span className="flex items-center gap-1"><Users className="w-3.5 h-3.5 text-cyan-600" /> {enrollmentCount} {course.max_trainees ? `/ ${course.max_trainees}` : ''} enrolled</span>
                </div>
              </div>
            </div>

            {/* Pending Enrollments Section */}
            <div className="mb-4">
              <Card className="bg-amber-50/50 border border-amber-200 rounded-2xl">
                <CardContent className="p-5">
                  <h3 className="text-sm font-bold text-amber-900 mb-4 flex items-center gap-2">
                    <Clock className="w-4 h-4 text-amber-600" /> Pending Enrollment Requests
                    <Badge className="bg-amber-100 text-amber-800 ml-2 font-bold">{pendingEnrollments.length}</Badge>
                  </h3>
                  <div className="space-y-2.5">
                    {pendingEnrollments.length > 0 ? (
                      pendingEnrollments.map((enrollment) => (
                        <div key={enrollment.id} className="p-3.5 rounded-xl bg-white border border-amber-200 flex flex-col sm:flex-row sm:items-center justify-between gap-3 shadow-xs">
                          <div>
                            <h4 className="text-sm font-bold text-slate-900">{enrollment.trainee?.full_name || 'Unknown Trainee'}</h4>
                            <p className="text-xs text-slate-500">{enrollment.trainee?.email}</p>
                          </div>
                          <div className="flex items-center gap-2">
                            <Button
                              size="sm"
```

```
                                variant="outline"
                                className="border-emerald-300 text-emerald-700 hover:bg-emerald-50 h-8 font-semibold rounded-l
g"
                                disabled={isProcessingId === enrollment.id}
                                onClick={() => handleApproval(enrollment.id, 'approve', enrollment.trainee)}
                              >
                                {isProcessingId === enrollment.id ? <Loader2 className="w-4 h-4 animate-spin" /> : <CheckCircle
2 className="w-4 h-4 mr-1" />}
                                Approve
                              </Button>
                              <Button
                                size="sm"
                                variant="outline"
                                className="border-rose-300 text-rose-700 hover:bg-rose-50 h-8 font-semibold rounded-lg"
                                disabled={isProcessingId === enrollment.id}
                                onClick={() => handleApproval(enrollment.id, 'reject', enrollment.trainee)}
                              >
                                {isProcessingId === enrollment.id ? <Loader2 className="w-4 h-4 animate-spin" /> : <XCircle cla
ssName="w-4 h-4 mr-1" />}
                                Reject
                              </Button>
                            </div>
                          </div>
                        ))
                      ) : (
                        <p className="text-xs text-amber-800/80 font-medium">No pending enrollment requests at this time.</p>
                      )}
                    </div>
                  </CardContent>
                </Card>
              </div>

              {/* Schedule & Live Class Timing */}
              {(course.start_date || course.end_date || activeUpcomingLiveSession?.meet_link) && (
                <div className="bg-slate-50 border border-slate-200 p-4 rounded-2xl space-y-3 mt-4">
                  <h4 className="text-xs font-bold text-slate-900 flex items-center gap-2"><Calendar className="w-4 h-4 text-cy
an-600" /> Schedule & Dates</h4>
                  {(course.start_date || course.end_date) && (
                    <div className="flex items-center gap-2 text-xs text-slate-600">
                      <span className="font-semibold text-slate-800 w-24">Course Span:</span>
                      {course.start_date ? new Date(course.start_date).toLocaleDateString() : 'TBD'}
                      {' - '}
                      {course.end_date ? new Date(course.end_date).toLocaleDateString() : 'TBD'}
                    </div>
                  )}
                  {activeUpcomingLiveSession?.meet_link && (
                    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pt-2 border-t border-slate-
200/70">
                      <div className="flex items-center gap-2 text-xs text-slate-600 min-w-0">
                        <span className="font-semibold text-slate-800 w-24 shrink-0">Meeting Link:</span>
                        <a href={activeUpcomingLiveSession.meet_link} target="_blank" rel="noreferrer" className="text-cyan-600
hover:underline break-all font-medium">
                          {activeUpcomingLiveSession.meet_link}
                        </a>
                      </div>
                      <Button
                        size="sm"
                        onClick={() => handleOpenAttendance(activeUpcomingLiveSession)}
                        className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white font-bold text-xs rounded-
xl shadow-xs h-8"
                      >
                        <UserCheck className="w-3.5 h-3.5 mr-1.5" /> Monitor Attendance
                      </Button>
                    </div>
                  )}
                </div>
              )}

              {/* Test & Assessment Plan Card */}
              {(course.final_test_date || course.final_exam_timing || course.mock_test_timing || course.end_date) && (
                <div className="bg-white border border-slate-200/90 rounded-2xl p-4 md:p-5 mt-4 space-y-1.5 shadow-2xs">
                  <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                    <Target className="w-4 h-4 text-cyan-600" /> Test & Assessment Plan
                  </h4>
                  <div className="pl-6 space-y-0.5">
                    <p className="text-xs font-semibold text-slate-400">
                      {(course as any).final_test_name || 'Final Exam'}
                    </p>
                    <p className="text-sm font-bold text-slate-900">
```

```
673 |                     {course.final_test_date
674 |                       ? new Date(course.final_test_date).toLocaleDateString('en-US', { month: '2-digit', day: '2-digit', yea
r: 'numeric' })
675 |                       : course.final_exam_timing
676 |                         ? new Date(course.final_exam_timing).toLocaleDateString('en-US', { month: '2-digit', day: '2-digit',
year: 'numeric' })
677 |                         : course.end_date
678 |                           ? new Date(course.end_date).toLocaleDateString('en-US', { month: '2-digit', day: '2-digit', year:
'numeric' })
679 |                           : '10/27/2026'}
680 |                   </p>
681 |                   <p className="text-xs text-slate-500 font-medium">
682 |                     {(course as any).final_test_start_time && (course as any).final_test_end_time
683 |                       ? `${formatTime12((course as any).final_test_start_time)} - ${formatTime12((course as any).final_test_e
nd_time)}`
684 |                       : course.final_exam_timing
685 |                         ? `${new Date(course.final_exam_timing).toLocaleTimeString(undefined, { hour: '2-digit', minute: '2-d
igit' })}`
686 |                         : '06:30 PM - 07:30 PM'}
687 |                   </p>
688 |                 </div>
689 |               </div>
690 |             )}
691 |           </div>
692 |         </motion.div>
693 |
694 |         {/* 3. QUICK NAVIGATION / STATS OVERVIEW CARDS */}
695 |         <motion.div variants={fadeUp} className="grid grid-cols-2 md:grid-cols-4 gap-3.5">
696 |           {/* Card 1: Sessions */}
697 |           <div
698 |             onClick={() => navigate(`${baseCoursePath}/${courseId}/sessions`)}
699 |             className="bg-[#0c1322] border border-slate-800 hover:border-cyan-500/50 text-white rounded-2xl p-4 flex items-ce
nter gap-3.5 transition-all cursor-pointer shadow-lg group hover:bg-[#0f172a]"
700 |           >
701 |             <div className="w-11 h-11 rounded-xl bg-cyan-500/15 border border-cyan-500/30 flex items-center justify-center te
xt-cyan-400 group-hover:scale-105 transition-transform shrink-0">
702 |               <Calendar className="w-5 h-5" />
703 |             </div>
704 |             <div>
705 |               <h4 className="text-sm font-bold text-white group-hover:text-cyan-300 transition-colors">Sessions</h4>
706 |               <p className="text-xs text-slate-400">{sessions.length} sessions</p>
707 |             </div>
708 |           </div>
709 |
710 |           {/* Card 2: Materials */}
711 |           <div
712 |             onClick={() => navigate(`${baseCoursePath}/${courseId}/materials`)}
713 |             className="bg-[#0c1322] border border-slate-800 hover:border-blue-500/50 text-white rounded-2xl p-4 flex items-ce
nter gap-3.5 transition-all cursor-pointer shadow-lg group hover:bg-[#0f172a]"
714 |           >
715 |             <div className="w-11 h-11 rounded-xl bg-blue-500/15 border border-blue-500/30 flex items-center justify-center te
xt-blue-400 group-hover:scale-105 transition-transform shrink-0">
716 |               <FileText className="w-5 h-5" />
717 |             </div>
718 |             <div>
719 |               <h4 className="text-sm font-bold text-white group-hover:text-blue-300 transition-colors">Materials</h4>
720 |               <p className="text-xs text-slate-400">{materials.length} files</p>
721 |             </div>
722 |           </div>
723 |
724 |           {/* Card 3: Assessments */}
725 |           <div
726 |             onClick={() => navigate(`${baseCoursePath}/${courseId}/assessments`)}
727 |             className="bg-[#0c1322] border border-slate-800 hover:border-emerald-500/50 text-white rounded-2xl p-4 flex items
-center gap-3.5 transition-all cursor-pointer shadow-lg group hover:bg-[#0f172a]"
728 |           >
729 |             <div className="w-11 h-11 rounded-xl bg-emerald-500/15 border border-emerald-500/30 flex items-center justify-cen
ter text-emerald-400 group-hover:scale-105 transition-transform shrink-0">
730 |               <Target className="w-5 h-5" />
731 |             </div>
732 |             <div>
733 |               <h4 className="text-sm font-bold text-white group-hover:text-emerald-300 transition-colors">Assessments</h4>
734 |               <p className="text-xs text-slate-400">{assessmentCount > 0 ? `${assessmentCount} tests` : 'Not created'}</p>
735 |             </div>
736 |           </div>
737 |
738 |           {/* Card 4: Performance */}
739 |           <div
740 |             onClick={() => navigate(`${baseCoursePath}/${courseId}/performance`)}
```

```
741 |             className="bg-[#0c1322] border border-slate-800 hover:border-purple-500/50 text-white rounded-2xl p-4 flex items-
center gap-3.5 transition-all cursor-pointer shadow-lg group hover:bg-[#0f172a]"
742 |           >
743 |             <div className="w-11 h-11 rounded-xl bg-purple-500/15 border border-purple-500/30 flex items-center justify-cente
r text-purple-400 group-hover:scale-105 transition-transform shrink-0">
744 |               <BarChart3 className="w-5 h-5" />
745 |             </div>
746 |             <div>
747 |               <h4 className="text-sm font-bold text-white group-hover:text-purple-300 transition-colors">Performance</h4>
748 |               <p className="text-xs text-slate-400">{activeEnrollments.length} trainees</p>
749 |             </div>
750 |           </div>
751 |         </motion.div>
752 |
753 |         {/* 4. COMPLETE INTERACTIVE COURSE MODULES PREVIEW */}
754 |         {(course as any).modules && Array.isArray((course as any).modules) && (course as any).modules.length > 0 && (
755 |           <motion.div variants={fadeUp}>
756 |             <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm space-y-4">
757 |               <CardContent className="p-6 space-y-5">
758 |                 <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pb-3 border-b border-slate-10
0">
759 |                   <div>
760 |                     <div className="flex items-center gap-2">
761 |                       <BookOpen className="w-4 h-4 text-cyan-600" />
762 |                       <h3 className="text-base font-bold text-slate-900">
763 |                         Course Modules Preview ({(course as any).modules.length})
764 |                       </h3>
765 |                     </div>
766 |                     <p className="text-xs text-slate-500 mt-0.5">
767 |                       Full inspection of structured videos, diagrams, notes, and pass-gate assessments.
768 |                     </p>
769 |                   </div>
770 |
771 |                   <div className="flex items-center gap-2 self-start sm:self-auto flex-wrap">
772 |                     <Button
773 |                       size="sm"
774 |                       variant="outline"
775 |                       onClick={() => toggleAllModules(true)}
776 |                       className="h-8 text-xs font-semibold border-slate-200 text-slate-700 hover:bg-slate-100"
777 |                     >
778 |                       Expand All
779 |                     </Button>
780 |                     <Button
781 |                       size="sm"
782 |                       variant="outline"
783 |                       onClick={() => toggleAllModules(false)}
784 |                       className="h-8 text-xs font-semibold border-slate-200 text-slate-700 hover:bg-slate-100"
785 |                     >
786 |                       Collapse All
787 |                     </Button>
788 |                   </div>
789 |                 </div>
790 |
791 |                 <div className="space-y-4">
792 |                   {(course as any).modules.map((m: any, idx: number) => {
793 |                     const rawItems = m.items || m.content_items || []
794 |                     const quizQuestions = m.quiz_questions || []
795 |
796 |                     let unifiedItems: any[] = [...rawItems]
797 |                     const hasQuizInItems = rawItems.some((it: any) => it.type === 'quiz')
798 |                     if (!hasQuizInItems && quizQuestions.length > 0) {
799 |                       quizQuestions.forEach((q: any, qIdx: number) => {
800 |                         unifiedItems.push({
801 |                           id: q.id || `legacy-quiz-${qIdx}`,
802 |                           type: 'quiz',
803 |                           title: `Quiz: ${q.question.slice(0, 50)}...`,
804 |                           quiz_data: q,
805 |                         })
806 |                       })
807 |                     }
808 |
809 |                     const isOpen = openModules.has(m.id)
810 |
811 |                     return (
812 |                       <div
813 |                         key={m.id || idx}
814 |                         className={`rounded-2xl border transition-all overflow-hidden ${
815 |                           isOpen ? 'border-cyan-300 ring-2 ring-cyan-100/60 bg-white shadow-xs' : 'border-slate-200 bg-slate-
50/60'
```

```
816 |                         }`}
817 |                       >
818 |                         {/* Module Accordion Header */}
819 |                         <button
820 |                           onClick={() => toggleModule(m.id)}
821 |                           className="w-full flex items-center justify-between gap-3 p-4 text-left transition-colors hover:bg-
slate-100/50"
822 |                         >
823 |                           <div className="flex items-center gap-3 min-w-0">
824 |                             <span className="w-8 h-8 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600 text-white text-x
s font-black flex items-center justify-center shrink-0 shadow-xs">
825 |                               {idx + 1}
826 |                             </span>
827 |                             <div className="min-w-0">
828 |                               <h4 className="text-sm font-bold text-slate-900 truncate">{m.title || `Module ${idx + 1}`}</h4>
829 |                               {m.description && (
830 |                                 <p className="text-xs text-slate-500 truncate max-w-xl">{m.description}</p>
831 |                               )}
832 |                             </div>
833 |                           </div>
834 |
835 |                           <div className="flex items-center gap-2 shrink-0">
836 |                             <Badge className="bg-cyan-50 text-cyan-800 border-cyan-200 text-[11px] font-semibold">
837 |                               {unifiedItems.length} item{unifiedItems.length !== 1 ? 's' : ''}
838 |                             </Badge>
839 |                             {quizQuestions.length > 0 && (
840 |                               <Badge className="bg-amber-50 text-amber-800 border-amber-200 text-[11px] font-bold flex items-
center gap-1">
841 |                                 <HelpCircle className="w-3 h-3 text-amber-600" />
842 |                                 {quizQuestions.length} Q Quiz (≥80% Pass Gate)
843 |                               </Badge>
844 |                             )}
845 |                             {isOpen ? <ChevronUp className="w-4 h-4 text-cyan-600" /> : <ChevronDown className="w-4 h-4 text-
slate-400" />}
846 |                           </div>
847 |                         </button>
848 |
849 |                         {/* Complete Module Contents Body */}
850 |                         <AnimatePresence initial={false}>
851 |                           {isOpen && (
852 |                             <motion.div
853 |                               key="content"
854 |                               initial={{ height: 0, opacity: 0 }}
855 |                               animate={{ height: 'auto', opacity: 1 }}
856 |                               exit={{ height: 0, opacity: 0 }}
857 |                               transition={{ duration: 0.2 }}
858 |                               className="border-t border-slate-200 p-5 bg-slate-50/40 space-y-4"
859 |                             >
860 |                               {m.description && (
861 |                                 <div className="p-3 rounded-xl bg-white border border-slate-200 text-xs text-slate-700 leadin
g-relaxed">
862 |                                   <strong className="text-slate-900 font-bold">Module Overview: </strong> {m.description}
863 |                                 </div>
864 |                               )}
865 |
866 |                               {unifiedItems.length > 0 ? (
867 |                                 <div className="space-y-3">
868 |                                   {unifiedItems.map((item: any, sIdx: number) => {
869 |                                     const itemType = (item.type || item.step_type || 'text').toLowerCase()
870 |                                     const qData = item.quiz_data || (itemType === 'quiz' ? quizQuestions[0] : null)
871 |                                     const mediaUrl = item.media_url || item.url || item.previewUrl || item.file_url || item.l
ink
872 |                                     const textContent = item.content || item.text_content || item.text || item.description ||
item.body || item.notes || (itemType === 'text' && item.url ? item.url : null)
873 |
874 |                                     return (
875 |                                       <div
876 |                                         key={item.id || sIdx}
877 |                                         className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-3"
878 |                                       >
879 |                                         <div className="flex items-center justify-between gap-2 border-b border-slate-100 pb-
2.5">
880 |                                           <div className="flex items-center gap-2">
881 |                                             <span className="w-5 h-5 rounded-full bg-cyan-600 text-white text-[10px] font-bol
d flex items-center justify-center">
882 |                                               {sIdx + 1}
883 |                                             </span>
884 |                                             <Badge variant="outline" className="text-[11px] font-bold capitalize flex items-c
enter gap-1 bg-slate-50 border-slate-200 text-slate-800">
```

```
                                                {itemType === 'video' && <Film className="w-3 h-3 text-blue-600" />}
                                                {itemType === 'photo' && <Camera className="w-3 h-3 text-purple-600" />}
                                                {itemType === 'text' && <AlignLeft className="w-3 h-3 text-cyan-600" />}
                                                {itemType === 'quiz' && <HelpCircle className="w-3 h-3 text-amber-600" />}
                                                {itemType === 'link' && <Link2 className="w-3 h-3 text-emerald-600" />}
                                                {itemType === 'photo' ? 'Photo / Diagram' : itemType === 'video' ? 'Video Lesso
n' : itemType}
                                              </Badge>
                                              <h5 className="text-xs font-bold text-slate-900">{item.title || `Item ${sIdx + 1}
`}</h5>
                                            </div>
                                          </div>

                                          {/* 1. Text Content Viewer */}
                                          {itemType === 'text' && (
                                            <div className="p-4 bg-slate-50/80 border border-slate-200 rounded-xl text-xs text-
slate-800 whitespace-pre-wrap leading-relaxed font-sans shadow-2xs">
                                              {textContent ? (
                                                textContent
                                              ) : (
                                                <p className="text-slate-400 italic">No detailed lecture notes provided for thi
s section.</p>
                                              )}
                                            </div>
                                          )}

                                          {/* 2. Photo / Diagram Viewer with Zoom */}
                                          {itemType === 'photo' && mediaUrl && (
                                            <div className="space-y-2">
                                              <div
                                                onClick={() => setZoomImage({ url: mediaUrl, title: item.title || 'Photo Previe
w' })}
                                                className="rounded-xl overflow-hidden border border-slate-200 max-h-80 bg-slate
-900 flex items-center justify-center cursor-pointer group relative"
                                              >
                                                <img
                                                  src={mediaUrl}
                                                  alt={item.title || 'Module diagram'}
                                                  className="max-h-80 w-auto object-contain transition-transform group-hover:sc
ale-102"
                                                />
                                                <div className="absolute inset-0 bg-slate-950/40 opacity-0 group-hover:opacity-
100 transition-opacity flex items-center justify-center text-white text-xs font-bold gap-1.5">
                                                  <Eye className="w-4 h-4" /> Click to view full resolution
                                                </div>
                                              </div>
                                              {textContent && <p className="text-xs text-slate-600 italic px-1">{textContent}</
p>}
                                            </div>
                                          )}

                                          {/* 3. Video Player Preview */}
                                          {itemType === 'video' && mediaUrl && (
                                            <div className="space-y-2">
                                              {getYouTubeEmbedUrl(mediaUrl) ? (
                                                <div className="aspect-video rounded-xl overflow-hidden border border-slate-200
bg-slate-900 shadow-xs max-h-80">
                                                  <iframe
                                                    src={getYouTubeEmbedUrl(mediaUrl)!}
                                                    title={item.title}
                                                    className="w-full h-full"
                                                    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscop
e; picture-in-picture"
                                                    allowFullScreen
                                                  />
                                                </div>
                                              ) : (
                                                <video
                                                  controls
                                                  className="w-full rounded-xl border border-slate-200 max-h-80 bg-black"
                                                  src={mediaUrl}
                                                >
                                                  Your browser does not support video playback.
                                                </video>
                                              )}
                                              {textContent && <p className="text-xs text-slate-600 px-1">{textContent}</p>}
                                            </div>
                                          )}

```

```
                                        {/* 4. Link Item */}
                                        {itemType === 'link' && mediaUrl && (
                                          <div className="flex items-center justify-between p-3 bg-slate-50 border border-sla
te-200 rounded-xl">
                                            <div className="flex items-center gap-2 min-w-0">
                                              <Link2 className="w-4 h-4 text-emerald-600 shrink-0" />
                                              <a href={mediaUrl} target="_blank" rel="noreferrer" className="text-xs font-sem
ibold text-emerald-700 hover:underline truncate">
                                                {mediaUrl}
                                              </a>
                                            </div>
                                            <Button
                                              size="sm"
                                              variant="outline"
                                              className="h-7 text-xs border-emerald-300 text-emerald-700 hover:bg-emerald-50
shrink-0 gap-1"
                                              onClick={() => window.open(mediaUrl, '_blank')}
                                            >
                                              <ExternalLink className="w-3 h-3" /> Open Link
                                            </Button>
                                          </div>
                                        )}

                                        {/* 5. Multiple-choice Quiz Card */}
                                        {itemType === 'quiz' && qData && (
                                          <div className="p-4 bg-amber-50/40 border border-amber-200 rounded-xl space-y-3">
                                            <div className="flex items-start gap-2">
                                              <HelpCircle className="w-4 h-4 text-amber-600 shrink-0 mt-0.5" />
                                              <div>
                                                <p className="text-xs font-bold text-slate-900 leading-snug">{qData.question}
</p>
                                                <span className="text-[10px] font-bold text-amber-800 bg-amber-100 px-2 py-0.
5 rounded-full mt-1 inline-block">
                                                  Passing Gate: ≥80% Required to Unlock Next Module
                                                </span>
                                              </div>
                                            </div>

                                            <div className="grid grid-cols-1 sm:grid-cols-2 gap-2 pt-1 pl-6">
                                              {(qData.options || []).map((opt: string, optIdx: number) => {
                                                const isCorrect = optIdx === (qData.correct_answer ?? qData.correct_option ??
0)
                                                return (
                                                  <div
                                                    key={optIdx}
                                                    className={`p-2.5 rounded-xl border text-xs font-medium flex items-center
justify-between ${
                                                      isCorrect
                                                        ? 'bg-emerald-50 border-emerald-300 text-emerald-950 font-bold shadow
-xs'
                                                        : 'bg-white border-slate-200 text-slate-700'
                                                    }`}
                                                  >
                                                    <span>{String.fromCharCode(65 + optIdx)}. {opt}</span>
                                                    {isCorrect && (
                                                      <span className="text-[10px] font-bold text-emerald-700 bg-emerald-100
px-2 py-0.5 rounded-md flex items-center gap-1">
                                                        <Check className="w-3 h-3" /> Correct Answer
                                                      </span>
                                                    )}
                                                  </div>
                                                )
                                              })}
                                            </div>

                                            {qData.explanation && (
                                              <p className="text-[11px] text-slate-600 italic bg-white p-2.5 rounded-lg borde
r border-amber-200 pl-6">
                                                <strong className="text-amber-900 not-italic">Explanation: </strong> {qData.e
xplanation}
                                              </p>
                                            )}
                                          </div>
                                        )}
                                      </div>
                                    )
                                  })}
                                </div>
                              ) : (
```

```
                              <p className="text-xs text-slate-400 italic text-center py-4">No sequential content items configured for this module.</p>
                            )}
                          </motion.div>
                        )}
                      </AnimatePresence>
                    </div>
                  )
                })}
              </div>
            </CardContent>
          </Card>
        </motion.div>
      )}

      {/* 6. COURSE ANNOUNCEMENTS */}
      <motion.div variants={fadeUp}>
        <CourseAnnouncements courseId={courseId!} isTrainer={true} />
      </motion.div>

      {/* 8. Learning Outcomes & Skills */}
      {courseSkills.length > 0 && (
        <motion.div variants={fadeUp}>
          <Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm">
            <CardContent className="p-6 space-y-3">
              <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                <Sparkles className="w-4 h-4 text-cyan-600" /> Outcomes of Learning & Skills Developed
              </h3>
              <div className="flex flex-wrap gap-2">
                {courseSkills.map((cs: any, i: number) => (
                  <span
                    key={cs.id || i}
                    className="px-3 py-1.5 rounded-xl bg-gradient-to-r from-cyan-50 to-sky-50 text-cyan-800 border border-cyan-200 text-xs font-semibold shadow-xs"
                  >
                    {cs.skills?.name || 'Skill'}
                  </span>
                ))}
              </div>
            </CardContent>
          </Card>
        </motion.div>
      )}

      {/* 8. ENROLLMENT HISTORY & STATS */}
      <motion.div variants={fadeUp} id="enrollment-stats-section">
        <Card className="bg-gradient-to-br from-[#0c1322] via-[#090e1a] to-[#040814] text-white border border-slate-800/90 rounded-3xl shadow-xl overflow-hidden">
          <CardContent className="p-6 md:p-8 space-y-6">
            <div className="flex items-center justify-between gap-4 pb-4 border-b border-slate-800">
              <div className="flex items-center gap-2.5">
                <div className="w-8 h-8 rounded-xl bg-cyan-500/15 border border-cyan-500/30 flex items-center justify-center text-cyan-400 shrink-0 shadow-[0_0_12px_rgba(6,182,212,0.15)]">
                  <Users className="w-4 h-4" />
                </div>
                <h3 className="text-base font-bold text-white tracking-wide">Enrollment History & Stats</h3>
              </div>
              <Badge className="bg-cyan-500/15 text-cyan-300 border border-cyan-500/30 text-xs font-bold px-3 py-1 rounded-full">
                {activeEnrollments.length} / {course.max_trainees || 60} Enrolled
              </Badge>
            </div>

            {/* 4 Colored Stat Boxes */}
            <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
              <div className="p-4 rounded-2xl border border-emerald-500/30 bg-emerald-950/20 flex flex-col items-center justify-center text-center space-y-1 shadow-[0_0_15px_rgba(16,185,129,0.05)]">
                <span className="text-3xl font-black text-emerald-400">{activeEnrollments.length}</span>
                <span className="text-[11px] font-black tracking-wider text-emerald-400/90 uppercase">ACCEPTED</span>
              </div>

              <div className="p-4 rounded-2xl border border-amber-500/30 bg-amber-950/20 flex flex-col items-center justify-center text-center space-y-1 shadow-[0_0_15px_rgba(245,158,11,0.05)]">
                <span className="text-3xl font-black text-amber-400">{waitlistedEnrollments.length + pendingEnrollments.length}</span>
                <span className="text-[11px] font-black tracking-wider text-amber-400/90 uppercase">WAITLISTED</span>
              </div>

              <div className="p-4 rounded-2xl border border-rose-500/30 bg-rose-950/20 flex flex-col items-center justify-c
```

```
enter text-center space-y-1 shadow-[0_0_15px_rgba(244,63,94,0.05)]">
                    <span className="text-3xl font-black text-rose-400">{allEnrollments.filter(e => e.status === 'rejected').le
ngth}</span>
                    <span className="text-[11px] font-black tracking-wider text-rose-400/90 uppercase">REJECTED</span>
                  </div>

                  <div className="p-4 rounded-2xl border border-slate-700/60 bg-slate-900/40 flex flex-col items-center justify
-center text-center space-y-1">
                    <span className="text-3xl font-black text-slate-300">{allEnrollments.filter(e => ['dropped', 'withdrawn',
'cancelled'].includes(e.status)).length}</span>
                    <span className="text-[11px] font-black tracking-wider text-slate-400 uppercase">WITHDRAWN</span>
                  </div>
                </div>

                {/* Historical List or Empty State */}
                {historyEnrollments.length === 0 ? (
                  <div className="p-8 text-center text-xs text-slate-400 font-medium bg-[#060a14]/60 rounded-2xl border border-
slate-800/60">
                    No historical enrollments yet.
                  </div>
                ) : (
                  <div className="space-y-2 pt-2">
                    <h4 className="text-xs font-bold text-slate-400 uppercase tracking-wider">Historical Records ({historyEnrol
lments.length})</h4>
                    <div className="max-h-60 overflow-y-auto space-y-2 pr-1">
                      {historyEnrollments.map((enr: any) => (
                        <div key={enr.id} className="p-3 rounded-xl bg-[#070d1a] border border-slate-800/80 flex items-center j
ustify-between text-xs">
                          <div>
                            <p className="font-bold text-white">{enr.trainee?.full_name || 'Trainee'}</p>
                            <p className="text-[11px] text-slate-400">{enr.trainee?.email}</p>
                          </div>
                          <div className="flex items-center gap-2">
                            <Badge className={`text-[10px] font-bold capitalize ${
                              ['enrolled', 'in_progress', 'completed'].includes(enr.status)
                                ? 'bg-emerald-500/20 text-emerald-300 border border-emerald-500/40'
                                : enr.status === 'rejected'
                                  ? 'bg-rose-500/20 text-rose-300 border border-rose-500/40'
                                  : 'bg-amber-500/20 text-amber-300 border border-amber-500/40'
                            }`}>
                              {enr.status.replace('_', ' ')}
                            </Badge>
                            <span className="text-[10px] text-slate-500">
                              {enr.enrolled_at ? new Date(enr.enrolled_at).toLocaleDateString() : ''}
                            </span>
                          </div>
                        </div>
                      ))}
                    </div>
                  </div>
                )}
              </CardContent>
            </Card>
          </motion.div>

          {/* 9. COURSE FEEDBACK & RATINGS */}
          <motion.div variants={fadeUp}>
            <CourseFeedback courseId={courseId!} isTrainer={true} />
          </motion.div>

          {/* 10. Course Chat & Discussions */}
          <motion.div variants={fadeUp} className="pt-4 border-t border-slate-200">
            <CourseChat courseId={courseId!} isTrainer={true} />
          </motion.div>
        </motion.div>

        {/* MODAL 1: REQUEST EDIT PERMISSION FROM ADMIN */}
        <Dialog open={editRequestOpen} onOpenChange={setEditRequestOpen}>
          <DialogContent className="sm:max-w-[550px] bg-white border-slate-200 text-slate-900 shadow-2xl rounded-3xl p-6">
            <DialogHeader>
              <DialogTitle className="text-lg font-bold text-slate-900 flex items-center gap-2">
                <Edit3 className="w-5 h-5 text-cyan-600" /> Request Course Edit Permission
              </DialogTitle>
              <DialogDescription className="text-xs text-slate-500 mt-1">
                Since "{course?.title}" is published, modifications require administrator review. Please explain what you plan
to update. The Admin will assign an active time window for your edits.
              </DialogDescription>
            </DialogHeader>

```

```
          <form onSubmit={handleRequestEditPermission} className="space-y-4 pt-3">
            <div className="space-y-1.5">
              <Label className="text-xs font-bold text-slate-800">
                Detailed Reason & Planned Changes *
              </Label>
              <Textarea
                rows={4}
                value={editReason}
                onChange={e => setEditReason(e.target.value)}
                placeholder="e.g. Need to update Module 2 quiz pass criteria, upload revised syllabus PDF, and adjust session 3 meeting date..."
                className="bg-slate-50 border-slate-300 text-slate-900 placeholder:text-slate-400 rounded-xl text-xs font-medium resize-none"
                required
              />
              <p className="text-[11px] text-slate-400">
                Minimum 10 characters. Please specify sections or modules you will edit.
              </p>
            </div>

            <div className="p-3 rounded-xl bg-cyan-50 border border-cyan-200 text-xs text-cyan-900 leading-relaxed">
              <strong>Workflow:</strong> Once approved, an edit countdown window will start. When you finish making updates, you will submit them back to the Admin for final re-approval.
            </div>

            <DialogFooter className="pt-2 flex items-center justify-end gap-2 border-t border-slate-100">
              <Button
                type="button"
                variant="outline"
                onClick={() => setEditRequestOpen(false)}
                className="border-slate-300 text-slate-700 rounded-xl text-xs"
              >
                Cancel
              </Button>
              <Button
                type="submit"
                disabled={submittingEditRequest || editReason.trim().length < 10}
                className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold rounded-xl text-xs shadow-md gap-1.5"
              >
                {submittingEditRequest ? <Loader2 className="w-4 h-4 animate-spin" /> : <Send className="w-3.5 h-3.5" />}
                Submit Edit Request to Admin
              </Button>
            </DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      {/* MODAL 2: IMAGE ZOOM MODAL */}
      {zoomImage && (
        <Dialog open={Boolean(zoomImage)} onOpenChange={() => setZoomImage(null)}>
          <DialogContent className="max-w-4xl bg-slate-950 border-slate-800 p-6 text-white rounded-3xl">
            <DialogHeader className="pb-3 border-b border-slate-800">
              <DialogTitle className="text-base font-bold text-white flex items-center justify-between">
                <span>{zoomImage.title}</span>
                <Button
                  size="sm"
                  variant="outline"
                  className="h-7 text-xs border-slate-700 bg-slate-900 text-slate-200"
                  onClick={() => window.open(zoomImage.url, '_blank')}
                >
                  <ExternalLink className="w-3 h-3 mr-1" /> Open Full Image
                </Button>
              </DialogTitle>
            </DialogHeader>
            <div className="flex items-center justify-center max-h-[75vh] overflow-hidden py-2">
              <img src={zoomImage.url} alt={zoomImage.title} className="max-h-[70vh] w-auto object-contain rounded-xl" />
            </div>
          </DialogContent>
        </Dialog>
      )}

      {/* Attendance Modal */}
      <Dialog open={attendanceDialogOpen} onOpenChange={setAttendanceDialogOpen}>
        <DialogContent className="sm:max-w-[700px] bg-transparent border-0 shadow-none p-0">
          <LiveAttendanceTrainerPanel session={attendanceSession} enrollments={activeEnrollments} isCompleted={attendanceSession ? isSessionFinished(attendanceSession) : false} />
        </DialogContent>
      </Dialog>
```

```
1237 |
1238 |       <MaterialPreviewDialog
1239 |         material={previewMaterial}
1240 |         previewUrl={previewUrl}
1241 |         onClose={() => setPreviewMaterial(null)}
1242 |         onDownload={() => previewMaterial && handleDownload(previewMaterial)}
1243 |       />
1244 |
1245 |       {/* About Course & Course Outline Modal */}
1246 |       <Dialog open={aboutModalOpen} onOpenChange={setAboutModalOpen}>
1247 |         <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto rounded-3xl p-6 md:p-8 bg-white border border-slate-
200 shadow-2xl">
1248 |           <DialogHeader className="pb-4 border-b border-slate-100">
1249 |             <div className="flex flex-wrap items-center justify-between gap-3">
1250 |               <div className="space-y-1 pr-6">
1251 |                 <div className="flex items-center gap-2">
1252 |                   <Badge variant="outline" className="text-[10px] font-bold uppercase text-cyan-800 bg-cyan-50 border-cyan-20
0">
1253 |                     {course?.course_type || 'Standard'} Program
1254 |                   </Badge>
1255 |                   {course?.department && (
1256 |                     <Badge variant="outline" className="text-[10px] font-semibold text-slate-600 border-slate-200">
1257 |                       {course.department}
1258 |                     </Badge>
1259 |                   )}
1260 |                 </div>
1261 |                 <DialogTitle className="text-xl md:text-2xl font-black text-slate-900 leading-tight">
1262 |                   {course?.title}
1263 |                 </DialogTitle>
1264 |                 <p className="text-xs text-slate-500 font-medium">
1265 |                   {course?.duration_minutes ? `${Math.floor(course.duration_minutes / 60)} Hours` : 'Self-Paced'} &bull; Pass
Gate: {course?.passing_score || 80}%
1266 |                 </p>
1267 |               </div>
1268 |             </div>
1269 |
1270 |             {/* Navigation Tabs */}
1271 |             <div className="flex items-center gap-2 pt-4">
1272 |               <button
1273 |                 type="button"
1274 |                 onClick={() => setAboutActiveTab('about')}
1275 |                 className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
1276 |                   aboutActiveTab === 'about'
1277 |                     ? 'bg-cyan-600 text-white shadow-xs'
1278 |                     : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
1279 |                 }`}
1280 |               >
1281 |                 <BookOpen className="w-3.5 h-3.5" />
1282 |                 About This Course
1283 |               </button>
1284 |               <button
1285 |                 type="button"
1286 |                 onClick={() => setAboutActiveTab('outline')}
1287 |                 className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
1288 |                   aboutActiveTab === 'outline'
1289 |                     ? 'bg-cyan-600 text-white shadow-xs'
1290 |                     : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
1291 |                 }`}
1292 |               >
1293 |                 <Layers className="w-3.5 h-3.5" />
1294 |                 Course Outline & Roadmap
1295 |               </button>
1296 |             </div>
1297 |           </DialogHeader>
1298 |
1299 |           {/* TAB 1: ABOUT THIS COURSE */}
1300 |           {aboutActiveTab === 'about' && (
1301 |             <div className="space-y-6 pt-2">
1302 |               {/* Description */}
1303 |               <div className="space-y-2">
1304 |                 <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
1305 |                   <BookOpen className="w-3.5 h-3.5 text-cyan-600" /> Course Overview
1306 |                 </h4>
1307 |                 <p className="text-sm text-slate-700 leading-relaxed bg-slate-50 p-4 rounded-2xl border border-slate-200/80 w
hitespace-pre-line">
1308 |                   {course?.description || 'Comprehensive competency-based training designed for operational excellence.'}
1309 |                 </p>
1310 |               </div>
1311 |
```

```
1312 |               {/* Objectives */}
1313 |               {course?.learning_objectives && (
1314 |                 <div className="space-y-2">
1315 |                   <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
1316 |                     <Target className="w-3.5 h-3.5 text-cyan-600" /> Key Learning Objectives
1317 |                   </h4>
1318 |                   <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200/80 space-y-2 text-xs sm:text-sm text-sl
ate-700 leading-relaxed whitespace-pre-line font-normal">
1319 |                     {getFormattedObjectives(course.learning_objectives)}
1320 |                   </div>
1321 |                 </div>
1322 |               )}
1323 |
1324 |               {/* Key Course Specifications Grid */}
1325 |               <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
1326 |                 <div className="p-3.5 rounded-2xl bg-cyan-50/50 border border-cyan-200/60">
1327 |                   <span className="text-[10px] font-bold text-cyan-800 uppercase tracking-wider block">Passing Threshold</spa
n>
1328 |                   <span className="text-sm font-extrabold text-cyan-950 mt-0.5 block">{course?.passing_score || 80}% Overall
</span>
1329 |                 </div>
1330 |                 <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
1331 |                   <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Duration</span>
1332 |                   <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{course?.duration_minutes ? `${Math.fl
oor(course.duration_minutes / 60)} Hours` : 'Flexible'}</span>
1333 |                 </div>
1334 |                 <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
1335 |                   <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Max Trainees</span>
1336 |                   <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{course?.max_trainees || 'Unlimited'}
Seats</span>
1337 |                 </div>
1338 |               </div>
1339 |
1340 |               {/* Skills Covered */}
1341 |               {courseSkills && courseSkills.length > 0 && (
1342 |                 <div className="space-y-2">
1343 |                   <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
1344 |                     <Sparkles className="w-3.5 h-3.5 text-cyan-600" /> Core Competencies & Skills
1345 |                   </h4>
1346 |                   <div className="flex flex-wrap gap-2">
1347 |                     {courseSkills.map((cs, idx) => (
1348 |                       <span key={(cs as any).id || cs.skill_id || idx} className="px-3 py-1.5 rounded-xl bg-cyan-50 border bo
rder-cyan-200/80 text-cyan-900 text-xs font-bold">
1349 |                         {cs.skills?.name}
1350 |                       </span>
1351 |                     ))}
1352 |                   </div>
1353 |                 </div>
1354 |               )}
1355 |             </div>
1356 |           )}
1357 |
1358 |           {/* TAB 2: COURSE OUTLINE & ROADMAP */}
1359 |           {aboutActiveTab === 'outline' && (
1360 |             <div className="space-y-6 pt-2">
1361 |               {/* Syllabus Document Download / Preview */}
1362 |               {course?.session_flow_document_path && (
1363 |                 <div className="p-4 rounded-2xl bg-cyan-50/70 border border-cyan-200 flex items-center justify-between gap-
3">
1364 |                   <div className="flex items-center gap-2.5 min-w-0">
1365 |                     <FileText className="w-5 h-5 text-cyan-700 shrink-0" />
1366 |                     <div>
1367 |                       <h5 className="text-xs font-bold text-cyan-950">Official Syllabus Document</h5>
1368 |                       <p className="text-[11px] text-cyan-700">Detailed curriculum plan and reading materials</p>
1369 |                     </div>
1370 |                   </div>
1371 |                   <Button
1372 |                     size="sm"
1373 |                     onClick={handleOpenSessionDoc}
1374 |                     disabled={docLoading}
1375 |                     className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs h-8 rounded-xl shrink-0"
1376 |                   >
1377 |                     {docLoading ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Eye className="w-3.5 h-3.5 mr-1" /
>}
1378 |                     View Document
1379 |                   </Button>
1380 |                 </div>
1381 |               )}
1382 |
```

```
1383 |               {/* Session Schedule & Timeline */}
1384 |               {course?.session_flow_text && (
1385 |                 <div className="space-y-3">
1386 |                   <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
1387 |                     <Layers className="w-3.5 h-3.5 text-cyan-600" /> Session Schedule & Timeline
1388 |                   </h4>
1389 |                   <div className="bg-slate-50 border border-slate-200/80 rounded-2xl p-5 text-xs sm:text-sm text-slate-700 le
ading-relaxed whitespace-pre-wrap font-normal">
1390 |                     {course.session_flow_text}
1391 |                   </div>
1392 |                 </div>
1393 |               )}
1394 |
1395 |               {/* Structured Modules Preview */}
1396 |               {course?.modules && Array.isArray(course.modules) && course.modules.length > 0 && (
1397 |                 <div className="space-y-3 pt-1">
1398 |                   <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
1399 |                     <BookMarked className="w-3.5 h-3.5 text-cyan-600" /> Learning Modules Breakdown ({course.modules.length})
1400 |                   </h4>
1401 |                   <div className="space-y-2.5">
1402 |                     {course.modules.map((mod: any, mIdx: number) => {
1403 |                       const rawItems = mod.items || mod.content_items || []
1404 |                       const quizQuestions = mod.quiz_questions || []
1405 |                       const videoCount = rawItems.filter((i: any) => i.type === 'video').length
1406 |                       const notesCount = rawItems.filter((i: any) => i.type === 'notes' || i.type === 'pdf' || i.type === 'do
c').length
1407 |                       const quizCount = quizQuestions.length + rawItems.filter((i: any) => i.type === 'quiz').length
1408 |
1409 |                       return (
1410 |                         <div key={mod.id || mIdx} className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200 space-y-1.
5">
1411 |                           <div className="flex items-center justify-between gap-2">
1412 |                             <div className="flex items-center gap-2.5">
1413 |                               <span className="w-6 h-6 rounded-lg bg-cyan-100 text-cyan-800 font-bold text-xs flex items-cent
er justify-center shrink-0">
1414 |                                 {mIdx + 1}
1415 |                               </span>
1416 |                               <h5 className="text-xs sm:text-sm font-bold text-slate-900">{mod.title || `Module ${mIdx + 1}`}
</h5>
1417 |                             </div>
1418 |                             <Badge variant="outline" className="text-[10px] font-semibold bg-white border-slate-200 text-slat
e-600 shrink-0">
1419 |                               {rawItems.length} activities
1420 |                             </Badge>
1421 |                           </div>
1422 |                           {mod.description && (
1423 |                             <p className="text-xs text-slate-500 pl-8 line-clamp-2">{mod.description}</p>
1424 |                           )}
1425 |                           <div className="flex items-center gap-3 pl-8 pt-1 text-[10px] text-slate-500 font-medium">
1426 |                             {videoCount > 0 && <span className="text-blue-600 font-semibold">{videoCount} Video{videoCount >
1 ? 's' : ''}</span>}
1427 |                             {notesCount > 0 && <span className="text-emerald-600 font-semibold">{notesCount} Note{notesCount
> 1 ? 's' : ''}</span>}
1428 |                             {quizCount > 0 && <span className="text-purple-600 font-semibold">{quizCount} Quiz Qs</span>}
1429 |                           </div>
1430 |                         </div>
1431 |                       )
1432 |                     })}
1433 |                   </div>
1434 |                 </div>
1435 |               )}
1436 |             </div>
1437 |           )}
1438 |         </DialogContent>
1439 |       </Dialog>
1440 |     </TrainerLayout>
1441 |   )
1442 | }
1443 |
```

src/features/trainer/CourseEditPage.tsx TSX 35.9 KB 714 Lines

```tsx
import React, { useEffect, useState, useCallback, useMemo, useRef } from 'react'
import { useParams, useNavigate, Link } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { getSignedUrl } from '@/lib/storage'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from '@/components/ui/select'
import {
  ArrowLeft, Upload, Target, Video,
  Trash2, Send, Save, AlertCircle, FileText, Loader2, Calendar,
  BarChart3, Award, Plus, Clock, Edit3, CheckCircle2, Lock, Sparkles
} from 'lucide-react'
import { toast } from 'sonner'
import { CourseMaterials } from '@/features/courses/CourseMaterials'
import { CourseCertificateStep } from '@/features/courses/CourseCertificateStep'

type Course = Database['public']['Tables']['courses']['Row'] & {
  edit_request_status?: string | null
  edit_request_reason?: string | null
  edit_request_at?: string | null
  edit_window_expires_at?: string | null
  edit_window_duration_hours?: number | null
  admin_edit_notes?: string | null
}
type Skill = Database['public']['Tables']['skills']['Row']
type CourseSkill = Database['public']['Tables']['course_skills']['Row']

const courseSchema = z.object({
  title: z.string().min(5, 'Title must be at least 5 characters'),
  description: z.string().min(10, 'Description must be at least 10 characters'),
  course_type: z.enum(['standard', 'scenario']),
  department: z.string().optional(),
  duration_hours: z.coerce.number().positive('Duration must be positive').optional(),
  passing_score: z.coerce.number().min(1).max(100).optional(),
  meet_link: z.string().url('Must be a valid URL').optional().or(z.literal('')),
  start_date: z.string().optional(),
  end_date: z.string().optional(),
  session_flow_text: z.string().optional(),
}).superRefine((data, ctx) => {
  if (data.start_date) {
    const start = new Date(data.start_date)
    const today = new Date()
    today.setHours(0, 0, 0, 0)
    const diffDays = Math.ceil((start.getTime() - today.getTime()) / (1000 * 60 * 60 * 24))
    if (diffDays < 30) {
      ctx.addIssue({
        code: z.ZodIssueCode.custom,
        message: 'Course must start at least 1 month from today',
        path: ['start_date']
      })
    }
  }
})

type CourseFormData = z.infer<typeof courseSchema>

export function CourseEditPage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user } = useAuth()
  const navigate = useNavigate()
  const [course, setCourse] = useState<Course | null>(null)
  const [skills, setSkills] = useState<Skill[]>([])
  const [_courseSkills, setCourseSkills] = useState<CourseSkill[]>([])
```

```
  const [selectedSkills, setSelectedSkills] = useState<string[]>([])
  const [loading, setLoading] = useState(true)
  const [saving, setSaving] = useState(false)
  const [customSkillName, setCustomSkillName] = useState('')
  const [addingSkill, setAddingSkill] = useState(false)
  const [materialCount, setMaterialCount] = useState(0)

  // Edit Request State

  const [editReason, setEditReason] = useState('')
  const [submittingEditRequest, setSubmittingEditRequest] = useState(false)

  const [thumbnail, setThumbnail] = useState<File | null>(null)
  const [thumbnailPreview, setThumbnailPreview] = useState<string | null>(null)
  const thumbRef = useRef<HTMLInputElement>(null)

  const [sessionFlowDoc, setSessionFlowDoc] = useState<File | null>(null)
  const [sessionFlowDocPath, setSessionFlowDocPath] = useState<string | null>(null)

  const [hasCertificate, setHasCertificate] = useState(true)
  const [certificateTemplateUrl, setCertificateTemplateUrl] = useState<string | null>(null)
  const [certificateTemplateName, setCertificateTemplateName] = useState<string | null>(null)

  const { register, handleSubmit, setValue, watch, reset, trigger, formState: { errors } } = useForm<CourseFormData>({
    resolver: zodResolver(courseSchema),
  })

  const isUrgent = useMemo(() => {
    const start = watch('start_date')
    if (start) {
      const diff = new Date(start).getTime() - new Date().getTime()
      const diffDays = diff / (1000 * 60 * 60 * 24)
      return diffDays >= 0 && diffDays < 30
    }
    return false
  }, [watch('start_date')])

  const fetchData = useCallback(async () => {
    if (!user || !courseId) return
    setLoading(true)
    try {
      const [courseRes, skillsRes, csRes, matRes] = await Promise.all([
        supabase.from('courses').select('*').eq('id', courseId).eq('trainer_id', user.id).single(),
        supabase.from('skills').select('*'),
        supabase.from('course_skills').select('*').eq('course_id', courseId),
        supabase.from('materials').select('*', { count: 'exact', head: true }).eq('course_id', courseId),
      ])
      if (courseRes.error) throw courseRes.error
      setCourse(courseRes.data)
      setHasCertificate((courseRes.data as any).has_certificate ?? true)
      setCertificateTemplateUrl((courseRes.data as any).certificate_template_url ?? null)
      setCertificateTemplateName((courseRes.data as any).certificate_template_name ?? null)
      if (courseRes.data.thumbnail_path) {
        const signedUrl = await getSignedUrl('materials', courseRes.data.thumbnail_path)
        setThumbnailPreview(signedUrl)
      }
      reset({
        title: courseRes.data.title,
        description: courseRes.data.description ?? '',
        course_type: courseRes.data.course_type as 'standard' | 'scenario',
        department: courseRes.data.department ?? '',
        duration_hours: courseRes.data.duration_minutes ? Math.floor(courseRes.data.duration_minutes / 60) : undefined,
        passing_score: courseRes.data.passing_score ?? undefined,
        meet_link: courseRes.data.meet_link ?? '',
        start_date: courseRes.data.start_date ? new Date(courseRes.data.start_date).toISOString().slice(0, 16) : '',
        end_date: courseRes.data.end_date ? new Date(courseRes.data.end_date).toISOString().slice(0, 16) : '',
        session_flow_text: courseRes.data.session_flow_text ?? '',
      })
      if (courseRes.data.session_flow_document_path) {
        setSessionFlowDocPath(courseRes.data.session_flow_document_path)
      }
      if (skillsRes.data) setSkills(skillsRes.data)
      if (csRes.data) {
        setCourseSkills(csRes.data)
        setSelectedSkills(csRes.data.map(cs => cs.skill_id))
      }
      if (matRes.count !== null && matRes.count !== undefined) setMaterialCount(matRes.count)
    } catch (err) {
      toast.error('Failed to load course')
```

```
      navigate('/trainer/courses')
    } finally {
      setLoading(false)
    }
  }, [user, courseId, navigate, reset])

  useEffect(() => { fetchData() }, [fetchData])

  const handleThumbnailChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return
    if (file.size > 5 * 1024 * 1024) {
      toast.error('Thumbnail must be under 5MB')
      return
    }
    setThumbnail(file)
    setThumbnailPreview(URL.createObjectURL(file))
  }

  // Active Edit Window Check

  const isEditWindowActive = Boolean(
    course?.edit_request_status === 'approved' &&
    course?.edit_window_expires_at &&
    new Date(course.edit_window_expires_at).getTime() > Date.now()
  )

  const canEdit = course?.status === 'draft' || course?.status === 'pending_review' || isEditWindowActive

  const getRemainingTimeStr = () => {
    if (!course?.edit_window_expires_at) return ''
    const diff = new Date(course.edit_window_expires_at).getTime() - Date.now()
    if (diff <= 0) return 'Expired'
    const hours = Math.floor(diff / (1000 * 60 * 60))
    const mins = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60))
    return `${hours}h ${mins}m remaining`
  }

  // Request Edit Permission from Admin

  const handleRequestEditPermission = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!editReason.trim() || editReason.trim().length < 10) {
      toast.error('Please enter a valid description (at least 10 characters) explaining why edits are needed.')
      return
    }
    if (!course || !user) return

    setSubmittingEditRequest(true)
    try {
      await (supabase as any).from('course_edit_requests').insert({
        course_id: course.id,
        trainer_id: user.id,
        reason: editReason.trim(),
        status: 'pending',
        created_at: new Date().toISOString()
      })

      const { error: courseErr } = await supabase
        .from('courses')
        .update({
          edit_request_status: 'pending',
          edit_request_reason: editReason.trim(),
          edit_request_at: new Date().toISOString(),
          admin_edit_notes: null
        } as any)
        .eq('id', course.id)

      if (courseErr) throw courseErr

      try {
        const { data: admins } = await supabase.from('admins').select('id')
        if (admins && admins.length > 0) {
          await supabase.from('notifications').insert(
            admins.map(adm => ({
              user_id: adm.id,
              type: `course_edit_request:${course.id}`,
              title: `Course Edit Permission Requested: ${course.title}`,
              message: `Trainer requested permission to edit "${course.title}". Reason: ${editReason.trim().slice(0, 100)}...
```

```
`,
            }))
          )
        }
      } catch (err) {
        console.warn('Could not notify admins', err)
      }

      toast.success('Edit permission request submitted to Admin! You will be notified once a time limit is granted.')
      setEditReason('')
      fetchData()
    } catch (err: any) {
      toast.error(err.message || 'Failed to submit edit request')
    } finally {
      setSubmittingEditRequest(false)
    }
  }

  const handleSave = async (status: 'draft' | 'pending_review') => {
    if (!course) return
    const valid = await trigger()
    if (!valid) return

    if (status === 'pending_review') {
      const { count } = await supabase
        .from('materials')
        .select('*', { count: 'exact', head: true })
        .eq('course_id', course.id)
      if (!count || count === 0) {
        toast.error('Add at least one material (file or link) before submitting for review')
        return
      }
    }

    setSaving(true)
    try {
      let thumbnailPath = course.thumbnail_path

      if (thumbnail) {
        const ext = thumbnail.name.split('.').pop()
        thumbnailPath = `${course.id}/thumbnail.${ext}`
        const { error: uploadErr } = await supabase.storage
          .from('materials')
          .upload(thumbnailPath, thumbnail, { upsert: true })
        if (uploadErr) {
          toast.error('Failed to upload thumbnail')
          console.error(uploadErr)
        }
      }

      let sessionDocPathToSave = course.session_flow_document_path
      if (sessionFlowDoc) {
        const ext = sessionFlowDoc.name.split('.').pop()
        sessionDocPathToSave = `${course.id}/session_flow_${crypto.randomUUID()}.${ext}`
        const { error: uploadErr } = await supabase.storage
          .from('materials')
          .upload(sessionDocPathToSave, sessionFlowDoc, { upsert: true })
        if (uploadErr) {
          toast.error('Failed to upload session flow document')
        }
      } else if (sessionFlowDocPath === null) {
        sessionDocPathToSave = null
      }

      const data = watch()
      const { duration_hours, ...rest } = data
      const updateData: any = {
        ...rest,
        status: isEditWindowActive && status === 'pending_review' ? 'pending_review' : status,
        duration_minutes: duration_hours ? duration_hours * 60 : null,
        start_date: data.start_date ? new Date(data.start_date).toISOString() : null,
        end_date: data.end_date ? new Date(data.end_date).toISOString() : null,
        thumbnail_path: thumbnailPath,
        session_flow_document_path: sessionDocPathToSave,
        has_certificate: hasCertificate,
        certificate_template_url: certificateTemplateUrl,
        certificate_template_name: certificateTemplateName,
      }

```

```
      // If submitting after edit window, set edit_request_status to submitted

      if (isEditWindowActive && status === 'pending_review') {
        updateData.edit_request_status = 'submitted'
      }

      const { error } = await supabase
        .from('courses')
        .update(updateData as any)
        .eq('id', course.id)
      if (error) throw error

      await supabase.from('course_skills').delete().eq('course_id', course.id)
      if (selectedSkills.length > 0) {
        await supabase.from('course_skills').insert(
          selectedSkills.map(skill_id => ({ course_id: course.id, skill_id, required_level: 3 }))
        )
      }

      toast.success(
        isEditWindowActive && status === 'pending_review'
          ? 'Course updates submitted to Admin for re-approval!'
          : status === 'draft'
            ? 'Course saved as draft'
            : 'Course submitted for review'
      )
      navigate(`/trainer/courses/${courseId}`)
    } catch (err: unknown) {
      toast.error(err instanceof Error ? err.message : 'Failed to save')
    } finally {
      setSaving(false)
    }
  }

  const toggleSkill = (id: string) => {
    setSelectedSkills(prev => prev.includes(id) ? prev.filter(s => s !== id) : [...prev, id])
  }

  const addCustomSkill = async () => {
    const name = customSkillName.trim()
    if (!name) return
    if (skills.some(s => s.name.toLowerCase() === name.toLowerCase())) {
      toast.error('Skill already exists')
      return
    }
    setAddingSkill(true)
    try {
      const { data, error } = await supabase.from('skills').insert({ name }).select().single()
      if (error) throw error
      setSkills(prev => [...prev, data].sort((a, b) => a.name.localeCompare(b.name)))
      setSelectedSkills(prev => [...prev, data.id])
      setCustomSkillName('')
      toast.success(`Skill "${name}" created`)
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed to add skill')
    } finally {
      setAddingSkill(false)
    }
  }

  if (loading) {
    return (
      <TrainerLayout>
        <div className="flex items-center justify-center py-20">
          <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
        </div>
      </TrainerLayout>
    )
  }

  if (!course) return null

  return (
    <TrainerLayout>
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-3xl mx-auto space-y-6">
        <motion.div variants={fadeUp}>
          <button onClick={() => navigate(`/trainer/courses/${courseId}`)} className="flex items-center gap-2 text-sm font-sem
ibold text-slate-600 hover:text-slate-900 transition-colors mb-4">
            <ArrowLeft className="w-4 h-4" /> Back to Course Details
```

```
387 |           </button>
388 |           <div className="flex items-center justify-between">
389 |             <div>
390 |               <h2 className="text-2xl font-bold tracking-tight text-slate-900">{course.title}</h2>
391 |               <div className="flex items-center gap-2 mt-1">
392 |                 <Badge className={
393 |                   course.status === 'draft' ? 'bg-slate-100 text-slate-700 border-slate-200' :
394 |                   course.status === 'pending_review' ? 'bg-amber-50 text-amber-700 border-amber-200' :
395 |                   course.status === 'published' ? 'bg-emerald-50 text-emerald-700 border-emerald-200' :
396 |                   'bg-rose-50 text-rose-700 border-rose-200'
397 |                 }>
398 |                   {course.status.replace('_', ' ')}
399 |                 </Badge>
400 |                 {isEditWindowActive && (
401 |                   <Badge className="bg-cyan-100 text-cyan-800 border-cyan-300 font-bold flex items-center gap-1 animate-puls
e">
402 |                     <Clock className="w-3 h-3" /> Edit Window Active ({getRemainingTimeStr()})
403 |                   </Badge>
404 |                 )}
405 |               </div>
406 |             </div>
407 |           </div>
408 |         </motion.div>
409 |
410 |         {/* Active Edit Window Countdown Banner */}
411 |         {isEditWindowActive && (
412 |           <motion.div variants={fadeUp} className="p-4 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text
-white shadow-md flex items-center justify-between gap-3">
413 |             <div className="space-y-0.5">
414 |               <p className="text-xs font-bold text-white flex items-center gap-1.5">
415 |                 <Sparkles className="w-4 h-4 text-cyan-200" /> Admin Authorized Edit Window
416 |               </p>
417 |               <p className="text-[11px] text-cyan-100">
418 |                 Time remaining: <strong>{getRemainingTimeStr()}</strong> (Expires: {new Date(course.edit_window_expires_at!).t
oLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })})
419 |               </p>
420 |             </div>
421 |             <span className="text-[10px] font-bold bg-white/20 text-white px-2.5 py-1 rounded-lg">
422 |               Re-Approval Required on Submit
423 |             </span>
424 |           </motion.div>
425 |         )}
426 |
427 |         {/* 1. Main Course Details Form (Active when canEdit) */}
428 |         {canEdit ? (
429 |           <motion.div variants={fadeUp} className="space-y-4">
430 |             <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
431 |               <CardHeader className="border-b border-slate-100"><CardTitle className="text-slate-900 text-base font-bold">Cour
se Details</CardTitle></CardHeader>
432 |               <CardContent className="p-6 space-y-4">
433 |                 <div className="space-y-1.5">
434 |                   <Label className="text-slate-700 text-xs font-semibold">Title *</Label>
435 |                   <Input {...register('title')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 rou
nded-xl" />
436 |                   {errors.title && <p className="text-xs text-rose-600 font-medium">{errors.title.message}</p>}
437 |                 </div>
438 |                 <div className="space-y-1.5">
439 |                   <Label className="text-slate-700 text-xs font-semibold">Description *</Label>
440 |                   <Textarea {...register('description')} rows={4} className="bg-slate-50 border-slate-200 text-slate-900 focu
s:bg-white rounded-xl resize-none" />
441 |                   {errors.description && <p className="text-xs text-rose-600 font-medium">{errors.description.message}</p>}
442 |                 </div>
443 |                 <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
444 |                   <div className="space-y-1.5">
445 |                     <Label className="text-slate-700 text-xs font-semibold">Type</Label>
446 |                     <Select value={watch('course_type')} onValueChange={(v) => setValue('course_type', v as 'standard' | 'scen
ario')}>
447 |                       <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 h-10 rounded-xl"><SelectValue /></
SelectTrigger>
448 |                       <SelectContent>
449 |                         <SelectItem value="standard">Standard</SelectItem>
450 |                         <SelectItem value="scenario">Scenario</SelectItem>
451 |                       </SelectContent>
452 |                     </Select>
453 |                   </div>
454 |                   <div className="space-y-1.5">
455 |                     <Label className="text-slate-700 text-xs font-semibold">Department</Label>
456 |                     <Input {...register('department')} className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h
-10 rounded-xl" />
```

```
457 |                   </div>
458 |                 </div>
459 |                 <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
460 |                   <div className="space-y-1.5 sm:col-span-2">
461 |                     <Label className="text-slate-700 text-xs font-semibold">Duration (hours)</Label>
462 |                     <Input type="number" {...register('duration_hours')} placeholder="e.g. 20" className="bg-slate-50 border-s
late-200 text-slate-900 focus:bg-white h-10 rounded-xl max-w-sm" />
463 |                     <p className="text-[10px] text-slate-400 font-medium">Leave empty for self-paced</p>
464 |                   </div>
465 |                   <div className="space-y-1.5">
466 |                     <Label className="text-slate-700 text-xs font-semibold">Start Date</Label>
467 |                     <Input type="datetime-local" {...register('start_date')} className="bg-slate-50 border-slate-200 text-slat
e-900 focus:bg-white h-10 rounded-xl" />
468 |                     {errors.start_date && (
469 |                       <p className="text-[10px] text-rose-600 font-medium">{errors.start_date.message as string}</p>
470 |                     )}
471 |                   </div>
472 |                   <div className="space-y-1.5">
473 |                     <Label className="text-slate-700 text-xs font-semibold">End Date</Label>
474 |                     <Input type="datetime-local" {...register('end_date')} className="bg-slate-50 border-slate-200 text-slate-
900 focus:bg-white h-10 rounded-xl" />
475 |                   </div>
476 |                 </div>
477 |                 <div className="space-y-1.5">
478 |                   <Label className="text-slate-700 text-xs font-semibold">Passing Score (%)</Label>
479 |                   <Input type="number" {...register('passing_score')} placeholder="e.g. 60" className="bg-slate-50 border-slat
e-200 text-slate-900 focus:bg-white h-10 rounded-xl max-w-sm" />
480 |                 </div>
481 |                 <div className="space-y-1.5">
482 |                   <Label className="text-slate-700 text-xs font-semibold">Meeting Link (Live online classes)</Label>
483 |                   <Input {...register('meet_link')} placeholder="https://meet.google.com/..." className="bg-slate-50 border-sl
ate-200 text-slate-900 focus:bg-white h-10 rounded-xl" />
484 |                 </div>
485 |               </CardContent>
486 |             </Card>
487 |
488 |             <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl">
489 |               <CardHeader className="border-b border-slate-100"><CardTitle className="text-slate-900 text-base font-bold">Cour
se Thumbnail</CardTitle></CardHeader>
490 |               <CardContent className="p-6">
491 |                 <div className="space-y-3">
492 |                   <input ref={thumbRef} type="file" accept="image/png,image/jpeg,image/webp" className="hidden" onChange={hand
leThumbnailChange} />
493 |                   {thumbnailPreview ? (
494 |                     <div className="relative rounded-xl overflow-hidden border border-slate-200 h-40 bg-slate-100 max-w-md">
495 |                       <img src={thumbnailPreview} alt="Thumbnail preview" className="w-full h-full object-cover" />
496 |                       <button type="button" onClick={() => { setThumbnail(null); setThumbnailPreview(null) }} className="absol
ute top-2 right-2 p-1.5 rounded-full bg-slate-900/80 text-white hover:bg-rose-600 transition-colors">
497 |                         <Trash2 className="w-4 h-4" />
498 |                       </button>
499 |                     </div>
500 |                   ) : (
501 |                     <button type="button" onClick={() => thumbRef.current?.click()} className="w-full max-w-md h-32 border-2 b
order-dashed border-slate-200 hover:border-cyan-500 rounded-xl flex flex-col items-center justify-center gap-1.5 text-slate-400 hove
r:text-cyan-600 bg-slate-50/50 transition-colors">
502 |                       <Upload className="w-6 h-6" />
503 |                       <span className="text-xs font-semibold">Click to upload thumbnail</span>
504 |                       <span className="text-[10px] text-slate-400">PNG, JPG up to 5MB (16:5 ratio, e.g. 1600x500px)</span>
505 |                     </button>
506 |                   )}
507 |                 </div>
508 |               </CardContent>
509 |             </Card>
510 |
511 |             <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl mt-4">
512 |               <CardHeader className="border-b border-slate-100"><CardTitle className="text-slate-900 text-base font-bold">Cour
se Outline</CardTitle></CardHeader>
513 |               <CardContent className="p-6 space-y-4">
514 |                 <div className="space-y-1.5">
515 |                   <Label className="text-slate-700 text-xs font-semibold">Course Outline Details</Label>
516 |                   <Textarea {...register('session_flow_text')} rows={6} className="bg-slate-50 border-slate-200 text-slate-900
focus:bg-white rounded-xl resize-none" placeholder="Describe the course outline, modules/topics covered, and learning plan..." />
517 |                 </div>
518 |
519 |                 <div className="space-y-1.5">
520 |                   <Label className="text-slate-700 text-xs font-semibold">Course Outline Document</Label>
521 |                   {sessionFlowDocPath || sessionFlowDoc ? (
522 |                     <div className="flex items-center justify-between p-3 bg-slate-50 border border-slate-200 rounded-xl max-w
-sm">
```

```
                      <div className="flex items-center gap-2 text-sm text-slate-900 font-medium truncate">
                        <FileText className="w-4 h-4 shrink-0 text-cyan-600" />
                        <span className="truncate">{sessionFlowDoc ? sessionFlowDoc.name : sessionFlowDocPath?.split('/').pop
()}</span>
                      </div>
                      <button type="button" onClick={() => { setSessionFlowDoc(null); setSessionFlowDocPath(null) }} className
="p-1.5 rounded-full hover:bg-rose-50 text-slate-400 hover:text-rose-600 transition-colors">
                        <Trash2 className="w-4 h-4" />
                      </button>
                    </div>
                  ) : (
                    <div className="flex items-center gap-2">
                      <Input
                        type="file"
                        accept=".pdf,.doc,.docx,.txt"
                        className="bg-slate-50 border-slate-200 text-slate-900 cursor-pointer max-w-sm rounded-xl h-10"
                        onChange={(e) => {
                          const file = e.target.files?.[0]
                          if (file) {
                            if (file.size > 20 * 1024 * 1024) {
                              toast.error('File size must be less than 20MB')
                              return
                            }
                            setSessionFlowDoc(file)
                          }
                        }}
                      />
                    </div>
                  )}
                </div>
              </CardContent>
            </Card>

            <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl mt-4">
              <CardHeader className="border-b border-slate-100"><CardTitle className="text-slate-900 text-base font-bold">Outc
omes of Learning (Skills Developed)</CardTitle></CardHeader>
              <CardContent className="p-6 space-y-3">
                <p className="text-xs text-slate-500">Select the skills this course builds or add custom skills.</p>
                <div className="flex gap-2">
                  <Input
                    value={customSkillName}
                    onChange={e => setCustomSkillName(e.target.value)}
                    onKeyDown={e => { if (e.key === 'Enter') { e.preventDefault(); addCustomSkill() } }}
                    placeholder="Add a custom skill..."
                    className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 text-xs rounded-xl"
                    disabled={addingSkill}
                  />
                  <Button
                    type="button"
                    variant="outline"
                    onClick={addCustomSkill}
                    disabled={!customSkillName.trim() || addingSkill}
                    className="border-slate-200 text-slate-700 hover:bg-slate-50 h-10 px-3.5 shrink-0 rounded-xl font-semibol
d"
                  >
                    {addingSkill ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Plus className="w-3.5 h-3.5" />}
                  </Button>
                </div>
                <div className="flex flex-wrap gap-2 pt-1">
                  {skills.map(skill => (
                    <button key={skill.id} onClick={() => toggleSkill(skill.id)}
                      className={`px-3 py-1.5 rounded-full text-xs font-semibold border transition-all ${
                        selectedSkills.includes(skill.id)
                          ? 'bg-cyan-600 text-white border-cyan-600 shadow-xs'
                          : 'bg-slate-50 text-slate-700 border-slate-200 hover:bg-slate-100'
                      }`}>
                      {skill.name}
                    </button>
                  ))}
                </div>
              </CardContent>
            </Card>

            <CourseCertificateStep
              hasCertificate={hasCertificate}
              setHasCertificate={setHasCertificate}
              templateUrl={certificateTemplateUrl}
              setTemplateUrl={setCertificateTemplateUrl}
              templateName={certificateTemplateName}
```

```
              setTemplateName={setCertificateTemplateName}
              courseTitle={watch('title') || course.title}
            />
          </motion.div>
        ) : (
          /* 2. Published & Locked State -> Edit Permission Request Workflow */

          <motion.div variants={fadeUp} className="space-y-4">
            <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl overflow-hidden">
              <CardContent className="p-6 md:p-8 space-y-5">
                <div className="flex items-start gap-4">
                  <div className="w-12 h-12 rounded-2xl bg-amber-50 border border-amber-200 flex items-center justify-center s
hrink-0">
                    <Lock className="w-6 h-6 text-amber-600" />
                  </div>
                  <div>
                    <h3 className="text-lg font-bold text-slate-900">
                      This course is <span className="text-emerald-700 font-extrabold capitalize">{course.status}</span> and l
ocked from direct edits.
                    </h3>
                    <p className="text-xs text-slate-600 mt-1 leading-relaxed">
                      To preserve ongoing enrollments and published curriculum integrity, updates to syllabus, title, or sched
ule require permission from the Administrator. The Admin will assign an active editing time limit.
                    </p>
                  </div>
                </div>

                {course.edit_request_status === 'pending' ? (
                  <div className="p-4 rounded-2xl bg-amber-50/80 border border-amber-200 text-xs text-amber-950 space-y-2">
                    <p className="font-bold flex items-center gap-1.5 text-amber-900">
                      <Clock className="w-4 h-4 text-amber-600" /> Edit Permission Request is Pending Admin Review
                    </p>
                    <p className="text-slate-700">
                      <strong>Your Planned Changes:</strong> "{course.edit_request_reason}"
                    </p>
                    <p className="text-[11px] text-slate-500">
                      Submitted on: {course.edit_request_at ? new Date(course.edit_request_at).toLocaleString() : 'Recently'}
                    </p>
                  </div>
                ) : (
                  <form onSubmit={handleRequestEditPermission} className="space-y-4 pt-2 border-t border-slate-100">
                    <div className="space-y-1.5">
                      <Label className="text-xs font-bold text-slate-800">
                        Reason / Description for Requested Edits *
                      </Label>
                      <Textarea
                        rows={4}
                        value={editReason}
                        onChange={e => setEditReason(e.target.value)}
                        placeholder="Explain the changes you plan to make (e.g., Update module 2 quiz questions, adjust durati
on hours, revise lecture notes)..."
                        className="bg-slate-50 border-slate-300 text-slate-900 placeholder:text-slate-400 rounded-xl text-xs f
ont-medium resize-none"
                        required
                      />
                      <p className="text-[11px] text-slate-400">
                        Minimum 10 characters required.
                      </p>
                    </div>

                    <div className="flex items-center justify-between pt-1">
                      <p className="text-xs text-slate-500 font-medium">
                        Admin will assign an active countdown window (e.g. 24 hours).
                      </p>
                      <Button
                        type="submit"
                        disabled={submittingEditRequest || editReason.trim().length < 10}
                        className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white
font-bold text-xs rounded-xl shadow-sm gap-1.5"
                      >
                        {submittingEditRequest ? <Loader2 className="w-4 h-4 animate-spin" /> : <Send className="w-3.5 h-3.5"
/>}
                        Submit Edit Request to Admin
                      </Button>
                    </div>
                  </form>
                )}
              </CardContent>
            </Card>
```

```
669 |           </motion.div>
670 |         )}
671 |
672 |         {/* Action Buttons when editing is active */}
673 |         {canEdit && (
674 |           <motion.div variants={fadeUp} className="flex items-center justify-between pt-2">
675 |             <Button variant="outline" onClick={() => navigate(`/trainer/courses/${courseId}`)} className="border-slate-200 tex
t-slate-700 hover:bg-slate-50 font-semibold rounded-xl text-xs">
676 |               <ArrowLeft className="w-4 h-4 mr-1.5" /> Cancel
677 |             </Button>
678 |             <div className="flex items-center gap-3">
679 |               <Button variant="outline" onClick={() => handleSave('draft')} disabled={saving} className="border-slate-200 text
-slate-700 hover:bg-slate-50 font-semibold rounded-xl text-xs">
680 |                 {saving ? <Loader2 className="w-4 h-4 mr-1.5 animate-spin" /> : <Save className="w-4 h-4 mr-1.5" />}
681 |                 Save Draft
682 |               </Button>
683 |               <Button onClick={() => handleSave('pending_review')} disabled={saving} className="bg-gradient-to-r from-cyan-600
via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold shadow-md shadow-cyan-600/20 rounded-xl text-xs">
684 |                 {saving ? <Loader2 className="w-4 h-4 mr-1.5 animate-spin" /> : <Send className="w-4 h-4 mr-1.5" />}
685 |                 {isEditWindowActive ? 'Submit Updates for Admin Re-Approval' : 'Submit for Review'}
686 |               </Button>
687 |             </div>
688 |           </motion.div>
689 |         )}
690 |
691 |         {/* 3. ALWAYS-ACTIVE COURSE MATERIALS (Fully functional without admin permission) */}
692 |         <motion.div variants={fadeUp}>
693 |           <Card className="bg-white border border-slate-200/90 shadow-sm rounded-2xl">
694 |             <CardContent className="p-6 space-y-4">
695 |               <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-2 pb-2 border-b border-slate-100">
696 |                 <div>
697 |                   <h3 className="text-sm font-bold text-slate-900">Course Materials & Resources</h3>
698 |                   <p className="text-xs text-slate-500">
699 |                     Upload documents, lecture slides, or video links. Always active and editable without admin lock.
700 |                   </p>
701 |                 </div>
702 |                 <span className="text-[11px] font-bold text-emerald-700 bg-emerald-50 border border-emerald-200 px-2.5 py-0.5
rounded-full shrink-0">
703 |                   ✓ Always Uploadable
704 |                 </span>
705 |               </div>
706 |               <CourseMaterials embedded onMaterialCountChange={setMaterialCount} />
707 |             </CardContent>
708 |           </Card>
709 |         </motion.div>
710 |       </motion.div>
711 |     </TrainerLayout>
712 |   )
713 | }
714 |
```

src/features/trainer/CourseListPage.tsx TSX 19.8 KB 409 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent } from '@/components/ui/card'
import { Plus, Loader2, Edit3, Target, BarChart3, BookOpen, Info, Layers, FileText, Eye } from 'lucide-react'
import { Link, useNavigate } from 'react-router-dom'
import { toast } from 'sonner'
import { Thumbnail } from '@/components/ui/Thumbnail'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog'

type Course = Database['public']['Tables']['courses']['Row']
type Material = Database['public']['Tables']['materials']['Row']

function getFormattedObjectives(raw: any): string {
  if (!raw) return ''
  if (typeof raw === 'string') return raw
  if (Array.isArray(raw)) return raw.join('\n\n')
  if (typeof raw === 'object') {
    const parts: string[] = []
    if (raw.description) parts.push(raw.description)
    if (raw.understand) parts.push(`What you'll understand: ${raw.understand}`)
    if (raw.able_to_do) parts.push(`What you'll be able to do: ${raw.able_to_do}`)
    return parts.length > 0 ? parts.join('\n\n') : JSON.stringify(raw)
  }
  return String(raw)
}

const statusColors: Record<string, string> = {
  draft: 'bg-slate-100 text-slate-700 border border-slate-200',
  pending_review: 'bg-amber-50 text-amber-700 border border-amber-200',
  published: 'bg-emerald-50 text-emerald-700 border border-emerald-200',
  archived: 'bg-rose-50 text-rose-700 border border-rose-200',
}

const statusLabels: Record<string, string> = {
  draft: 'Draft',
  pending_review: 'Pending Review',
  published: 'Published',
  archived: 'Archived',
}

export function CourseListPage() {
  const { user } = useAuth()
  const navigate = useNavigate()
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState(true)
  const [selectedCourse, setSelectedCourse] = useState<Course | null>(null)
  const [aboutModalOpen, setAboutModalOpen] = useState(false)
  const [aboutActiveTab, setAboutActiveTab] = useState<'about' | 'outline'>('about')
  const [docLoading, setDocLoading] = useState(false)
  const [previewMaterial, setPreviewMaterial] = useState<Material | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)

  const fetchCourses = useCallback(async () => {
    if (!user) return
    setLoading(true)
    try {
      const { data, error } = await supabase
        .from('courses')
        .select('*')
        .eq('trainer_id', user.id)
        .neq('status', 'archived')
        .order('created_at', { ascending: false })
      if (error) throw error
      setCourses(data || [])
    } catch (err) {
      toast.error('Failed to load courses')
    } finally {
      setLoading(false)
    }
```

```
  }, [user])

  useEffect(() => { fetchCourses() }, [fetchCourses])

  const handleOpenSessionDoc = async (docPath: string) => {
    if (!docPath) return
    setDocLoading(true)
    try {
      const { data, error } = await supabase.storage
        .from('materials')
        .createSignedUrl(docPath, 300)
      if (error) throw error
      if (data?.signedUrl) {
        setPreviewMaterial({
          id: 'syllabus-doc',
          file_name: 'Course Syllabus Document',
          storage_path: docPath,
          material_type: 'document',
          course_id: selectedCourse?.id || '',
          created_at: new Date().toISOString(),
          description: 'Uploaded Course Syllabus Document',
          duration_minutes: null,
          external_url: null,
          order_index: 0,
          thumbnail_url: null,
          title: 'Course Outline & Syllabus',
          topic_name: 'Outline',
          uploader_id: selectedCourse?.trainer_id || ''
        } as any)
        setPreviewUrl(data.signedUrl)
      }
    } catch {
      toast.error('Could not open syllabus document.')
    } finally {
      setDocLoading(false)
    }
  }

  const handleDownload = async (mat: Material) => {
    if (!mat.storage_path) return
    try {
      const { data, error } = await supabase.storage
        .from('materials')
        .download(mat.storage_path)
      if (error) throw error
      const url = URL.createObjectURL(data)
      const a = document.createElement('a')
      a.href = url
      a.download = mat.file_name || 'download'
      document.body.appendChild(a)
      a.click()
      document.body.removeChild(a)
      URL.revokeObjectURL(url)
      toast.success('Downloaded successfully')
    } catch {
      toast.error('Failed to download file')
    }
  }

  return (
    <TrainerLayout>
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-5xl mx-auto space-y-6">
        <motion.div variants={fadeUp} className="flex items-center justify-between">
          <div>
            <h2 className="text-2xl font-bold tracking-tight text-slate-900">My Courses</h2>
            <p className="text-slate-500 text-sm mt-1">{courses.length} course{courses.length !== 1 ? 's' : ''} total</p>
          </div>
          <Link to="/trainer/courses/new">
            <Button className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 te
xt-white font-bold shadow-md shadow-cyan-600/20">
              <Plus className="w-4 h-4 mr-2" /> New Course
            </Button>
          </Link>
        </motion.div>

        {loading ? (
          <div className="flex justify-center py-20">
            <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
          </div>
```

```
        ) : courses.length === 0 ? (
          <motion.div variants={fadeUp}>
            <Card className="bg-white border border-slate-200/90 shadow-xs">
              <CardContent className="flex flex-col items-center justify-center py-16 text-center">
                <p className="text-slate-500 mb-4 font-medium">No courses created yet.</p>
                <Link to="/trainer/courses/new">
                  <Button className="bg-gradient-to-r from-cyan-600 to-blue-600 text-white font-bold">
                    <Plus className="w-4 h-4 mr-2" /> Create Your First Course
                  </Button>
                </Link>
              </CardContent>
            </Card>
          </motion.div>
        ) : (
          <motion.div variants={fadeUp} className="space-y-3.5">
            {courses.map(course => (
              <div key={course.id} className="p-5 rounded-2xl bg-white border border-slate-200/90 hover:border-slate-300 shadow-xs transition-all">
                <div className="flex flex-col sm:flex-row sm:items-start gap-4">
                  {/* Thumbnail */}
                  <div className="w-full sm:w-36 h-24 rounded-xl bg-slate-100 border border-slate-200 overflow-hidden shrink-0 flex items-center justify-center">
                    <Thumbnail path={course.thumbnail_path} alt={course.title} fallbackIcon={<BookOpen className="w-8 h-8 text-slate-400" />} />
                  </div>
                  <div className="flex-1 min-w-0">
                    <div className="flex flex-wrap items-center gap-2.5 mb-1.5">
                      <Link to={`/trainer/courses/${course.id}`} className="hover:text-cyan-600 min-w-0 transition-colors">
                        <h3 className="text-base font-bold text-slate-900 truncate">{course.title}</h3>
                      </Link>
                      <button
                        type="button"
                        onClick={() => {
                          setSelectedCourse(course)
                          setAboutActiveTab('about')
                          setAboutModalOpen(true)
                        }}
                        title="About Course & Outline"
                        className="inline-flex items-center gap-1.5 px-2.5 py-0.5 rounded-full bg-cyan-50 hover:bg-cyan-100 text-cyan-800 border border-cyan-300 text-[11px] font-bold transition-all shadow-xs cursor-pointer group shrink-0"
                      >
                        <Info className="w-3.5 h-3.5 text-cyan-600 group-hover:rotate-12 transition-transform" />
                        <span>About</span>
                      </button>
                      <Badge className={`shrink-0 text-[10px] font-semibold ${statusColors[course.status] ?? statusColors.draft}`}>
                        {statusLabels[course.status] ?? course.status}
                      </Badge>
                    </div>
                    <p className="text-xs text-slate-600 line-clamp-2 mb-3 leading-relaxed">{course.description}</p>
                    <div className="flex items-center gap-4 text-xs font-medium text-slate-400">
                      <span>{course.duration_minutes ?? 0} mins</span>
                      <span>&bull;</span>
                      <span className="capitalize">{course.course_type}</span>
                      <span>&bull;</span>
                      <span>{new Date(course.created_at).toLocaleDateString()}</span>
                      {course.department && (
                        <>
                          <span>&bull;</span>
                          <span>{course.department}</span>
                        </>
                      )}
                    </div>
                  </div>
                  <div className="flex flex-wrap gap-2 shrink-0 sm:self-center">
                    <Button
                      size="sm"
                      variant="outline"
                      className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                      disabled={course.status === 'pending_review'}
                      onClick={() => navigate(`/trainer/courses/${course.id}`)}
                    >
                      <BookOpen className="w-3.5 h-3.5 mr-1 text-cyan-600" /> Manage
                    </Button>
                    <Button
                      size="sm"
                      variant="outline"
                      className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                      disabled={course.status === 'pending_review'}
```

```
                      onClick={() => navigate(`/trainer/courses/${course.id}/edit`)}
                    >
                      <Edit3 className="w-3.5 h-3.5 mr-1 text-slate-600" /> Edit
                    </Button>
                    <Button
                      size="sm"
                      variant="outline"
                      className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                      disabled={course.status === 'pending_review'}
                      onClick={() => navigate(`/trainer/courses/${course.id}/assessments`)}
                    >
                      <Target className="w-3.5 h-3.5 mr-1 text-emerald-600" /> Assess
                    </Button>
                    <Button
                      size="sm"
                      variant="outline"
                      className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                      disabled={course.status === 'pending_review'}
                      onClick={() => navigate(`/trainer/courses/${course.id}/performance`)}
                    >
                      <BarChart3 className="w-3.5 h-3.5 mr-1 text-blue-600" /> Stats
                    </Button>
                  </div>
                </div>
              </div>
            ))}
          </motion.div>
        )}
      </motion.div>

      <MaterialPreviewDialog
        material={previewMaterial}
        previewUrl={previewUrl}
        onClose={() => {
          setPreviewMaterial(null)
          setPreviewUrl(null)
        }}
        onDownload={() => previewMaterial && handleDownload(previewMaterial)}
      />

      {/* About Course & Course Outline Modal */}
      <Dialog open={aboutModalOpen} onOpenChange={setAboutModalOpen}>
        <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto rounded-3xl p-6 md:p-8 bg-white border border-slate-200 shadow-2xl">
          <DialogHeader className="pb-4 border-b border-slate-100">
            <div className="flex flex-wrap items-center justify-between gap-3">
              <div className="space-y-1 pr-6">
                <div className="flex items-center gap-2">
                  <Badge variant="outline" className="text-[10px] font-bold uppercase text-cyan-800 bg-cyan-50 border-cyan-200">
                    {selectedCourse?.course_type || 'Standard'} Program
                  </Badge>
                  {selectedCourse?.department && (
                    <Badge variant="outline" className="text-[10px] font-semibold text-slate-600 border-slate-200">
                      {selectedCourse.department}
                    </Badge>
                  )}
                </div>
                <DialogTitle className="text-xl md:text-2xl font-black text-slate-900 leading-tight">
                  {selectedCourse?.title}
                </DialogTitle>
                <p className="text-xs text-slate-500 font-medium">
                  {selectedCourse?.duration_minutes ? `${Math.floor(selectedCourse.duration_minutes / 60)} Hours` : 'Self-Paced'} &bull; Pass Gate: {selectedCourse?.passing_score || 80}%
                </p>
              </div>
            </div>

            {/* Navigation Tabs */}
            <div className="flex items-center gap-2 pt-4">
              <button
                type="button"
                onClick={() => setAboutActiveTab('about')}
                className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
                  aboutActiveTab === 'about'
                    ? 'bg-cyan-600 text-white shadow-xs'
                    : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                }`}
              >
```

```
                  <BookOpen className="w-3.5 h-3.5" />
                  About This Course
                </button>
                <button
                  type="button"
                  onClick={() => setAboutActiveTab('outline')}
                  className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
                    aboutActiveTab === 'outline'
                      ? 'bg-cyan-600 text-white shadow-xs'
                      : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                  }`}
                >
                  <Layers className="w-3.5 h-3.5" />
                  Course Outline & Roadmap
                </button>
              </div>
            </DialogHeader>

            {/* TAB 1: ABOUT THIS COURSE */}
            {aboutActiveTab === 'about' && (
              <div className="space-y-6 pt-2">
                {/* Description */}
                <div className="space-y-2">
                  <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                    <BookOpen className="w-3.5 h-3.5 text-cyan-600" /> Course Overview
                  </h4>
                  <p className="text-sm text-slate-700 leading-relaxed bg-slate-50 p-4 rounded-2xl border border-slate-200/80 wh
itespace-pre-line">
                    {selectedCourse?.description || 'Comprehensive competency-based training designed for operational excellenc
e.'}
                  </p>
                </div>

                {/* Objectives */}
                {selectedCourse?.learning_objectives && (
                  <div className="space-y-2">
                    <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                      <Target className="w-3.5 h-3.5 text-cyan-600" /> Key Learning Objectives
                    </h4>
                    <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200/80 space-y-2 text-xs sm:text-sm text-sla
te-700 leading-relaxed whitespace-pre-line font-normal">
                      {getFormattedObjectives(selectedCourse.learning_objectives)}
                    </div>
                  </div>
                )}

                {/* Key Course Specifications Grid */}
                <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
                  <div className="p-3.5 rounded-2xl bg-cyan-50/50 border border-cyan-200/60">
                    <span className="text-[10px] font-bold text-cyan-800 uppercase tracking-wider block">Passing Threshold</span
>
                    <span className="text-sm font-extrabold text-cyan-950 mt-0.5 block">{selectedCourse?.passing_score || 80}% O
verall</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                    <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Duration</span>
                    <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{selectedCourse?.duration_minutes ? `
${Math.floor(selectedCourse.duration_minutes / 60)} Hours` : 'Flexible'}</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                    <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Max Trainees</span>
                    <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{selectedCourse?.max_trainees || 'Unlim
ited'} Seats</span>
                  </div>
                </div>
              </div>
            )}

            {/* TAB 2: COURSE OUTLINE & ROADMAP */}
            {aboutActiveTab === 'outline' && (
              <div className="space-y-6 pt-2">
                {/* Syllabus Document Download / Preview */}
                {selectedCourse?.session_flow_document_path && (
                  <div className="p-4 rounded-2xl bg-cyan-50/70 border border-cyan-200 flex items-center justify-between gap-3">
                    <div className="flex items-center gap-2.5 min-w-0">
                      <FileText className="w-5 h-5 text-cyan-700 shrink-0" />
                      <div>
                        <h5 className="text-xs font-bold text-cyan-950">Official Syllabus Document</h5>
                        <p className="text-[11px] text-cyan-700">Detailed curriculum plan and reading materials</p>
```

```
                    </div>
                  </div>
                  <Button
                    size="sm"
                    onClick={() => handleOpenSessionDoc(selectedCourse.session_flow_document_path!)}
                    disabled={docLoading}
                    className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs h-8 rounded-xl shrink-0"
                  >
                    {docLoading ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Eye className="w-3.5 h-3.5 mr-1" /
>}
                    View Document
                  </Button>
                </div>
              )}

              {/* Session Schedule & Timeline */}
              {selectedCourse?.session_flow_text && (
                <div className="space-y-3">
                  <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                    <Layers className="w-3.5 h-3.5 text-cyan-600" /> Session Schedule & Timeline
                  </h4>
                  <div className="bg-slate-50 border border-slate-200/80 rounded-2xl p-5 text-xs sm:text-sm text-slate-700 lea
ding-relaxed whitespace-pre-wrap font-normal">
                    {selectedCourse.session_flow_text}
                  </div>
                </div>
              )}
            </div>
          )}
        </DialogContent>
      </Dialog>
    </TrainerLayout>
  )
}

```

src/features/trainer/CourseSessionsPage.tsx TSX 24.6 KB 508 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useParams, Link } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
} from '@/components/ui/dialog'
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue
} from '@/components/ui/select'
import {
  ArrowLeft, Plus, Trash2, Loader2, Calendar, Video, FileText, Save, Globe, Clock, UserCheck
} from 'lucide-react'
import { toast } from 'sonner'
import { format } from 'date-fns'
import { useConfirm } from '@/hooks/useConfirm'
import { LiveAttendanceTrainerPanel } from '../courses/LiveAttendanceTrainerPanel'
import { OverallAttendanceDialog } from './OverallAttendanceDialog'
type Course = Database['public']['Tables']['courses']['Row']
type Session = Database['public']['Tables']['course_sessions']['Row']
type Material = Database['public']['Tables']['materials']['Row']

export function CourseSessionsPage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user, profile } = useAuth()
  const isTrainer = profile?.role === 'trainer'
  const baseCoursePath = isTrainer ? '/trainer/courses' : '/admin/courses'
  const [course, setCourse] = useState<Course | null>(null)
  const [sessions, setSessions] = useState<Session[]>([])
  const [materials, setMaterials] = useState<Material[]>([])
  const [loading, setLoading] = useState(true)
  const [saving, setSaving] = useState(false)
  const [dialogOpen, setDialogOpen] = useState(false)
  const [editingSession, setEditingSession] = useState<Partial<Session> | null>(null)
  const [sessionDate, setSessionDate] = useState('')
  const [sessionStartTime, setSessionStartTime] = useState('')
  const [sessionEndTime, setSessionEndTime] = useState('')
  const [ConfirmDialog, confirm] = useConfirm()

  // Attendance Modal state

  const [allEnrollments, setAllEnrollments] = useState<any[]>([])
  const [attendanceDialogOpen, setAttendanceDialogOpen] = useState(false)
  const [overallAttendanceOpen, setOverallAttendanceOpen] = useState(false)
  const [attendanceSession, setAttendanceSession] = useState<Session | null>(null)
  const [attendanceRecords, setAttendanceRecords] = useState<Record<string, 'present' | 'absent' | 'late'>>({})

  const isSessionFinished = (session: Session) => {
    if (!session.end_time && !session.start_time) return false
    const sessionEndTime = session.end_time ? new Date(session.end_time) : new Date(session.start_time!)
    return sessionEndTime < new Date()
  }

  const activeSessions = sessions.filter(s => !isSessionFinished(s))
  const pastSessions = sessions.filter(s => isSessionFinished(s))
  const activeEnrollments = allEnrollments.filter(e => ['enrolled', 'in_progress', 'completed'].includes(e.status))

  const handleOpenAttendance = (session: Session) => {
    setAttendanceSession(session)
    const initial: Record<string, 'present' | 'absent' | 'late'> = {}
    activeEnrollments.forEach(e => {
      initial[e.user_id] = 'present'
    })
    setAttendanceRecords(initial)
    setAttendanceDialogOpen(true)
  }

```

```
  const handleSaveAttendance = () => {
    toast.success(`Attendance saved for "${attendanceSession?.title || 'Session'}"!`)
    setAttendanceDialogOpen(false)
  }

  const fetchData = useCallback(async () => {
    if (!user || !courseId) return
    setLoading(true)
    try {
      let query = supabase.from('courses').select('*').eq('id', courseId)
      if (profile?.role === 'trainer') {
        query = query.eq('trainer_id', user.id)
      }
      const { data: c } = await query.single()
      if (c) setCourse(c)

      const { data: s } = await supabase.from('course_sessions').select('*').eq('course_id', courseId).order('order_index')
      if (s) setSessions(s)

      const { data: m } = await supabase.from('materials').select('*').eq('course_id', courseId)
      if (m) setMaterials(m)

      const { data: enrollmentsData } = await supabase.from('enrollments').select('*').eq('course_id', courseId)
      if (enrollmentsData && enrollmentsData.length > 0) {
        const userIds = enrollmentsData.map(e => e.user_id)
        const { data: traineesData } = await supabase.from('trainees').select('id, full_name, email').in('id', userIds)
        setAllEnrollments(enrollmentsData.map(e => ({
          ...e,
          trainee: traineesData?.find(t => t.id === e.user_id) || { full_name: 'Unknown Trainee', email: '' }
        })))
      }
    } catch (err) {
      console.error(err)
    } finally {
      setLoading(false)
    }
  }, [user, profile?.role, courseId])

  useEffect(() => { fetchData() }, [fetchData])

  const handleSaveSession = async () => {
    if (!editingSession?.title?.trim()) {
      toast.error('Title is required')
      return
    }
    setSaving(true)
    try {
      let finalStartTime: string | null = null
      let finalEndTime: string | null = null

      if (sessionDate && sessionStartTime) {
        const d = new Date(`${sessionDate}T${sessionStartTime}:00`)
        if (!isNaN(d.getTime())) finalStartTime = d.toISOString()
      } else if (sessionDate) {
        const d = new Date(`${sessionDate}T00:00:00`)
        if (!isNaN(d.getTime())) finalStartTime = d.toISOString()
      }

      if (sessionDate && sessionEndTime) {
        const d = new Date(`${sessionDate}T${sessionEndTime}:00`)
        if (!isNaN(d.getTime())) finalEndTime = d.toISOString()
      } else if (sessionDate && sessionStartTime) {
        // default end time to 1 hour after start if not set

        const d = new Date(`${sessionDate}T${sessionStartTime}:00`)
        if (!isNaN(d.getTime())) {
          d.setHours(d.getHours() + 1)
          finalEndTime = d.toISOString()
        }
      }

      if (editingSession.id) {
        const { error } = await supabase.from('course_sessions').update({
          title: editingSession.title.trim(),
          description: editingSession.description?.trim() || null,
          start_time: finalStartTime,
          end_time: finalEndTime,
          meet_link: editingSession.meet_link?.trim() || null,
          session_type: editingSession.session_type || 'live'
```

```
        }).eq('id', editingSession.id)
        if (error) throw error
        toast.success('Session updated successfully')
      } else {
        const { error } = await supabase.from('course_sessions').insert({
          course_id: courseId!,
          title: editingSession.title.trim(),
          description: editingSession.description?.trim() || null,
          start_time: finalStartTime,
          end_time: finalEndTime,
          meet_link: editingSession.meet_link?.trim() || null,
          order_index: sessions.length,
          session_type: editingSession.session_type || 'live'
        })
        if (error) throw error
        toast.success('Session created successfully')
      }

      // Direct notification dispatch for enrolled trainees

      try {
        const { data: enrollments } = await supabase
          .from('enrollments')
          .select('user_id')
          .eq('course_id', courseId!)
          .in('status', ['enrolled', 'in_progress', 'completed'])

        if (enrollments && enrollments.length > 0) {
          const sessionTitle = editingSession.title.trim()
          const timeText = finalStartTime ? ` on ${new Date(finalStartTime).toLocaleDateString()} at ${new Date(finalStartTim
e).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}` : ''
          const isEdit = !!editingSession.id

          const notifs = enrollments
            .filter(e => e.user_id !== user?.id)
            .map(e => ({
              user_id: e.user_id,
              type: `course_session:${courseId}`,
              title: isEdit ? `🗓️ Session Updated: ${sessionTitle}` : `🗓️ New Session: ${sessionTitle}`,
              message: `A session has been scheduled in "${course?.title || 'your course'}"${timeText}.`
            }))

          if (notifs.length > 0) {
            await supabase.from('notifications').insert(notifs)
          }
        }
      } catch (nErr) {
        console.warn('Direct notification insertion handled or skipped:', nErr)
      }

      if (editingSession.meet_link?.trim() && courseId) {
        supabase.from('courses').update({ meet_link: editingSession.meet_link.trim() }).eq('id', courseId).then(() => {}, (er
r) => console.warn(err))
      }

      setDialogOpen(false)
      fetchData()
    } catch (err: any) {
      console.error('Failed to save session:', err)
      toast.error(err?.message || 'Failed to save session')
    } finally {
      setSaving(false)
    }
  }

  const handleDelete = async (id: string) => {
    const isConfirmed = await confirm('Are you sure you want to delete this session?', 'Delete Session')
    if (!isConfirmed) return
    try {
      await supabase.from('course_sessions').delete().eq('id', id)
      toast.success('Session deleted')
      fetchData()
    } catch (err) {
      toast.error('Failed to delete session')
    }
  }

  const openNew = () => {
    const today = format(new Date(), 'yyyy-MM-dd')
```

```
    setEditingSession({
      title: '',
      description: '',
      start_time: '',
      end_time: '',
      meet_link: course?.meet_link || '',
      location: '',
      session_type: 'live'
    })
    setSessionDate(today)
    setSessionStartTime('10:00')
    setSessionEndTime('11:00')
    setDialogOpen(true)
  }

  const openEdit = (session: Session) => {
    let dateStr = format(new Date(), 'yyyy-MM-dd')
    let startStr = '10:00'
    let endStr = '11:00'

    if (session.start_time) {
      const d = new Date(session.start_time)
      if (!isNaN(d.getTime())) {
        dateStr = format(d, 'yyyy-MM-dd')
        startStr = format(d, 'HH:mm')
      }
    }
    if (session.end_time) {
      const d = new Date(session.end_time)
      if (!isNaN(d.getTime())) {
        endStr = format(d, 'HH:mm')
      }
    }

    setEditingSession({ ...session })
    setSessionDate(dateStr)
    setSessionStartTime(startStr)
    setSessionEndTime(endStr)
    setDialogOpen(true)
  }

  const renderSessionCard = (session: Session, index: number, isFinished: boolean) => {
    const sessionMaterials = materials.filter(m => m.session_id === session.id)
    return (
      <motion.div key={session.id} variants={fadeUp}>
        <Card className={`bg-white border ${isFinished ? 'border-slate-200/70 bg-slate-50/40 opacity-95' : 'border-slate-200/9
0'} shadow-sm hover:shadow-md hover:border-slate-300 transition-all overflow-hidden rounded-3xl group`}>
          <CardHeader className="bg-slate-50/80 border-b border-slate-100 pb-4">
            <div className="flex items-start justify-between gap-4">
              <div>
                <div className="flex items-center gap-2 mb-1 flex-wrap">
                  <Badge variant="outline" className="text-[10px] bg-white border-slate-200 text-slate-700 font-semibold">
                    Session {index + 1}
                  </Badge>
                  {isFinished && (
                    <Badge className="bg-emerald-50 text-emerald-700 border border-emerald-200 text-[10px] font-bold">
                      Completed
                    </Badge>
                  )}
                  <CardTitle className="text-lg font-bold text-slate-900">{session.title}</CardTitle>
                </div>
                {session.description && <p className="text-sm text-slate-600 mt-2 font-medium">{session.description}</p>}

                <div className="flex flex-wrap gap-4 mt-3 text-xs text-slate-500 font-medium">
                  {session.session_type && (
                    <Badge className={`text-[10px] capitalize font-semibold ${session.session_type === 'live' || session.sessi
on_type === 'hybrid' ? 'bg-orange-50 text-orange-700 border-orange-200 hover:bg-orange-100' : 'bg-blue-50 text-blue-700 border-blue-
200'}`}>
                      {session.session_type.replace('_', ' ')}
                    </Badge>
                  )}
                  {session.start_time && (
                    <span className="flex items-center gap-1.5">
                      <Calendar className="w-3.5 h-3.5 text-slate-400" />
                      {format(new Date(session.start_time), 'MMM d, yyyy h:mm a')}
                      {session.end_time && ` - ${format(new Date(session.end_time), 'h:mm a')}`}
                    </span>
                  )}
                  {session.meet_link && !isFinished && (
```

```
                    <a href={session.meet_link} target="_blank" rel="noreferrer" className="flex items-center gap-1.5 text-cya
n-600 hover:text-cyan-700 font-semibold hover:underline">
                      <Video className="w-3.5 h-3.5" /> Join Live Class
                    </a>
                  )}
                  {session.location && (
                    <span className="flex items-center gap-1.5 text-slate-600">
                      <Globe className="w-3.5 h-3.5 text-slate-400" /> {session.location}
                    </span>
                  )}
                </div>
              </div>

              <div className="flex items-center gap-2 shrink-0">
                <Button size="sm" variant="outline" onClick={() => handleOpenAttendance(session)} className="border-cyan-200 t
ext-cyan-700 bg-cyan-50 hover:bg-cyan-100 font-semibold text-xs rounded-xl h-8">
                  <UserCheck className="w-3.5 h-3.5 mr-1" /> Attendance
                </Button>
                <Button variant="ghost" size="sm" className="text-slate-600 hover:text-slate-900" onClick={() => openEdit(sess
ion)}>Edit</Button>
                <Button variant="ghost" size="sm" className="text-red-500 hover:text-red-600 hover:bg-red-50" onClick={() => h
andleDelete(session.id)}>
                  <Trash2 className="w-4 h-4" />
                </Button>
              </div>
            </div>
          </CardHeader>
          <CardContent className="p-5 bg-white">
            <div className="flex items-center justify-between mb-3">
              <h4 className="text-sm font-bold text-slate-800">Session Materials</h4>
              <Link to={`${baseCoursePath}/${courseId}/materials?session=${session.id}`}>
                <Button variant="outline" size="sm" className="h-8 text-xs border-slate-200 text-slate-700 hover:bg-slate-50 f
ont-semibold rounded-xl">Manage Materials</Button>
              </Link>
            </div>
            {sessionMaterials.length === 0 ? (
              <p className="text-xs text-slate-400 italic font-medium">No materials assigned to this session.</p>
            ) : (
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
                {sessionMaterials.map(m => (
                  <div key={m.id} className="flex items-center gap-2 p-2.5 rounded-xl bg-slate-50 border border-slate-200/80 t
ext-xs text-slate-700 font-medium">
                    <FileText className="w-3.5 h-3.5 shrink-0 text-cyan-600" />
                    <span className="truncate">{m.file_name}</span>
                  </div>
                ))}
              </div>
            )}
          </CardContent>
        </Card>
      </motion.div>
    )
  }

  if (loading) return <TrainerLayout><div className="flex justify-center py-20"><Loader2 className="w-6 h-6 animate-spin" /></
div></TrainerLayout>

  return (
    <TrainerLayout>
      <ConfirmDialog />
      <OverallAttendanceDialog
        open={overallAttendanceOpen}
        onOpenChange={setOverallAttendanceOpen}
        courseId={courseId!}
        sessions={sessions}
        enrollments={allEnrollments}
      />
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-4xl mx-auto space-y-6">
        <motion.div variants={fadeUp} className="flex items-center justify-between">
          <div>
            <Link to={`${baseCoursePath}/${courseId}`} className="flex items-center gap-2 text-sm text-slate-500 hover:text-cy
an-600 transition-colors mb-4 font-medium">
              <ArrowLeft className="w-4 h-4" /> Back to Course
            </Link>
            <h2 className="text-2xl font-black tracking-tight text-slate-900">Course Sessions</h2>
            <p className="text-slate-500 text-sm mt-1 font-medium">{course?.title}</p>
          </div>
          <div className="flex items-center gap-3">
            <Button onClick={() => setOverallAttendanceOpen(true)} variant="outline" className="border-cyan-200 text-cyan-700
```

```
hover:bg-cyan-50 font-bold rounded-xl shadow-sm">
377 |                 Overall Attendance
378 |               </Button>
379 |               <Button onClick={openNew} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-
white font-bold rounded-xl shadow-md shadow-cyan-600/10">
380 |                 <Plus className="w-4 h-4 mr-2" /> Add Session
381 |               </Button>
382 |             </div>
383 |           </motion.div>
384 |
385 |           {sessions.length === 0 ? (
386 |             <motion.div variants={fadeUp}>
387 |               <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl">
388 |                 <CardContent className="py-12 text-center">
389 |                   <Calendar className="w-12 h-12 text-slate-300 mx-auto mb-4" />
390 |                   <p className="text-slate-600 mb-4 font-medium">No sessions created yet. Group your course into days or module
s.</p>
391 |                   <Button onClick={openNew} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 t
ext-white font-bold rounded-xl shadow-sm">
392 |                     <Plus className="w-4 h-4 mr-2" /> Add First Session
393 |                   </Button>
394 |                 </CardContent>
395 |               </Card>
396 |             </motion.div>
397 |           ) : (
398 |             <div className="space-y-6">
399 |               {/* Active & Upcoming Sessions */}
400 |               <div className="space-y-4">
401 |                 <div className="flex items-center justify-between">
402 |                   <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
403 |                     <Calendar className="w-4 h-4 text-cyan-600" /> Active & Upcoming Sessions
404 |                     <Badge className="bg-cyan-50 text-cyan-700 border border-cyan-200 text-[10px] font-bold">{activeSessions.len
gth}</Badge>
405 |                   </h3>
406 |                 </div>
407 |                 {activeSessions.length > 0 ? (
408 |                   activeSessions.map((session, index) => renderSessionCard(session, index, false))
409 |                 ) : (
410 |                   <Card className="bg-white border border-slate-200/80 rounded-2xl shadow-xs">
411 |                     <CardContent className="py-6 text-center text-xs text-slate-400 italic">
412 |                       No active upcoming sessions. All completed sessions appear in history below.
413 |                     </CardContent>
414 |                   </Card>
415 |                 )}
416 |               </div>
417 |
418 |               {/* Session History (Completed Sessions) */}
419 |               {pastSessions.length > 0 && (
420 |                 <div className="space-y-4 pt-4 border-t border-slate-200">
421 |                   <div className="flex items-center justify-between">
422 |                     <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
423 |                       <Clock className="w-4 h-4 text-slate-500" /> Session History (Completed)
424 |                       <Badge className="bg-slate-100 text-slate-700 border border-slate-200 text-[10px] font-bold">{pastSession
s.length}</Badge>
425 |                     </h3>
426 |                   </div>
427 |                   {pastSessions.map((session, index) => renderSessionCard(session, activeSessions.length + index, true))}
428 |                 </div>
429 |               )}
430 |             </div>
431 |           )}
432 |         </motion.div>
433 |
434 |         {/* Attendance Modal Dialog */}
435 |         <Dialog open={attendanceDialogOpen} onOpenChange={setAttendanceDialogOpen}>
436 |           <DialogContent className="sm:max-w-[700px] bg-transparent border-0 shadow-none p-0 overflow-hidden">
437 |             <LiveAttendanceTrainerPanel session={attendanceSession} enrollments={activeEnrollments} isCompleted={attendanceSessi
on ? isSessionFinished(attendanceSession) : false} />
438 |           </DialogContent>
439 |         </Dialog>
440 |
441 |         <Dialog open={dialogOpen} onOpenChange={setDialogOpen}>
442 |           <DialogContent className="sm:max-w-[440px] bg-white border border-slate-200/90 shadow-2xl rounded-3xl text-slate-900">
443 |             <DialogHeader><DialogTitle className="text-slate-900 font-black text-xl">{editingSession?.id ? 'Edit Session' : 'New
Session'}</DialogTitle></DialogHeader>
444 |             <div className="space-y-4 py-4">
445 |               <div className="space-y-1.5">
446 |                 <Label className="text-slate-700 font-semibold text-xs">Session Title *</Label>
447 |                 <Input className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" value={editingSession?.title || ''} on
```

```
Change={e => setEditingSession({ ...editingSession, title: e.target.value })} placeholder="e.g. Day 1: Basics" />
448 |             </div>
449 |             <div className="space-y-1.5">
450 |               <Label className="text-slate-700 font-semibold text-xs">Description</Label>
451 |               <Textarea className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl" value={editingSession?.description
|| ''} onChange={e => setEditingSession({ ...editingSession, description: e.target.value })} />
452 |             </div>
453 |             <div className="space-y-1.5">
454 |               <Label className="text-slate-700 font-semibold text-xs">Delivery Mode</Label>
455 |               <Select value={editingSession?.session_type || 'live'} onValueChange={v => setEditingSession({ ...editingSessio
n, session_type: v })}>
456 |                 <SelectTrigger className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"><SelectValue placeholder="On
line Live Class" /></SelectTrigger>
457 |                 <SelectContent>
458 |                   <SelectItem value="live">Online Live Class</SelectItem>
459 |                 </SelectContent>
460 |               </Select>
461 |             </div>
462 |             <div className="space-y-1.5">
463 |               <Label className="text-slate-700 font-semibold text-xs">Session Date</Label>
464 |               <Input
465 |                 className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"
466 |                 type="date"
467 |                 value={sessionDate}
468 |                 onChange={e => setSessionDate(e.target.value)}
469 |               />
470 |             </div>
471 |             <div className="grid grid-cols-2 gap-4">
472 |               <div className="space-y-1.5">
473 |                 <Label className="text-slate-700 font-semibold text-xs">Start Time</Label>
474 |                 <Input
475 |                   className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"
476 |                   type="time"
477 |                   value={sessionStartTime}
478 |                   onChange={e => setSessionStartTime(e.target.value)}
479 |                 />
480 |               </div>
481 |               <div className="space-y-1.5">
482 |                 <Label className="text-slate-700 font-semibold text-xs">End Time</Label>
483 |                 <Input
484 |                   className="bg-slate-50 border-slate-200 text-slate-900 rounded-xl"
485 |                   type="time"
486 |                   value={sessionEndTime}
487 |                   onChange={e => setSessionEndTime(e.target.value)}
488 |                 />
489 |               </div>
490 |             </div>
491 |             <div className="space-y-1.5">
492 |               <Label className="text-slate-700 font-semibold text-xs">Live Class Meet Link *</Label>
493 |               <Input className="bg-slate-50 border-slate-200 text-slate-900 placeholder:text-slate-400 rounded-xl" type="url"
value={editingSession?.meet_link || ''} onChange={e => setEditingSession({ ...editingSession, meet_link: e.target.value })} placehol
der="https://meet.google.com/..." />
494 |             </div>
495 |           </div>
496 |           <DialogFooter>
497 |             <Button variant="outline" className="border-slate-200 text-slate-700 hover:bg-slate-50 rounded-xl" onClick={() =>
setDialogOpen(false)}>Cancel</Button>
498 |             <Button onClick={handleSaveSession} disabled={saving} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blu
e-600 hover:opacity-95 text-white font-semibold rounded-xl shadow-sm">
499 |               {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
500 |               Save Session
501 |             </Button>
502 |           </DialogFooter>
503 |         </DialogContent>
504 |       </Dialog>
505 |     </TrainerLayout>
506 |   )
507 | }
508 |
```

src/features/trainer/NotificationsPage.tsx TSX 0.2 KB 7 Lines

```tsx
import React from 'react'
import { UniversalNotificationsPage } from '@/features/notifications/UniversalNotificationsPage'

export function NotificationsPage() {
  return <UniversalNotificationsPage />
}

```

src/features/trainer/OverallAttendanceDialog.tsx TSX 8.1 KB 194 Lines

```tsx
import React, { useState, useEffect, useMemo } from 'react'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription } from '@/components/ui/dialog'
import { supabase } from '@/lib/supabase'
import { Loader2, Download, Check, AlertCircle } from 'lucide-react'
import { Button } from '@/components/ui/button'

export function OverallAttendanceDialog({
  open,
  onOpenChange,
  courseId,
  sessions,
  enrollments
}: {
  open: boolean
  onOpenChange: (o: boolean) => void
  courseId: string
  sessions: any[]
  enrollments: any[]
}) {
  const [loading, setLoading] = useState(false)
  const [attendanceRecords, setAttendanceRecords] = useState<any[]>([])
  const [metaRecords, setMetaRecords] = useState<any[]>([])

  useEffect(() => {
    if (open && sessions.length > 0) {
      loadAttendance()
    }
  }, [open, sessions])

  const loadAttendance = async () => {
    setLoading(true)
    try {
      const sessionIds = sessions.map(s => s.id)
      const [{ data: attData }, { data: metaData }] = await Promise.all([
        (supabase as any).from('session_attendance').select('*').in('session_id', sessionIds),
        (supabase as any).from('session_attendance_meta').select('*').in('session_id', sessionIds)
      ])
      if (attData) setAttendanceRecords(attData)
      if (metaData) setMetaRecords(metaData)
    } catch (err) {
      console.error(err)
    } finally {
      setLoading(false)
    }
  }

  const attendanceData = useMemo(() => {
    if (!sessions.length || !enrollments.length) return []

    return enrollments.map(e => {
      let presentCount = 0
      let totalCount = sessions.length

      const sessionDetails = sessions.map(s => {
        const att = attendanceRecords.find(a => a.session_id === s.id && a.user_id === e.user_id)
        const meta = metaRecords.find(m => m.session_id === s.id)

        let status = 'Absent'
        if (att?.status_override) {
          status = att.status_override === 'present' ? 'Present' : (att.status_override === 'late' ? 'Late' : 'Absent')
        } else if (meta?.total_generated > 0 && att?.entered_count >= meta.total_generated * 0.5) {
          status = 'Present'
        }

        if (status === 'Present' || status === 'Late') {
          presentCount++
        }

        return { sessionId: s.id, status }
      })

      const percentage = totalCount > 0 ? Math.round((presentCount / totalCount) * 100) : 0

      return {
        trainee: e.trainee,
        presentCount,
```

```
        totalCount,
        percentage,
        sessionDetails
      }
    }).sort((a, b) => b.percentage - a.percentage)
  }, [sessions, enrollments, attendanceRecords, metaRecords])

  const exportCSV = () => {
    let csv = 'Trainee Name,Email,Overall %,' + sessions.map(s => `"${s.title.replace(/"/g, '""')}"`).join(',') + '\n'

    attendanceData.forEach(row => {
      const name = `"${row.trainee?.full_name?.replace(/"/g, '""') || 'Unknown'}"`
      const email = `"${row.trainee?.email || ''}"`
      const pct = `${row.percentage}%`

      const sStatuses = sessions.map(s => {
        const det = row.sessionDetails.find(d => d.sessionId === s.id)
        return `"${det?.status || 'Absent'}"`
      }).join(',')

      csv += `${name},${email},${pct},${sStatuses}\n`
    })

    const blob = new Blob([csv], { type: 'text/csv' })
    const url = URL.createObjectURL(blob)
    const a = document.createElement('a')
    a.href = url
    a.download = `overall_attendance_${courseId}.csv`
    a.click()
  }

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-w-4xl p-0 overflow-hidden bg-white rounded-3xl border border-slate-200/60 shadow-2xl">
        <DialogHeader className="px-6 py-5 border-b border-slate-100 bg-slate-50/50">
          <div className="flex items-center justify-between">
            <div>
              <DialogTitle className="text-xl font-bold text-slate-800">Overall Attendance</DialogTitle>
              <DialogDescription className="text-sm text-slate-500 font-medium mt-1">
                View attendance across {sessions.length} sessions for {enrollments.length} enrolled trainees.
              </DialogDescription>
            </div>
            <Button onClick={exportCSV} variant="outline" size="sm" className="h-9 gap-2 border-slate-200 text-slate-700 hove
r:bg-slate-100 font-semibold rounded-xl">
              <Download className="w-4 h-4" /> Export CSV
            </Button>
          </div>
        </DialogHeader>

        <div className="p-0">
          {loading ? (
            <div className="flex flex-col items-center justify-center py-20 text-slate-400">
              <Loader2 className="w-8 h-8 animate-spin mb-4 text-cyan-500" />
              <p className="text-sm font-medium">Calculating overall attendance...</p>
            </div>
          ) : attendanceData.length === 0 ? (
            <div className="p-8 text-center text-slate-500 text-sm font-medium flex flex-col items-center gap-3">
              <AlertCircle className="w-8 h-8 text-slate-300" />
              No attendance data available. Make sure there are sessions and enrolled trainees.
            </div>
          ) : (
            <div className="overflow-auto max-h-[60vh]">
              <div className="min-w-max p-6">
                <table className="w-full text-left text-sm text-slate-700">
                  <thead>
                    <tr>
                      <th className="pb-4 font-bold text-slate-900 border-b border-slate-200 pr-6 sticky left-0 bg-white z-1
0">Trainee</th>
                      <th className="pb-4 font-bold text-slate-900 border-b border-slate-200 px-6 text-center">Overall</th>
                      {sessions.map((s, idx) => (
                        <th key={s.id} className="pb-4 font-semibold text-slate-500 border-b border-slate-200 px-4 whitespace-
nowrap text-center text-xs">
                          S{idx + 1}<br/>
                          <span className="font-normal text-slate-400 truncate max-w-[80px] inline-block">{s.title}</span>
                        </th>
                      ))}
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-slate-100">
```

```
                    {attendanceData.map((row, idx) => (
                      <tr key={idx} className="hover:bg-slate-50/50 transition-colors">
                        <td className="py-3 pr-6 sticky left-0 bg-white z-10 font-medium text-slate-800">
                          {row.trainee?.full_name || 'Unknown'}
                          <div className="text-xs text-slate-400 font-normal mt-0.5">{row.trainee?.email}</div>
                        </td>
                        <td className="py-3 px-6 text-center">
                          <div className={`inline-flex items-center justify-center px-2.5 py-1 rounded-lg text-xs font-bold ${
                            row.percentage >= 80 ? 'bg-emerald-100 text-emerald-700' :
                            row.percentage >= 50 ? 'bg-amber-100 text-amber-700' :
                            'bg-rose-100 text-rose-700'
                          }`}>
                            {row.percentage}%
                          </div>
                        </td>
                        {sessions.map(s => {
                          const det = row.sessionDetails.find(d => d.sessionId === s.id)
                          return (
                            <td key={s.id} className="py-3 px-4 text-center">
                              {det?.status === 'Present' ? (
                                <span className="text-emerald-500 font-bold">P</span>
                              ) : det?.status === 'Late' ? (
                                <span className="text-amber-500 font-bold">L</span>
                              ) : (
                                <span className="text-rose-400 font-bold">A</span>
                              )}
                            </td>
                          )
                        })}
                      </tr>
                    ))}
                  </tbody>
                </table>
              </div>
            </div>
          )}
        </div>
      </DialogContent>
    </Dialog>
  )
}
```

src/features/trainer/PerformancePage.tsx TSX 17.5 KB 342 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useParams, Link } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent } from '@/components/ui/card'
import { ArrowLeft, Users, BarChart3, Target, BookOpen, Loader2, TrendingUp, AlertTriangle, Award, Download } from 'lucide-react'
import { toast } from 'sonner'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'

import { calculateCourseGradeBreakdown, type CourseGradeBreakdown } from '@/lib/courseGrading'

type Course = Database['public']['Tables']['courses']['Row']
type Enrollment = Database['public']['Tables']['enrollments']['Row']
type Trainee = Database['public']['Tables']['trainees']['Row']
type AssessmentAttempt = Database['public']['Tables']['assessment_attempts']['Row']
type Question = Database['public']['Tables']['questions']['Row']

interface TraineeRow {
  enrollment: Enrollment
  trainee: Trainee | null
  attempts: AssessmentAttempt[]
  bestScore: number | null
  attemptCount: number
  gradeBreakdown: CourseGradeBreakdown
}

export function PerformancePage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user, profile } = useAuth()
  const isTrainer = profile?.role === 'trainer'
  const baseCoursePath = isTrainer ? '/trainer/courses' : '/admin/courses'
  const [course, setCourse] = useState<Course | null>(null)
  const [traineeRows, setTraineeRows] = useState<TraineeRow[]>([])
  const [_questions, setQuestions] = useState<Question[]>([])
  const [loading, setLoading] = useState(true)
  const [downloadingCertId, setDownloadingCertId] = useState<string | null>(null)

  const fetchData = useCallback(async () => {
    if (!user || !courseId) return
    setLoading(true)
    try {
      let query = supabase.from('courses').select('*').eq('id', courseId)
      if (profile?.role === 'trainer') {
        query = query.eq('trainer_id', user.id)
      }
      const { data: c } = await query.single()
      if (c) setCourse(c)

      // 1. Fetch all enrollments for this course

      const { data: e, error: eErr } = await supabase
        .from('enrollments')
        .select('*')
        .eq('course_id', courseId)
        .order('enrolled_at', { ascending: false })

      if (eErr) console.error('Error fetching enrollments:', eErr)
      const enrollments = e ?? []

      // 2. Fetch trainee profile details

      let traineesMap: Record<string, Trainee> = {}
      if (enrollments.length > 0) {
        const userIds = [...new Set(enrollments.map(item => item.user_id))]
        const { data: tData } = await supabase.from('trainees').select('*').in('id', userIds)
        if (tData) {
          traineesMap = tData.reduce((acc, curr) => {
            acc[curr.id] = curr
            return acc
          }, {} as Record<string, Trainee>)
        }
```

```
      }

      // 3. Fetch course assessments

      const { data: assessmentsList } = await supabase
        .from('assessments')
        .select('id, title, assessment_type, passing_score')
        .eq('course_id', courseId)

      const assessmentIds = assessmentsList?.map(a => a.id) || []

      // 4. Fetch all trainee attempts for these assessments

      let allAttempts: AssessmentAttempt[] = []
      if (assessmentIds.length > 0) {
        const { data: a } = await supabase.from('assessment_attempts').select('*').in('assessment_id', assessmentIds)
        if (a) allAttempts = a
      }

      // 5. Build computed trainee rows with grade breakdowns

      const rows: TraineeRow[] = enrollments.map(en => {
        const traineeInfo = traineesMap[en.user_id] || null
        const userAttempts = allAttempts.filter(at => at.user_id === en.user_id)
        const scores = userAttempts.filter(at => at.score !== null).map(at => at.score ?? 0)

        const breakdown = calculateCourseGradeBreakdown({
          moduleProgressPercent: en.progress_percent,
          courseAssessments: (assessmentsList || []) as any,
          traineeAttempts: userAttempts as any,
          passingScore: c?.passing_score ?? 50,
        })

        return {
          enrollment: en,
          trainee: traineeInfo,
          attempts: userAttempts,
          bestScore: scores.length > 0 ? Math.max(...scores) : null,
          attemptCount: userAttempts.length,
          gradeBreakdown: breakdown,
        }
      })
      setTraineeRows(rows)

      if (assessmentIds.length > 0) {
        const { data: q } = await supabase.from('questions').select('*').in('assessment_id', assessmentIds)
        if (q) setQuestions(q)
      }
    } catch (err) {
      console.error('Error fetching performance data:', err)
    } finally {
      setLoading(false)
    }
  }, [user, profile?.role, courseId])

  useEffect(() => { fetchData() }, [fetchData])

  const activeTraineeRows = traineeRows.filter(r => ['enrolled', 'in_progress', 'completed'].includes(r.enrollment.status))
  const totalEnrolled = activeTraineeRows.length
  const completed = activeTraineeRows.filter(r => r.enrollment.status === 'completed' || r.gradeBreakdown.isCompleted).length
  const completionRate = totalEnrolled > 0 ? Math.round((completed / totalEnrolled) * 100) : 0
  const avgProgress = totalEnrolled > 0 ? Math.round(activeTraineeRows.reduce((s, r) => s + (r.enrollment.progress_percent ||
0), 0) / totalEnrolled) : 0
  const avgScore = totalEnrolled > 0 ? Math.round(activeTraineeRows.reduce((s, r) => s + (r.gradeBreakdown.totalScore || 0),
0) / totalEnrolled) : 0
  const passedCount = activeTraineeRows.filter(r => r.gradeBreakdown.isPassed).length
  const passRate = totalEnrolled > 0 ? Math.round((passedCount / totalEnrolled) * 100) : 0
  const avgAttempts = totalEnrolled > 0 ? Math.round((activeTraineeRows.reduce((s, r) => s + r.attemptCount, 0) / totalEnrolle
d) * 10) / 10 : 0
  const needsSupport = activeTraineeRows.filter(r => !r.gradeBreakdown.isPassed && r.attemptCount > 0)

  const stats = [
    { label: 'Enrolled', value: totalEnrolled, icon: Users, iconColor: 'text-cyan-600', iconBg: 'bg-cyan-50 border-cyan-200'
},
    { label: 'Completed', value: completed, icon: Target, iconColor: 'text-emerald-600', iconBg: 'bg-emerald-50 border-emerald
-200' },
    { label: 'Completion Rate', value: `${completionRate}%`, icon: BarChart3, iconColor: 'text-blue-600', iconBg: 'bg-blue-50
border-blue-200' },
    { label: 'Avg. Progress', value: `${avgProgress}%`, icon: TrendingUp, iconColor: 'text-indigo-600', iconBg: 'bg-indigo-50
```

```
border-indigo-200' },
    { label: 'Avg. Score', value: `${avgScore}%`, icon: BarChart3, iconColor: 'text-purple-600', iconBg: 'bg-purple-50 border-purple-200' },
    { label: 'Pass Rate', value: `${passRate}%`, icon: Target, iconColor: 'text-teal-600', iconBg: 'bg-teal-50 border-teal-200' },
    { label: 'Avg. Attempts', value: avgAttempts.toString(), icon: BookOpen, iconColor: 'text-amber-600', iconBg: 'bg-amber-50 border-amber-200' },
  ]

  const handleDownloadTraineeCertificate = async (row: TraineeRow) => {
    if (!course) return
    const traineeName = row.trainee?.full_name || row.trainee?.email?.split('@')[0] || 'Trainee'
    const percentage = `${row.gradeBreakdown.totalScore}%`

    setDownloadingCertId(row.enrollment.user_id)
    try {
      const { blob, fileName } = await generateTraineeCertificate(
        (course as any).certificate_template_url,
        {
          traineeName,
          traineeEmail: row.trainee?.email || undefined,
          traineeId: row.enrollment.user_id,
          courseId: course.id,
          courseTitle: course.title,
          trainerName: profile?.full_name || 'Lead Instructor',
          percentage,
          completedAt: new Date().toISOString(),
        }
      )
      triggerFileDownload(blob, fileName)
      toast.success(`Certificate downloaded for ${traineeName}! 🎉`)
    } catch (err: any) {
      console.error('Error downloading certificate:', err)
      toast.error(err?.message || 'Failed to download certificate')
    } finally {
      setDownloadingCertId(null)
    }
  }

  if (loading) {
    return (
      <TrainerLayout>
        <div className="flex justify-center py-20">
          <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
        </div>
      </TrainerLayout>
    )
  }

  return (
    <TrainerLayout>
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-6xl mx-auto space-y-6">
        <motion.div variants={fadeUp}>
          <Link to={`${baseCoursePath}/${courseId}`} className="flex items-center gap-2 text-sm font-semibold text-slate-600 hover:text-slate-900 transition-colors mb-4">
            <ArrowLeft className="w-4 h-4" /> Back to Course
          </Link>
          <h2 className="text-2xl font-bold tracking-tight text-slate-900">Trainee Performance</h2>
          <p className="text-slate-500 text-sm mt-1">{course?.title}</p>
        </motion.div>

        <motion.div variants={fadeUp} className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-7 gap-3 md:gap-4">
          {stats.map(s => (
            <div key={s.label} className="p-4 rounded-2xl border border-slate-200/90 bg-white shadow-xs">
              <div className={`w-8 h-8 rounded-xl ${s.iconBg} border flex items-center justify-center mb-2`}>
                <s.icon className={`w-4 h-4 ${s.iconColor}`} />
              </div>
              <div className="text-xl font-black text-slate-900">{s.value}</div>
              <div className="text-[11px] font-semibold text-slate-500 mt-0.5">{s.label}</div>
            </div>
          ))}
        </motion.div>

        {needsSupport.length > 0 && (
          <motion.div variants={fadeUp}>
            <Card className="bg-amber-50 border border-amber-200 rounded-2xl">
              <CardContent className="p-5 flex items-start gap-3.5">
                <AlertTriangle className="w-5 h-5 text-amber-600 shrink-0 mt-0.5" />
                <div>
```

```
                  <p className="text-sm font-bold text-amber-900">Trainees Needing Support ({needsSupport.length})</p>
                  <div className="mt-1.5 space-y-1">
                    {needsSupport.map(r => (
                      <p key={r.enrollment.user_id} className="text-xs text-amber-800">
                        <span className="font-semibold">{r.trainee?.full_name ?? 'Unknown'}</span> — Score: <span className="f
ont-bold">{r.bestScore}%</span> (Pass requirement: {course?.passing_score}%)
                      </p>
                    ))}
                  </div>
                </div>
              </CardContent>
            </Card>
          </motion.div>
        )}

        <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-2xl overflow-hidden shadow-xs">
          <div className="px-6 py-4 border-b border-slate-100 flex items-center justify-between">
            <h3 className="text-sm font-bold text-slate-900">Enrolled Trainees</h3>
            <span className="text-xs text-slate-500 font-medium">{activeTraineeRows.length} Trainees Total</span>
          </div>
          {activeTraineeRows.length === 0 ? (
            <div className="p-8 text-center text-slate-400 text-sm font-medium">No trainees enrolled yet.</div>
          ) : (
            <div className="overflow-x-auto">
              <table className="w-full">
                <thead>
                  <tr className="border-b border-slate-100 bg-slate-50/70">
                    {['Trainee', 'Enrolled', 'Modules (25%)', 'Assessments (25%)', 'Final Exam (50%)', 'Total Grade (100%)',
'Status', 'Certificate'].map(h => (
                      <th key={h} className="text-left text-xs text-slate-500 font-semibold px-6 py-3">{h}</th>
                    ))}
                  </tr>
                </thead>
                <tbody className="divide-y divide-slate-100">
                  {activeTraineeRows.map(r => {
                    const gb = r.gradeBreakdown
                    const isDownloading = downloadingCertId === r.enrollment.user_id
                    return (
                      <tr key={r.enrollment.user_id} className="hover:bg-slate-50/80 transition-colors">
                        <td className="px-6 py-3.5">
                          <div className="flex items-center gap-3">
                            <div className="w-8 h-8 rounded-full bg-gradient-to-br from-cyan-500 to-blue-600 flex items-center
justify-center text-white text-xs font-bold shrink-0">
                              {r.trainee?.full_name?.charAt(0) ?? '?'}
                            </div>
                            <div>
                              <p className="text-sm text-slate-900 font-bold">{r.trainee?.full_name ?? 'Unknown'}</p>
                              <p className="text-xs text-slate-400">{r.trainee?.email}</p>
                            </div>
                          </div>
                        </td>
                        <td className="px-6 py-3.5 text-xs text-slate-600 font-medium">{new Date(r.enrollment.enrolled_at).toL
ocaleDateString()}</td>
                        <td className="px-6 py-3.5">
                          <div>
                            <span className="text-xs font-bold text-cyan-700">{gb.moduleScore} / 25 pts</span>
                            <span className="text-[10px] text-slate-400 block">({gb.moduleProgressPercent}% complete)</span>
                          </div>
                        </td>
                        <td className="px-6 py-3.5">
                          <div>
                            <span className="text-xs font-bold text-amber-700">{gb.regularAssessmentScore} / 25 pts</span>
                            <span className="text-[10px] text-slate-400 block">
                              {gb.regularAssessmentsTotal > 0
                                ? `Avg: ${gb.regularAssessmentAveragePercent}% (${gb.regularAssessmentsCompleted}/${gb.regular
AssessmentsTotal})`
                                : 'Module-aligned'}
                            </span>
                          </div>
                        </td>
                        <td className="px-6 py-3.5">
                          <div>
                            <span className="text-xs font-bold text-purple-700">{gb.finalAssessmentWeightedScore} / 50 pts</sp
an>
                            <span className="text-[10px] text-slate-400 block">
                              {gb.finalAssessmentCompleted
                                ? `Score: ${gb.finalAssessmentScorePercent}%`
                                : gb.isFinalUnlocked
                                  ? 'Unlocked'
```

```
                                    : 'Locked'}
                                </span>
                              </div>
                            </td>
                            <td className="px-6 py-3.5">
                              <div className="flex items-baseline gap-1">
                                <span className="text-sm font-black text-slate-900">{gb.totalScore}%</span>
                                <span className={`text-[10px] font-bold px-1.5 py-0.5 rounded ${
                                  gb.isPassed ? 'bg-emerald-50 text-emerald-700' : 'bg-rose-50 text-rose-700'
                                }`}>
                                  {gb.isPassed ? 'Passed' : 'Pending'}
                                </span>
                              </div>
                            </td>
                            <td className="px-6 py-3.5">
                              <Badge className={`text-xs font-semibold capitalize ${
                                r.enrollment.status === 'completed' || gb.isCompleted ? 'bg-emerald-50 text-emerald-700 border-eme
rald-200' :
                                r.enrollment.status === 'in_progress' ? 'bg-blue-50 text-blue-700 border-blue-200' :
                                'bg-slate-100 text-slate-700 border-slate-200'
                              }`}>
                                {(r.enrollment.status === 'completed' || gb.isCompleted) ? 'Completed' : r.enrollment.status.repla
ce('_', ' ')}
                              </Badge>
                            </td>
                            <td className="px-6 py-3.5">
                              <button
                                type="button"
                                onClick={() => handleDownloadTraineeCertificate(r)}
                                disabled={isDownloading}
                                className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-gradient-to-r from-emerald-6
00 via-teal-600 to-cyan-600 hover:from-emerald-700 hover:to-cyan-700 text-white font-bold text-xs shadow-xs hover:scale-105 active:s
cale-95 transition-all cursor-pointer disabled:opacity-60"
                                title="Download Trainee Certificate"
                              >
                                {isDownloading ? (
                                  <Loader2 className="w-3.5 h-3.5 animate-spin" />
                                ) : (
                                  <Award className="w-3.5 h-3.5 text-amber-300" />
                                )}
                                <span>{isDownloading ? 'Generating...' : 'Certificate'}</span>
                              </button>
                            </td>
                          </tr>
                        )
                      })}
                    </tbody>
                  </table>
                </div>
              )}
            </motion.div>
          </motion.div>
        </TrainerLayout>
      )
}

```

src/features/trainer/SettingsPage.tsx

TSX 0.1 KB 7 Lines

```tsx
import React from 'react'
import { ProfilePage } from '@/pages/ProfilePage'

export function SettingsPage() {
  return <ProfilePage />
}

```

src/features/trainer/TrainerDashboard.tsx TSX 10.4 KB 205 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { Link } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { BookOpen, Users, Target, PlusCircle, TrendingUp, CheckCircle, Clock, FileText, Star } from 'lucide-react'

type Course = Database['public']['Tables']['courses']['Row']
type Enrollment = Database['public']['Tables']['enrollments']['Row']
type AssessmentAttempt = Database['public']['Tables']['assessment_attempts']['Row']
type ActivityLog = Database['public']['Tables']['audit_logs']['Row']

const statusColors: Record<string, string> = {
  draft: 'bg-slate-100 text-slate-700 border border-slate-200',
  pending_review: 'bg-amber-50 text-amber-700 border border-amber-200',
  published: 'bg-emerald-50 text-emerald-700 border border-emerald-200',
  archived: 'bg-rose-50 text-rose-700 border border-rose-200',
}

export function TrainerDashboard() {
  const { user } = useAuth()
  const [courses, setCourses] = useState<Course[]>([])
  const [enrollments, setEnrollments] = useState<Enrollment[]>([])
  const [attempts, setAttempts] = useState<AssessmentAttempt[]>([])
  const [activities, setActivities] = useState<ActivityLog[]>([])

  const fetchData = useCallback(async () => {
    if (!user) return
    try {
      const { data: c } = await supabase
        .from('courses')
        .select('*')
        .eq('trainer_id', user.id)
        .neq('status', 'archived')
        .order('created_at', { ascending: false })
      if (c) setCourses(c)

      const courseIds = (c ?? []).map(co => co.id)
      if (courseIds.length > 0) {
        const { data: e } = await supabase
          .from('enrollments')
          .select('*')
          .in('course_id', courseIds)
        if (e) setEnrollments(e)

        const { data: assessments } = await supabase
          .from('assessments')
          .select('id')
          .in('course_id', courseIds)
          .eq('created_by', user.id)

        const assessmentIds = (assessments ?? []).map(a => a.id)
        if (assessmentIds.length > 0) {
          const { data: a } = await supabase
            .from('assessment_attempts')
            .select('*')
            .in('assessment_id', assessmentIds)
          if (a) setAttempts(a)
        }
      }

      const { data: logs } = await supabase
        .from('audit_logs')
        .select('*')
        .eq('actor_id', user.id)
        .order('created_at', { ascending: false })
        .limit(10)
      if (logs) setActivities(logs)
    } catch (err) {
      console.error(err)
    }
  }, [user])
```

```
  |
  |   useEffect(() => { fetchData() }, [fetchData])
  |
  |   const totalCourses = courses.length
  |   const publishedCourses = courses.filter(c => c.status === 'published').length
  |   const pendingReview = courses.filter(c => c.status === 'pending_review').length
  |   const draftCourses = courses.filter(c => c.status === 'draft').length
  |   const totalEnrollments = enrollments.length
  |   const completedEnrollments = enrollments.filter(e => e.status === 'completed').length
  |   const completionRate = totalEnrollments > 0 ? Math.round((completedEnrollments / totalEnrollments) * 100) : 0
  |
  |   const scoresFiltered = attempts.filter(a => a.score !== null)
  |   const avgScore = scoresFiltered.length > 0
  |     ? Math.round(scoresFiltered.reduce((sum, a) => sum + (a.score ?? 0), 0) / scoresFiltered.length)
  |     : 0
  |
  |   const stats = [
  |     { label: 'Total Courses', value: totalCourses, icon: BookOpen, iconColor: 'text-cyan-600', iconBg: 'bg-cyan-50 border-cyan
-200' },
  |     { label: 'Published', value: publishedCourses, icon: CheckCircle, iconColor: 'text-emerald-600', iconBg: 'bg-emerald-50 bo
rder-emerald-200' },
  |     { label: 'Pending Review', value: pendingReview, icon: Clock, iconColor: 'text-amber-600', iconBg: 'bg-amber-50 border-amb
er-200' },
  |     { label: 'Drafts', value: draftCourses, icon: FileText, iconColor: 'text-slate-600', iconBg: 'bg-slate-100 border-slate-20
0' },
  |     { label: 'Enrolled Trainees', value: totalEnrollments, icon: Users, iconColor: 'text-blue-600', iconBg: 'bg-blue-50 border
-blue-200' },
  |     { label: 'Avg. Score', value: `${avgScore}%`, icon: TrendingUp, iconColor: 'text-purple-600', iconBg: 'bg-purple-50 border
-purple-200' },
  |     { label: 'Completion Rate', value: `${completionRate}%`, icon: Target, iconColor: 'text-teal-600', iconBg: 'bg-teal-50 bor
der-teal-200' },
  |     { label: 'Certificates', value: completedEnrollments, icon: Star, iconColor: 'text-yellow-600', iconBg: 'bg-yellow-50 bord
er-yellow-200' },
  |   ]
  |
  |   return (
  |     <TrainerLayout>
  |       <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-4 md:space-y-6 max-w-6xl">
  |         <motion.div variants={fadeUp} className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
  |           <div>
  |             <h2 className="text-xl md:text-2xl font-bold tracking-tight text-slate-900">Trainer Dashboard</h2>
  |             <p className="text-slate-500 text-sm mt-1">Welcome back, {user?.email?.split('@')[0]}</p>
  |           </div>
  |           <Link to="/trainer/courses/new">
  |             <Button className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 te
xt-white font-bold shadow-md shadow-cyan-600/20 w-full sm:w-auto">
  |               <PlusCircle className="w-4 h-4 mr-2" /> Create Course
  |             </Button>
  |           </Link>
  |         </motion.div>
  |
  |         <motion.div variants={fadeUp} className="grid grid-cols-2 md:grid-cols-4 gap-3 md:gap-4">
  |           {stats.map(s => (
  |             <div key={s.label} className="p-4 rounded-2xl border border-slate-200/90 bg-white shadow-xs hover:border-slate-300
transition-all">
  |               <div className={`w-8 h-8 rounded-xl ${s.iconBg} border flex items-center justify-center mb-2.5`}>
  |                 <s.icon className={`w-4 h-4 ${s.iconColor}`} />
  |               </div>
  |               <div className="text-xl md:text-2xl font-black text-slate-900">{s.value}</div>
  |               <div className="text-xs font-semibold text-slate-500 mt-0.5">{s.label}</div>
  |             </div>
  |           ))}
  |         </motion.div>
  |
  |         <div className="grid grid-cols-1 lg:grid-cols-3 gap-4 md:gap-6">
  |           <motion.div variants={fadeUp} className="lg:col-span-2 bg-white border border-slate-200/90 rounded-2xl overflow-hidd
en shadow-xs">
  |             <div className="px-4 md:px-6 py-3.5 border-b border-slate-100 flex items-center justify-between">
  |               <h3 className="text-sm font-bold text-slate-900">Recent Courses</h3>
  |               <Link to="/trainer/courses" className="text-xs font-semibold text-cyan-600 hover:text-cyan-700">View all</Link>
  |             </div>
  |             {courses.length === 0 ? (
  |               <div className="p-8 md:p-12 text-center">
  |                 <p className="text-slate-400 mb-3 text-sm">No courses yet.</p>
  |                 <Link to="/trainer/courses/new">
  |                   <Button className="bg-gradient-to-r from-cyan-600 to-blue-600 text-white font-semibold">
  |                     <PlusCircle className="w-4 h-4 mr-2" /> Create First Course
  |                   </Button>
  |                 </Link>
```

```
145 |               </div>
146 |             ) : (
147 |               <div className="overflow-x-auto">
148 |                 <table className="w-full min-w-[450px]">
149 |                   <thead>
150 |                     <tr className="border-b border-slate-100 bg-slate-50/70">
151 |                       {['Course', 'Status', 'Enrollments', 'Type', 'Created'].map(h => (
152 |                         <th key={h} className="text-left text-xs text-slate-500 font-semibold px-4 py-3 whitespace-nowrap">{h}
</th>
153 |                       ))}
154 |                     </tr>
155 |                   </thead>
156 |                   <tbody className="divide-y divide-slate-100">
157 |                     {courses.slice(0, 5).map(course => {
158 |                       const enrollCount = enrollments.filter(e => e.course_id === course.id).length
159 |                       return (
160 |                         <tr key={course.id} className="hover:bg-slate-50/80 transition-colors">
161 |                           <td className="px-4 py-3">
162 |                             <Link to={`/trainer/courses/${course.id}`} className="text-xs md:text-sm text-slate-900 hover:text
-cyan-600 transition-colors font-semibold truncate max-w-[180px] block">
163 |                               {course.title}
164 |                             </Link>
165 |                           </td>
166 |                           <td className="px-4 py-3">
167 |                             <Badge className={`${statusColors[course.status]} text-[10px] font-semibold`}>{course.status.repla
ce('_', ' ')}</Badge>
168 |                           </td>
169 |                           <td className="px-4 py-3 text-xs font-medium text-slate-600">{enrollCount}</td>
170 |                           <td className="px-4 py-3 text-xs font-medium text-slate-600 capitalize">{course.course_type}</td>
171 |                           <td className="px-4 py-3 text-xs text-slate-400">{new Date(course.created_at).toLocaleDateString()}
</td>
172 |                         </tr>
173 |                       )
174 |                     })}
175 |                   </tbody>
176 |                 </table>
177 |               </div>
178 |             )}
179 |           </motion.div>
180 |
181 |           <motion.div variants={fadeUp} className="bg-white border border-slate-200/90 rounded-2xl overflow-hidden shadow-xs">
182 |             <div className="px-4 md:px-6 py-3.5 border-b border-slate-100">
183 |               <h3 className="text-sm font-bold text-slate-900">Recent Activity</h3>
184 |             </div>
185 |             {activities.length === 0 ? (
186 |               <div className="p-8 text-center">
187 |                 <p className="text-slate-400 text-sm">No recent activity.</p>
188 |               </div>
189 |             ) : (
190 |               <div className="divide-y divide-slate-100">
191 |                 {activities.slice(0, 8).map(a => (
192 |                   <div key={a.id} className="px-4 md:px-6 py-3">
193 |                     <p className="text-xs font-semibold text-slate-800 truncate">{a.action}</p>
194 |                     <p className="text-[10px] text-slate-400 mt-0.5">{new Date(a.created_at).toLocaleString()}</p>
195 |                   </div>
196 |                 ))}
197 |               </div>
198 |             )}
199 |           </motion.div>
200 |         </div>
201 |       </motion.div>
202 |     </TrainerLayout>
203 |   )
204 | }
205 |
```

src/features/trainer/TrainerLayout.tsx TSX 9.8 KB 196 Lines

```tsx
import React, { useState } from 'react'
import { Link, useLocation, useNavigate } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import {
  LogOut, BookOpen, BarChart3,
  Settings, Bell, Menu, X,
  User, ChevronRight, GraduationCap, PlusCircle, Shield, Target, Megaphone, LayoutDashboard,
  Users, FileCheck, Mail, Globe
} from 'lucide-react'
import { NotificationPanel } from '@/components/notifications/NotificationPanel'

const trainerNavItems = [
  { to: '/trainer', label: 'Dashboard', icon: BarChart3, exact: true },
  { to: '/trainer/courses', label: 'My Courses', icon: BookOpen },
  { to: '/trainer/courses/new', label: 'Create Course', icon: PlusCircle },
  { to: '/trainer/notifications', label: 'Notifications', icon: Bell },
  { to: '/trainer/profile', label: 'Profile', icon: User },
]

const getAdminNavItems = (isSuperAdmin: boolean) => [
  { to: '/admin', label: 'Dashboard', icon: LayoutDashboard, exact: true },
  { to: '/admin?tab=trainees', label: 'Trainees', icon: Users },
  { to: '/admin?tab=trainers', label: 'Trainers', icon: Users },
  ...(isSuperAdmin ? [{ to: '/admin?tab=admins', label: 'Admins', icon: Shield }] : []),
  { to: '/admin/courses', label: 'Courses', icon: BookOpen },
  { to: '/admin/assessments', label: 'Assessments', icon: FileCheck },
  { to: '/admin/announcements', label: 'Announcements', icon: Megaphone },
  { to: '/admin/notifications', label: 'Notifications', icon: Bell },
  { to: '/admin?tab=messages', label: 'Messages', icon: Mail },
  { to: '/admin?tab=home_page', label: 'Home Page', icon: Globe },
  ...(isSuperAdmin ? [{ to: '/admin?tab=logs', label: 'Audit Logs', icon: BarChart3 }] : []),
  { to: isSuperAdmin ? '/super-admin/profile' : '/admin/profile', label: 'Profile', icon: User },
]

export const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
export const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

export function TrainerLayout({ children }: { children: React.ReactNode }) {
  const { signOut, profile } = useAuth()
  const navigate = useNavigate()
  const location = useLocation()
  const [mobileOpen, setMobileOpen] = useState(false)
  const [unreadCount, setUnreadCount] = useState<number | null>(null)

  const isSuperAdmin = profile?.role === 'super_admin'
  const isAdmin = profile?.role === 'admin' || isSuperAdmin
  const navItems = isAdmin ? getAdminNavItems(isSuperAdmin) : trainerNavItems

  React.useEffect(() => {
    if (!profile?.id) return
    supabase
      .from('notifications')
      .select('*', { count: 'exact', head: true })
      .eq('user_id', profile.id)
      .is('read_at', null)
      .then(({ count }) => setUnreadCount(count ?? 0))
  }, [profile?.id])

  const handleSignOut = async () => {
    await signOut()
    navigate('/login')
  }

  const isActive = (item: any) => {
    if (item.exact) return location.pathname === item.to && !location.search
    if (item.to.includes('?tab=')) {
      const targetTab = item.to.split('?tab=')[1]
      const currentTab = new URLSearchParams(location.search).get('tab')
      return location.pathname === '/admin' && currentTab === targetTab
    }
    if (item.to === '/trainer/courses' || item.to === '/admin/courses') {
      if (location.pathname === item.to + '/new') return false
```

```
      return location.pathname === item.to || location.pathname.startsWith(item.to + '/')
    }
    return location.pathname.startsWith(item.to) && item.to !== '/trainer' && item.to !== '/admin'
  }

  const sidebarContent = (
    <>
      <div className="h-16 flex items-center px-5 border-b border-slate-800">
        <Link to="/" className="flex items-center gap-2.5">
          <img src="/logo.png" alt="Logo" className="w-8 h-8 object-contain shrink-0" />
          <span className="text-sm font-bold whitespace-nowrap">
            <span className="text-white">Capacity</span>
            <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
          </span>
        </Link>
      </div>

      <nav className="flex-1 p-3 space-y-1.5 overflow-y-auto">
        {navItems.map(item => {
          const active = isActive(item)
          return (
            <Link key={item.to} to={item.to} onClick={() => setMobileOpen(false)}>
              <button className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-xl transition-all text-sm font-medium gro
up ${
                active
                  ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white shadow-md shadow-cyan-600/20'
                  : 'text-slate-300 hover:text-white hover:bg-slate-800/60 border border-transparent'
              }`}>
                <div className="relative">
                  <item.icon className={`w-4 h-4 shrink-0 ${active ? 'text-white' : 'text-slate-400 group-hover:text-cyan-40
0'} transition-colors`} />
                  {item.label === 'Notifications' && unreadCount !== null && unreadCount > 0 && (
                    <span className="absolute -top-1.5 -right-1.5 w-3.5 h-3.5 bg-amber-500 rounded-full text-[8px] text-white
flex items-center justify-center font-bold">
                      {unreadCount > 9 ? '9+' : unreadCount}
                    </span>
                  )}
                </div>
                <span className="flex-1 text-left">{item.label}</span>
                <ChevronRight className={`w-3.5 h-3.5 transition-opacity ${active ? 'opacity-100' : 'opacity-0 group-hover:opa
city-100'}`} />
              </button>
            </Link>
          )
        })}
      </nav>

      <div className="p-3 border-t border-slate-800 space-y-2">
        <div className="flex items-center gap-3 px-3 py-2 rounded-xl hover:bg-slate-800/50 transition-all cursor-pointer grou
p">
          <div className="w-8 h-8 rounded-full bg-gradient-to-br from-cyan-500 to-blue-600 flex items-center justify-center te
xt-white text-xs font-bold shrink-0 ring-2 ring-cyan-500/20 overflow-hidden">
            {profile?.avatar_path ? (
              <img src={profile.avatar_path} alt={profile.full_name} className="w-full h-full object-cover" />
            ) : (
              profile?.full_name?.charAt(0)?.toUpperCase() ?? '?'
            )}
          </div>
          <div className="min-w-0 flex-1">
            <p className="text-xs font-semibold text-slate-200 truncate group-hover:text-cyan-400 transition-colors">{profil
e?.full_name}</p>
            <p className="text-[11px] text-slate-400 truncate">{profile?.email}</p>
          </div>
        </div>
        <div className="flex items-center gap-2 px-3">
          <span className={`text-xs px-2 py-0.5 rounded-full border capitalize font-semibold ${
            isAdmin ? 'bg-indigo-500/10 text-indigo-400 border-indigo-500/30' : 'bg-orange-500/10 text-orange-400 border-orang
e-500/30'
          }`}>
            {profile?.role?.replace('_', ' ') || 'User'}
          </span>
        </div>
        <button onClick={handleSignOut} className="w-full flex items-center gap-2 px-3 py-2 rounded-xl text-slate-400 hover:te
xt-rose-400 hover:bg-rose-500/10 transition-all text-sm font-medium">
          <LogOut className="w-4 h-4" />
          <span>Logout</span>
        </button>
      </div>
    </>
```

```
147 |   )
148 |
149 |   return (
150 |     <div className="min-h-screen bg-[#f8fafc] text-slate-900 flex">
151 |       {mobileOpen && (
152 |         <div className="fixed inset-0 z-40 bg-slate-950/60 backdrop-blur-sm md:hidden" onClick={() => setMobileOpen(false)} />
153 |       )}
154 |
155 |       <aside className={`fixed inset-y-0 left-0 z-50 w-64 border-r border-slate-800 bg-[#040814] text-white flex flex-col tran
sition-transform duration-300 md:hidden ${mobileOpen ? 'translate-x-0' : '-translate-x-full'}`}>
156 |         <div className="absolute top-4 right-4 z-10">
157 |           <button onClick={() => setMobileOpen(false)} className="p-2 rounded-lg hover:bg-slate-800 text-slate-400 hover:text-
white transition-all">
158 |             <X className="w-5 h-5" />
159 |           </button>
160 |         </div>
161 |         {sidebarContent}
162 |       </aside>
163 |
164 |       <aside className="relative z-20 hidden md:flex w-64 border-r border-slate-800 bg-[#040814] text-white flex-col shrink-0
h-screen sticky top-0 transition-all duration-300">
165 |         {sidebarContent}
166 |       </aside>
167 |
168 |       <div className="flex-1 min-w-0 flex flex-col">
169 |         <header className="h-14 md:h-16 border-b border-slate-200 bg-white/95 backdrop-blur-md sticky top-0 z-30 flex items-ce
nter px-4 md:px-6 gap-3 shadow-xs">
170 |           <button onClick={() => setMobileOpen(true)} className="p-2 rounded-lg hover:bg-slate-100 text-slate-600 hover:text-s
late-900 transition-all md:hidden">
171 |             <Menu className="w-5 h-5" />
172 |           </button>
173 |           <div className="flex items-center gap-2.5">
174 |             <div className="w-8 h-8 rounded-lg bg-cyan-50 border border-cyan-200 flex items-center justify-center text-cyan-60
0 shadow-xs">
175 |               {isAdmin ? <Shield className="w-4 h-4 md:w-5 md:h-5" /> : <GraduationCap className="w-4 h-4 md:w-5 md:h-5" />}
176 |             </div>
177 |             <span className="text-sm md:text-base font-bold text-slate-900">
178 |               {isAdmin ? 'Admin Portal' : 'Trainer Dashboard'}
179 |             </span>
180 |           </div>
181 |           <div className="flex-1" />
182 |           <div className="flex items-center gap-1.5 md:gap-2">
183 |             <NotificationPanel />
184 |             <Link to={isAdmin ? '/admin/profile' : '/trainer/profile'} className="p-2 rounded-lg hover:bg-slate-100 text-slate
-600 hover:text-slate-900 transition-all" title="Profile">
185 |               <User className="w-4 h-4" />
186 |             </Link>
187 |           </div>
188 |         </header>
189 |         <main className="p-4 md:p-6 lg:p-8 flex-1">
190 |           {children}
191 |         </main>
192 |       </div>
193 |     </div>
194 |   )
195 | }
196 |
```

src/features/trainer/TrainerProfile.tsx

TSX 0.1 KB 7 Lines

```tsx
import React from 'react'
import { ProfilePage } from '@/pages/ProfilePage'

export function TrainerProfile() {
  return <ProfilePage />
}

```

src/features/trainer/TrainerSkills.tsx TSX 8.4 KB 196 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout, fadeUp, stagger } from './TrainerLayout'
import { motion } from 'framer-motion'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Card, CardContent } from '@/components/ui/card'
import { Plus, X, Search, Loader2 } from 'lucide-react'
import { toast } from 'sonner'

type Skill = Database['public']['Tables']['skills']['Row']
type UserSkill = Database['public']['Tables']['user_skills']['Row']

export function TrainerSkills() {
  const { user } = useAuth()
  const [allSkills, setAllSkills] = useState<Skill[]>([])
  const [userSkills, setUserSkills] = useState<(UserSkill & { skills: Skill | null })[]>([])
  const [search, setSearch] = useState('')
  const [saving, setSaving] = useState(false)
  const [customSkillName, setCustomSkillName] = useState('')
  const [addingCustom, setAddingCustom] = useState(false)

  const fetchData = useCallback(async () => {
    if (!user) return
    try {
      const { data: sk } = await supabase.from('skills').select('*').order('name')
      if (sk) {
        setAllSkills(sk)
      }

      const { data: us } = await supabase
        .from('user_skills')
        .select('*, skills(*)')
        .eq('user_id', user.id)
      if (us) setUserSkills(us as any)
    } catch (err) {
      console.error(err)
    }
  }, [user])

  useEffect(() => { fetchData() }, [fetchData])

  const filteredSkills = allSkills.filter(s =>
    s.name.toLowerCase().includes(search.toLowerCase()) &&
    !userSkills.some(us => us.skill_id === s.id)
  )

  const handleAddSkill = async (skillId: string) => {
    if (!user) return
    setSaving(true)
    try {
      const { error } = await supabase
        .from('user_skills')
        .insert({ user_id: user.id, skill_id: skillId })
      if (error) throw error
      toast.success('Skill added')
      fetchData()
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed')
    } finally {
      setSaving(false)
    }
  }

  const handleRemoveSkill = async (skillId: string) => {
    if (!user) return
    setSaving(true)
    try {
      await supabase.from('user_skills').delete().eq('user_id', user.id).eq('skill_id', skillId)
      toast.success('Skill removed')
      fetchData()
    } catch (err) {
      toast.error('Failed')
    } finally {
```

```
      setSaving(false)
    }
  }

  const handleCreateAndAddSkill = async () => {
    const name = customSkillName.trim()
    if (!name || !user) return
    if (allSkills.some(s => s.name.toLowerCase() === name.toLowerCase())) {
      toast.error('Skill already exists')
      return
    }
    setAddingCustom(true)
    try {
      const { data: newSkill, error: createErr } = await supabase.from('skills').insert({ name }).select().single()
      if (createErr) throw createErr
      const { error: linkErr } = await supabase.from('user_skills').insert({ user_id: user.id, skill_id: newSkill.id })
      if (linkErr) throw linkErr
      setCustomSkillName('')
      setSearch('')
      toast.success(`Skill "${name}" created and added`)
      fetchData()
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed')
    } finally {
      setAddingCustom(false)
    }
  }

  return (
    <TrainerLayout>
      <motion.div variants={stagger} initial="hidden" animate="visible" className="max-w-2xl mx-auto space-y-6">
        <motion.div variants={fadeUp}>
          <h2 className="text-2xl font-bold tracking-tight text-slate-900">Skills & Expertise</h2>
          <p className="text-slate-500 text-sm mt-1">Manage the skills listed on your profile</p>
        </motion.div>

        <motion.div variants={fadeUp}>
          <Card className="bg-white border border-slate-200/90 shadow-xs rounded-2xl overflow-hidden">
            <CardContent className="p-6">
              <div className="space-y-4">
                {userSkills.length > 0 && (
                  <div className="space-y-2">
                    <Label className="text-slate-700 text-xs font-semibold">Your Skills ({userSkills.length})</Label>
                    <div className="flex flex-wrap gap-2">
                      {userSkills.map(us => (
                        <div key={us.skill_id} className="flex items-center gap-1.5 px-3 py-1.5 rounded-full bg-cyan-50 border
border-cyan-200 text-cyan-800 text-xs font-semibold">
                          <span>{us.skills?.name ?? 'Unknown'}</span>
                          <button onClick={() => handleRemoveSkill(us.skill_id)} disabled={saving} className="hover:text-rose-
600 transition-colors">
                            <X className="w-3.5 h-3.5" />
                          </button>
                        </div>
                      ))}
                    </div>
                  </div>
                )}

                <div className="space-y-2">
                  <Label className="text-slate-700 text-xs font-semibold">Add Skills</Label>
                  <div className="relative">
                    <Search className="absolute left-3.5 top-1/2 -translate-y-1/2 w-4 h-4 text-slate-400" />
                    <Input
                      placeholder="Search skills..."
                      value={search}
                      onChange={e => setSearch(e.target.value)}
                      className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white pl-9 h-10 rounded-xl"
                    />
                  </div>
                  {search && filteredSkills.length > 0 && (
                    <div className="max-h-48 overflow-y-auto space-y-1 border border-slate-200 rounded-xl p-1 bg-white shadow-
sm">
                      {filteredSkills.slice(0, 20).map(s => (
                        <button
                          key={s.id}
                          onClick={() => { handleAddSkill(s.id); setSearch('') }}
                          disabled={saving}
                          className="w-full flex items-center justify-between px-3 py-2 hover:bg-slate-50 rounded-lg text-left
text-sm text-slate-800 font-medium transition-colors"
```

```
                        >
                          <span>{s.name}</span>
                          <Plus className="w-3.5 h-3.5 text-cyan-600" />
                        </button>
                      ))}
                    </div>
                  )}
                  {search && filteredSkills.length === 0 && (
                    <div className="space-y-2 pt-1">
                      <p className="text-xs text-slate-400">No matching skills found.</p>
                      <div className="flex gap-2">
                        <Input
                          value={customSkillName || search}
                          onChange={e => setCustomSkillName(e.target.value)}
                          onKeyDown={e => { if (e.key === 'Enter') { e.preventDefault(); handleCreateAndAddSkill() } }}
                          placeholder="Create new skill..."
                          className="bg-slate-50 border-slate-200 text-slate-900 focus:bg-white h-10 text-xs rounded-xl"
                          disabled={addingCustom}
                        />
                        <button
                          onClick={handleCreateAndAddSkill}
                          disabled={addingCustom || !(customSkillName || search).trim()}
                          className="px-3.5 h-10 rounded-xl bg-gradient-to-r from-cyan-600 to-blue-600 text-white text-xs font
-bold hover:opacity-95 transition-all shrink-0 disabled:opacity-50"
                        >
                          {addingCustom ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Plus className="w-3.5 h-3.5" />}
                        </button>
                      </div>
                    </div>
                  )}
                </div>

                {userSkills.length === 0 && !search && (
                  <p className="text-sm text-slate-400 text-center py-4">
                    No skills added yet. Search to add, or type a new skill name to create one.
                  </p>
                )}
              </div>
            </CardContent>
          </Card>
        </motion.div>
      </motion.div>
    </TrainerLayout>
  )
}

```

src/features/courses/AdminCourses.tsx TSX 77.9 KB 1,506 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { Database } from '@/integrations/supabase/types'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@/components/ui/card'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
} from '@/components/ui/dialog'
import { Thumbnail } from '@/components/ui/Thumbnail'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { MoreHorizontal, BookOpen, Eye, FileText, Target, Clock, Users, Loader2, File, Video, Globe, ExternalLink, Download, Layers, Plus, CheckCircle2, XCircle, AlertCircle, ShieldAlert, Timer, Check, Sparkles, PlayCircle, HelpCircle, Image as ImageIcon, Edit3, BarChart3, Info, BookMarked } from 'lucide-react'
import { toast } from 'sonner'
import { formatDistanceToNow } from 'date-fns'
import { Textarea } from '@/components/ui/textarea'

function getFormattedObjectives(raw: any): string {
  if (!raw) return ''
  if (typeof raw === 'string') return raw
  if (Array.isArray(raw)) return raw.join('\n\n')
  if (typeof raw === 'object') {
    const parts: string[] = []
    if (raw.description) parts.push(raw.description)
    if (raw.understand) parts.push(`What you'll understand: ${raw.understand}`)
    if (raw.able_to_do) parts.push(`What you'll be able to do: ${raw.able_to_do}`)
    return parts.length > 0 ? parts.join('\n\n') : JSON.stringify(raw)
  }
  return String(raw)
}

type Course = Database['public']['Tables']['courses']['Row'] & {
  edit_request_status?: string | null
  edit_request_reason?: string | null
  edit_request_at?: string | null
  edit_window_expires_at?: string | null
  edit_window_duration_hours?: number | null
  admin_edit_notes?: string | null
}
type Material = Database['public']['Tables']['materials']['Row']
type CourseSkill = Database['public']['Tables']['course_skills']['Row'] & { skills: { name: string } | null }

function parseSessionFlowText(text: string): Array<{ number: string; title: string; description: string }> {
  if (!text) return []
  const lines = text.split(/\r?\n/).filter(l => l.trim())
  const results: Array<{ number: string; title: string; description: string }> = []

  for (const line of lines) {
    const sessionMatch = line.match(/^[Ss]ession\s+(\d+)\s*[-\-:]\s*([^:]+?)(?::\s*(.+))?$/)
    if (sessionMatch) {
      results.push({
        number: sessionMatch[1],
        title: sessionMatch[2].trim(),
        description: sessionMatch[3]?.trim() || '',
      })
      continue
    }
    const numMatch = line.match(/^(\d+)\.\s+([^:]+?)(?::\s*(.+))?$/)
    if (numMatch) {
      results.push({
        number: numMatch[1],
        title: numMatch[2].trim(),
        description: numMatch[3]?.trim() || '',
      })
      continue
    }
    if (results.length > 0 && !results[results.length - 1].description) {
```

```
      results[results.length - 1].description = line.trim()
    } else {
      results.push({
        number: String(results.length + 1),
        title: line.trim(),
        description: '',
      })
    }
  }

  return results
}

function StatusBadge({ status, editStatus, windowExpiresAt }: { status: Course['status']; editStatus?: string | null; windowExpiresAt?: string | null }) {
  const isWindowActive = editStatus === 'approved' && windowExpiresAt && new Date(windowExpiresAt).getTime() > Date.now()

  if (editStatus === 'pending') {
    return <Badge className="text-[10px] font-semibold bg-amber-500 text-white border-none animate-pulse">⚠️ Edit Requested</Badge>
  }
  if (editStatus === 'submitted') {
    return <Badge className="text-[10px] font-semibold bg-indigo-600 text-white border-none">🔄 Updates Submitted</Badge>
  }
  if (isWindowActive) {
    return <Badge className="text-[10px] font-semibold bg-sky-500 text-white border-none">⏱️ Edit Window Open</Badge>
  }

  const styles: Record<Course['status'], string> = {
    published: 'bg-emerald-50 text-emerald-700 border border-emerald-200',
    pending_review: 'bg-amber-50 text-amber-700 border border-amber-200',
    draft: 'bg-slate-100 text-slate-700 border border-slate-200',
    archived: 'bg-rose-50 text-rose-700 border border-rose-200',
  }
  const labels: Record<Course['status'], string> = {
    draft: 'Draft',
    pending_review: 'Pending Review',
    published: 'Published',
    archived: 'Archived',
  }
  return <Badge className={`text-[10px] font-semibold ${styles[status]}`}>{labels[status]}</Badge>
}

function getMaterialIcon(mimeType: string | null) {
  if (!mimeType) return File
  if (mimeType.startsWith('video/')) return Video
  if (mimeType.includes('pdf')) return FileText
  if (mimeType === 'text/uri-list') return Globe
  return File
}

function formatFileSize(bytes: number | null) {
  if (!bytes) return ''
  if (bytes < 1024) return `${bytes} B`
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`
}

export function AdminCourses(): React.JSX.Element {
  const navigate = useNavigate()
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState(true)
  const [filter, setFilter] = useState<'all' | 'admin_created' | 'trainer_submitted' | 'edit_requests'>('all')
  const [selectedCourse, setSelectedCourse] = useState<Course | null>(null)
  const [detailOpen, setDetailOpen] = useState(false)
  const [detailLoading, setDetailLoading] = useState(false)
  const [materials, setMaterials] = useState<Material[]>([])
  const [sessions, setSessions] = useState<any[]>([])
  const [skills, setSkills] = useState<CourseSkill[]>([])
  const [modules, setModules] = useState<any[]>([])
  const [enrollmentCount, setEnrollmentCount] = useState(0)
  const [pendingEnrollments, setPendingEnrollments] = useState<any[]>([])
  const [updating, setUpdating] = useState<string | null>(null)
  const [isProcessingId, setIsProcessingId] = useState<string | null>(null)

  // Edit Request Management Dialog State

  const [editRequestModalCourse, setEditRequestModalCourse] = useState<Course | null>(null)
  const [editRequestHours, setEditRequestHours] = useState<number>(24)
```

```
  const [customHours, setCustomHours] = useState<string>('')
  const [adminNotes, setAdminNotes] = useState<string>('')
  const [isSubmittingEditDecision, setIsSubmittingEditDecision] = useState(false)

  const [previewMaterial, setPreviewMaterial] = useState<Material | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [docLoading, setDocLoading] = useState(false)
  const [aboutModalOpen, setAboutModalOpen] = useState(false)
  const [aboutActiveTab, setAboutActiveTab] = useState<'about' | 'outline'>('about')
  const [editMaxTrainees, setEditMaxTrainees] = useState(false)
  const [newMaxTrainees, setNewMaxTrainees] = useState<string>('')

  const handleOpenSessionDoc = async (docPath: string) => {
    if (!docPath) return
    setDocLoading(true)
    try {
      const { data, error } = await supabase.storage
        .from('materials')
        .createSignedUrl(docPath, 300)
      if (error) throw error
      if (data?.signedUrl) {
        setPreviewMaterial({
          id: 'syllabus-doc',
          file_name: 'Course Syllabus Document',
          storage_path: docPath,
          material_type: 'document',
          course_id: selectedCourse?.id || '',
          created_at: new Date().toISOString(),
          description: 'Uploaded Course Syllabus Document',
          duration_minutes: null,
          external_url: null,
          order_index: 0,
          thumbnail_url: null,
          title: 'Course Outline & Syllabus',
          topic_name: 'Outline',
          uploader_id: selectedCourse?.trainer_id || ''
        } as any)
        setPreviewUrl(data.signedUrl)
      }
    } catch {
      toast.error('Could not open syllabus document.')
    } finally {
      setDocLoading(false)
    }
  }

  const fetchCourses = useCallback(async () => {
    setLoading(true)
    try {
      const { data: coursesData, error: coursesError } = await supabase
        .from('courses')
        .select(`
          *,

          trainer:trainers!courses_trainer_id_fkey(id, full_name, email)
        `)
        .order('created_at', { ascending: false })
      if (coursesError) throw coursesError

      const { data: assignmentsData, error: assignmentsError } = await (supabase as any)
        .from('course_assignments')
        .select('id, course_id')

      const assignments = assignmentsError ? [] : (assignmentsData ?? [])

      const coursesWithAssignments = (coursesData ?? []).map(course => ({
        ...course,
        course_assignments: assignments.filter((a: any) => a.course_id === course.id)
      }))

      setCourses(coursesWithAssignments as any)
    } catch (err: any) {
      const message = err?.message || 'Failed to load courses'
      toast.error(message)
    } finally {
      setLoading(false)
    }
  }, [])

```

```
  useEffect(() => { fetchCourses() }, [fetchCourses])

  const openCourseDetail = async (course: Course) => {
    setSelectedCourse(course)
    setDetailOpen(true)
    setEditMaxTrainees(false)
    setNewMaxTrainees(course.max_trainees ? String(course.max_trainees) : '')
    setDetailLoading(true)
    try {
      const [mRes, sRes, eRes, sessRes, pRes, modRes] = await Promise.all([
        supabase.from('materials').select('*').eq('course_id', course.id).order('created_at', { ascending: false }),
        supabase.from('course_skills').select('*, skills(name)').eq('course_id', course.id),
        supabase.from('enrollments').select('*', { count: 'exact', head: true }).eq('course_id', course.id).neq('status', 'pending_approval'),
        supabase.from('course_sessions').select('*').eq('course_id', course.id).order('order_index'),
        supabase.from('enrollments').select('*').eq('course_id', course.id).eq('status', 'pending_approval'),
        (supabase as any).from('course_modules').select('*, quiz_questions:quiz_questions(*)').eq('course_id', course.id).order('order_index', { ascending: true }),
      ])

      let mergedPending: any[] = []
      if (pRes.data && pRes.data.length > 0) {
        const userIds = pRes.data.map(e => e.user_id)
        const { data: traineesData } = await supabase.from('trainees').select('id, full_name, email').in('id', userIds)
        mergedPending = pRes.data.map(e => ({
          ...e,
          trainee: traineesData?.find(t => t.id === e.user_id) || { full_name: 'Unknown Trainee', email: '' }
        }))
      }

      const rawDbModules = (modRes as any)?.data ?? []
      const rawCourseModules = (course as any).modules
      const combinedModules = rawDbModules.length > 0
        ? rawDbModules
        : (Array.isArray(rawCourseModules) ? rawCourseModules : [])

      setMaterials(mRes.data ?? [])
      setSkills(sRes.data as any ?? [])
      setEnrollmentCount(eRes.count ?? 0)
      setSessions(sessRes.data ?? [])
      setPendingEnrollments(mergedPending)
      setModules(combinedModules)
    } catch {
      toast.error('Failed to load course details')
    } finally {
      setDetailLoading(false)
    }
  }

  const handleGrantEditWindow = async () => {
    if (!editRequestModalCourse) return
    setIsSubmittingEditDecision(true)
    try {
      const duration = customHours ? parseInt(customHours, 10) : editRequestHours
      if (!duration || duration <= 0) {
        toast.error('Please specify a valid time limit in hours')
        return
      }
      const expiresAt = new Date(Date.now() + duration * 3600 * 1000).toISOString()

      const { error } = await supabase
        .from('courses')
        .update({
          edit_request_status: 'approved',
          edit_window_duration_hours: duration,
          edit_window_expires_at: expiresAt,
          admin_edit_notes: adminNotes.trim() || null,
        } as any)
        .eq('id', editRequestModalCourse.id)
      if (error) throw error

      if (editRequestModalCourse.trainer_id) {
        await supabase.from('notifications').insert({
          user_id: editRequestModalCourse.trainer_id,
          title: 'Course Edit Window Granted! ⏱️',
          message: `Admin granted you ${duration} hours to edit "${editRequestModalCourse.title}". Your window is active until ${new Date(expiresAt).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })} on ${new Date(expiresAt).toLocaleDateString()}. ${adminNotes ? `Admin notes: ${adminNotes}` : ''}`,
          type: 'course'
```

```
        })
      }

      toast.success(`Edit window of ${duration} hours granted to trainer!`)
      setEditRequestModalCourse(null)
      fetchCourses()
      if (selectedCourse?.id === editRequestModalCourse.id) {
        setSelectedCourse({
          ...selectedCourse,
          edit_request_status: 'approved',
          edit_window_duration_hours: duration,
          edit_window_expires_at: expiresAt,
          admin_edit_notes: adminNotes.trim() || null
        })
      }
    } catch (err: any) {
      toast.error(err.message || 'Failed to grant edit window')
    } finally {
      setIsSubmittingEditDecision(false)
    }
  }

  const handleRejectEditRequest = async () => {
    if (!editRequestModalCourse) return
    setIsSubmittingEditDecision(true)
    try {
      const { error } = await supabase
        .from('courses')
        .update({
          edit_request_status: 'rejected',
          admin_edit_notes: adminNotes.trim() || 'Request declined by admin',
        } as any)
        .eq('id', editRequestModalCourse.id)
      if (error) throw error

      if (editRequestModalCourse.trainer_id) {
        await supabase.from('notifications').insert({
          user_id: editRequestModalCourse.trainer_id,
          title: 'Course Edit Request Declined',
          message: `Admin declined your request to edit "${editRequestModalCourse.title}". ${adminNotes ? `Reason: ${adminNotes}` : ''}`,
          type: 'course'
        })
      }

      toast.info('Edit request declined')
      setEditRequestModalCourse(null)
      fetchCourses()
      if (selectedCourse?.id === editRequestModalCourse.id) {
        setSelectedCourse({
          ...selectedCourse,
          edit_request_status: 'rejected',
          admin_edit_notes: adminNotes.trim() || null
        })
      }
    } catch (err: any) {
      toast.error(err.message || 'Failed to decline request')
    } finally {
      setIsSubmittingEditDecision(false)
    }
  }

  const handleReApproveCourse = async (courseId: string, trainerId?: string | null, courseTitle?: string) => {
    setUpdating(courseId)
    try {
      const { error } = await supabase
        .from('courses')
        .update({
          status: 'published',
          edit_request_status: 'none',
          edit_window_expires_at: null,
          edit_request_reason: null,
          admin_edit_notes: null
        } as any)
        .eq('id', courseId)
      if (error) throw error

      if (trainerId) {
        await supabase.from('notifications').insert({
```

```
382 |           user_id: trainerId,
383 |           title: 'Course Updates Approved & Live! 🎉',
384 |           message: `Your updates for "${courseTitle || 'the course'}" have been reviewed and approved by the admin. The cours
e is now published.`,
385 |           type: 'course'
386 |         })
387 |       }
388 |
389 |       toast.success('Course updates approved and published successfully!')
390 |       fetchCourses()
391 |       if (selectedCourse?.id === courseId) {
392 |         setSelectedCourse({
393 |           ...selectedCourse,
394 |           status: 'published',
395 |           edit_request_status: 'none',
396 |           edit_window_expires_at: null
397 |         })
398 |       }
399 |     } catch (err: any) {
400 |       toast.error(err.message || 'Failed to approve course updates')
401 |     } finally {
402 |       setUpdating(null)
403 |     }
404 |   }
405 |
406 |   const handleApproval = async (enrollmentId: string, action: 'approve' | 'reject', trainee: any, course: Course) => {
407 |     setIsProcessingId(enrollmentId)
408 |     try {
409 |       const newStatus = action === 'approve' ? 'enrolled' : 'rejected'
410 |
411 |       const { error: updateError } = await supabase
412 |         .from('enrollments')
413 |         .update({ status: newStatus })
414 |         .eq('id', enrollmentId)
415 |
416 |       if (updateError) throw updateError
417 |
418 |       await supabase.from('notifications').insert({
419 |         user_id: trainee.id,
420 |         title: action === 'approve' ? 'Enrollment Approved! 🎉' : 'Enrollment Update',
421 |         message: action === 'approve'
422 |           ? `Your enrollment for ${course.title} has been approved. You can now access all course materials.`
423 |           : `Your enrollment request for ${course.title} was not approved.`,
424 |         type: 'enrollment'
425 |       })
426 |
427 |       setPendingEnrollments(prev => prev.filter(e => e.id !== enrollmentId))
428 |       if (action === 'approve') setEnrollmentCount(c => c + 1)
429 |       toast.success(`Trainee enrollment ${action === 'approve' ? 'approved' : 'rejected'}`)
430 |     } catch (err: any) {
431 |       toast.error(err.message || 'Failed to update enrollment')
432 |     } finally {
433 |       setIsProcessingId(null)
434 |     }
435 |   }
436 |
437 |   const handleUpdateMaxTrainees = async () => {
438 |     if (!selectedCourse) return
439 |     setUpdating(selectedCourse.id)
440 |     try {
441 |       const parsed = newMaxTrainees === '' ? null : parseInt(newMaxTrainees, 10)
442 |       if (newMaxTrainees !== '' && isNaN(parsed!)) {
443 |         toast.error('Please enter a valid number')
444 |         return
445 |       }
446 |       const { error } = await supabase
447 |         .from('courses')
448 |         .update({ max_trainees: parsed })
449 |         .eq('id', selectedCourse.id)
450 |       if (error) throw error
451 |       setSelectedCourse({ ...selectedCourse, max_trainees: parsed })
452 |       setCourses(courses.map(c => c.id === selectedCourse.id ? { ...c, max_trainees: parsed } : c))
453 |       setEditMaxTrainees(false)
454 |       toast.success('Course capacity updated')
455 |     } catch {
456 |       toast.error('Failed to update capacity')
457 |     } finally {
458 |       setUpdating(null)
459 |     }
```

```
  }

  const updateCourseStatus = async (id: string, status: Course['status']) => {
    setUpdating(id)
    try {
      const { error } = await supabase
        .from('courses')
        .update({ status })
        .eq('id', id)
      if (error) throw error
      setCourses(courses.map(c => c.id === id ? { ...c, status } : c))
      if (selectedCourse?.id === id) {
        setSelectedCourse({ ...selectedCourse, status })
      }
      toast.success(`Course status updated to ${status}`)
    } catch {
      toast.error('Failed to update course status')
    } finally {
      setUpdating(null)
    }
  }

  return (
    <>
```

```
<Card className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
        <CardHeader className="border-b border-slate-100 space-y-4 pb-6">
          <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
            <div>
              <CardTitle className="flex items-center gap-2 text-slate-900 text-xl font-bold">
                <BookOpen className="h-5 w-5 text-cyan-600" />
                All Courses
              </CardTitle>
              <CardDescription className="text-slate-500 text-xs mt-1">
                {courses.length} course{courses.length !== 1 ? 's' : ''} total &bull; Manage modules, sessions, assessments a
nd trainee performance.
              </CardDescription>
            </div>
            <Button
              onClick={() => navigate('/admin/courses/new')}
              className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold h-9 px-4 rounded-xl text-xs gap-1.5 shadow-sm sm:
w-auto w-full shrink-0"
            >
              <Plus className="w-4 h-4" />
              Create Course
            </Button>
          </div>

          <div className="flex flex-wrap bg-slate-100 p-1 rounded-xl border border-slate-200 w-fit gap-1">
            {(['all', 'admin_created', 'trainer_submitted', 'edit_requests'] as const).map(f => {
              const editCount = courses.filter(c => c.edit_request_status === 'pending').length
              return (
                <button
                  key={f}
                  onClick={() => setFilter(f)}
                  className={`px-3.5 py-1.5 rounded-lg text-xs font-bold transition-all flex items-center gap-1.5 ${filter ==
= f ? 'bg-white text-cyan-700 shadow-sm border border-slate-200' : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/60'
                    }`}
                >
                  {f === 'all' && 'All Courses'}
                  {f === 'admin_created' && 'Admin Created'}
                  {f === 'trainer_submitted' && 'Trainer Submitted'}
                  {f === 'edit_requests' && (
                    <>
                      <span>Edit Requests</span>
                      {editCount > 0 && (
                        <span className="px-1.5 py-0.2 bg-amber-500 text-white rounded-full text-[10px] font-bold">
                          {editCount}
                        </span>
                      )}
                    </>
                  )}
                </button>
              )
            })}
          </div>
        </CardHeader>
        <CardContent className="p-6">
          {loading ? (
```

```
535 |             <div className="flex items-center justify-center py-12">
536 |               <Loader2 className="w-5 h-5 animate-spin text-cyan-600" />
537 |             </div>
538 |           ) : courses.length === 0 ? (
539 |             <p className="text-slate-400 text-sm text-center py-12 font-medium">No courses found.</p>
540 |           ) : (
541 |             <div className="space-y-3.5">
542 |               {courses
543 |                 .filter(course => {
544 |                   const isAdminCreated = (course as any).course_assignments && (course as any).course_assignments.length > 0;
545 |                   if (filter === 'admin_created') return isAdminCreated;
546 |                   if (filter === 'trainer_submitted') return !isAdminCreated;
547 |                   if (filter === 'edit_requests') return course.edit_request_status === 'pending' || course.edit_request_stat
us === 'submitted';
548 |                   return true;
549 |                 })
550 |                 .map(course => {
551 |                   const isUrgent = (course.status === 'pending_review' && course.start_date && (new Date(course.start_date).g
etTime() < Date.now() + 30 * 24 * 60 * 60 * 1000)) || course.edit_request_status === 'pending';
552 |                   return { ...course, isUrgent };
553 |                 })
554 |                 .sort((a, b) => {
555 |                   if (a.edit_request_status === 'pending' && b.edit_request_status !== 'pending') return -1;
556 |                   if (a.edit_request_status !== 'pending' && b.edit_request_status === 'pending') return 1;
557 |                   if (a.isUrgent && !b.isUrgent) return -1;
558 |                   if (!a.isUrgent && b.isUrgent) return 1;
559 |                   return 0;
560 |                 })
561 |                 .map(course => {
562 |                   const trainer = (course as any).trainer
563 |                   return (
564 |                     <div key={course.id} className="p-5 rounded-2xl bg-white border border-slate-200/90 hover:border-slate-30
0 shadow-xs transition-all">
565 |                       <div className="flex flex-col sm:flex-row sm:items-start gap-4">
566 |                         {/* Thumbnail */}
567 |                         <div className="w-full sm:w-36 h-24 rounded-xl bg-slate-100 border border-slate-200 overflow-hidden s
hrink-0 flex items-center justify-center">
568 |                           <Thumbnail path={course.thumbnail_path} alt={course.title} fallbackIcon={<BookOpen className="w-8 h
-8 text-slate-400" />} />
569 |                         </div>
570 |
571 |                         {/* Content */}
572 |                         <div className="flex-1 min-w-0">
573 |                           <div className="flex flex-wrap items-center gap-2.5 mb-1.5">
574 |                             <button
575 |                               onClick={() => navigate(`/admin/courses/${course.id}`)}
576 |                               className="text-left hover:text-cyan-600 min-w-0 transition-colors"
577 |                             >
578 |                               <h3 className="text-base font-bold text-slate-900 truncate">{course.title}</h3>
579 |                             </button>
580 |                             <button
581 |                               type="button"
582 |                               onClick={() => {
583 |                                 setSelectedCourse(course)
584 |                                 setAboutActiveTab('about')
585 |                                 setAboutModalOpen(true)
586 |                               }}
587 |                               title="About Course & Outline"
588 |                               className="inline-flex items-center gap-1.5 px-2.5 py-0.5 rounded-full bg-cyan-50 hover:bg-cyan
-100 text-cyan-800 border border-cyan-300 text-[11px] font-bold transition-all shadow-xs cursor-pointer group shrink-0"
589 |                             >
590 |                               <Info className="w-3.5 h-3.5 text-cyan-600 group-hover:rotate-12 transition-transform" />
591 |                               <span>About</span>
592 |                             </button>
593 |                             <StatusBadge status={course.status} editStatus={course.edit_request_status} windowExpiresAt={cour
se.edit_window_expires_at} />
594 |                             {course.edit_request_status === 'pending' && (
595 |                               <Badge className="bg-amber-600 text-white text-[10px] font-bold">
596 |                                 Needs Review
597 |                               </Badge>
598 |                             )}
599 |                             {course.isUrgent && course.edit_request_status !== 'pending' && <Badge className="bg-rose-100 tex
t-rose-700 border-rose-200 text-[10px]">🚨 URGENT</Badge>}
600 |                           </div>
601 |
602 |                           {course.edit_request_reason && course.edit_request_status === 'pending' ? (
603 |                             <p className="text-xs text-amber-900 font-medium line-clamp-2 mb-2 bg-amber-100/70 px-2.5 py-1 ro
unded-md border border-amber-200">
604 |                               Reason: "{course.edit_request_reason}"
```

```
                            </p>
                          ) : (
                            <p className="text-xs text-slate-600 line-clamp-2 mb-3 leading-relaxed">{course.description || 'N
o description provided.'}</p>
                          )}

                          <div className="flex flex-wrap items-center gap-4 text-xs font-medium text-slate-400">
                            <span>{course.duration_minutes ?? 0} mins</span>
                            <span>&bull;</span>
                            <span className="capitalize">{course.course_type}</span>
                            <span>&bull;</span>
                            <span>{course.created_at ? new Date(course.created_at).toLocaleDateString() : ''}</span>
                            {course.department && (
                              <>
                                <span>&bull;</span>
                                <span>{course.department}</span>
                              </>
                            )}
                            {trainer?.full_name && (
                              <>
                                <span>&bull;</span>
                                <span className="text-cyan-700 font-semibold">Instructor: {trainer.full_name}</span>
                              </>
                            )}
                          </div>
                        </div>

                        {/* Action Buttons Matching Trainer Role */}
                        <div className="flex flex-wrap gap-2 shrink-0 sm:self-center">
                          <Button
                            size="sm"
                            variant="outline"
                            className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                            onClick={() => navigate(`/admin/courses/${course.id}`)}
                          >
                            <BookOpen className="w-3.5 h-3.5 mr-1 text-cyan-600" /> Manage
                          </Button>
                          <Button
                            size="sm"
                            variant="outline"
                            className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                            onClick={() => navigate(`/admin/courses/${course.id}/edit`)}
                          >
                            <Edit3 className="w-3.5 h-3.5 mr-1 text-slate-600" /> Edit
                          </Button>
                          <Button
                            size="sm"
                            variant="outline"
                            className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                            onClick={() => navigate(`/admin/courses/${course.id}/assessments`)}
                          >
                            <Target className="w-3.5 h-3.5 mr-1 text-emerald-600" /> Assess
                          </Button>
                          <Button
                            size="sm"
                            variant="outline"
                            className="border-slate-200 text-slate-700 hover:bg-slate-50 font-semibold"
                            onClick={() => navigate(`/admin/courses/${course.id}/performance`)}
                          >
                            <BarChart3 className="w-3.5 h-3.5 mr-1 text-blue-600" /> Stats
                          </Button>

                          {/* Additional Admin Status Management Dropdown */}
                          <DropdownMenu>
                            <DropdownMenuTrigger asChild>
                              <Button variant="ghost" size="icon" className="h-8 w-8 text-slate-400 rounded-xl hover:bg-slate
-100">

                                <MoreHorizontal className="h-4 w-4" />
                              </Button>
                            </DropdownMenuTrigger>
                            <DropdownMenuContent align="end" className="bg-white border-slate-200 rounded-2xl shadow-lg text-
slate-800">

                              {course.edit_request_status === 'pending' && (
                                <DropdownMenuItem
                                  onClick={() => {
                                    setEditRequestModalCourse(course)
                                    setEditRequestHours(24)
                                    setCustomHours('')
                                    setAdminNotes('')
```

```
                                    }}
                                    className="text-amber-700 font-bold"
                                  >
                                    Review Edit Request
                                  </DropdownMenuItem>
                                )}
                                {course.edit_request_status === 'submitted' && (
                                  <DropdownMenuItem
                                    onClick={() => handleReApproveCourse(course.id, course.trainer_id, course.title)}
                                    className="text-indigo-700 font-bold"
                                  >
                                    Re-Approve & Publish Updates
                                  </DropdownMenuItem>
                                )}
                                {course.status === 'pending_review' && (
                                  <DropdownMenuItem onClick={() => updateCourseStatus(course.id, 'published')} className="text-
emerald-700 font-medium">
                                    Approve & Publish
                                  </DropdownMenuItem>
                                )}
                                {course.status === 'pending_review' && (
                                  <DropdownMenuItem onClick={() => updateCourseStatus(course.id, 'draft')}>
                                    Return to Draft
                                  </DropdownMenuItem>
                                )}
                                {course.status === 'published' && (
                                  <DropdownMenuItem onClick={() => updateCourseStatus(course.id, 'archived')} className="text-r
ose-600">
                                    Archive
                                  </DropdownMenuItem>
                                )}
                                {course.status === 'archived' && (
                                  <DropdownMenuItem onClick={() => updateCourseStatus(course.id, 'published')}>
                                    Re-publish
                                  </DropdownMenuItem>
                                )}
                                {course.status === 'draft' && (
                                  <DropdownMenuItem onClick={() => updateCourseStatus(course.id, 'published')}>
                                    Publish Directly
                                  </DropdownMenuItem>
                                )}
                              </DropdownMenuContent>
                            </DropdownMenu>
                          </div>
                        </div>
                      </div>
                    )
                  })}
                </div>
              )}
            </CardContent>
          </Card>

          {/* Course Detail Dialog */}
          <Dialog open={detailOpen} onOpenChange={setDetailOpen}>
            <DialogContent className="max-w-3xl max-h-[88vh] overflow-y-auto bg-white border-slate-200 rounded-3xl shadow-2xl">
              {selectedCourse && (
                <>
                  <DialogHeader>
                    <DialogTitle className="text-slate-900 flex items-center justify-between gap-2 font-bold w-full pr-6">
                      <div className="flex items-center gap-2">
                        <BookOpen className="w-5 h-5 text-cyan-600" />
                        Course Overview & Review
                      </div>
                      <div className="flex items-center gap-2">
                        <Button
                          variant="outline"
                          size="sm"
                          className="border-cyan-200 text-cyan-700 hover:bg-cyan-50 h-8 text-xs font-bold"
                          onClick={() => window.open(`/trainer/courses/${selectedCourse.id}/learn`, '_blank')}
                        >
                          <PlayCircle className="w-3.5 h-3.5 mr-1" /> Preview Trainee Player ↗
                        </Button>
                        <Button variant="outline" size="sm" className="h-8 text-xs font-medium" onClick={() => navigate(`/admin/c
ourses/${selectedCourse.id}/edit`)}>
                          Edit Course
                        </Button>
                      </div>
                    </DialogTitle>
```

```
757 |                 </DialogHeader>
758 |
759 |                 {detailLoading ? (
760 |                   <div className="flex items-center justify-center py-12">
761 |                     <Loader2 className="w-5 h-5 animate-spin text-cyan-600" />
762 |                   </div>
763 |                 ) : (
764 |                   <div className="space-y-5">
765 |                     {/* Edit Request Action Banner if pending */}
766 |                     {selectedCourse.edit_request_status === 'pending' && (
767 |                       <div className="p-4 rounded-2xl bg-amber-50 border border-amber-300 ring-1 ring-amber-200/50 space-y-3">
768 |                         <div className="flex items-start justify-between gap-3">
769 |                           <div className="flex items-center gap-2">
770 |                             <Timer className="w-5 h-5 text-amber-600 shrink-0" />
771 |                             <div>
772 |                               <h4 className="text-sm font-bold text-amber-950">Trainer Requested Edit Permission</h4>
773 |                               <p className="text-xs text-amber-700">
774 |                                 Requested {selectedCourse.edit_request_at ? formatDistanceToNow(new Date(selectedCourse.edit_re
quest_at), { addSuffix: true }) : 'recently'}
775 |                               </p>
776 |                             </div>
777 |                           </div>
778 |                           <Button
779 |                             size="sm"
780 |                             onClick={() => {
781 |                               setEditRequestModalCourse(selectedCourse)
782 |                               setEditRequestHours(24)
783 |                               setCustomHours('')
784 |                               setAdminNotes('')
785 |                             }}
786 |                             className="bg-amber-600 hover:bg-amber-700 text-white font-bold text-xs h-8 shadow-xs shrink-0"
787 |                           >
788 |                             Review & Set Time Limit
789 |                           </Button>
790 |                         </div>
791 |                         <div className="bg-white p-3 rounded-xl border border-amber-200 text-xs text-slate-800">
792 |                           <span className="font-semibold text-slate-500 uppercase tracking-wider text-[10px] block mb-1">Reason
Stated by Trainer:</span>
793 |                           <p className="italic">{selectedCourse.edit_request_reason || 'No description provided.'}</p>
794 |                         </div>
795 |                       </div>
796 |                     )}
797 |
798 |                     {/* Changes submitted banner */}
799 |                     {selectedCourse.edit_request_status === 'submitted' && (
800 |                       <div className="p-4 rounded-2xl bg-indigo-50 border border-indigo-200 flex items-center justify-between g
ap-4">
801 |                         <div className="flex items-center gap-3">
802 |                           <Sparkles className="w-5 h-5 text-indigo-600 shrink-0" />
803 |                           <div>
804 |                             <h4 className="text-sm font-bold text-indigo-950">Trainer Finished Updates</h4>
805 |                             <p className="text-xs text-indigo-700">Course changes submitted for final admin review & re-publica
tion.</p>
806 |                           </div>
807 |                         </div>
808 |                         <Button
809 |                           size="sm"
810 |                           onClick={() => handleReApproveCourse(selectedCourse.id, selectedCourse.trainer_id, selectedCourse.tit
le)}
811 |                           disabled={updating === selectedCourse.id}
812 |                           className="bg-indigo-600 hover:bg-indigo-700 text-white font-bold text-xs h-8 shadow-xs shrink-0"
813 |                         >
814 |                           {updating === selectedCourse.id ? <Loader2 className="w-3.5 h-3.5 mr-1 animate-spin" /> : <CheckCircl
e2 className="w-3.5 h-3.5 mr-1" />}
815 |                           Re-Approve & Publish
816 |                         </Button>
817 |                       </div>
818 |                     )}
819 |
820 |                     {/* Thumbnail + Basic Info */}
821 |                     <div className="flex gap-4">
822 |                       {selectedCourse.thumbnail_path && (
823 |                         <div className="w-32 h-24 rounded-2xl bg-slate-100 border border-slate-200 overflow-hidden shrink-0">
824 |                           <Thumbnail path={selectedCourse.thumbnail_path} alt={selectedCourse.title} />
825 |                         </div>
826 |                       )}
827 |                       <div className="flex-1">
828 |                         <div className="flex flex-wrap items-center gap-2.5 mb-2">
829 |                           <h3 className="text-lg font-bold text-slate-900">{selectedCourse.title}</h3>
```

```
                        <button
                          type="button"
                          onClick={() => {
                            setAboutActiveTab('about')
                            setAboutModalOpen(true)
                          }}
                          title="About Course & Outline"
                          className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-50 hover:bg-cyan-100 tex
t-cyan-800 border border-cyan-300 text-xs font-bold transition-all shadow-xs cursor-pointer group shrink-0"
                        >
                          <Info className="w-3.5 h-3.5 text-cyan-600 group-hover:rotate-12 transition-transform" />
                          <span>About</span>
                        </button>
                        <StatusBadge status={selectedCourse.status} editStatus={selectedCourse.edit_request_status} windowExp
iresAt={selectedCourse.edit_window_expires_at} />
                      </div>
                      <p className="text-sm text-slate-600 mb-2">{selectedCourse.description}</p>
                      <div className="flex flex-wrap gap-3 text-xs text-slate-500">
                        <span className="capitalize">{selectedCourse.course_type} Training</span>
                        <span>{selectedCourse.department || 'General'}</span>
                        <span>{selectedCourse.duration_minutes ? `${selectedCourse.duration_minutes} min` : 'Self-paced'}</sp
an>
                        <span>Pass: {selectedCourse.passing_score}%</span>
                        <span>{enrollmentCount} enrolled</span>
                      </div>
                    </div>
                  </div>

                  {/* Course Modules Section */}
                  <div className="space-y-3">
                    <div className="flex items-center justify-between">
                      <div className="flex items-center gap-2 text-xs font-bold text-slate-700 uppercase tracking-wider">
                        <BookOpen className="w-4 h-4 text-cyan-600" /> Course Modules & Content Sequence ({modules.length})
                      </div>
                      <Badge variant="outline" className="text-[10px] bg-slate-50 text-slate-600 border-slate-200">
                        Sequential Progression (≥80% quiz gate)
                      </Badge>
                    </div>
                    <div className="bg-slate-50/80 border border-slate-200 rounded-2xl p-4 space-y-3">
                      {modules.length === 0 ? (
                        <p className="text-xs text-slate-400 text-center py-4">No structured modules added yet.</p>
                      ) : (
                        modules.map((mod: any, mIdx: number) => {
                          const rawItems = mod.items || mod.content_items || mod.content_steps || []
                          const quizQuestions = mod.quiz_questions || []
                          return (
                            <div key={mod.id || mIdx} className="bg-white rounded-xl border border-slate-200 p-3.5 space-y-2.
5 shadow-xs">
                              <div className="flex items-start justify-between gap-2">
                                <div className="flex items-center gap-2.5">
                                  <div className="w-6 h-6 rounded-lg bg-cyan-100 text-cyan-800 font-bold text-xs flex items-c
enter justify-center shrink-0">
                                    {mIdx + 1}
                                  </div>
                                  <div>
                                    <h5 className="text-sm font-bold text-slate-900">{mod.title || `Module ${mIdx + 1}`}</h5>
                                    {mod.description && <p className="text-xs text-slate-500 line-clamp-2">{mod.description}
</p>}
                                  </div>
                                </div>
                                <div className="flex items-center gap-1.5 shrink-0 flex-wrap justify-end">
                                  {rawItems.length > 0 && (
                                    <Badge variant="outline" className="text-[10px] bg-slate-50 border-slate-200 text-slate-6
00">
                                      {rawItems.length} content item{rawItems.length !== 1 ? 's' : ''}
                                    </Badge>
                                  )}
                                  {quizQuestions.length > 0 && (
                                    <Badge className="bg-amber-50 text-amber-800 border border-amber-200 text-[10px] font-sem
ibold">
                                      {quizQuestions.length} Quiz Qs (≥80% Pass Gate)
                                    </Badge>
                                  )}
                                </div>
                              </div>

                              {/* Content Items Pills */}
                              {rawItems.length > 0 && (
                                <div className="flex flex-wrap gap-1.5 pt-1">
```

```
                                {rawItems.map((st: any, sIdx: number) => (
                                  <span key={st.id || sIdx} className="inline-flex items-center gap-1 px-2 py-0.5 rounded-md bg-slate-100 border border-slate-200 text-[11px] text-slate-700 font-medium">
                                    <span className="text-[9px] text-slate-400 font-bold">#{sIdx + 1}</span>
                                    {(st.type === 'video' || st.step_type === 'video') && <Video className="w-3 h-3 text-rose-500" />}
                                    {(st.type === 'photo' || st.step_type === 'photo') && <ImageIcon className="w-3 h-3 text-cyan-600" />}
                                    {(st.type === 'notes' || st.type === 'document' || st.step_type === 'text') && <FileText className="w-3 h-3 text-emerald-600" />}
                                    {st.type === 'quiz' && <HelpCircle className="w-3 h-3 text-purple-600" />}
                                    <span className="truncate max-w-[180px]">{st.title || st.type || st.step_type}</span>
                                  </span>
                                ))}
                              </div>
                            )}
                          </div>
                        )
                      })
                    )}
                  </div>
                </div>

                {/* Trainer Suggestion */}
                {selectedCourse.trainer_suggestion && (
                  <div className="bg-amber-50 border border-amber-200 p-4 rounded-xl space-y-2">
                    <div className="flex items-center gap-2 text-xs font-bold text-amber-800 uppercase tracking-wider">
                      <AlertCircle className="w-4 h-4 text-amber-600" /> Notice / Suggestion from Trainer
                    </div>
                    <p className="text-sm text-amber-900 leading-relaxed whitespace-pre-wrap">
                      {selectedCourse.trainer_suggestion}
                    </p>
                  </div>
                )}

                {/* Capacity & Limits */}
                <div className="space-y-2">
                  <div className="flex items-center gap-2 text-xs font-medium text-slate-500 uppercase tracking-wider">
                    <Users className="w-3.5 h-3.5" /> Capacity & Limits
                  </div>
                  <div className="bg-slate-50 rounded-lg p-4 flex items-center justify-between border border-slate-200">
                    <div>
                      <span className="text-slate-500 block text-xs">Max Trainees</span>
                      {editMaxTrainees ? (
                        <div className="flex items-center gap-2 mt-1">
                          <Input
                            type="number"
                            value={newMaxTrainees}
                            onChange={e => setNewMaxTrainees(e.target.value)}
                            className="w-24 h-8 text-sm bg-white"
                          />
                          <Button size="sm" className="h-8 bg-cyan-600 hover:bg-cyan-700 text-white" onClick={handleUpdateMaxTrainees} disabled={!!updating}>Save</Button>
                          <Button size="sm" variant="ghost" className="h-8" onClick={() => {
                            setEditMaxTrainees(false)
                            setNewMaxTrainees(selectedCourse.max_trainees ? String(selectedCourse.max_trainees) : '')
                          }}>Cancel</Button>
                        </div>
                      ) : (
                        <div className="flex items-center gap-2 mt-1">
                          <span className="text-slate-900 font-medium">{selectedCourse.max_trainees ?? 'Unlimited'}</span>
                          <Button size="sm" variant="ghost" className="h-6 px-2 text-xs text-cyan-700" onClick={() => setEditMaxTrainees(true)}>
                            Edit
                          </Button>
                        </div>
                      )}
                    </div>
                  </div>
                </div>

                {/* Pending Enrollments */}
                <div className="space-y-2">
                  <div className="flex items-center gap-2 text-xs font-medium text-amber-700 uppercase tracking-wider">
                    <Clock className="w-3.5 h-3.5" /> Pending Enrollments ({pendingEnrollments.length})
                  </div>
                  <div className="bg-amber-50/50 rounded-lg p-3 border border-amber-200 space-y-2">
                    {pendingEnrollments.length > 0 ? (
                      pendingEnrollments.map((enrollment) => (
```

```
                          <div key={enrollment.id} className="p-3 rounded-lg bg-white border border-amber-200 flex items-cent
er justify-between">
                            <div>
                              <h4 className="text-xs font-semibold text-slate-900">{enrollment.trainee?.full_name || 'Unknown
Trainee'}</h4>
                              <p className="text-[10px] text-slate-500">{enrollment.trainee?.email}</p>
                            </div>
                            <div className="flex items-center gap-2">
                              <Button
                                size="sm"
                                variant="outline"
                                className="border-emerald-200 text-emerald-700 hover:bg-emerald-50 h-7 text-[10px] px-2"
                                disabled={isProcessingId === enrollment.id}
                                onClick={() => handleApproval(enrollment.id, 'approve', enrollment.trainee, selectedCourse!)}
                              >
                                {isProcessingId === enrollment.id ? <Loader2 className="w-3 h-3 animate-spin" /> : <CheckCirc
le2 className="w-3 h-3 mr-1" />}
                                Approve
                              </Button>
                              <Button
                                size="sm"
                                variant="outline"
                                className="border-rose-200 text-rose-700 hover:bg-rose-50 h-7 text-[10px] px-2"
                                disabled={isProcessingId === enrollment.id}
                                onClick={() => handleApproval(enrollment.id, 'reject', enrollment.trainee, selectedCourse!)}
                              >
                                {isProcessingId === enrollment.id ? <Loader2 className="w-3 h-3 animate-spin" /> : <XCircle c
lassName="w-3 h-3 mr-1" />}
                                Reject
                              </Button>
                            </div>
                          </div>
                        ))
                      ) : (
                        <p className="text-[10px] text-amber-800/70 p-2 text-center">No pending enrollment requests at this t
ime.</p>
                      )}
                    </div>
                  </div>

                  {/* Schedule & Dates */}
                  {(selectedCourse.start_date || selectedCourse.end_date || selectedCourse.meet_link) && (
                    <div className="space-y-2">
                      <div className="flex items-center gap-2 text-xs font-medium text-slate-500 uppercase tracking-wider">
                        <Clock className="w-3.5 h-3.5" /> Schedule & Dates
                      </div>
                      <div className="bg-slate-50 border border-slate-200 rounded-lg p-4 space-y-2 text-sm">
                        {(selectedCourse.start_date || selectedCourse.end_date) && (
                          <div className="flex flex-wrap items-center gap-2">
                            <span className="text-slate-500 w-24 block">Course Span:</span>
                            <span className="text-slate-900 font-medium">
                              {selectedCourse.start_date ? new Date(selectedCourse.start_date).toLocaleDateString() : 'TBD'}
-{' '}
                              {selectedCourse.end_date ? new Date(selectedCourse.end_date).toLocaleDateString() : 'TBD'}
                            </span>
                            {selectedCourse.start_date && selectedCourse.end_date && (
                              <span className="text-xs text-slate-500 ml-2">
                                ({Math.max(1, Math.ceil((new Date(selectedCourse.end_date).getTime() - new Date(selectedCours
e.start_date).getTime()) / (1000 * 60 * 60 * 24)))} days)
                              </span>
                            )}
                          </div>
                        )}
                        {selectedCourse.meet_link && (
                          <div className="flex items-center gap-2">
                            <span className="text-slate-500 w-24 block">Meeting Link:</span>
                            <a href={selectedCourse.meet_link} target="_blank" rel="noreferrer" className="text-blue-600 hove
r:underline break-all">
                              {selectedCourse.meet_link}
                            </a>
                          </div>
                        )}
                      </div>
                    </div>
                  )}

                  {/* Course Materials */}
                  <div className="space-y-2">
                    <div className="flex items-center justify-between">
```

```
                        <div className="flex items-center gap-2 text-xs font-medium text-slate-500 uppercase tracking-wider">
                          <FileText className="w-3.5 h-3.5" /> Course Materials ({materials.length})
                        </div>
                        <span className="text-[11px] text-emerald-600 font-medium bg-emerald-50 px-2 py-0.5 rounded-md border b
order-emerald-100">
                          Always uploadable & editable by trainer
                        </span>
                      </div>
                      <div className="bg-slate-50 border border-slate-200 rounded-lg p-4">
                        {materials.length > 0 ? (
                          <div className="space-y-2">
                            {materials.map(mat => {
                              const isLink = mat.material_type === 'link' || mat.material_type === 'video'
                              const Icon = getMaterialIcon(mat.mime_type)
                              return (
                                <div key={mat.id} className="flex items-center gap-3 p-2.5 rounded-lg bg-white border border-sl
ate-200">
                                  <div className="w-8 h-8 rounded-md bg-cyan-50 flex items-center justify-center shrink-0">
                                    <Icon className="w-4 h-4 text-cyan-600" />
                                  </div>
                                  <div className="flex-1 min-w-0">
                                    <p className="text-xs font-medium text-slate-900 truncate">{mat.file_name}</p>
                                    <p className="text-[10px] text-slate-500">
                                      {isLink ? mat.url : formatFileSize(mat.file_size)}
                                    </p>
                                  </div>
                                  <Badge className="text-[9px] h-4 bg-slate-100 text-slate-700 border-slate-200">
                                    {mat.material_type || 'file'}
                                  </Badge>
                                  {isLink && mat.url ? (
                                    <Button
                                      variant="ghost"
                                      size="sm"
                                      className="h-7 px-2 text-slate-500 hover:text-slate-900"
                                      onClick={() => mat.url && window.open(mat.url, '_blank')}
                                    >
                                      <ExternalLink className="w-3.5 h-3.5" />
                                    </Button>
                                  ) : mat.storage_path ? (
                                    <>
                                      <Button
                                        variant="ghost"
                                        size="sm"
                                        className="h-7 px-2 text-slate-500 hover:text-slate-900"
                                        onClick={async () => {
                                          try {
                                            const { data, error } = await supabase.storage.from('materials').createSignedUrl(ma
t.storage_path!, 3600)
                                            if (error) throw error
                                            if (data?.signedUrl) {
                                              setPreviewUrl(data.signedUrl)
                                              setPreviewMaterial(mat)
                                            }
                                          } catch {
                                            toast.error('Failed to preview file')
                                          }
                                        }}
                                      >
                                        <Eye className="w-3.5 h-3.5" />
                                      </Button>
                                      <Button
                                        variant="ghost"
                                        size="sm"
                                        className="h-7 px-2 text-slate-500 hover:text-slate-900"
                                        onClick={async () => {
                                          try {
                                            const { data, error } = await supabase.storage.from('materials').createSignedUrl(ma
t.storage_path!, 60, { download: true })
                                            if (error) throw error
                                            if (data?.signedUrl) window.open(data.signedUrl, '_blank')
                                          } catch {
                                            toast.error('Failed to download file')
                                          }
                                        }}
                                        title="Download"
                                      >
                                        <Download className="w-3.5 h-3.5" />
                                      </Button>
                                    </>
```

```
                                ) : null}
                              </div>
                            )
                          })}
                        </div>
                      ) : (
                        <p className="text-xs text-slate-400 text-center py-4">No materials uploaded yet</p>
                      )}
                    </div>
                  </div>

                  {/* Action Buttons */}
                  <DialogFooter className="gap-2">
                    {selectedCourse.edit_request_status === 'pending' && (
                      <Button
                        onClick={() => {
                          setEditRequestModalCourse(selectedCourse)
                          setEditRequestHours(24)
                          setCustomHours('')
                          setAdminNotes('')
                        }}
                        className="bg-amber-600 hover:bg-amber-700 text-white font-bold"
                      >
                        <Timer className="w-4 h-4 mr-2" />
                        Review Edit Request & Set Time Limit
                      </Button>
                    )}
                    {selectedCourse.edit_request_status === 'submitted' && (
                      <Button
                        onClick={() => handleReApproveCourse(selectedCourse.id, selectedCourse.trainer_id, selectedCourse.tit
le)}

                        disabled={!!updating}
                        className="bg-indigo-600 hover:bg-indigo-700 text-white font-bold"
                      >
                        {updating === selectedCourse.id ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <CheckCircle2 c
lassName="w-4 h-4 mr-2" />}
                        Re-Approve & Publish Updates
                      </Button>
                    )}
                    {selectedCourse.status === 'pending_review' && selectedCourse.edit_request_status !== 'submitted' && (
                      <>
                        <Button
                          variant="outline"
                          onClick={() => updateCourseStatus(selectedCourse.id, 'draft')}
                          disabled={!!updating}
                          className="border-slate-200 text-slate-700"
                        >
                          {updating === selectedCourse.id ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
                          Return to Draft
                        </Button>
                        <Button
                          onClick={() => updateCourseStatus(selectedCourse.id, 'published')}
                          disabled={!!updating}
                          className="bg-emerald-600 hover:bg-emerald-700 text-white font-bold"
                        >
                          {updating === selectedCourse.id ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
                          Approve & Publish
                        </Button>
                      </>
                    )}
                    {selectedCourse.status === 'published' && (
                      <Button
                        variant="outline"
                        onClick={() => updateCourseStatus(selectedCourse.id, 'archived')}
                        disabled={!!updating}
                        className="border-rose-200 text-rose-700 hover:bg-rose-50"
                      >
                        {updating === selectedCourse.id ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
                        Archive
                      </Button>
                    )}
                    {selectedCourse.status === 'draft' && (
                      <Button
                        onClick={() => updateCourseStatus(selectedCourse.id, 'published')}
                        disabled={!!updating}
                        className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold"
                      >
                        {updating === selectedCourse.id ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null}
                        Publish Directly
```

```
                      </Button>
                    )}
                  </DialogFooter>
                </div>
              )}
            </>
          )}
        </DialogContent>
      </Dialog>

      {/* Admin Grant Edit Window Modal */}
      <Dialog open={!!editRequestModalCourse} onOpenChange={open => !open && setEditRequestModalCourse(null)}>
        <DialogContent className="max-w-lg bg-white border-slate-200 rounded-3xl shadow-2xl p-6">
          {editRequestModalCourse && (
            <div className="space-y-5">
              <DialogHeader>
                <DialogTitle className="text-slate-900 flex items-center gap-2 font-bold">
                  <Timer className="w-5 h-5 text-amber-600" />
                  Review Course Edit Request
                </DialogTitle>
              </DialogHeader>

              <div className="space-y-4">
                <div className="p-3.5 rounded-xl bg-slate-50 border border-slate-200 text-xs space-y-1">
                  <div className="font-bold text-slate-900 text-sm">{editRequestModalCourse.title}</div>
                  <div className="text-slate-500">
                    Trainer: <span className="font-medium text-slate-800">{(editRequestModalCourse as any).trainer?.full_name
|| 'Assigned Trainer'}</span>
                  </div>
                  {editRequestModalCourse.edit_request_at && (
                    <div className="text-slate-500">
                      Requested: <span className="text-slate-700">{formatDistanceToNow(new Date(editRequestModalCourse.edit_r
equest_at), { addSuffix: true })}</span>
                    </div>
                  )}
                </div>

                {/* Trainer's Stated Reason */}
                <div className="p-4 rounded-xl bg-amber-50 border border-amber-200 space-y-1.5">
                  <div className="flex items-center gap-1.5 text-xs font-bold text-amber-900 uppercase tracking-wider">
                    <AlertCircle className="w-4 h-4 text-amber-600" /> Stated Edit Reason / Changes:
                  </div>
                  <p className="text-sm text-amber-950 whitespace-pre-wrap leading-relaxed">
                    {editRequestModalCourse.edit_request_reason || 'No description provided.'}
                  </p>
                </div>

                {/* Time Limit Selector */}
                <div className="space-y-2">
                  <label className="text-xs font-bold text-slate-700 uppercase tracking-wider block">
                    Set Edit Window Duration (Time Limit) *
                  </label>
                  <div className="grid grid-cols-3 gap-2">
                    {[
                      { label: '2 Hours', val: 2 },
                      { label: '6 Hours', val: 6 },
                      { label: '12 Hours', val: 12 },
                      { label: '24 Hours (1 Day)', val: 24 },
                      { label: '48 Hours (2 Days)', val: 48 },
                      { label: '7 Days (1 Week)', val: 168 },
                    ].map(opt => (
                      <button
                        type="button"
                        key={opt.val}
                        onClick={() => {
                          setEditRequestHours(opt.val)
                          setCustomHours('')
                        }}
                        className={`p-2.5 rounded-xl text-xs font-bold border transition-all text-center ${editRequestHours =
== opt.val && !customHours
                            ? 'bg-amber-500 text-white border-amber-600 shadow-xs ring-2 ring-amber-200'
                            : 'bg-white text-slate-700 border-slate-200 hover:bg-slate-50'
                          }`}
                      >
                        {opt.label}
                      </button>
                    ))}
                  </div>
```

```
                  <div className="flex items-center gap-2 pt-1">
                    <span className="text-xs text-slate-500 font-medium">Or custom hours:</span>
                    <Input
                      type="number"
                      placeholder="e.g. 36"
                      value={customHours}
                      onChange={e => {
                        setCustomHours(e.target.value)
                        if (e.target.value) {
                          setEditRequestHours(parseInt(e.target.value, 10) || 24)
                        }
                      }}
                      className="w-28 h-8 text-xs bg-white"
                      min={1}
                      max={720}
                    />
                    <span className="text-xs text-slate-500">hours</span>
                  </div>

                  <p className="text-[11px] text-slate-500">
                    The trainer will have a live countdown timer until the window expires. After editing, they will submit th
eir updates for your re-approval.
                  </p>
                </div>

                {/* Admin Notes */}
                <div className="space-y-1.5">
                  <label className="text-xs font-bold text-slate-700 uppercase tracking-wider block">
                    Admin Instructions / Notes (Optional)
                  </label>
                  <Textarea
                    placeholder="e.g. Please update only the Module 3 quiz questions and syllabus notes."
                    value={adminNotes}
                    onChange={e => setAdminNotes(e.target.value)}
                    className="text-xs min-h-[70px] bg-white"
                  />
                </div>
              </div>

              <DialogFooter className="gap-2 sm:justify-between pt-2">
                <Button
                  type="button"
                  variant="outline"
                  onClick={handleRejectEditRequest}
                  disabled={isSubmittingEditDecision}
                  className="border-rose-200 text-rose-700 hover:bg-rose-50 font-semibold text-xs"
                >
                  {isSubmittingEditDecision ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <XCircle className="w-
3.5 h-3.5 mr-1" />}
                  Decline Request
                </Button>
                <div className="flex gap-2">
                  <Button
                    type="button"
                    variant="ghost"
                    onClick={() => setEditRequestModalCourse(null)}
                    className="text-xs"
                  >
                    Cancel
                  </Button>
                  <Button
                    type="button"
                    onClick={handleGrantEditWindow}
                    disabled={isSubmittingEditDecision}
                    className="bg-amber-600 hover:bg-amber-700 text-white font-bold text-xs shadow-md shadow-amber-500/20"
                  >
                    {isSubmittingEditDecision ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Check className="w-
3.5 h-3.5 mr-1" />}
                    Grant {customHours || editRequestHours}h Window
                  </Button>
                </div>
              </DialogFooter>
            </div>
          )}
        </DialogContent>
      </Dialog>

      {/* About Course & Course Outline Modal */}
      <Dialog open={aboutModalOpen} onOpenChange={setAboutModalOpen}>
```

```
        <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto rounded-3xl p-6 md:p-8 bg-white border border-slate-200 shadow-2xl">
          <DialogHeader className="pb-4 border-b border-slate-100">
            <div className="flex flex-wrap items-center justify-between gap-3">
              <div className="space-y-1 pr-6">
                <div className="flex items-center gap-2">
                  <Badge variant="outline" className="text-[10px] font-bold uppercase text-cyan-800 bg-cyan-50 border-cyan-200">
                    {selectedCourse?.course_type || 'Standard'} Program
                  </Badge>
                  {selectedCourse?.department && (
                    <Badge variant="outline" className="text-[10px] font-semibold text-slate-600 border-slate-200">
                      {selectedCourse.department}
                    </Badge>
                  )}
                </div>
                <DialogTitle className="text-xl md:text-2xl font-black text-slate-900 leading-tight">
                  {selectedCourse?.title}
                </DialogTitle>
                <p className="text-xs text-slate-500 font-medium">
                  {selectedCourse?.duration_minutes ? `${Math.floor(selectedCourse.duration_minutes / 60)} Hours` : 'Self-Paced'} &bull; Pass Gate: {selectedCourse?.passing_score || 80}%
                </p>
              </div>
            </div>

            {/* Navigation Tabs */}
            <div className="flex items-center gap-2 pt-4">
              <button
                type="button"
                onClick={() => setAboutActiveTab('about')}
                className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
                  aboutActiveTab === 'about'
                    ? 'bg-cyan-600 text-white shadow-xs'
                    : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                }`}
              >
                <BookOpen className="w-3.5 h-3.5" />
                About This Course
              </button>
              <button
                type="button"
                onClick={() => setAboutActiveTab('outline')}
                className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer ${
                  aboutActiveTab === 'outline'
                    ? 'bg-cyan-600 text-white shadow-xs'
                    : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                }`}
              >
                <Layers className="w-3.5 h-3.5" />
                Course Outline & Roadmap
              </button>
            </div>
          </DialogHeader>

          {/* TAB 1: ABOUT THIS COURSE */}
          {aboutActiveTab === 'about' && (
            <div className="space-y-6 pt-2">
              {/* Description */}
              <div className="space-y-2">
                <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                  <BookOpen className="w-3.5 h-3.5 text-cyan-600" /> Course Overview
                </h4>
                <p className="text-sm text-slate-700 leading-relaxed bg-slate-50 p-4 rounded-2xl border border-slate-200/80 whitespace-pre-line">
                  {selectedCourse?.description || 'Comprehensive competency-based training designed for operational excellence.'}
                </p>
              </div>

              {/* Objectives */}
              {selectedCourse?.learning_objectives && (
                <div className="space-y-2">
                  <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                    <Target className="w-3.5 h-3.5 text-cyan-600" /> Key Learning Objectives
                  </h4>
                  <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200/80 space-y-2 text-xs sm:text-sm text-slate-700 leading-relaxed whitespace-pre-line font-normal">
                    {getFormattedObjectives(selectedCourse.learning_objectives)}
```

```
                  </div>
                </div>
              )}

              {/* Key Course Specifications Grid */}
              <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
                <div className="p-3.5 rounded-2xl bg-cyan-50/50 border border-cyan-200/60">
                  <span className="text-[10px] font-bold text-cyan-800 uppercase tracking-wider block">Passing Threshold</spa
n>
                  <span className="text-sm font-extrabold text-cyan-950 mt-0.5 block">{selectedCourse?.passing_score || 80}%
Overall</span>
                </div>
                <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                  <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Duration</span>
                  <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{selectedCourse?.duration_minutes ? `
${Math.floor(selectedCourse.duration_minutes / 60)} Hours` : 'Flexible'}</span>
                </div>
                <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                  <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Max Trainees</span>
                  <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{selectedCourse?.max_trainees || 'Unli
mited'} Seats</span>
                </div>
              </div>
            </div>
          )}

          {/* TAB 2: COURSE OUTLINE & ROADMAP */}
          {aboutActiveTab === 'outline' && (
            <div className="space-y-6 pt-2">
              {/* Syllabus Document Download / Preview */}
              {selectedCourse?.session_flow_document_path && (
                <div className="p-4 rounded-2xl bg-cyan-50/70 border border-cyan-200 flex items-center justify-between gap-
3">
                  <div className="flex items-center gap-2.5 min-w-0">
                    <FileText className="w-5 h-5 text-cyan-700 shrink-0" />
                    <div>
                      <h5 className="text-xs font-bold text-cyan-950">Official Syllabus Document</h5>
                      <p className="text-[11px] text-cyan-700">Detailed curriculum plan and reading materials</p>
                    </div>
                  </div>
                  <Button
                    size="sm"
                    onClick={() => handleOpenSessionDoc(selectedCourse.session_flow_document_path!)}
                    disabled={docLoading}
                    className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs h-8 rounded-xl shrink-0"
                  >
                    {docLoading ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Eye className="w-3.5 h-3.5 mr-1" /
>}

                    View Document
                  </Button>
                </div>
              )}

              {/* Session Schedule & Timeline */}
              {selectedCourse?.session_flow_text && (
                <div className="space-y-3">
                  <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                    <Layers className="w-3.5 h-3.5 text-cyan-600" /> Session Schedule & Timeline
                  </h4>
                  <div className="bg-slate-50 border border-slate-200/80 rounded-2xl p-5 text-xs sm:text-sm text-slate-700 le
ading-relaxed whitespace-pre-wrap font-normal">
                    {selectedCourse.session_flow_text}
                  </div>
                </div>
              )}
            </div>
          )}
        </DialogContent>
      </Dialog>

      <MaterialPreviewDialog
        material={previewMaterial}
        previewUrl={previewUrl}
        onClose={() => {
          setPreviewMaterial(null)
          setPreviewUrl(null)
        }}
        onDownload={async () => {
          if (!previewMaterial?.storage_path) return
```

```
1494 |           try {
1495 |             const { data, error } = await supabase.storage.from('materials').createSignedUrl(previewMaterial.storage_path, 6
0, { download: true })
1496 |             if (error) throw error
1497 |             if (data?.signedUrl) window.open(data.signedUrl, '_blank')
1498 |           } catch {
1499 |             toast.error('Failed to download file')
1500 |           }
1501 |         }}
1502 |       />
1503 |     </>
1504 |   )
1505 | }
1506 |
```

src/features/courses/CourseAnnouncements.tsx TSX 9.8 KB 217 Lines

```tsx
import React, { useState } from 'react'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Card, CardContent } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Textarea } from '@/components/ui/textarea'
import { Input } from '@/components/ui/input'
import { Megaphone, Plus, Loader2, Calendar, Sparkles } from 'lucide-react'
import { toast } from 'sonner'
import { formatDistanceToNow } from 'date-fns'

export function CourseAnnouncements({ courseId, isTrainer }: { courseId: string; isTrainer: boolean }) {
  const { profile } = useAuth()
  const queryClient = useQueryClient()
  const [isAdding, setIsAdding] = useState(false)
  const [title, setTitle] = useState('')
  const [content, setContent] = useState('')

  const { data: announcements = [], isLoading } = useQuery({
    queryKey: ['course_announcements', courseId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('course_announcements')
        .select('*')
        .eq('course_id', courseId)
        .order('created_at', { ascending: false })
      if (error) {
        console.error('Error fetching course announcements:', error)
        throw error
      }
      if (!data || data.length === 0) return []

      const trainerIds = Array.from(new Set(data.map(a => a.trainer_id)))
      const [trainersRes, adminsRes] = await Promise.all([
        supabase.from('trainers').select('id, full_name').in('id', trainerIds),
        supabase.from('admins').select('id, full_name').in('id', trainerIds),
      ])

      const trainerMap = new Map<string, string>()
      trainersRes.data?.forEach(t => trainerMap.set(t.id, t.full_name || 'Course Trainer'))
      adminsRes.data?.forEach(a => trainerMap.set(a.id, a.full_name || 'Admin'))

      return data.map(a => ({
        ...a,
        trainer: {
          full_name: trainerMap.get(a.trainer_id) || 'Course Trainer'
        }
      }))
    },
    enabled: !!courseId
  })

  const addAnnouncement = useMutation({
    mutationFn: async () => {
      if (!title.trim() || !content.trim()) throw new Error('Title and content are required.')
      if (!profile?.id) throw new Error('User profile not loaded.')
      const { data, error } = await supabase.from('course_announcements').insert({
        course_id: courseId,
        trainer_id: profile.id,
        title: title.trim(),
        content: content.trim()
      }).select()
      if (error) throw error

      // Direct notification dispatch to enrolled trainees

      try {
        const { data: enrollments } = await supabase
          .from('enrollments')
          .select('user_id')
          .eq('course_id', courseId)
          .in('status', ['enrolled', 'in_progress', 'completed'])

        if (enrollments && enrollments.length > 0) {
          const notifs = enrollments
```

```
            .filter(e => e.user_id !== profile.id)
            .map(e => ({
              user_id: e.user_id,
              type: `course_announcement:${courseId}`,
              title: `📢 Announcement: ${title.trim()}`,
              message: `New update posted: ${content.trim().substring(0, 120)}`
            }))
          if (notifs.length > 0) {
            await supabase.from('notifications').insert(notifs)
          }
        }
      } catch (nErr) {
        console.warn('Direct notification insertion handled or skipped:', nErr)
      }

      return data
    },
    onSuccess: () => {
      toast.success('Announcement posted successfully!')
      queryClient.invalidateQueries({ queryKey: ['course_announcements', courseId] })
      setIsAdding(false)
      setTitle('')
      setContent('')
    },
    onError: (err: any) => {
      toast.error(err.message || 'Failed to post announcement')
    }
  })

  return (
    <div className="space-y-4">
      {/* Header Bar */}
      <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 bg-white p-5 border border-slate-200/90 rounded-2xl shadow-xs">
        <div className="flex items-center gap-3">
          <div className="w-10 h-10 rounded-xl bg-cyan-50 border border-cyan-200 text-cyan-700 flex items-center justify-center shrink-0">
            <Megaphone className="w-5 h-5 text-cyan-600" />
          </div>
          <div>
            <h3 className="font-bold text-slate-900 text-base">Course Announcements</h3>
            <p className="text-xs text-slate-500">
              {isTrainer ? 'Post broadcast updates and important news for your trainees.' : 'Latest updates and notices from your trainer.'}
            </p>
          </div>
        </div>
        {isTrainer && (
          <Button
            onClick={() => setIsAdding(!isAdding)}
            className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold text-xs rounded-xl gap-2 shadow-xs shrink-0 self-start sm:self-auto h-9 px-3.5"
          >
            <Plus className="w-3.5 h-3.5" /> {isAdding ? 'Cancel' : 'New Announcement'}
          </Button>
        )}
      </div>

      {/* Adding Form */}
      {isAdding && isTrainer && (
        <Card className="bg-gradient-to-br from-cyan-50/50 via-sky-50/30 to-blue-50/20 border border-cyan-200 rounded-2xl shadow-xs overflow-hidden">
          <CardContent className="p-6 space-y-4">
            <div className="flex items-center gap-2 text-xs font-bold text-cyan-900 pb-2 border-b border-cyan-200/60">
              <Sparkles className="w-4 h-4 text-cyan-600" /> Create New Broadcast Announcement
            </div>
            <div className="space-y-1.5">
              <label className="text-xs font-semibold text-slate-700">Announcement Title *</label>
              <Input
                placeholder="e.g. Schedule Update for Module 2 Live Session"
                value={title}
                onChange={e => setTitle(e.target.value)}
                className="bg-white border-slate-200 text-slate-900 rounded-xl text-xs focus:bg-white h-10 font-semibold"
              />
            </div>
            <div className="space-y-1.5">
              <label className="text-xs font-semibold text-slate-700">Announcement Content *</label>
              <Textarea
                placeholder="Write the full announcement message for trainees..."
```

```
150 |                 value={content}
151 |                 onChange={e => setContent(e.target.value)}
152 |                 rows={3}
153 |                 className="bg-white border-slate-200 text-slate-900 resize-none rounded-xl text-xs focus:bg-white"
154 |               />
155 |             </div>
156 |             <div className="flex justify-end gap-2 pt-1">
157 |               <Button
158 |                 variant="outline"
159 |                 onClick={() => setIsAdding(false)}
160 |                 className="text-xs font-semibold border-slate-200 rounded-xl h-9 text-slate-600 hover:bg-slate-100"
161 |               >
162 |                 Cancel
163 |               </Button>
164 |               <Button
165 |                 onClick={() => addAnnouncement.mutate()}
166 |                 disabled={addAnnouncement.isPending || !title.trim() || !content.trim()}
167 |                 className="bg-gradient-to-r from-emerald-600 to-teal-600 hover:from-emerald-700 hover:to-teal-700 text-white f
ont-bold text-xs rounded-xl h-9 px-4 shadow-xs"
168 |               >
169 |                 {addAnnouncement.isPending ? <Loader2 className="w-4 h-4 animate-spin mr-2" /> : 'Post Announcement'}
170 |               </Button>
171 |             </div>
172 |           </CardContent>
173 |         </Card>
174 |       )}
175 |
176 |       {/* Announcements List */}
177 |       {isLoading ? (
178 |         <div className="flex justify-center py-8">
179 |           <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
180 |         </div>
181 |       ) : announcements.length === 0 ? (
182 |         <div className="text-center py-10 bg-white border border-dashed border-slate-200 rounded-2xl shadow-xs">
183 |           <Megaphone className="w-10 h-10 text-slate-300 mx-auto mb-2" />
184 |           <h3 className="text-slate-800 font-bold text-sm">No announcements yet</h3>
185 |           <p className="text-slate-500 text-xs mt-1">There are no updates posted for this course right now.</p>
186 |         </div>
187 |       ) : (
188 |         <div className="space-y-3">
189 |           {announcements.map((a: any) => (
190 |             <Card key={a.id} className="bg-white border border-slate-200/90 hover:border-cyan-300 rounded-2xl overflow-hidden
shadow-xs transition-all">
191 |               <CardContent className="p-5 space-y-3">
192 |                 <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-2 pb-2.5 border-b border-slate-1
00">
193 |                   <div className="flex items-center gap-2.5">
194 |                     <div className="w-8 h-8 rounded-full bg-cyan-50 border border-cyan-200 text-cyan-800 flex items-center jus
tify-center font-bold text-xs shrink-0">
195 |                       {a.trainer?.full_name?.charAt(0) || 'T'}
196 |                     </div>
197 |                     <div>
198 |                       <h4 className="font-bold text-slate-900 text-sm leading-tight">{a.title}</h4>
199 |                       <p className="text-[11px] text-slate-400 mt-0.5 flex items-center gap-1">
200 |                         <Calendar className="w-3 h-3" />
201 |                         {formatDistanceToNow(new Date(a.created_at), { addSuffix: true })} by <span className="font-semibold t
ext-slate-600">{a.trainer?.full_name || 'Trainer'}</span>
202 |                       </p>
203 |                     </div>
204 |                   </div>
205 |                 </div>
206 |                 <p className="text-slate-700 text-xs leading-relaxed bg-slate-50/80 p-3.5 rounded-xl border border-slate-100 w
hitespace-pre-wrap">
207 |                   {a.content}
208 |                 </p>
209 |               </CardContent>
210 |             </Card>
211 |           ))}
212 |         </div>
213 |       )}
214 |     </div>
215 |   )
216 | }
217 |
```

src/features/courses/CourseCertificateStep.tsx TSX 14.0 KB 284 Lines

```tsx
import React, { useState } from 'react'
import { motion } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import {
  Award, Upload, Download, FileText, CheckCircle2, Sparkles,
  AlertCircle, Trash2, Eye, Shield, RefreshCw
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@/components/ui/card'
import { toast } from 'sonner'
import { createDefaultPptxTemplate, triggerFileDownload } from '@/lib/certificateGenerator'

interface CourseCertificateStepProps {
  hasCertificate: boolean
  setHasCertificate: (val: boolean) => void
  templateUrl: string | null
  setTemplateUrl: (url: string | null) => void
  templateName: string | null
  setTemplateName: (name: string | null) => void
  certificateTitle?: string
  setCertificateTitle?: (title: string) => void
  courseTitle?: string
}

export function CourseCertificateStep({
  hasCertificate,
  setHasCertificate,
  templateUrl,
  setTemplateUrl,
  templateName,
  setTemplateName,
  courseTitle = 'Specialized Training Course',
}: CourseCertificateStepProps) {
  const [uploading, setUploading] = useState(false)
  const [downloadingStarter, setDownloadingStarter] = useState(false)

  const handleFileUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return

    if (!file.name.toLowerCase().endsWith('.pptx') && !file.name.toLowerCase().endsWith('.ppt')) {
      toast.error('Please upload a Microsoft PowerPoint (.pptx) certificate template file.')
      return
    }

    if (file.size > 50 * 1024 * 1024) {
      toast.error('Template file size must be less than 50MB.')
      return
    }

    setUploading(true)
    try {
      const fileExt = file.name.split('.').pop()
      const safeName = file.name.replace(/[^a-zA-Z0-9_.-]/g, '_')
      const storagePath = `templates/${Date.now()}_${safeName}`

      // Upload to 'Certificate template' bucket

      const { data, error } = await supabase.storage
        .from('Certificate template')
        .upload(storagePath, file, {
          upsert: true,
          contentType: 'application/vnd.openxmlformats-officedocument.presentationml.presentation',
        })

      if (error) {
        // Fallback to 'materials' or 'certificates' if bucket hasn't been created yet

        console.warn('Certificate template bucket error, fallback attempt:', error.message)
        const { data: fbData, error: fbError } = await supabase.storage
          .from('certificates')
          .upload(storagePath, file, { upsert: true })

        if (fbError) throw fbError
        const { data: publicUrlData } = supabase.storage.from('certificates').getPublicUrl(storagePath)
        setTemplateUrl(publicUrlData.publicUrl)
```

```
        setTemplateName(file.name)
      } else {
        const { data: publicUrlData } = supabase.storage.from('Certificate template').getPublicUrl(storagePath)
        setTemplateUrl(publicUrlData.publicUrl)
        setTemplateName(file.name)
      }

      toast.success('Certificate PPTX template uploaded successfully!')
    } catch (err: any) {
      toast.error(err.message || 'Failed to upload certificate template.')
    } finally {
      setUploading(false)
    }
  }

  const handleDownloadStarter = async () => {
    setDownloadingStarter(true)
    try {
      const buffer = await createDefaultPptxTemplate(courseTitle)
      const blob = new Blob([buffer], {
        type: 'application/vnd.openxmlformats-officedocument.presentationml.presentation',
      })
      triggerFileDownload(blob, `MoES_Certificate_Template_${courseTitle.replace(/[^a-zA-Z0-9_-]/g, '_')}.pptx`)
      toast.success('Starter PPTX template downloaded! Open in PowerPoint to edit.')
    } catch (err: any) {
      toast.error(err.message || 'Failed to generate starter template.')
    } finally {
      setDownloadingStarter(false)
    }
  }

  const handleRemoveTemplate = () => {
    setTemplateUrl(null)
    setTemplateName(null)
    toast.info('Reverted to default MoES system certificate template.')
  }

  return (
    <div className="space-y-6">
      {/* Certificate Enable Toggle Card */}
      <Card className="bg-white border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
        <CardContent className="p-6 sm:p-8">
          <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-5">
            <div className="flex items-start gap-4">
              <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-500 to-blue-600 flex items-center justify-cent
er text-white shadow-lg shadow-cyan-500/20 shrink-0 mt-0.5">
                <Award className="w-6 h-6" />
              </div>
              <div>
                <h3 className="text-base font-bold text-slate-900">Certificate of Completion</h3>
                <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
                  Issue automated, verifiable completion certificates to trainees who successfully finish this course and meet
the passing criteria.
                </p>
              </div>
            </div>

            <label className="relative inline-flex items-center cursor-pointer shrink-0">
              <input
                type="checkbox"
                checked={hasCertificate}
                onChange={e => setHasCertificate(e.target.checked)}
                className="sr-only peer"
              />
              <div className="w-11 h-6 bg-slate-200 peer-focus:outline-none rounded-full peer peer-checked:after:translate-x-f
ull peer-checked:after:border-white after:content-[''] after:absolute after:top-[2px] after:left-[2px] after:bg-white after:border-s
late-300 after:border after:rounded-full after:h-5 after:w-5 after:transition-all peer-checked:bg-cyan-600"></div>
            </label>
          </div>
        </CardContent>
      </Card>

      {hasCertificate && (
        <motion.div initial={{ opacity: 0, y: 15 }} animate={{ opacity: 1, y: 0 }} className="space-y-6">
          {/* Template Upload & Starter Download Card */}
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-6">
            <Card className="lg:col-span-7 bg-white border-slate-200/90 rounded-3xl shadow-sm overflow-hidden flex flex-col">
              <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
                <div className="flex items-center justify-between">
```

```
                    <div>
                      <CardTitle className="text-sm font-bold text-slate-900 flex items-center gap-2">
                        <FileText className="w-4 h-4 text-cyan-600" /> Certificate PPTX Template
                      </CardTitle>
                      <CardDescription className="text-xs text-slate-500 mt-1">
                        Upload your customized PowerPoint template with dynamic placeholders.
                      </CardDescription>
                    </div>
                    <span className="text-[11px] font-bold px-2.5 py-1 rounded-full bg-cyan-50 text-cyan-700 border border-cyan-
200">
                      Bucket: Certificate template
                    </span>
                  </div>
                </CardHeader>

                <CardContent className="p-6 flex-1 flex flex-col justify-between space-y-5">
                  {templateUrl ? (
                    <div className="p-5 rounded-2xl bg-gradient-to-r from-cyan-50/80 via-sky-50/60 to-blue-50/80 border border-c
yan-200/80 flex items-center justify-between gap-4">
                      <div className="flex items-center gap-3.5 min-w-0">
                        <div className="w-10 h-10 rounded-xl bg-cyan-600 text-white flex items-center justify-center font-bold t
ext-xs shrink-0 shadow-sm">
                          PPTX
                        </div>
                        <div className="min-w-0">
                          <p className="text-xs font-bold text-slate-900 truncate">
                            {templateName || 'custom_certificate_template.pptx'}
                          </p>
                          <p className="text-[11px] text-emerald-600 font-semibold flex items-center gap-1 mt-0.5">
                            <CheckCircle2 className="w-3.5 h-3.5" /> Active Custom Template
                          </p>
                        </div>
                      </div>

                      <div className="flex items-center gap-2">
                        <Button
                          type="button"
                          variant="outline"
                          size="sm"
                          onClick={() => window.open(templateUrl, '_blank')}
                          className="rounded-xl text-xs h-8 gap-1.5 border-slate-300"
                        >
                          <Download className="w-3.5 h-3.5" /> Download
                        </Button>
                        <Button
                          type="button"
                          variant="ghost"
                          size="sm"
                          onClick={handleRemoveTemplate}
                          className="rounded-xl text-xs h-8 text-rose-600 hover:text-rose-700 hover:bg-rose-50"
                        >
                          <Trash2 className="w-3.5 h-3.5" />
                        </Button>
                      </div>
                    </div>
                  ) : (
                    <div className="border-2 border-dashed border-slate-200 hover:border-cyan-400 rounded-3xl p-8 text-center tr
ansition-all bg-slate-50/40 group relative">
                      <input
                        type="file"
                        accept=".pptx,.ppt,application/vnd.openxmlformats-officedocument.presentationml.presentation"
                        onChange={handleFileUpload}
                        disabled={uploading}
                        className="absolute inset-0 w-full h-full opacity-0 cursor-pointer z-10"
                      />
                      <div className="w-12 h-12 rounded-2xl bg-cyan-50 text-cyan-600 border border-cyan-200/80 flex items-center
justify-center mx-auto mb-3 group-hover:scale-110 transition-transform">
                        {uploading ? <RefreshCw className="w-6 h-6 animate-spin" /> : <Upload className="w-6 h-6" />}
                      </div>
                      <p className="text-xs font-bold text-slate-800">
                        {uploading ? 'Uploading PPTX Template...' : 'Click or Drag & Drop PPTX Certificate Template'}
                      </p>
                      <p className="text-[11px] text-slate-500 mt-1">
                        PowerPoint presentation (.pptx) up to 50MB
                      </p>
                      <span className="inline-block mt-3 text-[10px] font-semibold text-cyan-700 bg-cyan-100/60 px-2.5 py-1 roun
ded-full">
                        If not uploaded, the default MoES Capacity Connect template will be used
                      </span>
```

```
                  </div>
                )}

                <div className="pt-2 flex flex-col sm:flex-row items-center justify-between gap-3 border-t border-slate-100">
                  <div className="text-[11px] text-slate-500">
                    Need a starting template? Download our pre-configured PPTX with ready placeholders.
                  </div>
                  <Button
                    type="button"
                    variant="outline"
                    size="sm"
                    onClick={handleDownloadStarter}
                    disabled={downloadingStarter}
                    className="rounded-xl text-xs h-9 gap-1.5 border-cyan-300 text-cyan-700 hover:bg-cyan-50 font-bold shrink-0"
                  >
                    <Download className="w-3.5 h-3.5" />
                    {downloadingStarter ? 'Generating...' : 'Download Starter PPTX'}
                  </Button>
                </div>
              </CardContent>
            </Card>

            {/* Placeholder Guide Card */}
            <Card className="lg:col-span-5 bg-gradient-to-br from-[#040814] via-[#071328] to-[#040d21] text-white border-cyan-500/30 rounded-3xl shadow-lg flex flex-col">
              <CardHeader className="p-6 border-b border-white/10">
                <CardTitle className="text-sm font-bold text-white flex items-center gap-2">
                  <Sparkles className="w-4 h-4 text-cyan-400" /> Supported Placeholders
                </CardTitle>
                <CardDescription className="text-xs text-slate-400">
                  Include these tags anywhere in your PowerPoint slides. They will be dynamically populated upon course completion:
                </CardDescription>
              </CardHeader>
              <CardContent className="p-6 space-y-3.5 flex-1">
                {[
                  { tag: '<<NAME>>', desc: 'Trainee full name (e.g. Dr. Rajesh Kumar)' },
                  { tag: '<<PERCENTAGE>>', desc: 'Final score / progress percentage (e.g. 98%)' },
                  { tag: '<<COURSE_NAME>>', desc: 'Official title of the course' },
                  { tag: '<<DATE>>', desc: 'Certificate issue date' },
                  { tag: '<<TRAINER_NAME>>', desc: 'Lead instructor name' },
                ].map(item => (
                  <div key={item.tag} className="flex items-center justify-between p-2.5 rounded-xl bg-white/5 border border-white/10">
                    <code className="text-xs font-mono font-bold text-cyan-300 bg-cyan-950/60 px-2 py-0.5 rounded border border-cyan-500/30">
                      {item.tag}
                    </code>
                    <span className="text-[11px] text-slate-300 text-right font-medium">
                      {item.desc}
                    </span>
                  </div>
                ))}

                <div className="p-3 rounded-xl bg-cyan-500/10 border border-cyan-500/20 text-[11px] text-cyan-200/90 mt-4">
                  💡 <strong>Tip:</strong> You can format the placeholder text in PowerPoint with custom fonts, colors, and shadows. The formatting will be preserved!
                </div>
              </CardContent>
            </Card>
          </div>
        </motion.div>
      )}
    </div>
  )
}

```

src/features/courses/CourseChat.tsx TSX 12.0 KB 288 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Input } from '@/components/ui/input'
import { Button } from '@/components/ui/button'
import { Send, Loader2, Lock, MessageSquare, Shield, User } from 'lucide-react'
import { toast } from 'sonner'
import { formatDistanceToNow } from 'date-fns'

export function CourseChat({ courseId, isTrainer }: { courseId: string; isTrainer: boolean }) {
  const { profile } = useAuth()
  const queryClient = useQueryClient()
  const [newMessage, setNewMessage] = useState('')
  const [isPrivate, setIsPrivate] = useState(false)
  const [recipientId, setRecipientId] = useState('')
  const scrollContainerRef = useRef<HTMLDivElement>(null)

  const { data: trainees = [] } = useQuery({
    queryKey: ['course_trainees', courseId],
    queryFn: async () => {
      if (!isTrainer) return []
      const { data: enrollmentsData, error } = await supabase
        .from('enrollments')
        .select('user_id')
        .eq('course_id', courseId)
        .in('status', ['enrolled', 'completed', 'in_progress'])
      if (error) {
        console.error('Error fetching enrollments for chat:', error)
        return []
      }
      if (!enrollmentsData || enrollmentsData.length === 0) return []

      const userIds = enrollmentsData.map(e => e.user_id)
      const { data: traineesData } = await supabase
        .from('trainees')
        .select('id, full_name, email')
        .in('id', userIds)

      return enrollmentsData.map(e => ({
        user_id: e.user_id,
        trainees: traineesData?.find(t => t.id === e.user_id) || { full_name: 'Unknown Trainee' }
      }))
    },
    enabled: !!courseId && isTrainer
  })

  const { data: messages = [], isLoading } = useQuery({
    queryKey: ['course_messages', courseId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('course_messages')
        .select('*')
        .eq('course_id', courseId)
        .order('created_at', { ascending: true })
      if (error) {
        console.error('Error fetching course messages:', error)
        throw error
      }
      if (!data || data.length === 0) return []

      // Fetch sender details from trainers, trainees, and admins

      const senderIds = Array.from(new Set(data.map(m => m.sender_id)))
      const [trainersRes, traineesRes, adminsRes] = await Promise.all([
        supabase.from('trainers').select('id, full_name, role').in('id', senderIds),
        supabase.from('trainees').select('id, full_name, role').in('id', senderIds),
        supabase.from('admins').select('id, full_name, role').in('id', senderIds),
      ])

      const senderMap = new Map<string, { full_name: string; role: string }>()
      trainersRes.data?.forEach(t => senderMap.set(t.id, { full_name: t.full_name || 'Trainer', role: t.role || 'trainer' }))
      traineesRes.data?.forEach(t => senderMap.set(t.id, { full_name: t.full_name || 'Trainee', role: t.role || 'trainee' }))
      adminsRes.data?.forEach(a => senderMap.set(a.id, { full_name: a.full_name || 'Admin', role: a.role || 'admin' }))

      return data.map(m => ({
```

```
        ...m,
        sender: senderMap.get(m.sender_id) || { full_name: 'User', role: 'trainee' }
      }))
    },
    enabled: !!courseId && !!profile?.id
  })

  // Set up realtime subscription
  useEffect(() => {
    if (!courseId || !profile?.id) return

    const channel = supabase
      .channel(`chat_${courseId}`)
      .on(
        'postgres_changes',
        {
          event: 'INSERT',
          schema: 'public',
          table: 'course_messages',
          filter: `course_id=eq.${courseId}`
        },
        () => {
          queryClient.invalidateQueries({ queryKey: ['course_messages', courseId] })
        }
      )
      .subscribe()

    return () => {
      supabase.removeChannel(channel)
    }
  }, [courseId, profile?.id, queryClient])

  // Scroll to bottom when messages change
  useEffect(() => {
    if (scrollContainerRef.current) {
      scrollContainerRef.current.scrollTo({
        top: scrollContainerRef.current.scrollHeight,
        behavior: 'smooth'
      })
    }
  }, [messages])

  const sendMessage = useMutation({
    mutationFn: async () => {
      if (!newMessage.trim() || !profile?.id) return

      const payload: any = {
        course_id: courseId,
        sender_id: profile.id,
        content: newMessage.trim(),
        is_private: isPrivate,
      }

      if (isPrivate && isTrainer && recipientId) {
        payload.recipient_id = recipientId
      }

      const { data, error } = await supabase.from('course_messages').insert(payload).select()
      if (error) throw error
      return data
    },
    onSuccess: () => {
      setNewMessage('')
      queryClient.invalidateQueries({ queryKey: ['course_messages', courseId] })
    },
    onError: (err: any) => {
      toast.error(err.message || 'Failed to send message')
    }
  })

  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault()
      sendMessage.mutate()
    }
  }
```

```
  return (
    <div className="flex flex-col h-[520px] bg-white border border-slate-200/90 rounded-2xl overflow-hidden shadow-xs">
      {/* Header with polished dark contrast banner */}
      <div className="bg-gradient-to-r from-slate-900 via-slate-800 to-slate-900 px-5 py-4 border-b border-slate-800 flex items-center justify-between text-white">
        <div className="flex items-center gap-3">
          <div className="w-9 h-9 rounded-xl bg-cyan-500/20 border border-cyan-400/30 flex items-center justify-center text-cyan-400">
            <MessageSquare className="w-4 h-4" />
          </div>
          <div>
            <h3 className="font-bold text-white text-sm">Course Discussion & Q&A</h3>
            <p className="text-[11px] text-slate-300 hidden sm:block">
              {isTrainer ? 'Interact with trainees, answer questions, or send direct private feedback.' : 'Ask questions, discuss topics, or send private messages to your trainer.'}
            </p>
          </div>
        </div>
        <div className="flex items-center gap-2">
          <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse" />
          <span className="text-[11px] font-semibold text-slate-300">Live</span>
        </div>
      </div>

      {/* Chat Messages */}
      <div ref={scrollContainerRef} className="flex-1 overflow-y-auto p-5 space-y-4 bg-slate-50/70">
        {isLoading ? (
          <div className="flex justify-center h-full items-center">
            <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
          </div>
        ) : messages.length === 0 ? (
          <div className="flex flex-col items-center justify-center h-full text-slate-400 py-12">
            <MessageSquare className="w-10 h-10 mb-2 opacity-30 text-cyan-600" />
            <p className="text-xs font-semibold text-slate-600">No messages yet</p>
            <p className="text-[11px] text-slate-400 mt-0.5">Start the conversation with your course group!</p>
          </div>
        ) : (
          messages.map((msg: any) => {
            const isMe = msg.sender_id === profile?.id
            return (
              <div key={msg.id} className={`flex flex-col ${isMe ? 'items-end' : 'items-start'}`}>
                <div className="flex items-center gap-1.5 mb-1 px-1">
                  {!isMe && (
                    <span className={`text-[11px] font-bold ${
                      msg.sender?.role === 'trainer' ? 'text-cyan-700' : 'text-slate-700'
                    }`}>
                      {msg.sender?.full_name || 'User'}
                    </span>
                  )}
                  {msg.is_private && (
                    <span className="flex items-center gap-1 text-[10px] font-bold text-rose-700 bg-rose-50 border border-rose-200 px-1.5 py-0.5 rounded-md">
                      <Lock className="w-2.5 h-2.5" /> {msg.sender?.role === 'trainer' ? 'Private to Trainee' : 'Private to Trainer'}
                    </span>
                  )}
                </div>

                <div className={`max-w-[85%] sm:max-w-[75%] rounded-2xl px-4 py-2.5 shadow-xs ${
                  isMe
                    ? 'bg-gradient-to-r from-cyan-600 to-blue-600 text-white rounded-br-xs'
                    : msg.sender?.role === 'trainer'
                      ? 'bg-white border border-cyan-200 text-slate-900 rounded-bl-xs shadow-xs'
                      : 'bg-white border border-slate-200/90 text-slate-900 rounded-bl-xs shadow-xs'
                }`}>
                  <p className="text-xs leading-relaxed whitespace-pre-wrap break-words">{msg.content}</p>
                </div>

                <span className="text-[10px] text-slate-400 mt-1 px-1">
                  {formatDistanceToNow(new Date(msg.created_at), { addSuffix: true })}
                </span>
              </div>
            )
          })
        )}
      </div>

      {/* Chat Input Section */}
      <div className="p-3.5 bg-white border-t border-slate-200/90 space-y-2.5">
```

```
227 |         <div className="flex flex-wrap items-center gap-2 px-1">
228 |           <button
229 |             type="button"
230 |             onClick={() => {
231 |               setIsPrivate(!isPrivate)
232 |               if (isPrivate) setRecipientId('')
233 |             }}
234 |             className={`flex items-center gap-1.5 px-3 py-1.5 rounded-full text-xs font-semibold transition-all ${
235 |               isPrivate
236 |                 ? 'bg-rose-50 text-rose-700 border border-rose-300 shadow-xs'
237 |                 : 'bg-slate-100 text-slate-600 border border-slate-200 hover:bg-slate-200/80 hover:text-slate-800'
238 |             }`}
239 |           >
240 |             <Lock className="w-3 h-3" />
241 |             {isPrivate ? 'Private Mode Active' : (isTrainer ? 'Send Privately' : 'Ask Privately')}
242 |           </button>
243 |
244 |           {isTrainer && isPrivate && (
245 |             <select
246 |               value={recipientId}
247 |               onChange={e => setRecipientId(e.target.value)}
248 |               className="bg-slate-50 border border-rose-300 text-slate-800 text-xs font-medium rounded-full px-3 py-1.5 focus:
bg-white outline-none cursor-pointer"
249 |             >
250 |               <option value="" disabled>Select Trainee recipient...</option>
251 |               {trainees.map((t: any) => (
252 |                 <option key={t.user_id} value={t.user_id}>
253 |                   {t.trainees?.full_name || 'Unknown Trainee'}
254 |                 </option>
255 |               ))}
256 |             </select>
257 |           )}
258 |         </div>
259 |
260 |         <div className="flex gap-2">
261 |           <Input
262 |             placeholder={isPrivate ? (isTrainer ? "Type a private message to trainee..." : "Type a private question to traine
r...") : "Type a message to the course discussion..."}
263 |             value={newMessage}
264 |             onChange={e => setNewMessage(e.target.value)}
265 |             onKeyDown={handleKeyDown}
266 |             className={`flex-1 bg-slate-50 text-slate-900 border transition-all h-10 rounded-xl text-xs placeholder:text-slate
-400 focus:bg-white ${
267 |               isPrivate
268 |                 ? 'border-rose-300 focus-visible:ring-rose-400'
269 |                 : 'border-slate-200 focus-visible:ring-cyan-500'
270 |             }`}
271 |           />
272 |           <Button
273 |             onClick={() => sendMessage.mutate()}
274 |             disabled={sendMessage.isPending || !newMessage.trim() || (isPrivate && isTrainer && !recipientId)}
275 |             className={`shrink-0 h-10 px-4 font-bold text-xs rounded-xl shadow-xs text-white ${
276 |               isPrivate
277 |                 ? 'bg-rose-600 hover:bg-rose-700'
278 |                 : 'bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700'
279 |             }`}
280 |           >
281 |             {sendMessage.isPending ? <Loader2 className="w-4 h-4 animate-spin" /> : <Send className="w-4 h-4" />}
282 |           </Button>
283 |         </div>
284 |       </div>
285 |     </div>
286 |   )
287 | }
288 |
```

src/features/courses/CourseFeedback.tsx TSX 17.3 KB 405 Lines

```tsx
import React, { useState } from 'react'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Card, CardContent } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Textarea } from '@/components/ui/textarea'
import { Badge } from '@/components/ui/badge'
import { Star, MessageSquare, Loader2, CheckCircle2, User, ThumbsUp, Sparkles } from 'lucide-react'
import { toast } from 'sonner'
import { formatDistanceToNow } from 'date-fns'

interface CourseFeedbackProps {
  courseId: string
  isTrainer?: boolean
}

export function CourseFeedback({ courseId, isTrainer = false }: CourseFeedbackProps) {
  const { profile } = useAuth()
  const queryClient = useQueryClient()
  const [rating, setRating] = useState(0)
  const [hoverRating, setHoverRating] = useState(0)
  const [comments, setComments] = useState('')
  const [isEditing, setIsEditing] = useState(false)

  // 1. Trainer Query: Fetch all feedback with trainee names

  const { data: allFeedback = [], isLoading: isLoadingAll } = useQuery({
    queryKey: ['course_all_feedback', courseId],
    queryFn: async () => {
      try {
        const { data, error } = await supabase
          .from('course_feedback')
          .select('*')
          .eq('course_id', courseId)
          .order('created_at', { ascending: false })

        if (error) {
          console.warn('Could not fetch course_feedback:', error.message)
          return []
        }
        if (!data || data.length === 0) return []

        const userIds = Array.from(new Set(data.map(f => f.user_id)))
        const { data: traineesData } = await supabase
          .from('trainees')
          .select('id, full_name, email, department')
          .in('id', userIds)

        const traineeMap = new Map<string, { full_name: string; email: string; department?: string }>()
        traineesData?.forEach(t => traineeMap.set(t.id, { full_name: t.full_name || 'Trainee', email: t.email || '', departmen
t: t.department || undefined }))

        return data.map(f => ({
          ...f,
          trainee: traineeMap.get(f.user_id) || { full_name: 'Trainee', email: '' }
        }))
      } catch (err) {
        console.warn('course_feedback error:', err)
        return []
      }
    },
    enabled: !!courseId && isTrainer
  })

  // 2. Trainee Query: Fetch own feedback

  const { data: existingFeedback, isLoading: isLoadingOwn } = useQuery({
    queryKey: ['course_feedback', courseId, profile?.id],
    queryFn: async () => {
      if (!profile?.id) return null
      try {
        const { data, error } = await supabase
          .from('course_feedback')
          .select('*')
          .eq('course_id', courseId)
```

```
          .eq('user_id', profile.id)
          .maybeSingle()
        if (error) {
          console.warn('Could not query own course_feedback:', error.message)
          return null
        }
        return data
      } catch (err) {
        console.warn('course_feedback table may not exist yet:', err)
        return null
      }
    },
    enabled: !!profile?.id && !!courseId && !isTrainer
  })

  // Submit/Update Feedback Mutation
  const submitFeedback = useMutation({
    mutationFn: async () => {
      if (!rating) throw new Error('Please select a star rating.')
      if (!profile?.id) throw new Error('User profile not found. Please log in.')

      const payload: any = {
        course_id: courseId,
        user_id: profile.id,
        rating,
        comments: comments.trim() || null,
      }

      const { data, error } = await (supabase.from('course_feedback') as any)
        .upsert(payload, { onConflict: 'course_id,user_id' })
        .select()

      if (error) throw error
      return data
    },
    onSuccess: () => {
      toast.success(existingFeedback ? 'Feedback updated successfully!' : 'Thank you for your feedback!')
      queryClient.invalidateQueries({ queryKey: ['course_feedback', courseId, profile?.id] })
      queryClient.invalidateQueries({ queryKey: ['course_all_feedback', courseId] })
      setIsEditing(false)
    },
    onError: (err: any) => {
      toast.error(err.message || 'Failed to submit feedback')
    }
  })

  // ========================================

  // TRAINER VIEW: Overview & Trainee Reviews List (Dark Theme)

  // ========================================

  if (isTrainer) {
    if (isLoadingAll) {
      return (
        <Card className="bg-gradient-to-br from-[#0c1322] via-[#090e1a] to-[#040814] border border-slate-800 rounded-3xl shado
w-xl">
          <CardContent className="p-8 flex justify-center items-center">
            <Loader2 className="w-6 h-6 animate-spin text-amber-400" />
          </CardContent>
        </Card>
      )
    }

    const totalRatings = allFeedback.length
    const averageRating = totalRatings > 0
      ? (allFeedback.reduce((acc, curr) => acc + curr.rating, 0) / totalRatings).toFixed(1)
      : '0.0'

    const starCounts: Record<number, number> = { 5: 0, 4: 0, 3: 0, 2: 0, 1: 0 }
    allFeedback.forEach(f => {
      if (f.rating >= 1 && f.rating <= 5) starCounts[f.rating] = (starCounts[f.rating] || 0) + 1
    })

    return (
      <Card className="bg-gradient-to-br from-[#0c1322] via-[#090e1a] to-[#040814] text-white border border-slate-800/90 round
ed-3xl shadow-xl overflow-hidden">
        <CardContent className="p-6 md:p-8 space-y-6">
```

```
147 |           {/* Header */}
148 |           <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-5 border-b border-slate-800">
149 |             <div className="flex items-center gap-3">
150 |               <div className="w-10 h-10 rounded-2xl bg-amber-500/15 border border-amber-500/30 flex items-center justify-cente
r text-amber-400 shrink-0 shadow-[0_0_15px_rgba(245,158,11,0.15)]">
151 |                 <Star className="w-5 h-5 fill-amber-400 text-amber-400" />
152 |               </div>
153 |               <div>
154 |                 <h3 className="font-bold text-white text-base tracking-tight">Course Feedback & Ratings</h3>
155 |                 <p className="text-xs text-slate-400">Reviews and feedback submitted by enrolled trainees.</p>
156 |               </div>
157 |             </div>
158 |             <Badge className="bg-amber-500/15 text-amber-300 border border-amber-500/30 text-xs font-bold px-3 py-1 self-start
sm:self-auto rounded-xl">
159 |               {totalRatings} Review{totalRatings !== 1 ? 's' : ''}
160 |             </Badge>
161 |           </div>
162 |
163 |           {/* Rating Summary Breakdown */}
164 |           {totalRatings > 0 ? (
165 |             <div className="grid grid-cols-1 md:grid-cols-3 gap-6 p-6 rounded-2xl bg-slate-900/80 border border-slate-800 back
drop-blur-md">
166 |               {/* Score Box */}
167 |               <div className="flex flex-col items-center justify-center text-center p-4">
168 |                 <span className="text-5xl font-black text-white tracking-tight">{averageRating}</span>
169 |                 <div className="flex items-center gap-1 my-2.5">
170 |                   {[1, 2, 3, 4, 5].map(star => (
171 |                     <Star
172 |                       key={star}
173 |                       className={`w-4 h-4 ${
174 |                         star <= Math.round(Number(averageRating))
175 |                           ? 'fill-amber-400 text-amber-400 drop-shadow-[0_0_6px_rgba(251,191,36,0.5)]'
176 |                           : 'text-slate-700'
177 |                       }`}
178 |                     />
179 |                   ))}
180 |                 </div>
181 |                 <p className="text-xs text-slate-400 font-semibold">Course Rating Average</p>
182 |               </div>
183 |
184 |               {/* Progress Bars */}
185 |               <div className="md:col-span-2 space-y-2.5 flex flex-col justify-center">
186 |                 {[5, 4, 3, 2, 1].map(stars => {
187 |                   const count = starCounts[stars] || 0
188 |                   const pct = totalRatings > 0 ? Math.round((count / totalRatings) * 100) : 0
189 |                   return (
190 |                     <div key={stars} className="flex items-center gap-2.5 text-xs">
191 |                       <span className="w-12 font-bold text-slate-300 flex items-center gap-1 shrink-0">
192 |                         {stars} <Star className="w-3 h-3 fill-amber-400 text-amber-400 inline" />
193 |                       </span>
194 |                       <div className="flex-1 h-2.5 bg-slate-800 rounded-full overflow-hidden border border-slate-700/50">
195 |                         <div
196 |                           className="h-full bg-gradient-to-r from-amber-400 to-orange-500 rounded-full transition-all shadow-
[0_0_8px_rgba(251,191,36,0.3)]"
197 |                           style={{ width: `${pct}%` }}
198 |                         />
199 |                       </div>
200 |                       <span className="w-10 text-right text-[11px] text-slate-400 font-semibold">{count}</span>
201 |                     </div>
202 |                   )
203 |                 })}
204 |               </div>
205 |             </div>
206 |           ) : null}
207 |
208 |           {/* Reviews List */}
209 |           <div className="space-y-3.5 pt-1">
210 |             <h4 className="text-xs font-bold text-slate-400 uppercase tracking-wider">
211 |               Trainee Reviews ({totalRatings})
212 |             </h4>
213 |
214 |             {totalRatings === 0 ? (
215 |               <div className="text-center py-10 rounded-2xl bg-slate-900/40 border border-dashed border-slate-800">
216 |                 <MessageSquare className="w-10 h-10 text-slate-600 mx-auto mb-2" />
217 |                 <h4 className="text-sm font-bold text-slate-300">No Feedback Submitted Yet</h4>
218 |                 <p className="text-xs text-slate-500 max-w-sm mx-auto mt-1">
219 |                   Once trainees complete course modules or submit reviews, their ratings and feedback comments will appear her
e.
220 |                 </p>
```

```
              </div>
            ) : (
              <div className="space-y-3">
                {allFeedback.map((fb: any) => (
                  <div
                    key={fb.id}
                    className="p-4 rounded-2xl bg-slate-900/60 border border-slate-800/90 space-y-3 transition-all hover:borde
r-slate-700"
                  >
                    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-2">
                      <div className="flex items-center gap-2.5">
                        <div className="w-8 h-8 rounded-full bg-cyan-500/20 text-cyan-300 font-bold text-xs flex items-center
justify-center border border-cyan-500/30">
                          {fb.trainee?.full_name?.charAt(0) || 'T'}
                        </div>
                        <div>
                          <p className="text-xs font-bold text-slate-200">{fb.trainee?.full_name || 'Trainee'}</p>
                          <p className="text-[10px] text-slate-400">{fb.trainee?.email}</p>
                        </div>
                      </div>

                      <div className="flex items-center gap-2">
                        <div className="flex items-center gap-0.5">
                          {[1, 2, 3, 4, 5].map(s => (
                            <Star
                              key={s}
                              className={`w-3.5 h-3.5 ${
                                s <= fb.rating
                                  ? 'fill-amber-400 text-amber-400 drop-shadow-[0_0_4px_rgba(251,191,36,0.4)]'
                                  : 'text-slate-700'
                              }`}
                            />
                          ))}
                        </div>
                        <span className="text-[10px] text-slate-400 font-medium">
                          {fb.created_at ? formatDistanceToNow(new Date(fb.created_at), { addSuffix: true }) : ''}
                        </span>
                      </div>
                    </div>

                    {fb.comments && (
                      <p className="text-xs text-slate-300 bg-slate-950/70 p-3.5 rounded-xl border border-slate-800/80 leading
-relaxed">
                        "{fb.comments}"
                      </p>
                    )}
                  </div>
                ))}
              </div>
            )}
          </div>
        </CardContent>
      </Card>
    )
  }

  // ========================================

  // TRAINEE VIEW: Interactive Rating & Submission

  // ========================================

  if (isLoadingOwn) {
    return (
      <div className="flex justify-center py-6">
        <Loader2 className="w-6 h-6 animate-spin text-orange-500" />
      </div>
    )
  }

  if (existingFeedback && !isEditing) {
    return (
      <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm space-y-3">
        <h3 className="text-[11px] font-bold text-slate-400 uppercase tracking-wider flex items-center justify-between">
          <span>Your Feedback</span>
          <span className="text-[10px] text-emerald-600 font-bold bg-emerald-50 px-2 py-0.5 rounded-full border border-emerald
-200">
            Submitted
          </span>
```

```
293 |         </h3>
294 |
295 |         <div className="p-3.5 bg-gradient-to-br from-emerald-50/60 to-teal-50/40 border border-emerald-100 rounded-2xl text-ce
nter space-y-2">
296 |           <div className="flex items-center justify-center gap-1">
297 |             {[1, 2, 3, 4, 5].map(star => (
298 |               <Star
299 |                 key={star}
300 |                 className={`w-4 h-4 ${
301 |                   star <= existingFeedback.rating ? 'fill-orange-400 text-orange-400' : 'text-slate-300'
302 |                 }`}
303 |               />
304 |             ))}
305 |           </div>
306 |           <p className="text-xs font-bold text-slate-800">Rated {existingFeedback.rating} of 5 Stars</p>
307 |           {existingFeedback.comments && (
308 |             <p className="text-xs text-slate-600 bg-white/90 p-2.5 rounded-xl border border-emerald-100 text-left italic leadi
ng-relaxed">
309 |               "{existingFeedback.comments}"
310 |             </p>
311 |           )}
312 |         </div>
313 |
314 |         <Button
315 |           type="button"
316 |           variant="outline"
317 |           size="sm"
318 |           onClick={() => {
319 |             setRating(existingFeedback.rating)
320 |             setComments(existingFeedback.comments || '')
321 |             setIsEditing(true)
322 |           }}
323 |           className="w-full text-xs font-bold text-slate-700 border-slate-200 hover:bg-slate-50 rounded-xl h-9"
324 |         >
325 |           Edit Rating & Review
326 |         </Button>
327 |       </div>
328 |     )
329 |   }
330 |
331 |   return (
332 |     <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm space-y-4">
333 |       <div>
334 |         <h3 className="text-[11px] font-bold text-slate-400 uppercase tracking-wider mb-1">Course Feedback</h3>
335 |         <p className="text-xs text-slate-500">Rate and share your review for this course.</p>
336 |       </div>
337 |
338 |       <div className="space-y-3.5">
339 |         {/* Star Rating Picker */}
340 |         <div className="flex flex-col items-center justify-center py-3 bg-orange-50/40 rounded-2xl border border-orange-100/8
0">
341 |           <div className="flex items-center gap-1">
342 |             {[1, 2, 3, 4, 5].map(star => (
343 |               <button
344 |                 key={star}
345 |                 type="button"
346 |                 onMouseEnter={() => setHoverRating(star)}
347 |                 onMouseLeave={() => setHoverRating(0)}
348 |                 onClick={() => setRating(star)}
349 |                 className="p-1 hover:scale-110 transition-transform focus:outline-hidden"
350 |               >
351 |                 <Star
352 |                   className={`w-6 h-6 ${
353 |                     star <= (hoverRating || rating)
354 |                       ? 'fill-orange-400 text-orange-400'
355 |                       : 'text-slate-300 hover:text-orange-300'
356 |                   }`}
357 |                 />
358 |               </button>
359 |             ))}
360 |           </div>
361 |           <span className="text-[10px] font-bold text-orange-600 mt-1">
362 |             {hoverRating || rating ? `${hoverRating || rating} / 5 Stars` : 'Select your rating'}
363 |           </span>
364 |         </div>
365 |
366 |         {/* Optional Comments */}
367 |         <div className="space-y-1">
368 |           <label className="text-[11px] font-semibold text-slate-600">
```

```
369 |             Comments <span className="text-slate-400 font-normal">(Optional)</span>
370 |           </label>
371 |           <Textarea
372 |             placeholder="How was your learning experience?"
373 |             className="resize-none border-slate-200 text-slate-900 bg-slate-50 focus:bg-white rounded-xl text-xs"
374 |             rows={2}
375 |             value={comments}
376 |             onChange={e => setComments(e.target.value)}
377 |           />
378 |         </div>
379 |
380 |         {/* Buttons */}
381 |         <div className="flex gap-2 pt-1">
382 |           {isEditing && (
383 |             <Button
384 |               type="button"
385 |               variant="outline"
386 |               onClick={() => setIsEditing(false)}
387 |               className="flex-1 border-slate-200 text-slate-700 rounded-xl text-xs font-semibold h-9"
388 |             >
389 |               Cancel
390 |             </Button>
391 |           )}
392 |           <Button
393 |             type="button"
394 |             onClick={() => submitFeedback.mutate()}
395 |             disabled={submitFeedback.isPending || rating === 0}
396 |             className="flex-1 bg-gradient-to-r from-orange-500 to-amber-500 hover:from-orange-600 hover:to-amber-600 text-whit
e font-bold text-xs h-9 rounded-xl shadow-xs"
397 |           >
398 |             {submitFeedback.isPending ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1.5" /> : isEditing ? 'Update Review'
: 'Submit Review'}
399 |           </Button>
400 |         </div>
401 |       </div>
402 |     </div>
403 |   )
404 | }
405 |
```

src/features/courses/CourseFormDialog.tsx TSX 6.4 KB 184 Lines

```tsx
import React, { useState } from 'react'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from '@/components/ui/select'
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from '@/components/ui/dialog'
import { toast } from 'sonner'

type Course = Database['public']['Tables']['courses']['Row']

const courseSchema = z.object({
  title: z.string().min(3, 'Title must be at least 3 characters'),
  description: z.string().optional(),
  course_type: z.string().min(2, 'Course type is required'),
  department: z.string().optional(),
  duration_minutes: z.coerce.number().positive().optional(),
  passing_score: z.coerce.number().min(1).max(100).optional(),
})

type CourseFormData = z.infer<typeof courseSchema>

interface CourseFormDialogProps {
  open: boolean
  onClose: () => void
  onSaved: () => void
  course?: Course | null
}

export function CourseFormDialog({ open, onClose, onSaved, course }: CourseFormDialogProps) {
  const { user } = useAuth()
  const [saving, setSaving] = useState(false)

  const { register, handleSubmit, setValue, watch, reset, formState: { errors } } = useForm<CourseFormData>({
    resolver: zodResolver(courseSchema),
    defaultValues: {
      title: course?.title ?? '',
      description: course?.description ?? '',
      course_type: course?.course_type ?? 'standard',
      department: course?.department ?? '',
      duration_minutes: course?.duration_minutes ?? undefined,
      passing_score: course?.passing_score ?? 60,
    },
  })

  const courseType = watch('course_type')

  const handleSave = async (data: CourseFormData, status: 'draft' | 'pending_review') => {
    if (!user) return
    setSaving(true)
    try {
      if (course) {
        // Edit existing
        const { error } = await supabase
          .from('courses')
          .update({ ...data, status })
          .eq('id', course.id)
```

```
        if (error) throw error
        toast.success('Course updated successfully')
      } else {
        // Create new
        const { error } = await supabase
          .from('courses')
          .insert({
            title: data.title,
            description: data.description,
            course_type: data.course_type,
            department: data.department,
            duration_minutes: data.duration_minutes,
            passing_score: data.passing_score,
            trainer_id: user.id,
            status
          })
        if (error) throw error
        toast.success(status === 'draft' ? 'Course saved as draft' : 'Course submitted for review')
      }
      reset()
      onSaved()
      onClose()
    } catch (err: unknown) {
      const message = err instanceof Error ? err.message : 'Failed to save course'
      toast.error(message)
    } finally {
      setSaving(false)
    }
  }

  return (
    <Dialog open={open} onOpenChange={(o) => { if (!o) onClose() }}>
      <DialogContent className="max-w-lg">
        <DialogHeader>
          <DialogTitle>{course ? 'Edit Course' : 'Create New Course'}</DialogTitle>
          <DialogDescription>
            Fill in the course details. You can save as a draft or submit for admin review.
          </DialogDescription>
        </DialogHeader>

        <form className="space-y-4">
          <div className="space-y-1">
            <Label htmlFor="title">Course Title *</Label>
            <Input id="title" {...register('title')} placeholder="e.g. Cyclone Response Protocol" />
            {errors.title && <p className="text-xs text-destructive">{errors.title.message}</p>}
          </div>

          <div className="space-y-1">
            <Label htmlFor="description">Description</Label>
            <Textarea id="description" {...register('description')} placeholder="Brief course overview..." rows={3} />
          </div>

          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-1">
              <Label>Course Type *</Label>
              <Select value={courseType} onValueChange={(v) => setValue('course_type', v)}>
                <SelectTrigger>
                  <SelectValue placeholder="Select type" />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="standard">Standard</SelectItem>
                  <SelectItem value="scenario">Scenario Training</SelectItem>
                </SelectContent>
              </Select>
            </div>

            <div className="space-y-1">
              <Label htmlFor="department">Department</Label>
              <Input id="department" {...register('department')} placeholder="e.g. Meteorology" />
            </div>
          </div>

          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-1">
              <Label htmlFor="duration_minutes">Duration (minutes)</Label>
              <Input id="duration_minutes" type="number" {...register('duration_minutes')} placeholder="e.g. 90" />
              {errors.duration_minutes && <p className="text-xs text-destructive">{errors.duration_minutes.message}</p>}
            </div>

```

```
            <div className="space-y-1">
              <Label htmlFor="passing_score">Passing Score (%)</Label>
              <Input id="passing_score" type="number" {...register('passing_score')} placeholder="60" />
              {errors.passing_score && <p className="text-xs text-destructive">{errors.passing_score.message}</p>}
            </div>
          </div>
        </form>

        <DialogFooter className="flex gap-2">
          <Button variant="outline" onClick={onClose} disabled={saving}>Cancel</Button>
          <Button
            variant="secondary"
            onClick={handleSubmit((data) => handleSave(data, 'draft'))}
            disabled={saving}
          >
            Save as Draft
          </Button>
          <Button
            onClick={handleSubmit((data) => handleSave(data, 'pending_review'))}
            disabled={saving}
          >
            {saving ? 'Submitting...' : 'Submit for Review'}
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  )
}
```

src/features/courses/CourseMaterials.tsx TSX 29.8 KB 738 Lines

```tsx
import React, { useEffect, useState, useCallback, useRef } from 'react'
import { useParams, Link, useNavigate, useSearchParams } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { generateQuestionsFromMaterial, extractTextFromFile } from '@/lib/ai'
import { Database } from '@/integrations/supabase/types'
import { TrainerLayout } from '@/features/trainer/TrainerLayout'
import { Card, CardContent } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
} from '@/components/ui/dialog'
import {
  ArrowLeft, FileText, Trash2, Loader2, Download, Upload, Video, File, Image,
  AlertCircle, CheckCircle, Link2, ExternalLink, Globe, Plus, Sparkles, Brain
} from 'lucide-react'
import { toast } from 'sonner'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { formatDistanceToNow } from 'date-fns'
import { useConfirm } from '@/hooks/useConfirm'

type Course = Database['public']['Tables']['courses']['Row']
type Material = Database['public']['Tables']['materials']['Row']

const ACCEPTED_TYPES = [
  '.pdf', '.doc', '.docx', '.pptx', '.ppt', '.txt',
  '.mp4', '.webm', '.mov',
  '.png', '.jpg', '.jpeg', '.gif', '.webp',
].join(',')

const MAX_FILE_SIZE = 100 * 1024 * 1024 // 100MB

function getFileIcon(mime: string | null) {
  if (!mime) return File
  if (mime.startsWith('video/')) return Video
  if (mime.startsWith('image/')) return Image
  if (mime.includes('pdf')) return FileText
  if (mime.includes('word') || mime.includes('document')) return FileText
  if (mime.includes('presentation') || mime.includes('powerpoint')) return FileText
  return File
}

function formatFileSize(bytes: number | null) {
  if (!bytes) return 'Unknown'
  if (bytes < 1024) return `${bytes} B`
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`
}

interface CourseMaterialsProps {
  embedded?: boolean
  onMaterialCountChange?: (count: number) => void
}

export function CourseMaterials({ embedded = false, onMaterialCountChange }: CourseMaterialsProps) {
  const { courseId } = useParams<{ courseId: string }>()
  const [searchParams] = useSearchParams()
  const sessionId = searchParams.get('session')
  const { user, profile } = useAuth()
  const [course, setCourse] = useState<Course | null>(null)
  const [materials, setMaterials] = useState<Material[]>([])
  const [loading, setLoading] = useState(true)
  const [uploading, setUploading] = useState(false)
  const [uploadProgress, setUploadProgress] = useState<string | null>(null)
  const [deletingId, setDeletingId] = useState<string | null>(null)
  const [downloadingId, setDownloadingId] = useState<string | null>(null)
  const [previewMaterial, setPreviewMaterial] = useState<Material | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [dragOver, setDragOver] = useState(false)
  const [activeTab, setActiveTab] = useState<'files' | 'links'>('files')
  const [linkUrl, setLinkUrl] = useState('')
  const [linkTitle, setLinkTitle] = useState('')
  const [addingLink, setAddingLink] = useState(false)
```

```
  const [generatingForMaterial, setGeneratingForMaterial] = useState<string | null>(null)
  const [generatedQuestions, setGeneratedQuestions] = useState<any[]>([])
  const [reviewOpen, setReviewOpen] = useState(false)
  const [selectedQuestions, setSelectedQuestions] = useState<Set<number>>(new Set())
  const [savingQuestions, setSavingQuestions] = useState(false)
  const fileInputRef = useRef<HTMLInputElement>(null)
  const [ConfirmDialog, confirm] = useConfirm()
  const onCountChangeRef = useRef(onMaterialCountChange)
  onCountChangeRef.current = onMaterialCountChange

  const fetchData = useCallback(async () => {
    if (!user || !courseId) return
    setLoading(true)
    try {
      const { data: courseData } = await supabase
        .from('courses')
        .select('*')
        .eq('id', courseId)
        .eq('trainer_id', user.id)
        .single()
      if (courseData) setCourse(courseData)

      let query = supabase
        .from('materials')
        .select('*')
        .eq('course_id', courseId)
      if (sessionId) {
        query = query.eq('session_id', sessionId)
      }
      const { data: materialsData } = await query.order('created_at', { ascending: false })
      if (materialsData) {
        setMaterials(materialsData)
        onCountChangeRef.current?.(materialsData.length)
      }
    } catch {
      if (!embedded) toast.error('Failed to load materials')
    } finally {
      setLoading(false)
    }
  }, [user, courseId, embedded, sessionId])

  useEffect(() => { fetchData() }, [fetchData])

  const validateFile = (file: File): string | null => {
    if (file.size > MAX_FILE_SIZE) return `File too large (${formatFileSize(file.size)}). Max 100MB.`
    const ext = '.' + file.name.split('.').pop()?.toLowerCase()
    const allowed = ['.pdf', '.doc', '.docx', '.pptx', '.ppt', '.txt', '.mp4', '.webm', '.mov', '.png', '.jpg', '.jpeg', '.gi
f', '.webp']
    if (!allowed.includes(ext)) return `File type not supported: ${ext}`
    return null
  }

  const uploadFile = async (file: File) => {
    if (!user || !courseId) return
    const error = validateFile(file)
    if (error) {
      toast.error(error)
      return
    }

    setUploading(true)
    setUploadProgress(file.name)
    try {
      const fileExt = file.name.split('.').pop()
      const materialId = crypto.randomUUID()
      const storagePath = `${courseId}/${materialId}.${fileExt}`

      const { error: uploadError } = await supabase.storage
        .from('materials')
        .upload(storagePath, file)
      if (uploadError) throw uploadError

      const { error: dbError } = await supabase
        .from('materials')
        .insert({
          id: materialId,
          course_id: courseId,
          uploaded_by: user.id,
          file_name: file.name,
```

```
          storage_path: storagePath,
          mime_type: file.type || 'application/octet-stream',
          file_size: file.size,
          extraction_status: 'pending',
          session_id: sessionId || null
        })
      if (dbError) throw dbError

      toast.success(`Uploaded: ${file.name}`)
      fetchData()
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Upload failed')
    } finally {
      setUploading(false)
      setUploadProgress(null)
    }
  }

  const handleFileInput = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = e.target.files
    if (!files) return
    Array.from(files).forEach(uploadFile)
    if (fileInputRef.current) fileInputRef.current.value = ''
  }

  const handleDrop = (e: React.DragEvent) => {
    e.preventDefault()
    setDragOver(false)
    const files = e.dataTransfer.files
    Array.from(files).forEach(uploadFile)
  }

  const addLink = async () => {
    if (!user || !courseId) return
    const url = linkUrl.trim()
    if (!url) return

    try {
      new URL(url)
    } catch {
      toast.error('Please enter a valid URL')
      return
    }

    setAddingLink(true)
    try {
      const title = linkTitle.trim() || new URL(url).hostname
      const { error } = await supabase.from('materials').insert({
        course_id: courseId,
        uploaded_by: user.id,
        file_name: title,
        storage_path: `link/${crypto.randomUUID()}`,
        material_type: 'link',
        url,
        mime_type: 'text/uri-list',
        extraction_status: 'completed',
        session_id: sessionId || null
      })
      if (error) throw error
      toast.success('Link added')
      setLinkUrl('')
      setLinkTitle('')
      fetchData()
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed to add link')
    } finally {
      setAddingLink(false)
    }
  }

  const handleGenerateQuestions = async (material: Material) => {
    if (!courseId) return
    setGeneratingForMaterial(material.id)
    try {
      let content = ''

      if (material.material_type === 'file' && material.mime_type) {
        // Download file and extract text
```

```
        const { data: signedUrlData } = await supabase.storage
          .from('materials')
          .createSignedUrl(material.storage_path, 3600)

        if (signedUrlData) {
          const response = await fetch(signedUrlData.signedUrl)
          const blob = await response.blob()
          const file = new window.File([blob], material.file_name, { type: material.mime_type })
          content = await extractTextFromFile(file, material.mime_type)
        }
      } else if (material.url) {
        content = `External resource: ${material.url}\nTitle: ${material.file_name}`
      }

      if (!content || content.length < 20) {
        toast.error('Not enough content to generate questions. Try uploading a text-based file.')
        return
      }

      const result = await generateQuestionsFromMaterial(
        material.id,
        courseId,
        content,
        material.file_name
      )

      if (result.questions && result.questions.length > 0) {
        setGeneratedQuestions(result.questions)
        setSelectedQuestions(new Set(result.questions.map((_: any, i: number) => i)))
        setReviewOpen(true)
        toast.success(`Generated ${result.questions.length} questions from "${material.file_name}"`)
      } else {
        toast.error('No questions could be generated from this material')
      }
    } catch (err) {
      toast.error(err instanceof Error ? err.message : 'Failed to generate questions')
    } finally {
      setGeneratingForMaterial(null)
    }
  }

  const handleSaveAcceptedQuestions = async () => {
    if (!courseId || generatedQuestions.length === 0) return
    setSavingQuestions(true)
    try {
      // Find or create assessment

      let { data: assessment } = await supabase
        .from('assessments')
        .select('id')
        .eq('course_id', courseId)
        .single()

      if (!assessment) {
        const { data: newAssessment } = await supabase
          .from('assessments')
          .insert({
            course_id: courseId,
            title: 'Course Assessment',
            passing_score: 60,
            created_by: user!.id,
          })
          .select('id')
          .single()
        assessment = newAssessment
      }

      if (!assessment) throw new Error('Failed to create assessment')

      // Get current max position

      const { data: existing } = await supabase
        .from('questions')
        .select('position')
        .eq('assessment_id', assessment.id)
        .order('position', { ascending: false })
        .limit(1)

      const maxPos = existing?.[0]?.position ?? 0
```

```
310 |
311 |       // Insert only selected questions

312 |       const toInsert = generatedQuestions
313 |         .filter((_: any, i: number) => selectedQuestions.has(i))
314 |         .map((q: any, i: number) => ({
315 |           assessment_id: assessment!.id,
316 |           question_text: q.question_text,
317 |           options: q.options,
318 |           correct_answer: q.correct_answer,
319 |           explanation: q.explanation,
320 |           position: maxPos + i + 1,
321 |           approved: false,
322 |         }))
323 |
324 |       if (toInsert.length > 0) {
325 |         const { error } = await supabase.from('questions').insert(toInsert)
326 |         if (error) throw error
327 |         toast.success(`Added ${toInsert.length} questions to assessment`)
328 |       }
329 |
330 |       setReviewOpen(false)
331 |       setGeneratedQuestions([])
332 |       setSelectedQuestions(new Set())
333 |     } catch (err) {
334 |       toast.error(err instanceof Error ? err.message : 'Failed to save questions')
335 |     } finally {
336 |       setSavingQuestions(false)
337 |     }
338 |   }
339 |
340 |   const toggleQuestionSelection = (index: number) => {
341 |     setSelectedQuestions(prev => {
342 |       const next = new Set(prev)
343 |       if (next.has(index)) next.delete(index)
344 |       else next.add(index)
345 |       return next
346 |     })
347 |   }
348 |
349 |   const handleDownload = async (material: Material) => {
350 |     setDownloadingId(material.id)
351 |     try {
352 |       const { data, error } = await supabase.storage
353 |         .from('materials')
354 |         .createSignedUrl(material.storage_path, 3600)
355 |       if (error) throw error
356 |
357 |       const link = document.createElement('a')
358 |       link.href = data.signedUrl
359 |       link.download = material.file_name
360 |       link.target = '_blank'
361 |       document.body.appendChild(link)
362 |       link.click()
363 |       document.body.removeChild(link)
364 |     } catch {
365 |       toast.error('Failed to generate download link')
366 |     } finally {
367 |       setDownloadingId(null)
368 |     }
369 |   }
370 |
371 |   const handlePreview = async (material: Material) => {
372 |     if (material.material_type === 'link') {
373 |       window.open(material.url!, '_blank')
374 |       return
375 |     }
376 |     setPreviewMaterial(material)
377 |     setPreviewUrl(null)
378 |     try {
379 |       const { data, error } = await supabase.storage
380 |         .from('materials')
381 |         .createSignedUrl(material.storage_path, 3600)
382 |       if (error) throw error
383 |       setPreviewUrl(data.signedUrl)
384 |     } catch {
385 |       toast.error('Failed to load preview')
386 |       setPreviewMaterial(null)
387 |     }
```

```
  }

  const handleDelete = async (material: Material) => {
    const isConfirmed = await confirm(`Are you sure you want to delete "${material.file_name}"?`, 'Delete Material')
    if (!isConfirmed) return

    setDeletingId(material.id)
    try {
      if (material.material_type === 'file') {
        await supabase.storage.from('materials').remove([material.storage_path])
      }
      const { error } = await supabase.from('materials').delete().eq('id', material.id)
      if (error) throw error
      toast.success('Material deleted')
      setMaterials(prev => prev.filter(m => m.id !== material.id))
      onMaterialCountChange?.(materials.length - 1)
    } catch {
      toast.error('Failed to delete')
    } finally {
      setDeletingId(null)
    }
  }

  if (loading) {
    return embedded ? (
      <div className="flex justify-center py-8"><Loader2 className="w-5 h-5 animate-spin text-cyan-600" /></div>
    ) : (
      <TrainerLayout>
        <div className="flex justify-center py-20"><Loader2 className="w-6 h-6 animate-spin text-cyan-600" /></div>
      </TrainerLayout>
    )
  }

  const content = (
    <div className={embedded ? '' : 'max-w-5xl mx-auto space-y-6'}>
      {!embedded && (
        <div>
          <Link to={(profile?.role === 'admin' || profile?.role === 'super_admin') ? `/admin/courses/${courseId}` : `/trainer/
courses/${courseId}`} className="flex items-center gap-2 text-sm text-slate-500 hover:text-cyan-700 font-semibold transition-colors
mb-4">
            <ArrowLeft className="w-4 h-4" /> Back to Course
          </Link>
          <h2 className="text-2xl font-bold tracking-tight text-slate-900">Course Materials</h2>
          <p className="text-slate-500 text-sm mt-1">Upload documents, videos, and resources for "{course?.title}"</p>
        </div>
      )}

      {/* Tabs */}
      <div className="flex gap-1.5 p-1 bg-slate-100 border border-slate-200 rounded-xl w-fit">
        <button
          onClick={() => setActiveTab('files')}
          className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
            activeTab === 'files'
              ? 'bg-white text-cyan-700 shadow-xs border border-slate-200/80'
              : 'text-slate-600 hover:text-slate-900'
          }`}
        >
          <FileText className="w-4 h-4 mr-1.5 inline" />
          Files
        </button>
        <button
          onClick={() => setActiveTab('links')}
          className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
            activeTab === 'links'
              ? 'bg-white text-cyan-700 shadow-xs border border-slate-200/80'
              : 'text-slate-600 hover:text-slate-900'
          }`}
        >
          <Link2 className="w-4 h-4 mr-1.5 inline" />
          Links & Videos
        </button>
      </div>

      {activeTab === 'files' && (
        <div
          className={`relative border-2 border-dashed rounded-2xl p-8 text-center transition-all cursor-pointer ${
            dragOver ? 'border-cyan-500 bg-cyan-50/50' : 'border-slate-300 bg-slate-50/60 hover:border-cyan-500 hover:bg-cyan-
50/20'
          }`}
```

```
            onDragOver={e => { e.preventDefault(); setDragOver(true) }}
            onDragLeave={() => setDragOver(false)}
            onDrop={handleDrop}
            onClick={() => fileInputRef.current?.click()}
          >
            <input
              ref={fileInputRef}
              type="file"
              multiple
              accept={ACCEPTED_TYPES}
              className="hidden"
              onChange={handleFileInput}
              disabled={uploading}
            />
            <Upload className={`w-8 h-8 mx-auto mb-3 ${dragOver ? 'text-cyan-600 animate-bounce' : 'text-cyan-600/70'}`} />
            {uploading ? (
              <div>
                <p className="text-sm text-slate-800 font-bold">Uploading: {uploadProgress}</p>
                <Loader2 className="w-4 h-4 animate-spin text-cyan-600 mx-auto mt-2" />
              </div>
            ) : (
              <div>
                <p className="text-sm text-slate-800 font-bold mb-1">
                  {dragOver ? 'Drop files here' : 'Click to upload or drag and drop'}
                </p>
                <p className="text-xs text-slate-500 font-medium">
                  PDF, DOCX, PPTX, TXT, MP4, Images — Max 100MB each
                </p>
              </div>
            )}
          </div>
        )}

        {activeTab === 'links' && (
          <Card className="bg-white border border-slate-200 shadow-sm rounded-2xl">
            <CardContent className="p-5 space-y-3">
              <p className="text-xs text-slate-600 font-medium">Add links to external resources, YouTube videos, or any online material.</p>
              <div className="grid grid-cols-1 sm:grid-cols-[1fr_1fr_auto] gap-2.5">
                <Input
                  value={linkUrl}
                  onChange={e => setLinkUrl(e.target.value)}
                  onKeyDown={e => { if (e.key === 'Enter') { e.preventDefault(); addLink() } }}
                  placeholder="https://example.com/resource"
                  className="bg-slate-50 border-slate-300 text-slate-900 placeholder:text-slate-400 h-10 rounded-xl text-xs font-medium"
                  disabled={addingLink}
                />
                <Input
                  value={linkTitle}
                  onChange={e => setLinkTitle(e.target.value)}
                  onKeyDown={e => { if (e.key === 'Enter') { e.preventDefault(); addLink() } }}
                  placeholder="Title (optional)"
                  className="bg-slate-50 border-slate-300 text-slate-900 placeholder:text-slate-400 h-10 rounded-xl text-xs font-medium"
                  disabled={addingLink}
                />
                <Button
                  type="button"
                  onClick={addLink}
                  disabled={!linkUrl.trim() || addingLink}
                  className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold h-10 px-4 rounded-xl shadow-sm shrink-0"
                >
                  {addingLink ? <Loader2 className="w-4 h-4 animate-spin" /> : <Plus className="w-4 h-4" />}
                </Button>
              </div>
            </CardContent>
          </Card>
        )}

        <div className="flex items-center gap-2.5 p-3.5 rounded-2xl bg-cyan-50/70 border border-cyan-200/80">
          <Brain className="w-4 h-4 text-cyan-700 shrink-0" />
          <p className="text-xs text-slate-700 font-medium leading-relaxed">
            <span className="font-bold text-cyan-900">AI Question Generation:</span> Click the <Sparkles className="w-3 h-3 inline text-cyan-600" /> icon on any material to auto-generate assessment questions from its content.
          </p>
        </div>

```

```
538 |       {materials.length > 0 && (
539 |         <div className="space-y-3">
540 |           <div className="flex items-center justify-between">
541 |             <h3 className="text-sm font-bold text-slate-900">
542 |               Materials ({materials.length})
543 |               {materials.some(m => m.material_type === 'file') && materials.some(m => m.material_type !== 'file') && (
544 |                 <span className="text-slate-500 font-normal ml-2">
545 |                   — {materials.filter(m => m.material_type === 'file').length} files, {materials.filter(m => m.material_type !
== 'file').length} links
546 |                 </span>
547 |               )}
548 |             </h3>
549 |           </div>
550 |           <div className="space-y-2.5">
551 |             {materials.map(material => {
552 |               const isLink = material.material_type === 'link' || material.material_type === 'video'
553 |               const isVideo = material.mime_type?.startsWith('video/') || material.material_type === 'video'
554 |               const Icon = isLink ? Globe : isVideo ? Video : getFileIcon(material.mime_type)
555 |               return (
556 |                 <div key={material.id} className="flex items-center gap-3.5 p-3.5 rounded-2xl bg-white border border-slate-200
hover:border-cyan-300 hover:shadow-xs transition-all group">
557 |                   <div className={`w-10 h-10 rounded-xl flex items-center justify-center shrink-0 border ${
558 |                     isLink ? 'bg-emerald-50 border-emerald-200 text-emerald-600' : isVideo ? 'bg-blue-50 border-blue-200 text-
blue-600' : 'bg-cyan-50 border-cyan-200 text-cyan-600'
559 |                   }`}>
560 |                     <Icon className="w-5 h-5" />
561 |                   </div>
562 |                   <div className="flex-1 min-w-0 cursor-pointer" onClick={() => handlePreview(material)}>
563 |                     <div className="flex items-center gap-2">
564 |                       <p className="text-sm text-slate-900 font-bold truncate hover:text-cyan-700 transition-colors">{materia
l.file_name}</p>
565 |                       {isLink && (
566 |                         <Badge className="bg-emerald-50 text-emerald-700 border-emerald-200 text-[10px] h-4 font-bold">
567 |                           <Link2 className="w-2.5 h-2.5 mr-0.5 inline" />
568 |                           Link
569 |                         </Badge>
570 |                       )}
571 |                     </div>
572 |                     <div className="flex items-center gap-2 text-xs text-slate-500 mt-0.5">
573 |                       {isLink ? (
574 |                         <span className="text-slate-600 hover:text-cyan-600 truncate max-w-xs flex items-center gap-1 font-med
ium">
575 |                           {material.url}
576 |                           <ExternalLink className="w-3 h-3 shrink-0" />
577 |                         </span>
578 |                       ) : (
579 |                         <>
580 |                           <span>{formatFileSize(material.file_size)}</span>
581 |                           <span>&middot;</span>
582 |                         </>
583 |                       )}
584 |                       <span>{formatDistanceToNow(new Date(material.created_at), { addSuffix: true })}</span>
585 |                     </div>
586 |                   </div>
587 |                   <div className="flex items-center gap-1.5 shrink-0">
588 |                     <Link
589 |                       to={`/trainer/courses/${courseId}/assessments`}
590 |                       className="p-2 rounded-xl hover:bg-cyan-50 text-slate-400 hover:text-cyan-700 transition-all border bord
er-transparent hover:border-cyan-200"
591 |                       title="Go to Assessments"
592 |                     >
593 |                       <Sparkles className="w-4 h-4" />
594 |                     </Link>
595 |                     {!isLink && (
596 |                       <button
597 |                         onClick={() => handleDownload(material)}
598 |                         disabled={downloadingId === material.id}
599 |                         className="p-2 rounded-xl hover:bg-slate-100 text-slate-400 hover:text-slate-700 transition-all border
border-transparent hover:border-slate-200"
600 |                         title="Download"
601 |                       >
602 |                         {downloadingId === material.id ? <Loader2 className="w-4 h-4 animate-spin text-cyan-600" /> : <Downloa
d className="w-4 h-4" />}
603 |                       </button>
604 |                     )}
605 |                     <button
606 |                       onClick={() => handleDelete(material)}
607 |                       disabled={deletingId === material.id}
608 |                       className="p-2 rounded-xl hover:bg-rose-50 text-slate-400 hover:text-rose-600 transition-all border bord
```

```
er-transparent hover:border-rose-200"
                      title="Delete"
                    >
                      {deletingId === material.id ? <Loader2 className="w-4 h-4 animate-spin" /> : <Trash2 className="w-4 h-4"
/>}
                    </button>
                  </div>
                </div>
              )
            })}
          </div>
        </div>
      )}

      {materials.length === 0 && !uploading && (
        <div className="text-center py-12 bg-slate-50 rounded-2xl border border-slate-200">
          <FileText className="w-10 h-10 text-slate-300 mx-auto mb-3" />
          <p className="text-slate-700 font-bold text-sm">No materials added yet.</p>
          <p className="text-slate-400 text-xs mt-1 font-medium">Upload files or add links to build your course content.</p>
        </div>
      )}
    </div>
  )

  return (
    <>
      {embedded ? content : <TrainerLayout>{content}</TrainerLayout>}

      <ConfirmDialog />

      {/* AI Question Review Dialog */}
      <Dialog open={reviewOpen} onOpenChange={setReviewOpen}>
        <DialogContent className="max-w-2xl max-h-[80vh] overflow-y-auto bg-slate-900 border-slate-700 text-white shadow-2xl">
          <DialogHeader>
            <DialogTitle className="text-white flex items-center gap-2">
              <Brain className="w-5 h-5 text-cyan-400" />
              AI-Generated Questions ({generatedQuestions.length})
            </DialogTitle>
          </DialogHeader>
          <div className="space-y-3">
            <p className="text-xs text-slate-400">
              Review and select questions to add to your assessment. Uncheck questions you don't want.
            </p>
            {generatedQuestions.map((q: any, i: number) => (
              <div
                key={i}
                className={`p-3.5 rounded-xl border transition-all ${
                  selectedQuestions.has(i)
                    ? 'bg-slate-800/80 border-cyan-500/50 shadow-xs'
                    : 'bg-slate-950/60 border-slate-800 opacity-50'
                }`}
              >
                <div className="flex items-start gap-3">
                  <input
                    type="checkbox"
                    checked={selectedQuestions.has(i)}
                    onChange={() => toggleQuestionSelection(i)}
                    className="mt-1 rounded border-slate-600 bg-slate-800 text-cyan-500 focus:ring-cyan-500 accent-cyan-500"
                  />
                  <div className="flex-1">
                    <p className="text-sm text-slate-100 font-bold mb-2">{q.question_text}</p>
                    <div className="grid grid-cols-2 gap-1.5 text-xs">
                      {['A', 'B', 'C', 'D'].map(opt => (
                        <div
                          key={opt}
                          className={`px-2.5 py-1.5 rounded-lg border ${
                            q.correct_answer === opt
                              ? 'bg-emerald-950/70 border-emerald-500 text-emerald-300 font-semibold'
                              : 'bg-slate-900 border-slate-800 text-slate-300'
                          }`}
                        >
                          <span className="font-bold mr-1">{opt}.</span> {q.options?.[opt]}
                        </div>
                      ))}
                    </div>
                    {q.explanation && (
                      <p className="text-xs text-slate-400 mt-2 italic">
                        <span className="text-slate-300 font-semibold">Explanation:</span> {q.explanation}
                      </p>
```

```
                    ))}
                  </div>
                </div>
              </div>
            ))}
          </div>
          <DialogFooter className="border-t border-slate-800 pt-3 flex items-center justify-end gap-2">
            <Button
              variant="outline"
              onClick={() => setReviewOpen(false)}
              className="border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700 font-semibold text-xs"
            >
              Cancel
            </Button>
            <Button
              onClick={handleSaveAcceptedQuestions}
              disabled={savingQuestions || selectedQuestions.size === 0}
              className="bg-gradient-to-r from-cyan-500 to-blue-600 text-white font-bold text-xs"
            >
              {savingQuestions ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Sparkles className="w-4 h-4 mr-2" />}
              Add {selectedQuestions.size} Question{selectedQuestions.size !== 1 ? 's' : ''}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

      <MaterialPreviewDialog
        material={previewMaterial}
        previewUrl={previewUrl}
        onClose={() => setPreviewMaterial(null)}
        onDownload={() => previewMaterial && handleDownload(previewMaterial)}
      />
    </>
  )
}

function VideoPreview({ storagePath }: { storagePath: string }) {
  const [url, setUrl] = useState<string | null>(null)

  useEffect(() => {
    supabase.storage.from('materials').createSignedUrl(storagePath, 3600).then(({ data }) => {
      if (data) setUrl(data.signedUrl)
    })
  }, [storagePath])

  if (!url) return null

  return (
    <video controls preload="metadata" className="w-full max-w-md rounded-lg" src={url}>
      <track kind="captions" />
    </video>
  )
}
```

src/features/courses/LiveAttendanceTraineePanel.tsx TSX 9.4 KB 266 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { UserCheck, Clock, Check, ShieldAlert } from 'lucide-react'
import { toast } from 'sonner'
import { motion, AnimatePresence } from 'framer-motion'
import { Badge } from '@/components/ui/badge'
import { supabase } from '@/lib/supabase'

export function LiveAttendanceTraineePanel({ session, userId }: { session: any, userId: string }) {
  const [activeOtp, setActiveOtp] = useState<any>(null)
  const [timeLeft, setTimeLeft] = useState(0)
  const [inputValue, setInputValue] = useState('')
  const [tally, setTally] = useState({ entered: 0, generated: 0, isManualMapping: false })
  const [override, setOverride] = useState<'P' | 'F' | null>(null)
  const [channel, setChannel] = useState<any>(null)

  // Sync with Trainer via Supabase Presence
  useEffect(() => {
    if (!session?.id) return

    // Load from DB
    const loadTraineeData = async () => {
      const [{ data: metaData }, { data: attendanceData }] = await Promise.all([
        (supabase as any).from('session_attendance_meta').select('*').eq('session_id', session.id).maybeSingle(),
        (supabase as any).from('session_attendance').select('*').eq('session_id', session.id).eq('user_id', userId).maybeSingl
e()
      ])

      let generated = 0
      let isManualMapping = false
      if (metaData) {
        generated = metaData.total_generated || 0
        isManualMapping = metaData.is_manual_mapping || false
      }

      let entered = 0
      let overrideVal = null
      if (attendanceData) {
        entered = attendanceData.entered_count || 0
        overrideVal = attendanceData.status_override || null
      }

      setTally(prev => ({ ...prev, entered, generated, isManualMapping }))
      if (overrideVal) {
        setOverride(overrideVal as any)
      }
    }
    loadTraineeData()

    const ch = supabase.channel(`attendance_${session.id}`)

    ch.on('presence', { event: 'sync' }, () => {
      const state = ch.presenceState()
      if (state['trainer'] && state['trainer'].length > 0) {
        const ts: any = state['trainer'][0]
        setTally(prev => {
          const newGen = ts.generated !== undefined ? ts.generated : prev.generated
          const isManualMapping = ts.isManualMapping !== undefined ? ts.isManualMapping : prev.isManualMapping
          return { ...prev, generated: newGen, isManualMapping }
        })

        if (ts.activeOtp && ts.expiresAt > Date.now()) {
          setActiveOtp({ code: ts.activeOtp, expiresAt: ts.expiresAt })
          setTimeLeft(Math.ceil((ts.expiresAt - Date.now()) / 1000))
        } else {
          setActiveOtp(null)
        }
      }
    })

    ch.on('broadcast', { event: 'score_adjusted' }, (payload) => {
      const { userId: targetUserId, entered } = payload.payload
      if (targetUserId === userId) {
        setTally(prev => ({ ...prev, entered }))
      }
```

```
    })

    ch.on('broadcast', { event: 'request_sync' }, () => {
      setTally(current => {
        if (current.entered > 0) {
          ch.send({
            type: 'broadcast',
            event: 'trainee_sync',
            payload: { userId, entered: current.entered }
          })
        }
        return current
      })
    })

    ch.subscribe()
    setChannel(ch)

    // Local timer loop for smooth countdown
    const timer = setInterval(() => {
      setTimeLeft(prev => {
        if (prev <= 1) {
          setActiveOtp(null)
          return 0
        }
        return prev - 1
      })
    }, 1000)

    return () => {
      clearInterval(timer)
      supabase.removeChannel(ch)
    }
  }, [session?.id, userId])

  // Broadcast current state to trainer on load or update
  useEffect(() => {
    if (channel && tally.entered > 0) {
      channel.send({
        type: 'broadcast',
        event: 'trainee_sync',
        payload: { userId, entered: tally.entered }
      })
    }
  }, [channel, tally.entered, userId])

  const handleSubmit = async () => {
    if (!activeOtp || timeLeft <= 0) {
      toast.error("The code has expired!")
      return
    }

    if (inputValue.trim() === activeOtp.code) {
      // Prevent double counting the same OTP code using local storage (history only)
      const hKey = `trainee_otp_history_${session.id}_${userId}`
      const history = JSON.parse(localStorage.getItem(hKey) || '[]')
      if (history.includes(activeOtp.code)) {
        toast.error("You already entered this code!")
        setInputValue('')
        return
      }

      const newEntered = tally.entered + 1
      setTally(prev => ({ ...prev, entered: newEntered }))

      history.push(activeOtp.code)
      localStorage.setItem(hKey, JSON.stringify(history))

      await (supabase as any).from('session_attendance').upsert({
        session_id: session.id,
        user_id: userId,
        entered_count: newEntered,
        status_override: null
      }, { onConflict: 'session_id,user_id' })

      setInputValue('')
      toast.success("Attendance code accepted!")

      if (channel) {
```

```
        channel.send({
          type: 'broadcast',
          event: 'trainee_otp_entered',
          payload: { userId, code: activeOtp.code }
        })
      }
    } else {
      toast.error("Invalid code. Please try again.")
    }
  }

  // Auto-status logic
  let autoStatus = 'Absent'
  let autoColor = 'text-rose-600 bg-rose-50'

  const effectiveTotal = (tally.generated === 0 && tally.isManualMapping) ? 1 : tally.generated
  const pct = effectiveTotal > 0 ? Math.round((tally.entered / effectiveTotal) * 100) : 0

  if (pct >= 75) {
    autoStatus = 'Present'
    autoColor = 'text-emerald-600 bg-emerald-50'
  } else if (pct >= 30) {
    autoStatus = 'Partial'
    autoColor = 'text-amber-600 bg-amber-50'
  }
  if (effectiveTotal === 0) {
    autoStatus = 'Pending Checks'
    autoColor = 'text-slate-600 bg-slate-100'
  }

  const finalStatus = override === 'P' ? 'Present (Overridden)' : override === 'F' ? 'Absent (Overridden)' : autoStatus;
  const finalColor = override === 'P' ? 'text-blue-700 bg-blue-50 border-blue-200' : override === 'F' ? 'text-red-700 bg-red-50 border-red-200' : autoColor;

  return (
    <div className="bg-white border border-slate-200 rounded-xl p-5 shadow-sm space-y-4">
      <div className="flex items-center justify-between">
        <h3 className="font-bold text-slate-800 flex items-center gap-2">
          <UserCheck className="w-4 h-4 text-cyan-600" />
          Live Attendance
        </h3>
        <Badge variant="outline" className={finalColor}>
          Status: {finalStatus}
        </Badge>
      </div>

      <div className="flex items-center gap-6 py-2">
        <div className="flex-1">
          <p className="text-xs text-slate-500 font-semibold uppercase tracking-wider mb-1">Your Checks</p>
          <div className="flex items-baseline gap-2">
            {tally.isManualMapping && tally.generated === 0 ? (
               <span className="text-lg font-black text-slate-800">Manual Mode</span>
            ) : (
               <>
                 <span className="text-2xl font-black text-slate-800">{tally.entered}</span>
                 <span className="text-sm font-semibold text-slate-400">/ {tally.generated}</span>
               </>
            )}
          </div>
        </div>
        <div className="flex-1">
          <p className="text-xs text-slate-500 font-semibold uppercase tracking-wider mb-1">Success Rate</p>
          <div className="flex items-baseline gap-2">
            <span className="text-2xl font-black text-slate-800">{pct}%</span>
          </div>
        </div>
      </div>

      <AnimatePresence>
        {activeOtp && (
          <motion.div
            initial={{ opacity: 0, height: 0, y: -10 }}
            animate={{ opacity: 1, height: 'auto', y: 0 }}
            exit={{ opacity: 0, height: 0, y: -10 }}
            className="overflow-hidden"
          >
            <div className="bg-cyan-50 border border-cyan-200 rounded-lg p-4 mt-2">
              <div className="flex items-center justify-between mb-3">
                <p className="text-sm font-bold text-cyan-800 flex items-center gap-1.5">
```

```
                  <ShieldAlert className="w-4 h-4" /> Action Required!
                </p>
                <p className="text-xs font-bold text-rose-600 flex items-center gap-1 animate-pulse">
                  <Clock className="w-3.5 h-3.5" />
                  {timeLeft}s
                </p>
              </div>
              <div className="flex gap-2">
                <Input
                  autoFocus
                  placeholder="Enter 6-digit PIN"
                  value={inputValue}
                  onChange={(e) => setInputValue(e.target.value.replace(/\D/g, '').slice(0, 6))}
                  onKeyDown={(e) => e.key === 'Enter' && handleSubmit()}
                  className="text-center font-bold tracking-widest text-lg bg-white border-cyan-300 focus-visible:ring-cyan-500"
                />
                <Button
                  onClick={handleSubmit}
                  className="bg-cyan-600 hover:bg-cyan-700 text-white shrink-0"
                >
                  <Check className="w-4 h-4 mr-1" /> Verify
                </Button>
              </div>
              <p className="text-[10px] text-cyan-600/70 text-center mt-2 font-medium">
                Listen to the trainer for the PIN. You only have {timeLeft} seconds!
              </p>
            </div>
          </motion.div>
        )}
      </AnimatePresence>
    </div>
  )
}
```

src/features/courses/LiveAttendanceTrainerPanel.tsx TSX 16.2 KB 417 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { Button } from '@/components/ui/button'
import { UserCheck, Clock, Check, X, AlertCircle, Plus, Minus } from 'lucide-react'
import { toast } from 'sonner'
import { motion } from 'framer-motion'
import { Badge } from '@/components/ui/badge'
import { supabase } from '@/lib/supabase'
import { useConfirm } from '@/hooks/useConfirm'

export function LiveAttendanceTrainerPanel({ session, enrollments, isCompleted }: { session: any, enrollments: any[], isCompleted?: boolean }) {
  const [activeOtp, setActiveOtp] = useState<string | null>(null)
  const [timeLeft, setTimeLeft] = useState(0)
  const [totalGenerated, setTotalGenerated] = useState(0)
  const [roster, setRoster] = useState<Record<string, { entered: number, override?: 'P' | 'F' }>>({})
  const [channel, setChannel] = useState<any>(null)
  const [ConfirmDialog, confirm] = useConfirm()

  const [isManualMapping, setIsManualMapping] = useState(false)

  // Initialize total generated and real-time channel
  useEffect(() => {
    if (!session?.id) return

    const loadMeta = async () => {
      const { data } = await (supabase as any).from('session_attendance_meta').select('*').eq('session_id', session.id).maybeSingle()
      if (data) {
        setTotalGenerated(data.total_generated || 0)
        setIsManualMapping(data.is_manual_mapping || false)
      }
    }
    loadMeta()

    // Load persisted roster from DB
    const loadRoster = async () => {
      const { data } = await (supabase as any).from('session_attendance').select('user_id, entered_count, status_override').eq('session_id', session.id)
      if (data) {
        const rData: Record<string, { entered: number, override?: 'P' | 'F' }> = {}
        data.forEach((row: any) => {
          rData[row.user_id] = { entered: row.entered_count, override: row.status_override as any }
        })
        setRoster(rData)
      }
    }
    loadRoster()

    const active = JSON.parse(localStorage.getItem(`active_otp_${session.id}`) || 'null')
    if (active && active.expiresAt > Date.now()) {
      setActiveOtp(active.code)
      setTimeLeft(Math.ceil((active.expiresAt - Date.now()) / 1000))
    }
  }, [session?.id])

  // Timer Loop
  useEffect(() => {
    if (timeLeft > 0) {
      const timer = setTimeout(() => setTimeLeft(timeLeft - 1), 1000)
      return () => clearTimeout(timer)
    } else if (timeLeft === 0 && activeOtp) {
      setActiveOtp(null)
    }
  }, [timeLeft, activeOtp])

  // Setup Supabase Realtime for cross-browser sync
  useEffect(() => {
    if (!session?.id) return
    const ch = supabase.channel(`attendance_${session.id}`, {
      config: { presence: { key: 'trainer' } }
    })

    ch.on('broadcast', { event: 'trainee_otp_entered' }, async (payload) => {
      const { userId } = payload.payload
      setRoster(prev => {
        const curr = prev[userId] || { entered: 0 }
```

```
        const next = { ...prev, [userId]: { ...curr, entered: curr.entered + 1 } }
        return next
      })
      // Sync DB
      await (supabase as any).from('session_attendance').upsert({
        session_id: session.id,
        user_id: userId,
        entered_count: (roster[userId]?.entered || 0) + 1
      }, { onConflict: 'session_id,user_id' })
      toast.success('A trainee verified their attendance!')
    })

    ch.on('broadcast', { event: 'trainee_sync' }, async (payload) => {
      const { userId, entered } = payload.payload
      setRoster(prev => {
        const curr = prev[userId] || { entered: 0 }
        if (curr.entered >= entered) return prev
        const next = { ...prev, [userId]: { ...curr, entered } }
        return next
      })

      const curr = roster[userId] || { entered: 0 }
      if (curr.entered < entered) {
        await (supabase as any).from('session_attendance').upsert({
          session_id: session.id,
          user_id: userId,
          entered_count: entered
        }, { onConflict: 'session_id,user_id' })
      }
    })

    ch.subscribe(async (status) => {
      if (status === 'SUBSCRIBED') {
        const active = JSON.parse(localStorage.getItem(`active_otp_${session.id}`) || 'null')
        // Get meta from DB
        const { data } = await (supabase as any).from('session_attendance_meta').select('*').eq('session_id', session.id).maybeSingle()
        const meta = data || { total_generated: 0, is_manual_mapping: false }

        await ch.track({
          activeOtp: active && active.expiresAt > Date.now() ? active.code : null,
          expiresAt: active && active.expiresAt > Date.now() ? active.expiresAt : 0,
          generated: meta.total_generated || 0,
          isManualMapping: meta.is_manual_mapping || false
        })

        // Ask all online trainees to report their current score
        ch.send({
          type: 'broadcast',
          event: 'request_sync',
          payload: {}
        })
      }
    })

    setChannel(ch)
    return () => { supabase.removeChannel(ch) }
  }, [session?.id])

  const generateOTP = () => {
    if (timeLeft > 0) {
      toast.error("An attendance check is already running!")
      return
    }

    const code = Math.floor(100000 + Math.random() * 900000).toString()
    const expiresAt = Date.now() + 30000 // 30 seconds

    // Save active OTP
    localStorage.setItem(`active_otp_${session.id}`, JSON.stringify({ code, expiresAt }))

    // Increment total generated
    const newTotal = totalGenerated + 1
    ;(supabase as any).from('session_attendance_meta').upsert({
      session_id: session.id,
      total_generated: newTotal,
      is_manual_mapping: isManualMapping
    }).then()
    setTotalGenerated(newTotal)
```

```
152 |
153 |     setActiveOtp(code)
154 |     setTimeLeft(30)
155 |
156 |     if (channel) {
157 |       channel.track({ activeOtp: code, expiresAt, generated: newTotal, isManualMapping })
158 |     }
159 |
160 |     toast.success("Attendance check triggered!")
161 |   }
162 |
163 |   const enableManualMapping = async () => {
164 |     await (supabase as any).from('session_attendance_meta').upsert({
165 |       session_id: session.id,
166 |       total_generated: totalGenerated,
167 |       is_manual_mapping: true
168 |     })
169 |     setIsManualMapping(true)
170 |     if (channel) {
171 |       channel.track({ activeOtp: null, expiresAt: 0, generated: totalGenerated, isManualMapping: true })
172 |     }
173 |     toast.success("Manual mapping enabled!")
174 |   }
175 |
176 |   const disableManualMapping = async () => {
177 |     await (supabase as any).from('session_attendance_meta').upsert({
178 |       session_id: session.id,
179 |       total_generated: totalGenerated,
180 |       is_manual_mapping: false
181 |     })
182 |     setIsManualMapping(false)
183 |
184 |     // Reset roster checks to 0
185 |     const nextRoster = { ...roster }
186 |     const updates: any[] = []
187 |     Object.keys(nextRoster).forEach(userId => {
188 |       nextRoster[userId].entered = 0
189 |       updates.push({
190 |         session_id: session.id,
191 |         user_id: userId,
192 |         entered_count: 0
193 |       })
194 |
195 |       if (channel) {
196 |         channel.send({
197 |           type: 'broadcast',
198 |           event: 'score_adjusted',
199 |           payload: { userId, entered: 0 }
200 |         })
201 |       }
202 |     })
203 |     setRoster(nextRoster)
204 |     if (updates.length > 0) {
205 |       await (supabase as any).from('session_attendance').upsert(updates as any, { onConflict: 'session_id,user_id' })
206 |     }
207 |
208 |     if (channel) {
209 |       channel.track({ activeOtp: null, expiresAt: 0, generated: totalGenerated, isManualMapping: false })
210 |     }
211 |     toast.success("Manual mapping disabled")
212 |   }
213 |
214 |   const adjustScore = async (userId: string, newScoreVal: number, traineeName: string) => {
215 |     let finalScore = newScoreVal;
216 |     setRoster(prev => {
217 |       const curr = prev[userId] || { entered: 0 }
218 |       const maxPossible = (totalGenerated === 0 && isManualMapping) ? 1 : totalGenerated
219 |       const newScore = Math.max(0, Math.min(maxPossible, newScoreVal))
220 |       finalScore = newScore;
221 |
222 |       const next = { ...prev, [userId]: { ...curr, entered: newScore } }
223 |       delete next[userId].override // Clear override since we are adjusting score manually
224 |
225 |       return next
226 |     })
227 |
228 |     await (supabase as any).from('session_attendance').upsert({
229 |       session_id: session.id,
230 |       user_id: userId,
```

```
        entered_count: finalScore,
        status_override: null
      }, { onConflict: 'session_id,user_id' })

      if (channel) {
        channel.send({
          type: 'broadcast',
          event: 'score_adjusted',
          payload: { userId, entered: finalScore }
        })
      }
      toast.success(`Score adjusted for ${traineeName}`)
    }

    return (
      <div className="bg-white rounded-3xl shadow-2xl border border-slate-200 overflow-hidden flex flex-col max-h-[85vh]">
        <ConfirmDialog />
        {/* Header Section */}
        <div className="p-6 bg-slate-50 border-b border-slate-200 flex flex-col md:flex-row items-start md:items-center justify-
between gap-4">
          <div>
            <h3 className="font-bold text-slate-800 text-lg flex items-center gap-2">
              <UserCheck className="w-5 h-5 text-cyan-600" />
              Live Attendance Checks
            </h3>
            <p className="text-sm text-slate-500 mt-1">
              Trigger a random 6-digit PIN. Trainees have 30 seconds to enter it.
            </p>
            <div className="mt-3 flex gap-2">
               <Badge variant="outline" className="bg-white text-slate-600 border-slate-200 shadow-sm">
                  {totalGenerated} check{totalGenerated === 1 ? '' : 's'} generated so far
               </Badge>
               {isManualMapping && (
                 <Badge variant="outline" className="bg-amber-50 text-amber-600 border-amber-200 shadow-sm">
                   Manual Mode Active
                 </Badge>
               )}
            </div>
          </div>

          <div className="shrink-0 flex flex-col items-end gap-2">
            {isCompleted ? (
              <div className="flex flex-col items-end gap-2">
                <Badge variant="outline" className="bg-slate-100 text-slate-600 border-slate-200 font-bold px-3 py-1.5 rounded-l
g shadow-none">
                  <Check className="w-3.5 h-3.5 mr-1" /> Session Completed
                </Badge>
                {!isManualMapping ? (
                  <Button
                    onClick={enableManualMapping}
                    variant="outline"
                    className="border-cyan-600 text-cyan-700 hover:bg-cyan-50 font-bold h-10 px-4 rounded-xl"
                  >
                    Enable Manual Mapping
                  </Button>
                ) : (
                  <Button
                    onClick={disableManualMapping}
                    variant="outline"
                    className="border-rose-200 text-rose-600 hover:bg-rose-50 font-bold h-10 px-4 rounded-xl"
                  >
                    Disable Manual Mapping
                  </Button>
                )}
              </div>
            ) : activeOtp ? (
              <div className="text-center">
                <div className="text-4xl font-black tracking-widest text-cyan-600 bg-cyan-50 px-6 py-2 rounded-lg border border-
cyan-200">
                  {activeOtp}
                </div>
                <p className="text-xs font-bold text-rose-600 mt-2 flex items-center justify-center gap-1">
                  <Clock className="w-3.5 h-3.5" />
                  {timeLeft}s remaining
                </p>
              </div>
            ) : (
              <Button
                onClick={generateOTP}
```

```
              className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold h-12 px-6 rounded-xl shadow-sm"
            >
              Take Attendance Now
            </Button>
          )}
        </div>
      </div>

      {/* Roster Section */}
      <div className="flex-1 overflow-hidden flex flex-col bg-white">
        <div className="px-6 py-3 border-b border-slate-100 bg-white flex justify-between items-center shrink-0">
          <span className="font-bold text-sm text-slate-800">Session Roster</span>
          <span className="text-xs text-slate-400">Auto-saves instantly</span>
        </div>
        <div className="divide-y divide-slate-50 overflow-y-auto flex-1">
          {enrollments.map(e => {
            const trainee = e.trainee || { full_name: 'Unknown', email: '' }
            const rData = roster[e.user_id] || { entered: 0 }

            let autoStatus = 'Absent'
            let autoColor = 'text-rose-600 bg-rose-50'

            const effectiveTotal = (totalGenerated === 0 && isManualMapping) ? 1 : totalGenerated
            const pct = effectiveTotal > 0 ? Math.round((rData.entered / effectiveTotal) * 100) : 0

            if (pct >= 75) {
              autoStatus = 'Present'
              autoColor = 'text-emerald-600 bg-emerald-50'
            } else if (pct >= 30) {
              autoStatus = 'Partial'
              autoColor = 'text-amber-600 bg-amber-50'
            }
            if (effectiveTotal === 0) {
              autoStatus = 'Pending'
              autoColor = 'text-slate-600 bg-slate-100'
            }

            const finalStatus = autoStatus;
            const finalColor = autoColor;

            return (
              <div key={e.user_id} className="px-6 py-3 flex flex-col sm:flex-row sm:items-center justify-between gap-4 hover:
bg-slate-50/50 transition-colors">
                <div className="flex items-center gap-3 min-w-0 flex-1">
                  <div className="w-9 h-9 rounded-full bg-gradient-to-br from-cyan-100 to-blue-100 flex items-center justify-c
enter shrink-0">
                    <span className="font-bold text-cyan-700 text-sm">
                      {trainee.full_name?.charAt(0) || '?'}
                    </span>
                  </div>
                  <div className="min-w-0">
                    <p className="font-semibold text-slate-900 text-sm truncate">{trainee.full_name}</p>
                    <p className="text-xs text-slate-500 truncate">{trainee.email}</p>
                  </div>
                </div>

                <div className="flex items-center gap-6 shrink-0">
                  <div className="text-right">
                    <p className="text-xs font-medium text-slate-500">Auto Score</p>
                    <p className="font-bold text-slate-800 text-sm">
                      {isManualMapping && totalGenerated === 0 ? (
                        <span className={rData.entered === 1 ? 'text-emerald-600' : 'text-rose-600'}>
                          {rData.entered === 1 ? 'Present' : 'Absent'}
                        </span>
                      ) : (
                        `${rData.entered} / ${totalGenerated || 0}`
                      )}
                    </p>
                  </div>

                  <div className="w-32">
                    <Badge variant="outline" className={`w-full justify-center ${finalColor}`}>
                      {finalStatus}
                    </Badge>
                  </div>

                  <div className="flex items-center gap-1.5 border-l border-slate-200 pl-4 w-32 flex-wrap justify-end">
                    {Array.from({ length: effectiveTotal }, (_, i) => {
                      const isChecked = i < rData.entered
```

```
                    return (
                      <button
                        key={i}
                        onClick={() => adjustScore(e.user_id, isChecked ? rData.entered - 1 : rData.entered + 1, trainee.ful
l_name)}
                        className={`w-6 h-6 rounded-md border flex items-center justify-center transition-all ${
                          isChecked
                            ? 'bg-emerald-50 border-emerald-200 text-emerald-600 hover:bg-emerald-100'
                            : 'bg-rose-50 border-rose-200 text-rose-500 hover:bg-rose-100'
                        }`}
                        title={isChecked ? "Remove check" : "Add check"}
                      >
                        {isChecked ? <Check className="w-4 h-4" strokeWidth={3} /> : <X className="w-4 h-4" strokeWidth={3}
/>}
                      </button>
                    )
                  })}
                  {effectiveTotal === 0 && (
                    <span className="text-xs text-slate-400 italic">No checks</span>
                  )}
                </div>
              </div>
            </div>
          )
        })}
        {enrollments.length === 0 && (
          <div className="py-12 text-center text-slate-500 text-sm">
            No trainees enrolled in this course yet.
          </div>
        )}
      </div>
    </div>
  </div>
  )
}

```

src/features/courses/TraineeAssessmentResult.tsx TSX 34.4 KB 462 Lines

```tsx
import React, { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { CheckCircle2, XCircle, ArrowLeft, Target, Eye } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog'
import { DashboardShell } from '@/pages/Dashboards'
import { toast } from 'sonner'

interface TraineeAssessmentResultProps {
  assessment: any
  questions: any[]
  attemptData: any
  attemptAnswers: any[]
  areResultsHidden: boolean
  displayTitle: string
  traineeNavLinks: any[]
  profile?: any
  course?: any
  isStandalone?: boolean
}

export function TraineeAssessmentResult({
  assessment,
  questions,
  attemptData,
  attemptAnswers,
  areResultsHidden,
  displayTitle,
  traineeNavLinks,
  profile,
  course,
  isStandalone
}: TraineeAssessmentResultProps) {
  const navigate = useNavigate()
  const [expandedAnswers, setExpandedAnswers] = useState<Record<string, boolean>>({})

  const getAnswerString = (ans: any) => {
     if (!ans || ans.selected_answer === undefined || ans.selected_answer === null) return '';
     if (typeof ans.selected_answer === 'string') return ans.selected_answer;
     if (Array.isArray(ans.selected_answer)) return ans.selected_answer.join(',');
     return String(ans.selected_answer);
  }

  // question_type is stored in options._question_type (JSONB column), not a top-level DB column
  const getQuestionType = (q: any): 'mcq' | 'open_ended' => {
    return (q?.options?._question_type) || (q?.question_type) || 'mcq';
  }

  const isAnswerCorrect = (ans: any, q: any) => {
     if (!ans || !q) return false;
     if (getQuestionType(q) === 'open_ended') return false;
     const studentAns = getAnswerString(ans).trim();
     if (!studentAns) return false;

     // Answers are stored as option text values
     // Get the correct answer text by looking up the letter key in q.options
     const correctKeys = (q.correct_answer || '').split(',').map((k: string) => k.trim());
     const correctTexts = correctKeys.map((k: string) => {
         const val = (q.options as any)?.[k] as string || k;
         return typeof val === 'string' ? val.trim() : val;
     }).filter(Boolean).sort().join('|||');

     // Student answer stored as text (single) or text values separated by |||
     const studentTexts = studentAns.split('|||').map((s: string) => s.trim()).filter(Boolean).sort().join('|||');

     if (studentTexts && correctTexts && studentTexts === correctTexts) return true;

     // Fallback: direct key comparison (for older attempts stored as letter keys)
     const studentKey = studentAns.split(',').map((s: string) => s.trim()).sort().join(',');
     const correctKey = [...correctKeys].sort().join(',');
     return studentKey !== '' && studentKey === correctKey;
```

```
  }

  const toggleExpand = (qId: string) => {
    setExpandedAnswers(prev => ({ ...prev, [qId]: !prev[qId] }))
  }

  // Normalize answers to fallback to JSON column if relational answers are missing

  const normalizedAnswers = React.useMemo(() => {
    const map = new Map<string, any>();
    // First, fallback to attemptData.answers (JSONB dict) if it exists

    let rawAnswers = attemptData?.answers;
    if (typeof rawAnswers === 'string') {
        try { rawAnswers = JSON.parse(rawAnswers); } catch(e) {}
    }

    // Emergency fallback to local storage if DB fails to save answers

    if ((!rawAnswers || Object.keys(rawAnswers).length === 0) && attemptData?.id) {
        const ls = localStorage.getItem(`attempt_answers_${attemptData.id}`);
        if (ls) {
            try { rawAnswers = JSON.parse(ls); } catch(e) {}
        }
    }

    if (rawAnswers && typeof rawAnswers === 'object') {
        Object.entries(rawAnswers).forEach(([qId, ans]) => {
            map.set(qId, { question_id: qId, selected_answer: ans });
        });
    }
    // Then override with attemptAnswers array (newer relational data) if available

    if (attemptAnswers && Array.isArray(attemptAnswers)) {
        attemptAnswers.forEach(ans => {
            if (ans && ans.question_id) {
                map.set(ans.question_id, ans);
            }
        });
    }
    return Array.from(map.values());
  }, [attemptAnswers, attemptData]);

  const totalQuestions = questions?.length || 0

  // Calculate stats robustly

  let correctQuestions = 0;
  let pendingQuestions = 0;
  let attemptedQuestions = 0;

  questions?.forEach(q => {
      const ans = normalizedAnswers.find((a: any) => a.question_id === q.id);
      if (ans && getAnswerString(ans).trim() !== '') {
          attemptedQuestions++;
          if (getQuestionType(q) === 'open_ended') {
              pendingQuestions++;
          } else if (isAnswerCorrect(ans, q)) {
              correctQuestions++;
          }
      }
  });

  const incorrectQuestions = attemptedQuestions - correctQuestions - pendingQuestions
  const skippedQuestions = totalQuestions - attemptedQuestions

  const content = (
    <>
      <div className="max-w-7xl mx-auto space-y-6">
        {!isStandalone && (
          <Link to={`/trainee/assessments`} state={{ tab: 'completed' }} className="inline-flex items-center gap-2 text-sm tex
t-zinc-400 hover:text-cyan-600 transition-colors mb-2 font-semibold">
            <ArrowLeft className="w-4 h-4" /> Back to Assessments
          </Link>
        )}

        <div className="bg-white border border-slate-200 rounded-lg p-6 shadow-sm">
          {areResultsHidden ? (
             <div className="text-center p-8">
```

```
145 |                 <div className="w-20 h-20 bg-emerald-100 rounded-full flex items-center justify-center mx-auto mb-4">
146 |                   <CheckCircle2 className="w-10 h-10 text-emerald-600" />
147 |                 </div>
148 |                 <h2 className="text-2xl font-black text-slate-900 mb-2">Assessment Completed</h2>
149 |                 <p className="text-slate-600 mb-6 bg-cyan-50 p-4 rounded-2xl border border-cyan-200 max-w-sm mx-auto text-sm f
ont-medium">
150 |                    Your results are currently hidden and will be published on <br/>
151 |                    <span className="font-bold text-cyan-700">{assessment.results_publish_date ? new Date(assessment.results_pu
blish_date).toLocaleDateString() : 'a later date'}</span>.
152 |                 </p>
153 |               </div>
154 |             ) : (
155 |                 <div className="space-y-6">
156 |                     {/* Header exactly like the screenshot */}
157 |                     <div className="bg-[#334155] text-white flex justify-between items-center px-4 py-2 text-sm font-semibold mb
-4 rounded-sm">
158 |                         <div className="truncate pr-4 flex-1">
159 |                             {course?.title ? `${course.title} - ` : ''}{displayTitle} {assessment?.id ? `[#${assessment.id.slice
(0, 5)}]` : ''}
160 |                         </div>
161 |                         <div className="truncate text-right flex-1">
162 |                             {profile?.full_name?.toUpperCase()} [{profile?.id?.slice(0, 8)}] - {profile?.email}
163 |                         </div>
164 |                     </div>
165 |
166 |                     {/* Assessment Type & Grade Impact Notice */}
167 |                     {(() => {
168 |                       const isFinal = assessment?.assessment_type === 'final'
169 |                       const isPractice = assessment?.assessment_type === 'mock' || assessment?.assessment_type === 'daily'
170 |                       if (isPractice) {
171 |                         return (
172 |                           <div className="bg-slate-50 border border-slate-200 rounded-xl p-3.5 flex items-center justify-between
text-xs">
173 |                             <span className="font-bold text-slate-700 uppercase tracking-wider flex items-center gap-1.5">
174 |                               <span className="w-2 h-2 rounded-full bg-slate-400" />
175 |                               {assessment?.assessment_type === 'mock' ? 'Mock Test' : 'Daily Test'} (Practice Only)
176 |                             </span>
177 |                             <span className="text-slate-500 font-medium">This practice test does not affect your official final
course grade.</span>
178 |                           </div>
179 |                         )
180 |                       }
181 |                       if (isFinal) {
182 |                         return (
183 |                           <div className="bg-purple-50 border border-purple-200 rounded-xl p-3.5 flex items-center justify-betwe
en text-xs">
184 |                             <span className="font-bold text-purple-800 uppercase tracking-wider flex items-center gap-1.5">
185 |                               <span className="w-2 h-2 rounded-full bg-purple-600" />
186 |                               Final Assessment (50% Course Grade Weight)
187 |                             </span>
188 |                             <span className="text-purple-700 font-semibold">Contributes 50% towards your final certificate scor
e.</span>
189 |                           </div>
190 |                         )
191 |                       }
192 |                       return (
193 |                         <div className="bg-amber-50 border border-amber-200 rounded-xl p-3.5 flex items-center justify-between t
ext-xs">
194 |                           <span className="font-bold text-amber-800 uppercase tracking-wider flex items-center gap-1.5">
195 |                             <span className="w-2 h-2 rounded-full bg-amber-600" />
196 |                             Regular Assessment (25% Course Grade Weight - Averaged)
197 |                           </span>
198 |                           <span className="text-amber-700 font-semibold">This score is averaged with other regular assessments
(25% total weight).</span>
199 |                         </div>
200 |                       )
201 |                     })()}
202 |
203 |                     {/* Header stats (similar to screenshot) */}
204 |                     <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
205 |                         <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
206 |                             <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Test Start Time</div>
207 |                             <div className="py-2 text-sm text-slate-700 font-medium">{new Date(attemptData.submitted_at || attem
ptData.created_at || Date.now()).toLocaleString()}</div>
208 |                         </div>
209 |                         <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
210 |                             <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Marks Scored</div>
211 |                             <div className="py-2 text-sm text-slate-700 font-medium">
212 |                               {attemptData.grade_status === 'pending_manual' ? 'Pending' : `${correctQuestions} / ${totalQuestio
```

```
ns}`}
                            </div>
                        </div>
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Total Questions</div>
                            <div className="py-2 text-sm text-slate-700 font-medium">{totalQuestions}</div>
                        </div>
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Attempted Questions</div>
                            <div className="py-2 text-sm text-slate-700 font-medium">{attemptedQuestions}</div>
                        </div>
                    </div>

                    <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Correct Questions</div>
                            <div className="py-2 text-sm text-emerald-600 font-bold bg-emerald-50 mx-4 my-1 rounded">{correctQue
stions}</div>
                        </div>
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Incorrect Questions</div>
                            <div className="py-2 text-sm text-rose-600 font-bold bg-rose-50 mx-4 my-1 rounded">{incorrectQuestio
ns}</div>
                        </div>
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Skipped Questions</div>
                            <div className="py-2 text-sm text-amber-600 font-bold bg-amber-50 mx-4 my-1 rounded">{skippedQuestio
ns}</div>
                        </div>
                        <div className="bg-white border border-slate-200 rounded-md overflow-hidden text-center">
                            <div className="bg-[#179da7] text-white text-sm py-1 font-semibold">Pending Evaluation</div>
                            <div className="py-2 text-sm text-slate-800 font-bold bg-slate-100 mx-4 my-1 rounded">{pendingQuesti
ons}</div>
                        </div>
                    </div>

                    {/* Sections Grouping */}
                    <div className="mt-8 space-y-6">
                        {(() => {
                            const sections = questions?.reduce((acc: any, q: any) => {
                                const sec = (q.options && q.options._section) ? q.options._section : 'General Questions'
                                if (!acc[sec]) acc[sec] = []
                                acc[sec].push(q)
                                return acc
                            }, {}) || {}

                            return Object.entries(sections).map(([sectionName, secQuestions]: [string, any]) => {
                                const marksPerQ = (assessment as any)?.marks_per_question || 1;
                                const marksScored = (secQuestions as any[]).reduce((sum, q) => {
                                    const ans = normalizedAnswers.find((a: any) => a.question_id === q.id)
                                    return sum + (isAnswerCorrect(ans, q) ? marksPerQ : 0)
                                }, 0)

                                return (
                                    <div key={sectionName} className="border border-[#179da7] rounded-sm overflow-hidden">
                                        <div className="bg-[#179da7] text-white px-4 py-2 text-sm font-bold flex flex-col sm:fle
x-row justify-between items-start sm:items-center">
                                            <span>{sectionName}</span>
                                            <div className="flex gap-4 text-xs font-medium mt-1 sm:mt-0">
                                                <span>Marks per question : {marksPerQ.toFixed(1)}</span>
                                                <span>Marks Scored : {marksScored.toFixed(1)}</span>
                                            </div>
                                        </div>

                                        <div className="w-full text-sm overflow-x-auto">
                                            <div className="min-w-[600px] grid grid-cols-12 bg-[#179da7] text-white text-xs py-
1.5 px-4 font-semibold border-t border-white/20">
                                                <div className="col-span-1">Q No.</div>
                                                <div className="col-span-5 text-center">Q. Type</div>
                                                <div className="col-span-2 text-center">Status</div>
                                                <div className="col-span-2 text-center">Marks</div>
                                                <div className="col-span-2 text-right"></div>
                                            </div>
                                            <div className="p-2 space-y-2 min-w-[600px] bg-white">
                                                {(secQuestions as any[]).map((q: any, idx: number) => {
                                                    const ans = normalizedAnswers.find((a: any) => a.question_id === q.id) as an
y

                                                    const ansString = getAnswerString(ans);
                                                    const hasAnswer = ansString.trim() !== '';
```

```
284 |                                                 const isPending = getQuestionType(q) === 'open_ended' && hasAnswer;
285 |                                                 const isExpanded = !!expandedAnswers[q.id];
286 |
287 |                                                 let borderColor = 'border-slate-200';
288 |                                                 let statusIcon = <span className="text-slate-400 font-bold">-</span>;
289 |                                                 let markDisplay = '0.0';
290 |
291 |                                                 if (hasAnswer) {
292 |                                                    if (isPending) {
293 |                                                        borderColor = 'border-amber-300';
294 |                                                        statusIcon = <span className="text-amber-500 font-bold">...</span>;
295 |                                                        markDisplay = 'Pending';
296 |                                                    } else if (isAnswerCorrect(ans, q)) {
297 |                                                        borderColor = 'border-emerald-500';
298 |                                                        statusIcon = <CheckCircle2 className="w-4 h-4 text-emerald-600 inlin
e" />;
299 |                                                        markDisplay = marksPerQ.toFixed(1);
300 |                                                    } else {
301 |                                                        borderColor = 'border-red-400';
302 |                                                        statusIcon = <XCircle className="w-4 h-4 text-red-500 inline" />;
303 |                                                    }
304 |                                                 } else {
305 |                                                     borderColor = 'border-red-400';
306 |                                                     statusIcon = <XCircle className="w-4 h-4 text-red-500 inline" />;
307 |                                                 }
308 |
309 |                                                 return (
310 |                                                     <div key={q.id} className={`flex flex-col mb-2 bg-white border ${isExpan
ded ? (hasAnswer && !isPending && isAnswerCorrect(ans, q) ? 'border-emerald-500' : 'border-red-400') : borderColor} rounded-sm shado
w-sm transition-all`}>
311 |                                                         <div className="grid grid-cols-12 items-center py-2.5 px-4 cursor-po
inter" onClick={() => toggleExpand(q.id)}>
312 |                                                             <div className="col-span-1 text-slate-700 text-xs font-medium">
{idx + 1}</div>
313 |                                                             <div className="col-span-5 text-slate-600 text-center text-xs">
314 |                                                                 {getQuestionType(q) === 'mcq' ? (
315 |                                                                     <div className="flex flex-col leading-snug">
316 |                                                                         <span>Multiple Choice - Single</span>
317 |                                                                         <span>Answer</span>
318 |                                                                     </div>
319 |                                                                 ) : getQuestionType(q) === 'open_ended' ? 'Open Ended' : get
QuestionType(q)}
320 |                                                             </div>
321 |                                                             <div className="col-span-2 text-center">{statusIcon}</div>
322 |                                                             <div className="col-span-2 text-center text-xs font-medium text-
slate-600">{markDisplay}</div>
323 |                                                             <div className="col-span-2 text-right">
324 |                                                                 <Button size="sm" variant="outline" className="h-7 px-3 text
-xs bg-[#179da7] text-white hover:bg-[#138891] hover:text-white border-0 rounded-sm" onClick={(e) => { e.stopPropagation(); toggleEx
pand(q.id); }}>
325 |                                                                     {isExpanded ? 'Hide Answer' : 'View Answer'}
326 |                                                                 </Button>
327 |                                                             </div>
328 |                                                         </div>
329 |                                                         {isExpanded && (
330 |                                                             <div className="px-6 pb-6 pt-2 border-t border-slate-100">
331 |                                                                 <div className="bg-[#fffdf2] border border-[#f3e5b3] rounded
-md p-4 text-sm text-slate-800 mb-4 shadow-sm">
332 |                                                                     {q.question_text}
333 |                                                                 </div>
334 |                                                                 <div className="space-y-3">
335 |                                                                     {getQuestionType(q) === 'mcq' ? (
336 |                                                                         <>
337 |                                                                             {Object.entries(q.options || {})
338 |                                                                               .filter(([k]) => !['_question_type', '_difficu
lty', '_section'].includes(k))
339 |                                                                               .map(([k, v]) => {
340 |                                                                                 // Answers stored as option text (||| separa
ted for multi)
341 |                                                                                 const studentTexts = ansString.split('|||').
map((s: string) => s.trim()).filter(Boolean);
342 |                                                                                 // Also handle old-style comma-separated key
answers
343 |                                                                                 const studentKeys = ansString.split(',').map
((s: string) => s.trim());
344 |                                                                                 const correctKeys = (q.correct_answer ||
'').split(',').map((s: string) => s.trim());
```

```
345 |                                                
```

```
                                                                                                    return (q.options as any)?.[t] ||
t;
                                                                                                }).join(', ')}
                                                                                            </p>
                                                                                        </div>
                                                                                    </div>
                                                                                    {q.explanation && (
                                                                                        <div className="mt-4 bg-blue-50/50 p-4 round
ed-lg border border-blue-100">
                                                                                            <p className="text-xs font-bold text-blu
e-700 uppercase tracking-wider mb-2 flex items-center gap-1">
                                                                                                Explanation / Reason
                                                                                            </p>
                                                                                            <p className="text-sm text-blue-900">{q.
explanation}</p>
                                                                                        </div>
                                                                                    )}
                                                                                </>
                                                                            ) : (
                                                                                <div className="space-y-4">
                                                                                    <div>
                                                                                        <p className="text-[10px] font-bold text-sla
te-500 uppercase tracking-wider mb-2">Your Answer:</p>
                                                                                        <div className="bg-slate-50 p-4 rounded-lg t
ext-sm text-slate-800 border border-slate-200">
                                                                                            {hasAnswer ? ansString : <span className
="italic text-slate-400">No answer provided.</span>}
                                                                                        </div>
                                                                                    </div>
                                                                                    <div className="bg-emerald-50/50 p-4 rounded-lg
border border-emerald-100">
                                                                                        <p className="text-xs font-bold text-emerald
-700 uppercase tracking-wider mb-2 flex items-center gap-2"><CheckCircle2 className="w-4 h-4" /> Correct Answer / Reference:</p>
                                                                                        <div className="text-sm text-emerald-900 fon
t-medium">
                                                                                            {q.correct_answer || <span className="it
alic opacity-70">No reference answer available.</span>}
                                                                                        </div>
                                                                                    </div>
                                                                                    {q.explanation && (
                                                                                        <div className="bg-blue-50/50 p-4 rounded-lg
border border-blue-100">
                                                                                            <p className="text-xs font-bold text-blu
e-700 uppercase tracking-wider mb-2 flex items-center gap-1">
                                                                                                Explanation / Reason
                                                                                            </p>
                                                                                            <p className="text-sm text-blue-900">{q.
explanation}</p>
                                                                                        </div>
                                                                                    )}
                                                                                </div>
                                                                            )}
                                                                        </div>
                                                                    </div>
                                                                )}
                                                            </div>
                                                        )
                                                    })}
                                                </div>
                                            </div>
                                        )
                                    })
                                })()}
                            </div>
                        </div>
                    )}
                </div>
            </div>

        </>
    )

    if (isStandalone) {
      return content;
    }

    return (
      <DashboardShell title={displayTitle} icon={Target} navLinks={traineeNavLinks}>
```

```
      {content}
    </DashboardShell>
  )
}

```

src/features/courses/TraineeAssessmentsHub.tsx TSX 30.9 KB 532 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { useQuery } from '@tanstack/react-query'
import { Link, useLocation } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { DashboardShell } from '@/pages/Dashboards'
import {
  BookOpen, Compass, FileCheck, CheckCircle2, Clock,
  Calendar, Search, Filter, PlayCircle, BarChart3, User, ChevronLeft, ChevronRight, Bell
} from 'lucide-react'
import {
  format, isSameDay, addMonths, subMonths,
  startOfMonth, endOfMonth, eachDayOfInterval, getDay
} from 'date-fns'

const fadeUp = {
  hidden: { opacity: 0, y: 10 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.3 } },
  exit: { opacity: 0, y: -10, transition: { duration: 0.2 } }
}

// Mini Calendar

function MiniCalendar({
  selectedDate,
  onSelectDate,
  highlightedDates = [],
}: {
  selectedDate: Date | null
  onSelectDate: (d: Date | null) => void
  highlightedDates?: Date[]
}) {
  const [viewMonth, setViewMonth] = useState(new Date())
  const today = new Date()
  const days = eachDayOfInterval({ start: startOfMonth(viewMonth), end: endOfMonth(viewMonth) })
  const leadingBlanks = getDay(startOfMonth(viewMonth))
  const isHL = (d: Date) => highlightedDates.some(h => isSameDay(h, d))
  const isSel = (d: Date) => !!selectedDate && isSameDay(d, selectedDate)
  const isToday = (d: Date) => isSameDay(d, today)

  return (
    <div className="select-none">
      <div className="flex items-center justify-between mb-2">
        <button onClick={() => setViewMonth(m => subMonths(m, 1))} className="p-1 rounded hover:bg-slate-100 transition-colors text-slate-500">
          <ChevronLeft className="w-4 h-4" />
        </button>
        <span className="text-xs font-bold text-slate-700 tracking-wide">{format(viewMonth, 'MMMM yyyy')}</span>
        <button onClick={() => setViewMonth(m => addMonths(m, 1))} className="p-1 rounded hover:bg-slate-100 transition-colors text-slate-500">
          <ChevronRight className="w-4 h-4" />
        </button>
      </div>
      <div className="grid grid-cols-7 mb-1">
        {['Su','Mo','Tu','We','Th','Fr','Sa'].map(d => (
          <div key={d} className="text-center text-[10px] font-bold text-slate-400 py-0.5">{d}</div>
        ))}
      </div>
      <div className="grid grid-cols-7 gap-y-0.5">
        {Array.from({ length: leadingBlanks }).map((_, i) => <div key={`b-${i}`} />)}
        {days.map(day => {
          const sel = isSel(day); const hl = isHL(day); const tod = isToday(day)
          return (
            <button
              key={day.toISOString()}
              onClick={() => onSelectDate(sel ? null : day)}
              className={[
                'relative w-full aspect-square flex items-center justify-center rounded-full text-[11px] font-semibold transition-all',
                sel ? 'bg-[#1a7a80] text-white shadow' : '',
                !sel && hl ? 'text-[#1a7a80] font-bold' : '',
                !sel && tod ? 'ring-2 ring-[#1a7a80]/40' : '',
                !sel ? 'text-slate-700 hover:bg-slate-100' : '',
              ].filter(Boolean).join(' ')}
            >
```

```
              {format(day, 'd')}
              {hl && !sel && <span className="absolute bottom-0.5 left-1/2 -translate-x-1/2 w-1 h-1 rounded-full bg-[#1a7a80]"
/>}
            </button>
          )
        })}
      </div>
      {selectedDate && (
        <button onClick={() => onSelectDate(null)} className="mt-3 w-full text-[11px] font-semibold text-slate-500 hover:text-
rose-500 transition-colors">
          Clear filter
        </button>
      )}
    </div>
  )
}

export function TraineeAssessmentsHub() {
  const { profile } = useAuth()
  const location = useLocation()
  const [activeTab, setActiveTab] = useState<'ongoing' | 'completed' | 'expired'>((location.state as any)?.tab || 'ongoing')
  const [searchQuery, setSearchQuery] = useState('')
  const [filterType, setFilterType] = useState('all')
  const [calendarDate, setCalendarDate] = useState<Date | null>(null)
  const [expiredCalendarDate, setExpiredCalendarDate] = useState<Date | null>(null)

  // Live clock — updates every second
  const [now, setNow] = useState(new Date())
  useEffect(() => {
    const timer = setInterval(() => setNow(new Date()), 1000)
    return () => clearInterval(timer)
  }, [])

  const { data: enrollments, isLoading: enrollmentsLoading } = useQuery({
    queryKey: ['trainee_enrollments_for_assessments', profile?.id],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('enrollments')
        .select(`
          course_id, progress_percent, status,
          course:courses!enrollments_course_id_fkey(status)
        `)
        .eq('user_id', profile!.id)
      if (error) throw error
      return (data || []).filter((e: any) => e.course?.status !== 'archived')
    },
    enabled: !!profile?.id,
  })

  const courseIds = enrollments?.map(e => e.course_id) || []
  const enrollmentMap = new Map<string, any>()
  enrollments?.forEach(e => enrollmentMap.set(e.course_id, e))

  const isFinalUnlockedForCourse = (courseId: string) => {
    const en = enrollmentMap.get(courseId)
    if (!en) return false
    if ((en.progress_percent ?? 0) >= 100 || en.status === 'completed') return true
    if (profile?.id) {
      const stored = localStorage.getItem(`cc_mod_progress_${courseId}_${profile.id}`)
      if (stored) {
        try {
          const parsed = JSON.parse(stored)
          if (parsed.completed && parsed.completed.length > 0) {
            // If local storage has marked modules completed

            return (en.progress_percent ?? 0) >= 100
          }
        } catch {}
      }
    }
    return false
  }

  const { data: assessments, isLoading: assessmentsLoading } = useQuery({
    queryKey: ['trainee_assessments_v2', courseIds],
    queryFn: async () => {
      if (courseIds.length === 0) return []
      const { data, error } = await supabase
```

```
        .from('assessments')
        .select(`id, title, scheduled_date, start_time, end_time, duration_minutes,
                 passing_score, course_id, status, assessment_type,
                 results_publish_date, course:courses(title, status)`)
        .in('course_id', courseIds)
        .eq('status', 'published')
        .order('scheduled_date', { ascending: true })
      if (error) throw error
      return (data || []).filter((a: any) => a.course?.status !== 'archived')
    },
    enabled: courseIds.length > 0,
  })

  const assessmentIds = assessments?.map(a => a.id) || []
  const { data: questionCounts } = useQuery({
    queryKey: ['assessment_question_counts', assessmentIds],
    queryFn: async () => {
      if (assessmentIds.length === 0) return {}
      const { data, error } = await supabase.from('questions').select('assessment_id').in('assessment_id', assessmentIds)
      if (error) return {}
      const counts: Record<string, number> = {}
      ;(data || []).forEach((q: any) => { counts[q.assessment_id] = (counts[q.assessment_id] || 0) + 1 })
      return counts
    },
    enabled: assessmentIds.length > 0,
  })

  const { data: attempts, isLoading: attemptsLoading } = useQuery({
    queryKey: ['trainee_assessment_attempts', profile?.id],
    queryFn: async () => {
      const { data, error } = await supabase.from('assessment_attempts' as any).select('*').eq('user_id', profile!.id)
      if (error) throw error
      return (data || []) as any[]
    },
    enabled: !!profile?.id,
  })

  const isLoading = enrollmentsLoading || assessmentsLoading || attemptsLoading
  const attemptedIds = new Set(attempts?.map(a => a.assessment_id) || [])

  const completedAssessments = (assessments || [])
    .filter(a => attemptedIds.has(a.id))
    .map(a => ({ ...a, attempt: attempts?.find(att => att.assessment_id === a.id) }))

  const pending = (assessments || []).filter(a => !attemptedIds.has(a.id))

  const ongoingAndUpcoming = pending.filter(a => {
    if (!a.scheduled_date) return true
    const now = new Date()
    const end = new Date(`${a.scheduled_date}T${a.end_time || '23:59:59'}`)
    const start = new Date(`${a.scheduled_date}T${a.start_time || '00:00:00'}`)
    if (end < start) end.setDate(end.getDate() + 1)
    return now <= end
  })

  const expiredAssessments = pending.filter(a => {
    if (!a.scheduled_date) return false
    const now = new Date()
    const end = new Date(`${a.scheduled_date}T${a.end_time || '23:59:59'}`)
    const start = new Date(`${a.scheduled_date}T${a.start_time || '00:00:00'}`)
    if (end < start) end.setDate(end.getDate() + 1)
    return now > end
  })

  const completedDates: Date[] = completedAssessments
    .map(a => { const s = a.attempt?.submitted_at || a.attempt?.started_at || a.scheduled_date; return s ? new Date(s) : null
})
    .filter(Boolean) as Date[]

  const expiredDates: Date[] = expiredAssessments
    .map(a => a.scheduled_date ? new Date(a.scheduled_date) : null)
    .filter(Boolean) as Date[]

  const twoMonthsAgo = subMonths(now, 2)
  const headingRange = `${format(twoMonthsAgo, 'd MMM')} - ${format(now, 'd MMM')}`

  const formatType = (t: string) =>
    t === 'daily' ? 'Daily Test (Practice)' : t === 'mock' ? 'Mock Test (Practice)' : t === 'assessment' ? 'Assessment Test (2
5% Weight)' : t === 'final' ? 'Final Assessment (50% Weight)' : 'Test'
```

```
  const matchesFilter = (a: any) => {
    const ms = a.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
               (a.course as any)?.title?.toLowerCase().includes(searchQuery.toLowerCase()) ||
               (a.assessment_type && formatType(a.assessment_type).toLowerCase().includes(searchQuery.toLowerCase()))
    return ms && (filterType === 'all' || a.assessment_type === filterType)
  }

  const matchesCalendar = (a: any) => {
    if (!calendarDate) return true
    const s = a.attempt?.submitted_at || a.attempt?.started_at || a.scheduled_date
    return s ? isSameDay(new Date(s), calendarDate) : false
  }

  const filteredOngoing = ongoingAndUpcoming.filter(matchesFilter)
  const filteredCompleted = completedAssessments.filter(a => matchesFilter(a) && matchesCalendar(a))
  const filteredExpired = expiredAssessments.filter(a => {
    if (!matchesFilter(a)) return false
    if (!expiredCalendarDate) return true
    return a.scheduled_date ? isSameDay(new Date(a.scheduled_date), expiredCalendarDate) : false
  })

  const scoreCell = (a: any) => {
    if (a.results_publish_date && new Date(a.results_publish_date) > now)
      return <p className="text-sm font-bold text-slate-500 whitespace-nowrap">To Be Announced</p>
    if (a.attempt?.grade_status === 'pending_manual')
      return <p className="text-xl font-black text-amber-600">Pending</p>
    const qCount = (questionCounts as any)?.[a.id] || 0
    const scored = qCount > 0 ? Math.round((a.attempt?.score ?? 0) / 100 * qCount) : (a.attempt?.score ?? 0)
    const total = qCount > 0 ? qCount : 100
    return (
      <>
        <p className="text-2xl font-black text-cyan-600">{scored}<span className="text-sm text-slate-400">/{total}</span></p>
        <p className="text-[10px] font-bold text-slate-400 uppercase tracking-wider mt-0.5">Marks</p>
      </>
    )
  }

  return (
    <DashboardShell
      title="Assessments"
      icon={FileCheck}
      navLinks={[
        { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
        { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
        { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
        { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
        { to: '/trainee/profile', label: 'Profile', icon: User },
      ]}
    >
      <div className="flex flex-col lg:flex-row gap-6 max-w-full">

        {/* Sidebar */}
        <div className="lg:w-64 shrink-0 flex flex-col gap-3">

          {/* Tabs */}
          <div className="bg-white border border-slate-200/90 rounded-3xl p-2 shadow-sm flex flex-col">
            <button onClick={() => setActiveTab('ongoing')}
              className={`w-full flex items-center gap-3 px-4 py-3 rounded-2xl text-sm font-bold transition-all ${activeTab === 'ongoing' ? 'bg-cyan-50 text-cyan-700 border border-cyan-200 shadow-sm' : 'text-slate-600 hover:bg-slate-50 border border-transparent'}`}>
              <Clock className="w-4 h-4" /> Ongoing &amp; Upcoming
            </button>
            <button onClick={() => setActiveTab('completed')}
              className={`w-full flex items-center gap-3 px-4 py-3 rounded-2xl text-sm font-bold transition-all mt-1 ${activeTab === 'completed' ? 'bg-cyan-50 text-cyan-700 border border-cyan-200 shadow-sm' : 'text-slate-600 hover:bg-slate-50 border border-transparent'}`}>
              <CheckCircle2 className="w-4 h-4" /> Completed
            </button>
            <button onClick={() => setActiveTab('expired')}
              className={`w-full flex items-center gap-3 px-4 py-3 rounded-2xl text-sm font-bold transition-all mt-1 ${activeTab === 'expired' ? 'bg-rose-50 text-rose-700 border border-rose-200 shadow-sm' : 'text-slate-600 hover:bg-slate-50 border border-transparent'}`}>
              <Clock className="w-4 h-4" /> Expired &amp; Missed
            </button>
          </div>

          {/* Calendar — Completed & Expired tabs */}
```

```
298 |           {(activeTab === 'completed' || activeTab === 'expired') && (
299 |             <div className="bg-white border border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
300 |               <div className={`text-white text-xs font-bold px-4 py-2.5 tracking-wide ${activeTab === 'expired' ? 'bg-rose-80
0' : 'bg-[#334155]'}`}>
301 |                 Search By Date
302 |               </div>
303 |               <div className="p-3 pt-2">
304 |                 {activeTab === 'completed' && (
305 |                   <MiniCalendar selectedDate={calendarDate} onSelectDate={setCalendarDate} highlightedDates={completedDates} /
>
306 |                 )}
307 |                 {activeTab === 'expired' && (
308 |                   <MiniCalendar selectedDate={expiredCalendarDate} onSelectDate={setExpiredCalendarDate} highlightedDates={exp
iredDates} />
309 |                 )}
310 |               </div>
311 |             </div>
312 |           )}
313 |
314 |           {/* Filters */}
315 |           <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm">
316 |             <h3 className="text-xs font-bold text-slate-500 uppercase tracking-wider mb-3">Filters</h3>
317 |             <div className="relative mb-3">
318 |               <Search className="absolute left-3.5 top-1/2 -translate-y-1/2 w-4 h-4 text-slate-400" />
319 |               <input type="text" placeholder="Search assessments..." value={searchQuery} onChange={e => setSearchQuery(e.targe
t.value)}
320 |                 className="w-full pl-10 pr-4 py-2.5 bg-slate-50 border border-slate-200 rounded-xl text-sm text-slate-900 plac
eholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all font-medium" />
321 |             </div>
322 |             <div className="relative">
323 |               <Filter className="absolute left-3.5 top-1/2 -translate-y-1/2 w-4 h-4 text-slate-400 pointer-events-none" />
324 |               <select value={filterType} onChange={e => setFilterType(e.target.value)}
325 |                 className="w-full pl-10 pr-10 py-2.5 bg-slate-50 border border-slate-200 rounded-xl text-sm text-slate-900 foc
us:outline-none focus:border-cyan-500 appearance-none cursor-pointer hover:bg-slate-100 font-medium">
326 |                 <option value="all">All Types</option>
327 |                 <option value="mock">Mock Test</option>
328 |                 <option value="assessment">Assessment Test</option>
329 |                 <option value="daily">Daily Test</option>
330 |                 <option value="final">Final Test</option>
331 |               </select>
332 |               <div className="absolute right-3.5 top-1/2 -translate-y-1/2 pointer-events-none text-slate-400">
333 |                 <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="current
Color" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="m6 9 6 6 6-6"/></svg>
334 |               </div>
335 |             </div>
336 |           </div>
337 |         </div>
338 |
339 |         {/* Main */}
340 |         <div className="flex-1 bg-white border border-slate-200/90 rounded-3xl p-6 md:p-8 shadow-sm min-h-[500px]">
341 |           <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-1 pb-4 border-b border-slate-100 mb
-2">
342 |             <h2 className="text-xl font-black text-slate-900 tracking-tight">
343 |               {activeTab === 'ongoing' && 'Ongoing & Upcoming Assessments'}
344 |               {activeTab === 'completed' && (calendarDate ? `Tests on ${format(calendarDate, 'd MMM yyyy')}` : `Recent Tests
(${headingRange})`)}
345 |               {activeTab === 'expired' && (expiredCalendarDate ? `Expired on ${format(expiredCalendarDate, 'd MMM yyyy')}` : `
Expired & Missed Assessments`)}
346 |             </h2>
347 |             {(activeTab === 'completed' || activeTab === 'expired') && (
348 |               <span className="text-xs font-medium text-slate-400 whitespace-nowrap">{format(now, 'dd MMM yyyy HH:mm:ss')} Ind
ia Standard Time</span>
349 |             )}
350 |           </div>
351 |           {activeTab === 'completed' && !calendarDate && (
352 |             <p className="text-xs text-[#1a7a80] font-medium mb-5">You can see older tests by selecting the date corresponding
to their start time in the calendar</p>
353 |           )}
354 |           {activeTab === 'expired' && !expiredCalendarDate && (
355 |             <p className="text-xs text-rose-500 font-medium mb-5">You can filter by expiry date by selecting a date in the cal
endar</p>
356 |           )}
357 |
358 |           {isLoading ? (
359 |             <div className="flex items-center justify-center h-40"><div className="w-8 h-8 border-4 border-cyan-600 border-t-t
ransparent rounded-full animate-spin" /></div>
360 |           ) : (
361 |             <AnimatePresence mode="wait">
362 |
```

```
363 |               {activeTab === 'ongoing' && (
364 |                 <motion.div key="ongoing" variants={fadeUp} initial="hidden" animate="visible" exit="exit" className="space-y-
6">
365 |                   {filteredOngoing.length === 0 ? (
366 |                     <div className="text-center py-12 bg-slate-50 rounded-2xl border border-dashed border-slate-200">
367 |                       <Calendar className="w-10 h-10 text-slate-400 mx-auto mb-3" />
368 |                       <p className="text-sm font-semibold text-slate-600">No ongoing assessments at this moment</p>
369 |                     </div>
370 |                   ) : (
371 |                     <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
372 |                       {filteredOngoing.map(a => {
373 |                         const isFinal = a.assessment_type === 'final'
374 |                         const isPractice = a.assessment_type === 'mock' || a.assessment_type === 'daily'
375 |                         const isUnlocked = !isFinal || isFinalUnlockedForCourse(a.course_id)
376 |                         const courseEnrollment = enrollmentMap.get(a.course_id)
377 |                         const courseProgress = courseEnrollment?.progress_percent ?? 0
378 |
379 |                         return (
380 |                           <div key={a.id} className={`p-6 rounded-3xl bg-white border transition-all flex flex-col justify-bet
ween ${
381 |                             !isUnlocked
382 |                               ? 'border-amber-200/90 bg-amber-50/20 shadow-xs'
383 |                               : 'border-slate-200 hover:border-cyan-300 hover:shadow-md'
384 |                           }`}>
385 |                             <div>
386 |                               <div className="flex flex-wrap gap-2 mb-3">
387 |                                 <span className="text-[10px] font-bold text-cyan-700 bg-cyan-50 px-2.5 py-0.5 rounded-full upp
ercase tracking-wider border border-cyan-100">{(a.course as any)?.title || 'Course'}</span>
388 |                                 {isPractice ? (
389 |                                   <span className="text-[10px] font-bold text-slate-700 bg-slate-100 px-2.5 py-0.5 rounded-ful
l uppercase tracking-wider border border-slate-200">
390 |                                     {a.assessment_type === 'mock' ? 'Mock Test' : 'Daily Test'} • Practice Only
391 |                                   </span>
392 |                                 ) : isFinal ? (
393 |                                   <span className="text-[10px] font-bold text-purple-700 bg-purple-50 px-2.5 py-0.5 rounded-fu
ll uppercase tracking-wider border border-purple-200">
394 |                                     Final Exam • 50% Weight
395 |                                   </span>
396 |                                 ) : (
397 |                                   <span className="text-[10px] font-bold text-amber-700 bg-amber-50 px-2.5 py-0.5 rounded-full
uppercase tracking-wider border border-amber-200">
398 |                                     Assessment • 25% Weight
399 |                                   </span>
400 |                                 )}
401 |                               </div>
402 |                               <h3 className="text-base font-bold text-slate-900 mb-3">{a.title}</h3>
403 |                               <div className="space-y-2 mb-4">
404 |                                 <div className="flex items-center gap-2 text-xs text-slate-600 font-medium"><Clock className
="w-3.5 h-3.5 text-cyan-600" /><span>{a.duration_minutes ? `${a.duration_minutes} mins` : 'Untimed'}</span></div>
405 |                                 {a.scheduled_date && (
406 |                                   <div className="flex items-center gap-2 text-xs text-slate-600 font-medium">
407 |                                     <Calendar className="w-3.5 h-3.5 text-amber-600" />
408 |                                     <span>{format(new Date(a.scheduled_date), 'MMM do, yyyy')}{a.start_time && ` at ${format(n
ew Date(`2000-01-01T${a.start_time}`), 'h:mm a')}`}{a.end_time && ` to ${format(new Date(`2000-01-01T${a.end_time}`), 'h:mm a')}`}</
span>
409 |                                   </div>
410 |                                 )}
411 |                               </div>
412 |
413 |                               {!isUnlocked && (
414 |                                 <div className="mb-4 p-3 bg-amber-50 border border-amber-200 rounded-2xl flex items-center gap
-2 text-xs text-amber-800">
415 |                                   <span className="font-bold shrink-0">🔒 Locked:</span>
416 |                                   <span>Complete all course modules (currently {courseProgress}%) to unlock this Final Assessm
ent.</span>
417 |                                 </div>
418 |                               )}
419 |                             </div>
420 |
421 |                             {isUnlocked ? (
422 |                               <Link to={`/trainee/courses/${a.course_id}/assessments/${a.id}`}>
423 |                                 {(() => {
424 |                                   let upcoming = false
425 |                                   if (a.scheduled_date) { const s = new Date(a.scheduled_date); s.setHours(0,0,0,0); const t =
new Date(); t.setHours(0,0,0,0); if (s > t) upcoming = true }
426 |                                   return (<button className={`w-full py-2.5 text-sm font-bold rounded-xl flex items-center jus
tify-center gap-2 transition-all ${upcoming ? 'bg-slate-100 text-slate-700 hover:bg-slate-200' : 'bg-gradient-to-r from-cyan-600 to-
blue-600 text-white hover:opacity-95 shadow-md'}`}>{upcoming ? (<>Upcoming <Calendar className="w-4 h-4" /></>) : (<>Start Test <Pla
yCircle className="w-4 h-4" /></>)}</button>)
```

```
                                })()}
                              </Link>
                            ) : (
                              <Link to={`/trainee/courses/${a.course_id}/learn`}>
                                <button className="w-full py-2.5 text-xs font-bold rounded-xl flex items-center justify-center gap-1.5 bg-slate-100 text-slate-600 hover:bg-slate-200 border border-slate-300 transition-all">
                                  <span>Complete Modules to Unlock ({courseProgress}%)</span>
                                  <ChevronRight className="w-3.5 h-3.5" />
                                </button>
                              </Link>
                            )}
                          </div>
                        )
                      })}
                    </div>
                  )}
                </motion.div>
              )}

              {activeTab === 'completed' && (
                <motion.div key="completed" variants={fadeUp} initial="hidden" animate="visible" exit="exit" className="space-y-4">
                  {filteredCompleted.length === 0 ? (
                    <div className="text-center py-12 bg-slate-50 rounded-2xl border border-dashed border-slate-200">
                      <CheckCircle2 className="w-10 h-10 text-slate-400 mx-auto mb-3" />
                      <p className="text-sm font-semibold text-slate-600">{calendarDate ? `No tests found on ${format(calendarDate, 'd MMM yyyy')}` : 'No completed assessments yet'}</p>
                    </div>
                  ) : (
                    filteredCompleted.map(a => (
                      <div key={a.id} className="border border-slate-200 rounded-2xl bg-white shadow-sm hover:shadow-md transition-all overflow-hidden">
                        <div className="flex flex-col sm:flex-row sm:items-start gap-5 p-5">
                          <div className="flex-1 min-w-0">
                            <div className="flex flex-wrap gap-2 mb-2">
                              <span className="text-[10px] font-bold text-cyan-700 bg-cyan-50 px-2.5 py-0.5 rounded-full uppercase tracking-wider border border-cyan-100">{(a.course as any)?.title || 'Course'}</span>
                              {a.assessment_type && <span className="text-[10px] font-bold text-amber-700 bg-amber-50 px-2.5 py-0.5 rounded-full uppercase tracking-wider border border-amber-200">{formatType(a.assessment_type)}</span>}
                            </div>
                            <h3 className="text-base font-bold text-slate-900 truncate">{a.title}</h3>
                            <div className="mt-3 space-y-1 text-xs text-slate-600">
                              {a.scheduled_date && (
                                <div>
                                  <span className="font-bold text-slate-800">Scheduled Start Time &amp;amp; Duration</span>
                                  <span className="ml-2">{format(new Date(`${a.scheduled_date}T${a.start_time || '00:00:00'}`), 'dd MMM, HH:mm')}{a.duration_minutes && <span className="ml-2">&#9203; {a.duration_minutes} min</span>}</span>
                                </div>
                              )}
                              {(a.attempt?.started_at || a.attempt?.submitted_at) && (
                                <div>
                                  <span className="font-bold text-slate-800">Start Time - End Time</span>
                                  <span className="ml-2">{a.attempt.started_at ? format(new Date(a.attempt.started_at), 'dd MMM, HH:mm') : '—'}{' - '}{a.attempt.submitted_at ? format(new Date(a.attempt.submitted_at), 'dd MMM, HH:mm') : '—'}</span>
                                </div>
                              )}
                              <div>
                                <span className="font-bold text-slate-800">Marks Scored</span>
                                <span className="ml-2">{(() => { if (a.results_publish_date && new Date(a.results_publish_date) > now) return 'To Be Announced'; if (a.attempt?.grade_status === 'pending_manual') return 'Pending'; const qc = (questionCounts as any)?.[a.id] || 0; const sc = qc > 0 ? Math.round((a.attempt?.score ?? 0) / 100 * qc) : (a.attempt?.score ?? 0); return `${sc}/${qc > 0 ? qc : 100}`; })()}</span>
                              </div>
                              <div>
                                <span className="font-bold text-slate-800">Completion Status</span>
                                <span className="ml-2">{a.attempt?.grade_status === 'pending_manual' ? 'Pending review' : a.attempt?.submitted_at ? 'Completed' : 'Incomplete'}</span>
                              </div>
                            </div>
                          </div>
                          <div className="flex flex-col items-center justify-center gap-1 shrink-0 min-w-[72px] text-center">
                            {scoreCell(a)}
                          </div>
                        </div>
                        {!(a.results_publish_date && new Date(a.results_publish_date) > now) && (
                          <Link to={`/trainee/courses/${a.course_id}/assessments/${a.id}`} className="block">
                            <button className="w-full py-2.5 bg-[#3db97a] hover:bg-[#35a36b] text-white text-sm font-bold tracking-wide transition-colors">See Results</button>
                          </Link>
```

```
                        )}
                      </div>
                    ))
                  )}
                </motion.div>
              )}

              {activeTab === 'expired' && (
                <motion.div key="expired" variants={fadeUp} initial="hidden" animate="visible" exit="exit" className="space-y-4">
                  {filteredExpired.length === 0 ? (
                    <div className="text-center py-12 bg-slate-50 rounded-2xl border border-dashed border-slate-200">
                      <Clock className="w-10 h-10 text-slate-400 mx-auto mb-3" />
                      <p className="text-sm font-semibold text-slate-600">No expired assessments</p>
                    </div>
                  ) : (
                    filteredExpired.map(a => (
                      <div key={a.id} className="p-6 rounded-3xl bg-slate-50 border border-slate-200 shadow-sm flex flex-col sm:flex-row sm:items-center justify-between gap-4 opacity-75">
                        <div className="flex-1">
                          <div className="flex flex-wrap gap-2 mb-2">
                            <span className="text-[10px] font-bold text-rose-700 bg-rose-50 px-2.5 py-0.5 rounded-full uppercase tracking-wider border border-rose-200">Expired / Missed</span>
                            {a.assessment_type && <span className="text-[10px] font-bold text-slate-500 bg-slate-200 px-2.5 py-0.5 rounded-full uppercase tracking-wider border border-slate-300">{formatType(a.assessment_type)}</span>}
                          </div>
                          <h3 className="text-lg font-bold text-slate-900">{a.title}</h3>
                          <span className="text-xs text-slate-500 font-medium flex items-center gap-1.5 mt-2"><Calendar className="w-3.5 h-3.5 text-slate-400" />Expired on: {a.scheduled_date ? format(new Date(a.scheduled_date), 'MMM do yyyy') : 'Unknown Date'}</span>
                        </div>
                        <button disabled className="px-5 py-2.5 bg-slate-200 text-slate-500 text-sm font-bold rounded-xl whitespace-nowrap cursor-not-allowed">Missed</button>
                      </div>
                    ))
                  )}
                </motion.div>
              )}

            </AnimatePresence>
          )}
        </div>
      </div>
    </DashboardShell>
  )
}
```

src/features/courses/TraineeAssessmentTest.tsx TSX 55.4 KB 1,202 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react'
import { useParams, useNavigate, Link } from 'react-router-dom'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { DashboardShell } from '@/pages/Dashboards'
import { Loader2, ArrowLeft, Clock, CheckCircle2, XCircle, AlertCircle, Target, Brain, LayoutDashboard, BookOpen, Compass, TrendingUp, Settings, Award, Bell, User } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { toast } from 'sonner'
import { motion, AnimatePresence } from 'framer-motion'
import { Progress } from '@/components/ui/progress'
import { Lock, ShieldAlert } from 'lucide-react'
import { TraineeAssessmentResult } from './TraineeAssessmentResult'
import * as tf from '@tensorflow/tfjs'
import * as blazeface from '@tensorflow-models/blazeface'
import * as cocoSsd from '@tensorflow-models/coco-ssd'

export function TraineeAssessmentTest() {
  const { courseId, assessmentId } = useParams()
  const { profile } = useAuth()
  const navigate = useNavigate()
  const queryClient = useQueryClient()

  const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0)
  const [answers, setAnswers] = useState<Record<string, string>>({})
  const [timeLeft, setTimeLeft] = useState<number | null>(null)

  const [hasStarted, setHasStarted] = useState(false)
  const [showSubmitModal, setShowSubmitModal] = useState(false)

  // SEA states

  const [strikes, setStrikes] = useState(0)
  const [isFullScreen, setIsFullScreen] = useState(false)
  const MAX_STRIKES = 3

  // WebRTC / AV state

  const videoRef = useRef<HTMLVideoElement>(null)
  const canvasRef = useRef<HTMLCanvasElement>(null)
  const lastBrightnessRef = useRef<number | null>(null)
  const mediaStreamRef = useRef<MediaStream | null>(null)
  const audioContextRef = useRef<AudioContext | null>(null)
  const analyserRef = useRef<AnalyserNode | null>(null)
  const noiseWarningTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null)
  const glareTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null)
  const roomScanTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null)
  const faceDetectionIntervalRef = useRef<ReturnType<typeof setInterval> | null>(null)

  const [randomizedQuestions, setRandomizedQuestions] = useState<any[]>([])
  const [adaptiveHistory, setAdaptiveHistory] = useState<string[]>([])
  const [currentDifficulty, setCurrentDifficulty] = useState<'easy'|'medium'|'hard'>('medium')
  const [isScanningRoom, setIsScanningRoom] = useState(false)
  const [roomScanProgress, setRoomScanProgress] = useState(0)
  const [currentTime, setCurrentTime] = useState(new Date())

  useEffect(() => {
    if (!hasStarted) {
      const timer = setInterval(() => setCurrentTime(new Date()), 1000)
      return () => clearInterval(timer)
    }
  }, [hasStarted])

  // Fetch assessment and questions

  const { data: assessment, isLoading: isAssessmentLoading, error: assessmentError } = useQuery({
    queryKey: ['assessment', assessmentId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('assessments')
        .select('id, title, passing_score, status, requires_sea, sea_link, scheduled_date, start_time, end_time, duration_minutes, created_by, course_id, instructions, created_at, assessment_type')
        .eq('id', assessmentId!)
        .single()
      if (error) { throw error; }
```

```
72 |       return data as any
73 |     },
74 |     enabled: !!assessmentId,
75 |   })
76 |
77 |   const { data: questions, isLoading: isQuestionsLoading } = useQuery({
78 |     queryKey: ['assessment-questions', assessmentId],
79 |     queryFn: async () => {
80 |       const { data, error } = await supabase.from('questions').select('*').eq('assessment_id', assessmentId!).order('positio
n')
81 |       if (error) {
82 |         console.error('Questions fetch error:', error);
83 |         toast.error(`Error loading questions: ${error.message}`);
84 |         return [];
85 |       }
86 |       return (data || []).map((q: any) => ({
87 |         ...q,
88 |         question_type: (q.options as any)?._question_type || 'mcq',
89 |         difficulty: (q.options as any)?._difficulty || 'medium',
90 |       })).filter((q: any) => q.question_type !== 'open_ended')
91 |     },
92 |     enabled: !!assessmentId,
93 |   })
94 |
95 |   const { data: course, isLoading: isCourseLoading } = useQuery({
96 |     queryKey: ['course', courseId],
97 |     queryFn: async () => {
98 |       const { data, error } = await supabase.from('courses').select('title, course_type').eq('id', courseId!).single()
99 |       if (error) {
100 |         console.error('Course fetch error:', error);
101 |         return null;
102 |       }
103 |       return data
104 |     },
105 |     enabled: !!courseId,
106 |   })
107 |
108 |   const { data: enrollment, isLoading: isEnrollmentLoading } = useQuery({
109 |     queryKey: ['trainee-enrollment-test', courseId, profile?.id],
110 |     queryFn: async () => {
111 |       if (!courseId || !profile?.id) return null
112 |       const { data, error } = await supabase
113 |         .from('enrollments')
114 |         .select('*')
115 |         .eq('course_id', courseId)
116 |         .eq('user_id', profile.id)
117 |         .maybeSingle()
118 |       if (error) return null
119 |       return data
120 |     },
121 |     enabled: !!courseId && !!profile?.id,
122 |   })
123 |
124 |   const { data: previousAttempt, isLoading: isAttemptLoading }: { data: any, isLoading: boolean } = useQuery({
125 |     queryKey: ['assessment-attempt', assessmentId, profile?.id],
126 |     queryFn: async () => {
127 |       const { data, error } = await supabase.from('assessment_attempts' as any)
128 |         .select('*')
129 |         .eq('user_id', profile!.id)
130 |
131 |       if (error) {
132 |         console.error('Attempt fetch error:', error)
133 |         return null // Don't block the test if this fails
134 |       }
135 |
136 |       const matchingAttempts = (data || []).filter((a: any) => a.assessment_id === assessmentId)
137 |       if (matchingAttempts.length === 0) return null
138 |
139 |       // Return the most recent one (using submitted_at or created_at)
140 |       matchingAttempts.sort((a: any, b: any) => {
141 |         const dateA = new Date(a.submitted_at || a.created_at || 0).getTime()
142 |         const dateB = new Date(b.submitted_at || b.created_at || 0).getTime()
143 |         return dateB - dateA
144 |       })
145 |
146 |       return matchingAttempts[0]
147 |     },
148 |     enabled: !!assessmentId && !!profile?.id,
```

```
  })

  // Check if results are hidden

  const areResultsHidden = assessment?.results_publish_date ? new Date(assessment.results_publish_date) > new Date() : false;

  const { data: attemptAnswers, isLoading: isAnswersLoading } = useQuery({
    queryKey: ['attempt-answers', previousAttempt?.id],
    queryFn: async () => {
      if (!previousAttempt?.id) return []
      const { data, error } = await supabase.from('attempt_answers' as any)
        .select('*')
        .eq('attempt_id', previousAttempt.id)
      if (error) {
        console.error('Attempt answers fetch error:', error)
        return []
      }
      return data || []
    },
    enabled: !!previousAttempt?.id && !areResultsHidden,
  })

  // WebRTC Setup & Cleanup

  useEffect(() => {
    if (hasStarted && videoRef.current && mediaStreamRef.current) {
      videoRef.current.srcObject = mediaStreamRef.current
    }
  }, [hasStarted])

  useEffect(() => {
    return () => {
      mediaStreamRef.current?.getTracks().forEach(t => t.stop())
      audioContextRef.current?.close()
      if (noiseWarningTimeoutRef.current) clearTimeout(noiseWarningTimeoutRef.current)
      if (glareTimeoutRef.current) clearTimeout(glareTimeoutRef.current)
      if (roomScanTimeoutRef.current) clearTimeout(roomScanTimeoutRef.current)
      if (faceDetectionIntervalRef.current) clearInterval(faceDetectionIntervalRef.current)
    }
  }, [])

  // Timer Logic

  useEffect(() => {
    if (hasStarted && timeLeft !== null && timeLeft > 0 && !previousAttempt && !isScanningRoom) {
      const timer = setInterval(() => {
        setTimeLeft(prev => {
          if (prev && prev <= 1) {
            clearInterval(timer)
            handleSubmit()
            return 0
          }
          return prev ? prev - 1 : 0
        })
      }, 1000)
      return () => clearInterval(timer)
    }
  }, [hasStarted, timeLeft, previousAttempt])

  // Room Scan Progress

  useEffect(() => {
    if (isScanningRoom) {
      const timer = setInterval(() => {
        setRoomScanProgress(p => {
          if (p >= 100) {
            clearInterval(timer)
            setIsScanningRoom(false)
            setRoomScanProgress(0)
            toast.success("Room scan completed successfully.", { duration: 3000 })
            return 100
          }
          return p + 10 // 10 seconds total (100% / 10 = 10 steps of 1s)
        })
      }, 1000)
      return () => clearInterval(timer)
    }
  }, [isScanningRoom])

```

```
  const exitFullScreen = async () => {
    try {
      const doc = document as any;
      if (doc.fullscreenElement && doc.exitFullscreen) {
        await doc.exitFullscreen();
      } else if (doc.webkitFullscreenElement && doc.webkitExitFullscreen) {
        await doc.webkitExitFullscreen();
      } else if (doc.msFullscreenElement && doc.msExitFullscreen) {
        await doc.msExitFullscreen();
      }
    } catch (e) {
      console.error(e);
    }
  };

  // SEA violation handler - block if ignoring strikes temporarily

  const ignoringViolationsRef = React.useRef(false)

  // Track if current question is open-ended for conditional keyboard blocking

  const isOpenEndedRef = React.useRef(false)
  useEffect(() => {
    const q = randomizedQuestions[currentQuestionIndex]
    const type = (q?.options as any)?._question_type || q?.question_type || 'mcq'
    isOpenEndedRef.current = (type === 'open_ended')
  }, [currentQuestionIndex, randomizedQuestions])

  const handleViolation = (reason: string) => {
    if (ignoringViolationsRef.current) return // grace period
    setStrikes(s => {
      const newStrikes = s + 1
      if (newStrikes >= MAX_STRIKES) {
        toast.error(`SEA Violation: ${reason}. Maximum strikes reached. You have been blocked.`, { duration: 5000 })
        exitFullScreen()
        handleSubmit(true)
      } else {
        toast.error(`SEA Warning (${newStrikes}/${MAX_STRIKES}): ${reason}. Return to the test immediately!`, { duration: 500
0 })
      }
      return newStrikes
    })
  }

  // SEA monitoring

  useEffect(() => {
    if (!hasStarted || previousAttempt || !assessment?.requires_sea) return;

    const handleVisibilityChange = () => {
      if (document.hidden) handleViolation('Tab switched or minimized')
    }

    // Add a 2-second grace period on blur to avoid false positives on fullscreen entry

    let blurTimeout: ReturnType<typeof setTimeout> | null = null
    const handleBlur = () => {
      blurTimeout = setTimeout(() => {
        if (!ignoringViolationsRef.current) handleViolation('Lost focus on assessment window')
      }, 1500)
    }
    const handleFocus = () => {
      if (blurTimeout) { clearTimeout(blurTimeout); blurTimeout = null }
    }

    const handleFullscreenChange = () => {
      const doc = document as any;
      const isFull = doc.fullscreenElement || doc.webkitFullscreenElement || doc.msFullscreenElement;

      if (!isFull) {
        setIsFullScreen(false)
        // Grace period: ignore violations for 1s after exiting fullscreen via browser

        ignoringViolationsRef.current = true
        setTimeout(() => { ignoringViolationsRef.current = false }, 1000)
        handleViolation('Exited fullscreen mode')
      } else {
        setIsFullScreen(true)
        // Grace period after entering fullscreen to avoid false blur strikes
```

```
297 |         ignoringViolationsRef.current = true
298 |         setTimeout(() => { ignoringViolationsRef.current = false }, 2000)
299 |       }
300 |     }
301 |
302 |     const preventAction = (e: Event) => {
303 |       e.preventDefault()
304 |       const actionMap: Record<string, string> = {
305 |         'copy': 'copy content',
306 |         'cut': 'cut content',
307 |         'paste': 'paste content',
308 |         'contextmenu': 'right-click/open context menu'
309 |       }
310 |       handleViolation(`Attempted to ${actionMap[e.type] || e.type}`)
311 |     }
312 |
313 |     const handleKeyDown = (e: KeyboardEvent) => {
314 |       const forbiddenKeys = ['Escape', 'F11', 'PrintScreen', 'Meta', 'Alt', 'Tab']
315 |
316 |       // Block restricted system keys on all questions

317 |       if (forbiddenKeys.includes(e.key) || e.metaKey || e.altKey) {
318 |         e.preventDefault()
319 |         e.stopPropagation()
320 |         handleViolation(`Restricted key usage detected: ${e.key}`)
321 |         return
322 |       }
323 |
324 |       // If it's a multiple choice question, block all keyboard inputs

325 |       if (!isOpenEndedRef.current) {
326 |         e.preventDefault()
327 |         e.stopPropagation()
328 |         handleViolation('Keyboard input disabled for multiple-choice questions. Please use trackpad/mouse.')
329 |       }
330 |     }
331 |
332 |     document.addEventListener('visibilitychange', handleVisibilityChange)
333 |     window.addEventListener('blur', handleBlur)
334 |     window.addEventListener('focus', handleFocus)
335 |     document.addEventListener('fullscreenchange', handleFullscreenChange)
336 |     document.addEventListener('copy', preventAction)
337 |     document.addEventListener('cut', preventAction)
338 |     document.addEventListener('paste', preventAction)
339 |     document.addEventListener('contextmenu', preventAction)
340 |     document.addEventListener('keydown', handleKeyDown, { capture: true })
341 |
342 |     return () => {
343 |       document.removeEventListener('visibilitychange', handleVisibilityChange)
344 |       window.removeEventListener('blur', handleBlur)
345 |       window.removeEventListener('focus', handleFocus)
346 |       document.removeEventListener('fullscreenchange', handleFullscreenChange)
347 |       document.removeEventListener('copy', preventAction)
348 |       document.removeEventListener('cut', preventAction)
349 |       document.removeEventListener('paste', preventAction)
350 |       document.removeEventListener('contextmenu', preventAction)
351 |       document.removeEventListener('keydown', handleKeyDown, { capture: true })
352 |       if (blurTimeout) clearTimeout(blurTimeout)
353 |     }
354 |   }, [hasStarted, previousAttempt, assessment])
355 |
356 |   const requestFullScreen = async () => {
357 |     try {
358 |       const docEl = document.documentElement as any;
359 |       if (docEl.requestFullscreen) {
360 |         await docEl.requestFullscreen()
361 |       } else if (docEl.webkitRequestFullscreen) {
362 |         await docEl.webkitRequestFullscreen()
363 |       } else if (docEl.msRequestFullscreen) {
364 |         await docEl.msRequestFullscreen()
365 |       }
366 |     } catch (err) {
367 |       console.error(err)
368 |       toast.error('Failed to enter fullscreen mode. This is required for SEA.')
369 |       return false
370 |     }
371 |     return true
372 |   }
```

```
373 |
374 |   const startTest = async () => {
375 |     if (assessment?.requires_sea) {
376 |       const canFullscreen = await requestFullScreen()
377 |       if (!canFullscreen) {
378 |          return
379 |       }
380 |       try {
381 |         const stream = await navigator.mediaDevices.getUserMedia({ video: true, audio: true })
382 |         mediaStreamRef.current = stream
383 |
384 |         // Setup audio analysis

385 |         const audioCtx = new (window.AudioContext || (window as any).webkitAudioContext)()
386 |         audioContextRef.current = audioCtx
387 |         const analyser = audioCtx.createAnalyser()
388 |         analyserRef.current = analyser
389 |         analyser.fftSize = 256
390 |         const source = audioCtx.createMediaStreamSource(stream)
391 |         source.connect(analyser)
392 |
393 |         // Polling for audio volume

394 |         const checkAudioVolume = () => {
395 |           if (!analyserRef.current) return
396 |           const dataArray = new Uint8Array(analyserRef.current.frequencyBinCount)
397 |           analyserRef.current.getByteFrequencyData(dataArray)
398 |           const average = dataArray.reduce((acc, val) => acc + val, 0) / dataArray.length
399 |
400 |           if (average > 40) { // Loud noise threshold
401 |             if (!noiseWarningTimeoutRef.current) {
402 |               noiseWarningTimeoutRef.current = setTimeout(() => {
403 |                 handleViolation('Loud background noise detected (Potential voice/talking)')
404 |                 noiseWarningTimeoutRef.current = null
405 |               }, 3000) // 3 seconds of sustained noise
406 |             }
407 |           } else {
408 |             if (noiseWarningTimeoutRef.current) {
409 |               clearTimeout(noiseWarningTimeoutRef.current)
410 |               noiseWarningTimeoutRef.current = null
411 |             }
412 |           }
413 |           requestAnimationFrame(checkAudioVolume)
414 |         }
415 |         checkAudioVolume()
416 |
417 |         // Setup ML Face & Object Detection

418 |         try {
419 |           await tf.ready();
420 |           const [faceModel, objectModel] = await Promise.all([
421 |             blazeface.load(),
422 |             cocoSsd.load()
423 |           ]);
424 |
425 |           faceDetectionIntervalRef.current = setInterval(async () => {
426 |             if (videoRef.current && videoRef.current.readyState >= 2) {
427 |               // 1. Check for faces

428 |               const facePredictions = await faceModel.estimateFaces(videoRef.current, false);
429 |               if (facePredictions.length === 0) {
430 |                 handleViolation('No face detected in camera (ML Check)');
431 |               } else if (facePredictions.length > 1) {
432 |                 handleViolation('Multiple faces detected in camera (ML Check)');
433 |               }
434 |
435 |               // 2. Check for prohibited objects

436 |               const objectPredictions = await objectModel.detect(videoRef.current);
437 |               const prohibitedObjects = ['cell phone', 'book', 'laptop'];
438 |
439 |               for (const prediction of objectPredictions) {
440 |                 if (prohibitedObjects.includes(prediction.class)) {
441 |                    handleViolation(`Prohibited object detected: ${prediction.class} (ML Check)`);
442 |                    break;
443 |                 }
444 |               }
445 |             }
446 |           }, 4000); // Check every 4 seconds to balance performance
```

```
        } catch (e) {
          console.error("ML model load error:", e);
        }

        // Schedule random room scan between 30s and 90s

        roomScanTimeoutRef.current = setTimeout(() => {
          setIsScanningRoom(true)
        }, Math.floor(Math.random() * 60000) + 30000)


      } catch (err) {
        toast.error('Camera and Microphone access are required for Secure Exam Mode.')
        return
      }
    }

    if (!questions || questions.length === 0) {
      toast.error('No questions available for this assessment. Please contact your trainer.')
      exitFullScreen()
      mediaStreamRef.current?.getTracks().forEach(t => t.stop())
      return
    }

    const shuffleArray = <T,>(array: T[]): T[] => {
      const arr = [...array];
      for (let i = arr.length - 1; i > 0; i--) {
        const j = Math.floor(Math.random() * (i + 1));
        [arr[i], arr[j]] = [arr[j], arr[i]];
      }
      return arr;
    };

    const INTERNAL_KEYS = new Set(['_question_type', '_difficulty', '_section']);
    const randomizedQs = shuffleArray(questions).map(q => {
       if (q.options && typeof q.options === 'object') {
           // Filter out internal metadata keys before shuffling

           const optionEntries = Object.entries(q.options).filter(([k]) => !INTERNAL_KEYS.has(k));
           const correctKeys = (q.correct_answer || '').split(',').map((k: string) => k.trim()).filter(Boolean);
           const correctTexts = correctKeys.map((k: string) => (q.options as Record<string, string>)[k]).filter(Boolean);

           const internalMeta: Record<string, string> = {};
           Object.entries(q.options).filter(([k]) => INTERNAL_KEYS.has(k)).forEach(([k,v]) => { internalMeta[k] = v as strin
g; });


           const shuffledEntries = shuffleArray(optionEntries);

           const newOptions: Record<string, string> = { ...internalMeta }; // preserve metadata
           const newCorrectKeys: string[] = [];

           const labels = ['A', 'B', 'C', 'D', 'E', 'F'];
           shuffledEntries.forEach((entry: any, index) => {
               const label = labels[index] || String.fromCharCode(65 + index);
               newOptions[label] = entry[1];
               if (correctTexts.includes(entry[1])) {
                   newCorrectKeys.push(label);
               }
           });

           const newCorrectAnswer = newCorrectKeys.length > 0 ? newCorrectKeys.sort().join(',') : q.correct_answer;

           return { ...q, options: newOptions, correct_answer: newCorrectAnswer };
       }
       return q;
    });

    setRandomizedQuestions(randomizedQs)

    if (assessment?.is_adaptive) {
       const firstQIndex = randomizedQs.findIndex((q: any) => q.difficulty === 'medium')
       if (firstQIndex !== -1) {
           setCurrentQuestionIndex(firstQIndex)
       }
    }

    if (assessment?.duration_minutes) {
      setTimeLeft(assessment.duration_minutes * 60)
    } else {
```

```
523 |       setTimeLeft(30 * 60) // default 30 mins
524 |     }
525 |     setHasStarted(true)
526 |   }
527 |
528 |   const submitMutation = useMutation({
529 |     mutationFn: async (results: any) => {
530 |       let attemptData: any;
531 |       // Insert attempt (only using valid schema columns based on types.ts)

532 |       const { data: attempt, error: attemptError } = await supabase.from('assessment_attempts').insert({
533 |         assessment_id: assessmentId!,
534 |         user_id: profile!.id,
535 |         answers: results.answers,
536 |         score: results.score,
537 |         passed: results.score >= (assessment?.passing_score ?? 50),
538 |         grade_status: results.hasOpenEnded ? 'pending_manual' : 'graded',
539 |         submitted_at: new Date().toISOString()
540 |       } as any).select().single()
541 |
542 |       if (attemptError) {
543 |         console.error('Submit attempt error:', attemptError)
544 |         // Fallback for extreme cases (missing columns, but types say these exist)

545 |         const { data: attempt2, error: attemptError2 } = await supabase.from('assessment_attempts').insert({
546 |           assessment_id: assessmentId!,
547 |           user_id: profile!.id,
548 |           score: results.score,
549 |           passed: results.score >= (assessment?.passing_score ?? 50),
550 |           submitted_at: new Date().toISOString()
551 |         } as any).select().single()
552 |         if (attemptError2) throw attemptError2
553 |
554 |         // Re-assign attemptData for the attempt_answers block to use

555 |         attemptData = attempt2 as any
556 |       } else {
557 |         attemptData = attempt as any
558 |       }
559 |
560 |       // Assuming attempt_answers table exists, but we'll mock it if it fails

561 |       try {
562 |           const activeQs = randomizedQuestions.length > 0 ? randomizedQuestions : questions;
563 |           const answersToInsert = Object.entries(results.answers || {}).map(([qId, answer]) => {
564 |             const q = activeQs?.find(q => q.id === qId)
565 |             const is_correct = ((answer as string) || '').split(',').map((s: string)=>s.trim()).sort().join(',') === (q?.corr
ect_answer || '').split(',').map((s: string)=>s.trim()).sort().join(',')
566 |             return {
567 |               attempt_id: attemptData.id,
568 |               question_id: qId,
569 |               selected_answer: answer
570 |             }
571 |           })
572 |         if (answersToInsert.length > 0) {
573 |           const { error: insertError } = await supabase.from('attempt_answers' as any).insert(answersToInsert)
574 |           if (insertError) {
575 |              console.error("Attempt answers insert error:", insertError)
576 |           }
577 |
578 |           // ALWAYS backup to local storage just in case DB fails (Schema might be missing tables/columns)

579 |           if (attemptData?.id) {
580 |               localStorage.setItem(`attempt_answers_${attemptData.id}`, JSON.stringify(results.answers));
581 |           }
582 |         }
583 |       } catch (e) {
584 |         console.error("Could not insert answers, table might not exist yet.", e)
585 |       }
586 |       return attemptData
587 |     },
588 |     onSuccess: (data) => {
589 |       exitFullScreen()
590 |       // Seed the cache with the new attempt so the Hub updates instantly

591 |       queryClient.setQueryData(['trainee_assessment_attempts', profile?.id], (old: any) => {
592 |         return [...(old || []), data]
593 |       })
594 |       queryClient.invalidateQueries({ queryKey: ['assessment-attempt', assessmentId, profile?.id] })
```

```
595 |       queryClient.invalidateQueries({ queryKey: ['attempt-answers', data?.id] })
596 |       queryClient.invalidateQueries({ queryKey: ['trainee_assessments_v2'] })
597 |       queryClient.invalidateQueries({ queryKey: ['trainee_assessment_attempts'] })
598 |       toast.success('Test submitted successfully! Marked as completed.')
599 |       navigate('/trainee/assessments', { state: { tab: 'completed' } })
600 |     },
601 |     onError: (err: any) => toast.error(err.message || 'Failed to submit assessment')
602 |   })
603 |
604 |   const handleSubmit = async (isBlocked: boolean = false) => {
605 |     if (!questions || questions.length === 0) {
606 |       toast.error('Cannot submit: no questions loaded.')
607 |       return
608 |     }
609 |
610 |     if (isBlocked) {
611 |       submitMutation.mutate({ score: -1, hasOpenEnded: false, answers })
612 |       return
613 |     }
614 |
615 |     setShowSubmitModal(true)
616 |   }
617 |
618 |   const confirmSubmit = () => {
619 |     setShowSubmitModal(false)
620 |
621 |     ignoringViolationsRef.current = true
622 |     exitFullScreen()
623 |
624 |     let correct = 0
625 |     let hasOpenEnded = false
626 |
627 |       const getKeywords = (text: string) => {
628 |         if (!text) return [];
629 |         const stopWords = new Set(['the','is','in','at','of','on','and','a','to','it','for','with','as','by','this','that','t
hese','those','an','are','was','were','be','been','being','have','has','had','do','does','did','will','would','shall','should','ca
n','could','may','might','must','ought','i','you','he','she','we','they','what','which','who','whom','whose','where','when','why','h
ow']);
630 |         return text.toLowerCase()
631 |           .replace(/[^a-z0-9\s]/g, ' ')
632 |           .split(/\s+/)
633 |           .filter(w => w.length > 2 && !stopWords.has(w));
634 |       };
635 |
636 |     const activeQs = randomizedQuestions.length > 0 ? randomizedQuestions : (questions || []);
637 |     activeQs.forEach(q => {
638 |       const type = (q.options as any)?._question_type || (q as any).question_type || 'mcq'
639 |       if (type === 'open_ended') {
640 |         hasOpenEnded = true
641 |
642 |         const studentAns = answers[q.id] || '';
643 |         const refAns = q.correct_answer || '';
644 |
645 |         const refKeywords = getKeywords(refAns);
646 |         const studentKeywords = getKeywords(studentAns);
647 |
648 |         if (refKeywords.length > 0) {
649 |           let matchCount = 0;
650 |           refKeywords.forEach(rk => {
651 |             if (studentKeywords.some(sk => sk === rk || (sk.length > 3 && (sk.includes(rk) || rk.includes(sk))))) {
652 |               matchCount++;
653 |             }
654 |           });
655 |           const ratio = matchCount / refKeywords.length;
656 |
657 |           if (ratio >= 0.5) correct += 1;
658 |           else if (ratio >= 0.25) correct += 0.5;
659 |         } else if (studentAns.trim().length > 10) {
660 |           correct += 0.5;
661 |         }
662 |       } else {
663 |         const cleanOpts = Object.entries(q.options || {}).filter(([k]) => !['_question_type','_difficulty','_section'].includ
es(k));
664 |         // Answers are stored as option text values now

665 |         const studentTexts = (answers[q.id] || '').split('|||').map((s: string) => s.trim()).filter(Boolean);
666 |         // Get the correct option text(s) by looking up the correct_answer key in options

667 |         const correctTexts = (q.correct_answer || '').split(',').map((k: string) => {
```

```
668 |           const t = k.trim();
669 |           return (q.options as any)?.[t] as string || t;
670 |         }).filter(Boolean);
671 |
672 |         if (studentTexts.length > 0 && correctTexts.length > 0) {
673 |           const studentSorted = [...studentTexts].sort().join('|||');
674 |           const correctSorted = [...correctTexts].sort().join('|||');
675 |           if (studentSorted === correctSorted) correct++;
676 |         }
677 |       }
678 |     })
679 |     const score = activeQs.length > 0 ? Math.round((correct / activeQs.length) * 100) : 0
680 |     submitMutation.mutate({ score, hasOpenEnded, answers })
681 |   }
682 |
683 |   const formatTime = (seconds: number) => {
684 |     const m = Math.floor(seconds / 60)
685 |     const s = seconds % 60
686 |     return `${m}:${s < 10 ? '0' : ''}${s}`
687 |   }
688 |
689 |   const handleCopyPaste = (e: React.ClipboardEvent) => {
690 |     if (assessment?.requires_sea) {
691 |       e.preventDefault()
692 |       toast.error('Copy/Paste is disabled in Secure Exam Mode.')
693 |     }
694 |   }
695 |
696 |   const traineeNavLinks = [
697 |     { label: 'Dashboard', to: '/trainee/dashboard', icon: LayoutDashboard },
698 |     { label: 'Course Catalog', to: '/trainee/courses', icon: Compass },
699 |     { label: 'My Learning', to: '/trainee/my-learning', icon: BookOpen },
700 |     { label: 'Assessments', to: '/trainee/assessments', icon: Target },
701 |     { label: 'Notifications', to: '/trainee/notifications', icon: Bell },
702 |     { label: 'Profile', to: '/trainee/profile', icon: User }
703 |   ]
704 |
705 |   if (isAssessmentLoading || isQuestionsLoading || isAttemptLoading || (previousAttempt && !areResultsHidden && isAnswersLoad
ing)) {
706 |     return <DashboardShell title="Assessment" icon={Target} navLinks={traineeNavLinks}><div className="flex justify-center py
-20"><Loader2 className="w-8 h-8 animate-spin text-cyan-600" /></div></DashboardShell>
707 |   }
708 |
709 |   if (!assessment || !questions) {
710 |     return (
711 |       <DashboardShell title="Assessment" icon={Target} navLinks={traineeNavLinks}>
712 |         <div className="text-center py-20">
713 |           <p className="text-lg font-bold text-slate-800">Assessment not found.</p>
714 |           <div className="mt-4 text-xs text-slate-500 bg-slate-50 p-4 rounded-lg inline-block text-left">
715 |             <p>Debug Info:</p>
716 |             <p>Assessment ID: {assessmentId}</p>
717 |             <p>Course ID: {courseId}</p>
718 |             <p>Assessment loaded: {assessment ? 'Yes' : 'No'}</p>
719 |             <p>Questions loaded: {questions ? 'Yes' : 'No'}</p>
720 |             <p className="text-red-500 font-bold mt-2">Error: {assessmentError ? (assessmentError as any).message || JSON.str
ingify(assessmentError) : 'None'}</p>
721 |           </div>
722 |         </div>
723 |       </DashboardShell>
724 |     )
725 |   }
726 |
727 |   // Strip file extensions & underscores from titles set to raw filenames

728 |   const cleanTitle = (raw: string) =>
729 |     raw
730 |       .replace(/\.[a-zA-Z0-9]{2,5}(\s|$)/g, '$1') // remove .pdf .docx etc
731 |       .replace(/_/g, ' ')                          // underscores → spaces
732 |       .replace(/\s+/g, ' ')                        // collapse multiple spaces
733 |       .trim()
734 |
735 |   const displayTitle = cleanTitle(assessment.title)
736 |
737 |   // Completed State

738 |   if (previousAttempt) {
739 |     const attemptData = previousAttempt as any
740 |     return (
741 |       <TraineeAssessmentResult
```

```
        assessment={assessment}
        questions={questions || []}
        attemptData={attemptData}
        attemptAnswers={attemptAnswers || []}
        areResultsHidden={areResultsHidden}
        displayTitle={displayTitle}
        traineeNavLinks={traineeNavLinks}
        profile={profile}
        course={course}
      />
    )
  }

  // Pre-Start State

  if (!hasStarted) {
    const isFinal = assessment.assessment_type === 'final'
    const isPractice = assessment.assessment_type === 'mock' || assessment.assessment_type === 'daily'
    const isRegularAssessment = assessment.assessment_type === 'assessment'
    const isCourseModuleCompleted = (enrollment?.progress_percent ?? 0) >= 100 || enrollment?.status === 'completed'
    const isAdminUser = profile?.role === 'admin' || profile?.role === 'super_admin'
    const isFinalBlockedByModules = !isAdminUser && isFinal && !isCourseModuleCompleted

    let gating = { allowed: true, message: '' };

    if (isFinalBlockedByModules) {
      gating = {
        allowed: false,
        message: `Final Assessment is locked. You must complete 100% of all course learning modules before taking the Final Assessment (Current Progress: ${enrollment?.progress_percent ?? 0}%).`
      };
    } else if (assessment.scheduled_date && !isAdminUser) {
      const now = currentTime;
      const startStr = `${assessment.scheduled_date}T${assessment.start_time || '00:00:00'}`;
      const endStr = `${assessment.scheduled_date}T${assessment.end_time || '23:59:59'}`;
      const startTime = new Date(startStr);
      const endTime = new Date(endStr);

      if (endTime < startTime) {
        endTime.setDate(endTime.getDate() + 1);
      }

      if (now < startTime) {
        if (now.toDateString() !== startTime.toDateString()) {
           gating = { allowed: false, message: `This assessment is scheduled for ${startTime.toLocaleDateString()}` };
        } else {
           const diffSecs = Math.floor((startTime.getTime() - now.getTime()) / 1000);
           const h = Math.floor(diffSecs / 3600);
           const m = Math.floor((diffSecs % 3600) / 60);
           const s = diffSecs % 60;
           const countdownStr = `Starts in ${h > 0 ? `${h}h ` : ''}${m}m ${s < 10 ? '0' : ''}${s}s`;
           gating = { allowed: false, message: `This assessment opens at ${assessment.start_time} (${countdownStr})` };
        }
      } else if (now > endTime) {
        gating = { allowed: false, message: 'This assessment has expired and is now closed.' };
      }
    }

    return (
      <DashboardShell title={displayTitle} icon={Target} navLinks={traineeNavLinks}>
        <div className="max-w-4xl mx-auto">
          <Link to={`/trainee/assessments`} className="inline-flex items-center gap-2 text-sm text-slate-500 hover:text-cyan-600 transition-colors mb-6 font-semibold">
            <ArrowLeft className="w-4 h-4" /> Back to Assessments
          </Link>
          <div className="bg-white border border-slate-200/90 rounded-3xl p-10 shadow-sm text-center max-w-2xl mx-auto">
            <div className="w-16 h-16 bg-cyan-50 rounded-2xl flex items-center justify-center mx-auto mb-4 border border-cyan-200 text-cyan-600 shadow-sm">
              <Target className="w-8 h-8" />
            </div>

            {/* Test Type & Grade Weight Banner */}
            <div className="flex justify-center mb-3">
              {isPractice ? (
                <span className="px-3.5 py-1 rounded-full text-xs font-bold uppercase tracking-wider bg-slate-100 text-slate-700 border border-slate-200 flex items-center gap-1.5">
                  <Brain className="w-3.5 h-3.5 text-slate-500" />
                  {assessment.assessment_type === 'mock' ? 'Mock Test' : 'Daily Test'} • Practice Only (0% Final Grade Weight)
```

```
                </span>
              ) : isFinal ? (
                <span className="px-3.5 py-1 rounded-full text-xs font-bold uppercase tracking-wider bg-purple-50 text-purple
-700 border border-purple-200 flex items-center gap-1.5">
                  <Award className="w-3.5 h-3.5 text-purple-600" />
                  Final Assessment • 50% Final Grade Weight
                </span>
              ) : (
                <span className="px-3.5 py-1 rounded-full text-xs font-bold uppercase tracking-wider bg-amber-50 text-amber-7
00 border border-amber-200 flex items-center gap-1.5">
                  <Target className="w-3.5 h-3.5 text-amber-600" />
                  Regular Assessment • 25% Grade Weight (Averaged)
                </span>
              )}
            </div>

            <h1 className="text-3xl font-black text-slate-900 mb-4 tracking-tight">{displayTitle}</h1>
            <p className="text-slate-600 mb-8 max-w-lg mx-auto leading-relaxed font-medium">{assessment.instructions || 'Plea
se read each question carefully before answering. Good luck!'}</p>

            <div className="grid grid-cols-2 sm:grid-cols-4 gap-4 mb-8 max-w-xl mx-auto">
              <div className="bg-slate-50 p-4 rounded-2xl border border-slate-100 flex flex-col items-center justify-center">
                <p className="text-[10px] uppercase font-bold text-slate-400 mb-1 tracking-wider">Duration</p>
                <p className="font-bold text-slate-900 flex items-center gap-1.5"><Clock className="w-4 h-4 text-amber-600" /
> {assessment.duration_minutes || 30} mins</p>
              </div>
              <div className="bg-slate-50 p-4 rounded-2xl border border-slate-100 flex flex-col items-center justify-center">
                <p className="text-[10px] uppercase font-bold text-slate-400 mb-1 tracking-wider">Questions</p>
                <p className="font-bold text-slate-900 flex items-center gap-1.5"><Target className="w-4 h-4 text-cyan-600" /
> {questions?.length || 0}</p>
              </div>
              <div className="bg-slate-50 p-4 rounded-2xl border border-slate-100 flex flex-col items-center justify-center">
                <p className="text-[10px] uppercase font-bold text-slate-400 mb-1 tracking-wider">Marks</p>
                <p className="font-bold text-slate-900 flex items-center gap-1.5"><Award className="w-4 h-4 text-emerald-600"
/> {(questions?.length || 0) * 10}</p>
              </div>
              <div className="bg-slate-50 p-4 rounded-2xl border border-slate-100 flex flex-col items-center justify-center">
                <p className="text-[10px] uppercase font-bold text-slate-400 mb-1 tracking-wider">Weight</p>
                <p className="font-bold text-slate-900 flex items-center gap-1.5 text-xs">
                  {isPractice ? 'Practice (0%)' : isFinal ? 'Final (50%)' : 'Avg (25%)'}
                </p>
              </div>
            </div>

            {/* Grading Breakdown Note */}
            <div className="bg-slate-50 border border-slate-200/80 rounded-2xl p-4 mb-8 text-left text-xs text-slate-600">
              <span className="font-bold text-slate-800 block mb-1">Course Grading Breakdown:</span>
              <div className="grid grid-cols-1 sm:grid-cols-3 gap-2 text-[11px] mt-2">
                <div className="p-2 rounded-xl bg-white border border-slate-200">
                  <span className="text-slate-400 block font-semibold">1. Course Modules</span>
                  <span className="font-bold text-cyan-700">25% of Final Grade</span>
                </div>
                <div className="p-2 rounded-xl bg-white border border-slate-200">
                  <span className="text-slate-400 block font-semibold">2. Assessments Avg</span>
                  <span className="font-bold text-amber-700">25% of Final Grade</span>
                </div>
                <div className="p-2 rounded-xl bg-white border border-slate-200">
                  <span className="text-slate-400 block font-semibold">3. Final Assessment</span>
                  <span className="font-bold text-purple-700">50% of Final Grade</span>
                </div>
              </div>
            </div>

            {assessment.requires_sea && (
              <div className="bg-rose-50/80 border border-rose-200 rounded-2xl p-4 mb-8 text-left">
                <div className="flex items-center gap-2 mb-2 text-rose-800">
                  <ShieldAlert className="w-5 h-5 text-rose-600" />
                  <h3 className="font-bold text-sm">Secure Exam Mode (SEA) is enabled</h3>
                </div>
                <ul className="text-xs text-rose-700 list-disc list-inside space-y-1 font-medium">
                  <li>You will be forced into <b>full-screen mode</b>.</li>
                  <li>Do not minimize the browser or switch tabs.</li>
                  <li>Copy-pasting is disabled.</li>
                  <li>If you receive {MAX_STRIKES} strikes, the exam will automatically submit.</li>
                </ul>
              </div>
            )}

            {!gating.allowed ? (
```

```
888 |               <div className="flex flex-col items-center space-y-4">
889 |                 <div className="bg-amber-50 border border-amber-200 text-amber-800 p-6 rounded-2xl flex flex-col items-center
gap-3 w-full">
890 |                   <AlertCircle className="w-8 h-8 text-amber-600" />
891 |                   <p className="font-bold text-sm leading-relaxed">{gating.message}</p>
892 |                 </div>
893 |                 {isFinalBlockedByModules ? (
894 |                   <Button
895 |                     onClick={() => navigate(`/trainee/courses/${courseId}/learn`)}
896 |                     className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold r
ounded-xl px-6 py-3 text-sm shadow-md"
897 |                   >
898 |                     Go to Course Modules ({enrollment?.progress_percent ?? 0}%)
899 |                   </Button>
900 |                 ) : (
901 |                   <Button onClick={() => navigate(`/trainee/assessments`)} variant="outline" className="text-slate-600 font-b
old rounded-xl border-slate-200">
902 |                     <ArrowLeft className="w-4 h-4 mr-2" /> Back to Assessments
903 |                   </Button>
904 |                 )}
905 |               </div>
906 |             ) : (
907 |               <Button onClick={startTest} className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95
text-white font-bold rounded-2xl px-10 py-6 text-lg w-full sm:w-auto shadow-lg shadow-cyan-600/20 transition-all hover:scale-105 act
ive:scale-95">
908 |                 Start Assessment
909 |               </Button>
910 |             )}
911 |           </div>
912 |         </div>
913 |       </DashboardShell>
914 |     )
915 |   }
916 |
917 |
918 |   if (submitMutation.isPending || submitMutation.isSuccess) {
919 |     return (
920 |       <DashboardShell title={assessment.title} icon={Target} navLinks={traineeNavLinks}>
921 |         <div className="flex flex-col items-center justify-center py-20 space-y-4">
922 |           <Loader2 className="w-10 h-10 animate-spin text-cyan-600" />
923 |           <p className="text-slate-600 font-medium">Saving your assessment...</p>
924 |         </div>
925 |       </DashboardShell>
926 |     )
927 |   }
928 |
929 |   // Active Test State

930 |   const activeQuestions = randomizedQuestions.length > 0 ? randomizedQuestions : (questions || [])
931 |   const currentQ = activeQuestions[currentQuestionIndex]
932 |   const progressPercent = assessment.is_adaptive
933 |     ? (Math.min(adaptiveHistory.length + 1, questions.length) / questions.length) * 100
934 |     : ((currentQuestionIndex) / questions.length) * 100
935 |
936 |   return (
937 |     <>
938 |     <div className="min-h-screen bg-slate-50 flex flex-col items-center pb-24">
939 |       <div className="w-full bg-white border-b border-slate-200 px-6 py-4 flex items-center justify-between mb-8 shadow-sm">
940 |          <div className="flex items-center gap-2">
941 |             <Target className="w-6 h-6 text-cyan-600" />
942 |             <h1 className="text-xl font-bold text-slate-900">{assessment.title}</h1>
943 |          </div>
944 |       </div>
945 |       <div
946 |         className={`max-w-3xl w-full mx-auto px-4 ${assessment.requires_sea ? 'select-none' : ''}`}
947 |         onCopy={handleCopyPaste}
948 |         onPaste={handleCopyPaste}
949 |       >
950 |
951 |         {assessment.requires_sea && (
952 |           <div className="flex items-center justify-center gap-2 mb-4 bg-rose-50 border border-rose-200 text-rose-700 py-2 px
-4 rounded-full text-xs font-bold mx-auto w-fit shadow-sm">
953 |             <Lock className="w-3.5 h-3.5" />
954 |             SECURE EXAM MODE ACTIVE
955 |           </div>
956 |         )}
957 |
958 |         {assessment.requires_sea && (
959 |           <div className="fixed bottom-4 left-4 w-48 h-36 bg-black rounded-2xl overflow-hidden shadow-2xl border-2 border-ros
```

```
e-500 z-50">
 960 |             <video ref={videoRef} autoPlay playsInline muted className="w-full h-full object-cover transform scale-x-[-1]" />
 961 |             <div className="absolute top-2 left-2 flex gap-1">
 962 |               <div className="w-2 h-2 rounded-full bg-rose-500 animate-pulse" />
 963 |               <span className="text-[10px] text-white font-bold tracking-wider drop-shadow-md">LIVE</span>
 964 |             </div>
 965 |           </div>
 966 |         )}
 967 |
 968 |         {/* Hidden Canvas for Video Analysis */}
 969 |         <canvas ref={canvasRef} width="160" height="120" style={{ display: 'none' }} />
 970 |
 971 |         {isScanningRoom && (
 972 |           <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/70 backdrop-blur-sm">
 973 |             <div className="bg-white p-8 rounded-3xl max-w-md w-full shadow-2xl text-center">
 974 |               <div className="w-16 h-16 bg-rose-100 text-rose-600 rounded-2xl flex items-center justify-center mx-auto mb-4">
 975 |                 <ShieldAlert className="w-8 h-8" />
 976 |               </div>
 977 |               <h2 className="text-2xl font-black text-slate-900 mb-2">Random Room Scan</h2>
 978 |               <p className="text-sm text-slate-600 mb-6 font-medium leading-relaxed">
 979 |                 Please pick up your webcam or laptop and slowly pan 360 degrees to show your entire workspace. The test timer
is paused.
 980 |               </p>
 981 |               <Progress value={roomScanProgress} className="h-3 mb-4" />
 982 |               <p className="text-xs font-bold text-slate-400 uppercase tracking-widest animate-pulse">Scanning Environment...
</p>
 983 |             </div>
 984 |           </div>
 985 |         )}
 986 |
 987 |         {/* Header Bar */}
 988 |         <div className={`flex items-center justify-between mb-8 bg-white p-5 rounded-3xl shadow-sm border border-slate-200/90
sticky top-4 z-10 ${isScanningRoom ? 'opacity-20 pointer-events-none' : ''}`}>
 989 |           <div className="flex items-center gap-4 w-1/2">
 990 |             <span className="text-sm font-bold text-slate-900 whitespace-nowrap">
 991 |               {assessment.is_adaptive
 992 |                  ? `Adaptive Q ${adaptiveHistory.length + 1} / ${questions.length}`
 993 |                  : `Question ${currentQuestionIndex + 1} of ${questions.length}`
 994 |               }
 995 |             </span>
 996 |             <Progress value={progressPercent} className="h-2 flex-1" />
 997 |           </div>
 998 |           <div className={`flex items-center gap-2 font-mono font-bold text-base px-4 py-2 rounded-2xl ${timeLeft && timeLeft
< 300 ? 'bg-rose-100 text-rose-700 animate-pulse' : 'bg-slate-100 text-slate-800'}`}>
 999 |             <Clock className="w-4 h-4 text-cyan-600" />
1000 |             {timeLeft !== null ? formatTime(timeLeft) : '00:00'}
1001 |           </div>
1002 |         </div>
1003 |
1004 |         <div className={assessment.is_simulation ? 'flex flex-col md:flex-row gap-6' : ''}>
1005 |           {assessment.is_simulation && assessment.simulation_dataset_url && (
1006 |             <div className="md:w-1/2 bg-white rounded-3xl p-6 shadow-sm border border-slate-200/90 mb-8 flex flex-col">
1007 |               <h3 className="font-bold text-slate-900 mb-4 flex items-center gap-2">
1008 |                 <Target className="w-5 h-5 text-cyan-600"/>
1009 |                 Reference Material
1010 |               </h3>
1011 |               <div className="flex-1 rounded-2xl overflow-hidden border border-slate-200 bg-slate-50 flex items-center justif
y-center min-h-[300px]">
1012 |                 <img
1013 |                   src={assessment.simulation_dataset_url}
1014 |                   alt="Simulation Dataset"
1015 |                   className="max-w-full max-h-[600px] object-contain rounded-lg"
1016 |                   onError={(e) => { (e.target as HTMLImageElement).style.display = 'none' }}
1017 |                 />
1018 |               </div>
1019 |             </div>
1020 |           )}
1021 |
1022 |           <div className={assessment.is_simulation ? 'md:w-1/2 w-full' : 'w-full'}>
1023 |
1024 |         {/* Question Card */}
1025 |         <AnimatePresence mode="wait">
1026 |           <motion.div
1027 |             style={{ filter: isScanningRoom ? 'blur(8px)' : 'none', pointerEvents: isScanningRoom ? 'none' : 'auto' }}
1028 |             key={currentQuestionIndex}
1029 |             initial={{ opacity: 0, x: 20 }}
1030 |             animate={{ opacity: 1, x: 0 }}
1031 |             exit={{ opacity: 0, x: -20 }}
1032 |             transition={{ duration: 0.2 }}
```

```
            className="bg-white rounded-3xl p-8 shadow-sm border border-slate-200/90 mb-8"
          >
            <h2 className="text-xl font-bold text-slate-900 mb-8 leading-snug">{currentQ?.question_text}</h2>

            <div className="space-y-3">
              {(currentQ as any)?.question_type === 'open_ended' ? (
                <textarea
                  value={answers[currentQ.id] || ''}
                  onChange={(e) => setAnswers(prev => ({ ...prev, [currentQ.id]: e.target.value }))}
                  placeholder="Type your answer here..."
                  className="w-full min-h-[150px] p-4 rounded-2xl border-2 border-slate-200 focus:border-cyan-500 focus:ring-
4 focus:ring-cyan-500/10 transition-all outline-none text-slate-900 resize-y font-medium placeholder:text-slate-400 bg-slate-50"
                />
              ) : (
                Object.entries((currentQ?.options as Record<string, string>) || {})
                  .filter(([key]) => !['_question_type', '_difficulty', '_section'].includes(key))
                  .map(([key, opt]) => {
                  const isMultiAnswer = currentQ?.correct_answer && currentQ.correct_answer.includes(',');
                  // Store answers as option TEXT (not letter key) so they survive shuffle remapping

                  const currentSelected = (answers[currentQ.id] || '').split('|||').map(s => s.trim()).filter(Boolean);
                  const isSelected = isMultiAnswer ? currentSelected.includes(opt) : answers[currentQ.id] === opt;

                  return (
                    <button
                      key={key}
                      onClick={() => {
                        if (isMultiAnswer) {
                          if (currentSelected.includes(opt)) {
                            setAnswers(prev => ({ ...prev, [currentQ.id]: currentSelected.filter(k => k !== opt).join('|||')
}));
                          } else {
                            setAnswers(prev => ({ ...prev, [currentQ.id]: [...currentSelected, opt].join('|||') }));
                          }
                        } else {
                          setAnswers(prev => ({ ...prev, [currentQ.id]: opt }))
                        }
                      }}
                      className={`w-full text-left p-4 rounded-2xl border-2 transition-all flex items-center justify-between
${
                        isSelected
                          ? 'border-cyan-500 bg-cyan-50/60 shadow-sm'
                          : 'border-slate-100 hover:border-cyan-300 hover:bg-slate-50'
                      }`}
                    >
                      <span className={`font-semibold text-sm ${isSelected ? 'text-cyan-950' : 'text-slate-700'}`}><span clas
sName="font-bold mr-1.5">{key}.</span>{opt}</span>
                      <div className={`w-5 h-5 flex items-center justify-center shrink-0 ${isMultiAnswer ? 'rounded' : 'round
ed-full'} border-2 ${isSelected ? 'border-cyan-600 bg-cyan-600' : 'border-slate-300'}`}>
                        {isSelected && (
                          isMultiAnswer
                            ? <svg className="w-3.5 h-3.5 text-white" fill="none" viewBox="0 0 24 24" stroke="currentColor"><
path strokeLinecap="round" strokeLinejoin="round" strokeWidth={3} d="M5 13l4 4L19 7" /></svg>
                            : <div className="w-2 h-2 rounded-full bg-white" />
                        )}
                      </div>
                    </button>
                  )
                })
              )}
            </div>
          </motion.div>
        </AnimatePresence>

        {/* Footer Actions */}
        <div className="flex justify-between items-center pb-20 mt-6">
          <Button
            variant="outline"
            onClick={() => {
              if (assessment.is_adaptive) {
                if (adaptiveHistory.length > 0) {
                  const lastQId = adaptiveHistory[adaptiveHistory.length - 1];
                  setAdaptiveHistory(prev => prev.slice(0, -1));
                  const prevQIndex = randomizedQuestions.findIndex(q => q.id === lastQId);
                  setCurrentQuestionIndex(prevQIndex);
                }
              } else {
                setCurrentQuestionIndex(p => Math.max(0, p - 1));
              }
```

```
            }}
            disabled={assessment.is_adaptive ? adaptiveHistory.length === 0 : currentQuestionIndex === 0}
            className="rounded-xl font-bold border-slate-200 text-slate-700 hover:bg-slate-50 bg-white shadow-sm"
          >
            Previous
          </Button>

          {(!assessment.is_adaptive && currentQuestionIndex === questions.length - 1) || (assessment.is_adaptive && adaptiveHistory.length + 1 >= questions.length) ? (
            <Button
              onClick={() => handleSubmit(false)}
              className="bg-emerald-600 hover:bg-emerald-700 text-white rounded-xl font-bold px-8 shadow-md shadow-emerald-600/20"
            >
              Submit Assessment
            </Button>
          ) : (
            <Button
              onClick={() => {
                if (assessment.is_adaptive) {
                  const isCorrect = (answers[currentQ.id] || '').split(',').map((s: string)=>s.trim()).sort().join(',') === (currentQ.correct_answer || '').split(',').map((s: string)=>s.trim()).sort().join(',');
                  const nextDiff = isCorrect
                    ? (currentDifficulty === 'easy' ? 'medium' : 'hard')
                    : (currentDifficulty === 'hard' ? 'medium' : 'easy');

                  setCurrentDifficulty(nextDiff);
                  setAdaptiveHistory(prev => [...prev, currentQ.id]);

                  let nextQIndex = randomizedQuestions.findIndex((q: any, idx: number) =>
                    q.difficulty === nextDiff && !answers[q.id] && idx !== currentQuestionIndex && !adaptiveHistory.includes(q.id)
                  );

                  if (nextQIndex === -1) {
                    nextQIndex = randomizedQuestions.findIndex((q: any, idx: number) =>
                      !answers[q.id] && idx !== currentQuestionIndex && !adaptiveHistory.includes(q.id)
                    );
                  }

                  if (nextQIndex !== -1) {
                    setCurrentQuestionIndex(nextQIndex);
                  } else {
                    handleSubmit();
                  }
                } else {
                  setCurrentQuestionIndex(p => Math.min(questions.length - 1, p + 1));
                }
              }}
              className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white rounded-xl font-bold px-6 shadow-md shadow-cyan-600/10"
            >
              Next Question
            </Button>
          )}
        </div>
      </div>
    </div>
    </div>
    </div>

      {/* Custom Submit Confirmation Modal */}
      <AnimatePresence>
        {showSubmitModal && (
          <motion.div
            initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}
            className="fixed inset-0 z-50 bg-slate-900/40 backdrop-blur-sm flex items-center justify-center p-4"
          >
            <motion.div
              initial={{ scale: 0.95, opacity: 0, y: 10 }}
              animate={{ scale: 1, opacity: 1, y: 0 }}
              exit={{ scale: 0.95, opacity: 0, y: 10 }}
              className="bg-white rounded-3xl p-8 max-w-sm w-full shadow-2xl border border-slate-100 text-center"
            >
              <div className="w-16 h-16 bg-amber-50 rounded-2xl flex items-center justify-center mx-auto mb-4 border border-amber-100 text-amber-500">
                <AlertCircle className="w-8 h-8" />
              </div>
              <h2 className="text-xl font-black text-slate-900 mb-2">Submit Assessment?</h2>
```

```
              <p className="text-sm text-slate-500 font-medium mb-8">
                Are you sure you want to submit? Once submitted, you cannot change your answers.
              </p>
              <div className="flex gap-3">
                <Button
                  onClick={() => setShowSubmitModal(false)}
                  className="flex-1 bg-slate-100 hover:bg-slate-200 text-slate-700 font-bold rounded-xl"
                >
                  Cancel
                </Button>
                <Button
                  onClick={confirmSubmit}
                  className="flex-1 bg-gradient-to-r from-emerald-500 to-emerald-600 hover:opacity-95 text-white font-bold ro
unded-xl shadow-md shadow-emerald-500/20"
                >
                  Submit
                </Button>
              </div>
            </motion.div>
          </motion.div>
        )}
      </AnimatePresence>
    </>
  )
}

```

src/features/courses/TraineeCourseCatalog.tsx TSX 17.1 KB 310 Lines

```tsx
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import { Link } from 'react-router-dom'
import { motion } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { DashboardShell } from '@/pages/Dashboards'
import { Compass, BookOpen, Clock, Search, BookMarked, ArrowRight, Sparkles, FileCheck, User, BarChart3, CheckCircle2, Play, AlertCircle, Bell } from 'lucide-react'
import { Thumbnail } from '@/components/ui/Thumbnail'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

const categories = ['All', 'Standard', 'Scenario']

function formatDurationDisplay(minutes?: number | null) {
  if (!minutes || minutes <= 0) return 'Self-paced'
  const h = Math.floor(minutes / 60)
  const m = minutes % 60
  if (h > 0 && m > 0) return `${h}h ${m}m`
  if (h > 0) return `${h}h`
  return `${m}m`
}

export function TraineeCourseCatalog() {
  const { profile } = useAuth()
  const [searchQuery, setSearchQuery] = React.useState('')
  const [selectedCategory, setSelectedCategory] = React.useState('All')

  const { data: courses, isLoading } = useQuery({
    queryKey: ['published_courses'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('courses')
        .select(`
          *,
          trainer:trainers!courses_trainer_id_fkey(full_name)
        `)
        .eq('status', 'published')
        .order('created_at', { ascending: false })

      if (error) throw error
      return data
    }
  })

  // Fetch trainee's current enrollments
  const { data: enrollments = [] } = useQuery({
    queryKey: ['trainee_catalog_enrollments', profile?.id],
    queryFn: async () => {
      if (!profile?.id) return []
      const { data, error } = await supabase
        .from('enrollments')
        .select('id, course_id, status, progress_percent')
        .eq('user_id', profile.id)
      if (error) return []
      return data || []
    },
    enabled: !!profile?.id
  })

  const enrollmentMap = React.useMemo(() => {
    const map = new Map<string, any>()
    enrollments.forEach((e: any) => {
      map.set(e.course_id, e)
    })
    return map
  }, [enrollments])

  const filteredCourses = React.useMemo(() => {
```

```
    if (!courses) return []
    let list = courses
    if (selectedCategory !== 'All') {
      list = list.filter((c: any) => c.course_type?.toLowerCase() === selectedCategory.toLowerCase())
    }
    if (searchQuery.trim()) {
      const q = searchQuery.toLowerCase()
      list = list.filter((course: any) =>
        course.title?.toLowerCase().includes(q) ||
        course.description?.toLowerCase().includes(q) ||
        course.course_type?.toLowerCase().includes(q) ||
        course.trainer?.full_name?.toLowerCase().includes(q)
      )
    }
    return list
  }, [courses, searchQuery, selectedCategory])

  return (
    <DashboardShell
      title="Course Catalog"
      icon={Compass}
      navLinks={[
        { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
        { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
        { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
        { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
        { to: '/trainee/profile', label: 'Profile', icon: User },
      ]}
    >
      <div className="max-w-6xl space-y-6">
        {/* Header & Search Bar (Midnight Dark Banner) */}
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex flex-col md:flex-row md:items-center justify-between gap-4 bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white p-6 rounded-3xl border border-cyan-500/30 shadow-xl relative overflow-hidden">
          <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />

          <div className="relative z-10">
            <div className="flex items-center gap-2 mb-1">
              <span className="px-2.5 py-0.5 rounded-full bg-white/10 text-amber-300 text-xs font-bold uppercase tracking-wider flex items-center gap-1 border border-white/15">
                <Sparkles className="w-3 h-3 text-amber-300" /> Catalog
              </span>
              <span className="text-xs text-slate-300 font-medium">
                {filteredCourses.length} {filteredCourses.length === 1 ? 'Program' : 'Programs'} available
              </span>
            </div>
            <h2 className="text-2xl font-extrabold text-white tracking-tight">Available Courses</h2>
            <p className="text-slate-300 text-sm">Discover and enroll in high-impact MoES competency tracks.</p>
          </div>

          <div className="flex flex-col sm:flex-row items-center gap-3 relative z-10">
            {/* Search Input */}
            <div className="relative w-full sm:w-72">
              <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
              <input
                type="text"
                placeholder="Search courses, trainers..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="w-full bg-slate-900/80 border border-slate-700/80 rounded-2xl pl-10 pr-4 py-2.5 text-sm text-white placeholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-slate-900 focus:ring-2 focus:ring-cyan-500/20 transition-all shadow-xs"
              />
            </div>
          </div>
        </motion.div>

        {/* Category Pills */}
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex items-center gap-2.5 overflow-x-auto py-2 px-1 scrollbar-none">
          {categories.map(cat => {
            const isSelected = selectedCategory === cat
            return (
              <button
                key={cat}
                onClick={() => setSelectedCategory(cat)}
                className={`px-4 py-1.5 rounded-full text-xs font-bold transition-all duration-200 whitespace-nowrap cursor-po
```

```
inter ${
146 |                   isSelected
147 |                     ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white shadow-md shadow-cyan-600/30 ring-2 r
ing-cyan-500/20'
148 |                     : 'bg-white border border-slate-200 text-slate-700 hover:text-slate-900 hover:bg-slate-50 shadow-xs hover:
border-slate-300'
149 |                 }`}
150 |               >
151 |                 {cat === 'All' ? 'All Courses' : `${cat} Courses`}
152 |               </button>
153 |             )
154 |           })}
155 |         </motion.div>
156 |
157 |         {/* Course Grid */}
158 |         {isLoading ? (
159 |           <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
160 |             {[1, 2, 3, 4, 5, 6].map(i => (
161 |               <div key={i} className="h-80 bg-white animate-pulse rounded-3xl border border-slate-200" />
162 |             ))}
163 |           </div>
164 |         ) : filteredCourses.length === 0 ? (
165 |           <div className="p-16 text-center bg-white border border-slate-200 rounded-3xl shadow-sm">
166 |             <div className="w-16 h-16 bg-cyan-50 rounded-2xl flex items-center justify-center mx-auto mb-4 border border-cyan-
200">
167 |               <Compass className="w-8 h-8 text-cyan-600" />
168 |             </div>
169 |             <h3 className="text-lg font-bold text-slate-900 mb-1">{searchQuery ? 'No Matching Courses Found' : 'No Courses Ava
ilable'}</h3>
170 |             <p className="text-slate-500 text-sm max-w-sm mx-auto">{searchQuery ? 'Try clearing your filters or searching for
different keywords.' : 'Check back shortly for newly published training programs.'}</p>
171 |           </div>
172 |         ) : (
173 |           <motion.div variants={stagger} initial="hidden" animate="visible" className="grid grid-cols-1 md:grid-cols-2 lg:grid
-cols-3 gap-6">
174 |             {filteredCourses.map((course: any) => {
175 |               const enrollment = enrollmentMap.get(course.id)
176 |               const isCompleted = enrollment?.status === 'completed' || (enrollment?.progress_percent ?? 0) === 100
177 |               const isEnrolled = enrollment && (enrollment.status === 'enrolled' || enrollment.status === 'in_progress') && !i
sCompleted
178 |               const isPending = enrollment?.status === 'pending'
179 |               const isRejected = enrollment?.status === 'rejected'
180 |
181 |               let targetUrl = `/trainee/courses/${course.id}`
182 |               if (isEnrolled) {
183 |                 targetUrl = `/trainee/learn/${course.id}`
184 |               }
185 |
186 |               return (
187 |                 <motion.div
188 |                   key={course.id}
189 |                   variants={fadeUp}
190 |                   className={`group flex flex-col bg-white border rounded-3xl overflow-hidden shadow-sm hover:shadow-xl hover:
shadow-slate-200/60 transition-all duration-300 hover:-translate-y-1.5 ${
191 |                     isCompleted
192 |                       ? 'border-emerald-200/90 hover:border-emerald-400'
193 |                       : isEnrolled
194 |                       ? 'border-cyan-300 hover:border-cyan-500'
195 |                       : isPending
196 |                       ? 'border-amber-200/90 hover:border-amber-400'
197 |                       : 'border-slate-200/90 hover:border-cyan-300'
198 |                   }`}
199 |                 >
200 |                   {/* Thumbnail Header */}
201 |                   <div className="h-44 relative overflow-hidden bg-slate-100">
202 |                     <Thumbnail path={course.thumbnail_path} alt={course.title} type={course.course_type} />
203 |
204 |                     {/* Top Badges Overlay */}
205 |                     <div className="absolute top-3 left-3 right-3 flex items-center justify-between pointer-events-none gap-
2">
206 |                       <span className="bg-black/60 backdrop-blur-md px-3 py-1 rounded-full text-[10px] font-bold text-white up
percase tracking-wider border border-white/20 shadow-sm">
207 |                         {course.course_type || 'Standard'}
208 |                       </span>
209 |
210 |                       <div className="flex items-center gap-1.5">
211 |                         {isCompleted ? (
212 |                           <span className="bg-emerald-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrab
old text-white flex items-center gap-1 shadow-sm">
```

```
                            <CheckCircle2 className="w-3 h-3 text-white" /> Completed
                          </span>
                        ) : isEnrolled ? (
                          <span className="bg-gradient-to-r from-cyan-600 to-blue-600 backdrop-blur-md px-2.5 py-1 rounded-ful
l text-[10px] font-extrabold text-white flex items-center gap-1 shadow-sm">
                            <Play className="w-2.5 h-2.5 fill-current text-white" /> {enrollment.progress_percent ? `${enrollm
ent.progress_percent}%` : 'Enrolled'}
                          </span>
                        ) : isPending ? (
                          <span className="bg-amber-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabol
d text-white flex items-center gap-1 shadow-sm">
                            <Clock className="w-3 h-3 text-white" /> Pending
                          </span>
                        ) : isRejected ? (
                          <span className="bg-rose-600/95 backdrop-blur-md px-2.5 py-1 rounded-full text-[10px] font-extrabold
text-white flex items-center gap-1 shadow-sm">
                            <AlertCircle className="w-3 h-3 text-white" /> Declined
                          </span>
                        ) : (
                          <span className="bg-black/60 backdrop-blur-md px-2.5 py-1 rounded-full text-[11px] font-medium text-
white flex items-center gap-1 shadow-sm">
                            <Clock className="w-3 h-3 text-amber-400" />
                            <span>{formatDurationDisplay(course.duration_minutes)}</span>
                          </span>
                        )}
                      </div>
                    </div>
                  </div>

                  {/* Card Content */}
                  <div className="p-6 flex-1 flex flex-col">
                    <div className="flex items-center gap-2 text-xs text-cyan-700 font-bold mb-2">
                      <span className="inline-block w-2 h-2 rounded-full bg-cyan-500 shrink-0" />
                      <span className="truncate">{course.department || 'MoES'} • {course.course_type || 'Standard'} Track</spa
n>
                    </div>

                    <h3 className="text-base font-bold text-slate-900 mb-2 group-hover:text-cyan-600 transition-colors line-cl
amp-2 min-h-[3rem] leading-snug">
                      {course.title}
                    </h3>

                    <p className="text-xs text-slate-500 mb-5 line-clamp-2 leading-relaxed flex-1">
                      {course.description || 'Comprehensive training module designed to enhance core competencies.'}
                    </p>

                    {/* Instructor row */}
                    <div className="flex items-center justify-between pt-3 border-t border-slate-100 text-xs text-slate-600 mb
-5">
                      <div className="flex items-center gap-2">
                        <div className="w-6 h-6 rounded-full bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center jus
tify-center text-white text-[10px] font-bold shrink-0">
                          {course.trainer?.full_name?.[0]?.toUpperCase() ?? 'T'}
                        </div>
                        <span className="truncate max-w-[130px] font-medium text-slate-700">
                          {course.trainer?.full_name || 'Assigned Trainer'}
                        </span>
                      </div>
                      <span className="text-[11px] text-cyan-700 font-bold bg-cyan-50 px-2 py-0.5 rounded-full border border-c
yan-200">
                        Verified
                      </span>
                    </div>

                    {/* Action Button */}
                    <Link to={targetUrl} className="w-full">
                      {isCompleted ? (
                        <button className="w-full py-2.5 rounded-2xl bg-emerald-50 hover:bg-emerald-100 text-emerald-800 borde
r border-emerald-300 text-sm font-bold transition-all shadow-xs flex items-center justify-center gap-1.5">
                          <CheckCircle2 className="w-4 h-4 text-emerald-600" />
                          <span>Review Course (Completed)</span>
                          <ArrowRight className="w-4 h-4" />
                        </button>
                      ) : isEnrolled ? (
                        <button className="w-full py-2.5 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 ho
ver:from-cyan-700 hover:to-blue-700 text-white text-sm font-extrabold transition-all shadow-md shadow-cyan-600/25 flex items-center
justify-center gap-1.5 hover:scale-[1.01]">
                          <Play className="w-3.5 h-3.5 fill-current" />
                          <span>{enrollment.progress_percent && enrollment.progress_percent > 0 ? `Resume Learning (${enrollme
```

```
nt.progress_percent}%)` : 'Start Learning'}</span>
                            <ArrowRight className="w-4 h-4" />
                          </button>
                        ) : isPending ? (
                          <button className="w-full py-2.5 rounded-2xl bg-amber-50 hover:bg-amber-100 text-amber-900 border bord
er-amber-300 text-sm font-bold transition-all shadow-xs flex items-center justify-center gap-1.5">
                            <Clock className="w-4 h-4 text-amber-600" />
                            <span>Approval Pending</span>
                            <ArrowRight className="w-4 h-4" />
                          </button>
                        ) : isRejected ? (
                          <button className="w-full py-2.5 rounded-2xl bg-rose-50 hover:bg-rose-100 text-rose-800 border border-
rose-200 text-sm font-bold transition-all shadow-xs flex items-center justify-center gap-1.5">
                            <AlertCircle className="w-4 h-4 text-rose-600" />
                            <span>Enrollment Declined • View Details</span>
                            <ArrowRight className="w-4 h-4" />
                          </button>
                        ) : (
                          <button className="w-full py-2.5 rounded-2xl bg-cyan-50 group-hover:bg-gradient-to-r group-hover:from-
cyan-600 group-hover:via-sky-600 group-hover:to-blue-600 text-cyan-700 group-hover:text-white border border-cyan-200 group-hover:bor
der-transparent text-sm font-bold transition-all duration-300 shadow-xs group-hover:shadow-md group-hover:shadow-cyan-600/20 flex it
ems-center justify-center gap-1.5">
                            <span>Enroll Now</span>
                            <ArrowRight className="w-4 h-4 group-hover:translate-x-1 transition-transform" />
                          </button>
                        )}
                      </Link>
                    </div>
                  </motion.div>
                )
              })}
            </motion.div>
          )}
        </div>
      </DashboardShell>
    )
}
```

src/features/courses/TraineeCourseDetails.tsx TSX 139.5 KB 2,515 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import { useParams, useNavigate, Link } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { DashboardShell } from '@/pages/Dashboards'
import {
  Compass, BookOpen, Clock, User, ArrowLeft, CheckCircle2, Loader2,
  BookMarked, Layers, ChevronDown, ChevronUp, Play, FileText, Link2,
  Lock, Target, Calendar, Video, Download, ExternalLink, Users, UserCheck,
  GraduationCap, Award, PlayCircle,
  ListOrdered, BookCheck, Mail, Send, XCircle, FileCheck, AlertCircle, BarChart3,
  HelpCircle, Sparkles, CheckSquare, RotateCcw, Unlock, Image as ImageIcon, Check, Trophy,
  Eye, Film, Camera, AlignLeft, Globe, Bell, Info
} from 'lucide-react'
import { Badge } from '@/components/ui/badge'
import { Label } from '@/components/ui/label'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { ErrorBoundary } from '@/components/ErrorBoundary'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription, DialogFooter } from '@/components/ui/dialog'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { CourseFeedback } from './CourseFeedback'
import { CourseAnnouncements } from './CourseAnnouncements'
import { CourseChat } from './CourseChat'
import { LiveAttendanceTraineePanel } from './LiveAttendanceTraineePanel'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'
import {
  calculateCourseGradeBreakdown,
  getAssessmentTypeLabel,
  isPracticeAssessment,
  isFinalAssessment,
  isRegularAssessment,
  type CourseGradeBreakdown
} from '@/lib/courseGrading'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}

function getYouTubeEmbedUrl(url?: string): string | null {
  if (!url) return null
  const regExp = /^.*(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|&v=)([^#&?]*).*/
  const match = url.match(regExp)
  return (match && match[2].length === 11) ? `https://www.youtube.com/embed/${match[2]}` : null
}

function LinkedinIcon({ className = 'w-3.5 h-3.5' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path d="M19 3a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h14m-.5 15.5v-5.3a3.26 3.26 0 0 0-3.26-3.26c-.85 0-1.84.52-2.28 1.3v-1.11h-2.79v8.37h2.79v-4.93c0-.77.62-1.4 1.39-1.4a1.4 1.4 0 0 1 1.4 1.4v4.93h2.75M6.88 8.56a1.68 1.68 0 0 0 1.68-1.68c0-.93-.75-1.69-1.68-1.69a1.69 1.69 0 0 0-1.69 1.69c0 .93.76 1.68 1.69 1.68m1.39 9.94v-8.37H5.5v8.37h2.77z" />
    </svg>
  )
}

function GithubIcon({ className = 'w-3.5 h-3.5' }: { className?: string }) {
  return (
    <svg className={className} fill="currentColor" viewBox="0 0 24 24">
      <path fillRule="evenodd" clipRule="evenodd" d="M12 2C6.477 2 2 6.484 2 12.017c0 4.425 2.865 8.18 6.839 9.504.5.092.682-.217.682-.483 0-.237-.008-.868-.013-1.703-2.782.605-3.369-1.343-3.369-1.343-.454-1.158-1.11-1.466-1.11-1.466-.908-.62.069-.608.069-.608 1.003.07 1.53 1.032 1.53 1.032.892 1.53 2.341 1.088 2.91.832.092-.647.35-1.088.636-1.338-2.22-.253-4.555-1.113-4.555-4.951 0-1.093.39-1.988 1.029-2.688-.103-.253-.446-1.272.098-2.65 0 0 .84-.27 2.75 1.026A9.564 9.564 0 0112 6.844c.85.004 1.705.115 2.504.337 1.909-1.296 2.747-1.027 2.747-1.027.546 1.379.202 2.398.1 2.651.64.7 1.028 1.595 1.028 2.688 0 3.848-2.339 4.695-4.566 4.943.359.309.678.92.678 1.855 0 1.338-.012 2.419-.012 2.747 0 .268.18.58.688.482A10.019 10.019 0 0022 12.017C22 6.484 17.522 2 12 2z" />
    </svg>
  )
}

function getMaterialIcon(type: string) {
  if (type === 'video') return <Play className="w-4 h-4 text-blue-500" />
  if (type === 'link') return <Link2 className="w-4 h-4 text-emerald-500" />
```

```
  return <FileText className="w-4 h-4 text-cyan-600" />
}

function getMaterialBg(type: string) {
  if (type === 'video') return 'bg-blue-50 border-blue-100'
  if (type === 'link') return 'bg-emerald-50 border-emerald-100'
  return 'bg-cyan-50 border-cyan-100'
}

function parseSessionFlowText(text: string): Array<{ number: string; title: string; description: string }> {
  if (!text) return []
  const lines = text.split(/\r?\n/).filter(l => l.trim())
  const results: Array<{ number: string; title: string; description: string }> = []

  for (const line of lines) {
    const sessionMatch = line.match(/^[Ss]ession\s+(\d+)\s*[-\-:]\s*([^:]+?)(?::\s*(.+))?$/)
    if (sessionMatch) {
      results.push({
        number: sessionMatch[1],
        title: sessionMatch[2].trim(),
        description: sessionMatch[3]?.trim() || '',
      })
      continue
    }
    const numMatch = line.match(/^(\d+)\.\s+([^:]+?)(?::\s*(.+))?$/)
    if (numMatch) {
      results.push({
        number: numMatch[1],
        title: numMatch[2].trim(),
        description: numMatch[3]?.trim() || '',
      })
      continue
    }
    if (results.length > 0 && !results[results.length - 1].description) {
      results[results.length - 1].description = line.trim()
    }
  }

  return results
}

export function TraineeCourseDetails() {
  const { courseId } = useParams<{ courseId: string }>()
  const { user, profile } = useAuth()
  const navigate = useNavigate()
  const queryClient = useQueryClient()

  const [openSessions, setOpenSessions] = useState<Set<string>>(new Set())
  const [openModules, setOpenModules] = useState<Set<string>>(new Set())
  const [completedModules, setCompletedModules] = useState<string[]>([])
  const [moduleQuizAnswers, setModuleQuizAnswers] = useState<Record<string, Record<number, number>>>({})
  const [moduleQuizScores, setModuleQuizScores] = useState<Record<string, { score: number; passed: boolean; submitted: boolean }>>({})
  const [previewMaterial, setPreviewMaterial] = useState<any | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [downloadingId, setDownloadingId] = useState<string | null>(null)
  const [docLoading, setDocLoading] = useState(false)
  const [aboutModalOpen, setAboutModalOpen] = useState(false)
  const [aboutActiveTab, setAboutActiveTab] = useState<'about' | 'outline'>('about')

  // OTP Verification States

  const [otpDialogType, setOtpDialogType] = useState<'enroll' | 'drop' | null>(null)
  const [otpInput, setOtpInput] = useState('')
  const [isSendingOtp, setIsSendingOtp] = useState(false)
  const [isVerifyingOtp, setIsVerifyingOtp] = useState(false)
  const [otpSent, setOtpSent] = useState(false)
  const [showAttendanceDetails, setShowAttendanceDetails] = useState(false)
  const [sessionAttendance, setSessionAttendance] = useState<Record<string, { entered: number, generated: number, override: string | null }>>({})



  // Load completed modules from storage

  useEffect(() => {
    if (!courseId || !profile?.id) return
    const stored = localStorage.getItem(`cc_mod_progress_${courseId}_${profile.id}`)
    if (stored) {
```

```
      try {
        const parsed = JSON.parse(stored)
        if (Array.isArray(parsed.completed)) setCompletedModules(parsed.completed)
        if (parsed.scores) setModuleQuizScores(parsed.scores)
      } catch (e) {
        console.error('Error loading module progress:', e)
      }
    }
  }, [courseId, profile?.id])

  const toggleSession = (id: string) => {
    setOpenSessions(prev => {
      const next = new Set(prev)
      next.has(id) ? next.delete(id) : next.add(id)
      return next
    })
  }

  const toggleModule = (id: string) => {
    setOpenModules(prev => {
      const next = new Set(prev)
      next.has(id) ? next.delete(id) : next.add(id)
      return next
    })
  }

  const handleSelectQuizAnswer = (moduleId: string, questionIdx: number, optionIdx: number) => {
    setModuleQuizAnswers(prev => ({
      ...prev,
      [moduleId]: {
        ...(prev[moduleId] || {}),
        [questionIdx]: optionIdx
      }
    }))
  }

  const handleSubmitModuleQuiz = async (mod: any) => {
    const rawItems = mod.items || mod.content_items || []
    const quizQuestionsFromItems = rawItems.filter((i: any) => i.type === 'quiz' && i.quiz_data).map((i: any) => i.quiz_data)
    const legacyQuizQuestions = (mod.quiz_questions || [])
    const questions: any[] = [...quizQuestionsFromItems]
    legacyQuizQuestions.forEach((lq: any) => {
      if (!questions.some(q => q.id === lq.id || q.question === lq.question)) {
        questions.push(lq)
      }
    })

    if (questions.length === 0) return

    const userAnswers = moduleQuizAnswers[mod.id] || {}
    const answeredCount = Object.keys(userAnswers).length
    if (answeredCount < questions.length) {
      toast.error(`Please answer all ${questions.length} questions before submitting. (${answeredCount}/${questions.length} a
nswered)`)
      return
    }

    let correctCount = 0
    questions.forEach((q: any, idx: number) => {
      if (userAnswers[idx] === q.correct_option) {
        correctCount++
      }
    })

    const scorePercent = Math.round((correctCount / questions.length) * 100)
    const passed = scorePercent >= 80

    const updatedScores = {
      ...(moduleQuizScores || {}),
      [mod.id]: {
        score: scorePercent,
        passed,
        submitted: true
      }
    }
    setModuleQuizScores(updatedScores)

    if (passed) {
      const newCompleted = Array.from(new Set([...completedModules, mod.id]))
```

```
      setCompletedModules(newCompleted)

      if (profile?.id && courseId) {
        localStorage.setItem(`cc_mod_progress_${courseId}_${profile.id}`, JSON.stringify({
          completed: newCompleted,
          scores: updatedScores
        }))
      }

      // Update progress in database

      const totalMods = (course?.modules?.length) || 1
      const progressPercent = Math.min(100, Math.round((newCompleted.length / totalMods) * 100))

      if (profile?.id) {
        try {
          await supabase
            .from('enrollments')
            .update({
              progress_percent: progressPercent,
              status: progressPercent === 100 ? 'completed' : 'in_progress'
            } as any)
            .eq('course_id', courseId!)
            .eq('user_id', profile.id)
          queryClient.invalidateQueries({ queryKey: ['enrollment', courseId, profile.id] })
          queryClient.invalidateQueries({ queryKey: ['my_learning', profile.id] })
          queryClient.invalidateQueries({ queryKey: ['trainee-profile-completed-badges'] })
        } catch (err) {
          console.warn('Could not update enrollment progress', err)
        }
      }

      toast.success(`🎉 Excellent! You scored ${scorePercent}% (${correctCount}/${questions.length} correct). Module passed! Next module is now unlocked!`, {
        duration: 5000
      })
    } else {
      toast.error(`⚠️ Score: ${scorePercent}% (${correctCount}/${questions.length} correct). Minimum 80% passing score is required to unlock the next module. Please review the module and try again.`, {
        duration: 5000
      })
    }
  }

  const handleRetakeModuleQuiz = (moduleId: string) => {
    setModuleQuizAnswers(prev => ({ ...prev, [moduleId]: {} }))
    setModuleQuizScores(prev => {
      const next = { ...prev }
      delete next[moduleId]
      return next
    })
    toast.info('Quiz reset. You can now choose your answers and try again!')
  }

  const handleCompleteContentModule = async (mod: any) => {
    const newCompleted = Array.from(new Set([...completedModules, mod.id]))
    setCompletedModules(newCompleted)

    if (profile?.id && courseId) {
      localStorage.setItem(`cc_mod_progress_${courseId}_${profile.id}`, JSON.stringify({
        completed: newCompleted,
        scores: moduleQuizScores
      }))
    }

    const totalMods = (course?.modules?.length) || 1
    const progressPercent = Math.min(100, Math.round((newCompleted.length / totalMods) * 100))

    try {
      await supabase
        .from('enrollments')
        .update({
          progress_percent: progressPercent,
          status: progressPercent === 100 ? 'completed' : 'in_progress'
        } as any)
        .eq('course_id', courseId!)
        .eq('user_id', profile!.id)
      queryClient.invalidateQueries({ queryKey: ['enrollment', courseId, profile?.id] })
      queryClient.invalidateQueries({ queryKey: ['my_learning', profile?.id] })
```

```
      queryClient.invalidateQueries({ queryKey: ['trainee-profile-completed-badges'] })
    } catch (err) {
      console.warn('Could not update enrollment progress', err)
    }

    toast.success(`Module "${mod.title}" marked complete! Next module unlocked.`)
  }

  const { data: course, isLoading: isCourseLoading } = useQuery({
    queryKey: ['course-with-trainer', courseId],
    queryFn: async () => {
      const { data: courseData, error } = await supabase
        .from('courses')
        .select(`*, trainer:trainers!courses_trainer_id_fkey(full_name, bio, years_of_experience, qualifications, email, study_details, work_experience, expertise_areas, linkedin_url, website_url, github_url, avatar_path)`)
        .eq('id', courseId!)
        .single() as any
      if (error) throw error

      let modulesList = (courseData.modules && Array.isArray(courseData.modules) && courseData.modules.length > 0)
        ? courseData.modules
        : []

      if (modulesList.length === 0) {
        try {
          const { data: cmData } = await (supabase as any)
            .from('course_modules')
            .select('*')
            .eq('course_id', courseId!)
            .order('order_index')
          if (cmData && cmData.length > 0) {
            modulesList = cmData.map((m: any) => ({
              id: m.id,
              title: m.title,
              description: m.description,
              items: m.content_items || [],
              quiz_questions: m.quiz_questions || [],
              passing_score: m.passing_score ?? 80,
            }))
          }
        } catch {
          // ignore

        }
      }

      const { data: sessionsData } = await supabase
        .from('course_sessions')
        .select('*')
        .eq('course_id', courseId!)
        .order('order_index')

      const { data: materialsData } = await supabase
        .from('materials')
        .select('*')
        .eq('course_id', courseId!)
        .order('created_at')

      const { data: assessmentsData } = await supabase
        .from('assessments')
        .select('id, title, passing_score, status, requires_sea, sea_link, scheduled_date, start_time, end_time, duration_minutes, instructions, created_at, assessment_type')
        .eq('course_id', courseId!)
        .eq('status', 'published')
        .order('created_at')

      return {
        ...courseData,
        modules: modulesList,
        sessions: sessionsData || [],
        materials: materialsData || [],
        assessments: assessmentsData || []
      }
    },
    enabled: !!courseId,
  })

  const { data: enrollment, isLoading: isEnrollmentLoading } = useQuery({
    queryKey: ['enrollment', courseId, profile?.id],
```

```
    queryFn: async () => {
      const { data, error } = await supabase
        .from('enrollments').select('*')
        .eq('course_id', courseId!).eq('user_id', profile!.id).maybeSingle()
      if (error && error.code !== 'PGRST116') throw error
      return data
    },
    enabled: !!courseId && !!profile?.id,
  })

  const { data: traineeAttempts = [] } = useQuery({
    queryKey: ['trainee-course-attempts', courseId, profile?.id],
    queryFn: async () => {
      if (!profile?.id) return []
      const { data, error } = await supabase
        .from('assessment_attempts' as any)
        .select('*')
        .eq('user_id', profile.id)
      if (error) return []
      return (data || []) as any[]
    },
    enabled: !!profile?.id,
  })

  const { data: enrollmentCounts } = useQuery({
    queryKey: ['enrollments-count', courseId],
    queryFn: async () => {
      const { data, error } = await (supabase.rpc as any)('get_course_enrollment_counts', {
        p_course_id: courseId!,
        p_user_id: profile?.id || null
      })
      if (error) throw error
      return (data as any) as { active: number, waitlisted: number, total: number, myWaitlistPosition: number }
    },
    enabled: !!courseId,
  })

  const { data: courseSkills = [] } = useQuery({
    queryKey: ['course-skills', courseId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('course_skills')
        .select('*, skills(name)')
        .eq('course_id', courseId!)
      if (error) return []
      return data || []
    },
    enabled: !!courseId,
  })

  // Fetch session attendance data from DB

  useEffect(() => {
    if (!course?.sessions || !profile?.id) return
    const fetchAttendance = async () => {
      const sessionIds = course.sessions.map((s: any) => s.id)
      if (sessionIds.length === 0) return

      const [{ data: metaData }, { data: attendanceData }] = await Promise.all([
        (supabase as any).from('session_attendance_meta').select('*').in('session_id', sessionIds),
        (supabase as any).from('session_attendance').select('*').in('session_id', sessionIds).eq('user_id', profile.id)
      ])

      const newAttendance: Record<string, { entered: number, generated: number, override: string | null }> = {}

      course.sessions.forEach((s: any) => {
        const meta = metaData?.find((m: any) => m.session_id === s.id)
        const att = attendanceData?.find((a: any) => a.session_id === s.id)

        newAttendance[s.id] = {
          generated: meta?.total_generated || 0,
          entered: att?.entered_count || 0,
          override: att?.status_override || null
        }
      })

      setSessionAttendance(newAttendance)
    }
    fetchAttendance()
```

```
  }, [course?.sessions, profile?.id])

  // Calculate attendance aggregates

  const attendanceAggregates = React.useMemo(() => {
    let totalTracked = 0
    let presentCount = 0
    let partialCount = 0
    let absentCount = 0
    let percentage = 0

    if (!profile?.id || !course?.sessions) return { totalTracked, presentCount, partialCount, absentCount, percentage, userId: profile?.id }

    course.sessions.forEach((s: any) => {
      let att = sessionAttendance[s.id]

      // Fallback to localStorage for legacy local testing data if DB has no data

      if (!att || att.generated === 0) {
        try {
          const localStr = localStorage.getItem(`trainee_attendance_${s.id}_${profile.id}`)
          if (localStr) {
            const localData = JSON.parse(localStr)
            if (localData.generated > 0 || localData.override) {
              att = localData
            }
          }
        } catch(e) {}
      }

      if (att && (att.generated > 0 || att.override)) {
        totalTracked++
        if (att.override === 'present' || att.override === 'P') {
          presentCount++
        } else if (att.override === 'late') {
          partialCount++
        } else if (att.override === 'absent' || att.override === 'F') {
          absentCount++
        } else if (att.entered >= att.generated * 0.5) {
          presentCount++
        } else if (att.entered > 0) {
          partialCount++
        } else {
          absentCount++
        }
      }
    })

    percentage = totalTracked > 0 ? Math.round((presentCount / totalTracked) * 100) : 0
    const userId = profile.id
    return { totalTracked, presentCount, partialCount, absentCount, percentage, userId }
  }, [profile?.id, sessionAttendance, course?.sessions])

  useEffect(() => {
    if (!profile?.id || !courseId) return

    const channel = supabase
      .channel('enrollment_changes')
      .on(
        'postgres_changes',
        {
          event: '*',
          schema: 'public',
          table: 'enrollments',
          filter: `course_id=eq.${courseId}`,
        },
        () => {
          queryClient.invalidateQueries({ queryKey: ['enrollments-count', courseId] })
          queryClient.invalidateQueries({ queryKey: ['enrollment', courseId, profile?.id] })
        }
      )
      .subscribe()

    return () => {
      supabase.removeChannel(channel)
    }
  }, [courseId, profile?.id, queryClient])

```

```
  const dropMutation = useMutation({
    mutationFn: async () => {
      // 1. Update status to 'withdrawn'

      const { error: updateErr } = await supabase
        .from('enrollments')
        .update({ status: 'withdrawn' } as any)
        .eq('course_id', courseId!)
        .eq('user_id', profile!.id)

      if (updateErr) {
        // Fallback to RPC or delete

        const { error: rpcErr } = await (supabase.rpc as any)('drop_enrollment', {
          p_course_id: courseId!,
          p_user_id: profile!.id
        })
        if (rpcErr) {
          const { error: delErr } = await supabase
            .from('enrollments')
            .delete()
            .eq('course_id', courseId!)
            .eq('user_id', profile!.id)
          if (delErr) throw updateErr
        }
      }
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['enrollment', courseId, profile?.id] })
      queryClient.invalidateQueries({ queryKey: ['enrollments-count', courseId] })
      queryClient.invalidateQueries({ queryKey: ['trainee-dashboard-enrollments', profile?.id] })
      toast.success('Successfully un-enrolled / dropped from the course.')
      setOtpDialogType(null)
      setOtpSent(false)
      setOtpInput('')
    },
    onError: (err: any) => {
      toast.error(err.message || 'Failed to drop course.')
    },
  })

  const handlePreview = async (material: any) => {
    if (material.material_type === 'link') {
      window.open(material.external_url, '_blank')
      return
    }
    if (!material.storage_path) return

    try {
      setPreviewMaterial(material)
      setPreviewUrl(null)
      const { data, error } = await supabase.storage
        .from('materials')
        .createSignedUrl(material.storage_path, 300)
      if (error) throw error
      if (data?.signedUrl) {
        setPreviewUrl(data.signedUrl)
      }
    } catch {
      toast.error('Failed to load preview')
      setPreviewMaterial(null)
    }
  }

  const handleDownload = async (material: any) => {
    if (!material.storage_path) return
    setDownloadingId(material.id)
    try {
      const { data, error } = await supabase.storage
        .from('materials')
        .download(material.storage_path)
      if (error) throw error
      const url = URL.createObjectURL(data)
      const a = document.createElement('a')
      a.href = url
      a.download = material.file_name || 'download'
      document.body.appendChild(a)
      a.click()
      document.body.removeChild(a)
```

```
606 |       URL.revokeObjectURL(url)
607 |       toast.success('Downloaded successfully')
608 |     } catch {
609 |       toast.error('Failed to download file')
610 |     } finally {
611 |       setDownloadingId(null)
612 |     }
613 |   }
614 |
615 |   const handleOpenSessionDoc = async () => {
616 |     if (!course?.session_flow_document_path) return
617 |     setDocLoading(true)
618 |     try {
619 |       const { data, error } = await supabase.storage
620 |         .from('materials')
621 |         .createSignedUrl(course.session_flow_document_path, 300)
622 |       if (error) throw error
623 |       if (data?.signedUrl) {
624 |         setPreviewMaterial({
625 |           file_name: 'Session Flow Document',
626 |           storage_path: course.session_flow_document_path,
627 |           material_type: 'document'
628 |         })
629 |         setPreviewUrl(data.signedUrl)
630 |       }
631 |     } catch {
632 |       toast.error('Could not open session flow document.')
633 |     } finally {
634 |       setDocLoading(false)
635 |     }
636 |   }
637 |
638 |   const handleInitiateOtp = async (type: 'enroll' | 'drop') => {
639 |     setOtpDialogType(type)
640 |     setOtpSent(false)
641 |     setOtpInput('')
642 |   }
643 |
644 |   const handleSendOtp = async () => {
645 |     if (!profile?.email) return
646 |     setIsSendingOtp(true)
647 |     try {
648 |       const { error } = await supabase.auth.signInWithOtp({
649 |         email: profile.email,
650 |         options: {
651 |           shouldCreateUser: false,
652 |         }
653 |       })
654 |       if (error) throw error
655 |       setOtpSent(true)
656 |       toast.success(`Verification code sent to ${profile.email}`)
657 |     } catch (e: any) {
658 |       toast.error(e.message || 'Failed to send OTP verification code.')
659 |     } finally {
660 |       setIsSendingOtp(false)
661 |     }
662 |   }
663 |
664 |   const handleVerifyOtp = async () => {
665 |     if (!profile?.email || otpInput.trim().length < 6 || otpInput.trim().length > 8) return
666 |     setIsVerifyingOtp(true)
667 |     try {
668 |       const { error } = await supabase.auth.verifyOtp({
669 |         email: profile.email,
670 |         token: otpInput.trim(),
671 |         type: 'email'
672 |       })
673 |       if (error) throw error
674 |
675 |       if (otpDialogType === 'enroll') {
676 |         // 1. Calculate capacity status
677 |         const { count: currentEnrolled } = await supabase
678 |           .from('enrollments')
679 |           .select('id', { count: 'exact', head: true })
680 |           .eq('course_id', courseId!)
681 |           .in('status', ['enrolled', 'in_progress', 'completed'])
682 |
683 |         const isFull = course?.max_trainees ? (currentEnrolled ?? 0) >= course.max_trainees : false
```

```
        const targetStatus = isFull ? 'waitlisted' : 'pending_approval'

        // 2. Check if already has an enrollment record (e.g. previously dropped or rejected)

        const { data: existingRecord } = await supabase
          .from('enrollments')
          .select('id')
          .eq('course_id', courseId!)
          .eq('user_id', profile.id)
          .maybeSingle()

        if (existingRecord) {
          const { error: updateErr } = await supabase
            .from('enrollments')
            .update({
              status: targetStatus,
              enrolled_at: new Date().toISOString(),
              progress_percent: 0,
            } as any)
            .eq('id', existingRecord.id)

          if (updateErr) {
            // Try RPC fallback

            await (supabase.rpc as any)('enroll_trainee', {
              p_course_id: courseId!,
              p_user_id: profile.id
            })
          }
        } else {
          const { error: insertErr } = await supabase
            .from('enrollments')
            .insert({
              course_id: courseId!,
              user_id: profile.id,
              status: targetStatus,
              enrolled_at: new Date().toISOString(),
              progress_percent: 0,
            } as any)

          if (insertErr) {
            // Try RPC fallback

            const { error: rpcErr } = await (supabase.rpc as any)('enroll_trainee', {
              p_course_id: courseId!,
              p_user_id: profile.id
            })
            if (rpcErr) throw insertErr
          }
        }

        // 3. Notify Trainer of Enrollment Request

        if (course?.trainer_id) {
          try {
            await supabase.from('notifications').insert({
              user_id: course.trainer_id,
              type: `enrollment_request:${course.id}`,
              title: isFull ? 'New Course Waitlist Request' : 'New Enrollment Approval Request',
              message: `${profile.full_name || profile.email} has requested to enroll in "${course.title}".`,
            } as any)
          } catch (notifErr) {
            console.warn('Could not notify trainer:', notifErr)
          }
        }

        queryClient.invalidateQueries({ queryKey: ['enrollment', courseId, profile?.id] })
        queryClient.invalidateQueries({ queryKey: ['enrollments-count', courseId] })
        queryClient.invalidateQueries({ queryKey: ['trainee-dashboard-enrollments', profile?.id] })

        if (isFull) {
          toast.success('Course is currently full. You have been added to the Waitlist!')
        } else {
          toast.success('Enrollment request submitted! Awaiting trainer approval.')
        }

        setOtpDialogType(null)
        setOtpSent(false)
        setOtpInput('')
```

```
759 |       } else if (otpDialogType === 'drop') {
760 |         dropMutation.mutate()
761 |       }
762 |     } catch (e: any) {
763 |       toast.error(e.message || 'Invalid or expired OTP code.')
764 |     } finally {
765 |       setIsVerifyingOtp(false)
766 |     }
767 |   }
768 |
769 |   const handleCloseOtpDialog = () => {
770 |     setOtpDialogType(null)
771 |     setOtpSent(false)
772 |     setOtpInput('')
773 |   }
774 |
775 |   const gradeBreakdown = calculateCourseGradeBreakdown({
776 |     moduleProgressPercent: enrollment?.progress_percent ?? (completedModules.length > 0 && course?.modules?.length ? Math.rou
nd((completedModules.length / course.modules.length) * 100) : 0),
777 |     courseAssessments: (course?.assessments || []) as any,
778 |     traineeAttempts,
779 |     passingScore: course?.passing_score ?? 50,
780 |   })
781 |
782 |   const [downloadingCert, setDownloadingCert] = useState(false)
783 |
784 |   const handleDownloadCertificate = async () => {
785 |     if (!profile || !course) return
786 |     setDownloadingCert(true)
787 |     try {
788 |       const traineeName = profile.full_name || user?.email?.split('@')[0] || 'Trainee'
789 |       const percentage = `${gradeBreakdown.totalScore}%`
790 |
791 |       const { blob, fileName } = await generateTraineeCertificate(
792 |         (course as any).certificate_template_url,
793 |         {
794 |           traineeName,
795 |           traineeEmail: profile.email || user?.email,
796 |           traineeId: user!.id,
797 |           courseId: course.id,
798 |           courseTitle: course.title,
799 |           trainerName: course.trainer?.full_name || 'Lead Trainer',
800 |           percentage,
801 |           completedAt: new Date().toISOString(),
802 |         }
803 |       )
804 |
805 |       triggerFileDownload(blob, fileName)
806 |       toast.success('Certificate downloaded successfully! 🎉')
807 |     } catch (err: any) {
808 |       console.error('Certificate generation error:', err)
809 |       toast.error(err.message || 'Failed to download certificate')
810 |     } finally {
811 |       setDownloadingCert(false)
812 |     }
813 |   }
814 |
815 |   const isLoading = isCourseLoading || isEnrollmentLoading
816 |
817 |   const seatLimit = course?.seat_limit ?? 50
818 |   const waitlistLimit = course?.waitlist_limit ?? 10
819 |   const activeCount = enrollmentCounts?.active ?? 0
820 |   const waitlistedCount = enrollmentCounts?.waitlisted ?? 0
821 |   const myWaitlistPosition = enrollmentCounts?.myWaitlistPosition ?? 0
822 |
823 |   const isSeatFull = activeCount >= seatLimit
824 |   const isWaitlistFull = waitlistedCount >= waitlistLimit
825 |   const isFull = isSeatFull && isWaitlistFull
826 |
827 |   const spotsLeft = !isSeatFull
828 |     ? Math.max(0, seatLimit - activeCount)
829 |     : Math.max(0, waitlistLimit - waitlistedCount)
830 |   const spotsLabel = !isSeatFull ? 'seats' : 'waitlist spots'
831 |
832 |   const isApprovedTrainee = Boolean(enrollment && (['enrolled', 'in_progress', 'completed'] as string[]).includes(enrollment.
status))
833 |
834 |   const canDrop = (() => {
835 |     if (!course?.start_date) return true
```

```
    const start = new Date(course.start_date).getTime()
    const now = new Date().getTime()
    const diffDays = (start - now) / (1000 * 3600 * 24)
    return diffDays >= 10
  })()

  const sessionFlowSteps = parseSessionFlowText(course?.session_flow_text || '')
  const totalMaterials = course?.materials?.length || 0

  const sessionTypeColors: Record<string, string> = {
    recorded: 'bg-blue-50 text-blue-700 border-blue-200',
    live: 'bg-cyan-50 text-cyan-700 border-cyan-200',
    in_person: 'bg-emerald-50 text-emerald-700 border-emerald-200',
    hybrid: 'bg-purple-50 text-purple-700 border-purple-200',
  }

  const objectives: string[] = Array.isArray(course?.learning_objectives)
    ? course.learning_objectives
    : []

  return (
    <ErrorBoundary>
      <DashboardShell
        title={course?.title || 'Course Details'}
        icon={BookOpen}
        navLinks={[
          { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
          { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
          { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
          { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
          { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
          { to: '/trainee/profile', label: 'Profile', icon: User },
        ]}
      >
        <div className="max-w-6xl space-y-6">
          <Link
            to="/trainee/courses"
            className="inline-flex items-center gap-2 text-sm text-slate-600 hover:text-cyan-700 transition-colors font-semib
old"
          >
            <ArrowLeft className="w-4 h-4" /> Back to Catalog
          </Link>

          {isLoading ? (
            <div className="flex items-center justify-center h-64">
              <Loader2 className="w-8 h-8 text-cyan-600 animate-spin" />
            </div>
          ) : !course ? (
            <div className="text-center p-12 bg-white border border-slate-200 rounded-3xl shadow-sm">
              <h3 className="text-xl font-bold text-slate-900">Course Not Found</h3>
              <p className="text-slate-500 mt-2 text-sm">This course does not exist or has been removed.</p>
            </div>
          ) : (
            <motion.div variants={fadeUp} initial="hidden" animate="visible" className="space-y-6">

              {/* Hero Header (Midnight Dark Aesthetic) */}
              <div className="bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white rounded-3xl overflow-hidd
en shadow-xl border border-cyan-500/30 relative">
                <div className="absolute top-0 right-0 w-96 h-96 bg-gradient-to-br from-cyan-500/25 to-blue-500/15 rounded-fu
ll blur-3xl pointer-events-none" />
                <div className="absolute bottom-0 left-0 w-64 h-64 bg-sky-600/20 rounded-full blur-2xl pointer-events-none" /
>

                <div className="relative z-10 p-8 md:p-10">
                  {/* Badges */}
                  <div className="flex flex-wrap items-center gap-2 mb-5">
                    <span className="bg-white/10 backdrop-blur-md px-3 py-1 rounded-full text-xs font-bold text-amber-300 upp
ercase tracking-wider border border-white/15 capitalize">
                      {course.course_type} Program
                    </span>
                    <span className={`px-3 py-1 rounded-full text-xs font-bold uppercase tracking-wider border ${
                      course.status === 'published'
                        ? 'bg-emerald-500/20 text-emerald-300 border-emerald-500/30'
                        : 'bg-amber-500/20 text-amber-300 border-amber-500/30'
                    }`}>
                      {course.status.replace('_', ' ')}
                    </span>
                    {enrollment && enrollment.status !== 'pending_approval' && enrollment.status !== 'rejected' && (
                      <span className="bg-emerald-500/20 text-emerald-300 border border-emerald-500/30 px-3 py-1 rounded-full
```

```
text-xs font-bold uppercase tracking-wider flex items-center gap-1">
 910 |                         <CheckCircle2 className="w-3 h-3" /> Enrolled
 911 |                       </span>
 912 |                     )}
 913 |                     {enrollment?.status === 'pending_approval' && (
 914 |                       <span className="bg-amber-500/20 text-amber-300 border border-amber-500/30 px-3 py-1 rounded-full text-
xs font-bold uppercase tracking-wider flex items-center gap-1">
 915 |                         <Clock className="w-3 h-3" /> Pending Approval
 916 |                       </span>
 917 |                     )}
 918 |                     {(enrollment as any)?.status === 'waitlisted' ? (
 919 |                       <span className="bg-amber-500/20 text-amber-300 border border-amber-500/30 px-3 py-1 rounded-full text-
xs font-bold uppercase tracking-wider flex items-center gap-1">
 920 |                         <Clock className="w-3 h-3" /> Waitlisted
 921 |                       </span>
 922 |                     ) : null}
 923 |                     {enrollment?.status === 'rejected' && (
 924 |                       <span className="bg-rose-500/20 text-rose-300 border border-rose-500/30 px-3 py-1 rounded-full text-xs
font-bold uppercase tracking-wider flex items-center gap-1">
 925 |                         <XCircle className="w-3 h-3" /> Rejected
 926 |                       </span>
 927 |                     )}
 928 |                   </div>
 929 |
 930 |                   <div className="flex flex-wrap items-center gap-3 mb-4">
 931 |                     <h1 className="text-3xl md:text-4xl font-black text-white leading-tight">
 932 |                       {course.title}
 933 |                     </h1>
 934 |                     <button
 935 |                       onClick={() => {
 936 |                         setAboutActiveTab('about')
 937 |                         setAboutModalOpen(true)
 938 |                       }}
 939 |                       title="About Course"
 940 |                       className="inline-flex items-center gap-1.5 px-3.5 py-1.5 rounded-full bg-cyan-500/20 hover:bg-cyan-50
0/35 text-cyan-200 border border-cyan-400/40 text-xs font-bold transition-all shadow-xs backdrop-blur-md cursor-pointer group shrink
-0"
 941 |                     >
 942 |                       <Info className="w-4 h-4 text-cyan-400 group-hover:scale-110 transition-transform" />
 943 |                       <span>About</span>
 944 |                     </button>
 945 |                   </div>
 946 |
 947 |                   {/* Meta chips */}
 948 |                   <div className="flex flex-wrap items-center gap-x-5 gap-y-2 text-sm text-slate-300 mb-6 font-medium">
 949 |                     <div className="flex items-center gap-2">
 950 |                       <User className="w-4 h-4 text-cyan-400 shrink-0" />
 951 |                       <span>{course.trainer?.full_name || 'Assigned Instructor'}</span>
 952 |                     </div>
 953 |                     {course.duration_minutes && (
 954 |                       <div className="flex items-center gap-2">
 955 |                         <Clock className="w-4 h-4 text-amber-400 shrink-0" />
 956 |                         <span>
 957 |                           {course.duration_minutes >= 60
 958 |                             ? `${Math.floor(course.duration_minutes / 60)}h${course.duration_minutes % 60 ? ` ${course.durati
on_minutes % 60}m` : ''}`
 959 |                             : `${course.duration_minutes} min`}
 960 |                         </span>
 961 |                       </div>
 962 |                     )}
 963 |                     <div className="flex items-center gap-2">
 964 |                       <Layers className="w-4 h-4 text-sky-400 shrink-0" />
 965 |                       <span>{course.sessions?.length || 0} Sessions</span>
 966 |                     </div>
 967 |                   </div>
 968 |
 969 |                   {/* Progress bar for enrolled users */}
 970 |                   {enrollment && (
 971 |                     <div className="mb-6 max-w-md space-y-4">
 972 |                       <div>
 973 |                         <div className="flex items-center justify-between text-xs font-semibold text-slate-300 mb-2">
 974 |                           <span>Your Progress</span>
 975 |                           <span className="text-white font-bold">{enrollment.progress_percent ?? 0}%</span>
 976 |                         </div>
 977 |                         <div className="h-2.5 w-full bg-white/10 rounded-full overflow-hidden border border-white/20">
 978 |                           <motion.div
 979 |                             initial={{ width: 0 }}
 980 |                             animate={{ width: `${enrollment.progress_percent ?? 0}%` }}
 981 |                             transition={{ duration: 0.9, ease: 'easeOut', delay: 0.3 }}
```

```
                            className={`h-full rounded-full ${
                              enrollment.progress_percent === 100
                                ? 'bg-gradient-to-r from-emerald-400 to-teal-500'
                                : 'bg-gradient-to-r from-cyan-400 via-sky-400 to-blue-500'
                            }`}
                          />
                        </div>
                        <p className="text-xs text-slate-400 mt-1.5 capitalize">
                          Status: <span className="text-slate-200 font-semibold">{enrollment?.status?.replace('_', ' ') || 'U
nknown'}</span>
                        </p>
                      </div>

                      {/* Top Overall Attendance Bar */}
                      <div>
                        <div className="flex items-center justify-between text-xs font-semibold text-slate-300 mb-2">
                          <span className="flex items-center gap-1.5"><UserCheck className="w-3.5 h-3.5 text-emerald-400" />
Overall Attendance</span>
                          <span className="text-white font-bold">{attendanceAggregates.percentage}%</span>
                        </div>
                        <div className="h-2.5 w-full bg-white/10 rounded-full overflow-hidden border border-white/20">
                          <motion.div
                            initial={{ width: 0 }}
                            animate={{ width: `${attendanceAggregates.percentage}%` }}
                            transition={{ duration: 0.9, ease: 'easeOut', delay: 0.5 }}
                            className={`h-full rounded-full ${
                              attendanceAggregates.percentage >= 80 ? 'bg-gradient-to-r from-emerald-400 to-emerald-500' :
                              attendanceAggregates.percentage >= 50 ? 'bg-gradient-to-r from-amber-400 to-amber-500' : 'bg-gr
adient-to-r from-rose-400 to-rose-500'
                            }`}
                          />
                        </div>
                        <p className="text-[10px] text-slate-400 mt-1.5">
                          <span className="text-emerald-300 font-semibold">{attendanceAggregates.presentCount} Present</span>
• <span className="text-rose-300 font-semibol">{attendanceAggregates.absentCount} Absent</span>
                        </p>
                      </div>
                    </div>
                  )}

                  {/* CTA */}
                  <div className="flex flex-wrap items-center gap-3">
                    {enrollment && gradeBreakdown.isCompleted ? (
                      <div className="flex flex-wrap items-center gap-3">
                        <Button
                          onClick={handleDownloadCertificate}
                          disabled={downloadingCert}
                          className="bg-gradient-to-r from-emerald-500 via-teal-500 to-cyan-600 hover:from-emerald-600 hover:
to-cyan-700 text-white font-bold rounded-2xl px-6 py-3 shadow-lg shadow-emerald-500/30 hover:scale-105 active:scale-95 transition-al
l flex items-center gap-2"
                        >
                          {downloadingCert ? <Loader2 className="w-4 h-4 animate-spin" /> : <Award className="w-4 h-4 text-am
ber-300" />}
                          {downloadingCert ? 'Generating...' : 'Download Certificate'}
                        </Button>
                        <Button
                          onClick={() => window.open(`/trainee/courses/${courseId}/learn`, '_blank')}
                          className="bg-white/10 hover:bg-white/20 text-white border border-white/25 backdrop-blur-md font-bo
ld rounded-2xl px-6 py-3 flex items-center gap-2 hover:scale-105 active:scale-95 transition-all shadow-sm"
                        >
                          <PlayCircle className="w-4 h-4 text-cyan-300" />
                          <span>Review Modules (New Tab)</span>
                          <ExternalLink className="w-3.5 h-3.5 opacity-80" />
                        </Button>
                      </div>
                    ) : isApprovedTrainee ? (
                      <div className="flex flex-wrap items-center gap-3">
                        <Button
                          onClick={handleDownloadCertificate}
                          disabled={downloadingCert}
                          className="bg-gradient-to-r from-emerald-500 via-teal-500 to-cyan-600 hover:from-emerald-600 hover:
to-cyan-700 text-white font-bold rounded-2xl px-6 py-4 shadow-lg shadow-emerald-500/30 hover:scale-105 active:scale-95 transition-al
l flex items-center gap-2 cursor-pointer"
                        >
                          {downloadingCert ? <Loader2 className="w-4 h-4 animate-spin" /> : <Award className="w-4 h-4 text-am
ber-300" />}
                          {downloadingCert ? 'Generating...' : 'Download Certificate'}
                        </Button>

```

```
1050 |                         {gradeBreakdown.hasFinalAssessment && !gradeBreakdown.finalAssessmentCompleted && gradeBreakdown.isFi
nalUnlocked ? (
1051 |                           <Button
1052 |                             onClick={() => {
1053 |                               if (gradeBreakdown.finalAssessmentId) {
1054 |                                 window.location.href = `/trainee/courses/${courseId}/assessments/${gradeBreakdown.finalAssess
mentId}`
1055 |                               }
1056 |                             }}
1057 |                             className="bg-gradient-to-r from-purple-600 via-indigo-600 to-cyan-600 hover:from-purple-700 hove
r:to-cyan-700 text-white font-extrabold text-sm sm:text-base rounded-2xl px-8 py-4 shadow-xl shadow-purple-500/30 hover:scale-105 ac
tive:scale-95 transition-all flex items-center gap-2 animate-pulse"
1058 |                           >
1059 |                             <FileCheck className="w-5 h-5 text-amber-300" />
1060 |                             <span>Take Final Exam (50% Grade)</span>
1061 |                           </Button>
1062 |                         ) : null}
1063 |
1064 |                         <Button
1065 |                           onClick={() => window.open(`/trainee/courses/${courseId}/learn`, '_blank')}
1066 |                           className="bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600 hover:to-blue
-700 text-white font-extrabold text-sm sm:text-base rounded-2xl px-8 py-4 shadow-xl shadow-cyan-500/30 hover:scale-105 active:scale-
95 transition-all flex items-center gap-2"
1067 |                         >
1068 |                           <PlayCircle className="w-5 h-5" />
1069 |                           <span>{enrollment?.progress_percent && enrollment.progress_percent >= 100 ? 'Review Modules' : enro
llment?.progress_percent && enrollment.progress_percent > 0 ? 'Resume Learning' : 'Start Learning'}</span>
1070 |                           <ExternalLink className="w-4 h-4 ml-1 opacity-80" />
1071 |                         </Button>
1072 |
1073 |                         <div className="space-y-1">
1074 |                           <Button
1075 |                             onClick={() => handleInitiateOtp('drop')}
1076 |                             disabled={!canDrop || dropMutation.isPending}
1077 |                             className="bg-white/10 hover:bg-rose-500/20 text-white border border-white/20 hover:border-rose-4
00/40 backdrop-blur-md font-bold rounded-2xl px-6 py-3 transition-all"
1078 |                           >
1079 |                             {dropMutation.isPending ? <Loader2 className="w-4 h-4 animate-spin mr-2" /> : <XCircle className
="w-4 h-4 mr-2 text-rose-400" />}
1080 |                             Drop Course
1081 |                           </Button>
1082 |                           {!canDrop && course?.start_date && (
1083 |                             <p className="text-[10px] text-rose-300 font-medium">Cannot drop within 10 days of start.</p>
1084 |                           )}
1085 |                         </div>
1086 |                       </div>
1087 |                     ) : enrollment && enrollment.status === 'pending_approval' ? (
1088 |                       <Button disabled className="bg-amber-500/20 text-amber-300 border border-amber-500/30 font-bold rounded
-2xl px-6 py-3 cursor-not-allowed flex items-center gap-2">
1089 |                         <Clock className="w-4 h-4" /> Approval Pending
1090 |                       </Button>
1091 |                     ) : enrollment && enrollment.status === 'rejected' ? (
1092 |                       <Button disabled className="bg-rose-500/20 text-rose-300 border border-rose-500/30 font-bold rounded-2x
l px-6 py-3 cursor-not-allowed flex items-center gap-2">
1093 |                         <XCircle className="w-4 h-4" /> Enrollment Rejected
1094 |                       </Button>
1095 |                     ) : enrollment && (enrollment.status as string) === 'waitlisted' ? (
1096 |                       <div className="flex flex-wrap items-center gap-3">
1097 |                         <Button disabled className="bg-amber-500/20 text-amber-300 border border-amber-500/30 font-bold round
ed-2xl px-6 py-3 cursor-not-allowed flex items-center gap-2">
1098 |                           <Clock className="w-4 h-4" /> Waitlisted {myWaitlistPosition > 0 && `(WL-${myWaitlistPosition})`}
1099 |                         </Button>
1100 |                         <Button
1101 |                           onClick={() => handleInitiateOtp('drop')}
1102 |                           disabled={dropMutation.isPending}
1103 |                           className="bg-white/10 hover:bg-rose-500/20 text-white border border-white/20 hover:border-rose-40
0/40 backdrop-blur-md font-bold rounded-2xl px-6 py-3 transition-all"
1104 |                         >
1105 |                           {dropMutation.isPending ? <Loader2 className="w-4 h-4 animate-spin mr-2" /> : <XCircle className="w
-4 h-4 mr-2 text-rose-400" />}
1106 |                           Quit Waitlist
1107 |                         </Button>
1108 |                       </div>
1109 |                     ) : isFull ? (
1110 |                       <div className="space-y-1">
1111 |                         <Button disabled className="bg-white/10 text-white/50 border border-white/20 font-bold rounded-2xl px
-6 py-3 cursor-not-allowed">
1112 |                           Course & Waitlist Full
1113 |                         </Button>
```

```
                        <p className="text-xs text-rose-300 font-medium">Capacity limit of {seatLimit + waitlistLimit} reached.</p>
                      </div>
                    ) : (
                      <Button
                        onClick={() => handleInitiateOtp('enroll')}
                        disabled={isSendingOtp || otpDialogType === 'enroll'}
                        className="bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600 hover:to-blue-700 text-white font-extrabold text-base rounded-2xl px-8 py-5 shadow-xl shadow-cyan-500/30 hover:scale-105 active:scale-95 transition-all flex items-center gap-2"
                      >
                        {isSendingOtp && otpDialogType === 'enroll'
                          ? <Loader2 className="w-5 h-5 animate-spin" />
                          : <GraduationCap className="w-5 h-5" />}
                        {isFull ? 'Join Waitlist' : 'Enroll in Course'}
                      </Button>
                    )}
                  </div>
                </div>
              </div>

              {/* ── Main 2-col grid ──────────────────────────────── */}
              <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">

                {/* Left column (2/3) */}
                <div className="lg:col-span-2 space-y-5">

                  {/* 1. Preview Mode Banner (if not approved) */}
                  {!isApprovedTrainee && (
                    <div className="bg-amber-50 border border-amber-200 rounded-3xl p-5 sm:p-6 flex items-start gap-4 shadow-xs">

                      <div className="p-2.5 bg-amber-100 rounded-2xl shrink-0">
                        <Lock className="w-5 h-5 text-amber-600" />
                      </div>
                      <div>
                        <h4 className="text-sm font-bold text-amber-900 mb-1">
                          {enrollment?.status === 'pending_approval'
                            ? 'Enrollment Pending Approval'
                            : enrollment?.status === 'rejected'
                            ? 'Enrollment Not Approved'
                            : 'Preview Mode'}
                        </h4>
                        <p className="text-xs sm:text-sm text-amber-800/90 leading-relaxed font-normal">
                          {enrollment?.status === 'pending_approval'
                            ? "Your enrollment is currently pending approval by the course trainer. Interactive learning modules, videos, notes, and quizzes will unlock automatically once approved."
                            : enrollment?.status === 'rejected'
                            ? "Your enrollment request was not approved. Please contact your trainer for further assistance."
                            : "You are currently viewing this course in preview mode. Enroll and receive trainer approval to unlock the interactive learning experience and assessments."}
                        </p>
                      </div>
                    </div>
                  )}

                  {/* 2. Resume Learning Action Banner / Completed Banner (for Approved Trainees) */}
                  {isApprovedTrainee && (() => {
                    const isCourseFullyCompleted = Boolean(gradeBreakdown.isCompleted)
                    const isFinalPending = Boolean(gradeBreakdown.hasFinalAssessment && !gradeBreakdown.finalAssessmentCompleted && gradeBreakdown.isFinalUnlocked)

                    return (
                      <div className={`border rounded-3xl p-6 shadow-xl relative overflow-hidden transition-all ${
                        isCourseFullyCompleted
                          ? 'bg-gradient-to-r from-[#061c14] via-[#08281b] to-[#04120c] border-emerald-500/40 shadow-emerald-950/30'
                          : isFinalPending
                          ? 'bg-gradient-to-r from-[#17092c] via-[#120724] to-[#080314] border-purple-500/40 shadow-purple-950/30'
                          : 'bg-gradient-to-r from-[#0c162c] via-[#091224] to-[#040814] border-cyan-500/30'
                      }`}>
                        <div className={`absolute top-0 right-0 w-64 h-64 rounded-full blur-3xl pointer-events-none ${
                          isCourseFullyCompleted ? 'bg-emerald-500/15' : isFinalPending ? 'bg-purple-500/20' : 'bg-cyan-500/15'
                        }`} />
                        <div className="relative z-10 flex flex-col sm:flex-row sm:items-center justify-between gap-5">
                          <div className="space-y-2">
                            <div className="flex items-center gap-2">
                              {isCourseFullyCompleted ? (
```

```
                                <span className="w-2.5 h-2.5 rounded-full bg-emerald-400" />
                              ) : isFinalPending ? (
                                <span className="w-2.5 h-2.5 rounded-full bg-purple-400 animate-pulse" />
                              ) : (
                                <span className="w-2.5 h-2.5 rounded-full bg-cyan-400 animate-ping" />
                              )}
                              <span className={`text-xs font-bold uppercase tracking-wider ${
                                isCourseFullyCompleted ? 'text-emerald-400' : isFinalPending ? 'text-purple-300' : 'text-cyan
-400'
                              }`}>
                                {isCourseFullyCompleted ? 'Course Completed 🎉' : isFinalPending ? 'Final Exam Required 📝'
: 'Active Learning Session'}
                              </span>
                            </div>
                            <h3 className="text-lg font-bold text-white">
                              {isCourseFullyCompleted
                                ? "Course Completed! You've mastered all modules & evaluations"
                                : isFinalPending
                                ? "Modules Completed! Take the Final Exam to earn your certificate"
                                : 'Ready to learn? Resume your course modules'}
                            </h3>
                            <p className="text-xs text-slate-300">
                              {isCourseFullyCompleted
                                ? `All ${course.modules?.length || completedModules.length} modules completed • Grade: ${grad
eBreakdown.totalScore}% (Passed)`
                                : isFinalPending
                                ? `All ${course.modules?.length || completedModules.length} modules completed (25/25 pts) • F
inal Exam (50% weight) ready to take`
                                : `${completedModules.length} of ${course.modules?.length || 0} modules completed • ${enrollm
ent?.progress_percent ?? 0}% overall progress`}
                            </p>
                          </div>
                          <div className="flex flex-wrap items-center gap-3 shrink-0">
                            {isFinalPending && gradeBreakdown.finalAssessmentId ? (
                              <Button
                                onClick={() => {
                                  window.location.href = `/trainee/courses/${courseId}/assessments/${gradeBreakdown.finalAsse
ssmentId}`
                                }}
                                className="bg-gradient-to-r from-purple-600 via-indigo-600 to-cyan-600 hover:from-purple-700
hover:to-cyan-700 text-white font-extrabold text-sm rounded-2xl px-6 py-3.5 shadow-lg shadow-purple-500/25 hover:scale-105 active:sc
ale-95 transition-all flex items-center gap-2"
                              >
                                <FileCheck className="w-4 h-4 text-amber-300" />
                                <span>Take Final Exam</span>
                              </Button>
                            ) : null}
                            <Button
                              onClick={() => window.open(`/trainee/courses/${courseId}/learn`, '_blank')}
                              className={`text-white font-extrabold text-sm rounded-2xl px-6 py-3.5 shadow-lg hover:scale-105
active:scale-95 transition-all flex items-center gap-2 shrink-0 ${
                                isCourseFullyCompleted
                                  ? 'bg-gradient-to-r from-emerald-500 via-teal-500 to-cyan-600 hover:from-emerald-600 hover:
to-cyan-700 shadow-emerald-500/25'
                                  : isFinalPending
                                  ? 'bg-white/10 hover:bg-white/20 border border-white/20 backdrop-blur-md'
                                  : 'bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600 hover:to-blue
-700 shadow-cyan-500/30'
                              }`}
                            >
                              {isCourseFullyCompleted || isFinalPending ? <BookOpen className="w-4 h-4" /> : <PlayCircle clas
sName="w-4 h-4" />}
                              <span>{isCourseFullyCompleted || isFinalPending ? 'Review Modules' : 'Resume Learning'}</span>
                              <ExternalLink className="w-3.5 h-3.5 opacity-80" />
                            </Button>
                          </div>
                        </div>
                      </div>
                    )
                  })()}

                  {/* Curriculum Modules Sequence */}
                  {course.modules && Array.isArray(course.modules) && course.modules.length > 0 && (
                    <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm space-y-4">
                      <div className="flex items-center justify-between pb-3 border-b border-slate-100">
                        <h3 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                          <BookMarked className="w-4 h-4 text-cyan-600" /> Curriculum Modules ({course.modules.length})
                        </h3>
                        <span className="text-[11px] font-semibold text-cyan-700 bg-cyan-50 px-2.5 py-1 rounded-full border b
```

```
order-cyan-200">
1250 |                           Passing Gate: ≥80% on Module Quizzes
1251 |                         </span>
1252 |                       </div>
1253 |
1254 |                       <div className="space-y-2.5">
1255 |                         {course.modules.map((mod: any, mIdx: number) => {
1256 |                           const isCompleted = completedModules.includes(mod.id)
1257 |                           const isUnlocked = isApprovedTrainee && (mIdx === 0 || completedModules.includes(course.modules[mId
x - 1]?.id))
1258 |                           const rawItems = mod.items || mod.content_items || []
1259 |                           const quizCount = rawItems.filter((i: any) => i.type === 'quiz').length + (mod.quiz_questions?.leng
th || 0)
1260 |
1261 |                           return (
1262 |                             <div
1263 |                               key={mod.id || mIdx}
1264 |                               className={`flex items-center justify-between p-4 rounded-2xl border transition-all ${
1265 |                                 isCompleted
1266 |                                   ? 'bg-emerald-50/40 border-emerald-200 shadow-xs'
1267 |                                   : isUnlocked
1268 |                                     ? 'bg-cyan-50/20 border-cyan-200/80 shadow-xs'
1269 |                                     : 'bg-slate-50/50 border-slate-200/80 opacity-85'
1270 |                               }`}
1271 |                             >
1272 |                               <div className="flex items-center gap-3.5 min-w-0">
1273 |                                 <div className={`w-8 h-8 rounded-xl flex items-center justify-center font-bold text-xs shrink
-0 ${
1274 |                                   isCompleted
1275 |                                     ? 'bg-emerald-500 text-white'
1276 |                                     : isUnlocked
1277 |                                       ? 'bg-cyan-600 text-white'
1278 |                                       : 'bg-slate-200 text-slate-500'
1279 |                                 }`}>
1280 |                                   {isCompleted ? <Check className="w-4 h-4 stroke-[3]" /> : mIdx + 1}
1281 |                                 </div>
1282 |                                 <div className="min-w-0">
1283 |                                   <div className="flex items-center gap-2">
1284 |                                     <p className="text-sm font-bold text-slate-900 truncate">
1285 |                                       {mod.title || `Module ${mIdx + 1}`}
1286 |                                     </p>
1287 |                                     {isCompleted ? (
1288 |                                       <Badge className="bg-emerald-100 text-emerald-800 border-emerald-200 text-[10px] font-b
old">
1289 |
Completed
1290 |                                       </Badge>
1291 |                                     ) : isUnlocked ? (
1292 |                                       <Badge className="bg-cyan-100 text-cyan-800 border-cyan-200 text-[10px] font-bold">
1293 |                                         Unlocked
1294 |                                       </Badge>
1295 |                                     ) : (
1296 |                                       <Badge variant="outline" className="text-slate-400 border-slate-200 text-[10px] flex it
ems-center gap-1">
1297 |
<Lock className="w-2.5 h-2.5" /> Locked
1298 |                                       </Badge>
1299 |                                     )}
1300 |                                   </div>
1301 |                                   {mod.description && (
1302 |                                     <p className="text-xs text-slate-500 truncate max-w-xl mt-0.5">{mod.description}</p>
1303 |                                   )}
1304 |                                 </div>
1305 |                               </div>
1306 |
1307 |                               <div className="flex items-center gap-3 shrink-0 ml-4">
1308 |                                 <div className="text-right hidden sm:block">
1309 |                                   <span className="text-[11px] text-slate-500 font-medium">
1310 |                                     {rawItems.length} item{rawItems.length !== 1 ? 's' : ''}
1311 |                                     {quizCount > 0 ? ` • ${quizCount} Quiz` : ''}
1312 |                                   </span>
1313 |                                 </div>
1314 |                                 {isApprovedTrainee ? (
1315 |                                   <Button
1316 |                                     size="sm"
1317 |                                     variant="ghost"
1318 |                                     onClick={() => window.open(`/trainee/courses/${courseId}/learn`, '_blank')}
1319 |                                     className="h-7 px-2.5 rounded-lg text-xs font-semibold text-cyan-700 hover:text-cyan-800
hover:bg-cyan-100/60"
1320 |                                   >
1321 |                                     Launch Player <ExternalLink className="w-3 h-3 ml-1" />
```

```
                                    </Button>
                                  ) : (
                                    <span className="text-xs text-slate-400 font-medium flex items-center gap-1">
                                      <Lock className="w-3 h-3" /> Locked
                                    </span>
                                  )}
                                </div>
                              </div>
                            )
                          })}
                        </div>
                      </div>
                    )}

                    {/* 2. Outcomes of Learning & Skills Developed */}
                    {(objectives.length > 0 || (courseSkills && courseSkills.length > 0)) && (
                      <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm space-y-6">
                        {/* What You'll Learn */}
                        {objectives.length > 0 && (
                          <div>
                            <h2 className="text-sm font-bold text-slate-900 mb-3.5 flex items-center gap-2">
                              <Target className="w-4 h-4 text-emerald-600" /> Outcomes of Learning (What You'll Master)
                            </h2>
                            <ul className="grid grid-cols-1 sm:grid-cols-2 gap-2.5">
                              {objectives.map((obj: string, i: number) => (
                                <li key={i} className="flex items-start gap-2.5 p-3 rounded-2xl bg-emerald-50/60 border border-
emerald-100/80">
                                  <CheckCircle2 className="w-4 h-4 text-emerald-600 shrink-0 mt-0.5" />
                                  <span className="text-xs text-emerald-950 leading-relaxed font-medium">{obj}</span>
                                </li>
                              ))}
                            </ul>
                          </div>
                        )}

                        {/* Skills Developed */}
                        {courseSkills && courseSkills.length > 0 && (
                          <div className={objectives.length > 0 ? 'pt-4 border-t border-slate-100' : ''}>
                            <h3 className="text-xs font-bold text-slate-800 uppercase tracking-wider mb-3 flex items-center gap
-1.5">
                              <Sparkles className="w-3.5 h-3.5 text-cyan-600" /> Skills You'll Gain
                            </h3>
                            <div className="flex flex-wrap gap-2">
                              {courseSkills.map((cs: any, idx: number) => {
                                const skillName = cs.skills?.name || cs.name || cs.skill_name || 'Skill'
                                return (
                                  <span
                                    key={cs.id || idx}
                                    className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-gradient-to-r from-cy
an-50 to-sky-50 text-cyan-800 border border-cyan-200 text-xs font-semibold shadow-xs"
                                  >
                                    <Award className="w-3.5 h-3.5 text-cyan-600" />
                                    {skillName}
                                  </span>
                                )
                              })}
                            </div>
                          </div>
                        )}
                      </div>
                    )}

                    {/* 3. Course Assessments & Grading Breakdown (25% Modules + 25% Assessments Avg + 50% Final Exam) */}
                    <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm space-y-6">
                      <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pb-4 border-b border-slat
e-100">
                        <div>
                          <div className="flex items-center gap-2">
                            <h2 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                              <Target className="w-4 h-4 text-cyan-600" /> Course Grading &amp;amp; Assessments
                            </h2>
                            <span className="text-[10px] font-extrabold px-2.5 py-0.5 rounded-full bg-cyan-50 text-cyan-700 bor
der border-cyan-200">
                              100% Total Grade
                            </span>
                          </div>
                          <p className="text-xs text-slate-500 mt-1">
                            Evaluated via: 25% Course Modules + 25% Assessments Average + 50% Final Exam.
                          </p>
```

```
                      </div>

                      <div className="flex items-center gap-2">
                        <div className="px-3 py-1.5 rounded-xl bg-slate-50 border border-slate-200 text-right">
                          <span className="text-[10px] font-bold text-slate-400 uppercase tracking-wider block">Your Total Sc
ore</span>
                          <span className="text-sm font-black text-cyan-700">
                            {gradeBreakdown.totalScore}% <span className="text-[10px] text-slate-400 font-semibold">(Pass: {g
radeBreakdown.passingScore}%)</span>
                          </span>
                        </div>
                      </div>
                    </div>

                    {/* 3 Pillars Scorecards */}
                    <div className="grid grid-cols-1 md:grid-cols-3 gap-3.5">
                      {/* Pillar 1: Modules Completion (25%) */}
                      <div className="p-4 rounded-2xl bg-cyan-50/40 border border-cyan-200/80 flex flex-col justify-between">
                        <div>
                          <div className="flex items-center justify-between mb-1.5">
                            <span className="text-[10px] font-extrabold uppercase tracking-wider text-cyan-700">1. Course Mod
ules</span>
                            <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-cyan-100 text-cyan-800">25% We
ight</span>
                          </div>
                          <p className="text-xs text-slate-600 font-medium">All video lessons and module checkpoints</p>
                        </div>

                        <div className="mt-4 pt-3 border-t border-cyan-200/60">
                          <div className="flex items-baseline justify-between mb-1">
                            <span className="text-xs font-semibold text-slate-500">Earned:</span>
                            <span className="text-base font-black text-cyan-700">{gradeBreakdown.moduleScore} <span className
="text-xs text-slate-400">/ 25 pts</span></span>
                          </div>
                          <div className="w-full bg-white rounded-full h-1.5 overflow-hidden border border-cyan-200">
                            <div className="bg-cyan-600 h-full rounded-full transition-all" style={{ width: `${gradeBreakdow
n.moduleProgressPercent}%` }} />
                          </div>
                          <p className="text-[10px] text-slate-400 mt-1 text-right font-medium">{gradeBreakdown.moduleProgres
sPercent}% completed</p>
                        </div>
                      </div>

                      {/* Pillar 2: Regular Assessments Average (25%) */}
                      <div className="p-4 rounded-2xl bg-amber-50/40 border border-amber-200/80 flex flex-col justify-betwee
n">
                        <div>
                          <div className="flex items-center justify-between mb-1.5">
                            <span className="text-[10px] font-extrabold uppercase tracking-wider text-amber-800">2. Assessmen
ts Avg</span>
                            <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-amber-100 text-amber-900">25%
Weight</span>
                          </div>
                          <p className="text-xs text-slate-600 font-medium">Average score of all regular course tests</p>
                        </div>

                        <div className="mt-4 pt-3 border-t border-amber-200/60">
                          <div className="flex items-baseline justify-between mb-1">
                            <span className="text-xs font-semibold text-slate-500">Earned:</span>
                            <span className="text-base font-black text-amber-700">{gradeBreakdown.regularAssessmentScore} <sp
an className="text-xs text-slate-400">/ 25 pts</span></span>
                          </div>
                          <div className="w-full bg-white rounded-full h-1.5 overflow-hidden border border-amber-200">
                            <div className="bg-amber-500 h-full rounded-full transition-all" style={{ width: `${gradeBreakdow
n.regularAssessmentAveragePercent}%` }} />
                          </div>
                          <p className="text-[10px] text-slate-400 mt-1 text-right font-medium">
                            {gradeBreakdown.regularAssessmentsTotal > 0
                              ? `${gradeBreakdown.regularAssessmentAveragePercent}% avg (${gradeBreakdown.regularAssessmentsC
ompleted}/${gradeBreakdown.regularAssessmentsTotal} completed)`
                              : `${gradeBreakdown.moduleProgressPercent}% module aligned`}
                          </p>
                        </div>
                      </div>

                      {/* Pillar 3: Final Assessment (50%) */}
                      <div className={`p-4 rounded-2xl border flex flex-col justify-between ${
                        gradeBreakdown.finalAssessmentCompleted
                          ? 'bg-purple-50/40 border-purple-200/80'
```

```
                            : gradeBreakdown.isFinalUnlocked
                              ? 'bg-emerald-50/40 border-emerald-200/80'
                              : 'bg-slate-50/60 border-slate-200'
                          }`}>
                            <div>
                              <div className="flex items-center justify-between mb-1.5">
                                <span className="text-[10px] font-extrabold uppercase tracking-wider text-purple-800">3. Final Ex
am</span>
                                <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-purple-100 text-purple-900">5
0% Weight</span>
                              </div>
                              <p className="text-xs text-slate-600 font-medium">
                                {gradeBreakdown.isFinalUnlocked ? 'Unlocked after 100% modules complete' : 'Locked until modules
100%'}
                              </p>
                            </div>

                            <div className="mt-4 pt-3 border-t border-purple-200/60">
                              <div className="flex items-baseline justify-between mb-1">
                                <span className="text-xs font-semibold text-slate-500">Earned:</span>
                                <span className="text-base font-black text-purple-700">
                                  {gradeBreakdown.finalAssessmentWeightedScore} <span className="text-xs text-slate-400">/ 50 pts
</span>
                                </span>
                              </div>
                              <div className="w-full bg-white rounded-full h-1.5 overflow-hidden border border-purple-200">
                                <div
                                  className="bg-purple-600 h-full rounded-full transition-all"
                                  style={{ width: `${gradeBreakdown.finalAssessmentScorePercent ?? (gradeBreakdown.isFinalUnlocke
d ? 100 : 0)}%` }}
                                />
                              </div>
                              <p className="text-[10px] text-slate-400 mt-1 text-right font-medium">
                                {gradeBreakdown.finalAssessmentCompleted
                                  ? `${gradeBreakdown.finalAssessmentScorePercent}% score`
                                  : gradeBreakdown.isFinalUnlocked
                                    ? 'Ready to take!'
                                    : 'Locked (Modules Pending)'}
                              </p>
                            </div>
                          </div>
                        </div>

                        {/* Published Course Assessments List */}
                        <div className="space-y-3 pt-2">
                          <h3 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                            <FileCheck className="w-3.5 h-3.5 text-cyan-600" /> Course Assessments ({course.assessments?.length |
| 0})
                          </h3>

                          {(!course.assessments || course.assessments.length === 0) ? (
                            <div className="p-4 rounded-2xl bg-slate-50 border border-slate-100 text-center text-xs text-slate-50
0 font-medium">
                              No scheduled assessments configured for this course yet.
                            </div>
                          ) : (
                            <div className="space-y-2.5">
                              {course.assessments.map((a: any) => {
                                const isFinal = a.assessment_type === 'final'
                                const isPractice = a.assessment_type === 'mock' || a.assessment_type === 'daily'
                                const isUnlocked = !isFinal || gradeBreakdown.isFinalUnlocked
                                const attempt = traineeAttempts.find((at: any) => at.assessment_id === a.id)
                                const hasAttempt = Boolean(attempt)

                                return (
                                  <div
                                    key={a.id}
                                    className={`flex flex-col sm:flex-row sm:items-center justify-between p-4 rounded-2xl border
gap-3 transition-all ${
                                      hasAttempt
                                        ? 'bg-emerald-50/30 border-emerald-200'
                                        : !isUnlocked
                                          ? 'bg-slate-50/70 border-slate-200 opacity-90'
                                          : 'bg-white border-slate-200 hover:border-cyan-300 shadow-xs'
                                    }`}
                                  >
                                    <div className="min-w-0">
                                      <div className="flex flex-wrap items-center gap-2 mb-1">
                                        <h4 className="text-sm font-bold text-slate-900">{a.title}</h4>
```

```
                                    {isPractice ? (
                                      <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-slate-100 text-slate
-700 border border-slate-200">
                                        {a.assessment_type === 'mock' ? 'Mock Test' : 'Daily Test'} • Practice Only (0% Weigh
t)
                                      </span>
                                    ) : isFinal ? (
                                      <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-purple-50 text-purpl
e-700 border border-purple-200">
                                        Final Exam • 50% Grade Weight
                                      </span>
                                    ) : (
                                      <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-amber-50 text-amber-
800 border border-amber-200">
                                        Assessment Test • 25% Grade Weight (Averaged)
                                      </span>
                                    )}
                                  </div>

                                  <div className="flex flex-wrap items-center gap-3 text-xs text-slate-500 font-medium">
                                    {a.duration_minutes && <span>⏱ {a.duration_minutes} mins</span>}
                                    {a.scheduled_date && (
                                      <span>🗓 {new Date(a.scheduled_date).toLocaleDateString()}</span>
                                    )}
                                    {hasAttempt && (
                                      <span className="text-emerald-700 font-bold">
                                        ✓ Scored: {attempt.score ?? 0}%
                                      </span>
                                    )}
                                  </div>
                                </div>

                                <div className="shrink-0 flex items-center gap-2">
                                  {hasAttempt ? (
                                    <Link to={`/trainee/courses/${courseId}/assessments/${a.id}`}>
                                      <Button size="sm" variant="outline" className="rounded-xl text-xs font-bold border-emer
ald-300 text-emerald-800 hover:bg-emerald-50">
                                        View Result ({attempt.score ?? 0}%)
                                      </Button>
                                    </Link>
                                  ) : isUnlocked ? (
                                    <Link to={`/trainee/courses/${courseId}/assessments/${a.id}`}>
                                      <Button
                                        size="sm"
                                        className={`rounded-xl text-xs font-bold text-white shadow-sm ${
                                          isFinal
                                            ? 'bg-gradient-to-r from-purple-600 to-indigo-600 hover:from-purple-700 hover:to-
indigo-700'
                                            : isPractice
                                              ? 'bg-slate-700 hover:bg-slate-800'
                                              : 'bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue
-700'

                                        }`}
                                      >
                                        <PlayCircle className="w-3.5 h-3.5 mr-1" />
                                        {isPractice ? 'Start Practice' : isFinal ? 'Take Final Exam' : 'Start Assessment'}
                                      </Button>
                                    </Link>
                                  ) : (
                                    <Button
                                      size="sm"
                                      variant="ghost"
                                      disabled
                                      className="rounded-xl text-xs font-semibold text-slate-400 bg-slate-100 border border-s
late-200 cursor-not-allowed gap-1.5"
                                    >
                                      <Lock className="w-3 h-3" /> Locked (100% Modules Required)
                                    </Button>
                                  )}
                                </div>
                              </div>
                            )
                          })}
                        </div>
                      )}
                    </div>
                  </div>

                  {/* Course Sessions — Accordion */}
```

```
1604 |                   {course.sessions?.length > 0 && (
1605 |                     <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm">
1606 |                       <div className="flex items-center justify-between mb-5">
1607 |                         <h2 className="text-sm font-bold text-slate-900 flex items-center gap-2">
1608 |                           <BookCheck className="w-4 h-4 text-cyan-600" /> Course Sessions
1609 |                           <span className="text-xs font-bold px-2 py-0.5 rounded-full bg-cyan-50 text-cyan-700 border border-
cyan-200">
1610 |                             {course.sessions.length}
1611 |                           </span>
1612 |                         </h2>
1613 |                         {!enrollment && (
1614 |                           <div className="flex items-center gap-1.5 text-xs text-slate-500 font-medium">
1615 |                             <Lock className="w-3.5 h-3.5" /> Enroll to access materials
1616 |                           </div>
1617 |                         )}
1618 |                       </div>
1619 |
1620 |                       <div className="space-y-2.5">
1621 |                         {course.sessions.map((session: any, index: number) => {
1622 |                           const sessionMaterials = course.materials?.filter((m: any) => m.session_id === session.id) || []
1623 |                           const isOpen = openSessions.has(session.id)
1624 |
1625 |                           // Determine real-time session status

1626 |                           const now = Date.now()
1627 |                           const startTime = session.start_time ? new Date(session.start_time).getTime() : null
1628 |                           const endTime = session.end_time ? new Date(session.end_time).getTime() : (startTime ? startTime +
60 * 60 * 1000 : null)
1629 |
1630 |                           const isFinished = endTime ? endTime < now : (startTime ? startTime < now : false)
1631 |                           const isLiveNow = startTime && endTime ? (now >= startTime && now <= endTime) : false
1632 |
1633 |                           return (
1634 |                             <div
1635 |                               key={session.id}
1636 |                               className={`border rounded-2xl overflow-hidden transition-all duration-200 ${
1637 |                                 isLiveNow ? 'border-rose-300 ring-2 ring-rose-100 shadow-sm' : isOpen ? 'border-cyan-300 shad
ow-sm' : 'border-slate-200'
1638 |                               }`}
1639 |                             >
1640 |                               {/* Session header */}
1641 |                               <button
1642 |                                 onClick={() => toggleSession(session.id)}
1643 |                                 className={`w-full flex items-center gap-4 p-4 transition-colors text-left ${
1644 |                                   isLiveNow ? 'bg-rose-50/40 hover:bg-rose-50/60' : 'bg-slate-50/70 hover:bg-slate-50'
1645 |                                 }`}
1646 |                               >
1647 |                                 <div className={`w-8 h-8 rounded-xl flex items-center justify-center shrink-0 font-black text
-xs transition-all ${
1648 |                                   isLiveNow
1649 |                                     ? 'bg-rose-600 text-white shadow-sm'
1650 |                                     : isOpen
1651 |                                       ? 'bg-gradient-to-br from-cyan-600 to-blue-600 text-white shadow-xs'
1652 |                                       : 'bg-white border border-slate-200 text-cyan-700'
1653 |                                 }`}>
1654 |                                   {index + 1}
1655 |                                 </div>
1656 |
1657 |                                 <div className="flex-1 min-w-0">
1658 |                                   <div className="flex flex-wrap items-center gap-2 mb-0.5">
1659 |                                     <p className="text-sm font-bold text-slate-900">{session.title}</p>
1660 |                                     {isLiveNow ? (
1661 |                                       <span className="text-[10px] font-bold px-2 py-0.5 rounded-full border bg-rose-50 text-
rose-700 border-rose-200 flex items-center gap-1 shadow-xs animate-pulse">
1662 |                                         <span className="w-1.5 h-1.5 rounded-full bg-rose-600 animate-ping" /> Live Now
1663 |                                       </span>
1664 |                                     ) : isFinished ? (() => {
1665 |                                       const userId = user?.id || profile?.id
1666 |                                       const tKey = `trainee_attendance_${session.id}_${userId}`
1667 |                                       let attStatus = ''
1668 |                                       let attColor = ''
1669 |                                       if (enrollment && (session.session_type === 'live' || session.session_type === 'hybri
d')) {
1670 |                                         try {
1671 |                                           const tData = JSON.parse(localStorage.getItem(tKey) || '{"entered":0, "generated":
0}')
1672 |                                           if (tData.override === 'P') {
1673 |                                             attStatus = 'Present'
1674 |                                             attColor = 'text-emerald-700 bg-emerald-50 border-emerald-200'
```

```
                                        } else if (tData.override === 'F') {
                                          attStatus = 'Absent'
                                          attColor = 'text-rose-700 bg-rose-50 border-rose-200'
                                        } else if (tData.generated > 0) {
                                          if (tData.entered === tData.generated) {
                                            attStatus = 'Present'
                                            attColor = 'text-emerald-700 bg-emerald-50 border-emerald-200'
                                          } else if (tData.entered > 0) {
                                            attStatus = 'Partial'
                                            attColor = 'text-amber-700 bg-amber-50 border-amber-200'
                                          } else {
                                            attStatus = 'Absent'
                                            attColor = 'text-rose-700 bg-rose-50 border-rose-200'
                                          }
                                        }
                                      } catch(e) {}
                                    }

                                    return (
                                      <div className="flex items-center gap-1.5">
                                        <span className="text-[10px] font-bold px-2 py-0.5 rounded-full border bg-slate-100
text-slate-600 border-slate-200 flex items-center gap-1">
                                          <CheckCircle2 className="w-3 h-3 text-slate-400" /> Completed
                                        </span>
                                        {attStatus && (
                                          <span className={`text-[10px] font-bold px-2 py-0.5 rounded-full border flex item
s-center gap-1 ${attColor}`}>

                                            {attStatus}
                                          </span>
                                        )}
                                      </div>
                                    )
                                  })() : session.session_type ? (
                                    <span className={`text-[10px] font-bold px-2 py-0.5 rounded-full border capitalize ${se
ssionTypeColors[session.session_type] || 'bg-slate-100 text-slate-700 border-slate-200'}`}>
                                      {session.session_type === 'recorded' ? 'Video' : session.session_type === 'live' ? 'L
ive Online' : session.session_type.replace('_', ' ')}
                                    </span>
                                  ) : null}
                                </div>
                                {session.start_time && (
                                  <p className="text-xs text-slate-500 flex items-center gap-1">
                                    <Calendar className="w-3 h-3 text-cyan-600" />
                                    {new Date(session.start_time).toLocaleString(undefined, {
                                      month: 'short', day: 'numeric', hour: 'numeric', minute: '2-digit',
                                    })}
                                    {session.end_time && ` - ${new Date(session.end_time).toLocaleTimeString(undefined, { h
our: 'numeric', minute: '2-digit' })}`}
                                  </p>
                                )}
                              </div>

                                <div className="flex items-center gap-2 shrink-0">
                                {sessionMaterials.length > 0 && (
                                  <span className="text-[11px] text-slate-600 font-semibold bg-white border border-slate-20
0 px-2 py-0.5 rounded-full">
                                    {sessionMaterials.length} file{sessionMaterials.length !== 1 ? 's' : ''}
                                  </span>
                                )}
                                {session.meet_link && enrollment && (session.session_type === 'live' || session.session_typ
e === 'hybrid') && (
                                  isFinished ? (
                                    <span
                                      onClick={e => e.stopPropagation()}
                                      className="text-[11px] font-semibold text-slate-400 bg-slate-100 border border-slate-
200 px-2.5 py-1 rounded-lg"
                                    >
                                      Ended
                                    </span>
                                  ) : (
                                    <a
                                      href={session.meet_link}
                                      target="_blank"
                                      rel="noreferrer"
                                      onClick={e => e.stopPropagation()}
                                      className={`flex items-center gap-1 text-xs px-2.5 py-1 rounded-lg border transition-
all font-semibold ${

                                        isLiveNow
                                          ? 'text-white bg-gradient-to-r from-red-600 to-rose-600 border-rose-500 hover:opa
```

```
city-90 shadow-sm animate-pulse'
1745 |                                             : 'text-blue-700 bg-blue-50 border-blue-200 hover:bg-blue-100'
1746 |                                         }`}
1747 |                                       >
1748 |                                         <Video className="w-3.5 h-3.5" /> {isLiveNow ? 'Join Live Now' : 'Join'}
1749 |                                       </a>
1750 |                                     )
1751 |                                   )}
1752 |                                   {session.location && enrollment && (session.session_type === 'in_person' || session.session
_type === 'hybrid') && (
1753 |                                     <span className="flex items-center gap-1 text-xs text-emerald-700 bg-emerald-50 px-2.5 py
-1 rounded-lg border border-emerald-200 font-semibold truncate max-w-[120px]">
1754 |                                       <Target className="w-3.5 h-3.5" /> {session.location}
1755 |                                     </span>
1756 |                                   )}
1757 |                                   {isOpen
1758 |                                     ? <ChevronUp className="w-4 h-4 text-cyan-600 shrink-0" />
1759 |                                     : <ChevronDown className="w-4 h-4 text-slate-400 shrink-0" />}
1760 |                                 </div>
1761 |                               </button>
1762 |
1763 |                               {/* Session body */}
1764 |                               <AnimatePresence initial={false}>
1765 |                                 {isOpen && (
1766 |                                   <motion.div
1767 |                                     key="body"
1768 |                                     initial={{ height: 0, opacity: 0 }}
1769 |                                     animate={{ height: 'auto', opacity: 1 }}
1770 |                                     exit={{ height: 0, opacity: 0 }}
1771 |                                     transition={{ duration: 0.2, ease: 'easeInOut' }}
1772 |                                     style={{ overflow: 'hidden' }}
1773 |                                   >
1774 |                                     <div className="px-4 pb-4 pt-3 bg-white border-t border-slate-200">
1775 |                                       {session.description && (
1776 |                                         <p className="text-xs text-slate-600 mb-3 leading-relaxed">{session.description}</p>
1777 |                                       )}
1778 |
1779 |                                       {/* LIVE ATTENDANCE OTP PANEL */}
1780 |                                       {enrollment && (
1781 |                                         <div className="mb-4">
1782 |                                           <LiveAttendanceTraineePanel session={session} userId={profile!.id} />
1783 |                                         </div>
1784 |                                       )}
1785 |
1786 |                                       {sessionMaterials.length === 0 ? (
1787 |                                         <div className="flex items-center gap-2 py-3 px-3 rounded-xl bg-slate-50 border borde
r-slate-100">
1788 |                                           <FileText className="w-4 h-4 text-slate-400 shrink-0" />
1789 |                                           <p className="text-xs text-slate-500 italic">No materials uploaded for this session
yet.</p>
1790 |                                         </div>
1791 |                                       ) : (
1792 |                                         <ul className="space-y-2">
1793 |                                           {sessionMaterials.map((m: any) => (
1794 |                                             <li
1795 |                                               key={m.id}
1796 |                                               onClick={() => handlePreview(m)}
1797 |                                               className={`flex items-center gap-3 p-2.5 rounded-xl border transition-all ${
1798 |                                                 enrollment
1799 |                                                   ? `cursor-pointer hover:shadow-xs ${getMaterialBg(m.material_type)}`
1800 |                                                   : 'bg-slate-50 border-slate-100 opacity-60 cursor-not-allowed'
1801 |                                               }`}
1802 |                                             >
1803 |                                               <div className={`w-8 h-8 rounded-lg flex items-center justify-center shrink-0 b
order ${getMaterialBg(m.material_type)}`}>
1804 |                                                 {getMaterialIcon(m.material_type)}
1805 |                                               </div>
1806 |                                               <div className="flex-1 min-w-0">
1807 |                                                 <p className="text-xs font-semibold text-slate-900 truncate">{m.file_name}</p
>
1808 |                                                 <p className="text-[10px] text-slate-500 capitalize">{m.material_type}</p>
1809 |                                               </div>
1810 |                                               {enrollment ? (
1811 |                                                 m.material_type === 'link'
1812 |                                                   ? <ExternalLink className="w-3.5 h-3.5 text-slate-400 shrink-0" />
1813 |                                                   : downloadingId === m.id
1814 |                                                     ? <Loader2 className="w-3.5 h-3.5 text-cyan-600 animate-spin shrink-0" />
1815 |                                                     : <Download className="w-3.5 h-3.5 text-slate-400 shrink-0" />
1816 |                                               ) : (
```

```
                                            <Lock className="w-3.5 h-3.5 text-slate-400 shrink-0" />
                                          )}
                                        </li>
                                      ))}
                                    </ul>
                                  )}
                                </div>
                              </motion.div>
                            )}
                          </AnimatePresence>
                        </div>
                      )
                    })}
                  </div>
                </div>
              )}

              {/* Announcements */}
              <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm">
                <CourseAnnouncements courseId={courseId!} isTrainer={false} />
              </div>

              {/* Course Chat */}
              {enrollment && (enrollment.status === 'enrolled' || enrollment.status === 'completed' || enrollment.status === 'in_progress') && (
                <div className="space-y-6 mt-6 pt-6 border-t border-slate-200">
                  <CourseChat courseId={courseId!} isTrainer={false} />
                </div>
              )}
            </div>

            {/* Right sidebar (1/3) */}
            <div className="space-y-4">

              {/* Quick Stats */}
              <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm">
                <h3 className="text-[11px] font-bold text-slate-400 uppercase tracking-wider mb-4">Course Stats</h3>
                <div className="space-y-3">
                  {[
                    {
                      icon: <Clock className="w-3.5 h-3.5 text-amber-500" />,
                      label: 'Duration',
                      value: course.duration_minutes
                        ? course.duration_minutes >= 60
                          ? `${Math.floor(course.duration_minutes / 60)}h${course.duration_minutes % 60 ? ` ${course.duration_minutes % 60}m` : ''}`
                          : `${course.duration_minutes}m`
                        : 'Self-paced',
                    },
                    { icon: <Layers className="w-3.5 h-3.5 text-cyan-600" />, label: 'Sessions', value: String(course.sessions?.length || 0) },
                    { icon: <FileText className="w-3.5 h-3.5 text-blue-500" />, label: 'Materials', value: String(totalMaterials) },
                    { icon: <Target className="w-3.5 h-3.5 text-emerald-500" />, label: 'Passing Score', value: `${course.passing_score ?? '—'}%` },
                    { icon: <Users className="w-3.5 h-3.5 text-sky-500" />, label: 'Enrolled', value: `${activeCount} / ${seatLimit}` },
                    { icon: <Video className="w-3.5 h-3.5 text-indigo-500" />, label: 'Delivery', value: course.delivery_mode ? course.delivery_mode.charAt(0).toUpperCase() + course.delivery_mode.slice(1) : 'Recorded' },
                  ].map(item => (
                    <div key={item.label} className="flex items-center gap-2.5">
                      <div className="w-7 h-7 rounded-xl bg-slate-50 border border-slate-100 flex items-center justify-center shrink-0">
                        {item.icon}
                      </div>
                      <span className="text-xs text-slate-500 font-medium flex-1">{item.label}</span>
                      <span className="text-xs font-bold text-slate-900">{item.value}</span>
                    </div>
                  ))}
                </div>
              </div>

              {/* Instructor */}
              <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm">
                <h3 className="text-[11px] font-bold text-slate-400 uppercase tracking-wider mb-4">Instructor</h3>
                <div className="flex items-start gap-3.5 mb-3.5">
                  <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white font-black text-base shadow-md shadow-cyan-600/20 shrink-0 overflow-hidden border border-cyan-100">
                    {course.trainer?.avatar_path ? (
```

```
                          <img src={course.trainer.avatar_path} alt={course.trainer.full_name || 'Trainer'} className="w-full
h-full object-cover" />
                        ) : (
                          course.trainer?.full_name?.charAt(0) || 'T'
                        )}
                      </div>
                      <div className="min-w-0 flex-1">
                        <p className="text-sm font-bold text-slate-900 leading-tight">{course.trainer?.full_name || 'Assigned
Instructor'}</p>
                        {course.trainer?.years_of_experience && (
                          <p className="text-xs text-slate-500 font-medium mt-0.5">{course.trainer.years_of_experience} yrs e
xperience</p>
                        )}

                        {/* Trainer Social Links below name */}
                        <div className="flex items-center gap-1.5 mt-2 flex-wrap">
                          <a
                            href={
                              course.trainer?.linkedin_url
                                ? (course.trainer.linkedin_url.startsWith('http') ? course.trainer.linkedin_url : `https://
${course.trainer.linkedin_url}`)
                                : `https://www.linkedin.com/search/results/all/?keywords=${encodeURIComponent(course.traine
r?.full_name || 'MoES Trainer')}`
                            }
                            target="_blank"
                            rel="noreferrer"
                            className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-blue-50 hover:bg-blue-100 tex
t-blue-700 border border-blue-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                            title={course.trainer?.linkedin_url ? "LinkedIn Profile" : "Search LinkedIn Profile"}
                          >
                            <LinkedinIcon className="w-3.5 h-3.5" />
                            <span>LinkedIn</span>
                          </a>

                          <a
                            href={
                              course.trainer?.github_url
                                ? (course.trainer.github_url.startsWith('http') ? course.trainer.github_url : `https://${cour
se.trainer.github_url}`)
                                : `https://github.com/search?q=${encodeURIComponent(course.trainer?.full_name || 'MoES')}`
                            }
                            target="_blank"
                            rel="noreferrer"
                            className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-slate-100 hover:bg-slate-200
text-slate-800 border border-slate-300 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                            title={course.trainer?.github_url ? "GitHub / Research Profile" : "Search GitHub Profile"}
                          >
                            <GithubIcon className="w-3.5 h-3.5" />
                            <span>GitHub</span>
                          </a>

                          <a
                            href={
                              course.trainer?.website_url
                                ? (course.trainer.website_url.startsWith('http') ? course.trainer.website_url : `https://${co
urse.trainer.website_url}`)
                                : 'https://moes.gov.in'
                            }
                            target="_blank"
                            rel="noreferrer"
                            className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-emerald-50 hover:bg-emerald-1
00 text-emerald-700 border border-emerald-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                            title="Official Website / Portal"
                          >
                            <Globe className="w-3.5 h-3.5" />
                            <span>Website</span>
                          </a>

                          <a
                            href={`mailto:${course.trainer?.email || 'trainer@capacityconnect.in'}?subject=Regarding: ${encod
eURIComponent(course.title || 'Course')}`}
                            className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-cyan-50 hover:bg-cyan-100 tex
t-cyan-700 border border-cyan-200 text-[11px] font-bold transition-all shadow-2xs hover:scale-105"
                            title="Email Instructor"
                          >
                            <Mail className="w-3.5 h-3.5" />
                            <span>Email</span>
                          </a>
                        </div>
```

```
                      </div>
                    </div>

                    {/* Professional Summary / Biography */}
                    {course.trainer?.bio && (
                      <div className="mb-3.5 p-3.5 bg-slate-50 rounded-2xl border border-slate-200/80">
                        <p className="text-[10px] font-bold text-cyan-700 uppercase tracking-wider mb-1.5 flex items-center g
ap-1.5">
                          <User className="w-3 h-3 text-cyan-600" />
                          Professional Summary / Biography
                        </p>
                        <p className="text-xs text-slate-700 leading-relaxed whitespace-pre-line font-normal">
                          {course.trainer.bio}
                        </p>
                      </div>
                    )}

                    {/* Academic & Professional Qualifications */}
                    {course.trainer?.qualifications && (
                      <div className="mb-3.5 p-3.5 bg-blue-50/70 rounded-2xl border border-blue-100">
                        <p className="text-[10px] font-bold text-blue-700 uppercase tracking-wider mb-1.5 flex items-center g
ap-1.5">
                          <GraduationCap className="w-3.5 h-3.5 text-blue-600" />
                          Academic & Professional Qualifications
                        </p>
                        <p className="text-xs text-slate-800 leading-relaxed whitespace-pre-line font-medium">
                          {course.trainer.qualifications}
                        </p>
                      </div>
                    )}

                    {/* Expertise Areas */}
                    {course.trainer?.expertise_areas && course.trainer.expertise_areas.length > 0 && (
                      <div className="mt-3.5">
                        <p className="text-[10px] font-bold text-slate-400 uppercase tracking-wide mb-1.5">Expertise</p>
                        <div className="flex flex-wrap gap-1.5">
                          {course.trainer.expertise_areas.map((skill: string, idx: number) => (
                            <span key={idx} className="px-2 py-0.5 rounded-md bg-emerald-50 border border-emerald-100 text-[1
0px] font-semibold text-emerald-700">
                              {skill}
                            </span>
                          ))}
                        </div>
                      </div>
                    )}
                  </div>

                  {/* Schedule */}
                  {(course.start_date || course.end_date || course.live_class_timing ||
                    course.mock_test_timing || course.final_exam_timing || course.final_test_date) && (
                    <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-sm">
                      <h3 className="text-[11px] font-bold text-slate-400 uppercase tracking-wider mb-4 flex items-center gap
-1.5">
                        <Calendar className="w-3.5 h-3.5 text-cyan-600" /> Schedule
                      </h3>
                      <div className="space-y-2.5">
                        {(course.start_date || course.end_date) && (
                          <div className="p-3 rounded-xl bg-slate-50 border border-slate-200">
                            <p className="text-[10px] font-bold text-cyan-700 uppercase tracking-wide mb-0.5">Course Period</
p>
                            <p className="text-xs font-semibold text-slate-900">
                              {course.start_date && new Date(course.start_date).toLocaleDateString(undefined, { month: 'shor
t', day: 'numeric', year: 'numeric' })}
                              {course.start_date && course.end_date && ' – '}
                              {course.end_date && new Date(course.end_date).toLocaleDateString(undefined, { month: 'short', d
ay: 'numeric', year: 'numeric' })}
                            </p>
                          </div>
                        )}
                        {course.live_class_timing && (
                          <div className="p-3 rounded-xl bg-amber-50 border border-amber-100">
                            <p className="text-[10px] font-bold text-amber-700 uppercase tracking-wide mb-0.5">Live Classes</
p>
                            <p className="text-xs font-semibold text-slate-900">{course.live_class_timing}</p>
                          </div>
                        )}
                        {course.mock_test_timing && (
                          <div className="p-3 rounded-xl bg-blue-50 border border-blue-100">
                            <p className="text-[10px] font-bold text-blue-700 uppercase tracking-wide mb-0.5">Mock Test</p>
```

```
                            <p className="text-xs font-semibold text-slate-900">
                              {new Date(course.mock_test_timing).toLocaleString(undefined, { month: 'short', day: 'numeric',
hour: 'numeric', minute: '2-digit' })}
                            </p>
                          </div>
                        )}
                        {course.final_exam_timing && (
                          <div className="p-3 rounded-xl bg-rose-50 border border-rose-100">
                            <p className="text-[10px] font-bold text-rose-700 uppercase tracking-wide mb-0.5">Final Exam</p>
                            <p className="text-xs font-semibold text-slate-900">
                              {new Date(course.final_exam_timing).toLocaleString(undefined, { month: 'short', day: 'numeric',
hour: 'numeric', minute: '2-digit' })}
                            </p>
                          </div>
                        )}
                        {course.final_test_date && (
                          <div className="p-3 rounded-xl bg-emerald-50 border border-emerald-100">
                            <p className="text-[10px] font-bold text-emerald-700 uppercase tracking-wide mb-0.5">Final Test</
p>
                            <p className="text-xs font-semibold text-slate-900">
                              {new Date(course.final_test_date).toLocaleDateString(undefined, { month: 'short', day: 'numeri
c', year: 'numeric' })}
                              {course.final_test_start_time && ` at ${course.final_test_start_time}`}
                              {course.final_test_end_time && ` – ${course.final_test_end_time}`}
                            </p>
                          </div>
                        )}
                      </div>
                    </div>
                  )}

                  {/* Overall Live Attendance */}
                  {(() => {
                    if (!enrollment || !course?.sessions || !profile?.id) return null

                    let totalTracked = 0
                    let presentCount = 0
                    let partialCount = 0
                    let absentCount = 0
                    const trackedSessions: any[] = []

                    const userId = profile.id

                    course.sessions.forEach((session: any) => {
                      if (session.session_type !== 'live' && session.session_type !== 'hybrid') return

                      let tData = sessionAttendance[session.id]

                      // Fallback to localStorage for legacy local testing data if DB has no data
                      if (!tData || tData.generated === 0) {
                        try {
                          const localStr = localStorage.getItem(`trainee_attendance_${session.id}_${profile.id}`)
                          if (localStr) {
                            const localData = JSON.parse(localStr)
                            if (localData.generated > 0 || localData.override) {
                              tData = localData
                            }
                          }
                        } catch(e) {}
                      }

                      if (!tData) return

                      let isTracked = false
                      let sStatus = ''
                      let sColor = ''

                      if (tData.override === 'P' || tData.override === 'present') {
                        presentCount++
                        isTracked = true
                        sStatus = 'Present (Override)'
                        sColor = 'text-emerald-700 bg-emerald-50 border-emerald-200'
                      } else if (tData.override === 'F' || tData.override === 'absent') {
                        absentCount++
                        isTracked = true
                        sStatus = 'Absent (Override)'
                        sColor = 'text-rose-700 bg-rose-50 border-rose-200'
                      } else if (tData.override === 'late') {
```

```
                        partialCount++
                        isTracked = true
                        sStatus = 'Partial (Override)'
                        sColor = 'text-amber-700 bg-amber-50 border-amber-200'
                      } else if (tData.generated > 0) {
                        isTracked = true
                        if (tData.entered >= tData.generated * 0.5) {
                          presentCount++
                          sStatus = 'Present'
                          sColor = 'text-emerald-700 bg-emerald-50 border-emerald-200'
                        } else if (tData.entered > 0) {
                          partialCount++
                          sStatus = 'Partial'
                          sColor = 'text-amber-700 bg-amber-50 border-amber-200'
                        } else {
                          absentCount++
                          sStatus = 'Absent'
                          sColor = 'text-rose-700 bg-rose-50 border-rose-200'
                        }
                      }

                      if (isTracked) {
                        totalTracked++
                        trackedSessions.push({
                          session,
                          status: sStatus,
                          color: sColor,
                          score: tData.override ? null : `${tData.entered}/${tData.generated}`
                        })
                      }
                    })

                    return (
                      <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm">
                        <h2 className="text-sm font-bold text-slate-900 flex items-center justify-between mb-4">
                          <div className="flex items-center gap-2">
                            <UserCheck className="w-4 h-4 text-cyan-600" /> Overall Attendance
                          </div>
                          <button
                            onClick={() => setShowAttendanceDetails(!showAttendanceDetails)}
                            className="p-1 hover:bg-slate-100 rounded-full transition-colors"
                          >
                            {showAttendanceDetails ? <ChevronUp className="w-4 h-4 text-slate-500" /> : <ChevronDown classNam
e="w-4 h-4 text-slate-500" />}
                          </button>
                        </h2>

                        <div className="flex flex-col gap-3">
                          <div className="flex items-center justify-between">
                            <span className="text-xs font-semibold text-slate-500">Live Sessions Tracked</span>
                            <span className="text-sm font-black text-slate-900">{totalTracked}</span>
                          </div>

                          <div className="grid grid-cols-3 gap-2">
                            <div className="bg-emerald-50 border border-emerald-200 rounded-xl p-2.5 text-center">
                              <p className="text-[10px] font-bold text-emerald-700 uppercase mb-1">Present</p>
                              <p className="text-base font-black text-emerald-600">{presentCount}</p>
                            </div>
                            <div className="bg-amber-50 border border-amber-200 rounded-xl p-2.5 text-center">
                              <p className="text-[10px] font-bold text-amber-700 uppercase mb-1">Partial</p>
                              <p className="text-base font-black text-amber-600">{partialCount}</p>
                            </div>
                            <div className="bg-rose-50 border border-rose-200 rounded-xl p-2.5 text-center">
                              <p className="text-[10px] font-bold text-rose-700 uppercase mb-1">Absent</p>
                              <p className="text-base font-black text-rose-600">{absentCount}</p>
                            </div>
                          </div>

                          <div className="w-full bg-slate-100 rounded-full h-1.5 mt-2 flex overflow-hidden">
                            <div className="bg-emerald-500 h-full" style={{ width: `${(presentCount / totalTracked) * 100}%`
}} />
                            <div className="bg-amber-500 h-full" style={{ width: `${(partialCount / totalTracked) * 100}%` }}
/>
                            <div className="bg-rose-500 h-full" style={{ width: `${(absentCount / totalTracked) * 100}%` }} /
>
                          </div>

                          <AnimatePresence>
                            {showAttendanceDetails && (
```

```
                                <motion.div
                                  initial={{ height: 0, opacity: 0 }}
                                  animate={{ height: 'auto', opacity: 1 }}
                                  exit={{ height: 0, opacity: 0 }}
                                  className="overflow-hidden"
                                >
                                  <div className="mt-4 pt-4 border-t border-slate-100 flex flex-col gap-2">
                                    {trackedSessions.map((ts, idx) => (
                                      <div key={idx} className="flex items-center justify-between p-2 rounded-lg bg-slate-50 bo
rder border-slate-100">
                                        <div className="min-w-0">
                                          <p className="text-xs font-bold text-slate-800 truncate">{ts.session.title}</p>
                                          {ts.score && (
                                            <p className="text-[10px] font-medium text-slate-500 mt-0.5">Score: {ts.score}</p>
                                          )}
                                        </div>
                                        <span className={`text-[10px] font-bold px-2 py-0.5 rounded-full border ${ts.color} whi
tespace-nowrap ml-2`}>
                                          {ts.status}
                                        </span>
                                      </div>
                                    ))}
                                  </div>
                                </motion.div>
                              )}
                            </AnimatePresence>
                          </div>
                        </div>
                      )
                    })()}

                    {/* Feedback Card in Sidebar */}
                    {enrollment && (enrollment.status === 'enrolled' || enrollment.status === 'completed' || enrollment.status
=== 'in_progress') && (
                      <CourseFeedback courseId={courseId!} isTrainer={false} />
                    )}

                    {/* Enroll prompt for non-enrolled */}
                    {!enrollment && (
                      <div className="p-4 bg-amber-50 border border-amber-200 rounded-2xl flex items-start gap-3">
                        <AlertCircle className="w-4 h-4 text-amber-600 shrink-0 mt-0.5" />
                        <p className="text-xs text-amber-900 font-medium leading-relaxed">
                          Enroll to access all materials, track your progress, and earn a certificate upon completion.
                        </p>
                      </div>
                    )}
                  </div>
                </div>
              </motion.div>
            )}
          </div>

          <MaterialPreviewDialog
            material={previewMaterial}
            previewUrl={previewUrl}
            onClose={() => setPreviewMaterial(null)}
            onDownload={() => previewMaterial && handleDownload(previewMaterial)}
          />

          {/* About Course & Course Outline Modal */}
          <Dialog open={aboutModalOpen} onOpenChange={setAboutModalOpen}>
            <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto rounded-3xl p-6 md:p-8 bg-white border border-slat
e-200 shadow-2xl">
              <DialogHeader className="pb-4 border-b border-slate-100">
                <div className="flex flex-wrap items-center justify-between gap-3">
                  <div className="space-y-1 pr-6">
                    <div className="flex items-center gap-2">
                      <Badge variant="outline" className="text-[10px] font-bold uppercase text-cyan-800 bg-cyan-50 border-cyan-
200">
                        {course?.course_type || 'Standard'} Program
                      </Badge>
                      {course?.department && (
                        <Badge variant="outline" className="text-[10px] font-semibold text-slate-600 border-slate-200">
                          {course.department}
                        </Badge>
                      )}
                    </div>
                    <DialogTitle className="text-xl md:text-2xl font-black text-slate-900 leading-tight">
                      {course?.title}
```

```
                    </DialogTitle>
                    <p className="text-xs text-slate-500 font-medium">
                      Instructor: {course?.trainer?.full_name || 'Assigned Instructor'} &bull; {course?.duration_minutes ? `${M
ath.floor(course.duration_minutes / 60)}h` : 'Self-Paced'} &bull; Pass Gate: {course?.passing_score || 80}%
                    </p>
                  </div>
                </div>

                {/* Navigation Tabs */}
                <div className="flex items-center gap-2 pt-4">
                  <button
                    onClick={() => setAboutActiveTab('about')}
                    className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer
${
                      aboutActiveTab === 'about'
                        ? 'bg-cyan-600 text-white shadow-xs'
                        : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                    }`}
                  >
                    <BookOpen className="w-3.5 h-3.5" />
                    About This Course
                  </button>
                  <button
                    onClick={() => setAboutActiveTab('outline')}
                    className={`flex items-center gap-1.5 px-4 py-2 rounded-xl text-xs font-bold transition-all cursor-pointer
${
                      aboutActiveTab === 'outline'
                        ? 'bg-cyan-600 text-white shadow-xs'
                        : 'bg-slate-100 text-slate-700 hover:bg-slate-200'
                    }`}
                  >
                    <Layers className="w-3.5 h-3.5" />
                    Course Outline & Roadmap
                  </button>
                </div>
              </DialogHeader>

              {/* TAB 1: ABOUT THIS COURSE */}
              {aboutActiveTab === 'about' && (
                <div className="space-y-6 pt-2">
                  {/* Description */}
                  <div className="space-y-2">
                    <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                      <BookOpen className="w-3.5 h-3.5 text-cyan-600" /> Course Overview
                    </h4>
                    <p className="text-sm text-slate-700 leading-relaxed bg-slate-50 p-4 rounded-2xl border border-slate-200/80
whitespace-pre-line">
                      {course?.description || 'Comprehensive competency-based training designed for operational excellence acro
ss MoES divisions.'}
                    </p>
                  </div>

                  {/* Objectives */}
                  {(objectives.length > 0 || course?.learning_objectives) && (
                    <div className="space-y-2">
                      <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                        <Target className="w-3.5 h-3.5 text-cyan-600" /> Key Learning Objectives
                      </h4>
                      <div className="bg-slate-50 p-4 rounded-2xl border border-slate-200/80 space-y-2">
                        {objectives.length > 0 ? (
                          <ul className="space-y-2">
                            {objectives.map((obj, i) => (
                              <li key={i} className="flex items-start gap-2.5 text-xs sm:text-sm text-slate-700">
                                <CheckCircle2 className="w-4 h-4 text-cyan-600 shrink-0 mt-0.5" />
                                <span>{obj}</span>
                              </li>
                            ))}
                          </ul>
                        ) : (
                          <p className="text-xs sm:text-sm text-slate-700 whitespace-pre-line">
                            {typeof course?.learning_objectives === 'string'
                              ? course.learning_objectives
                              : JSON.stringify(course?.learning_objectives)}
                          </p>
                        )}
                      </div>
                    </div>
                  )}

```

```
                {/* Key Course Specifications Grid */}
                <div className="grid grid-cols-2 sm:grid-cols-3 gap-3">
                  <div className="p-3.5 rounded-2xl bg-cyan-50/50 border border-cyan-200/60">
                    <span className="text-[10px] font-bold text-cyan-800 uppercase tracking-wider block">Passing Threshold</s
pan>
                    <span className="text-sm font-extrabold text-cyan-950 mt-0.5 block">{course?.passing_score || 80}% Overal
l</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                    <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Duration</span>
                    <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">{course?.duration_minutes ? `${Math.
floor(course.duration_minutes / 60)} Hours` : 'Flexible'}</span>
                  </div>
                  <div className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200">
                    <span className="text-[10px] font-bold text-slate-500 uppercase tracking-wider block">Curriculum Gate</sp
an>
                    <span className="text-sm font-extrabold text-slate-900 mt-0.5 block">≥80% Module Quiz</span>
                  </div>
                </div>

                {/* Instructor Information */}
                {course?.trainer && (
                  <div className="p-4 rounded-2xl bg-slate-50 border border-slate-200 flex items-center gap-4">
                    <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 text-white font-extrabo
ld text-base flex items-center justify-center shrink-0 shadow-xs">
                      {course.trainer.full_name?.charAt(0) || 'T'}
                    </div>
                    <div className="min-w-0">
                      <span className="text-[10px] font-bold text-cyan-700 uppercase tracking-wider block">Assigned Instructo
r</span>
                      <h5 className="text-sm font-bold text-slate-900 truncate">{course.trainer.full_name}</h5>
                      <p className="text-xs text-slate-500 truncate">{course.trainer.email || 'MoES Certified Trainer'}</p>
                    </div>
                  </div>
                )}
              </div>
            )}

            {/* TAB 2: COURSE OUTLINE & ROADMAP */}
            {aboutActiveTab === 'outline' && (
              <div className="space-y-6 pt-2">
                {/* Syllabus Document Download / Preview */}
                {course?.session_flow_document_path && (
                  <div className="p-4 rounded-2xl bg-cyan-50/70 border border-cyan-200 flex items-center justify-between gap-
3">
                    <div className="flex items-center gap-2.5 min-w-0">
                      <FileText className="w-5 h-5 text-cyan-700 shrink-0" />
                      <div>
                        <h5 className="text-xs font-bold text-cyan-950">Official Syllabus Document</h5>
                        <p className="text-[11px] text-cyan-700">Detailed curriculum plan and reading materials</p>
                      </div>
                    </div>
                    <Button
                      size="sm"
                      onClick={handleOpenSessionDoc}
                      disabled={docLoading}
                      className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs h-8 rounded-xl shrink-0"
                    >
                      {docLoading ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Eye className="w-3.5 h-3.5 mr-1"
/>}

                      View Document
                    </Button>
                  </div>
                )}

                {/* Session Schedule & Timeline */}
                {course?.session_flow_text && (
                  <div className="space-y-3">
                    <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                      <Layers className="w-3.5 h-3.5 text-cyan-600" /> Session Schedule & Timeline
                    </h4>
                    <div className="bg-slate-50 border border-slate-200/80 rounded-2xl p-5 text-xs sm:text-sm text-slate-700
leading-relaxed whitespace-pre-wrap font-normal">
                      {course.session_flow_text}
                    </div>
                  </div>
                )}

                {/* Structured Modules Preview */}
```

```
                {course?.modules && Array.isArray(course.modules) && course.modules.length > 0 && (
                  <div className="space-y-3 pt-1">
                    <h4 className="text-xs font-bold text-slate-800 uppercase tracking-wider flex items-center gap-1.5">
                      <BookMarked className="w-3.5 h-3.5 text-cyan-600" /> Learning Modules Breakdown ({course.modules.lengt
h})
                    </h4>
                    <div className="space-y-2.5">
                      {course.modules.map((mod: any, mIdx: number) => {
                        const rawItems = mod.items || mod.content_items || []
                        const quizQuestions = mod.quiz_questions || []
                        const videoCount = rawItems.filter((i: any) => i.type === 'video').length
                        const notesCount = rawItems.filter((i: any) => i.type === 'notes' || i.type === 'pdf' || i.type ===
'doc').length
                        const quizCount = quizQuestions.length + rawItems.filter((i: any) => i.type === 'quiz').length

                        return (
                          <div key={mod.id || mIdx} className="p-3.5 rounded-2xl bg-slate-50 border border-slate-200 space-y-
1.5">
                            <div className="flex items-center justify-between gap-2">
                              <div className="flex items-center gap-2.5">
                                <span className="w-6 h-6 rounded-lg bg-cyan-100 text-cyan-800 font-bold text-xs flex items-ce
nter justify-center shrink-0">
                                  {mIdx + 1}
                                </span>
                                <h5 className="text-xs sm:text-sm font-bold text-slate-900">{mod.title || `Module ${mIdx + 1}
`}</h5>
                              </div>
                              <Badge variant="outline" className="text-[10px] font-semibold bg-white border-slate-200 text-sl
ate-600 shrink-0">
                                {rawItems.length} activities
                              </Badge>
                            </div>
                            {mod.description && (
                              <p className="text-xs text-slate-500 pl-8 line-clamp-2">{mod.description}</p>
                            )}
                            <div className="flex items-center gap-3 pl-8 pt-1 text-[10px] text-slate-500 font-medium">
                              {videoCount > 0 && <span className="text-blue-600 font-semibold">{videoCount} Video{videoCount
> 1 ? 's' : ''}</span>}
                              {notesCount > 0 && <span className="text-emerald-600 font-semibold">{notesCount} Note{notesCoun
t > 1 ? 's' : ''}</span>}
                              {quizCount > 0 && <span className="text-purple-600 font-semibold">{quizCount} Quiz Qs (≥80% gat
e)</span>}
                            </div>
                          </div>
                        )
                      })}
                    </div>
                  </div>
                )}
              </div>
            </DialogContent>
          </Dialog>

          <Dialog open={otpDialogType !== null} onOpenChange={(open) => !open && handleCloseOtpDialog()}>
            <DialogContent className="max-w-sm rounded-3xl p-6 bg-white border border-slate-200 shadow-2xl">
              <DialogHeader>
                <DialogTitle className="text-xl font-bold text-slate-900 text-center mb-1">
                  {otpDialogType === 'enroll' ? 'Confirm Enrollment' : 'Confirm Drop Course'}
                </DialogTitle>
                <DialogDescription className="text-center text-slate-600 text-sm">
                  {otpSent
                    ? <>We've sent a verification code to <strong>{profile?.email}</strong>. Enter it below to confirm.</>
                    : <>Are you sure you want to {otpDialogType === 'enroll' ? 'enroll in' : 'drop'} this course? We will send
a verification code to <strong>{profile?.email}</strong>.</>}
                </DialogDescription>
              </DialogHeader>

              {!otpSent ? (
                <DialogFooter className="flex-col sm:flex-col gap-2 mt-4">
                  <Button
                    onClick={handleSendOtp}
                    disabled={isSendingOtp}
                    className={`w-full font-bold h-11 rounded-xl text-white ${
                      otpDialogType === 'enroll'
                        ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 shadow-
cyan-600/20'
                        : 'bg-gradient-to-r from-rose-500 to-orange-500 hover:from-rose-600 hover:to-orange-600 shadow-rose-50
0/20'
```

```
                  } shadow-lg transition-all`}
                >
                  {isSendingOtp ? <Loader2 className="w-5 h-5 animate-spin" /> : 'Send Verification Code'}
                </Button>
                <Button
                  variant="ghost"
                  onClick={handleCloseOtpDialog}
                  disabled={isSendingOtp}
                  className="w-full text-slate-600 hover:text-slate-900 h-11 rounded-xl"
                >
                  Cancel
                </Button>
              </DialogFooter>
            ) : (
              <>
                <div className="py-4 space-y-4">
                  <div className="space-y-2">
                    <Input
                      placeholder="Enter verification OTP"
                      value={otpInput}
                      onChange={(e) => setOtpInput(e.target.value.replace(/[^0-9]/g, '').slice(0, 8))}
                      className="text-center text-lg tracking-[0.25em] font-bold h-12 rounded-xl border-slate-200 focus-visib
le:ring-cyan-500 bg-slate-50 text-slate-900"
                      maxLength={8}
                    />
                  </div>
                </div>

                <DialogFooter className="flex-col sm:flex-col gap-2">
                  <Button
                    onClick={handleVerifyOtp}
                    disabled={isVerifyingOtp || otpInput.trim().length < 6 || otpInput.trim().length > 8}
                    className={`w-full font-bold h-11 rounded-xl text-white ${
                      otpDialogType === 'enroll'
                        ? 'bg-gradient-to-r from-emerald-600 to-teal-600 hover:from-emerald-700 hover:to-teal-700 shadow-emer
ald-500/20'
                        : 'bg-gradient-to-r from-rose-500 to-orange-500 hover:from-rose-600 hover:to-orange-600 shadow-rose-5
00/20'
                    } shadow-lg transition-all`}
                  >
                    {isVerifyingOtp ? <Loader2 className="w-5 h-5 animate-spin" /> : 'Verify Code'}
                  </Button>
                  <Button
                    variant="ghost"
                    onClick={handleCloseOtpDialog}
                    disabled={isVerifyingOtp}
                    className="w-full text-slate-600 hover:text-slate-900 h-11 rounded-xl"
                  >
                    Cancel
                  </Button>
                </DialogFooter>
              </>
            )}
          </DialogContent>
        </Dialog>
      </DashboardShell>
    </ErrorBoundary>
  )
}
```

src/features/courses/TraineeCourseLearnPage.tsx TSX 93.5 KB 1,900 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react'
import { useParams, useNavigate, Link } from 'react-router-dom'
import { useQuery, useQueryClient } from '@tanstack/react-query'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import {
  ChevronRight, ChevronLeft, CheckCircle2, Lock, Sparkles,
  HelpCircle, Video, Image as ImageIcon, ExternalLink,
  Award, ArrowLeft, RotateCcw, CheckSquare, AlertCircle, Check,
  X, Layers, Eye, AlignLeft, Loader2, Sun, Moon, Maximize2,
  Clock, Play, Pause
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { toast } from 'sonner'
import { MaterialPreviewDialog } from '@/components/ui/MaterialPreviewDialog'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'
import { calculateCourseGradeBreakdown } from '@/lib/courseGrading'

function getYouTubeEmbedUrl(url?: string): string | null {
  if (!url) return null
  const regExp = /^.*(youtu.be\/|v\/|u\/\w\/|embed\/|watch\?v=|&v=)([^#&?]*).*/
  const match = url.match(regExp)
  return match && match[2].length === 11 ? `https://www.youtube.com/embed/${match[2]}` : null
}

function parseQuizData(raw: any): any {
  if (!raw) return null
  if (typeof raw === 'object') return raw
  if (typeof raw === 'string') {
    try {
      return JSON.parse(raw)
    } catch {
      return null
    }
  }
  return null
}

interface ResolvedQuizQuestion {
  id: string
  question: string
  options: string[]
  correct_option: number
  explanation: string
}

function shuffleArray<T>(array: T[]): T[] {
  const arr = [...array]
  for (let i = arr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1))
    const temp = arr[i]
    arr[i] = arr[j]
    arr[j] = temp
  }
  return arr
}

function resolveQuizQuestion(
  item: any,
  itemIdx: number,
  moduleQuizQuestions: any[],
  moduleTitle: string,
  shuffleOptions: boolean = true
): ResolvedQuizQuestion {
  let qData = parseQuizData(item?.quiz_data)

  // 1. Try match by ID from module quiz_questions

  if (!qData || !Array.isArray(qData.options) || qData.options.length === 0) {
    const matchById = moduleQuizQuestions.find(
      q => (q.id && item.id && q.id === item.id) || (qData?.id && q.id === qData.id)
    )
    if (matchById) {
      qData = { ...(qData || {}), ...matchById }
```

```
    }
  }

  // 2. Try match by question text similarity

  if (!qData || !Array.isArray(qData.options) || qData.options.length === 0) {
    const rawPrompt = (item?.title || item?.question || '')
      .replace(/^Quiz:\s*/i, '')
      .replace(/\.\.\.$/, '')
      .trim()
      .toLowerCase()

    if (rawPrompt.length >= 5) {
      const matchByText = moduleQuizQuestions.find(q => {
        const targetQ = (q.question || '').toLowerCase()
        return targetQ.startsWith(rawPrompt.slice(0, 20)) || rawPrompt.startsWith(targetQ.slice(0, 20))
      })
      if (matchByText) {
        qData = { ...(qData || {}), ...matchByText }
      }
    }
  }

  // 3. Try match by sequential index in moduleQuizQuestions

  if (!qData || !Array.isArray(qData.options) || qData.options.length === 0) {
    if (moduleQuizQuestions[itemIdx]) {
      qData = { ...(qData || {}), ...moduleQuizQuestions[itemIdx] }
    }
  }

  // 4. Resolve question string

  let fullQuestion = qData?.question || item?.question
  if (!fullQuestion || fullQuestion.endsWith('...')) {
    if (qData?.question && !qData.question.endsWith('...')) {
      fullQuestion = qData.question
    } else {
      const rawPrompt = (item?.title || item?.question || '').replace(/^Quiz:\s*/i, '').trim()
      const topic = moduleTitle.replace(/^Module \d+:\s*/i, '') || 'this module'
      if (rawPrompt.toLowerCase().includes('prerequisite') || rawPrompt.toLowerCase().includes('protocol')) {
        fullQuestion = `What is the core prerequisite protocol for ${topic}?`
      } else if (rawPrompt.toLowerCase().includes('passing score') || rawPrompt.toLowerCase().includes('benchmark')) {
        fullQuestion = `What minimum passing score is required to unlock subsequent modules in Capacity Connect?`
      } else if (rawPrompt.toLowerCase().includes('abnormal') || rawPrompt.toLowerCase().includes('telemetry') || rawPrompt.t
oLowerCase().includes('discrepanc')) {
        fullQuestion = `How should abnormal telemetry flags or critical discrepancies in ${topic} be handled?`
      } else {
        fullQuestion = rawPrompt.replace(/\.\.\.$/, '') || `Key concept evaluation for ${topic}`
      }
    }
  }

  // 5. Resolve options array

  let options: string[] = []
  if (Array.isArray(qData?.options) && qData.options.length > 0) {
    options = qData.options.map((o: any) => (typeof o === 'string' ? o : String(o?.text || o?.title || o)))
  } else if (Array.isArray(item?.options) && item.options.length > 0) {
    options = item.options.map((o: any) => (typeof o === 'string' ? o : String(o?.text || o?.title || o)))
  } else if (typeof item?.options === 'string') {
    try {
      const parsed = JSON.parse(item.options)
      if (Array.isArray(parsed)) options = parsed
    } catch {}
  }

  // 6. If options are still empty, supply meaningful topic-aligned options

  if (!options || options.length === 0) {
    const qLower = fullQuestion.toLowerCase()
    const topic = moduleTitle.replace(/^Module \d+:\s*/i, '') || 'the subject'

    if (qLower.includes('prerequisite') || qLower.includes('protocol')) {
      options = [
        'Adhere to verified operational checklists and standard ministry guidelines',
        'Bypass preliminary verification to speed up execution',
        'Conduct unrecorded subjective assessments',
        'Defer telemetry log synchronization until post-incident review'
```

```
        ]
      } else if (qLower.includes('passing score') || qLower.includes('benchmark') || qLower.includes('threshold')) {
        options = [
          '80% passing threshold in module checkpoint verification',
          '50% general participation threshold',
          '60% optional assessment threshold',
          'No benchmark requirement'
        ]
      } else if (qLower.includes('abnormal') || qLower.includes('telemetry') || qLower.includes('discrepanc') || qLower.include
s('critical')) {
        options = [
          'Log anomaly immediately and escalate to emergency response command',
          'Suppress the alert to prevent alarm fatigue',
          'Recalibrate sensor data without logging',
          'Wait for end-of-day reconciliation before taking action'
        ]
      } else {
        options = [
          `Apply standard ${topic} verification procedures and validate schema integrity`,
          `Skip baseline relational constraint validation`,
          `Perform unindexed queries without foreign key constraints`,
          `Disable transaction safety logging`
        ]
      }
    }

    const correctOpt =
      typeof qData?.correct_option === 'number'
        ? qData.correct_option
        : typeof item?.correct_option === 'number'
        ? item.correct_option
        : 0

    const explanation =
      qData?.explanation ||
      item?.explanation ||
      item?.content ||
      'Standard operational protocols ensure consistent execution, safety, and data integrity.'

    // Shuffle options for the trainee side and map correct_option accordingly

    let finalOptions = options
    let finalCorrectOpt = correctOpt

    if (shuffleOptions && Array.isArray(options) && options.length > 1) {
      const correctOptionText = options[correctOpt] ?? options[0]
      finalOptions = shuffleArray(options)
      const newIdx = finalOptions.findIndex(o => o === correctOptionText)
      finalCorrectOpt = newIdx !== -1 ? newIdx : 0
    }

    return {
      id: item?.id || qData?.id || `quiz-q-${itemIdx}`,
      question: fullQuestion,
      options: finalOptions,
      correct_option: finalCorrectOpt,
      explanation
    }
  }

export function TraineeCourseLearnPage() {
  const { courseId } = useParams<{ courseId: string }>()
  const { profile } = useAuth()
  const navigate = useNavigate()
  const queryClient = useQueryClient()

  // Balanced Light/Dark Theme state

  const [isDarkMode, setIsDarkMode] = useState<boolean>(() => {
    const saved = localStorage.getItem('cc_learn_theme')
    if (saved === 'dark') return true
    if (saved === 'light') return false
    return false // default to clean balanced light mode
  })

  const toggleTheme = () => {
    setIsDarkMode(prev => {
      const next = !prev
      localStorage.setItem('cc_learn_theme', next ? 'dark' : 'light')
```

```
      return next
    })
  }

  const [activeModuleIndex, setActiveModuleIndex] = useState<number>(0)
  const [completedModules, setCompletedModules] = useState<string[]>([])
  const [moduleQuizAnswers, setModuleQuizAnswers] = useState<Record<string, Record<number, number>>>({})
  const [moduleQuizScores, setModuleQuizScores] = useState<Record<string, { score: number; passed: boolean; submitted: boolean }>>({})
  const [videoWatchProgress, setVideoWatchProgress] = useState<Record<string, number>>({})
  const [activePlayingVideoId, setActivePlayingVideoId] = useState<string | null>(null)
  const [retakeKey, setRetakeKey] = useState<number>(0)
  const [previewMaterial, setPreviewMaterial] = useState<any | null>(null)
  const [previewUrl, setPreviewUrl] = useState<string | null>(null)
  const [isSidebarOpen, setIsSidebarOpen] = useState(false)
  const [downloadingCert, setDownloadingCert] = useState(false)
  const contentTopRef = useRef<HTMLDivElement>(null)

  // Fetch Course details

  const { data: course, isLoading: courseLoading } = useQuery({
    queryKey: ['course-learn', courseId],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('courses')
        .select(`
          *,

          trainer:trainers!courses_trainer_id_fkey(full_name, bio, years_of_experience, qualifications, email, study_details)
        `)
        .eq('id', courseId!)
        .single() as any

      if (error) throw error
      return data
    },
    enabled: !!courseId,
  })

  // Fetch Enrollment details to verify approval

  const { data: enrollment, isLoading: enrollmentLoading } = useQuery({
    queryKey: ['enrollment-learn', courseId, profile?.id],
    queryFn: async () => {
      if (!profile?.id) return null
      const { data, error } = await supabase
        .from('enrollments')
        .select('*')
        .eq('course_id', courseId!)
        .eq('user_id', profile!.id)
        .maybeSingle()

      if (error) throw error
      return data
    },
    enabled: !!courseId && !!profile?.id,
  })

  // Fetch course assessments to know if a Final Assessment exists

  const { data: courseAssessments = [] } = useQuery({
    queryKey: ['learn-course-assessments', courseId],
    queryFn: async () => {
      if (!courseId) return []
      const { data, error } = await supabase
        .from('assessments')
        .select('*')
        .eq('course_id', courseId)
        .eq('status', 'published')
      if (error) return []
      return data || []
    },
    enabled: !!courseId,
  })

  // Fetch Trainee assessment attempts for grade breakdown

  const { data: traineeAttempts = [] } = useQuery({
    queryKey: ['trainee-attempts-learn', courseId, profile?.id],
```

```
    queryFn: async () => {
      if (!profile?.id || !courseId) return []
      const { data, error } = await supabase
        .from('assessment_attempts')
        .select('*, assessment:assessments!inner(course_id, assessment_type, title, passing_score)')
        .eq('user_id', profile.id)
        .eq('assessment.course_id', courseId)
      if (error) return []
      return data || []
    },
    enabled: !!courseId && !!profile?.id,
  })

  const finalAssessment = courseAssessments?.find((a: any) => a.assessment_type === 'final')

  const gradeBreakdown = calculateCourseGradeBreakdown({
    moduleProgressPercent: enrollment?.progress_percent ?? (course?.modules?.length && completedModules.length >= course.modules.length ? 100 : 0),
    courseAssessments: (courseAssessments || []) as any,
    traineeAttempts: traineeAttempts as any,
    passingScore: course?.passing_score ?? 50,
  })

  const isApproved = Boolean(enrollment && (['enrolled', 'in_progress', 'completed'] as string[]).includes((enrollment as any).status))

  const hasInitializedActiveModule = useRef(false)

  // Load and reconcile completed modules from localStorage and enrollment

  useEffect(() => {
    if (!courseId || !profile?.id) return
    let localCompleted: string[] = []
    let localScores: Record<string, any> = {}

    const stored = localStorage.getItem(`cc_mod_progress_${courseId}_${profile.id}`)
    if (stored) {
      try {
        const parsed = JSON.parse(stored)
        if (Array.isArray(parsed.completed)) {
          localCompleted = parsed.completed
        }
        if (parsed.scores) {
          localScores = parsed.scores
        }
      } catch (e) {
        console.error('Error loading module progress:', e)
      }
    }

    // Load video watch progress from local storage

    const storedWatch = localStorage.getItem(`cc_video_watch_${courseId}_${profile.id}`)
    if (storedWatch) {
      try {
        setVideoWatchProgress(JSON.parse(storedWatch))
      } catch (e) {
        console.error('Error loading video watch progress:', e)
      }
    }

    // Reconcile from enrollment progress_percent if local storage was empty

    if (localCompleted.length === 0 && enrollment?.progress_percent && course?.modules?.length) {
      const count = Math.min(
        course.modules.length,
        Math.round((enrollment.progress_percent / 100) * course.modules.length)
      )
      localCompleted = course.modules.slice(0, count).map((m: any) => m.id)
    }

    setCompletedModules(localCompleted)
    setModuleQuizScores(localScores)
  }, [courseId, profile?.id, enrollment?.progress_percent, course?.modules])

  // Set initial active module ONLY ONCE on initial load

  useEffect(() => {
    if (hasInitializedActiveModule.current) return
```

```
    if (!course?.modules || !Array.isArray(course.modules) || course.modules.length === 0) return

    const firstIncompleteIdx = course.modules.findIndex((m: any) => !completedModules.includes(m.id))
    if (firstIncompleteIdx !== -1) {
      setActiveModuleIndex(firstIncompleteIdx)
    } else if (completedModules.length >= course.modules.length) {
      setActiveModuleIndex(course.modules.length - 1)
    }
    hasInitializedActiveModule.current = true
  }, [course?.modules, completedModules])

  // Scroll to top when active module changes
  useEffect(() => {
    contentTopRef.current?.scrollIntoView({ behavior: 'smooth' })
  }, [activeModuleIndex])

  const formatDuration = (totalSeconds: number) => {
    const mins = Math.floor(totalSeconds / 60)
    const secs = Math.floor(totalSeconds % 60)
    return `${mins}m ${secs < 10 ? '0' : ''}${secs}s`
  }

  const updateVideoWatchProgress = React.useCallback(
    (itemId: string, secondsWatched: number, totalSeconds: number) => {
      setVideoWatchProgress(prev => {
        const current = prev[itemId] || 0
        const updated = Math.min(totalSeconds, Math.max(current, Math.floor(secondsWatched)))
        if (updated === current) return prev
        const next = { ...prev, [itemId]: updated }
        if (profile?.id && courseId) {
          localStorage.setItem(`cc_video_watch_${courseId}_${profile.id}`, JSON.stringify(next))
        }
        return next
      })
    },
    [courseId, profile?.id]
  )

  const modules = Array.isArray(course?.modules) ? course.modules : []
  const currentModule = modules[activeModuleIndex] || null

  const checkModuleVideoRequirement = React.useCallback(
    (mod: any) => {
      if (!mod) return { passed: true, unmetList: [] }
      const rawItems = mod.items || mod.content_items || []
      const videoItems = rawItems.filter((i: any) => i.type === 'video')

      const unmetList: Array<{
        id: string
        title: string
        watched: number
        required: number
        total: number
        percent: number
      }> = []

      for (const v of videoItems) {
        const totalSec =
          v.duration_seconds || (v.duration_minutes ? v.duration_minutes * 60 : 300)
        const requiredSec = Math.ceil(totalSec * 0.9)
        const watchedSec = videoWatchProgress[v.id] || 0
        if (watchedSec < requiredSec) {
          unmetList.push({
            id: v.id,
            title: v.title || 'Video Lesson',
            watched: watchedSec,
            required: requiredSec,
            total: totalSec,
            percent: Math.min(100, Math.round((watchedSec / totalSec) * 100)),
          })
        }
      }

      return {
        passed: unmetList.length === 0,
        unmetList,
      }
    },
```

```
    [videoWatchProgress]
  )

  const currentVideoCheck = checkModuleVideoRequirement(currentModule)

  // Process and resolve all sequential items and quizzes for the current module (with shuffled options for trainees)
  const { sequentialItems, allQuizQuestions } = React.useMemo(() => {
    if (!currentModule) return { sequentialItems: [], allQuizQuestions: [] }
    const rawItems = currentModule.items || currentModule.content_items || []
    const legacyQuizQuestions: any[] = Array.isArray(currentModule.quiz_questions) ? currentModule.quiz_questions : []

    const seqItems: any[] = []
    const allQuiz: ResolvedQuizQuestion[] = []

    let quizCounter = 0
    rawItems.forEach((item: any) => {
      if (item.type === 'quiz') {
        const resolved = resolveQuizQuestion(item, quizCounter, legacyQuizQuestions, currentModule.title || '', true)
        allQuiz.push(resolved)
        seqItems.push({
          ...item,
          title: resolved.question,
          quiz_data: resolved,
        })
        quizCounter++
      } else {
        seqItems.push(item)
      }
    })

    legacyQuizQuestions.forEach((lq: any) => {
      const isAlreadyIncluded = allQuiz.some(
        q => q.id === lq.id || q.question.toLowerCase() === (lq.question || '').toLowerCase()
      )
      if (!isAlreadyIncluded) {
        const resolved = resolveQuizQuestion(
          { id: lq.id, type: 'quiz', quiz_data: lq },
          allQuiz.length,
          legacyQuizQuestions,
          currentModule.title || '',
          true
        )
        allQuiz.push(resolved)
        seqItems.push({
          id: resolved.id,
          type: 'quiz',
          title: resolved.question,
          quiz_data: resolved,
        })
      }
    })

    return { sequentialItems: seqItems, allQuizQuestions: allQuiz }
  }, [currentModule, retakeKey])

  // Active video watch interval ticker
  useEffect(() => {
    if (!activePlayingVideoId) return
    const currentItem = sequentialItems.find((i: any) => i.id === activePlayingVideoId)
    const totalSec =
      currentItem?.duration_seconds ||
      (currentItem?.duration_minutes ? currentItem.duration_minutes * 60 : 300)

    const interval = setInterval(() => {
      setVideoWatchProgress(prev => {
        const current = prev[activePlayingVideoId] || 0
        if (current >= totalSec) {
          setActivePlayingVideoId(null)
          return prev
        }
        const nextSec = current + 1
        const updated = { ...prev, [activePlayingVideoId]: nextSec }
        if (profile?.id && courseId) {
          localStorage.setItem(`cc_video_watch_${courseId}_${profile.id}`, JSON.stringify(updated))
        }
        return updated
      })
```

```
526 |     }, 1000)
527 |
528 |     return () => clearInterval(interval)
529 |   }, [activePlayingVideoId, sequentialItems, courseId, profile?.id])
530 |
531 |   const handleSelectQuizAnswer = (moduleId: string, questionIdx: number, optionIdx: number) => {
532 |     setModuleQuizAnswers(prev => ({
533 |       ...prev,
534 |       [moduleId]: {
535 |         ...(prev[moduleId] || {}),
536 |         [questionIdx]: optionIdx,
537 |       },
538 |     }))
539 |   }
540 |
541 |   const handleSubmitModuleQuiz = async (mod: any) => {
542 |     // 90% Video playback requirement check

543 |     const videoCheck = checkModuleVideoRequirement(mod)
544 |     if (!videoCheck.passed) {
545 |       const firstUnmet = videoCheck.unmetList[0]
546 |       toast.error(
547 |         `⚠️ Video Requirement: Please watch at least 90% of "${firstUnmet.title}" before submitting the quiz. (Watched: ${for
matDuration(firstUnmet.watched)} / Required: ${formatDuration(firstUnmet.required)})`,
548 |         { duration: 6000 }
549 |       )
550 |       return
551 |     }
552 |
553 |     const questions = allQuizQuestions
554 |     if (questions.length === 0) return
555 |
556 |     const userAnswers = moduleQuizAnswers[mod.id] || {}
557 |     const answeredCount = Object.keys(userAnswers).length
558 |     if (answeredCount < questions.length) {
559 |       toast.error(
560 |         `Please answer all ${questions.length} questions before submitting. (${answeredCount}/${questions.length} answered)`
561 |       )
562 |       return
563 |     }
564 |
565 |     let correctCount = 0
566 |     questions.forEach((q: any, idx: number) => {
567 |       const correctOptionIndex = q.correct_option ?? 0
568 |       if (userAnswers[idx] === correctOptionIndex) {
569 |         correctCount++
570 |       }
571 |     })
572 |
573 |     const scorePercent = Math.round((correctCount / questions.length) * 100)
574 |     const passed = scorePercent >= 80
575 |
576 |     const updatedScores = {
577 |       ...(moduleQuizScores || {}),
578 |       [mod.id]: {
579 |         score: scorePercent,
580 |         passed,
581 |         submitted: true,
582 |       },
583 |     }
584 |     setModuleQuizScores(updatedScores)
585 |
586 |     if (passed) {
587 |       const newCompleted = Array.from(new Set([...completedModules, mod.id]))
588 |       setCompletedModules(newCompleted)
589 |
590 |       if (profile?.id && courseId) {
591 |         localStorage.setItem(
592 |           `cc_mod_progress_${courseId}_${profile.id}`,
593 |           JSON.stringify({
594 |             completed: newCompleted,
595 |             scores: updatedScores,
596 |           })
597 |         )
598 |       }
599 |
600 |       // Update enrollment progress in Supabase

601 |       if (enrollment?.id && modules.length > 0) {
```

```
602 |         const progress = Math.min(100, Math.round((newCompleted.length / modules.length) * 100))
603 |         await supabase
604 |           .from('enrollments')
605 |           .update({
606 |             progress_percent: progress,
607 |             status: progress === 100 ? 'completed' : 'in_progress',
608 |           })
609 |           .eq('id', enrollment.id)
610 |         queryClient.invalidateQueries({ queryKey: ['enrollment-learn'] })
611 |         queryClient.invalidateQueries({ queryKey: ['trainee-enrollment'] })
612 |       }
613 |
614 |       toast.success(`🎉 Excellent! You scored ${scorePercent}% (${correctCount}/${questions.length} correct). Next module unl
ocked!`, {
615 |         duration: 4500,
616 |       })
617 |     } else {
618 |       toast.error(`Score: ${scorePercent}% (${correctCount}/${questions.length} correct). Please review the questions and try
again.`, {
619 |         duration: 4500,
620 |       })
621 |     }
622 |   }
623 |
624 |   const handleRetakeModuleQuiz = (moduleId: string) => {
625 |     setModuleQuizAnswers(prev => ({ ...prev, [moduleId]: {} }))
626 |     setModuleQuizScores(prev => {
627 |       const next = { ...prev }
628 |       delete next[moduleId]
629 |       return next
630 |     })
631 |     setRetakeKey(prev => prev + 1)
632 |     toast.info('Quiz reset with refreshed options. Review the lesson and select your answers.')
633 |   }
634 |
635 |   const handleCompleteContentModule = async (mod: any) => {
636 |     // 90% Video playback requirement check

637 |     const videoCheck = checkModuleVideoRequirement(mod)
638 |     if (!videoCheck.passed) {
639 |       const firstUnmet = videoCheck.unmetList[0]
640 |       toast.error(
641 |         `⚠️ Video Requirement: Please watch at least 90% of "${firstUnmet.title}" before completing this module. (Watched:
${formatDuration(firstUnmet.watched)} / Required: ${formatDuration(firstUnmet.required)})`,
642 |         { duration: 6000 }
643 |       )
644 |       return
645 |     }
646 |
647 |     const newCompleted = Array.from(new Set([...completedModules, mod.id]))
648 |     setCompletedModules(newCompleted)
649 |
650 |     if (profile?.id && courseId) {
651 |       localStorage.setItem(
652 |         `cc_mod_progress_${courseId}_${profile.id}`,
653 |         JSON.stringify({
654 |           completed: newCompleted,
655 |           scores: moduleQuizScores,
656 |         })
657 |       )
658 |     }
659 |
660 |     if (enrollment?.id && modules.length > 0) {
661 |       const progress = Math.min(100, Math.round((newCompleted.length / modules.length) * 100))
662 |       await supabase
663 |         .from('enrollments')
664 |         .update({
665 |           progress_percent: progress,
666 |           status: progress === 100 ? 'completed' : 'in_progress',
667 |         })
668 |         .eq('id', enrollment.id)
669 |       queryClient.invalidateQueries({ queryKey: ['enrollment-learn'] })
670 |       queryClient.invalidateQueries({ queryKey: ['trainee-enrollment'] })
671 |     }
672 |
673 |     toast.success('Module completed!')
674 |     if (activeModuleIndex < modules.length - 1) {
675 |       setActiveModuleIndex(prev => prev + 1)
676 |     }
```

```
  }

  const handleNextModule = () => {
    if (activeModuleIndex < modules.length - 1) {
      setActiveModuleIndex(prev => prev + 1)
    }
  }

  const handlePrevModule = () => {
    if (activeModuleIndex > 0) {
      setActiveModuleIndex(prev => prev - 1)
    }
  }

  const handleDownloadCertificate = async () => {
    if (!course || !profile) return
    setDownloadingCert(true)
    try {
      const result = await generateTraineeCertificate((course as any).certificate_template_url || null, {
        traineeName: profile.full_name || 'Trainee',
        traineeEmail: profile.email || undefined,
        traineeId: profile.id,
        courseId: course.id,
        courseTitle: course.title,
        trainerName: (course as any).trainer?.full_name || 'Assigned Instructor',
        percentage: `${gradeBreakdown.totalScore}%`,
        completedAt: new Date().toISOString(),
        certificateId: 'CC-' + course.id.slice(0, 6).toUpperCase() + '-' + profile.id.slice(0, 4).toUpperCase(),
      })
      triggerFileDownload(result.blob, result.fileName)
      toast.success('Certificate downloaded successfully!')
    } catch (e: any) {
      toast.error(e?.message || 'Failed to generate certificate.')
    } finally {
      setDownloadingCert(false)
    }
  }

  if (courseLoading || enrollmentLoading) {
    return (
      <div className={`min-h-screen flex flex-col items-center justify-center gap-4 ${isDarkMode ? 'bg-[#040814] text-white'
: 'bg-[#FAF9F6] text-slate-800'}`}>
        <Loader2 className="w-10 h-10 text-cyan-500 animate-spin" />
        <p className="text-sm font-medium tracking-wide opacity-80">Loading learning experience...</p>
      </div>
    )
  }

  if (!course) {
    return (
      <div className={`min-h-screen flex flex-col items-center justify-center p-6 text-center ${isDarkMode ? 'bg-[#040814] te
xt-white' : 'bg-[#FAF9F6] text-slate-800'}`}>
        <h2 className="text-2xl font-bold mb-2">Course Not Found</h2>
        <p className="text-sm opacity-75 mb-6">This course is not available.</p>
        <Button onClick={() => navigate('/trainee/courses')} className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold ro
unded-xl shadow-md">
          Back to Catalog
        </Button>
      </div>
    )
  }

  // Access Control Guard

  if (!isApproved) {
    return (
      <div className={`min-h-screen flex items-center justify-center p-6 ${isDarkMode ? 'bg-[#040814] text-white' : 'bg-[#FAF
9F6] text-slate-800'}`}>
        <div className={`max-w-md w-full rounded-3xl p-8 text-center shadow-2xl space-y-5 border ${isDarkMode ? 'bg-[#0b1329]
border-amber-500/30' : 'bg-white border-amber-200'}`}>
          <div className="w-16 h-16 rounded-2xl bg-amber-500/10 border border-amber-500/30 text-amber-500 flex items-center j
ustify-center mx-auto">
            <Lock className="w-8 h-8" />
          </div>
          <h2 className="text-xl font-black">Enrollment Approval Required</h2>
          <p className="text-sm opacity-80 leading-relaxed">
            {enrollment?.status === 'pending_approval'
              ? 'Your enrollment request is pending approval from the trainer. You will gain full access once your request is
approved.'
```

```
748 |               : enrollment?.status === 'rejected'
749 |               ? 'Your enrollment request was not approved. Please contact your instructor.'
750 |               : 'You have not enrolled in this course yet. Please enroll and wait for trainer approval to access the learning
modules.'}
751 |           </p>
752 |           <div className="flex flex-col gap-2.5 pt-2">
753 |             <Button
754 |               onClick={() => navigate(`/trainee/courses/${courseId}`)}
755 |               className="w-full bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white f
ont-bold rounded-xl py-2.5 shadow-lg shadow-cyan-500/20"
756 |             >
757 |               View Course Details & Enroll
758 |             </Button>
759 |             <Button
760 |               variant="outline"
761 |               onClick={() => navigate('/trainee/courses')}
762 |               className={`w-full rounded-xl ${isDarkMode ? 'border-slate-700 text-slate-300 hover:bg-slate-800' : 'border-sla
te-300 text-slate-700 hover:bg-slate-100'}`}
763 |             >
764 |               Back to Catalog
765 |             </Button>
766 |           </div>
767 |         </div>
768 |       </div>
769 |     )
770 |   }
771 |
772 |   const isCurrentCompleted = currentModule
773 |     ? completedModules.includes(currentModule.id) || activeModuleIndex < completedModules.length
774 |     : false
775 |   const quizResult = currentModule ? moduleQuizScores[currentModule.id] : null
776 |   const userAnswers = currentModule ? moduleQuizAnswers[currentModule.id] || {} : {}
777 |   const totalCompleted = completedModules.length
778 |   const progressPercent = modules.length > 0 ? Math.round((totalCompleted / modules.length) * 100) : 0
779 |
780 |   // The trainee can advance forward if current module is completed, passed, or next module is already unlocked

781 |   const isNextUnlocked =
782 |     activeModuleIndex < modules.length - 1 &&
783 |     (isCurrentCompleted ||
784 |       (quizResult?.passed ?? false) ||
785 |       activeModuleIndex < completedModules.length ||
786 |       completedModules.includes(modules[activeModuleIndex]?.id))
787 |
788 |   const canGoNext = isNextUnlocked
789 |   const isLastModule = activeModuleIndex === modules.length - 1
790 |
791 |   return (
792 |     <div className={`min-h-screen flex flex-col transition-colors duration-300 selection:bg-cyan-500 selection:text-white ${i
sDarkMode ? 'bg-[#040814] text-slate-100' : 'bg-[#FAF9F6] text-slate-800'}`}>
793 |       {/* ── TOP NAV BAR (Capacity Connect Header) ──────────────── */}
794 |       <header className={`sticky top-0 z-40 backdrop-blur-md px-4 lg:px-8 py-3.5 flex items-center justify-between gap-4 bord
er-b transition-colors duration-300 ${
795 |         isDarkMode
796 |           ? 'bg-[#071124]/90 border-cyan-500/20 shadow-lg'
797 |           : 'bg-white/95 border-slate-200/90 shadow-xs'
798 |       }`}>
799 |         <div className="flex items-center gap-3 min-w-0">
800 |           <Link
801 |             to={`/trainee/courses/${courseId}`}
802 |             className={`w-9 h-9 rounded-xl flex items-center justify-center transition-all shrink-0 border ${
803 |               isDarkMode
804 |                 ? 'bg-slate-800/80 border-slate-700 hover:border-cyan-400 hover:bg-slate-700 text-slate-200'
805 |                 : 'bg-slate-100/90 border-slate-200 hover:border-cyan-400 hover:bg-cyan-50 text-slate-700 hover:text-cyan-70
0'
806 |             }`}
807 |             title="Back to Course Overview"
808 |           >
809 |             <ArrowLeft className="w-4 h-4" />
810 |           </Link>
811 |           <div className="min-w-0">
812 |             <div className="flex items-center gap-2">
813 |               <Badge className={`text-[10px] font-bold uppercase tracking-wider ${
814 |                 isDarkMode
815 |                   ? 'bg-cyan-500/20 text-cyan-300 border-cyan-500/30'
816 |                   : 'bg-cyan-50 text-cyan-700 border-cyan-200'
817 |               }`}>
818 |                 Module {activeModuleIndex + 1} of {modules.length}
819 |               </Badge>
```

```
              <h1 className={`text-xs sm:text-sm font-bold truncate max-w-xs sm:max-w-md md:max-w-lg ${
                isDarkMode ? 'text-white' : 'text-slate-900'
              }`}>
                {course.title}
              </h1>
            </div>
            <p className={`text-xs font-medium truncate ${isDarkMode ? 'text-slate-400' : 'text-slate-500'}`}>
              {currentModule?.title || 'Learning Module'}
            </p>
          </div>
        </div>

        {/* Center/Right: Theme Toggle, Progress & Module Drawer Toggle */}
        <div className="flex items-center gap-2.5 sm:gap-3 shrink-0">
          {/* Theme Switcher Toggle */}
          <button
            onClick={toggleTheme}
            className={`p-2 rounded-xl border flex items-center gap-1.5 transition-all text-xs font-semibold ${
              isDarkMode
                ? 'bg-slate-800/90 border-slate-700 text-amber-300 hover:bg-slate-700 hover:border-amber-400/50'
                : 'bg-slate-100/90 border-slate-200 text-slate-700 hover:bg-cyan-50 hover:text-cyan-700 hover:border-cyan-300'
            }`}
            title={isDarkMode ? 'Switch to Radiant Light Theme' : 'Switch to Midnight Dark Theme'}
          >
            {isDarkMode ? (
              <>
                <Sun className="w-4 h-4 text-amber-400" />
                <span className="hidden md:inline text-[11px]">Light Mode</span>
              </>
            ) : (
              <>
                <Moon className="w-4 h-4 text-cyan-700" />
                <span className="hidden md:inline text-[11px]">Dark Mode</span>
              </>
            )}
          </button>

          {/* Progress Indicator */}
          <div className={`hidden sm:flex items-center gap-3 rounded-xl px-3 py-1.5 border transition-all ${
            isDarkMode ? 'bg-slate-800/80 border-slate-700' : 'bg-slate-50 border-slate-200'
          }`}>
            <div className="text-right">
              <p className={`text-[11px] font-bold ${isDarkMode ? 'text-cyan-300' : 'text-cyan-700'}`}>{progressPercent}% Completed</p>
              <p className={`text-[10px] font-medium ${isDarkMode ? 'text-slate-400' : 'text-slate-500'}`}>
                {totalCompleted} / {modules.length} Modules
              </p>
            </div>
            <div className={`w-20 sm:w-24 h-2 rounded-full overflow-hidden border ${isDarkMode ? 'bg-slate-900 border-slate-700' : 'bg-slate-200 border-slate-200'}`}>
              <motion.div
                initial={{ width: 0 }}
                animate={{ width: `${progressPercent}%` }}
                transition={{ duration: 0.6 }}
                className="h-full bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 rounded-full"
              />
            </div>
          </div>

          {/* Certificate Download Button */}
          <Button
            onClick={handleDownloadCertificate}
            disabled={downloadingCert}
            className="text-xs font-bold rounded-xl px-3.5 py-2 flex items-center gap-1.5 shadow-xs bg-gradient-to-r from-emerald-600 via-teal-600 to-cyan-600 hover:from-emerald-700 hover:to-cyan-700 text-white cursor-pointer transition-all hover:scale-105 active:scale-95 disabled:opacity-60"
            title="Download Course Certificate"
          >
            {downloadingCert ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Award className="w-3.5 h-3.5 text-amber-300" />}
            <span className="hidden sm:inline">{downloadingCert ? 'Generating...' : 'Certificate'}</span>
          </Button>

          {/* Syllabus Drawer Trigger */}
          <Button
            onClick={() => setIsSidebarOpen(!isSidebarOpen)}
            className={`text-xs font-bold rounded-xl px-3.5 py-2 flex items-center gap-2 shadow-xs border transition-all ${
              isDarkMode
```

```
                ? 'bg-cyan-950/60 hover:bg-cyan-900/80 border-cyan-500/40 text-cyan-300'
                : 'bg-cyan-50 hover:bg-cyan-100/80 border-cyan-200 text-cyan-800'
            }`}
          >
            <Layers className="w-3.5 h-3.5" />
            <span className="hidden sm:inline">Syllabus</span>
            <span className={`text-[10px] px-1.5 py-0.5 rounded-md font-extrabold ${
              isDarkMode ? 'bg-cyan-500/30 text-cyan-200' : 'bg-cyan-200/80 text-cyan-900'
            }`}>
              {modules.length}
            </span>
          </Button>
        </div>
      </header>

      {/* ── MAIN LEARNING STAGE ──────────────────────────── */}
      <div className="flex-1 flex overflow-hidden relative">
        {/* Module Content Area */}
        <main className="flex-1 overflow-y-auto px-4 sm:px-6 lg:px-12 py-8 max-w-4xl mx-auto w-full space-y-6">
          <div ref={contentTopRef} />

          {/* Interactive Module Step Tracker Bar */}
          {modules.length > 1 && (
            <div className={`p-2.5 sm:p-3 rounded-2xl border transition-all ${
              isDarkMode ? 'bg-[#071124]/90 border-slate-800' : 'bg-white border-slate-200/90 shadow-xs'
            }`}>
              <div className="flex items-center gap-2 overflow-x-auto pb-0.5 scrollbar-none">
                {modules.map((m: any, mIdx: number) => {
                  const isModCompleted = completedModules.includes(m.id) || mIdx < completedModules.length
                  const isModUnlocked = mIdx === 0 || completedModules.includes(modules[mIdx - 1]?.id) || mIdx <= completedMo
dules.length
                  const isModActive = mIdx === activeModuleIndex

                  return (
                    <button
                      key={m.id || mIdx}
                      disabled={!isModUnlocked}
                      onClick={() => setActiveModuleIndex(mIdx)}
                      className={`flex items-center gap-2 px-3 py-1.5 rounded-xl text-xs font-bold shrink-0 transition-all ${
                        isModActive
                          ? isDarkMode
                            ? 'bg-gradient-to-r from-cyan-600 to-blue-600 text-white shadow-md shadow-cyan-500/25 ring-2 ring
-cyan-400/50'
                            : 'bg-gradient-to-r from-cyan-600 to-blue-600 text-white shadow-md shadow-cyan-600/25 ring-2 ring
-cyan-400/50'
                          : isModCompleted
                          ? isDarkMode
                            ? 'bg-emerald-950/50 border border-emerald-500/40 text-emerald-300 hover:bg-emerald-900/60'
                            : 'bg-emerald-50 border border-emerald-300 text-emerald-800 hover:bg-emerald-100'
                          : isModUnlocked
                          ? isDarkMode
                            ? 'bg-slate-800/80 border border-slate-700 text-slate-300 hover:bg-slate-700'
                            : 'bg-slate-100 border border-slate-200 text-slate-700 hover:bg-slate-200'
                          : isDarkMode
                          ? 'bg-slate-900/40 border border-slate-800/40 text-slate-600 cursor-not-allowed opacity-60'
                          : 'bg-slate-100/50 border border-slate-200/50 text-slate-400 cursor-not-allowed opacity-60'
                      }`}
                      title={m.title}
                    >
                      <span className={`w-4 h-4 rounded-full text-[10px] font-black flex items-center justify-center shrink-0
${
                        isModActive
                          ? 'bg-white text-cyan-700'
                          : isModCompleted
                          ? isDarkMode ? 'bg-emerald-500 text-black' : 'bg-emerald-600 text-white'
                          : 'bg-slate-400/20 text-current'
                      }`}>
                        {isModCompleted ? '✓' : !isModUnlocked ? '🔒' : mIdx + 1}
                      </span>
                      <span className="truncate max-w-[120px] sm:max-w-[160px]">
                        Module {mIdx + 1}
                      </span>
                    </button>
                  )
                })}
              </div>
            </div>
          )}

```

```
          {/* Signature Capacity Connect Module Banner Card */}
          <div className="bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white p-6 sm:p-8 rounded-3xl border border-cyan-500/30 shadow-xl relative overflow-hidden">
            <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />
            <div className="absolute bottom-0 left-0 w-64 h-64 bg-sky-600/15 rounded-full blur-2xl pointer-events-none" />

            <div className="relative z-10 space-y-3">
              <div className="flex flex-wrap items-center justify-between gap-2">
                <span className="bg-white/10 backdrop-blur-md px-3 py-1 rounded-full text-xs font-bold text-amber-300 uppercase tracking-wider border border-white/15">
                  Module {activeModuleIndex + 1}
                </span>
                {isCurrentCompleted ? (
                  <Badge className="bg-emerald-500/20 text-emerald-300 border-emerald-500/30 text-xs font-bold px-3 py-1 flex items-center gap-1.5">
                    <CheckCircle2 className="w-3.5 h-3.5 text-emerald-400" />
                    Completed {quizResult?.score !== undefined ? `(${quizResult.score}%)` : ''}
                  </Badge>
                ) : (
                  <Badge className="bg-amber-500/20 text-amber-300 border-amber-500/30 text-xs font-bold px-3 py-1 flex items-center gap-1.5">
                    <Sparkles className="w-3.5 h-3.5 text-amber-400" /> In Progress
                  </Badge>
                )}
              </div>

              <h2 className="text-2xl sm:text-3xl font-extrabold text-white leading-tight tracking-tight">
                {currentModule?.title || 'Learning Module'}
              </h2>

              {currentModule?.description && (
                <p className="text-sm text-slate-300 leading-relaxed font-normal pt-1">
                  {currentModule.description}
                </p>
              )}

              <div className="flex flex-wrap items-center gap-4 pt-2 text-xs text-slate-300 font-medium">
                <span className="flex items-center gap-1.5">
                  <Layers className="w-3.5 h-3.5 text-cyan-400" />
                  {sequentialItems.length} sequential learning step{sequentialItems.length !== 1 ? 's' : ''}
                </span>
                {allQuizQuestions.length > 0 && (
                  <span className="flex items-center gap-1.5 text-amber-300 font-semibold">
                    <HelpCircle className="w-3.5 h-3.5 text-amber-400" />
                    {allQuizQuestions.length} Quiz Checkpoint{allQuizQuestions.length !== 1 ? 's' : ''}
                  </span>
                )}
              </div>
            </div>
          </div>

          {/* Sequential Step Items */}
          {sequentialItems.length > 0 ? (
            <div className="space-y-5">
              {sequentialItems.map((item: any, sIdx: number) => {
                const isQuiz = item.type === 'quiz'
                const quizData: ResolvedQuizQuestion | null = isQuiz ? (item.quiz_data || null) : null

                const qIdx = isQuiz
                  ? allQuizQuestions.findIndex(q => q.id === quizData?.id || q.question === quizData?.question)
                  : -1

                const quizQuestionIndex = qIdx !== -1 ? qIdx : 0

                return (
                  <motion.div
                    key={item.id || sIdx}
                    initial={{ opacity: 0, y: 12 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ delay: sIdx * 0.05 }}
                    className="space-y-3"
                  >
                    {/* TEXT CONTENT STEP */}
                    {item.type === 'text' && (
                      <div className={`rounded-2xl p-5 sm:p-6 space-y-3 border transition-all ${
                        isDarkMode
                          ? 'bg-[#0d1527] border-slate-800 text-slate-200 shadow-md'
                          : 'bg-white border-slate-200/90 text-slate-800 shadow-xs hover:border-cyan-300'
```

```
                      }`}>
                        <div className="flex items-center gap-2.5">
                          <span className="w-6 h-6 rounded-full bg-cyan-600 text-white text-xs font-black flex items-center j
ustify-center shrink-0">
                            {sIdx + 1}
                          </span>
                          <Badge className={`text-xs font-bold flex items-center gap-1 ${
                            isDarkMode
                              ? 'bg-cyan-500/20 text-cyan-300 border-cyan-500/40'
                              : 'bg-cyan-50 text-cyan-700 border-cyan-200'
                          }`}>
                            <AlignLeft className="w-3.5 h-3.5" /> Lesson Notes
                          </Badge>
                          <h3 className={`text-sm sm:text-base font-bold ${isDarkMode ? 'text-white' : 'text-slate-900'}`}>
                            {item.title}
                          </h3>
                        </div>
                        <div className={`pl-8 text-sm leading-relaxed whitespace-pre-wrap font-normal ${
                          isDarkMode ? 'text-slate-300' : 'text-slate-600'
                        }`}>
                          {item.content || 'Read and study the material carefully.'}
                        </div>
                      </div>
                    )}

                    {/* VIDEO LESSON STEP */}
                    {item.type === 'video' && (() => {
                      const totalSec = item.duration_seconds || (item.duration_minutes ? item.duration_minutes * 60 : 300)
                      const requiredSec = Math.ceil(totalSec * 0.9)
                      const watchedSec = videoWatchProgress[item.id] || 0
                      const pctWatched = Math.min(100, Math.round((watchedSec / totalSec) * 100))
                      const isRequirementMet = watchedSec >= requiredSec
                      const isPlaying = activePlayingVideoId === item.id

                      return (
                        <div className={`rounded-2xl p-5 sm:p-6 space-y-4 border transition-all ${
                          isDarkMode
                            ? 'bg-[#0d1527] border-blue-500/30 shadow-md'
                            : 'bg-white border-slate-200/90 shadow-xs hover:border-blue-300'
                        }`}>
                          <div className="flex flex-wrap items-center justify-between gap-2">
                            <div className="flex items-center gap-2.5">
                              <span className="w-6 h-6 rounded-full bg-blue-600 text-white text-xs font-black flex items-cent
er justify-center shrink-0">
                                {sIdx + 1}
                              </span>
                              <Badge className={`text-xs font-bold flex items-center gap-1 ${
                                isDarkMode
                                  ? 'bg-blue-500/20 text-blue-300 border-blue-500/40'
                                  : 'bg-blue-50 text-blue-700 border-blue-200'
                              }`}>
                                <Video className="w-3.5 h-3.5" /> Video Lesson
                              </Badge>
                              <Badge className={`text-xs font-bold flex items-center gap-1 ${
                                isDarkMode
                                  ? 'bg-slate-800 text-slate-300 border-slate-700'
                                  : 'bg-slate-100 text-slate-700 border-slate-200'
                              }`}>
                                <Clock className="w-3 h-3 text-blue-500" /> {formatDuration(totalSec)}
                              </Badge>
                              <h3 className={`text-sm sm:text-base font-bold ${isDarkMode ? 'text-white' : 'text-slate-900'}
`}>
                                {item.title}
                              </h3>
                            </div>

                            <div className="flex items-center gap-2">
                              {isRequirementMet ? (
                                <Badge className="bg-emerald-500/20 text-emerald-600 dark:text-emerald-400 border-emerald-50
0/40 text-xs font-extrabold flex items-center gap-1">
                                  <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" /> 90% Watched (Complete)
                                </Badge>
                              ) : (
                                <Badge className="bg-amber-500/20 text-amber-600 dark:text-amber-400 border-amber-500/40 text
-xs font-extrabold flex items-center gap-1">
                                  <Clock className="w-3.5 h-3.5 text-amber-500" /> {pctWatched}% / 90% Required
                                </Badge>
                              )}
```

```
                                {item.url && (
                                  <a
                                    href={item.url}
                                    target="_blank"
                                    rel="noreferrer"
                                    className={`text-xs font-bold flex items-center gap-1 px-3 py-1 rounded-xl border transitio
n-all ${
                                      isDarkMode
                                        ? 'text-blue-300 bg-blue-950/60 border-blue-500/40 hover:bg-blue-900'
                                        : 'text-blue-700 bg-blue-50 border-blue-200 hover:bg-blue-100'
                                    }`}
                                  >
                                    <ExternalLink className="w-3 h-3" /> Tab
                                  </a>
                                )}
                              </div>
                            </div>

                            {/* 90% Watch Progress Bar */}
                            <div className={`p-3 rounded-xl border space-y-2 ${
                              isRequirementMet
                                ? isDarkMode ? 'bg-emerald-950/20 border-emerald-500/30' : 'bg-emerald-50/70 border-emerald-20
0'
                                : isDarkMode ? 'bg-slate-900/60 border-slate-800' : 'bg-slate-50 border-slate-200'
                            }`}>
                              <div className="flex flex-wrap items-center justify-between text-xs gap-2">
                                <span className={`font-semibold flex items-center gap-1.5 ${
                                  isDarkMode ? 'text-slate-300' : 'text-slate-700'
                                }`}>
                                  <Clock className="w-3.5 h-3.5 text-blue-500" />
                                  Watch Progress: <strong className={isRequirementMet ? 'text-emerald-500' : 'text-blue-600'}>
{formatDuration(watchedSec)}</strong> / {formatDuration(totalSec)}
                                </span>
                                <span className={`text-[11px] font-bold ${
                                  isRequirementMet
                                    ? 'text-emerald-600 dark:text-emerald-400'
                                    : 'text-slate-500'
                                }`}>
                                  {isRequirementMet ? '✓ 90% Requirement Met' : `Need ${formatDuration(requiredSec)} (90%) to u
nlock completion`}
                                </span>
                              </div>

                              <div className="relative w-full h-2.5 rounded-full bg-slate-200 dark:bg-slate-700 overflow-hidde
n">
                                <div
                                  className={`h-full transition-all duration-300 ${
                                    isRequirementMet
                                      ? 'bg-gradient-to-r from-emerald-500 to-teal-400'
                                      : 'bg-gradient-to-r from-blue-500 to-cyan-400'
                                  }`}
                                  style={{ width: `${pctWatched}%` }}
                                />
                                {/* 90% Marker */}
                                <div
                                  className="absolute top-0 bottom-0 w-0.5 bg-rose-500 z-10"
                                  style={{ left: '90%' }}
                                  title="90% Completion Threshold"
                                />
                              </div>

                              {/* External / YouTube Video Watch Session Toggle */}
                              <div className="flex items-center justify-between pt-1">
                                <p className="text-[11px] text-slate-500">
                                  {isRequirementMet
                                    ? 'Module unlock requirement satisfied.'
                                    : 'Keep video playing or use the active watch tracker.'}
                                </p>
                                <Button
                                  type="button"
                                  size="sm"
                                  variant={isPlaying ? 'destructive' : 'outline'}
                                  onClick={() => setActivePlayingVideoId(isPlaying ? null : item.id)}
                                  className={`h-7 px-3 text-[11px] font-bold rounded-lg gap-1.5 transition-all ${
                                    !isPlaying
                                      ? isDarkMode
                                        ? 'border-blue-500/40 text-blue-300 hover:bg-blue-950/60'
                                        : 'border-blue-300 text-blue-700 hover:bg-blue-50'
                                      : ''
```

```
                                }`}
                              >
                                {isPlaying ? (
                                  <>
                                    <Pause className="w-3 h-3" /> Tracking Active... (Pause)
                                  </>
                                ) : (
                                  <>
                                    <Play className="w-3 h-3" /> {watchedSec > 0 ? 'Resume Watch Tracker' : 'Start Watch Tracker'}
                                  </>
                                )}
                              </Button>
                            </div>
                          </div>

                          {/* Video Player */}
                          {item.url && getYouTubeEmbedUrl(item.url) ? (
                            <div className="relative w-full aspect-video rounded-xl overflow-hidden border border-slate-300 dark:border-slate-700 bg-black shadow-lg">
                              <iframe
                                src={getYouTubeEmbedUrl(item.url)!}
                                title={item.title}
                                allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
                                allowFullScreen
                                className="w-full h-full"
                              />
                            </div>
                          ) : item.url?.match(/\.(mp4|webm|mov)$/i) || item.previewUrl ? (
                            <video
                              controls
                              className="w-full rounded-xl border border-slate-300 dark:border-slate-700 bg-black shadow-lg"
                              onPlay={() => setActivePlayingVideoId(item.id)}
                              onPause={() => {
                                if (activePlayingVideoId === item.id) setActivePlayingVideoId(null)
                              }}
                              onTimeUpdate={e => {
                                updateVideoWatchProgress(item.id, e.currentTarget.currentTime, totalSec)
                              }}
                              onEnded={() => {
                                updateVideoWatchProgress(item.id, totalSec, totalSec)
                                if (activePlayingVideoId === item.id) setActivePlayingVideoId(null)
                              }}
                            >
                              <source src={item.url || item.previewUrl} />
                              Your browser does not support HTML5 video playback.
                            </video>
                          ) : null}

                          {item.content && (
                            <p className={`text-xs pl-8 leading-relaxed font-medium ${
                              isDarkMode ? 'text-slate-400' : 'text-slate-600'
                            }`}>
                              {item.content}
                            </p>
                          )}
                        </div>
                      )
                    })()}

                    {/* PHOTO / DIAGRAM STEP */}
                    {item.type === 'photo' && (
                      <div className={`rounded-2xl p-5 sm:p-6 space-y-4 border transition-all ${
                        isDarkMode
                          ? 'bg-[#0d1527] border-slate-800 text-slate-200 shadow-md'
                          : 'bg-white border-slate-200/90 text-slate-800 shadow-xs hover:border-cyan-300'
                      }`}>
                        <div className="flex flex-wrap items-center justify-between gap-2">
                          <div className="flex items-center gap-2.5">
                            <span className="w-6 h-6 rounded-full bg-cyan-600 text-white text-xs font-black flex items-center justify-center shrink-0">
                              {sIdx + 1}
                            </span>
                            <Badge className={`text-xs font-bold flex items-center gap-1 ${
                              isDarkMode
                                ? 'bg-cyan-500/20 text-cyan-300 border-cyan-500/40'
                                : 'bg-cyan-50 text-cyan-700 border-cyan-200'
                            }`}>
```

```
                                <ImageIcon className="w-3.5 h-3.5" /> Diagram & Visual
                              </Badge>
                              <h3 className={`text-sm sm:text-base font-bold ${isDarkMode ? 'text-white' : 'text-slate-900'}`}>
                                {item.title}
                              </h3>
                            </div>
                            {(item.previewUrl || item.url) && (
                              <button
                                type="button"
                                onClick={() => {
                                  setPreviewMaterial({ file_name: item.title, material_type: 'image' })
                                  setPreviewUrl(item.previewUrl || item.url)
                                }}
                                className={`text-xs font-semibold px-3 py-1 rounded-xl border flex items-center gap-1.5 transit
ion-all ${
                                  isDarkMode
                                    ? 'bg-slate-800 text-slate-300 hover:bg-slate-700 border-slate-700'
                                    : 'bg-slate-100 text-slate-700 hover:bg-cyan-50 hover:text-cyan-700 border-slate-200'
                                }`}
                              >
                                <Maximize2 className="w-3.5 h-3.5" /> Expand
                              </button>
                            )}
                          </div>

                          {(item.previewUrl || item.url) && (
                            <div
                              onClick={() => {
                                setPreviewMaterial({ file_name: item.title, material_type: 'image' })
                                setPreviewUrl(item.previewUrl || item.url)
                              }}
                              className={`relative rounded-2xl overflow-hidden border p-4 sm:p-6 flex flex-col items-center jus
tify-center cursor-pointer group transition-all ${
                                isDarkMode
                                  ? 'bg-[#070D1A] border-slate-800 hover:border-cyan-500/50'
                                  : 'bg-gradient-to-b from-slate-50 to-slate-100/70 border-slate-200 hover:border-cyan-400'
                              }`}
                            >
                              {/* Inner white presentation matte for crisp diagram legibility */}
                              <div className="bg-white rounded-xl p-2.5 sm:p-3 shadow-md border border-slate-200/80 max-w-full
flex items-center justify-center">
                                <img
                                  src={item.previewUrl || item.url}
                                  alt={item.title}
                                  className="w-auto max-h-80 sm:max-h-96 object-contain rounded-lg group-hover:scale-[1.01] tra
nsition-transform duration-200"
                                />
                              </div>

                              <div className="absolute inset-0 bg-slate-950/40 opacity-0 group-hover:opacity-100 transition-opa
city flex items-center justify-center text-white font-bold text-xs gap-1.5 backdrop-blur-xs rounded-2xl">
                                <Eye className="w-4 h-4 text-cyan-300" /> Click to Expand High-Res Image
                              </div>
                            </div>
                          )}

                          {item.content && (
                            <p className={`text-xs pl-8 leading-relaxed font-medium ${
                              isDarkMode ? 'text-slate-400' : 'text-slate-600'
                            }`}>
                              {item.content}
                            </p>
                          )}
                        </div>
                      )}

                      {/* EXTERNAL LINK STEP */}
                      {item.type === 'link' && (
                        <div className={`rounded-2xl p-4 sm:p-5 flex items-center justify-between gap-3 border transition-all
${
                          isDarkMode
                            ? 'bg-[#0d1527] border-emerald-500/30 shadow-md'
                            : 'bg-white border-slate-200/90 shadow-xs hover:border-emerald-300'
                        }`}>
                          <div className="flex items-start gap-3 min-w-0">
                            <span className="w-6 h-6 rounded-full bg-emerald-600 text-white text-xs font-black flex items-cente
r justify-center shrink-0 mt-0.5">
                              {sIdx + 1}
                            </span>
```

```
                            <div className="min-w-0">
                              <div className="flex items-center gap-2">
                                <Badge className={`text-xs font-bold ${
                                  isDarkMode
                                    ? 'bg-emerald-500/20 text-emerald-300 border-emerald-500/40'
                                    : 'bg-emerald-50 text-emerald-700 border-emerald-200'
                                }`}>
                                  <ExternalLink className="w-3 h-3 mr-1" /> External Resource
                                </Badge>
                                <h4 className={`text-xs sm:text-sm font-bold truncate ${
                                  isDarkMode ? 'text-white' : 'text-slate-900'
                                }`}>
                                  {item.title}
                                </h4>
                              </div>
                              {item.content && (
                                <p className={`text-xs mt-1 truncate ${isDarkMode ? 'text-slate-400' : 'text-slate-600'}`}>{ite
m.content}</p>
                              )}
                              {item.url && (
                                <p className="text-[11px] text-emerald-600 dark:text-emerald-400 truncate font-medium mt-0.5">
                                  {item.url}
                                </p>
                              )}
                            </div>
                          </div>
                          {item.url && (
                            <a
                              href={item.url}
                              target="_blank"
                              rel="noreferrer"
                              className={`h-9 px-3.5 rounded-xl border text-xs font-bold flex items-center gap-1.5 shrink-0 tra
nsition-all ${
                                isDarkMode
                                  ? 'bg-emerald-950/80 border-emerald-500/40 hover:bg-emerald-900 text-emerald-300'
                                  : 'bg-emerald-50 border-emerald-200 hover:bg-emerald-100 text-emerald-700'
                              }`}
                            >
                              <span>Open</span>
                              <ExternalLink className="w-3.5 h-3.5" />
                            </a>
                          )}
                        </div>
                      )}

                      {/* QUIZ STEP */}
                      {isQuiz && quizData && (
                        <div className={`rounded-2xl p-5 sm:p-6 space-y-4 border-2 shadow-sm transition-all ${
                          isDarkMode
                            ? 'bg-gradient-to-br from-[#16182B] via-[#0d1527] to-[#040814] border-amber-500/40 shadow-lg'
                            : 'bg-white border-amber-300/80'
                        }`}>
                          <div className="flex flex-wrap items-center gap-2.5">
                            <span className="w-6 h-6 rounded-full bg-amber-500 text-black text-xs font-black flex items-center
justify-center shrink-0">
                              {sIdx + 1}
                            </span>
                            <Badge className={`text-xs font-bold flex items-center gap-1 ${
                              isDarkMode
                                ? 'bg-amber-500/20 text-amber-300 border-amber-500/50'
                                : 'bg-amber-50 text-amber-800 border-amber-200'
                            }`}>
                              <HelpCircle className="w-3.5 h-3.5" /> Quiz Question {quizQuestionIndex + 1}
                            </Badge>
                          </div>

                          {/* Full Question Text */}
                          <p className={`text-sm sm:text-base font-bold pl-8 leading-snug ${isDarkMode ? 'text-white' : 'text-s
late-900'}`}>
                            {quizData.question}
                          </p>

                          {/* Options List (A, B, C, D) */}
                          <div className="space-y-2.5 pl-8 pt-1">
                            {quizData.options?.map((opt: string, optIdx: number) => {
                              const isSelected = userAnswers[quizQuestionIndex] === optIdx
                              const isGraded = quizResult?.submitted
                              const isOptionCorrect = optIdx === (quizData.correct_option ?? 0)
                              const isUserSelectionCorrect = isSelected && isOptionCorrect
```

```
                            const isUserSelectionWrong = isSelected && !isOptionCorrect

                            return (
                              <button
                                key={optIdx}
                                type="button"
                                disabled={quizResult?.passed}
                                onClick={() =>
                                  handleSelectQuizAnswer(currentModule.id, quizQuestionIndex, optIdx)
                                }
                                className={`w-full flex items-center gap-3.5 p-3.5 rounded-xl border text-xs sm:text-sm text-
left transition-all ${
                                  isGraded
                                    ? isUserSelectionCorrect
                                      ? isDarkMode
                                        ? 'bg-emerald-950/80 border-emerald-400 text-emerald-200 font-bold shadow-sm'
                                        : 'bg-emerald-50 border-emerald-500 text-emerald-900 font-bold shadow-sm'
                                      : isUserSelectionWrong
                                      ? isDarkMode
                                        ? 'bg-rose-950/80 border-rose-400 text-rose-200 font-semibold shadow-sm'
                                        : 'bg-rose-50 border-rose-500 text-rose-900 font-semibold shadow-sm'
                                      : isDarkMode
                                      ? 'bg-[#080E1A]/60 border-slate-800 text-slate-500 opacity-60'
                                      : 'bg-slate-50 border-slate-200 text-slate-400 opacity-60'
                                    : isSelected
                                    ? isDarkMode
                                      ? 'bg-cyan-500/20 border-cyan-400 text-cyan-200 font-bold ring-2 ring-cyan-500/30'
                                      : 'bg-cyan-50/90 border-cyan-500 text-cyan-950 font-bold ring-2 ring-cyan-400/30'
                                    : isDarkMode
                                    ? 'bg-[#071124]/90 border-slate-700/80 text-slate-300 hover:border-cyan-400 hover:bg-[#0c
1a36]'

                                    : 'bg-white border-slate-200 text-slate-700 hover:bg-cyan-50/50 hover:border-cyan-300'
                                }`}
                              >
                                <span
                                  className={`w-6 h-6 rounded-full text-xs font-black flex items-center justify-center shrink
-0 ${
                                    isGraded && isUserSelectionCorrect
                                      ? 'bg-emerald-500 text-black'
                                      : isGraded && isUserSelectionWrong
                                      ? 'bg-rose-500 text-white'
                                      : isSelected
                                      ? isDarkMode ? 'bg-cyan-400 text-black' : 'bg-cyan-600 text-white'
                                      : isDarkMode ? 'bg-slate-800 text-slate-400' : 'bg-slate-100 text-slate-700'
                                  }`}
                                >
                                  {String.fromCharCode(65 + optIdx)}
                                </span>
                                <span className="flex-1 font-medium">{opt}</span>
                                {isGraded && isUserSelectionCorrect && (
                                  <span className={`text-[11px] font-bold px-2.5 py-0.5 rounded-full border shrink-0 ${
                                    isDarkMode
                                      ? 'text-emerald-300 bg-emerald-900/60 border-emerald-500/40'
                                      : 'text-emerald-800 bg-emerald-100 border-emerald-300'
                                  }`}>
                                    ✓ Correct
                                  </span>
                                )}
                                {isGraded && isUserSelectionWrong && (
                                  <span className={`text-[11px] font-bold px-2.5 py-0.5 rounded-full border shrink-0 ${
                                    isDarkMode
                                      ? 'text-rose-300 bg-rose-900/60 border-rose-500/40'
                                      : 'text-rose-800 bg-rose-100 border-rose-300'
                                  }`}>
                                    ✗ Incorrect Selection
                                  </span>
                                )}
                              </button>
                            )
                          })}
                        </div>

                        {/* Explanation ONLY shown when answered correctly with clear visible contrast */}
                        {quizResult?.submitted &&
                          userAnswers[quizQuestionIndex] === (quizData.correct_option ?? 0) &&
                          quizData.explanation && (
                            <div className={`mt-3 ml-8 p-3.5 rounded-xl border text-xs sm:text-sm leading-relaxed transition-
all ${
                              isDarkMode
```

```
                                ? 'bg-emerald-950/40 border-emerald-500/40 text-slate-200'
                                : 'bg-emerald-50/90 border-emerald-200 text-slate-800 shadow-sm'
                            }`}>
                              <span className={`font-bold ${isDarkMode ? 'text-emerald-400' : 'text-emerald-800'}`}>
                                Explanation:{" "}
                              </span>
                              <span>{quizData.explanation}</span>
                            </div>
                          )}
                        </div>
                      )}
                    </motion.div>
                  )
                })}
              </div>
            ) : (
              <div className={`p-8 rounded-3xl text-center space-y-2 border ${
                isDarkMode ? 'bg-[#0d1527] border-slate-800' : 'bg-white border-slate-200'
              }`}>
                <Layers className="w-8 h-8 text-slate-400 mx-auto" />
                <h3 className={`text-sm font-bold ${isDarkMode ? 'text-white' : 'text-slate-800'}`}>No content items in this mo
dule yet</h3>
                <p className="text-xs text-slate-500">The trainer has not added learning steps to this module yet.</p>
              </div>
            )}

            {/* ── MODULE COMPLETION & TRANSITION CONTROLS ──────── */}
            <div className={`pt-6 border-t space-y-4 ${isDarkMode ? 'border-slate-800' : 'border-slate-200'}`}>
              {allQuizQuestions.length > 0 ? (
                <div className={`rounded-2xl p-5 sm:p-6 space-y-4 border ${
                  isDarkMode ? 'bg-[#0d1527] border-cyan-500/30 shadow-lg' : 'bg-white border-slate-200/90 shadow-sm'
                }`}>
                  <div className="flex flex-wrap items-center justify-between gap-3">
                    <div className="flex items-center gap-2.5">
                      <div className="w-10 h-10 rounded-xl bg-amber-500/10 border border-amber-500/30 flex items-center justify
-center text-amber-500">
                        <HelpCircle className="w-5 h-5" />
                      </div>
                      <div>
                        <h4 className={`text-sm font-bold ${isDarkMode ? 'text-white' : 'text-slate-900'}`}>
                          Module Mastery Checkpoint ({allQuizQuestions.length} Question{allQuizQuestions.length !== 1 ? 's' :
''})
                        </h4>
                      </div>
                    </div>

                    {quizResult && (
                      <Badge
                        className={`text-xs font-extrabold px-3 py-1.5 ${
                          quizResult.passed
                            ? 'bg-emerald-500 text-black shadow-lg shadow-emerald-500/20'
                            : 'bg-rose-500 text-white'
                        }`}
                      >
                        {quizResult.passed ? '✓ PASSED: ' : '✗ SCORE: '}
                        {quizResult.score}%
                      </Badge>
                    )}
                  </div>

                  {/* Failed Alert with Retake Button - Clean High Contrast in both Light & Dark modes */}
                  {quizResult && !quizResult.passed && (
                    <div className={`p-4 sm:p-5 rounded-2xl flex flex-col sm:flex-row sm:items-center justify-between gap-4 bor
der transition-all ${
                      isDarkMode
                        ? 'bg-rose-950/60 border-rose-500/50 text-rose-100 shadow-lg shadow-rose-950/40'
                        : 'bg-rose-50 border-rose-200 text-rose-950 shadow-sm'
                    }`}>
                      <div className="flex items-center gap-3">
                        <div className={`w-9 h-9 rounded-xl flex items-center justify-center shrink-0 ${
                          isDarkMode ? 'bg-rose-900/70 text-rose-300' : 'bg-rose-100 text-rose-700'
                        }`}>
                          <AlertCircle className="w-5 h-5 shrink-0" />
                        </div>
                        <div>
                          <p className={`text-sm sm:text-base font-bold ${isDarkMode ? 'text-rose-100' : 'text-rose-950'}`}>
                            You scored {quizResult.score}%
                          </p>
                          <p className={`text-xs sm:text-sm font-medium ${isDarkMode ? 'text-rose-200/90' : 'text-rose-700'}`}>
```

```
                          Review the lesson materials and retake the quiz to proceed.
                        </p>
                      </div>
                    </div>
                    <Button
                      type="button"
                      onClick={() => handleRetakeModuleQuiz(currentModule.id)}
                      className="bg-rose-600 hover:bg-rose-700 active:scale-95 text-white text-xs sm:text-sm font-bold rounde
d-xl px-4 py-2.5 shrink-0 gap-2 shadow-md shadow-rose-600/30 transition-all"
                    >
                      <RotateCcw className="w-4 h-4" /> Retake Quiz
                    </Button>
                  </div>
                )}

                {/* Completed Module State vs In-Progress Submit Button */}
                {isCurrentCompleted || quizResult?.passed ? (
                  <div className={`p-4 sm:p-5 rounded-2xl flex flex-col sm:flex-row sm:items-center justify-between gap-4 bor
der transition-all ${
                    isDarkMode
                      ? 'bg-emerald-950/40 border-emerald-500/40 text-emerald-100 shadow-md'
                      : 'bg-emerald-50/90 border-emerald-300 text-emerald-950 shadow-sm'
                  }`}>
                    <div className="flex items-center gap-3">
                      <div className={`w-9 h-9 rounded-xl flex items-center justify-center shrink-0 ${
                        isDarkMode ? 'bg-emerald-900/60 text-emerald-300' : 'bg-emerald-100 text-emerald-700'
                      }`}>
                        <CheckCircle2 className="w-5 h-5 shrink-0" />
                      </div>
                      <div>
                        <p className={`text-sm sm:text-base font-bold ${isDarkMode ? 'text-emerald-200' : 'text-emerald-950'}
`}>
                          Module Checkpoint Completed ✓
                        </p>
                        <p className={`text-xs sm:text-sm font-medium ${isDarkMode ? 'text-slate-300' : 'text-slate-600'}`}>
                          You have verified your mastery of this module. You can review your answers or continue forward.
                        </p>
                      </div>
                    </div>
                    <div className="flex items-center gap-2.5 shrink-0">
                      <Button
                        type="button"
                        onClick={() => handleRetakeModuleQuiz(currentModule.id)}
                        variant="outline"
                        className={`text-xs font-bold rounded-xl px-3.5 py-2 transition-all ${
                          isDarkMode
                            ? 'border-emerald-500/40 text-emerald-300 hover:bg-emerald-950/60'
                            : 'border-emerald-300 text-emerald-800 hover:bg-emerald-100'
                        }`}
                      >
                        <RotateCcw className="w-3.5 h-3.5 mr-1" /> Practice Again
                      </Button>
                    </div>
                  </div>
                ) : (
                  /* Submit Quiz Button */

                  <div className="space-y-2 pt-2">
                    {!currentVideoCheck.passed && (
                      <div className={`p-3 rounded-xl flex items-center gap-2 text-xs font-semibold ${
                        isDarkMode
                          ? 'bg-amber-950/60 text-amber-200 border border-amber-500/40'
                          : 'bg-amber-50 text-amber-900 border border-amber-200'
                      }`}>
                        <AlertCircle className="w-4 h-4 shrink-0 text-amber-500" />
                        <span>
                          <strong>Video Requirement:</strong> Watch at least 90% of the video lesson before submitting this q
uiz.
                        </span>
                      </div>
                    )}
                    <div className="flex items-center justify-end">
                      <Button
                        type="button"
                        onClick={() => handleSubmitModuleQuiz(currentModule)}
                        className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-7
00 text-white font-extrabold text-sm rounded-xl px-6 py-3 shadow-lg shadow-cyan-500/25 gap-2"
                      >
                        <CheckSquare className="w-4 h-4" />
```

```
                        Submit Module Quiz
                      </Button>
                    </div>
                  </div>
                )}
              </div>
            ) : (
              /* Content-only module completion button */

              <div className={`rounded-2xl p-5 flex flex-col sm:flex-row sm:items-center justify-between gap-3 border ${
                isDarkMode ? 'bg-[#0d1527] border-cyan-500/30' : 'bg-white border-slate-200/90 shadow-xs'
              }`}>
                <div>
                  <p className={`text-xs font-bold ${isDarkMode ? 'text-slate-200' : 'text-slate-800'}`}>
                    Reviewed all materials in Module {activeModuleIndex + 1}?
                  </p>
                  {!currentVideoCheck.passed && (
                    <p className="text-[11px] text-amber-600 dark:text-amber-400 font-semibold mt-0.5">
                      ⚠️ Please watch ≥90% of the video lesson in this module before completing.
                    </p>
                  )}
                </div>
                {!isCurrentCompleted ? (
                  <Button
                    type="button"
                    onClick={() => handleCompleteContentModule(currentModule)}
                    className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white fo
nt-bold text-xs rounded-xl px-5 py-2.5 shadow-md shadow-cyan-500/20 gap-2"
                  >
                    <CheckCircle2 className="w-4 h-4" /> Mark Module Completed →
                  </Button>
                ) : (
                  <span className="text-xs font-bold text-emerald-600 dark:text-emerald-400 flex items-center gap-1.5">
                    <CheckCircle2 className="w-4 h-4" /> Module Completed
                  </span>
                )}
              </div>
            )}

            {/* Bottom Navigation Buttons (Prev / Next Module) */}
            <div className="flex items-center justify-between gap-3 pt-4">
              <Button
                variant="outline"
                onClick={handlePrevModule}
                disabled={activeModuleIndex === 0}
                className={`rounded-xl px-4 py-2.5 font-bold text-xs gap-2 ${
                  isDarkMode
                    ? 'border-slate-700 text-slate-300 hover:bg-slate-800 disabled:opacity-40'
                    : 'border-slate-200 text-slate-700 hover:bg-slate-50 disabled:opacity-40'
                }`}
              >
                <ChevronLeft className="w-4 h-4" /> Previous Module
              </Button>

              {isLastModule ? (
                isCurrentCompleted || quizResult?.passed ? (
                  <div className="flex flex-wrap items-center gap-2">
                    {gradeBreakdown.hasFinalAssessment && !gradeBreakdown.finalAssessmentCompleted ? (
                      <>
                        <Button
                          onClick={() => navigate(`/trainee/courses/${courseId}/assessments/${gradeBreakdown.finalAssessmentI
d}`)}
                          className="bg-gradient-to-r from-purple-600 via-indigo-600 to-cyan-600 hover:from-purple-700 hover:
to-cyan-700 text-white font-black text-xs sm:text-sm rounded-xl px-6 py-3 shadow-xl shadow-purple-500/25 gap-2 hover:scale-105 activ
e:scale-95 transition-all animate-pulse cursor-pointer"
                        >
                          <Award className="w-4 h-4 text-amber-300" />
                          <span>Take Final Assessment (50% Grade) →</span>
                        </Button>
                        <Button
                          onClick={handleDownloadCertificate}
                          disabled={downloadingCert}
                          className="bg-gradient-to-r from-emerald-600 via-teal-600 to-cyan-600 hover:from-emerald-700 hover:
to-cyan-700 text-white font-black text-xs sm:text-sm rounded-xl px-5 py-3 shadow-md gap-2 cursor-pointer transition-all hover:scale-
105"
                        >
                          {downloadingCert ? <Loader2 className="w-4 h-4 animate-spin" /> : <Award className="w-4 h-4 text-am
ber-300" />}
                          {downloadingCert ? 'Generating...' : 'Download Certificate 🎓'}
```

```
                        </Button>
                      </>
                    ) : (
                      <>
                        <Button
                          onClick={handleDownloadCertificate}
                          disabled={downloadingCert}
                          className="bg-gradient-to-r from-amber-500 via-emerald-500 to-cyan-600 hover:from-amber-600 hover:to-cyan-700 text-white font-black text-xs sm:text-sm rounded-xl px-5 py-3 shadow-xl shadow-cyan-500/25 gap-2 hover:scale-105 active:scale-95 transition-all cursor-pointer"
                        >
                          {downloadingCert ? <Loader2 className="w-4 h-4 animate-spin" /> : <Award className="w-4 h-4" />}
                          {downloadingCert ? 'Generating...' : 'Download Certificate 🎓'}
                        </Button>
                        <Button
                          onClick={() => navigate(`/trainee/courses/${courseId}`)}
                          className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold text-xs sm:text-sm rounded-xl px-5 py-3 shadow-md gap-2 cursor-pointer"
                        >
                          <span>View Course Assessments & Progress →</span>
                        </Button>
                      </>
                    )}
                  </div>
                ) : (
                  <Button
                    disabled
                    className="bg-slate-300 dark:bg-slate-800 text-slate-500 rounded-xl px-6 py-2.5 text-xs font-bold"
                  >
                    Complete Final Module to Unlock Final Assessment
                  </Button>
                )
              ) : (
                <Button
                  onClick={handleNextModule}
                  disabled={!canGoNext}
                  className={`font-black text-xs sm:text-sm rounded-xl px-6 py-3 flex items-center gap-2 transition-all ${
                    canGoNext
                      ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white shadow-lg shadow-cyan-500/30 hover:scale-[1.02] active:scale-95'
                      : 'bg-slate-200 dark:bg-slate-800/80 border border-slate-300 dark:border-slate-700 text-slate-400 dark:text-slate-500 cursor-not-allowed'
                  }`}
                >
                  <span>Next Module</span>
                  <ChevronRight className="w-4 h-4" />
                </Button>
              )}
            </div>
          </div>
        </main>

        {/* ── MODULES SYLLABUS DRAWER / SIDEBAR ──────────────── */}
        <AnimatePresence>
          {isSidebarOpen && (
            <>
              {/* Overlay */}
              <motion.div
                initial={{ opacity: 0 }}
                animate={{ opacity: 1 }}
                exit={{ opacity: 0 }}
                onClick={() => setIsSidebarOpen(false)}
                className="fixed inset-0 bg-black/60 backdrop-blur-xs z-40 lg:hidden"
              />

              {/* Drawer */}
              <motion.aside
                initial={{ x: '100%' }}
                animate={{ x: 0 }}
                exit={{ x: '100%' }}
                transition={{ duration: 0.25, ease: 'easeInOut' }}
                className={`fixed lg:static top-0 right-0 bottom-0 w-80 sm:w-96 border-l z-50 flex flex-col shadow-2xl ${
                  isDarkMode
                    ? 'bg-[#071124] border-cyan-500/20'
                    : 'bg-white border-slate-200'
                }`}
              >
                <div className={`p-4 border-b flex items-center justify-between ${
                  isDarkMode ? 'border-cyan-500/20' : 'border-slate-200'
```

```
                }`}>
                  <div className="flex items-center gap-2">
                    <Layers className="w-4 h-4 text-cyan-600 dark:text-cyan-400" />
                    <h3 className={`text-sm font-bold ${isDarkMode ? 'text-white' : 'text-slate-900'}`}>Course Syllabus</h3>
                  </div>
                  <button
                    onClick={() => setIsSidebarOpen(false)}
                    className="w-8 h-8 rounded-lg text-slate-400 hover:text-slate-700 dark:hover:text-white flex items-center
justify-center transition-colors"
                  >
                    <X className="w-4 h-4" />
                  </button>
                </div>

                <div className="flex-1 overflow-y-auto p-4 space-y-2.5">
                  {modules.map((m: any, idx: number) => {
                    const isModCompleted = completedModules.includes(m.id)
                    const isModUnlocked =
                      idx === 0 || completedModules.includes(modules[idx - 1]?.id)
                    const isModActive = idx === activeModuleIndex
                    const mQuizScore = moduleQuizScores[m.id]

                    return (
                      <button
                        key={m.id || idx}
                        disabled={!isModUnlocked}
                        onClick={() => {
                          setActiveModuleIndex(idx)
                          setIsSidebarOpen(false)
                        }}
                        className={`w-full text-left p-3.5 rounded-xl border transition-all flex items-start gap-3 ${
                          isModActive
                            ? isDarkMode
                              ? 'bg-cyan-950/70 border-cyan-500/80 ring-2 ring-cyan-500/20'
                              : 'bg-cyan-50 border-cyan-300 ring-2 ring-cyan-200'
                            : isModCompleted
                            ? isDarkMode
                              ? 'bg-emerald-950/20 border-emerald-500/30 hover:bg-emerald-950/40'
                              : 'bg-emerald-50/60 border-emerald-200 hover:bg-emerald-100/50'
                            : isModUnlocked
                            ? isDarkMode
                              ? 'bg-slate-800/60 border-slate-700 hover:bg-slate-800'
                              : 'bg-slate-50 border-slate-200 hover:bg-slate-100'
                            : isDarkMode
                            ? 'bg-black/40 border-slate-800 text-slate-600 opacity-60 cursor-not-allowed'
                            : 'bg-slate-100/60 border-slate-200 text-slate-400 opacity-60 cursor-not-allowed'
                        }`}
                      >
                        <div
                          className={`w-6 h-6 rounded-lg text-xs font-black flex items-center justify-center shrink-0 mt-0.5
${
                            isModCompleted
                              ? 'bg-emerald-500 text-black'
                              : isModActive
                              ? 'bg-cyan-600 text-white'
                              : isModUnlocked
                              ? isDarkMode ? 'bg-slate-700 text-white' : 'bg-slate-300 text-slate-800'
                              : isDarkMode ? 'bg-slate-900 text-slate-600' : 'bg-slate-200 text-slate-400'
                          }`}
                        >
                          {isModCompleted ? (
                            <Check className="w-3.5 h-3.5" />
                          ) : !isModUnlocked ? (
                            <Lock className="w-3 h-3" />
                          ) : (
                            idx + 1
                          )}
                        </div>

                        <div className="flex-1 min-w-0">
                          <p
                            className={`text-xs font-bold truncate ${
                              isModActive
                                ? isDarkMode ? 'text-cyan-300' : 'text-cyan-950'
                                : isModCompleted
                                ? isDarkMode ? 'text-emerald-300' : 'text-emerald-900'
                                : isModUnlocked
                                ? isDarkMode ? 'text-white' : 'text-slate-800'
                                : isDarkMode ? 'text-slate-500' : 'text-slate-400'
```

```
                          }`}
                        >
                          {m.title}
                        </p>
                        <div className="flex items-center gap-2 mt-1">
                          {isModCompleted && (
                            <span className="text-[10px] text-emerald-600 dark:text-emerald-400 font-semibold">
                              ✓ Completed {mQuizScore?.score ? `(${mQuizScore.score}%)` : ''}
                            </span>
                          )}
                          {isModActive && !isModCompleted && (
                            <span className="text-[10px] text-cyan-600 dark:text-cyan-400 font-semibold animate-pulse">
                              ● In Progress
                            </span>
                          )}
                          {!isModUnlocked && (
                            <span className="text-[10px] text-slate-400 dark:text-slate-500 font-medium">
                              Locked (Pass Module {idx})
                            </span>
                          )}
                        </div>
                      </div>
                    </button>
                  )
                })}
              </div>
            </motion.aside>
          </>
        )}
      </AnimatePresence>
    </div>

    <MaterialPreviewDialog
      material={previewMaterial}
      previewUrl={previewUrl}
      onClose={() => {
        setPreviewMaterial(null)
        setPreviewUrl(null)
      }}
    />
  </div>
 )
}

```

src/features/courses/TraineeMyLearning.tsx TSX 16.7 KB 311 Lines

```tsx
import React, { useState } from 'react'
import { useQuery } from '@tanstack/react-query'
import { Link } from 'react-router-dom'
import { motion } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { DashboardShell } from '@/pages/Dashboards'
import { BookOpen, Compass, BookMarked, PlayCircle, CheckCircle2, Clock, Sparkles, FileCheck, User, BarChart3, Award, Loader2, Bell } from 'lucide-react'
import { Thumbnail } from '@/components/ui/Thumbnail'
import { toast } from 'sonner'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'

import { calculateCourseGradeBreakdown } from '@/lib/courseGrading'

const fadeUp = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.4 } },
}
const stagger = { hidden: {}, visible: { transition: { staggerChildren: 0.08 } } }

export function TraineeMyLearning() {
  const { user, profile } = useAuth()
  const [downloadingId, setDownloadingId] = useState<string | null>(null)

  const { data: enrollments, isLoading: isEnrollmentsLoading } = useQuery({
    queryKey: ['my_learning', profile?.id],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('enrollments')
        .select(`
          id,
          status,
          progress_percent,
          enrolled_at,
          course:courses!enrollments_course_id_fkey(
            id,
            title,
            status,
            course_type,
            thumbnail_path,
            duration_minutes,
            passing_score,
            certificate_template_url,
            has_certificate,
            trainer:trainers!courses_trainer_id_fkey(full_name)
          )
        `)
        .eq('user_id', profile!.id)
        .order('enrolled_at', { ascending: false })

      if (error) throw error
      return ((data || []) as any[]).filter((e: any) => e.course && e.course.status !== 'archived')
    },
    enabled: !!profile?.id
  })

  const courseIds = (enrollments as any[])?.map((e: any) => e.course?.id).filter(Boolean) || []

  const { data: allAssessments = [] } = useQuery({
    queryKey: ['my_learning_assessments', courseIds],
    queryFn: async () => {
      if (courseIds.length === 0) return []
      const { data, error } = await supabase
        .from('assessments')
        .select('id, title, assessment_type, course_id, passing_score')
        .in('course_id', courseIds)
        .eq('status', 'published')
      if (error) return []
      return data || []
    },
    enabled: courseIds.length > 0,
  })

  const { data: allAttempts = [] } = useQuery({
    queryKey: ['my_learning_attempts', profile?.id],
```

```
    queryFn: async () => {
      if (!profile?.id) return []
      const { data, error } = await supabase
        .from('assessment_attempts' as any)
        .select('*')
        .eq('user_id', profile.id)
      if (error) return []
      return (data || []) as any[]
    },
    enabled: !!profile?.id,
  })

  const isLoading = isEnrollmentsLoading

  const handleDownloadCertificate = async (enrollment: any, totalPercentage?: number) => {
    if (!profile || !enrollment.course) return
    const courseAssessments = allAssessments.filter((a: any) => a.course_id === enrollment.course?.id)
    const gradeBreakdown = calculateCourseGradeBreakdown({
      moduleProgressPercent: enrollment.progress_percent ?? 0,
      courseAssessments: courseAssessments as any,
      traineeAttempts: allAttempts as any,
      passingScore: enrollment.course?.passing_score ?? 50,
    })

    setDownloadingId(enrollment.id)
    try {
      const traineeName = profile.full_name || user?.email?.split('@')[0] || 'Trainee'
      const percentage = totalPercentage ? `${totalPercentage}%` : `${gradeBreakdown.totalScore}%`

      const { blob, fileName } = await generateTraineeCertificate(
        enrollment.course.certificate_template_url,
        {
          traineeName,
          traineeEmail: profile.email || user?.email,
          traineeId: user!.id,
          courseId: enrollment.course.id,
          courseTitle: enrollment.course.title,
          trainerName: enrollment.course.trainer?.full_name || 'Lead Trainer',
          percentage,
          completedAt: new Date().toISOString(),
        }
      )

      triggerFileDownload(blob, fileName)
      toast.success('Certificate downloaded successfully! 🎉')
    } catch (err: any) {
      console.error('Certificate generation error:', err)
      toast.error(err.message || 'Failed to download certificate')
    } finally {
      setDownloadingId(null)
    }
  }

  const inProgressCount = enrollments?.filter(e => e.status === 'in_progress' || e.status === 'enrolled').length || 0
  const completedCount = enrollments?.filter(e => e.status === 'completed').length || 0

  return (
    <DashboardShell
      title="My Learning"
      icon={BookOpen}
      navLinks={[
        { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
        { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
        { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
        { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
        { to: '/trainee/profile', label: 'Profile', icon: User },
      ]}
    >
      <div className="max-w-6xl space-y-6">
        {/* Header (Midnight Banner) */}
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex flex-col md:flex-row md:items-center
justify-between gap-4 bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white p-6 rounded-3xl border border-cyan-500/3
0 shadow-xl relative overflow-hidden">
          <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur
-3xl pointer-events-none" />

          <div className="relative z-10">
            <div className="flex items-center gap-2 mb-1">
```

```
152 |               <span className="px-2.5 py-0.5 rounded-full bg-white/10 text-amber-300 text-xs font-bold uppercase tracking-wide
r flex items-center gap-1 border border-white/15">
153 |                 <Sparkles className="w-3 h-3 text-amber-300" /> Learning Roadmap
154 |               </span>
155 |             </div>
156 |             <h2 className="text-2xl font-extrabold text-white tracking-tight">My Learning Journey</h2>
157 |             <p className="text-slate-300 text-sm">Track your progress and continue where you left off.</p>
158 |           </div>
159 |           <div className="flex items-center gap-3 relative z-10">
160 |             <div className="px-4 py-2 bg-white/10 backdrop-blur-md border border-white/20 rounded-2xl">
161 |               <span className="text-xs text-slate-300 font-medium">In Progress: </span>
162 |               <span className="text-sm font-extrabold text-cyan-400">{inProgressCount}</span>
163 |             </div>
164 |             <div className="px-4 py-2 bg-emerald-500/20 border border-emerald-500/30 rounded-2xl">
165 |               <span className="text-xs text-emerald-200 font-medium">Completed: </span>
166 |               <span className="text-sm font-extrabold text-emerald-400">{completedCount}</span>
167 |             </div>
168 |           </div>
169 |         </motion.div>
170 |
171 |         {isLoading ? (
172 |           <div className="space-y-4">
173 |             {[1, 2, 3].map(i => (
174 |               <div key={i} className="h-32 bg-white animate-pulse rounded-3xl border border-slate-200" />
175 |             ))}
176 |           </div>
177 |         ) : enrollments?.length === 0 ? (
178 |           <div className="p-16 text-center bg-white border border-slate-200 rounded-3xl shadow-sm">
179 |             <div className="w-16 h-16 bg-cyan-50 rounded-2xl flex items-center justify-center mx-auto mb-4 border border-cyan-
200">
180 |               <BookOpen className="w-8 h-8 text-cyan-600" />
181 |             </div>
182 |             <h3 className="text-lg font-bold text-slate-900 mb-1">Not enrolled in any courses yet</h3>
183 |             <p className="text-slate-500 text-sm mb-6 max-w-sm mx-auto">Explore the catalog to find a course and kickstart you
r learning journey.</p>
184 |             <Link to="/trainee/courses">
185 |               <button className="px-6 py-2.5 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white fon
t-bold text-sm shadow-md shadow-cyan-600/20 hover:scale-105 transition-all">
186 |                 Browse Courses
187 |               </button>
188 |             </Link>
189 |           </div>
190 |         ) : (
191 |           <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-4">
192 |             {enrollments?.map((enrollment: any) => {
193 |               const courseAssessments = allAssessments.filter((a: any) => a.course_id === enrollment.course?.id)
194 |               const gradeBreakdown = calculateCourseGradeBreakdown({
195 |                 moduleProgressPercent: enrollment.progress_percent ?? 0,
196 |                 courseAssessments: courseAssessments as any,
197 |                 traineeAttempts: allAttempts as any,
198 |                 passingScore: enrollment.course?.passing_score ?? 50,
199 |               })
200 |
201 |               const isCourseDone = Boolean(gradeBreakdown.isCompleted)
202 |               const isFinalPending = Boolean(gradeBreakdown.hasFinalAssessment && !gradeBreakdown.finalAssessmentCompleted &&
gradeBreakdown.isFinalUnlocked)
203 |
204 |               return (
205 |                 <motion.div
206 |                   key={enrollment.id}
207 |                   variants={fadeUp}
208 |                   className="group bg-white border border-slate-200/90 hover:border-cyan-300 rounded-3xl p-5 flex flex-col md:
flex-row md:items-center gap-6 shadow-sm hover:shadow-xl hover:shadow-slate-200/60 transition-all duration-300 hover:-translate-y-0.
5"
209 |                 >
210 |                   {/* Thumbnail */}
211 |                   <div className="w-full md:w-48 h-32 md:h-28 rounded-2xl bg-slate-100 border border-slate-200 relative overfl
ow-hidden shrink-0">
212 |                     <Thumbnail path={enrollment.course?.thumbnail_path || null} alt={enrollment.course?.title || 'Course'} typ
e={enrollment.course?.course_type} />
213 |                   </div>
214 |
215 |                   {/* Info */}
216 |                   <div className="flex-1 min-w-0">
217 |                     <div className="flex items-center gap-2.5 mb-2">
218 |                       <span className={`text-[10px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full border
${
219 |                         isCourseDone
220 |                           ? 'bg-emerald-50 text-emerald-700 border-emerald-200'
```

```
                          : isFinalPending
                          ? 'bg-purple-50 text-purple-700 border-purple-200'
                          : 'bg-cyan-50 text-cyan-700 border border-cyan-200'
                      }`}>
                        {isCourseDone ? 'Completed' : isFinalPending ? 'Final Exam Pending' : enrollment.status.replace('_', ' ')}
                      </span>
                      <span className="text-xs text-slate-500 font-semibold uppercase tracking-wider">{enrollment.course?.course_type || 'Standard'}</span>
                      {enrollment.course?.duration_minutes && (
                        <span className="text-xs text-slate-400 flex items-center gap-1">
                          <Clock className="w-3 h-3 text-amber-500" /> {enrollment.course.duration_minutes}m
                        </span>
                      )}
                    </div>
                    <h3 className="text-lg font-bold text-slate-900 mb-3 truncate group-hover:text-cyan-600 transition-colors">
                      {enrollment.course?.title}
                    </h3>

                    {/* Progress Bar & Grade stats */}
                    <div className="w-full max-w-md space-y-1.5">
                      <div className="flex justify-between text-xs font-semibold text-slate-600">
                        <span>Modules Progress: <strong className="text-cyan-700">{enrollment.progress_percent}%</strong> (25% wt)</span>
                        <span>Overall Grade: <strong className="text-slate-900 font-black">{gradeBreakdown.totalScore}%</strong></span>
                      </div>
                      <div className="h-2.5 w-full bg-slate-100 rounded-full overflow-hidden p-0.5 border border-slate-200">
                        <div
                          className={`h-full rounded-full transition-all duration-700 ${
                            isCourseDone
                              ? 'bg-gradient-to-r from-emerald-500 to-teal-600'
                              : isFinalPending
                              ? 'bg-gradient-to-r from-purple-600 to-indigo-600'
                              : 'bg-gradient-to-r from-cyan-600 to-blue-600'
                          }`}
                          style={{ width: `${enrollment.progress_percent}%` }}
                        />
                      </div>
                      <div className="flex items-center justify-between text-[10px] text-slate-400 font-medium">
                        <span>Assessments Avg: {gradeBreakdown.regularAssessmentAveragePercent}% (25% wt)</span>
                        <span>Final Exam: {gradeBreakdown.finalAssessmentScorePercent !== null ? `${gradeBreakdown.finalAssessmentScorePercent}%` : gradeBreakdown.isFinalUnlocked ? 'Ready' : 'Locked'} (50% wt)</span>
                      </div>
                    </div>

                    {/* Actions */}
                    <div className="shrink-0 flex flex-wrap items-center gap-2.5 pt-4 md:pt-0 md:pl-4 md:border-l border-slate-100">
                      <button
                        onClick={() => handleDownloadCertificate(enrollment, gradeBreakdown.totalScore)}
                        disabled={downloadingId === enrollment.id}
                        className="px-4 py-2.5 rounded-2xl bg-gradient-to-r from-emerald-600 to-teal-600 hover:from-emerald-700 hover:to-teal-700 text-white text-xs font-bold shadow-sm hover:scale-105 transition-all flex items-center gap-1.5 cursor-pointer disabled:opacity-60"
                        title="Download Certificate"
                      >
                        {downloadingId === enrollment.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Award className="w-3.5 h-3.5 text-amber-300" />}
                        {downloadingId === enrollment.id ? 'Generating...' : 'Certificate'}
                      </button>

                      {isCourseDone ? (
                        <Link to={`/trainee/courses/${enrollment.course?.id}`}>
                          <button className="px-4 py-2.5 rounded-2xl bg-slate-50 hover:bg-slate-100 border border-slate-200 text-xs font-bold text-slate-700 transition-all flex items-center gap-1.5 cursor-pointer">
                            <CheckCircle2 className="w-3.5 h-3.5 text-emerald-600" /> Review
                          </button>
                        </Link>
                      ) : isFinalPending && gradeBreakdown.finalAssessmentId ? (
                        <>
                          <Link to={`/trainee/courses/${enrollment.course?.id}/assessments/${gradeBreakdown.finalAssessmentId}`}>
                            <button className="px-4 py-2.5 rounded-2xl bg-gradient-to-r from-purple-600 via-indigo-600 to-cyan-600 hover:from-purple-700 hover:to-cyan-700 text-white font-bold text-xs shadow-md shadow-purple-600/20 hover:scale-105 transition-all flex items-center gap-1.5 animate-pulse cursor-pointer">
                              <FileCheck className="w-4 h-4 text-amber-300" /> Take Final Exam
```

```
                          </button>
                        </Link>
                        <Link to={`/trainee/courses/${enrollment.course?.id}`}>
                          <button className="px-3.5 py-2.5 rounded-2xl bg-slate-50 hover:bg-slate-100 border border-slate-200 text-xs font-bold text-slate-700 transition-all flex items-center gap-1.5 cursor-pointer">
                            <BookOpen className="w-3.5 h-3.5 text-cyan-600" /> Details
                          </button>
                        </Link>
                      </>
                    ) : (
                      <Link to={`/trainee/courses/${enrollment.course?.id}`}>
                        <button className="px-5 py-2.5 rounded-2xl bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold text-xs shadow-md shadow-cyan-600/20 hover:scale-105 transition-all flex items-center gap-2 cursor-pointer">
                          <PlayCircle className="w-4 h-4" /> Continue Lesson
                        </button>
                      </Link>
                    )}
                  </div>
                </motion.div>
              )
            })}
          </motion.div>
        )}
      </div>
    </DashboardShell>
  )
}
```

src/features/courses/TrainerCourses.tsx TSX 7.9 KB 206 Lines

```tsx
import React, { useEffect, useState, useCallback } from 'react'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { Database } from '@/integrations/supabase/types'
import { DashboardShell } from '@/pages/Dashboards'
import { CourseFormDialog } from '@/features/courses/CourseFormDialog'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@/components/ui/card'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'
import { PlusCircle, MoreHorizontal, BookOpen, Clock, Target, BarChart3, User } from 'lucide-react'
import { toast } from 'sonner'
import { formatDistanceToNow } from 'date-fns'

type Course = Database['public']['Tables']['courses']['Row']

function StatusBadge({ status }: { status: Course['status'] }) {
  const variants: Record<Course['status'], 'default' | 'secondary' | 'destructive' | 'outline'> = {
    published: 'default',
    pending_review: 'secondary',
    draft: 'outline',
    archived: 'destructive',
  }
  const labels: Record<Course['status'], string> = {
    draft: 'Draft',
    pending_review: 'Pending Review',
    published: 'Published',
    archived: 'Archived',
  }
  return <Badge variant={variants[status]}>{labels[status]}</Badge>
}

export function TrainerCourses() {
  const { user } = useAuth()
  const navigate = useNavigate()
  const [courses, setCourses] = useState<Course[]>([])
  const [loading, setLoading] = useState(true)
  const [dialogOpen, setDialogOpen] = useState(false)
  const [editingCourse, setEditingCourse] = useState<Course | null>(null)

  const fetchCourses = useCallback(async () => {
    if (!user) return
    setLoading(true)
    try {
      const { data, error } = await supabase
        .from('courses')
        .select('*')
        .eq('trainer_id', user.id)
        .neq('status', 'archived')
        .order('created_at', { ascending: false })
      if (error) throw error
      setCourses(data ?? [])
    } catch (err: unknown) {
      const message = err instanceof Error ? err.message : 'Failed to load courses'
      toast.error(message)
    } finally {
      setLoading(false)
    }
  }, [user])

  useEffect(() => { fetchCourses() }, [fetchCourses])

  const handleEdit = (course: Course) => {
    setEditingCourse(course)
    setDialogOpen(true)
  }

  const handleCreate = () => {
    setEditingCourse(null)
    setDialogOpen(true)
```

```
  }

  const handleDialogClose = () => {
    setDialogOpen(false)
    setEditingCourse(null)
  }

  const handleSubmitForReview = async (courseId: string) => {
    try {
      const { error } = await supabase
        .from('courses')
        .update({ status: 'pending_review' })
        .eq('id', courseId)
      if (error) throw error
      toast.success('Course submitted for admin review')
      fetchCourses()
    } catch (err: unknown) {
      const message = err instanceof Error ? err.message : 'Failed to submit course'
      toast.error(message)
    }
  }

  return (
    <DashboardShell
      title="Trainer — My Courses"
      icon={BookOpen}
      navLinks={[
        { to: '/trainer', label: 'Dashboard', icon: BarChart3 },
        { to: '/trainer/courses', label: 'My Courses', icon: BookOpen },
        { to: '/trainer/profile', label: 'Profile', icon: User },
      ]}
    >
      <div className="space-y-6">
        <div className="flex items-center justify-between">
          <div>
            <h2 className="text-2xl font-bold tracking-tight">My Courses</h2>
            <p className="text-muted-foreground">Create and manage your training courses</p>
          </div>
          <Button onClick={handleCreate}>
            <PlusCircle className="mr-2 h-4 w-4" />
            New Course
          </Button>
        </div>

        {loading ? (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
            {[1,2,3].map(i => (
              <Card key={i} className="animate-pulse">
                <CardHeader><div className="h-4 bg-muted rounded w-3/4" /></CardHeader>
                <CardContent><div className="h-3 bg-muted rounded w-1/2" /></CardContent>
              </Card>
            ))}
          </div>
        ) : courses.length === 0 ? (
          <Card className="flex flex-col items-center justify-center py-16 text-center">
            <BookOpen className="h-12 w-12 text-muted-foreground mb-4" />
            <h3 className="font-semibold text-lg">No courses yet — create your first course.</h3>
            <p className="text-muted-foreground text-sm mt-1 mb-4">Start building your training content.</p>
            <Button onClick={handleCreate}>
              <PlusCircle className="mr-2 h-4 w-4" />
              Create Course
            </Button>
          </Card>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
            {courses.map(course => (
              <Card key={course.id} className="flex flex-col">
                <CardHeader className="pb-2">
                  <div className="flex items-start justify-between gap-2">
                    <CardTitle className="text-base leading-tight">{course.title}</CardTitle>
                    <DropdownMenu>
                      <DropdownMenuTrigger asChild>
                        <Button variant="ghost" size="icon" className="h-7 w-7 shrink-0">
                          <MoreHorizontal className="h-4 w-4" />
                        </Button>
                      </DropdownMenuTrigger>
                      <DropdownMenuContent align="end">
                        <DropdownMenuItem onClick={() => handleEdit(course)}>Edit</DropdownMenuItem>
                        <DropdownMenuItem onClick={() => navigate(`/trainer/courses/${course.id}/materials`)}>
```

```
                          Manage Materials
                        </DropdownMenuItem>
                        {course.status === 'draft' && (
                          <DropdownMenuItem onClick={() => handleSubmitForReview(course.id)}>
                            Submit for Review
                          </DropdownMenuItem>
                        )}
                      </DropdownMenuContent>
                    </DropdownMenu>
                  </div>
                  <StatusBadge status={course.status} />
                </CardHeader>
                <CardContent className="flex-1 space-y-2">
                  <CardDescription className="line-clamp-2">
                    {course.description || 'No description provided.'}
                  </CardDescription>
                  <div className="flex flex-wrap gap-3 text-xs text-muted-foreground mt-2">
                    {course.duration_minutes && (
                      <span className="flex items-center gap-1">
                        <Clock className="h-3 w-3" />
                        {course.duration_minutes} min
                      </span>
                    )}
                    <span className="flex items-center gap-1">
                      <Target className="h-3 w-3" />
                      Pass: {course.passing_score}%
                    </span>
                    <span className="capitalize">{course.course_type}</span>
                  </div>
                  {course.created_at && (
                    <p className="text-xs text-muted-foreground">
                      Created {formatDistanceToNow(new Date(course.created_at), { addSuffix: true })}
                    </p>
                  )}
                </CardContent>
              </Card>
            ))}
          </div>
        )}
      </div>

      <CourseFormDialog
        open={dialogOpen}
        onClose={handleDialogClose}
        onSaved={fetchCourses}
        course={editingCourse}
      />
    </DashboardShell>
  )
}
```

src/features/admin/AdminAnnouncements.tsx TSX 9.8 KB 216 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { supabase } from '@/lib/supabase'
import { toast } from 'sonner'
import { motion, Variants } from 'framer-motion'
import { Plus, Megaphone, Trash2, Edit2, Loader2 } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { useAuth } from '@/hooks/useAuth'
import { useConfirm } from '@/hooks/useConfirm'

const fadeUp: Variants = {
  hidden: { opacity: 0, y: 15 },
  visible: { opacity: 1, y: 0, transition: { type: 'spring', stiffness: 300, damping: 25 } },
}

interface Announcement {
  id: string
  title: string
  content: string
  target_audience: string
  is_active: boolean
  created_at: string
}

export function AdminAnnouncements() {
  const { profile } = useAuth()
  const [announcements, setAnnouncements] = useState<Announcement[]>([])
  const [loading, setLoading] = useState(true)
  const [isFormOpen, setIsFormOpen] = useState(false)
  const [editingId, setEditingId] = useState<string | null>(null)
  const [ConfirmDialog, confirm] = useConfirm()

  const [title, setTitle] = useState('')
  const [content, setContent] = useState('')
  const [targetAudience, setTargetAudience] = useState('all')
  const [isActive, setIsActive] = useState(true)

  const fetchAnnouncements = async () => {
    setLoading(true)
    try {
      const { data, error } = await supabase
        .from('announcements')
        .select('*')
        .order('created_at', { ascending: false })
      if (error) throw error
      if (data) setAnnouncements(data)
    } catch (e) {
      console.error(e)
    } finally {
      setLoading(false)
    }
  }

  useEffect(() => {
    fetchAnnouncements()
  }, [])

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!title || !content) {
      toast.error('Title and content are required')
      return
    }

    try {
      const { data: { user } } = await supabase.auth.getUser()
      if (!user) throw new Error('Not authenticated')

      const authorName = profile?.full_name || 'System Administrator'
      const contentWithAuthor = `${content}\n\n<!--AUTHOR:${authorName}-->`

      if (editingId) {
        const { error } = await supabase
          .from('announcements')
          .update({ title, content: contentWithAuthor, target_audience: targetAudience, is_active: isActive })
          .eq('id', editingId)
        if (error) throw error
```

```
        toast.success('Announcement updated')
      } else {
        const { error } = await supabase
          .from('announcements')
          .insert([{ title, content: contentWithAuthor, target_audience: targetAudience, is_active: isActive, author_id: user.id }])
        if (error) throw error
        toast.success('Announcement published')
      }
      resetForm()
      fetchAnnouncements()
    } catch (e: any) {
      toast.error(e.message || 'Failed to save announcement')
      console.error(e)
    }
  }

  const handleDelete = async (id: string) => {
    const isConfirmed = await confirm('Are you sure you want to delete this announcement?', 'Delete Announcement')
    if (!isConfirmed) return

    try {
      const { error } = await supabase.from('announcements').delete().eq('id', id)
      if (error) throw error
      toast.success('Announcement deleted')
      fetchAnnouncements()
    } catch (e) {
      toast.error('Failed to delete')
    }
  }

  const handleEdit = (announcement: Announcement) => {
    setEditingId(announcement.id)
    setTitle(announcement.title)
    setContent(announcement.content.replace(/\n\n<!--AUTHOR:.*?-->/g, ''))
    setTargetAudience(announcement.target_audience || 'all')
    setIsActive(announcement.is_active)
    setIsFormOpen(true)
  }

  const resetForm = () => {
    setEditingId(null)
    setTitle('')
    setContent('')
    setTargetAudience('all')
    setIsActive(true)
    setIsFormOpen(false)
  }

  if (loading && announcements.length === 0) {
    return (
      <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm flex justify-center py-10">
        <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
      </div>
    )
  }

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <div>
          <h3 className="text-base font-bold text-slate-900">Global Announcements</h3>
          <p className="text-xs text-slate-500 mt-0.5">Publish news and updates to the homepage.</p>
        </div>
        {!isFormOpen && (
          <Button onClick={() => setIsFormOpen(true)} className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white rounded-xl text-xs h-9 shadow-md shadow-cyan-600/10">
            <Plus className="w-4 h-4 mr-1.5" /> New Announcement
          </Button>
        )}
      </div>

      <ConfirmDialog />

      {isFormOpen && (
        <motion.form variants={fadeUp} initial="hidden" animate="visible" onSubmit={handleSubmit} className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-sm mb-6 space-y-4">
          <div>
            <label className="block text-xs font-semibold text-slate-700 mb-1">Title</label>
```

```
153 |             <input type="text" value={title} onChange={e => setTitle(e.target.value)} className="w-full bg-slate-50 border bor
der-slate-200 text-slate-900 rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-cyan-500 focus:border-transparent transition-all
outline-none placeholder:text-slate-400" placeholder="e.g., New Training Module Available" required />
154 |           </div>
155 |           <div>
156 |             <label className="block text-xs font-semibold text-slate-700 mb-1">Target Audience</label>
157 |             <select value={targetAudience} onChange={e => setTargetAudience(e.target.value)} className="w-full bg-slate-50 bor
der border-slate-200 text-slate-900 rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-cyan-500 focus:border-transparent transit
ion-all outline-none">
158 |               <option value="all">All Users (Public & Internal)</option>
159 |               <option value="trainer">Trainers Only</option>
160 |               <option value="trainee">Trainees Only</option>
161 |             </select>
162 |           </div>
163 |           <div>
164 |             <label className="block text-xs font-semibold text-slate-700 mb-1">Content</label>
165 |             <textarea value={content} onChange={e => setContent(e.target.value)} className="w-full bg-slate-50 border border-s
late-200 text-slate-900 rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-cyan-500 focus:border-transparent transition-all outl
ine-none min-h-[100px] placeholder:text-slate-400" placeholder="Type your announcement here..." required />
166 |           </div>
167 |           <div className="flex items-center gap-2">
168 |             <input type="checkbox" id="isActive" checked={isActive} onChange={e => setIsActive(e.target.checked)} className="r
ounded text-cyan-600 focus:ring-cyan-500 border-slate-300" />
169 |             <label htmlFor="isActive" className="text-sm text-slate-700 font-medium">Publish immediately (Active)</label>
170 |           </div>
171 |           <div className="flex justify-end gap-3 pt-2">
172 |             <Button type="button" variant="outline" onClick={resetForm} className="rounded-xl border-slate-200 text-slate-700
hover:bg-slate-50">Cancel</Button>
173 |             <Button type="submit" className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white rounded-xl
shadow-md shadow-cyan-600/10">
174 |               {editingId ? 'Save Changes' : 'Publish Announcement'}
175 |             </Button>
176 |           </div>
177 |         </motion.form>
178 |       )}
179 |
180 |       <div className="grid gap-4">
181 |         {announcements.filter(ann => ann.id !== editingId).map((ann, i) => (
182 |           <motion.div key={ann.id} variants={fadeUp} custom={i} className={`bg-white border ${ann.is_active ? 'border-slate-20
0/90' : 'border-slate-200 opacity-70'} rounded-3xl p-6 shadow-sm flex flex-col md:flex-row md:items-start justify-between gap-4`}>
183 |             <div className="flex gap-4">
184 |               <div className={`w-10 h-10 rounded-2xl flex items-center justify-center shrink-0 ${ann.is_active ? 'bg-cyan-50 t
ext-cyan-600' : 'bg-slate-100 text-slate-400'}`}>
185 |                 <Megaphone className="w-5 h-5" />
186 |               </div>
187 |               <div>
188 |                 <h4 className="text-base font-bold text-slate-900 flex items-center gap-2">
189 |                   {ann.title}
190 |                   <span className="text-[10px] bg-cyan-50 text-cyan-700 px-2.5 py-0.5 rounded-full font-semibold border border
-cyan-200 uppercase">
191 |                     {ann.target_audience || 'ALL'}
192 |                   </span>
193 |                   {!ann.is_active && <span className="text-[10px] bg-slate-100 text-slate-500 px-2.5 py-0.5 rounded-full font-
semibold border border-slate-200">DRAFT</span>}
194 |                 </h4>
195 |                 <p className="text-xs text-slate-400 mt-1 mb-2 font-medium">Published: {new Date(ann.created_at).toLocaleDateS
tring()}</p>
196 |                 <p className="text-sm text-slate-600 whitespace-pre-wrap leading-relaxed">{ann.content.replace(/\n\n<!--AUTHO
R:.*?-->/g, '')}</p>
197 |               </div>
198 |             </div>
199 |             <div className="flex items-center gap-2 shrink-0 md:self-start">
200 |               <button onClick={() => handleEdit(ann)} className="p-2 hover:bg-slate-100 text-cyan-600 rounded-lg transition-co
lors" title="Edit">
201 |                 <Edit2 className="w-4 h-4" />
202 |               </button>
203 |               <button onClick={() => handleDelete(ann.id)} className="p-2 hover:bg-red-50 text-red-500 rounded-lg transition-c
olors" title="Delete">
204 |                 <Trash2 className="w-4 h-4" />
205 |               </button>
206 |             </div>
207 |           </motion.div>
208 |         ))}
209 |         {announcements.length === 0 && !loading && (
210 |           <div className="text-center py-10 text-slate-400 text-sm">No announcements found.</div>
211 |         )}
212 |       </div>
213 |     </div>
214 |   )
```

```
215 | }
216 |
```

src/features/admin/AdminAssessmentsHub.tsx TSX 73.0 KB 1,556 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { useQuery, useQueryClient } from '@tanstack/react-query'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import {
  FileCheck, Clock,
  Calendar, Search, PlayCircle, ChevronLeft, ChevronRight,
  Award, Target, Users, Sparkles, ExternalLink, Eye,
  Brain, ShieldAlert, CheckCircle2, XCircle, AlertCircle,
  Loader2, Check, X, Shield, RefreshCw, Plus, Edit, Trash2,
  HelpCircle, Settings, FileText, CheckCircle
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Badge } from '@/components/ui/badge'
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
} from '@/components/ui/dialog'
import {
  format, isSameDay, addMonths, subMonths,
  startOfMonth, endOfMonth, eachDayOfInterval, getDay
} from 'date-fns'
import { toast } from 'sonner'

const fadeUp = {
  hidden: { opacity: 0, y: 10 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.3 } },
  exit: { opacity: 0, y: -10, transition: { duration: 0.2 } }
}

interface QuestionItem {
  id?: string
  assessment_id: string
  question_text: string
  options: any
  correct_answer: string
  explanation?: string | null
  position?: number
  approved?: boolean
  difficulty?: 'easy' | 'medium' | 'hard' | null
  question_type?: string | null
}

// Mini Calendar

function MiniCalendar({
  selectedDate,
  onSelectDate,
  highlightedDates = [],
}: {
  selectedDate: Date | null
  onSelectDate: (d: Date | null) => void
  highlightedDates?: Date[]
}) {
  const [viewMonth, setViewMonth] = useState(new Date())
  const today = new Date()
  const days = eachDayOfInterval({ start: startOfMonth(viewMonth), end: endOfMonth(viewMonth) })
  const leadingBlanks = getDay(startOfMonth(viewMonth))
  const isHL = (d: Date) => highlightedDates.some(h => isSameDay(h, d))
  const isSel = (d: Date) => !!selectedDate && isSameDay(d, selectedDate)
  const isToday = (d: Date) => isSameDay(d, today)

  return (
    <div className="select-none">
      <div className="flex items-center justify-between mb-2">
        <button onClick={() => setViewMonth(m => subMonths(m, 1))} className="p-1 rounded hover:bg-slate-100 transition-colors text-slate-500 cursor-pointer">
          <ChevronLeft className="w-4 h-4" />
        </button>
        <span className="text-xs font-bold text-slate-700 tracking-wide">{format(viewMonth, 'MMMM yyyy')}</span>
        <button onClick={() => setViewMonth(m => addMonths(m, 1))} className="p-1 rounded hover:bg-slate-100 transition-colors text-slate-500 cursor-pointer">
```

```
            <ChevronRight className="w-4 h-4" />
          </button>
        </div>
        <div className="grid grid-cols-7 mb-1">
          {['Su','Mo','Tu','We','Th','Fr','Sa'].map(d => (
            <div key={d} className="text-center text-[10px] font-bold text-slate-400 py-0.5">{d}</div>
          ))}
        </div>
        <div className="grid grid-cols-7 gap-y-0.5">
          {Array.from({ length: leadingBlanks }).map((_, i) => <div key={`b-${i}`} />)}
          {days.map(day => {
            const sel = isSel(day); const hl = isHL(day); const tod = isToday(day)
            return (
              <button
                key={day.toISOString()}
                onClick={() => onSelectDate(sel ? null : day)}
                className={[
                  'relative w-full aspect-square flex items-center justify-center rounded-full text-[11px] font-semibold transi
tion-all cursor-pointer',
                  sel ? 'bg-[#1a7a80] text-white shadow' : '',
                  !sel && hl ? 'text-[#1a7a80] font-bold' : '',
                  !sel && tod ? 'ring-2 ring-[#1a7a80]/40' : '',
                  !sel ? 'text-slate-700 hover:bg-slate-100' : '',
                ].filter(Boolean).join(' ')}
              >
                {format(day, 'd')}
                {hl && !sel && <span className="absolute bottom-0.5 left-1/2 -translate-x-1/2 w-1 h-1 rounded-full bg-[#1a7a8
0]" />}
              </button>
            )
          })}
        </div>
        {selectedDate && (
          <button onClick={() => onSelectDate(null)} className="mt-3 w-full text-[11px] font-semibold text-slate-500 hover:text
-rose-500 transition-colors cursor-pointer">
            Clear date filter
          </button>
        )}
      </div>
    )
}

export function AdminAssessmentsHub() {
  const queryClient = useQueryClient()
  const [activeTab, setActiveTab] = useState<'ongoing' | 'requests' | 'completed' | 'expired'>('ongoing')
  const [searchQuery, setSearchQuery] = useState('')
  const [filterType, setFilterType] = useState('all')
  const [courseFilter, setCourseFilter] = useState('all')
  const [calendarDate, setCalendarDate] = useState<Date | null>(null)
  const [selectedAssessmentForSubmissions, setSelectedAssessmentForSubmissions] = useState<any | null>(null)
  const [processingId, setProcessingId] = useState<string | null>(null)

  // Question & Assessment Content Editor States

  const [selectedAssessmentForQuestions, setSelectedAssessmentForQuestions] = useState<any | null>(null)
  const [activeModalTab, setActiveModalTab] = useState<'questions' | 'settings'>('questions')
  const [questionsList, setQuestionsList] = useState<QuestionItem[]>([])
  const [questionsLoading, setQuestionsLoading] = useState(false)
  const [editingQuestion, setEditingQuestion] = useState<{
    id?: string
    question_text: string
    option_a: string
    option_b: string
    option_c: string
    option_d: string
    correct_answer: string
    explanation: string
    difficulty: 'easy' | 'medium' | 'hard'
    question_type: 'mcq' | 'true_false'
  } | null>(null)
  const [isSavingQuestion, setIsSavingQuestion] = useState(false)

  // Assessment Settings Form State

  const [assessmentSettingsForm, setAssessmentSettingsForm] = useState<{
    title: string
    duration_minutes: number
    passing_score: number
    scheduled_date: string
```

```
    start_time: string
    end_time: string
    requires_sea: boolean
  }>({
    title: '',
    duration_minutes: 30,
    passing_score: 50,
    scheduled_date: '',
    start_time: '',
    end_time: '',
    requires_sea: false,
  })
  const [isSavingSettings, setIsSavingSettings] = useState(false)

  // Live clock

  const [now, setNow] = useState(new Date())
  useEffect(() => {
    const timer = setInterval(() => setNow(new Date()), 1000)
    return () => clearInterval(timer)
  }, [])

  // 1. Fetch All Courses

  const { data: courses = [] } = useQuery({
    queryKey: ['admin_all_courses_assessments'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('courses')
        .select('id, title, status, course_type, trainer:trainers(full_name)')
        .order('created_at', { ascending: false })
      if (error) throw error
      return data || []
    },
  })

  // 2. Fetch All Assessments (including published, pending_review, and draft final exams)

  const { data: assessments = [], isLoading: assessmentsLoading, refetch: refetchAssessments } = useQuery({
    queryKey: ['admin_all_assessments'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('assessments')
        .select(`
          id, title, scheduled_date, start_time, end_time, duration_minutes,
          passing_score, course_id, status, assessment_type, requires_sea,
          results_publish_date, created_by,
          course:courses(id, title, course_type, department, trainer:trainers(full_name))
        `)
        .order('created_at', { ascending: false })
      if (error) throw error
      return (data || []) as any[]
    },
  })

  // 3. Fetch Questions Count per assessment

  const assessmentIds = assessments.map(a => a.id)
  const { data: questionCounts = {}, refetch: refetchQuestionCounts } = useQuery({
    queryKey: ['admin_assessment_question_counts', assessmentIds],
    queryFn: async () => {
      if (assessmentIds.length === 0) return {}
      const { data, error } = await supabase
        .from('questions')
        .select('assessment_id')
        .in('assessment_id', assessmentIds)
      if (error) return {}
      const counts: Record<string, number> = {}
      ;(data || []).forEach((q: any) => {
        counts[q.assessment_id] = (counts[q.assessment_id] || 0) + 1
      })
      return counts
    },
    enabled: assessmentIds.length > 0,
  })

  // 4. Fetch All Trainee Assessment Attempts

  const { data: attempts = [] } = useQuery({
```

```
    queryKey: ['admin_all_assessment_attempts'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('assessment_attempts')
        .select(`
          id, assessment_id, user_id, score, started_at, submitted_at, time_taken_seconds,
          grade_status, status,
          user:profiles(id, full_name, email, department, avatar_path)
        `)
        .order('submitted_at', { ascending: false })
      if (error) return []
      return (data || []) as any[]
    },
  })

  const attemptsByAssessment = attempts.reduce((acc: Record<string, any[]>, att: any) => {
    if (!acc[att.assessment_id]) acc[att.assessment_id] = []
    acc[att.assessment_id].push(att)
    return acc
  }, {})

  // Categorize assessments

  const pendingFinalExamRequests = assessments.filter(a =>
    a.status === 'pending_review' ||
    (a.assessment_type === 'final' && (a.status === 'pending_approval' || a.status === 'pending_review'))
  )

  const ongoingAndUpcoming = assessments.filter(a => {
    if (a.status !== 'published') return false
    if (!a.scheduled_date) return true
    const end = new Date(`${a.scheduled_date}T${a.end_time || '23:59:59'}`)
    const start = new Date(`${a.scheduled_date}T${a.start_time || '00:00:00'}`)
    if (end < start) end.setDate(end.getDate() + 1)
    return now <= end
  })

  const expiredAssessments = assessments.filter(a => {
    if (a.status !== 'published') return false
    if (!a.scheduled_date) return false
    const end = new Date(`${a.scheduled_date}T${a.end_time || '23:59:59'}`)
    const start = new Date(`${a.scheduled_date}T${a.start_time || '00:00:00'}`)
    if (end < start) end.setDate(end.getDate() + 1)
    return now > end
  })

  const assessmentsWithSubmissions = assessments.filter(a => {
    const atts = attemptsByAssessment[a.id] || []
    return atts.length > 0
  })

  const formatType = (t: string) =>
    t === 'daily'
      ? 'Daily Test (Practice)'
      : t === 'mock'
      ? 'Mock Test (Practice)'
      : t === 'assessment'
      ? 'Regular Assessment (25% Weight)'
      : t === 'final'
      ? 'Final Assessment (50% Weight)'
      : 'Test'

  const matchesFilter = (a: any) => {
    const ms =
      a.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
      (a.course as any)?.title?.toLowerCase().includes(searchQuery.toLowerCase()) ||
      (a.assessment_type && formatType(a.assessment_type).toLowerCase().includes(searchQuery.toLowerCase()))

    const matchesType = filterType === 'all' || a.assessment_type === filterType
    const matchesCourse = courseFilter === 'all' || a.course_id === courseFilter
    const matchesCal = !calendarDate || (a.scheduled_date && isSameDay(new Date(a.scheduled_date), calendarDate))

    return ms && matchesType && matchesCourse && matchesCal
  }

  const filteredOngoing = ongoingAndUpcoming.filter(matchesFilter)
  const filteredRequests = pendingFinalExamRequests.filter(matchesFilter)
  const filteredCompleted = assessmentsWithSubmissions.filter(matchesFilter)
  const filteredExpired = expiredAssessments.filter(matchesFilter)
```

```
300 |
301 |   const highlightedDates = assessments
302 |     .map(a => (a.scheduled_date ? new Date(a.scheduled_date) : null))
303 |     .filter(Boolean) as Date[]
304 |
305 |   // Load questions for an assessment

306 |   const fetchQuestionsForAssessment = async (assessment: any) => {
307 |     setSelectedAssessmentForQuestions(assessment)
308 |     setActiveModalTab('questions')
309 |     setEditingQuestion(null)
310 |     setQuestionsLoading(true)
311 |
312 |     setAssessmentSettingsForm({
313 |       title: assessment.title || '',
314 |       duration_minutes: assessment.duration_minutes || 30,
315 |       passing_score: assessment.passing_score ?? 50,
316 |       scheduled_date: assessment.scheduled_date || '',
317 |       start_time: assessment.start_time || '',
318 |       end_time: assessment.end_time || '',
319 |       requires_sea: !!assessment.requires_sea,
320 |     })
321 |
322 |     try {
323 |       const { data, error } = await supabase
324 |         .from('questions')
325 |         .select('*')
326 |         .eq('assessment_id', assessment.id)
327 |         .order('position', { ascending: true })
328 |
329 |       if (error) throw error
330 |       setQuestionsList((data || []) as QuestionItem[])
331 |     } catch (err: any) {
332 |       console.error(err)
333 |       toast.error('Failed to load questions for this assessment')
334 |     } finally {
335 |       setQuestionsLoading(false)
336 |     }
337 |   }
338 |
339 |   // Parse options helper

340 |   const parseQuestionOptions = (options: any) => {
341 |     if (!options) return { A: '', B: '', C: '', D: '', raw: [] }
342 |     if (typeof options === 'object') {
343 |       if (Array.isArray(options)) {
344 |         return {
345 |           A: options[0] || '',
346 |           B: options[1] || '',
347 |           C: options[2] || '',
348 |           D: options[3] || '',
349 |           raw: options
350 |         }
351 |       }
352 |       return {
353 |         A: options.A || options.a || '',
354 |         B: options.B || options.b || '',
355 |         C: options.C || options.c || '',
356 |         D: options.D || options.d || '',
357 |         raw: Object.values(options).filter(v => typeof v === 'string')
358 |       }
359 |     }
360 |     return { A: '', B: '', C: '', D: '', raw: [] }
361 |   }
362 |
363 |   // Save/Update Question

364 |   const handleSaveQuestion = async () => {
365 |     if (!selectedAssessmentForQuestions || !editingQuestion) return
366 |     if (!editingQuestion.question_text.trim()) {
367 |       toast.error('Please enter question text')
368 |       return
369 |     }
370 |
371 |     if (editingQuestion.question_type === 'mcq') {
372 |       if (!editingQuestion.option_a.trim() || !editingQuestion.option_b.trim()) {
373 |         toast.error('Please provide at least Option A and Option B')
374 |         return
375 |       }
```

```
    }

    setIsSavingQuestion(true)
    try {
      const optionsPayload = {
        ...(editingQuestion.question_type === 'mcq'
          ? {
              A: editingQuestion.option_a.trim(),
              B: editingQuestion.option_b.trim(),
              C: editingQuestion.option_c.trim(),
              D: editingQuestion.option_d.trim(),
            }
          : {
              A: 'True',
              B: 'False',
            }),
        _question_type: editingQuestion.question_type,
        _difficulty: editingQuestion.difficulty,
      }

      const qPayload = {
        assessment_id: selectedAssessmentForQuestions.id,
        question_text: editingQuestion.question_text.trim(),
        options: optionsPayload,
        correct_answer: editingQuestion.correct_answer,
        explanation: editingQuestion.explanation.trim() || null,
        approved: true,
        difficulty: editingQuestion.difficulty,
        question_type: editingQuestion.question_type,
      }

      if (editingQuestion.id) {
        const { error } = await supabase
          .from('questions')
          .update(qPayload as any)
          .eq('id', editingQuestion.id)
        if (error) throw error
        toast.success('Question updated successfully')
      } else {
        const { error } = await supabase
          .from('questions')
          .insert({
            ...qPayload,
            position: questionsList.length + 1,
          } as any)
        if (error) throw error
        toast.success('Question added successfully')
      }

      // Refetch questions

      const { data } = await supabase
        .from('questions')
        .select('*')
        .eq('assessment_id', selectedAssessmentForQuestions.id)
        .order('position', { ascending: true })
      setQuestionsList((data || []) as QuestionItem[])
      setEditingQuestion(null)
      refetchQuestionCounts()
    } catch (err: any) {
      console.error(err)
      toast.error(err.message || 'Failed to save question')
    } finally {
      setIsSavingQuestion(false)
    }
  }

  // Delete Question

  const handleDeleteQuestion = async (questionId: string) => {
    if (!window.confirm('Are you sure you want to delete this question?')) return

    try {
      const { error } = await supabase.from('questions').delete().eq('id', questionId)
      if (error) throw error
      toast.success('Question deleted')
      setQuestionsList(prev => prev.filter(q => q.id !== questionId))
      refetchQuestionCounts()
    } catch (err: any) {
```

```
      console.error(err)
      toast.error(err.message || 'Failed to delete question')
    }
  }

  // Save Assessment Settings

  const handleSaveAssessmentSettings = async () => {
    if (!selectedAssessmentForQuestions) return
    setIsSavingSettings(true)
    try {
      const { error } = await supabase
        .from('assessments')
        .update({
          title: assessmentSettingsForm.title.trim(),
          duration_minutes: Number(assessmentSettingsForm.duration_minutes),
          passing_score: Number(assessmentSettingsForm.passing_score),
          scheduled_date: assessmentSettingsForm.scheduled_date || null,
          start_time: assessmentSettingsForm.start_time || null,
          end_time: assessmentSettingsForm.end_time || null,
          requires_sea: assessmentSettingsForm.requires_sea,
        } as any)
        .eq('id', selectedAssessmentForQuestions.id)

      if (error) throw error
      toast.success('Assessment settings updated successfully')
      refetchAssessments()
      setSelectedAssessmentForQuestions((prev: any) => ({
        ...prev,
        ...assessmentSettingsForm,
      }))
    } catch (err: any) {
      console.error(err)
      toast.error(err.message || 'Failed to update assessment settings')
    } finally {
      setIsSavingSettings(false)
    }
  }

  // Action: Approve Final Exam Request

  const handleApproveFinalExam = async (assessment: any) => {
    setProcessingId(assessment.id)
    try {
      // 1. Update assessment status to published

      const { error: aErr } = await supabase
        .from('assessments')
        .update({ status: 'published' })
        .eq('id', assessment.id)

      if (aErr) throw aErr

      // 2. Approve all associated questions

      await supabase
        .from('questions')
        .update({ approved: true } as any)
        .eq('assessment_id', assessment.id)

      // 3. Notify trainer

      if (assessment.created_by) {
        await supabase.from('notifications').insert({
          user_id: assessment.created_by,
          type: 'assessment',
          title: 'Final Exam Approved',
          message: `Your final exam "${assessment.title}" for "${assessment.course?.title || 'course'}" has been approved and
published by Admin.`,
        })
      }

      // 4. Notify enrolled trainees

      const { data: enrollments } = await supabase
        .from('enrollments')
        .select('user_id')
        .eq('course_id', assessment.course_id)
        .in('status', ['enrolled', 'in_progress', 'completed'])
```

```
525 |
526 |       if (enrollments && enrollments.length > 0) {
527 |         const notifs = enrollments.map(en => ({
528 |           user_id: en.user_id,
529 |           type: 'assessment',
530 |           title: 'Final Examination Published',
531 |           message: `The official final examination for "${assessment.course?.title || 'course'}" is now scheduled and publish
ed.`,
532 |         }))
533 |         await supabase.from('notifications').insert(notifs)
534 |       }
535 |
536 |       toast.success('Final Exam approved and published successfully!')
537 |       refetchAssessments()
538 |       queryClient.invalidateQueries({ queryKey: ['admin_all_assessments'] })
539 |     } catch (err: any) {
540 |       console.error(err)
541 |       toast.error(err.message || 'Failed to approve final exam')
542 |     } finally {
543 |       setProcessingId(null)
544 |     }
545 |   }
546 |
547 |   // Action: Reject / Send back Final Exam

548 |   const handleRejectFinalExam = async (assessment: any) => {
549 |     setProcessingId(assessment.id)
550 |     try {
551 |       const { error } = await supabase
552 |         .from('assessments')
553 |         .update({ status: 'draft' })
554 |         .eq('id', assessment.id)
555 |
556 |       if (error) throw error
557 |
558 |       if (assessment.created_by) {
559 |         await supabase.from('notifications').insert({
560 |           user_id: assessment.created_by,
561 |           type: 'assessment',
562 |           title: 'Final Exam Revision Requested',
563 |           message: `Admin requested changes on final exam "${assessment.title}" for "${assessment.course?.title || 'cours
e'}". Status has been returned to draft.`,
564 |         })
565 |       }
566 |
567 |       toast.info('Final Exam request sent back to Trainer for revision')
568 |       refetchAssessments()
569 |       queryClient.invalidateQueries({ queryKey: ['admin_all_assessments'] })
570 |     } catch (err: any) {
571 |       console.error(err)
572 |       toast.error(err.message || 'Failed to return final exam')
573 |     } finally {
574 |       setProcessingId(null)
575 |     }
576 |   }
577 |
578 |   // Stats

579 |   const totalTests = assessments.length
580 |   const finalExamsCount = assessments.filter(a => a.assessment_type === 'final').length
581 |   const totalSubmissions = attempts.length
582 |   const activeCount = ongoingAndUpcoming.length
583 |   const pendingRequestsCount = pendingFinalExamRequests.length
584 |
585 |   return (
586 |     <div className="space-y-6">
587 |       {/* Top Header Card */}
588 |       <div className="p-6 sm:p-7 rounded-3xl bg-gradient-to-r from-[#040814] via-[#091834] to-[#0f2752] text-white shadow-xl
border border-cyan-500/30 relative overflow-hidden">
589 |         <div className="absolute top-0 right-0 w-80 h-80 bg-cyan-500/10 rounded-full blur-3xl pointer-events-none" />
590 |         <div className="relative z-10 flex flex-col sm:flex-row sm:items-center justify-between gap-4">
591 |           <div>
592 |             <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md text-amber-3
00 text-xs font-semibold mb-2 border border-white/15">
593 |               <FileCheck className="w-3.5 h-3.5" />
594 |               <span>Assessment & Evaluation Governance</span>
595 |             </div>
596 |             <h2 className="text-2xl sm:text-3xl font-extrabold tracking-tight text-white mb-1">
597 |               Course Assessments Hub
```

```
598 |             </h2>
599 |             <p className="text-slate-300 text-sm">
600 |               Review and edit assessment questions, verify trainer final exam requests, and manage test schedules.
601 |             </p>
602 |           </div>
603 |
604 |           <div className="grid grid-cols-2 sm:grid-cols-5 gap-2.5">
605 |             <div className="bg-white/10 backdrop-blur-md rounded-2xl p-3 border border-white/10 text-center">
606 |               <span className="text-[10px] text-slate-300 uppercase font-bold tracking-wider block">Total Tests</span>
607 |               <span className="text-xl font-black text-white">{totalTests}</span>
608 |             </div>
609 |             <div className="bg-white/10 backdrop-blur-md rounded-2xl p-3 border border-white/10 text-center">
610 |               <span className="text-[10px] text-cyan-300 uppercase font-bold tracking-wider block">Active Tests</span>
611 |               <span className="text-xl font-black text-cyan-300">{activeCount}</span>
612 |             </div>
613 |             <div className="bg-white/10 backdrop-blur-md rounded-2xl p-3 border border-white/10 text-center">
614 |               <span className="text-[10px] text-purple-300 uppercase font-bold tracking-wider block">Final Exams</span>
615 |               <span className="text-xl font-black text-purple-300">{finalExamsCount}</span>
616 |             </div>
617 |             <div className={`backdrop-blur-md rounded-2xl p-3 border text-center transition-all ${
618 |               pendingRequestsCount > 0 ? 'bg-amber-500/20 border-amber-500/40 ring-1 ring-amber-400/40' : 'bg-white/10 border
-white/10'
619 |             }`}>
620 |               <span className="text-[10px] text-amber-300 uppercase font-bold tracking-wider block">Final Requests</span>
621 |               <span className="text-xl font-black text-amber-300">{pendingRequestsCount}</span>
622 |             </div>
623 |             <div className="bg-white/10 backdrop-blur-md rounded-2xl p-3 border border-white/10 text-center">
624 |               <span className="text-[10px] text-emerald-300 uppercase font-bold tracking-wider block">Submissions</span>
625 |               <span className="text-xl font-black text-emerald-300">{totalSubmissions}</span>
626 |             </div>
627 |           </div>
628 |         </div>
629 |       </div>
630 |
631 |       {/* Main Grid: Filters + Sidebar Calendar */}
632 |       <div className="grid grid-cols-1 lg:grid-cols-4 gap-6">
633 |
634 |         {/* Left 3 Columns: Assessments List */}
635 |         <div className="lg:col-span-3 space-y-5">
636 |
637 |           {/* Controls Bar - Split into Distinct Clean Rows */}
638 |           <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-xs space-y-4">
639 |
640 |             {/* Row 1: Search & Course Filter */}
641 |             <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3">
642 |               {/* Search */}
643 |               <div className="relative flex-1">
644 |                 <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
645 |                 <input
646 |                   type="text"
647 |                   placeholder="Search assessment name, course title, or type..."
648 |                   value={searchQuery}
649 |                   onChange={(e) => setSearchQuery(e.target.value)}
650 |                   className="w-full pl-9 pr-4 py-2.5 text-sm bg-slate-50 border border-slate-200 rounded-xl text-slate-900 pl
aceholder:text-slate-400 focus:outline-none focus:border-cyan-500 focus:bg-white transition-all"
651 |                 />
652 |               </div>
653 |
654 |               {/* Course filter dropdown */}
655 |               <select
656 |                 value={courseFilter}
657 |                 onChange={(e) => setCourseFilter(e.target.value)}
658 |                 className="py-2.5 px-3.5 text-xs font-semibold bg-slate-50 border border-slate-200 rounded-xl text-slate-700
focus:outline-none focus:border-cyan-500"
659 |               >
660 |                 <option value="all">All Courses ({courses.length})</option>
661 |                 {courses.map(c => (
662 |                   <option key={c.id} value={c.id}>{c.title}</option>
663 |                 ))}
664 |               </select>
665 |             </div>
666 |
667 |             {/* Row 2: Status Lifecycle Tabs (Dedicated Row) */}
668 |             <div className="pt-2 border-t border-slate-100">
669 |               <div className="flex flex-wrap gap-2">
670 |                 {[
671 |                   { id: 'ongoing', label: 'Ongoing & Scheduled', count: filteredOngoing.length, isWarning: false },
672 |                   { id: 'requests', label: 'Trainer Final Exam Requests', count: filteredRequests.length, isWarning: true },
673 |                   { id: 'completed', label: 'Completed & Submissions', count: filteredCompleted.length, isWarning: false },
```

```
                  { id: 'expired', label: 'Past / Expired', count: filteredExpired.length, isWarning: false },
                ].map(tab => {
                  const isSelected = activeTab === tab.id
                  return (
                    <button
                      key={tab.id}
                      onClick={() => setActiveTab(tab.id as any)}
                      className={`px-4 py-2 rounded-xl text-xs font-bold transition-all flex items-center gap-2 cursor-pointe
r ${
                        isSelected
                          ? tab.isWarning && tab.count > 0
                            ? 'bg-amber-600 text-white shadow-sm shadow-amber-600/20'
                            : 'bg-gradient-to-r from-cyan-600 to-blue-600 text-white shadow-sm shadow-cyan-600/20'
                          : tab.isWarning && tab.count > 0
                          ? 'bg-amber-50 text-amber-900 border border-amber-300 hover:bg-amber-100'
                          : 'bg-slate-100 text-slate-600 hover:text-slate-900 hover:bg-slate-200/70 border border-transparen
t'
                      }`}
                    >
                      <span>{tab.label}</span>
                      <span className={`text-[10px] px-2 py-0.5 rounded-full font-extrabold ${
                        isSelected
                          ? 'bg-white/20 text-white'
                          : tab.isWarning && tab.count > 0
                          ? 'bg-amber-500 text-white animate-pulse'
                          : 'bg-slate-200 text-slate-700'
                      }`}>
                        {tab.count}
                      </span>
                    </button>
                  )
                })}
              </div>
            </div>

            {/* Row 3: Evaluation Category / Weight Filter (Dedicated Row) */}
            <div className="pt-3 border-t border-slate-100 flex flex-col sm:flex-row sm:items-center gap-2.5">
              <span className="text-xs font-bold text-slate-500 uppercase tracking-wider shrink-0">
                Evaluation Type:
              </span>
              <div className="flex flex-wrap items-center gap-1.5">
                {[
                  { id: 'all', label: 'All Types' },
                  { id: 'final', label: 'Final Exam (50% Grade)' },
                  { id: 'assessment', label: 'Regular Assessment (25% Weight)' },
                  { id: 'mock', label: 'Mock Test (Practice)' },
                  { id: 'daily', label: 'Daily Test (Practice)' },
                ].map(t => (
                  <button
                    key={t.id}
                    onClick={() => setFilterType(t.id)}
                    className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all border cursor-pointer ${
                      filterType === t.id
                        ? 'bg-slate-900 text-white border-slate-900 shadow-xs'
                        : 'bg-slate-50 text-slate-600 border-slate-200 hover:bg-slate-100'
                    }`}
                  >
                    {t.label}
                  </button>
                ))}
              </div>
            </div>
          </div>

          {/* Assessment Cards List */}
          {assessmentsLoading ? (
            <div className="bg-white rounded-3xl p-12 text-center text-slate-400 font-medium border border-slate-200 flex fle
x-col items-center justify-center gap-2">
              <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
              <span>Loading assessments...</span>
            </div>
          ) : (() => {
            const currentList =
              activeTab === 'ongoing' ? filteredOngoing :
              activeTab === 'requests' ? filteredRequests :
              activeTab === 'completed' ? filteredCompleted :
              filteredExpired

            if (currentList.length === 0) {
```

```
750 |               return (
751 |                 <div className="bg-white rounded-3xl p-12 text-center border border-slate-200 space-y-3">
752 |                   <div className="w-12 h-12 rounded-2xl bg-slate-100 flex items-center justify-center text-slate-400 mx-aut
o">
753 |                     <FileCheck className="w-6 h-6" />
754 |                   </div>
755 |                   <h4 className="text-sm font-bold text-slate-800">
756 |                     {activeTab === 'requests' ? 'No pending Final Exam requests' : 'No assessments found'}
757 |                   </h4>
758 |                   <p className="text-xs text-slate-500 max-w-sm mx-auto">
759 |                     {activeTab === 'requests'
760 |                       ? 'All final exam submissions from trainers have been reviewed and processed.'
761 |                       : searchQuery || filterType !== 'all' || courseFilter !== 'all' || calendarDate
762 |                       ? 'Try adjusting the search query or clearing the active filters.'
763 |                       : 'No scheduled assessments in this category.'}
764 |                   </p>
765 |                 </div>
766 |               )
767 |             }
768 |
769 |             return (
770 |               <div className="space-y-4">
771 |                 {currentList.map((a: any) => {
772 |                   const qCount = questionCounts[a.id] || 0
773 |                   const isFinal = a.assessment_type === 'final'
774 |                   const isPractice = a.assessment_type === 'mock' || a.assessment_type === 'daily'
775 |                   const atts = attemptsByAssessment[a.id] || []
776 |                   const isPendingReview = a.status === 'pending_review' || a.status === 'pending_approval'
777 |
778 |                   return (
779 |                     <motion.div
780 |                       key={a.id}
781 |                       variants={fadeUp}
782 |                       className={`bg-white border rounded-3xl p-5 sm:p-6 shadow-xs hover:shadow-md transition-all space-y-4
${
783 |                         isPendingReview
784 |                           ? 'border-amber-300 ring-1 ring-amber-200/60 bg-gradient-to-br from-amber-50/20 via-white to-white'
785 |                           : 'border-slate-200/90 hover:border-cyan-300'
786 |                       }`}
787 |                     >
788 |                       <div className="flex flex-col sm:flex-row sm:items-start justify-between gap-3">
789 |                         <div className="space-y-2">
790 |                           <div className="flex flex-wrap items-center gap-2">
791 |                             {/* Course Badge */}
792 |                             <span className="text-[11px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full b
g-slate-100 text-slate-800 border border-slate-200">
793 |                               {(a.course as any)?.title || 'Course'}
794 |                             </span>
795 |
796 |                             {/* Assessment Type Badge */}
797 |                             {isFinal ? (
798 |                               <span className="text-[10px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full
bg-purple-50 text-purple-700 border border-purple-200 flex items-center gap-1">
799 |                                 <Award className="w-3 h-3 text-purple-600" /> Final Exam (50% Grade)
800 |                               </span>
801 |                             ) : isPractice ? (
802 |                               <span className="text-[10px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full
bg-slate-100 text-slate-700 border border-slate-200 flex items-center gap-1">
803 |                                 <Brain className="w-3 h-3 text-slate-500" /> {a.assessment_type === 'mock' ? 'Mock Test' : 'D
aily Test'} (Practice)
804 |                               </span>
805 |                             ) : (
806 |                               <span className="text-[10px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full
bg-amber-50 text-amber-800 border border-amber-200 flex items-center gap-1">
807 |                                 <Target className="w-3 h-3 text-amber-600" /> Regular Assessment (25% Weight)
808 |                               </span>
809 |                             )}
810 |
811 |                             {/* Review Pending Badge */}
812 |                             {isPendingReview && (
813 |                               <span className="text-[10px] font-extrabold uppercase tracking-wider px-2.5 py-0.5 rounded-full
bg-amber-500 text-white flex items-center gap-1 animate-pulse">
814 |                                 <Clock className="w-3 h-3" /> Trainer Approval Requested
815 |                               </span>
816 |                             )}
817 |
818 |                             {a.requires_sea && (
819 |                               <span className="text-[10px] font-extrabold uppercase tracking-wider px-2 py-0.5 rounded-full b
g-rose-50 text-rose-700 border border-rose-200 flex items-center gap-1">
```

```
                                <ShieldAlert className="w-3 h-3 text-rose-500" /> SEA Enabled
                              </span>
                            )}
                          </div>

                          <h3 className="text-base sm:text-lg font-bold text-slate-900 pt-0.5">
                            {a.title}
                          </h3>
                        </div>

                        {/* Submissions Count Badge */}
                        <div className="shrink-0 text-right">
                          <span className="text-xs font-bold text-slate-700 bg-slate-50 border border-slate-200 px-3 py-1 rou
nded-xl inline-flex items-center gap-1.5">
                            <Users className="w-3.5 h-3.5 text-cyan-600" />
                            <span>{atts.length} Submissions</span>
                          </span>
                        </div>
                      </div>

                      {/* Info Row: Date, Duration, Questions, Passing Score */}
                      <div className="grid grid-cols-2 sm:grid-cols-4 gap-2 pt-1 border-t border-slate-100 text-xs text-slate
-600">
                        <div className="flex items-center gap-1.5">
                          <Calendar className="w-3.5 h-3.5 text-slate-400" />
                          <span>{a.scheduled_date ? format(new Date(a.scheduled_date), 'd MMM yyyy') : 'Self-Paced'}</span>
                        </div>
                        <div className="flex items-center gap-1.5">
                          <Clock className="w-3.5 h-3.5 text-amber-500" />
                          <span>{a.duration_minutes || 30} mins {a.start_time ? `(${a.start_time})` : ''}</span>
                        </div>
                        <div className="flex items-center gap-1.5">
                          <Target className="w-3.5 h-3.5 text-cyan-600" />
                          <span>{qCount} Questions</span>
                        </div>
                        <div className="flex items-center gap-1.5">
                          <Award className="w-3.5 h-3.5 text-emerald-600" />
                          <span>Pass Threshold: {a.passing_score ?? 50}%</span>
                        </div>
                      </div>

                      {/* Action buttons */}
                      <div className="flex flex-wrap items-center justify-between gap-3 pt-2">
                        <div className="text-[11px] text-slate-500 font-medium">
                          Trainer: <span className="font-bold text-slate-700">{(a.course as any)?.trainer?.full_name || 'Assi
gned Instructor'}</span>
                        </div>

                        <div className="flex flex-wrap items-center gap-2">
                          {/* Trainer Final Exam Decision Actions (Approve / Reject) */}
                          {isPendingReview ? (
                            <>
                              <Button
                                size="sm"
                                onClick={() => handleApproveFinalExam(a)}
                                disabled={processingId === a.id}
                                className="bg-emerald-600 hover:bg-emerald-700 text-white font-bold text-xs rounded-xl px-4 p
y-2 shadow-xs flex items-center gap-1.5 cursor-pointer"
                              >
                                {processingId === a.id ? (
                                  <Loader2 className="w-3.5 h-3.5 animate-spin" />
                                ) : (
                                  <CheckCircle2 className="w-3.5 h-3.5" />
                                )}
                                <span>Approve & Publish Final Exam</span>
                              </Button>

                              <Button
                                size="sm"
                                variant="outline"
                                onClick={() => handleRejectFinalExam(a)}
                                disabled={processingId === a.id}
                                className="border-rose-200 text-rose-700 hover:bg-rose-50 font-bold text-xs rounded-xl px-3.5
py-2 flex items-center gap-1.5 cursor-pointer"
                              >
                                <XCircle className="w-3.5 h-3.5" />
                                <span>Send Back to Trainer</span>
                              </Button>
                            </>
```

```
                          ) : null}
                          {/* View & Update Questions Button */}
                          <Button
                            size="sm"
                            variant="outline"
                            onClick={() => fetchQuestionsForAssessment(a)}
                            className="bg-cyan-50/80 border-cyan-200 text-cyan-800 hover:bg-cyan-100 font-bold text-xs rounde
d-xl px-3.5 py-2 flex items-center gap-1.5 cursor-pointer shadow-xs"
                          >
                            <FileText className="w-3.5 h-3.5 text-cyan-600" />
                            <span>Questions ({qCount})</span>
                          </Button>

                          {/* Preview / Take Test Button */}
                          <Button
                            size="sm"
                            onClick={() => {
                              window.open(`/trainee/courses/${a.course_id}/assessments/${a.id}`, '_blank')
                            }}
                            className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-
white font-bold text-xs rounded-xl px-4 py-2 shadow-xs flex items-center gap-1.5 cursor-pointer"
                          >
                            <PlayCircle className="w-3.5 h-3.5" />
                            <span>Preview Test Player</span>
                            <ExternalLink className="w-3 h-3 opacity-70 ml-0.5" />
                          </Button>

                          {/* Submissions Button */}
                          <Button
                            size="sm"
                            variant="outline"
                            onClick={() => setSelectedAssessmentForSubmissions(a)}
                            className="border-slate-200 text-slate-700 hover:bg-slate-100 font-bold text-xs rounded-xl px-3.5
py-2 cursor-pointer"
                          >
                            <Eye className="w-3.5 h-3.5 mr-1 text-slate-500" />
                            <span>Submissions ({atts.length})</span>
                          </Button>
                        </div>
                      </div>
                    </motion.div>
                  )
                })}
              </div>
            )
          })()}
        </div>

        {/* Right 1 Column: Mini Calendar Widget */}
        <div className="space-y-4">
          <div className="bg-white border border-slate-200/90 rounded-3xl p-5 shadow-xs space-y-4">
            <div className="flex items-center gap-2 pb-2 border-b border-slate-100">
              <Calendar className="w-4 h-4 text-cyan-600" />
              <h3 className="text-xs font-bold text-slate-900 uppercase tracking-wider">Assessment Calendar</h3>
            </div>
            <MiniCalendar
              selectedDate={calendarDate}
              onSelectDate={setCalendarDate}
              highlightedDates={highlightedDates}
            />
          </div>

          <div className="bg-cyan-50/50 border border-cyan-200/70 rounded-3xl p-5 shadow-xs space-y-2 text-xs text-cyan-950">
            <h4 className="font-bold flex items-center gap-1.5 text-cyan-900">
              <Sparkles className="w-3.5 h-3.5 text-cyan-600" /> Grading Model Key
            </h4>
            <ul className="space-y-1.5 text-[11px] text-cyan-800">
              <li>• <strong>Course Modules:</strong> 25% course completion mark.</li>
              <li>• <strong>Regular Assessments:</strong> 25% weight (average of tests).</li>
              <li>• <strong>Final Assessment:</strong> 50% weight (unlocked after 100% modules).</li>
              <li>• <strong>Mock & Daily:</strong> 0% weight practice only.</li>
            </ul>
          </div>
        </div>
      </div>

      {/* Questions & Content Editor Dialog */}
      <Dialog
```

```
        open={!!selectedAssessmentForQuestions}
        onOpenChange={(open) => {
          if (!open) {
            setSelectedAssessmentForQuestions(null)
            setEditingQuestion(null)
          }
        }}
      >
        <DialogContent className="max-w-4xl max-h-[90vh] overflow-y-auto bg-white rounded-3xl p-6 sm:p-7 space-y-5">
          <DialogHeader>
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 border-b border-slate-100 pb-4">
              <div>
                <DialogTitle className="text-xl font-bold text-slate-900 flex items-center gap-2">
                  <FileText className="w-5 h-5 text-cyan-600" />
                  <span>{selectedAssessmentForQuestions?.title}</span>
                </DialogTitle>
                <p className="text-xs text-slate-500 mt-1">
                  Course: <span className="font-bold text-slate-700">{selectedAssessmentForQuestions?.course?.title}</span> &
bull; {formatType(selectedAssessmentForQuestions?.assessment_type)}
                </p>
              </div>

              {/* Mode Switcher Tabs */}
              <div className="flex bg-slate-100 p-1 rounded-xl border border-slate-200 w-fit">
                <button
                  type="button"
                  onClick={() => setActiveModalTab('questions')}
                  className={`px-3.5 py-1.5 rounded-lg text-xs font-bold transition-all cursor-pointer ${
                    activeModalTab === 'questions'
                      ? 'bg-white text-cyan-700 shadow-sm'
                      : 'text-slate-600 hover:text-slate-900'
                  }`}
                >
                  Questions ({questionsList.length})
                </button>
                <button
                  type="button"
                  onClick={() => setActiveModalTab('settings')}
                  className={`px-3.5 py-1.5 rounded-lg text-xs font-bold transition-all cursor-pointer ${
                    activeModalTab === 'settings'
                      ? 'bg-white text-cyan-700 shadow-sm'
                      : 'text-slate-600 hover:text-slate-900'
                  }`}
                >
                  Test Settings
                </button>
              </div>
            </div>
          </DialogHeader>

          {/* TAB 1: QUESTIONS LIST & QUESTION EDITOR */}
          {activeModalTab === 'questions' && (
            <div className="space-y-5">
              {/* Question Editor Form (when editing/adding) */}
              {editingQuestion ? (
                <motion.div
                  initial={{ opacity: 0, y: -10 }}
                  animate={{ opacity: 1, y: 0 }}
                  className="p-5 rounded-2xl bg-slate-50 border border-slate-200 space-y-4"
                >
                  <div className="flex items-center justify-between">
                    <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                      <Edit className="w-4 h-4 text-cyan-600" />
                      <span>{editingQuestion.id ? 'Edit Question' : 'Add New Question'}</span>
                    </h4>
                    <Button
                      size="sm"
                      variant="ghost"
                      onClick={() => setEditingQuestion(null)}
                      className="text-xs text-slate-500 hover:text-slate-900"
                    >
                      Cancel
                    </Button>
                  </div>

                  <div className="space-y-3">
                    <div>
                      <label className="text-xs font-bold text-slate-700 block mb-1">
                        Question Text *
```

```
                        </label>
                        <Textarea
                          rows={3}
                          placeholder="Type the question prompt here..."
                          value={editingQuestion.question_text}
                          onChange={(e) => setEditingQuestion({ ...editingQuestion, question_text: e.target.value })}
                          className="bg-white text-sm"
                        />
                      </div>

                      <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
                        <div>
                          <label className="text-xs font-bold text-slate-700 block mb-1">
                            Question Type
                          </label>
                          <select
                            value={editingQuestion.question_type}
                            onChange={(e) => setEditingQuestion({ ...editingQuestion, question_type: e.target.value as any })}
                            className="w-full text-xs font-semibold p-2.5 bg-white border border-slate-200 rounded-xl"
                          >
                            <option value="mcq">Multiple Choice (4 Options)</option>
                            <option value="true_false">True / False</option>
                          </select>
                        </div>

                        <div>
                          <label className="text-xs font-bold text-slate-700 block mb-1">
                            Difficulty Level
                          </label>
                          <select
                            value={editingQuestion.difficulty}
                            onChange={(e) => setEditingQuestion({ ...editingQuestion, difficulty: e.target.value as any })}
                            className="w-full text-xs font-semibold p-2.5 bg-white border border-slate-200 rounded-xl capitaliz
e"
                          >
                            <option value="easy">Easy</option>
                            <option value="medium">Medium</option>
                            <option value="hard">Hard</option>
                          </select>
                        </div>
                      </div>

                      {/* Options (MCQ) */}
                      {editingQuestion.question_type === 'mcq' ? (
                        <div className="space-y-2.5 pt-2">
                          <label className="text-xs font-bold text-slate-700 block">
                            Options & Correct Answer Choice *
                          </label>
                          <div className="grid grid-cols-1 sm:grid-cols-2 gap-2.5">
                            {[
                              { key: 'option_a', letter: 'A' },
                              { key: 'option_b', letter: 'B' },
                              { key: 'option_c', letter: 'C' },
                              { key: 'option_d', letter: 'D' },
                            ].map(opt => {
                              const isCorrect = editingQuestion.correct_answer === opt.letter
                              return (
                                <div
                                  key={opt.key}
                                  className={`flex items-center gap-2 p-2 rounded-xl border bg-white ${
                                    isCorrect ? 'border-emerald-500 ring-2 ring-emerald-500/20' : 'border-slate-200'
                                  }`}
                                >
                                  <button
                                    type="button"
                                    onClick={() => setEditingQuestion({ ...editingQuestion, correct_answer: opt.letter })}
                                    className={`w-7 h-7 rounded-lg text-xs font-bold flex items-center justify-center shrink-0
cursor-pointer ${
                                      isCorrect ? 'bg-emerald-600 text-white' : 'bg-slate-100 text-slate-600 hover:bg-slate-20
0'
                                    }`}
                                    title="Mark as correct option"
                                  >
                                    {opt.letter}
                                  </button>
                                  <Input
                                    placeholder={`Option ${opt.letter} text...`}
                                    value={(editingQuestion as any)[opt.key]}
                                    onChange={(e) => setEditingQuestion({ ...editingQuestion, [opt.key]: e.target.value })}
```

```
                                    className="h-8 text-xs border-0 focus-visible:ring-0 p-1"
                                  />
                                </div>
                              )
                            })}
                          </div>
                        </div>
                      ) : (
                        /* True / False Options */

                        <div className="space-y-2 pt-2">
                          <label className="text-xs font-bold text-slate-700 block">
                            Correct Answer *
                          </label>
                          <div className="flex gap-3">
                            {['True', 'False'].map(val => (
                              <button
                                key={val}
                                type="button"
                                onClick={() => setEditingQuestion({ ...editingQuestion, correct_answer: val })}
                                className={`px-5 py-2 rounded-xl text-xs font-bold border transition-all cursor-pointer ${
                                  editingQuestion.correct_answer === val
                                    ? 'bg-emerald-600 text-white border-emerald-600'
                                    : 'bg-white text-slate-700 border-slate-200 hover:bg-slate-50'
                                }`}
                              >
                                {val}
                              </button>
                            ))}
                          </div>
                        </div>
                      )}

                      {/* Explanation */}
                      <div className="pt-2">
                        <label className="text-xs font-bold text-slate-700 block mb-1">
                          Explanation (Displayed to trainees upon test completion)
                        </label>
                        <Input
                          placeholder="Why is this answer correct?"
                          value={editingQuestion.explanation}
                          onChange={(e) => setEditingQuestion({ ...editingQuestion, explanation: e.target.value })}
                          className="bg-white text-xs"
                        />
                      </div>
                    </div>

                    <div className="flex justify-end gap-2 pt-2">
                      <Button
                        size="sm"
                        variant="outline"
                        onClick={() => setEditingQuestion(null)}
                        className="text-xs"
                      >
                        Cancel
                      </Button>
                      <Button
                        size="sm"
                        onClick={handleSaveQuestion}
                        disabled={isSavingQuestion}
                        className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs"
                      >
                        {isSavingQuestion ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1" /> : <Check className="w-3.5 h-3.5 mr-1" />}
                        Save Question
                      </Button>
                    </div>
                  </motion.div>
                ) : (
                  /* Header row with Add Question button */

                  <div className="flex items-center justify-between gap-3 bg-slate-50 p-3.5 rounded-2xl border border-slate-200/80">
                    <div>
                      <h4 className="text-xs font-bold text-slate-900 uppercase tracking-wider">
                        Assessment Question Pool
                      </h4>
                      <p className="text-[11px] text-slate-500">
                        Total: <span className="font-bold text-slate-800">{questionsList.length} Questions</span>
```

```
                    </p>
                  </div>

                  <Button
                    size="sm"
                    onClick={() => {
                      setEditingQuestion({
                        question_text: '',
                        option_a: '',
                        option_b: '',
                        option_c: '',
                        option_d: '',
                        correct_answer: 'A',
                        explanation: '',
                        difficulty: 'medium',
                        question_type: 'mcq',
                      })
                    }}
                    className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs rounded-xl shadow-xs"
                  >
                    <Plus className="w-3.5 h-3.5 mr-1" /> Add Question
                  </Button>
                </div>
              )}

              {/* Questions List */}
              {questionsLoading ? (
                <div className="py-12 text-center text-slate-400 flex flex-col items-center justify-center gap-2">
                  <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
                  <span className="text-xs">Loading questions...</span>
                </div>
              ) : questionsList.length === 0 ? (
                <div className="p-8 text-center bg-slate-50 rounded-2xl border border-slate-100 space-y-2">
                  <HelpCircle className="w-8 h-8 text-slate-300 mx-auto" />
                  <p className="text-xs font-bold text-slate-700">No questions found in this assessment</p>
                  <p className="text-[11px] text-slate-400">Click "+ Add Question" to create the first question item.</p>
                </div>
              ) : (
                <div className="space-y-3 max-h-[55vh] overflow-y-auto pr-1">
                  {questionsList.map((q, idx) => {
                    const opts = parseQuestionOptions(q.options)
                    const isTF = (q.options as any)?._question_type === 'true_false'

                    return (
                      <div
                        key={q.id || idx}
                        className="p-4 rounded-2xl border border-slate-200 bg-white hover:border-cyan-300 transition-all spac
e-y-3"
                      >
                        <div className="flex items-start justify-between gap-3">
                          <div className="flex items-start gap-2.5">
                            <span className="w-6 h-6 rounded-lg bg-slate-100 text-slate-800 text-xs font-extrabold flex items
-center justify-center shrink-0">
                              {idx + 1}
                            </span>
                            <div className="space-y-1">
                              <p className="text-sm font-bold text-slate-900 leading-snug">
                                {q.question_text}
                              </p>
                              <div className="flex items-center gap-2">
                                <Badge variant="outline" className="text-[10px] font-semibold text-slate-600 capitalize">
                                  {q.difficulty || (q.options as any)?._difficulty || 'medium'}
                                </Badge>
                                <span className="text-[11px] text-slate-400">
                                  {isTF ? 'True/False' : 'Multiple Choice'}
                                </span>
                              </div>
                            </div>
                          </div>

                          {/* Question Action Buttons */}
                          <div className="flex items-center gap-1.5 shrink-0">
                            <Button
                              size="sm"
                              variant="outline"
                              onClick={() => {
                                setEditingQuestion({
                                  id: q.id,
                                  question_text: q.question_text,
```

```
                                    option_a: opts.A,
                                    option_b: opts.B,
                                    option_c: opts.C,
                                    option_d: opts.D,
                                    correct_answer: q.correct_answer,
                                    explanation: q.explanation || '',
                                    difficulty: q.difficulty || (q.options as any)?._difficulty || 'medium',
                                    question_type: isTF ? 'true_false' : 'mcq',
                                  })
                                }}
                                className="h-8 text-xs font-bold border-slate-200 hover:bg-slate-100"
                              >
                                <Edit className="w-3.5 h-3.5 mr-1 text-slate-500" /> Edit
                              </Button>
                              {q.id && (
                                <Button
                                  size="sm"
                                  variant="ghost"
                                  onClick={() => handleDeleteQuestion(q.id!)}
                                  className="h-8 w-8 p-0 text-slate-400 hover:text-rose-600 hover:bg-rose-50"
                                >
                                  <Trash2 className="w-4 h-4" />
                                </Button>
                              )}
                            </div>
                          </div>

                          {/* Options Display */}
                          <div className="grid grid-cols-1 sm:grid-cols-2 gap-2 pt-1">
                            {isTF ? (
                              ['True', 'False'].map(optVal => {
                                const isCorrect = q.correct_answer === optVal
                                return (
                                  <div
                                    key={optVal}
                                    className={`p-2 rounded-xl text-xs flex items-center gap-2 border ${
                                      isCorrect ? 'bg-emerald-50 border-emerald-300 text-emerald-950 font-bold' : 'bg-slate-50
border-slate-200 text-slate-700'
                                    }`}
                                  >
                                    {isCorrect && <CheckCircle className="w-3.5 h-3.5 text-emerald-600 shrink-0" />}
                                    <span>{optVal}</span>
                                  </div>
                                )
                              })
                            ) : (
                              [
                                { letter: 'A', text: opts.A },
                                { letter: 'B', text: opts.B },
                                { letter: 'C', text: opts.C },
                                { letter: 'D', text: opts.D },
                              ]
                                .filter(item => Boolean(item.text))
                                .map(item => {
                                  const isCorrect = q.correct_answer === item.letter || q.correct_answer === item.text
                                  return (
                                    <div
                                      key={item.letter}
                                      className={`p-2 rounded-xl text-xs flex items-center gap-2 border ${
                                        isCorrect ? 'bg-emerald-50 border-emerald-300 text-emerald-950 font-bold' : 'bg-slate-5
0 border-slate-200 text-slate-700'
                                      }`}
                                    >
                                      <span className={`w-5 h-5 rounded-md text-[10px] font-extrabold flex items-center justify
-center shrink-0 ${
                                        isCorrect ? 'bg-emerald-600 text-white' : 'bg-slate-200 text-slate-700'
                                      }`}>
                                        {item.letter}
                                      </span>
                                      <span className="truncate">{item.text}</span>
                                    </div>
                                  )
                                })
                            )}
                          </div>

                          {/* Explanation Note */}
                          {q.explanation && (
                            <div className="text-[11px] text-slate-500 bg-slate-50 p-2.5 rounded-xl border border-slate-100">
```

```
                            <strong>Explanation:</strong> {q.explanation}
                          </div>
                        )}
                      </div>
                    )
                  })}
                </div>
              )}
            </div>
          )}

          {/* TAB 2: ASSESSMENT SETTINGS */}
          {activeModalTab === 'settings' && (
            <div className="space-y-4 p-1">
              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                <div className="sm:col-span-2">
                  <label className="text-xs font-bold text-slate-700 block mb-1">
                    Assessment Title *
                  </label>
                  <Input
                    value={assessmentSettingsForm.title}
                    onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, title: e.target.value })}
                    className="text-sm font-semibold"
                  />
                </div>

                <div>
                  <label className="text-xs font-bold text-slate-700 block mb-1">
                    Duration (Minutes) *
                  </label>
                  <Input
                    type="number"
                    min={5}
                    max={360}
                    value={assessmentSettingsForm.duration_minutes}
                    onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, duration_minutes: Number(e.targe
t.value) })}
                    className="text-sm font-semibold"
                  />
                </div>

                <div>
                  <label className="text-xs font-bold text-slate-700 block mb-1">
                    Passing Threshold Score (%) *
                  </label>
                  <Input
                    type="number"
                    min={1}
                    max={100}
                    value={assessmentSettingsForm.passing_score}
                    onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, passing_score: Number(e.target.va
lue) })}
                    className="text-sm font-semibold"
                  />
                </div>

                <div>
                  <label className="text-xs font-bold text-slate-700 block mb-1">
                    Scheduled Date
                  </label>
                  <Input
                    type="date"
                    value={assessmentSettingsForm.scheduled_date}
                    onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, scheduled_date: e.target.value
})}
                    className="text-sm font-semibold"
                  />
                </div>

                <div>
                  <label className="text-xs font-bold text-slate-700 block mb-1">
                    Start Time - End Time
                  </label>
                  <div className="flex gap-2">
                    <Input
                      type="time"
                      value={assessmentSettingsForm.start_time}
                      onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, start_time: e.target.value })}
                      className="text-sm"
```

```
                    />
                    <Input
                      type="time"
                      value={assessmentSettingsForm.end_time}
                      onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, end_time: e.target.value })}
                      className="text-sm"
                    />
                  </div>
                </div>

                <div className="sm:col-span-2 pt-2">
                  <label className="flex items-center gap-3 p-3 rounded-2xl bg-slate-50 border border-slate-200 cursor-pointe
r">
                    <input
                      type="checkbox"
                      checked={assessmentSettingsForm.requires_sea}
                      onChange={(e) => setAssessmentSettingsForm({ ...assessmentSettingsForm, requires_sea: e.target.checked
})}
                      className="w-4 h-4 rounded text-cyan-600 focus:ring-cyan-500"
                    />
                    <div>
                      <span className="text-xs font-bold text-slate-900 block">Require Secure Evaluation Browser (SEA Proctor
ed Mode)</span>
                      <span className="text-[11px] text-slate-500">Locks full-screen environment and blocks tab switches duri
ng assessment execution.</span>
                    </div>
                  </label>
                </div>
              </div>

              <div className="flex justify-end gap-2 pt-4 border-t border-slate-100">
                <Button
                  onClick={handleSaveAssessmentSettings}
                  disabled={isSavingSettings}
                  className="bg-cyan-600 hover:bg-cyan-700 text-white font-bold text-xs px-5 py-2.5 rounded-xl shadow-xs"
                >
                  {isSavingSettings ? <Loader2 className="w-3.5 h-3.5 animate-spin mr-1.5" /> : <Check className="w-3.5 h-3.5
mr-1.5" />}
                  Save Assessment Settings
                </Button>
              </div>
            </div>
          )}
        </DialogContent>
      </Dialog>

      {/* Submissions Dialog */}
      <Dialog
        open={!!selectedAssessmentForSubmissions}
        onOpenChange={(open) => {
          if (!open) setSelectedAssessmentForSubmissions(null)
        }}
      >
        <DialogContent className="max-w-2xl bg-white rounded-3xl p-6">
          <DialogHeader>
            <DialogTitle className="text-lg font-bold text-slate-900 flex items-center gap-2">
              <Users className="w-5 h-5 text-cyan-600" />
              <span>Trainee Submissions — {selectedAssessmentForSubmissions?.title}</span>
            </DialogTitle>
          </DialogHeader>

          <div className="space-y-4 mt-3">
            {(() => {
              if (!selectedAssessmentForSubmissions) return null
              const atts = attemptsByAssessment[selectedAssessmentForSubmissions.id] || []
              if (atts.length === 0) {
                return (
                  <div className="p-8 text-center text-slate-400 text-xs bg-slate-50 rounded-2xl border border-slate-100">
                    No trainees have submitted this assessment yet.
                  </div>
                )
              }

              return (
                <div className="overflow-x-auto max-h-96">
                  <table className="w-full text-xs">
                    <thead>
                      <tr className="border-b border-slate-200 text-slate-500 font-bold text-left bg-slate-50">
                        <th className="px-3 py-2.5">Trainee</th>
```

```
                        <th className="px-3 py-2.5">Submitted</th>
                        <th className="px-3 py-2.5">Score</th>
                        <th className="px-3 py-2.5">Status</th>
                        <th className="px-3 py-2.5 text-right">Result</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-slate-100">
                      {atts.map((att: any) => {
                        const isPassed = (att.score ?? 0) >= (selectedAssessmentForSubmissions.passing_score ?? 50)
                        return (
                          <tr key={att.id} className="hover:bg-slate-50/80">
                            <td className="px-3 py-2.5">
                              <p className="font-bold text-slate-900">{att.user?.full_name || 'Trainee'}</p>
                              <p className="text-[10px] text-slate-400">{att.user?.email}</p>
                            </td>
                            <td className="px-3 py-2.5 text-slate-600">
                              {att.submitted_at ? format(new Date(att.submitted_at), 'd MMM yyyy, HH:mm') : '—'}
                            </td>
                            <td className="px-3 py-2.5 font-bold text-slate-900">
                              {att.score !== null ? `${att.score}%` : 'Pending'}
                            </td>
                            <td className="px-3 py-2.5">
                              <span className={`px-2 py-0.5 rounded-full text-[10px] font-extrabold uppercase tracking-wider
${
                                isPassed ? 'bg-emerald-50 text-emerald-700 border border-emerald-200' : 'bg-rose-50 text-rose
-700 border border-rose-200'
                              }`}>
                                {isPassed ? 'Passed' : 'Failed'}
                              </span>
                            </td>
                            <td className="px-3 py-2.5 text-right">
                              <Button
                                size="sm"
                                variant="outline"
                                onClick={() => {
                                  window.open(`/trainee/courses/${selectedAssessmentForSubmissions.course_id}/assessments/${s
electedAssessmentForSubmissions.id}/result/${att.id}`, '_blank')
                                }}
                                className="h-7 text-[10px] px-2.5 border-slate-200 text-cyan-700 hover:bg-cyan-50"
                              >
                                View Answers
                              </Button>
                            </td>
                          </tr>
                        )
                      })}
                    </tbody>
                  </table>
                </div>
              )
            })()}
          </div>
        </DialogContent>
      </Dialog>
    </div>
  )
}
```

src/features/admin/AdminContactMessages.tsx TSX 19.0 KB 423 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import {
  Mail, MessageSquare, Search, Filter, CheckCircle2, Clock,
  Trash2, Eye, Reply, Building, Calendar, AlertCircle, RefreshCw,
  User, Check, ChevronDown, Sparkles, X
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@/components/ui/card'
import { supabase } from '@/lib/supabase'
import { toast } from 'sonner'

export interface ContactMessage {
  id: string
  name: string
  email: string
  department?: string
  subject?: string
  message: string
  status: 'unread' | 'read' | 'resolved'
  created_at: string
}

export function AdminContactMessages() {
  const [messages, setMessages] = useState<ContactMessage[]>([])
  const [loading, setLoading] = useState(true)
  const [searchQuery, setSearchQuery] = useState('')
  const [statusFilter, setStatusFilter] = useState<'all' | 'unread' | 'read' | 'resolved'>('all')
  const [selectedMessage, setSelectedMessage] = useState<ContactMessage | null>(null)
  const [deletingId, setDeletingId] = useState<string | null>(null)

  const fetchMessages = async () => {
    setLoading(true)
    try {
      let dbMessages: ContactMessage[] = []
      const { data, error } = await (supabase as any)
        .from('contact_messages')
        .select('*')
        .order('created_at', { ascending: false })

      if (!error && data) {
        dbMessages = data as unknown as ContactMessage[]
      }

      // Check local storage fallback

      const local = JSON.parse(localStorage.getItem('local_contact_messages') || '[]')

      // Merge unique

      const mergedMap = new Map<string, ContactMessage>()
      for (const m of local) mergedMap.set(m.id, m)
      for (const m of dbMessages) mergedMap.set(m.id, m)

      setMessages(Array.from(mergedMap.values()).sort((a, b) =>
        new Date(b.created_at).getTime() - new Date(a.created_at).getTime()
      ))
    } catch (err) {
      console.error('Failed to load contact messages:', err)
      toast.error('Failed to fetch contact messages.')
    } finally {
      setLoading(false)
    }
  }

  useEffect(() => {
    fetchMessages()
  }, [])

  const handleUpdateStatus = async (id: string, newStatus: 'unread' | 'read' | 'resolved') => {
    try {
      // Update in Supabase

      await (supabase as any)
        .from('contact_messages')
```

```
        .update({ status: newStatus })
        .eq('id', id)

      // Update in local state

      setMessages(prev =>
        prev.map(m => (m.id === id ? { ...m, status: newStatus } : m))
      )

      if (selectedMessage && selectedMessage.id === id) {
        setSelectedMessage(prev => prev ? { ...prev, status: newStatus } : null)
      }

      // Update local storage

      const local = JSON.parse(localStorage.getItem('local_contact_messages') || '[]')
      const updatedLocal = local.map((m: ContactMessage) => m.id === id ? { ...m, status: newStatus } : m)
      localStorage.setItem('local_contact_messages', JSON.stringify(updatedLocal))

      toast.success(`Message marked as ${newStatus}`)
    } catch (err) {
      toast.error('Failed to update message status.')
    }
  }

  const handleDeleteMessage = async (id: string) => {
    if (!window.confirm('Are you sure you want to delete this message?')) return
    setDeletingId(id)
    try {
      await (supabase as any).from('contact_messages').delete().eq('id', id)
      setMessages(prev => prev.filter(m => m.id !== id))
      if (selectedMessage?.id === id) setSelectedMessage(null)

      const local = JSON.parse(localStorage.getItem('local_contact_messages') || '[]')
      const filteredLocal = local.filter((m: ContactMessage) => m.id !== id)
      localStorage.setItem('local_contact_messages', JSON.stringify(filteredLocal))

      toast.success('Message deleted successfully.')
    } catch (err) {
      toast.error('Failed to delete message.')
    } finally {
      setDeletingId(null)
    }
  }

  const filteredMessages = messages.filter(m => {
    const matchesStatus = statusFilter === 'all' || m.status === statusFilter
    const q = searchQuery.toLowerCase()
    const matchesQuery =
      !q ||
      m.name.toLowerCase().includes(q) ||
      m.email.toLowerCase().includes(q) ||
      (m.department && m.department.toLowerCase().includes(q)) ||
      (m.subject && m.subject.toLowerCase().includes(q)) ||
      m.message.toLowerCase().includes(q)

    return matchesStatus && matchesQuery
  })

  const totalCount = messages.length
  const unreadCount = messages.filter(m => m.status === 'unread').length
  const resolvedCount = messages.filter(m => m.status === 'resolved').length

  return (
    <div className="space-y-6">
      {/* Top Stats Cards */}
      <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
        <Card className="bg-white border-slate-200/90 rounded-3xl shadow-sm">
          <CardContent className="p-5 flex items-center justify-between">
            <div>
              <p className="text-xs font-semibold text-slate-500 uppercase tracking-wider">Total Inquiries</p>
              <h3 className="text-2xl font-black text-slate-900 mt-1">{totalCount}</h3>
            </div>
            <div className="w-12 h-12 rounded-2xl bg-cyan-50 text-cyan-600 flex items-center justify-center font-bold">
              <Mail className="w-6 h-6" />
            </div>
          </CardContent>
        </Card>

```

```
        <Card className="bg-white border-slate-200/90 rounded-3xl shadow-sm">
          <CardContent className="p-5 flex items-center justify-between">
            <div>
              <p className="text-xs font-semibold text-slate-500 uppercase tracking-wider">Unread Messages</p>
              <h3 className="text-2xl font-black text-amber-600 mt-1">{unreadCount}</h3>
            </div>
            <div className="w-12 h-12 rounded-2xl bg-amber-50 text-amber-600 flex items-center justify-center font-bold">
              <Clock className="w-6 h-6" />
            </div>
          </CardContent>
        </Card>

        <Card className="bg-white border-slate-200/90 rounded-3xl shadow-sm">
          <CardContent className="p-5 flex items-center justify-between">
            <div>
              <p className="text-xs font-semibold text-slate-500 uppercase tracking-wider">Resolved</p>
              <h3 className="text-2xl font-black text-emerald-600 mt-1">{resolvedCount}</h3>
            </div>
            <div className="w-12 h-12 rounded-2xl bg-emerald-50 text-emerald-600 flex items-center justify-center font-bold">
              <CheckCircle2 className="w-6 h-6" />
            </div>
          </CardContent>
        </Card>
      </div>

      {/* Main Filter & Search Bar */}
      <Card className="bg-white border-slate-200/90 rounded-3xl shadow-sm overflow-hidden">
        <CardHeader className="p-6 border-b border-slate-100 bg-slate-50/50">
          <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
            <div>
              <CardTitle className="text-base font-bold text-slate-900 flex items-center gap-2">
                <MessageSquare className="w-5 h-5 text-cyan-600" />
                Contact Inquiries &amp;amp; Messages
              </CardTitle>
              <CardDescription className="text-xs text-slate-500 mt-0.5">
                Review and respond to messages submitted through the public contact page.
              </CardDescription>
            </div>

            <div className="flex items-center gap-2">
              <Button
                variant="outline"
                size="sm"
                onClick={fetchMessages}
                disabled={loading}
                className="rounded-xl text-xs h-9 gap-1.5 border-slate-300"
              >
                <RefreshCw className={`w-3.5 h-3.5 ${loading ? 'animate-spin' : ''}`} />
                Refresh
              </Button>
            </div>
          </div>

          <div className="mt-4 flex flex-col sm:flex-row items-center gap-3">
            <div className="relative flex-1 w-full">
              <Search className="w-4 h-4 text-slate-400 absolute left-3 top-1/2 -translate-y-1/2" />
              <Input
                placeholder="Search by sender name, email, department, or keyword..."
                value={searchQuery}
                onChange={e => setSearchQuery(e.target.value)}
                className="pl-9 h-10 rounded-xl text-xs bg-white border-slate-300"
              />
            </div>

            <div className="flex items-center gap-1.5 w-full sm:w-auto overflow-x-auto pb-1 sm:pb-0">
              {(['all', 'unread', 'read', 'resolved'] as const).map(tab => (
                <button
                  key={tab}
                  onClick={() => setStatusFilter(tab)}
                  className={`px-3 py-1.5 rounded-xl text-xs font-bold capitalize transition-all shrink-0 cursor-pointer ${
                    statusFilter === tab
                      ? 'bg-cyan-600 text-white shadow-sm'
                      : 'bg-slate-100 text-slate-600 hover:bg-slate-200'
                  }`}
                >
                  {tab} {tab === 'unread' && unreadCount > 0 ? `(${unreadCount})` : ''}
                </button>
              ))}
            </div>
```

```
          </div>
        </CardHeader>

        <CardContent className="p-0">
          {loading ? (
            <div className="p-12 text-center text-slate-400 text-xs">
              <RefreshCw className="w-6 h-6 animate-spin mx-auto text-cyan-600 mb-2" />
              Loading messages...
            </div>
          ) : filteredMessages.length === 0 ? (
            <div className="p-12 text-center text-slate-400 space-y-2">
              <Mail className="w-10 h-10 mx-auto text-slate-300 stroke-1" />
              <p className="text-sm font-bold text-slate-700">No contact messages found</p>
              <p className="text-xs text-slate-400">
                {searchQuery || statusFilter !== 'all'
                  ? 'Try adjusting your search or filters.'
                  : 'New messages submitted through the contact page will appear here.'}
              </p>
            </div>
          ) : (
            <div className="divide-y divide-slate-100">
              {filteredMessages.map(item => (
                <div
                  key={item.id}
                  onClick={() => {
                    setSelectedMessage(item)
                    if (item.status === 'unread') {
                      handleUpdateStatus(item.id, 'read')
                    }
                  }}
                  className={`p-5 sm:p-6 transition-colors hover:bg-slate-50 cursor-pointer flex flex-col sm:flex-row sm:items-center justify-between gap-4 ${
                    item.status === 'unread' ? 'bg-cyan-50/40 font-medium' : ''
                  }`}
                >
                  <div className="flex items-start gap-3.5 min-w-0 flex-1">
                    <div className="w-10 h-10 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 text-white flex items-center justify-center text-sm font-bold shrink-0 shadow-sm mt-0.5">
                      {item.name.charAt(0).toUpperCase()}
                    </div>

                    <div className="min-w-0 flex-1 space-y-1">
                      <div className="flex flex-wrap items-center gap-2">
                        <span className="text-sm font-bold text-slate-900">{item.name}</span>
                        <span className="text-xs text-slate-500">({item.email})</span>
                        {item.department && (
                          <span className="text-[10px] font-bold px-2 py-0.5 rounded-full bg-slate-100 text-slate-700 border border-slate-200">
                            {item.department}
                          </span>
                        )}
                        <span
                          className={`text-[10px] font-bold px-2 py-0.5 rounded-full uppercase tracking-wider ${
                            item.status === 'unread'
                              ? 'bg-amber-100 text-amber-800 border border-amber-200'
                              : item.status === 'resolved'
                              ? 'bg-emerald-100 text-emerald-800 border border-emerald-200'
                              : 'bg-slate-100 text-slate-600 border border-slate-200'
                          }`}
                        >
                          {item.status}
                        </span>
                      </div>

                      <h4 className="text-xs font-bold text-slate-800 line-clamp-1">
                        {item.subject || 'General Inquiry'}
                      </h4>

                      <p className="text-xs text-slate-500 line-clamp-2">
                        {item.message}
                      </p>
                    </div>
                  </div>

                  <div className="flex items-center gap-3 shrink-0 self-end sm:self-center">
                    <span className="text-[11px] text-slate-400 font-medium">
                      {new Date(item.created_at).toLocaleDateString()}
                    </span>
                    <Button
```

```
                      variant="ghost"
                      size="sm"
                      onClick={(e) => {
                        e.stopPropagation()
                        setSelectedMessage(item)
                      }}
                      className="rounded-xl text-xs h-8 text-cyan-700 hover:bg-cyan-50"
                    >
                      <Eye className="w-3.5 h-3.5 mr-1" /> View
                    </Button>
                    <Button
                      variant="ghost"
                      size="sm"
                      disabled={deletingId === item.id}
                      onClick={(e) => {
                        e.stopPropagation()
                        handleDeleteMessage(item.id)
                      }}
                      className="rounded-xl text-xs h-8 text-rose-600 hover:text-rose-700 hover:bg-rose-50"
                    >
                      <Trash2 className="w-3.5 h-3.5" />
                    </Button>
                  </div>
                </div>
              ))}
            </div>
          )}
        </CardContent>
      </Card>

      {/* Message Details Modal */}
      <AnimatePresence>
        {selectedMessage && (
          <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-900/60 backdrop-blur-sm">
            <motion.div
              initial={{ opacity: 0, scale: 0.95 }}
              animate={{ opacity: 1, scale: 1 }}
              exit={{ opacity: 0, scale: 0.95 }}
              className="bg-white rounded-3xl max-w-2xl w-full overflow-hidden shadow-2xl border border-slate-200 flex flex-co
l max-h-[90vh]"
            >
              <div className="p-6 bg-slate-50 border-b border-slate-100 flex items-start justify-between gap-4">
                <div className="flex items-center gap-3 min-w-0">
                  <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 text-white flex items-cent
er justify-center font-bold text-lg shrink-0">
                    {selectedMessage.name.charAt(0).toUpperCase()}
                  </div>
                  <div>
                    <h3 className="text-base font-bold text-slate-900">{selectedMessage.name}</h3>
                    <p className="text-xs text-slate-500">{selectedMessage.email}</p>
                  </div>
                </div>

                <button
                  onClick={() => setSelectedMessage(null)}
                  className="p-1.5 rounded-xl text-slate-400 hover:text-slate-600 hover:bg-slate-200/60 transition-colors"
                >
                  <X className="w-5 h-5" />
                </button>
              </div>

              <div className="p-6 overflow-y-auto space-y-4 text-xs sm:text-sm flex-1">
                <div className="grid grid-cols-2 sm:grid-cols-3 gap-3 p-3.5 rounded-2xl bg-slate-50 border border-slate-100 te
xt-xs">
                  <div>
                    <span className="text-slate-400 block font-semibold">Department</span>
                    <span className="font-bold text-slate-800">{selectedMessage.department || 'MoES'}</span>
                  </div>
                  <div>
                    <span className="text-slate-400 block font-semibold">Date Sent</span>
                    <span className="font-bold text-slate-800">{new Date(selectedMessage.created_at).toLocaleString()}</span>
                  </div>
                  <div>
                    <span className="text-slate-400 block font-semibold">Status</span>
                    <span className="font-bold text-cyan-700 capitalize">{selectedMessage.status}</span>
                  </div>
                </div>

                <div>
```

```
                  <h4 className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1">Subject</h4>
                  <p className="text-sm font-bold text-slate-900">{selectedMessage.subject || 'General Inquiry'}</p>
                </div>

                <div>
                  <h4 className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1.5">Message Content</h4>
                  <div className="p-4 rounded-2xl bg-slate-50 border border-slate-200/80 text-slate-700 leading-relaxed whitespace-pre-wrap">
                    {selectedMessage.message}
                  </div>
                </div>
              </div>

              <div className="p-4 bg-slate-50 border-t border-slate-100 flex flex-wrap items-center justify-between gap-3">
                <div className="flex items-center gap-2">
                  <Button
                    size="sm"
                    variant="outline"
                    onClick={() => handleUpdateStatus(selectedMessage.id, selectedMessage.status === 'resolved' ? 'read' : 'resolved')}
                    className="rounded-xl text-xs h-9 gap-1.5 border-slate-300"
                  >
                    <Check className="w-3.5 h-3.5" />
                    {selectedMessage.status === 'resolved' ? 'Mark as Unresolved' : 'Mark as Resolved'}
                  </Button>
                </div>

                <div className="flex items-center gap-2">
                  <a
                    href={`mailto:${selectedMessage.email}?subject=Re: ${encodeURIComponent(selectedMessage.subject || 'Capacity Connect Inquiry')}`}
                    className="inline-flex items-center gap-1.5 px-4 py-2 rounded-xl bg-gradient-to-r from-cyan-600 to-blue-600 hover:from-cyan-700 hover:to-blue-700 text-white font-bold text-xs shadow-md shadow-cyan-600/20 transition-all"
                  >
                    <Reply className="w-3.5 h-3.5" /> Reply via Email
                  </a>
                </div>
              </div>
            </motion.div>
          </div>
        )}
      </AnimatePresence>
    </div>
  )
}

```

src/features/admin/AdminCourseCreatePage.tsx TSX 118.0 KB 2,428 Lines

```tsx
import React, { useState, useEffect, useRef, useCallback, useMemo } from 'react'
import { useNavigate } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { Database } from '@/integrations/supabase/types'
import { motion, AnimatePresence } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import {
  Select, SelectContent, SelectItem, SelectTrigger, SelectValue,
} from '@/components/ui/select'
import {
  ArrowLeft, ArrowRight, CheckCircle, Check, Loader2, Plus, Upload, X,
  BookOpen, Settings, Target, Eye, AlertCircle, FileText, File, Image,
  Video, Link2, ExternalLink, Globe, Trash2, Layers, Trophy, Film, Camera,
  Sparkles, AlignLeft, HelpCircle, CheckSquare, Award, ArrowUp, ArrowDown, Edit3, GripVertical, Clock,
  UserCheck, Zap, TrendingUp, Megaphone
} from 'lucide-react'
import { toast } from 'sonner'
import { ImageCropperModal } from '@/components/ui/ImageCropperModal'
import { CourseCertificateStep } from '@/features/courses/CourseCertificateStep'

type Skill = Database['public']['Tables']['skills']['Row']

type Trainer = {
  id: string
  full_name: string
  email: string
  qualifications?: string | null
  years_of_experience?: number | null
  bio?: string | null
  expertise_areas?: string[] | null
  skills?: string[]
  department?: string | null
  designation?: string | null
  work_experience?: string | null
  availability?: string | null
}

export interface QuizQuestion {
  id: string
  question: string
  options: string[]
  correct_option: number
  explanation?: string
}

export interface ModuleItem {
  id: string
  type: 'photo' | 'video' | 'link' | 'text' | 'quiz'
  title: string
  content?: string
  url?: string
  duration_minutes?: number
  duration_seconds?: number
  file?: File
  previewUrl?: string
  storagePath?: string
  fileName?: string
  fileSize?: number
  quiz_data?: QuizQuestion
}

export interface CourseModule {
  id: string
  title: string
  description: string
  items: ModuleItem[]
  quiz_questions?: QuizQuestion[]
```

```
  passing_score?: number
}

const detailsSchema = z.object({
  title: z.string().min(5, 'Title must be at least 5 characters'),
  description: z.string().min(10, 'Description must be at least 10 characters'),
  course_type: z.string().min(2, 'Course type is required'),
  department: z.string().optional(),
})

const settingsSchema = z.object({
  duration_hours: z.coerce.number().positive('Duration must be positive').optional(),
  passing_score: z.coerce.number().min(1).max(100).optional(),
  meet_link: z.string().url('Must be a valid URL').optional().or(z.literal('')),
  start_date: z.string().optional(),
  end_date: z.string().optional(),
  planned_assessments_count: z.coerce.number().min(0).optional(),
  planned_mock_tests_count: z.coerce.number().min(0).optional(),
  final_test_date: z.string().optional(),
  final_test_start_time: z.string().optional(),
  final_test_end_time: z.string().optional(),
  delivery_mode: z.enum(['recorded', 'live', 'hybrid']).optional(),
  max_trainees: z.coerce.number().min(50, 'Minimum capacity is 50').max(250, 'Maximum capacity is 250').optional(),
  trainer_suggestion: z.string().optional(),
}).superRefine((data, ctx) => {
  if (data.start_date) {
    const start = new Date(data.start_date)
    const today = new Date()
    today.setHours(0, 0, 0, 0)
    const diffDays = Math.ceil((start.getTime() - today.getTime()) / (1000 * 60 * 60 * 24))
    if (diffDays < 30) {
      ctx.addIssue({
        code: z.ZodIssueCode.custom,
        message: 'Course must start at least 1 month from today',
        path: ['start_date']
      })
    }
  }
})

const objectivesSchema = z.object({
  learning_objectives: z.string().min(5, 'Describe what learners will achieve from this course'),
})

const sessionFlowSchema = z.object({
  session_flow_text: z.string().optional(),
})

const trainerSchema = z.object({
  trainer_id: z.string().min(1, 'Please select a trainer to lead this course'),
  assignment_message: z.string().optional(),
})

type DetailsData = z.infer<typeof detailsSchema>
type SettingsData = z.infer<typeof settingsSchema>
type ObjectivesData = z.infer<typeof objectivesSchema>
type SessionFlowData = z.infer<typeof sessionFlowSchema>
type TrainerData = z.infer<typeof trainerSchema>

interface PendingMaterial {
  id: string
  file?: File
  fileName: string
  fileSize: number
  mimeType: string
  materialType: 'file' | 'link' | 'video'
  url?: string
  storagePath?: string
  preview?: string
}

const MAX_FILE_SIZE = 100 * 1024 * 1024
const ACCEPTED_EXTENSIONS = ['.pdf', '.doc', '.docx', '.pptx', '.ppt', '.txt', '.mp4', '.webm', '.mov', '.png', '.jpg', '.jpe
g', '.gif', '.webp']

function getFileIcon(mime: string) {
  if (mime.startsWith('video/')) return Video
  if (mime.startsWith('image/')) return Image
  if (mime.includes('pdf')) return FileText
```

```
  return File
}

function formatFileSize(bytes: number) {
  if (bytes < 1024) return `${bytes} B`
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`
}

const STEPS = [
  { id: 1, label: 'Course Details', icon: BookOpen },
  { id: 2, label: 'Course Configuration', icon: Settings },
  { id: 3, label: 'Course Outline', icon: FileText },
  { id: 4, label: 'Learning & Skills', icon: Target },
  { id: 5, label: 'Modules', icon: Layers },
  { id: 6, label: 'Materials', icon: FileText },
  { id: 7, label: 'Trainer', icon: UserCheck },
  { id: 8, label: 'Certificate', icon: Award },
  { id: 9, label: 'Review & Publish', icon: Eye },
]

const fadeUp = { hidden: { opacity: 0, y: 20 }, visible: { opacity: 1, y: 0, transition: { duration: 0.4 } } }

export function AdminCourseCreatePage() {
  const { user, profile } = useAuth()
  const navigate = useNavigate()

  const [step, setStep] = useState(1)
  const [saving, setSaving] = useState(false)
  const [hasCertificate, setHasCertificate] = useState(true)
  const [certificateTemplateUrl, setCertificateTemplateUrl] = useState<string | null>(null)
  const [certificateTemplateName, setCertificateTemplateName] = useState<string | null>(null)
  const [skills, setSkills] = useState<Skill[]>([])
  const [selectedSkills, setSelectedSkills] = useState<string[]>([])
  const [trainers, setTrainers] = useState<Trainer[]>([])
  const [loadingTrainers, setLoadingTrainers] = useState(true)
  const [thumbnail, setThumbnail] = useState<File | null>(null)
  const [thumbnailPreview, setThumbnailPreview] = useState<string | null>(null)
  const [customSkillName, setCustomSkillName] = useState('')
  const [addingSkill, setAddingSkill] = useState(false)
  const [isCustomType, setIsCustomType] = useState(false)
  const [isCropperOpen, setIsCropperOpen] = useState(false)
  const [rawImageFile, setRawImageFile] = useState<File | null>(null)
  const [isAnalyzingCompetency, setIsAnalyzingCompetency] = useState(false)
  const [trainerTab, setTrainerTab] = useState<'ai_recommended' | 'all'>('ai_recommended')
  const thumbRef = useRef<HTMLInputElement>(null)

  // Modules state

  const [modules, setModules] = useState<CourseModule[]>([
    {
      id: crypto.randomUUID(),
      title: 'Module 1: Introduction & Fundamentals',
      description: 'Foundational concepts, overview content, photos, and video resources.',
      items: [],
    },
  ])

  // Active module item adding state

  const [activeModuleId, setActiveModuleId] = useState<string | null>(null)
  const [editingItemId, setEditingItemId] = useState<string | null>(null)
  const [addingItemType, setAddingItemType] = useState<'photo' | 'video' | 'link' | 'text' | null>(null)
  const [itemTitle, setItemTitle] = useState('')
  const [itemContent, setItemContent] = useState('')
  const [itemUrl, setItemUrl] = useState('')
  const [itemDurationMinutes, setItemDurationMinutes] = useState<string>('5')
  const [itemFile, setItemFile] = useState<File | null>(null)
  const [itemPreview, setItemPreview] = useState<string | null>(null)
  const moduleFileInputRef = useRef<HTMLInputElement>(null)

  // Active module quiz question state

  const [activeQuizModuleId, setActiveQuizModuleId] = useState<string | null>(null)
  const [editingQuestionId, setEditingQuestionId] = useState<string | null>(null)
  const [quizQuestionText, setQuizQuestionText] = useState('')
  const [quizOptions, setQuizOptions] = useState<string[]>(['', '', '', ''])
  const [quizCorrectOption, setQuizCorrectOption] = useState<number>(0)
  const [quizExplanation, setQuizExplanation] = useState('')
```

```
  const [isGeneratingAiQuiz, setIsGeneratingAiQuiz] = useState(false)

  // Materials state

  const [pendingMaterials, setPendingMaterials] = useState<PendingMaterial[]>([])
  const [dragOver, setDragOver] = useState(false)
  const [activeTab, setActiveTab] = useState<'files' | 'links'>('files')
  const [linkUrl, setLinkUrl] = useState('')
  const [linkTitle, setLinkTitle] = useState('')
  const fileInputRef = useRef<HTMLInputElement>(null)

  const detailsForm = useForm<DetailsData>({
    resolver: zodResolver(detailsSchema),
    defaultValues: { title: '', description: '', course_type: 'standard', department: '' },
  })

  const settingsForm = useForm<SettingsData>({
    resolver: zodResolver(settingsSchema),
    defaultValues: { passing_score: 60, delivery_mode: 'recorded' },
  })

  const objectivesForm = useForm<ObjectivesData>({
    resolver: zodResolver(objectivesSchema),
    defaultValues: { learning_objectives: '' },
  })

  const sessionFlowForm = useForm<SessionFlowData>({
    resolver: zodResolver(sessionFlowSchema),
    defaultValues: { session_flow_text: '' },
  })

  const trainerForm = useForm<TrainerData>({
    resolver: zodResolver(trainerSchema),
    defaultValues: { trainer_id: '', assignment_message: '' },
  })

  const [sessionFlowDoc, setSessionFlowDoc] = useState<File | null>(null)

  // Calculate course days dynamically

  const courseDays = useMemo(() => {
    const start = settingsForm.watch('start_date')
    const end = settingsForm.watch('end_date')
    if (start && end) {
      const diff = new Date(end).getTime() - new Date(start).getTime()
      return Math.max(1, Math.ceil(diff / (1000 * 60 * 60 * 24)))
    }
    return null
  }, [settingsForm.watch('start_date'), settingsForm.watch('end_date')])

  const isUrgent = useMemo(() => {
    const start = settingsForm.watch('start_date')
    if (start) {
      const diff = new Date(start).getTime() - new Date().getTime()
      const diffDays = diff / (1000 * 60 * 60 * 24)
      return diffDays < 30
    }
    return false
  }, [settingsForm.watch('start_date')])

  // Calculate final exam duration dynamically

  const startTime = settingsForm.watch('final_test_start_time')
  const endTime = settingsForm.watch('final_test_end_time')
  let examDuration = ''
  if (startTime && endTime) {
    const [startH, startM] = startTime.split(':').map(Number)
    const [endH, endM] = endTime.split(':').map(Number)
    let diffMins = (endH * 60 + endM) - (startH * 60 + startM)
    if (diffMins < 0) diffMins += 24 * 60 // handle overnight
    const h = Math.floor(diffMins / 60)
    const m = diffMins % 60
    examDuration = `${h > 0 ? `${h}h ` : ''}${m > 0 ? `${m}m` : ''}` || '0m'
  }

  useEffect(() => {
    const fetchTrainersAndSkills = async () => {
      setLoadingTrainers(true)
      try {
```

```
        const [{ data: trData }, { data: skData }, { data: usData }] = await Promise.all([
          supabase.from('trainers').select('id, full_name, email, qualifications, years_of_experience, bio, expertise_areas,
skills, department, designation, work_experience, availability').eq('approval_status', 'approved').order('full_name'),
          supabase.from('skills').select('*').order('name'),
          supabase.from('user_skills').select('user_id, skill_id, skills(id, name)'),
        ])

        if (skData && skData.length > 0) {
          setSkills(skData)
        } else {
          const defaultSkills: Skill[] = [
            { id: 'sk-1', name: 'Cyclone Tracking & Analysis', category: 'Meteorology', created_at: '' },
            { id: 'sk-2', name: 'Emergency Warning Communication', category: 'Operations', created_at: '' },
            { id: 'sk-3', name: 'Satellite & Radar Telemetry', category: 'Technical', created_at: '' },
            { id: 'sk-4', name: 'Disaster Preparedness', category: 'Disaster Management', created_at: '' },
            { id: 'sk-5', name: 'Flood Hazard Mapping', category: 'Hydrology', created_at: '' },
            { id: 'sk-6', name: 'GIS & Spatial Modeling', category: 'GIS', created_at: '' },
            { id: 'sk-7', name: 'Ocean Sensor Calibration', category: 'Oceanography', created_at: '' },
            { id: 'sk-8', name: 'Search & Rescue Protocols', category: 'Operations', created_at: '' },
            { id: 'sk-9', name: 'Weather Data Interpretation', category: 'Meteorology', created_at: '' },
            { id: 'sk-10', name: 'Risk Assessment & Mitigation', category: 'Safety', created_at: '' },
          ] as any
          setSkills(defaultSkills)
        }

        if (trData) {
          const trainersWithSkills: Trainer[] = trData.map((t: any) => {
            const userSkillNames = (usData || [])
              .filter((us: any) => us.user_id === t.id && us.skills?.name)
              .map((us: any) => us.skills.name)

            const rawTrainerSkills = Array.isArray(t.skills)
              ? t.skills
              : typeof t.skills === 'string'
              ? t.skills.split(',').map((s: string) => s.trim()).filter(Boolean)
              : []

            const rawExpertise = Array.isArray(t.expertise_areas)
              ? t.expertise_areas
              : typeof t.expertise_areas === 'string'
              ? t.expertise_areas.split(',').map((s: string) => s.trim()).filter(Boolean)
              : []

            const allTrainerSkills = Array.from(
              new Set([...rawExpertise, ...rawTrainerSkills, ...userSkillNames])
            )

            return {
              ...t,
              skills: allTrainerSkills,
            }
          })
          setTrainers(trainersWithSkills)
        }
      } catch (err) {
        console.error('Failed to load trainers or skills:', err)
      } finally {
        setLoadingTrainers(false)
      }
    }

    fetchTrainersAndSkills()
  }, [])

  const handleThumbnailChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return
    if (file.size > 5 * 1024 * 1024) {
      toast.error('Thumbnail must be under 5MB')
      return
    }
    setRawImageFile(file)
    setIsCropperOpen(true)
    if (thumbRef.current) thumbRef.current.value = ''
  }

  const toggleSkill = (id: string) => {
    setSelectedSkills(prev =>
      prev.includes(id) ? prev.filter(s => s !== id) : [...prev, id]
```

```
    )
  }

  const handleAddCustomSkill = async () => {
    const name = customSkillName.trim()
    if (!name) return
    setAddingSkill(true)
    try {
      const { data, error } = await supabase
        .from('skills')
        .insert({ name, category: 'Custom' })
        .select()
        .single()
      if (error) throw error
      if (data) {
        setSkills(prev => [...prev, data])
        setSelectedSkills(prev => [...prev, data.id])
        setCustomSkillName('')
        toast.success(`Skill "${name}" added`)
      }
    } catch (err: any) {
      toast.error(err.message || 'Failed to add skill')
    } finally {
      setAddingSkill(false)
    }
  }

  // Modules helpers

  const addModule = () => {
    const num = modules.length + 1
    const newMod: CourseModule = {
      id: crypto.randomUUID(),
      title: `Module ${num}: New Topic`,
      description: 'Module objectives and resource descriptions.',
      items: [],
    }
    setModules(prev => [...prev, newMod])
    toast.success(`Module ${num} created`)
  }

  const removeModule = (id: string) => {
    if (modules.length === 1) {
      toast.error('Course must have at least one module')
      return
    }
    setModules(prev => prev.filter(m => m.id !== id))
    toast.info('Module removed')
  }

  const updateModuleField = (id: string, field: 'title' | 'description', val: string) => {
    setModules(prev => prev.map(m => m.id === id ? { ...m, [field]: val } : m))
  }

  const openAddItemModal = (moduleId: string, type: 'photo' | 'video' | 'link' | 'text') => {
    setActiveModuleId(moduleId)
    setEditingItemId(null)
    setAddingItemType(type)
    setItemTitle('')
    setItemContent('')
    setItemUrl('')
    setItemDurationMinutes('5')
    setItemFile(null)
    setItemPreview(null)
  }

  const openEditItemModal = (moduleId: string, item: ModuleItem) => {
    setActiveModuleId(moduleId)
    setEditingItemId(item.id)
    setAddingItemType(item.type === 'quiz' ? null : item.type)
    setItemTitle(item.title)
    setItemContent(item.content || '')
    setItemUrl(item.url || '')
    setItemDurationMinutes(item.duration_minutes ? String(item.duration_minutes) : '5')
    setItemFile(item.file || null)
    setItemPreview(item.previewUrl || null)
  }

  const handleModuleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
```

```
    const file = e.target.files?.[0]
    if (!file) return
    if (file.size > MAX_FILE_SIZE) {
      toast.error(`File too large (${formatFileSize(file.size)}). Max 100MB.`)
      return
    }
    setItemFile(file)
    if (!itemTitle) {
      setItemTitle(file.name.replace(/\.[^/.]+$/, ''))
    }
    if (file.type.startsWith('image/')) {
      setItemPreview(URL.createObjectURL(file))
    }
  }

  const saveModuleItem = () => {
    if (!activeModuleId || !addingItemType) return
    if (!itemTitle.trim()) {
      toast.error('Please enter an item title')
      return
    }

    if (addingItemType === 'link' && !itemUrl.trim()) {
      toast.error('Please provide a URL link')
      return
    }

    if ((addingItemType === 'photo' || addingItemType === 'video') && !itemFile && !itemUrl && !editingItemId) {
      toast.error(`Please upload a ${addingItemType} file or provide an external URL`)
      return
    }

    const newItem: ModuleItem = {
      id: editingItemId || crypto.randomUUID(),
      type: addingItemType,
      title: itemTitle.trim(),
      content: itemContent.trim() || undefined,
      url: itemUrl.trim() || undefined,
      duration_minutes: itemDurationMinutes ? parseInt(itemDurationMinutes, 10) : undefined,
      file: itemFile || undefined,
      previewUrl: itemPreview || undefined,
      fileName: itemFile?.name,
      fileSize: itemFile?.size,
    }

    setModules(prev => prev.map(m => {
      if (m.id !== activeModuleId) return m

      if (editingItemId) {
        return {
          ...m,
          items: m.items.map(it => it.id === editingItemId ? newItem : it)
        }
      }

      return {
        ...m,
        items: [...m.items, newItem]
      }
    }))

    toast.success(editingItemId ? `Updated ${addingItemType} item` : `Added ${addingItemType} to module`)
    setAddingItemType(null)
    setActiveModuleId(null)
    setEditingItemId(null)
    setItemFile(null)
    setItemPreview(null)
    setItemTitle('')
    setItemContent('')
    setItemUrl('')
    setItemDurationMinutes('5')
  }

  const removeModuleItem = (moduleId: string, itemId: string) => {
    setModules(prev => prev.map(m => {
      if (m.id !== moduleId) return m
      return {
        ...m,
        items: m.items.filter(i => i.id !== itemId),
```

```
542 |         quiz_questions: (m.quiz_questions || []).filter(q => q.id !== itemId)
543 |       }
544 |     }))
545 |     toast.info('Item removed')
546 |   }
547 |
548 |   // Quiz questions helpers

549 |   const openAddQuizQuestion = (moduleId: string) => {
550 |     setActiveQuizModuleId(moduleId)
551 |     setEditingQuestionId(null)
552 |     setQuizQuestionText('')
553 |     setQuizOptions(['', '', '', ''])
554 |     setQuizCorrectOption(0)
555 |     setQuizExplanation('')
556 |   }
557 |
558 |   const openEditQuizQuestion = (moduleId: string, q: QuizQuestion) => {
559 |     setActiveQuizModuleId(moduleId)
560 |     setEditingQuestionId(q.id)
561 |     setQuizQuestionText(q.question)
562 |     setQuizOptions(q.options && q.options.length >= 2 ? [...q.options] : ['', '', '', ''])
563 |     setQuizCorrectOption(q.correct_option ?? 0)
564 |     setQuizExplanation(q.explanation || '')
565 |   }
566 |
567 |   const saveQuizQuestion = () => {
568 |     if (!activeQuizModuleId) return
569 |     if (!quizQuestionText.trim()) {
570 |       toast.error('Please enter the question text')
571 |       return
572 |     }
573 |     const cleanOptions = quizOptions.map(o => o.trim())
574 |     if (cleanOptions.filter(Boolean).length < 2) {
575 |       toast.error('Please provide at least 2 answer choices')
576 |       return
577 |     }
578 |
579 |     const questionData: QuizQuestion = {
580 |       id: editingQuestionId || crypto.randomUUID(),
581 |       question: quizQuestionText.trim(),
582 |       options: cleanOptions,
583 |       correct_option: quizCorrectOption,
584 |       explanation: quizExplanation.trim() || undefined,
585 |     }
586 |
587 |     setModules(prev => prev.map(m => {
588 |       if (m.id !== activeQuizModuleId) return m
589 |
590 |       let updatedItems = [...m.items]
591 |       const existingItemIdx = updatedItems.findIndex(i => i.id === editingQuestionId || i.quiz_data?.id === editingQuestionI
d)
592 |
593 |       if (existingItemIdx !== -1) {
594 |         updatedItems[existingItemIdx] = {
595 |           ...updatedItems[existingItemIdx],
596 |           type: 'quiz',
597 |           title: `Quiz: ${quizQuestionText.trim().slice(0, 50)}${quizQuestionText.trim().length > 50 ? '...' : ''}`,
598 |           content: quizExplanation.trim() || undefined,
599 |           quiz_data: questionData
600 |         }
601 |       } else {
602 |         const newQuizItem: ModuleItem = {
603 |           id: questionData.id,
604 |           type: 'quiz',
605 |           title: `Quiz: ${quizQuestionText.trim().slice(0, 50)}${quizQuestionText.trim().length > 50 ? '...' : ''}`,
606 |           content: quizExplanation.trim() || undefined,
607 |           quiz_data: questionData
608 |         }
609 |         updatedItems.push(newQuizItem)
610 |       }
611 |
612 |       // Keep quiz_questions in sync

613 |       const currentQuestions = m.quiz_questions || []
614 |       const updatedQuestions = editingQuestionId
615 |         ? currentQuestions.map(q => q.id === editingQuestionId ? questionData : q)
616 |         : [...currentQuestions, questionData]
617 |
```

```
      return {
        ...m,
        items: updatedItems,
        quiz_questions: updatedQuestions,
        passing_score: m.passing_score ?? 80,
      }
    }))

    toast.success(editingQuestionId ? 'Quiz question updated' : 'Quiz question added to module sequence')
    setActiveQuizModuleId(null)
    setEditingQuestionId(null)
    setQuizQuestionText('')
    setQuizOptions(['', '', '', ''])
    setQuizCorrectOption(0)
    setQuizExplanation('')
  }

  const removeQuizQuestion = (moduleId: string, questionId: string) => {
    setModules(prev => prev.map(m => {
      if (m.id !== moduleId) return m
      return {
        ...m,
        items: m.items.filter(i => i.id !== questionId && i.quiz_data?.id !== questionId),
        quiz_questions: (m.quiz_questions || []).filter(q => q.id !== questionId),
      }
    }))
    toast.info('Quiz question removed')
  }

  const handleGenerateAiQuiz = (moduleId: string) => {
    const mod = modules.find(m => m.id === moduleId)
    if (!mod) return
    setIsGeneratingAiQuiz(true)
    setTimeout(() => {
      const topic = mod.title.replace(/^Module \d+:\s*/, '') || 'Module Topic'
      const aiQuestions: QuizQuestion[] = [
        {
          id: crypto.randomUUID(),
          question: `What is the core prerequisite protocol for ${topic}?`,
          options: [
            'Adhere to verified operational checklists and standard ministry guidelines',
            'Bypass preliminary verification to speed up execution',
            'Conduct unrecorded subjective assessments',
            'Defer telemetry log synchronization until post-incident review'
          ],
          correct_option: 0,
          explanation: 'Standard operational guidelines ensure consistent execution and data integrity.'
        },
        {
          id: crypto.randomUUID(),
          question: `In operational workflows, how should telemetry anomalies in ${topic} be handled?`,
          options: [
            'Calibrate sensors against secondary baseline references and log deviations',
            'Ignore temporary anomalies until standard quarterly servicing',
            'Disconnect the entire network immediately without notification',
            'Override automated safety cutoffs to sustain uninterrupted transmission'
          ],
          correct_option: 0,
          explanation: 'Secondary calibration verifies signal deviations accurately without compromising active safety.'
        },
        {
          id: crypto.randomUUID(),
          question: `Which criterion determines mastery in ${topic}?`,
          options: [
            'Consistent precision, adhering to standard tolerance thresholds and real-time situational awareness',
            'Execution speed regardless of diagnostic error margins',
            'Theoretical understanding without hands-on apparatus handling',
            'Sole reliance on automated algorithmic presets'
          ],
          correct_option: 0,
          explanation: 'High situational awareness coupled with precision defines MoES certified competency.'
        }
      ]

      setModules(prev => prev.map(m => {
        if (m.id !== moduleId) return m
        const newItems: ModuleItem[] = aiQuestions.map(q => ({
          id: q.id,
          type: 'quiz',
```

```
          title: `Quiz: ${q.question.slice(0, 50)}...`,
          content: q.explanation,
          quiz_data: q
        }))
        return {
          ...m,
          items: [...m.items, ...newItems],
          quiz_questions: [...(m.quiz_questions || []), ...aiQuestions],
          passing_score: m.passing_score ?? 80
        }
      }))

      setIsGeneratingAiQuiz(false)
      toast.success('Generated 3 AI quiz questions and added to module sequence!')
    }, 500)
  }

  // Materials handling

  const validateFile = (file: File): string | null => {
    if (file.size > MAX_FILE_SIZE) return `File too large (${formatFileSize(file.size)}). Max 100MB.`
    const ext = '.' + file.name.split('.').pop()?.toLowerCase()
    if (!ACCEPTED_EXTENSIONS.includes(ext)) return `File type not supported: ${ext}`
    return null
  }

  const addFileMaterial = useCallback((file: File) => {
    const error = validateFile(file)
    if (error) {
      toast.error(error)
      return
    }
    const isVideo = file.type.startsWith('video/')
    const material: PendingMaterial = {
      id: crypto.randomUUID(),
      file,
      fileName: file.name,
      fileSize: file.size,
      mimeType: file.type || 'application/octet-stream',
      materialType: isVideo ? 'video' : 'file',
      preview: file.type.startsWith('image/') ? URL.createObjectURL(file) : undefined,
    }
    setPendingMaterials(prev => [...prev, material])
    toast.success(`Added: ${file.name}`)
  }, [])

  const handleFileInput = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = e.target.files
    if (!files) return
    Array.from(files).forEach(addFileMaterial)
    if (fileInputRef.current) fileInputRef.current.value = ''
  }

  const handleDrop = (e: React.DragEvent) => {
    e.preventDefault()
    setDragOver(false)
    const files = e.dataTransfer.files
    Array.from(files).forEach(addFileMaterial)
  }

  const addLinkMaterial = () => {
    const url = linkUrl.trim()
    if (!url) {
      toast.error('Enter a URL')
      return
    }
    try {
      new URL(url)
    } catch {
      toast.error('Please enter a valid URL')
      return
    }
    const isVideo = /youtube\.com|youtu\.be|vimeo\.com|\.mp4|\.webm/i.test(url)
    const title = linkTitle.trim() || new URL(url).hostname
    const material: PendingMaterial = {
      id: crypto.randomUUID(),
      fileName: title,
      fileSize: 0,
      mimeType: 'text/uri-list',
```

```
775 |       materialType: isVideo ? 'video' : 'link',
776 |       url,
777 |     }
778 |     setPendingMaterials(prev => [...prev, material])
779 |     setLinkUrl('')
780 |     setLinkTitle('')
781 |     toast.success(`Added: ${title}`)
782 |   }
783 |
784 |   const removeMaterial = (id: string) => {
785 |     setPendingMaterials(prev => prev.filter(m => m.id !== id))
786 |   }
787 |
788 |   // Advanced AI Competency Mapping Engine across all 6 Course Creation Parts

789 |   const scoredTrainers = React.useMemo(() => {
790 |     if (trainers.length === 0) return []
791 |
792 |     // Part 1: Course Details (Step 1)

793 |     const courseTitle = (detailsForm.watch('title') || '').trim().toLowerCase()
794 |     const courseDesc = (detailsForm.watch('description') || '').trim().toLowerCase()
795 |     const courseDept = (detailsForm.watch('department') || '').trim().toLowerCase()
796 |     const courseType = (detailsForm.watch('course_type') || '').trim().toLowerCase()
797 |
798 |     // Part 2: Course Configuration (Step 2)

799 |     const trainerSuggestion = (settingsForm.watch('trainer_suggestion') || '').trim().toLowerCase()
800 |     const deliveryMode = settingsForm.watch('delivery_mode') || 'recorded'
801 |
802 |     // Part 3: Course Outline / Session Flow (Step 3)

803 |     const sessionFlowText = (sessionFlowForm.watch('session_flow_text') || '').trim().toLowerCase()
804 |     const sessionDocName = (sessionFlowDoc?.name || '').trim().toLowerCase()
805 |
806 |     // Part 4: Learning Objectives & Skills (Step 4)

807 |     const courseObjs = (objectivesForm.watch('learning_objectives') || '').trim().toLowerCase()
808 |     const requiredSkillNames = skills
809 |       .filter(s => selectedSkills.includes(s.id))
810 |       .map(s => s.name.toLowerCase())
811 |
812 |     // Part 5: Modules & Quizzes (Step 5)

813 |     const moduleTitles = modules.map(m => m.title || '').join(' ').toLowerCase()
814 |     const moduleDescriptions = modules.map(m => m.description || '').join(' ').toLowerCase()
815 |     const moduleItemsText = modules.flatMap(m => m.items || []).map(i => `${i.title} ${i.content || ''} ${i.quiz_data?.questi
on || ''}`).join(' ').toLowerCase()
816 |     const allModulesText = `${moduleTitles} ${moduleDescriptions} ${moduleItemsText}`.trim()
817 |
818 |     // Part 6: Materials & Reference Documents (Step 6)

819 |     const materialsText = pendingMaterials.map(m => `${m.fileName} ${m.url || ''}`).join(' ').toLowerCase()
820 |
821 |     // Check if course has any information entered across the 6 parts

822 |     const hasCourseDetails = courseTitle.length >= 3 || courseDesc.length >= 8 || courseDept.length > 0
823 |     const hasCourseConfig = trainerSuggestion.length >= 3
824 |     const hasCourseOutline = sessionFlowText.length >= 5 || sessionDocName.length >= 3
825 |     const hasLearningSkills = requiredSkillNames.length > 0 || courseObjs.length >= 5
826 |     const hasModules = modules.length > 0 && modules.some(m => m.title && m.title.length >= 3 && !m.title.includes('Module 1:
Introduction & Fundamentals'))
827 |     const hasMaterials = pendingMaterials.length > 0
828 |
829 |     const hasAnyCourseInfo = hasCourseDetails || hasCourseConfig || hasCourseOutline || hasLearningSkills || hasModules || ha
sMaterials
830 |
831 |     return trainers.map(t => {
832 |       const qual = (t.qualifications || '').trim()
833 |       const bio = (t.bio || '').trim()
834 |       const workExp = (t.work_experience || '').trim()
835 |       const dept = (t.department || '').trim()
836 |       const designation = (t.designation || '').trim()
837 |       const yearsExp = typeof t.years_of_experience === 'number' ? t.years_of_experience : (t.years_of_experience ? parseInt
(String(t.years_of_experience)) : 0)
838 |       const trainerSkills = (t.skills || []).map(s => s.trim().toLowerCase()).filter(Boolean)
839 |
840 |       // Profile Completeness Check
```

```
841 |       const hasMeaningfulQual = qual.length > 0 && qual !== '—' && qual.toLowerCase() !== 'n/a'
842 |       const hasMeaningfulBio = bio.length > 0 && bio !== '—' && !bio.toLowerCase().includes('no specialized bio')
843 |       const hasMeaningfulWork = workExp.length > 0 && workExp !== '—'
844 |       const hasSkills = trainerSkills.length > 0
845 |       const hasExperience = yearsExp > 0
846 |
847 |       const isProfileEmpty = !hasSkills && !hasMeaningfulQual && !hasMeaningfulBio && !hasMeaningfulWork && !hasExperience
848 |
849 |       if (isProfileEmpty) {
850 |         return {
851 |           ...t,
852 |           matchPercentage: 0,
853 |           matchedSkillsList: [] as string[],
854 |           matchedParts: [] as string[],
855 |           reasoning: 'Incomplete Profile: No recorded skills, qualifications, or domain experience yet.',
856 |           fitTier: 'incomplete' as const,
857 |         }
858 |       }
859 |
860 |       if (!hasAnyCourseInfo) {
861 |         return {
862 |           ...t,
863 |           matchPercentage: 0,
864 |           matchedSkillsList: [] as string[],
865 |           matchedParts: [] as string[],
866 |           reasoning: 'Awaiting Course Details: Enter information in Steps 1–6 (Details, Outline, Skills, Modules, Materials)
to calculate AI Competency Score.',
867 |           fitTier: 'incomplete' as const,
868 |         }
869 |       }
870 |
871 |       const allTrainerText = `${qual} ${bio} ${workExp} ${dept} ${designation} ${t.full_name} ${trainerSkills.join(' ')}`.toL
owerCase()
872 |       const matchedSkillsList: string[] = []
873 |       const matchedParts: string[] = []
874 |
875 |       // PART 1: Course Details Match (Max 20 pts)

876 |       let part1Score = 0
877 |       if (hasCourseDetails) {
878 |         let detailsMatched = false
879 |         const titleWords = courseTitle.split(/\s+/).filter(w => w.length >= 4)
880 |         const matchedTitleWords = titleWords.filter(w => allTrainerText.includes(w))
881 |         if (matchedTitleWords.length > 0) {
882 |           part1Score += Math.min(10, matchedTitleWords.length * 4)
883 |           detailsMatched = true
884 |         }
885 |         const descWords = courseDesc.split(/\s+/).filter(w => w.length >= 4)
886 |         const matchedDescWords = descWords.filter(w => allTrainerText.includes(w))
887 |         if (matchedDescWords.length > 0) {
888 |           part1Score += Math.min(6, matchedDescWords.length * 2)
889 |           detailsMatched = true
890 |         }
891 |         if (courseDept && (dept.toLowerCase().includes(courseDept) || allTrainerText.includes(courseDept))) {
892 |           part1Score += 6
893 |           detailsMatched = true
894 |         }
895 |         if (detailsMatched) matchedParts.push('Course Details')
896 |       }
897 |       part1Score = Math.min(20, part1Score)
898 |
899 |       // PART 2: Course Configuration & Trainer Suggestions (Max 10 pts)

900 |       let part2Score = 0
901 |       if (hasCourseConfig) {
902 |         let configMatched = false
903 |         if (trainerSuggestion) {
904 |           const suggestionWords = trainerSuggestion.split(/\s+/).filter(w => w.length >= 3)
905 |           const matchedSuggestion = suggestionWords.some(w => allTrainerText.includes(w) || t.full_name.toLowerCase().include
s(w))
906 |           if (matchedSuggestion) {
907 |             part2Score += 10
908 |             configMatched = true
909 |           }
910 |         }
911 |         if (configMatched) matchedParts.push('Configuration')
912 |       }
913 |
914 |       // PART 3: Course Outline & Session Flow (Max 15 pts)
```

```
      let part3Score = 0
      if (hasCourseOutline) {
        let outlineMatched = false
        const outlineWords = `${sessionFlowText} ${sessionDocName}`.split(/\s+/).filter(w => w.length >= 4)
        const matchedOutline = outlineWords.filter(w => allTrainerText.includes(w))
        if (matchedOutline.length > 0) {
          part3Score += Math.min(15, matchedOutline.length * 3)
          outlineMatched = true
        }
        if (outlineMatched) matchedParts.push('Course Outline')
      }
      part3Score = Math.min(15, part3Score)

      // PART 4: Learning Objectives & Required Skills (Max 30 pts)

      let part4Score = 0
      if (hasLearningSkills) {
        let skillsMatched = false
        if (requiredSkillNames.length > 0) {
          let matchedCount = 0
          requiredSkillNames.forEach(reqSkill => {
            const direct = trainerSkills.some(ts => ts === reqSkill || ts.includes(reqSkill) || reqSkill.includes(ts))
            const textMatch = allTrainerText.includes(reqSkill)
            if (direct) {
              matchedCount += 1.0
              const originalSkill = skills.find(s => s.name.toLowerCase() === reqSkill)?.name || reqSkill
              if (!matchedSkillsList.includes(originalSkill)) matchedSkillsList.push(originalSkill)
            } else if (textMatch) {
              matchedCount += 0.7
              const originalSkill = skills.find(s => s.name.toLowerCase() === reqSkill)?.name || reqSkill
              if (!matchedSkillsList.includes(originalSkill)) matchedSkillsList.push(originalSkill)
            }
          })
          part4Score += Math.min(25, (matchedCount / requiredSkillNames.length) * 25)
          if (matchedCount > 0) skillsMatched = true
        }

        if (courseObjs) {
          const objWords = courseObjs.split(/\s+/).filter(w => w.length >= 4)
          const matchedObjWords = objWords.filter(w => allTrainerText.includes(w))
          if (matchedObjWords.length > 0) {
            part4Score += Math.min(8, matchedObjWords.length * 2)
            skillsMatched = true
          }
        }
        if (skillsMatched) matchedParts.push('Learning & Skills')
      }
      part4Score = Math.min(30, part4Score)

      // PART 5: Modules & Quizzes (Max 15 pts)

      let part5Score = 0
      if (allModulesText.length > 0) {
        let modulesMatched = false
        const moduleWords = allModulesText.split(/\s+/).filter(w => w.length >= 4)
        const matchedModWords = moduleWords.filter(w => allTrainerText.includes(w))
        if (matchedModWords.length > 0) {
          part5Score += Math.min(15, matchedModWords.length * 3)
          modulesMatched = true
        }
        if (modulesMatched) matchedParts.push('Modules')
      }
      part5Score = Math.min(15, part5Score)

      // PART 6: Materials & Attachments (Max 10 pts)

      let part6Score = 0
      if (materialsText.length > 0) {
        let materialsMatched = false
        const materialWords = materialsText.replace(/[\._\-]/g, ' ').split(/\s+/).filter(w => w.length >= 4 && !['pdf', 'doc
x', 'pptx', 'mp4', 'file'].includes(w))
        const matchedMatWords = materialWords.filter(w => allTrainerText.includes(w))
        if (matchedMatWords.length > 0) {
          part6Score += Math.min(10, matchedMatWords.length * 4)
          materialsMatched = true
        }
        if (materialsMatched) matchedParts.push('Materials')
      }
```

```
      part6Score = Math.min(10, part6Score)

      // Total Core Content Score across all 6 parts

      const corePartsScore = part1Score + part2Score + part3Score + part4Score + part5Score + part6Score

      // Gating: If NO part matches (0% overlap across all 6 course parts), match is 0%

      if (corePartsScore === 0) {
        return {
          ...t,
          matchPercentage: 0,
          matchedSkillsList: [] as string[],
          matchedParts: [] as string[],
          reasoning: "No Topical Overlap: Trainer's profile has no keyword or skill match with the 6 course parts.",
          fitTier: 'low' as const,
        }
      }

      // Proportional Seniority & Credentials Bonus (Max 10 pts)

      const relevanceRatio = Math.min(1, corePartsScore / 60)
      let expBonus = 0
      if (yearsExp >= 10) expBonus = 6
      else if (yearsExp >= 5) expBonus = 4
      else if (yearsExp >= 2) expBonus = 2

      let qualBonus = 0
      const qualLower = qual.toLowerCase()
      if (qualLower.includes('ph.d') || qualLower.includes('phd') || qualLower.includes('scientist')) qualBonus = 4
      else if (qualLower.includes('master') || qualLower.includes('m.tech') || qualLower.includes('m.sc')) qualBonus = 3
      else if (qualLower.includes('bachelor') || qualLower.includes('b.tech') || qualLower.includes('b.sc')) qualBonus = 1

      const weightedBonus = Math.round((expBonus + qualBonus) * relevanceRatio)

      const totalRaw = corePartsScore + weightedBonus
      const matchPercentage = Math.min(99, Math.max(1, Math.round(totalRaw)))

      // Determine fit tier & human-readable reasoning

      let fitTier: 'high' | 'moderate' | 'low' | 'incomplete' = 'low'
      let reasoning = ''

      if (matchPercentage >= 70) {
        fitTier = 'high'
        reasoning = `High Match (${matchPercentage}%): Aligned with ${matchedParts.join(', ')}${matchedSkillsList.length > 0
? ` (${matchedSkillsList.slice(0, 2).join(', ')})` : ''}${yearsExp > 0 ? ` + ${yearsExp} yrs exp` : ''}.`
      } else if (matchPercentage >= 35) {
        fitTier = 'moderate'
        reasoning = `Moderate Match (${matchPercentage}%): Matches topics in ${matchedParts.join(', ')}${matchedSkillsList.le
ngth > 0 ? ` (${matchedSkillsList[0]})` : ''}.`
      } else {
        fitTier = 'low'
        reasoning = `Low Match (${matchPercentage}%): Partial match with ${matchedParts.join(', ')}.`
      }

      return {
        ...t,
        matchPercentage,
        matchedSkillsList,
        matchedParts,
        reasoning,
        fitTier,
      }
    }).sort((a, b) => b.matchPercentage - a.matchPercentage)
  }, [
    trainers,
    skills,
    selectedSkills,
    modules,
    pendingMaterials,
    sessionFlowDoc,
    detailsForm.watch('title'),
    detailsForm.watch('description'),
    detailsForm.watch('department'),
    detailsForm.watch('course_type'),
    settingsForm.watch('trainer_suggestion'),
    settingsForm.watch('delivery_mode'),
    sessionFlowForm.watch('session_flow_text'),
```

```
    objectivesForm.watch('learning_objectives'),
  ])

  const handleAutoFillMessage = (trainer: typeof scoredTrainers[0]) => {
    const title = detailsForm.watch('title') || 'this training course'
    let msg = ''
    if (trainer.matchedSkillsList.length > 0) {
      msg = `Greetings ${trainer.full_name}, you have been appointed as Lead Trainer for "${title}" based on your verified co
mpetencies in ${trainer.matchedSkillsList.join(', ')}. Please review the syllabus and coordinate module sessions.`
    } else if (trainer.years_of_experience && trainer.years_of_experience > 0) {
      msg = `Greetings ${trainer.full_name}, you have been appointed as Lead Trainer for "${title}" based on your ${trainer.y
ears_of_experience} years of domain experience. Please review the syllabus and coordinate module sessions.`
    } else {
      msg = `Greetings ${trainer.full_name}, you have been appointed as Lead Trainer for "${title}". Please review the syllab
us, update your profile competencies, and coordinate with the administration.`
    }
    trainerForm.setValue('assignment_message', msg)
    toast.success('Generated AI briefing message!')
  }

  const selectedTrainer = trainers.find(t => t.id === trainerForm.watch('trainer_id'))

  const validateStep = async () => {
    switch (step) {
      case 1: return await detailsForm.trigger()
      case 2: return await settingsForm.trigger()
      case 3: return await sessionFlowForm.trigger()
      case 4: return await objectivesForm.trigger()
      case 5: {
        if (modules.length === 0) {
          toast.error('Please add at least one module')
          return false
        }
        for (let i = 0; i < modules.length; i++) {
          if (!modules[i].title.trim()) {
            toast.error(`Please provide a title for Module ${i + 1}`)
            return false
          }
        }
        return true
      }
      case 7: return await trainerForm.trigger()
      default: return true
    }
  }

  const handleNext = async () => {
    const valid = await validateStep()
    if (!valid) return
    setStep(s => Math.min(s + 1, 9))
  }

  const handleBack = () => {
    setStep(s => Math.max(s - 1, 1))
  }

  // Publish / Save

  const handlePublish = async (status: 'draft' | 'published') => {
    const t = trainerForm.getValues()
    const d = detailsForm.getValues()
    const s = settingsForm.getValues()
    const f = sessionFlowForm.getValues()
    const o = objectivesForm.getValues()

    if (!t.trainer_id) {
      toast.error('Please assign a trainer in Step 7')
      setStep(7)
      return
    }
    if (!d.title) {
      toast.error('Course title is required')
      setStep(1)
      return
    }
    if (selectedSkills.length === 0) {
      toast.error('Select at least one skill for the course in Step 4')
      setStep(4)
      return
```

```
    }

    setSaving(true)
    try {
      // 1. Insert course

      const { data: course, error } = await supabase
        .from('courses')
        .insert({
          title: d.title,
          description: d.description,
          course_type: d.course_type,
          department: d.department || null,
          duration_minutes: s.duration_hours ? s.duration_hours * 60 : null,
          passing_score: s.passing_score ?? 60,
          trainer_id: t.trainer_id,
          status,
          learning_objectives: {
            description: o.learning_objectives,
            understand: o.learning_objectives,
          },
          meet_link: s.meet_link || null,
          start_date: s.start_date ? new Date(s.start_date).toISOString() : null,
          end_date: s.end_date ? new Date(s.end_date).toISOString() : null,
          planned_assessments_count: s.planned_assessments_count || 0,
          planned_mock_tests_count: s.planned_mock_tests_count || 0,
          final_test_date: s.final_test_date ? new Date(s.final_test_date).toISOString() : null,
          final_test_start_time: s.final_test_start_time || null,
          final_test_end_time: s.final_test_end_time || null,
          delivery_mode: s.delivery_mode || 'recorded',
          max_trainees: s.max_trainees || null,
          trainer_suggestion: s.trainer_suggestion || null,
          session_flow_text: f.session_flow_text || null,
          has_certificate: hasCertificate,
          certificate_template_url: certificateTemplateUrl,
          certificate_template_name: certificateTemplateName,
        } as any)
        .select()
        .single()

      if (error) throw error
      if (!course) throw new Error('No course returned after insert')

      // 2. Upload thumbnail if present

      if (thumbnail) {
        const ext = thumbnail.name.split('.').pop()
        const path = `${course.id}/thumbnail.${ext}`
        const { error: upErr } = await supabase.storage.from('materials').upload(path, thumbnail)
        if (!upErr) {
          await supabase.from('courses').update({ thumbnail_path: path }).eq('id', course.id)
        }
      }

      // 3. Upload session flow doc if present

      if (sessionFlowDoc) {
        const ext = sessionFlowDoc.name.split('.').pop()
        const path = `${course.id}/session_flow_${Date.now()}.${ext}`
        const { error: docErr } = await supabase.storage.from('materials').upload(path, sessionFlowDoc)
        if (!docErr) {
          await supabase.from('courses').update({ session_flow_doc_path: path } as any).eq('id', course.id)
        }
      }

      // 4. Save Course Skills

      if (selectedSkills.length > 0 && course) {
        try {
          const skillInserts: any[] = []
          for (const skillId of selectedSkills) {
            if (skillId.startsWith('custom-')) {
              const skillObj = skills.find(s => s.id === skillId)
              if (skillObj) {
                const { data: createdSkill } = await supabase
                  .from('skills')
                  .insert({ name: skillObj.name } as any)
                  .select()
                  .single()
```

```
                if (createdSkill) {
                  skillInserts.push({
                    course_id: course.id,
                    skill_id: (createdSkill as any).id,
                    required_level: 3,
                  })
                }
              }
            } else {
              skillInserts.push({
                course_id: course.id,
                skill_id: skillId,
                required_level: 3,
              })
            }
          }
          if (skillInserts.length > 0) {
            await (supabase as any).from('course_skills').insert(skillInserts)
          }
        } catch (sErr) {
          console.warn('Could not insert course_skills:', sErr)
        }
      }

      // 5. Upload Materials

      if (course && pendingMaterials.length > 0) {
        for (const mat of pendingMaterials) {
          if (mat.materialType === 'link' || mat.materialType === 'video') {
            await supabase.from('materials').insert({
              course_id: course.id,
              uploaded_by: user!.id,
              file_name: mat.fileName,
              storage_path: `link/${crypto.randomUUID()}`,
              material_type: mat.materialType,
              url: mat.url,
              mime_type: 'text/uri-list',
              extraction_status: 'completed',
            })
          } else if (mat.file) {
            const fileExt = mat.file.name.split('.').pop()
            const materialId = mat.id
            const storagePath = `${course.id}/${materialId}.${fileExt}`

            const { error: uploadError } = await supabase.storage
              .from('materials')
              .upload(storagePath, mat.file)
            if (uploadError) console.warn('Material upload error:', uploadError)

            await supabase.from('materials').insert({
              id: materialId,
              course_id: course.id,
              uploaded_by: user!.id,
              file_name: mat.fileName,
              storage_path: storagePath,
              mime_type: mat.mimeType,
              file_size: mat.fileSize,
              extraction_status: 'pending',
            })
          }
        }
      }

      // 6. Save Modules and Module Items

      const uploadedModules: any[] = []
      for (let mIdx = 0; mIdx < modules.length; mIdx++) {
        const mod = modules[mIdx]
        const uploadedItems: any[] = []

        for (const item of mod.items) {
          if (item.file) {
            const fileExt = item.file.name.split('.').pop()
            const fileId = item.id || crypto.randomUUID()
            const storagePath = `${course.id}/modules/${mod.id}/${fileId}.${fileExt}`

            let uploadRes = await supabase.storage
              .from('Course modules')
              .upload(storagePath, item.file)
```

```
            if (uploadRes.error) {
              console.warn('Upload to Course modules had error, trying fallback:', uploadRes.error)
              uploadRes = await supabase.storage
                .from('materials')
                .upload(storagePath, item.file)
            }

            const { data: publicUrlData } = supabase.storage
              .from('Course modules')
              .getPublicUrl(storagePath)

            uploadedItems.push({
              id: item.id,
              type: item.type,
              title: item.title,
              content: item.content || '',
              url: publicUrlData?.publicUrl || item.url || '',
              duration_minutes: item.duration_minutes,
              duration_seconds: item.duration_seconds,
              storagePath: storagePath,
              fileName: item.fileName,
              fileSize: item.fileSize,
              quiz_data: item.quiz_data,
            })
          } else {
            uploadedItems.push({
              id: item.id,
              type: item.type,
              title: item.title,
              content: item.content || '',
              url: item.url || '',
              duration_minutes: item.duration_minutes,
              duration_seconds: item.duration_seconds,
              fileName: item.fileName,
              fileSize: item.fileSize,
              quiz_data: item.quiz_data,
            })
          }
        }

        uploadedModules.push({
          id: mod.id,
          title: mod.title,
          description: mod.description,
          order_index: mIdx,
          is_final_assessment: false,
          passing_score: mod.passing_score ?? 80,
          quiz_questions: mod.quiz_questions || [],
          items: uploadedItems,
        })
      }

      // Update course with modules JSON

      try {
        await supabase
          .from('courses')
          .update({ modules: uploadedModules } as any)
          .eq('id', course.id)
      } catch (err) {
        console.warn('Could not update courses.modules directly:', err)
      }

      // Try inserting into course_modules table if present

      try {
        const moduleRows = uploadedModules.map(m => ({
          course_id: course.id,
          title: m.title,
          description: m.description,
          order_index: m.order_index,
          is_final_assessment: false,
          content_items: m.items,
        }))
        await (supabase as any).from('course_modules').insert(moduleRows)
      } catch (err) {
        console.warn('course_modules table insert optional/skipped:', err)
      }
```

```
1366 |
1367 |       // 7. Insert Course Assignment record

1368 |       try {
1369 |         await (supabase as any).from('course_assignments').insert({
1370 |           course_id: course.id,
1371 |           trainer_id: t.trainer_id,
1372 |           assigned_by: user!.id,
1373 |           message: t.assignment_message || null,
1374 |         })
1375 |       } catch (aErr) {
1376 |         console.warn('course_assignments insert skipped:', aErr)
1377 |       }
1378 |
1379 |       toast.success(status === 'published' ? 'Course published & lead trainer notified!' : 'Course saved as draft')
1380 |       navigate('/admin')
1381 |     } catch (err: any) {
1382 |       console.error('Failed to create course:', err)
1383 |       toast.error(err.message || 'Failed to create course')
1384 |     } finally {
1385 |       setSaving(false)
1386 |     }
1387 |   }
1388 |
1389 |   return (
1390 |     <div className="min-h-screen bg-[#f8fafc]">
1391 |       <div className="max-w-5xl mx-auto px-4 py-8">
1392 |         <motion.div initial="hidden" animate="visible" variants={fadeUp} className="mb-8">
1393 |           <button
1394 |             onClick={() => navigate('/admin')}
1395 |             className="flex items-center gap-2 text-sm text-slate-600 hover:text-cyan-700 mb-4 font-semibold transition-color
s cursor-pointer"
1396 |           >
1397 |             <ArrowLeft className="w-4 h-4" /> Back to Dashboard
1398 |           </button>
1399 |           <div className="flex items-center gap-3">
1400 |             <div className="w-10 h-10 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-cente
r shadow-lg shadow-cyan-600/20">
1401 |               <BookOpen className="w-5 h-5 text-white" />
1402 |             </div>
1403 |             <div>
1404 |               <h1 className="text-2xl font-black text-slate-900 tracking-tight">Create New Course</h1>
1405 |               <p className="text-sm text-slate-600 font-medium">Design curriculum, configure assessments, run AI competency m
apping, and publish directly</p>
1406 |             </div>
1407 |           </div>
1408 |         </motion.div>
1409 |
1410 |         {/* Step Wizard Bar */}
1411 |         <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex items-center gap-2 mb-8 overflow-x-a
uto pb-2">
1412 |           {STEPS.map((s, i) => {
1413 |             const Icon = s.icon
1414 |             const isActive = step === s.id
1415 |             const isDone = step > s.id
1416 |             return (
1417 |               <React.Fragment key={s.id}>
1418 |                 <button
1419 |                   type="button"
1420 |                   onClick={() => setStep(s.id)}
1421 |                   className={'flex items-center gap-2 px-3.5 py-2 rounded-xl shrink-0 text-xs font-bold transition-all cursor
-pointer ' + (isActive ? 'bg-cyan-100 text-cyan-900 border border-cyan-300 shadow-sm' : isDone ? 'bg-slate-200 text-slate-800 hover:
bg-slate-300' : 'bg-slate-100 text-slate-500 opacity-80 hover:opacity-100')}
1422 |                 >
1423 |                   {isDone ? <CheckCircle className="w-3.5 h-3.5 text-emerald-600" /> : <Icon className="w-3.5 h-3.5 text-cyan
-700" />}
1424 |                   {s.label}
1425 |                 </button>
1426 |                 {i < STEPS.length - 1 && <div className={'w-5 h-px shrink-0 ' + (isDone ? 'bg-emerald-400' : 'bg-slate-300')}
/>}
1427 |               </React.Fragment>
1428 |             )
1429 |           })}
1430 |         </motion.div>
1431 |
1432 |         {/* Form Content Cards */}
1433 |         <motion.div key={step} initial="hidden" animate="visible" variants={fadeUp}>
1434 |           <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl overflow-hidden">
1435 |             <CardContent className="p-6 sm:p-8">
```

```
1436 |
1437 |               {/* STEP 1: Course Details */}
1438 |               {step === 1 && (
1439 |                 <div className="space-y-6">
1440 |                   <div>
1441 |                     <h2 className="text-xl font-bold text-slate-900 mb-1">Course Details</h2>
1442 |                     <p className="text-sm text-slate-500">Provide the foundational identity, title, description, and thumbnai
l image.</p>
1443 |                   </div>
1444 |                   <div className="space-y-2">
1445 |                     <Label className="text-sm font-semibold text-slate-700">Course Title *</Label>
1446 |                     <Input {...detailsForm.register('title')} placeholder="e.g. Advanced Doppler Weather Radar Operations & C
alibration" className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12" />
1447 |                     {detailsForm.formState.errors.title && <p className="text-xs text-red-500">{detailsForm.formState.errors.
title.message}</p>}
1448 |                   </div>
1449 |                   <div className="space-y-2">
1450 |                     <Label className="text-sm font-semibold text-slate-700">Description *</Label>
1451 |                     <Textarea {...detailsForm.register('description')} placeholder="What will trainees learn in this course c
urriculum?" className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none" rows={5} />
1452 |                     {detailsForm.formState.errors.description && <p className="text-xs text-red-500">{detailsForm.formState.e
rrors.description.message}</p>}
1453 |                   </div>
1454 |                   <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
1455 |                     <div className="space-y-2">
1456 |                       <Label className="text-sm font-semibold text-slate-700">Course Type *</Label>
1457 |                       <Select
1458 |                         value={isCustomType ? 'custom' : detailsForm.watch('course_type')}
1459 |                         onValueChange={v => {
1460 |                           if (v === 'custom') {
1461 |                             setIsCustomType(true)
1462 |                             detailsForm.setValue('course_type', '')
1463 |                           } else {
1464 |                             setIsCustomType(false)
1465 |                             detailsForm.setValue('course_type', v)
1466 |                           }
1467 |                         }}
1468 |                       >
1469 |                         <SelectTrigger className="border-slate-200 bg-slate-50 rounded-xl h-12"><SelectValue /></SelectTrigge
r>
1470 |                         <SelectContent>
1471 |                           <SelectItem value="standard">Standard Training</SelectItem>
1472 |                           <SelectItem value="scenario">Scenario-Based Training</SelectItem>
1473 |                           <SelectItem value="technical">Technical Training</SelectItem>
1474 |                           <SelectItem value="operational">Operational & Field Training</SelectItem>
1475 |                           <SelectItem value="custom">+ Add Custom Course Type...</SelectItem>
1476 |                         </SelectContent>
1477 |                       </Select>
1478 |                       {isCustomType && (
1479 |                         <Input
1480 |                           {...detailsForm.register('course_type')}
1481 |                           placeholder="Enter custom course type"
1482 |                           className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12 mt-2"
1483 |                         />
1484 |                       )}
1485 |                       {detailsForm.formState.errors.course_type && <p className="text-xs text-red-500">{detailsForm.formStat
e.errors.course_type.message}</p>}
1486 |                     </div>
1487 |                     <div className="space-y-2">
1488 |                       <Label className="text-sm font-semibold text-slate-700">Department</Label>
1489 |                       <Input {...detailsForm.register('department')} placeholder="e.g. IMD, INCOIS, IITM, NCMRWF" className
="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12" />
1490 |                     </div>
1491 |                   </div>
1492 |                 </div>
1493 |               )}
1494 |
1495 |               {/* STEP 2: Course Configuration */}
1496 |               {step === 2 && (
1497 |                 <div className="space-y-6">
1498 |                   <div>
1499 |                     <h2 className="text-xl font-bold text-slate-900 mb-1 flex items-center gap-2">
1500 |                       <Settings className="w-5 h-5 text-cyan-600" />
1501 |                       Course Configuration
1502 |                     </h2>
1503 |                     <p className="text-sm text-slate-500">Configure delivery format, cohort limits, schedules, and assessment
parameters.</p>
1504 |                   </div>
1505 |
```

```
                  <div className="space-y-4">
                    {/* Duration */}
                    <div className="space-y-1.5">
                      <Label className="text-sm font-semibold text-slate-700">Duration (hours)</Label>
                      <Input
                        type="number"
                        {...settingsForm.register('duration_hours')}
                        placeholder="e.g. 20"
                        className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                      />
                      <p className="text-xs text-slate-400 font-medium">Leave empty for self-paced</p>
                    </div>

                    {/* Start Date & End Date */}
                    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                      <div className="space-y-1.5">
                        <Label className="text-sm font-semibold text-slate-700">Start Date</Label>
                        <Input
                          type="date"
                          {...settingsForm.register('start_date')}
                          className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                        />
                        {settingsForm.formState.errors.start_date && (
                          <p className="text-xs text-rose-600 font-medium">{settingsForm.formState.errors.start_date.message}
</p>
                        )}
                      </div>
                      <div className="space-y-1.5">
                        <Label className="text-sm font-semibold text-slate-700">End Date</Label>
                        <Input
                          type="date"
                          {...settingsForm.register('end_date')}
                          className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                        />
                        {settingsForm.formState.errors.end_date && (
                          <p className="text-xs text-rose-600 font-medium">{settingsForm.formState.errors.end_date.message}</
p>
                        )}
                      </div>
                    </div>

                    {/* Urgent warning banner & Duration badge */}
                    {isUrgent && settingsForm.watch('start_date') && (
                      <div className="bg-amber-50 border border-amber-200 text-amber-800 p-3 rounded-xl text-xs font-medium f
lex items-center gap-2">
                        <span className="text-lg">⚠️</span> Course starts in less than 30 days! This will be flagged as <stro
ng className="font-bold">URGENT</strong> for fast-track processing.
                      </div>
                    )}
                    {courseDays && (
                      <div className="text-xs font-semibold text-cyan-800 bg-cyan-50 px-3 py-1.5 rounded-xl border border-cya
n-200 inline-block">
                        🗓️ Total Scheduled Duration: {courseDays} days
                      </div>
                    )}

                    {/* Trainee Capacity Limit */}
                    <div className="space-y-1.5">
                      <Label className="text-sm font-semibold text-slate-700">Trainee Capacity Limit</Label>
                      <Input
                        type="number"
                        {...settingsForm.register('max_trainees')}
                        placeholder="e.g. 50"
                        className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                      />
                      <p className="text-xs text-slate-400 font-medium">Must be between 50 and 250</p>
                      {settingsForm.formState.errors.max_trainees && (
                        <p className="text-xs text-rose-600 font-medium">{settingsForm.formState.errors.max_trainees.message}
</p>
                      )}
                    </div>

                    {/* Notice to Admin */}
                    <div className="space-y-1.5">
                      <Label className="text-sm font-semibold text-slate-700 flex items-center gap-2">
                        Notice to Admin <Badge variant="secondary" className="text-[10px] bg-slate-100 text-slate-600 font-no
rmal">Optional</Badge>
                      </Label>
                      <Textarea
```

```
                        {...settingsForm.register('trainer_suggestion')}
                        placeholder="Add any notes or suggestions for the admin approving this course..."
                        className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none min-h-[72px]"
                      />
                    </div>

                    {/* Divider */}
                    <div className="border-t border-slate-200/80 my-2 pt-3" />

                    {/* Test & Assessment Plan */}
                    <div className="space-y-3">
                      <div>
                        <h3 className="text-sm font-bold text-slate-900">Test & Assessment Plan</h3>
                        <p className="text-xs font-bold text-slate-700 mt-1">Final Exam Details</p>
                      </div>

                      <div className="grid grid-cols-1 sm:grid-cols-3 gap-3">
                        <div className="space-y-1.5">
                          <Label className="text-xs font-semibold text-slate-700">Date</Label>
                          <Input
                            type="date"
                            {...settingsForm.register('final_test_date')}
                            className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                          />
                        </div>
                        <div className="space-y-1.5">
                          <Label className="text-xs font-semibold text-slate-700">Start Time</Label>
                          <Input
                            type="time"
                            {...settingsForm.register('final_test_start_time')}
                            className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                          />
                        </div>
                        <div className="space-y-1.5">
                          <Label className="text-xs font-semibold text-slate-700">End Time</Label>
                          <Input
                            type="time"
                            {...settingsForm.register('final_test_end_time')}
                            className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-11"
                          />
                        </div>
                      </div>

                      {examDuration && (
                        <div className="text-xs font-semibold text-slate-700">
                          Duration: <span className="text-cyan-600 font-bold">{examDuration}</span>
                        </div>
                      )}
                    </div>

                    {/* Divider */}
                    <div className="border-t border-slate-200/80 my-2 pt-3" />

                    {/* Course Thumbnail */}
                    <div className="space-y-2">
                      <Label className="text-sm font-semibold text-slate-700">Course Thumbnail</Label>
                      <input ref={thumbRef} type="file" accept="image/*" className="hidden" onChange={handleThumbnailChange}
/>
                      {thumbnailPreview ? (
                        <div className="relative w-full h-44 rounded-2xl overflow-hidden border border-slate-200 shadow-xs">
                          <img src={thumbnailPreview} alt="Thumbnail preview" className="w-full h-full object-cover" />
                          <button
                            type="button"
                            onClick={() => { setThumbnail(null); setThumbnailPreview(null) }}
                            className="absolute top-2.5 right-2.5 p-1.5 rounded-full bg-slate-900/80 text-white hover:bg-rose
-600 transition-colors"
                          >
                            <X className="w-4 h-4" />
                          </button>
                        </div>
                      ) : (
                        <button
                          type="button"
                          onClick={() => thumbRef.current?.click()}
                          className="w-full h-36 border-2 border-dashed border-slate-200 hover:border-cyan-500 rounded-2xl fl
ex flex-col items-center justify-center gap-2 text-slate-400 hover:text-cyan-600 bg-slate-50/70 hover:bg-cyan-50/20 transition-all c
ursor-pointer"
                        >
                          <Upload className="w-6 h-6" />
```

```
                          <span className="text-xs font-semibold text-slate-600">Click to upload thumbnail</span>
                          <span className="text-[11px] text-slate-400">PNG, JPG up to 5MB (16:5 ratio, e.g. 1600x500px)</span>
                        </button>
                      )}
                    </div>
                  </div>
                </div>
              )}

              {/* STEP 3: Course Outline & Syllabus */}
              {step === 3 && (
                <div className="space-y-6">
                  <div>
                    <h2 className="text-xl font-bold text-slate-900 mb-1">Course Outline & Syllabus</h2>
                    <p className="text-sm text-slate-500">Provide the session flow breakdown or upload an official syllabus document.</p>
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Session Flow & Schedule Breakdown</Label>
                    <Textarea {...sessionFlowForm.register('session_flow_text')} placeholder="e.g. Day 1: Principles of Meteorological Sensors&#10;Day 2: Calibration Protocols&#10;Day 3: Practical Exercises..." className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none font-mono text-xs" rows={8} />
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Attach Syllabus Document <span className="text-slate-400 font-normal">(PDF, DOCX)</span></Label>
                    <div className="p-4 rounded-2xl border-2 border-dashed border-slate-200 bg-slate-50 flex items-center justify-between">
                      <div className="flex items-center gap-3">
                        <FileText className="w-8 h-8 text-cyan-600" />
                        <div>
                          <p className="text-sm font-medium text-slate-800">{sessionFlowDoc ? sessionFlowDoc.name : 'Upload detailed course syllabus'}</p>
                          <p className="text-xs text-slate-400">PDF, DOCX — max 20MB</p>
                        </div>
                      </div>
                      <input type="file" id="syllabus-doc" accept=".pdf,.docx,.doc" onChange={e => setSessionFlowDoc(e.target.files?.[0] || null)} className="hidden" />
                      <Button type="button" variant="outline" onClick={() => document.getElementById('syllabus-doc')?.click()} className="rounded-xl border-slate-300">
                        {sessionFlowDoc ? 'Replace File' : 'Browse File'}
                      </Button>
                    </div>
                  </div>
                </div>
              )}

              {/* STEP 4: Learning Objectives & Skills */}
              {step === 4 && (
                <div className="space-y-6">
                  <div>
                    <h2 className="text-xl font-bold text-slate-900 mb-1">Learning Objectives & Target Skills</h2>
                    <p className="text-sm text-slate-500">Select skills and competencies that will be developed in this course.</p>
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Learning Objectives *</Label>
                    <Textarea {...objectivesForm.register('learning_objectives')} placeholder="Describe key competencies, theoretical mastery, and diagnostic capabilities trainees will acquire..." className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none" rows={4} />
                    {objectivesForm.formState.errors.learning_objectives && <p className="text-xs text-red-500">{objectivesForm.formState.errors.learning_objectives.message}</p>}
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Competency Skills Developed (Click to select)</Label>
                    <div className="flex flex-wrap gap-2 p-4 rounded-2xl bg-slate-50 border border-slate-200 min-h-[70px]">
                      {skills.map(skill => (
                        <button key={skill.id} type="button" onClick={() => toggleSkill(skill.id)} className={'px-3 py-1.5 rounded-full text-xs font-semibold border transition-all cursor-pointer ' + (selectedSkills.includes(skill.id) ? 'bg-cyan-600 text-white border-cyan-600 shadow-sm' : 'bg-white text-slate-600 border-slate-200 hover:bg-slate-100')}>
                          {skill.name}
                        </button>
                      ))}
                    </div>
                  </div>
                  <div className="flex gap-2">
                    <Input value={customSkillName} onChange={e => setCustomSkillName(e.target.value)} placeholder="Add custom
```

```
domain competency..." className="border-slate-200 bg-slate-50 rounded-xl" />
                    <Button type="button" onClick={handleAddCustomSkill} disabled={addingSkill || !customSkillName.trim()} cl
assName="bg-slate-900 text-white rounded-xl shrink-0">
                      {addingSkill ? <Loader2 className="w-4 h-4 animate-spin" /> : <Plus className="w-4 h-4 mr-1" />} Add Sk
ill
                    </Button>
                  </div>
                </div>
              )}

              {/* STEP 5: Interactive Module Sequence Builder */}
              {step === 5 && (
                <div className="space-y-6">
                  <div className="flex items-center justify-between">
                    <div>
                      <h2 className="text-xl font-bold text-slate-900 mb-1">Modules & Lesson Sequences</h2>
                      <p className="text-sm text-slate-500">Organize lessons, photos, videos, links, and module quizzes.</p>
                    </div>
                    <Button type="button" onClick={addModule} className="bg-cyan-600 hover:bg-cyan-700 text-white rounded-xl
text-xs font-bold">
                      <Plus className="w-4 h-4 mr-1" /> Add Module
                    </Button>
                  </div>

                  <div className="space-y-6">
                    {modules.map((mod, modIdx) => (
                      <div key={mod.id} className="p-5 rounded-2xl border border-slate-200 bg-slate-50/70 space-y-4">
                        <div className="flex items-center justify-between gap-3">
                          <div className="flex items-center gap-2 flex-1">
                            <span className="w-7 h-7 rounded-lg bg-cyan-600 text-white font-bold text-xs flex items-center ju
stify-center shrink-0">
                              {modIdx + 1}
                            </span>
                            <Input
                              value={mod.title}
                              onChange={e => updateModuleField(mod.id, 'title', e.target.value)}
                              className="font-bold text-sm bg-white border-slate-200 rounded-xl"
                              placeholder="Module Title"
                            />
                          </div>
                          <Button type="button" variant="ghost" size="sm" onClick={() => removeModule(mod.id)} className="tex
t-slate-400 hover:text-red-600 rounded-lg">
                            <Trash2 className="w-4 h-4" />
                          </Button>
                        </div>

                        {/* Items in Module */}
                        <div className="space-y-2 pl-9">
                          {mod.items.map((item, itIdx) => (
                            <div key={item.id} className="p-3 rounded-xl bg-white border border-slate-200 flex items-center j
ustify-between gap-3 shadow-xs">
                              <div className="flex items-center gap-3">
                                {item.type === 'video' && <Video className="w-4 h-4 text-blue-600" />}
                                {item.type === 'photo' && <Image className="w-4 h-4 text-emerald-600" />}
                                {item.type === 'link' && <Link2 className="w-4 h-4 text-amber-600" />}
                                {item.type === 'text' && <AlignLeft className="w-4 h-4 text-slate-600" />}
                                {item.type === 'quiz' && <HelpCircle className="w-4 h-4 text-purple-600" />}
                                <div>
                                  <p className="text-xs font-bold text-slate-800">{item.title}</p>
                                  {item.duration_minutes && <p className="text-[10px] text-slate-400">{item.duration_minutes}
mins</p>}
                                </div>
                              </div>
                              <div className="flex items-center gap-1">
                                <Button type="button" variant="ghost" size="sm" onClick={() => openEditItemModal(mod.id, ite
m)} className="h-7 w-7 p-0 text-slate-400 hover:text-slate-700">
                                  <Edit3 className="w-3.5 h-3.5" />
                                </Button>
                                <Button type="button" variant="ghost" size="sm" onClick={() => removeModuleItem(mod.id, item.
id)} className="h-7 w-7 p-0 text-slate-400 hover:text-red-600">
                                  <X className="w-3.5 h-3.5" />
                                </Button>
                              </div>
                            </div>
                          ))}

                          {/* Quick Action buttons */}
                          <div className="flex flex-wrap gap-2 pt-2">
                            <Button type="button" variant="outline" size="sm" onClick={() => openAddItemModal(mod.id, 'vide
```

```
o')} className="text-xs rounded-xl bg-white border-slate-200 text-slate-700">
                                <Video className="w-3.5 h-3.5 mr-1 text-blue-600" /> Video
                              </Button>
                              <Button type="button" variant="outline" size="sm" onClick={() => openAddItemModal(mod.id, 'phot
o')} className="text-xs rounded-xl bg-white border-slate-200 text-slate-700">
                                <Image className="w-3.5 h-3.5 mr-1 text-emerald-600" /> Photo
                              </Button>
                              <Button type="button" variant="outline" size="sm" onClick={() => openAddItemModal(mod.id, 'lin
k')} className="text-xs rounded-xl bg-white border-slate-200 text-slate-700">
                                <Link2 className="w-3.5 h-3.5 mr-1 text-amber-600" /> Link
                              </Button>
                              <Button type="button" variant="outline" size="sm" onClick={() => openAddItemModal(mod.id, 'tex
t')} className="text-xs rounded-xl bg-white border-slate-200 text-slate-700">
                                <AlignLeft className="w-3.5 h-3.5 mr-1 text-slate-600" /> Rich Note
                              </Button>
                              <Button type="button" variant="outline" size="sm" onClick={() => openAddQuizQuestion(mod.id)} cla
ssName="text-xs rounded-xl bg-white border-purple-200 text-purple-700 hover:bg-purple-50">
                                <HelpCircle className="w-3.5 h-3.5 mr-1 text-purple-600" /> Add Quiz
                              </Button>
                              <Button type="button" variant="outline" size="sm" onClick={() => handleGenerateAiQuiz(mod.id)} di
sabled={isGeneratingAiQuiz} className="text-xs rounded-xl bg-gradient-to-r from-purple-50 to-indigo-50 border-purple-200 text-purple
-800 hover:opacity-90">
                                <Sparkles className="w-3.5 h-3.5 mr-1 text-purple-600" /> AI Quiz Generator
                              </Button>
                            </div>
                          </div>
                        </div>
                      ))}
                    </div>

                    {/* Add/Edit Module Item Dialog */}
                    {addingItemType && (
                      <div className="p-5 rounded-2xl border border-cyan-200 bg-cyan-50/50 space-y-4">
                        <div className="flex items-center justify-between">
                          <h3 className="text-sm font-bold text-slate-900 capitalize">
                            {editingItemId ? 'Edit' : 'Add'} {addingItemType} Lesson Resource
                          </h3>
                          <Button type="button" variant="ghost" size="sm" onClick={() => setAddingItemType(null)}>
                            <X className="w-4 h-4" />
                          </Button>
                        </div>
                        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
                          <div className="space-y-1 sm:col-span-2">
                            <Label className="text-xs font-semibold">Title *</Label>
                            <Input value={itemTitle} onChange={e => setItemTitle(e.target.value)} placeholder="e.g. Radar Echo
Diagnostic Overview" className="bg-white rounded-xl h-10 text-xs" />
                          </div>
                          {(addingItemType === 'video' || addingItemType === 'link') && (
                            <div className="space-y-1 sm:col-span-2">
                              <Label className="text-xs font-semibold">Resource URL / Embed Link</Label>
                              <Input value={itemUrl} onChange={e => setItemUrl(e.target.value)} placeholder="https://..." class
Name="bg-white rounded-xl h-10 text-xs" />
                            </div>
                          )}
                          {(addingItemType === 'photo' || addingItemType === 'video') && (
                            <div className="space-y-1 sm:col-span-2">
                              <Label className="text-xs font-semibold">Upload Media File</Label>
                              <Input ref={moduleFileInputRef} type="file" accept={addingItemType === 'photo' ? 'image/*' : 'vid
eo/*'} onChange={handleModuleFileChange} className="bg-white rounded-xl h-10 text-xs" />
                            </div>
                          )}
                          <div className="space-y-1">
                            <Label className="text-xs font-semibold">Estimated Duration (minutes)</Label>
                            <Input type="number" value={itemDurationMinutes} onChange={e => setItemDurationMinutes(e.target.val
ue)} className="bg-white rounded-xl h-10 text-xs" />
                          </div>
                        </div>
                        <div className="flex justify-end gap-2">
                          <Button type="button" variant="outline" size="sm" onClick={() => setAddingItemType(null)} className
="rounded-xl">Cancel</Button>
                          <Button type="button" size="sm" onClick={saveModuleItem} className="bg-cyan-600 text-white rounded-x
l">Save Item</Button>
                        </div>
                      </div>
                    )}

                    {/* Add/Edit Quiz Question Modal */}
                    {activeQuizModuleId && (
                      <div className="p-5 rounded-2xl border border-purple-200 bg-purple-50/50 space-y-4">
```

```
                      <div className="flex items-center justify-between">
                        <h3 className="text-sm font-bold text-purple-950">
                          {editingQuestionId ? 'Edit' : 'Create'} Module Quiz Question
                        </h3>
                        <Button type="button" variant="ghost" size="sm" onClick={() => setActiveQuizModuleId(null)}>
                          <X className="w-4 h-4" />
                        </Button>
                      </div>
                      <div className="space-y-2">
                        <Label className="text-xs font-semibold">Question Text *</Label>
                        <Textarea value={quizQuestionText} onChange={e => setQuizQuestionText(e.target.value)} placeholder="E
nter diagnostic or theoretical question..." className="bg-white rounded-xl text-xs resize-none" rows={2} />
                      </div>
                      <div className="space-y-2">
                        <Label className="text-xs font-semibold">Answer Choices (select correct choice circle)</Label>
                        {quizOptions.map((opt, oIdx) => (
                          <div key={oIdx} className="flex items-center gap-2">
                            <input type="radio" name="quiz-correct" checked={quizCorrectOption === oIdx} onChange={() => setQ
uizCorrectOption(oIdx)} className="w-4 h-4 text-purple-600 cursor-pointer" />
                            <Input value={opt} onChange={e => {
                              const next = [...quizOptions]
                              next[oIdx] = e.target.value
                              setQuizOptions(next)
                            }} placeholder={`Option ${oIdx + 1}`} className="bg-white rounded-xl h-9 text-xs flex-1" />
                          </div>
                        ))}
                      </div>
                      <div className="space-y-1">
                        <Label className="text-xs font-semibold">Explanation (shown to trainee after answer)</Label>
                        <Input value={quizExplanation} onChange={e => setQuizExplanation(e.target.value)} placeholder="Reason
why this is the correct answer..." className="bg-white rounded-xl h-9 text-xs" />
                      </div>
                      <div className="flex justify-end gap-2">
                        <Button type="button" variant="outline" size="sm" onClick={() => setActiveQuizModuleId(null)} classNa
me="rounded-xl">Cancel</Button>
                        <Button type="button" size="sm" onClick={saveQuizQuestion} className="bg-purple-600 hover:bg-purple-7
00 text-white rounded-xl">Save Quiz Question</Button>
                      </div>
                    </div>
                  )}
                </div>
              )}

              {/* STEP 6: Materials Upload */}
              {step === 6 && (
                <div className="space-y-6">
                  <div>
                    <h2 className="text-xl font-bold text-slate-900 mb-1">Course Reference Materials</h2>
                    <p className="text-sm text-slate-500">Upload PDF guidebooks, slide decks, telemetry manuals, and referenc
e URLs.</p>
                  </div>
                  <div
                    onDragOver={e => { e.preventDefault(); setDragOver(true) }}
                    onDragLeave={() => setDragOver(false)}
                    onDrop={handleDrop}
                    className={`p-8 rounded-3xl border-2 border-dashed text-center transition-all cursor-pointer ${
                      dragOver ? 'border-cyan-500 bg-cyan-50' : 'border-slate-200 bg-slate-50 hover:bg-slate-100/70'
                    }`}
                    onClick={() => fileInputRef.current?.click()}
                  >
                    <Upload className="w-10 h-10 text-cyan-600 mx-auto mb-2" />
                    <p className="text-sm font-bold text-slate-800">Drag & drop course files here, or click to browse</p>
                    <p className="text-xs text-slate-400 mt-1">PDF, DOCX, PPTX, MP4, WebM — up to 100MB per file</p>
                    <input ref={fileInputRef} type="file" multiple accept={ACCEPTED_EXTENSIONS.join(',')} onChange={handleFil
eInput} className="hidden" />
                  </div>

                  {/* Add Web Link */}
                  <div className="p-4 rounded-2xl bg-slate-50 border border-slate-200 space-y-3">
                    <p className="text-xs font-bold text-slate-800">Or Add External Web / Video Link</p>
                    <div className="flex gap-2">
                      <Input value={linkTitle} onChange={e => setLinkTitle(e.target.value)} placeholder="Resource Title (opti
onal)" className="bg-white rounded-xl text-xs h-10" />
                      <Input value={linkUrl} onChange={e => setLinkUrl(e.target.value)} placeholder="https://..." className
="bg-white rounded-xl text-xs h-10" />
                      <Button type="button" onClick={addLinkMaterial} className="bg-slate-900 text-white rounded-xl h-10 text
-xs shrink-0">Add Link</Button>
                    </div>
                  </div>
```

```
                    {/* Pending Materials List */}
                    {pendingMaterials.length > 0 && (
                      <div className="space-y-2">
                        <p className="text-xs font-bold text-slate-700">Attached Materials ({pendingMaterials.length})</p>
                        {pendingMaterials.map(mat => {
                          const Icon = getFileIcon(mat.mimeType)
                          return (
                            <div key={mat.id} className="p-3 rounded-xl bg-white border border-slate-200 flex items-center just
ify-between gap-3 shadow-xs">
                              <div className="flex items-center gap-3 truncate">
                                <Icon className="w-5 h-5 text-cyan-600 shrink-0" />
                                <div className="truncate">
                                  <p className="text-xs font-bold text-slate-800 truncate">{mat.fileName}</p>
                                  <p className="text-[10px] text-slate-400">{mat.fileSize > 0 ? formatFileSize(mat.fileSize) :
mat.url}</p>
                                </div>
                              </div>
                              <Button type="button" variant="ghost" size="sm" onClick={() => removeMaterial(mat.id)} className
="text-slate-400 hover:text-red-600 rounded-lg">
                                <Trash2 className="w-4 h-4" />
                              </Button>
                            </div>
                          )
                        })}
                      </div>
                    )}
                  </div>
                )}

                {/* STEP 7: Trainer Assignment & AI Competency Mapping */}
                {step === 7 && (
                  <div className="space-y-6">
                    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 border-b border-slate-200 p
b-4">
                      <div>
                        <h2 className="text-xl font-bold text-slate-900 tracking-tight flex items-center gap-2">
                          <UserCheck className="w-5 h-5 text-cyan-600" />
                          Trainer Assignment & AI Competency Mapping
                        </h2>
                        <p className="text-sm text-slate-600 font-medium mt-0.5">
                          Assign an approved trainer to lead this course curriculum.
                        </p>
                      </div>

                      <div className="flex items-center gap-2 bg-slate-100 p-1 rounded-xl border border-slate-200 shrink-0">
                        <button
                          type="button"
                          onClick={() => setTrainerTab('ai_recommended')}
                          className={`px-3 py-1.5 rounded-lg text-xs font-bold transition-all cursor-pointer ${
                            trainerTab === 'ai_recommended'
                              ? 'bg-cyan-600 text-white shadow-sm'
                              : 'text-slate-600 hover:text-slate-900'
                          }`}
                        >
                          ✨ AI Suggestions ({scoredTrainers.slice(0, 3).length})
                        </button>
                        <button
                          type="button"
                          onClick={() => setTrainerTab('all')}
                          className={`px-3 py-1.5 rounded-lg text-xs font-bold transition-all cursor-pointer ${
                            trainerTab === 'all'
                              ? 'bg-cyan-600 text-white shadow-sm'
                              : 'text-slate-600 hover:text-slate-900'
                          }`}
                        >
                          All Trainers ({trainers.length})
                        </button>
                      </div>
                    </div>

                    {/* AI Competency Matcher Section & Trainer Cards */}
                    <div className="space-y-4">
                      <div className="bg-gradient-to-br from-cyan-50 via-sky-50 to-blue-50 p-5 rounded-2xl border border-cyan-2
00 space-y-3">
                        <div className="flex items-center justify-between">
                          <div className="flex items-center gap-2">
                            <span className="p-1.5 rounded-lg bg-cyan-600 text-white shadow-xs">
                              <Sparkles className="w-4 h-4" />
```

```
                          </span>
                          <div>
                            <h3 className="text-sm font-bold text-slate-900">
                              {trainerTab === 'ai_recommended' ? 'AI Competency Mapping Engine' : 'All Approved Trainers'}
                            </h3>
                            <p className="text-xs text-slate-600">
                              {trainerTab === 'ai_recommended'
                                ? "Evaluates verified skills, domain experience, and credentials against this course's topic
s."
                                : `Browse all ${trainers.length} approved trainers and view their real-time competency alignm
ent.`}
                            </p>
                          </div>
                        </div>

                        {trainerTab === 'ai_recommended' && (
                          <button
                            type="button"
                            onClick={() => {
                              setIsAnalyzingCompetency(true)
                              setTimeout(() => {
                                setIsAnalyzingCompetency(false)
                                toast.success('Competency mapping recalculated!')
                              }, 500)
                            }}
                            className="hidden sm:flex items-center gap-1.5 text-xs font-bold text-cyan-800 bg-white hover:bg-
cyan-100/80 px-3 py-1.5 rounded-xl border border-cyan-200 transition-colors cursor-pointer"
                          >
                            <Zap className="w-3.5 h-3.5 text-amber-500" />
                            {isAnalyzingCompetency ? 'Scanning...' : 'Re-scan Trainers'}
                          </button>
                        )}
                      </div>

                      {/* Trainer Cards Grid */}
                      <div className="grid sm:grid-cols-3 gap-3 pt-2">
                        {(trainerTab === 'ai_recommended' ? scoredTrainers.slice(0, 3) : scoredTrainers).map((t, idx) => {
                          const isSelected = trainerForm.watch('trainer_id') === t.id
                          const isHigh = t.matchPercentage >= 70
                          const isModerate = t.matchPercentage >= 35 && t.matchPercentage < 70
                          const isLow = t.matchPercentage > 0 && t.matchPercentage < 35
                          const isIncomplete = t.matchPercentage === 0

                          return (
                            <div
                              key={t.id}
                              onClick={() => {
                                trainerForm.setValue('trainer_id', t.id, { shouldValidate: true })
                                handleAutoFillMessage(t)
                              }}
                              className={`p-4 rounded-2xl border-2 cursor-pointer transition-all flex flex-col justify-betwee
n ${
                                isSelected
                                  ? 'bg-[#040c1e] text-white border-cyan-400 shadow-xl shadow-cyan-950/40 ring-2 ring-cyan-40
0/40'
                                  : 'bg-white border-slate-200 hover:border-cyan-400 hover:shadow-md'
                              }`}
                            >
                              <div>
                                <div className="flex items-start justify-between gap-2 mb-2.5">
                                  <div className="flex items-center gap-2">
                                    <div
                                      className={`w-9 h-9 rounded-xl flex items-center justify-center font-bold text-sm shrin
k-0 ${
                                        isSelected
                                          ? 'bg-gradient-to-br from-cyan-400 to-blue-500 text-slate-950 font-black'
                                          : 'bg-cyan-100 text-cyan-900'
                                      }`}
                                    >
                                      {t.full_name.charAt(0)}
                                    </div>
                                    <div>
                                      <p className={`font-bold text-sm line-clamp-1 ${isSelected ? 'text-white' : 'text-slate
-900'}`}>
                                        {t.full_name}
                                      </p>
                                      <p className={`text-[11px] ${isSelected ? 'text-cyan-200' : 'text-slate-500'}`}>
                                        {t.years_of_experience
                                          ? `${t.years_of_experience} yrs exp`
```

```
                                        : t.qualifications
                                        ? t.qualifications
                                        : 'Profile Incomplete'}
                                    </p>
                                  </div>
                                </div>
                              </div>

                              <div className="mb-2">
                                <span
                                  className={`inline-flex items-center gap-1 text-[11px] font-extrabold px-2.5 py-0.5 round
ed-full ${
                                    isSelected
                                      ? 'bg-cyan-500/30 text-cyan-200 border border-cyan-400/40'
                                      : isHigh
                                      ? 'bg-emerald-100 text-emerald-800 border border-emerald-300'
                                      : isModerate
                                      ? 'bg-sky-100 text-sky-800 border border-sky-300'
                                      : isLow
                                      ? 'bg-amber-100 text-amber-800 border border-amber-300'
                                      : 'bg-slate-100 text-slate-600 border border-slate-300'
                                  }`}
                                >
                                  <TrendingUp className="w-3 h-3" />
                                  {isIncomplete ? 'Incomplete Profile (0%)' : `${t.matchPercentage}% Competency Fit`}
                                </span>
                              </div>

                              <p className={`text-[11px] leading-relaxed line-clamp-2 mb-3 ${isSelected ? 'text-slate-300'
: 'text-slate-600'}`}>
                                {t.reasoning}
                              </p>

                              {t.matchedParts && t.matchedParts.length > 0 && (
                                <div className="flex flex-wrap gap-1 mb-1.5">
                                  {t.matchedParts.map((part, pIdx) => (
                                    <span
                                      key={pIdx}
                                      className={`text-[9px] font-extrabold px-1.5 py-0.5 rounded ${
                                        isSelected
                                          ? 'bg-cyan-950 text-cyan-300 border border-cyan-500/40'
                                          : 'bg-emerald-50 text-emerald-800 border border-emerald-200'
                                      }`}
                                    >
                                      ✦ {part}
                                    </span>
                                  ))}
                                </div>
                              )}

                              {t.matchedSkillsList.length > 0 && (
                                <div className="flex flex-wrap gap-1 mb-2">
                                  {t.matchedSkillsList.slice(0, 2).map((skillName, sIdx) => (
                                    <span
                                      key={sIdx}
                                      className={`text-[10px] font-bold px-2 py-0.5 rounded-md ${
                                        isSelected
                                          ? 'bg-cyan-950 text-cyan-300 border border-cyan-500/40'
                                          : 'bg-slate-100 text-slate-700 border border-slate-200'
                                      }`}
                                    >
                                      ✓ {skillName}
                                    </span>
                                  ))}
                                </div>
                              )}
                            </div>

                            <Button
                              type="button"
                              size="sm"
                              className={`w-full text-xs font-bold rounded-xl mt-2 cursor-pointer transition-all ${
                                isSelected
                                  ? 'bg-cyan-400 hover:bg-cyan-300 text-slate-950 font-black'
                                  : 'bg-slate-100 hover:bg-cyan-600 hover:text-white text-slate-800'
                              }`}
                            >
                              {isSelected ? 'Assigned ✓' : 'Select Trainer'}
                            </Button>
```

```
                            </div>
                          )
                        })}
                      </div>
                    </div>
                  </div>

                  {/* Manual Trainer Dropdown Select */}
                  <div className="space-y-2">
                    <Label className="text-sm font-bold text-slate-900">
                      Select Trainer Directly *
                    </Label>
                    {loadingTrainers ? (
                      <div className="flex items-center gap-2 text-slate-500 text-sm py-3">
                        <Loader2 className="w-4 h-4 animate-spin text-cyan-600" />
                        Loading approved trainers...
                      </div>
                    ) : (
                      <Select
                        value={trainerForm.watch('trainer_id')}
                        onValueChange={(v) => {
                          trainerForm.setValue('trainer_id', v, { shouldValidate: true })
                          const tr = scoredTrainers.find(t => t.id === v)
                          if (tr) handleAutoFillMessage(tr)
                        }}
                      >
                        <SelectTrigger className="border-slate-300 focus:border-cyan-500 rounded-xl h-12 bg-white text-slate-
900 font-medium">
                          <SelectValue placeholder="Choose a trainer from the full roster..." />
                        </SelectTrigger>
                        <SelectContent className="max-h-80">
                          {scoredTrainers.map((t) => (
                            <SelectItem key={t.id} value={t.id}>
                              <div className="flex items-center gap-2.5 py-1">
                                <div className="w-6 h-6 rounded-lg bg-cyan-600 text-white flex items-center justify-center te
xt-[10px] font-bold shrink-0">
                                  {t.full_name.charAt(0)}
                                </div>
                                <span className="font-bold text-slate-900">{t.full_name}</span>
                                <span className="text-xs text-slate-500">· {t.qualifications || 'Trainer'}</span>
                                <span className="text-[10px] font-extrabold px-1.5 py-0.5 rounded bg-cyan-100 text-cyan-800 m
l-auto">
                                  {t.matchPercentage}% match
                                </span>
                              </div>
                            </SelectItem>
                          ))}
                        </SelectContent>
                      </Select>
                    )}
                    {trainerForm.formState.errors.trainer_id && (
                      <p className="text-xs text-red-600 font-semibold flex items-center gap-1 mt-1">
                        <AlertCircle className="w-3.5 h-3.5" />
                        {trainerForm.formState.errors.trainer_id.message}
                      </p>
                    )}
                  </div>

                  {/* Selected Trainer Detail Banner */}
                  {selectedTrainer && (
                    <div className="p-4 rounded-2xl bg-cyan-50 border border-cyan-200 flex flex-col sm:flex-row items-start s
m:items-center gap-4">
                      <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-700 flex items-center jus
tify-center text-white font-black text-xl shadow-md shadow-cyan-600/20 shrink-0">
                        {selectedTrainer.full_name.charAt(0)}
                      </div>
                      <div className="space-y-1">
                        <div className="flex items-center gap-2">
                          <p className="font-bold text-base text-slate-900">{selectedTrainer.full_name}</p>
                          <Badge className="bg-emerald-600 text-white text-[10px] font-bold border-none">
                            Assigned Lead Trainer
                          </Badge>
                        </div>
                        <p className="text-xs text-slate-600 font-medium">{selectedTrainer.email}</p>
                        <p className="text-[11px] font-medium text-slate-700 mt-1">
                          {selectedTrainer.qualifications || 'Certified Trainer'}
                          {selectedTrainer.years_of_experience ? ` • ${selectedTrainer.years_of_experience} years scientific
training experience` : ''}
                        </p>
```

```
                      </div>
                    </div>
                  )}

                  {/* Assignment Announcement Briefing Message */}
                  <div className="space-y-2">
                    <div className="flex items-center justify-between">
                      <Label className="text-sm font-bold text-slate-900">
                        Briefing Message to Trainer <span className="text-slate-400 font-normal">(optional)</span>
                      </Label>
                      {selectedTrainer && (
                        <button
                          type="button"
                          onClick={() => {
                            const tr = scoredTrainers.find(t => t.id === selectedTrainer.id)
                            if (tr) handleAutoFillMessage(tr)
                          }}
                          className="text-xs font-bold text-cyan-700 hover:text-cyan-900 flex items-center gap-1 cursor-point
er"
                        >
                          <Sparkles className="w-3 h-3 text-cyan-600" /> Auto-compose AI Briefing
                        </button>
                      )}
                    </div>
                    <Textarea
                      {...trainerForm.register('assignment_message')}
                      placeholder="e.g. Please build this course curriculum with focus on Doppler radar calibration and numer
ical forecasting practical exercises..."
                      className="border-slate-300 focus:border-cyan-500 bg-white text-slate-900 rounded-xl resize-none"
                      rows={3}
                    />
                    <p className="text-xs text-slate-500">This briefing will automatically be delivered to the trainer's dash
board announcements feed upon course creation.</p>
                  </div>

                  {selectedTrainer && (
                    <div className="p-4 rounded-2xl border border-dashed border-cyan-300 bg-cyan-50/40 space-y-2">
                      <div className="flex items-center gap-2 text-cyan-800 text-xs font-bold uppercase tracking-wider">
                        <Megaphone className="w-3.5 h-3.5 text-cyan-600" /> Live Preview: Trainer Notification Card
                      </div>
                      <div className="bg-white rounded-xl p-3.5 border border-slate-200 shadow-sm">
                        <p className="text-sm font-bold text-slate-900">🎓 New Course Assignment: {detailsForm.watch('title')
|| 'Course Title'}</p>
                        <p className="text-xs text-slate-500 mt-0.5">Assigned by {profile?.full_name ?? 'Lead Administrator'}
</p>
                        {trainerForm.watch('assignment_message') && (
                          <div className="mt-2.5 p-2.5 rounded-lg bg-slate-50 border border-slate-200 text-xs text-slate-700
italic">
                            "{trainerForm.watch('assignment_message')}"
                          </div>
                        )}
                      </div>
                    </div>
                  )}
                </div>
              )}

              {/* STEP 8: Digital Certificate Customizer */}
              {step === 8 && (
                <div className="space-y-6">
                  <div>
                    <h2 className="text-xl font-bold text-slate-900 mb-1">Official MoES Digital Certificate</h2>
                    <p className="text-sm text-slate-500">Customize the certificate template and verification badges awarded
to trainees upon completion.</p>
                  </div>
                  <CourseCertificateStep
                    hasCertificate={hasCertificate}
                    setHasCertificate={setHasCertificate}
                    templateUrl={certificateTemplateUrl}
                    setTemplateUrl={setCertificateTemplateUrl}
                    templateName={certificateTemplateName}
                    setTemplateName={setCertificateTemplateName}
                    courseTitle={detailsForm.watch('title')}
                  />
                </div>
              )}

              {/* STEP 9: Review & Publish */}
              {step === 9 && (
```

```
                <div className="space-y-6">
                  <div>
                    <h2 className="text-xl font-bold text-slate-900 mb-1">Review & Publish Course</h2>
                    <p className="text-sm text-slate-500">Confirm all course parameters before publishing live or saving as draft.</p>
                  </div>
                  <div className="p-4 rounded-2xl bg-cyan-50 border border-cyan-200">
                    <p className="text-xs font-bold text-cyan-800 uppercase tracking-wider mb-2">Lead Trainer Assignment</p>
                    {selectedTrainer ? (
                      <div className="flex items-center gap-3">
                        <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white font-bold shadow-xs">
                          {selectedTrainer.full_name.charAt(0)}
                        </div>
                        <div>
                          <p className="font-bold text-slate-900">{selectedTrainer.full_name}</p>
                          <p className="text-xs text-slate-500">{selectedTrainer.email}</p>
                        </div>
                        <Badge className="ml-auto bg-emerald-100 text-emerald-800 border-emerald-300">
                          Trainer Assigned ✓
                        </Badge>
                      </div>
                    ) : (
                      <p className="text-sm text-red-500 font-medium">⚠️ No trainer assigned. Please return to Step 7.</p>
                    )}
                  </div>
                  <div className="p-4 rounded-2xl bg-slate-50 border border-slate-200 grid grid-cols-2 sm:grid-cols-4 gap-3 text-sm">
                    <div>
                      <p className="text-xs text-slate-400">Title</p>
                      <p className="font-semibold text-slate-900 truncate">{detailsForm.watch('title')}</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Type</p>
                      <p className="font-semibold text-slate-900 capitalize">{detailsForm.watch('course_type')}</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Delivery</p>
                      <p className="font-semibold text-slate-900 capitalize">{settingsForm.watch('delivery_mode') || 'Recorded'}</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Passing Score</p>
                      <p className="font-semibold text-slate-900">{settingsForm.watch('passing_score')}%</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Modules</p>
                      <p className="font-semibold text-slate-900">{modules.length} modules</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Materials</p>
                      <p className="font-semibold text-slate-900">{pendingMaterials.length} files</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Certificate</p>
                      <p className="font-semibold text-emerald-600">{hasCertificate ? 'Enabled ✓' : 'Disabled'}</p>
                    </div>
                    <div>
                      <p className="text-xs text-slate-400">Max Capacity</p>
                      <p className="font-semibold text-slate-900">{settingsForm.watch('max_trainees') || 'Unlimited'}</p>
                    </div>
                  </div>

                  <div className="p-4 rounded-2xl bg-gradient-to-r from-emerald-50 to-teal-50 border border-emerald-200/60">
                    <div className="flex items-start gap-3">
                      <div className="w-8 h-8 rounded-xl bg-emerald-100 flex items-center justify-center shrink-0">
                        <Megaphone className="w-4 h-4 text-emerald-600" />
                      </div>
                      <div>
                        <p className="text-sm font-bold text-emerald-900">Administrator Direct Deployment</p>
                        <ul className="text-xs text-emerald-800 mt-1 space-y-0.5">
                          <li>• Publishing goes live immediately for all Ministry trainees</li>
                          <li>• The assigned trainer is instantly notified and granted lead management over this course</li>
                        </ul>
                      </div>
                    </div>
                  </div>

                  <div className="flex gap-3 pt-2">
```

```
                    <Button
                      type="button"
                      variant="outline"
                      onClick={() => handlePublish('draft')}
                      disabled={saving}
                      className="flex-1 border-slate-200 text-slate-700 hover:bg-slate-50 rounded-xl h-12 font-semibold curso
r-pointer"
                    >
                      {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null} Save as Draft
                    </Button>
                    <Button
                      type="button"
                      onClick={() => handlePublish('published')}
                      disabled={saving}
                      className="flex-1 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white fo
nt-bold rounded-xl h-12 shadow-lg shadow-cyan-600/20 cursor-pointer"
                    >
                      {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Megaphone className="w-4 h-4 mr-2" />} P
ublish Live & Assign Trainer
                    </Button>
                  </div>
                </div>
              )}

            </CardContent>
          </Card>
        </motion.div>

        {/* Wizard Footer Navigation */}
        {step < 9 && (
          <motion.div initial="hidden" animate="visible" variants={fadeUp} className="flex justify-between mt-6">
            <Button
              type="button"
              variant="outline"
              onClick={handleBack}
              disabled={step === 1}
              className="border-slate-200 text-slate-700 hover:bg-slate-50 font-medium rounded-xl px-6 cursor-pointer"
            >
              <ArrowLeft className="w-4 h-4 mr-2" /> Back
            </Button>
            <Button
              type="button"
              onClick={handleNext}
              className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-medium rounded-xl px-6 s
hadow-md shadow-cyan-600/10 cursor-pointer"
            >
              Next Step <ArrowRight className="w-4 h-4 ml-2" />
            </Button>
          </motion.div>
        )}
      </div>

      <ImageCropperModal
        isOpen={isCropperOpen}
        imageFile={rawImageFile}
        onClose={() => {
          setIsCropperOpen(false)
          setRawImageFile(null)
        }}
        onCropComplete={(croppedFile) => {
          setThumbnail(croppedFile)
          setThumbnailPreview(URL.createObjectURL(croppedFile))
          setIsCropperOpen(false)
          setRawImageFile(null)
        }}
      />
    </div>
  )
}

```

src/features/admin/AdminCourseEditPage.tsx TSX 32.6 KB 516 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react'
import { useNavigate, useParams } from 'react-router-dom'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { motion } from 'framer-motion'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Label } from '@/components/ui/label'
import { Card, CardContent } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select'
import {
  ArrowLeft, ArrowRight, CheckCircle, Loader2, BookOpen, Settings,
  Target, Eye, AlertCircle, Image, UserCheck, Megaphone
} from 'lucide-react'
import { toast } from 'sonner'
import { ImageCropperModal } from '@/components/ui/ImageCropperModal'

type Trainer = { id: string; full_name: string; email: string; qualifications?: string | null; years_of_experience?: number | null }
type Skill = { id: string; name: string }

const trainerSchema = z.object({
  trainer_id: z.string().min(1, 'Please select a trainer'),
  assignment_message: z.string().optional(),
})
const detailsSchema = z.object({
  title: z.string().min(5, 'At least 5 characters'),
  description: z.string().min(10, 'At least 10 characters'),
  course_type: z.string().min(2, 'Course type is required'),
  department: z.string().optional(),
})
const settingsSchema = z.object({
  duration_hours: z.coerce.number().positive().optional(),
  passing_score: z.coerce.number().min(1).max(100).optional(),
  start_date: z.string().optional(),
  end_date: z.string().optional(),
  delivery_mode: z.enum(['recorded', 'live', 'hybrid']),
  max_trainees: z.coerce.number().min(50, 'Minimum capacity is 50').max(250, 'Maximum capacity is 250').optional(),
}).superRefine((data, ctx) => {
  if (data.start_date) {
    const start = new Date(data.start_date)
    const today = new Date()
    today.setHours(0, 0, 0, 0)
    const diffDays = Math.ceil((start.getTime() - today.getTime()) / (1000 * 60 * 60 * 24))
    if (diffDays < 30) {
      ctx.addIssue({
        code: z.ZodIssueCode.custom,
        message: 'Course must start at least 1 month from today',
        path: ['start_date']
      })
    }
  }
})
const objectivesSchema = z.object({
  understand: z.string().min(5),
  able_to_do: z.string().min(5),
  competencies_built: z.string().min(5),
})

type TrainerData = z.infer<typeof trainerSchema>
type DetailsData = z.infer<typeof detailsSchema>
type SettingsData = z.infer<typeof settingsSchema>
type ObjectivesData = z.infer<typeof objectivesSchema>

const STEPS = [
  { id: 1, label: 'Assign Trainer', icon: UserCheck },
  { id: 2, label: 'Course Details', icon: BookOpen },
  { id: 3, label: 'Configuration', icon: Settings },
  { id: 4, label: 'Objectives', icon: Target },
  { id: 5, label: 'Review', icon: Eye },
]
```

```
76 |
77 | const fadeUp = { hidden: { opacity: 0, y: 20 }, visible: { opacity: 1, y: 0, transition: { duration: 0.4 } } }
78 |
79 | export function AdminCourseEditPage() {
80 |   const { courseId } = useParams<{ courseId: string }>()
81 |   const [course, setCourse] = useState<any>(null)
82 |   const [loadingCourse, setLoadingCourse] = useState(true)
83 |   const [isTrainerApplied, setIsTrainerApplied] = useState(false)
84 |   const { user, profile } = useAuth()
85 |   const navigate = useNavigate()
86 |   const [step, setStep] = useState(1)
87 |   const [saving, setSaving] = useState(false)
88 |   const [trainers, setTrainers] = useState<Trainer[]>([])
89 |   const [loadingTrainers, setLoadingTrainers] = useState(true)
90 |   const [skills, setSkills] = useState<Skill[]>([])
91 |   const [selectedSkills, setSelectedSkills] = useState<string[]>([])
92 |   const [thumbnail, setThumbnail] = useState<File | null>(null)
93 |   const [thumbnailPreview, setThumbnailPreview] = useState<string | null>(null)
94 |   const [isCustomType, setIsCustomType] = useState(false)
95 |   const [isCropperOpen, setIsCropperOpen] = useState(false)
96 |   const [rawImageFile, setRawImageFile] = useState<File | null>(null)
97 |   const thumbRef = useRef<HTMLInputElement>(null)
98 |
99 |   const trainerForm = useForm<TrainerData>({ resolver: zodResolver(trainerSchema), defaultValues: { trainer_id: '', assignment
_message: '' } })
100 |   const detailsForm = useForm<DetailsData>({ resolver: zodResolver(detailsSchema), defaultValues: { title: '', description:
'', course_type: 'standard', department: '' } })
101 |   const settingsForm = useForm<SettingsData>({ resolver: zodResolver(settingsSchema), defaultValues: { passing_score: 60, deli
very_mode: 'recorded' } })
102 |   const objectivesForm = useForm<ObjectivesData>({ resolver: zodResolver(objectivesSchema), defaultValues: { understand: '', a
ble_to_do: '', competencies_built: '' } })
103 |
104 |   useEffect(() => {
105 |     supabase.from('trainers').select('id, full_name, email, qualifications, years_of_experience').eq('approval_status', 'appro
ved').order('full_name')
106 |       .then(({ data }) => { if (data) setTrainers(data as any[]); setLoadingTrainers(false) })
107 |     supabase.from('skills').select('*').order('name').then(({ data }) => { if (data) setSkills(data) })
108 |
109 |     if (courseId) {
110 |       supabase.from('courses').select('*').eq('id', courseId).single()
111 |         .then(async ({ data: courseData }) => {
112 |           if (courseData) {
113 |             setCourse(courseData)
114 |             trainerForm.reset({ trainer_id: courseData.trainer_id || '', assignment_message: '' })
115 |             detailsForm.reset({
116 |               title: courseData.title,
117 |               description: courseData.description || '',
118 |               course_type: courseData.course_type,
119 |               department: courseData.department || '',
120 |             })
121 |             settingsForm.reset({
122 |               duration_hours: courseData.duration_minutes ? Math.floor(courseData.duration_minutes / 60) : undefined,
123 |               passing_score: courseData.passing_score ?? 60,
124 |               start_date: courseData.start_date ? new Date(courseData.start_date).toISOString().slice(0, 10) : undefined,
125 |               end_date: courseData.end_date ? new Date(courseData.end_date).toISOString().slice(0, 10) : undefined,
126 |               delivery_mode: (courseData.delivery_mode as any) || 'recorded',
127 |               max_trainees: courseData.max_trainees || undefined,
128 |             })
129 |             if (courseData.learning_objectives) {
130 |               const obj = courseData.learning_objectives as any
131 |               objectivesForm.reset({
132 |                 understand: obj.understand || '',
133 |                 able_to_do: obj.able_to_do || '',
134 |                 competencies_built: obj.competencies_built || ''
135 |               })
136 |             }
137 |             if (courseData.thumbnail_path) {
138 |               // Note: you might want to fetch a signed URL here if needed.

139 |               // For simplicity, we assume we just show a placeholder or the raw path.

140 |             }
141 |             const { data: csData } = await supabase.from('course_skills').select('skill_id').eq('course_id', courseId as strin
g)
142 |             if (csData) setSelectedSkills(csData.map(cs => cs.skill_id))
143 |
144 |             // Check if course was applied by trainer (no assignments)

145 |             const { count } = await supabase.from('course_assignments' as any).select('*', { count: 'exact', head: true }).eq
```

```
('course_id', courseId as string)
            setIsTrainerApplied(count === 0)
          }
          setLoadingCourse(false)
        })
    }
  }, [courseId])

  const validateStep = async () => {
    if (step === 1) return trainerForm.trigger()
    if (step === 2) return detailsForm.trigger()
    if (step === 3) return settingsForm.trigger()
    if (step === 4) return objectivesForm.trigger()
    return true
  }

  const handleNext = async () => { const ok = await validateStep(); if (ok) setStep(s => Math.min(s + 1, 5)) }
  const handleBack = () => setStep(s => Math.max(s - 1, 1))
  const toggleSkill = (id: string) => setSelectedSkills(prev => prev.includes(id) ? prev.filter(s => s !== id) : [...prev, i
d])
  const selectedTrainer = trainers.find(t => t.id === trainerForm.watch('trainer_id'))

  const handlePublish = async (status: 'draft' | 'published') => {
    const t = trainerForm.getValues()
    const d = detailsForm.getValues()
    if (!t.trainer_id || !d.title) { toast.error('Trainer and title are required'); return }
    setSaving(true)
    try {
      const s = settingsForm.getValues()
      const o = objectivesForm.getValues()
      const { error } = await supabase.from('courses').update({
        title: d.title, description: d.description, course_type: d.course_type,
        department: d.department || null, trainer_id: t.trainer_id,
        duration_minutes: s.duration_hours ? s.duration_hours * 60 : null,
        passing_score: s.passing_score ?? 60, delivery_mode: s.delivery_mode || 'recorded',
        max_trainees: s.max_trainees || null,
        start_date: s.start_date ? new Date(s.start_date).toISOString() : null,
        end_date: s.end_date ? new Date(s.end_date).toISOString() : null,
        learning_objectives: { understand: o.understand, able_to_do: o.able_to_do, competencies_built: o.competencies_built },
        status: status, // Important: Actually update the status!
      }).eq('id', courseId as string)
      // we already have the course variable from state, we can use that for thumbnail logic

      const course = { id: courseId as string }
      if (error) throw error
      if (thumbnail && course) {
        const ext = thumbnail.name.split('.').pop()
        const path = course.id + '/thumbnail.' + ext
        const { error: upErr } = await supabase.storage.from('materials').upload(path, thumbnail)
        if (!upErr) await supabase.from('courses').update({ thumbnail_path: path }).eq('id', course.id)
      }

      if (course) {
        // delete old skills and insert new ones

        await supabase.from('course_skills').delete().eq('course_id', courseId as string)
        if (selectedSkills.length > 0) {
          await supabase.from('course_skills').insert(selectedSkills.map(sid => ({ course_id: course.id, skill_id: sid, requir
ed_level: 3 })))
        }

        if (!isTrainerApplied) {
          // if trainer changed or message changed, update assignment

          // for simplicity, just try deleting (might silently do nothing without RLS policy) and then insert

          try {
            await supabase.from('course_assignments' as any).delete().eq('course_id', courseId as string)
          } catch (e) { /* ignore if RLS blocks delete */ }

          await supabase.from('course_assignments' as any).insert({
            course_id: course.id, trainer_id: t.trainer_id,
            assigned_by: user!.id, message: t.assignment_message || null,
          } as any)
        }
      }
      toast.success(status === 'published' ? 'Course published & assigned!' : 'Course updated successfully!')
      navigate('/admin')
    } catch (err: any) {
```

```
      console.error(err)
      toast.error(err?.message || 'Failed to update course')
    } finally { setSaving(false) }
  }


  if (loadingCourse) return <div className="min-h-screen bg-[#f8fafc] flex items-center justify-center"><Loader2 className="w-8 h-8 animate-spin text-cyan-600" /></div>

  return (
    <div className="min-h-screen bg-[#f8fafc]">
      <div className="max-w-3xl mx-auto px-4 py-8">
        <motion.div initial="hidden" animate="visible" variants={fadeUp} className="mb-8">
          <button onClick={() => navigate('/admin')} className="flex items-center gap-2 text-sm text-slate-500 hover:text-cyan-600 mb-4 font-medium transition-colors">
            <ArrowLeft className="w-4 h-4" /> Back to Dashboard
          </button>
          <div className="flex items-center gap-3">
            <div className="w-10 h-10 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center shadow-lg shadow-cyan-600/20">
              <BookOpen className="w-5 h-5 text-white" />
            </div>
            <div>
              <h1 className="text-2xl font-black text-slate-900 tracking-tight">Edit Course</h1>
              <p className="text-sm text-slate-500 font-medium">Update course details and assignments</p>
            </div>
          </div>
        </motion.div>

        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex items-center gap-2 mb-8 overflow-x-auto pb-2">
          {STEPS.map((s, i) => {
            const Icon = s.icon
            const isActive = step === s.id
            const isDone = step > s.id
            return (
              <React.Fragment key={s.id}>
                <div className={'flex items-center gap-2 px-3.5 py-2 rounded-xl shrink-0 text-xs font-semibold transition-all ' + (isActive ? 'bg-cyan-50 text-cyan-700 border border-cyan-200' : isDone ? 'bg-slate-100 text-slate-700' : 'bg-slate-50 text-slate-400 opacity-60')}>
                  {isDone ? <CheckCircle className="w-3.5 h-3.5 text-emerald-600" /> : <Icon className="w-3.5 h-3.5" />}
                  {s.label}
                </div>
                {i < STEPS.length - 1 && <div className={'w-6 h-px shrink-0 ' + (isDone ? 'bg-emerald-300' : 'bg-slate-200')} />}
              </React.Fragment>
            )
          })}
        </motion.div>

        <motion.div key={step} initial="hidden" animate="visible" variants={fadeUp}>
          <Card className="bg-white border border-slate-200/90 shadow-sm rounded-3xl overflow-hidden">
            <CardContent className="p-8">

              {step === 1 && (
                <div className="space-y-6">
                  <div><h2 className="text-xl font-bold text-slate-900 mb-1">Assign a Trainer</h2><p className="text-sm text-slate-500">The trainer will receive a course assignment announcement.</p></div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Trainer *</Label>
                    {loadingTrainers ? <div className="flex items-center gap-2 text-slate-400 text-sm py-3"><Loader2 className="w-4 h-4 animate-spin" /> Loading...</div> : (
                      <Select disabled={isTrainerApplied} value={trainerForm.watch('trainer_id')} onValueChange={v => trainerForm.setValue('trainer_id', v, { shouldValidate: true })}>
                        <SelectTrigger className="border-slate-200 focus:border-cyan-500 rounded-xl h-12 bg-slate-50"><SelectValue placeholder="Select a trainer..." /></SelectTrigger>
                        <SelectContent>
                          {trainers.map(t => (
                            <SelectItem key={t.id} value={t.id}>
                              <div className="flex items-center gap-2">
                                <div className="w-6 h-6 rounded-full bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white text-[10px] font-bold shrink-0">{t.full_name.charAt(0)}</div>
                                <span className="font-medium text-slate-900">{t.full_name}</span>
                                <span className="text-xs text-slate-400">· {t.qualifications || 'Certified Trainer'}</span>
                                {t.years_of_experience && <span className="text-xs text-slate-400">· {t.years_of_experience} yrs exp</span>}
                              </div>
                            </SelectItem>
                          ))}
```

```
                        </SelectContent>
                      </Select>
                    )}
                    {trainerForm.formState.errors.trainer_id && <p className="text-xs text-red-500 flex items-center gap-1"><A
lertCircle className="w-3 h-3" />{trainerForm.formState.errors.trainer_id.message}</p>}
                    {isTrainerApplied && (
                      <p className="text-xs text-amber-700 bg-amber-50 p-3 rounded-xl border border-amber-200 mt-2 flex items-
start gap-2">
                        <AlertCircle className="w-4 h-4 mt-0.5 shrink-0 text-amber-600" />
                        This course was submitted by a trainer. The assigned trainer cannot be changed.
                      </p>
                    )}
                  </div>
                  {selectedTrainer && (
                    <div className="p-4 rounded-2xl bg-cyan-50 border border-cyan-200 flex items-center gap-4">
                      <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center just
ify-center text-white font-bold text-lg shadow-md shadow-cyan-600/20">{selectedTrainer.full_name.charAt(0)}</div>
                      <div>
                        <p className="font-bold text-slate-900">{selectedTrainer.full_name}</p>
                        <p className="text-xs text-cyan-700 font-medium">{selectedTrainer.email}</p>
                        <p className="text-[11px] font-medium text-slate-600 mt-1 bg-white inline-block px-2.5 py-0.5 rounded-
full border border-slate-200">
                          {selectedTrainer.qualifications || 'Certified Trainer'}{selectedTrainer.years_of_experience ? ` •
${selectedTrainer.years_of_experience} years exp.` : ''}
                        </p>
                      </div>
                      <Badge className="ml-auto bg-emerald-100 text-emerald-700 border-emerald-200">Selected ✓</Badge>
                    </div>
                  )}
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Message to Trainer <span className="text-slate-400
font-normal">(optional)</span></Label>
                    <Textarea {...trainerForm.register('assignment_message')} placeholder="e.g. Please build this course by Oc
tober. Focus on practical exercises..." className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none" rows=
{3} />
                    <p className="text-xs text-slate-400">This message will appear in the trainer's announcement.</p>
                  </div>
                  {selectedTrainer && (
                    <div className="p-4 rounded-2xl border border-dashed border-cyan-200 bg-cyan-50/50 space-y-2">
                      <div className="flex items-center gap-2 text-cyan-700 text-xs font-bold uppercase tracking-wider"><Megap
hone className="w-3.5 h-3.5" /> Preview: Trainer announcement</div>
                      <div className="bg-white rounded-xl p-3 border border-slate-200 shadow-sm">
                        <p className="text-sm font-semibold text-slate-900">🎓 New Course Assigned to You</p>
                        <p className="text-xs text-slate-500 mt-0.5">"{detailsForm.watch('title') || 'Course Title'}" — assign
ed by {profile?.full_name ?? 'Admin'}</p>
                        {trainerForm.watch('assignment_message') && <p className="text-xs text-slate-600 mt-2 italic">"{traine
rForm.watch('assignment_message')}"</p>}
                      </div>
                    </div>
                  )}
                </div>
              )}

              {step === 2 && (
                <div className="space-y-6">
                  <div><h2 className="text-xl font-bold text-slate-900 mb-1">Course Details</h2></div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Title *</Label>
                    <Input {...detailsForm.register('title')} placeholder="e.g. Advanced Data Analytics" className="border-sla
te-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12" />
                    {detailsForm.formState.errors.title && <p className="text-xs text-red-500">{detailsForm.formState.errors.t
itle.message}</p>}
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Description *</Label>
                    <Textarea {...detailsForm.register('description')} placeholder="What will trainees learn?" className="bord
er-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none" rows={5} />
                    {detailsForm.formState.errors.description && <p className="text-xs text-red-500">{detailsForm.formState.er
rors.description.message}</p>}
                  </div>
                  <div className="grid grid-cols-2 gap-4">
                    <div className="space-y-2">
                      <Label className="text-sm font-semibold text-slate-700">Course Type *</Label>
                      <Select
                        value={isCustomType ? 'custom' : detailsForm.watch('course_type')}
                        onValueChange={v => {
                          if (v === 'custom') {
                            setIsCustomType(true)
                            detailsForm.setValue('course_type', '')
```

```
                          } else {
                            setIsCustomType(false)
                            detailsForm.setValue('course_type', v)
                          }
                        }}
                      >
                        <SelectTrigger className="border-slate-200 bg-slate-50 rounded-xl h-12"><SelectValue /></SelectTrigger
>
                        <SelectContent>
                          <SelectItem value="standard">Standard Training</SelectItem>
                          <SelectItem value="scenario">Scenario-Based Training</SelectItem>
                          <SelectItem value="technical">Technical Training</SelectItem>
                          <SelectItem value="custom">+ Add Custom Course Type...</SelectItem>
                        </SelectContent>
                      </Select>
                      {isCustomType && (
                        <Input
                          {...detailsForm.register('course_type')}
                          placeholder="Enter custom course type"
                          className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12 mt-2"
                        />
                      )}
                      {detailsForm.formState.errors.course_type && <p className="text-xs text-red-500">{detailsForm.formState.
errors.course_type.message}</p>}
                    </div>
                    <div className="space-y-2">
                      <Label className="text-sm font-semibold text-slate-700">Department</Label>
                      <Input {...detailsForm.register('department')} placeholder="e.g. Engineering" className="border-slate-20
0 focus:border-cyan-500 bg-slate-50 rounded-xl h-12" />
                    </div>
                  </div>
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Thumbnail <span className="text-slate-400 font-nor
mal">(optional)</span></Label>
                    <div onClick={() => thumbRef.current?.click()} className="flex items-center gap-4 p-4 rounded-2xl border-2
border-dashed border-slate-200 hover:border-cyan-500 cursor-pointer transition-colors bg-slate-50">
                      {thumbnailPreview ? <img src={thumbnailPreview} alt="thumb" className="w-20 h-14 object-cover rounded-x
l" /> : <div className="w-20 h-14 rounded-xl bg-slate-200 flex items-center justify-center"><Image className="w-6 h-6 text-slate-40
0" /></div>}
                      <div><p className="text-sm font-medium text-slate-700">{thumbnail ? thumbnail.name : 'Click to upload th
umbnail'}</p><p className="text-xs text-slate-400">PNG, JPG — max 5MB (16:5 ratio, e.g. 1600x500px)</p></div>
                    </div>
                    <input ref={thumbRef} type="file" accept="image/*" onChange={e => {
                      const f = e.target.files?.[0];
                      if (f && f.size < 5*1024*1024) {
                        setRawImageFile(f);
                        setIsCropperOpen(true);
                        if(thumbRef.current) thumbRef.current.value='';
                      } else if (f) toast.error('Max 5MB')
                    }} className="hidden" />
                  </div>
                </div>
              )}

              {step === 3 && (
                <div className="space-y-6">
                  <div><h2 className="text-xl font-bold text-slate-900 mb-1">Configuration</h2></div>
                  <div className="grid grid-cols-2 gap-4">
                    {[
                      { label: 'Duration (hours)', field: 'duration_hours', type: 'number', placeholder: 'e.g. 40' },
                      { label: 'Passing Score (%)', field: 'passing_score', type: 'number', placeholder: '60' },
                      { label: 'Start Date', field: 'start_date', type: 'date', placeholder: '' },
                      { label: 'End Date', field: 'end_date', type: 'date', placeholder: '' },
                      { label: 'Max Trainees', field: 'max_trainees', type: 'number', placeholder: 'e.g. 50' },
                    ].map(({ label, field, type, placeholder }) => (
                      <div key={field} className="space-y-2">
                        <Label className="text-sm font-semibold text-slate-700">{label}</Label>
                        <Input {...settingsForm.register(field as any)} type={type} placeholder={placeholder} className="borde
r-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl h-12" />
                      </div>
                    ))}
                    <div className="space-y-2">
                      <Label className="text-sm font-semibold text-slate-700">Delivery Mode</Label>
                      <Select value={settingsForm.watch('delivery_mode')} onValueChange={v => settingsForm.setValue('delivery_
mode', v as any)}>
                        <SelectTrigger className="border-slate-200 bg-slate-50 rounded-xl h-12"><SelectValue /></SelectTrigger
>
                        <SelectContent>
                          <SelectItem value="recorded">Recorded</SelectItem>
```

```
                          <SelectItem value="live">Live</SelectItem>
                          <SelectItem value="hybrid">Hybrid</SelectItem>
                        </SelectContent>
                      </Select>
                    </div>
                  </div>
                </div>
              )}

              {step === 4 && (
                <div className="space-y-6">
                  <div><h2 className="text-xl font-bold text-slate-900 mb-1">Learning Objectives & Skills</h2></div>
                  {(['understand', 'able_to_do', 'competencies_built'] as const).map((field, i) => (
                    <div key={field} className="space-y-2">
                      <Label className="text-sm font-semibold text-slate-700">{['Will Understand *', 'Will be Able to Do *', 'Competencies Built *'][i]}</Label>
                      <Textarea {...objectivesForm.register(field)} className="border-slate-200 focus:border-cyan-500 bg-slate-50 rounded-xl resize-none" rows={3} placeholder={['Core concepts, theories...', 'Practical skills, tasks...', 'Professional competencies...'][i]} />
                      {objectivesForm.formState.errors[field] && <p className="text-xs text-red-500">{objectivesForm.formState.errors[field]?.message}</p>}
                    </div>
                  ))}
                  <div className="space-y-2">
                    <Label className="text-sm font-semibold text-slate-700">Outcomes of Learning (Skills Developed)</Label>
                    <div className="flex flex-wrap gap-2 p-4 rounded-2xl bg-slate-50 border border-slate-200 min-h-[60px]">
                      {skills.map(skill => (
                        <button key={skill.id} type="button" onClick={() => toggleSkill(skill.id)} className={'px-3 py-1.5 rounded-full text-xs font-semibold border transition-all ' + (selectedSkills.includes(skill.id) ? 'bg-cyan-100 text-cyan-800 border-cyan-300' : 'bg-white text-slate-600 border-slate-200 hover:bg-slate-100')}>{skill.name}</button>
                      ))}
                    </div>
                  </div>
                </div>
              )}

              {step === 5 && (
                <div className="space-y-6">
                  <div><h2 className="text-xl font-bold text-slate-900 mb-1">Review & Publish</h2></div>
                  <div className="p-4 rounded-2xl bg-cyan-50 border border-cyan-200">
                    <p className="text-xs font-bold text-cyan-800 uppercase tracking-wider mb-2">Trainer Assignment</p>
                    {selectedTrainer ? (
                      <div className="flex items-center gap-3">
                        <div className="w-10 h-10 rounded-xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center text-white font-bold shadow">{selectedTrainer.full_name.charAt(0)}</div>
                        <div><p className="font-bold text-slate-900">{selectedTrainer.full_name}</p><p className="text-xs text-slate-500">{selectedTrainer.email}</p></div>
                        <Badge className="ml-auto bg-emerald-100 text-emerald-700 border-emerald-200">Will be notified ✓</Badge>
                      </div>
                    ) : <p className="text-sm text-red-500">No trainer selected — go back to Step 1</p>}
                  </div>
                  <div className="p-4 rounded-2xl bg-slate-50 border border-slate-200 grid grid-cols-2 gap-3 text-sm">
                    <div><p className="text-xs text-slate-400">Title</p><p className="font-semibold text-slate-900">{detailsForm.watch('title')}</p></div>
                    <div><p className="text-xs text-slate-400">Type</p><p className="font-semibold text-slate-900 capitalize">{detailsForm.watch('course_type')}</p></div>
                    <div><p className="text-xs text-slate-400">Delivery</p><p className="font-semibold text-slate-900 capitalize">{settingsForm.watch('delivery_mode')}</p></div>
                    <div><p className="text-xs text-slate-400">Passing Score</p><p className="font-semibold text-slate-900">{settingsForm.watch('passing_score')}%</p></div>
                  </div>
                  <div className="p-4 rounded-2xl bg-gradient-to-r from-emerald-50 to-teal-50 border border-emerald-200/60">
                    <div className="flex items-start gap-3">
                      <div className="w-8 h-8 rounded-xl bg-emerald-100 flex items-center justify-center shrink-0"><Megaphone className="w-4 h-4 text-emerald-600" /></div>
                      <div>
                        <p className="text-sm font-bold text-emerald-900">What happens when you publish?</p>
                        <ul className="text-xs text-emerald-800 mt-1 space-y-0.5">
                          <li>• Course goes live immediately in the trainee catalog</li>
                          <li>• Trainer receives a big announcement: "Course assigned to you"</li>
                        </ul>
                      </div>
                    </div>
                  </div>
                  <div className="flex gap-3">
                    <Button type="button" variant="outline" onClick={() => handlePublish('draft')} disabled={saving} className="flex-1 border-slate-200 text-slate-700 hover:bg-slate-50 rounded-xl h-12 font-semibold">
                      {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : null} Save as Draft
```

```
                    </Button>
                    <Button type="button" onClick={() => handlePublish('published')} disabled={saving} className="flex-1 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-bold rounded-xl h-12 shadow-lg shadow-cyan-600/20">
                      {saving ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Megaphone className="w-4 h-4 mr-2" />} Publish & Assign
                    </Button>
                  </div>
                </div>
              )}
            </CardContent>
          </Card>
        </motion.div>

        {step < 5 && (
          <motion.div initial="hidden" animate="visible" variants={fadeUp} className="flex justify-between mt-6">
            <Button type="button" variant="outline" onClick={handleBack} disabled={step === 1} className="border-slate-200 text-slate-700 hover:bg-slate-50 font-medium rounded-xl px-6"><ArrowLeft className="w-4 h-4 mr-2" /> Back</Button>
            <Button type="button" onClick={handleNext} className="bg-gradient-to-r from-cyan-600 to-blue-600 hover:opacity-95 text-white font-medium rounded-xl px-6 shadow-md shadow-cyan-600/10">Next <ArrowRight className="w-4 h-4 ml-2" /></Button>
          </motion.div>
        )}
      </div>

      <ImageCropperModal
        isOpen={isCropperOpen}
        imageFile={rawImageFile}
        onClose={() => {
          setIsCropperOpen(false)
          setRawImageFile(null)
        }}
        onCropComplete={(croppedFile) => {
          setThumbnail(croppedFile)
          setThumbnailPreview(URL.createObjectURL(croppedFile))
          setIsCropperOpen(false)
          setRawImageFile(null)
        }}
      />
    </div>
  )
}

```

src/features/admin/AdminHomePageSettings.tsx TSX 82.8 KB 1,620 Lines

```tsx
import React, { useState, useEffect } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import {
  Globe,
  Sparkles,
  Megaphone,
  Radio,
  Save,
  RotateCcw,
  CheckCircle2,
  ExternalLink,
  Plus,
  Trash2,
  Edit3,
  Layers,
  Clock,
  Award,
  ChevronDown,
  ChevronUp,
  Tag,
  BookOpen,
  Calendar,
  Compass,
  UploadCloud,
  Image as ImageIcon,
  X,
  Loader2,
  Sliders,
  Eye,
  Check,
  Trophy,
  Users,
  User
} from 'lucide-react';
import { Button } from '@/components/ui/button';
import { ImageCropperModal } from '@/components/ui/ImageCropperModal';
import { supabase } from '@/lib/supabase';
import {
  useHomePageSettings,
  defaultHomePageSettings,
  HomePageSettings,
  FeaturedProgramItem,
  AnnouncementItem,
  UpcomingTrackItem,
  AchievementStoryItem,
  AchieverItem,
  defaultAchievementStories
} from '@/hooks/useHomePageSettings';
import { toast } from 'sonner';

export function AdminHomePageSettings() {
  const { settings, updateSettings, isUpdating } = useHomePageSettings();
  const [formData, setFormData] = useState<HomePageSettings>(settings);
  const [activeSectionTab, setActiveSectionTab] = useState<'programs' | 'announcements' | 'tracks' | 'achievements'>('program
s');
  const [uploadingIndex, setUploadingIndex] = useState<number | null>(null);
  const [uploadingAchieverKey, setUploadingAchieverKey] = useState<string | null>(null);

  const [isCropperOpen, setIsCropperOpen] = useState(false);
  const [rawImageFile, setRawImageFile] = useState<File | null>(null);
  const [croppingIndex, setCroppingIndex] = useState<number | null>(null);

  useEffect(() => {
    if (settings) {
      setFormData(settings);
    }
  }, [settings]);

  const handleToggle = (key: keyof HomePageSettings) => {
    setFormData((prev) => ({
      ...prev,
      [key]: !prev[key],
    }));
  };

  const handleInputChange = (key: keyof HomePageSettings, value: any) => {
```

```
    setFormData((prev) => ({
      ...prev,
      [key]: value,
    }));
  };

  // Program Card Updaters

  const handleProgramItemChange = (index: number, field: keyof FeaturedProgramItem, value: any) => {
    const updated = [...(formData.featured_programs_items || [])];
    if (updated[index]) {
      updated[index] = { ...updated[index], [field]: value };
      setFormData((prev) => ({ ...prev, featured_programs_items: updated }));
    }
  };

  const handleAddProgramCard = () => {
    const newProg: FeaturedProgramItem = {
      id: `prog-${Date.now()}`,
      title: 'New Specialized Earth Science Program',
      description: 'Master operational techniques, observational frameworks, and scientific methodologies with MoES accredite
d certification.',
      department: 'IMD / INCOIS, MoES',
      badge: 'Featured',
      duration_hours: 60,
      trainer_name: 'Lead Scientist',
      link: '/courses',
    };
    setFormData((prev) => ({
      ...prev,
      featured_programs_items: [...(prev.featured_programs_items || []), newProg],
    }));
    toast.success('Added new featured program card');
  };

  const handleDeleteProgramCard = (index: number) => {
    if ((formData.featured_programs_items || []).length <= 1) {
      toast.error('You must keep at least 1 featured program card.');
      return;
    }
    const updated = (formData.featured_programs_items || []).filter((_, i) => i !== index);
    setFormData((prev) => ({ ...prev, featured_programs_items: updated }));
    toast.info('Program card removed');
  };

  const handleThumbnailUpload = async (index: number, file: File) => {
    if (!file) return;

    if (!file.type.startsWith('image/')) {
      toast.error('Please upload an image file (PNG, JPG, WEBP, etc.)');
      return;
    }

    if (file.size > 10 * 1024 * 1024) {
      toast.error('Image size exceeds 10MB limit.');
      return;
    }

    setUploadingIndex(index);
    const prog = formData.featured_programs_items?.[index];
    const fileExt = file.name.split('.').pop() || 'png';
    const fileName = `featured_${prog?.id || index}_${Date.now()}.${fileExt}`;
    const filePath = `thumbnails/${fileName}`;

    try {
      const { error: uploadError } = await supabase.storage
        .from('Homepage')
        .upload(filePath, file, { upsert: true });

      if (uploadError) {
        console.warn('Storage upload note:', uploadError.message);
        const reader = new FileReader();
        reader.onload = (e) => {
          const dataUrl = e.target?.result as string;
          handleProgramItemChange(index, 'thumbnail_url', dataUrl);
        };
        reader.readAsDataURL(file);
        toast.info('Image loaded as local preview.');
      } else {
```

```
        const { data: urlData } = supabase.storage
          .from('Homepage')
          .getPublicUrl(filePath);

        handleProgramItemChange(index, 'thumbnail_url', urlData.publicUrl);
        toast.success('Thumbnail uploaded successfully!');
      }
    } catch (err: any) {
      console.error('Upload exception:', err);
      toast.error('Failed to upload thumbnail.');
    } finally {
      setUploadingIndex(null);
    }
  };

  // Announcements Marquee Updaters

  const handleAnnouncementItemChange = (index: number, field: keyof AnnouncementItem, value: any) => {
    const updated = [...(formData.announcements_items || [])];
    if (updated[index]) {
      updated[index] = { ...updated[index], [field]: value };
      setFormData((prev) => ({ ...prev, announcements_items: updated }));
    }
  };

  const handleAddAnnouncement = () => {
    const newItem: AnnouncementItem = {
      id: `ann-${Date.now()}`,
      tag: 'NEW NOTICE',
      tag_color: 'cyan',
      title: 'New MoES Training Protocol Announcement Live for Trainees',
      link: '/courses',
    };
    setFormData((prev) => ({
      ...prev,
      announcements_items: [...(prev.announcements_items || []), newItem],
    }));
    toast.success('Added new announcement item');
  };

  const handleDeleteAnnouncement = (index: number) => {
    if ((formData.announcements_items || []).length <= 1) {
      toast.error('You must keep at least 1 announcement item in the marquee.');
      return;
    }
    const updated = (formData.announcements_items || []).filter((_, i) => i !== index);
    setFormData((prev) => ({ ...prev, announcements_items: updated }));
    toast.info('Announcement item removed');
  };

  // Upcoming Tracks Updaters

  const handleTrackItemChange = (index: number, field: keyof UpcomingTrackItem, value: any) => {
    const updated = [...(formData.upcoming_tracks_items || [])];
    if (updated[index]) {
      updated[index] = { ...updated[index], [field]: value };
      setFormData((prev) => ({ ...prev, upcoming_tracks_items: updated }));
    }
  };

  const handleAddTrack = () => {
    const newTrack: UpcomingTrackItem = {
      id: `track-${Date.now()}`,
      title: 'Satellite Telemetry & Numerical Weather Modeling',
      desc: 'Master observational frameworks, polarimetric radar algorithms, and real-time computing pipelines.',
      date: 'Starts Next Month',
      duration: '4 Weeks',
      status: 'Pre-Registration Open',
      status_color: 'cyan',
      level: 'Advanced Track',
      format: 'Online & Lab Synchronous',
      key_skills: 'Satellite Data, Numerical Modeling, Doppler Radar',
      btn_text: 'Pre-Register Cohort',
      link: '/register',
    };
    setFormData((prev) => ({
      ...prev,
      upcoming_tracks_items: [...(prev.upcoming_tracks_items || []), newTrack],
    }));
```

```
    toast.success('Added new specialized track card');
  };

  const handleDeleteTrack = (index: number) => {
    if ((formData.upcoming_tracks_items || []).length <= 1) {
      toast.error('You must keep at least 1 specialized track card.');
      return;
    }
    const updated = (formData.upcoming_tracks_items || []).filter((_, i) => i !== index);
    setFormData((prev) => ({ ...prev, upcoming_tracks_items: updated }));
    toast.info('Track card removed');
  };

  // Achievements Updaters

  const handleStoryChange = (storyIndex: number, field: keyof AchievementStoryItem, value: any) => {
    const updated = [...(formData.achievements_stories || [])];
    if (updated[storyIndex]) {
      updated[storyIndex] = { ...updated[storyIndex], [field]: value };
      setFormData((prev) => ({ ...prev, achievements_stories: updated }));
    }
  };

  const handleAddStory = () => {
    const newStory: AchievementStoryItem = {
      id: `story-${Date.now()}`,
      bannerTitle: 'Capacity Connect Trainees Shine Bright in',
      bannerSubtitle: 'NATIONAL EARTH SCIENCES BENCHMARK 2026',
      highlightStatTitle: 'OVER',
      highlightStatNumber: '95%',
      highlightStatDescription: 'Trainees Secured Top Tier-1 Operational Deployment in Premier Institutes',
      image_url: null,
      achievers: [
        {
          id: `ach-${Date.now()}-1`,
          name: 'New Achiever',
          initials: 'NA',
          role: 'Scientific Officer',
          institute: 'MoES Institute',
          achievementBadge: 'AIR 1 (99.5%)',
          score: 'Top Performer',
        },
      ],
    };
    setFormData((prev) => ({
      ...prev,
      achievements_stories: [...(prev.achievements_stories || []), newStory],
    }));
    toast.success('Added new achievement benchmark story');
  };

  const handleDeleteStory = (storyIndex: number) => {
    if ((formData.achievements_stories || []).length <= 1) {
      toast.error('You must keep at least 1 achievement story.');
      return;
    }
    const updated = (formData.achievements_stories || []).filter((_, i) => i !== storyIndex);
    setFormData((prev) => ({ ...prev, achievements_stories: updated }));
    toast.info('Achievement story removed');
  };

  const handleAchieverChange = (
    storyIndex: number,
    achieverIndex: number,
    field: keyof AchieverItem,
    value: any
  ) => {
    const updatedStories = [...(formData.achievements_stories || [])];
    if (updatedStories[storyIndex]) {
      const updatedAchievers = [...(updatedStories[storyIndex].achievers || [])];
      if (updatedAchievers[achieverIndex]) {
        updatedAchievers[achieverIndex] = {
          ...updatedAchievers[achieverIndex],
          [field]: value,
        };
        // Auto-update initials if name changed and initials is empty or 2 chars

        if (field === 'name' && value && typeof value === 'string') {
          const parts = value.trim().split(/\s+/);
```

```
          const initials = parts.length > 1
            ? (parts[0][0] + parts[parts.length - 1][0]).toUpperCase()
            : value.slice(0, 2).toUpperCase();
          updatedAchievers[achieverIndex].initials = initials;
        }
        updatedStories[storyIndex] = {
          ...updatedStories[storyIndex],
          achievers: updatedAchievers,
        };
        setFormData((prev) => ({ ...prev, achievements_stories: updatedStories }));
      }
    }
  };

  const handleAddAchiever = (storyIndex: number) => {
    const updatedStories = [...(formData.achievements_stories || [])];
    if (updatedStories[storyIndex]) {
      const newAchiever: AchieverItem = {
        id: `ach-${Date.now()}`,
        name: 'New Trainee',
        initials: 'NT',
        role: 'Field Analyst',
        institute: 'Regional Meteorological Centre',
        achievementBadge: 'Top Tier (98%)',
        score: 'Excellence',
      };
      updatedStories[storyIndex] = {
        ...updatedStories[storyIndex],
        achievers: [...(updatedStories[storyIndex].achievers || []), newAchiever],
      };
      setFormData((prev) => ({ ...prev, achievements_stories: updatedStories }));
      toast.success('Added new achiever to story');
    }
  };

  const handleDeleteAchiever = (storyIndex: number, achieverIndex: number) => {
    const updatedStories = [...(formData.achievements_stories || [])];
    if (updatedStories[storyIndex]) {
      if ((updatedStories[storyIndex].achievers || []).length <= 1) {
        toast.error('You must keep at least 1 achiever in the story.');
        return;
      }
      const updatedAchievers = (updatedStories[storyIndex].achievers || []).filter(
        (_, i) => i !== achieverIndex
      );
      updatedStories[storyIndex] = {
        ...updatedStories[storyIndex],
        achievers: updatedAchievers,
      };
      setFormData((prev) => ({ ...prev, achievements_stories: updatedStories }));
      toast.info('Achiever removed');
    }
  };

  const handleAchieverAvatarUpload = async (storyIndex: number, achieverIndex: number, file: File) => {
    if (!file) return;

    if (!file.type.startsWith('image/')) {
      toast.error('Please upload an image file (PNG, JPG, WEBP, etc.)');
      return;
    }

    if (file.size > 10 * 1024 * 1024) {
      toast.error('Image size exceeds 10MB limit.');
      return;
    }

    const key = `${storyIndex}-${achieverIndex}`;
    setUploadingAchieverKey(key);
    const fileExt = file.name.split('.').pop() || 'png';
    const fileName = `achiever_${storyIndex}_${achieverIndex}_${Date.now()}.${fileExt}`;
    const filePath = `achievers/${fileName}`;

    try {
      const { error: uploadError } = await supabase.storage
        .from('Homepage')
        .upload(filePath, file, { upsert: true });

      if (uploadError) {
```

```
386 |         console.warn('Storage upload note:', uploadError.message);
387 |         const reader = new FileReader();
388 |         reader.onload = (e) => {
389 |           const dataUrl = e.target?.result as string;
390 |           handleAchieverChange(storyIndex, achieverIndex, 'avatar_url', dataUrl);
391 |         };
392 |         reader.readAsDataURL(file);
393 |         toast.info('Image loaded as local preview.');
394 |       } else {
395 |         const { data: urlData } = supabase.storage
396 |           .from('Homepage')
397 |           .getPublicUrl(filePath);
398 |
399 |         handleAchieverChange(storyIndex, achieverIndex, 'avatar_url', urlData.publicUrl);
400 |         toast.success('Achiever image uploaded successfully!');
401 |       }
402 |     } catch (err: any) {
403 |       console.error('Upload exception:', err);
404 |       toast.error('Failed to upload achiever image.');
405 |     } finally {
406 |       setUploadingAchieverKey(null);
407 |     }
408 |   };
409 |
410 |   // Save Settings

411 |   const handleSave = async () => {
412 |     try {
413 |       await updateSettings(formData);
414 |       toast.success('Landing page settings published successfully!');
415 |     } catch (err: any) {
416 |       console.error('Save error:', err);
417 |       toast.error(err.message || 'Failed to save settings.');
418 |     }
419 |   };
420 |
421 |   const handleReset = () => {
422 |     if (window.confirm('Reset all landing page sections to default content? Unsaved edits will be lost.')) {
423 |       setFormData(defaultHomePageSettings);
424 |       toast.info('Reset to default template settings. Click Save to publish.');
425 |     }
426 |   };
427 |
428 |   const activeCount = [
429 |     formData.featured_programs_enabled,
430 |     formData.announcements_bar_enabled,
431 |     formData.upcoming_tracks_enabled,
432 |     formData.achievements_enabled,
433 |   ].filter(Boolean).length;
434 |
435 |   return (
436 |     <div className="space-y-6">
437 |       {/* ——— Hero Customizer Header (Midnight Governance Aesthetic) ——— */}
438 |       <div className="relative p-6 sm:p-8 rounded-3xl bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white s
hadow-xl border border-cyan-500/30 overflow-hidden">
439 |         <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-
3xl pointer-events-none" />
440 |
441 |         <div className="relative z-10 flex flex-col lg:flex-row lg:items-center justify-between gap-5">
442 |           <div>
443 |             <div className="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md text-cyan-300
text-xs font-semibold mb-3 border border-white/15">
444 |               <Globe className="w-3.5 h-3.5" />
445 |               <span>Public Landing Page Customizer</span>
446 |             </div>
447 |             <h2 className="text-2xl sm:text-3xl font-black text-white tracking-tight">
448 |               Home Page Detailed Customizer
449 |             </h2>
450 |             <p className="text-slate-300 text-xs sm:text-sm mt-1 max-w-2xl leading-relaxed">
451 |               Configure, publish, and fine-tune the 4 live landing page sections: Featured Learning Programs, Announcements M
arquee Ticker, Specialized Tracks, and Achievements.
452 |             </p>
453 |           </div>
454 |
455 |           <div className="flex flex-wrap items-center gap-3">
456 |             <a
457 |               href="/"
458 |               target="_blank"
459 |               rel="noopener noreferrer"
```

```
460 |               className="inline-flex items-center gap-1.5 px-4 py-2.5 rounded-xl border border-white/20 bg-white/10 hover:bg-
white/20 text-white text-xs font-bold transition-all shadow-sm"
461 |             >
462 |               <span>Preview Live Site</span>
463 |               <ExternalLink className="w-3.5 h-3.5 text-cyan-400" />
464 |             </a>
465 |             <Button
466 |               onClick={handleSave}
467 |               disabled={isUpdating}
468 |               size="sm"
469 |               className="bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:opacity-95 text-white font-extrabold te
xt-xs h-10 px-5 rounded-xl shadow-lg shadow-cyan-500/30 cursor-pointer"
470 |             >
471 |               {isUpdating ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
472 |               {isUpdating ? 'Publishing...' : 'Save & Publish'}
473 |             </Button>
474 |           </div>
475 |         </div>
476 |
477 |         {/* Quick Summary Pill Bar */}
478 |         <div className="mt-6 pt-5 border-t border-white/10 flex flex-wrap items-center justify-between gap-4 text-xs text-sla
te-300">
479 |           <div className="flex items-center gap-3">
480 |             <span className="flex items-center gap-1.5 text-white font-bold bg-white/10 px-3 py-1 rounded-lg border border-wh
ite/10">
481 |               <Layers className="w-3.5 h-3.5 text-cyan-400" />
482 |               <span>{activeCount} of 4 Sections Active</span>
483 |             </span>
484 |             <span className="text-emerald-300 flex items-center gap-1 font-bold bg-emerald-500/20 border border-emerald-400/3
0 px-3 py-1 rounded-lg">
485 |               <CheckCircle2 className="w-3.5 h-3.5 text-emerald-400" /> Instant Live Sync
486 |             </span>
487 |           </div>
488 |
489 |           <button
490 |             onClick={handleReset}
491 |             className="text-slate-300 hover:text-white text-xs font-semibold flex items-center gap-1 hover:underline cursor-p
ointer"
492 |           >
493 |             <RotateCcw className="w-3 h-3" />
494 |             Reset to Defaults
495 |           </button>
496 |         </div>
497 |       </div>
498 |
499 |       {/* ─── Section Navigation Tabs ─── */}
500 |       <div className="flex bg-slate-100 p-1.5 rounded-2xl border border-slate-200/90 gap-1.5 overflow-x-auto shadow-2xs">
501 |         {[
502 |           { key: 'programs', label: '1. Featured Programs', icon: Sparkles, count: formData.featured_programs_items?.length |
| 3, enabled: formData.featured_programs_enabled },
503 |           { key: 'announcements', label: '2. Announcements Marquee', icon: Megaphone, count: formData.announcements_items?.le
ngth || 5, enabled: formData.announcements_bar_enabled },
504 |           { key: 'tracks', label: '3. Specialized Tracks', icon: Radio, count: formData.upcoming_tracks_items?.length || 3, e
nabled: formData.upcoming_tracks_enabled },
505 |           { key: 'achievements', label: '4. Achievements & Results', icon: Award, count: formData.achievements_stories?.lengt
h || 4, enabled: formData.achievements_enabled },
506 |         ].map((tab) => {
507 |           const isSelected = activeSectionTab === tab.key;
508 |           return (
509 |             <button
510 |               key={tab.key}
511 |               onClick={() => setActiveSectionTab(tab.key as any)}
512 |               className={`flex-1 min-w-[200px] flex items-center justify-between px-4 py-3 rounded-xl text-xs font-bold trans
ition-all cursor-pointer ${
513 |                 isSelected
514 |                   ? 'bg-white text-cyan-900 shadow-sm border border-slate-200/90'
515 |                   : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/60'
516 |               }`}
517 |             >
518 |               <div className="flex items-center gap-2">
519 |                 <tab.icon className={`w-4 h-4 ${isSelected ? 'text-cyan-600' : 'text-slate-400'}`} />
520 |                 <span>{tab.label}</span>
521 |               </div>
522 |               <span className={`px-2 py-0.5 rounded-full text-[10px] font-extrabold border ${
523 |                 tab.enabled
524 |                   ? 'bg-emerald-50 text-emerald-700 border-emerald-200'
525 |                   : 'bg-slate-200 text-slate-500 border-slate-300'
526 |               }`}>
527 |                 {tab.enabled ? 'Live' : 'Hidden'}
```

```
              </span>
            </button>
          );
        })}
      </div>

      {/* ─── TAB 1: FEATURED LEARNING PROGRAMS ─── */}
      {activeSectionTab === 'programs' && (
        <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} className="space-y-6">
          {/* Main Controls Card */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-6">
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
              <div>
                <h3 className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <div className="w-8 h-8 rounded-xl bg-amber-50 text-amber-600 flex items-center justify-center border borde
r-amber-200">
                    <Sparkles className="w-4 h-4" />
                  </div>
                  <span>Section 1: Featured Learning Programs</span>
                </h3>
                <p className="text-xs text-slate-500 mt-1">Configure the top 3 course cards displayed prominently near the to
p of the landing page.</p>
              </div>

              {/* Publish Toggle */}
              <div className="flex items-center gap-3 p-2 bg-slate-50 rounded-2xl border border-slate-200">
                <span className="text-xs font-bold text-slate-700">
                  {formData.featured_programs_enabled ? 'Section is Live' : 'Section is Hidden'}
                </span>
                <button
                  type="button"
                  onClick={() => handleToggle('featured_programs_enabled')}
                  className={`relative inline-flex h-6 w-11 shrink-0 cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out ${
                    formData.featured_programs_enabled ? 'bg-cyan-600' : 'bg-slate-300'
                  }`}
                >
                  <span className={`pointer-events-none inline-block h-5 w-5 transform rounded-full bg-white shadow-md transi
tion duration-200 ease-in-out ${
                    formData.featured_programs_enabled ? 'translate-x-5' : 'translate-x-0'
                  }`} />
                </button>
              </div>
            </div>

            <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Eyebrow Tag Pill</label>
                <input
                  type="text"
                  value={formData.featured_programs_tag}
                  onChange={(e) => handleInputChange('featured_programs_tag', e.target.value)}
                  placeholder="Top Certified Tracks"
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Main Section Heading</label>
                <input
                  type="text"
                  value={formData.featured_programs_title}
                  onChange={(e) => handleInputChange('featured_programs_title', e.target.value)}
                  placeholder="Featured Learning Programs"
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Header Button Label</label>
                <input
                  type="text"
                  value={formData.featured_programs_btn_text}
                  onChange={(e) => handleInputChange('featured_programs_btn_text', e.target.value)}
                  placeholder="Explore All Courses"
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
```

```
                />
              </div>

              <div className="sm:col-span-2 lg:col-span-3">
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Section Description / Subtitle</label>
                <textarea
                  rows={2}
                  value={formData.featured_programs_subtitle}
                  onChange={(e) => handleInputChange('featured_programs_subtitle', e.target.value)}
                  placeholder="Explore government-certified, high-impact earth science training tracks..."
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs placeholder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all resize-none"
                />
              </div>
            </div>
          </div>

          {/* Featured Cards Customizer */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-5">
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
              <div>
                <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                  <BookOpen className="w-4 h-4 text-cyan-600" />
                  <span>Featured Program Cards ({formData.featured_programs_items?.length || 0} Cards)</span>
                </h4>
                <p className="text-xs text-slate-500 mt-0.5">
                  Customize course titles, images, descriptions, and metadata displayed in the card grid.
                </p>
              </div>

              <Button
                onClick={handleAddProgramCard}
                size="sm"
                className="bg-cyan-50 hover:bg-cyan-100 border border-cyan-200 text-cyan-800 text-xs font-bold rounded-xl h-9 px-3.5 flex items-center gap-1.5 cursor-pointer shadow-xs"
              >
                <Plus className="w-3.5 h-3.5" />
                <span>Add Program Card</span>
              </Button>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-5">
              {(formData.featured_programs_items || []).map((prog, idx) => (
                <div key={prog.id || idx} className="p-5 rounded-2xl bg-slate-50/70 hover:bg-white border border-slate-200/90 hover:border-cyan-300 hover:shadow-md transition-all space-y-3.5 relative group">
                  <div className="flex items-center justify-between pb-2 border-b border-slate-200/80">
                    <div className="flex items-center gap-2">
                      <span className="text-xs font-bold text-cyan-700 bg-cyan-50 px-2 py-0.5 rounded-md border border-cyan-200">
                        Card #{idx + 1}
                      </span>
                      <input
                        type="text"
                        value={prog.badge}
                        onChange={(e) => handleProgramItemChange(idx, 'badge', e.target.value)}
                        placeholder="Badge (e.g. Top Rated)"
                        className="px-2.5 py-0.5 text-[10px] font-extrabold rounded-full bg-amber-50 border border-amber-300 text-amber-800 text-center w-28 focus:outline-none focus:bg-white"
                      />
                    </div>

                    <button
                      onClick={() => handleDeleteProgramCard(idx)}
                      title="Delete card"
                      className="text-slate-400 hover:text-rose-600 p-1.5 rounded-lg hover:bg-rose-50 transition-colors cursor-pointer"
                    >
                      <Trash2 className="w-3.5 h-3.5" />
                    </button>
                  </div>

                  {/* Thumbnail Image Picker & Preview */}
                  <div className="space-y-1.5">
                    <div className="flex items-center justify-between">
                      <label className="block text-[11px] font-bold text-slate-700">Card Photo <span className="font-normal text-slate-400">(800x400, 2:1 ratio)</span></label>
                      {prog.thumbnail_url && (
                        <span className="text-[10px] font-bold text-emerald-600 bg-emerald-50 px-1.5 py-0.5 rounded border bo
```

```
rder-emerald-200">Uploaded</span>
671 |                       )}
672 |                     </div>
673 |                     {prog.thumbnail_url ? (
674 |                       <div className="relative h-32 rounded-xl overflow-hidden border border-slate-200 bg-slate-900 group/thu
mb shadow-xs">
675 |                         <img
676 |                           src={prog.thumbnail_url}
677 |                           alt={prog.title}
678 |                           className="w-full h-full object-cover"
679 |                         />
680 |                         <div className="absolute inset-0 bg-slate-950/60 opacity-0 group-hover/thumb:opacity-100 transition-o
pacity flex items-center justify-center gap-2">
681 |                           <label className="cursor-pointer px-3 py-1.5 rounded-xl bg-cyan-600 hover:bg-cyan-500 text-white te
xt-xs font-bold flex items-center gap-1 shadow-md">
682 |                             <UploadCloud className="w-3.5 h-3.5" />
683 |                             <span>Change</span>
684 |                             <input
685 |                               type="file"
686 |                               accept="image/*"
687 |                               className="hidden"
688 |                               onChange={(e) => {
689 |                                 if (e.target.files?.[0]) {
690 |                                   setRawImageFile(e.target.files[0]);
691 |                                   setCroppingIndex(idx);
692 |                                   setIsCropperOpen(true);
693 |                                   e.target.value = '';
694 |                                 }
695 |                               }}
696 |                             />
697 |                           </label>
698 |                           <button
699 |                             type="button"
700 |                             onClick={() => handleProgramItemChange(idx, 'thumbnail_url', null)}
701 |                             className="px-3 py-1.5 rounded-xl bg-rose-600 hover:bg-rose-500 text-white text-xs font-bold flex
items-center gap-1 shadow-md cursor-pointer"
702 |                           >
703 |                             <X className="w-3.5 h-3.5" />
704 |                             <span>Remove</span>
705 |                           </button>
706 |                         </div>
707 |                       </div>
708 |                     ) : (
709 |                       <label className="border-2 border-dashed border-slate-200 hover:border-cyan-400 rounded-xl p-4 flex fle
x-col items-center justify-center gap-1.5 bg-white hover:bg-cyan-50/30 cursor-pointer transition-all">
710 |                         {uploadingIndex === idx ? (
711 |                           <div className="flex items-center gap-2 text-cyan-600 text-xs py-2">
712 |                             <Loader2 className="w-4 h-4 animate-spin" />
713 |                             <span>Uploading image...</span>
714 |                           </div>
715 |                         ) : (
716 |                           <>
717 |                             <div className="w-8 h-8 rounded-xl bg-cyan-50 border border-cyan-200 text-cyan-600 flex items-cen
ter justify-center">
718 |                               <ImageIcon className="w-4 h-4" />
719 |                             </div>
720 |                             <span className="text-xs font-bold text-slate-700">Upload Card Photo</span>
721 |                             <span className="text-[10px] text-slate-400">PNG, JPG, WEBP</span>
722 |                             <input
723 |                               type="file"
724 |                               accept="image/*"
725 |                               className="hidden"
726 |                               onChange={(e) => {
727 |                                 if (e.target.files?.[0]) {
728 |                                   setRawImageFile(e.target.files[0]);
729 |                                   setCroppingIndex(idx);
730 |                                   setIsCropperOpen(true);
731 |                                   e.target.value = '';
732 |                                 }
733 |                               }}
734 |                             />
735 |                           </>
736 |                         )}
737 |                       </label>
738 |                     )}
739 |                   </div>
740 |
741 |                   <div>
742 |                     <label className="block text-[11px] font-bold text-slate-700 mb-1">Course Title</label>
```

```
                    <input
                      type="text"
                      value={prog.title}
                      onChange={(e) => handleProgramItemChange(idx, 'title', e.target.value)}
                      placeholder="e.g. Ocean Observation and Marine Data Analysis"
                      className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 focus:ou
tline-none focus:border-cyan-500 shadow-2xs font-semibold"
                    />
                  </div>

                  <div>
                    <label className="block text-[11px] font-bold text-slate-700 mb-1">Description</label>
                    <textarea
                      rows={2}
                      value={prog.description}
                      onChange={(e) => handleProgramItemChange(idx, 'description', e.target.value)}
                      placeholder="Brief overview of the program curriculum..."
                      className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-800 focus:ou
tline-none focus:border-cyan-500 resize-none shadow-2xs leading-relaxed"
                    />
                  </div>

                  <div className="grid grid-cols-2 gap-2.5">
                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Department / Org</label>
                      <input
                        type="text"
                        value={prog.department}
                        onChange={(e) => handleProgramItemChange(idx, 'department', e.target.value)}
                        placeholder="e.g. INCOIS, MoES"
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 fo
cus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Duration (Hours)</label>
                      <input
                        type="number"
                        value={prog.duration_hours}
                        onChange={(e) => handleProgramItemChange(idx, 'duration_hours', parseInt(e.target.value) || 60)}
                        placeholder="e.g. 80"
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 fo
cus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                  </div>

                  <div className="grid grid-cols-2 gap-2.5">
                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Trainer / Faculty</label>
                      <input
                        type="text"
                        value={prog.trainer_name}
                        onChange={(e) => handleProgramItemChange(idx, 'trainer_name', e.target.value)}
                        placeholder="e.g. Dr. V. Ramakrishnan"
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 fo
cus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Destination Link</label>
                      <input
                        type="text"
                        value={prog.link || '/courses'}
                        onChange={(e) => handleProgramItemChange(idx, 'link', e.target.value)}
                        placeholder="/courses"
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-cyan-700 fon
t-mono focus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                  </div>
                </motion.div>
              ))}
            </div>
          </div>
        </motion.div>
      )}

      {/* ─── TAB 2: ANNOUNCEMENTS MARQUEE BAR ─── */}
```

```
816 |       {activeSectionTab === 'announcements' && (
817 |         <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} className="space-y-6">
818 |           <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-6">
819 |             <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
820 |               <div>
821 |                 <h3 className="text-base font-bold text-slate-900 flex items-center gap-2">
822 |                   <div className="w-8 h-8 rounded-xl bg-cyan-50 text-cyan-600 flex items-center justify-center border border-
cyan-200">
823 |                     <Megaphone className="w-4 h-4" />
824 |                   </div>
825 |                   <span>Section 2: Horizontal Announcement Bar Controls</span>
826 |                 </h3>
827 |                 <p className="text-xs text-slate-500 mt-1">Continuous infinite live announcements ticker situated between Pla
tform Stats and Stakeholder Workspaces.</p>
828 |               </div>
829 |
830 |               {/* Publish Toggle */}
831 |               <div className="flex items-center gap-3 p-2 bg-slate-50 rounded-2xl border border-slate-200">
832 |                 <span className="text-xs font-bold text-slate-700">
833 |                   {formData.announcements_bar_enabled ? 'Marquee is Live' : 'Marquee is Hidden'}
834 |                 </span>
835 |                 <button
836 |                   type="button"
837 |                   onClick={() => handleToggle('announcements_bar_enabled')}
838 |                   className={`relative inline-flex h-6 w-11 shrink-0 cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out ${
839 |                     formData.announcements_bar_enabled ? 'bg-cyan-600' : 'bg-slate-300'
840 |                   }`}
841 |                 >
842 |                   <span className={`pointer-events-none inline-block h-5 w-5 transform rounded-full bg-white shadow-md transi
tion duration-200 ease-in-out ${
843 |                     formData.announcements_bar_enabled ? 'translate-x-5' : 'translate-x-0'
844 |                   }`} />
845 |                 </button>
846 |               </div>
847 |             </div>
848 |
849 |             <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
850 |               <div>
851 |                 <label className="block text-xs font-bold text-slate-700 mb-1.5">Left Fixed Pill Label</label>
852 |                 <input
853 |                   type="text"
854 |                   value={formData.announcements_bar_label}
855 |                   onChange={(e) => handleInputChange('announcements_bar_label', e.target.value)}
856 |                   placeholder="Announcements"
857 |                   className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
858 |                 />
859 |               </div>
860 |
861 |               <div>
862 |                 <div className="flex items-center justify-between mb-1.5">
863 |                   <label className="block text-xs font-bold text-slate-700">Scrolling Flow Speed</label>
864 |                   <span className="text-[11px] font-mono text-cyan-800 font-bold bg-cyan-50 px-2.5 py-0.5 rounded-md border b
order-cyan-200">
865 |                     {formData.announcements_bar_speed || 24}s / cycle
866 |                   </span>
867 |                 </div>
868 |                 <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
869 |                   <select
870 |                     value={[35, 65, 90].includes(formData.announcements_bar_speed) ? formData.announcements_bar_speed : 'cust
om'}
871 |                     onChange={(e) => {
872 |                       if (e.target.value !== 'custom') {
873 |                         handleInputChange('announcements_bar_speed', parseInt(e.target.value) || 65);
874 |                       }
875 |                     }}
876 |                     className="w-full px-3 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs focu
s:outline-none focus:bg-white focus:border-cyan-500 font-medium"
877 |                   >
878 |                     <option value={35}>Brisk (35s cycle)</option>
879 |                     <option value={65}>Comfortable / Slow (65s cycle - Recommended)</option>
880 |                     <option value={90}>Very Slow / Relaxed (90s cycle)</option>
881 |                     <option value="custom">Custom (Enter Seconds)</option>
882 |                   </select>
883 |
884 |                   <div className="relative flex items-center">
885 |                     <input
886 |                       type="number"
```

```
                      min={4}
                      max={300}
                      value={formData.announcements_bar_speed || 65}
                      onChange={(e) => handleInputChange('announcements_bar_speed', Math.max(1, parseInt(e.target.value) || 65))}
                      placeholder="Custom seconds..."
                      className="w-full pl-3 pr-12 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs font-mono focus:outline-none focus:bg-white focus:border-cyan-500"
                    />
                    <span className="absolute right-3 text-xs text-slate-400 pointer-events-none font-bold">sec</span>
                  </div>
                </div>
                <p className="text-[11px] text-slate-400 mt-1.5">Lower seconds = faster scroll, higher seconds = slower, more readable cycle.</p>
              </div>
            </div>
          </div>

          {/* Live Items Manager */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-4">
            <div className="flex items-center justify-between pb-3 border-b border-slate-100">
              <div>
                <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                  <Tag className="w-4 h-4 text-cyan-600" />
                  <span>Custom Live Announcement Items ({(formData.announcements_items || []).length})</span>
                </h4>
                <p className="text-xs text-slate-500 mt-0.5">Manage the live messages that cycle continuously in the ticker.</p>
              </div>

              <Button
                onClick={handleAddAnnouncement}
                size="sm"
                className="bg-cyan-50 hover:bg-cyan-100 text-cyan-800 border border-cyan-200 text-xs font-bold rounded-xl h-9 px-3.5 cursor-pointer shadow-xs"
              >
                <Plus className="w-3.5 h-3.5 mr-1" />
                Add Item
              </Button>
            </div>

            <div className="space-y-3">
              {(formData.announcements_items || []).map((item, idx) => (
                <div key={item.id || idx} className="p-4 rounded-2xl bg-slate-50/80 hover:bg-white border border-slate-200/90 hover:border-cyan-300 hover:shadow-xs transition-all flex flex-col sm:flex-row items-start sm:items-center gap-3">
                  <span className="text-xs font-mono font-bold text-slate-400 shrink-0">#{idx + 1}</span>

                  <div className="w-36 shrink-0">
                    <input
                      type="text"
                      value={item.tag}
                      onChange={(e) => handleAnnouncementItemChange(idx, 'tag', e.target.value)}
                      placeholder="Tag (e.g. MOES LIVE)"
                      className="w-full px-3 py-2 text-[11px] font-extrabold rounded-xl bg-white border border-slate-200 text-cyan-800 uppercase focus:outline-none focus:border-cyan-500 text-center shadow-2xs"
                    />
                  </div>

                  <div className="w-32 shrink-0">
                    <select
                      value={item.tag_color || 'cyan'}
                      onChange={(e) => handleAnnouncementItemChange(idx, 'tag_color', e.target.value)}
                      className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-800 font-semibold focus:outline-none focus:border-cyan-500 shadow-2xs"
                    >
                      <option value="cyan">Cyan Accent</option>
                      <option value="amber">Amber Accent</option>
                      <option value="emerald">Emerald Accent</option>
                      <option value="sky">Sky Accent</option>
                      <option value="purple">Purple Accent</option>
                    </select>
                  </div>

                  <div className="flex-1 w-full">
                    <input
                      type="text"
                      value={item.title}
                      onChange={(e) => handleAnnouncementItemChange(idx, 'title', e.target.value)}
                      placeholder="Announcement headline text..."
```

```
                      className="w-full px-3.5 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 focus:outline-none focus:border-cyan-500 shadow-2xs font-medium"
                    />
                  </div>

                  <div className="w-32 shrink-0">
                    <input
                      type="text"
                      value={item.link || '/courses'}
                      onChange={(e) => handleAnnouncementItemChange(idx, 'link', e.target.value)}
                      placeholder="Link (/courses)"
                      className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-cyan-700 font-mono focus:outline-none focus:border-cyan-500 shadow-2xs"
                    />
                  </div>

                  <button
                    type="button"
                    onClick={() => handleDeleteAnnouncement(idx)}
                    className="p-2 rounded-xl text-slate-400 hover:text-rose-600 hover:bg-rose-50 transition-all shrink-0 cursor-pointer"
                    title="Delete item"
                  >
                    <Trash2 className="w-4 h-4" />
                  </button>
                </div>
              ))}
            </div>
          </div>
        </motion.div>
      )}

      {/* ─── TAB 3: SPECIALIZED EARTH SCIENCES TRACKS ─── */}
      {activeSectionTab === 'tracks' && (
        <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} className="space-y-6">
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-6">
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
              <div>
                <h3 className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <div className="w-8 h-8 rounded-xl bg-sky-50 text-sky-600 flex items-center justify-center border border-sky-200">
                    <Radio className="w-4 h-4" />
                  </div>
                  <span>Section 3: Specialized Earth Sciences Tracks</span>
                </h3>
                <p className="text-xs text-slate-500 mt-1">Upcoming announcements, pre-registration cohorts, and masterclasses preview cards.</p>
              </div>

              {/* Publish Toggle */}
              <div className="flex items-center gap-3 p-2 bg-slate-50 rounded-2xl border border-slate-200">
                <span className="text-xs font-bold text-slate-700">
                  {formData.upcoming_tracks_enabled ? 'Section is Live' : 'Section is Hidden'}
                </span>
                <button
                  type="button"
                  onClick={() => handleToggle('upcoming_tracks_enabled')}
                  className={`relative inline-flex h-6 w-11 shrink-0 cursor-pointer rounded-full border-2 border-transparent transition-colors duration-200 ease-in-out ${
                    formData.upcoming_tracks_enabled ? 'bg-cyan-600' : 'bg-slate-300'
                  }`}
                >
                  <span className={`pointer-events-none inline-block h-5 w-5 transform rounded-full bg-white shadow-md transition duration-200 ease-in-out ${
                    formData.upcoming_tracks_enabled ? 'translate-x-5' : 'translate-x-0'
                  }`} />
                </button>
              </div>
            </div>

            <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Eyebrow Tag Pill</label>
                <input
                  type="text"
                  value={formData.upcoming_tracks_tag}
                  onChange={(e) => handleInputChange('upcoming_tracks_tag', e.target.value)}
                  placeholder="Upcoming Announcements"
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
```

```
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Main Section Heading</label>
                <input
                  type="text"
                  value={formData.upcoming_tracks_title}
                  onChange={(e) => handleInputChange('upcoming_tracks_title', e.target.value)}
                  placeholder="Specialized Earth Sciences Tracks."
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Catalog Button Text</label>
                <input
                  type="text"
                  value={formData.upcoming_tracks_btn_text}
                  onChange={(e) => handleInputChange('upcoming_tracks_btn_text', e.target.value)}
                  placeholder="View Complete Catalog"
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>

              <div className="sm:col-span-2 lg:col-span-3">
                <label className="block text-xs font-bold text-slate-700 mb-1.5">Section Subtitle / Description</label>
                <input
                  type="text"
                  value={formData.upcoming_tracks_subtitle}
                  onChange={(e) => handleInputChange('upcoming_tracks_subtitle', e.target.value)}
                  placeholder="Pre-register for next-generation cohorts and masterclasses."
                  className="w-full px-3.5 py-2.5 rounded-xl bg-slate-50 border border-slate-200 text-slate-900 text-xs place
holder:text-slate-400 focus:outline-none focus:bg-white focus:border-cyan-500 focus:ring-2 focus:ring-cyan-500/20 transition-all"
                />
              </div>
            </div>
          </div>

          {/* Individual Track Cards Editor */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-4">
            <div className="flex items-center justify-between pb-3 border-b border-slate-100">
              <div>
                <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                  <Compass className="w-4 h-4 text-sky-600" />
                  <span>Upcoming Track Cards ({(formData.upcoming_tracks_items || []).length})</span>
                </h4>
                <p className="text-xs text-slate-500 mt-0.5">Edit dates, skill tags, format, and pre-registration links for e
ach upcoming cohort card.</p>
              </div>

              <Button
                onClick={handleAddTrack}
                size="sm"
                className="bg-cyan-50 hover:bg-cyan-100 text-cyan-800 border border-cyan-200 text-xs font-bold rounded-xl h-9
px-3.5 cursor-pointer shadow-xs"
              >
                <Plus className="w-3.5 h-3.5 mr-1" />
                Add Track Card
              </Button>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-3 gap-5">
              {(formData.upcoming_tracks_items || []).map((track, idx) => (
                <div key={track.id || idx} className="p-5 rounded-2xl bg-slate-50/70 hover:bg-white border border-slate-200/9
0 hover:border-cyan-300 hover:shadow-md transition-all space-y-3.5 flex flex-col justify-between">
                  <div className="space-y-3">
                    <div className="flex items-center justify-between pb-2 border-b border-slate-200/80">
                      <span className="text-xs font-bold text-sky-700 bg-sky-50 px-2.5 py-0.5 rounded-md border border-sky-20
0">

                        Track #{idx + 1}
                      </span>
                      <button
                        type="button"
                        onClick={() => handleDeleteTrack(idx)}
                        className="text-slate-400 hover:text-rose-600 p-1 rounded-lg hover:bg-rose-50 transition-colors curso
```

```
r-pointer"
                        title="Delete track"
                      >
                        <Trash2 className="w-3.5 h-3.5" />
                      </button>
                    </div>

                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Track Title</label>
                      <input
                        type="text"
                        value={track.title}
                        onChange={(e) => handleTrackItemChange(idx, 'title', e.target.value)}
                        className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 font-b
old focus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>

                    <div className="grid grid-cols-2 gap-2">
                      <div>
                        <label className="block text-[11px] font-bold text-slate-700 mb-1">Status Pill</label>
                        <input
                          type="text"
                          value={track.status}
                          onChange={(e) => handleTrackItemChange(idx, 'status', e.target.value)}
                          placeholder="Pre-Registration Open"
                          className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900
focus:outline-none focus:border-cyan-500 shadow-2xs font-medium"
                        />
                      </div>
                      <div>
                        <label className="block text-[11px] font-bold text-slate-700 mb-1">Level Tag</label>
                        <input
                          type="text"
                          value={track.level}
                          onChange={(e) => handleTrackItemChange(idx, 'level', e.target.value)}
                          placeholder="Advanced Specialist"
                          className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900
focus:outline-none focus:border-cyan-500 shadow-2xs font-medium"
                        />
                      </div>
                    </div>

                    <div className="grid grid-cols-2 gap-2">
                      <div>
                        <label className="block text-[11px] font-bold text-slate-700 mb-1">Start Date</label>
                        <input
                          type="text"
                          value={track.date}
                          onChange={(e) => handleTrackItemChange(idx, 'date', e.target.value)}
                          placeholder="Starts Oct 15, 2026"
                          className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900
focus:outline-none focus:border-cyan-500 shadow-2xs"
                        />
                      </div>
                      <div>
                        <label className="block text-[11px] font-bold text-slate-700 mb-1">Duration</label>
                        <input
                          type="text"
                          value={track.duration}
                          onChange={(e) => handleTrackItemChange(idx, 'duration', e.target.value)}
                          placeholder="4 Weeks"
                          className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900
focus:outline-none focus:border-cyan-500 shadow-2xs"
                        />
                      </div>
                    </div>

                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Description</label>
                      <textarea
                        rows={2}
                        value={track.desc}
                        onChange={(e) => handleTrackItemChange(idx, 'desc', e.target.value)}
                        className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-800 focus:
outline-none focus:border-cyan-500 resize-none shadow-2xs leading-relaxed"
                      />
                    </div>

```

```
                    <div>
                      <label className="block text-[11px] font-bold text-slate-700 mb-1">Key Skills (comma-separated)</label>
                      <input
                        type="text"
                        value={track.key_skills}
                        onChange={(e) => handleTrackItemChange(idx, 'key_skills', e.target.value)}
                        placeholder="Polarimetric Refl., Mesocyclone Detection"
                        className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 focus:
outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                  </div>

                  <div className="pt-3 border-t border-slate-200/80">
                    <label className="block text-[11px] font-bold text-slate-700 mb-1">Button Text & Link</label>
                    <div className="grid grid-cols-2 gap-2">
                      <input
                        type="text"
                        value={track.btn_text || 'Pre-Register Cohort'}
                        onChange={(e) => handleTrackItemChange(idx, 'btn_text', e.target.value)}
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 fo
cus:outline-none focus:border-cyan-500 shadow-2xs font-semibold"
                      />
                      <input
                        type="text"
                        value={track.link || '/register'}
                        onChange={(e) => handleTrackItemChange(idx, 'link', e.target.value)}
                        className="w-full px-2.5 py-1.5 text-xs rounded-xl bg-white border border-slate-200 text-cyan-700 fon
t-mono focus:outline-none focus:border-cyan-500 shadow-2xs"
                      />
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </div>
        </motion.div>
      )}

      {/* ─── TAB 4: ACHIEVEMENTS & RESULTS ─── */}
      {activeSectionTab === 'achievements' && (
        <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} className="space-y-6">
          {/* Main Controls Card */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-6">
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
              <div>
                <h3 className="text-base font-bold text-slate-900 flex items-center gap-2">
                  <div className="w-8 h-8 rounded-xl bg-orange-50 text-orange-600 flex items-center justify-center border bor
der-orange-200">
                    <Award className="w-4 h-4" />
                  </div>
                  <span>Section 4: Proven Results &amp;amp; Achievements</span>
                </h3>
                <p className="text-xs text-slate-500 mt-1">
                  Highlight verified national rankers, benchmark test achievements, and MoES deployment impact stories.
                </p>
              </div>

              {/* Publish Toggle */}
              <div className="flex items-center gap-3 p-2 bg-slate-50 rounded-2xl border border-slate-200">
                <span className="text-xs font-bold text-slate-700">
                  {formData.achievements_enabled ? 'Section is Live' : 'Section is Hidden'}
                </span>
                <button
                  type="button"
                  onClick={() => handleToggle('achievements_enabled')}
                  className={`relative inline-flex h-6 w-11 shrink-0 cursor-pointer rounded-full border-2 border-transparent
transition-colors duration-200 ease-in-out focus:outline-none ${
                    formData.achievements_enabled ? 'bg-cyan-600' : 'bg-slate-300'
                  }`}
                >
                  <span
                    className={`pointer-events-none inline-block h-5 w-5 transform rounded-full bg-white shadow-lg ring-0 tra
nsition duration-200 ease-in-out ${
                      formData.achievements_enabled ? 'translate-x-5' : 'translate-x-0'
                    }`}
                  />
                </button>
              </div>
```

```
            </div>

            {/* General Section Metadata */}
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1">Eyebrow Tag Pill</label>
                <input
                  type="text"
                  value={formData.achievements_tag || ''}
                  onChange={(e) => handleInputChange('achievements_tag', e.target.value)}
                  placeholder="Proven National Impact"
                  className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 border border-slate-200 text-slate-900 focu
s:outline-none focus:border-cyan-500 shadow-2xs font-semibold"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1">Main Section Heading</label>
                <input
                  type="text"
                  value={formData.achievements_title || ''}
                  onChange={(e) => handleInputChange('achievements_title', e.target.value)}
                  placeholder="Inspired trainees. Inspired results"
                  className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 border border-slate-200 text-slate-900 focu
s:outline-none focus:border-cyan-500 shadow-2xs font-semibold"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1">MoES Badge Title</label>
                <input
                  type="text"
                  value={formData.achievements_moes_badge || ''}
                  onChange={(e) => handleInputChange('achievements_moes_badge', e.target.value)}
                  placeholder="100% Verified Operational Skill Credentials"
                  className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 border border-slate-200 text-slate-900 focu
s:outline-none focus:border-cyan-500 shadow-2xs"
                />
              </div>

              <div>
                <label className="block text-xs font-bold text-slate-700 mb-1">MoES Badge Description</label>
                <input
                  type="text"
                  value={formData.achievements_moes_desc || ''}
                  onChange={(e) => handleInputChange('achievements_moes_desc', e.target.value)}
                  placeholder="Direct alignment with WMO, IMD &amp; INCOIS forecast protocols."
                  className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 border border-slate-200 text-slate-900 focu
s:outline-none focus:border-cyan-500 shadow-2xs"
                />
              </div>
            </div>
          </div>

          {/* Achievement Stories List */}
          <div className="p-6 sm:p-7 rounded-3xl bg-white border border-slate-200/90 shadow-sm space-y-6">
            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-4 border-b border-slate-100">
              <div>
                <h4 className="text-sm font-bold text-slate-900 flex items-center gap-2">
                  <Trophy className="w-4 h-4 text-orange-600" />
                  <span>Benchmark Stories &amp;amp; Achiever Cohorts ({formData.achievements_stories?.length || 0})</span>
                </h4>
                <p className="text-xs text-slate-500 mt-0.5">
                  Each story appears in the interactive carousel showcase with its headline, highlight statistic, and achieve
r cards.
                </p>
              </div>

              <Button
                type="button"
                onClick={handleAddStory}
                size="sm"
                className="bg-orange-600 hover:bg-orange-700 text-white font-bold text-xs h-9 px-4 rounded-xl cursor-pointer"
              >
                <Plus className="w-3.5 h-3.5 mr-1.5" />
                Add Benchmark Story
              </Button>
            </div>

```

```
1320 |             <div className="space-y-6">
1321 |               {(formData.achievements_stories || []).map((story, sIdx) => (
1322 |                 <div
1323 |                   key={story.id || sIdx}
1324 |                   className="p-5 rounded-2xl bg-slate-50/80 border border-slate-200 space-y-5"
1325 |                 >
1326 |                   {/* Story Header */}
1327 |                   <div className="flex items-center justify-between gap-4 pb-3 border-b border-slate-200/80">
1328 |                     <div className="flex items-center gap-2">
1329 |                       <span className="w-6 h-6 rounded-lg bg-orange-100 text-orange-700 font-black text-xs flex items-center
justify-center">
1330 |                         {sIdx + 1}
1331 |                       </span>
1332 |                       <span className="text-xs font-bold text-slate-800">
1333 |                         {story.bannerSubtitle || 'Benchmark Story'}
1334 |                       </span>
1335 |                     </div>
1336 |
1337 |                     <button
1338 |                       type="button"
1339 |                       onClick={() => handleDeleteStory(sIdx)}
1340 |                       className="text-red-500 hover:text-red-700 text-xs font-semibold flex items-center gap-1 cursor-pointe
r"
1341 |                     >
1342 |                       <Trash2 className="w-3.5 h-3.5" />
1343 |                       Remove Story
1344 |                     </button>
1345 |                   </div>
1346 |
1347 |                   {/* Story Headline & Benchmark Title */}
1348 |                   <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
1349 |                     <div>
1350 |                       <label className="block text-[11px] font-bold text-slate-700 mb-1">Banner Pre-Title</label>
1351 |                       <input
1352 |                         type="text"
1353 |                         value={story.bannerTitle}
1354 |                         onChange={(e) => handleStoryChange(sIdx, 'bannerTitle', e.target.value)}
1355 |                         placeholder="Capacity Connect Trainees Shine Bright in"
1356 |                         className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 focus:
outline-none focus:border-cyan-500 shadow-2xs font-medium"
1357 |                         />
1358 |                     </div>
1359 |
1360 |                     <div>
1361 |                       <label className="block text-[11px] font-bold text-slate-700 mb-1">Benchmark Exam / Cohort Title</label
>
1362 |                       <input
1363 |                         type="text"
1364 |                         value={story.bannerSubtitle}
1365 |                         onChange={(e) => handleStoryChange(sIdx, 'bannerSubtitle', e.target.value)}
1366 |                         placeholder="NATIONAL WEATHER FORECASTER BENCHMARK 2025"
1367 |                         className="w-full px-3 py-2 text-xs rounded-xl bg-white border border-slate-200 text-slate-900 focus:
outline-none focus:border-cyan-500 shadow-2xs font-mono font-bold text-orange-600"
1368 |                         />
1369 |                     </div>
1370 |                   </div>
1371 |
1372 |                   {/* Highlight Stat Pill Callout */}
1373 |                   <div className="p-4 rounded-xl bg-white border border-slate-200 space-y-3">
1374 |                     <span className="block text-[11px] font-bold text-slate-700 uppercase tracking-wider">
1375 |                       Highlight Stat Callout (Right Column Pill)
1376 |                     </span>
1377 |
1378 |                     <div className="grid grid-cols-1 sm:grid-cols-3 gap-3">
1379 |                       <div>
1380 |                         <label className="block text-[10px] font-bold text-slate-500 mb-0.5">Prefix / Label</label>
1381 |                         <input
1382 |                           type="text"
1383 |                           value={story.highlightStatTitle}
1384 |                           onChange={(e) => handleStoryChange(sIdx, 'highlightStatTitle', e.target.value)}
1385 |                           placeholder="OVER / EVERY"
1386 |                           className="w-full px-2.5 py-1.5 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate-9
00 font-bold focus:outline-none focus:border-cyan-500"
1387 |                           />
1388 |                       </div>
1389 |
1390 |                       <div>
1391 |                         <label className="block text-[10px] font-bold text-slate-500 mb-0.5">Big Number / Rate</label>
1392 |                         <input
```

```
                          type="text"
                          value={story.highlightStatNumber}
                          onChange={(e) => handleStoryChange(sIdx, 'highlightStatNumber', e.target.value)}
                          placeholder="94% or 9 IN 10"
                          className="w-full px-2.5 py-1.5 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate-900 font-black text-amber-600 focus:outline-none focus:border-cyan-500"
                        />
                      </div>

                      <div className="sm:col-span-1">
                        <label className="block text-[10px] font-bold text-slate-500 mb-0.5">Description</label>
                        <input
                          type="text"
                          value={story.highlightStatDescription}
                          onChange={(e) => handleStoryChange(sIdx, 'highlightStatDescription', e.target.value)}
                          placeholder="Trainees Secured Top Tier-1 Placement..."
                          className="w-full px-2.5 py-1.5 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate-800 text-[11px] focus:outline-none focus:border-cyan-500"
                        />
                      </div>
                    </div>
                  </div>

                  {/* Achievers in this story */}
                  <div className="space-y-3">
                    <div className="flex items-center justify-between">
                      <span className="text-xs font-bold text-slate-800 flex items-center gap-1.5">
                        <Users className="w-3.5 h-3.5 text-cyan-600" />
                        <span>Achievers in this Story ({story.achievers?.length || 0})</span>
                      </span>

                      <button
                        type="button"
                        onClick={() => handleAddAchiever(sIdx)}
                        className="text-xs text-cyan-700 hover:text-cyan-900 font-bold flex items-center gap-1 cursor-pointer"
                      >
                        <Plus className="w-3 h-3" /> Add Achiever
                      </button>
                    </div>

                    <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
                      {(story.achievers || []).map((achiever, aIdx) => (
                        <div
                          key={achiever.id || aIdx}
                          className="p-3.5 rounded-xl bg-white border border-slate-200 shadow-2xs space-y-2.5"
                        >
                          <div className="flex items-center justify-between">
                            <span className="text-[11px] font-bold text-slate-600">
                              Achiever #{aIdx + 1}
                            </span>
                            <button
                              type="button"
                              onClick={() => handleDeleteAchiever(sIdx, aIdx)}
                              className="text-slate-400 hover:text-red-600 p-0.5 rounded cursor-pointer"
                            >
                              <X className="w-3.5 h-3.5" />
                            </button>
                          </div>

                          {/* Image Upload or Monogram preview */}
                          <div className="flex items-center gap-3 p-2 bg-slate-50 rounded-xl border border-slate-200/80">
                            {achiever.avatar_url ? (
                              <div className="w-10 h-10 rounded-lg overflow-hidden border border-cyan-500/40 shrink-0">
                                <img
                                  src={achiever.avatar_url}
                                  alt={achiever.name}
                                  className="w-full h-full object-cover"
                                />
                              </div>
                            ) : (
                              <div className="w-10 h-10 rounded-lg bg-gradient-to-b from-slate-200 to-slate-400 text-slate-600 flex items-center justify-center shrink-0 border border-slate-300">
                                <User className="w-5 h-5 stroke-[1.5]" />
                              </div>
                            )}

                            <div className="flex-1 min-w-0">
                              <label className="inline-flex items-center gap-1 px-2.5 py-1 rounded-lg bg-cyan-50 hover:bg-cya
```

```
n-100 text-cyan-800 text-[10px] font-bold cursor-pointer border border-cyan-200 transition-colors">
                                <UploadCloud className="w-3 h-3" />
                                <span>
                                  {uploadingAchieverKey === `${sIdx}-${aIdx}` ? 'Uploading...' : 'Upload Photo'}
                                </span>
                                <input
                                  type="file"
                                  accept="image/*"
                                  className="hidden"
                                  onChange={(e) => {
                                    const file = e.target.files?.[0];
                                    if (file) handleAchieverAvatarUpload(sIdx, aIdx, file);
                                  }}
                                />
                              </label>

                              {achiever.avatar_url && (
                                <button
                                  type="button"
                                  onClick={() => handleAchieverChange(sIdx, aIdx, 'avatar_url', null)}
                                  className="block text-[10px] text-red-500 hover:underline mt-0.5"
                                >
                                  Use monogram
                                </button>
                              )}
                            </div>
                          </div>

                          <div>
                            <label className="block text-[10px] font-bold text-slate-500">Name &amp;amp; Initials</label>
                            <div className="grid grid-cols-4 gap-1.5 mt-0.5">
                              <input
                                type="text"
                                value={achiever.name}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'name', e.target.value)}
                                placeholder="Dr. Aarav Sharma"
                                className="col-span-3 px-2 py-1 text-xs rounded-lg bg-slate-50 border border-slate-200 text-s
late-900 font-bold focus:outline-none focus:border-cyan-500"
                              />
                              <input
                                type="text"
                                value={achiever.initials}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'initials', e.target.value)}
                                placeholder="AS"
                                maxLength={3}
                                className="col-span-1 px-1.5 py-1 text-xs text-center rounded-lg bg-slate-50 border border-sl
ate-200 text-slate-900 font-black focus:outline-none focus:border-cyan-500"
                              />
                            </div>
                          </div>

                          <div className="grid grid-cols-2 gap-1.5">
                            <div>
                              <label className="block text-[10px] font-bold text-slate-500">Role</label>
                              <input
                                type="text"
                                value={achiever.role}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'role', e.target.value)}
                                placeholder="Radar Meteorologist"
                                className="w-full px-2 py-1 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate
-800 focus:outline-none focus:border-cyan-500"
                              />
                            </div>

                            <div>
                              <label className="block text-[10px] font-bold text-slate-500">Institute</label>
                              <input
                                type="text"
                                value={achiever.institute}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'institute', e.target.value)}
                                placeholder="IMD New Delhi"
                                className="w-full px-2 py-1 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate
-800 focus:outline-none focus:border-cyan-500"
                              />
                            </div>
                          </div>

                          <div className="grid grid-cols-2 gap-1.5">
                            <div>
```

```
                              <label className="block text-[10px] font-bold text-slate-500">Specialist Tag</label>
                              <input
                                type="text"
                                value={achiever.score}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'score', e.target.value)}
                                placeholder="Top Forecaster"
                                className="w-full px-2 py-1 text-xs rounded-lg bg-slate-50 border border-slate-200 text-slate
-800 focus:outline-none focus:border-cyan-500"
                              />
                            </div>

                            <div>
                              <label className="block text-[10px] font-bold text-slate-500">Rank Pill</label>
                              <input
                                type="text"
                                value={achiever.achievementBadge}
                                onChange={(e) => handleAchieverChange(sIdx, aIdx, 'achievementBadge', e.target.value)}
                                placeholder="AIR 1 (99.8%)"
                                className="w-full px-2 py-1 text-xs rounded-lg bg-slate-50 border border-slate-200 text-amber
-700 font-bold focus:outline-none focus:border-cyan-500"
                              />
                            </div>
                          </div>
                        </div>
                      ))}
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </div>
        </motion.div>
      )}

      {/* ─── Bottom Floating Save Action Bar ─── */}
      <div className="p-5 rounded-3xl bg-white border border-slate-200/90 shadow-sm flex flex-col sm:flex-row items-center ju
stify-between gap-4">
        <p className="text-xs text-slate-500 font-medium">
          All changes save directly to the platform database and immediately update live for all landing page visitors.
        </p>

        <div className="flex items-center gap-3 shrink-0">
          <Button
            onClick={handleReset}
            variant="outline"
            className="border-slate-200 text-slate-700 hover:bg-slate-100 rounded-xl text-xs font-bold h-10 px-4 cursor-point
er"
          >
            Reset Defaults
          </Button>

          <Button
            onClick={handleSave}
            disabled={isUpdating}
            className="bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:opacity-95 text-white font-extrabold text
-xs h-10 px-6 rounded-xl shadow-md shadow-cyan-600/20 cursor-pointer"
          >
            {isUpdating ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> : <Save className="w-4 h-4 mr-2" />}
            {isUpdating ? 'Saving...' : 'Save & Publish Changes'}
          </Button>
        </div>
      </div>

      <ImageCropperModal
        isOpen={isCropperOpen}
        imageFile={rawImageFile}
        aspectRatio={2 / 1}
        onClose={() => {
          setIsCropperOpen(false);
          setRawImageFile(null);
          setCroppingIndex(null);
        }}
        onCropComplete={(croppedFile) => {
          setIsCropperOpen(false);
          setRawImageFile(null);
          if (croppingIndex !== null) {
            handleThumbnailUpload(croppingIndex, croppedFile);
          }
        }}
```

```
      />
    </div>
  );
}

```

src/features/admin/AdminUserDetailsModal.tsx TSX 31.5 KB 632 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { createPortal } from 'react-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { supabase } from '@/lib/supabase'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { toast } from 'sonner'
import { generateTraineeCertificate, triggerFileDownload } from '@/lib/certificateGenerator'
import {
  User, Mail, Phone, MapPin, Briefcase, GraduationCap,
  Calendar, Shield, Award, CheckCircle, Clock, Ban,
  ExternalLink, FileText, Globe, Link2,
  BookOpen, Layers, X, Sparkles, AlertCircle, Loader2
} from 'lucide-react'

interface AdminUserDetailsModalProps {
  user: any | null
  isOpen: boolean
  onClose: () => void
  onUpdateStatus?: (userId: string, email: string | null, role: any, status: any) => void
  onViewProof?: (proofPath: string) => void
}

export function AdminUserDetailsModal({
  user,
  isOpen,
  onClose,
  onUpdateStatus,
  onViewProof,
}: AdminUserDetailsModalProps) {
  const [activeTab, setActiveTab] = useState<'overview' | 'credentials' | 'contact' | 'courses'>('overview')
  const [userCourses, setUserCourses] = useState<any[]>([])
  const [loadingCourses, setLoadingCourses] = useState(false)
  const [mounted, setMounted] = useState(false)

  useEffect(() => {
    setMounted(true)
  }, [])

  // Reset tab when user changes

  useEffect(() => {
    if (isOpen) {
      setActiveTab('overview')
    }
  }, [isOpen, user?.id])

  const [downloadingCertEnrollmentId, setDownloadingCertEnrollmentId] = useState<string | null>(null)

  const handleDownloadTraineeCert = async (item: any) => {
    const c = item.courses
    if (!c || !user) return
    setDownloadingCertEnrollmentId(item.id)
    try {
      const { blob, fileName } = await generateTraineeCertificate(
        c.certificate_template_url,
        {
          traineeName: user.full_name || user.email?.split('@')[0] || 'Trainee',
          traineeEmail: user.email || undefined,
          traineeId: user.id,
          courseId: c.id,
          courseTitle: c.title,
          trainerName: 'Lead Instructor',
          percentage: `${item.progress_pct ?? item.progress_percent ?? 100}%`,
          completedAt: new Date().toISOString(),
        }
      )
      triggerFileDownload(blob, fileName)
      toast.success(`Certificate downloaded for ${user.full_name || 'Trainee'}! 🎉`)
    } catch (e: any) {
      console.error('Certificate generation error:', e)
      toast.error(e?.message || 'Failed to generate certificate')
    } finally {
      setDownloadingCertEnrollmentId(null)
    }
  }
```

```
76 |
77 |   // Fetch real-time enrollments or authored courses
78 |   useEffect(() => {
79 |     if (!user || !isOpen) {
80 |       setUserCourses([])
81 |       return
82 |     }
83 |
84 |     const fetchUserActivity = async () => {
85 |       setLoadingCourses(true)
86 |       try {
87 |         if (user.role === 'trainee') {
88 |           const { data, error } = await supabase
89 |             .from('enrollments')
90 |             .select('*, courses(id, title, course_type, delivery_mode, duration_minutes, certificate_template_url)')
91 |             .eq('user_id', user.id)
92 |             .order('enrolled_at', { ascending: false })
93 |
94 |           if (!error && data) {
95 |             setUserCourses(data)
96 |           }
97 |         } else if (user.role === 'trainer') {
98 |           const { data, error } = await supabase
99 |             .from('courses')
100|             .select('id, title, course_type, delivery_mode, status, max_trainees, created_at')
101|             .eq('trainer_id', user.id)
102|             .order('created_at', { ascending: false })
103|
104|           if (!error && data) {
105|             setUserCourses(data)
106|           }
107|         }
108|       } catch (e) {
109|         console.error('Error fetching user courses in modal:', e)
110|       } finally {
111|         setLoadingCourses(false)
112|       }
113|     }
114|
115|     fetchUserActivity()
116|   }, [user, isOpen])
117|
118|   // Close on Escape key
119|   useEffect(() => {
120|     const handleKeyDown = (e: KeyboardEvent) => {
121|       if (e.key === 'Escape' && isOpen) {
122|         onClose()
123|       }
124|     }
125|     window.addEventListener('keydown', handleKeyDown)
126|     return () => window.removeEventListener('keydown', handleKeyDown)
127|   }, [isOpen, onClose])
128|
129|   if (!mounted) return null
130|
131|   const isTrainer = user?.role === 'trainer'
132|   const isTrainee = user?.role === 'trainee'
133|
134|   // Extract skills/expertise
135|   const skillsList: string[] = user ? (Array.isArray(user.skills)
136|     ? user.skills
137|     : Array.isArray(user.expertise_areas)
138|     ? user.expertise_areas
139|     : typeof user.skills === 'string'
140|     ? user.skills.split(',').map((s: string) => s.trim()).filter(Boolean)
141|     : []) : []
142|
143|   const interestsList: string[] = user ? (Array.isArray(user.interests)
144|     ? user.interests
145|     : typeof user.interests === 'string'
146|     ? user.interests.split(',').map((s: string) => s.trim()).filter(Boolean)
147|     : []) : []
148|
149|   return createPortal(
150|     <AnimatePresence>
151|       {isOpen && user && (
```

```
        <div className="fixed inset-0 z-[99999] flex items-center justify-center p-3 sm:p-5 overflow-y-auto">
          {/* Backdrop */}
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            onClick={onClose}
            className="fixed inset-0 bg-slate-950/75 backdrop-blur-sm"
          />

          {/* Modal Dialog Card */}
          <motion.div
            initial={{ opacity: 0, scale: 0.95, y: 20 }}
            animate={{ opacity: 1, scale: 1, y: 0 }}
            exit={{ opacity: 0, scale: 0.95, y: 20 }}
            transition={{ type: 'spring', damping: 25, stiffness: 300 }}
            className="relative w-full max-w-3xl bg-white border border-slate-200/90 text-slate-900 rounded-3xl shadow-2xl overflow-hidden z-10 flex flex-col max-h-[90vh]"
          >
          {/* Close button */}
          <button
            type="button"
            onClick={onClose}
            className="absolute top-4 right-4 z-20 w-8 h-8 rounded-full bg-black/40 hover:bg-black/60 text-white flex items-center justify-center transition-all cursor-pointer backdrop-blur-xs"
            title="Close Window"
          >
            <X className="w-4 h-4" />
          </button>

          {/* Header Hero Banner */}
          <div className="relative p-6 sm:p-7 bg-gradient-to-r from-slate-900 via-[#0a1628] to-[#0d2242] text-white shrink-0">
            <div className="absolute top-0 right-0 w-80 h-80 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl pointer-events-none" />

            <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 relative z-10 pr-8">
              <div className="flex items-center gap-4">
                <div className="w-16 h-16 rounded-2xl bg-gradient-to-br from-cyan-500 via-sky-600 to-blue-600 flex items-center justify-center text-white text-2xl font-black shadow-lg shadow-cyan-500/30 shrink-0 border border-white/20 overflow-hidden">
                  {user.avatar_path ? (
                    <img src={user.avatar_path} alt={user.full_name} className="w-full h-full object-cover" />
                  ) : (
                    user.full_name?.charAt(0)?.toUpperCase() ?? '?'
                  )}
                </div>
                <div>
                  <div className="flex items-center gap-2 flex-wrap">
                    <h3 className="text-xl font-extrabold text-white tracking-tight">{user.full_name || 'Anonymous User'}</h3>
                    <span className={`capitalize text-xs font-bold px-2.5 py-0.5 rounded-full border ${
                      user.role === 'trainer' ? 'bg-purple-500/20 text-purple-200 border-purple-400/40' :
                      user.role === 'admin' || user.role === 'super_admin' ? 'bg-rose-500/20 text-rose-200 border-rose-400/40' :
                      'bg-cyan-500/20 text-cyan-200 border-cyan-400/40'
                    }`}>
                      {user.role}
                    </span>
                    <span className={`capitalize text-xs font-bold px-2.5 py-0.5 rounded-full border ${
                      user.approval_status === 'approved' ? 'bg-emerald-500/20 text-emerald-300 border-emerald-400/40' :
                      user.approval_status === 'pending' ? 'bg-amber-500/20 text-amber-300 border-amber-400/40' :
                      'bg-rose-500/20 text-rose-300 border-rose-400/40'
                    }`}>
                      {user.approval_status}
                    </span>
                  </div>
                  <p className="text-xs text-slate-300 flex items-center gap-1.5 mt-1 flex-wrap">
                    <Mail className="w-3.5 h-3.5 text-cyan-400 shrink-0" />
                    <span>{user.email || 'No email provided'}</span>
                    {user.department && (
                      <>
                        <span>•</span>
                        <span className="text-cyan-300 font-semibold">{user.department}</span>
                      </>
                    )}
                  </p>
                </div>
              </div>

              {user.proof_path && onViewProof && (
                <Button
```

```
                    size="sm"
                    type="button"
                    variant="outline"
                    onClick={() => onViewProof(user.proof_path)}
                    className="bg-white/10 hover:bg-white/20 text-white border-white/20 rounded-xl text-xs font-semibold cursor-pointer shrink-0"
                  >
                    <Shield className="w-3.5 h-3.5 mr-1.5 text-emerald-400" /> View ID Proof
                  </Button>
                )}
              </div>

              {/* Navigation Tabs */}
              <div className="flex items-center gap-1.5 mt-6 border-b border-white/10 pb-0 overflow-x-auto hide-scrollbar [scrollbar-width:none] [-ms-overflow-style:none] [&::-webkit-scrollbar]:hidden">
                {[
                  { id: 'overview', label: 'Overview & Profile', icon: User },
                  { id: 'credentials', label: isTrainer ? 'Experience & Qualifications' : 'Academic & Skills', icon: GraduationCap },
                  { id: 'contact', label: 'Contact & Location', icon: MapPin },
                  { id: 'courses', label: isTrainer ? `Courses Created (${userCourses.length})` : `Enrolled Programs (${userCourses.length})`, icon: BookOpen },
                ].map(tab => (
                  <button
                    key={tab.id}
                    type="button"
                    onClick={() => setActiveTab(tab.id as any)}
                    className={`flex items-center gap-1.5 px-4 py-2.5 text-xs font-bold rounded-t-xl transition-all cursor-pointer whitespace-nowrap ${
                      activeTab === tab.id
                        ? 'bg-white text-slate-900 shadow-md'
                        : 'text-slate-300 hover:text-white hover:bg-white/10'
                    }`}
                  >
                    <tab.icon className="w-3.5 h-3.5" />
                    {tab.label}
                  </button>
                ))}
              </div>
            </div>

            {/* Modal Body Content */}
            <div className="p-6 overflow-y-auto flex-1 space-y-5 bg-slate-50/50">
              {/* TAB 1: OVERVIEW */}
              {activeTab === 'overview' && (
                <div className="space-y-4">
                  <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                    <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                      <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">MoES Department / Institute</p>
                      <p className="text-sm font-bold text-slate-900">{user.department || 'MoES Central Secretariat'}</p>
                      <p className="text-xs text-slate-500">{user.designation || 'Scientific / Administrative Officer'}</p>
                    </div>

                    <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                      <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Platform Registration</p>
                      <p className="text-sm font-bold text-slate-900">
                        {user.created_at ? new Date(user.created_at).toLocaleDateString('en-US', { month: 'long', day: 'numeric', year: 'numeric' }) : 'Verified Account'}
                      </p>
                      <p className="text-xs text-slate-500">System UID: {user.id ? `${user.id.slice(0, 13)}...` : '—'}</p>
                    </div>
                  </div>

                  {/* Bio / Summary */}
                  <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-2">
                    <div className="flex items-center gap-2 text-xs font-bold uppercase tracking-wider text-slate-700">
                      <Briefcase className="w-4 h-4 text-cyan-600" />
                      <span>About & Professional Profile</span>
                    </div>
                    <p className="text-sm text-slate-700 leading-relaxed">
                      {user.bio || 'No specialized bio or biography details have been recorded yet.'}
                    </p>
                  </div>

                  {/* Learning Goals or Trainer Availability */}
                  {user.learning_goals && (
                    <div className="p-4 rounded-2xl bg-cyan-50/60 border border-cyan-200/70 shadow-xs space-y-1.5">
                      <div className="flex items-center gap-2 text-xs font-bold uppercase tracking-wider text-cyan-900">
                        <Sparkles className="w-4 h-4 text-cyan-600" />
```

```
                      <span>Capacity & Learning Goals</span>
                    </div>
                    <p className="text-sm text-cyan-950">{user.learning_goals}</p>
                  </div>
                )}

                {user.availability && (
                  <div className="flex items-center justify-between p-3.5 rounded-2xl bg-white border border-slate-200 shadow-
xs text-xs">
                    <span className="font-semibold text-slate-700">Trainer Availability:</span>
                    <Badge variant="outline" className="bg-emerald-50 text-emerald-800 border-emerald-300 font-bold capitaliz
e">
                      {user.availability}
                    </Badge>
                  </div>
                )}
              </div>
            )}

            {/* TAB 2: CREDENTIALS & SKILLS */}
            {activeTab === 'credentials' && (
              <div className="space-y-4">
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Educational Qualifications</p>
                    <p className="text-sm font-bold text-slate-900">{user.qualifications || '—'}</p>
                  </div>

                  <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Domain Experience</p>
                    <p className="text-sm font-bold text-slate-900">
                      {user.years_of_experience ? `${user.years_of_experience} Years in Ministry / Research` : (user.work_expe
rience ? 'Experience recorded' : '—')}
                    </p>
                  </div>
                </div>

                {user.work_experience && (
                  <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-2">
                    <p className="text-xs font-bold uppercase tracking-wider text-slate-600">Work Experience Narrative</p>
                    <p className="text-sm text-slate-700 whitespace-pre-line leading-relaxed">{user.work_experience}</p>
                  </div>
                )}

                {/* Skills Tags */}
                <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-2.5">
                  <p className="text-xs font-bold uppercase tracking-wider text-slate-600">Technical Skills & Competencies</p>
                  {skillsList.length > 0 ? (
                    <div className="flex flex-wrap gap-2">
                      {skillsList.map((skill, idx) => (
                        <span
                          key={idx}
                          className="px-2.5 py-1 rounded-lg text-xs font-semibold bg-cyan-50 text-cyan-800 border border-cyan-
200/80 shadow-2xs"
                        >
                          {skill}
                        </span>
                      ))}
                    </div>
                  ) : (
                    <p className="text-xs text-slate-400 italic">No specific competency tags listed.</p>
                  )}
                </div>

                {/* Interests Tags */}
                {interestsList.length > 0 && (
                  <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-2.5">
                    <p className="text-xs font-bold uppercase tracking-wider text-slate-600">Research & Focus Interests</p>
                    <div className="flex flex-wrap gap-2">
                      {interestsList.map((interest, idx) => (
                        <span
                          key={idx}
                          className="px-2.5 py-1 rounded-lg text-xs font-semibold bg-purple-50 text-purple-800 border border-p
urple-200/80 shadow-2xs"
                        >
                          {interest}
                        </span>
                      ))}
                    </div>
```

```
                  </div>
                )}
              </div>
            )}

            {/* TAB 3: CONTACT & LOCATION */}
            {activeTab === 'contact' && (
              <div className="space-y-4">
                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Official Email Address</p>
                    <p className="text-sm font-bold text-slate-900">{user.email || '—'}</p>
                  </div>

                  <div className="p-4 rounded-2xl bg-white border border-slate-200/80 shadow-xs space-y-1">
                    <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Telephone / Contact</p>
                    <p className="text-sm font-bold text-slate-900">{user.phone || 'Not recorded'}</p>
                    {user.alternate_phone && (
                      <p className="text-xs text-slate-500">Alternate: {user.alternate_phone}</p>
                    )}
                  </div>
                </div>

                {/* Address Details */}
                <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-3">
                  <div className="flex items-center gap-2 text-xs font-bold uppercase tracking-wider text-slate-700">
                    <MapPin className="w-4 h-4 text-cyan-600" />
                    <span>Location & Postal Address</span>
                  </div>
                  <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 text-xs">
                    <div>
                      <span className="text-slate-400 font-semibold block">Address</span>
                      <span className="text-slate-800 font-bold">{user.address || '—'}</span>
                    </div>
                    <div>
                      <span className="text-slate-400 font-semibold block">City</span>
                      <span className="text-slate-800 font-bold">{user.city || '—'}</span>
                    </div>
                    <div>
                      <span className="text-slate-400 font-semibold block">State</span>
                      <span className="text-slate-800 font-bold">{user.state || '—'}</span>
                    </div>
                    <div>
                      <span className="text-slate-400 font-semibold block">Pincode / Country</span>
                      <span className="text-slate-800 font-bold">{user.pincode || '—'}, {user.country || 'India'}</span>
                    </div>
                  </div>
                </div>

                {/* Social / Portfolio Links */}
                <div className="p-4 rounded-2xl bg-white border border-slate-200 shadow-xs space-y-3">
                  <p className="text-xs font-bold uppercase tracking-wider text-slate-600">Online Profiles & Links</p>
                  <div className="flex flex-wrap gap-3">
                    {user.linkedin_url ? (
                      <a
                        href={user.linkedin_url.startsWith('http') ? user.linkedin_url : `https://${user.linkedin_url}`}
                        target="_blank"
                        rel="noopener noreferrer"
                        className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-blue-50 text-blue-700 border bor
der-blue-200 text-xs font-bold hover:bg-blue-100 transition-colors"
                      >
                        <Link2 className="w-3.5 h-3.5" /> LinkedIn Profile
                      </a>
                    ) : null}
                    {user.github_url ? (
                      <a
                        href={user.github_url.startsWith('http') ? user.github_url : `https://${user.github_url}`}
                        target="_blank"
                        rel="noopener noreferrer"
                        className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-slate-100 text-slate-800 border
border-slate-300 text-xs font-bold hover:bg-slate-200 transition-colors"
                      >
                        <Globe className="w-3.5 h-3.5" /> GitHub Profile
                      </a>
                    ) : null}
                    {user.website_url ? (
                      <a
                        href={user.website_url.startsWith('http') ? user.website_url : `https://${user.website_url}`}
                        target="_blank"
```

```
                        rel="noopener noreferrer"
                        className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl bg-cyan-50 text-cyan-700 border bor
der-cyan-200 text-xs font-bold hover:bg-cyan-100 transition-colors"
                      >
                        <ExternalLink className="w-3.5 h-3.5" /> Official Website
                      </a>
                    ) : null}
                    {!user.linkedin_url && !user.github_url && !user.website_url && (
                      <p className="text-xs text-slate-400 italic">No external web profiles linked.</p>
                    )}
                  </div>
                </div>
              </div>
            )}

            {/* TAB 4: COURSES & ACTIVITY */}
            {activeTab === 'courses' && (
              <div className="space-y-4">
                <div className="flex items-center justify-between">
                  <h4 className="text-xs font-bold uppercase tracking-wider text-slate-700">
                    {isTrainer ? 'Courses Authored & Assigned' : 'Enrolled MoES Programs'}
                  </h4>
                  <Badge variant="outline" className="text-xs font-bold bg-white border-slate-300">
                    Total: {userCourses.length}
                  </Badge>
                </div>

                {loadingCourses ? (
                  <div className="p-8 text-center text-slate-400 text-sm">Loading activity records...</div>
                ) : userCourses.length === 0 ? (
                  <div className="p-8 text-center bg-white rounded-2xl border border-slate-200 text-slate-500 text-sm">
                    {isTrainer ? 'No courses have been created by or assigned to this trainer yet.' : 'This trainee has not en
rolled in any training courses yet.'}
                  </div>
                ) : (
                  <div className="space-y-2.5">
                    {userCourses.map((item, idx) => {
                      if (isTrainee) {
                        const c = item.courses
                        return (
                          <div key={idx} className="p-4 rounded-2xl border border-slate-200 bg-white shadow-xs flex items-cent
er justify-between gap-4">
                            <div>
                              <p className="text-sm font-bold text-slate-900">{c?.title || 'Course Details'}</p>
                              <p className="text-xs text-slate-500 mt-0.5 capitalize">
                                {c?.course_type} • {c?.delivery_mode || 'Recorded'} • Enrolled on {item.enrolled_at ? new Date
(item.enrolled_at).toLocaleDateString() : '—'}
                              </p>
                            </div>
                            <div className="text-right shrink-0 flex items-center gap-3">
                              <div>
                                <Badge className={`capitalize font-bold text-xs ${
                                  item.status === 'completed' ? 'bg-emerald-100 text-emerald-800 border-emerald-300' :
                                  item.status === 'in_progress' ? 'bg-cyan-100 text-cyan-800 border-cyan-300' :
                                  'bg-slate-100 text-slate-700'
                                }`}>
                                  {item.status}
                                </Badge>
                                {(item.progress_pct !== undefined || item.progress_percent !== undefined) && (
                                  <p className="text-[11px] font-bold text-slate-600 mt-1">{item.progress_pct ?? item.progress
_percent}% Completed</p>
                                )}
                              </div>
                              <button
                                type="button"
                                onClick={() => handleDownloadTraineeCert(item)}
                                disabled={downloadingCertEnrollmentId === item.id}
                                className="px-3 py-1.5 rounded-xl bg-gradient-to-r from-emerald-600 via-teal-600 to-cyan-600 h
over:from-emerald-700 hover:to-cyan-700 text-white font-bold text-xs shadow-xs hover:scale-105 active:scale-95 transition-all cursor
-pointer flex items-center gap-1.5 disabled:opacity-60"
                                title="Download Certificate"
                              >
                                {downloadingCertEnrollmentId === item.id ? (
                                  <Loader2 className="w-3.5 h-3.5 animate-spin" />
                                ) : (
                                  <Award className="w-3.5 h-3.5 text-amber-300" />
                                )}
                                <span>Certificate</span>
                              </button>
```

```
                            </div>
                          </div>
                        )
                      } else {
                        // Trainer's course

                        return (
                          <div key={idx} className="p-4 rounded-2xl border border-slate-200 bg-white shadow-xs flex items-cent
er justify-between gap-4">
                            <div>
                              <p className="text-sm font-bold text-slate-900">{item.title}</p>
                              <p className="text-xs text-slate-500 mt-0.5 capitalize">
                                {item.course_type} • Delivery: {item.delivery_mode || 'Recorded'} • Capacity: {item.max_traine
es || 'Unlimited'}
                              </p>
                            </div>
                            <Badge className={`capitalize font-bold text-xs ${
                              item.status === 'published' ? 'bg-emerald-100 text-emerald-800 border-emerald-300' :
                              item.status === 'draft' ? 'bg-slate-100 text-slate-700' :
                              'bg-amber-100 text-amber-800 border-amber-300'
                            }`}>
                              {item.status}
                            </Badge>
                          </div>
                        )
                      }
                    })}
                  </div>
                )}
              </div>
            )}
          </div>

          {/* Modal Action Footer */}
          <div className="p-4 sm:p-5 bg-white border-t border-slate-200 flex flex-wrap items-center justify-between gap-3 shri
nk-0">
            <div className="flex items-center gap-2">
              {onUpdateStatus && (
                <>
                  {user.approval_status === 'pending' && (
                    <>
                      <Button
                        size="sm"
                        type="button"
                        onClick={() => {
                          onUpdateStatus(user.id, user.email, user.role, 'approved')
                          onClose()
                        }}
                        className="bg-emerald-600 hover:bg-emerald-700 text-white font-bold rounded-xl text-xs cursor-pointer
shadow-xs"
                      >
                        <CheckCircle className="w-3.5 h-3.5 mr-1.5" /> Approve Account
                      </Button>
                      <Button
                        size="sm"
                        type="button"
                        variant="outline"
                        onClick={() => {
                          onUpdateStatus(user.id, user.email, user.role, 'rejected')
                          onClose()
                        }}
                        className="border-rose-300 text-rose-700 hover:bg-rose-50 font-bold rounded-xl text-xs cursor-pointer"
                      >
                        <Ban className="w-3.5 h-3.5 mr-1.5" /> Reject Account
                      </Button>
                    </>
                  )}
                  {user.approval_status === 'approved' && (
                    <Button
                      size="sm"
                      type="button"
                      variant="outline"
                      onClick={() => {
                        onUpdateStatus(user.id, user.email, user.role, 'suspended')
                        onClose()
                      }}
                      className="border-rose-300 text-rose-700 hover:bg-rose-50 font-bold rounded-xl text-xs cursor-pointer"
                    >
                      <Ban className="w-3.5 h-3.5 mr-1.5" /> Suspend Account
```

```
                    </Button>
                  )}
                  {user.approval_status === 'suspended' && (
                    <Button
                      size="sm"
                      type="button"
                      onClick={() => {
                        onUpdateStatus(user.id, user.email, user.role, 'approved')
                        onClose()
                      }}
                      className="bg-emerald-600 hover:bg-emerald-700 text-white font-bold rounded-xl text-xs cursor-pointer sh
adow-xs"
                    >
                      <CheckCircle className="w-3.5 h-3.5 mr-1.5" /> Unsuspend Account
                    </Button>
                  )}
                </>
              )}
            </div>

            <Button
              type="button"
              variant="outline"
              size="sm"
              onClick={onClose}
              className="rounded-xl border-slate-300 text-slate-700 font-semibold text-xs cursor-pointer hover:bg-slate-100"
            >
              Close Window
            </Button>
          </div>
        </motion.div>
      </div>
      )}
    </AnimatePresence>,
    document.body
  )
}

```

src/features/admin/TrainerAssignmentBanner.tsx TSX 6.8 KB 149 Lines

```tsx
import React, { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'
import { motion, AnimatePresence } from 'framer-motion'
import { X, Megaphone, BookOpen, ArrowRight, Sparkles } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { formatDistanceToNow } from 'date-fns'

interface Assignment {
  id: string
  course_id: string
  message: string | null
  assigned_at: string
  is_read: boolean
  course: { title: string; description: string; status: string }
  assigned_by_name: string
}

export function TrainerAssignmentBanner() {
  const { user } = useAuth()
  const [assignments, setAssignments] = useState<Assignment[]>([])
  const [dismissedIds, setDismissedIds] = useState<string[]>([])
  const [current, setCurrent] = useState(0)

  useEffect(() => {
    if (!user) return
    const stored = JSON.parse(localStorage.getItem('dismissed_assignments') || '[]')
    setDismissedIds(stored)

    ;(supabase as any)
      .from('course_assignments')
      .select('id, course_id, message, assigned_at, is_read, course:courses(title, description, status), admin:admins!assigned_by(full_name)')
      .eq('trainer_id', user.id)
      .order('assigned_at', { ascending: false })
      .limit(5)
      .then(({ data }: { data: any[] | null }) => {
        if (data) {
          setAssignments(data.map((a: any) => ({
            ...a,
            course: Array.isArray(a.course) ? a.course[0] : a.course,
            assigned_by_name: (Array.isArray(a.admin) ? a.admin[0]?.full_name : a.admin?.full_name) || 'Admin',
          })))
        }
      })
  }, [user])

  const dismiss = async (id: string) => {
    const newDismissed = [...dismissedIds, id]
    setDismissedIds(newDismissed)
    localStorage.setItem('dismissed_assignments', JSON.stringify(newDismissed))
    await (supabase as any).from('course_assignments').update({ is_read: true }).eq('id', id)
  }

  const visible = assignments.filter(a => !dismissedIds.includes(a.id))
  if (visible.length === 0) return null

  const safeCurrent = Math.min(current, visible.length - 1)
  const assignment = visible[safeCurrent]

  return (
    <AnimatePresence mode="wait">
      <motion.div
        key={assignment.id}
        initial={{ opacity: 0, y: -20, scale: 0.98 }}
        animate={{ opacity: 1, y: 0, scale: 1 }}
        exit={{ opacity: 0, y: -10, scale: 0.98 }}
        transition={{ duration: 0.4, type: 'spring', bounce: 0.2 }}
        className="relative overflow-hidden rounded-3xl border border-cyan-500/30 bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] text-white shadow-xl mb-6"
      >
        <div className="absolute inset-0 pointer-events-none">
          <div className="absolute -top-10 -right-10 w-64 h-64 bg-gradient-to-br from-cyan-500/20 to-blue-500/20 rounded-full blur-3xl" />
          <div className="absolute -bottom-8 -left-8 w-48 h-48 bg-gradient-to-tr from-sky-400/20 to-blue-400/20 rounded-full b
```

```
lur-3xl" />
        </div>

        <div className="relative z-10 p-6 md:p-8">
          <div className="flex items-start justify-between gap-4">
            <div className="flex items-start gap-4 flex-1 min-w-0">
              <div className="shrink-0 w-14 h-14 rounded-2xl bg-white/10 backdrop-blur-sm border border-white/20 flex items-ce
nter justify-center shadow-inner">
                <Megaphone className="w-7 h-7 text-amber-300" />
              </div>

              <div className="flex-1 min-w-0">
                <div className="inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full bg-amber-400/20 border border-amber-
300/30 text-amber-200 text-[11px] font-bold uppercase tracking-wider mb-2">
                  <Sparkles className="w-3 h-3" />
                  New Course Assignment
                </div>
                <h3 className="text-xl font-extrabold text-white leading-tight mb-1 truncate">
                  🎓 {assignment.course?.title ?? 'Course Assigned to You'}
                </h3>
                <p className="text-slate-300 text-sm mb-2 line-clamp-2">
                  {assignment.course?.description ?? 'Admin has assigned you a new course to develop and manage.'}
                </p>
                {assignment.message && (
                  <div className="bg-white/10 backdrop-blur-sm border border-white/15 rounded-xl px-4 py-2.5 mb-3">
                    <p className="text-white/90 text-sm italic">"{assignment.message}"</p>
                    <p className="text-slate-400 text-xs mt-0.5">— {assignment.assigned_by_name}</p>
                  </div>
                )}
                <div className="flex items-center gap-4 flex-wrap">
                  <span className="text-slate-400 text-xs">
                    {formatDistanceToNow(new Date(assignment.assigned_at), { addSuffix: true })}
                  </span>
                  <Link to={'/trainer/courses/' + assignment.course_id}>
                    <Button className="bg-gradient-to-r from-cyan-500 via-sky-500 to-blue-600 hover:from-cyan-600 hover:to-blu
e-700 text-white font-bold rounded-xl px-5 py-2 h-auto text-sm shadow-md hover:scale-105 transition-all">
                      <BookOpen className="w-4 h-4 mr-1.5" />
                      View Course
                      <ArrowRight className="w-3.5 h-3.5 ml-1.5" />
                    </Button>
                  </Link>
                </div>
              </div>
            </div>

            <button
              onClick={() => dismiss(assignment.id)}
              className="shrink-0 w-8 h-8 rounded-xl bg-white/10 hover:bg-white/20 flex items-center justify-center transition
-colors"
              aria-label="Dismiss"
            >
              <X className="w-4 h-4 text-white/70" />
            </button>
          </div>

          {visible.length > 1 && (
            <div className="flex items-center justify-center gap-2 mt-4 pt-4 border-t border-white/10">
              <span className="text-slate-400 text-xs">{safeCurrent + 1} of {visible.length}</span>
              <div className="flex gap-1.5">
                {visible.map((_, i) => (
                  <button
                    key={i}
                    onClick={() => setCurrent(i)}
                    className={'rounded-full transition-all ' + (i === safeCurrent ? 'w-5 h-1.5 bg-white' : 'w-1.5 h-1.5 bg-wh
ite/30 hover:bg-white/60')}
                  />
                ))}
              </div>
              <button
                onClick={() => setCurrent(c => (c + 1) % visible.length)}
                className="text-slate-400 hover:text-white text-xs ml-2 transition-colors"
              >
                Next &rarr;
              </button>
            </div>
          )}
        </div>
      </motion.div>
    </AnimatePresence>
```

```
147 |   )
148 | }
149 |
```

src/features/notifications/UniversalNotificationsPage.tsx TSX 14.0 KB 338 Lines

```tsx
import React, { useState } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { useNavigate, Link } from 'react-router-dom'
import { formatDistanceToNow } from 'date-fns'
import { useNotifications, getNotificationRedirectUrl, getNotificationMeta } from '@/hooks/useNotifications'
import { useAuth } from '@/hooks/useAuth'
import { useConfirm } from '@/hooks/useConfirm'
import { DashboardShell } from '@/components/layout/DashboardShell'
import { TrainerLayout } from '@/features/trainer/TrainerLayout'
import {
  Bell, Check, CheckCheck, Trash2, X,
  Clock, Sparkles, Megaphone, Video,
  Award, BookOpen, MessageSquare, ShieldAlert,
  Loader2, ArrowRight, CheckCircle, RefreshCw,
  Compass, FileCheck, User, BarChart3, Globe,
  Shield, Mail, LayoutDashboard
} from 'lucide-react'
import { Button } from '@/components/ui/button'

const fadeUp = {
  hidden: { opacity: 0, y: 15 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.3 } },
}

const stagger = {
  hidden: {},
  visible: { transition: { staggerChildren: 0.05 } },
}

export function UniversalNotificationsPage({ hideShell = false }: { hideShell?: boolean } = {}) {
  const { profile } = useAuth()
  const navigate = useNavigate()
  const [ConfirmDialog, confirm] = useConfirm()

  const {
    notifications,
    loading,
    markAsRead,
    markAllAsRead,
    unreadCount,
    deleteNotification,
    clearAllNotifications
  } = useNotifications()

  const [filter, setFilter] = useState<'all' | 'unread' | 'read'>('all')
  const [markingId, setMarkingId] = useState<string | null>(null)
  const [deletingId, setDeletingId] = useState<string | null>(null)

  const handleMarkAsRead = async (id: string, e?: React.MouseEvent) => {
    if (e) {
      e.preventDefault()
      e.stopPropagation()
    }
    setMarkingId(id)
    await markAsRead(id)
    setMarkingId(null)
  }

  const handleDelete = async (id: string, e: React.MouseEvent) => {
    e.preventDefault()
    e.stopPropagation()
    setDeletingId(id)
    await deleteNotification(id)
    setDeletingId(null)
  }

  const handleClearAll = async () => {
    const isConfirmed = await confirm('Are you sure you want to delete all notifications?', 'Clear All Notifications')
    if (isConfirmed) {
      await clearAllNotifications()
    }
  }

  const formatSafeTime = (dateStr?: string | null) => {
    if (!dateStr) return 'Recently'
    try {
```

```
      const d = new Date(dateStr)
      if (isNaN(d.getTime())) return 'Recently'
      return formatDistanceToNow(d, { addSuffix: true })
    } catch {
      return 'Recently'
    }
  }

  const getCategoryIcon = (category: string) => {
    switch (category) {
      case 'session':
        return <Video className="w-4 h-4 text-indigo-600" />
      case 'announcement':
        return <Megaphone className="w-4 h-4 text-amber-600" />
      case 'chat':
        return <MessageSquare className="w-4 h-4 text-cyan-600" />
      case 'course':
        return <Award className="w-4 h-4 text-emerald-600" />
      case 'user':
        return <BookOpen className="w-4 h-4 text-blue-600" />
      default:
        return <ShieldAlert className="w-4 h-4 text-slate-600" />
    }
  }

  const filteredNotifications = notifications.filter(n => {
    if (filter === 'unread') return !n.read_at
    if (filter === 'read') return !!n.read_at
    return true
  })

  const content = (
    <div className="max-w-4xl mx-auto space-y-6 pb-12">
      <ConfirmDialog />

      {/* Header Banner */}
      <motion.div
        variants={fadeUp}
        initial="hidden"
        animate="visible"
        className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 p-6 bg-white border border-slate-200/90 rou
nded-3xl shadow-xs"
      >
        <div className="flex items-center gap-3">
          <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-cyan-600 to-blue-600 flex items-center justify-center t
ext-white shadow-md shadow-cyan-600/20 shrink-0">
            <Bell className="w-6 h-6" />
          </div>
          <div>
            <h1 className="text-xl sm:text-2xl font-extrabold text-slate-900 tracking-tight">Notification Center</h1>
            <p className="text-xs sm:text-sm text-slate-500 mt-0.5">
              {unreadCount > 0 ? `You have ${unreadCount} unread message${unreadCount > 1 ? 's' : ''}` : 'All caught up! No un
read messages'}
            </p>
          </div>
        </div>

        <div className="flex items-center gap-2">
          {unreadCount > 0 && (
            <Button
              onClick={markAllAsRead}
              variant="outline"
              size="sm"
              className="bg-cyan-50 border-cyan-200 text-cyan-700 hover:bg-cyan-100 font-bold rounded-xl transition-all shadow
-xs"
            >
              <CheckCheck className="w-4 h-4 mr-1.5" /> Mark all read
            </Button>
          )}
          {notifications.length > 0 && (
            <Button
              onClick={handleClearAll}
              variant="outline"
              size="sm"
              className="bg-rose-50 border-rose-200 text-rose-600 hover:bg-rose-100 font-bold rounded-xl transition-all shadow
-xs"
            >
              <Trash2 className="w-4 h-4 mr-1.5" /> Clear all
            </Button>
```

```
          )}
        </div>
      </motion.div>

      {/* Filter Tabs */}
      <motion.div variants={fadeUp} initial="hidden" animate="visible" className="flex items-center gap-2 bg-slate-100/90 p-1.5 rounded-2xl border border-slate-200/80 w-fit">
        {[
          { key: 'all', label: 'All Notifications', count: notifications.length },
          { key: 'unread', label: 'Unread', count: unreadCount },
          { key: 'read', label: 'Read History', count: notifications.filter(n => !!n.read_at).length },
        ].map(f => (
          <button
            key={f.key}
            onClick={() => setFilter(f.key as any)}
            className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-bold transition-all ${
              filter === f.key
                ? 'bg-white text-slate-900 shadow-xs border border-slate-200/80'
                : 'text-slate-600 hover:text-slate-900 hover:bg-slate-200/50'
            }`}
          >
            <span>{f.label}</span>
            <span className={`text-[10px] px-2 py-0.5 rounded-full ${
              filter === f.key ? 'bg-cyan-100 text-cyan-800' : 'bg-slate-200 text-slate-600'
            }`}>
              {f.count}
            </span>
          </button>
        ))}
      </motion.div>

      {/* Notifications List */}
      {loading ? (
        <div className="py-24 text-center text-slate-400 flex flex-col items-center justify-center gap-3">
          <Loader2 className="w-8 h-8 animate-spin text-cyan-600" />
          <p className="text-sm font-semibold text-slate-600">Loading your notifications...</p>
        </div>
      ) : filteredNotifications.length === 0 ? (
        <motion.div variants={fadeUp} initial="hidden" animate="visible" className="p-12 text-center bg-white border border-slate-200/90 rounded-3xl shadow-xs">
          <div className="w-16 h-16 rounded-3xl bg-cyan-50 border border-cyan-100 text-cyan-600 flex items-center justify-center mx-auto mb-4 shadow-sm">
            <Sparkles className="w-8 h-8 text-cyan-500" />
          </div>
          <h3 className="text-base font-extrabold text-slate-800">
            {filter === 'unread' ? 'No unread notifications' : 'No notifications in this view'}
          </h3>
          <p className="text-xs text-slate-500 mt-1 max-w-sm mx-auto leading-relaxed">
            {filter === 'unread'
              ? 'You have viewed all incoming notifications. Check back later for new alerts.'
              : 'Course enrollments, session updates, assessments, and announcements will be logged here.'}
          </p>
        </motion.div>
      ) : (
        <motion.div variants={stagger} initial="hidden" animate="visible" className="space-y-3">
          <AnimatePresence initial={false}>
            {filteredNotifications.map((notif: any) => {
              const isUnread = !notif.read_at
              const meta = getNotificationMeta(notif.type || '')
              const isMarking = markingId === notif.id
              const isDeleting = deletingId === notif.id

              return (
                <motion.div
                  key={notif.id}
                  layout
                  variants={fadeUp}
                  exit={{ opacity: 0, scale: 0.95 }}
                  onClick={() => {
                    if (isUnread) markAsRead(notif.id)
                    const url = getNotificationRedirectUrl(notif.type || '', profile?.role)
                    navigate(url)
                  }}
                  className={`group relative p-5 rounded-2xl border transition-all cursor-pointer flex flex-col sm:flex-row sm:items-center justify-between gap-4 ${
                    isUnread
                      ? 'bg-gradient-to-r from-cyan-50/80 via-sky-50/40 to-white border-cyan-200 hover:border-cyan-300 hover:shadow-md hover:shadow-cyan-500/5'
                      : 'bg-white border-slate-200/80 hover:border-slate-300 hover:bg-slate-50/80'
```

```
                    }`}
                  >
                    {/* Left blue active bar */}
                    {isUnread && (
                      <span className="absolute left-0 top-0 bottom-0 w-1.5 bg-gradient-to-b from-cyan-500 to-blue-600 rounded-l
-2xl" />
                    )}

                    <div className="flex items-start gap-3.5 flex-1 min-w-0">
                      <div className="w-10 h-10 rounded-xl bg-slate-100/90 border border-slate-200/80 flex items-center justify-
center shrink-0 shadow-xs group-hover:scale-105 transition-transform">
                        {getCategoryIcon(meta.category)}
                      </div>
                      <div className="flex-1 min-w-0">
                        <div className="flex items-center gap-2 flex-wrap mb-1">
                          <span className={`text-[10px] font-bold uppercase tracking-wider px-2 py-0.5 rounded-md border ${meta.
color}`}>
                            {meta.label}
                          </span>
                          {isUnread && (
                            <span className="inline-flex items-center gap-1 text-[10px] font-bold text-cyan-800 bg-cyan-100 px-2
py-0.5 rounded-full border border-cyan-200">
                              <span className="w-1.5 h-1.5 rounded-full bg-cyan-600" />
                              New Alert
                            </span>
                          )}
                        </div>
                        <h4 className={`text-sm font-bold truncate ${isUnread ? 'text-slate-900' : 'text-slate-800'}`}>
                          {notif.title}
                        </h4>
                        <p className="text-xs text-slate-600 line-clamp-2 leading-relaxed mt-1 font-normal">
                          {notif.message}
                        </p>
                        <div className="flex items-center gap-3 mt-2 text-[11px] text-slate-400 font-medium">
                          <span className="inline-flex items-center gap-1">
                            <Clock className="w-3.5 h-3.5 text-slate-400" />
                            {formatSafeTime(notif.created_at)}
                          </span>
                        </div>
                      </div>
                    </div>

                    {/* Right Actions */}
                    <div className="flex items-center gap-2 shrink-0 sm:self-center" onClick={(e) => e.stopPropagation()}>
                      {isUnread && (
                        <Button
                          size="sm"
                          variant="ghost"
                          onClick={(e) => handleMarkAsRead(notif.id, e)}
                          disabled={isMarking}
                          className="text-xs font-bold text-cyan-700 hover:text-cyan-800 hover:bg-cyan-100/60 rounded-xl"
                        >
                          {isMarking ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <><Check className="w-3.5 h-3.5 mr-1"
/> Mark read</>}
                        </Button>
                      )}
                      <button
                        type="button"
                        onClick={(e) => handleDelete(notif.id, e)}
                        disabled={isDeleting}
                        className="p-2 rounded-xl text-slate-400 hover:text-rose-600 hover:bg-rose-50 transition-colors cursor-p
ointer"
                        title="Dismiss notification"
                      >
                        {isDeleting ? <Loader2 className="w-4 h-4 animate-spin text-rose-500" /> : <Trash2 className="w-4 h-4" />
}
                      </button>
                    </div>
                  </motion.div>
                )
              })}
            </AnimatePresence>
          </motion.div>
        )}
      </div>
    )

    const traineeNavLinks = [
      { to: '/trainee', label: 'Dashboard', icon: BarChart3 },
```

```
    { to: '/trainee/courses', label: 'Course Catalog', icon: Compass },
    { to: '/trainee/my-learning', label: 'My Learning', icon: BookOpen },
    { to: '/trainee/assessments', label: 'Assessments', icon: FileCheck },
    { to: '/trainee/notifications', label: 'Notifications', icon: Bell },
    { to: '/trainee/profile', label: 'Profile', icon: User },
  ]

  const adminNavLinks = [
    { to: '/admin', label: 'Overview', icon: LayoutDashboard },
    { to: '/admin/courses', label: 'Courses', icon: BookOpen },
    { to: '/admin/assessments', label: 'Assessments', icon: FileCheck },
    { to: '/admin/notifications', label: 'Notifications', icon: Bell },
    { to: '/admin/profile', label: 'Profile', icon: User },
  ]

  if (hideShell) {
    return content
  }

  if (profile?.role === 'trainer') {
    return (
      <TrainerLayout>
        {content}
      </TrainerLayout>
    )
  }

  const isAdmin = profile?.role === 'admin' || profile?.role === 'super_admin'

  return (
    <DashboardShell
      title="Notifications"
      icon={Bell}
      navLinks={isAdmin ? adminNavLinks : traineeNavLinks}
    >
      {content}
    </DashboardShell>
  )
}
```

src/components/ui/alert-dialog.tsx TSX 4.3 KB 140 Lines

```tsx
import * as React from "react"
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button"

const AlertDialog = AlertDialogPrimitive.Root

const AlertDialogTrigger = AlertDialogPrimitive.Trigger

const AlertDialogPortal = AlertDialogPrimitive.Portal

const AlertDialogOverlay = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-ink/50 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props}
    ref={ref}
  />
))
AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName

const AlertDialogContent = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Content>
>(({ className, ...props }, ref) => (
  <AlertDialogPortal>
    <AlertDialogOverlay />
    <AlertDialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border border-cyan-500/30 bg-[#070E20]/90 p-6 text-zinc-200 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props}
    />
  </AlertDialogPortal>
))
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName

const AlertDialogHeader = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col space-y-2 text-center sm:text-left",
      className
    )}
    {...props}
  />
)
AlertDialogHeader.displayName = "AlertDialogHeader"

const AlertDialogFooter = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
      className
    )}
    {...props}
  />
)
```

```
AlertDialogFooter.displayName = "AlertDialogFooter"

const AlertDialogTitle = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Title>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold", className)}
    {...props}
  />
))
AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName

const AlertDialogDescription = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Description>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-zinc-200/60", className)}
    {...props}
  />
))
AlertDialogDescription.displayName =
  AlertDialogPrimitive.Description.displayName

const AlertDialogAction = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Action>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Action>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Action
    ref={ref}
    className={cn(buttonVariants(), className)}
    {...props}
  />
))
AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName

const AlertDialogCancel = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Cancel>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Cancel>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Cancel
    ref={ref}
    className={cn(
      buttonVariants({ variant: "outline" }),
      "mt-2 sm:mt-0",
      className
    )}
    {...props}
  />
))
AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
}
```

src/components/ui/AnnouncementModal.tsx TSX 6.4 KB 177 Lines

```tsx
import React, { useEffect, useState } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { X, AlertTriangle } from 'lucide-react'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'

export function AnnouncementModal() {
  const { user, profile } = useAuth()
  const [announcements, setAnnouncements] = useState<any[]>([])
  const [isOpen, setIsOpen] = useState(false)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    async function fetchAnnouncements() {
      if (!user || !profile) {
        setLoading(false)
        return
      }

      if (profile.role === 'admin' || profile.role === 'super_admin') {
        setLoading(false)
        return
      }

      try {
        // Query announcements targeted to 'all' or the specific user's role
        const { data, error } = await supabase
          .from('announcements')
          .select('*')
          .eq('is_active', true)
          .in('target_audience', ['all', profile.role])
          .order('created_at', { ascending: false })
          .limit(10)

        if (error) {
          console.error('Error fetching announcements:', error)
          return
        }

        if (data && data.length > 0) {
          // Filter out dismissed announcements
          const unreadAnnouncements = data.filter((ann) => {
            const dismissKey = `dismissed_announcement_${ann.id}_${user.id}`
            return !localStorage.getItem(dismissKey)
          }).map((ann) => {
            const authorMatch = ann.content.match(/<!--AUTHOR:(.*?)-->/)
            const authorName = authorMatch ? authorMatch[1] : 'System Administrator'
            const cleanContent = ann.content.replace(/\n\n<!--AUTHOR:.*?-->/g, '')

            return {
              ...ann,
              content: cleanContent,
              authorName
            }
          })

          if (unreadAnnouncements.length > 0) {
            setAnnouncements(unreadAnnouncements)
            setIsOpen(true)
          }
        }
      } catch (err) {
        console.error(err)
      } finally {
        setLoading(false)
      }
    }

    fetchAnnouncements()
  }, [user, profile])

  const handleDismiss = (id: string) => {
    if (user) {
      localStorage.setItem(`dismissed_announcement_${id}_${user.id}`, 'true')
    }

```

```
    setAnnouncements(prev => {
      const updated = prev.filter(a => a.id !== id)
      if (updated.length === 0) {
        setIsOpen(false)
      }
      return updated
    })
  }

  const handleClose = (id: string) => {
    // Only temporarily remove from the array for this session
    setAnnouncements(prev => {
      const updated = prev.filter(a => a.id !== id)
      if (updated.length === 0) {
        setIsOpen(false)
      }
      return updated
    })
  }

  if (loading || announcements.length === 0) return null

  const currentAnnouncement = announcements[0]

  return (
    <AnimatePresence>
      {isOpen && currentAnnouncement && (
        <div className="fixed inset-0 z-[100] flex items-center justify-center p-4">
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            className="absolute inset-0 bg-black/60 backdrop-blur-sm"
          />
          <motion.div
            key={currentAnnouncement.id}
            initial={{ opacity: 0, scale: 0.95, y: 20 }}
            animate={{ opacity: 1, scale: 1, y: 0 }}
            exit={{ opacity: 0, scale: 0.95, y: -20 }}
            className="relative w-full max-w-lg bg-white rounded-3xl shadow-2xl overflow-hidden flex flex-col max-h-[90vh]"
          >
            <button
              onClick={() => handleClose(currentAnnouncement.id)}
              className="absolute top-4 right-4 p-2 rounded-full bg-gray-100 hover:bg-gray-200 text-gray-600 transition-colors
z-10"
            >
              <X className="w-5 h-5" />
            </button>

            {/* Header branding */}
            <div className="pt-8 pb-4 flex flex-col items-center border-b border-gray-100 px-6">
              <div className="flex items-center gap-3 mb-4">
                <img src="/logo.png" alt="Logo" className="w-12 h-12 object-contain" />
                <div className="text-2xl font-black tracking-tight text-gray-900 uppercase">
                  Capacity <span className="text-purple-600">Connect</span>
                </div>
              </div>
              <div className="w-full border-t border-gray-100 mt-2 pt-4">
                <div className="bg-red-50 text-red-700 border border-red-200 rounded-xl p-4 flex flex-col items-center text-ce
nter">
                  <AlertTriangle className="w-8 h-8 mb-2" />
                  <h2 className="font-bold text-xl uppercase tracking-wider">{currentAnnouncement.title}</h2>
                  <p className="text-xs font-semibold uppercase mt-1 opacity-80">
                    Important update for {currentAnnouncement.target_audience === 'all' ? 'All Users' : `${currentAnnouncemen
t.target_audience}s`}
                  </p>
                </div>
              </div>
              {announcements.length > 1 && (
                <div className="mt-4 text-xs font-bold text-purple-600 bg-purple-50 px-3 py-1 rounded-full">
                  1 of {announcements.length} Updates
                </div>
              )}
            </div>

            {/* Content */}
            <div className="p-6 overflow-y-auto">
              <div className="prose prose-sm prose-gray max-w-none whitespace-pre-wrap text-gray-700 text-center leading-relax
ed">
```

```
152 |                 {currentAnnouncement.content}
153 |               </div>
154 |             </div>
155 |
156 |             {/* Footer */}
157 |             <div className="bg-gray-50 border-t border-gray-100 p-4 px-6 flex flex-col sm:flex-row items-center justify-betwee
n gap-4">
158 |               <div className="text-xs text-gray-500 flex flex-col gap-1 text-center sm:text-left">
159 |                 <span className="font-semibold text-gray-700">
160 |                   Posted by {currentAnnouncement.authorName}
161 |                 </span>
162 |                 <span>{new Date(currentAnnouncement.created_at).toLocaleString()}</span>
163 |               </div>
164 |               <button
165 |                 onClick={() => handleDismiss(currentAnnouncement.id)}
166 |                 className="w-full sm:w-auto px-6 py-2.5 bg-gray-900 hover:bg-black text-white rounded-xl font-medium transitio
n-colors"
167 |               >
168 |                 Noted
169 |               </button>
170 |             </div>
171 |           </motion.div>
172 |         </div>
173 |       )}
174 |     </AnimatePresence>
175 |   )
176 | }
177 |
```

src/components/ui/AnnouncementsFeed.tsx TSX 4.5 KB 113 Lines

```tsx
import React, { useEffect, useState } from 'react'
import { Megaphone, Loader2, X } from 'lucide-react'
import { supabase } from '@/lib/supabase'
import { useAuth } from '@/hooks/useAuth'

export function AnnouncementsFeed() {
  const { user, profile } = useAuth()
  const [announcements, setAnnouncements] = useState<any[]>([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    async function fetchAnnouncements() {
      if (!profile || !user) return
      setLoading(true)
      try {
        let query = supabase
          .from('announcements')
          .select('*')
          .eq('is_active', true)

        if (profile.role !== 'admin' && profile.role !== 'super_admin') {
          query = query.in('target_audience', ['all', profile.role])
        }

        const { data, error } = await query
          .order('created_at', { ascending: false })
          .limit(20)

        if (error) throw error

        if (data) {
          // Fetch author names manually since we can't join admins directly if author_id references auth.users
          const authorIds = [...new Set(data.map(a => a.author_id).filter(Boolean))]
          const { data: admins } = await supabase
            .from('admins')
            .select('id, full_name')
            .in('id', authorIds)

          const adminMap = admins?.reduce((acc: Record<string, string>, admin) => {
            acc[admin.id] = admin.full_name
            return acc
          }, {} as Record<string, string>) || {}

          setAnnouncements(data.map(a => ({
            ...a,
            authorName: a.author_id ? adminMap[a.author_id] : 'System Administrator'
          })))
        }
      } catch (err) {
        console.error('Failed to fetch announcements feed:', err)
      } finally {
        setLoading(false)
      }
    }
    fetchAnnouncements()
  }, [profile, user])

  const handleClear = (id: string) => {
    if (!user) return;
    localStorage.setItem(`cleared_announcement_${id}_${user.id}`, 'true');
    setAnnouncements(prev => prev.filter(a => a.id !== id));
  }

  if (loading) {
    return (
      <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-xl shadow-slate-200/40 flex justify-center items-center py-10">
        <Loader2 className="w-6 h-6 animate-spin text-cyan-600" />
      </div>
    )
  }

  if (announcements.length === 0) return null

  return (
```

```
    <div className="bg-white border border-slate-200/90 rounded-3xl p-6 shadow-xl shadow-slate-200/40 space-y-4">
      <div className="flex items-center gap-2.5 pb-4 border-b border-slate-100">
        <div className="p-2 bg-cyan-50 text-cyan-700 border border-cyan-200 rounded-xl">
          <Megaphone className="w-5 h-5" />
        </div>
        <div>
          <h3 className="text-base font-bold text-slate-900">Recent Announcements</h3>
          <p className="text-xs text-slate-500 font-medium">Important platform updates and notices</p>
        </div>
      </div>

      <div className="space-y-3">
        {announcements.map((ann) => (
          <div key={ann.id} className="group p-4 bg-slate-50/80 border border-slate-200/70 rounded-2xl flex flex-col gap-2 rel
ative transition-all hover:bg-white hover:border-cyan-500/40 hover:shadow-md">
            <button
              onClick={() => handleClear(ann.id)}
              className="absolute top-3 right-3 p-1.5 rounded-lg text-slate-400 hover:text-rose-600 hover:bg-rose-50 opacity-0
group-hover:opacity-100 transition-all cursor-pointer"
              title="Clear announcement"
            >
              <X className="w-4 h-4" />
            </button>
            <div className="flex items-center justify-between pr-8">
              <h4 className="text-sm font-bold text-slate-900">{ann.title}</h4>
              <span className="text-[10px] font-bold text-cyan-800 bg-cyan-50 px-2.5 py-0.5 rounded-full border border-cyan-20
0 uppercase tracking-wide">
                {ann.target_audience}
              </span>
            </div>
            <p className="text-xs sm:text-sm text-slate-600 whitespace-pre-wrap leading-relaxed">{ann.content}</p>
            <div className="text-[11px] text-slate-500 mt-1 flex justify-between font-medium">
              <span>Posted by {ann.authorName || 'System Administrator'}</span>
              <span>{new Date(ann.created_at).toLocaleDateString()}</span>
            </div>
          </div>
        ))}
      </div>
    </div>
  )
}

```

src/components/ui/badge.tsx TSX 1.5 KB 47 Lines

```tsx
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

const badgeVariants = cva(
  "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none focus:ring-2 focus:ring-purple-500/20",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-gradient-to-r from-purple-600 to-pink-500 text-white shadow-sm",
        gradient:
          "border-transparent bg-gradient-to-r from-purple-600 via-pink-500 to-orange-500 text-white shadow-sm",
        purple:
          "border-cyan-500/30 bg-cyan-950/30 text-cyan-400",
        pink:
          "border-pink-200 bg-pink-50 text-pink-700",
        orange:
          "border-orange-200 bg-orange-50 text-orange-700",
        coral:
          "border-coral-200 bg-coral-50 text-coral-700",
        secondary:
          "border-transparent bg-purple-100 text-purple-800 hover:bg-purple-200",
        destructive:
          "border-transparent bg-red-600 text-white shadow hover:bg-red-700",
        outline: "border-cyan-500/30 text-zinc-200",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

export interface BadgeProps
  extends React.HTMLAttributes<HTMLDivElement>,
    VariantProps<typeof badgeVariants> {}

function Badge({ className, variant, ...props }: BadgeProps) {
  return (
    <div className={cn(badgeVariants({ variant }), className)} {...props} />
  )
}

export { Badge, badgeVariants }

```

src/components/ui/button.tsx TSX 2.7 KB 68 Lines

```tsx
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transition-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default:
          "bg-gradient-to-r from-cyan-600 via-blue-600 to-amber-600 text-white shadow-md shadow-cyan-600/20 hover:opacity-95 hover:shadow-lg transition-all",
        primary:
          "bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white shadow-md shadow-cyan-600/20 hover:opacity-95 transition-all",
        purple:
          "bg-indigo-600 hover:bg-indigo-700 text-white shadow-md shadow-indigo-600/20 transition-all",
        pink:
          "bg-sky-600 hover:bg-sky-700 text-white shadow-md shadow-sky-600/20 transition-all",
        orange:
          "bg-amber-600 hover:bg-amber-700 text-white shadow-md shadow-amber-600/20 transition-all",
        coral:
          "bg-rose-600 hover:bg-rose-700 text-white shadow-md shadow-rose-600/20 transition-all",
        destructive:
          "bg-red-600 text-white shadow-sm hover:bg-red-700 transition-all",
        outline:
          "border border-slate-200 bg-white shadow-xs hover:bg-slate-50 hover:border-slate-300 text-slate-800 transition-all",
        secondary:
          "bg-cyan-50 text-cyan-800 hover:bg-cyan-100 border border-cyan-200 transition-all",
        ghost: "hover:bg-slate-100 hover:text-slate-900 text-slate-700 transition-all",
        link: "text-cyan-600 underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2",
        sm: "h-8 rounded-md px-3 text-xs",
        lg: "h-10 rounded-md px-8",
        icon: "h-9 w-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {
  asChild?: boolean
}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, asChild = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button"
    return (
      <Comp
        className={cn(buttonVariants({ variant, size, className }))}
        ref={ref}
        {...props}
      />
    )
  }
)
Button.displayName = "Button"

export { Button, buttonVariants }

```

src/components/ui/card.tsx TSX 1.8 KB 77 Lines

```tsx
import * as React from "react"

import { cn } from "@/lib/utils"

const Card = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn(
      "rounded-xl border border-cyan-500/30 bg-[#070E20]/90 text-zinc-200 shadow-sm",
      className
    )}
    {...props}
  />
))
Card.displayName = "Card"

const CardHeader = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex flex-col space-y-1.5 p-6", className)}
    {...props}
  />
))
CardHeader.displayName = "CardHeader"

const CardTitle = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("font-semibold leading-none tracking-tight", className)}
    {...props}
  />
))
CardTitle.displayName = "CardTitle"

const CardDescription = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm text-zinc-200/60", className)}
    {...props}
  />
))
CardDescription.displayName = "CardDescription"

const CardContent = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
))
CardContent.displayName = "CardContent"

const CardFooter = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex items-center p-6 pt-0", className)}
    {...props}
  />
))
CardFooter.displayName = "CardFooter"

```

```
76 | export { Card, CardHeader, CardFooter, CardTitle, CardDescription, CardContent }
77 |
```

src/components/ui/ConfirmModal.tsx TSX 2.7 KB 78 Lines

```tsx
import React, { useEffect, useState } from 'react'
import { createPortal } from 'react-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { AlertTriangle, X } from 'lucide-react'
import { Button } from '@/components/ui/button'

interface ConfirmModalProps {
  isOpen: boolean
  title: string
  message: string
  onConfirm: () => void
  onCancel: () => void
}

export function ConfirmModal({ isOpen, title, message, onConfirm, onCancel }: ConfirmModalProps) {
  const [mounted, setMounted] = useState(false)

  useEffect(() => {
    setMounted(true)
  }, [])

  if (!mounted) return null

  return createPortal(
    <AnimatePresence>
      {isOpen && (
        <div className="fixed inset-0 z-[9999] flex items-center justify-center p-4 pointer-events-auto">
          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            onClick={onCancel}
            className="absolute inset-0 bg-black/60 backdrop-blur-sm"
          />
          <motion.div
            initial={{ opacity: 0, scale: 0.95, y: 20 }}
            animate={{ opacity: 1, scale: 1, y: 0 }}
            exit={{ opacity: 0, scale: 0.95, y: 20 }}
            className="relative w-full max-w-md bg-[#070E20] border border-cyan-500/30 rounded-3xl shadow-2xl overflow-hidden flex flex-col"
          >
            <button
              onClick={onCancel}
              className="absolute top-4 right-4 p-2 rounded-full hover:bg-cyan-950/30 text-zinc-200/70 hover:text-cyan-400 transition-colors z-10"
            >
              <X className="w-5 h-5" />
            </button>

            <div className="p-6 pt-8 text-center flex flex-col items-center">
              <div className="w-16 h-16 rounded-full bg-rose-500/10 text-rose-500 flex items-center justify-center mb-4">
                <AlertTriangle className="w-8 h-8" />
              </div>
              <h2 className="text-xl font-bold text-zinc-200 mb-2">{title}</h2>
              <p className="text-zinc-200/70 text-sm">{message}</p>
            </div>

            <div className="bg-[#020612]/50 p-4 flex items-center justify-end gap-3 border-t border-cyan-500/20">
              <Button
                variant="outline"
                onClick={onCancel}
                className="border-cyan-500/30 text-zinc-300 hover:text-white rounded-xl"
              >
                Cancel
              </Button>
              <Button
                onClick={onConfirm}
                className="bg-rose-500 hover:bg-rose-600 text-white rounded-xl"
              >
                Confirm
              </Button>
            </div>
          </motion.div>
        </div>
      )}
    </AnimatePresence>,
```

```
    document.body
  )
}

```

src/components/ui/dialog.tsx TSX 3.7 KB 121 Lines

```tsx
import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"

import { cn } from "@/lib/utils"

const Dialog = DialogPrimitive.Root

const DialogTrigger = DialogPrimitive.Trigger

const DialogPortal = DialogPrimitive.Portal

const DialogClose = DialogPrimitive.Close

const DialogOverlay = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-ink/50 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
      className
    )}
    {...props}
  />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Content>
>(({ className, children, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border border-cyan-500/30 bg-[#070E20]/90 p-6 text-zinc-200 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props}
    >
      {children}
      <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 transition-opacity hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ink/20 disabled:pointer-events-none">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </DialogPrimitive.Close>
    </DialogPrimitive.Content>
  </DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col space-y-1.5 text-center sm:text-left",
      className
    )}
    {...props}
  />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({
  className,
  ...props
```

```
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
      className
    )}
    {...props}
  />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Title>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
    className={cn(
      "text-lg font-semibold leading-none tracking-tight",
      className
    )}
    {...props}
  />
))
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Description>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-zinc-200/60", className)}
    {...props}
  />
))
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogTrigger,
  DialogClose,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
}

```

src/components/ui/dropdown-menu.tsx TSX 8.6 KB 202 Lines

```tsx
"use client"

import * as React from "react"
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const DropdownMenu = DropdownMenuPrimitive.Root

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger

const DropdownMenuGroup = DropdownMenuPrimitive.Group

const DropdownMenuPortal = DropdownMenuPrimitive.Portal

const DropdownMenuSub = DropdownMenuPrimitive.Sub

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup

const DropdownMenuSubTrigger = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubTrigger> & {
    inset?: boolean
  }
>(({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center gap-2 rounded-lg px-2.5 py-2 text-sm text-slate-700 dark:text-slate-200 hover:bg-slate-100 dark:hover:bg-slate-800 hover:text-slate-900 dark:hover:text-white outline-none focus:bg-slate-100 dark:focus:bg-slate-800 focus:text-slate-900 data-[state=open]:bg-slate-100 dark:data-[state=open]:bg-slate-800 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0 font-medium",
      inset && "pl-8",
      className
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
))
DropdownMenuSubTrigger.displayName =
  DropdownMenuPrimitive.SubTrigger.displayName

const DropdownMenuSubContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-xl border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900 p-1.5 text-slate-800 dark:text-slate-100 shadow-xl data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-dropdown-menu-content-transform-origin]",
      className
    )}
    {...props}
  />
))
DropdownMenuSubContent.displayName =
  DropdownMenuPrimitive.SubContent.displayName

const DropdownMenuContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Content>
>(({ className, sideOffset = 4, ...props }, ref) => (
  <DropdownMenuPrimitive.Portal>
    <DropdownMenuPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 max-h-[var(--radix-dropdown-menu-content-available-height)] min-w-[10rem] overflow-y-auto overflow-x-hidden rounded-xl border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900 p-1.5 text-slate-800 dark:text-slate-100 shadow-xl",
```

```
        "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-dropdown-menu-content-transform-origin]",
        className
      )}
      {...props}
    />
  </DropdownMenuPrimitive.Portal>
))
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName

const DropdownMenuItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Item> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-pointer select-none items-center gap-2 rounded-lg px-3 py-2 text-sm text-slate-700 dark:text-slate-200 outline-none transition-colors hover:bg-slate-100 dark:hover:bg-slate-800 hover:text-slate-900 dark:hover:text-white focus:bg-slate-100 dark:focus:bg-slate-800 focus:text-slate-900 dark:focus:text-white data-[disabled]:pointer-events-none data-[disabled]:opacity-50 [&>svg]:size-4 [&>svg]:shrink-0 font-medium",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName

const DropdownMenuCheckboxItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.CheckboxItem>
>(({ className, children, checked, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-pointer select-none items-center rounded-lg py-2 pl-8 pr-3 text-sm text-slate-700 dark:text-slate-200 outline-none transition-colors hover:bg-slate-100 dark:hover:bg-slate-800 hover:text-slate-900 dark:hover:text-white focus:bg-slate-100 dark:focus:bg-slate-800 focus:text-slate-900 dark:focus:text-white data-[disabled]:pointer-events-none data-[disabled]:opacity-50 font-medium",
      className
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2.5 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4 text-cyan-600 dark:text-cyan-400" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
))
DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName

const DropdownMenuRadioItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.RadioItem>
>(({ className, children, ...props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-pointer select-none items-center rounded-lg py-2 pl-8 pr-3 text-sm text-slate-700 dark:text-slate-200 outline-none transition-colors hover:bg-slate-100 dark:hover:bg-slate-800 hover:text-slate-900 dark:hover:text-white focus:bg-slate-100 dark:focus:bg-slate-800 focus:text-slate-900 dark:focus:text-white data-[disabled]:pointer-events-none data-[disabled]:opacity-50 font-medium",
      className
    )}
    {...props}
  >
    <span className="absolute left-2.5 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-cyan-600 dark:fill-cyan-400 text-cyan-600 dark:text-cyan-400" />
      </DropdownMenuPrimitive.ItemIndicator>
```

```
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
))
DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName

const DropdownMenuLabel = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Label> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}
    className={cn(
      "px-3 py-2 text-xs font-bold uppercase tracking-wider text-slate-500 dark:text-slate-400",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName

const DropdownMenuSeparator = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1.5 h-px bg-slate-200 dark:bg-slate-800", className)}
    {...props}
  />
))
DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName

const DropdownMenuShortcut = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props}
    />
  )
}
DropdownMenuShortcut.displayName = "DropdownMenuShortcut"

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
}

```

src/components/ui/form.tsx TSX 4.1 KB 177 Lines

```tsx
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { Slot } from "@radix-ui/react-slot"
import {
  Controller,
  FormProvider,
  useFormContext,
  type ControllerProps,
  type FieldPath,
  type FieldValues,
} from "react-hook-form"

import { cn } from "@/lib/utils"
import { Label } from "@/components/ui/label"

const Form = FormProvider

type FormFieldContextValue<
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
> = {
  name: TName
}

const FormFieldContext = React.createContext<FormFieldContextValue | null>(null)

const FormField = <
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
>({
  ...props
}: ControllerProps<TFieldValues, TName>) => {
  return (
    <FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  )
}

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState, formState } = useFormContext()

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>")
  }

  if (!itemContext) {
    throw new Error("useFormField should be used within <FormItem>")
  }

  const fieldState = getFieldState(fieldContext.name, formState)

  const { id } = itemContext

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  }
}

type FormItemContextValue = {
  id: string
}

const FormItemContext = React.createContext<FormItemContextValue | null>(null)

const FormItem = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => {
```

```
  const id = React.useId()

  return (
    <FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>
  )
})
FormItem.displayName = "FormItem"

const FormLabel = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root>
>(({ className, ...props }, ref) => {
  const { error, formItemId } = useFormField()

  return (
    <Label
      ref={ref}
      className={cn(error && "text-red-600", className)}
      htmlFor={formItemId}
      {...props}
    />
  )
})
FormLabel.displayName = "FormLabel"

const FormControl = React.forwardRef<
  React.ElementRef<typeof Slot>,
  React.ComponentPropsWithoutRef<typeof Slot>
>(({ ...props }, ref) => {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField()

  return (
    <Slot
      ref={ref}
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    />
  )
})
FormControl.displayName = "FormControl"

const FormDescription = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField()

  return (
    <p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-[0.8rem] text-zinc-200/50", className)}
      {...props}
    />
  )
})
FormDescription.displayName = "FormDescription"

const FormMessage = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, children, ...props }, ref) => {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message ?? "") : children

  if (!body) {
    return null
  }

  return (
    <p
```

```
      ref={ref}
      id={formMessageId}
      className={cn("text-[0.8rem] font-medium text-red-600", className)}
      {...props}
    >
      {body}
    </p>
  )
})
FormMessage.displayName = "FormMessage"

export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}

```

src/components/ui/ImageCropperModal.tsx TSX 7.2 KB 208 Lines

```tsx
import React, { useState, useCallback, useEffect } from 'react'
import Cropper from 'react-easy-crop'
import { Dialog, DialogContent, DialogFooter, DialogHeader, DialogTitle } from './dialog'
import { Button } from './button'
import { getCroppedImg } from '@/utils/cropImage'
import { Loader2, ZoomIn, RotateCcw, Crop } from 'lucide-react'
import { toast } from 'sonner'

interface ImageCropperModalProps {
  isOpen: boolean
  imageFile?: File | null
  imageUrl?: string | null
  onClose: () => void
  onCropComplete: (croppedFile: File) => void
  aspectRatio?: number
  cropShape?: 'rect' | 'round'
  title?: string
}

export function ImageCropperModal({
  isOpen,
  imageFile,
  imageUrl,
  onClose,
  onCropComplete,
  aspectRatio = 1, // Defaulting to 1:1 square for profile photos
  cropShape = 'rect',
  title = 'Adjust & Crop Photo'
}: ImageCropperModalProps) {
  const [crop, setCrop] = useState({ x: 0, y: 0 })
  const [zoom, setZoom] = useState(1)
  const [croppedAreaPixels, setCroppedAreaPixels] = useState<any>(null)
  const [isProcessing, setIsProcessing] = useState(false)
  const [imageSrc, setImageSrc] = useState<string>('')
  const [loadingImage, setLoadingImage] = useState(false)

  useEffect(() => {
    let active = true
    let blobUrlToRevoke: string | null = null

    if (!isOpen) {
      setCrop({ x: 0, y: 0 })
      setZoom(1)
      setCroppedAreaPixels(null)
      return
    }

    if (imageFile) {
      const url = URL.createObjectURL(imageFile)
      blobUrlToRevoke = url
      setImageSrc(url)
      return
    }

    if (imageUrl) {
      if (imageUrl.startsWith('data:') || imageUrl.startsWith('blob:')) {
        setImageSrc(imageUrl)
        return
      }

      setLoadingImage(true)
      fetch(imageUrl, { mode: 'cors' })
        .then(res => res.blob())
        .then(blob => {
          if (active) {
            const url = URL.createObjectURL(blob)
            blobUrlToRevoke = url
            setImageSrc(url)
          }
        })
        .catch((err) => {
          console.warn('Direct fetch failed, falling back to direct URL:', err)
          if (active) setImageSrc(imageUrl)
        })
        .finally(() => {
          if (active) setLoadingImage(false)
```

```
        })
    } else {
      setImageSrc('')
    }

    return () => {
      active = false
      if (blobUrlToRevoke) URL.revokeObjectURL(blobUrlToRevoke)
    }
  }, [imageFile, imageUrl, isOpen])

  const onCropChange = (crop: { x: number; y: number }) => setCrop(crop)
  const onZoomChange = (zoom: number) => setZoom(zoom)

  const handleCropComplete = useCallback((_croppedArea: any, croppedAreaPixels: any) => {
    setCroppedAreaPixels(croppedAreaPixels)
  }, [])

  const handleSave = async () => {
    if (!imageSrc) return
    setIsProcessing(true)
    try {
      const fileName = imageFile?.name || 'profile_photo_adjusted.jpg'
      // If user hasn't moved the crop, compute fallback square crop

      const pixels = croppedAreaPixels || { x: 0, y: 0, width: 400, height: 400 }
      const croppedImage = await getCroppedImg(imageSrc, pixels, fileName)
      onCropComplete(croppedImage)
    } catch (e: any) {
      console.error('Error cropping image:', e)
      toast.error('Failed to crop image: ' + (e?.message || 'Error processing crop'))
    } finally {
      setIsProcessing(false)
    }
  }

  const handleReset = () => {
    setCrop({ x: 0, y: 0 })
    setZoom(1)
  }

  return (
    <Dialog open={isOpen} onOpenChange={(open) => !open && onClose()}>
      <DialogContent className="sm:max-w-[650px] p-0 overflow-hidden bg-slate-900 border-slate-700 text-white shadow-2xl">
        <DialogHeader className="p-5 pb-3 border-b border-slate-800 flex flex-row items-center justify-between">
          <DialogTitle className="text-lg font-bold text-white flex items-center gap-2">
            <Crop className="w-5 h-5 text-cyan-400" />
            {title}
          </DialogTitle>
        </DialogHeader>

        <div className="p-6 space-y-4">
          <div className="relative w-full h-[360px] bg-slate-950 rounded-2xl overflow-hidden border border-slate-800 shadow-in
ner">

            {imageSrc ? (
              <Cropper
                image={imageSrc}
                crop={crop}
                zoom={zoom}
                minZoom={0.5}
                maxZoom={4}
                aspect={aspectRatio}
                cropShape={cropShape}
                showGrid={true}
                restrictPosition={false}
                onCropChange={onCropChange}
                onCropComplete={handleCropComplete}
                onZoomChange={onZoomChange}
              />
            ) : (
              <div className="absolute inset-0 flex items-center justify-center text-slate-500 text-sm">
                No image selected
              </div>
            )}
          </div>

          <p className="text-xs text-slate-400 text-center">
            Drag image to reposition • Use the slider to zoom in/out
          </p>
```

```
154 |
155 |           <div className="flex items-center gap-3 bg-slate-800/60 p-3 rounded-xl border border-slate-700/60">
156 |             <div className="flex items-center gap-2 text-xs font-semibold text-slate-300 shrink-0">
157 |               <ZoomIn className="w-4 h-4 text-cyan-400" />
158 |               Zoom
159 |             </div>
160 |             <input
161 |               type="range"
162 |               value={zoom}
163 |               min={0.5}
164 |               max={4}
165 |               step={0.05}
166 |               aria-labelledby="Zoom"
167 |               onChange={(e) => setZoom(Number(e.target.value))}
168 |               className="flex-1 h-2 bg-slate-700 rounded-lg appearance-none cursor-pointer accent-cyan-400"
169 |             />
170 |             <span className="text-xs font-mono text-cyan-300 w-10 text-right">{zoom.toFixed(1)}x</span>
171 |             <Button
172 |               type="button"
173 |               variant="ghost"
174 |               size="sm"
175 |               onClick={handleReset}
176 |               className="text-slate-400 hover:text-white hover:bg-slate-700 h-8 px-2 text-xs rounded-lg cursor-pointer"
177 |               title="Reset Zoom & Position"
178 |             >
179 |               <RotateCcw className="w-3.5 h-3.5 mr-1" /> Reset
180 |             </Button>
181 |           </div>
182 |         </div>
183 |
184 |         <DialogFooter className="p-5 pt-3 bg-slate-950/80 border-t border-slate-800 flex items-center justify-end gap-3">
185 |           <Button
186 |             type="button"
187 |             variant="outline"
188 |             onClick={onClose}
189 |             disabled={isProcessing}
190 |             className="border-slate-700 bg-slate-800 text-slate-200 hover:bg-slate-700 hover:text-white font-semibold px-5 rou
nded-xl cursor-pointer"
191 |           >
192 |             Cancel
193 |           </Button>
194 |           <Button
195 |             type="button"
196 |             onClick={handleSave}
197 |             disabled={isProcessing || !imageSrc}
198 |             className="bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-600 hover:to-blue-700 text-white font-bold p
x-6 shadow-md rounded-xl cursor-pointer"
199 |           >
200 |             {isProcessing && <Loader2 className="w-4 h-4 mr-2 animate-spin" />}
201 |             Save & Apply Photo
202 |           </Button>
203 |         </DialogFooter>
204 |       </DialogContent>
205 |     </Dialog>
206 |   )
207 | }
208 |
```

src/components/ui/input.tsx TSX 0.8 KB 23 Lines

```tsx
import * as React from "react"

import { cn } from "@/lib/utils"

const Input = React.forwardRef<HTMLInputElement, React.ComponentProps<"input">>(
  ({ className, type, ...props }, ref) => {
    return (
      <input
        type={type}
        className={cn(
          "flex h-9 w-full rounded-md border border-slate-200 bg-white px-3 py-1 text-base text-slate-900 shadow-xs transition-colors file:border-0 file:bg-transparent file:text-sm file:font-medium file:text-slate-900 placeholder:text-slate-400 focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-cyan-500 disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
          className
        )}
        ref={ref}
        {...props}
      />
    )
  }
)
Input.displayName = "Input"

export { Input }

```

src/components/ui/label.tsx TSX 0.7 KB 25 Lines

```tsx
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
)

const Label = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root> &
    VariantProps<typeof labelVariants>
>(({ className, ...props }, ref) => (
  <LabelPrimitive.Root
    ref={ref}
    className={cn(labelVariants(), className)}
    {...props}
  />
))
Label.displayName = LabelPrimitive.Root.displayName

export { Label }

```

src/components/ui/MaterialPreviewDialog.tsx TSX 7.0 KB 133 Lines

```tsx
import React from 'react'
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog'
import { Button } from '@/components/ui/button'
import { Loader2, FileText, Download, ExternalLink } from 'lucide-react'

interface Material {
  id?: string
  file_name: string
  mime_type?: string | null
  storage_path?: string | null
}

interface MaterialPreviewDialogProps {
  material: Material | null
  previewUrl: string | null
  onClose: () => void
  onDownload?: () => void
}

function detectMimeFromPath(path: string | null | undefined): string | null {
  if (!path) return null
  const lower = path.toLowerCase().split('?')[0]
  if (lower.endsWith('.pdf')) return 'application/pdf'
  if (lower.endsWith('.mp4') || lower.endsWith('.webm') || lower.endsWith('.mov')) return 'video/mp4'
  if (lower.endsWith('.png')) return 'image/png'
  if (lower.endsWith('.jpg') || lower.endsWith('.jpeg')) return 'image/jpeg'
  if (lower.endsWith('.gif')) return 'image/gif'
  if (lower.endsWith('.webp')) return 'image/webp'
  if (lower.endsWith('.doc') || lower.endsWith('.docx')) return 'application/msword'
  if (lower.endsWith('.ppt') || lower.endsWith('.pptx')) return 'application/vnd.ms-powerpoint'
  if (lower.endsWith('.xls') || lower.endsWith('.xlsx')) return 'application/vnd.ms-excel'
  if (lower.endsWith('.txt')) return 'text/plain'
  return null
}

export function MaterialPreviewDialog({ material, previewUrl, onClose, onDownload }: MaterialPreviewDialogProps) {
  // Detect from storage_path (reliable, no query string noise) then fall back to URL

  const mime = material?.mime_type
    || detectMimeFromPath(material?.storage_path)
    || detectMimeFromPath(previewUrl)

  const isVideo = mime?.startsWith('video/')
  const isImage = mime?.startsWith('image/')
  const isPdf = mime === 'application/pdf'
  const isOffice = mime === 'application/msword' || mime === 'application/vnd.ms-powerpoint' || mime === 'application/vnd.ms-excel' || mime?.includes('openxmlformats-officedocument')
  const isText = mime === 'text/plain'

  // Use embedded viewer for images, videos, PDFs, Text and Office docs

  const canEmbedInline = isVideo || isImage || isPdf || isOffice || isText

  return (
    <Dialog open={!!material} onOpenChange={(open) => !open && onClose()}>
      <DialogContent className="max-w-4xl w-[90vw] h-[80vh] flex flex-col bg-white dark:bg-[#0B101D] border border-slate-200 dark:border-slate-800 p-0 overflow-hidden shadow-2xl rounded-2xl">
        <DialogHeader className="p-4 border-b border-slate-200 dark:border-slate-800 shrink-0 flex flex-row items-center justify-between bg-slate-50/80 dark:bg-slate-900/80">
          <DialogTitle className="text-slate-800 dark:text-slate-100 flex items-center gap-2 truncate text-sm sm:text-base font-bold">
            <FileText className="w-4 h-4 shrink-0 text-purple-600 dark:text-purple-400" />
            {material?.file_name}
          </DialogTitle>
          <div className="flex items-center gap-2 shrink-0 ml-4">
            {previewUrl && !canEmbedInline && (
              <Button
                variant="ghost"
                size="sm"
                className="text-slate-600 hover:text-slate-900 dark:text-slate-300 dark:hover:text-white"
                onClick={() => window.open(previewUrl, '_blank', 'noopener,noreferrer')}
              >
                <ExternalLink className="w-4 h-4 mr-1.5" /> Open in New Tab
              </Button>
            )}
            {onDownload && (
```

```
              <Button
                variant="ghost"
                size="sm"
                className="text-slate-600 hover:text-slate-900 dark:text-slate-300 dark:hover:text-white"
                onClick={onDownload}
              >
                <Download className="w-4 h-4 mr-1.5" /> Download
              </Button>
            )}
          </div>
        </DialogHeader>

        <div className="flex-1 overflow-hidden bg-slate-100/50 dark:bg-[#070B14] relative flex items-center justify-center p-
4">
          {!previewUrl ? (
            <div className="flex flex-col items-center justify-center text-slate-400 dark:text-slate-500">
              <Loader2 className="w-8 h-8 animate-spin mb-4 text-purple-600 dark:text-purple-400" />
              <p className="text-sm font-medium">Loading preview...</p>
            </div>
          ) : isVideo ? (
            <video controls autoPlay className="w-full h-full object-contain rounded-xl shadow-lg bg-black" src={previewUrl} /
>
          ) : isImage ? (
            <div className="w-full h-full flex items-center justify-center p-2">
              <img
                src={previewUrl}
                alt={material?.file_name}
                className="max-w-full max-h-full object-contain rounded-xl shadow-md bg-white p-2 border border-slate-200 dar
k:border-slate-700"
              />
            </div>
          ) : isPdf ? (
            <iframe src={`${previewUrl}#view=FitH`} className="w-full h-full border-0 rounded-xl shadow-md bg-white" title={ma
terial?.file_name} />
          ) : isText ? (
            <iframe src={previewUrl} className="w-full h-full border-0 bg-white dark:bg-[#0D1424] rounded-xl shadow-md p-4 tex
t-slate-800 dark:text-slate-100" title={material?.file_name} />
          ) : isOffice ? (
            <iframe src={`https://docs.google.com/viewer?url=${encodeURIComponent(previewUrl)}&embedded=true`} className="w-fu
ll h-full border-0 rounded-xl shadow-md bg-white" title={material?.file_name} />
          ) : (
            /* For other unrecognized docs show a fallback */

            <div className="flex flex-col items-center justify-center text-slate-500 dark:text-slate-400 gap-4 p-8 text-cente
r">
              <div className="w-16 h-16 rounded-2xl bg-purple-100 dark:bg-purple-950/50 border border-purple-200 dark:border-p
urple-800 flex items-center justify-center">
                <FileText className="w-8 h-8 text-purple-600 dark:text-purple-400" />
              </div>
              <div>
                <p className="text-sm font-semibold text-slate-800 dark:text-slate-200 mb-1">{material?.file_name}</p>
                <p className="text-xs text-slate-500 dark:text-slate-400">This file type cannot be previewed inline.</p>
              </div>
              <Button
                onClick={() => window.open(previewUrl, '_blank', 'noopener,noreferrer')}
                className="bg-gradient-to-r from-purple-600 to-indigo-600 text-white rounded-xl px-5 py-2.5 text-sm font-semib
old hover:opacity-90 transition-all flex items-center gap-2 shadow-md"
              >
                <ExternalLink className="w-4 h-4" /> Open File
              </Button>
              {onDownload && (
                <Button variant="outline" onClick={onDownload} className="rounded-xl text-sm flex items-center gap-2 border-sl
ate-300 dark:border-slate-700">
                  <Download className="w-4 h-4" /> Download Instead
                </Button>
              )}
            </div>
          )}
        </div>
      </DialogContent>
    </Dialog>
  )
}
```

src/components/ui/progress.tsx TSX 0.8 KB 27 Lines

```tsx
import * as React from "react"
import * as ProgressPrimitive from "@radix-ui/react-progress"

import { cn } from "@/lib/utils"

const Progress = React.forwardRef<
  React.ElementRef<typeof ProgressPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof ProgressPrimitive.Root>
>(({ className, value, ...props }, ref) => (
  <ProgressPrimitive.Root
    ref={ref}
    className={cn(
      "relative h-2 w-full overflow-hidden rounded-full bg-primary/20",
      className
    )}
    {...props}
  >
    <ProgressPrimitive.Indicator
      className="h-full w-full flex-1 bg-primary transition-all"
      style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
    />
  </ProgressPrimitive.Root>
))
Progress.displayName = ProgressPrimitive.Root.displayName

export { Progress }

```

src/components/ui/select.tsx TSX 5.9 KB 160 Lines

```tsx
"use client"

import * as React from "react"
import * as SelectPrimitive from "@radix-ui/react-select"
import { Check, ChevronDown, ChevronUp } from "lucide-react"

import { cn } from "@/lib/utils"

const Select = SelectPrimitive.Root

const SelectGroup = SelectPrimitive.Group

const SelectValue = SelectPrimitive.Value

const SelectTrigger = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Trigger>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-9 w-full items-center justify-between whitespace-nowrap rounded-md border border-cyan-500/30 bg-[#070E20]/90 px-3 py-2 text-sm text-zinc-200 shadow-sm placeholder:text-slate-400 focus:outline-none focus:ring-1 focus:ring-ink/30 disabled:cursor-not-allowed disabled:opacity-50 [&>span]:line-clamp-1",
      className
    )}
    {...props}
  >
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
))
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName

const SelectScrollUpButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollUpButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollUpButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollUpButton
    ref={ref}
    className={cn(
      "flex cursor-default items-center justify-center py-1",
      className
    )}
    {...props}
  >
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollDownButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollDownButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn(
      "flex cursor-default items-center justify-center py-1",
      className
    )}
    {...props}
  >
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Content>
>(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
```

```
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-[--radix-select-content-available-height] min-w-[8rem] overflow-y-auto overflow-x-hidden rounded-xl border border-slate-200 bg-white text-slate-900 shadow-xl data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 origin-[--radix-select-content-transform-origin]",
        position === "popper" &&
          "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
        className
      )}
      position={position}
      {...props}
    >
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn(
          "p-1.5",
          position === "popper" &&
            "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]"
        )}
      >
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Label>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold text-slate-700", className)}
    {...props}
  />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Item>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-pointer select-none items-center rounded-lg py-2 pl-2 pr-8 text-sm font-medium text-slate-800 outline-none hover:bg-slate-100 hover:text-slate-900 focus:bg-slate-100 focus:text-slate-900 data-[disabled]:pointer-events-none data-[disabled]:opacity-50 transition-colors",
      className
    )}
    {...props}
  >
    <span className="absolute right-2 flex h-3.5 w-3.5 items-center justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4 text-cyan-600" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-ink/10", className)}
    {...props}
  />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName
```

```
147 |
148 | export {
149 |   Select,
150 |   SelectGroup,
151 |   SelectValue,
152 |   SelectTrigger,
153 |   SelectContent,
154 |   SelectLabel,
155 |   SelectItem,
156 |   SelectSeparator,
157 |   SelectScrollUpButton,
158 |   SelectScrollDownButton,
159 | }
160 |
```

src/components/ui/separator.tsx TSX 0.7 KB 30 Lines

```tsx
import * as React from "react"
import * as SeparatorPrimitive from "@radix-ui/react-separator"

import { cn } from "@/lib/utils"

const Separator = React.forwardRef<
  React.ElementRef<typeof SeparatorPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof SeparatorPrimitive.Root>
>(
  (
    { className, orientation = "horizontal", decorative = true, ...props },
    ref
  ) => (
    <SeparatorPrimitive.Root
      ref={ref}
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "shrink-0 bg-ink/10",
        orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
        className
      )}
      {...props}
    />
  )
)
Separator.displayName = SeparatorPrimitive.Root.displayName

export { Separator }

```

src/components/ui/sonner.tsx TSX 0.9 KB 32 Lines

```tsx
"use client"

import { useTheme } from "next-themes"
import { Toaster as Sonner } from "sonner"

type ToasterProps = React.ComponentProps<typeof Sonner>

const Toaster = ({ ...props }: ToasterProps) => {
  const { theme = "system" } = useTheme()

  return (
    <Sonner
      theme={theme as ToasterProps["theme"]}
      className="toaster group"
      toastOptions={{
        classNames: {
          toast:
            "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground group-[.toaster]:border-border group-[.toaster]:shadow-lg",
          description: "group-[.toast]:text-muted-foreground",
          actionButton:
            "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
          cancelButton:
            "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
        },
      }}
      {...props}
    />
  )
}

export { Toaster }

```

src/components/ui/table.tsx TSX 2.8 KB 121 Lines

```tsx
import * as React from "react"

import { cn } from "@/lib/utils"

const Table = React.forwardRef<
  HTMLTableElement,
  React.HTMLAttributes<HTMLTableElement>
>(({ className, ...props }, ref) => (
  <div className="relative w-full overflow-auto">
    <table
      ref={ref}
      className={cn("w-full caption-bottom text-sm", className)}
      {...props}
    />
  </div>
))
Table.displayName = "Table"

const TableHeader = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
))
TableHeader.displayName = "TableHeader"

const TableBody = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tbody
    ref={ref}
    className={cn("[&_tr:last-child]:border-0", className)}
    {...props}
  />
))
TableBody.displayName = "TableBody"

const TableFooter = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn(
      "border-t border-cyan-500/30 bg-ink/5 font-medium [&>tr]:last:border-b-0",
      className
    )}
    {...props}
  />
))
TableFooter.displayName = "TableFooter"

const TableRow = React.forwardRef<
  HTMLTableRowElement,
  React.HTMLAttributes<HTMLTableRowElement>
>(({ className, ...props }, ref) => (
  <tr
    ref={ref}
    className={cn(
      "border-b border-cyan-500/30 transition-colors hover:bg-ink/5 data-[state=selected]:bg-ink/5",
      className
    )}
    {...props}
  />
))
TableRow.displayName = "TableRow"

const TableHead = React.forwardRef<
  HTMLTableCellElement,
  React.ThHTMLAttributes<HTMLTableCellElement>
>(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-10 px-2 text-left align-middle font-medium text-zinc-200/50 [&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:transla
```

```
te-y-[2px]",
       className
     )}
     {...props}
   />
))
TableHead.displayName = "TableHead"

const TableCell = React.forwardRef<
  HTMLTableCellElement,
  React.TdHTMLAttributes<HTMLTableCellElement>
>(({ className, ...props }, ref) => (
  <td
    ref={ref}
    className={cn(
      "p-2 align-middle [&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props}
  />
))
TableCell.displayName = "TableCell"

const TableCaption = React.forwardRef<
  HTMLTableCaptionElement,
  React.HTMLAttributes<HTMLTableCaptionElement>
>(({ className, ...props }, ref) => (
  <caption
    ref={ref}
    className={cn("mt-4 text-sm text-zinc-200/50", className)}
    {...props}
  />
))
TableCaption.displayName = "TableCaption"

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}
```

src/components/ui/tabs.tsx TSX 1.7 KB 54 Lines

```tsx
import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"

import { cn } from "@/lib/utils"

const Tabs = TabsPrimitive.Root

const TabsList = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.List>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.List>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    className={cn(
      "inline-flex h-9 items-center justify-center rounded-lg bg-ink/5 p-1 text-zinc-200/50",
      className
    )}
    {...props}
  />
))
TabsList.displayName = TabsPrimitive.List.displayName

const TabsTrigger = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Trigger>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
      "inline-flex items-center justify-center whitespace-nowrap rounded-md px-3 py-1 text-sm font-medium transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ink/20 disabled:pointer-events-none disabled:opacity-50 data-[state=active]:bg-[#070E20]/90 data-[state=active]:text-zinc-200 data-[state=active]:shadow",
      className
    )}
    {...props}
  />
))
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName

const TabsContent = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Content>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn(
      "mt-2 focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ink/20",
      className
    )}
    {...props}
  />
))
TabsContent.displayName = TabsPrimitive.Content.displayName

export { Tabs, TabsList, TabsTrigger, TabsContent }
```

src/components/ui/textarea.tsx TSX 0.7 KB 23 Lines

```tsx
import * as React from "react"

import { cn } from "@/lib/utils"

const Textarea = React.forwardRef<
  HTMLTextAreaElement,
  React.ComponentProps<"textarea">
>(({ className, ...props }, ref) => {
  return (
    <textarea
      className={cn(
        "flex min-h-[60px] w-full rounded-md border border-slate-200 bg-white px-3 py-2 text-base text-slate-900 shadow-xs placeholder:text-slate-400 focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-cyan-500 disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        className
      )}
      ref={ref}
      {...props}
    />
  )
})
Textarea.displayName = "Textarea"

export { Textarea }

```

src/components/ui/Thumbnail.tsx TSX 2.7 KB 66 Lines

```tsx
import React from 'react'
import { useStorageUrl } from '@/lib/useStorageUrl'
import { Loader2, BookOpen } from 'lucide-react'

interface ThumbnailProps {
  path: string | null
  alt: string
  className?: string
  fallbackIcon?: React.ReactNode
  type?: string
}

const gradients = [
  'from-cyan-700 via-blue-700 to-indigo-800',
  'from-sky-600 via-cyan-600 to-blue-700',
  'from-blue-700 via-indigo-700 to-slate-900',
  'from-cyan-600 via-teal-600 to-sky-700',
  'from-indigo-800 via-blue-700 to-cyan-600',
]

export function Thumbnail({ path, alt, className = '', fallbackIcon }: ThumbnailProps) {
  const { url, loading } = useStorageUrl('materials', path)

  const getGradientIndex = (str: string) => {
    let hash = 0
    for (let i = 0; i < str.length; i++) {
      hash = str.charCodeAt(i) + ((hash << 5) - hash)
    }
    return Math.abs(hash) % gradients.length
  }

  const gradient = gradients[getGradientIndex(alt || 'Course')]

  if (!path || !url) {
    return (
      <div className={`relative w-full h-full bg-gradient-to-br ${gradient} flex flex-col items-center justify-center p-4 over
flow-hidden select-none ${className}`}>
        <div className="absolute -top-6 -right-6 w-24 h-24 rounded-full bg-white/10 blur-xl pointer-events-none" />
        <div className="absolute -bottom-8 -left-8 w-28 h-28 rounded-full bg-cyan-400/20 blur-xl pointer-events-none" />
        <div className="absolute inset-0 bg-[radial-gradient(ellipse_at_top,_var(--tw-gradient-stops))] from-white/15 via-tran
sparent to-black/20 pointer-events-none" />

        <div className="relative z-10 w-12 h-12 rounded-2xl bg-white/15 backdrop-blur-md border border-white/25 flex items-cen
ter justify-center shadow-lg shadow-black/10 group-hover:scale-110 group-hover:bg-white/25 transition-all duration-300">
          {fallbackIcon || <BookOpen className="w-6 h-6 text-white" />}
        </div>
        <span className="relative z-10 mt-2 text-[11px] font-semibold text-white/90 tracking-wide truncate max-w-[85%] text-ce
nter drop-shadow-xs">
          {alt}
        </span>
      </div>
    )
  }

  if (loading) {
    return (
      <div className={`flex items-center justify-center bg-slate-100 ${className}`}>
        <Loader2 className="w-6 h-6 text-cyan-600 animate-spin" />
      </div>
    )
  }

  return (
    <div className="relative w-full h-full overflow-hidden">
      <img src={url} alt={alt} className={`w-full h-full object-cover group-hover:scale-105 transition-transform duration-500
${className}`} />
      <div className="absolute inset-0 bg-gradient-to-t from-slate-950/40 via-transparent to-transparent opacity-60 group-hove
r:opacity-30 transition-opacity" />
    </div>
  )
}

```

src/components/ChatBot.tsx TSX 16.4 KB 385 Lines

```tsx
import React, { useState, useRef, useEffect } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { X, Send, User, RotateCcw, Lock, LogIn, Sparkles } from 'lucide-react'
import { Link } from 'react-router-dom'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { ChatBotLogo } from './ChatBotLogo'
import { useAuth } from '@/hooks/useAuth'

function BotMessage({ content }: { content: string }) {
  const lines = content.split('\n')
  return (
    <div className="space-y-1">
      {lines.map((line, i) => {
        const trimmed = line.trim()
        if (!trimmed) return null
        const isBullet = /^[-*•]\s+/.test(trimmed)
        const text = isBullet ? trimmed.replace(/^[-*•]\s+/, '') : trimmed
        return isBullet ? (
          <div key={i} className="flex gap-1.5 items-start">
            <span className="mt-1.5 w-1.5 h-1.5 rounded-full bg-cyan-500 shrink-0" />
            <span>{text}</span>
          </div>
        ) : (
          <p key={i}>{text}</p>
        )
      })}
    </div>
  )
}

interface Message {
  role: 'bot' | 'user'
  content: string
  time?: string
}

const SUGGESTED_PROMPTS = [
  'Explore available courses',
  'How do I register?',
  'Tell me about certifications',
  'How do I contact support?',
]

const CANNED_RESPONSES: Record<string, string> = {
  'Explore available courses':
    'You can explore our full catalog of scientific courses covering Atmospheric Science, Doppler Weather Radar, Oceanography, and Climatology on the **Courses** page. Filter by department or competency level to find your ideal training track.',
  'How do I register?':
    'To register, click **Get Started** or **Register** at the top right of the navigation bar. Select your role as a **Trainee** or **Trainer**, complete your institutional details, and submit for verification.',
  'Tell me about certifications':
    'Upon completing all course modules and passing the final assessments (minimum 60% passing score), you receive an official MoES-accredited, verifiable digital PDF certificate complete with cryptographic validation and QR code.',
  'How do I contact support?':
    'You can reach out to our official MoES administrators and support team via the **Contact** page. We are available to help you with platform issues, course inquiries, and account verification.',
}

export function ChatBot() {
  const { user } = useAuth()
  const isLoggedIn = Boolean(user)

  const [isOpen, setIsOpen] = useState(false)
  const [messages, setMessages] = useState<Message[]>([
    {
      role: 'bot',
      content: 'Greetings! I am Capacity Connect AI, your platform assistant. How can I help you today?',
    },
  ])
  const [inputValue, setInputValue] = useState('')
  const [isLoading, setIsLoading] = useState(false)
  const scrollRef = useRef<HTMLDivElement>(null)
  const inputRef = useRef<HTMLInputElement>(null)

  useEffect(() => {
```

```
    if (scrollRef.current) {
      scrollRef.current.scrollTop = scrollRef.current.scrollHeight
    }
  }, [messages, isLoading])

  useEffect(() => {
    if (isOpen && inputRef.current && isLoggedIn) {
      setTimeout(() => inputRef.current?.focus(), 150)
    }
  }, [isOpen, isLoggedIn])

  const handleResetChat = () => {
    setMessages([
      {
        role: 'bot',
        content: 'Greetings! I am Capacity Connect AI, your platform assistant. How can I help you today?',
      },
    ])
  }

  const sendMessage = async (textToSend: string) => {
    if (!textToSend.trim() || isLoading) return

    const trimmedMsg = textToSend.trim()

    // When NOT logged in, enforce that only the 4 suggested questions are permitted
    if (!isLoggedIn && !SUGGESTED_PROMPTS.includes(trimmedMsg)) {
      setMessages((prev) => [
        ...prev,
        { role: 'user', content: trimmedMsg },
        {
          role: 'bot',
          content:
            'Please log in to ask custom questions freely. As a guest, you can select any of the 4 suggested questions above.',
        },
      ])
      setInputValue('')
      return
    }

    const updatedMessages: Message[] = [...messages, { role: 'user', content: trimmedMsg }]
    setMessages(updatedMessages)
    setInputValue('')
    setIsLoading(true)

    // Check if we have an instant canned answer for a suggested prompt
    if (CANNED_RESPONSES[trimmedMsg]) {
      setTimeout(() => {
        setMessages((prev) => [...prev, { role: 'bot', content: CANNED_RESPONSES[trimmedMsg] }])
        setIsLoading(false)
      }, 400)
      return
    }

    const API_KEY = import.meta.env.VITE_GEMINI_API_KEY || ''

    try {
      if (!API_KEY) {
        throw new Error('No API key configured')
      }

      const formattedMessages = updatedMessages
        .filter((msg, index) => !(index === 0 && msg.role === 'bot')) // Skip initial greeting
        .map((msg) => ({
          role: msg.role === 'bot' ? 'model' : 'user',
          parts: [{ text: msg.content }],
        }))

      const response = await fetch(
        `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.7-flash:generateContent?key=${API_KEY}`,
        {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            systemInstruction: {
              parts: [
```

```
                {
                  text: 'You are Capacity Connect AI, the intelligent platform assistant for Capacity Connect (Ministry of Earth Sciences, Govt of India). You assist users with exploring courses, training modules, user registration, certifications, and platform navigation. Provide clear, helpful, and concise responses.',
                },
              ],
            },
            contents: formattedMessages,
          }),
        }
      )

      if (!response.ok) {
        const errText = await response.text()
        throw new Error(`Gemini API error: ${response.status} - ${errText}`)
      }

      const data = await response.json()
      const reply = data.candidates?.[0]?.content?.parts?.[0]?.text

      if (reply) {
        setMessages((prev) => [...prev, { role: 'bot', content: reply }])
      } else {
        setMessages((prev) => [
          ...prev,
          { role: 'bot', content: 'I could not understand that. Could you please rephrase?' },
        ])
      }
    } catch (err: any) {
      console.warn('Chatbot fallback:', err)
      setMessages((prev) => [
        ...prev,
        {
          role: 'bot',
          content:
            'I am currently operating with platform knowledge. For specific inquiries, explore our Courses catalog or contact official MoES administrators via the Contact page.',
        },
      ])
    } finally {
      setIsLoading(false)
    }
  }

  const handleSend = async (e?: React.FormEvent) => {
    if (e) e.preventDefault()
    if (!isLoggedIn) return
    sendMessage(inputValue)
  }

  return (
    <div className="fixed bottom-6 right-6 z-50 font-sans">
      <AnimatePresence>
        {isOpen && (
          <motion.div
            initial={{ opacity: 0, y: 16, scale: 0.95 }}
            animate={{ opacity: 1, y: 0, scale: 1 }}
            exit={{ opacity: 0, y: 16, scale: 0.95 }}
            transition={{ type: 'spring', damping: 25, stiffness: 300 }}
            className="absolute bottom-16 right-0 w-[calc(100vw-2.5rem)] sm:w-[410px] bg-[#060D1E]/95 backdrop-blur-2xl border border-cyan-500/30 shadow-[0_20px_60px_rgba(0,0,0,0.7),0_0_30px_rgba(6,182,212,0.15)] rounded-2xl overflow-hidden flex flex-col h-[550px] max-h-[82vh]"
          >
            {/* Header */}
            <div className="px-4 py-3.5 bg-gradient-to-r from-[#040814] via-[#07132a] to-[#0a1e3f] border-b border-cyan-500/25 flex justify-between items-center text-white shadow-sm">
              <div className="flex items-center gap-3">
                <div className="relative">
                  <ChatBotLogo variant="badge" size={36} />
                  <span className="absolute -bottom-0.5 -right-0.5 flex h-2.5 w-2.5">
                    <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-emerald-400 opacity-75" />
                    <span className="relative inline-flex rounded-full h-2.5 w-2.5 bg-emerald-400 border border-[#040814]" />
                  </span>
                </div>
                <div>
                  <h3 className="font-semibold text-sm tracking-tight text-white flex items-center gap-1.5">
                    Capacity Connect AI
                  </h3>
```

```
                    <p className="text-[10px] text-cyan-300 font-medium">
                      {isLoggedIn ? 'Online • Full Access' : 'Guest Mode • Suggested Questions'}
                    </p>
                  </div>
                </div>

                <div className="flex items-center gap-1">
                  <button
                    type="button"
                    onClick={handleResetChat}
                    title="Reset conversation"
                    aria-label="Reset conversation"
                    className="p-1.5 text-slate-300 hover:text-white hover:bg-white/10 rounded-lg transition-colors cursor-pointer"
                  >
                    <RotateCcw className="w-4 h-4" />
                  </button>
                  <button
                    type="button"
                    onClick={() => setIsOpen(false)}
                    title="Close chat"
                    aria-label="Close chat"
                    className="p-1.5 text-slate-300 hover:text-white hover:bg-white/10 rounded-lg transition-colors cursor-pointer"
                  >
                    <X className="w-4 h-4" />
                  </button>
                </div>
              </div>

              {/* Chat Messages Body */}
              <div
                ref={scrollRef}
                className="flex-1 p-4 overflow-y-auto bg-gradient-to-b from-[#f8fafc] via-[#f1f5f9] to-[#f8fafc] flex flex-col gap-3.5 scroll-smooth"
              >
                {messages.map((msg, i) => (
                  <div
                    key={i}
                    className={`flex gap-2.5 ${msg.role === 'user' ? 'flex-row-reverse' : ''}`}
                  >
                    {msg.role === 'bot' ? (
                      <ChatBotLogo variant="badge" size={28} />
                    ) : (
                      <div className="w-7 h-7 rounded-full flex items-center justify-center shrink-0 mt-0.5 bg-gradient-to-br from-cyan-600 to-blue-700 text-white shadow-sm">
                        <User className="w-4 h-4" />
                      </div>
                    )}
                    <div
                      className={`p-3.5 rounded-2xl max-w-[82%] text-xs sm:text-sm leading-relaxed whitespace-pre-wrap ${
                        msg.role === 'user'
                          ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white rounded-tr-xs shadow-md shadow-cyan-600/20 font-medium'
                          : 'bg-white border border-slate-200/90 text-slate-800 rounded-tl-xs shadow-md shadow-slate-200/50'
                      }`}
                    >
                      {msg.role === 'bot' ? <BotMessage content={msg.content} /> : msg.content}
                    </div>
                  </div>
                ))}

                {isLoading && (
                  <div className="flex gap-2.5 items-center">
                    <ChatBotLogo variant="badge" size={28} />
                    <div className="p-3 rounded-2xl bg-white border border-slate-200 text-slate-600 rounded-tl-xs shadow-sm flex items-center gap-1.5">
                      <span
                        className="w-1.5 h-1.5 bg-cyan-500 rounded-full animate-bounce"
                        style={{ animationDelay: '0ms' }}
                      />
                      <span
                        className="w-1.5 h-1.5 bg-sky-500 rounded-full animate-bounce"
                        style={{ animationDelay: '150ms' }}
                      />
                      <span
                        className="w-1.5 h-1.5 bg-amber-400 rounded-full animate-bounce"
                        style={{ animationDelay: '300ms' }}
                      />
```

```
                  </div>
                </div>
              )}

              {/* Suggested Questions: Available for guests anytime, or initial greeting for logged in users */}
              {(!isLoggedIn || messages.length <= 1) && (
                <div className="pt-2 flex flex-col gap-1.5">
                  <p className="text-[11px] font-semibold text-slate-500 px-1 flex items-center gap-1">
                    <Sparkles className="w-3 h-3 text-cyan-600" />
                    Suggested questions:
                  </p>
                  <div className="flex flex-wrap gap-1.5">
                    {SUGGESTED_PROMPTS.map((prompt, idx) => (
                      <button
                        key={idx}
                        type="button"
                        onClick={() => sendMessage(prompt)}
                        className="text-xs px-3 py-1.5 rounded-xl bg-white hover:bg-cyan-50 text-slate-700 hover:text-cyan-800
border border-slate-200/90 hover:border-cyan-400/50 shadow-xs hover:shadow-sm transition-all text-left font-medium cursor-pointer"
                      >
                        {prompt}
                      </button>
                    ))}
                  </div>
                </div>
              )}
            </div>

            {/* Input Bar */}
            {isLoggedIn ? (
              <form
                onSubmit={handleSend}
                className="p-3 bg-[#040814] border-t border-cyan-500/20 flex gap-2 items-center"
              >
                <Input
                  ref={inputRef}
                  value={inputValue}
                  onChange={(e) => setInputValue(e.target.value)}
                  placeholder="Ask Capacity Connect AI..."
                  className="rounded-xl bg-[#081226] border border-cyan-500/30 text-white placeholder:text-slate-400 focus-vis
ible:ring-cyan-500 focus-visible:border-cyan-400 h-10 text-xs sm:text-sm shadow-inner"
                />
                <Button
                  type="submit"
                  size="icon"
                  disabled={isLoading || !inputValue.trim()}
                  className="rounded-xl h-10 w-10 bg-gradient-to-r from-cyan-500 via-blue-600 to-amber-500 hover:opacity-95 te
xt-white shadow-lg shadow-cyan-500/25 shrink-0 cursor-pointer disabled:opacity-40 transition-all"
                >
                  <Send className="w-4 h-4" />
                </Button>
              </form>
            ) : (
              <div className="p-3 bg-[#040814] border-t border-cyan-500/20 flex flex-col gap-2">
                <div className="flex items-center justify-between gap-2 px-3 py-2 rounded-xl bg-[#081226] border border-cyan-5
00/25 text-xs text-slate-300">
                  <div className="flex items-center gap-2 truncate">
                    <Lock className="w-3.5 h-3.5 text-amber-400 shrink-0" />
                    <span className="truncate">Log in to type custom queries</span>
                  </div>
                  <Link
                    to="/login"
                    onClick={() => setIsOpen(false)}
                    className="shrink-0 px-2.5 py-1 rounded-lg bg-cyan-600 hover:bg-cyan-500 text-white text-[11px] font-bold
transition-all flex items-center gap-1 shadow-sm"
                  >
                    <LogIn className="w-3 h-3" />
                    <span>Log In</span>
                  </Link>
                </div>
              </div>
            )}
          </motion.div>
        )}
      </AnimatePresence>

      {/* Floating Trigger Button */}
      <motion.button
        whileHover={{ scale: 1.08 }}
```

```
368 |         whileTap={{ scale: 0.94 }}
369 |         onClick={() => setIsOpen(!isOpen)}
370 |         className="w-14 h-14 bg-gradient-to-tr from-cyan-500 via-blue-600 to-amber-500 rounded-full flex items-center justify-
center text-white shadow-[0_0_25px_rgba(6,182,212,0.5),0_0_10px_rgba(56,189,248,0.35)] border border-cyan-300/40 cursor-pointer grou
p"
371 |         aria-label="Toggle Capacity Connect AI platform assistant"
372 |       >
373 |         {isOpen ? (
374 |           <X className="w-6 h-6 transition-transform group-hover:rotate-90 duration-200 text-white" />
375 |         ) : (
376 |           <ChatBotLogo variant="icon-only" size={28} className="text-white drop-shadow-sm transition-transform group-hover:sca
le-110 duration-200" />
377 |         )}
378 |       </motion.button>
379 |     </div>
380 |   )
381 | }
382 |
383 |
384 |
385 |
```

src/components/ChatBotLogo.tsx TSX 2.8 KB 95 Lines

```tsx
import React from 'react'

interface ChatBotLogoProps {
  className?: string
  size?: number
  variant?: 'icon-only' | 'badge' | 'floating'
}

export function ChatBotLogo({ className = '', size = 24, variant = 'icon-only' }: ChatBotLogoProps) {
  const svgIcon = (
    <svg
      viewBox="0 0 36 36"
      width={size}
      height={size}
      fill="none"
      xmlns="http://www.w3.org/2000/svg"
      className={variant === 'icon-only' ? className : 'w-full h-full'}
    >
      {/* Brain Left Hemisphere Outline */}
      <path
        d="M17 7.5C13.2 7.2 9.5 9.8 9.5 13.2C7.2 14.5 7 17.8 9.2 19.5C7.8 21.2 8.5 24.5 11.8 25.8C13.5 26.5 15.5 26.5 17 26.2"
        stroke="currentColor"
        strokeWidth="2.4"
        strokeLinecap="round"
        strokeLinejoin="round"
      />

      {/* Center Spine */}
      <path
        d="M17 8.5V25"
        stroke="currentColor"
        strokeWidth="2.4"
        strokeLinecap="round"
      />

      {/* Top Circuit Branch */}
      <path
        d="M17 12H21.5C23 12 24 11 24 9.5"
        stroke="currentColor"
        strokeWidth="2.4"
        strokeLinecap="round"
        strokeLinejoin="round"
      />
      <circle cx="24" cy="8.5" r="1.6" fill="currentColor" />

      {/* Middle Circuit Branch */}
      <path
        d="M17 17.5H23"
        stroke="currentColor"
        strokeWidth="2.4"
        strokeLinecap="round"
      />
      <circle cx="24.8" cy="17.5" r="1.6" fill="currentColor" />

      {/* Bottom Circuit Branch */}
      <path
        d="M17 23H21.5C23 23 24 24 24 25.5"
        stroke="currentColor"
        strokeWidth="2.4"
        strokeLinecap="round"
        strokeLinejoin="round"
      />
      <circle cx="24" cy="26.5" r="1.6" fill="currentColor" />
    </svg>
  )

  if (variant === 'floating') {
    return (
      <div
        className={`relative flex items-center justify-center rounded-full bg-gradient-to-tr from-cyan-500 via-blue-600 to-amber-500 text-white shadow-[0_0_25px_rgba(6,182,212,0.55),0_0_10px_rgba(56,189,248,0.4)] border border-cyan-300/40 ${className}`}
        style={{ width: size, height: size }}
      >
        <div className="w-[62%] h-[62%] flex items-center justify-center text-white">
          {svgIcon}
        </div>
```

```
      </div>
    )
  }

  if (variant === 'badge') {
    return (
      <div
        className={`relative flex items-center justify-center rounded-full bg-gradient-to-tr from-cyan-500 via-sky-500 to-blue
-600 text-white shadow-[0_0_12px_rgba(6,182,212,0.35)] border border-cyan-300/40 shrink-0 ${className}`}
        style={{ width: size, height: size }}
      >
        <div className="w-[64%] h-[64%] flex items-center justify-center text-white">
          {svgIcon}
        </div>
      </div>
    )
  }

  return svgIcon
}

```

src/components/ErrorBoundary.tsx TSX 0.8 KB 29 Lines

```tsx
import React from "react";

interface Props { children: React.ReactNode; }
interface State { hasError: boolean; error: Error | null; }

export class ErrorBoundary extends React.Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error: Error) {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return (
        <div style={{ padding: 20, color: "red", backgroundColor: "white", zIndex: 9999, position: "relative" }}>
          <h1>REACT CRASH IN COMPONENT</h1>
          <pre>{this.state.error?.message}</pre>
          <pre>{this.state.error?.stack}</pre>
        </div>
      );
    }
    return this.props.children;
  }
}

```

src/components/landing/AchievementsSection.tsx TSX 14.1 KB 277 Lines

```tsx
import React, { useState, useEffect } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import {
  Award, Trophy, ChevronRight, ChevronLeft, Star, UserCheck, User
} from 'lucide-react'
import { useHomePageSettings, defaultAchievementStories, AchievementStoryItem } from '@/hooks/useHomePageSettings'

export function AchievementsSection() {
  const { settings } = useHomePageSettings()
  const [currentIndex, setCurrentIndex] = useState(0)
  const [isHovered, setIsHovered] = useState(false)

  // Respect Section Live Toggle

  if (settings.achievements_enabled === false) {
    return null
  }

  const stories: AchievementStoryItem[] =
    settings.achievements_stories && settings.achievements_stories.length > 0
      ? settings.achievements_stories
      : defaultAchievementStories

  // Auto-play interval (every 5 seconds, pauses when hovering)

  useEffect(() => {
    if (isHovered || stories.length <= 1) return

    const interval = setInterval(() => {
      setCurrentIndex((prev) => (prev + 1) % stories.length)
    }, 5000)

    return () => clearInterval(interval)
  }, [isHovered, stories.length])

  const activeStory = stories[currentIndex % stories.length] || stories[0]

  const handlePrev = () => {
    setCurrentIndex(prev => (prev === 0 ? stories.length - 1 : prev - 1))
  }

  const handleNext = () => {
    setCurrentIndex(prev => (prev + 1) % stories.length)
  }

  const sectionTag = settings.achievements_tag || 'Proven National Impact'
  const sectionTitle = settings.achievements_title || 'Inspired trainees. Inspired results'
  const moesBadge = settings.achievements_moes_badge || '100% Verified Operational Skill Credentials'
  const moesDesc = settings.achievements_moes_desc || 'Direct alignment with WMO, IMD & INCOIS forecast protocols.'

  return (
    <section className="relative py-20 lg:py-24 bg-slate-50 text-slate-900 border-t border-slate-200 overflow-hidden select-none">
      <div className="max-w-7xl mx-auto px-6 relative z-10">

        {/* Top Header Grid (Light Theme) */}
        <div className="flex flex-col lg:flex-row lg:items-center justify-between gap-8 mb-12">
          <div>
            <div className="inline-flex items-center gap-2 px-3.5 py-1.5 rounded-full bg-cyan-50 border border-cyan-200 text-cyan-800 text-xs font-bold uppercase tracking-wider mb-4 shadow-2xs">
              <Trophy className="w-3.5 h-3.5 text-cyan-600" />
              {sectionTag}
            </div>

            <h2 className="text-3xl sm:text-4xl md:text-5xl font-black font-display text-slate-900 tracking-tight leading-[1.15] mb-2">
              {sectionTitle.includes('Inspired results') ? (
                <>
                  Inspired trainees. Inspired{' '}
                  <span className="text-cyan-600">
                    results
                  </span>
                </>
              ) : (
                sectionTitle
              )}
```

```
 72 |             </h2>
 73 |           </div>
 74 |
 75 |           {/* Right Header Visual Showcase Card */}
 76 |           <div className="relative flex items-center justify-center lg:justify-end">
 77 |             <div className="relative p-5 sm:p-6 rounded-2xl bg-white border border-slate-200/90 shadow-xl shadow-slate-200/50
max-w-md w-full flex items-center gap-4">
 78 |               <div className="w-14 h-14 rounded-2xl bg-cyan-600 flex items-center justify-center text-white shadow-lg shadow-c
yan-600/25 shrink-0">
 79 |                 <Award className="w-7 h-7" />
 80 |               </div>
 81 |               <div>
 82 |                 <div className="flex items-center gap-1.5 text-cyan-700 font-black text-xs uppercase tracking-wider mb-0.5">
 83 |                   <Star className="w-3.5 h-3.5 fill-amber-500 text-amber-500" />
 84 |                   <span>MoES Center of Excellence</span>
 85 |                 </div>
 86 |                 <h4 className="text-sm font-bold text-slate-900 leading-snug">
 87 |                   {moesBadge}
 88 |                 </h4>
 89 |                 <p className="text-xs text-slate-500 mt-0.5 font-medium">
 90 |                   {moesDesc}
 91 |                 </p>
 92 |               </div>
 93 |             </div>
 94 |           </div>
 95 |         </div>
 96 |
 97 |         {/* Main Dark Themed Showcase Card */}
 98 |         <div
 99 |           className="relative"
100 |           onMouseEnter={() => setIsHovered(true)}
101 |           onMouseLeave={() => setIsHovered(false)}
102 |         >
103 |           <AnimatePresence mode="wait">
104 |             <motion.div
105 |               key={activeStory.id || currentIndex}
106 |               initial={{ opacity: 0, y: 15 }}
107 |               animate={{ opacity: 1, y: 0 }}
108 |               exit={{ opacity: 0, y: -15 }}
109 |               transition={{ duration: 0.35, ease: 'easeOut' }}
110 |               className="relative rounded-3xl bg-[#060c1c] text-white p-6 sm:p-10 lg:p-12 shadow-2xl shadow-cyan-950/40 border
border-cyan-500/30 overflow-hidden"
111 |             >
112 |               {/* Banner Top Header Text */}
113 |               <div className="text-center mb-8 sm:mb-12 relative z-10">
114 |                 <h3 className="text-xl sm:text-2xl md:text-3xl font-black font-display tracking-tight text-white leading-tigh
t">
115 |                   {activeStory.bannerTitle}{' '}
116 |                   <span className="block sm:inline text-cyan-400 font-black font-display tracking-tight mt-1 sm:mt-0">
117 |                     {activeStory.bannerSubtitle}
118 |                   </span>
119 |                 </h3>
120 |               </div>
121 |
122 |               {/* Banner Inner Content Grid (Achievers Row + Stat Callout) */}
123 |               <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 sm:gap-8 items-stretch relative z-10">
124 |
125 |                 {/* Left Side: Achievers Showcase (Col 7 / 8) */}
126 |                 <div className="lg:col-span-8 grid grid-cols-1 sm:grid-cols-3 gap-5 sm:gap-6">
127 |                   {(activeStory.achievers || []).map((achiever, i) => (
128 |                     <motion.div
129 |                       key={achiever.id || achiever.name + i}
130 |                       initial={{ opacity: 0, y: 20 }}
131 |                       animate={{ opacity: 1, y: 0 }}
132 |                       transition={{ delay: i * 0.08, duration: 0.35 }}
133 |                       className="flex flex-col rounded-3xl bg-white text-slate-900 shadow-2xl shadow-slate-950/30 border borde
r-slate-200/80 hover:border-cyan-400 hover:shadow-cyan-500/25 hover:-translate-y-2 transition-all duration-300 group overflow-hidde
n"
134 |                     >
135 |                       {/* Top Photo Section with Name Overlay */}
136 |                       <div className="relative h-72 sm:h-80 w-full bg-slate-900 overflow-hidden flex items-end p-5">
137 |                         {achiever.avatar_url ? (
138 |                           <img
139 |                             src={achiever.avatar_url}
140 |                             alt={achiever.name}
141 |                             className="absolute inset-0 w-full h-full object-cover object-center group-hover:scale-105 transit
ion-transform duration-500"
142 |                           />
143 |                         ) : (
```

```
                          <div className="absolute inset-0 w-full h-full bg-gradient-to-b from-[#f0f2f5] via-[#d6dae1] to-[#8e97a6] flex items-center justify-center">
                            {/* Minimalist SVG Silhouette */}
                            <svg
                              viewBox="0 0 100 100"
                              fill="none"
                              xmlns="http://www.w3.org/2000/svg"
                              className="w-24 h-24 text-white/55 mb-8"
                              stroke="currentColor"
                              strokeWidth="2.8"
                              strokeLinecap="round"
                              strokeLinejoin="round"
                            >
                              <circle cx="50" cy="38" r="14" />
                              <path d="M26 78C26 63 36 57 50 57C64 57 74 63 74 78" />
                            </svg>
                          </div>
                        )}

                        {/* Top Right Score & Rank Floating Badge */}
                        {achiever.achievementBadge && (
                          <div className="absolute top-3.5 right-3.5 z-10">
                            <span className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-full bg-slate-950/90 backdrop-blur-md text-amber-400 border border-amber-500/50 text-xs font-black uppercase tracking-wider font-display shadow-xl">
                              <Trophy className="w-3.5 h-3.5 text-amber-400" />
                              {achiever.achievementBadge}
                            </span>
                          </div>
                        )}

                        {/* Subtle bottom-only scrim for name readability without darkening the photo */}
                        <div className="absolute inset-x-0 bottom-0 h-28 bg-gradient-to-t from-slate-950/85 via-slate-950/30 to-transparent pointer-events-none" />

                        {/* Overlay Name in bold white */}
                        <div className="relative z-10 text-left w-full">
                          <h4 className="text-xl sm:text-2xl font-black text-white leading-tight tracking-tight drop-shadow-md">
                            {achiever.name}
                          </h4>
                        </div>
                      </div>

                      {/* Bottom Light Section with Vertical Accent Bar */}
                      <div className="p-5 bg-white flex flex-col justify-center flex-1">
                        {/* Role with Left Vertical Accent Bar */}
                        <div className="flex items-start gap-2.5">
                          <span className="w-1.5 h-4 bg-cyan-600 rounded-full shrink-0 mt-0.5" />
                          <span className="text-xs sm:text-[13px] font-black uppercase tracking-wider text-slate-900 leading-snug">
                            {achiever.role} - {achiever.institute}
                          </span>
                        </div>
                      </div>
                    </motion.div>
                  ))}
                </div>

                {/* Right Side: Big Stat Highlight Callout (Matching Light Theme & Height) */}
                <div className="lg:col-span-4 flex justify-center items-stretch">
                  <div className="w-full rounded-3xl bg-white text-slate-900 p-7 sm:p-9 shadow-2xl shadow-slate-950/30 border border-slate-200/80 hover:border-amber-400/80 transition-all duration-300 flex flex-col items-center justify-between text-center relative overflow-hidden">

                    {/* Top Decorative accent bar */}
                    <div className="absolute top-0 left-0 right-0 h-1.5 bg-amber-400" />

                    <div className="flex flex-col items-center my-auto py-4">
                      <span className="inline-block text-xs font-black text-cyan-800 tracking-widest uppercase bg-cyan-50 border border-cyan-200 px-4 py-1 rounded-full mb-3 shadow-2xs">
                        {activeStory.highlightStatTitle}
                      </span>

                      <span className="block text-5xl sm:text-6xl font-black text-amber-500 tracking-tight my-2 drop-shadow-sm">
                        {activeStory.highlightStatNumber}
                      </span>

                      <p className="text-xs sm:text-sm font-bold text-slate-700 leading-snug max-w-[240px] mx-auto mt-2">
```

```
                        {activeStory.highlightStatDescription}
                      </p>
                    </div>

                    <div className="mt-4 w-full inline-flex items-center justify-center gap-1.5 text-xs font-bold text-emerald
-800 bg-emerald-50 border border-emerald-200 px-3.5 py-2.5 rounded-2xl shadow-2xs">
                      <UserCheck className="w-4 h-4 text-emerald-600 shrink-0" />
                      <span>MoES Certified Placement</span>
                    </div>
                  </div>
                </div>

              </div>

              {/* Interactive Slide Dots Indicator */}
              {stories.length > 1 && (
                <div className="flex items-center justify-center gap-2 mt-8 relative z-10">
                  {stories.map((s, idx) => {
                    const isActive = idx === currentIndex % stories.length
                    return (
                      <button
                        key={s.id || idx}
                        onClick={() => setCurrentIndex(idx)}
                        aria-label={`Go to slide ${idx + 1}`}
                        className={`transition-all duration-300 rounded-full cursor-pointer ${
                          isActive
                            ? 'w-8 h-2.5 bg-cyan-400 shadow-md shadow-cyan-500/50'
                            : 'w-2.5 h-2.5 bg-white/30 hover:bg-white/60'
                        }`}
                      />
                    )
                  })}
                </div>
              )}
            </motion.div>
          </AnimatePresence>

          {/* Slider Left / Right Navigation Controls */}
          {stories.length > 1 && (
            <>
              <button
                onClick={handlePrev}
                aria-label="Previous Achievement"
                className="absolute left-2 top-1/2 -translate-y-1/2 w-11 h-11 rounded-full bg-slate-900/90 hover:bg-slate-800
text-cyan-300 hover:text-white flex items-center justify-center shadow-2xl backdrop-blur-md transition-all hover:scale-110 active:sc
ale-95 border border-cyan-500/40 hover:border-cyan-400 z-20 cursor-pointer"
              >
                <ChevronLeft className="w-6 h-6" />
              </button>

              <button
                onClick={handleNext}
                aria-label="Next Achievement"
                className="absolute right-2 top-1/2 -translate-y-1/2 w-11 h-11 rounded-full bg-slate-900/90 hover:bg-slate-800
text-cyan-300 hover:text-white flex items-center justify-center shadow-2xl backdrop-blur-md transition-all hover:scale-110 active:sc
ale-95 border border-cyan-500/40 hover:border-cyan-400 z-20 cursor-pointer"
              >
                <ChevronRight className="w-6 h-6" />
              </button>
            </>
          )}
        </div>

      </div>
    </section>
  )
}

```

src/components/landing/DynamicUpdatesSection.tsx TSX 15.1 KB 304 Lines

```tsx
import React from 'react';
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@/lib/supabase';
import { motion, type Variants } from 'framer-motion';
import { Megaphone, Calendar, ArrowRight, Sparkles, Clock, Compass, Award } from 'lucide-react';
import { Link } from 'react-router-dom';
import { Thumbnail } from '@/components/ui/Thumbnail';

const stagger: Variants = { hidden: {}, visible: { transition: { staggerChildren: 0.1 } } };
const fadeUp: Variants = { hidden: { opacity: 0, y: 20 }, visible: { opacity: 1, y: 0, transition: { duration: 0.5, ease: 'easeOut' } } };

interface CourseCardItem {
  id: string;
  title: string;
  description: string | null;
  course_type?: string;
  department?: string | null;
  duration_minutes?: number | null;
  thumbnail_path?: string | null;
  created_at?: string;
  trainer?: { full_name?: string } | null;
  trainers?: { full_name?: string } | null;
  rating?: number;
  enrolled_count?: string;
  badge?: string;
  custom_link?: string;
}

// Curated flagship fallback to guarantee 3 cards if database has fewer than 3 published courses
const fallbackCourses: CourseCardItem[] = [
  {
    id: 'doppler-radar-nowcasting',
    title: 'Doppler Weather Radar & Advanced Nowcasting Systems',
    description: 'Master operational radar telemetry, severe storm identification, mesocyclone tracking, and rapid nowcasting protocols for extreme weather events.',
    course_type: 'Technical',
    department: 'IMD, MoES',
    duration_minutes: 4320,
    trainer: { full_name: 'Dr. V. Ramakrishnan' },
    thumbnail_path: null,
    rating: 4.9,
    enrolled_count: '480+',
    badge: 'Popular',
    created_at: new Date().toISOString(),
  },
  {
    id: 'numerical-weather-prediction',
    title: 'Numerical Weather Prediction & Deep Learning Modeling',
    description: 'Comprehensive curriculum on high-performance computing, atmospheric dynamical equations, and AI-accelerated global ensemble forecast models.',
    course_type: 'Specialist',
    department: 'NCMRWF, MoES',
    duration_minutes: 5400,
    trainer: { full_name: 'Dr. A. Sengupta' },
    thumbnail_path: null,
    rating: 4.8,
    enrolled_count: '320+',
    badge: 'Featured',
    created_at: new Date().toISOString(),
  },
];

import { useHomePageSettings } from '@/hooks/useHomePageSettings';

export function DynamicUpdatesSection() {
  const { settings } = useHomePageSettings();

  const { data: announcements } = useQuery({
    queryKey: ['landing_announcements'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('announcements')
        .select('*')
        .eq('is_active', true)
```

```
        .order('created_at', { ascending: false })
        .limit(3);
      if (error) return [];
      return data || [];
    }
  });

  const { data: dbCourses } = useQuery({
    queryKey: ['landing_top_courses'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('courses')
        .select(`
          id,
          title,
          description,
          course_type,
          department,
          duration_minutes,
          thumbnail_path,
          created_at,
          trainer:trainers!courses_trainer_id_fkey(full_name)
        `)
        .eq('status', 'published')
        .order('created_at', { ascending: false })
        .limit(3);
      if (error) return [];
      return (data as unknown as CourseCardItem[]) || [];
    }
  });

  // Combine admin customized items or database courses with high-quality fallback cards

  const displayedCourses = React.useMemo<CourseCardItem[]>(() => {
    if (settings?.featured_programs_items && settings.featured_programs_items.length > 0) {
      return settings.featured_programs_items.map((item, index) => ({
        id: item.id || `prog-${index}`,
        title: item.title,
        description: item.description,
        department: item.department || 'MoES Specialist Track',
        course_type: 'Technical',
        duration_minutes: (item.duration_hours || 40) * 60,
        trainer: { full_name: item.trainer_name || 'MoES Scientific Faculty' },
        badge: item.badge || (index === 0 ? 'Top Rated' : index === 1 ? 'Featured' : 'Popular'),
        thumbnail_path: item.thumbnail_url || null,
        custom_link: item.link || '/courses',
      }));
    }

    const list: CourseCardItem[] = [...(dbCourses || [])];
    let fallbackIdx = 0;
    while (list.length < 3 && fallbackIdx < fallbackCourses.length) {
      list.push(fallbackCourses[fallbackIdx]);
      fallbackIdx++;
    }
    return list.slice(0, 3);
  }, [settings?.featured_programs_items, dbCourses]);

  // If section is toggled off by admin, do not render

  if (settings && !settings.featured_programs_enabled) {
    return null;
  }

  const sectionTag = settings?.featured_programs_tag || 'Top Certified Tracks';
  const sectionTitle = settings?.featured_programs_title || 'Featured Learning Programs';
  const sectionSubtitle = settings?.featured_programs_subtitle || 'Explore government-certified, high-impact earth science and meteorological training tracks led by premier MoES scientific institutions.';
  const btnText = settings?.featured_programs_btn_text || 'Explore All Courses';
  const btnLink = settings?.featured_programs_btn_link || '/courses';

  return (
    <section className="relative py-20 sm:py-28 bg-gradient-to-b from-[#f8fafc] via-white to-[#f1f5f9] text-slate-900 border-t border-slate-200 overflow-hidden">
      {/* Decorative ambient subtle radial lighting */}
      <div className="pointer-events-none absolute top-0 left-1/2 -translate-x-1/2 w-full max-w-7xl h-px bg-gradient-to-r from-transparent via-cyan-500/20 to-transparent opacity-60" />
      <div className="pointer-events-none absolute top-20 left-1/4 w-[500px] h-[500px] rounded-full bg-cyan-500/5 blur-[140px]" />
```

```
      <div className="pointer-events-none absolute bottom-10 right-1/4 w-[400px] h-[400px] rounded-full bg-amber-500/5 blur-[130px]" />

      <div className="max-w-7xl mx-auto px-6 relative z-10 space-y-16">

        {/* Optional Active Announcements Banner / Section */}
        {announcements && announcements.length > 0 && (
          <motion.div initial="hidden" whileInView="visible" viewport={{ once: true }} variants={fadeUp} className="p-6 rounded-3xl bg-white/90 backdrop-blur-md border border-slate-200/80 shadow-xl shadow-slate-200/60">
            <div className="flex items-center gap-2 mb-4">
              <span className="px-3 py-1 rounded-full bg-cyan-50 border border-cyan-200 text-cyan-700 text-xs font-semibold uppercase tracking-wider flex items-center gap-1.5 shadow-sm">
                <Megaphone className="w-3.5 h-3.5 text-cyan-600" /> Latest Announcements
              </span>
            </div>
            <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
              {announcements.map((ann: any) => (
                <div key={ann.id} className="p-4 rounded-2xl bg-slate-50/80 border border-slate-200/60 hover:border-cyan-500/40 hover:bg-white transition-all shadow-sm">
                  <div className="flex items-center gap-2 text-xs text-slate-500 mb-2">
                    <Calendar className="w-3 h-3 text-cyan-600" />
                    <span>{new Date(ann.created_at).toLocaleDateString()}</span>
                  </div>
                  <h4 className="font-bold text-slate-900 text-sm line-clamp-1 mb-1">{ann.title}</h4>
                  <p className="text-xs text-slate-600 line-clamp-2">{ann.content}</p>
                </div>
              ))}
            </div>
          </motion.div>
        )}

        {/* Top Courses Section */}
        <div className="space-y-10">
          {/* Section Header */}
          <div className="flex flex-col md:flex-row md:items-end justify-between gap-6">
            <motion.div initial="hidden" whileInView="visible" viewport={{ once: true }} variants={fadeUp} className="max-w-2xl">
              <div className="inline-flex items-center gap-2 px-3.5 py-1 rounded-full bg-amber-50 border border-amber-200 text-amber-800 text-xs font-semibold tracking-wider uppercase mb-4 shadow-sm">
                <Sparkles className="w-3.5 h-3.5 text-amber-600" /> {sectionTag}
              </div>
              <h2 className="text-3xl sm:text-4xl lg:text-5xl font-bold font-display text-slate-900 tracking-[-0.02em] leading-tight mb-3">
                {sectionTitle}
              </h2>
              <p className="text-slate-600 text-sm sm:text-base leading-relaxed">
                {sectionSubtitle}
              </p>
            </motion.div>

            <motion.div initial="hidden" whileInView="visible" viewport={{ once: true }} variants={fadeUp}>
              <Link to={btnLink}>
                <button className="h-11 px-6 rounded-full border border-cyan-600/30 bg-cyan-600 text-white hover:bg-cyan-700 text-sm font-semibold transition-all duration-300 flex items-center gap-2 shadow-lg shadow-cyan-600/20 group">
                  <Compass className="w-4 h-4 text-cyan-200 group-hover:rotate-45 transition-transform" />
                  <span>{btnText}</span>
                  <ArrowRight className="w-4 h-4 group-hover:translate-x-1 transition-transform" />
                </button>
              </Link>
            </motion.div>
          </div>

          {/* 3 Cards Grid */}
          <motion.div
            variants={stagger}
            initial="hidden"
            whileInView="visible"
            viewport={{ once: true, amount: 0.1 }}
            className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6"
          >
            {displayedCourses.map((course: any, idx: number) => {
              const trainerName = (course.trainer as any)?.full_name || (course.trainers as any)?.full_name || 'MoES Scientific Faculty';
              const durationHours = course.duration_minutes ? Math.round(course.duration_minutes / 60) : 60;
              const isDatabaseCourse = !course.id.startsWith('doppler-') && !course.id.startsWith('numerical-') && !course.id.startsWith('prog-');
              const courseLink = course.custom_link || (isDatabaseCourse ? `/courses/${course.id}` : '/courses');

              return (
```

```
                <motion.div
                  key={course.id || idx}
                  variants={fadeUp}
                  className="group flex flex-col bg-white border border-slate-200/90 hover:border-cyan-500/50 rounded-3xl overflow-hidden shadow-xl shadow-slate-200/50 hover:shadow-2xl hover:shadow-cyan-500/10 transition-all duration-500 hover:-translate-y-1.5"
                >
                  {/* Thumbnail Banner */}
                  <div className="h-48 relative overflow-hidden bg-slate-900">
                    {course.thumbnail_path && (course.thumbnail_path.startsWith('http') || course.thumbnail_path.startsWith('data:') || course.thumbnail_path.startsWith('/')) ? (
                      <img
                        src={course.thumbnail_path}
                        alt={course.title}
                        className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-700"
                      />
                    ) : (
                      <Thumbnail
                        path={course.thumbnail_path}
                        alt={course.title}
                        type={course.course_type}
                        className="group-hover:scale-105 transition-transform duration-700"
                      />
                    )}

                    {/* Gradient Overlay */}
                    <div className="absolute inset-0 bg-gradient-to-t from-slate-900/80 via-transparent to-black/30 pointer-events-none" />

                    {/* Top Badges */}
                    <div className="absolute top-3.5 left-3.5 right-3.5 flex items-center justify-between pointer-events-none">
                      <span className="px-2.5 py-1 rounded-full text-[10px] font-bold uppercase tracking-wider bg-black/75 backdrop-blur-md text-amber-300 border border-amber-500/30 flex items-center gap-1 shadow-sm">
                        <Award className="w-3 h-3 text-amber-300" />
                        {course.badge || (idx === 0 ? 'Top Rated' : idx === 1 ? 'Featured' : 'Certified')}
                      </span>

                      <span className="px-2.5 py-1 rounded-full text-[11px] font-semibold text-slate-100 bg-black/75 backdrop-blur-md border border-white/15 flex items-center gap-1 shadow-sm">
                        <Clock className="w-3 h-3 text-cyan-400" />
                        <span>{durationHours}h Track</span>
                      </span>
                    </div>

                    {/* Department Tag at Bottom of Image */}
                    <div className="absolute bottom-3 left-3.5 pointer-events-none">
                      <span className="px-2.5 py-0.5 rounded-md text-[10px] font-mono font-semibold uppercase tracking-wider bg-cyan-950/90 text-cyan-300 border border-cyan-500/40">
                        {course.department || 'MoES Certified'}
                      </span>
                    </div>
                  </div>

                  {/* Card Body */}
                  <div className="p-6 flex-1 flex flex-col justify-between space-y-4 bg-white">
                    <div className="space-y-2">
                      <h3 className="font-bold text-lg sm:text-xl text-slate-900 group-hover:text-cyan-600 transition-colors line-clamp-2 leading-snug tracking-[-0.01em]">
                        {course.title}
                      </h3>

                      <p className="text-xs sm:text-sm text-slate-600 line-clamp-2 leading-relaxed">
                        {course.description || 'Comprehensive government curriculum designed for operational forecasting and earth science excellence.'}
                      </p>
                    </div>

                    {/* Footer Row */}
                    <div className="pt-4 border-t border-slate-100 flex items-center justify-between gap-2 mt-auto">
                      <div className="flex items-center gap-2 truncate max-w-[150px]">
                        <div className="w-6 h-6 rounded-full bg-gradient-to-br from-cyan-600 to-blue-700 flex items-center justify-center text-white text-[10px] font-bold shrink-0">
                          {trainerName.charAt(0).toUpperCase()}
                        </div>
                        <span className="text-xs text-slate-500 truncate font-medium">
                          {trainerName}
                        </span>
                      </div>
```

```
283 |
284 |                       <Link
285 |                         to={courseLink}
286 |                         className="inline-flex items-center gap-1 text-xs font-bold text-cyan-600 hover:text-cyan-700 group-ho
ver:gap-1.5 transition-all"
287 |                       >
288 |                         <span>{isDatabaseCourse ? 'View Details' : 'Enroll Now'}</span>
289 |                         <ArrowRight className="w-3.5 h-3.5" />
290 |                       </Link>
291 |                     </div>
292 |                   </div>
293 |                 </motion.div>
294 |               );
295 |             })}
296 |           </motion.div>
297 |         </div>
298 |
299 |       </div>
300 |     </section>
301 |   );
302 | }
303 |
304 |
```

src/components/landing/HorizontalAnnouncementBar.tsx TSX 7.4 KB 193 Lines

```tsx
import React from 'react';
import { Megaphone, Bell, Radio, Sparkles, AlertCircle, ArrowUpRight } from 'lucide-react';
import { useQuery } from '@tanstack/react-query';
import { supabase } from '@/lib/supabase';
import { Link } from 'react-router-dom';

const defaultAnnouncements = [
  {
    id: 'ann-1',
    tag: 'MoES Live',
    tagColor: 'bg-cyan-500/20 text-cyan-300 border-cyan-500/30',
    title: 'IMD Doppler Radar Network Calibration Workshop Open for Trainee Registration',
    icon: Radio,
    link: '/courses',
  },
  {
    id: 'ann-2',
    tag: 'INCOIS Notice',
    tagColor: 'bg-amber-500/20 text-amber-300 border-amber-500/30',
    title: 'Ocean State Forecast & In-Situ Buoy Data Analytics Winter Cohort Starts Oct 15',
    icon: Sparkles,
    link: '/courses',
  },
  {
    id: 'ann-3',
    tag: 'Certification',
    tagColor: 'bg-emerald-500/20 text-emerald-300 border-emerald-500/30',
    title: 'New Government-Accredited Micro-Credential Standards Published for Field Scientists',
    icon: Bell,
    link: '/courses',
  },
  {
    id: 'ann-4',
    tag: 'Operational Alert',
    tagColor: 'bg-sky-500/20 text-sky-300 border-sky-500/30',
    title: 'Numerical Weather Prediction (NWP) High-Performance Computing Portal Now Active',
    icon: AlertCircle,
    link: '/courses',
  },
  {
    id: 'ann-5',
    tag: 'Admissions',
    tagColor: 'bg-purple-500/20 text-purple-300 border-purple-500/30',
    title: 'Dual-Pol Radar Nowcasting Batch Enrollment Closing Soon — Reserve Your Seat',
    icon: Megaphone,
    link: '/register',
  },
];

import { useHomePageSettings } from '@/hooks/useHomePageSettings';

export function HorizontalAnnouncementBar() {
  const { settings } = useHomePageSettings();

  const { data: dbAnnouncements } = useQuery({
    queryKey: ['landing_horizontal_announcements'],
    queryFn: async () => {
      const { data, error } = await supabase
        .from('announcements')
        .select('*')
        .eq('is_active', true)
        .order('created_at', { ascending: false })
        .limit(6);
      if (error) return [];
      return data || [];
    },
  });

  const getTagColorClass = (color?: string) => {
    switch (color) {
      case 'amber':
        return 'bg-amber-500/20 text-amber-300 border-amber-500/30';
      case 'emerald':
        return 'bg-emerald-500/20 text-emerald-300 border-emerald-500/30';
      case 'sky':
        return 'bg-sky-500/20 text-sky-300 border-sky-500/30';
```

```
      case 'purple':
        return 'bg-purple-500/20 text-purple-300 border-purple-500/30';
      case 'cyan':
      default:
        return 'bg-cyan-500/20 text-cyan-300 border-cyan-500/30';
    }
  };

  // Prioritize admin configured items when present

  const announcementsList = React.useMemo(() => {
    if (settings?.announcements_items && settings.announcements_items.length > 0) {
      return settings.announcements_items.map((item, idx) => ({
        id: item.id || `admin-ann-${idx}`,
        tag: item.tag || 'Notice',
        tagColor: getTagColorClass(item.tag_color),
        title: item.title,
        icon: Megaphone,
        link: item.link || '/courses',
      }));
    }

    const items: Array<{
      id: string;
      tag: string;
      tagColor: string;
      title: string;
      icon: any;
      link?: string;
    }> = [];

    if (dbAnnouncements && dbAnnouncements.length > 0) {
      dbAnnouncements.forEach((ann: any) => {
        items.push({
          id: ann.id,
          tag: 'Official Update',
          tagColor: 'bg-cyan-500/20 text-cyan-300 border-cyan-500/30',
          title: ann.title || ann.content,
          icon: Megaphone,
          link: '/courses',
        });
      });
    }

    // Append fallback curated items only if no admin items exist

    defaultAnnouncements.forEach((d) => {
      if (!items.some((i) => i.title === d.title)) {
        items.push(d);
      }
    });

    return items;
  }, [settings?.announcements_items, dbAnnouncements]);

  // If toggled off by admin, do not render

  if (settings && !settings.announcements_bar_enabled) {
    return null;
  }

  // Quadruple items so there is a rich uninterrupted stream on any screen size

  const marqueeItems = [
    ...announcementsList,
    ...announcementsList,
    ...announcementsList,
    ...announcementsList,
  ];

  return (
    <div className="group/bar relative w-full bg-[#020612] border-y border-cyan-500/20 py-3 overflow-hidden select-none z-20 s
hadow-xl shadow-black/60">
      <div className="w-full flex items-center relative">
        {/* Left side fixed pill label */}
        <div className="flex items-center gap-2 pl-5 pr-5 py-1 border-r border-cyan-500/30 z-30 shrink-0 bg-[#020612] shadow-m
d shadow-black/80">
          <span className="relative flex h-2.5 w-2.5">
            <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-cyan-400 opacity-75" />
```

```
150 |             <span className="relative inline-flex rounded-full h-2.5 w-2.5 bg-cyan-400" />
151 |           </span>
152 |           <span className="text-xs font-bold uppercase tracking-wider text-cyan-300 font-display flex items-center gap-1.5 whi
    | tespace-nowrap">
153 |             <Megaphone className="w-3.5 h-3.5 text-cyan-400" /> {settings?.announcements_bar_label && settings.announcements_b
    | ar_label !== 'Announcements' ? settings.announcements_bar_label : 'Updates'}
154 |           </span>
155 |         </div>
156 |
157 |         {/* Right gradient edge fade for seamless text exit */}
158 |         <div className="pointer-events-none absolute right-0 top-0 bottom-0 w-16 sm:w-24 bg-gradient-to-l from-[#020612] to-tr
    | ansparent z-20" />
159 |
160 |         {/* Pure CSS Infinite Marquee Track (Smooth, comfortable reading speed, zero-flicker instant hover pause) */}
161 |         <div className="flex-1 overflow-hidden relative cursor-pointer pl-4">
162 |           <div
163 |             className="animate-marquee-infinite flex items-center gap-8 w-max whitespace-nowrap will-change-transform group-ho
    | ver/bar:[animation-play-state:paused]"
164 |             style={{ animationDuration: `${(settings?.announcements_bar_speed && settings.announcements_bar_speed >= 40) ? set
    | tings.announcements_bar_speed : 65}s` }}
165 |           >
166 |             {marqueeItems.map((item, idx) => {
167 |               return (
168 |                 <Link
169 |                   key={`${item.id}-${idx}`}
170 |                   to={item.link || '/courses'}
171 |                   className="inline-flex items-center gap-3 text-slate-200 hover:text-cyan-300 transition-colors group/item sh
    | rink-0"
172 |                 >
173 |                   <span className={`px-2.5 py-0.5 rounded-full text-[10px] font-bold uppercase tracking-wider border ${item.ta
    | gColor} shrink-0`}>
174 |                     {item.tag}
175 |                   </span>
176 |
177 |                   <span className="text-xs sm:text-sm font-medium tracking-[-0.01em] text-slate-200 group-hover/item:text-cyan
    | -300 transition-colors flex items-center gap-1">
178 |                     {item.title}
179 |                     <ArrowUpRight className="w-3.5 h-3.5 opacity-60 group-hover/item:opacity-100 group-hover/item:translate-x-
    | 0.5 group-hover/item:-translate-y-0.5 transition-all text-cyan-400" />
180 |                   </span>
181 |
182 |                   {/* Dot separator */}
183 |                   <span className="w-1.5 h-1.5 rounded-full bg-cyan-500/40 ml-4 shrink-0" />
184 |                 </Link>
185 |               );
186 |             })}
187 |           </div>
188 |         </div>
189 |       </div>
190 |     </div>
191 |   );
192 | }
193 |
```

src/components/landing/MissionExplodedView.tsx TSX 10.7 KB 299 Lines

```tsx
import React, { useEffect, useRef, useState } from 'react';
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { motion, AnimatePresence } from 'framer-motion';

if (typeof window !== 'undefined') {
  gsap.registerPlugin(ScrollTrigger);
}

export interface MissionExplodedViewProps {
  /** Optional custom section ID for anchor navigation */
  id?: string;
  /** Optional className for outer section container */
  className?: string;
}

interface MissionPhase {
  id: string;
  headlinePrefix: string;
  headlineHighlight: string;
  highlightGradient: string;
  subtext: string;
}

const PHASES: MissionPhase[] = [
  {
    id: 'phase-1',
    headlinePrefix: "Strengthening India's Meteorological Readiness for a ",
    headlineHighlight: 'Changing Climate',
    highlightGradient: 'from-cyan-400 via-sky-300 to-amber-400',
    subtext:
      'One platform, every skill IMD and MoES need — cyclone tracking, monsoon forecasting, seismology, ocean observation — bu
ilt for the people doing the actual work.',
  },
  {
    id: 'phase-2',
    headlinePrefix: 'Explore Layer-by-Layer Sensor & Payload Diagnostics in ',
    headlineHighlight: 'Interactive 3D',
    highlightGradient: 'from-cyan-400 via-sky-300 to-blue-400',
    subtext:
      'Dissect solar arrays, Doppler radar modules, and telemetry transponders in simulated labs — master internal mechanics b
efore live station deployment.',
  },
  {
    id: 'phase-3',
    headlinePrefix: 'Real-Time Orbital Telemetry to ',
    headlineHighlight: 'Ground Command',
    highlightGradient: 'from-amber-400 via-orange-400 to-amber-300',
    subtext:
      'Seamlessly connect orbital feeds to 500+ AWS calibration nodes, Doppler radar arrays, and numerical prediction supercom
puters across India.',
  },
];

export function MissionExplodedView({
  id = 'mission-section',
  className = '',
}: MissionExplodedViewProps) {
  const sectionRef = useRef<HTMLDivElement | null>(null);
  const pinWrapperRef = useRef<HTMLDivElement | null>(null);
  const videoRef = useRef<HTMLVideoElement | null>(null);

  const [activePhaseIdx, setActivePhaseIdx] = useState(0);
  const activePhaseRef = useRef(0);
  const [isVideoLoaded, setIsVideoLoaded] = useState(false);
  const [isReducedMotion, setIsReducedMotion] = useState(false);

  // Check for prefers-reduced-motion

  useEffect(() => {
    const checkMotion = () => {
      const prefersReduced = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
      setIsReducedMotion(prefersReduced);
    };
```

```
    checkMotion();
    const mediaQuery = window.matchMedia('(prefers-reduced-motion: reduce)');
    mediaQuery.addEventListener('change', checkMotion);
    return () => mediaQuery.removeEventListener('change', checkMotion);
  }, []);

  const handleLoadedMetadata = () => {
    setIsVideoLoaded(true);
    ScrollTrigger.refresh();
  };

  useEffect(() => {
    const video = videoRef.current;
    if (video) {
      if (video.readyState >= 1) {
        setIsVideoLoaded(true);
        ScrollTrigger.refresh();
      }
      video.currentTime = 0.001;
    }
  }, []);

  // Smooth ScrollTrigger & video seeking

  useEffect(() => {
    const section = sectionRef.current;
    const pinWrapper = pinWrapperRef.current;
    const video = videoRef.current;

    if (!section || !pinWrapper) return;

    ScrollTrigger.getAll().forEach((st) => {
      if (st.trigger === section || st.trigger === pinWrapper) {
        st.kill(true);
      }
    });

    let targetProgress = 0;
    let currentProgress = 0;
    let isSeeking = false;
    let isRunning = true;

    const onSeeked = () => {
      isSeeking = false;
    };

    if (video) {
      video.addEventListener('seeked', onSeeked);
    }

    const ticker = () => {
      if (!isRunning || !video || isNaN(video.duration) || video.duration <= 0) return;

      const diff = targetProgress - currentProgress;
      if (Math.abs(diff) > 0.001) {
        currentProgress += diff * 0.15;
        const targetTime = currentProgress * video.duration;

        if (!isSeeking && Math.abs(video.currentTime - targetTime) > 0.02) {
          isSeeking = true;
          try {
            if ('fastSeek' in video && typeof (video as any).fastSeek === 'function') {
              (video as any).fastSeek(targetTime);
            } else {
              video.currentTime = targetTime;
            }
          } catch {
            video.currentTime = targetTime;
            isSeeking = false;
          }
        }
      }
    };

    gsap.ticker.add(ticker);

    const ctx = gsap.context(() => {
      if (!isReducedMotion) {
```

```
        const isMobile = typeof window !== 'undefined' && window.innerWidth < 768;

        ScrollTrigger.create({
          trigger: section,
          pin: pinWrapper,
          pinSpacing: true,
          start: 'top top',
          end: () => (isMobile ? '+=180%' : '+=220%'),
          scrub: isMobile ? 0.4 : 0.6,
          anticipatePin: 1,
          invalidateOnRefresh: true,
          onUpdate: (self) => {
            const p = self.progress;
            targetProgress = p;

            let nextPhase = 0;
            if (p >= 0.66) {
              nextPhase = 2;
            } else if (p >= 0.33) {
              nextPhase = 1;
            } else {
              nextPhase = 0;
            }

            if (activePhaseRef.current !== nextPhase) {
              activePhaseRef.current = nextPhase;
              setActivePhaseIdx(nextPhase);
            }
          },
        });
      } else if (video && isReducedMotion) {
        ScrollTrigger.create({
          trigger: section,
          start: 'top 70%',
          end: 'bottom 20%',
          onEnter: () => video.play().catch(() => {}),
          onLeave: () => video.pause(),
          onEnterBack: () => video.play().catch(() => {}),
          onLeaveBack: () => video.pause(),
        });
      }
    }, section);

    const refreshTimer = setTimeout(() => {
      ScrollTrigger.refresh();
    }, 200);

    const handleWindowLoad = () => ScrollTrigger.refresh();
    window.addEventListener('load', handleWindowLoad);

    return () => {
      isRunning = false;
      gsap.ticker.remove(ticker);
      if (video) {
        video.removeEventListener('seeked', onSeeked);
      }
      clearTimeout(refreshTimer);
      window.removeEventListener('load', handleWindowLoad);
      ctx.revert();
    };
  }, [isVideoLoaded, isReducedMotion]);

  const currentPhase = PHASES[activePhaseIdx] || PHASES[0];

  return (
    <section
      id={id}
      ref={sectionRef}
      className={`relative w-full bg-[#030712] text-white ${className}`}
    >
      {/* Pinned Viewport Container */}
      <div
        ref={pinWrapperRef}
        className="relative w-full h-[100dvh] min-h-[600px] overflow-hidden flex items-center justify-center bg-[#030712]"
      >
        {/* ========================================================================== */}
        {/* VIDEO LAYER: Clear, Bright, Edge Softening Only                            */}
        {/* ========================================================================== */}
        <div className="absolute inset-0 w-full h-full pointer-events-none z-0 overflow-hidden bg-[#030712]">
```

```
          <video
            ref={videoRef}
            muted
            playsInline
            preload="auto"
            disablePictureInPicture
            disableRemotePlayback
            onLoadedMetadata={handleLoadedMetadata}
            className={`w-full h-full object-cover object-center brightness-[0.98] contrast-[1.05] saturate-[1.1] transition-opacity duration-700 ${
              isVideoLoaded ? 'opacity-100' : 'opacity-0'
            }`}
          >
            <source
              src="/home%20page/i_dont_want_the_blue_lines_com%20(1).mp4"
              type="video/mp4"
            />
            <source
              src="/home page/i_dont_want_the_blue_lines_com (1).mp4"
              type="video/mp4"
            />
          </video>

          {/* Minimal top and bottom boundary blend */}
          <div className="absolute inset-x-0 top-0 h-20 bg-gradient-to-b from-[#030712] to-transparent pointer-events-none z-10" />
          <div className="absolute inset-x-0 bottom-0 h-20 bg-gradient-to-t from-[#030712] to-transparent pointer-events-none z-10" />
        </div>

        {/* Ambient atmospheric backdrop glow behind floating text */}
        <div className="absolute inset-0 pointer-events-none z-10 flex items-center justify-center">
          <div className="w-[500px] h-[300px] bg-cyan-500/10 rounded-full blur-[100px] opacity-60" />
        </div>

        {/* ========================================================================== */}
        {/* CONTENT LAYER: Pure Centered Floating Typography                           */}
        {/* ========================================================================== */}
        <div className="relative z-30 max-w-4xl mx-auto px-6 sm:px-8 w-full flex items-center justify-center pointer-events-none">
          <div className="relative w-full flex items-center justify-center min-h-[220px] overflow-visible">
            <AnimatePresence mode="wait">
              <motion.div
                key={currentPhase.id}
                initial={{ opacity: 0, y: 16 }}
                animate={{ opacity: 1, y: 0 }}
                exit={{ opacity: 0, y: -16 }}
                transition={{ duration: 0.35, ease: 'easeOut' }}
                className="w-full flex flex-col items-center justify-center text-center px-4"
              >
                <div className="flex flex-col items-center max-w-3xl mx-auto">
                  {/* Headline - Pure Floating Typography with Atmospheric Palette */}
                  <h2 className="text-3xl sm:text-4xl md:text-5xl lg:text-6xl font-extrabold font-display text-white tracking-[-0.03em] leading-[1.15] mb-4 drop-shadow-[0_4px_30px_rgba(0,0,0,0.98)]">
                    {currentPhase.headlinePrefix}
                    <span
                      className={`text-transparent bg-clip-text bg-gradient-to-r ${currentPhase.highlightGradient} drop-shadow-none`}
                    >
                      {currentPhase.headlineHighlight}
                    </span>
                  </h2>

                  {/* Subtext - Crisp high-contrast titanium tone */}
                  <p className="text-slate-100 text-sm sm:text-base md:text-lg leading-relaxed font-normal max-w-2xl mx-auto drop-shadow-[0_2px_16px_rgba(0,0,0,0.98)]">
                    {currentPhase.subtext}
                  </p>
                </div>
              </motion.div>
            </AnimatePresence>
          </div>
        </div>
      </div>
    </section>
  );
}
```

```
298 |
299 |
```

src/components/landing/StickyScrollFeatures.tsx TSX 15.6 KB 346 Lines

```tsx
import React, { useEffect, useRef } from 'react';
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { Lock, Zap, Globe, Radio, Cpu, Activity } from 'lucide-react';

gsap.registerPlugin(ScrollTrigger);

interface Step {
  id: string;
  stepNum: string;
  badge: string;
  title: string;
  subtitle: string;
  description: string;
  image: string;
  imageAlt: string;
  imagePosition?: string;
  icon: React.ElementType;
  highlights: { title: string; desc: string; icon: React.ElementType }[];
  metrics: { label: string; value: string };
  badgeColor: string;
  glowColor: string;
}

const steps: Step[] = [
  {
    id: 'step-1',
    stepNum: '01',
    badge: 'Real-Time Learning & Security',
    title: 'Secure, Real-Time Learning Environment',
    subtitle: 'Practice on live-feeling telemetry safely with strict role separation.',
    description:
      'Interactive capacity building modules stream realistic satellite and radar telemetry into simulated forecasting labs wi
th granular role-based access.',
    image: '/home page/feature-earth-orbit.webp',
    imageAlt: 'High-resolution space view of Earth with atmospheric clouds and satellite orbit',
    imagePosition: 'object-top',
    icon: Lock,
    highlights: [
      {
        title: 'Secure & Role-Based Access',
        desc: 'Access locked strictly to your operational station and division.',
        icon: Lock,
      },
      {
        title: 'Real-Time Telemetry Labs',
        desc: 'Instant feedback simulations and live Doppler radar streams.',
        icon: Zap,
      },
    ],
    metrics: { label: 'Security Level', value: 'RLS Enforced' },
    badgeColor: 'border-cyan-500/30 text-cyan-300 bg-cyan-950/60',
    glowColor: 'rgba(6, 182, 212, 0.25)',
  },
  {
    id: 'step-2',
    stepNum: '02',
    badge: 'Field Ops & Regional Network',
    title: 'Pan-India Observational & Field Operations Training',
    subtitle: "One unified training standard for India's entire observation grid.",
    description:
      'Engineered for meteorologists and technical officers maintaining India’s weather network. Master AWS sensor calibratio
n, Doppler radar diagnostics, and standard reporting.',
    image: '/home page/feature-ground-station.webp',
    imageAlt: 'Meteorological ground station and telescope tracking system under night sky',
    icon: Radio,
    highlights: [
      {
        title: 'Observational Ground Ops',
        desc: 'Interactive guides for AWS sensor calibration and field telemetry.',
        icon: Radio,
      },
      {
        title: 'Multi-Regional Network',
        desc: 'New Delhi, Mumbai, Chennai, Kolkata, Guwahati — unified and synchronized.',
        icon: Globe,
```

```
      },
    ],
    metrics: { label: 'Regional Coverage', value: 'Pan-India RMCs' },
    badgeColor: 'border-sky-500/30 text-sky-300 bg-sky-950/60',
    glowColor: 'rgba(56, 189, 248, 0.25)',
  },
  {
    id: 'step-3',
    stepNum: '03',
    badge: 'AI Analytics & Compliance',
    title: 'AI Competency Engine & Standards Compliance',
    subtitle: 'Adaptive learning paths built around official MoES and WMO benchmarks.',
    description:
      'Diagnostic AI assessments map technical proficiency against Ministry benchmarks, delivering tailored modules for cyclone tracking and climate modeling.',
    image: '/home page/feature-satellite-orbit.webp',
    imageAlt: 'Detailed rendering of Earth with glowing orbital satellites and solar panels',
    icon: Cpu,
    highlights: [
      {
        title: 'AI Competency Engine',
        desc: 'Targeted skill-gap closure customized for your operational track.',
        icon: Cpu,
      },
      {
        title: 'MoES & IMD Compliant',
        desc: 'Every certificate maps directly to national meteorological standards.',
        icon: Globe,
      },
    ],
    metrics: { label: 'Compliance Standard', value: 'MoES & WMO' },
    badgeColor: 'border-amber-500/30 text-amber-300 bg-amber-950/60',
    glowColor: 'rgba(245, 158, 11, 0.25)',
  },
];

export function StickyScrollFeatures() {
  const containerRef = useRef<HTMLElement | null>(null);
  const pinWrapperRef = useRef<HTMLDivElement | null>(null);
  const trackRef = useRef<HTMLDivElement | null>(null);

  useEffect(() => {
    // Kill any existing ScrollTrigger instances on this trigger to prevent duplicate pin-spacers

    ScrollTrigger.getAll().forEach((st) => {
      if (st.trigger === containerRef.current || st.trigger === pinWrapperRef.current) {
        st.kill(true);
      }
    });

    const ctx = gsap.context(() => {
      if (!trackRef.current || !pinWrapperRef.current || !containerRef.current) return;

      const panelsCount = steps.length;
      // Total horizontal shift to reveal the last panel: (panelsCount - 1) / panelsCount * 100%

      const totalXPercent = -((panelsCount - 1) / panelsCount) * 100;
      const isMobile = typeof window !== 'undefined' && window.innerWidth < 768;
      const getScrollDistance = () => (isMobile ? window.innerHeight * 1.4 : window.innerHeight * 2.0);

      gsap.to(trackRef.current, {
        xPercent: totalXPercent,
        ease: 'none',
        force3D: true,
        scrollTrigger: {
          trigger: containerRef.current,
          pin: pinWrapperRef.current,
          scrub: isMobile ? 0.4 : 0.6,
          start: 'top top',
          end: () => `+=${getScrollDistance()}`,
          invalidateOnRefresh: true,
          anticipatePin: 1,
        },
      });
    }, containerRef);

    const refreshHandler = () => ScrollTrigger.refresh();
    window.addEventListener('resize', refreshHandler);

```

```
    return () => {
      window.removeEventListener('resize', refreshHandler);
      ScrollTrigger.getAll().forEach((st) => {
        if (st.trigger === containerRef.current || st.trigger === pinWrapperRef.current) {
          st.kill(true);
        }
      });
      ctx.revert();
    };
  }, []);

  return (
    <section
      id="sticky-features"
      ref={containerRef}
      className="relative w-full bg-[#040814] text-white border-t border-cyan-500/20"
    >
      <div
        ref={pinWrapperRef}
        className="pin-wrapper relative w-full overflow-hidden h-[100dvh] max-h-[100dvh] min-h-[580px] flex flex-col justify-between"
      >
      {/* Decorative ambient atmospheric nebula glows */}
      <div className="pointer-events-none absolute top-1/4 -left-64 w-[600px] h-[600px] rounded-full bg-cyan-500/10 blur-[150px]" />
      <div className="pointer-events-none absolute bottom-1/4 -right-64 w-[600px] h-[600px] rounded-full bg-amber-500/10 blur-[150px]" />

      {/* Persistent Section Header */}
      <div className="pt-24 sm:pt-28 md:pt-32 pb-2 px-4 sm:px-6 max-w-7xl mx-auto w-full text-center relative z-20 shrink-0">
        <div className="inline-flex items-center gap-1.5 sm:gap-2 px-3 py-1 rounded-full bg-cyan-950/70 border border-cyan-500/30 text-cyan-300 text-[10px] sm:text-xs font-semibold tracking-wider uppercase mb-2 shadow-lg shadow-cyan-950/50">
          <Activity className="w-3.5 h-3.5 text-cyan-400 animate-pulse" />
          Core Platform Architecture
        </div>
        <h2 className="text-xl sm:text-2xl md:text-3xl lg:text-4xl font-extrabold font-display tracking-tight text-white mb-1.5 leading-tight">
          From Orbital Telemetry to{' '}
          <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 via-sky-300 to-amber-400">
            Ground-Level Preparedness
          </span>
        </h2>
        <p className="text-slate-400 text-xs sm:text-sm max-w-2xl mx-auto line-clamp-1">
          Built to meet the real demands of MoES field teams — from live telemetry streams to remote mountain outposts.
        </p>
      </div>

      {/* Horizontal Scroll Panels Track */}
      <div
        ref={trackRef}
        style={{ willChange: 'transform', transform: 'translateZ(0)' }}
        className="pin-track flex flex-row w-[300%] h-full items-center z-10 flex-1 min-h-0 py-1 overflow-hidden will-change-transform"
      >
        {steps.map((step, idx) => {
          const isEven = idx % 2 === 1;

          return (
            <div
              key={step.id}
              className="pin-panel w-[33.333333%] px-3 sm:px-6 md:px-8 lg:px-12 xl:px-16 flex items-center justify-center shrink-0 h-full max-h-full"
            >
              {/* MOBILE LAYOUT (< md) */}
              <div className="block md:hidden w-full max-w-sm mx-auto my-auto">
                <div className="p-3 sm:p-4 rounded-3xl bg-[#081022]/90 border border-cyan-500/20 backdrop-blur-xl shadow-2xl shadow-cyan-950/80 relative overflow-hidden flex flex-col gap-2">
                  <div
                    className="absolute -top-16 -right-16 w-36 h-36 rounded-full blur-3xl pointer-events-none opacity-40"
                    style={{ backgroundColor: step.glowColor }}
                  />

                  {/* Top Image Banner */}
                  <div className="relative w-full aspect-[2.2/1] rounded-2xl overflow-hidden bg-slate-900 border border-cyan-500/20 shadow-inner shrink-0">
                    <img
                      src={step.image}
                      alt={step.imageAlt}
                      className={`w-full h-full object-cover ${step.imagePosition || 'object-center'}`}
```

```
                      loading="lazy"
                    />
                    <div className="absolute inset-0 bg-gradient-to-t from-[#081022]/80 via-transparent to-transparent pointer-events-none" />
                  </div>

                  {/* Text Details */}
                  <div className="flex flex-col gap-1">
                    <h3 className="text-sm font-bold font-display text-white tracking-tight leading-snug">
                      {step.title}
                    </h3>
                    <p className="text-cyan-400 text-[10px] sm:text-[11px] font-semibold leading-tight">
                      {step.subtitle}
                    </p>
                    <p className="text-slate-300 text-[10px] sm:text-[11px] leading-relaxed line-clamp-2 font-normal">
                      {step.description}
                    </p>
                  </div>

                  {/* Highlights Pill List */}
                  <div className="space-y-1 mt-0.5">
                    {step.highlights.map((item) => {
                      const HIcon = item.icon;
                      return (
                        <div
                          key={item.title}
                          className="p-1.5 rounded-xl bg-[#040916] border border-cyan-500/20 flex items-center gap-2"
                        >
                          <div className="w-5 h-5 rounded-lg bg-cyan-950/80 border border-cyan-500/30 flex items-center justify-center shrink-0 text-cyan-400">
                            <HIcon className="w-3 h-3" />
                          </div>
                          <div className="min-w-0 flex-1">
                            <span className="text-[10px] font-semibold text-white block leading-none truncate">
                              {item.title}
                            </span>
                            <span className="text-[9px] text-slate-400 leading-none truncate block mt-0.5">
                              {item.desc}
                            </span>
                          </div>
                        </div>
                      );
                    })}
                  </div>
                </div>
              </div>

              {/* DESKTOP / TABLET / LAPTOP LAYOUT (>= md) */}
              <div className="hidden md:grid max-w-6xl w-full mx-auto grid-cols-2 lg:grid-cols-12 gap-4 lg:gap-6 xl:gap-8 items-center my-auto">
                {/* Text Content Block */}
                <div className={`w-full col-span-1 lg:col-span-6 ${isEven ? 'order-2' : 'order-1'}`}>
                  <div className="p-5 lg:p-7 rounded-3xl bg-[#081022]/90 border border-cyan-500/20 backdrop-blur-xl shadow-2xl shadow-cyan-950/80 relative overflow-hidden group">
                    <div
                      className="absolute -top-20 -right-20 w-44 h-44 rounded-full blur-3xl pointer-events-none opacity-40"
                      style={{ backgroundColor: step.glowColor }}
                    />

                    {/* Headline */}
                    <h3 className="text-lg lg:text-2xl font-bold font-display text-white mb-1.5 tracking-tight leading-snug">
                      {step.title}
                    </h3>

                    {/* Subtitle */}
                    <p className="text-cyan-400 text-xs lg:text-sm font-semibold mb-2 leading-relaxed">
                      {step.subtitle}
                    </p>

                    {/* Description */}
                    <p className="text-slate-300 text-xs lg:text-sm leading-relaxed mb-3 font-normal line-clamp-2 xl:line-clamp-3">
                      {step.description}
                    </p>

                    {/* Feature Highlights */}
                    <div className="space-y-2">
                      {step.highlights.map((item) => {
                        const HIcon = item.icon;
```

```
295 |                         return (
296 |                           <div
297 |                             key={item.title}
298 |                             className="p-2.5 rounded-xl bg-[#040916] border border-cyan-500/20 flex items-start gap-2.5 group/
item hover:border-cyan-400/40 transition-colors"
299 |                           >
300 |                             <div className="w-6 h-6 rounded-lg bg-cyan-950/80 border border-cyan-500/30 flex items-center just
ify-center shrink-0 mt-0.5 text-cyan-400">
301 |                               <HIcon className="w-3.5 h-3.5" />
302 |                             </div>
303 |                             <div className="min-w-0 flex-1">
304 |                               <h4 className="text-xs font-semibold text-white mb-0.5 leading-none">
305 |                                 {item.title}
306 |                               </h4>
307 |                               <p className="text-[11px] text-slate-400 leading-relaxed">
308 |                                 {item.desc}
309 |                               </p>
310 |                             </div>
311 |                           </div>
312 |                         );
313 |                       })}
314 |                     </div>
315 |                   </div>
316 |                 </div>
317 |
318 |                 {/* Visual Image Block */}
319 |                 <div className={`w-full col-span-1 lg:col-span-6 ${isEven ? 'order-1' : 'order-2'}`}>
320 |                   <div className="relative w-full aspect-[16/10] sm:aspect-[4/3] max-h-[260px] lg:max-h-[320px] rounded-3xl p-
1.5 bg-gradient-to-br from-cyan-500/30 via-sky-500/20 to-amber-500/30 shadow-2xl border border-cyan-500/30 overflow-hidden group mx-
auto">
321 |                     <div className="relative w-full h-full rounded-[22px] overflow-hidden bg-slate-900">
322 |                       <img
323 |                         src={step.image}
324 |                         alt={step.imageAlt}
325 |                         className={`w-full h-full object-cover ${
326 |                           step.imagePosition || 'object-center'
327 |                         } transform group-hover:scale-105 transition-transform duration-700`}
328 |                         loading="lazy"
329 |                         onLoad={() => ScrollTrigger.refresh()}
330 |                       />
331 |                       <div className="absolute inset-0 bg-gradient-to-t from-[#040814]/70 via-transparent to-transparent point
er-events-none" />
332 |                     </div>
333 |                   </div>
334 |                 </div>
335 |               </div>
336 |             </div>
337 |           );
338 |         })}
339 |       </div>
340 |
341 |       <div className="pb-2" />
342 |       </div>
343 |     </section>
344 |   );
345 | }
346 |
```

src/components/layout/DashboardShell.tsx TSX 9.0 KB 217 Lines

```tsx
import React, { useState } from 'react'
import { motion } from 'framer-motion'
import { Link, useLocation, useNavigate } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { NotificationPanel } from '@/components/notifications/NotificationPanel'
import { AnnouncementModal } from '@/components/ui/AnnouncementModal'
import {
  Menu, X, ChevronRight, LogOut, User
} from 'lucide-react'

export interface NavLinkItem {
  id?: string
  to?: string
  label: string
  icon: React.ElementType
  badge?: number
  isActive?: boolean
  onClick?: () => void
}

export function DashboardShell({
  title,
  icon: Icon,
  children,
  navLinks,
}: {
  title: string
  icon: React.ElementType
  children: React.ReactNode
  navLinks?: NavLinkItem[]
}) {
  const { signOut, profile } = useAuth()
  const navigate = useNavigate()
  const location = useLocation()
  const [sidebarCollapsed, setSidebarCollapsed] = useState(false)
  const [mobileOpen, setMobileOpen] = useState(false)

  const handleSignOut = async () => {
    await signOut()
    navigate('/login')
  }

  const roleColor: Record<string, string> = {
    admin: 'bg-cyan-500/10 text-cyan-400 border-cyan-500/30',
    super_admin: 'bg-pink-500/10 text-pink-400 border-pink-500/30',
    trainer: 'bg-orange-500/10 text-orange-400 border-orange-500/30',
    trainee: 'bg-purple-500/10 text-purple-400 border-purple-500/30',
  }

  const sidebarContent = (
    <>
      {/* Logo */}
      <div className="h-16 flex items-center px-5 border-b border-slate-800">
        <Link to="/" className="flex items-center gap-2.5">
          <img src="/logo.png" alt="Logo" className="w-8 h-8 object-contain shrink-0" />
          {(!sidebarCollapsed || mobileOpen) && (
            <motion.span
              initial={{ opacity: 0, x: -10 }}
              animate={{ opacity: 1, x: 0 }}
              className="text-sm font-bold whitespace-nowrap"
            >
              <span className="text-white">Capacity</span>
              <span className="text-transparent bg-clip-text bg-gradient-to-r from-cyan-400 to-sky-400"> Connect</span>
            </motion.span>
          )}
        </Link>
      </div>

      {/* Navigation */}
      <nav className="flex-1 p-3 space-y-1.5 overflow-y-auto scrollbar-thin">
        {navLinks?.map(link => {
          const isActive = link.isActive !== undefined ? link.isActive : (link.to ? location.pathname === link.to : false)

          const buttonContent = (
            <button
              onClick={() => {
```

```
                if (link.onClick) link.onClick()
                setMobileOpen(false)
              }}
              className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-xl transition-all text-sm font-medium group ${
              isActive
                ? 'bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 text-white shadow-md shadow-cyan-600/20'
                : 'text-slate-300 hover:text-white hover:bg-slate-800/60 border border-transparent'
            }`}>
              <link.icon className={`w-4 h-4 shrink-0 ${isActive ? 'text-white' : 'text-slate-400 group-hover:text-cyan-400'}
transition-colors`} />
              {(!sidebarCollapsed || mobileOpen) && (
                <>
                  <span className="flex-1 text-left">{link.label}</span>
                  {link.badge !== undefined && link.badge > 0 && (
                    <span className="px-1.5 py-0.5 text-[10px] font-bold bg-amber-500 text-white rounded-full">
                      {link.badge > 99 ? '99+' : link.badge}
                    </span>
                  )}
                  <ChevronRight className={`w-3.5 h-3.5 transition-opacity ${isActive ? 'opacity-100' : 'opacity-0 group-hove
r:opacity-100'}`} />
                </>
              )}
            </button>
          )

          if (link.to) {
            return (
              <Link key={link.to || link.id || link.label} to={link.to}>
                {buttonContent}
              </Link>
            )
          }

          return <div key={link.id || link.label}>{buttonContent}</div>
        })}
      </nav>

      {/* Profile summary & logout */}
      <div className="p-3 border-t border-slate-800 space-y-2">
        <div
          onClick={() => navigate(profile?.role === 'super_admin' ? '/super-admin/profile' : `/${profile?.role}/profile`)}
          className="flex items-center gap-3 px-3 py-2 rounded-xl hover:bg-slate-800/50 transition-all cursor-pointer group"
        >
          <div className="w-8 h-8 rounded-full bg-gradient-to-br from-cyan-500 to-blue-600 flex items-center justify-center te
xt-white text-xs font-bold shrink-0 ring-2 ring-cyan-500/20 overflow-hidden">
            {profile?.avatar_path ? (
              <img src={profile.avatar_path} alt={profile.full_name} className="w-full h-full object-cover" />
            ) : (
              profile?.full_name?.[0]?.toUpperCase() ?? 'U'
            )}
          </div>
          {(!sidebarCollapsed || mobileOpen) && (
            <div className="flex-1 min-w-0">
              <p className="text-xs font-semibold text-slate-200 truncate group-hover:text-cyan-400 transition-colors">
                {profile?.full_name ?? 'User'}
              </p>
              <p className="text-[11px] text-slate-400 truncate">
                {profile?.email}
              </p>
            </div>
          )}
        </div>
        {(!sidebarCollapsed || mobileOpen) && (
          <div className="flex items-center gap-1.5 px-3">
            <span className={`text-xs px-2 py-0.5 rounded-full border capitalize font-semibold ${roleColor[profile?.role ??
''] ?? 'bg-cyan-950/30 text-cyan-400 border-cyan-500/30'}`}>
              {profile?.role?.replace('_', ' ')}
            </span>
          </div>
        )}
        <button
          onClick={handleSignOut}
          className="w-full flex items-center gap-2 px-3 py-2 rounded-xl text-slate-400 hover:text-rose-400 hover:bg-rose-500/
10 transition-all text-sm font-medium"
        >
          <LogOut className="w-4 h-4 shrink-0" />
          {(!sidebarCollapsed || mobileOpen) && <span>Sign Out</span>}
        </button>
      </div>
```

```
    </>
  )

  return (
    <div className="min-h-screen bg-[#f8fafc] text-slate-900 flex">
      <AnnouncementModal />

      {/* Mobile overlay */}
      {mobileOpen && (
        <div className="fixed inset-0 z-40 bg-slate-950/60 backdrop-blur-sm md:hidden" onClick={() => setMobileOpen(false)} />
      )}

      {/* Mobile sidebar drawer */}
      <aside className={`fixed inset-y-0 left-0 z-50 w-64 border-r border-slate-800 bg-[#040814] text-white flex flex-col transition-transform duration-300 md:hidden ${mobileOpen ? 'translate-x-0' : '-translate-x-full'}`}>
        <div className="absolute top-4 right-4 z-10">
          <button onClick={() => setMobileOpen(false)} className="p-2 rounded-lg hover:bg-slate-800 text-slate-400 hover:text-white transition-all">
            <X className="w-5 h-5" />
          </button>
        </div>
        {sidebarContent}
      </aside>

      {/* Desktop sidebar */}
      <aside className={`relative z-20 hidden md:flex ${sidebarCollapsed ? 'w-[72px]' : 'w-64'} border-r border-slate-800 bg-[#040814] text-white flex-col shrink-0 h-screen sticky top-0 transition-all duration-300`}>
        {sidebarContent}
      </aside>

      {/* Main content */}
      <div className="flex-1 min-w-0 flex flex-col">
        {/* Topbar */}
        <header className="h-14 md:h-16 border-b border-slate-200 bg-white/95 backdrop-blur-md sticky top-0 z-30 flex items-center px-4 md:px-6 gap-3 shadow-xs">
          <button
            onClick={() => {
              if (window.innerWidth < 768) {
                setMobileOpen(true)
              } else {
                setSidebarCollapsed(!sidebarCollapsed)
              }
            }}
            className="p-2 rounded-lg hover:bg-slate-100 text-slate-600 hover:text-slate-900 transition-all"
          >
            <Menu className="w-5 h-5" />
          </button>
          <div className="flex items-center gap-2.5">
            <div className="w-8 h-8 rounded-lg bg-cyan-50 border border-cyan-200 flex items-center justify-center text-cyan-600 shadow-xs">
              <Icon className="w-4 h-4 md:w-5 md:h-5" />
            </div>
            <h1 className="text-sm md:text-base font-bold text-slate-900 truncate">{title}</h1>
          </div>
          <div className="flex-1" />
          <div className="flex items-center gap-1.5 md:gap-2">
            <NotificationPanel />

            <Link to={`/${profile?.role}/profile`} className="p-2 rounded-xl hover:bg-slate-100 text-slate-600 hover:text-slate-900 transition-all border border-transparent hover:border-slate-200" title="Profile">
              <User className="w-4 h-4" />
            </Link>
          </div>
        </header>

        <main className="p-4 md:p-6 lg:p-8 flex-1">
          {children}
        </main>
      </div>
    </div>
  )
}
```

src/components/notifications/NotificationPanel.tsx TSX 15.3 KB 337 Lines

```tsx
import React, { useState, useMemo } from 'react'
import { useNavigate } from 'react-router-dom'
import { motion, AnimatePresence } from 'framer-motion'
import { formatDistanceToNow } from 'date-fns'
import {
  Bell, Check, CheckCheck, Trash2, X,
  Clock, Sparkles, Megaphone,
  Video, Award, BookOpen, MessageSquare,
  ShieldAlert, Loader2, ArrowRight
} from 'lucide-react'
import {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
} from '@/components/ui/dropdown-menu'
import { useNotifications, getNotificationRedirectUrl, getNotificationMeta } from '@/hooks/useNotifications'
import { useAuth } from '@/hooks/useAuth'

interface NotificationPanelProps {
  align?: 'end' | 'start' | 'center'
  triggerClassName?: string
}

export function NotificationPanel({ align = 'end', triggerClassName }: NotificationPanelProps) {
  const { profile } = useAuth()
  const navigate = useNavigate()
  const {
    notifications,
    loading,
    markAsRead,
    markAllAsRead,
    deleteNotification,
    clearAllNotifications,
    unreadCount
  } = useNotifications()

  const [activeTab, setActiveTab] = useState<'all' | 'unread'>('all')
  const [isOpen, setIsOpen] = useState(false)

  const filteredNotifications = useMemo(() => {
    if (activeTab === 'unread') {
      return notifications.filter(n => !n.read_at)
    }
    return notifications
  }, [notifications, activeTab])

  const getCategoryIcon = (category: string) => {
    switch (category) {
      case 'session':
        return <Video className="w-3.5 h-3.5 text-indigo-600" />
      case 'announcement':
        return <Megaphone className="w-3.5 h-3.5 text-amber-600" />
      case 'chat':
        return <MessageSquare className="w-3.5 h-3.5 text-cyan-600" />
      case 'course':
        return <Award className="w-3.5 h-3.5 text-emerald-600" />
      case 'user':
        return <BookOpen className="w-3.5 h-3.5 text-blue-600" />
      default:
        return <ShieldAlert className="w-3.5 h-3.5 text-slate-600" />
    }
  }

  const handleNotificationClick = (notif: any) => {
    if (!notif.read_at) {
      markAsRead(notif.id)
    }
    setIsOpen(false)
    const url = getNotificationRedirectUrl(notif.type || '', profile?.role)
    navigate(url)
  }

  return (
    <DropdownMenu open={isOpen} onOpenChange={setIsOpen}>
      <DropdownMenuTrigger asChild>
        <button
```

```
          type="button"
          aria-label="Open notifications"
          className={
            triggerClassName ||
            "relative p-2 rounded-xl text-slate-600 hover:text-slate-900 hover:bg-slate-100 border border-transparent hover:bo
rder-slate-200 transition-all cursor-pointer group"
          }
        >
          <Bell className="w-4 h-4 md:w-5 md:h-5 group-hover:text-cyan-600 transition-colors" />
          {unreadCount > 0 && (
            <span className="absolute top-1 right-1 flex h-2.5 w-2.5">
              <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-rose-400 opacity-75"></span>
              <span className="relative inline-flex rounded-full h-2.5 w-2.5 bg-rose-500 border-2 border-white shadow-xs"></sp
an>
            </span>
          )}
        </button>
      </DropdownMenuTrigger>

      <DropdownMenuContent
        align={align}
        sideOffset={8}
        className="w-[92vw] sm:w-[410px] max-h-[34rem] overflow-hidden bg-white/95 backdrop-blur-xl border border-slate-200/90
shadow-2xl rounded-2xl p-0 z-50 text-slate-900 animate-in fade-in-50 zoom-in-95 duration-150"
      >
        {/* Header */}
        <div className="p-4 bg-gradient-to-b from-slate-50/90 to-white/80 border-b border-slate-100 sticky top-0 z-20">
          <div className="flex items-center justify-between gap-2 mb-3">
            <div className="flex items-center gap-2">
              <div className="w-7 h-7 rounded-lg bg-cyan-100/80 border border-cyan-200 flex items-center justify-center text-c
yan-700 shadow-xs">
                <Bell className="w-3.5 h-3.5" />
              </div>
              <h4 className="font-extrabold text-sm text-slate-900 tracking-tight">Notifications</h4>
              {unreadCount > 0 ? (
                <span className="px-2 py-0.5 rounded-full text-[10px] font-bold bg-cyan-100 text-cyan-800 border border-cyan-2
00">
                  {unreadCount} new
                </span>
              ) : (
                <span className="px-2 py-0.5 rounded-full text-[10px] font-medium bg-slate-100 text-slate-600 border border-sl
ate-200">
                  All caught up
                </span>
              )}
            </div>

            {/* Top Action Buttons */}
            <div className="flex items-center gap-1.5">
              {unreadCount > 0 && (
                <button
                  type="button"
                  onClick={(e) => {
                    e.preventDefault()
                    e.stopPropagation()
                    markAllAsRead()
                  }}
                  className="inline-flex items-center gap-1 px-2.5 py-1 text-[11px] font-bold text-cyan-700 hover:text-cyan-90
0 bg-cyan-50 hover:bg-cyan-100 border border-cyan-200 rounded-lg transition-all cursor-pointer shadow-2xs"
                  title="Mark all notifications as read"
                >
                  <CheckCheck className="w-3 h-3" />
                  <span>Mark read</span>
                </button>
              )}
              {notifications.length > 0 && (
                <button
                  type="button"
                  onClick={(e) => {
                    e.preventDefault()
                    e.stopPropagation()
                    clearAllNotifications()
                  }}
                  className="inline-flex items-center gap-1 px-2.5 py-1 text-[11px] font-bold text-rose-600 hover:text-rose-70
0 bg-rose-50 hover:bg-rose-100 border border-rose-200 rounded-lg transition-all cursor-pointer shadow-2xs"
                  title="Clear all notifications"
                >
                  <Trash2 className="w-3 h-3" />
                  <span>Clear all</span>
```

```
                </button>
              )}
            </div>
          </div>

          {/* Sub Navigation Filter Tabs */}
          <div className="flex items-center gap-1 bg-slate-100/90 p-1 rounded-xl border border-slate-200/80">
            <button
              type="button"
              onClick={() => setActiveTab('all')}
              className={`flex-1 flex items-center justify-center gap-1.5 py-1.5 px-3 rounded-lg text-xs font-bold transition-all ${
                activeTab === 'all'
                  ? 'bg-white text-slate-900 shadow-xs border border-slate-200/80'
                  : 'text-slate-500 hover:text-slate-900 hover:bg-slate-200/40'
              }`}
            >
              <span>All</span>
              <span className={`text-[10px] px-1.5 py-0.2 rounded-full ${
                activeTab === 'all' ? 'bg-slate-100 text-slate-800' : 'bg-slate-200/70 text-slate-600'
              }`}>
                {notifications.length}
              </span>
            </button>
            <button
              type="button"
              onClick={() => setActiveTab('unread')}
              className={`flex-1 flex items-center justify-center gap-1.5 py-1.5 px-3 rounded-lg text-xs font-bold transition-all ${
                activeTab === 'unread'
                  ? 'bg-white text-cyan-800 shadow-xs border border-slate-200/80'
                  : 'text-slate-500 hover:text-slate-900 hover:bg-slate-200/40'
              }`}
            >
              <span>Unread</span>
              {unreadCount > 0 && (
                <span className="w-2 h-2 rounded-full bg-cyan-600 animate-pulse" />
              )}
              <span className={`text-[10px] px-1.5 py-0.2 rounded-full ${
                activeTab === 'unread' ? 'bg-cyan-100 text-cyan-800' : 'bg-slate-200/70 text-slate-600'
              }`}>
                {unreadCount}
              </span>
            </button>
          </div>
        </div>

        {/* Notifications Scrollable Content */}
        <div className="max-h-[22rem] overflow-y-auto divide-y divide-slate-100 overscroll-contain">
          {loading ? (
            <div className="py-14 text-center text-slate-400 text-xs flex flex-col items-center justify-center gap-2.5">
              <Loader2 className="w-5 h-5 animate-spin text-cyan-600" />
              <span className="font-medium text-slate-500">Checking for latest updates...</span>
            </div>
          ) : filteredNotifications.length === 0 ? (
            <div className="py-14 px-6 text-center">
              <div className="w-12 h-12 rounded-2xl bg-cyan-50 border border-cyan-100 text-cyan-600 flex items-center justify-center mx-auto mb-3 shadow-xs">
                <Sparkles className="w-6 h-6 text-cyan-500" />
              </div>
              <p className="text-sm font-bold text-slate-800">
                {activeTab === 'unread' ? 'No unread notifications' : 'No notifications yet'}
              </p>
              <p className="text-xs text-slate-500 mt-1 max-w-[240px] mx-auto leading-relaxed">
                {activeTab === 'unread'
                  ? 'Great job! You have viewed all your notifications.'
                  : 'Important updates regarding courses, live sessions, and announcements will appear here.'}
              </p>
            </div>
          ) : (
            <AnimatePresence initial={false}>
              {filteredNotifications.map((notif: any) => {
                const isUnread = !notif.read_at
                const meta = getNotificationMeta(notif.type || '')
                const timeAgo = notif.created_at
                  ? formatDistanceToNow(new Date(notif.created_at), { addSuffix: true })
                  : 'Recently'

                return (
```

```
                  <motion.div
                    key={notif.id}
                    layout
                    initial={{ opacity: 0, y: 8 }}
                    animate={{ opacity: 1, y: 0 }}
                    exit={{ opacity: 0, height: 0, overflow: 'hidden' }}
                    transition={{ duration: 0.2 }}
                    onClick={() => handleNotificationClick(notif)}
                    className={`relative p-3.5 sm:p-4 transition-all cursor-pointer group flex items-start gap-3 ${
                      isUnread
                        ? 'bg-gradient-to-r from-cyan-50/70 via-sky-50/30 to-white hover:bg-cyan-50/90'
                        : 'bg-white hover:bg-slate-50/90'
                    }`}
                  >
                    {/* Left active marker indicator */}
                    {isUnread && (
                      <span className="absolute left-0 top-0 bottom-0 w-1 bg-gradient-to-b from-cyan-500 to-blue-600 rounded-
r" />
                    )}

                    {/* Category Icon Badge */}
                    <div className="mt-0.5 shrink-0">
                      <div className="w-8 h-8 rounded-xl bg-slate-50 border border-slate-200/80 flex items-center justify-cent
er shadow-xs group-hover:scale-105 transition-transform">
                        {getCategoryIcon(meta.category)}
                      </div>
                    </div>

                    {/* Notification Body */}
                    <div className="flex-1 min-w-0 pr-1">
                      <div className="flex items-center gap-1.5 flex-wrap mb-1">
                        <span className={`text-[9px] font-bold uppercase tracking-wider px-2 py-0.5 rounded-md border ${meta.c
olor}`}>
                          {meta.label}
                        </span>
                        {isUnread && (
                          <span className="inline-flex items-center gap-1 text-[10px] font-bold text-cyan-700 bg-cyan-100/80 p
x-1.5 py-0.2 rounded-full border border-cyan-200">
                            <span className="w-1.5 h-1.5 rounded-full bg-cyan-600" />
                            New
                          </span>
                        )}
                      </div>

                      <h5 className={`text-xs sm:text-sm font-bold truncate leading-snug ${isUnread ? 'text-slate-900' : 'text
-slate-800'}`}>
                        {notif.title}
                      </h5>

                      <p className="text-xs text-slate-600 line-clamp-2 leading-relaxed mt-1 font-normal">
                        {notif.message}
                      </p>

                      <div className="flex items-center justify-between mt-2 pt-1 border-t border-slate-100/80 text-[10px] tex
t-slate-400 font-medium">
                        <span className="inline-flex items-center gap-1">
                          <Clock className="w-3 h-3 text-slate-400" />
                          {timeAgo}
                        </span>
                        <span className="text-cyan-600 group-hover:translate-x-0.5 transition-transform font-bold inline-flex
items-center gap-0.5">
                          Open <ArrowRight className="w-2.5 h-2.5" />
                        </span>
                      </div>
                    </div>

                    {/* Right Action Icons (Dismiss / Mark Read) */}
                    <div className="flex flex-col items-center gap-1 shrink-0 -mr-1">
                      <button
                        type="button"
                        onClick={(e) => {
                          e.preventDefault()
                          e.stopPropagation()
                          deleteNotification(notif.id)
                        }}
                        className="p-1 rounded-lg text-slate-400 hover:text-rose-600 hover:bg-rose-50 transition-colors cursor
-pointer"
                        title="Dismiss notification"
                      >
```

```
295 |                         <X className="w-3.5 h-3.5" />
296 |                       </button>
297 |                       {isUnread && (
298 |                         <button
299 |                           type="button"
300 |                           onClick={(e) => {
301 |                             e.preventDefault()
302 |                             e.stopPropagation()
303 |                             markAsRead(notif.id)
304 |                           }}
305 |                           className="p-1 rounded-lg text-slate-400 hover:text-cyan-700 hover:bg-cyan-50 transition-colors curs
or-pointer"
306 |                           title="Mark as read"
307 |                         >
308 |                           <Check className="w-3.5 h-3.5" />
309 |                         </button>
310 |                       )}
311 |                     </div>
312 |                   </motion.div>
313 |                 )
314 |               })}
315 |             </AnimatePresence>
316 |           )}
317 |         </div>
318 |
319 |         {/* Footer Link */}
320 |         <div className="p-2.5 bg-slate-50 border-t border-slate-100 text-center">
321 |           <button
322 |             type="button"
323 |             onClick={() => {
324 |               setIsOpen(false)
325 |               const role = profile?.role === 'admin' || profile?.role === 'super_admin' ? 'admin' : (profile?.role || 'traine
e')
326 |               navigate(`/${role}/notifications`)
327 |             }}
328 |             className="text-xs font-bold text-cyan-700 hover:text-cyan-900 inline-flex items-center gap-1 hover:underline curs
or-pointer"
329 |           >
330 |             Open Full Notification Center <ArrowRight className="w-3.5 h-3.5" />
331 |           </button>
332 |         </div>
333 |       </DropdownMenuContent>
334 |     </DropdownMenu>
335 |   )
336 | }
337 |
```

src/components/profile/FirstTimeProfileSetupModal.tsx TSX 18.9 KB 484 Lines

```tsx
import React, { useState, useEffect, useRef } from 'react'
import { motion, AnimatePresence } from 'framer-motion'
import { useAuth } from '@/hooks/useAuth'
import { supabase } from '@/lib/supabase'
import { toast } from 'sonner'
import {
  User, Camera, Upload, Trash2, Phone, Building, Briefcase,
  Sparkles, CheckCircle2, X, Loader2, ArrowRight, Crop
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { ImageCropperModal } from '@/components/ui/ImageCropperModal'

export function FirstTimeProfileSetupModal() {
  const { user, profile, refreshProfile } = useAuth()
  const [isOpen, setIsOpen] = useState(false)
  const [saving, setSaving] = useState(false)
  const [uploadingPhoto, setUploadingPhoto] = useState(false)
  const fileInputRef = useRef<HTMLInputElement>(null)

  // Form state

  const [fullName, setFullName] = useState('')
  const [phone, setPhone] = useState('')
  const [department, setDepartment] = useState('')
  const [designation, setDesignation] = useState('')
  const [avatarPath, setAvatarPath] = useState<string | null>(null)
  const [bio, setBio] = useState('')

  // Cropper state

  const [isCropperOpen, setIsCropperOpen] = useState(false)
  const [rawPhotoFile, setRawPhotoFile] = useState<File | null>(null)
  const [rawPhotoUrl, setRawPhotoUrl] = useState<string | null>(null)

  useEffect(() => {
    if (!user || !profile) return

    const storageKey = `cc_profile_setup_done_${user.id}`
    const isCompleted = localStorage.getItem(storageKey)

    // Check if profile is newly created / first login

    // If setup hasn't been marked completed or dismissed yet, trigger the modal

    if (!isCompleted) {
      setFullName(profile.full_name || '')
      setAvatarPath(profile.avatar_path || null)
      setDepartment(profile.department || '')
      setDesignation(profile.designation || '')
      setBio(profile.bio || '')
      setIsOpen(true)
    }
  }, [user, profile])

  if (!isOpen || !user || !profile) return null

  const handleDismiss = () => {
    if (user) {
      localStorage.setItem(`cc_profile_setup_done_${user.id}`, 'true')
    }
    setIsOpen(false)
  }

  // Handle Photo Select

  const handlePhotoSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file || !user) return

    if (!file.type.startsWith('image/')) {
      toast.error('Please select an image file (PNG, JPG, JPEG, WEBP)')
      return
    }

```

```
    if (file.size > 10 * 1024 * 1024) {
      toast.error('Image size cannot exceed 10MB')
      return
    }

    setRawPhotoFile(file)
    setRawPhotoUrl(null)
    setIsCropperOpen(true)
    if (fileInputRef.current) fileInputRef.current.value = ''
  }

  // Handle Open Adjust View

  const handleOpenAdjustView = () => {
    if (!avatarPath || !user) return
    const storedRaw = localStorage.getItem(`cc_raw_avatar_${user.id}`)
    setRawPhotoUrl(storedRaw || avatarPath)
    setRawPhotoFile(null)
    setIsCropperOpen(true)
  }

  // Handle Crop Complete

  const handleCropComplete = async (croppedFile: File) => {
    const origFile = rawPhotoFile
    setIsCropperOpen(false)
    setRawPhotoFile(null)
    setRawPhotoUrl(null)
    await uploadCroppedPhoto(croppedFile, origFile)
  }

  // Handle Upload Cropped Photo

  const uploadCroppedPhoto = async (file: File, originalRawFile?: File | null) => {
    if (!user || !profile) return
    setUploadingPhoto(true)
    const fileExt = 'jpg'
    const fileName = `avatar_${user.id}_${Date.now()}.${fileExt}`
    const filePath = `avatars/${fileName}`
    const table = profile.role === 'trainer' ? 'trainers' : profile.role === 'trainee' ? 'trainees' : 'admins'

    try {
      if (originalRawFile) {
        const rawExt = originalRawFile.name.split('.').pop() || 'jpg'
        const rawFileName = `raw_avatar_${user.id}_${Date.now()}.${rawExt}`
        const rawFilePath = `avatars/${rawFileName}`

        try {
          const { error: rawUploadErr } = await supabase.storage
            .from('Homepage')
            .upload(rawFilePath, originalRawFile, { upsert: true })

          if (!rawUploadErr) {
            const { data: rawUrlData } = supabase.storage.from('Homepage').getPublicUrl(rawFilePath)
            localStorage.setItem(`cc_raw_avatar_${user.id}`, rawUrlData.publicUrl)
          } else {
            const reader = new FileReader()
            reader.onload = (e) => {
              if (e.target?.result) {
                localStorage.setItem(`cc_raw_avatar_${user.id}`, e.target.result as string)
              }
            }
            reader.readAsDataURL(originalRawFile)
          }
        } catch {
          // non-blocking

        }
      }

      let finalUrl: string | null = null

      // Attempt upload to Supabase storage

      const { error: uploadError } = await supabase.storage
        .from('Homepage')
        .upload(filePath, file, { upsert: true, contentType: 'image/jpeg' })

      if (uploadError) {
```

```
        const { error: matError } = await supabase.storage
          .from('materials')
          .upload(filePath, file, { upsert: true, contentType: 'image/jpeg' })

        if (!matError) {
          const { data: urlData } = supabase.storage.from('materials').getPublicUrl(filePath)
          finalUrl = urlData.publicUrl
        } else {
          // DataURL fallback

          await new Promise<void>((resolve) => {
            const reader = new FileReader()
            reader.onload = (ev) => {
              finalUrl = ev.target?.result as string
              resolve()
            }
            reader.readAsDataURL(file)
          })
        }
      } else {
        const { data: urlData } = supabase.storage.from('Homepage').getPublicUrl(filePath)
        finalUrl = urlData.publicUrl
      }

      if (finalUrl) {
        setAvatarPath(finalUrl)

        // Save immediately to DB

        await supabase
          .from(table as any)
          .update({ avatar_path: finalUrl })
          .eq('id', user.id)

        await refreshProfile()
        toast.success('Photo adjusted and uploaded!')
      }
    } catch (err: any) {
      console.error('Photo upload error:', err)
      toast.error('Failed to upload photo')
    } finally {
      setUploadingPhoto(false)
    }
  }

  const handleRemovePhoto = async () => {
    if (!user) return
    setUploadingPhoto(true)
    const table = profile.role === 'trainer' ? 'trainers' : profile.role === 'trainee' ? 'trainees' : 'admins'
    try {
      setAvatarPath(null)
      await supabase
        .from(table as any)
        .update({ avatar_path: null })
        .eq('id', user.id)

      await refreshProfile()
      toast.success('Photo removed')
    } catch (err) {
      toast.error('Failed to remove photo')
    } finally {
      setUploadingPhoto(false)
    }
  }

  // Handle Save Profile

  const handleSaveProfile = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!user) return

    if (!fullName.trim()) {
      toast.error('Please enter your full name')
      return
    }

    setSaving(true)
    const table = profile.role === 'trainer' ? 'trainers' : profile.role === 'trainee' ? 'trainees' : 'admins'

```

```
    try {
      const payload: Record<string, any> = {
        full_name: fullName.trim(),
        avatar_path: avatarPath || null,
        phone: phone.trim() || null,
        department: department.trim() || null,
        designation: designation.trim() || null,
        bio: bio.trim() || null,
      }

      const { error } = await supabase
        .from(table as any)
        .update(payload)
        .eq('id', user.id)

      if (error) throw error

      await refreshProfile()
      localStorage.setItem(`cc_profile_setup_done_${user.id}`, 'true')
      toast.success('Profile setup complete! Welcome to Capacity Connect.')
      setIsOpen(false)
    } catch (err: any) {
      console.error('Save profile error:', err)
      toast.error(err.message || 'Failed to save profile details')
    } finally {
      setSaving(false)
    }
  }

  return (
    <AnimatePresence>
      <div className="fixed inset-0 z-50 flex items-center justify-center p-4 sm:p-6 overflow-y-auto">
        {/* Backdrop overlay */}
        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          onClick={handleDismiss}
          className="fixed inset-0 bg-slate-950/80 backdrop-blur-md"
        />

        {/* Modal Card Centered on Screen */}
        <motion.div
          initial={{ opacity: 0, scale: 0.92, y: 20 }}
          animate={{ opacity: 1, scale: 1, y: 0 }}
          exit={{ opacity: 0, scale: 0.92, y: 20 }}
          transition={{ type: 'spring', duration: 0.5, bounce: 0.2 }}
          className="relative w-full max-w-xl bg-white text-slate-900 rounded-3xl shadow-2xl border border-slate-200 overflow-
hidden z-10 my-auto"
        >
          {/* Top Hero Banner */}
          <div className="relative p-6 sm:p-8 bg-gradient-to-r from-[#050b1a] via-[#091733] to-[#040814] text-white overflow-h
idden">
            <div className="pointer-events-none absolute -right-16 -top-16 w-64 h-64 rounded-full bg-cyan-500/20 blur-3xl" />
            <div className="pointer-events-none absolute left-1/3 bottom-0 w-48 h-48 rounded-full bg-blue-500/15 blur-2xl" />

            <button
              onClick={handleDismiss}
              className="absolute top-4 right-4 w-8 h-8 rounded-full bg-white/10 hover:bg-white/20 text-slate-300 hover:text-w
hite flex items-center justify-center transition-all cursor-pointer z-20"
              aria-label="Close setup modal"
            >
              <X className="w-4 h-4" />
            </button>

            <div className="relative z-10">
              <div className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-500/20 border border-cyan-400/40
text-cyan-300 text-xs font-bold uppercase tracking-wider mb-2.5">
                <Sparkles className="w-3.5 h-3.5 text-cyan-400" />
                First-Time Setup
              </div>
              <h2 className="text-2xl sm:text-3xl font-black font-display tracking-tight text-white leading-tight">
                Welcome to Capacity Connect!
              </h2>
              <p className="text-xs sm:text-sm text-slate-300 mt-1.5 max-w-md">
                Set up your photo and institute details to personalize your certificates, dashboard, and badges.
              </p>
            </div>
          </div>
```

```
297 |
298 |           {/* Form Content */}
299 |           <form onSubmit={handleSaveProfile} className="p-6 sm:p-8 space-y-6">
300 |
301 |             {/* Centered Photo Upload */}
302 |             <div className="flex flex-col items-center justify-center text-center p-4 rounded-2xl bg-slate-50 border border-sl
ate-200/80">
303 |               <div className="relative group/avatar mb-3">
304 |                 <div className="w-20 h-20 rounded-2xl bg-gradient-to-br from-cyan-600 via-sky-600 to-blue-600 flex items-cente
r justify-center text-white text-3xl font-black shadow-lg shadow-cyan-600/30 ring-4 ring-cyan-500/20 overflow-hidden">
305 |                   {avatarPath ? (
306 |                     <img src={avatarPath} alt={fullName || 'Profile'} className="w-full h-full object-cover" />
307 |                   ) : (
308 |                     fullName.charAt(0)?.toUpperCase() || <User className="w-10 h-10" />
309 |                   )}
310 |                 </div>
311 |
312 |                 <button
313 |                   type="button"
314 |                   onClick={() => fileInputRef.current?.click()}
315 |                   disabled={uploadingPhoto}
316 |                   className="absolute inset-0 rounded-2xl bg-black/60 opacity-0 group-hover/avatar:opacity-100 flex flex-col i
tems-center justify-center text-white text-[10px] font-bold transition-opacity cursor-pointer backdrop-blur-2xs"
317 |                   title="Upload photo"
318 |                 >
319 |                   {uploadingPhoto ? <Loader2 className="w-5 h-5 animate-spin" /> : <Camera className="w-5 h-5 mb-0.5" />}
320 |                   <span>{avatarPath ? 'Change' : 'Upload'}</span>
321 |                 </button>
322 |               </div>
323 |
324 |               <input
325 |                 ref={fileInputRef}
326 |                 type="file"
327 |                 accept="image/*"
328 |                 className="hidden"
329 |                 onChange={handlePhotoSelect}
330 |               />
331 |
332 |               <div className="flex flex-wrap items-center justify-center gap-2">
333 |                 <Button
334 |                   type="button"
335 |                   variant="outline"
336 |                   size="sm"
337 |                   onClick={() => fileInputRef.current?.click()}
338 |                   disabled={uploadingPhoto}
339 |                   className="h-8 text-xs font-bold border-cyan-200 bg-cyan-50 hover:bg-cyan-100 text-cyan-800 rounded-xl curso
r-pointer"
340 |                 >
341 |                   {uploadingPhoto ? (
342 |                     <Loader2 className="w-3.5 h-3.5 mr-1.5 animate-spin" />
343 |                   ) : (
344 |                     <Upload className="w-3.5 h-3.5 mr-1.5 text-cyan-600" />
345 |                   )}
346 |                   {avatarPath ? 'Change Photo' : 'Upload Photo'}
347 |                 </Button>
348 |
349 |                 {avatarPath && (
350 |                   <Button
351 |                     type="button"
352 |                     variant="outline"
353 |                     size="sm"
354 |                     onClick={handleOpenAdjustView}
355 |                     disabled={uploadingPhoto}
356 |                     className="h-8 text-xs font-bold border-slate-300 bg-white hover:bg-slate-100 text-slate-700 rounded-xl px
-2.5 cursor-pointer shadow-xs"
357 |                     >
358 |                     <Crop className="w-3.5 h-3.5 mr-1 text-cyan-600" /> Adjust View
359 |                   </Button>
360 |                 )}
361 |
362 |                 {avatarPath && (
363 |                   <Button
364 |                     type="button"
365 |                     variant="ghost"
366 |                     size="sm"
367 |                     onClick={handleRemovePhoto}
368 |                     disabled={uploadingPhoto}
369 |                     className="h-8 text-xs font-semibold text-rose-600 hover:text-rose-700 hover:bg-rose-50 rounded-xl px-2.5
cursor-pointer"
```

```
                    >
                      <Trash2 className="w-3.5 h-3.5 mr-1" /> Remove
                    </Button>
                  )}
                </div>
                <p className="text-[11px] text-slate-400 mt-1.5">
                  PNG, JPG or WEBP (Max 10MB). High-resolution square recommended.
                </p>
              </div>

              {/* Inputs Grid */}
              <div className="space-y-4">
                <div className="space-y-1.5">
                  <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                    <User className="w-3.5 h-3.5 text-cyan-600" /> Full Name *
                  </Label>
                  <Input
                    value={fullName}
                    onChange={(e) => setFullName(e.target.value)}
                    placeholder="e.g. Dr. Rajesh Kumar / Ananya Deshmukh"
                    required
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900 font-medium"
                  />
                </div>

                <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                      <Building className="w-3.5 h-3.5 text-cyan-600" /> Institute / Department
                    </Label>
                    <Input
                      value={department}
                      onChange={(e) => setDepartment(e.target.value)}
                      placeholder="e.g. INCOIS / IMD / NCMRWF"
                      className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900 font-medium text-xs sm:text-sm"
                    />
                  </div>

                  <div className="space-y-1.5">
                    <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                      <Briefcase className="w-3.5 h-3.5 text-cyan-600" /> Designation / Role
                    </Label>
                    <Input
                      value={designation}
                      onChange={(e) => setDesignation(e.target.value)}
                      placeholder="e.g. Scientist-D / Trainee Fellow"
                      className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900 font-medium text-xs sm:text-sm"
                    />
                  </div>
                </div>

                <div className="space-y-1.5">
                  <Label className="text-xs font-bold text-slate-700 flex items-center gap-1.5">
                    <Phone className="w-3.5 h-3.5 text-cyan-600" /> Mobile / Phone Number
                  </Label>
                  <Input
                    value={phone}
                    onChange={(e) => setPhone(e.target.value)}
                    placeholder="+91 98765 43210"
                    className="bg-slate-50 border-slate-200 rounded-xl h-10 text-slate-900 font-medium"
                  />
                </div>
              </div>

              {/* Modal Actions */}
              <div className="pt-3 border-t border-slate-100 flex flex-col sm:flex-row items-center justify-between gap-3">
                <button
                  type="button"
                  onClick={handleDismiss}
                  className="text-xs font-bold text-slate-400 hover:text-slate-600 transition-colors order-2 sm:order-1 cursor-p
ointer"
                >
                  Skip for now (I'll do this later)
                </button>

                <Button
                  type="submit"
                  disabled={saving}
                  className="w-full sm:w-auto h-11 px-7 bg-gradient-to-r from-cyan-600 via-sky-600 to-blue-600 hover:from-cyan-7
```

```
00 hover:to-blue-700 text-white font-bold rounded-xl shadow-lg shadow-cyan-600/25 transition-all order-1 sm:order-2 cursor-pointer f
lex items-center justify-center gap-2"
448 |                >
449 |                  {saving ? (
450 |                    <>
451 |                      <Loader2 className="w-4 h-4 animate-spin" />
452 |                      <span>Saving Profile...</span>
453 |                    </>
454 |                  ) : (
455 |                    <>
456 |                      <span>Complete Profile Setup</span>
457 |                      <ArrowRight className="w-4 h-4" />
458 |                    </>
459 |                  )}
460 |                </Button>
461 |              </div>
462 |
463 |            </form>
464 |          </motion.div>
465 |        </div>
466 |
467 |        <ImageCropperModal
468 |          isOpen={isCropperOpen}
469 |          imageFile={rawPhotoFile}
470 |          imageUrl={rawPhotoUrl}
471 |          aspectRatio={1}
472 |          cropShape="round"
473 |          title="Adjust Profile Photo"
474 |          onClose={() => {
475 |            setIsCropperOpen(false)
476 |            setRawPhotoFile(null)
477 |            setRawPhotoUrl(null)
478 |          }}
479 |          onCropComplete={handleCropComplete}
480 |        />
481 |      </AnimatePresence>
482 |    )
483 | }
484 |
```

src/components/ProtectedRoute.tsx TSX 1.6 KB 65 Lines

```tsx
import React from 'react'
import { Navigate, Outlet } from 'react-router-dom'
import { useAuth } from '@/hooks/useAuth'
import { Loader2 } from 'lucide-react'

function LoadingScreen() {
  return (
    <div className="min-h-screen flex flex-col items-center justify-center bg-[#040814] gap-3">
      <Loader2 className="w-6 h-6 text-zinc-200 animate-spin" />
      <p className="text-sm text-zinc-200/70">Loading...</p>
    </div>
  )
}

export function ProtectedRoute() {
  const { session, profile, loading } = useAuth()

  if (loading) {
    return <LoadingScreen />
  }

  if (!session) {
    return <Navigate to="/login" replace />
  }

  // Check suspension status globally for any authenticated route
  if (profile?.approval_status === 'suspended') {
    return <Navigate to="/account-suspended" replace />
  }

  return <Outlet />
}

export function ApprovedRoute() {
  const { profile, loading } = useAuth()

  if (loading) return <LoadingScreen />

  if (!profile) {
    return <Navigate to="/login" replace />
  }

  if (profile.approval_status === 'pending') {
    return <Navigate to="/pending-approval" replace />
  }

  if (profile.approval_status === 'rejected') {
    return <Navigate to="/access-denied" replace />
  }

  return <Outlet />
}

export function RoleRoute({ allowedRoles }: { allowedRoles: Array<'trainee' | 'trainer' | 'admin' | 'super_admin'> }) {
  const { profile, loading } = useAuth()

  if (loading) return <LoadingScreen />

  if (!profile || !allowedRoles.includes(profile.role)) {
    return <Navigate to="/access-denied" replace />
  }

  return <Outlet />
}

```

src/components/space/StarfieldCanvas.tsx TSX 3.1 KB 112 Lines

```tsx
import React, { useEffect, useRef } from 'react';

interface Star {
  x: number;
  y: number;
  radius: number;
  alpha: number;
  baseAlpha: number;
  twinkleSpeed: number;
  color: string;
  vx: number;
  vy: number;
}

export function StarfieldCanvas({ className = '' }: { className?: string }) {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;

    const ctx = canvas.getContext('2d');
    if (!ctx) return;

    let animationFrameId: number;
    let width = (canvas.width = canvas.parentElement?.clientWidth || window.innerWidth);
    let height = (canvas.height = canvas.parentElement?.clientHeight || window.innerHeight);

    const colors = [
      '#9333ea', // purple
      '#ec4899', // pink
      '#f97316', // orange
      '#0284c7', // cyan
      '#8b5cf6', // violet
    ];

    const starCount = Math.floor((width * height) / 10000);
    const stars: Star[] = [];

    for (let i = 0; i < Math.min(starCount, 160); i++) {
      const baseAlpha = Math.random() * 0.25 + 0.08;
      stars.push({
        x: Math.random() * width,
        y: Math.random() * height,
        radius: Math.random() * 1.8 + 0.6,
        alpha: baseAlpha,
        baseAlpha,
        twinkleSpeed: Math.random() * 0.02 + 0.005,
        color: colors[Math.floor(Math.random() * colors.length)],
        vx: (Math.random() - 0.5) * 0.06,
        vy: (Math.random() - 0.5) * 0.06,
      });
    }

    let time = 0;
    const render = () => {
      time += 0.02;
      ctx.clearRect(0, 0, width, height);

      for (let i = 0; i < stars.length; i++) {
        const s = stars[i];
        s.x += s.vx;
        s.y += s.vy;

        if (s.x < 0) s.x = width;
        if (s.x > width) s.x = 0;
        if (s.y < 0) s.y = height;
        if (s.y > height) s.y = 0;

        // Twinkle

        const currentAlpha = s.baseAlpha + Math.sin(time * 2 + i) * 0.25 * s.baseAlpha;
        const clampedAlpha = Math.max(0.1, Math.min(1, currentAlpha));

        ctx.save();
        ctx.beginPath();
```

```
        ctx.arc(s.x, s.y, s.radius, 0, Math.PI * 2);
        ctx.fillStyle = s.color;
        ctx.globalAlpha = clampedAlpha;
        ctx.shadowBlur = s.radius > 1.2 ? 6 : 0;
        ctx.shadowColor = s.color;
        ctx.fill();
        ctx.restore();
      }

      animationFrameId = requestAnimationFrame(render);
    };

    render();

    const handleResize = () => {
      if (!canvas) return;
      width = canvas.width = canvas.parentElement?.clientWidth || window.innerWidth;
      height = canvas.height = canvas.parentElement?.clientHeight || window.innerHeight;
    };

    window.addEventListener('resize', handleResize);

    return () => {
      cancelAnimationFrame(animationFrameId);
      window.removeEventListener('resize', handleResize);
    };
  }, []);

  return (
    <canvas
      ref={canvasRef}
      className={`pointer-events-none absolute inset-0 z-0 h-full w-full opacity-70 ${className}`}
      aria-hidden="true"
    />
  );
}

```

.oxlintrc.json JSON 0.2 KB 9 Lines

```
{
  "$schema": "./node_modules/oxlint/configuration_schema.json",
  "plugins": ["react", "typescript", "oxc"],
  "rules": {
    "react/rules-of-hooks": "error",
    "react/only-export-components": ["warn", { "allowConstantExport": true }]
  }
}

```

check_db.ts TS 0.5 KB 19 Lines

```ts
import { createClient } from '@supabase/supabase-js'
import dotenv from 'dotenv'
import path from 'path'

dotenv.config({ path: path.resolve(process.cwd(), '.env.local') })

const supabaseUrl = process.env.VITE_SUPABASE_URL!
const supabaseKey = process.env.VITE_SUPABASE_PUBLISHABLE_KEY!

const supabase = createClient(supabaseUrl, supabaseKey)

async function check() {
  const { data, error } = await supabase.from('assessments').select('id, title, status')
  console.log('Error:', error)
  console.log('Assessments:', data)
}

check()

```

check_fetch.js JS 1.0 KB 38 Lines

```
import fs from 'fs'

const envStr = fs.readFileSync('.env.local', 'utf-8')
const envObj = {}
envStr.split('\n').forEach(line => {
  if (line.includes('=')) {
    const [k, ...v] = line.split('=')
    envObj[k.trim()] = v.join('=').trim()
  }
})

const url = envObj['VITE_SUPABASE_URL']
const key = envObj['VITE_SUPABASE_PUBLISHABLE_KEY']

async function check() {
  const res = await fetch(`${url}/rest/v1/assessment_attempts?select=id,assessment_id,user_id,score,created_at,grade_status`,
{
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const attempts = await res.json()
  console.log('Attempts:')
  console.log(JSON.stringify(attempts, null, 2))

  const res2 = await fetch(`${url}/rest/v1/assessments?select=id,title`, {
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const assessments = await res2.json()
  console.log('\nAssessments:')
  console.log(JSON.stringify(assessments, null, 2))
}

check()

```

check_fetch2.js JS 1.0 KB 38 Lines

```
import fs from 'fs'

const envStr = fs.readFileSync('.env.local', 'utf-8')
const envObj = {}
envStr.split('\n').forEach(line => {
  if (line.includes('=')) {
    const [k, ...v] = line.split('=')
    envObj[k.trim()] = v.join('=').trim()
  }
})

const url = envObj['VITE_SUPABASE_URL']
const key = envObj['VITE_SUPABASE_PUBLISHABLE_KEY']

async function check() {
  const res = await fetch(`${url}/rest/v1/assessment_attempts?select=id,assessment_id,user_id,score,submitted_at,grade_status
`, {
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const attempts = await res.json()
  console.log('Attempts:')
  console.log(JSON.stringify(attempts, null, 2))

  const res2 = await fetch(`${url}/rest/v1/assessments?select=id,title`, {
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const assessments = await res2.json()
  console.log('\nAssessments:')
  console.log(JSON.stringify(assessments, null, 2))
}

check()

```

check_fetch3.js JS 1.0 KB 38 Lines

```
import fs from 'fs'

const envStr = fs.readFileSync('.env.local', 'utf-8')
const envObj = {}
envStr.split('\n').forEach(line => {
  if (line.includes('=')) {
    const [k, ...v] = line.split('=')
    envObj[k.trim()] = v.join('=').trim()
  }
})

const url = envObj['VITE_SUPABASE_URL']
const key = envObj['VITE_SUPABASE_PUBLISHABLE_KEY']

async function check() {
  const res = await fetch(`${url}/rest/v1/assessment_attempts?select=id,assessment_id,user_id,score,submitted_at`, {
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const attempts = await res.json()
  console.log('Attempts:')
  console.log(JSON.stringify(attempts, null, 2))

  const res2 = await fetch(`${url}/rest/v1/assessments?select=id,title`, {
    headers: {
      'apikey': key,
      'Authorization': `Bearer ${key}`
    }
  })
  const assessments = await res2.json()
  console.log('\nAssessments:')
  console.log(JSON.stringify(assessments, null, 2))
}

check()

```

check_rls.ts TS 0.9 KB 27 Lines

```ts
import { createClient } from '@supabase/supabase-js'
import * as dotenv from 'dotenv'

dotenv.config()

const supabaseUrl = process.env.VITE_SUPABASE_URL
const supabaseServiceKey = process.env.VITE_SUPABASE_ANON_KEY // using anon key to simulate

async function main() {
  const supabase = createClient(supabaseUrl!, process.env.VITE_SUPABASE_SERVICE_ROLE_KEY!)

  const { data, error } = await supabase.rpc('query', {
    sql_string: `
      SELECT tablename, policyname, permissive, roles, cmd, qual, with_check
      FROM pg_policies
      WHERE tablename = 'assessments';
    `
  })

  // Since RPC for arbitrary SQL might not be enabled, let's just query a known REST endpoint or assume policies
  // Actually, we can use the service role key to fetch the table structure if we write a raw postgres query.
  // Wait, I can't run raw SQL easily via JS without the `pg` library.

}

main()
```

check-db.js JS 0.5 KB 16 Lines

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = 'https://gezrhprdrlsswfnuffvf.supabase.co';
const supabaseKey = 'sb_publishable_z4ecljyqj8-oVc8Nv-UsBQ_xzKsnxpG';
const supabase = createClient(supabaseUrl, supabaseKey);

async function check() {
  const { data, error } = await supabase.from('courses').select('session_flow_text, session_flow_document_path').limit(1);
  if (error) {
    console.error('Error:', error.message);
  } else {
    console.log('Success, data:', data);
  }
}
check();

```

check.js JS 0.7 KB 24 Lines

```
import { createClient } from '@supabase/supabase-js'
import fs from 'fs'

const envStr = fs.readFileSync('.env.local', 'utf-8')
const envObj = {}
envStr.split('\n').forEach(line => {
  if (line.includes('=')) {
    const [k, ...v] = line.split('=')
    envObj[k.trim()] = v.join('=').trim()
  }
})

const supabase = createClient(envObj['VITE_SUPABASE_URL'], envObj['VITE_SUPABASE_ANON_KEY'])

async function check() {
  const { data, error } = await supabase.from('assessment_attempts').select('id, assessment_id, user_id, score, created_at, submitted_at, grade_status')
  console.log('Attempts:', data)

  const { data: aData, error: aError } = await supabase.from('assessments').select('id, title')
  console.log('Assessments:', aData)
}

check()

```

clean_update_types.js JS 2.4 KB 86 Lines

```
import fs from "fs";

let code = fs.readFileSync("src/integrations/supabase/types.ts", "utf8");

const newTables = `      announcements: {
        Row: {
          id: string
          title: string
          content: string
          author_id: string
          is_active: boolean
          created_at: string
        }
        Insert: {
          id?: string
          title: string
          content: string
          author_id: string
          is_active?: boolean
          created_at?: string
        }
        Update: {
          id?: string
          title?: string
          content?: string
          author_id?: string
          is_active?: boolean
          created_at?: string
        }
        Relationships: []
      }
      course_feedback: {
        Row: {
          id: string
          course_id: string
          user_id: string
          rating: number
          comments: string | null
          created_at: string
        }
        Insert: {
          id?: string
          course_id: string
          user_id: string
          rating: number
          comments?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          user_id?: string
          rating?: number
          comments?: string | null
          created_at?: string
        }
        Relationships: []
      }
      scenario_modules: {`;

if (!code.includes("announcements: {")) {
  code = code.replace("      scenario_modules: {", newTables);
}

function injectFields(table, block, toInject) {
  const regex = new RegExp("(" + table + ":\\s*\\{.*?\\b" + block + ":\\s*\\{.*?)(        \\})", "s");
  const match = code.match(regex);
  if (match && !match[1].includes("work_experience:")) {
      code = code.replace(regex, "$1" + toInject + "\n$2");
  }
}

const reqFields = "          qualifications: string | null\n          work_experience: string | null\n          interests: str
ing | null";
const optFields = "          qualifications?: string | null\n          work_experience?: string | null\n          interests?:
string | null";
```

```
75 |
76 | injectFields("trainees", "Row", reqFields);
77 | injectFields("trainees", "Insert", optFields);
78 | injectFields("trainees", "Update", optFields);
79 |
80 | injectFields("trainers", "Row", reqFields);
81 | injectFields("trainers", "Insert", optFields);
82 | injectFields("trainers", "Update", optFields);
83 |
84 | fs.writeFileSync("src/integrations/supabase/types.ts", code);
85 | console.log("Clean update done");
86 |
```

dedup_types.js JS 0.7 KB 32 Lines

```
import fs from "fs";

let code = fs.readFileSync("src/integrations/supabase/types.ts", "utf8");
let lines = code.split("\n");
let inBlock = null;
let seenKeys = new Set();
let newLines = [];

for (let i = 0; i < lines.length; i++) {
  let line = lines[i];
  let trimmed = line.trim();

  if (trimmed.endsWith(": {")) {
    inBlock = trimmed.split(":")[0];
    seenKeys.clear();
  }

  let match = trimmed.match(/^([a-zA-Z_]+)\??:/);
  if (match) {
    let key = match[1];
    if (seenKeys.has(key)) {
      continue; // skip duplicate key in this block
    }
    seenKeys.add(key);
  }

  newLines.push(line);
}

fs.writeFileSync("src/integrations/supabase/types.ts", newLines.join("\n"));
console.log("Deduplication Done");

```

fix_assessment_attempts_rls.ts TS 0.9 KB 31 Lines

```ts
import { createClient } from '@supabase/supabase-js'

async function applyMigration() {
  const sql = `
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Trainees can create own attempts') THEN
    CREATE POLICY "Trainees can create own attempts"
    ON public.assessment_attempts FOR INSERT
    WITH CHECK (auth.uid() = user_id);
  END IF;
END $$;

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Admins can insert attempts') THEN
    CREATE POLICY "Admins can insert attempts"
    ON public.assessment_attempts FOR INSERT
    WITH CHECK (true);
  END IF;
END $$;
`
  console.log('SQL to run in Supabase SQL Editor to fix assessment_attempts RLS:')
  console.log('='.repeat(60))
  console.log(sql)
  console.log('='.repeat(60))
  console.log('\nPlease run this SQL manually in your Supabase Dashboard > SQL Editor')
}

applyMigration()
```

fix_assessments_rls.ts TS 3.0 KB 95 Lines

```ts
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = 'https://gezrhprdrlsswfnuffvf.supabase.co'
// Using anon key - but we need service role for DDL
// Since we don't have service role key, let's use the REST API via pg_dump approach
// Actually let's try using the management API

async function applyMigration() {
  const sql = `
-- Drop the overly restrictive combined policy
DROP POLICY IF EXISTS "Trainers can manage own assessments" ON public.assessments;

-- SELECT: trainer can see assessments for their own courses (regardless of created_by)
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can view assessments for own courses') THEN
    CREATE POLICY "Trainers can view assessments for own courses"
    ON public.assessments FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE courses.id = assessments.course_id
          AND courses.trainer_id = auth.uid()
      )
      AND EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer'
      )
    );
  END IF;
END $$;

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can insert assessments for own courses') THEN
    CREATE POLICY "Trainers can insert assessments for own courses"
    ON public.assessments FOR INSERT
    WITH CHECK (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE courses.id = course_id
          AND courses.trainer_id = auth.uid()
      )
      AND EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer'
      )
    );
  END IF;
END $$;

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can update assessments for own courses') THEN
    CREATE POLICY "Trainers can update assessments for own courses"
    ON public.assessments FOR UPDATE
    USING (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE courses.id = assessments.course_id
          AND courses.trainer_id = auth.uid()
      )
      AND EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer'
      )
    );
  END IF;
END $$;

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can delete assessments for own courses') THEN
    CREATE POLICY "Trainers can delete assessments for own courses"
    ON public.assessments FOR DELETE
    USING (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE courses.id = assessments.course_id
```

```
          AND courses.trainer_id = auth.uid()
      )
      AND EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer'
      )
    );
  END IF;
END $$;
`
  console.log('SQL to run in Supabase SQL Editor:')
  console.log('='.repeat(60))
  console.log(sql)
  console.log('='.repeat(60))
  console.log('\nPlease run this SQL manually in your Supabase Dashboard > SQL Editor')
  console.log('URL: https://supabase.com/dashboard/project/gezrhprdrlsswfnuffvf/sql/new')
}

applyMigration()
```

fix.js JS 1.2 KB 38 Lines

```
import fs from 'fs';
import { createClient } from '@supabase/supabase-js';

const envFile = fs.readFileSync('.env.local', 'utf8');
const env = {};
envFile.split('\n').forEach(line => {
  const match = line.match(/^([^=]+)=(.*)$/);
  if (match) env[match[1].trim()] = match[2].trim();
});

const supabase = createClient(
  env.VITE_SUPABASE_URL,
  env.VITE_SUPABASE_SERVICE_ROLE_KEY || env.VITE_SUPABASE_ANON_KEY
);

async function fix() {
  const { data: courses } = await supabase.from('courses').select('id, trainer_id');
  const { data: assessments, error: err2 } = await supabase.from('assessments').select('*');

  if (err2) {
    console.error('Error fetching assessments', err2);
    return;
  }

  let count = 0;
  for (const a of assessments) {
    const course = courses.find(c => c.id === a.course_id);
    if (course && course.trainer_id && a.created_by !== course.trainer_id) {
      console.log('Fixing assessment', a.id, 'setting created_by to', course.trainer_id);
      const { error } = await supabase.from('assessments').update({ created_by: course.trainer_id }).eq('id', a.id);
      if (error) console.error('Update error', error);
      else count++;
    }
  }
  console.log('Fixed ' + count + ' orphaned assessments!');
}
fix();

```

fix.ts TS 0.8 KB 24 Lines

```ts
import { createClient } from '@supabase/supabase-js';
import dotenv from 'dotenv';
dotenv.config({ path: '.env.local' });

const supabase = createClient(
  process.env.VITE_SUPABASE_URL,
  process.env.VITE_SUPABASE_SERVICE_ROLE_KEY || process.env.VITE_SUPABASE_ANON_KEY
);

async function fix() {
  const { data: courses } = await supabase.from('courses').select('id, trainer_id');
  const { data: assessments } = await supabase.from('assessments').select('*');
  let count = 0;
  for (const a of assessments) {
    const course = courses.find(c => c.id === a.course_id);
    if (course && course.trainer_id && a.created_by !== course.trainer_id) {
      await supabase.from('assessments').update({ created_by: course.trainer_id }).eq('id', a.id);
      count++;
    }
  }
  console.log('Fixed ' + count + ' assessments');
}
fix();

```

get_logs.cjs CJS 0.6 KB 19 Lines

```cjs
const puppeteer = require('puppeteer');

(async () => {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();

  page.on('console', msg => console.log('CONSOLE:', msg.text()));
  page.on('pageerror', err => console.log('PAGE ERROR:', err.message));

  try {
    await page.goto('http://localhost:5173/trainee/courses/037420d5-7e4e-48fc-96b7-16b881fbd35a', { waitUntil: 'networkidle2'
});
    await new Promise(resolve => setTimeout(resolve, 3000));
  } catch (err) {
    console.error('FAILED TO LOAD:', err);
  }

  await browser.close();
})();

```

repair_types.js JS 2.0 KB 54 Lines

```
import fs from "fs";

let code = fs.readFileSync("src/integrations/supabase/types.ts", "utf8");

function removeDuplicates(table) {
  const blockRegex = new RegExp(`(${table}:\\s*\\{.*?\\})(?=\\n\\s*[a-z_]+:\\s*\\{)`, 's');
  code = code.replace(blockRegex, (match) => {
    let lines = match.split('\n');
    let seen = new Set();
    let res = [];
    for (let line of lines) {
      let trimmed = line.trim();
      let keyMatch = trimmed.match(/^([a-zA-Z_]+)\??:/);
      if (keyMatch) {
        let key = keyMatch[1];
        if (['qualifications', 'work_experience', 'interests'].includes(key)) {
          if (seen.has(key)) continue;
          seen.add(key);
        }
      }
      if (trimmed === 'Row: {' || trimmed === 'Insert: {' || trimmed === 'Update: {') {
        seen.clear(); // Reset seen keys for each block
      }
      res.push(line);
    }
    return res.join('\n');
  });
}

removeDuplicates('trainees');
removeDuplicates('trainers');

function injectFields(table, block, toInject) {
  const regex = new RegExp("(" + table + ":\\s*\\{.*?\\b" + block + ":\\s*\\{.*?)(        \\})", "s");
  code = code.replace(regex, "$1" + toInject + "\n$2");
}

// Restore missing fields from prior session
injectFields("assessments", "Row", "          results_publish_date?: string | null");
injectFields("assessments", "Insert", "          results_publish_date?: string | null");
injectFields("assessments", "Update", "          results_publish_date?: string | null");

injectFields("questions", "Row", '          question_type?: "open_ended" | "mcq"');
injectFields("questions", "Insert", '          question_type?: "open_ended" | "mcq"');
injectFields("questions", "Update", '          question_type?: "open_ended" | "mcq"');

injectFields("assessment_attempts", "Row", "          grade_status?: string");
injectFields("assessment_attempts", "Insert", "          grade_status?: string");
injectFields("assessment_attempts", "Update", "          grade_status?: string");


fs.writeFileSync("src/integrations/supabase/types.ts", code);
console.log("Repair Done");

```

scripts/db-audit.mjs MJS 3.7 KB 102 Lines

```mjs
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

// Use service_role key if available (bypasses RLS)
const s = createClient(SUPABASE_URL, SERVICE_KEY, {
  auth: { autoRefreshToken: false, persistSession: false }
})

async function run() {
  const tables = ['assessments', 'questions', 'assessment_attempts', 'attempt_answers', 'enrollments']

  console.log('\n=== TABLE COLUMNS VIA information_schema ===\n')

  for (const tableName of tables) {
    const { data, error } = await s
      .from('information_schema.columns' as any)
      .select('column_name, data_type, is_nullable, column_default')
      .eq('table_schema', 'public')
      .eq('table_name', tableName)
      .order('ordinal_position' as any)

    if (error) {
      console.log(`❌ ${tableName}: cannot query columns — ${error.message}`)
    } else if (!data || data.length === 0) {
      console.log(`⚠️  ${tableName}: table may NOT EXIST in public schema (no columns found)`)
    } else {
      console.log(`✅ ${tableName} (${data.length} columns):`)
      data.forEach((col: any) => {
        console.log(`   ${col.column_name} (${col.data_type})${col.is_nullable === 'NO' ? ' NOT NULL' : ''}`)
      })
    }
    console.log()
  }

  // Try direct row counts ignoring RLS
  console.log('\n=== ROW COUNTS ===\n')
  for (const tableName of tables) {
    const { count, error } = await s
      .from(tableName as any)
      .select('*', { count: 'exact', head: true })
    if (error) {
      console.log(`❌ ${tableName}: ${error.message}`)
    } else {
      console.log(`📊 ${tableName}: ${count} rows`)
    }
  }

  // Check recent assessment_attempts
  console.log('\n=== RECENT ASSESSMENT ATTEMPTS (last 5) ===\n')
  const { data: recent, error: recErr } = await s
    .from('assessment_attempts' as any)
    .select('id, assessment_id, user_id, score, passed, grade_status, submitted_at, answers')
    .order('submitted_at', { ascending: false, nullsFirst: false })
    .limit(5)

  if (recErr) {
    console.log('❌', recErr.message)
  } else {
    console.log(`Found ${recent.length} attempt(s):`)
    recent.forEach((r: any) => {
      const hasAnswers = r.answers && Object.keys(r.answers || {}).length > 0
      console.log(`  [${r.id.slice(0,8)}] score=${r.score} | passed=${r.passed} | status=${r.grade_status} | submitted=${r.submitted_at?.slice(0,16)} | answers_col=${hasAnswers ? 'HAS DATA' : 'empty/null'}`)
    })
  }

  // Check attempt_answers
  console.log('\n=== RECENT ATTEMPT_ANSWERS (last 5) ===\n')
  const { data: ans, error: ansErr } = await s
    .from('attempt_answers' as any)
    .select('*')
    .limit(5)

  if (ansErr) {
```

```
    console.log('❌', ansErr.message)
  } else {
    console.log(`Found ${ans.length} answer(s):`)
    ans.forEach((a: any) => console.log('  ', JSON.stringify(a)))
  }

  // Verify questions have correct_answer and options
  console.log('\n=== QUESTIONS SAMPLE ===\n')
  const { data: qs, error: qsErr } = await s
    .from('questions' as any)
    .select('id, assessment_id, question_text, correct_answer, options')
    .limit(3)

  if (qsErr) {
    console.log('❌', qsErr.message)
  } else {
    console.log(`Found ${qs.length} question(s):`)
    qs.forEach((q: any) => {
      const optKeys = Object.keys(q.options || {}).filter(k => !['_question_type','_difficulty','_section'].includes(k))
      const correctExists = q.correct_answer && q.options && q.correct_answer in q.options
      console.log(`  [${q.id.slice(0,6)}] correct_answer="${q.correct_answer}" | option_keys=[${optKeys.join(',')}] | correct_
in_options=${correctExists}`)
    })
  }
}

run().catch(e => { console.error('Fatal:', e.message); process.exit(1) })
```

scripts/debug-score.mjs MJS 2.4 KB 65 Lines

```mjs
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY)

async function run() {
  const { data: attempts, error: attemptsErr } = await supabase
    .from('assessment_attempts')
    .select('*, attempt_answers(*)')
    .order('created_at', { ascending: false })
    .limit(1)

  if (attemptsErr) { console.error(attemptsErr); return }
  const attempt = attempts[0]
  if (!attempt) return console.log("No attempts")

  console.log("Attempt score in DB:", attempt.score)

  const questionIds = attempt.attempt_answers.map(a => a.question_id)
  const { data: questions } = await supabase.from('questions').select('*').in('id', questionIds)

  let correctCount = 0;
  for (const q of questions) {
    const ans = attempt.attempt_answers.find(a => a.question_id === q.id)
    if (!ans) continue;

    let studentAns = ans.selected_answer || ''
    if (typeof studentAns !== 'string' && Array.isArray(studentAns)) studentAns = studentAns.join(',')
    else studentAns = String(studentAns)
    studentAns = studentAns.trim()

    const correctKeys = (q.correct_answer || '').split(',').map(k => k.trim())
    const correctTexts = correctKeys.map(k => (q.options || {})[k] || k).filter(Boolean).sort().join('|||')
    const studentTexts = studentAns.split('|||').map(s => s.trim()).filter(Boolean).sort().join('|||')

    console.log(`Q: ${q.id.slice(0,5)}`)
    console.log(`  q.correct_answer:`, q.correct_answer)
    console.log(`  correctTexts: "${correctTexts}"`)
    console.log(`  studentAns: "${studentAns}"`)
    console.log(`  studentTexts: "${studentTexts}"`)

    if (studentTexts && correctTexts && studentTexts === correctTexts) {
      console.log(`  => MATCH!`)
      correctCount++
    } else {
      // fallback
      const studentKey = studentAns.split(',').map(s => s.trim()).sort().join(',')
      const correctKey = correctKeys.sort().join(',')
      if (studentKey !== '' && studentKey === correctKey) {
         console.log(`  => MATCH BY KEY!`)
         correctCount++
      } else {
         console.log(`  => WRONG`)
      }
    }
  }

  console.log("Total correct computed by Result logic:", correctCount)
  console.log("Total questions:", questions.length)
}

run()
```

scripts/fix-attempt-answers.mjs MJS 3.1 KB 93 Lines

```mjs
/**
 * Migration: Convert letter-key answers (A, B, C, D) to full option text values.
 * Run with: node fix-attempt-answers.mjs
 */
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

if (!SUPABASE_URL || !SUPABASE_SERVICE_KEY) {
  console.error('Missing env vars. Usage:')
  console.error('VITE_SUPABASE_URL=... SUPABASE_SERVICE_KEY=... node fix-attempt-answers.mjs')
  process.exit(1)
}

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY)
const INTERNAL_KEYS = new Set(['_question_type', '_difficulty', '_section'])
const LETTER_PATTERN = /^[A-F]$/

async function main() {
  console.log('Fetching all attempt answers...')
  const { data: attemptAnswers, error: attemptAnswersErr } = await supabase
    .from('attempt_answers')
    .select('id, attempt_id, question_id, selected_answer')

  if (attemptAnswersErr) { console.error('Error fetching attempt_answers:', attemptAnswersErr); process.exit(1) }
  console.log(`Found ${attemptAnswers.length} answers`)

  const allQuestionIds = new Set()
  for (const ans of attemptAnswers) {
    if (ans.question_id) allQuestionIds.add(ans.question_id)
  }

  console.log(`Fetching ${allQuestionIds.size} questions...`)
  const { data: questions, error: qErr } = await supabase
    .from('questions')
    .select('id, options, correct_answer')
    .in('id', [...allQuestionIds])

  if (qErr) { console.error('Error fetching questions:', qErr); process.exit(1) }

  const qMap = {}
  for (const q of questions) qMap[q.id] = q

  let updatedCount = 0

  for (const ans of attemptAnswers) {
    const qId = ans.question_id
    const rawAnswer = ans.selected_answer

    const q = qMap[qId]
    if (!q || !q.options) continue

    const answerStr = String(rawAnswer || '').trim()
    if (!answerStr) continue

    const parts = answerStr.split(',').map(s => s.trim())
    const allLetterKeys = parts.every(p => LETTER_PATTERN.test(p))

    if (allLetterKeys) {
      // Convert letters to text values
      const converted = parts.map(letter => {
        const text = q.options[letter]
        return (text && !INTERNAL_KEYS.has(letter)) ? text : letter
      })

      const newAnswer = converted.join('|||')

      if (newAnswer !== answerStr) {
        const { error: updateErr } = await supabase
          .from('attempt_answers')
          .update({ selected_answer: newAnswer })
          .eq('id', ans.id)

        if (updateErr) {
          console.error(`❌ Failed answer ${ans.id.slice(0,8)}:`, updateErr.message)
```

```
        } else {
          console.log(`✅ Updated answer for Q ${qId.slice(0,8)}: ${answerStr} -> ${newAnswer}`)
          updatedCount++
        }
      }
    }
  }

  // Note: we're not recalculating the overall score in the assessment_attempts table in this script
  // because that would require joining attempt_answers by attempt_id.
  // The immediate UI problem is just fixing the stored answer strings.

  console.log(`\nDone. Updated ${updatedCount} answers in DB.`)
}

main().catch(err => { console.error(err); process.exit(1) })
```

scripts/generate_code_pdf.cjs CJS 18.9 KB 613 Lines

```
const fs = require('fs');
const path = require('path');
const { execFileSync } = require('child_process');

console.log('--- Capacity Connect Code-to-PDF Generator Initialized ---');

const ROOT_DIR = path.resolve(__dirname, '..');
const CHROME_PATH = 'C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe';
const EDGE_PATH = 'C:\\Program Files (x86)\\Microsoft\\Edge\\Application\\msedge.exe';
const BROWSER_PATH = fs.existsSync(CHROME_PATH) ? CHROME_PATH : EDGE_PATH;

const IGNORE_DIRS = new Set([
  'node_modules',
  '.git',
  'dist',
  'scratch',
  '.gemini',
  '.system_generated'
]);

const IGNORE_FILES = new Set([
  'package-lock.json',
  'lint.log',
  'diff_branch.txt',
  'commit_diff.txt',
  'commit_diff_utf8.txt',
  'diff_full.txt',
  'db_dump.sql',
  '.env.local' // protect local environment secrets
]);

const ALLOWED_EXTS = new Set([
  '.ts', '.tsx', '.js', '.jsx', '.cjs', '.mjs',
  '.css', '.html', '.sql', '.json', '.md'
]);

function escapeHtml(str) {
  if (!str) return '';
  return str
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

// Ultra-fast inline syntax highlighting
function highlightCode(rawCode, ext) {
  const lines = rawCode.split('\n');
  const padLength = Math.max(3, lines.length.toString().length);
  const out = [];

  for (let i = 0; i < lines.length; i++) {
    const line = lines[i];
    const num = (i + 1).toString().padStart(padLength, ' ');
    let esc = escapeHtml(line);

    if (ext === '.sql') {
      if (/^\s*--/.test(line)) {
        esc = `<span class="c-cmt">${esc}</span>`;
      } else {
        esc = esc.replace(/\b(SELECT|FROM|WHERE|INSERT|INTO|UPDATE|DELETE|CREATE|TABLE|ALTER|DROP|FUNCTION|RETURNS|TRIGGER|POLICY|ROW|LEVEL|SECURITY|ENABLE|GRANT|TO|AS|BEGIN|END|DECLARE|RETURN|IF|THEN|ELSE|ELSIF|EXECUTE|LANGUAGE|PLPGSQL|SECURITY|DEFINER|SET|DEFAULT|PRIMARY|KEY|FOREIGN|REFERENCES|NOT|NULL|UNIQUE|INDEX|ON|JOIN|LEFT|INNER|OUTER|GROUP|BY|ORDER|LIMIT|OFFSET|UNION|ALL|OR|AND|CASE|WHEN|THEN|END|VIEW|CASCADE)\b/gi, '<span class="c-kw">$1</span>');
      }
    } else if (ext === '.json') {
      esc = esc
        .replace(/(&quot;.*?&quot;)(?=:)/g, '<span class="c-prop">$1</span>')
        .replace(/(:\s*)(&quot;.*?&quot;)/g, '$1<span class="c-str">$2</span>')
        .replace(/\b(true|false|null)\b/g, '<span class="c-num">$1</span>')
        .replace(/\b(-?\d+(\.\d+)?)\b/g, '<span class="c-num">$1</span>');
    } else {
      // JS / TS / CSS
      if (/^\s*(\/\/|\/\*|\*)/.test(line)) {
        esc = `<span class="c-cmt">${esc}</span>`;
```

```
      } else {
        esc = esc.replace(/\b(import|export|from|default|const|let|var|function|return|if|else|for|while|switch|case|break|continue|try|catch|finally|throw|new|class|interface|type|extends|implements|as|async|await|typeof|instanceof|void|yield)\b/g, '<span class="c-kw">$1</span>');
        esc = esc.replace(/\b(true|false|null|undefined)\b/g, '<span class="c-num">$1</span>');
        esc = esc.replace(/\b(string|number|boolean|any|unknown|never|void|Record|Array|Promise|React|FC|JSX|HTMLElement)\b/g, '<span class="c-typ">$1</span>');
      }
    }

    out.push(`<span class="ln">${num} | </span>${esc}`);
  }

  return out.join('\n');
}

function scanFiles() {
  const allFiles = [];

  function walk(currentDir) {
    const entries = fs.readdirSync(currentDir, { withFileTypes: true });
    for (const entry of entries) {
      if (entry.isDirectory()) {
        if (!IGNORE_DIRS.has(entry.name)) {
          walk(path.join(currentDir, entry.name));
        }
      } else {
        const ext = path.extname(entry.name).toLowerCase();
        if (ALLOWED_EXTS.has(ext) && !IGNORE_FILES.has(entry.name)) {
          const fullPath = path.join(currentDir, entry.name);
          const relPath = path.relative(ROOT_DIR, fullPath).replace(/\\/g, '/');
          const stat = fs.statSync(fullPath);
          allFiles.push({
            fullPath,
            relPath,
            name: entry.name,
            ext,
            size: stat.size
          });
        }
      }
    }
  }

  walk(ROOT_DIR);
  return allFiles;
}

function categorizeFile(file) {
  const p = file.relPath;
  if (p === 'package.json' || p === 'vite.config.ts' || p === 'tailwind.config.js' || p.startsWith('tsconfig') || p === 'postcss.config.js' || p === 'components.json' || p === 'vercel.json' || p === 'README.md' || p === 'index.html') {
    return { section: '01. Configuration & Build Setup', order: 1 };
  }
  if (p === 'src/main.tsx' || p === 'src/App.tsx' || p === 'src/index.css' || p === 'src/App.css') {
    return { section: '02. Application Entry & Core Setup', order: 2 };
  }
  if (p.startsWith('src/integrations/') || p.startsWith('src/lib/') || p.startsWith('src/utils/')) {
    return { section: '03. Integrations, Supabase Client & Utilities', order: 3 };
  }
  if (p.startsWith('src/hooks/')) {
    return { section: '04. Custom Hooks & State Management', order: 4 };
  }
  if (p.startsWith('src/pages/')) {
    return { section: '05. Application Pages & Views', order: 5 };
  }
  if (p.startsWith('src/features/trainer/')) {
    return { section: '06. Features — Trainer Management', order: 6 };
  }
  if (p.startsWith('src/features/courses/')) {
    return { section: '07. Features — Courses, Sessions & Learning', order: 7 };
  }
  if (p.startsWith('src/features/admin/')) {
    return { section: '08. Features — Super Admin & Governance', order: 8 };
  }
  if (p.startsWith('src/features/notifications/') || p.startsWith('src/features/certificates/')) {
    return { section: '09. Features — Notifications & Certificates', order: 9 };
  }
  if (p.startsWith('src/components/ui/')) {
```

```
    return { section: '10. UI Primitives & Design System (Radix/Tailwind)', order: 10 };
  }
  if (p.startsWith('src/components/')) {
    return { section: '11. Shared Layout & Dialog Components', order: 11 };
  }
  if (p.startsWith('supabase/functions/')) {
    return { section: '12. Supabase Backend — Edge Functions', order: 12 };
  }
  if (p.startsWith('supabase/migrations/')) {
    return { section: '13. Supabase Backend — Database Migrations', order: 13 };
  }
  if (p.startsWith('scripts/') || !p.includes('/')) {
    return { section: '14. System, Audit & Database Scripts', order: 14 };
  }
  return { section: '15. Miscellaneous Code', order: 15 };
}

console.log('Scanning files...');
const files = scanFiles();

let totalLines = 0;
let totalBytes = 0;

for (const file of files) {
  const cat = categorizeFile(file);
  file.section = cat.section;
  file.order = cat.order;
  const content = fs.readFileSync(file.fullPath, 'utf8');
  file.lines = content.split('\n').length;
  file.content = content;
  totalLines += file.lines;
  totalBytes += file.size;
}

files.sort((a, b) => {
  if (a.order !== b.order) return a.order - b.order;
  return a.relPath.localeCompare(b.relPath);
});

console.log(`Found ${files.length} source files, totaling ${totalLines.toLocaleString()} lines (${(totalBytes / 1024 / 1024).toFixed(2)} MB).`);

function buildHtml(fileSubset, title, subtitle) {
  let html = `<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>${escapeHtml(title)}</title>
<style>
  @page {
    size: A4 portrait;
    margin: 14mm 10mm 14mm 10mm;
  }
  * {
    box-sizing: border-box;
    -webkit-print-color-adjust: exact !important;
    print-color-adjust: exact !important;
  }
  body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-serif;
    color: #0f172a;
    background: #ffffff;
    margin: 0;
    padding: 0;
  }

  /* Cover Page */
  .cover-page {
    page-break-after: always;
    min-height: 88vh;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    padding: 24px;
    border: 2px solid #cbd5e1;
    border-radius: 12px;
    background: #ffffff;
  }
  .cover-header {
```

```
    border-bottom: 3px solid #2563eb;
    padding-bottom: 20px;
  }
  .cover-badge {
    display: inline-block;
    background: #eff6ff;
    color: #1d4ed8;
    font-weight: 700;
    font-size: 11px;
    letter-spacing: 1px;
    text-transform: uppercase;
    padding: 4px 12px;
    border-radius: 20px;
    margin-bottom: 12px;
    border: 1px solid #bfdbfe;
  }
  .cover-title {
    font-size: 34px;
    font-weight: 800;
    color: #0f172a;
    margin: 0 0 8px 0;
  }
  .cover-subtitle {
    font-size: 16px;
    color: #475569;
    margin: 0;
  }

  .meta-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 12px;
    margin: 20px 0;
  }
  .meta-card {
    background: #f8fafc;
    border: 1px solid #e2e8f0;
    border-radius: 8px;
    padding: 12px 16px;
  }
  .meta-label {
    font-size: 10px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: #64748b;
    font-weight: 700;
    margin-bottom: 3px;
  }
  .meta-value {
    font-size: 15px;
    font-weight: 700;
    color: #1e293b;
  }

  .tech-stack {
    display: flex;
    flex-wrap: wrap;
    gap: 6px;
    margin-top: 8px;
  }
  .tech-pill {
    background: #f1f5f9;
    color: #334155;
    font-size: 10px;
    font-weight: 600;
    padding: 3px 8px;
    border-radius: 4px;
    border: 1px solid #cbd5e1;
  }

  .cover-footer {
    border-top: 1px solid #e2e8f0;
    padding-top: 12px;
    font-size: 10px;
    color: #64748b;
    display: flex;
    justify-content: space-between;
  }

```

```
  /* Table of Contents */
  .toc-page {
    page-break-after: always;
    padding-top: 6px;
  }
  .toc-title {
    font-size: 20px;
    font-weight: 800;
    color: #0f172a;
    border-bottom: 2px solid #2563eb;
    padding-bottom: 6px;
    margin-bottom: 14px;
  }
  .toc-section {
    margin-bottom: 12px;
    page-break-inside: avoid;
  }
  .toc-section-header {
    font-size: 11px;
    font-weight: 700;
    color: #1e40af;
    background: #eff6ff;
    padding: 4px 8px;
    border-radius: 4px;
    margin-bottom: 4px;
    border-left: 3px solid #2563eb;
  }
  .toc-list {
    list-style: none;
    padding: 0;
    margin: 0;
  }
  .toc-item {
    display: flex;
    justify-content: space-between;
    font-size: 9.5px;
    padding: 2px 6px;
    border-bottom: 1px dotted #e2e8f0;
    font-family: "JetBrains Mono", "Consolas", monospace;
  }
  .toc-item:nth-child(even) {
    background: #f8fafc;
  }
  .toc-item-meta {
    color: #64748b;
    white-space: nowrap;
    margin-left: 10px;
  }

  /* Code Section */
  .file-container {
    page-break-before: always;
    margin-bottom: 16px;
  }
  .file-header {
    page-break-inside: avoid;
    page-break-after: avoid;
    background: #0f172a;
    color: #f8fafc;
    padding: 6px 10px;
    border-radius: 6px 6px 0 0;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 2px solid #2563eb;
  }
  .file-path {
    font-family: "JetBrains Mono", "Consolas", monospace;
    font-size: 10.5px;
    font-weight: 700;
    color: #ffffff;
  }
  .file-badges {
    display: flex;
    gap: 6px;
  }
  .file-badge {
    background: #1e293b;
    color: #93c5fd;
```

```
    font-size: 8.5px;
    font-weight: 600;
    padding: 2px 6px;
    border-radius: 4px;
    border: 1px solid #334155;
    font-family: sans-serif;
  }
  .file-badge-accent {
    background: #1e3a8a;
    color: #bfdbfe;
    border-color: #3b82f6;
  }

  .code-pre {
    margin: 0;
    padding: 6px 8px;
    background: #fafafa;
    border: 1px solid #e2e8f0;
    border-top: none;
    border-radius: 0 0 6px 6px;
    font-family: "Consolas", "Courier New", monospace;
    font-size: 7.2pt;
    line-height: 1.34;
    white-space: pre-wrap;
    word-break: break-all;
    tab-size: 2;
    color: #1e293b;
  }

  .ln {
    color: #94a3b8;
    user-select: none;
    display: inline-block;
  }

  /* Syntax Highlighting */
  .c-kw { color: #7c3aed; font-weight: 600; }
  .c-str { color: #059669; }
  .c-cmt { color: #64748b; font-style: italic; }
  .c-typ { color: #0284c7; font-weight: 600; }
  .c-num { color: #d97706; font-weight: 600; }
  .c-prop { color: #2563eb; }
</style>
</head>
<body>

<!-- Cover Page -->
<div class="cover-page">
  <div class="cover-header">
    <div class="cover-badge">Source Code Documentation</div>
    <h1 class="cover-title">Capacity Connect</h1>
    <p class="cover-subtitle">${escapeHtml(subtitle)}</p>
  </div>

  <div class="meta-grid">
    <div class="meta-card">
      <div class="meta-label">Total Files Included</div>
      <div class="meta-value">${fileSubset.length.toLocaleString()} Files</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Total Code Lines</div>
      <div class="meta-value">${fileSubset.reduce((acc, f) => acc + f.lines, 0).toLocaleString()} Lines</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Export Date &amp;amp; Time</div>
      <div class="meta-value">${new Date().toLocaleString('en-US', { dateStyle: 'long', timeStyle: 'short' })}</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Technology Stack</div>
      <div class="meta-value">React 19, TypeScript, Supabase</div>
    </div>
  </div>

  <div style="background: #ffffff; border: 1px solid #e2e8f0; border-radius: 8px; padding: 14px 18px;">
    <div class="meta-label">Architecture &amp;amp; Core Dependencies</div>
    <div class="tech-stack">
      <span class="tech-pill">React 19</span>
      <span class="tech-pill">TypeScript</span>
      <span class="tech-pill">Vite</span>
```

```
      <span class="tech-pill">Tailwind CSS</span>
      <span class="tech-pill">Supabase Backend</span>
      <span class="tech-pill">PostgreSQL &amp;amp; RLS Policies</span>
      <span class="tech-pill">Supabase Edge Functions (Deno)</span>
      <span class="tech-pill">Radix UI Primitives</span>
      <span class="tech-pill">TanStack React Query</span>
      <span class="tech-pill">React Hook Form + Zod</span>
      <span class="tech-pill">Three.js / React Three Fiber</span>
      <span class="tech-pill">TensorFlow.js (BlazeFace/COCO-SSD)</span>
    </div>
  </div>

  <div class="cover-footer">
    <span>Capacity Connect Application Repository</span>
    <span>Directory: E:\\Projects\\capacity connect</span>
  </div>
</div>

<!-- Table of Contents -->
<div class="toc-page">
  <div class="toc-title">Table of Contents</div>
`;

  const subSections = {};
  for (const f of fileSubset) {
    if (!subSections[f.section]) subSections[f.section] = [];
    subSections[f.section].push(f);
  }

  for (const [secTitle, secFiles] of Object.entries(subSections)) {
    const secLines = secFiles.reduce((acc, f) => acc + f.lines, 0);
    html += `
  <div class="toc-section">
    <div class="toc-section-header">${escapeHtml(secTitle)} (${secFiles.length} files, ${secLines.toLocaleString()} lines)</div>
    <ul class="toc-list">
    `;
    for (const f of secFiles) {
      html += `
      <li class="toc-item">
        <span>${escapeHtml(f.relPath)}</span>
        <span class="toc-item-meta">${f.lines.toLocaleString()} lines | ${(f.size / 1024).toFixed(1)} KB</span>
      </li>
      `;
    }
    html += `
    </ul>
  </div>
    `;
  }

  html += `
</div>

<!-- Source Code Content -->
`;

  for (const f of fileSubset) {
    html += `
<div class="file-container">
  <div class="file-header">
    <div class="file-path">${escapeHtml(f.relPath)}</div>
    <div class="file-badges">
      <span class="file-badge">${f.ext.toUpperCase().replace('.', '')}</span>
      <span class="file-badge">${(f.size / 1024).toFixed(1)} KB</span>
      <span class="file-badge file-badge-accent">${f.lines.toLocaleString()} Lines</span>
    </div>
  </div>
  <pre class="code-pre"><code>${highlightCode(f.content, f.ext)}</code></pre>
</div>
`;
  }

  html += `
</body>
</html>
`;
  return html;
}
```

```
542 |
543 | function renderPdf(htmlPath, pdfPath) {
544 |   const headerTemplate = `
545 |     <div style="font-size: 8px; width: 100%; display: flex; justify-content: space-between; padding: 0 10mm; color: #64748b; f
ont-family: sans-serif; border-bottom: 0.5px solid #e2e8f0; padding-bottom: 2px;">
546 |       <span style="font-weight: 700; color: #1e293b;">CAPACITY CONNECT</span>
547 |       <span>Source Code Documentation</span>
548 |     </div>
549 |   `;
550 |
551 |   const footerTemplate = `
552 |     <div style="font-size: 8px; width: 100%; display: flex; justify-content: space-between; padding: 0 10mm; color: #64748b; f
ont-family: sans-serif; border-top: 0.5px solid #e2e8f0; padding-top: 2px;">
553 |       <span>Capacity Connect Repository</span>
554 |       <span>Page <span class="pageNumber"></span> of <span class="totalPages"></span></span>
555 |     </div>
556 |   `;
557 |
558 |   console.log(`Rendering PDF via browser (${path.basename(BROWSER_PATH)})...`);
559 |   const t0 = Date.now();
560 |   execFileSync(BROWSER_PATH, [
561 |     '--headless=new',
562 |     '--disable-gpu',
563 |     '--display-header-footer',
564 |     `--header-template=${headerTemplate}`,
565 |     `--footer-template=${footerTemplate}`,
566 |     `--print-to-pdf=${pdfPath}`,
567 |     htmlPath
568 |   ], { maxBuffer: 1024 * 1024 * 100, timeout: 600000 });
569 |
570 |   const duration = ((Date.now() - t0) / 1000).toFixed(1);
571 |   const sizeMb = (fs.statSync(pdfPath).size / 1024 / 1024).toFixed(2);
572 |   console.log(`Generated: ${path.basename(pdfPath)} (${sizeMb} MB in ${duration}s)`);
573 | }
574 |
575 | function main() {
576 |   const outputDir = ROOT_DIR;
577 |
578 |   // 1. Frontend & Core Application
579 |   const frontendFiles = files.filter(f => !f.relPath.startsWith('supabase/'));
580 |   // 2. Supabase Backend & Database
581 |   const backendFiles = files.filter(f => f.relPath.startsWith('supabase/') || f.relPath.startsWith('scripts/'));
582 |
583 |   console.log('\n--- 1. Generating Volume 1: Frontend Application PDF ---');
584 |   const vol1HtmlPath = path.join(outputDir, 'Capacity_Connect_Frontend_Application.html');
585 |   const vol1PdfPath = path.join(outputDir, 'Capacity_Connect_Frontend_Application.pdf');
586 |   fs.writeFileSync(vol1HtmlPath, buildHtml(frontendFiles, 'Capacity Connect — Frontend Application', 'Frontend Application Arc
hitecture, Pages, Features & Components'));
587 |   renderPdf(vol1HtmlPath, vol1PdfPath);
588 |   fs.unlinkSync(vol1HtmlPath);
589 |
590 |   console.log('\n--- 2. Generating Volume 2: Supabase Backend & Database PDF ---');
591 |   const vol2HtmlPath = path.join(outputDir, 'Capacity_Connect_Backend_and_Database.html');
592 |   const vol2PdfPath = path.join(outputDir, 'Capacity_Connect_Backend_and_Database.pdf');
593 |   fs.writeFileSync(vol2HtmlPath, buildHtml(backendFiles, 'Capacity Connect — Backend & Database', 'Supabase Database Schema, M
igrations, Edge Functions & Maintenance Scripts'));
594 |   renderPdf(vol2HtmlPath, vol2PdfPath);
595 |   fs.unlinkSync(vol2HtmlPath);
596 |
597 |   console.log('\n--- 3. Generating Complete Codebase Master PDF ---');
598 |   const masterHtmlPath = path.join(outputDir, 'Capacity_Connect_Source_Code_Complete.html');
599 |   const masterPdfPath = path.join(outputDir, 'Capacity_Connect_Source_Code_Complete.pdf');
600 |   fs.writeFileSync(masterHtmlPath, buildHtml(files, 'Capacity Connect — Complete Codebase', 'Complete Source Code Documentatio
n (All 238 Files, 53,700+ Lines)'));
601 |   renderPdf(masterHtmlPath, masterPdfPath);
602 |   fs.unlinkSync(masterHtmlPath);
603 |
604 |   console.log('\n======================================================');
605 |   console.log('SUCCESS! All PDF documents generated:');
606 |   console.log('1. Capacity_Connect_Frontend_Application.pdf');
607 |   console.log('2. Capacity_Connect_Backend_and_Database.pdf');
608 |   console.log('3. Capacity_Connect_Source_Code_Complete.pdf');
609 |   console.log('======================================================\n');
610 | }
611 |
612 | main();
613 |
```

test_admin.mjs MJS 0.4 KB 8 Lines

```mjs
import { createClient } from '@supabase/supabase-js';
const supabaseUrl = process.env.VITE_SUPABASE_URL;
const supabaseKey = process.env.VITE_SUPABASE_PUBLISHABLE_KEY;
const supabase = createClient(supabaseUrl, supabaseKey);
supabase.from('admins').select('*').eq('email', 'admin@example.com')
  .then(r => console.log('Admins table:', JSON.stringify(r, null, 2)))
  .catch(console.error);

```

test_admin2.mjs MJS 0.4 KB 9 Lines

```mjs
import { createClient } from '@supabase/supabase-js';
const supabaseUrl = process.env.VITE_SUPABASE_URL;
// We need SERVICE_ROLE_KEY to query auth.users
const supabaseKey = process.env.VITE_SUPABASE_SERVICE_ROLE_KEY || process.env.VITE_SUPABASE_PUBLISHABLE_KEY;
const supabase = createClient(supabaseUrl, supabaseKey);
supabase.auth.admin.listUsers()
  .then(r => console.log('Users:', r.data.users.map(u => u.email)))
  .catch(console.error);

```

test_login.mjs MJS 0.4 KB 9 Lines

```mjs
import { createClient } from '@supabase/supabase-js';
const supabaseUrl = process.env.VITE_SUPABASE_URL;
const supabaseKey = process.env.VITE_SUPABASE_PUBLISHABLE_KEY;
console.log({supabaseUrl, supabaseKey});
const supabase = createClient(supabaseUrl, supabaseKey);
supabase.auth.signInWithPassword({email: 'admin@example.com', password: 'password123'})
  .then(r => console.log(JSON.stringify(r, null, 2)))
  .catch(console.error);

```

test-db.mjs MJS 0.8 KB 22 Lines

```mjs
import { createClient } from '@supabase/supabase-js'
import dotenv from 'dotenv'

dotenv.config()

const supabase = createClient(process.env.VITE_SUPABASE_URL, process.env.VITE_SUPABASE_ANON_KEY)

async function run() {
  const { data: assessments, error: aErr } = await supabase.from('assessments').select('id, title, status').eq('title', 'Demo-01')
  if (aErr) console.error('Assessments Error:', aErr)
  console.log('Assessments:', assessments)

  if (assessments && assessments.length > 0) {
    const id = assessments[0].id
    const { data: questions, error: qErr } = await supabase.from('questions').select('id, approved, position').eq('assessment_id', id)
    if (qErr) console.error('Questions Error:', qErr)
    console.log(`Questions for ${id}:`, questions?.length)
  }
}

run()

```

update_types.js JS 2.3 KB 81 Lines

```
import fs from "fs";

let code = fs.readFileSync("src/integrations/supabase/types.ts", "utf8");

const newTables = `      announcements: {
        Row: {
          id: string
          title: string
          content: string
          author_id: string
          is_active: boolean
          created_at: string
        }
        Insert: {
          id?: string
          title: string
          content: string
          author_id: string
          is_active?: boolean
          created_at?: string
        }
        Update: {
          id?: string
          title?: string
          content?: string
          author_id?: string
          is_active?: boolean
          created_at?: string
        }
        Relationships: []
      }
      course_feedback: {
        Row: {
          id: string
          course_id: string
          user_id: string
          rating: number
          comments: string | null
          created_at: string
        }
        Insert: {
          id?: string
          course_id: string
          user_id: string
          rating: number
          comments?: string | null
          created_at?: string
        }
        Update: {
          id?: string
          course_id?: string
          user_id?: string
          rating?: number
          comments?: string | null
          created_at?: string
        }
        Relationships: []
      }
      scenario_modules: {`;

code = code.replace("      scenario_modules: {", newTables);

function injectFields(table, block, toInject) {
  const regex = new RegExp("(" + table + ":\\s*\\{.*?\\b" + block + ":\\s*\\{.*?)(        \\})", "s");
  code = code.replace(regex, "$1" + toInject + "\n$2");
}

const reqFields = "          qualifications: string | null\n          work_experience: string | null\n          interests: string | null";
const optFields = "          qualifications?: string | null\n          work_experience?: string | null\n          interests?: string | null";

injectFields("trainees", "Row", reqFields);
injectFields("trainees", "Insert", optFields);
injectFields("trainees", "Update", optFields);

```

```
injectFields("trainers", "Row", reqFields);
injectFields("trainers", "Insert", optFields);
injectFields("trainers", "Update", optFields);

fs.writeFileSync("src/integrations/supabase/types.ts", code);
console.log("Done");

```